

Milkyway™ Environment Online Help

Decemeber 2007

Comments?

Send comments on the documentation by going to <http://solvnnet.synopsys.com>, then clicking "Enter a Call to the Support Center."

SYNOPSYS®

Copyright Notice and Proprietary Information

Copyright © 2007 Synopsys, Inc. All rights reserved. This software and documentation contain confidential and proprietary information that is the property of Synopsys, Inc. The software and documentation are furnished under a license agreement and may be used or copied only in accordance with the terms of the license agreement. No part of the software and documentation may be reproduced, transmitted, or translated, in any form or by any means, electronic, mechanical, manual, optical, or otherwise, without prior written permission of Synopsys, Inc., or as expressly provided by the license agreement.

Right to Copy Documentation

The license agreement with Synopsys permits licensee to make copies of the documentation for its internal use only. Each copy shall include all copyrights, trademarks, service marks, and proprietary rights notices, if any. Licensee must assign sequential numbers to all copies. These copies shall contain the following legend on the cover page:

"This document is duplicated with the permission of Synopsys, Inc., for the exclusive use of _____ and its employees. This is copy number _____."

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Registered Trademarks (®)

Synopsys, AMPS, Cadabra, CATS, CRITIC, CSim, Design Compiler, DesignPower, DesignWare, EPIC, Formality, HSIM, HSPICE, iN-Phase, in-Sync, Leda, MAST, ModelTools, NanoSim, OpenVera, PathMill, Photolynx, Physical Compiler, PrimeTime, SiVL, SNUG, SolvNet, System Compiler, TetraMAX, VCS, Vera, and YIELDirector are registered trademarks of Synopsys, Inc.

Trademarks (™)

AFGen, Apollo, Astro, Astro-Rail, Astro-Xtalk, Aurora, AvanWaves, Columbia, Columbia-CE, Cosmos, CosmosEnterprise, CosmosLE, CosmosScope, CosmosSE, DC Expert, DC Professional, DC Ultra, Design Analyzer, Design Vision, DesignerHDL, Direct Silicon Access, Discovery, Encore, Galaxy, HANEX, HDL Compiler, Hercules,

Hierarchical Optimization Technology, HSIM^{plus}, HSPICE-Link, iN-Tandem, i-Virtual Stepper, Jupiter, Jupiter-DP, JupiterXT, JupiterXT-ASIC, Liberty, Libra-Passport, Library Compiler, Magellan, Mars, Mars-Xtalk, Milkyway, ModelSource, Module Compiler, Planet, Planet-PL, Polaris, Power Compiler, Raphael, Raphael-NES, Saturn, Scirocco, Scirocco-i, Star-RCXT, Star-SimXT, Taurus, TSUPREM-4, VCS Express, VCSi, VHDL Compiler, VirSim, and VMC are trademarks of Synopsys, Inc.

Service Marks (SM)

MAP-in, SVP Café, and TAP-in are service marks of Synopsys, Inc.

SystemC is a trademark of the Open SystemC Initiative and is used under license.

ARM and AMBA are registered trademarks of ARM Limited.

Saber is a registered trademark of SabreMark Limited Partnership and is used under license.

All other product or company names may be trademarks of their respective owners.

Contents

1. apr Commands for Milkyway

aprDumpScanChain	2-2
aprFixCell	2-3
aprPDEF30DisplIgnoredInfo	2-4
aprPDEFIn.	2-4
aprPDEFOut	2-6
aprPDEFOutIgnoreFillerCells	2-8
aprPDEFPrintRegionName	2-9
aprSetPDEFVersionTo30.	2-10

2. apr Commands for Astro Family Products and Milkyway

aprHiliteCell	3-2
aprSelectFlyline	3-4
aprSetFlyline	3-8

aprShowPinNet	3-11
aprShowPinNetByName	3-13
aprShowSelCellFlyline.	3-15
3. aprCmd Commands for Milkyway	
aprCmdFixCell.	4-2
4. ast Commands for Milkyway	
astDeleteHierPreservation	5-2
astDumpHierPreservation	5-3
astDumpHierVerilog	5-4
astExtrLEQ	5-12
astInitHierPreservation	5-14
astPostVerification	5-15
astSetCellInstDontTouch	5-16
5. ast Commands (No License)	
astSetDebug	6-2
astUnsetDebug	6-2
6. au Commands for Milkyway	
auAbortStreamInIfNoMapFile	1-7
auAbortStreamOutIfNoMapFile	1-7

auAlfToCLF (Unsupported)	1-8
auCellTextOut (Unsupported)	1-8
auConvertLayerDataType	1-8
auCreateBaseArray	1-9
auCreateHierNetlist	1-11
auCreateHierVlogOut	1-18
auDefIn	1-29
auDefOut	1-31
auDumpCLF	1-34
auECOByChangeFile	1-35
auECOByNetCmp	1-39
auECODumpDiff	1-46
auEdifIn	1-46
auEdifOut	1-52
auExtractBlockagePinVia	1-57
auExtractMacroPins	1-74
auGenCellBndry	1-78
auGenCellBoundary	1-81
auGenLibCellBoundary	1-82
auGenSynopBackAnn	1-84
auGetEmptyCellOnly	1-86

auHECOByFile	1-87
auHECODiff.	1-88
auHierECO	1-90
auHierVerilogOut	1-97
auLefIn	1-102
auLib2LCIf	1-104
auLoadCLF	1-105
auNDIApi	1-109
auNDOApi	1-118
auNLIApi	1-135
auNLOApi	1-144
auNewECOByNetCmp	1-153
auSDFToConstraint	1-159
auSetEdifOutExternal	1-160
auSetEmptyCellOnly	1-161
auSetHierInstanceAndNetNameLong	1-161
auSetHierInstanceNameLong	1-162
auSetHierNetNameLong	1-163
auSetHierRefLibTwoLevels	1-164
auSetMasterCellNameLong	1-164
auSetMultiLEFForeignName	1-165

auSetPRBdry.	1-166
auStampToDB	1-176
auStampToCLF	1-180
auStreamIn	1-187
auStreamInCellBndryIgnoreText	1-192
auStreamInDuplicateLayerAsBoundary	1-193
auStreamInRectilinearBoundary	1-194
auStreamInSetCellBoundaryDataType	1-195
auStreamInTextConversionFactor	1-195
auStreamOut	1-197
Stream Out Data File Dialog Box (Pin/Net Options)	1-211
Design Intent	1-214
auStreamOutAllHierPinNetData	1-218
auStreamOutArefContactArray	1-219
auStreamOutByLayerNumber	1-220
auStreamOutCellInstanceNameAsText	1-220
auStreamOutCellRenamePrefix	1-221
auStreamOutCheckInvalidCells.	1-222
auStreamOutChildCellFillData	1-223
auStreamOutConfirmOverWriteGDSIIFile.	1-224
auStreamOutContactArrayMatrix	1-225

auStreamOutControlFileToLog	1-225
auStreamOutCustomViewInstance	1-226
auStreamOutFillDataForAllCellTypes	1-228
auStreamOutFillDataMapping	1-229
auStreamOutGDSIIVersion	1-229
auStreamOutKeepDataType	1-230
auStreamOutNetNameBasedLayerMapping	1-231
auStreamOutNetWithPathTypeZero	1-232
auStreamOutOddShapePin.	1-233
auStreamOutOptionsToLog.	1-234
auStreamOutPropertyByObject.	1-234
auStreamOutRemoveEmptyCells	1-235
auStreamOutRenameCellName	1-236
auStreamOutRenameRefLibDuplicateCellFirst.	1-237
auStreamOutSelectedCells	1-238
auStreamOutShortCellName.	1-239
auStreamOutShowCellInfo	1-239
auStreamOutSkipGRouteContact	1-240
auStreamOutSkipTheseCells	1-241
auStreamOutSplitPath.	1-242
auStreamOutUnlinkGDSIIFile	1-243

auStreamOutWillAbortIfCellsMissing	1-244
auStreamOutWithExtendedLayerMapping	1-245
auSuffixDuplicateLongCellName.	1-246
auSynopToCLF	1-247
auVerilogIn	1-254
auVerilogOut	1-261
auVerilogToCell	1-266
auVhdlIn	1-277
auVhdlOut	1-283
7. awe Commands for Milkyway	
aweSetOrder	2-2
8. ax Commands for Milkyway	
axCreateContactArray	3-2
axCreateRowRecord	3-4
axCreateTilePatternRecord	3-6
axCreateTrackRecord	3-7
axCreateWireDirRecord	3-10
axSetUnitTileWidth	3-11
9. axg Commands for Milkyway	
axgCheckWireTrack.	4-2

axgDefineWireTracks	4-3
axgDisplayCellWireTracks	4-9
axgDisplayDesignWireTracks	4-10
axgGenerateInstName	4-12

10. clf Commands for Milkyway

clfCreateTable	5-3
clfCreateVectorTable	5-12
clfDefineDesignCellName	5-19
clfDump	5-20
clfDumpAntennaProp	5-21
clfDumpStaticNoiseMargin	5-22
clfDumpStaticNoiseProperty	5-23
clfLoad	5-23
clfPurgeBooleanFunction	5-24
clfPurgePad	5-25

11. cm Commands for Milkyway

cmAttachStarRCXT	6-4
cmCheckCellDRC	6-5
cmCheckLibrary	6-6
cmCheckNetList	6-8

cmCheckNetlist	6-8
cmCheckTechFile	6-9
cmCopyLib.	6-10
cmCreateCapModel	6-11
cmCreateHierPorts (Unsupported)	6-17
cmCreateLib	6-18
cmDumpHierPorts (Unsupported).	6-21
cmDumpPlib	6-23
cmDumpPseudoPlib	6-25
cmDumpRefLibList	6-26
cmDumpTech	6-27
cmDumpTLUPlus	6-28
cmExecSmash.	6-29
cmFlattenNetlist.	6-30
cmItfToTLUPlus	6-31
cmLoadTech	6-34
cmItfToTLUTest	6-35
cmMarkCellType	6-36
cmMarkEEQPins	6-39
cmMarkLefSite	6-41
cmMasterTilePattern	6-43

cmPDBIn	6-47
cmPurgeTech	6-49
cmQueryHierPorts.	6-49
cmRefLib	6-50
cmRenameCell	6-51
cmRenameLib	6-52
cmReplaceRefLib	6-53
cmReplaceTech.	6-55
cmReplaceTLUPlus.	6-56
cmSetBusNameStyle	6-57
cmSetMultiHeightProperty.	6-60
cmShowInstOrigin	6-62
cmShowRefLib	6-63
cmSmash	6-65

12. cmCmd Commands for Milkyway

cmCmdDumpMultipleECOs	7-2
cmCmdDumpSlackGraph	7-3
cmCmdECODump.	7-5
cmCmdECOHistory.	7-8
cmCmdExpand	7-11
cmCmdExpandBindNet.	7-17

cmCmdLoadConstraints	7-23
cmCmdLoadSlackGraph	7-25
cmCmdLoadTimingRequirements	7-26
13. db Commands for Milkyway and Astro Family Products	
dbAddBlockage	8-2
dbAddSoftBlockage	8-3
dbDumpVTFillers	8-3
dbSetEcoWeight	8-4
dbSetReqArrivalTime	8-5
14. db Commands for Milkyway	
dbAbortCell	9-27
dbAddAntennaLayerRule	9-27
dbAddCellInstToPlanGroupByName	9-29
dbAddCellInstToPlanGroupByNames	9-30
dbAddCellInstToCellRegion	9-31
dbAddFixedEdgeToScanChain	9-33
dbAddFreeEdgeToScanChain	9-34
dbAddInstGroupToScanChain	9-36
dbAddIOText	9-37
dbAddLibSchemeFile	9-38

dbAddLibTclFile.	9-39
dbAddNetSubTypeByName.	9-40
dbAddObjectToGroup	9-41
dbAddPartialScanToScanChain	9-42
dbAddRefLib	9-43
dbAddViaSpot	9-45
dbAllowToAddAllPGIOText	9-46
dbAllowToAddPGIOText	9-47
dbArchiveCellToLib	9-48
dbArchiveHierDesign.	9-54
dbAssignNetClassXtalkConstraints.	9-55
dbAssignNetMinMaxLayer.	9-56
dbAssignNetRouteRuleByHalfPerimeter.	9-58
dbAssignNetTimingSpacing	9-59
dbAssignNetTopLayerProbe	9-60
dbAssignNetVarRouteRule	9-61
dbAssignNetXtalkClass.	9-63
dbAssignNetXtalkConstraints	9-64
dbAssignVarRouteRule	9-66
dbBrowser	9-67
dbCellChecksum.	9-68

dbCellCompare	9-72
dbCellInstPlaceAndStatus	9-76
dbCellReportHighFanoutNet	9-77
dbChangeAccessMode	9-79
dbChangeAllInstNames	9-79
dbChangeAllNetNames	9-81
dbChangeAllNetNamesByNetId	9-83
dbChangeInstanceName	9-84
dbChangeLibDbVersion19To20	9-85
dbChangeRefLibAccessMode	9-86
dbChangeTextLayer	9-87
dbCheckCellData	9-88
dbCheckCellTypeAndPortMatching	9-88
dbCheckIdenticalCell	9-90
dbCheckNetList	9-91
dbCheckNetlistVsFram	9-92
dbCheckRefLibControl	9-94
dbClearAllNetXtalkProp	9-94
dbClearAttachment	9-95
dbClearCellDifferentialGroup	9-95
dbClearCellEEQClass	9-96

dbClearCellEQClass	9-97
dbClearCellLEQClass	9-97
dbClearCellFootPrintEQClass	9-98
dbClearCellVTType	9-98
dbClearCellXtalkConstraints	9-99
dbClearCellXtalkLengthWeight	9-100
dbClearLengthRouteRuleTable	9-101
dbClearLibAntennaRules	9-102
dbClearLibraryCellsVTType	9-103
dbClearLRFillerRule	9-104
dbClearMustJoinPortByCell	9-104
dbClearNetLengthRouteRuleFromTable	9-105
dbClearRefLibControl	9-106
dbClearVTFiller	9-107
dbCloseCell	9-108
dbCloseLib	9-108
dbCloseQueryFile	9-110
dbCompNetHalfBBox	9-110
dbConnect	9-111
dbConnectPortInstByName	9-112
dbConvertPortToDiodePort	9-114

dbConvertRegionToMacroCell.	9-115
dbCopyCell	9-116
dbCopyCellFromAnotherLib	9-117
dbCopyCellToAnotherLibDisk	9-119
dbCopyCellWithViewFromAnotherLib	9-120
dbCopyLib	9-121
dbCopyTextAndRectangle	9-122
dbCreateBaseArray	9-122
dbCreateBaseArrayRow	9-124
dbCreateCell	9-125
dbCreateCellBndyForChildCell	9-127
dbCreateCellBoundary	9-128
dbCreateCellBoundaryForCell	9-129
dbCreateCellInst	9-130
dbCreateCellInstArray	9-132
dbCreateCellEEQClass.	9-134
dbCreateCellInstIgnoreECO	9-135
dbCreateCellInstWithoutCheckStatus	9-137
dbCreateCellLEQClass	9-139
dbCreateCellProperty	9-140
dbCreateCellRegion	9-142

dbCreateCellRow	9-143
dbCreateChildNet	9-144
dbCreateChipPort	9-146
dbCreateContact	9-148
dbCreateError	9-149
dbCreateErrorClass.	9-151
dbCreateErrorComponent.	9-152
dbCreateErrorObj	9-154
dbCreateErrorType	9-155
dbCreateGLink	9-156
dbCreateGlobalNetByScanChain	9-157
dbCreateGroup	9-158
dbCreateHierNet	9-160
dbCreateHorizontalWire	9-161
dbCreateLeqPortGroup	9-162
dbCreateLib.	9-163
dbCreateLogicView	9-165
dbCreateModelView	9-165
dbCreateMustJoinNets	9-166
dbCreateNet	9-167
dbCreateNode	9-169

dbCreateParamSet	9-171
dbCreateParentNet	9-172
dbCreatePath	9-173
dbCreatePcellDefinition	9-175
dbCreatePin	9-176
dbCreatePlanGroupAndStatus	9-177
dbCreatePolygon	9-180
dbCreatePort	9-181
dbCreatePreRoute	9-182
dbCreateProperty	9-184
dbCreatePseudoPath	9-185
dbCreateRectangle	9-187
dbCreateRegionPin	9-188
dbCreateRegionPort	9-190
dbCreateScanChain	9-191
dbCreateSGEdge	9-192
dbCreateSGPrimaryNode	9-193
dbCreateText	9-195
dbCreateVerticalWire	9-197
dbCreateWireMaster	9-198
dbDefineAbsXtalkConstraint	9-200

dbDefineAntennaAccumMode	9-202
dbDefineAntennaLayerRatioScale	9-202
dbDefineAntennaRule	9-203
dbDefineCTSPin	9-205
dbDefineCTSPort	9-207
dbDefineDifferentialGroup	9-209
dbDefineIgnorePin	9-209
dbDefineIgnorePort	9-211
dbDefineLengthRouteRuleTable	9-212
dbDefineLeqPortGroup	9-213
dbDefineSyncPin	9-214
dbDefineSyncPort	9-219
dbDefineVarRouteRule	9-224
dbDefineXtalkConstraint	9-228
dbDeleteAllVarRouteRules	9-230
dbDeleteCell	9-231
dbDeleteCellAttachedFile	9-232
dbDeleteCellInstFromCellRegion	9-233
dbDeleteCellRowDoubleBack	9-234
dbDeleteFloatingCell	9-235
dbDeleteInstAndStitch	9-235

dbDeleteLibAttachedFile	9-236
dbDeleteLeqPortGroup	9-237
dbDeleteLogicData	9-238
dbDeleteLogicView	9-239
dbDeleteModelData.	9-240
dbDeleteModelView.	9-241
dbDeleteNetClass	9-241
dbDeleteObject	9-243
dbDeleteOneLayerRectangle	9-244
dbDeletePcellDefinition	9-244
dbDeletePin.	9-245
dbDeletePlacementChain	9-246
dbDeletePolygonByLayer	9-247
dbDeleteProperty	9-248
dbDeletePropertyById	9-248
dbDeletePropertyByName.	9-249
dbDeleteRectangleByLayer.	9-250
dbDeleteRefLibByName	9-251
dbDeleteScanChain	9-252
dbDeleteTieHiLoCell	9-253
dbDisconnect.	9-254

dbDisconnectPortInstByName	9-255
dbDisJoinPort	9-256
dbDisplayCellRailProperty.	9-257
dbDisplayPlanGroup	9-258
dbDowngradeLib	9-259
dbDumpAllCriticalNets	9-259
dbDumpAllNetLengthRouteRuleFromTable.	9-260
dbDumpAllNetMinMaxLayer	9-261
dbDumpAllNetTimingSpacing	9-262
dbDumpAllNetTopLayerProbe	9-262
dbDumpAllVarRouteRules.	9-263
dbDumpAllVTFillers.	9-264
dbDumpCell.	9-265
dbDumpCellAttachedFile.	9-266
dbDumpCellDifferentialGroup	9-267
dbDumpCellPlacement	9-267
dbDumpCellVTType	9-269
dbDumpCheckNetlistVsFram	9-270
dbDumpChildParentInfo	9-271
dbDumpCTSPin	9-273
dbDumpFootPrintEQ	9-274

dbDumpFreezeRoutingNets	9-275
dbDumpGPortTable	9-276
dbDumpHierRefLibControl	9-277
dbDumpIgnorePin	9-278
dbDumpLengthRouteRuleTable	9-279
dbDumpLeqPortGroup	9-280
dbDumpLeqPortGroupCandidates	9-281
dbDumpLibAntennaRules	9-282
dbDumpLibAttachedFile	9-283
dbDumpLibCatalog	9-284
dbDumpLibraryCellsVTType	9-285
dbDumpLRFillerRule	9-286
dbDumpNetXtalkProp	9-287
dbDumpPlacementChains	9-288
dbDumpPropertyTable	9-288
dbDumpRefLibControl	9-289
dbDumpScanChain	9-290
dbDumpScanPortPairs	9-290
dbDumpSingleViewPageNumber	9-291
dbDumpSyncPin	9-292
dbDumpTechFile	9-293

dbDumpViewPageNumber	9-295
dbDumpViewRecordNumber	9-295
dbDumpVTFiller	9-296
dbDumpXtalkConstraints	9-297
dbEliminatePins	9-298
dbExistsCellAttachedFile	9-299
dbExistsLibAttachedFile	9-300
dbExpungeAllObjByTypeId	9-300
dbExpungeAllObjByTypeName	9-301
dbExpungeObjTypeById	9-302
dbExpungeObjTypeByName	9-303
dbExtractCellPGPosition	9-304
dbExtractPGPosition	9-304
dbExtractPins	9-305
dbFetchCellPropertyValueByName	9-305
dbFetchContactCodeInfo	9-306
dbFetchContactCodeTypes	9-307
dbFetchContactGeom	9-308
dbFetchContactNames	9-309
dbFetchDesignRuleInfo	9-310
dbFetchDesignRuleTypes	9-311

dbFetchFringeCapInfo	9-312
dbFetchLayerIdByName	9-313
dbFetchLayerIdByNumber	9-314
dbFetchLayerIdList	9-315
dbFetchLayerInfo	9-316
dbFetchLayerTypes	9-317
dbFetchNonDefaultContactCodeInfo	9-318
dbFetchObject	9-319
dbFetchObjectField	9-321
dbFetchPinInstByPortInst	9-324
dbFetchProperty	9-325
dbFetchPropertyFieldAndName	9-325
dbFetchPropertyValueByName	9-326
dbFetchScanTypes	9-328
dbFetchTechInfo	9-329
dbFetchTechTypes	9-330
dbFileCompare	9-330
dbFixCellData	9-332
dbFixNetList	9-333
dbForceRecompCellBBox	9-334
dbForceRecursiveUpdateAllHierarchy	9-335

dbForceUpdateHierarchy	9-336
db-foreach	9-337
dbFormNetClass	9-346
dbFreeCleanPages	9-348
dbFreezeRoutingOnNets	9-348
dbGenNetList	9-350
dbGetLayerNumberFromName	9-351
dbGetLayerNameFromNumber	9-352
dbGetLibIdByCellId	9-353
dbGetNetByName	9-354
dbGetPinByName	9-356
dbGetPortByName	9-357
dbGetRegionIdByCellInstId	9-358
dbGetRegionPortByName	9-359
dbGetRegionPortDirection	9-360
dbGetAccessMode	9-360
dbGetAllInstFromFence	9-362
dbGetAllInstFromGroup	9-363
dbGetAllLibCellInfo	9-363
dbGetCellAttachedFile	9-364
dbGetCellByInstPath	9-365

dbGetCellFullNameById	9-366
dbGetCellInstById	9-366
dbGetCellInstByName	9-367
dbGetCellInstRefLib	9-368
dbGetCellNameAndViewById	9-369
dbGetCellNameById	9-370
dbGetCellViewVersions	9-371
dbGetCheckLibMode	9-372
dbGetConnectedObject	9-372
dbGetConstraintsCellId	9-373
dbGetCurrentLibId	9-373
dbGetDiodePortByName	9-374
dbGetLayerNameByMaskName	9-374
dbGetLayerNumberByMaskName	9-375
dbGetLibAttachedFile	9-376
dbGetLibBusNameStyle	9-376
dbGetLibDbVersion	9-377
dbGetLibIdByName	9-378
dbGetLibNameById	9-379
dbGetLibraryPath	9-379
dbGetLibRefCtrlFileMode	9-380

dbGetLModelSubType	9-381
dbGetMaxMetalRoutingLayer	9-382
dbGetMsgLevel	9-383
dbGetNetVarRouteRuleName	9-383
dbGetObjectTypeName	9-384
dbGetOneLibCellInfo	9-385
dbGetPinNetByName	9-385
dbGetPlanGroupByName	9-386
dbGetRefControlFileMode	9-386
dbGetRefLib	9-387
dbGetTolerantMode	9-388
dbGetUserHierDelimiter	9-389
dbGetVarRouteRule	9-389
dbHierCopyToExistLib	9-390
dbHierCopyToNewLib	9-392
dbHierDelete	9-394
dbHierPurge	9-395
dbHierSave	9-396
dbHierSplit	9-397
dbId?	9-401
dbImportLogicData	9-401

dbImportModelData.	9-402
dbInitShapeStruct	9-404
dbIOPinTextToPin	9-405
dbIsCaseSensitive.	9-406
dbIsCellOpen.	9-407
dbLastECONum	9-408
dbLibraryCompare.	9-408
dbLibReportTiming	9-410
dbListPcellDefinitions	9-411
dbListUnitRange	9-412
dbLoadPropertyDef.	9-412
dbMakeScanChainByCellInst	9-413
dbMakeScanChainByMaster.	9-414
dbMakeWireMasterId	9-416
dbMarkCellType.	9-417
dbMarkScanPortPairs	9-419
dbMemProfile	9-420
dbMemProfileReport	9-421
dbMergeLib	9-421
dbMoveAllCellRegions	9-424
dbMoveAllNetMember	9-425

dbMoveCellOrigin	9-429
dbNetCheckSum	9-430
dbObjId?	9-439
dbOpenCell	9-440
dbOpenLib.	9-441
dbOpenLibrary.	9-442
dbOpenQueryFile	9-443
dbOpenRefLibTree	9-443
dbPadToPin	9-444
dbPinToPad	9-444
dbPinToPadAndSnap	9-445
dbPrintAllNetClasses.	9-446
dbPrintApplParam	9-447
dbPrintCellHierReport	9-448
dbPurgeAllEmptyNet.	9-451
dbPurgeAllPlanGroup	9-451
dbPurgeAllSingleNet.	9-452
dbPurgeCellInstByName	9-453
dbPurgeCellInstMaster	9-453
dbPurgeCTSPin	9-454
dbPurgeIgnorePin	9-456

dbPurgeNet	9-457
dbPurgeNetByName	9-458
dbPurgeRefLib	9-458
dbPurgeLibStaticNoiseMargin	9-459
dbPurgeLibStaticNoiseProperty	9-460
dbPurgeObject	9-460
dbPurgePin	9-462
dbPurgePort	9-463
dbPurgeRegionPin	9-464
dbPurgeRegionPort	9-465
dbPurgeSyncPin	9-465
dbPurgeUnusedWireMasters	9-466
dbQuery	9-467
dbQueryCPUTime	9-468
dbQueryMem	9-469
dbQueryNetClassByName	9-469
dbQueryObjectByExtendDist	9-470
dbQueryPeakMem	9-472
dbQueryPeriodPeakMem	9-472
dbQueryPeriodPeakMemBegin	9-473
dbQuitShapeStruct	9-473

dbRebuildLib	9-474
dbRecoverLib	9-475
dbRecursiveUpdateAllHierarchy	9-476
dbRefLibCheckSum.	9-477
dbRefreshLib	9-483
dbRegisterProperty	9-484
dbRemoveInstGroupFromScanChain	9-486
dbRemoveObjectFromGroup.	9-487
dbRenameCell.	9-488
dbRenameInstAndNet	9-489
dbRenameLib	9-490
dbReOpenRefLibForWrite	9-491
dbReplaceAllCellBoundary	9-492
dbReplaceCellBoundary	9-493
dbReplaceCellInst	9-493
dbReplaceCellTextChar.	9-495
dbReplaceCellTextStr	9-496
dbReplaceChildCell.	9-497
dbReplaceRegionWithInst.	9-498
dbReplaceRefLibByName	9-499
dbReplaceScanCell.	9-500

dbReplaceTechFile	9-501
dbResetAllNetTimingSpacing	9-502
dbResetAllNetTopLayerProbe	9-503
dbResetCellMustJoinPin	9-504
dbRestoreHierDesign	9-505
dbRestorePin.	9-507
dbRollBackCell	9-509
dbSaveCell	9-510
dbSetArrivalTime	9-511
dbSetBaseArraySubTile	9-511
dbSetCellInstECOStatus	9-512
dbSetCellInstECOStatusByName	9-514
dbSetCellInstFixStatusByName	9-515
dbSetCellInstOrigin	9-516
dbSetCellInstGroupId	9-517
dbSetCellInstPlacement	9-518
dbSetCellInstStatus	9-520
dbSetCellMustJoin2Pin	9-521
dbSetCellMinMaxLayer	9-522
dbSetCellMustJoinAllPin	9-524
dbSetCellMustJoinPin	9-525

dbSetCellMustJoinPinPattern	9-527
dbSetCellPattern	9-528
dbSetCellPortTypes	9-530
dbSetCellPortTypeExtraView	9-534
dbSetCellPowerValue	9-535
dbSetCellPRBoundaryPolygon	9-536
dbSetCellRegionBBox	9-537
dbSetCellRegionTargetPercent	9-539
dbSetCellRowMirror	9-540
dbSetCellType	9-541
dbSetCellVTType	9-542
dbSetCheckLibMode	9-543
dbSetCurrentRegionNumber	9-544
dbSetEcoWeight	9-545
dbSetErrorClassColor	9-546
dbSetErrorClassPattern	9-547
dbSetErrorClassStatus	9-548
dbSetErrorObjStatus	9-550
dbSetErrorStatus	9-551
dbSetErrorTypeStatus	9-552
dbSetFringeFactor	9-553

dbSetGroupParentId	9-554
dbSetHierRefLibMode	9-554
dbSetIOPad	9-555
dbSetLayerTimeInfo	9-556
dbSetLayoutSubType	9-557
dbSetLibBusAppendName	9-559
dbSetLibBusNameStyle	9-559
dbSetLibDefaultCreationAccess	9-561
dbSetLibRefCtrlFileMode	9-562
dbSetLibraryCellsVTType	9-563
dbSetMacroCellOrient	9-564
dbSetMaxMetalRoutingLayer	9-567
dbSetMsgLevel	9-568
dbSetLModelSubType	9-569
dbSetMaxCapacitance	9-571
dbSetMaxIntraCapDistRatio	9-572
dbSetMaxTransition	9-573
dbSetNetCriticality	9-574
dbSetNetSubTypeByName	9-575
dbSetNetTrace	9-577
dbSetNetTraceById	9-578

dbSetNetType	9-578
dbSetNetTypeByName	9-580
dbSetOperatingVoltage	9-581
dbSetPathConstraint	9-581
dbSetPinAccessDir	9-584
dbSetPinAccessDirByName	9-586
dbSetPinLayerNumber	9-588
dbSetPinLocation	9-589
dbSetPinPointArray	9-591
dbSetPinStatusFlag1.	9-592
dbSetPlanGroupAndStatus	9-593
dbSetPlanGroupBBox	9-595
dbSetPlanGroupBBoxByName	9-596
dbSetPolygonPointArray	9-597
dbSetPortInstLocalStatus	9-598
dbSetPortDirection	9-599
dbSetPortInstNetId	9-600
dbSetRefLibControl	9-601
dbSetRefControlFileMode	9-602
dbSetReqArrivalTime	9-603
dbSetScanChainDefaultPriority.	9-603

dbSetSkipRefLibReadLock	9-604
dbSetStdCell	9-605
dbSetStdCellOrient	9-605
dbSetTechFileName	9-607
dbSetUserLogFile	9-608
dbSetUseUmask	9-609
dbSetXtalkLengthWeight	9-610
dbSkipRouteOnNet	9-611
dbSleep	9-612
dbSpecifyLRFiller	9-612
dbSpecifyVTFiller	9-614
dbStartCellRowDoubleBack	9-616
dbStoreCellAttachedFile	9-617
dbStoreLibAttachedFile	9-618
dbSwapCellViewVersions	9-620
dbSyncRefLib	9-621
dbTruncateCellText	9-621
dbTruncateText	9-622
dbUpdateCellInstNumberOfCellRow	9-623
dbUpdateCellProperty	9-624
dbUpdateCellRailProperty	9-625

dbUpdateHierarchy	9-626
dbUpdateNetStatus	9-627
dbUpdatePcellDefinition	9-628
dbUpdateProperty	9-629
dbVerifyCellEEQClass	9-630
dbVerifyCellLEQClass	9-631
dbVerilogTranslateString	9-631

15. define Commands for Milkyway

defineAntennaArea	10-4
defineBindKey	10-5
defineBooleanFunction	10-7
defineCellDontTouch	10-7
defineCellDontUse	10-7
defineCellFanoutLoad	10-8
defineCellMaxCapacitance	10-8
defineCellMaxFanout	10-9
defineCellMaxTransition	10-10
defineCellMinCapacitance	10-10
defineCellMinFanout	10-11
defineCellMinTransition	10-12
defineCondMinClkPeriod	10-12

defineCondMinClkPulseWidth	10-14
defineCondTimeIntrinsic	10-17
defineCondTimeTLU	10-38
defineDiodeProtection	10-46
defineExtAntennaArea	10-47
defineFlipFlopFunction	10-48
defineFootPrintEQCell.	10-50
defineGateSize	10-51
defineHierAntennaProp	10-53
defineInternalPower.	10-54
defineLatchFunction	10-58
defineLeakagePower	10-59
defineMinClkPeriod	10-63
defineMinClkPulseWidth	10-64
definePad	10-66
definePortCapacitance	10-67
definePortFeedthru	10-70
definePortPhaseDelay.	10-71
definePortTieHigh	10-72
definePortTieLow.	10-73
defineStaticNoiseMargin	10-74

defineStaticNoiseWidth	10-75
defineSwitchingPower	10-76
defineTimeIntrinsic	10-80
defineTimeTLU	10-101
defineTriStateFunction.	10-108
16. eip Commands for Milkyway	
eipPop	11-2
eipPush	11-2
eipQuit.	11-4
17. ek Commands for Milkyway	
ekGetPostRouteCapMultiplier	12-2
ekGetPostRouteResMultiplier	12-2
ekSetPostRouteCapMultiplier	12-3
ekSetPostRouteResMultiplier	12-4
18. ek Commands (No license)	
ekGetCoupleCapThreshold.	13-2
ekSetCoupleCapThreshold	13-2
ekSetLPEMode	13-3
ekSetLPEOperatingConds	13-4
ekSetParasiticSource	13-4

19. fc Commands for Milkyway

fcPad	14-2
-----------------	------

20. ge Commands for Milkyway

geAddBlockage	15-9
geAddBoundary	15-10
geAddCell	15-11
geAddContact	15-13
geAddContactArray	15-15
geAddPath	15-17
geAddPolygon	15-20
geAddRectangle	15-22
geAddRectilinearBoundary	15-23
geAddRectilinearBoundaryForChildCell	15-23
geAddRectPin	15-24
geAddRoutingBlockage	15-27
geAddSMPin	15-30
geAddText	15-31
geAddWire	15-33
geAllError	15-39
geBackupCell	15-39
geCheckMinGrid	15-40

geCheckPinsOnUserGrid	15-42
geCloseLib	15-45
geCloseWindow.	15-47
geConfirmCloseLib	15-50
geConvertInstPinToPin	15-51
geConvertWireToPin	15-52
geCopy	15-53
geCopyCell	15-56
geCreateCell	15-57
geCreateConnView	15-58
geCreateLogicView	15-66
geCreateModelView	15-69
geCurrentError	15-69
geCurrentErrorType.	15-70
geDefineBindKey.	15-70
geDelete	15-70
geDeleteCell	15-71
geDeleteRuler	15-73
geDeselectAll	15-74
geDeselectAllObject	15-74
geDeselectObject	15-75

geErrorBrowser	15-76
geFirstError	15-80
geFirstErrorType	15-81
geFit	15-81
geFitBorder	15-82
geFitSelSet	15-82
geGetEditCell	15-83
geGetEnvValue	15-83
geGetInputData	15-85
geGetSelectedSet	15-86
geGoToError	15-87
geGridOption	15-88
geImportPDB	15-90
geImposeCell	15-91
geIsLayerSelectable	15-92
geIsLayerVisible	15-93
geIsObjectSelectable	15-94
geIsObjectVisible	15-94
geLastView	15-95
geLayerPanel	15-96
geLineDeselect	15-100

geLineSelect	15-100
geListCoordinatesOfAllErrorType	15-101
geListErrorSummary	15-101
geListHierarchy	15-104
geLoadDetailRouteErrCell.	15-105
geLoadErrorCell	15-105
geMerge	15-106
geMergeFatBlockage.	15-107
geModify	15-108
geMove	15-126
geMoveCell	15-128
geMoveGroup	15-131
geMovePoint	15-132
geMoveRegion	15-133
geMoveSMPin	15-135
geNameSelect.	15-136
geNewMakeMacro.	15-139
geNewShortLocator.	15-153
geNextError	15-155
geNextErrorType	15-155
geOpenCell	15-156

geOpenLib	15-158
geOpenLocator	15-160
geOutputBBox	15-161
geOutputLine	15-161
geOutputPoint	15-162
gePan	15-162
gePan2	15-162
gePointDeselect	15-163
gePointSelect	15-163
gePrepLibs	15-164
gePreviousError.	15-174
gePurgeCell.	15-174
geQueryError.	15-176
geQueryGroup.	15-177
geQueryGroupName.	15-178
geQueryNet.	15-180
geQueryNetConstraint.	15-181
geQueryNetName	15-182
geQueryObject	15-185
geRedo	15-185
geRedraw	15-186

geReShape	15-186
geRollbackCell.	15-188
geRuler	15-188
geSaveAllOpenCells	15-193
geSaveAs	15-194
geSelectAll	15-195
geSelectInstancesOnNet.	15-195
geSelectObject	15-196
geSetCellInstColor	15-197
geSetConfirmPinDeletionFlag.	15-199
geSetCustomRouteTypesVis.	15-199
geSetErrorType	15-200
geSetEnvValue	15-203
Environment Variables That Are Unique to Each Window.	15-204
Environment Variables That Are Unique to Each Cell	15-204
Environment Variables That Are Global and Apply to All Windows	
15-205	
geSetLayerSelectable	15-207
geSetLayerVisible	15-208
geSetObjectSelectable	15-209
geSetObjectVisible	15-210
geSetNetTrace.	15-211

geSetNetTypesVis	15-213
geSetViewCell	15-214
geShowCellList	15-215
geSmashCellInst	15-216
geSplit	15-217
geStretch	15-218
geSummary	15-219
geSuppressHltRedraw.	15-220
geSwapCells	15-221
geTransform.	15-222
geTurnErrorOff.	15-224
getXWindowId	15-224
geUndo	15-225
geUnpackCellInstArray	15-226
geViewDown	15-226
geViewLeft.	15-227
geViewRight	15-227
geViewUp	15-228
geWindowDeselect	15-228
geWindowOption	15-229
geWindowSelect	15-233

geWindowStretch	15-234
geZoomIn	15-236
geZoomInBy2	15-236
geZoomOutBy2	15-236
geZoomOutBy4	15-237
geZoomOutBy8	15-237
setInstViewLevel	15-237
addRelativePoint	15-238
layoutWindowGrabFocus.	15-239

21. hdl Commands for Milkyway

hdlGetBusDirectionAsSameWay	16-2
hdlGetNoPGPortForStdCellsOnly	16-2
hdlGetNoPortNetInconsistent	16-3
hdlSetNotCloseCell	16-3
hdlGetOutputLowerCase	16-4
hdlSetBusDirectionAsSameWay	16-5
hdlSetNoPortNetInconsistent	16-5
hdlSetOutputLowerCase	16-6

22. hi Commands for Milkyway

hiGetCurrentWindow	17-2
------------------------------	------

hiQuit.	17-2
23. hmi Commands for Milkyway	
hmiA2DRC	18-3
hmiA2LVS	18-7
hmiADRC	18-10
hmiADRCAddFile	18-11
hmiADRCDelFile	18-12
hmiExplorerDRC	18-14
hmiExplorerLVS.	18-14
hmiExplorerOptions.	18-14
hmiHercules	18-15
hmiHierAntenna.	18-19
24. hnet Commands for Milkyway	
hnetChangeInstName	19-2
hnetCreate.	19-3
hnetCreateHierNetlist	19-3
hnetCreateHierVlogOut.	19-7
hnetDetachPortInstFromPGNet.	19-15
25. io Commands for Milkyway	
ioDumpIOLocation.	20-2

ioDumpPinLocation	20-3
ioDumpTdf.	20-4
ioDumpTdfForChildCell	20-5
ioLoadTdf	20-6
ioReplacePads	20-8

26. Commands With No Prefix (No License)

addUserMenu	21-2
advSearch	21-2
cnvGetBuffering.	21-7
cnvSetBuffering	21-8
exit.	21-8
FeatureCheckedOut	21-9
functions	21-9
getFeature	21-10
getFunctionLicensing	21-15
listFeature	21-16
releaseFeature	21-17
removeUserMenu	21-21
setPostCallBack.	21-21
setPreCallBack	21-23
vars	21-24

varValue.	21-25
27. Commands With No Prefix for Milkyway	
->.	22-6
->type	22-6
augLibCompiler	22-7
batchMode.	22-13
bell.	22-14
boolNotPolygons	22-14
clearAll.	22-16
closeImageCell	22-17
CONSTANT	22-18
cPin	22-19
cPinF	22-20
createImageCell	22-21
createRowRecord	22-22
createTilePatternRecord	22-23
createTrackRecord	22-25
createWireDirRecord.	22-27
cycle	22-27
debug.	22-28
demOpen.	22-38

dvSnapWindow	22-49
FileExists?	22-49
FileIsExecutable?	22-50
FileIsReadable?	22-50
FileIsWritable?	22-51
flipChipMatrix.	22-51
flipChipPad	22-52
formDefault	22-54
fpGetFPOption	22-56
fpSetFPOption.	22-58
genUniformTileInLib	22-64
getCaseSensitive.	22-66
getDisplayTrackingMsg	22-67
getEditWindow.	22-67
getFormField	22-67
getFormFieldValue	22-68
go	22-69
hiliteNet	22-70
hvSetDebugNetMapping	22-70
IsApplicationIn64BitMode	22-71
lib2lclf	22-71

listBindKey	22-72
loadFromPath	22-72
loadUserForm	22-73
loadUserFormFromPath	22-74
make-db.	22-74
net	22-75
netCapacitance	22-76
netDelay	22-77
netLength	22-79
netTransition	22-79
notDoneYet	22-81
obsoleteCmd	22-81
pad	22-82
pin	22-83
rPin	22-85
Plot Commands.	22-86
plotAddBoxText	22-86
plotAddPath.	22-87
plotAddPolygon	22-88
plotAddRectangle	22-89
plotAddScreenText	22-89

plotAddSegment	22-90
plotAddText	22-91
plotClear	22-92
plotFlush	22-92
plotSelectWindow	22-93
plotSetDrawAttr	22-94
plotSetScale	22-95
plotWindow	22-95
printSystemInfo	22-96
repeatTile	22-98
replay.	22-98
rPin	22-99
scmxConcatFunction.	22-100
scmxLoad	22-102
scmxPreprocess	22-102
setCaseSensitive.	22-103
setConfigurationPrinting	22-104
setDisplayTrackingMsg	22-105
setFieldDefault.	22-106
setIconificationPrinting	22-107
ssClearAllSSViewports	22-108

ssSetViewportMode	22-109
strToRegExp	22-109
system	22-110
systemExec	22-111
tcl	22-112
tclLoad	22-112
undefineBindKey	22-113
userFormCancel	22-114
userGetTclVar	22-114

28. po Commands for Milkyway

poAndPolygons	23-3
poConnViewPreparation	23-3
poCreateConnView	23-17
poCreateConnViewByHerc	23-27
poDumpCurrSource	23-36
poFillPolygonsToRectangles	23-38
poGenCurrSource	23-39
poNotPolygons	23-41
poOrPolygons	23-42
poOversizePolygon	23-42
poOversizeUndersizePolygon	23-43

poPathToPolygon	23-43
poPolygonArea	23-44
poPolygonToRectangles	23-44
poPurgeCurrSource	23-45
poXorPolygons	23-47
29. pvr Commands for Milkyway	
pvrDSPFIn	24-2
pvrSPEFIn	24-6
30. tdf Commands for Milkyway	
tdfDump	25-4
tdfDumpIOToTDF	25-5
tdfDumpPinToTDF	25-6
tdfDumpPowerInfo	25-8
tdfPurgeClks	25-8
tdfPurgeClockPhaseDelays	25-9
tdfPurgeClockWaveforms	25-9
tdfPurgeDrivingCell	25-10
tdfPurgeInputDelay	25-10
tdfPurgeIOSpaceConstr	25-11
tdfPurgeNetCapAndTransTimes	25-11

tdfPurgeNetCapTransAndDelayTimes.	25-12
tdfPurgeNetConstr.	25-12
tdfPurgeNetTransitionDelays.	25-13
tdfPurgeNoTouchCellInst.	25-13
tdfPurgeNoTouchNet.	25-14
tdfPurgeOutputDelay.	25-14
tdfPurgePadConstr	25-15
tdfPurgeTimeRequirements	25-15
tdfResetPath	25-16
tdfSetCaseAnalysis	25-19
tdfSetClkDef	25-19
tdfSetClockPhaseDelay.	25-22
tdfSetClockPortTransition	25-29
tdfSetClockWaveform	25-31
tdfSetDisableTiming	25-32
tdfSetDrivingCell	25-35
tdfSetFalsePath	25-37
tdfSetGeneratedClk.	25-40
tdfSetInputDelay	25-42
tdfSetInputPortTransition	25-46
tdfSetLoadCap	25-48

tdfSetMaxDelayPath	25-49
tdfSetMinDelayPath	25-52
tdfSetMinIOSpacing	25-54
tdfSetMultiCyclePath	25-55
tdfSetNetCapAndTransTime	25-61
tdfSetNetCapTransAndDelayTime.	25-63
tdfSetNetTransitionDelay	25-65
tdfSetNoTouchCellInst.	25-67
tdfSetNoTouchNet	25-68
tdfSetOutputDelay	25-69
tdfSetPowerSupply	25-72
tdfSetRestoreTiming	25-73
tdfSetSrcCapacitance	25-75
tdfSetSrcCapacitanceList	25-78
tdfSetSrcResistance	25-82
tdfSetSrcResistanceList	25-85
tdfSetTimeBorrowLimit	25-88
31. tg Commands for Astro Family Products and Milkyway	
tgCheckTimingConstraint	26-2

32. tr Commands for Milkyway

trCancel 27-3

33. tu Commands for Milkyway

tuHelp 28-2

tuHelpCancel 28-2

34. wv Commands for Milkyway

wvCloseSource 29-3

wvCloseView 29-3

wvDeleteGraph 29-3

wvDeleteView 29-4

wvDeleteWave 29-4

wvNewGraph 29-5

wvNewView 29-5

wvOpenSource 29-6

wvOpenView 29-6

wvPlotWave 29-7

wvSelectWave 29-7

wvSetGraphCurrent 29-8

wvSetSourceCurrent 29-8

wvSetView 29-8

wwSetViewCurrent.	29-9
wwViewClear	29-9
wwViewDebug	29-10

About Milkyway Help

The following sections describe basic information about the Synopsys Online Help:

- [Starting Online Help](#)
- [What is New in This Release](#)
- [Additional Information](#)
- [Customer Support](#)

Starting Online Help

To access tool Help, type `help` at the command line in the tool GUI. When the Help system loads into a standard browser, click the "Help QuickStart link" at the bottom of the a Help page for brief information on using Help. Alternatively, you can click the Help button in a dialog box to view Help on a command or topic.

You can also click the "PDF Documentation" link at the bottom of a Help page to view and print the Acrobat PDF versions of the manuals. For best performance, set the `setHelpBrowser` variable to Firefox or Mozilla. This is covered in detail in the *Help Quick Start*.

The online Help is in HyperText Markup Language (HTML) format. You can view these files with any standard browser.

Starting Online Help

Type `help "command_name"` in the command area of the tool or click Help in any tool dialog box to view the online Help for that command.

For example,

```
help "geOpenLib"
```

This opens the `geOpenLib` topic in the online Help.

What is New in This Release

Information about new features, enhancements, and changes; known problems and limitations; and resolved Synopsys Technical Action Requests (STARs) is available in the in SolvNet.

To see release notes for Synopsys products,

1. Go to <http://solvnet.synopsys.com/ReleaseNotes>. (If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to register with SolvNet.)
2. Click a product name, then click the release you want in the list that appears at the bottom.

Additional Information

The following section covers related publication for to supliment the information provided in Help. Also, this section includes information on conventions used in this documentation.

Related Publications

For additional information about Synopsys tools, see

- Synopsys Online Documentation (SOLD), which is included with the software for CD users or is available to download through the Synopsys Electronic Software Transfer (EST) system
- Documentation on the Web, which is available through SolvNet at <http://solvnet.synopsys.com>

You might also want to refer to the documentation for the following related Synopsys products:

- Astro
- Astro Interactive Ultra and Columbia
- Astro-Rail
- JupiterXT
- Milkyway
- PrimeRail

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
menuName > commandName	Indicates the name of the menu and the command name. For example, <i>Cell > Open</i> refers to the <i>Open</i> command in the Cell menu.
Tool: menuName > commandName	Indicates that a command is accessible only through an application tool. Tool is the tool through which you access the command, <i>menuName</i> is the name of the menu, and <i>commandName</i> is the name of the command.
<code>Courier</code>	Indicates command syntax, a function, or a keyword that you must type exactly as shown. In examples, this font indicates system prompts, text from files, and messages printed by the system.

Convention	Description
<i>Courier italic</i>	Indicates a user-defined value in Synopsys syntax, such as <i>object_name</i> . (A user-defined value that is not Synopsys syntax, such as a user-defined value in a Verilog or VHDL statement, is indicated by regular text font italic.)
	Arguments to commands are strings and must be enclosed in quotation marks.
Courier bold	Indicates user input—text you type verbatim—in Synopsys syntax and examples. (User input that is not Synopsys syntax, such as a user name or password you enter in a GUI, is indicated by regular text font bold.)
[]	Denotes optional parameters, such as <i>pin1 [pin2 ... pinN]</i>
'(item1 item2)	An apostrophe followed by parentheses indicates that the text within the parentheses is a list. When the list contains multiple items, the items are separated by spaces. Type this information exactly as it appears in the syntax.
	Indicates a choice among alternatives, such as <i>low medium high</i> (This example indicates that you can enter one of three possible values for an option: low, medium, or high.)
—	Connects terms that are read as a single term by the system, such as <i>set_annotated_delay</i>
Control-c	Indicates a keyboard combination, such as holding down the Control key and pressing c.
\	Indicates a continuation of a command line.
/	Indicates levels of directory structure.
Edit > Copy	Indicates a path to a menu command, such as opening the Edit menu and choosing Copy.

Using Swish-e With Physical Implementation Online Help

The Swish-e program, which is provided for online Help, is not “Licensed Software” or “Licensed Product” in your license agreement with Synopsys. The Swish-e program is subject to the license provisions found at <http://swish-e.org>, where you can also find the source code for Swish-e.

Customer Support

Customer support is available through SolvNet online customer support and through contacting the Synopsys Technical Support Center.

Accessing SolvNet

SolvNet includes an electronic knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. SolvNet also gives you access to a wide range of Synopsys online services including software downloads, documentation on the Web, and “Enter a Call to the Support Center.”

To access SolvNet,

1. Go to the SolvNet Web page at <http://solvnet.synopsys.com>.
2. If prompted, enter your user name and password. (If you do not have a Synopsys user name and password, follow the instructions to register with SolvNet.)

If you need help using SolvNet, click HELP in the top-right menu bar or in the footer.

Contacting the Synopsys Technical Support Center

If you have problems, questions, or suggestions, you can contact the Synopsys Technical Support Center in the following ways:

- Open a call to your local support center from the Web by going to <http://solvnet.synopsys.com> (Synopsys user name and password required), then clicking “Enter a Call to the Support Center.”
- Send an e-mail message to your local support center.
 - E-mail support_center@synopsys.com from within North America.
 - Find other local support center e-mail addresses at http://www.synopsys.com/support/support_ctr.
- Telephone your local support center.
 - Call (800) 245-8005 from within the continental United States.
 - Call (650) 584-4200 from Canada.
 - Find other local support center telephone numbers at http://www.synopsys.com/support/support_ctr.

4

apr Commands for Milkyway

This chapter describes commands with the prefix *apr*. The commands are listed alphabetically.

- [aprDumpScanChain](#)
- [aprFixCell](#)
- [aprPDEF30DisplIgnoredInfo](#)
- [aprPDEFIn](#)
- [aprPDEFOut](#)
- [aprPDEFOutIgnoreFillerCells](#)
- [aprPDEFPrintRegionName](#)
- [aprSetPDEFVersionTo30](#)

aprDumpScanChain

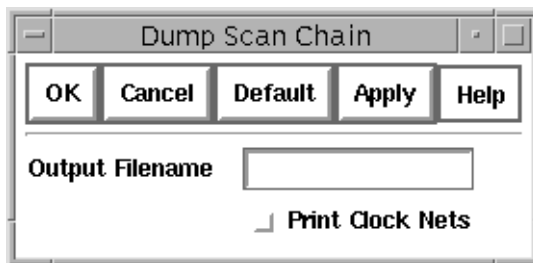
Writes scan chain information to the specified file.

Tool and Menu Command

Astro: PostPlace > Scan Chain – Dump Scan Chain

Dump Scan Chain Dialog Box

The menu command opens the following dialog box. Its options are described below.



Output Filename

Type the name of the file to which you want to save the scan chain information.

Print Clock Nets

The clock nets can be printed out in the scan chain dump file to verify the results of scan chain optimization with clock net reordering.

aprFixCell

Sets the placement status of selected cell instances, pads, region pins, and pins.

Syntax

```
aprFixCell <int mode>
```

Argument:

mode

Use the following settings:

- 0 to set selected objects as placed (i.e., unfixed)
- 1 to set selected objects as fixed and create pin/pad constraints
- 2 to set selected standard cell instances as soft fixed and all other selected objects as fixed.

Description

The `aprFixCell` command sets the placement status of selected cell instances, pads, region pins, and pins depending upon the specified mode. Before calling this command, the design cell must be opened and the desired objects must be selected.

Example

```
; open design cell
geOpenCell
setFormField "Open Cell" "Cell Name" "myDesignCell"
formOK "Open Cell"

; select objects
gePointSelect
addPoint 2 '(12.700000 -1.450000)
addPoint 2 '(2.150000 7.300000)
```

```
; call command to mark objects as fixed
aprFixCell 1
```

aprPDEF30DispIgnoredInfo

Turns on the flag so that the `aprPDEFOut` command for version 3.0 displays all the semantics that are being ignored. The argument is either `#t` or `#f`.

Tools

Astro

Jupiter

Syntax

```
aprPDEF30DispIgnoredInfo BOOLEAN
```

See Also

[aprPDEFOut](#) – Write s your layout to a file in Physical Design Exchange Format (PDEF) for export to a synthesis tool.

aprPDEFIn

Note:

The `aprPDEFIn` command has been replaced by the `read_pdef` command. The `aprPDEFIn` command is no longer supported. For information about the `read_pdef` command, see the man pages or the *Milkyway Environment Data Preparation User Guide*.

Imports a new or changed floorplan in Physical Design Exchange Format (PDEF).

The PDEF file contains information about the relationship of standard and pad cells to groups and regions. The `aprPDEFIn` command lets you bring in new or changed information from a synthesis tool, so you do not have to return to the data preparation step.

When importing a PDEF 1.0 file, which does not specify the locations of standard cells, the Synopsys application places standard cells in the center of the group or region to which they belong. When importing a PDEF 2.0 file, which specifies the locations of standard cells, the Synopsys application places standard cells as close as possible to the specified locations.

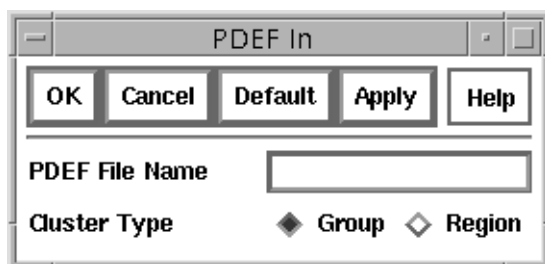
Tools and Menu Commands

Astro: Design Setup > PDEF – PDEF In

Jupiter: Setup > PDEF – In

PDEF In Dialog Box

The menu command opens the following dialog box. Its options are described below.



PDEF File Name

Type the name of the PDEF file you want to import.

Cluster Type

Select the type of clusters used in the PDEF file you want to import.

Option	Cluster type
Group	Groups
Region	Regions

aprPDEFOut

Writes your layout to a file in Physical Design Exchange Format (PDEF) for export to a synthesis tool.

Note:

The `aprPDEFOut` command has been replaced by the `write_pdef` command. The `aprPDEFOut` command is no longer supported. For information about the `write_pdef` command, see the man pages or the *Milkyway Environment Data Preparation User Guide*.

Information related to filler cells is ignored if this option was set with the [aprPDEFOutIgnoreFillerCells](#) Scheme command. This Scheme command takes #t or #f as an argument.

Tools and Menu Commands

Astro: Design Setup > PDEF – PDEF Out

Jupiter: Design Setup > PDEF – Out

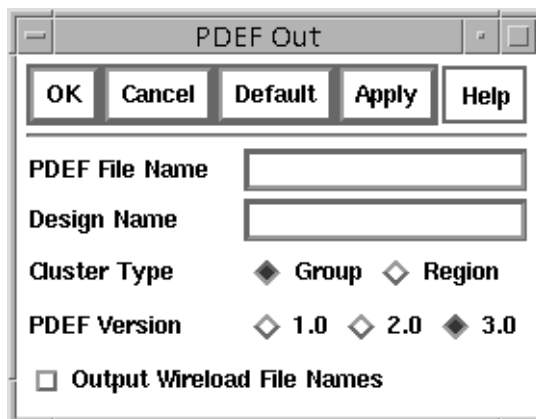
Prerequisites

If you have a hierarchical design with soft macros, do the following:

- Estimate the area of these groups by using `fphEditToolbar`.

PDEF Out Dialog Box

The menu command opens the following dialog box. Its options are described below.



PDEF File Name

Type the name you want to assign to the PDEF file created by this command.

Design Name

Type the design name.

Note:

For back-annotation, this name must match the design name used by the synthesis tool.

Cluster Type

Indicate the type of clusters you are working with in your design.

Option	Cluster type
Group	Groups
Region	Regions

If you select Region as the cluster type, the region name can be written instead of the region number if this option was set with the [aprPDEFOutIgnoreFillerCells](#) Scheme command. This Scheme command takes #t or #f as an argument.

PDEF Version

If you select Region as the cluster type, indicate the PDEF version to which you want to write your layout. Version 3.0 is the default.

Output Wireload File Names

If you select this option, the names of the wire load files will be written to the output file.

aprPDEFOutIgnoreFillerCells

Sets or resets the flag to ignore filler cells to be dumped out by the [aprPDEFOut](#) command.

Tools

Astro

Jupiter

Syntax

```
aprPDEFOutIgnoreFillerCells BOOLEAN
```

See Also

[aprPDEFOut](#) – Writes your layout to a file in Physical Design Exchange Format (PDEF) for export to a synthesis tool.

aprPDEFPrintRegionName

If you selected Region as the cluster type for `aprPDEFOut`, allows the region name to be written instead of the region number when the flag is turned on with this command.

Tools

Astro

Jupiter

Syntax

```
aprPDEFPrintRegionName BOOLEAN
```

See Also

[aprPDEFOut](#)

aprSetPDEFVersionTo30

Reads in a design netlist to version 3.0 of the Physical Design Exchange Format (PDEF). This Scheme command takes #t or #f as the argument.

Tools

Astro

Jupiter

Syntax

```
aprSetPDEFVersionTo30 BOOLEAN
```

See Also

[aprPDEFOut](#)

2

apr Commands for Astro Family Products and Milkyway

This chapter describes commands with the prefix *apr*. The commands are listed alphabetically.

- [aprHiliteCell](#)
- [aprSelectFlyline](#)
- [aprSetFlyline](#)
- [aprShowPinNet](#)
- [aprShowPinNetByName](#)
- [aprShowSelCellFlyline](#)

aprHiliteCell

Highlights a specified cell instance or all instances of a specified cell master. To remove the highlighting, click Clear.

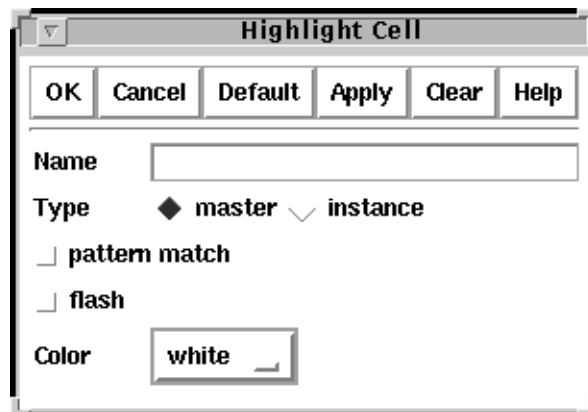
Tools and Menu Command

Astro: Query > Object – High Light Cells

Jupiter: Query > Object – High Light Cells

Highlight Cell Dialog Box

The menu command opens the following dialog box. Its options are described below.



Name

Type the name (local or hierarchical) of the cell instance you want to highlight or the name of the cell master for which you want to highlight all instances, or select “pattern match” and type a pattern to match the names of multiple cell instances or cell masters.

Use the pattern matching option to select all nets with names that match a pattern. For more information about pattern matching, see “Specifying Objects by Pattern Matching” in *Physical Implementation Getting Started*.

Type

Select the option that indicates whether the name you typed is a master or an instance.

Option	Cells highlighted
master	All cell instances of the specified cell master <i>Name</i> .
instance	The cell instance with the specified <i>Name</i> .

pattern match

Select to highlight all cells matching a pattern specified in the Name field. For example, if you type N.* in the Name field and select “pattern match,” all cell names beginning with the letter N will be highlighted.

Use the pattern matching option to select all nets with names that match a pattern. For more information about pattern matching, see .

flash

Select if you want the cell to flash (blink).

Color

Select the color you want for the cell display.

aprSelectFlyline

Displays flylines for nets (except for power, ground, and clock nets) that cross all specified edges of blocks. Optionally, you can display the Select Flyline Report (see [Select Flyline Report](#)), which contains the following information:

- The number of nets common to all specified edges
- The number of nodes (cell blocks) in the selection
- The following information for each node:
 - Type of node - Cell instance (block)
 - Number for a cell block, or instance and master names for a block
 - Number of nets crossing the node, in the format

netsCrossingSpecEdges/netsCrossingAllEdges

where *netsCrossingSpecEdges* is the number of nets that cross the specified edges of the node, and *netsCrossingAllEdges* is the number of nets that cross all edges of the node.

Note that the number given for *netsCrossingSpecEdges* includes common nets that connect to internal pins that have pin access from the direction of any of the specified edges.

- The specified edges of the node (left, right, top, bottom)
- Names and IDs for all common nets

Note:

Flylines are straight lines between net connection points. If an internal pin connecting to a common net has access from the direction of a selected edge, the display includes a flyline for the connection.

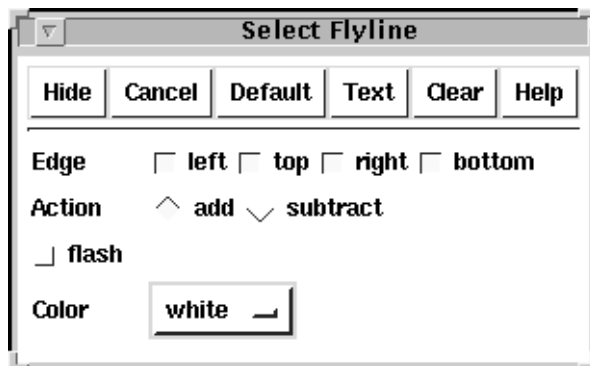
Tools and Menu Commands

Astro: Query > Flyline – Show Flyline by Object Edge

Jupiter: Query > Net – By Object Edge

Select Flyline Dialog Box

The menu command opens the following dialog box:



Use the Select Flyline dialog box as follows:

- To add a node to the display, select the “add” option and the edges of the node you want considered, then click the object in the graphics window.
- To remove a node, select the “subtract” option and click the node in the graphics window.
- To display the Select Flyline Report, click Text.

- To remove the flylines, click Clear.

The dialog box options are described below.

Edge

Select the edge or edges for which you want to display (or remove) flylines: left, top, right, bottom.

Action

Indicate whether you want to add or remove the node you select.

Option	Action
add	Adds the node to the display.
subtract	Removes the node from the display.

flash

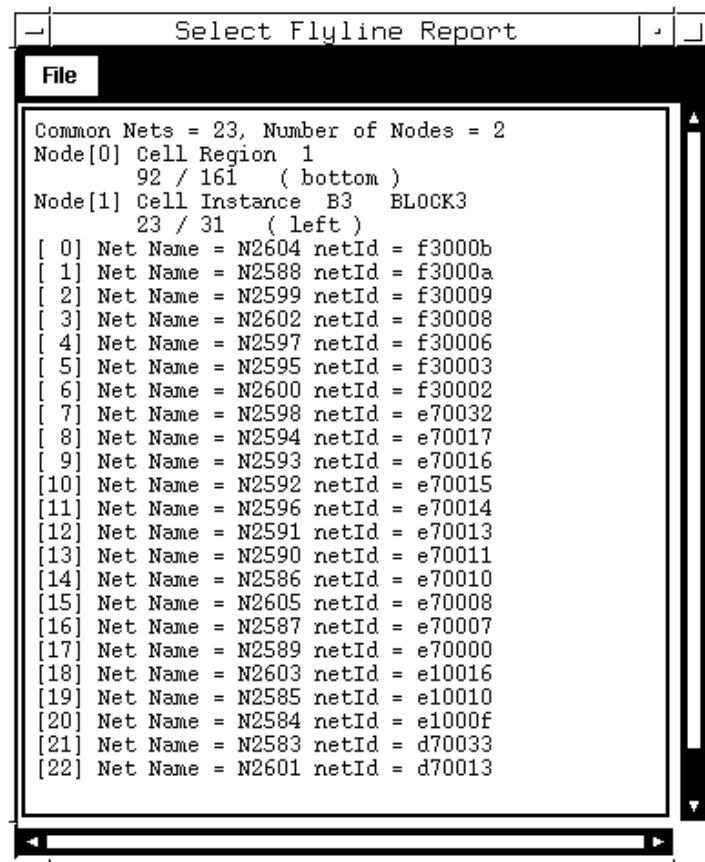
Select if you want the flylines to flash (blink).

Color

Select the color you want for displaying flylines.

Select Flyline Report

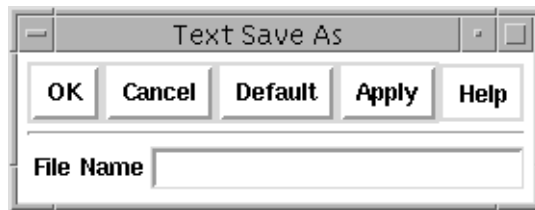
Open the Select Flyline Report by clicking Text in the Select Flyline dialog box. Use of the report window is described below.



File

Use the commands on this menu to save the text in the window to a file, load another file into the window, or close the window.

- Save saves the text in the window to a file with a name you assign by using Set File Name.
- Save As displays the Text Save As dialog box.



To save the text in the window, type a file name and click OK. The Synopsys application saves the file to the name you type.

- Load loads an existing file into the window.
- Set File Name assigns a name to the text in the window. You can then save the text to a file by using Save.
- Close closes the window.

aprSetFlyline

Displays flylines for nets in a design and reports wire length based on minimum spanning tree routing.

Flylines are straight lines between net connection points.

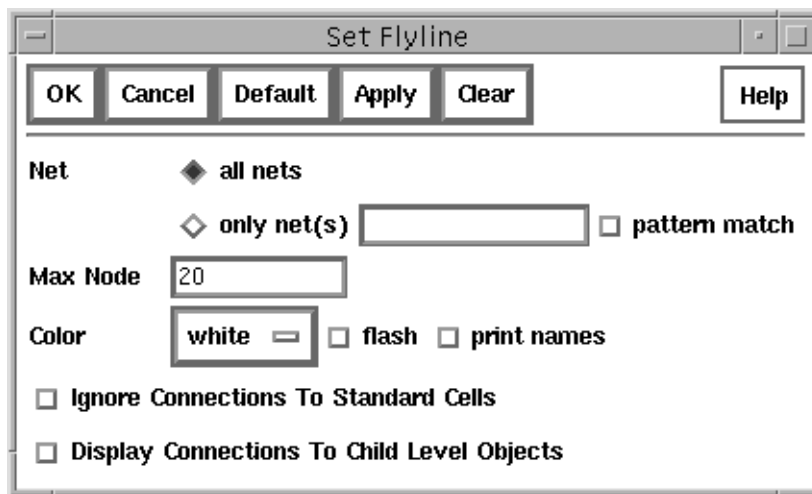
Tools and Menu Commands

Astro: Query > Flyline – Show Net Connections

Jupiter: Query > Flyline – Show Net Connections

Set Flyline Dialog Box

The menu command opens the following dialog box. Its options are described below.



Clear

Click to remove the flylines.

Net

Indicate the nets for which you want to display flylines.

Option	Net for which flylines displayed
all nets	All nets in the design.
only net(s)	Only the specified net or nets. You specify a net by typing its name in the “only net(s)” field. To specify multiple nets, separate the names with commas.
pattern match	All nets matching the pattern specified in the “only net(s)” field. For example, if you type N.* in the field and select “pattern match,” flylines will be shown for all net names beginning with the letter <i>N</i>. Use the pattern matching option to select all nets with names that match a pattern. For more information about pattern matching, see “Specifying Objects by Pattern Matching” in <i>Physical Implementation Getting Started</i>.

Max Node

Type the maximum number of nodes you want the displayed nets to have. When a net has more nodes than the value specified, the net is not displayed.

Color

Select the color you want to use for displaying flylines.

flash

Select if you want the flylines to flash (blink).

print names

Select when you want to display the net names and pins in the message/input area. This option is useful only when combined with the “only net(s)” option.

Ignore Connections To Standard Cells

Select when you do not want the connections to standard cells displayed. The flyline display ignores such connections when this option is selected.

Display Connections To Child Level Objects

Select when you want to see flylines displayed for child-level objects that the top-level net connects to. If the “print names” option is selected along with this option, the name of the child-level object and its location (with respect to top-level origin) is displayed in the message window.

aprShowPinNet

Displays flylines for a pin, region pin, cell region, or cell instance you specify by clicking in the graphics window. Flylines are straight lines between net connection points. If you select a cell instance or cell region, the flyline display excludes power, ground, and clock nets.

This command also displays information about the selected object in the message/input area.

If you select a pin or region pin, the information includes the following:

- Name of the pin (optional)
- Name of the net to which the pin is connected (optional)

If you select a cell instance, the information includes the following:

- Name of the cell master
- Name of the cell instance
- Name of each pin in the cell (optional)
- Name of the net to which each pin is connected (optional)

If you select a cell region, the information gives the number of the cell region.

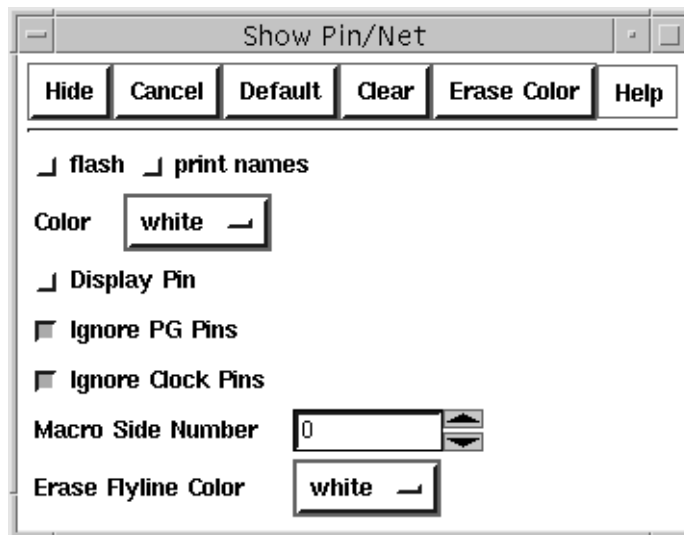
Tools and Menu Commands

Astro: Query > Flyline – Show Object Connections

Jupiter: Query > Flyline – Show Object Connections

Show Pin/Net Dialog Box

The menu command opens the following dialog box. Its options are described below.



Clear

Click to remove the flylines.

Erase Color

Click to erase flylines with the selected color.

flash

Select if you want the pins and nets to flash (blink).

print names

Select if you want to display pin and net names in the message/ input area.

Color

Select the color you want to use for displaying pins and nets.

Display Pin

Select to graphically display pins without using the Window Option dialog box ([geWindowOption](#)). Turn off this option to make macro pins invisible. The default mode is off.

Ignore PG Pins

Select (turn off) if you do not want to show flylines for PG pins. The default mode is on.

Ignore Clock Pins

Select (turn off) if you do not want to show flylines for clock pins. The default mode is on.

Macro Side Number

Type or select a number for displaying flylines for pins located at a particular side of the macro instance. The default is 0, meaning that the flyline will be shown for all pins of a macro instance.

Erase Flyline Color

Use to erase flylines that have the selected color. Click the Erase Color button at the top of the dialog box to execute the erasing function.

aprShowPinNetByName

Displays flylines for a pin instance you specify by name, and displays net information in the message/input area.

Flylines are straight lines between net connection points.

Tools and Menu Commands

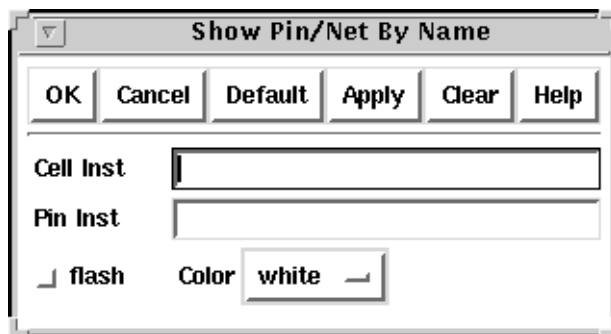
Astro: Query > Flyline – Show Pin Instance Flyline

Jupiter: Query > Pin – Instance

Jupiter, Register Level Transfer: Query > Pin – Instance

Show Pin/Net By Name Dialog Box

The menu command opens the following dialog box. Its options are described below.



Clear

Click to remove flylines.

Cell Inst

Type the name of the cell instance containing the pin you want to query.

Pin Inst

Type the instance name of the pin you want to query.

flash

Select if you want the flylines to flash (blink).

Color

Select the color you want to use for displaying flylines.

aprShowSelCellFlyline

Displays flylines between two or more selected cell instances.

GUI Form

Form Title: "Show Flyline Between Cells"

Use the following options:

Color

Select color to be used for displaying flylines. By default, the color is white.

Ignore PG Pins

Select to suppress flylines between power and ground net connection points. By default, this option is selected.

Ignore Clock Pins

Select to suppress flylines between clock net connection points. By default, this option is selected.

Description

The `aprShowSelCellFlyline` command displays flylines between two or more selected cell instances. Before calling this command, the design cell should be opened and the cell instances should be selected by clicking in the graphics window. The selected cell instances may be standard cells or macro cells.

Flylines are straight lines between net connection points (i.e., pin-to-pin connections). Flylines are used to view and debug net connections in the current session only and are not actually saved to

the database. The flyline color may be specified on the GUI form. Power, ground, and clock net flylines may be suppressed, by selecting the appropriate ignore options on the GUI form.

Example

```
aprShowSelCellFlyline
setFormField "Show Flyline Between Cells" "Color" "white"
setFormField "Show Flyline Between Cells" "Ignore PG Pins"
    "1"
setFormField "Show Flyline Between Cells" "Ignore Clock Pins"
    "1"
formOK "Show Flyline Between Cells"
```

6

aprCmd Commands for Milkyway

This chapter describes the following command with the prefix *aprCmd*:

- [aprCmdFixCell](#)

aprCmdFixCell

Places or removes a constraint on selected cells (including soft macros) or floorplan groups to prevent or allow their movement during automatic placement.

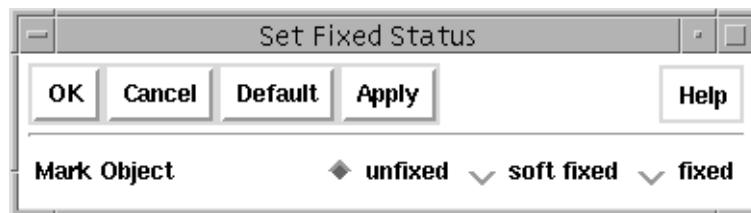
Tools and Menu Commands

Astro: PrePlace > Constraints – Set Fixed Status

Jupiter: Setup > Set Fixed Status

Set Fixed Status Dialog Box

The menu command opens the following dialog box. Its options are described below.



Mark Object

Select the type of movement constraint you want on the selected object.

Table 6-3

Option	Description
unfixed	(Also called unset or placed) Cell instances of unfixed status can be placed, moved, or optimized. However, when being placed, they cannot be placed within any (hard or soft) placement blockages.

Table 6-3

Option	Description
soft fixed	Cell instances of soft-fixed status cannot be placed or moved, but they can be optimized with netlist optimization commands. A soft-fixed cell instance cannot legally stay within hard placement blockages, but it can stay within a soft placement blockage. When necessary, netlist optimization commands can create buffers or inverters in soft placement blockage areas with soft-fixed status (for use during in-place optimization, see astPlaceDesign ; for use during postplacement optimization, see astPostPS).
fixed	Cell instances of fixed status cannot be placed or optimized. They can be moved through the geMove command only. A fixed cell instance automatically prevents netlist optimization commands from changing. That is, a fixed status implies don't touch status. A fixed cell instance can legally stay within any placement blockage (hard or soft).

See Also

[astPlaceDesign](#) – Performs standard cell instance placement for a given design.

[astPostPS](#) – Performs post-place optimizations for overall timing improvement.

[geMove](#) – Moves a selected object.

8

ast Commands for Milkyway

This chapter describes commands with the prefix *ast*. The commands are listed alphabetically.

- [astDeleteHierPreservation](#)
- [astDumpHierPreservation](#)
- [astDumpHierVerilog](#)
- [astExtrLEQ](#)
- [astInitHierPreservation](#)
- [astPostVerification](#)
- [astSetCellInstDontTouch](#)

Note:

Unless otherwise noted, all command examples are shown as you would enter them from the command line, which can differ from Scheme syntax. If you want to add commands to startup files, or use them in user-defined programs, you must use correct Scheme syntax. For more information, see the *Milkyway Environment Extension Language Reference Manual*.

astDeleteHierPreservation

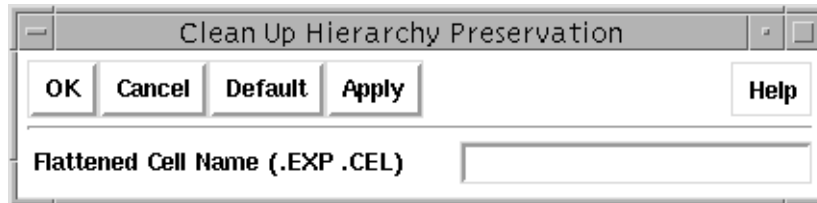
This Scheme command cleans up all the hierarchical information created by the [astInitHierPreservation](#) command in the given flattened top cell. Can also be used to clean up existing hierarchy information before it is recreated. Once the hierarchy information is deleted, the module instances cannot be preserved and the hierarchical Verilog cannot be generated using the [astDumpHierVerilog](#) command.

Tool and Menu Command

Astro: Cell > Hierarchy Preservation - Delete Hierarchy Information

Clean Up Hierarchy Preservation Dialog Box

The menu command opens the following dialog box. Its options are described below.



Flattened Cell Name (.EXP .CEL)

Enter the name of the flattened top cell that hierarchy information is to be cleaned up.

Example

Top.EXP, Top.CEL

See Also

[astDumpHierVerilog](#) – Generates a hierarchical Verilog netlist for a given flat cell when the cell contains hierarchical information.

[astMarkHierAsPreserved](#) – Marks all the module instances to be preserved hierarchically.

[astInitHierPreservation](#) – Extracts necessary information from the original hierarchical netlist and writes it into the flattened top cell.

astDumpHierPreservation

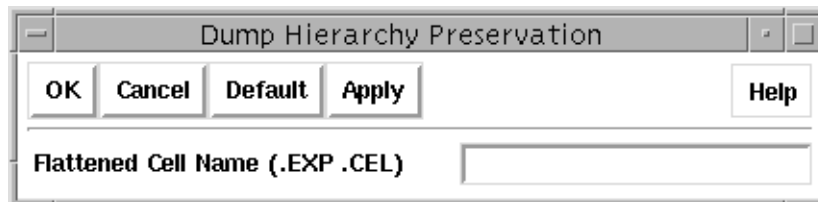
Dumps the hierarchy preservation data base objects into the message window and the log file. These data objects are maintained throughout timing optimization process.

Tool

Astro

Dump Hierarchical Preservation Dialog Box

The menu command opens the following dialog box. Its options are described below.



Flattened Cell Name (.EXP .CEL)

Enter the name of the flattened top cell that hierarchy preservation data objects are to be dumped from.

See Also

[astDeleteHierPreservation](#) – Cleans up all the hierarchical information created by the [astInitHierPreservation](#) command in the given flattened top cell.

[astInitHierPreservation](#) – Extracts necessary information from the original hierarchical netlist and writes it into the flattened top cell.

astDumpHierVerilog

Generates a hierarchical Verilog netlist for a given flat cell when the cell contains hierarchical information. You must use the [astInitHierPreservation](#) command to create the initial hierarchical information after binding the cell with an expanded netlist (.EXP) before any netlist changes have been done by optimization.

The `astDumpHierVerilog` command generates bus or vector port connections for a standard cell instance or a macro cell instance either when a netlist view is available for the cell or when a .CEL view with hierarchy information exists. When the command cannot find either a .NETL view or a .CEL view with hierarchy information, it generates individual bits for the port connections and the port names are escaped just like scalar ports with special characters.

For example:

```
A macro instances with input port IN[3:0]:
macro macro_inst(.IN({net1, net2, net3, net4}, .OUT(net5))
is
generated as follows if macro.NETL or macro.CEL with
hierarchy information exist:
macro macro_inst(.IN({net1, net2, net3, net4}, .OUT(net5))
if
not, it is generated as:
macro macro_inst(.\IN[0] (net1), .\IN[1] (net2), .\IN[2]
(net3),
.\IN[3] (net4), .OUT(net5));
```

Equivalent Tcl Command

`write_verilog`

See the man pages for Tcl command descriptions and syntax. For more information about the man pages, enter `man help` on the command line, or enter `man command_name` for information about a specific Tcl command.

Tool and Menu Command

Astro: Cell > Hierarchy Preservation - Hierarchical Verilog Out

Dump Hierarchical Verilog Dialog Box

The menu command opens the following dialog box. Its options are described below.

Dump Hierarchical Verilog

OK Cancel Default Apply Help

Flattened Cell Name (.EXP .CEL)

Enter File Name

Must Dump Cell instance Master(s)

☒ No power/ground ports

☐ No power/ground nets

☐ Output bus as individual bits

☒ No empty Cell Module Definitions

☐ No Corner Pad Cell instances

☐ No Pad Filler Cell instances

☐ No Core Filler Cell instances

☐ No Unconnected Cell instances

☒ No Unconnected Ports

☒ Strip BackSlash before hierarchy separator

☒ No diode ports

☐ Output Wire Declaration

☒ Output 1'b1 for Power(VDD, vdd, ...) and 1'b0 for Ground(VSS,gnd, ...)

1'b1 Net Name 1'b0 Net Name

☐ Generate macro definitions

Flattened Cell Name (.EXP .CEL)

Enter the name of the cell in the current open library for which the hierarchical Verilog description is to be generated.

Enter File Name

Enter the name of the file to which the hierarchical Verilog description is written.

Must Dump Cell instance Master(s)

Enter in the box the cell master names, separated with a space, for which all cell instances must be generated overriding the following switches:

- No Corner Pad Cell instances
- No Pad Filler Cell instances
- No Core Filler Cell instances
- No Unconnected Cell instances

The maximum allowed string size is 1023 chars.

For example:

Consider a cell "top" with four types of core filler cells: "FILL1," "FILL2," "FILL4," and "FILL8."

To generate the Verilog description with only "FILL4" and "FILL8" instances, select the "No Core Filler Cell instances" option and enter "FILL4 FILL8" in this field:

```
astDumpHierVerilog
setFormField "Dump Hierarchical Verilog" "Flattened Cell
Name"
"top"
setFormField "Dump Hierarchical Verilog" "Hierarcical
Verilog
File Name" "top.hv"
setFormField "Dump Hierarchical Verilog" "Must Dump Cell
instance Master(s)" "FILL4 FILL8"
setFormField "Dump Hierarchical Verilog" "No Core Filler
Cell
instances" "1"
formOK "Dump Hierarchical Verilog"
```

No power/ground ports

Select this option to prevent power and ground ports from being generated for cell instances in the Verilog description. The option is selected by default to skip power and ground ports.

No power/ground nets

Select this option to prevent the power and ground ports from being generated for all the cell instances, module instances and module definitions including the top-most module.

For each power or ground net in the cell, there is a port generated for the module instances and module definitions. If such a port exists in the original description, then no new port is generated, but the original port is considered as the power or ground port. Also, such a port from the original description should never be ignored and are always generated regardless of this switch.

Output bus as individual bits:

All the bus (vector) ports are generated as individual bits. The port names are treated as scalar port names and are escaped.

For example:

```
A module definition with vector port;
module top (out, in);
output [3:0] out;
input in;
...
endmodule
is generated as follows:
module top(\out[3] , \out[2] , \out[1] , \out[0] , in);
output \out[3] ;
output \out[2] ;
output \out[1] ;
output \out[0] ;
input in;
...
endmodule
```

No empty Cell Module Definitions

This option is used to suppress generating empty module definitions for the leaf level cells such as standard cells and macro cells. When this option is switched off, the command generates module definitions for all the “.FRAM view only” cells that contain port definitions but no internal cell instances.

No Corner Pad Cell instances

Select this option to prevent Corner Pad Cell instances from being generated in the hierarchical Verilog description.

No Pad Filler Cell instances

Select this option to prevent Pad Filler Cell instances from being generated in the hierarchical Verilog description.

No Core Filler Cell instances

Select this option to prevent Core Filler Cell instances from being generated in the hierarchical Verilog description.

No Unconnected Cell instances

Select this option to prevent unconnected cells from being generated in the hierarchical Verilog description. Unconnected cells are the cells with no ports connected, except for power and ground ports with “No power and ground ports” selected.

No Unconnected Ports

Select this option to prevent the generation of unconnected ports. This includes ports on module instances, macro instances and standard cell instances. When this option is deselected, all the unconnected ports are generated with a “SYNOPSIS_UNCONNECTED_%d” net. The “%d” portion of the name, represents the unconnected net number, and is used

for uniqueness. The number is first initialized at the maximum of existing unconnected nets and is incrementally increased every time a net is generated.

Strip BackSlash before hierarchy separator

Removes the backslash character preceding a hierarchy separator for all the instances identified. The option is selected by default.

No diode ports

Select this option to prevent diode ports from being generated for cell instances in the hierarchical Verilog description. The option is selected by default to skip generating diode ports.

Output Wire Declaration

Select this button to generate wire declarations in the hierarchical Verilog out. This option controls only the scalar wires. For vector wires, a declaration with vector limits is generated regardless of this switch.

For example:

With “Output Wire Declaration:”

```
wire n246;  
wire [1:0] n245;  
BUF U12 (.A(n245[0]), .Y(n246));
```

Without “Output Wire Declaration:”

```
wire [1:0] n245;  
BUF U12 (.A(n245[0]), .Y(n246));
```

Output 1'b1 for Power (VDD, vdd, ...) and 1'b0 for Ground (VSS, gnd...)

Select this option to generate 1'b1 for Power and 1'b0 for Ground nets in the hierarchical Verilog description. By default this switch is on. You can deselect the switch to generate the Power (VDD)

and Ground (VSS) net names. When deselected, the command generates power and ground supply nets at the top-most module instance and connect them to the power and ground ports (created when not found) of all the module instances in the hierarchy. The switch supports only one power and one ground net specified in the [auVerilogIn](#) command dialog box.

- 1'b0 net name - If needed, enter the correct 1'b0 net name for the cell. The cell should be the same as the one listed in the [auVerilogIn](#) command's: "Net Name for 1'b0." The default values are taken from the design library. The [auVerilogIn](#) command sets the properties for special net names like 1'b0 in the design library, and are specified in the "Net Name for 1'b0" boxes of the [auVerilogIn](#) command. These boxes override the previously set (or not set) values in the design library.
- 1'b1 net name - If needed, enter the correct 1'b1 net name for the cell. The cell should be the same as the one listed in the [auVerilogIn](#) command's: "Net Name for 1'b1." The default values are taken from the design library. The [auVerilogIn](#) command sets the properties for special net names like 1'b1 in the design library, and are specified in the "Net Name for 1'b1" boxes of the [auVerilogIn](#) command. These boxes override the previously set (or not set) values in the design library.

Generate macro definitions

Select this option to generate module definitions for all the macros with hierarchy information in the CEL views.

See Also

[astInitHierPreservation](#) – Extracts necessary information from the original hierarchical netlist and writes it into the flattened top cell.

[auVerilogIn](#) – Translates a netlist from Verilog Hardware Description Language (HDL) format to Synopsys format, creating a library (if the library does not already exist) and a netlist cell.

astExtrLEQ

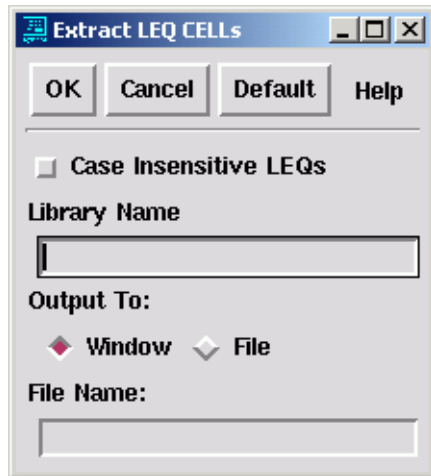
Extracts logically equivalent cells from the reference library. The extracted logic equivalence classes are stored back into the Milkyway database. This should be done for all reference libraries during data preparation.

Tool and Menu Command

Milkyway: CLF > Extract LEQ

Extract LEQ Cells Dialog Box

The menu command opens the following dialog box. Its options are described below.



Case Insensitive LEQs

When selected, disregards the case of cell names during logically equivalent cell extraction. This option is off by default.

Library Name

Enter the name of the reference library that logically equivalent extraction is to be performed.

Output To

Select where you want the report to be written.

Option	Writes the report to ...
Window	A text window.
File	The file specified by <i>File Name</i> .

File Name

Enter the name of the file that you want the report written.

astInitHierPreservation

Extracts necessary information from the original hierarchical netlist and writes it into the flattened top cell. The information is represented in the form of an object record in the Milkyway database.

Note:

This command must be called prior to doing optimization in order to obtain a hierarchical verilog netlist or before creating a hierarchical timing view (HTV) model (see [astCreateHTV](#)).

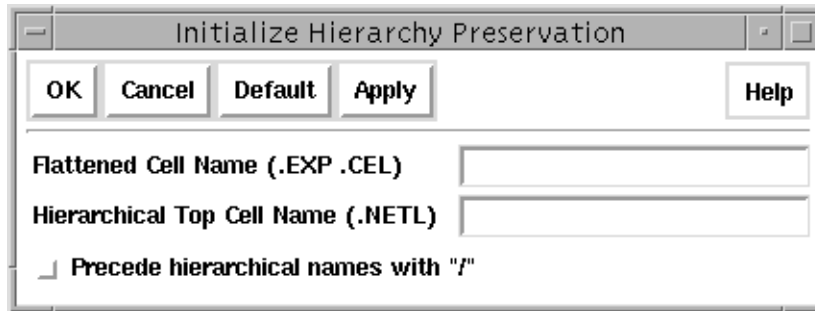
When hierarchy preservation is initialized with the `astInitHierPreservation` command, it is not necessary to run the “[cmCreateHierPorts \(Unsupported\)](#)” on page 22-17 command in order for the timer to identify the hierarchical port information in the SDC file.

Tool and Menu Command

Astro: Cell > Hierarchy Preservation - Initialize Hierarchy Information

Initialize Hierarchy Preservation Dialog Box

The menu command opens the following dialog box. Its options are described below.



Flattened Cell Name (.EXP .CEL)

Enter the flattened top cell name for the information to be written into.

Hierarchical Top Cell Name (.NETL)

Enter the top netlist cell name to extract hierarchy information from.

Precede hierarchical names with "/"

All the hierarchical names in Milkyway are preceded by a forward slash "/" for consistency in naming with other Scheme commands.

See Also

[astDeleteHierPreservation](#) – Cleans up all the hierarchical information created by the [astInitHierPreservation](#) command in the given flattened top cell.

[astDumpHierPreservation](#) – Dumps the hierarchy preservation data base objects into the message window and the log file.

astPostVerification

Documentation not available.

astSetCellInstDontTouch

Enables the setting and resetting of the *Don't Touch* status in a cell instance. A cell instance with the *Don't Touch* status set is not changed during netlist optimization for timing. This command also reveals the previous *Don't Touch* status before it is changed.

Tool

Milkyway

Syntax

```
astSetCellInstDontTouch topCellId cellInstName [#t | #f]
```

where the arguments are as follows:

<i>topCellId</i>	ID of the cell that you want to route. Valid values: ID of any cell in the open library. If the cell is open, you can substitute the following for <i>topCellId</i> : (geGetEditCell).
<i>cellInstName</i>	Name of the cell instance whose <i>Don't Touch</i> status you want to modify. Valid values: Instance name of any cell instance in the design.

Example

```
astSetCellInstDontTouch (geGetEditCell) "U123/U4" #t  
set cell instance U123/U4 to be Don't Touch.  
astSetCellInstDontTouch (geGetEditCell) "U123/U4" #f  
set cell instance U123/U4 to be modifiable.
```

Note:

If the Don't Touch situation is derived from its master-based Don't Touch, the cell instance would remain Don't Touch in netlist optimization).

9

ast Commands (No License)

This chapter describes commands with the prefix *ast*. The commands are listed alphabetically.

- [astSetDebug](#)
- [astUnsetDebug](#)

Note:

Unless otherwise noted, all command examples are shown as you would enter them from the command line, which can differ from Scheme syntax. If you want to add commands to startup files, or use them in user-defined programs, you must use correct Scheme syntax. For more information, see the *Milkyway Environment Extension Language Reference Manual*.

astSetDebug

Documentation not available.

astUnsetDebug

Documentation not available.

14

au Commands for Milkyway

This chapter describes commands with the prefix *au*. All of the *au* commands are accessed through Milkyway. The commands are listed alphabetically.

- [auAbortStreamInIfNoMapFile](#)
- [auAbortStreamOutIfNoMapFile](#)
- [auAlfToCLF \(Unsupported\)](#)
- [auCellTextOut \(Unsupported\)](#)
- [auConvertLayerDataType](#)
- [auCreateBaseArray](#)
- [auCreateHierNetlist](#)
- [auCreateHierVlogOut](#)

- `auDefIn`
- `auDefOut`
- `auDumpCLF`
- `auECOByChangeFile`
- `auECOByNetCmp`
- `auECODumpDiff`
- `auEdifIn`
- `auEdifOut`
- `auExtractBlockagePinVia`
- `auExtractMacroPins`
- `auGenCellBndry`
- `auGenCellBoundary`
- `auGenLibCellBoundary`
- `auGenSynopBackAnn`
- `auGetEmptyCellOnly`
- `auHECOByFile`
- `auHECODiff`
- `auHierECO`
- `auHierVerilogOut`
- `auLefIn`
- `auLib2LCIf`

- auLoadCLF
- auNDIApi
- auNDOApi
- auNLIApi
- auNLOApi
- auNewECOByNetCmp
- auSDFToConstraint
- auSetEdifOutExternal
- auSetEmptyCellOnly
- auSetHierInstanceAndNetNameLong
- auSetHierInstanceNameLong
- auSetHierNetNameLong
- auSetHierRefLibTwoLevels
- auSetMasterCellNameLong
- auSetMultiLEFForeignName
- auSetPRBdry
- auStampToDB
- auStampToCLF
- auStreamIn
- auStreamInCellBndryIgnoreText
- auStreamInDuplicateLayerAsBoundary

- `auStreamInRectilinearBoundary`
- `auStreamInSetCellBoundaryDataType`
- `auStreamInTextConversionFactor`
- `auStreamOut`
- `auStreamOutAllHierPinNetData`
- `auStreamOutArefContactArray`
- `auStreamOutByLayerNumber`
- `auStreamOutCellInstanceNameAsText`
- `auStreamOutCellRenamePrefix`
- `auStreamOutCheckInvalidCells`
- `auStreamOutChildCellFillData`
- `auStreamOutConfirmOverWriteGDSIIFile`
- `auStreamOutContactArrayMatrix`
- `auStreamOutControlFileToLog`
- `auStreamOutCustomViewInstance`
- `auStreamOutFillDataForAllCellTypes`
- `auStreamOutFillDataMapping`
- `auStreamOutGDSIIVersion`
- `auStreamOutKeepDataType`
- `auStreamOutNetNameBasedLayerMapping`
- `auStreamOutNetWithPathTypeZero`

- `auStreamOutOddShapePin`
- `auStreamOutOptionsToLog`
- `auStreamOutPropertyByObject`
- `auStreamOutRemoveEmptyCells`
- `auStreamOutRenameCellName`
- `auStreamOutRenameRefLibDuplicateCellFirst`
- `auStreamOutSelectedCells`
- `auStreamOutShortCellName`
- `auStreamOutShowCellInfo`
- `auStreamOutSkipGRouteContact`
- `auStreamOutSkipTheseCells`
- `auStreamOutSplitPath`
- `auStreamOutUnlinkGDSIIFile`
- `auStreamOutWillAbortIfCellsMissing`
- `auStreamOutWithExtendedLayerMapping`
- `auSuffixDuplicateLongCellName`
- `auSynopToCLF`
- `auVerilogIn`
- `auVerilogOut`
- `auVerilogToCell`
- `auVhdlIn`

- auVhdlOut

auAbortStreamInIfNoMapFile

This switch enables [auStreamIn](#) to stop operation without a layer map file or a cell type file.

Tool

Milkyway

Syntax

```
auAbortStreamInIfNoMapFile Value?
```

where the argument is as follows:

<i>Value?</i>	Indicates whether or not to abort streamIn when the layer map file or cell type file is absent. Valid values: • #t means enable the Scheme function. • #f means disable the Scheme function.
---------------	---

Example

```
auAbortStreamInIfNoMapFile #t
```

See Also

[auStreamIn](#)

auAbortStreamOutIfNoMapFile

This switch disables [auStreamOut](#) from doing anything without a layer map file.

Tool

Milkyway

Syntax

`auAbortStreamOutIfNoMapFile` *Value?*

where the argument is as follows:

<i>Value?</i>	Indicates whether or not to abort streamOut when the layer map file is absent. Valid values:
	• #t means enable the Scheme function. • #f means disable the Scheme function.

See Also

[auStreamOut](#)

auAlfToCLF (Unsupported)

This command is no longer supported, beginning with the V-2004.06 release.

auCellTextOut (Unsupported)

This command is no longer supported, beginning with the V-2004.06 release.

auConvertLayerDataType

Converts cell layers and data types from a given layer mapping file. The map file format is the same as the one used in [auStreamOut](#).

Tool

Milkyway

Syntax

```
auConvertLayerDataType "libName" "mapFile"
```

where the arguments are as follows:

<i>libName</i>	Name of the library containing cells that you want to convert the layer and data type.
<i>mapFile</i>	Name of the file containing the layer mapping instructions.

Example

This example converts layer and data type cells in the library named DESIGNS.

```
auConvertLayerDataType "DESIGNS" "layerMap"
```

auCreateBaseArray

Creates a base array in the specified design cell.

Syntax

```
auCreateBaseArray <string cellName> <string className>  
    <boolean isHorizontal> <float tileWidth>  
    <float tileHeight> <float originX> <float originY>  
    <float offsetLL> <float offsetUR>  
    <integer numberOfTiles> <integer numberOfRows>
```

where the arguments are as follows:

cellName

Design cell name.

className

Base array name.

isHorizontal

TRUE if base array is horizontal, FALSE if base array is vertical.

tileWidth

Tile width.

tileHeight

Tile height.

originX

X-coordinate of the base array origin.

originY

Y-coordinate of the base array origin.

offsetLL

Lower row vertical offset if base array is horizontal, Left row horizontal offset if base array is vertical.

offsetUR

Upper row vertical offset if base array is horizontal, Right row horizontal offset if base array is vertical.

numberOfTiles

Number of tiles in the base array.

numberOfRows

Number of rows in the base array.

Description

The `auCreateBaseArray` command creates a base array in the specified design cell according to the specified parameters.

Example

```
auCreateBaseArray "myDesign" "unit" #t 1.50 1.80 0.00 0.00  
0.00 0.00 2 200
```

auCreateHierNetlist

Regenerates a hierarchical netlist for a flat layout cell after netlist changes in place and route, based on a reference hierarchical netlist. These netlist changes can result from clock tree synthesis (CTS), scan chain optimization (reordering), Solar and Saturn optimization (cell sizing, buffer or repeater insertion, logic optimization), or any other netlist changes.

To reflect the changes back to the original hierarchical netlist, the .NETL view cells corresponding to the original hierarchical netlist need to be used. The cell view name for the top module of the .NETL view cell is required for command input.

HNET is the view name for the newly created hierarchical netlist cells. After the HNET view cell creation, you can use the hierarchical VerilogOut command to output the hierarchical netlist. This output contains the changes resulting from place and route activities.

If your netlist is hierarchical, expand it with [cmCmdExpand](#). Subsequent place and route steps, such as clock tree synthesis and scan chain optimization, change the expanded hierarchical netlist, making it difficult for the logic designer to work with. The `auCreateHierNetlist` command generates a new hierarchical netlist reflecting the changes made to the design during place and route.

The newly generated hierarchical netlist is created by the [auHierVerilogOut](#) command.

Tool

Milkyway

Create Hierarchical Netlist Dialog Box

The menu command opens the following dialog box. Its options are described below.

Create Hierarchical Netlist

OK Cancel Default Apply Help

Library Name

Cell Name

Reference Netlist Cell Name

Output Cell Name

☐ Create Scan Chains Nets

☒ Create Unused Ports

☐ No Assign Created In Verilog

☐ Replace Leaf Macro Cells With HNET Cells

☒ Overwrite Existing Cells

☐ Open NETL Cells For Leaf Cells

☐ Keep Port Name For Top Cell

☒ Preserve Boundary For All Modules

Module Names

Module List File Name

☒ Post Verification

sed Script To Convert Net Name For Back Annotation

☐ Print Detail Information

Library Name

The name of the library containing the cell for which you want to build a hierarchical netlist, and the reference cell.

Cell Name

The name of the top-level expanded cell for which a new netlist hierarchy is being created. By default, this is a .CEL view cell. For example, top.CEL.

Reference Netlist Cell

The name of the top-level, unexpanded netlist cell from which the place and route cell was generated.

Output Cell Name

The name of the hierarchical netlist cell to be created and used during VerilogOut or hierarchical VerilogOut.

Create Scan Chain Nets

When selected, the tool includes scan chain instances when building a new hierarchical netlist.

Create Unused Ports

When selected, the tool includes unused ports when creating the new hierarchical netlist.

No Assign Created In Verilog

When selected, the tool will remove assign statements (some port names will be modified to remove these assign statements).

After layout, some connections may become changed. To retain original port names, usually a new net is created which is connected with the ports. An assignment statement is generated for this connection whenever you dump the cell through commands [auVerilogOut](#) or [auHierVerilogOut](#).

Replace Leaf Macro Cells With HNET Cells

When selected, the tool will check the database to find a HNET cell that is matched with the macro cell. If found, the HNET cell is used as the cell instance master allowing you to dump a single Verilog file reflecting all levels of cell hierarchy.

This option is useful for designers who design hierarchically, and want a complete Verilog file reflecting cells from the bottom to the top. When a design is performed hierarchically, some cells at the

lowest level may be macro cells. The [auCreateHierNetlist](#) command will stop creating hierarchical netlist at this level. If you dump this cell using [auVerilogOut](#), you cannot find cells that are below these macro cells. If you want a complete Verilog file containing all cells from top to bottom, you need to merge all Verilog files for every macro cell into a large file.

Overwrite Existing Cells

Select to overwrite existing HNET cells.

Open NETL Cells For Leaf Cells

Select to open .NETL view cells for leaf cells. Leaf cells are the cells at the lowest level of the hierarchy. Otherwise, you will open .FRAM view cells.

Keep Port Name For Top Cell

Select to retain the port name for the top cell in the hierarchy.

Preserve Boundary For All Modules

This option is selected by default. When selected, the tools will create a hierarchical netlist for every cell.

Module Names

The name for the modules which you want to remain unchanged by optimization.

Note:

The Preserve Boundary For All Modules option must be deselected to use this field.

After Saturn optimization, you may want to retain optimized cells unchanged. If so, type either the module names in this field, or, if the number of modules that need to be preserved is large, create a file with multiple module names, and type the file name in the Module List File Name field.

Example format:

```
name1 name2 name3
```

Module List File Name

The name for the file containing module names which you want to remain unchanged by optimization.

Note:

The Preserve Boundary For All Models option must be deselected to use this field.

After Saturn optimization, you may want to retain optimized cells unchanged. If so, type either the module names in the Module Names field, or, if the number of modules that need to be preserved is large, create a file with multiple module names, and type the file name in this field.

Example format:

```
name1  
name2  
name3
```

Post Verification

When selected, the tool performs a quick check of the cell created by this command. If found, the tool will report the following five types of problems:

Problem Type	Description
Directions of Ports are Changed	After layout, the original port direction cannot drive all port instances it connected. Therefore, the direction has been changed.
Extra Ports are Found	Some extra ports may be created when using this command. This is acceptable as long as they do not cause net loop or break the connection.

Problem Type	Description
Net Loops	Net loops may be formed from the hierarchical ports and nets in the netlist. Net loops may cause problems in front-end tools.
Mismatch Between Flattened Cell and Layout Cell	Corresponding connections in both cells cannot be found. This is a serious problem and can result in shorts or opens. Please call Synopsys Technical Support for help.
Net Mapping Between Flatten Cell and Layout Cell	Can result in net name changes. Corresponding nets can be determined using the reported mapping messages. Names can be replaced in back-annotation files.

sedScript To Convert Net Name For Back Annotation

Because some nets may be renamed after running this command, you need to use the 'sed' function to replace nets for your timing information file with the new ones (LPE, DSPF, SPEF, or SPICE).

Whenever you enter a file name in this field, the tool creates a file containing the required sed script. You can use this script to run the following sed function:

```
sed -f script <oldTimeFile> newTimeFile
```

After you perform this function, all nets in oldTimeFile are renamed, as needed.

Print Detail Information

When selected, the tool shows a detailed listing of the build process.

auCreateHierVlogOut

Regenerates a hierarchical netlist for a flat layout cell after netlist changes in place and route, based on a reference hierarchical netlist. These netlist changes can result from clock tree synthesis (CTS), scan chain optimization (reordering), Solar and Saturn optimization (cell sizing, buffer or repeater insertion, logic optimization), or any other netlist changes.

To reflect the changes back to the original hierarchical netlist, the .NETL view cells corresponding to the original hierarchical netlist need to be used. The cell view name for the top module of the .NETL view cell is required for command input.

HNET is the view name for the newly created hierarchical netlist cells. After the HNET view cell creation, you can use the hierarchical VerilogOut command to output the hierarchical netlist. This output contains the changes resulting from place and route activities.

If your netlist is hierarchical, expand it with [cmCmdExpand](#). Subsequent place and route steps, such as clock tree synthesis and scan chain optimization, change the expanded hierarchical netlist, making it difficult for the logic designer to work with. The auCreateHierVlogOut command generates a new hierarchical netlist reflecting the changes made to the design during place and route.

The newly generated hierarchical netlist is created automatically.

Tools and Menu Commands

Milkyway: Output > Create Hierarchical Netlist

Create Hierarchical Verilog Out Dialog Box

The menu command opens the following dialog box. Its options are described below.

The dialog box titled "Create Hierarchical Verilog Out" contains the following elements:

- Buttons: OK, Cancel, Default, Hierarchical Verilog Out (selected), Help.
- Library Name: [Text Field]
- Cell Name: [Text Field]
- Reference Netlist Cell Name: [Text Field]
- Output Cell Name: [Text Field]
- ☐ Create Scan Chains Nets
- ☒ Create Unused Ports
- ☐ No Assign Created In Verilog
- ☐ Replace Leaf Macro Cells With HNET Cells
- ☒ Overwrite Existing Cells
- ☐ Open NETL Cells For Leaf Cells
- ☐ Keep Port Name For Top Cell
- ☒ Preserve Boundary For All Modules
- Module Names: [Text Field]
- Module List File Name: [Text Field]
- ☒ Post Verification
- sed Script To Convert Net Name For Back Annotation: cel2Hnet.map [Text Field]
- ☐ Print Detail Information

Library Name

The name of the library containing the cell for which you want to build a hierarchical netlist, and the reference cell.

Cell Name

The name of the top-level expanded cell for which a new netlist hierarchy is being created. By default, this is a .CEL view cell. For example, top.CEL.

Reference Netlist Cell Name

The name of the top-level, unexpanded netlist cell from which the place and route cell was generated.

Output Cell Name

The name of the hierarchical netlist cell to be created and used during VerilogOut or hierarchical VerilogOut.

Create Scan Chain Nets

When selected, the tool includes scan chain instances when building a new hierarchical netlist.

Create Unused Ports

When selected, the tool includes unused ports when creating the new hierarchical netlist.

No Assign Created In Verilog

When selected, the tool will remove assign statements (some port names will be modified to remove these assign statements).

After layout, some connections may become changed. To retain original port names, usually a new net is created which is connected with the ports. An assignment statement is generated for this connection whenever you dump the cell through commands [auVerilogOut](#) or [auHierVerilogOut](#).

Replace Leaf Macro Cells With HNET Cells

When selected, the tool will check the database to find a HNET cell that is matched with the macro cell. If found, the HNET cell is used as the cell instance master allowing you to dump a single Verilog file reflecting all levels of cell hierarchy.

This option is useful for designers who design hierarchically, and want a complete Verilog file reflecting cells from the bottom to the top. When a design is performed hierarchically, some cells at the lowest level may be macro cells. The [auCreateHierNetlist](#) command will stop creating hierarchical netlist at this level. If you dump this cell using [auVerilogOut](#), you cannot find cells that are below these macro cells. If you want a complete Verilog file containing all cells from top to bottom, you need to merge all Verilog files for every macro cell into a large file.

Overwrite Existing Cells

Select to overwrite existing HNET cells.

Open NETL Cells For Leaf Cells

Select to open .NETL view cells for leaf cells. Leaf cells are the cells at the lowest level of the hierarchy. Otherwise, you will open .FRAM view cells.

Keep Port Name For Top Cell

Select to retain the port name for the top cell in the hierarchy.

Preserve Boundary For All Modules

This option is selected by default. When selected, the tools will create a hierarchical netlist for every cell.

Module Names

The name for the modules which you want to remain unchanged by optimization.

Note:

The Preserve Boundary For All Modules option must be deselected to use this field.

After Saturn optimization, you may want to retain optimized cells unchanged. If so, type either the module names in this field, or, if the number of modules that need to be preserved is large, create a file with multiple module names, and type the file name in the Module List File Name field.

Example format:

```
name1 name2 name3
```

Module List File Name

The name for the file containing module names which you want to remain unchanged by optimization.

Note:

The Preserve Boundary For All Models option must be deselected to use this field.

After Saturn optimization, you may want to retain optimized cells unchanged. If so, type either the module names in the Module Names field, or, if the number of modules that need to be preserved is large, create a file with multiple module names, and type the file name in this field.

Example format:

```
name1  
name2  
name3
```


Post Verification

When selected, the tool performs a quick check of the cell created by this command. If found, the tool will report the following five types of problems:

Problem Type	Description
Directions of Ports are Changed	After layout, the original port direction cannot drive all port instances it connected. Therefore, the direction has been changed.
Extra Ports are Found	Some extra ports may be created when using this command. This is acceptable as long as they do not cause net loop or break the connection.
Net Loops	Net loops may be formed from the hierarchical ports and nets in the netlist. Net loops may cause problem in front-end tools.
Mismatch Between Flattened Cell and Layout Cell	Corresponding connections in both cells cannot be found. This is a serious problem and can result in shorts or opens. Please call Synopsys Technical Support for help.
Net Mapping Between Flatten Cell and Layout Cell	Can result in net name changes. Corresponding nets can be determined using the reported mapping messages. Names can be replaced in back-annotation files.

sedScript To Convert Net Name For Back Annotation

Because some nets may be renamed after running this command, you need to use the 'sed' function to replace nets for your timing information file with the new ones (LPE, DSPF, SPEF, or SPICE).

Whenever you enter a file name in this field, the tool creates a file containing the required sed script. You can use this script to run the following sed function:

```
sed -f script <oldTimeFile> newTimeFile
```

After you perform this function, all nets in oldTimeFile are renamed, as needed.

Print Detail Information

When selected, the tool shows a detailed listing of the build process.

Hierarchical Verilog Out Dialog Box

Hide

Verilog Out Data File

☒ No empty Cell Module Definitions

☒ No Corner Pad Instances

☒ No Pad Filler Cell Instances ☐ No Ports connected to Pad Filler Cell Instances

☒ No Core Filler Cell Instances

☒ No Unconnected Instances ☒ No Unconnected Ports

☐ No P/G nets ☒ No P/G ports

☒ No Diode Ports

Cell Rename Map File

☒ Output Wire Declaration

☒ Output 1'b1 for Power(VDD, vdd, ...) and 1'b0 for Ground(VSS, gnd, ...)

☐ Output Bus As Individual Bits

Reference Top NETL Cell To Output Bus

☒ Strip BackSlash Before Hierarchy Separator

☒ Output Simple Verilog File

☐ Output Instances By Sorted Names

Verilog Out Data File

Type the name of the file to which you want to write the verilog information.

No empty Cell Module Definitions

Select to prevent Milkyway from translating empty cells.

No Corner Pad Instances

Select when you do not want Milkyway to translate corner pad cell instances.

No Pad Filler Cell Instances

Select when you do not want Milkyway to translate pad filler cell instances.

No Ports connected to Pad Filler Cell Instances

Select when you do not want Milkyway to translate the ports that are connected to the pad filler cell instances.

No Core Filler Cell Instances

Select if you do not want Milkyway to translate core filler cell instances.

No Unconnected Instances

Select if you do not want Milkyway to translate unconnected instances.

No Unconnected Ports

Select if you do not want Milkyway to translate unconnected ports. If this option is not selected, unconnected ports will be generated and new nets will be created to connect the ports. The naming style for these nets is AVANT_UNCONNECTED_%d.

No P/G nets

Select if you do not want Milkyway to translate power and ground nets.

No P/G ports

Select if you do not want Milkyway to translate power and ground ports.

No Diode Ports

When selected, Milkyway diode ports are not written to the LEF file. If not selected, Milkyway diode ports are written.

Cell Rename Map File

Type the name of the cell mapping file you want Milkyway to use to map cell names during translation, or leave blank to use GDSII cell names. If you used an EDIF to GDSII map file to map cell names when you imported your netlist, use this same file to map names back to EDIF. For more information, see “Specifying Cell Name Mapping” in the *Milkyway Environment Data Preparation User Guide*.

Output Wire Declaration

Select to add wire information to the output. By default, this option is not selected and no wire declaration is generated.

Output 1'b1 for Power(VDD, vdd, ...) and 1'b0 for Ground(VSS, gnd, ...)

Select to output power nets as 1'b1 and ground nets as 1'b0. For example, if you have power net VDD connected to port1, and ground net VSS connected to port2

And the Option is ...	Milkyway outputs the nets as ...
Selected	assign port1 = 1'b1 ; assign port2 = 1'b0 ;
Not selected	assign port1 = VDD ; assign port2 = VSS ;

When this option is not selected, “supply1” and “supply0” are generated for power and ground nets at the net declaration section of the verilog file. When this option is selected, as it is by default, “1'b1” and “1'b0” are created at the cell instantiation section and no net declaration is created for them.

Output Bus As Individual Bits

Select to output buses as individual bits. For example, if you have a bus with bits A[0], A[1], and A[2]:

And the Option is ...	Milkyway outputs the bus as ...
Selected	<pre>module X (A[2], A[1], A[0]); input A[2]; /*Individual Bits*/ input A[1]; /*Individual Bits*/ input A[0]; /*Individual Bits*/ endmodule</pre>
Not selected	<pre>module X (A); input [2:0] A; /*As a Bus*/ endmodule</pre>

When this options is not selected, all bits of a bus are combined into a bus port (or net). In such case, Milkyway ignores No Unconnected Ports and No P/G Nets, so that unconnected ports and tie-off nets are generated.

Reference Top NETL Cell TO Output Bus

Type the name of the .NETL cell which contains the bus information you want to output.

Very often, a hierarchical .NETL cell is expanded and then bound to a top cell, usually with a different name for placement and routing. Because the bus information is stored in the .NETL cells, you need to specify the original top .NETL cell name when outputting bus information.

Strip BackSlash Before Hierarchy Separator

When selected, the tool removes backslashes that appear before hierarchy separators for every identifier, including ports, nets, and instances.

Output Simple Verilog File

When selected, the tool generates a Verilog file that is smaller and easier to read. Used for backward compatibility, selecting this option allows you to generate a simple format Verilog file similar to version 2.2.

Output Instances By Sorted Names

Select this feature to control the order of instances in the Verilog Files. When selected, the feature will output filenames alphabetically. When deselected, filenames will order arbitrarily.

auDefIn

Imports a DEF file to Milkyway.

DEF (Design Exchange Format) contains the design-specific information of a circuit and is a representation of the physical layout of the design during the layout process.

Note:

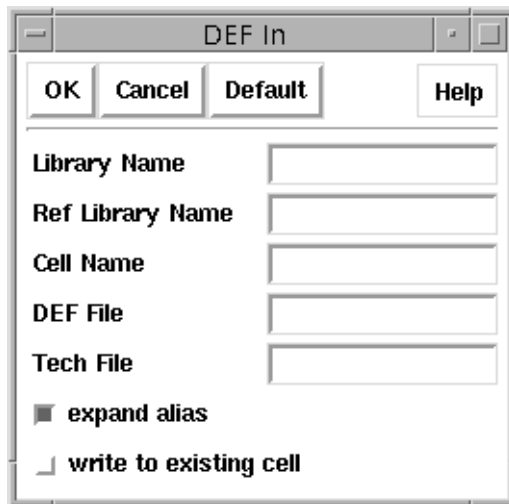
Supports version 4.5 and some features in version 5.2 of DEF.

Tool

Milkyway

DEF In Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Specify the name of the library.

Ref Library Name

Specify the name of the reference library.

Cell Name

Specify the name of the top cell.

DEF File

Specify the name of the DEF file.

Tech File

Specify the name of the technology file.

expand alias

When checked, this will expand the alias.

write to existing cell

When checked, this will overwrite the existing cell.

auDefOut

Exports a DEF file from Milkyway.

DEF (Design Exchange Format) contains the design-specific information of a circuit and is a representation of the physical layout of the design during the layout process.

Note:

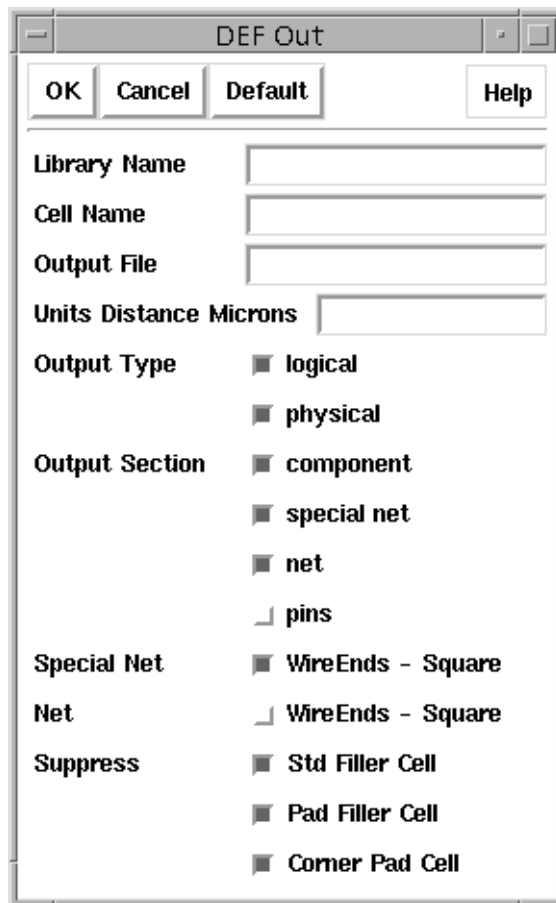
Supports version 4.5 and some features in version 5.2 of DEF.

Tool

Milkyway

DEF Out Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Specify the name of the library.

Cell Name

Specify the name of the top cell.

Output File

Specify the name of the output DEF file.

Units Distance Microns

Specify the unit distance in microns used in the output DEF file.

OutPut Type

The Output Type options are as follows:

- *logical* outputs logical connection information to the DEF file.
- *physical* outputs physical layout information to the DEF file.

OutPut Section

The Output Section options are as follows:

- *component* outputs component section to the DEF file.
- *special net* outputs special net section to the DEF file.
- *net* outputs net section to the DEF file.
- *pins* outputs pins section to the DEF file.

Special Net

WireEnds Square outputs special nets with square ends.

Net

WireEnds Square outputs regular nets with square ends.

Suppress

The Suppress options are as follows:

- *Std Filler Cell* suppresses standard filler cells.
- *Pad Filler Cell* suppresses pad filler cells.
- *Corner Pad Cell* suppresses corner pad cells.

auDumpCLF

Writes the CLF information associated with a library to a file. The CLF file provides timing information for timing-driven place and route. For more information about creating and loading a CLF file, see “Creating and Loading a CLF File” on page C-4 of the *Milkyway Data Preparation User Guide*.

Note:

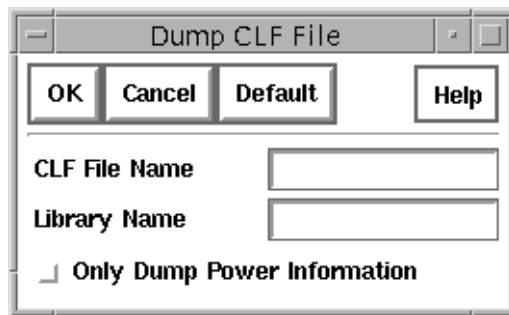
Writes only CLF functions to the file. Although you can include a “dbSetCellPortTypes” on page 3-221 function in a CLF file, it is a database function, not a CLF function.

Tools and Menu Commands

Milkyway: CLF > Write To File

Dump CLF File Dialog Box

The menu command opens the following dialog box. Its options are described below.



CLF File Name

Type the name of the file to which you want to write CLF information.

Library Name

Type the name of the library for which you want CLF information.

Only Dump Power Information

When dumping out the clf file, the dumped file (usually quite large) contains a mixture of logic, timing, and power information. Often you may need only work on the power portion. It is desirable to have an option to only dump out the clf power models and tables.

After this fix, once you click the button, you can only get power information and not all the logic, timing, and power information.

In addition, the Scheme command [clfDump](#) was also changed to make give you the choice to only dump the power information when you use the Scheme command instead of the menu.

See Also

[clfDump](#) writes the CLF information associated with a library to a file. (This function does not display a dialog box.)

auECOByChangeFile

Performs engineering changes on a design, using an ECO file you create.

You produce the change file with “ECO Header Information” by using the `cmCmdECODump` command. For information about creating the change file, including naming conventions to be used, see “[cmCmdECODump](#).”

Note:

The file dumped with `cmCmdECODump` can't be used directly with `auECOByChangeFile` if it contains hierarchical names.

The `auECOByChangeFile` command differs from the `auECOByNetCmp` command in that it makes changes based on ECO information that you create instead of comparing the netlists.

Caution!

You should make a backup of the cell, using `geBackupCell`, before you execute `auECOByChangeFile`.

Tool

Milkyway

ECO By Change File Dialog Box

The command opens the following dialog box. Its options are described below.



Library Name

Enter the name of the library containing the layout cells (and the new netlist cell) you want to update.

Design Cell Name

Enter the name of the layout cell for which you want to perform ECO changes.

File Name

Enter the name of the file you created that contains the ECO information.

ECO Report File

Enter the name of the ECO file created with `auECOByNetCmp` or `cmCmdECODump`.

Keep all metal

Keeps old and obsolete metals for the ECO routing process (`axgECORouteDesign`) to reuse later, rather than marking them for deletion at the end of this operation.

You must enable the silicon freeze status, using the `axSetFreezeSiliconECO` command, before you use the keep-all-metal capability. You can check the status by using the `axgGetECOStatus` command.

Delete Unused Pins

Deletes the pins of all the ports that are disconnected by this operation.

Delete Unused Nets

Deletes the nets that are disconnected by this operation.

Delete Cell Instances Immediately

If the operation is “-I”, the default behavior is to mark the cell for deletion, and it gets deleted later during ECO placement. Select this option if you want the cell instances to be deleted immediately.

See Also

[cmCmdECODump](#)

[auECOByNetCmp](#)

`axSetFreezeSiliconECO`

`axgECORouteDesign`

`axgGetECOStatus`

auECOByNetCmp

Compares the current netlist for a design with an updated version of the netlist and makes the following changes to the layout:

- When you replace a cell instance in the netlist with another cell (using the same instance name), Milkyway replaces the instance in the layout with the cell used in the netlist.

Note:

The replacement cell must have the same instance name, pin names, and pin-to-net connections as the cell it is replacing. Otherwise, Milkyway performs a deletion and an addition operation.

- When you delete a cell instance in the netlist, Milkyway deletes the instance in the layout, unless the freeze silicon option is turned on (see [axSetFreezeSiliconECO](#)).

Note:

When the freeze silicon option is turned on, Milkyway marks deleted instances as spare cells and places them in a spare group called ecoBonus.

- When you add a cell instance in the netlist, Milkyway adds the instance in the layout.

The `auECOByNetCmp` command works on hierarchical designs, as well as on flattened designs (designs with no hierarchical information). The command automatically updates the hierarchical structure (hierarchical common graph), such that removal and re-initialization of the graph is not necessary. In most cases you do not need to run the [astRepairHierPreservation](#) command after `auECOByNetCmp`. However, because additional updates are

sometimes needed, it is best to run `astRepairHierPreservation` after `auECOByNetCmp`. You should see zero repair performed in most cases.

You can choose to import a new or changed floor plan that is in the Physical Design Exchange Format (PDEF).

The `auECOByNetCmp` command generates an engineering change order (ECO) report, which contains detailed information about the comparison and update.

Caution!

You should make a backup of the cell, using [geBackupCell](#), before you execute `auECOByNetCmp`.

Tools and Menu Commands

Milkyway: ECO > By Net Compare

Prerequisites

Create a netlist view of the cell in the Milkyway database with the updated netlist in the library that contains the top-level cell. Use the `auVerilogToCell` command or a command such as `auEdifIn`, `auVerilogIn`, or `auVhdlIn`.

Verify that the scan chain groups in the design cell are consistent with its netlists. If they are not, retrace the scan chains to update scan chain groups in the database.

ECO By Netlist Compare Dialog Box

The command opens the following dialog box. Its options are described below.

ECO By Netlist Compare

Library Name

Design Cell Name

ECO Change Source

☒ **From Verilog File**

Verilog File Name

Top Module Name

☒ **From Milkyway Cell**

Cell Name

Ignore Tie High/Low Net Name Pattern

☐ **Use PDEF Clusters**

PDEF File

Cluster Type ☒ **Group** ☒ **Region**

☐ **Keep all metal**

☒ **Ignore Power/Ground**

☒ **Freeze Via Cell Instances**

☐ **Freeze Synthesized Clock Nets**

☐ **Soft Freeze Synthesized Clock Nets**

☐ **Freeze Pad Ring**

☐ **Freeze Standard Filler Cells**

☐ **Delete Unused Pins**

☒ **Delete Cell Instance Immediately**

☒ **Delete Unused Nets Immediately**

☐ **Ignore Floating Net**

☒ **No Freeze Scan Chains** ☐ **Freeze Scan Chains** ☐ **Soft Freeze Scan Chains**

ECO Report File

Library Name

Enter the name of the library containing the layout cells (and the new netlist cell) you want to update.

Design Cell Name

Enter the name of the layout cell for which you want to perform ECO changes.

ECO Change Source

Select one of the following types of ECO designs:

- From Verilog File – The input design file is in Verilog format.

Verilog File Name – Enter the name of the Verilog file, such as design.v.

Top Module Name– Enter the name of the top-level module.

- From Milkyway Cell – The input design file is from a Milkyway cell, which can be a flattened design or a hierarchical design.

Cell Name – Enter the name of the cell, such as design.CEL, design.HNET, or design.NETL.

Ignore Tie High/Low Net Name Pattern

Enter the tie-high and tie-low net name pattern(s). The engine will ignore all the matched nets during the netlist compare process. The net names or patterns must be separated by commas.

Use PDEF Clusters

Uses clusters that are defined as groups or regions in a PDEF file.

- PDEF File – Enter the name of the PDEF input file.
- Cluster Type – Choose either Group or Region

Keep all metal

Keeps old and obsolete metals for the ECO routing process (`axgECORouteDesign`) to reuse later, rather than marking them for deletion at the end of this operation.

You must enable the silicon freeze option, using the `axSetFreezeSiliconECO` command, before you use the keep-all-metal capability. You can check the status by using the `axgGetECOStatus` command.

Ignore Power/Ground

Instructs the comparison to ignore power and ground nets.

- If the netlist does not define power and ground connections, select this option.
- If you have only one power net and one ground net, Milkyway attempts to connect any new cell instances to power and ground.
- If you have multiple power and ground nets, specify port-to-net connections for any new cell instances, using `aprPGConnect`.
- If the netlist defines power and ground connections, deselect this option. Milkyway connects any new cell instances to power and ground as defined.

Note:

In any case, check the ECO report file that is generated to be sure all power and ground connections are made correctly.

Freeze Via Cell Instances

Preserves special via cells added by power and ground routing. The Ignore Power/Ground option must be selected also (both Ignore Power/Ground and Freeze Via Cell Instances are on by default).

Freeze Synthesized Clock Nets

Preserves buffer cells and nets added by clock tree (or trunk) synthesis.

Soft Freeze Synthesized Clock Nets

Preserves buffer cells and nets added by clock tree synthesis and delete or add the flip-flops.

Note:

You must turn on the freeze silicon option, using `fphEditHierarchy` or `axgECOStatus`. If you do not want any gate changes in the design during the net compare process, use the `auECODoNotChangeGates` command to set the flag. It takes `#t` or `#f` as an argument.

Freeze Pad Ring

Preserves the pad ring. If this option is turned on, the existing connections to the pad cells or pad filler cells will not be modified.

Freeze Standard Filler Cells

Preserves the standard filler cells.

Delete Unused Pins

Deletes the pins of all the ports that would be disconnected by this operation.

Delete Cell Instance Immediately

“-I” cell instances are marked for deletion, but not actually deleted. They will be deleted during ECO placement. You can see that a cell is marked for deletion if the cell instance name is blank. Select this option if you want the cell instances to be deleted immediately.

Delete Unused Nets Immediately

Unused nets are marked for deletion but are not actually deleted. They will be deleted during ECO routing. Select this option if you want the nets to be deleted immediately.

Ignore Floating Net

Ignores (does not preserve) the floating nets.

No Freeze Scan Chains

Ignores (does not preserve) the scan chains.

Freeze Scan Chains

Preserves the scan chains.

Soft Freeze Scan Chains

Preserves the scan chains except allowing the changes “+R” and “-I&+I.” For the change “-I,” deleting the beginning scan-chain cell or the ending scan-chain cell is not allowed. For the change “-I&+I,” the number of “ScanIn” or “ScanOut” ports of the “+I” cell instance in the netlist should be equal to that of the “-I” cell instance in the design cell.

ECO Report File

Enter the name of the file to which you want Milkyway to write the report.

See Also

[cmCmdECODump](#)
[astRepairHierPreservation](#)
[auECOByChangeFile](#)
[axSetFreezeSiliconECO](#)
[axgECORouteDesign](#)
[axgGetECOStatus](#)

auECODumpDiff

Dumps an ECO change file [ECOFileName] containing the netlist changes between cells [origCell] and [newCell]. It can also provide a list of netlist changes that have occurred during optimization.

Tool

Milkyway

Syntax

```
auECODumpDiff [library] [origCell] [newCell] [ECOFileName]
```

auEdifIn

Translates a netlist from EDIF format to Synopsys format, creating a library (if the library does not already exist) and a netlist cell.

Tools and Menu Commands

Milkyway: Netlist In > Edif In

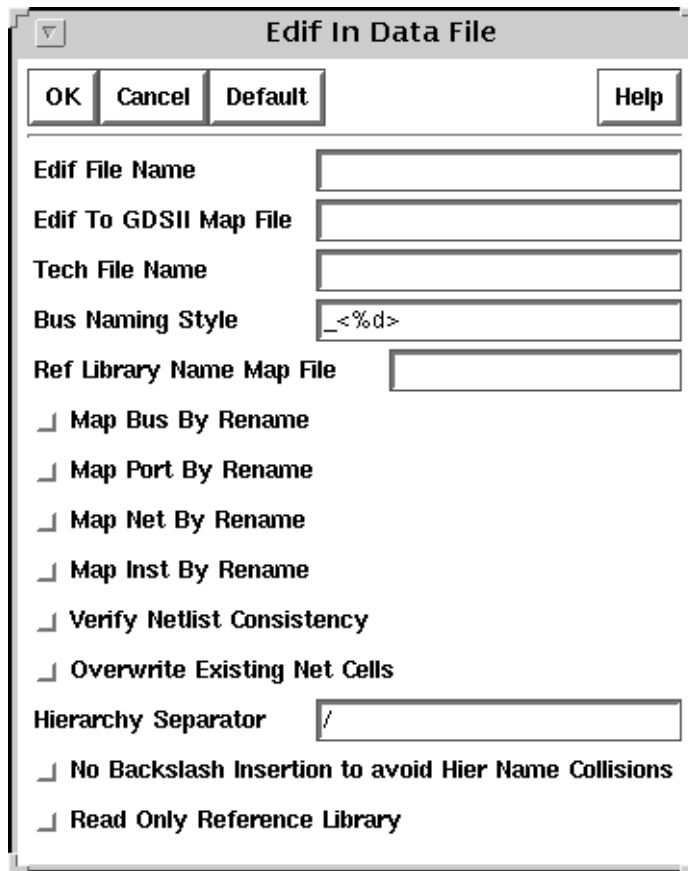
Jupiter: Design Setup > Edif In

Prerequisites

Specify the library in which you want to place the netlist cell. If you want to specify a Scheme for mapping EDIF cell names to GDSII cell names, create an EDIF To GDSII Map file.

Edif In Data File Dialog Box

The menu command opens the following dialog box. Its options are described below.



The image shows a dialog box titled "Edif In Data File". It has a standard Windows-style title bar with a small icon on the left. Below the title bar are four buttons: "OK", "Cancel", "Default", and "Help". The main area of the dialog contains several input fields and checkboxes. The "Edif File Name" field is empty. The "Edif To GDSII Map File" field is empty. The "Tech File Name" field is empty. The "Bus Naming Style" field contains the text "<%d>". The "Ref Library Name Map File" field is empty. There are seven checkboxes, all of which are unchecked: "Map Bus By Rename", "Map Port By Rename", "Map Net By Rename", "Map Inst By Rename", "Verify Netlist Consistency", "Overwrite Existing Net Cells", and "Hierarchy Separator". The "Hierarchy Separator" field contains the character "/". There are two more checkboxes at the bottom, both unchecked: "No Backslash Insertion to avoid Hier Name Collisions" and "Read Only Reference Library".

Field/Option	Value/State
Edif File Name	
Edif To GDSII Map File	
Tech File Name	
Bus Naming Style	<%d>
Ref Library Name Map File	
Map Bus By Rename	☐
Map Port By Rename	☐
Map Net By Rename	☐
Map Inst By Rename	☐
Verify Netlist Consistency	☐
Overwrite Existing Net Cells	☐
Hierarchy Separator	/
No Backslash Insertion to avoid Hier Name Collisions	☐
Read Only Reference Library	☐

Edif File Name

Type the name of the EDIF file containing the connectivity information.

Edif To GDSII Map File

Type the name of the file you want Milkyway to use to map cell names during translation or leave blank to use EDIF cell names. For information about creating an Edif to GDSII map file, see "Specifying Cell Name Mapping" in Chapter 5 of the *Milkyway Environment Data Preparation User Guide*.

Tech File Name

Type the name of the technology file you want loaded to the library. If you do not want to load a technology file, leave this blank.

Note:

If the library already exists, Milkyway ignores this option.

Bus Naming Style

Type any string you want Milkyway to append to the bus name specified in the EDIF file when it translates the bus to individual ports, using *%d* to indicate the port number. The requirements are as follows:

- The string cannot contain any blanks.
- The string can contain only one *%d*.
- To specify a percent (%) in the string, use *%%*.

For example, if the EDIF file defines a bus as:

```
port (array sys 4)
```

and you specify a *Bus Naming Style* of *_<<%%%d>*, Milkyway translates the bus defined by this statement as ports *sys_<%0>*, *sys_<%1>*, *sys_<%2>*, and *sys_<%3>*.

Ref Library Name Map File

Type the name of the file you want Milkyway to use to map external libraries to their corresponding path during translation. If no reference library mapping file is given, the required external library will be created in the current working directory, unless it already exists.

A syntax example of a reference library map file is:

```
Stecell /usr1/engineer/lib/Stdcell  
ECL_H /usr1/engineer/lib/ECL_H
```

Map Bus By Rename

Select if you want Milkyway to use a special bus name mapping Scheme when the EDIF file defines buses in the format generated by certain schematic editors.

- For information about translating data from the Mentor system, see “Specifying Bus and Scalar Name Mapping for Mentor Data” in Chapter 5 of the *Milkyway Environment Data Preparation User Guide*.
- For information about translating data from Synopsys, see “Specifying Bus Name Mapping for Synopsys Data” in Chapter 5 of the *Milkyway Environment Data Preparation User Guide*.

Note:

If you select this option, Milkyway applies the Bus Naming Style after it renames the buses.

Map Port By Rename

Select if you want Milkyway to use the port names specified by the rename statements in the EDIF file. For example, if the EDIF file defines a port as

```
port (rename pcgpf "port1-a")
```

Milkyway translates the port with the name *port1-a*.

Note:

If you are translating data from Mentor, you need to select Map Port By Rename. If you are translating data from Synopsys, you need to deselect Map Port By Rename.

Map Net By Rename

Select if you want Milkyway to use the net names specified by the rename statements in the EDIF file. For example, if the EDIF file defines a net as

```
net (rename vdd "net1-a")
```

Milkyway translates the net with the name *net1-a*.

Map Inst By Rename

Select if you want Milkyway to use the instance names specified by the rename statements in the EDIF file. For example, if the EDIF file defines an instance

```
instance (rename I20 "cella/cellb/cnt_op[3]")
```

Milkyway translates the instance with the name *cella/cellb/cnt_op[3]*.

Verify Netlist Consistency

Select if you want Milkyway to check that

- All nets are driven by one output
- All nets have at least one input

If you select this option, Milkyway writes the results of the check to the *edif2Arcs* log file.

Overwrite Existing Net Cells

Select if you want Milkyway to overwrite existing netlist cells.

Hierarchy Separator

Type the character you want to use as a separator in hierarchical names. If this character is used in cell instance or net names, Milkyway translates the name with a backslash (\) preceding the hierarchy separator character.

No Backslash Insertion to avoid Hier Name Collisions

Select to prevent the insertion of backslashes in file names during translation which may conflict with hierarchical name separators.

Read Only Reference Library

Select to set the access to the cells in your libraries as read only.

auEdifOut

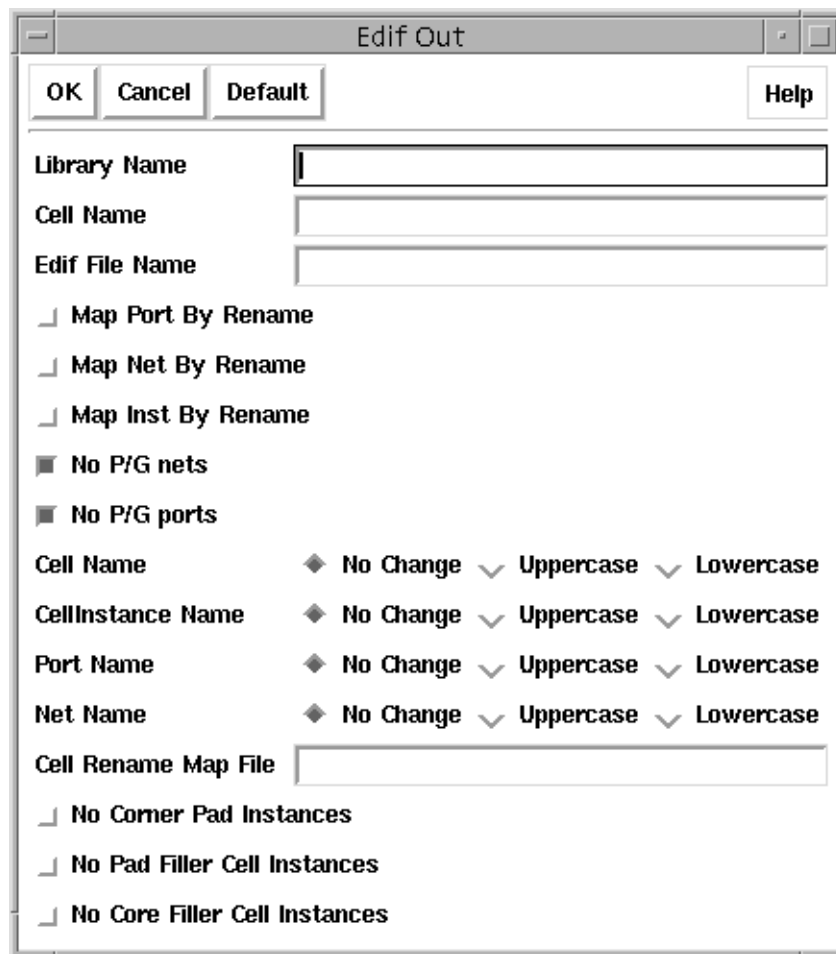
Translates connectivity information contained in a Synopsys top-level cell to a file in EDIF netlist format.

Tools and Menu Commands

Milkyway: Output > Edif Out

Edif Out Dialog Box

The menu command opens the following dialog box. Its options are described below.



The image shows a dialog box titled "Edif Out". It has a standard Windows-style title bar with minimize, maximize, and close buttons. Below the title bar are four buttons: "OK", "Cancel", "Default", and "Help". The main area of the dialog contains several input fields and checkboxes. At the top are three text boxes labeled "Library Name", "Cell Name", and "Edif File Name". Below these are three unchecked checkboxes: "Map Port By Rename", "Map Net By Rename", and "Map Inst By Rename". Following these are two checked checkboxes: "No P/G nets" and "No P/G ports". Then there are four rows, each with a label and a set of three options: "Cell Name" (No Change, Uppercase, Lowercase), "CellInstance Name" (No Change, Uppercase, Lowercase), "Port Name" (No Change, Uppercase, Lowercase), and "Net Name" (No Change, Uppercase, Lowercase). Each row has a diamond icon before "No Change" and a downward arrow before the other two options. At the bottom is a text box labeled "Cell Rename Map File". Finally, there are three unchecked checkboxes at the very bottom: "No Corner Pad Instances", "No Pad Filler Cell Instances", and "No Core Filler Cell Instances".

Library Name

Type the name of the library containing the cell for which you want to export connectivity information.

Cell Name

Type the name of the top-level cell for which you want to export connectivity information.

Edif File Name

Type the name of the file to which you want to write the EDIF netlist.

Map Port By Rename

Select if you want Milkyway to create rename statements for ports in the EDIF file. Select this option if your Synopsys data has port names that are illegal EDIF names (for example, *VDD!*).

Map Net By Rename

Select if you want Milkyway to create rename statements for nets in the EDIF file. Select this option if your Synopsys data has net names that are illegal EDIF names (for example, */I23/I96/N1*).

Map Inst By Rename

Select if you want Milkyway to create rename statements for instances in the EDIF file. Select this option if your Synopsys data has instance names that are illegal EDIF names (for example, *Inst1-a*). Your Synopsys data could have illegal EDIF names for any of the following reasons:

- You used the rename options when you imported the data using the *Netlist > Edif In* command.
- You made modifications that created illegal names (for example, *VDD!*).
- You expanded the netlist using the *Netlist > Expand* command, which places slashes (/) in the names.

No P/G nets

Select if you do not want Milkyway to translate power and ground nets.

No P/G ports

Select if you do not want Milkyway to translate power and ground ports.

Cell Name

Select to control the case of the name of the cell that is to be translated.

- No Change: The cell name will be written out as it is.
- Uppercase: The cell name will be written out in uppercase.
- Lowercase: The cell name will be written out in lowercase.

Cell Instance Name

Select to control the case of the name of the cell instance that is to be translated.

- No Change: The cell instance name will be written out as it is.
- Uppercase: The cell instance name will be written out in uppercase.
- Lowercase: The cell instance name will be written out in lowercase.

Port Name

Select to control the case of the name of the port that is to be translated.

- No Change: The port name will be written out as it is.
- Uppercase: The port name will be written out in uppercase.
- Lowercase: The port name will be written out in lowercase.

Net Name

Select to control the case of the name of the net that is to be translated.

- No Change: The net name will be written out as it is.
- Uppercase: The net name will be written out in uppercase.
- Lowercase: The net name will be written out in lowercase.

Cell Rename Map File

Type the name of the cell mapping file you want Milkyway to use to map cell names during translation, or leave blank to use GDSII cell names. If you used an EDIF to GDSII map file to map cell names when you imported your netlist, use this same file to map names back to EDIF. For more information, see “Specifying Cell Name Mapping” in Chapter 5 of the *Milkyway Environment Data Preparation User Guide*.

No Corner Pad Instances

Select if you do not want Milkyway to translate corner pad cell instances.

No Pad Filler Cell Instances

Select if you do not want Milkyway to translate pad filler cell instances.

No Core Filler Cell Instances

Select if you do not want Milkyway to translate core filler cell instances.

auExtractBlockagePinVia

Creates pin/blockage cells for layout cells. A pin/blockage cell defines pins and blockage areas (areas where no routing on a particular layer can occur) and via regions (areas where vias can be placed during routing). For cell-based layout, it defines pin solutions (solutions for routing to pins) for standard cells.

Milkyway creates blockage areas by merging any geometries on the same layer that are within a “merging distance” of each other (except pins). By default, the merging distance is equal to

$$(2 \times \text{minSpacing}) + \text{minWidth}$$

where *minSpacing* and *minWidth* are the minimum spacing and minimum width specified in the technology file for the layer. You can change the merging distance by using the Threshold options in the Extract Blockage dialog box.

For more information about cells, see “Translating Physical Design Data” in Chapter 3 of the *Milkyway Environment Data Preparation User Guide*.

Tools and Menu Commands

Milkyway: Cell Library > Blockage, Pin & Via

Prerequisites

You must have layout cells in the library. To create the layout cells, use [auStreamIn](#). Identify the global net ports in the cells using the `dbSetCellPortTypes` command.

Note:

If your library contains large macro cells with many small blockage areas, you might want to use [auExtractMacroPins](#) to extract pin geometries and delete blockage geometries in these cells before you create the pin/blockage cells.

Extract Blockage Dialog Box

The menu command opens the following dialog box. Its options are described below.

Extract Blockage

OK Cancel Default Apply Help

Library Name

Cell Name

Generate Boundary ☐ left ☐ right ☐ bottom ☐ top

Verti Via Grid Offset ☒ auto ☐ specify

☐ std/module cell ☐ macro ☐ pad

Extract Blockage Extract Pin by Text Extract Via

Extract Pin

☐ Extract Connectivity

through ☐ polyCont ☐ via1 ☐ via2 ☐ via3 ☐ via4 ☐ via5 ☐ via6

☐ via7 ☐ via8 ☐ via9 ☐ via10 ☐ via11

transfer pin on layer ☐ m2 ☐ m3 ☐ m4 ☐ m5 ☐ m6

☐ m7 ☐ m8 ☐ m9 ☐ m10 ☐ m11 ☐ m12

☐ text fall through ☐ double fall through

Poly Text Metal1 Text

Metal2 Text Metal3 Text

Metal4 Text Metal5 Text

Metal6 Text Metal7 Text

Metal8 Text Metal9 Text

Metal10 Text Metal11 Text

Metal12 Text Fall Through Text

☐ ignore internal macro pin access edges

☐ expand small pin on blockage for contact connection

Pin must-connect area layer number

Pin Must-Connect Area Layer Number

Poly Pin		M1 Pin		M2 Pin		M3 Pin	
M4 Pin		M5 Pin		M6 Pin		M7 Pin	
M8 Pin		M9 Pin		M10 Pin		M11 Pin	
M12 Pin							

Library Name

Type the name of the library containing the layout cells for which you want to create blockages.

Cell Name

Indicate the layout cells for which you want to create pin/blockage cells.

To indicate ...

Do the following:

One cell

Type the name of that cell and select its type.

All cells of the types you select

Type an asterisk (*) and select from the cell types: *std/module cell*, *macro*, *pad*.

Generate Boundary

Select the edges of the cell boundaries you want Milkyway to generate for the cells.

- If you imported cells with boundaries defined and you want Milkyway to use these existing boundaries, do not select any edges.
- If you imported cells with no boundaries defined, Milkyway generates all boundary edges, regardless of the edges you select.

Verti Via Grid Offset

Indicate how you want Astro to place standard cells in order to achieve optimum horizontal via placement.

Option

Instructs Astro to place cells such that...

auto

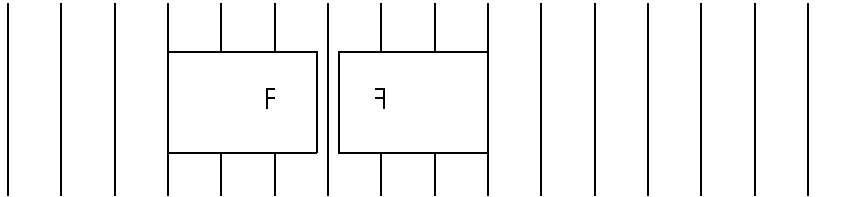
The maximum number of vias are on a vertical wire track.

specify

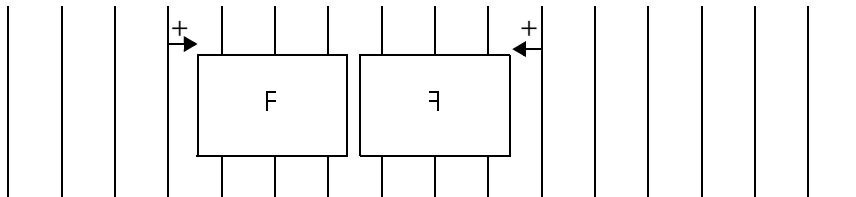
The left side of an unflipped cell (or the right side of a flipped cell) is a distance equal to the value you type from a vertical wire track.

The “specify” option works as follows:

- To instruct Astro to place an unflipped cell such that its left side is on a vertical wire track (or a flipped cell such that its right side is on a vertical wire track), select *specify* and type a zero (0).



- To instruct Astro to place an unflipped cell such that its left side is a specified distance to the right of a vertical wire track (or a flipped cell such that its right side is a specified distance to the left of a vertical wire track), select *specify* and type a positive number (less than the pitch of the vertical routing layer) to indicate the distance.



std/module cell

Select to process all standard or module cells in the library when you do not specify a *Cell Name*.

macro

Select to process all blocks in the library when you do not specify a *Cell Name*.

pad

Select to process all pads in the library when you do not specify a *Cell Name*.

Extract Blockage

Click to display the extraction blockage options, as described in [Extract Blockage Options](#).

Extract Pin by Text

Click to display the extract pin by text options, as described in [Extract Pin by Text Options](#).

Extract Via

Click to display the extract via options, as described in [Extract Via Options](#).

Extract Blockage Options

In the Make Macro dialog box, click the Extract Blockage button to change to the extract pin options, as shown in the following dialog box and described below. Select the appropriate extract blockage options and clicking Apply.

preserve all metal blockage

By default, this option is off. When selected, all metal blockages will not be cut around pins, of distance “minSpacing”, in both of horizontal and vertical directions. The purpose is to preserve the critical 45-degree portion of a blockage, when the metal blockage is of exact “minWidth” and has a 45-degree shape, and the 45-degree section is of distance “minSpacing” to other pins.

Treat All Blockage as Thin Wire

Enable to have the router treat all blockages as thin wires.

This option is off by default. You can also use the As Thin option to specify thin wire individually for each layer.

Routing Blockage Output Layer

Metal Blockage - This option controls the output layer of routing blockages generated by this command.

Route Guide - The default value is Route-Guide for the backward compatibility with Apollon. However, Astro users should set the output layer to Metal Blockage, which is more convenient than the route-guide layer from a viewing and editing perspective. The Astro router honors both Route-Guide and Metal Blockage as routing blockages.

zero min-spacing Route Guide – Select to allow a Route Guide with a minimum spacing of zero.

Treat Blockage – As Thin

Specifies thin wire for individual layers.

Merge Blockage

Select to control whether metal blockages need to perform a merge within a 'merging distance' or not. When selected, you can specify a 'merging distance' on each metal layer, or use the default merging distance which is equal to

$$(2 * minSpacing) + minWidth$$

where *minSpacing* and *minWidth* are the minimum spacing and minimum width specified in the technology file for the layer.

Poly Threshold

Type the value you want to substitute for *minWidth* in the formula for calculating the distance within which geometry merging on the Poly layer occurs.

Metal# Threshold

Type the value you want to substitute for *minWidth* in the formula for calculating the distance within which geometry merging on the metal# layer occurs.

PolyCont Threshold

Type the value by which you want to increase the distance within which geometry merging on the PolyCont layer occurs.

Via1 Threshold

Type the value by which you want to increase the distance within which geometry merging on the Via1 layer occurs.

Note: By default, any metal blockage that is less than the minimal width of the corresponding layer will be deleted by the command. However, if you want to keep those metal blockages that are less than the minimal width, type the following Scheme command in the Milkyway command area:

```
define BPV_KEEP_SMALL_METALBLOCKAGE
```

Extract Pin by Text Options

In the Make Macro dialog box, click the Extract Pin by Text button to change to the extract pin options, as shown in the following dialog box and described below.

Extract Blockage

OK Cancel Default Apply Help

Library Name

Cell Name

Generate Boundary ☐ left ☐ right ☐ bottom ☐ top

Verti Via Grid Offset ☒ auto ☐ specify

☐ std/module cell ☐ macro ☐ pad

Extract Blockage Extract Pin by Text Extract Via

Extract Blockage ☐ preserve all metal blockage

☐ Treat All Blockage as Thin Wire

☐ Protect Macro/Pad Pin By Route Guide

☐ Merge Blockage

Poly Threshold	0.000	Metal1 Threshold	0.000
Metal2 Threshold	0.000	Metal3 Threshold	0.000
Metal4 Threshold	0.000	Metal5 Threshold	0.000
Metal6 Threshold	0.000	Metal7 Threshold	0.000
Metal8 Threshold	0.000	Metal9 Threshold	0.000
Metal10 Threshold	0.000	Metal11 Threshold	0.000
Metal12 Threshold	0.000	PolyCont Threshold	0.000
Via1 Threshold	0.000		

Extract Connectivity

Select if you want Milkyway to trace the connectivity of a specified

multiple-layer pin and extract the entire pin.

through

The options are as follows:

- polyCont
- via1
- via2
- via3
- via4
- via5
- via6
- via7
- via8
- via9
- via10
- via11

The connectivity can be extracted through any specified via(s).

The default is to extract connectivity through via1 to via11 (metal1 - metal12) when “Extract Connectivity” is selected.

transfer pin on layer

When two adjacent layer geometries are connected through a via, and the higher layer geometry can be identified as a pin, this function removes the higher layer pin (and connecting via) and marks the lower layer geometry as the same pin. This reduces

the higher layer geometry, allowing more empty space in the higher layer, and also provides a resource for further higher layer routing.

For example, when the button m2 is selected (or on), the transfer pin function transfers the metal (2) pins to metal (2-1) pins.

Note:

The “transfer pin on layer” option only works when the `auExtractBlockagePinVia` command’s Extract Connectivity option is selected.

text fall through

Select this option and specify the layer if you want Milkyway to look at the layer(s) beneath the origin of each text label to identify the geometry annotated by the text. If more than one geometry exists beneath the origin of the text label, Milkyway selects the geometry to associate with the label according to the following priorities from highest to lowest:

- metal3
- metal2
- metal1
- poly

For example, if one of the geometries is metal3, Milkyway selects this geometry. If not, Milkyway looks for metal2, then metal1, then poly.

When the text layers match the layers for the geometries they identify, deselect the “text fall through” option. You do not need to specify the text layers associated with the interconnection

metals. When text layers are not specified (and the “text fall through” option is not selected), Milkyway assumes they match the geometry layers.

double fall through

Select this option and specify the layer if you want Milkyway to look at the layers beneath the origin of each text label on a metal2 geometry to identify a geometry (in addition to the metal2 geometry) annotated by the text. If more than one geometry (except the metal2 geometry) exists beneath the origin of the text label, Milkyway selects the second geometry to associate with the label according to the following priorities, from highest to lowest:

- metal1
- poly

For example, if one of the geometries is metal1, Milkyway selects this geometry. If not, Milkyway looks for poly.

Poly Text

Type the name or number of the layer used for the text identifying poly geometries (or leave blank if the text layer matches the geometry layer).

Metal# Text

Type the name or number of the layer used for the text identifying metal# geometries (or leave blank if the text layer matches the geometry layer).

Fall Through Text

Type the name or number of the layer used for text if you select either the “text fall through” or “double fall through” option.

ignore internal macro pin access edges

Select to speed up the macro-cell processing and reduce the runtime memory when all macro pins which need “pin access edges” are located near the cell boundary. The default value is off.

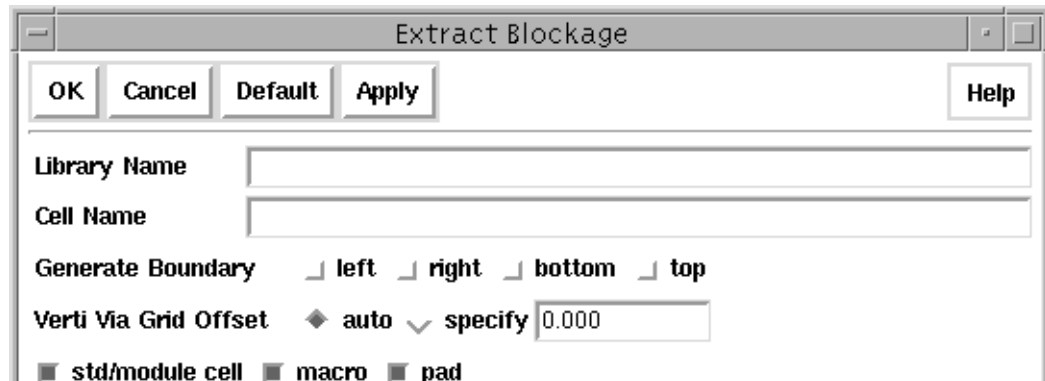
expand small pin on blockage for contact connection

Select if you have a small metal pin whose size is too small, or insufficient to the minimal width inside the metal blockage.

- If you do not select this option, the `auExtractBlockagePinVia` command cuts around pins of the normal “minimal spacing”. The router will not be able to access an “insufficient minimal-width pin” through dropping a contact under this situation. The default value is off because the `auExtractBlockagePinVia` command assumes there is no “insufficient minimal-width pin” inside a cell.
- If you select this option, the `auExtractBlockagePinVia` command cuts blockage around an “insufficient minimal-width pin” that is larger than the minimal spacing, allowing the router to access the small pin by dropping a normal contact among the blockage.

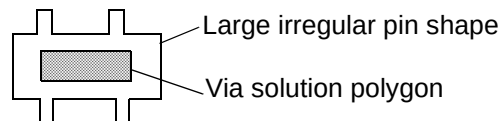
Pin must-connect area layer number

Click the “Pin must-connect area layer number” button to display the additional option, as shown below.



Select this option when you want the router to connect a pin through a specified area. Enter the corresponding layer number with metal pins, for example *poly*, *metal1*, ..., *metal12*. The `auExtractBlockagePinVia` command reads in the specified areas from the entered layer number to generate a proper routing guide to have the router connect the pin through the specified area.

For example, to specify a via solution for a large irregular pin, as shown in the following figure:



1. Open the cell.
2. Draw a polygon for the via solution area (for example, a rectangle) on the layer (for example, *metal2*).
3. Save the cell.
4. Enter the layer number in the box "Pin must-connect area layer number" box for the corresponding pin mask.
5. Run the `gauExtractBlockagePinVia` command.

You will see the resulting .FRAM with the route guide blockage around the large irregular pin to prevent the router from dropping vias into the forbidden area.

Note:
The layer number entered in a field must be defined in the technology file of the library.

Auto-compute pin must-connect area

As an alternate to drawing a polygon to designate the via solution, click the “Auto-compute pin must-connect area” option to display the Pin Must-Connect Area Width Threshold options.

☐ expand small pin on blockage for contact connection

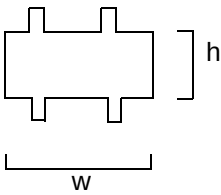
Pin must-connect area layer number

Auto-compute pin must-connect area

Pin Must-Connect Area Width Threshold

Poly	0.000	M1	0.000	M2	0.000	M3	0.000
M4	0.000	M5	0.000	M6	0.000	M7	0.000
M8	0.000	M9	0.000	M10	0.000	M11	0.000
M12	0.000						

This option lets you specify a threshold value for automatically computing an area in an irregular pin that is large enough for the router to connect and excludes small areas where you do not want a pin connected. For example, the following figure shows an irregular pin.



- Enter a threshold value for computing the via solution area in the appropriate layer.
- Run the `auExtractBlockagePinVia` command.

Extract Via Options

In the Make Macro dialog box, click Extract Via to display the Extract Via options, as described below.

Extract Blockage

OK Cancel Default Apply Help

Library Name

Cell Name

Generate Boundary ☐ left ☐ right ☐ bottom ☐ top

Verti Via Grid Offset ☒ auto ☐ specify

☒ std/module cell ☒ macro ☒ pad

Extract Blockage Extract Pin by Text Extract Via

Extract Pin

☐ Extract Connectivity

through ☐ polyCont ☒ via1 ☒ via2 ☒ via3 ☒ via4 ☒ via5 ☒ via6

☒ via7 ☒ via8 ☒ via9 ☒ via10 ☒ via11

transfer pin on layer ☐ m2 ☐ m3 ☐ m4 ☐ m5 ☐ m6

☐ m7 ☐ m8 ☐ m9 ☐ m10 ☐ m11 ☐ m12

☐ text fall through ☐ double fall through

Poly Text		Metal1 Text	
Metal2 Text		Metal3 Text	
Metal4 Text		Metal5 Text	
Metal6 Text		Metal7 Text	
Metal8 Text		Metal9 Text	
Metal10 Text		Metal11 Text	

Extract Via—skip macro via region

Select to prevent Milkyway from defining via regions for macros. If you are processing a large number of macros, selecting this option can speed up processing significantly.

Extract Via—skip rotated vias

Select if you do not want the router to drop any rotated vias to connect a pin. When deselected, the router chooses to drop a regular via or a rotated via to connect a pin. The default value is off.

Extract Via Region Up To Metal *

Select to specify the mask that the via-region is calculated up to. By default, the via-region information is provided up to Metal 2.

Contact Selection

The `auExtractBlockagePinVia` command extracts via-regions for multiple contacts whose contact numbers (or contact names) are typed into this field.

auExtractMacroPins

Extracts pin geometries and deletes blockage geometries in the specified layout cell(s).

Caution!

Because this command modifies your layout cells, use it only for very large macro cells with many small blockage areas.

Performing this processing on large macro cells with many small blockage areas speeds up pin and blockage extraction (see [auExtractBlockagePinVia](#)).

Note:

After pin and blockage extraction, replace the blockage.

Tools and Menu Commands

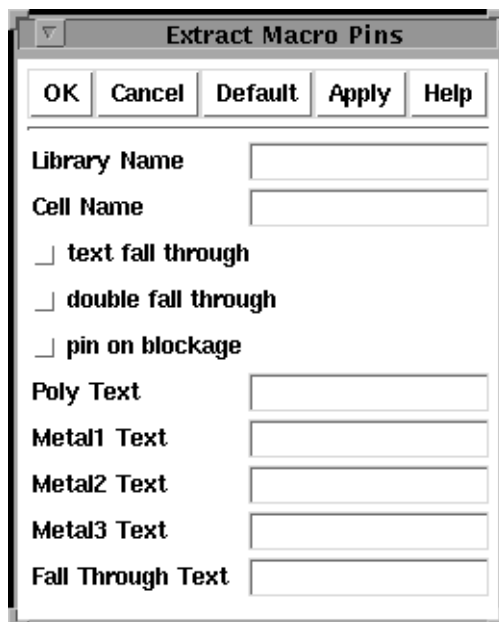
Milkyway: Cell Library > Extract Macro Pins

Prerequisites

Smash any hierarchical layout cell(s) to a level sufficient to provide the access required to perform the processing.

Extract Macro Pins Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Type the name of the library containing the cell(s) you want to process.

Cell Name

Type the name of the macro cell you want to process, or an asterisk (*) to process all macro cells.

text fall through

Select and specify the *Fall Through Text* layer if you want Milkyway to look at the layer(s) beneath the origin of each text label to identify the geometry annotated by the text. If more than one geometry exists beneath the origin of the text label, Milkyway selects the geometry to associate with the label according to the following priorities, from highest to lowest:

- metal3
- metal2
- metal1
- poly

For example, if one of the geometries is metal3, Milkyway selects this geometry. If not, Milkyway looks for metal2, then metal1, then poly.

double fall through

Select and specify the *Fall Through Text* layer if you want Milkyway to look at the layer(s) beneath the origin of each text label on a metal2 geometry to identify a geometry (in addition to the metal2 geometry) annotated by the text. If more than one geometry (except the metal2 geometry) exists beneath the origin of the text label, Milkyway selects the second geometry to associate with the label according to the following priorities, from highest to lowest:

- metal1

- poly

pin on blockage

Select if you have metal1 pins inside metal1 blockage.

- If you do not select this option, the router cannot access the metal1 pins.
- If you select this option, Milkyway cuts through the metal1 blockage where there are metal1 pins to allow the router to access the metal1 pins. In this case, the router can route in metal2 over the metal1 blockage and drop vias to access the metal1 pins.

Poly Text

Type the name or number of the layer used for the text identifying poly geometries (or leave blank if the text layer matches the geometry layer).

Metal# Text

Type the name or number of the layer used for the text identifying metal# geometries (or leave blank if the text layer matches the geometry layer).

Fall Through Text

Type the name or number of the layer used for text if you select *text fall through* or *double fall through*.

See Also

[auExtractBlockagePinVia](#) removes any metal2 that can be removed without changing the internal connectivity of the cell and changes metal2 pins (and pin text) to metal1.

auGenCellBndry

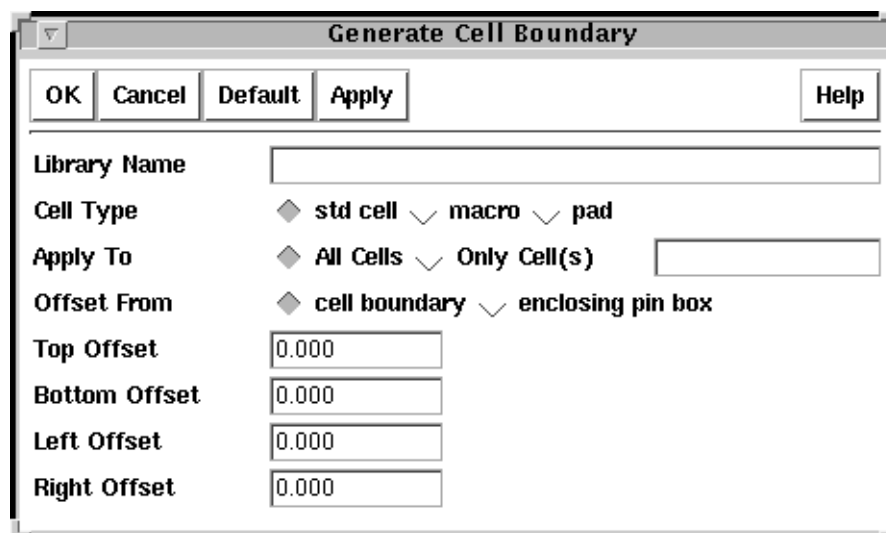
Regenerates cell boundaries.

Tools and Menu Commands

Milkyway: Cell Library > Generate Cell Boundary

Generate Cell Boundary Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Type the name of the library for which you want to regenerate cell boundaries.

Cell Type

Select the type of cells for which you want to regenerate cell boundaries.

Table 14-6

Option	Regenerates cell boundaries for ...
std cell	Standard cells.
macro	Macro cells.
pad	Pad cells.

Apply To

Indicate the cells for which you want to regenerate cell boundaries.

Table 14-7

Option	Regenerates cell boundaries for ...
All Cells	All cells of the specified <i>Cell Type</i> .
Only Cell(s)	The cell(s) you specify by typing the cell name(s). To specify multiple cells, separate the names with commas.

Offset From

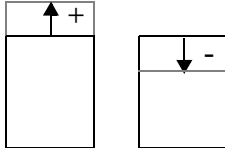
Indicate whether the offset options you specify apply to the cell boundary or the enclosing pin box.

Table 14-8

Option	Offsets the generated boundary from ...
cell boundary	The existing cell boundary.
enclosing pin box	A box that encloses the I/O pins.

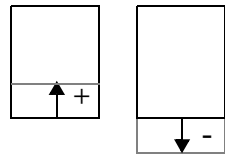
Top Offset

Type the value (in user units) by which you want to move the top edge of the existing cell boundary (or enclosing pin box) upward (positive number) or downward (negative number).



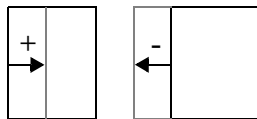
Bottom Offset

Type the value (in user units) by which you want to move the bottom edge of the existing cell boundary (or enclosing pin box) upward (positive number) or downward (negative number).



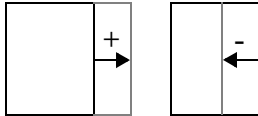
Left Offset

Type the number (in user units) by which you want to move the left edge of the existing cell boundary (or enclosing pin box) to the right (positive number) or left (negative number).



Right Offset

Type the number (in user units) by which you want to move the right edge of the existing cell boundary (or enclosing pin box) to the right (positive number) or left (negative number).



auGenCellBoundary

Generates a cell boundary in the specified cell.

Syntax

```
auGenCellBoundary <objectId cellId> <int offsetType>  
    <float leftOffset> <float bottomOffset>  
    <float rightOffset> <float topOffset>
```

where the arguments are as follows:

cellId

Database object ID of the cell.

offsetType

0 if the offset is based on the current cell boundary, 1 if the offset is based on the bounding box of all the cell's pins.

leftOffset

Distance by which to offset the cell's left boundary.

bottomOffset

Distance by which to offset the cell's bottom boundary.

rightOffset

Distance by which to offset the cell's right boundary.

topOffset

Distance by which to offset the cell's top boundary.

Description

The `auGenCellBoundary` command generates a cell boundary in the specified cell. If the `offsetType` is 0, then the cell boundary is based upon the existing cell boundary's bounding box. If the `offsetType` is 1, then the cell boundary is based upon the bounding box of the cell's pins (that is, the smallest area enclosing all of the cell's pins). The cell boundary's dimensions are taken from this bounding box and enlarged by the specified offset values (or reduced if the specified offset values are negative).

Example

```
auGenCellBoundary (geGetEditCell) 1 1.00 2.00 1.00 2.00
```

auGenLibCellBoundary

Generates cell boundaries for cells in the specified library.

Syntax

```
auGenLibCellBoundary <objectId libId> <string cellType>  
    <int offsetType>  
    <float leftOffset> <float bottomOffset>  
    <float rightOffset> <float topOffset>
```

where the arguments are as follows:

`libId`

Database object ID of the library.

cellType

Type of cells for which to generate boundaries (such as “padCell”, “moduleCell”, or “macroCell”).

offsetType

0 if the offset is based on the current cell boundary, 1 if the offset is based on the bounding box of all the cell’s pins.

leftOffset

Distance by which to offset the cell’s left boundary.

bottomOffset

Distance by which to offset the cell’s bottom boundary.

rightOffset

Distance by which to offset the cell’s right boundary.

topOffset

Distance by which to offset the cell’s top boundary.

Description

The `auGenLibCellBoundary` command generates cell boundaries for cells in the specified library. Cell boundaries are generated (in the FRAM views) only for those cells with the specified cell type.

For each library cell, if the `offsetType` is 0, then the cell boundary is based upon the existing cell boundary’s bounding box. If the `offsetType` is 1, then the cell boundary is based upon the bounding box of the cell’s pins (that is, the smallest area enclosing all of the cell’s pins). Each cell boundary’s dimensions are taken from this bounding box and enlarged by the specified offset values (or reduced if the specified offset values are negative).

Example

```
auGenLibCellBoundary (dbGetCurrentLibId) "moduleCell" 1  
1.00 2.00 1.00 2.00
```

auGenSynopBackAnn

Generates a file containing a Synopsys Design Compiler script based on an ECO. You can import this file into Design Compiler to reflect a change made by a SolarII operation.

Note:

Supports all types of ECOs except for those generated by scan chain optimization. The original netlist must be in Verilog format (translated using [auVerilogIn](#)).

Tools and Menu Commands

Milkyway: Output > Generate Backannotation Script

Prerequisites

You need to write out the ECO using [cmCmdECODump](#).

Generate Synopsys Backannotation Script Dialog Box

The menu command opens the following dialog box. Its options are described below.

Generate Synopsys Backannotation Script

OK Cancel Default Help

Library Name

Netlist Cell Name

Design Cell Name

ECO Filename

Synopsys Script Filename

Standard Cell Library Name

Hierarchy Separator

Name Separator

☐ Ignore Hierarchy Separator in ECO Dump File

Library Name

Type the name of the library containing the netlist (*.netl*) cells.

Netlist Cell Name

Type the name of the netlist cell corresponding to the top-level netlist module.

Design Cell Name

Type the name of the layout cell from which the ECO file was derived.

ECO Filename

Type the path to the ECO file to be used. This is the file containing detailed ECO information (see [cmCmdECODump](#))

Synopsys Script Filename

Type the path to the file to which the output script should be written.

Standard Cell Library Name

Type the name of the library containing the standard cells.

Hierarchy Separator

Type the character used as a separator in hierarchical names in Astro.

Name Separator

Type the character used as a separator in names of pins and nets added by ECOs.

Ignore Hierarchy Separator in ECO Dump File

Select to disregard the designated hierarchy separator in the ECO dump file.

auGetEmptyCellOnly

Retrieves the current value for [auSetEmptyCellOnly](#) flag.

Tool

Milkyway

Syntax

`auGetEmptyCellOnly`

auHECOByFile

Reads engineering change order (ECO) change files and performs ECO operations on Milkyway design cells.

The `auHECOByFile` command performs hierarchical ECO changes on the specified cell according to the specified change file. If all operations are successful, the results are saved in the current revision of the cell, the original design data is overwritten, and the cell is closed. If one or more ECO changes fail, the subsequent ECO changes continue until the change file ends, but no changes are saved in the library.

Equivalent Tcl Command

```
read_eco_list
```

See the man pages for Tcl command descriptions and syntax. For more information about the man pages, enter `man help` on the command line, or enter `man command_name` for information about a specific Tcl command.

Syntax

```
auHECOByFile lib_name cell_name change_file
```

where the arguments are as follows:

`lib_name`

Specifies the name of the library in which the design cell is stored.

`cell_name`

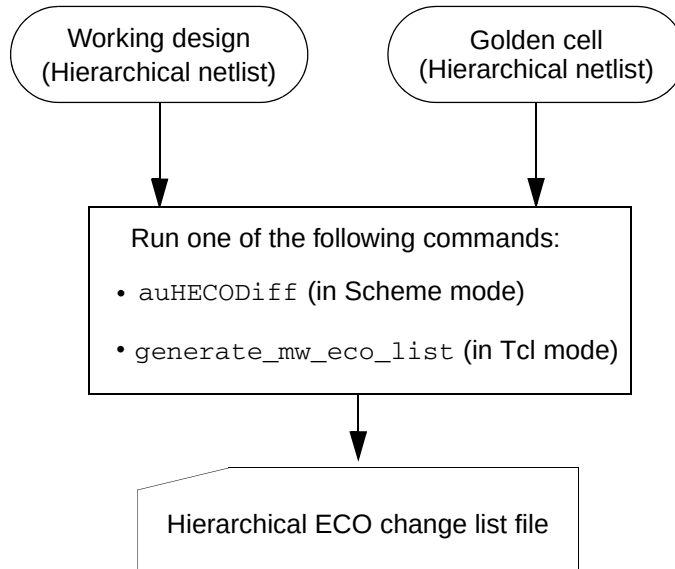
Specifies the name of the Milkyway design cell to be changed.

change_file

Specifies the name of the change file with ECO data.

auHECODiff

Compares two hierarchical design netlists and automatically generates a hierarchical engineering change order (ECO) change list file.



Syntax

```
auHECODiff library_name design_cell_name change_cell_name  
output_change_file_name
```

where the arguments are as follows:

`library_name`

Specifies the name of the working library that is opened for Milkyway commands. This library includes the working design cell and the golden cell (ECO change cell).

`design_cell_name`

Specifies the name of the design cell that is opened and compared with the golden cell. This is the cell that you are working on.

`change_cell_name`

Specifies the name of the golden cell that is opened for comparison to the design cell.

`output_change_file_name`

Specifies the name of the change list file that the `auHECODiff` command outputs to. This change list describes the differences between the hierarchical netlists of the design cell and golden cell.

Equivalent Tcl Command

`generate_mw_eco_list`

See the man pages for Tcl command descriptions and syntax. For more information about the man pages, enter `man help` on the command line, or enter `man command_name` for information about a specific Tcl command.

auHierECO

Reads engineering change order (ECO) Verilog files and updates the top-level design cell, as well as all softmacro cells, with both flat and hierarchical information. The hierarchical information is extracted from the Verilog input file.

The `auHierECO` command reads the Verilog file, or the files listed in the list file. Then it expands the hierarchical design into a flat cell named top module. Next, the `auHierECO` command compares the current design cell with the expanded ECO cell and makes changes in the design cell. After the flat design is updated, `auHierECO` deletes the original hierarchy data and rebuilds the hierarchy preservation information from the specified Verilog file or files. The ECO change list is attached to the ECO history.

If the design contains physical hierarchy data in different softmacro cells, `auHierECO` updates each softmacro cell and its hierarchy preservation information from the bottom up and attaches its own change list to the softmacro cell's ECO change history.

The `auHierECO` command maintains design consistency with complete bus properties for both flat and hierarchical data. This is true even if the current design and ECO design have different properties or different bus ranges on the same bus objects.

Important:

If all operations are successful, `auHierECO` deletes the original hierarchy data; therefore, it is recommended that you make a back-up copy of the cell before executing the `auHierECO` command.

Equivalent Tcl Command

`update_mw_design_eco`

See the man pages for Tcl command descriptions and syntax. For more information about the man pages, enter `man help` on the command line, or enter `man command_name` for information about a specific Tcl command.

Hierarchical ECO By Netlist Compare Dialog Box

The `auHierECO` command opens the following dialog box. Its options are described below.

Hierarchical ECO By Netlist Compare

OK Cancel Default Help

Library Name

Top Design Cell Name

Verilog File Name

☐ Specified Verilog File is a Verilog List File

Top Module name

Ignore Tie High/Low Net Name Pattern

☐ Keep all metal

☒ Ignore Power/Ground

☒ Freeze Via Cell Instances

☐ Freeze Synthesized Clock Nets

☐ Soft Freeze Synthesized Clock Nets

☐ Freeze Pad Ring

☐ Freeze Standard Filler Cells

☐ Delete Unused Pins

☒ Delete Cell Instance Immediately

☒ Delete Unused Nets Immediately

☐ Ignore Floating Net

☒ No Freeze Scan Chains ☐ Freeze Scan Chains ☐ Soft Freeze Scan Chains

ECO Report File

Library Name

Specifies the name of the library in which the design cell is stored.

You must enter a library name; the current library is not automatically used if you do not specify a library name.

Top Design Cell Name

Specifies the name of the Milkyway design cell to be changed.

You must enter the name of the top design cell; the current cell is not automatically used if you do not specify a top design cell name.

Verilog File Name

Specifies the name of the Verilog source file with ECO hierarchical information. The top module containing the name of the design cell must be listed in one of the Verilog files.

Specified Verilog File is a Verilog List File

Indicates that the specified Verilog file includes a list of all Verilog source files with ECO change data. There must be a top module with the name of the design cell in one of the listed Verilog files.

Top Module name

Specifies the Verilog module name. The hierarchical design is expanded from the top module down to a leaf instance and saved to a cell named *top_cell_name*.ECO.

Ignore Tie High/Low Net Name Pattern

If this option is set to one or more patterns, Milkyway ignores all the matched nets during the netlist comparison process. If the option is an empty string, no net name patterns are used.

Enter the tie-high and tie-low net name patterns to be ignored. The net names or patterns must be separated by commas. The default is an empty string.

Keep all metal

If this option is selected, Milkyway preserves all metal layers. By default, Milkyway modifies metal layers.

Ignore Power/Ground

If this option is selected, Milkyway ignores power and ground nets. This option is selected by default.

Freeze Via Cell Instances

If this option is selected, Milkyway preserves the via cell instances. This option is selected by default.

Freeze Synthesized Clock Nets

If this option is selected, Milkyway preserves synthesized clock buffer cells and nets added by clock tree synthesis. By default, Milkyway does not keep the synthesized clock buffers and nets.

Soft Freeze Synthesized Clock Nets

If this option is selected, Milkyway preserves the synthesized clock buffer cells and nets added by clock tree synthesis and deletes or adds flip-flops. By default, Milkyway does not keep the synthesized clock buffers and nets.

Freeze Pad Ring

If this option is selected, Milkyway preserves the pad ring and the existing connections to the pad cells or pad filler cells. By default, Milkyway does not keep the pad ring and its connections.

Freeze Standard Filler Cells

If this option is selected, Milkyway preserves the standard filler cells. By default, Milkyway does not keep the filler cells.

Delete Unused Pins

If this option is selected, Milkyway deletes the pins of all the ports that would be disconnected. By default, the Milkyway ECO process does not delete the unconnected pins.

Delete Cell Instance Immediately

If this option is selected, the cell instances that do not exist in the new ECO design are removed immediately from the design cell; if the option is deselected, the cell instances to be removed are instead marked for deletion later during ECO placement and the names of the cell instances are cleared. This option is selected by default.

Delete Unused Nets Immediately

If this option is selected, the unused nets and all their child nets are immediately deleted; if the option is deselected, the nets remain in the design cell. This option is selected by default.

Ignore Floating Net

If this option is selected, Milkyway ignores all unconnected nets in both the design cell and the new ECO design during netlist comparison. (Floating nets are unconnected nets or single pin nets.) Milkyway searches all ports in the ECO design for connected nets and then determines whether the nets differ from those in the design cell.

If the option is not selected, changes in the floating nets are made in the design cell. This option is not selected by default.

No Freeze Scan Chains

Specifies that the scan chain ports are to be compared and changes made in the design cell. If this option is deselected, the scan chain ports will be ignored (and preserved). This option is selected by default.

Note:

There must be scan chains in the design cell.

Freeze Scan Chains

Specifies rules for preserving scan chains. If this option is selected, none of the scan chain ports are changed and the cell instances with scan chain ports are not deleted. This option is not selected by default.

Note:

There must be scan chains in the design cell.

Soft Freeze Scan Chains

Specifies rules for preserving scan chains. If this option is selected, the scan chain ports (except for top-level ports) and the cell instances with scan chain ports (except for beginning or ending scan chain cells) can be changed. This option is not selected by default.

Note:

There must be scan chains in the design cell.

ECO Report File

Specifies the name of the file to which Milkyway will write the report.

auHierVerilogOut

Translates connectivity information contained in a Synopsys top-level cell to a file in hierarchical Verilog format.

Note that this command is replaced with `astDumpHierVerilogOut`. Also, the `auHierVerilogOut` command cannot be used to output the netlist of a cell created with `auVerilogToCell`.

Tools and Menu CommandsVerilog

Milkyway: Output > Hierarchical Verilog Out

Hierarchical Verilog Out Dialog Box

The menu command opens the following dialog box. Its options are described below.

Hierarchical Verilog Out

OK Cancel Default Help

Library Name

Cell Name

Verilog Out Data File

☒ No empty Cell Module Definitions

☒ No Corner Pad Instances

☒ No Pad Filler Cell Instances ☐ No Ports connected to Pad Filler Cell Instances

☒ No Core Filler Cell Instances

☒ No Unconnected Instances ☒ No Unconnected Ports

☐ No P/G nets ☒ No P/G ports

☒ No Diode Ports

Cell Rename Map File

☐ Output Wire Declaration

☒ Output 1'b1 for Power(VDD, vdd, ...) and 1'b0 for Ground(VSS, gnd, ...)

☒ Output Bus As Individual Bits

Reference Top NETL Cell To Output Bus

☒ Strip BackSlash Before Hierarchy Separator

☒ Output Simple Verilog File

☐ Output Instances By Sorted Names

Library Name

Type the name of the library containing the cell which you want to translate.

Cell Name

Type the name of the top-level cell that you created in “create hierarchical netlist.”

Verilog Out Data File

Type the name of the file to which you want to write the cell information.

No Empty Cell Module Definitions

Select to prevent Milkyway from translating empty cells.

No Corner Pad Instances

Select if you do not want Milkyway to translate corner pad cell instances.

No Pad Filler Cell Instances

Select if you do not want Milkyway to translate pad filler cell instances.

No Core Filler Cell Instances

Select if you do not want Milkyway to translate core filler cell instances.

No Unconnected Instances

Select if you do not want Milkyway to translate unconnected instances.

No Unconnected Ports

Select if you do not want Milkyway to translate unconnected ports. If this option is not selected, unconnected ports will be generated and new nets will be created to connect the ports. The naming style for these nets is AVANT_UNCONNECTED_%d.

No P/G nets

Select if you do not want Milkyway to translate power and ground nets. By default, this option is not selected, and power and ground nets (tie-up or tie-down only) are translated and output.

No P/G ports

Deselect if you want Milkyway to translate the power and ground ports. By default, this option is selected, and the power and ground ports will not be translated and output.

Cell Rename Map File

Type the name of the cell mapping file you want Milkyway to use to map cell names during translation, or leave blank to use GDSII cell names. If you used a VHDL to GDSII map file to map cell names when you imported your netlist, use this same file to map names back to HDL.

Output Wire Declaration

Select to add wire information to the output. By default, this option is not selected and no wire declaration is generated.

Output 1'b1 for Power (VDD, vdd, ...) and Output 1'b0 for Ground (VSS, gnd, ...)

Select to output power nets as 1'b1 and ground nets as 1'b0. For example, if you have power net VDD connected to *port1*, and ground net VSS connected to *port2*

And the Option is ...	Milkyway outputs the nets as ...
Selected	assign port1 = 1'b1 ; assign port2 = 1'b0 ;
Not selected	assign port1 = VDD ; assign port2 = VSS ;

When this option is not selected, “supply1” and “supply0” are generated for power and ground nets at the net declaration section of the verilog file. When this option is selected, as it is by default, “1'b1” and “1'b0” are created at the cell instantiation section and no net declaration is created for them.

Output Bus As Individual Bits

Select to output buses as individual bits. For example, if you have a bus with bits A[0], A[1], and A[2]:

And the Option is ...	Milkyway outputs the bus as ...
Selected	<pre>module X (A[2], A[1], A[0]); input A[2]; /*Individual Bits*/ input A[1]; /*Individual Bits*/ input A[0]; /*Individual Bits*/ endmodule</pre>
Not selected	<pre>module X (A); input [2:0] A; /*As a Bus*/ endmodule</pre>

When this options is not selected, all bits of a bus are combined into a bus port (or net). In such case, Milkyway ignores *No Unconnected Ports* and *No P/G Nets*, so that unconnected ports and tie-off nets are generated.

Reference Top NETL Cell To Output Bus

Type the name of the .NETL cell which contains the bus information you want to output.

Very often, a hierarchical .NETL cell is expanded and then bound to a top cell, usually with a different name for placement and routing. Because the bus information is stored in the .NETL cells, you need to specify the original top .NETL cell name when outputting bus information.

Strip BackSlash Before Hierarchy Separator

When selected, the tool removes backslashes that appear before hierarchy separators for every identifier, including ports, nets, and instances.

Output Simple Verilog File

When selected, the tool generates a Verilog file that is smaller and easier to read. Used for backward compatibility, selecting this option allows you to generate a simple format Verilog file similar to version 2.2.

Output Instances By Sorted Names

Select this feature to control the order of instances in the Verilog Files. When selected, the feature will output filenames alphabetically. When deselected, filenames will order arbitrarily.

auLefIn

Imports a cell library from Library Exchange Format (LEF). It contains library information for a class of designs. Creates a *.CEL* view of cells in Milkyway.

Tool

Milkyway

Prerequisites

A technology file is needed to create a library or open an existing library.

LEF In Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Specify the library where the LEF file will be imported.

LEF File

Type the name of the LEF file.

skip techfile

Skips the technology file creation.

over write existing cells

Select if you want over write mode.

poly layer

Select to define the poly layer in the technology file.

expand aliases

Select to expand aliases. When selected, a new LEF file named “lef_file_name.alias_exp” is automatically generated and contains expanded aliases. When any error messages are displayed in the log file, the error line numbers will correspond to this new LEF file.

GDSII

Select to use translated GDSII data as physical data.

annotate GDSII text

Select for text annotation while GDSII option is on.

via pin

Select to generate via pins.

Cell Type Definition File

Specify the name of cell type definition file.

auLib2LCIf

Extracts logical information and creates a logical information file from your Synopsys .lib file.

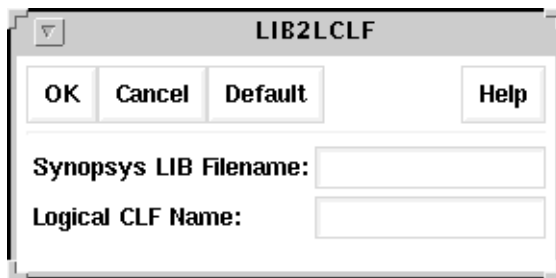
Also created is a new file with the extension, “.con.” which contains the converted constraints imported from the Synopsys .lib file, including the transition delay constraints and maximum capacitance constraints. You can load the constraint file using [auLoadCLF](#) to ensure that your constraints are in place when you perform optimization operations.

Tool

Milkyway

LIB2LCLF Dialog Box

The menu command opens the following dialog box. Its options are described below.



Synopsys LIB Filename

Type the name of the Synopsys .lib file from which you want to extract logical information.

Logical CLF Name

Type a name for the CLF file you wish to create from the extracted logical information.

auLoadCLF

Loads a Cell Library Format (CLF) file into a library you specify.

Tools and Menu Commands

Milkyway: CLF > Load

Prerequisites

Before using this command you need to extract logical information and create a logical information file from your Synopsys .lib file. See [auLib2LClf](#). You can use [augLibCompiler](#) to get the CLF information.

Load CLF File Dialog Box

The menu command opens the following dialog box. Its options are described below.

Load CLF File

OK Cancel Default Help

☐ Load CLF File Without Timing Related Information

Best CLF File Name Best Value ☐ min ☒ nom ☐ max

Typical CLF File Name Typical Value ☐ min ☒ nom ☐ max

Worst CLF File Name Worst Value ☐ min ☒ nom ☐ max

Library Name

☐ Translate Linear Model To Table Lookup Model

Input Net Transition: Min 0.2 Nom 2.0 Max 4.0

Output Capacitance: Min 0.01 Nom 0.1 Max 1.0

Constrained Pin Transition: Min 0.2 Nom 2.0 Max 4.0

Related Pin Transition: Min 0.01 Nom 0.1 Max 1.0

Load CLF File Without Timing Related Information

Allows you to load non-timing CLF data. (Supports the .LM view-based flow, which is the preferred library data preparation flow.)

When you enable this option, a dialog box is displayed for entering the CLF file name and library.

Best CLF File Name

Type the name of the best case CLF file you want to load.

Best Value

Choose the value you want to use when loading the CLF for the Best-Worst Case.

Typical CLF File Name

Type the name of the typical case CLF file you want to load.

Typical Value

Choose the value you want to use when loading the CLF for the Best-Worst Case.

Worst CLF File Name

Type the name of the worst case CLF file you want to load.

Worst Value

Choose the value you want to use when loading the CLF for the Best-Worst Case.

Note:

You need to specify all three values (Best, Typical, Worst) on the dialog box for this feature to work properly. You can use the same value for all, or choose different conditions. For example, enter a CLF file and choose min for the Best Value, nom for the Typical value, and max for the Worst value. This variation will produce the same CLF as the original one.

Library Name

Type the name of the library to which you want to load the CLF file. You can only load CLF information in to one library at a time. Execute this command separately for each reference library into which you want to load CLF information.

Translate Linear Model To Table Lookup Model

Select to translate delay information specified using the linear delay model to the table lookup delay model when loading the CLF file.

Note:

This option is also available on the Synopsys To CLF Translator dialog box. If you generate your CLF file using [auSynopToCLF](#), select the option on the Synopsys To CLF Translator dialog box and deselect it here.

Input Net Transition

Specify index values of Input Net Transition in timing table when translating a linear model to a table lookup model. See also [clfCreateTable](#).

Output Capacitance

Specify index values of Output Capacitance in timing table when translating a linear model to a table lookup model. See also [clfCreateTable](#).

Constrained Pin Transition

Specify index values of Constrained Pin Transition in timing table when translating a linear model to a table lookup model. See also [clfCreateTable](#).

Related Pin Transition

Specify index values of Related Pin Transition in timing table when translating a linear model to a table lookup model. See also [clfCreateTable](#).

auNDIApi

Note:

The `auNDIApi` command has been replaced by the `read_def` command. The `auNDIApi` command is no longer supported. For information about the `read_def` command, see the man pages or the *Milkyway Environment Data Preparation User Guide*.

Imports a DEF file to a Milkyway design cell. DEF (Design Exchange Format) defines the elements of an IC design relevant to physical layout, including the netlist and design constraints. It contains the design-specific information of a circuit and is a representation of the design at any point during the layout process.

Note:

Supports DEF syntax version 5.3.

The LEF/DEF syntax and Milkyway database use conflicting object names. LEF/DEF macros are equivalent to Milkyway reference cells (including macro cells, standard cells, pad cells, etc). LEF/DEF designs are equivalent to Milkyway (top level) design cells. Unless otherwise noted, all object names refer to the LEF/DEF object definition. See the “LEF/DEF Supported Syntax” document for a description of supported objects and the mapping between LEF/DEF and Milkyway.

During the DEF input process, files used by other functions are automatically generated or that need to be loaded into the design cell.

`cellName.assignVarRoute.sc` - Scheme file that assigns the Milkyway variable route rules in the design cell. This file is created from the DEF net nondefault rule statements.

cellName.tdf - TDF file containing the Milkyway design cell's cell instance placement locations. This file is created from DEF component statements.

cellName.viaNameMap - List of via name mappings. Each via name is mapped from the actual via name (in the Milkyway design) to the original via name (as specified in the DEF input file). (This file is optionally passed to the "DEF Out API" dialog box.)

cellName.LongViaNameMap - List of long via name mappings. Via names longer than 31 characters are shortened. This file maps the original long name to the new shorter name. During DEF output, this file is automatically loaded to retrieve and output the original via name.

Tool

Milkyway

DEF In Api Dialog Box

The menu command opens the following dialog box. Its options are described below.

The image shows a dialog box titled "DEF In Api". It has a standard Windows-style title bar with minimize, maximize, and close buttons. The dialog contains the following elements:

- Buttons at the top: "OK", "Cancel", "Default", and "Help".
- Five text input fields:
 - Library Name
 - Ref Library Name(s)
 - Cell Name
 - DEF File
 - Tech File
- A checkbox labeled "Overwrite Existing Cell" below the input fields.
- Three buttons at the bottom: "Row Options", "Track Options", and "Via Options".

Library Name

Specifies the Milkyway library in which the design cell will be created. The string may include the path of the library, otherwise the library is resolved from the current working directory.

Ref Library Name(s)

Specifies the Milkyway reference libraries. The string may be a single library or a comma delimited list of libraries. The libraries are added to the main library's reference list. Duplicate and existing library references are ignored. Each library name string may include the path of the library, otherwise the library is resolved from the current working directory.

Cell Name

Specifies the name of the Milkyway design cell which will be created.

DEF file

Specifies the DEF input file.

Tech file

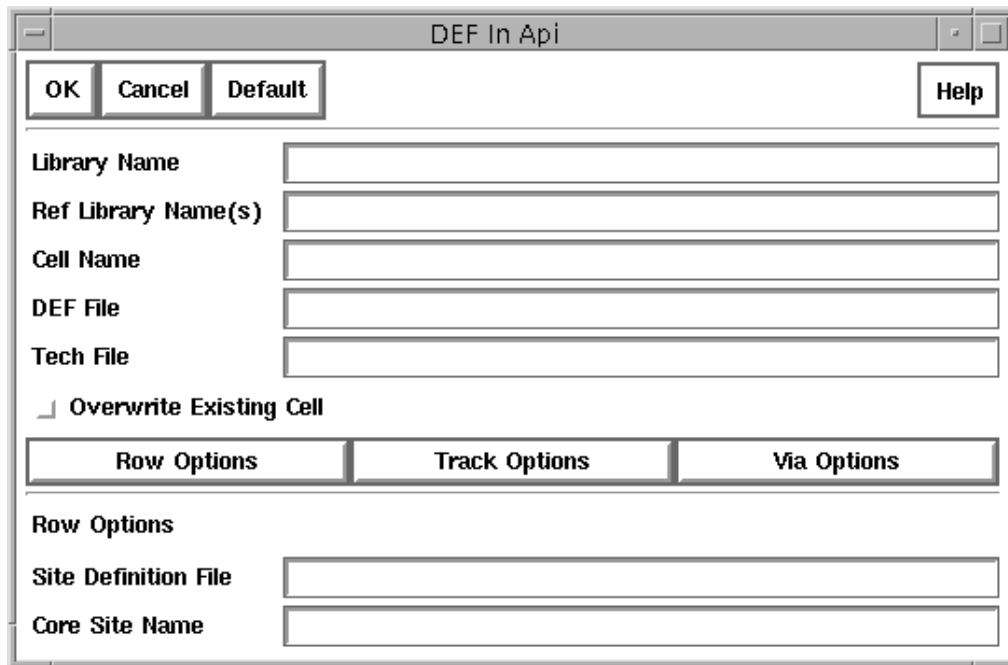
Specifies the Milkyway technology file that is used to create the Milkyway library. If the library already exists, this technology file is not used.

Overwrite Existing Cell

When selected, the most recent Milkyway design cell version is overwritten by the new cell. When not selected, a new Milkyway design cell version is created. This option only applies when a cell exists with the name as specified in the “Cell Name” field. If the cell does not exist, a new cell is always created.

Row Options Dialog Box

The menu command opens the following dialog box. Its options are described below.



The image shows a dialog box titled "DEF In Api". At the top, there are four buttons: "OK", "Cancel", "Default", and "Help". Below these buttons are five text input fields with labels: "Library Name", "Ref Library Name(s)", "Cell Name", "DEF File", and "Tech File". Below the input fields is a checkbox labeled "Overwrite Existing Cell". At the bottom, there are three tabs: "Row Options", "Track Options", and "Via Options". The "Row Options" tab is currently selected, showing two more text input fields labeled "Site Definition File" and "Core Site Name".

Row Options

Cell rows are created, when the corresponding Milkyway core sites and unit tiles are found in the database.

Sites are created in the design using either the definitions in the Site Definition File, or from the sites and tiles already existing in the library. Please see the "Site Definition File" option below.

Site Definition File

Specifies the Site Definition File. This file contains the core and pad site definitions, and is generated during LEF input from the site statements. Sites are created in the Milkyway design cell using these definitions.

If the Site Definition File is not specified, sites are created in the Milkyway design cell using sites and tiles that are found within the library. Searching begins in the top library and proceeds through the library reference hierarchy.

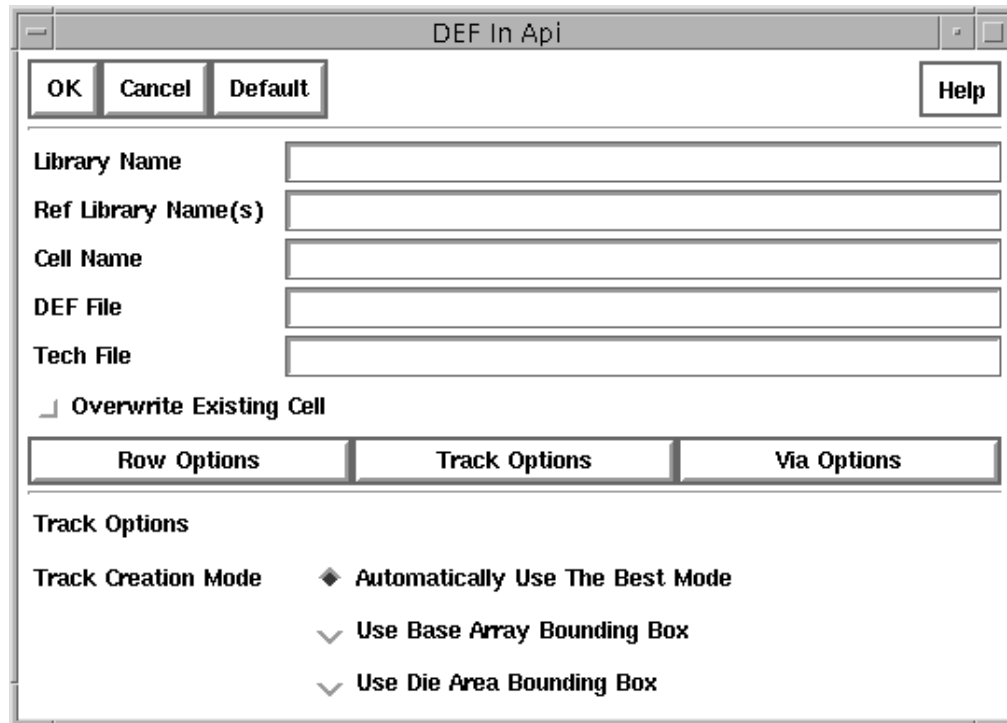
Core Site Name

Specifies the core site name. Only those rows, which utilize the specified core site, are created in the Milkyway design cell.

When the core site is not specified, all rows are created in the Milkyway design cell.

Track Options Dialog Box

The menu command opens the following dialog box. Its options are described below.



DEF In Api

OK Cancel Default Help

Library Name

Ref Library Name(s)

Cell Name

DEF File

Tech File

☐ Overwrite Existing Cell

Row Options Track Options Via Options

Track Options

Track Creation Mode

- ◆ Automatically Use The Best Mode
- ✓ Use Base Array Bounding Box
- ✓ Use Die Area Bounding Box

Track Creation Mode

Specifies the method by which to input the DEF track statements.

- Automatically Use The Best Mode - When selected, the tracks are created using the best method. The best method is determined by searching for the Milkyway base array and DEF die area. One of these objects is needed to create wire tracks. If the Milkyway base array exists, it is used to create wire tracks. Otherwise the DEF die area is used to create wire tracks. If neither of these objects are found, then an error message is displayed.
- Use Base Array Bounding Box - When selected. the tracks are created and bounded by the Milkyway base array bounding box. If the Milkyway base array does not exist, an error message is displayed.

- Use Die Area Bounding Box - When selected, the tracks are created and bounded by the DEF die area bounding box. If the DEF die area does not exist, an error message is displayed.

Via Options Dialog Box

The menu command opens the following dialog box. Its options are described below.

The screenshot shows a dialog box titled "DEF In Api". It has buttons for "OK", "Cancel", "Default", and "Help". Below these are input fields for "Library Name", "Ref Library Name(s)", "Cell Name", "DEF File", and "Tech File". There is a checkbox for "Overwrite Existing Cell". Below this is a tabbed interface with three tabs: "Row Options", "Track Options", and "Via Options". The "Via Options" tab is selected, showing options for "Translate Turn Vias To" (with radio buttons for "Cell Instances" and "Paths"), "Translate Inexactly Matched Vias To" (with radio buttons for "Cell Instances" and "Contact Arrays And Wires"), and "Via Cell Name Base" (with a text field containing "ViaCell").

Via Options

For each definition in the via section, a corresponding Milkyway via cell is created in the library. Each of these via cells have the physical geometries as specified by the DEF VIA statements.

For each via referenced in the net section wiring, it is matched to either a via cell or a Milkyway technology contact code. When a match is found, the via is translated into the design either as a Milkyway contact or contact array. If a match is not found, the via is translated as a Milkyway via cell instance.

A simple via (single cut rectangle) must match a Milkyway technology contact code by name; or by its layer information, enclosure dimensions and cut dimensions. A via array (multiple cut rectangles) must match a Milkyway technology contact code by its layer information, enclosure dimensions, cut dimensions, and cut spacing. Layer information, enclosure dimensions, and cut dimensions must match exactly. The DEF VIA cut spacing must be greater or equal to the Milkyway technology contact codes's minimum cut spacing.

There are two exceptions to these rules for Turn Vias and Inexactly Matched Vias. Please see the options below...

Translate Turn Vias To

Specifies how Turn Vias are translated. Turn Vias are represented in DEF as a single rectangle (usually a piece of dangling metal).

- Cell Instances - When selected, Turn Vias are created as via cell instances. The vias remain marked as Turn Vias.
- Paths - When selected, Turn Vias are created as path objects. The vias are no longer marked as Turn Vias.

Translate Inexactly Matched Vias To

Specifies how Inexactly Matched Vias are to be translated. Inexactly Matched Vias are vias which match a Milkyway technology contact code's cut dimensions, but do not match the contact code's enclosure dimensions.

- Cell Instances - When selected, Inexactly Matched Vias are created as via cell instances.
- Contact Arrays And Wires - When selected, Inexactly Matched Vias are created as contact arrays and extra wires on the enclosure layers. The extra wires are needed to meet the via's enclosure requirements.

Note:

This option is valid only for via arrays with enclosure dimensions that are greater than the Milkyway technology contact code's enclosure dimensions.

- Via Cell Name Base - Specifies the via cell name base string. Long DEF via names are renamed and mapped to shorter names. These shorter names are formed by appending a unique integer to this base string.

See Also

[auNDOApi](#)
[auNLIApi](#)
[auNLOApi](#)

auNDOApi

Note:

The `auNDOApi` command has been replaced by the `write_def` command. The `auNDOApi` command is no longer supported. For information about the `write_def` command, see the man pages or the *Milkyway Environment Data Preparation User Guide*.

Exports a Milkyway design cell to a DEF file. DEF (Design Exchange Format) defines the elements of an IC design relevant to physical layout, including the netlist and design constraints. It contains the design-specific information of a circuit and is a representation of the design at any point during the layout process.

Note:

Supports DEF syntax version 5.3.

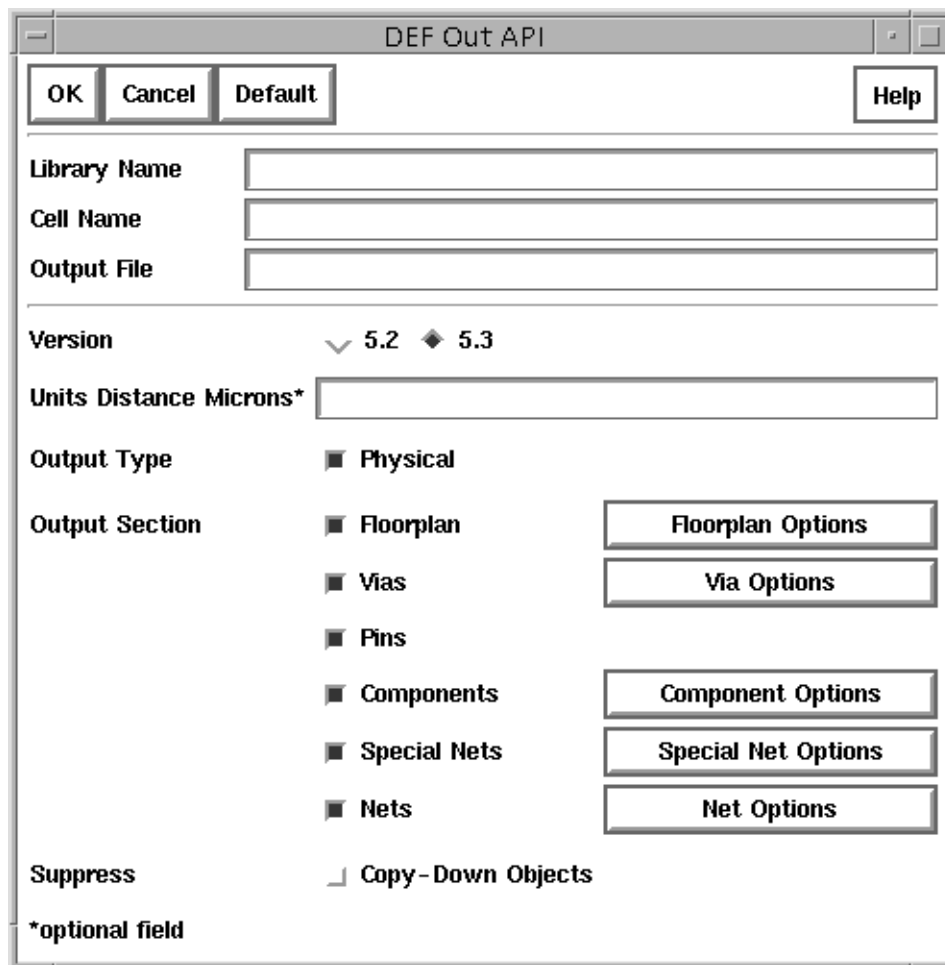
The LEF/DEF syntax and Milkyway database use conflicting object names. LEF/DEF macros are equivalent to Milkyway reference cells (includes macro cells, standard cells, pad cells, and so on). LEF/DEF designs are equivalent to Milkyway (top level) design cells. Unless otherwise noted, all object names refer to the LEF/DEF object definition. Please see the “LEF/DEF Supported Syntax” document for a description of supported objects and its mapping between LEF/DEF and Milkyway.

Tool

Milkyway

DEF Out API Dialog Box

The menu command opens the following dialog box. Its options are described below.



The image shows a dialog box titled "DEF Out API". It has a standard Windows-style title bar with minimize, maximize, and close buttons. Below the title bar are four buttons: "OK", "Cancel", "Default", and "Help". The dialog contains several input fields and checkboxes:

- Library Name:** A text input field.
- Cell Name:** A text input field.
- Output File:** A text input field.
- Version:** A dropdown menu showing "5.2" and "5.3" as options.
- Units Distance Microns*:** A text input field.
- Output Type:** A checkbox labeled "Physical".
- Output Section:** A list of checkboxes: "Floorplan", "Vias", "Pins", "Components", "Special Nets", and "Nets".
- Options Buttons:** To the right of the "Output Section" list are six buttons: "Floorplan Options", "Via Options", "Component Options", "Special Net Options", and "Net Options".
- Suppress:** A checkbox labeled "Copy-Down Objects".
- *optional field:** A label at the bottom left.

Library Name

Specifies the Milkyway library which contains the design cell. The string may include the path of the library, otherwise the library is resolved from the current working directory.

Cell Name

Specify the name of the design cell.

Output File

Specifies the name of the DEF file to which the design cell is written. If the file already exists, it is overwritten. When the file does not exist, it will be created. The string may include the path of the output file, otherwise the file is written to the current working directory.

Version

Specifies which DEF syntax version should be written to the DEF file.

When “5.4” is selected, then the DEF file complies with the 5.4 syntax. In the PIN section, the “.extra” keyword is used to separate each extra physical pin name and number. When the rectangle objects exist in the “FILL” view cell with null net IDs, the FILLS section is written to include those rectangles.

When “5.3” is selected, then the DEF file complies with the 5.3 syntax. In the PIN section, the “.extra” keyword is used to separate each extra physical pin name and number.

When “5.2” is selected, then the DEF file complies with the 5.2 syntax. In the PIN section, the “_” character is used to separate each extra physical pins name and number.

Note:

If the “Row Output Mode” option is set to “Use Original Row Statements From DEF Input” (Mode 0), then version 4.5 may be written, if the original row statements contain 4.5 syntax statements.

If the value is invalid or not specified, then the conversion factor is set according to the Milkyway library technology file’s length precision value.

If neither of these values are valid, then the default conversion factor is 100.

Design Name*

Specifies output design name. If not specified, using input cell name.

Units Distance Microns*

Specifies the conversion factor that is written to the DEF UNITS DISTANCE MICRONS statement. All distance conversions from Milkyway to DEF are based on this factor. The valid values are 100, 200, 1000, and 2000. If no value or an invalid value is specified, 100 will be used.

* optional field

Output Type

- physical - When selected, physical layout information is written. The physical layout information includes the via section, vias in the special and regular nets, component placement information, and special and regular net wiring.

When not selected, only the logical connectivity information is written.

Output Section

- Floorplan - When selected, the floorplan sections are written. The floorplan sections include the row, track, and global cell grid (GCell grid) statements.

When not selected, the floorplan sections are not written.

- Vias - When selected, the via section is written.

When not selected, the via section is not written.

- Pins - When selected, the pin section is written.

When not selected, the pin section is not written.

- Components - When selected, the component section is written.

When not selected, the design components are not written; instead an empty component section with zero components is written. The component section is required in the DEF syntax.

- Special Nets - When selected, the special net section is written.

When not selected, the special net section is not written.

- Nets - When selected, the regular net is written. When not selected, the design regular nets are not written; instead an empty regular net section with zero nets is written. The net section is required in the DEF syntax.
- Scan Chains - When selected, the scan chain section is written. When not selected, the scan chain section is not written.

Suppress

- Copy-Down Objects - When selected, copy-down objects (components, pins, and net wiring that have been marked as copy-down) are not written. Copy-down components are omitted from the component section. Copy-down pins are omitted from the pins, nets, and special nets sections. Copy-down net wiring is omitted from the special net and regular net sections. When not selected, copy-down objects are written.

Floorplan Options Dialog Box

The menu command opens the following dialog box. Its options are described below.

The dialog box is titled "DEF Out API" and contains the following fields and options:

- Buttons:** OK, Cancel, Default, Help.
- Library Name:** Text input field.
- Cell Name:** Text input field.
- Output File:** Text input field.
- Version:** Dropdown menu showing 5.2 and 5.3.
- Units Distance Microns*:** Text input field.
- Output Type:** Radio button for Physical.
- Output Section:** Radio buttons for Floorplan, Vias, Pins, Components, Special Nets, and Nets. Each has an associated "Options" button (Floorplan Options, Via Options, Component Options, Special Net Options, Net Options).
- Suppress:** Check box for Copy - Down Objects.
- *optional field:** Section header for optional fields.
- Floorplan Options:**
 - GCellGrid Output:** Radio button for Use Global Route Cell Data.
 - Row Output Mode:** Radio button for Use Original Row Statements From DEF Input.
 - Use Library Site Definition:** Check box with a dropdown for Library Site Name*.
 - Use Site Definition File:** Check box with a dropdown for Site Definition File and a text field for Site Name*.
 - Use Unit Tile Translation:** Check box.
- Row Name*:** Text input field containing STD_ROW.

Floorplan Options

Track Output

- Output Extra Dummy Tracks on Boundary - When selected, it outputs extra dummy tracks on the boundary. Other EDA tools may need these dummy tracks so that other wire tracks are expanded to the boundary.

GCellGrid Output

- Use Global Route Cell Data - If selected, and the cell GCellGrid attached file cannot be found, the data is derived from the Milkyway design cell's global routing properties and written to the GCELLGRID section.

Row Output Mode

Default: "Use Original Row Statements From DEF Input"

Specifies the method by which to output the DEF row statements.

- Use Original Row Statements From DEF Input - When selected, the original row statements (from the original DEF input file, which were stored internally during input) are written. This option only applies to designs that were created from DEF input.
- Use Library Site Definition - When selected, the row statements are written based upon a library core site.
- Library Site Name - Specifies the library core site name. When not specified, the function searches through the library hierarchy for a core site, and uses the first core site found. Sites are created during LEF input.

- Use Site Definition File - When selected, the row statements are written based upon the specified core site that is defined in the site definition file.
- Site Definition File - Required Field (if using “Use Site Definition File” mode).

Specifies the site definition file. This file contains the core and pad site definitions, and is generated during LEF input.

- Site Name - Specifies the core site name. When not specified, the function searches through the site definition file for a core site, and uses the first core site found.
- Use Unit Tile Translation - When selected, the row statements are written based on the database row unit tiles. The row type is the name of the unit tile.
- Set Row Type String To* - Specifies the name of row type. When not specified, use the name of unit tile.
* optional field
- Row Name - Default: “STD_ROW” Specifies the row name that is written in the row statements. A unique integer is appended to the row name of each row. For example, when the specified row name is “STD_ROW”, then:

Row 1 would be named “STD_ROW_1”

Row 2 would be named “STD_ROW_2”

Row 3 would be named “STD_ROW_3”

and so on...

The row name is ignored if the row output mode is “Use Original Row Statements From DEF Input”.

Via Options Dialog Box

The menu command opens the following dialog box. Its options are described below.

DEF Out API

OK Cancel Default Help

Library Name

Cell Name

Output File

Version 5.2 5.3

Units Distance Microns*

Output Type ☒ Physical

Output Section ☒ Floorplan

☒ Vias

☒ Pins

☒ Components

☒ Special Nets

☒ Nets

Suppress ☐ Copy-Down Objects ☒

*optional field

Via Options

☐ Output Elementary Vias

Via Name Map File*

Via Options

Output Elementary Vias

When selected, Milkyway library contacts are written to the DEF via section. When not selected, Milkyway library contacts are not written to the DEF via section.

In general, library contacts should not be written to the DEF file, because the DEF file should only contain design specific information. Library contacts are not design specified. They are defined in the library technology file, and are available to any design in the library. Library contact information should only be written to the LEF file.

Via Name Map File*

Specifies the via name map file used to map the actual via names to the via names in the DEF output file. This map file is only necessary if the via names need to be changed in the output file.

A map file is automatically generated during DEF input, which maps the actual via names to the original names in the DEF input file.

When specified, the via names are written based upon this map file.

When not specified and the design contains an internal map file (created during DEF input), the via names are written based upon the internal map file.

When not specified and the design does not contain an internal map file, then no via name mapping occurs. The design via names are written as is.

Component Options Dialog Box

The menu command opens the following dialog box. Its options are described below.

DEF Out API

OK Cancel Default Help

Library Name

Cell Name

Output File

Version 5.2 5.3

Units Distance Microns*

Output Type ☒ Physical

Output Section ☒ Floorplan

☒ Vias

☒ Pins

☒ Components

☒ Special Nets

☒ Nets

Suppress ☐ Copy-Down Objects

*optional field

Component Options

Suppress ☐ Standard Filler Cell

☐ Pad Filler Cell

☐ Corner Pad Cell

Component Options

Suppress

- Standard Filler Cell - When selected, standard filler cell information is not written. It is suppressed from the component, special net component pin list, and regular net component pin list sections. When not selected, standard filler cell information is written.
- Pad Filler Cell - When selected, pad filler cell information is not written. It is suppressed from the component, special net component pin list, and regular net component pin list sections. When not selected, pad filler cell information is written.
- Corner Pad Cell - When selected, corner pad cell information is not written. It is suppressed from the component, special net component pin list, and regular net component pin list sections. When not selected, corner pad cell information is written.

Special Net Options Dialog Box

The menu command opens the following dialog box. Its options are described below.

DEF Out API

OK Cancel Default Help

Library Name

Cell Name

Output File

Version 5.2 5.3

Units Distance Microns*

Output Type ☒ Physical

Output Section ☒ Floorplan

☒ Vias

☒ Pins

☒ Components

☒ Special Nets

☒ Nets

Suppress ☐ Copy-Down Objects

*optional field

Special Net Options

Special Net ☒ WireEnds - Square

☐ Signal Detail Route Type Only

☐ Fill View Geometry

☐ Reserved Blockage

Special Net Options

Special Net

- WireEnds - Square - When selected, special net wires are written with squared end points (no extension). The written end points are on the actual wire ends. When not selected, special net wires are written with half-width extension end points. The written end points are a half-width short of the actual wire ends.

Note:

The DEF input function, [auNDIApi](#), requires that all special net wires have squared end points. If you intend to read this DEF file with [auNDIApi](#), this option must be selected.

- Signal Detail Route Type Only - When selected, only special net wires with the “Signal Detail” route type are written to the special net section. When not selected, all special net wires are written to the special net section.
- Fill View Geometry - When selected, the rectangle and contact objects in the “FILL” view cell with non-null net ids are written as special wires in the special net section. When not selected, the rectangle and contact objects in the “FILL” view cell are not written.
- Reserved Blockage - When selected, disconnected net objects (objects that are not connected to any net) are written to the special net section under the net name, “_RESERVED_BLOCKAGE”. When not selected, disconnected net objects are not written.

Net Options Dialog Box

The menu command opens the following dialog box. Its options are described below.

DEF Out API

OK Cancel Default Help

Library Name

Cell Name

Output File

Version 5.2 5.3

Units Distance Microns*

Output Type ☒ Physical

Output Section ☒ Floorplan

☒ Vias

☒ Pins

☒ Components

☒ Special Nets

☒ Nets

Suppress ☐ Copy-Down Objects

*optional field

Net Options

Net ☐ WireEnds - Square

☐ Signal Detail Route Type Only

☐ Output User Enter As Signal Route Type

☒ Output Diode Pins

Net Options

Net

- WireEnds - Square - When selected, regular net wires are written with squared end points (no extension). The written end points are on the actual wire ends. When not selected, regular net wires are written with half-width extension end points. The written end points are a half-width short of the actual wire ends.

Note:

The DEF input function, [auNDIApi](#), requires that all regular net wires have half-width extension end points. If you intend to read this DEF file with [auNDIApi](#), this option must not be selected.

- Signal Detail Route Type Only - When selected, only regular net wires with the “Signal Detail” route type are written to the regular net section. When not selected, all regular net wires are written to the regular net section.
- Output User Enter As Signal Route Type - When selected, wires with the “User Enter” route type are treated the same as wires with the “Signal Detail” route type. These wires, which have the default width or a non-default rule width, are written to the regular net section, instead of the special net section. When not selected, all wires with the “User Enter” route type are written to the special net section.
- Output Diode Pins - When selected, diode (extra) pins are written to the regular net section. If a special net has diode pins, its diode pins are written to the regular net section, but its special pins and wiring remain in the special net section. When not selected, diode pins are not written.
- Output VarRouteRule Nets As Special Nets - When selected, the NONDEFAULTRULE statement is no longer used for varRouteRule nets and all varRouteRule nets are output to the special nets section. When not selected, varRouteRule nets

are output to the regular net section and NONDEFAULTRULE statement specifies the LEF-defined nondefault rule to use when creating the net and wiring.

- Output Generated Via To Special Nets - When selected, the generated vias, that have no definition in the technology file, are written to a special net section.

See Also

[auNDIApi](#)
[auNLIApi](#)
[auNLOApi](#)

auNLIApi

Note:

The `auNLIApi` command has been replaced by the `read_lef` command. The `auNLIApi` command is no longer supported. For information about the `read_lef` command, see the man pages or the *Milkyway Environment Data Preparation User Guide*.

Imports a LEF file into a Milkyway library, creating the technology information and cells in the library. LEF (Library Exchange Format) defines the elements of an IC process technology and associated library of cell models. It contains library information for a class of designs. Library data includes layer, via, placement site type, and macro cell definitions.

Note:

Supports LEF syntax version 5.3. The LEF/DEF syntax and Milkyway database use conflicting object names. LEF/DEF macros are equivalent to Milkyway reference cells (includes macro cells, standard cells, pad cells, etc.). LEF/DEF designs are

equivalent to Milkyway (top level) design cells. Unless otherwise noted, all object names refer to the LEF/DEF object definition. Please see the “LEF/DEF Supported Syntax” document for a description of supported objects and its mapping between LEF/DEF and Milkyway.

During the LEF input process, files are automatically generated that are used by other functions or need to be loaded into the design cell.

lefdef.alias.sum - List of all alias/define statements within the LEF file.

lefFileName.alias_exp - Copy of the LEF input file with expanded alias names and define variables. Each alias name is replaced with its corresponding alias definition. Each define variable is replaced with its corresponding define expression.

libraryName.wtdSet.script - Scheme file which sets the Milkyway design cell wire track directions. This file is created from the LEF layer direction statements.

lefFileName.site_def - Site Definition File containing the placement site definitions derived from the LEF site statements. (This file is optionally passed to the DEF input and output functions.)

lefFileName.site_info - Site Information File containing the macro placement site information derived from the LEF macro site statements.

lefFileName.defineVarRoute.sc - Scheme file which defines the Milkyway design cell variable route rules. This file is created from the LEF non-default rule statements.

lefPinOrder2def - List of macros and their pins which describes the macro pin order as defined in the LEF file.

lefFileName.clf - Scheme (CLF) file which sets the Milkyway cell library gate sizes. This file is created from the LEF macro pin antenna size statements.

libraryName.eeqSet.script - Scheme file which sets the Milkyway electrically-equivalent cell information. This file is created from the LEF macro pin port statements.

libraryName.mjSet.script - Scheme file which sets the Milkyway must join port information. This file is created from the LEF macro pin must join statements.

Prerequisites

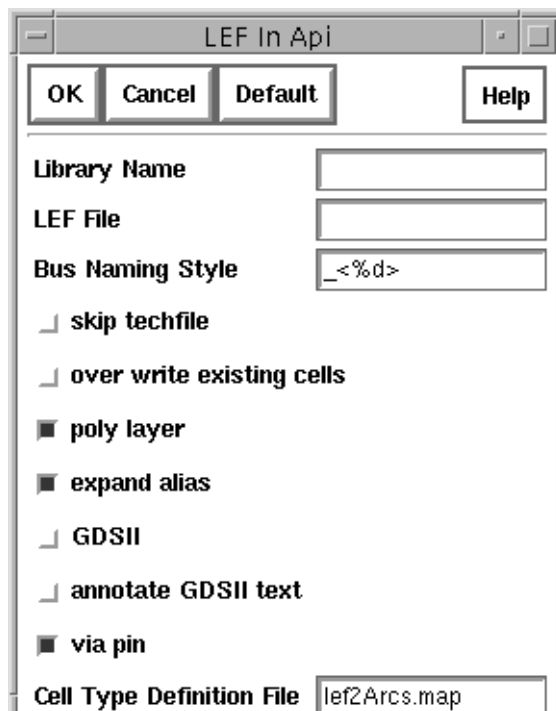
Use a technology file to create a library or open an existing library.

Tools and Menu Commands

Milkyway: DataPrep: Cell Library > LEF In

LEF In Api Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Specify the Milkyway library in which the library information and reference cells will be created. The string may include the path of the library, otherwise the library is resolved from the current working directory.

LEF File

Specify the name of the LEF input file.

Bus Naming Style

Specify the bus naming style used in Milkyway. All LEF bus names are converted to this style.

Cell Type Definition File

Specify the name of cell type definition file. (Optional field)

expand aliases

When selected, the input file alias/define names are expanded before parsing. A file (lefFileName.alias_exp) is created containing the original LEF file statements with the alias/define names replaced with the alias/define value. A file (lefdef.alias.sum) is created containing only the alias and define statements. If a lefdef.alias.sum file already exists, it will be saved as lefdef.alias.sum.old and loaded in memory before expansion occurs. If not selected, the input file alias/define names are not expanded.

GDSII

When selected, pin, rectangle, polygon, and via objects are not created. After LEF input, you are expected to input the GDSII data to create the physical information. If not selected, all objects are created.

annotate GDSII text

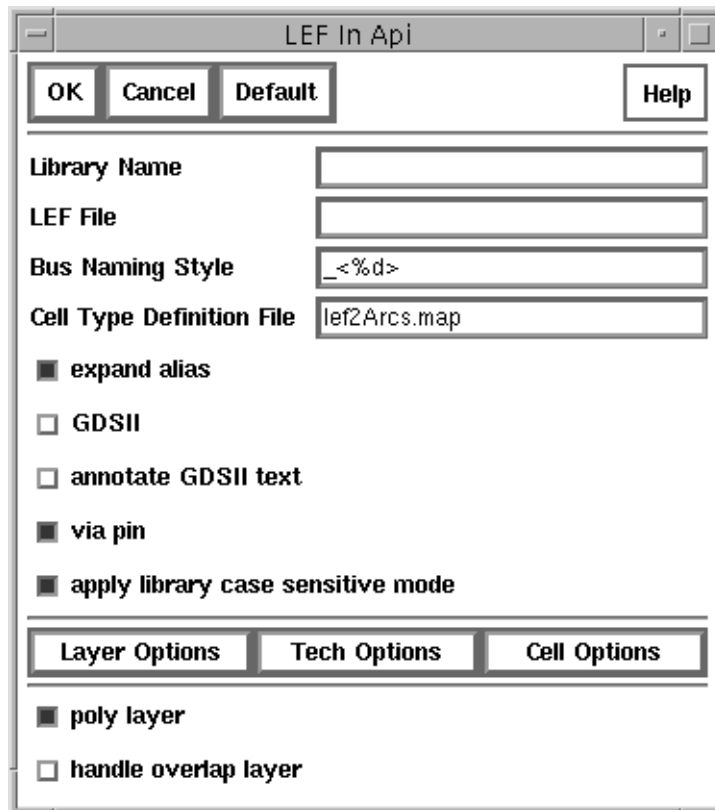
This option only takes effect when the “GDSII” option is selected (see above). When selected, the GDSII text is annotated. Pin text is created, but the actual pins are not created. When not selected, GDSII text is not annotated.

via pin

When selected, via pins are created. If not selected, via pins are not created.

LEF In Api Layer Options Dialog Box

The menu command opens the following dialog box. Its options are described below.



poly layer

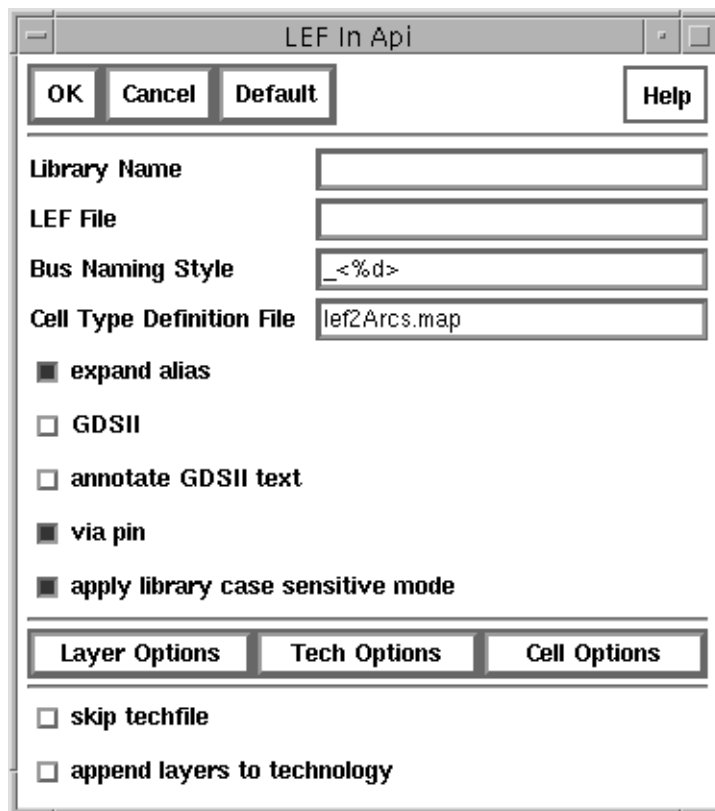
When selected, the poly layer is defined in the Milkyway library technology information. If not selected, the poly layer is not defined in the Milkyway library technology information.

handle overlap layer

When selected, Milkyway sets all objects on the overlap layer on to the Overlap Check layer, and the [auExtractBlockagePinVia](#) command can extract these objects to the .FRAM view.

LEF In Api Tech Options Dialog Box

The menu command opens the following dialog box. Its options are described below.



skip techfile

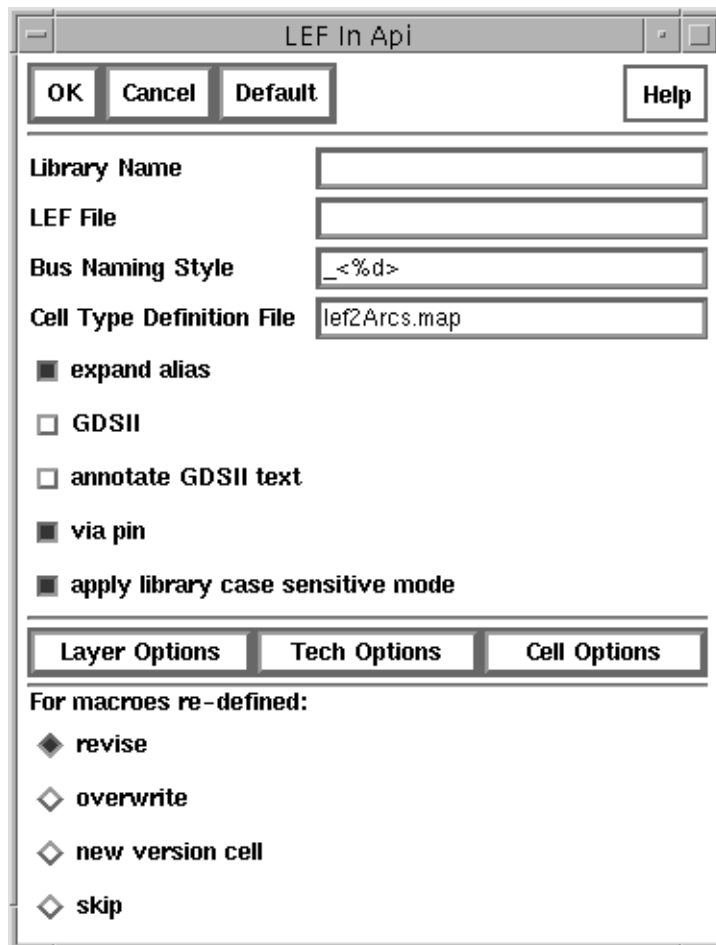
When selected, the Milkyway library technology information is not modified. When not selected, the Milkyway library technology information is modified according to the statements in the LEF file.

append layers to technology

When selected, the Milkyway library layer information is not modified. When not selected, the Milkyway library layer information is modified according to the statements in the LEF file.

LEF In Api Cell Options Dialog Box

The menu command opens the following dialog box. Its options are described below.



This set of options affects only those cells that pre-exist in both the Milkyway library and the LEF file. Cells that exist only in the library are never modified. Cells that exist only in the LEF file are always created as new Milkyway cells

revise

When selected, the latest version cell is revised base on the information defined in the LEF file. Cell related information such as macros, macro pins, and macro obstacles can be modified or appended.

overwrite

When selected, the latest version cells are overwritten by the LEF macro cells' definition.

new version cell

When selected, new version cells are created by the LEF macro cells' definition.

skip

When selected, the same name macros defined in the LEF file are ignored when imported.

See Also

[auNDIApi](#)

[auNDOApi](#)

[auNLOApi](#)

auNLOApi

Note:

The `auNLOApi` command has been replaced by the `write_lef` command. The `auNLOApi` command is no longer supported. For information about the `write_lef` command, see the man pages or the *Milkyway Environment Data Preparation User Guide*.

Exports a Milkyway design cell's library technology and reference cell information to a LEF file.

LEF (Library Exchange Format) defines the elements of an IC process technology and associated library of cell models. It contains library information for a class of designs. Library data includes layer, via, placement site type, and macro cell definitions.

Note:

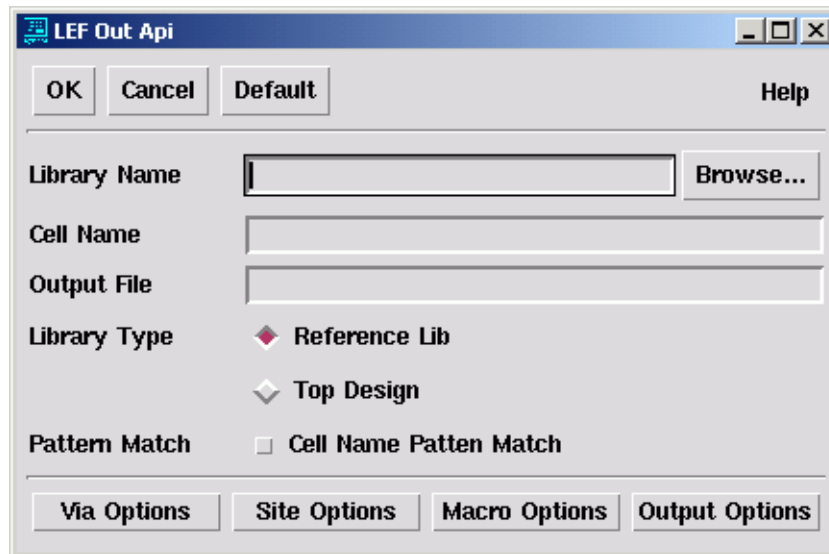
Supports LEF syntax version 5.3. The LEF/DEF syntax and Milkyway database use conflicting object names. LEF/DEF macros are equivalent to Milkyway reference cells (includes macro cells, standard cells, pad cells, etc.). LEF/DEF designs are equivalent to Milkyway (top level) design cells. Unless otherwise noted, all object names refer to the LEF/DEF object definition. Please see the "LEF/DEF Supported Syntax" document for a description of supported objects and its mapping between LEF/DEF and Milkyway.

Tools and Menu Commands

Milkyway: DataPrep: Output > LEF Out

LEF Out Api Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Specifies the Milkyway library that contains the design cell. The string may include the path of the library, otherwise the library is resolved from the current working directory.

Cell Name

Specifies the name of the design cell. The physical layout information of its reference cells (Milkyway standard, macro, I/O pad, corner pad, and cover cells) are written to the macro section of the LEF file.

Output File

Specifies the name of the LEF file to which the library information is written. If the file already exists, it is overwritten. If the file does not exist, it will be created. The string may include the path of the output file, otherwise the file is written to the current working directory.

Library Type

Library Type has the following options:

- Top Design - When selected, the physical layout information of the design cells are written to the macro section of the LEF file, in addition to the physical layout information of its reference cells. Only one cell name can be accepted. The option “Pattern Match” is inactive.
- Reference Lib - When selected, the physical layout information of the reference cells are written to the macro section of the LEF file. The reference cell must be marked as standard, macro, I/O pad, corner pad, or cover cell. The reference cell must have a FRAM view.

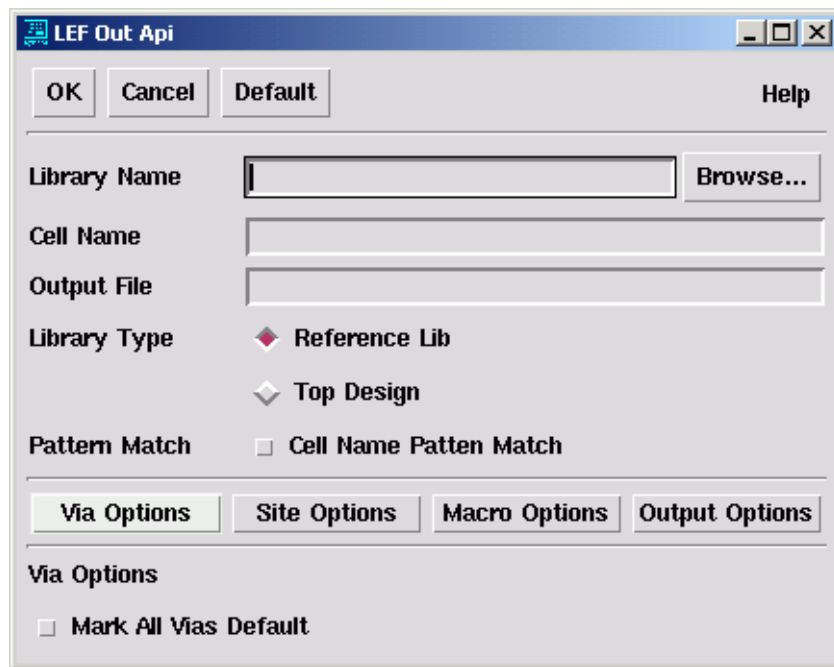
Pattern Match

Pattern Match has the following option:

- Cell Name Pattern Match - When selected, all reference cells matching the pattern name specified in the “Cell Name” field are written to the LEF MACRO section. The “Reference Lib” must be selected.

LEF Out Api Via Options Dialog Box

The menu command opens the following dialog box. Its options are described below.



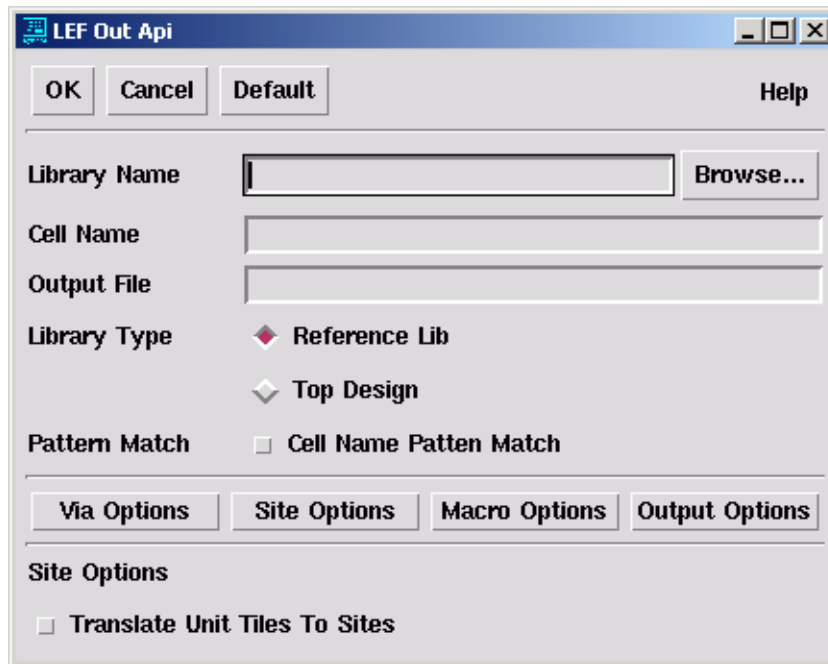
Via Options

Via Options has the following option:

- Mark All Vias Default - When selected, all vias output in the VIA section are marked as default vias. When not selected, the behavior is the same as before.

LEF Out Api Site Options Dialog Box

The menu command opens the following dialog box. Its options are described below.



Site Options

Site Options has the following option:

- Translate Unit Tiles To Sites - When selected, Milkyway library unit tiles are written to the site section. When not selected, Milkyway library unit tiles are not written.

LEF Out Api Macro Options Dialog Box

The menu command opens the following dialog box. Its options are described below.

LEF Out Api

OK Cancel Default Help

Library Name Browse...

Cell Name

Output File

Library Type ☒ Reference Lib
☐ Top Design

Pattern Match ☐ Cell Name Patten Match

Via Options Site Options **Macro Options** Output Options

Macro Options

☐ Diode Port

☐ Output via blockages

☐ Output Macro Symmetry

☐ Output Macro Foreign Mapping File

Pin Capacitance Options

Output From ☒ From LEF in ☐ From CLF

Output Value ☐ Rise Max ☐ Rise Nom ☐ Rise Min
☐ Fall Max ☐ Fall Nom ☐ Fall Min
☒ No Output

Macro Options

Macro Options has the following options:

- Diode Port - When selected, Milkyway diode ports are written to the LEF file. When not selected, Milkyway diode ports are not written.

- Output via blockages - When selected, Milkyway output via blockages are written to the LEF file. When not selected, they are not written.
- Output Macro Symmetry - When selected, Milkyway output macro symmetry is written to the LEF file. When not selected, it is not written.
- Output Macro Foreign - When selected, Milkyway

Pin Capacitance Options

Output From

Output From has the following options:

- From LEF in - Specifies the source of pin capacitance. When selected, output pin capacitance from LEF in.
- From CLF - When selected, output from capacitance is stored in the database when read in a .CLF file. (Capacitance is stored by the function: definePortCapacitance).

Output Value

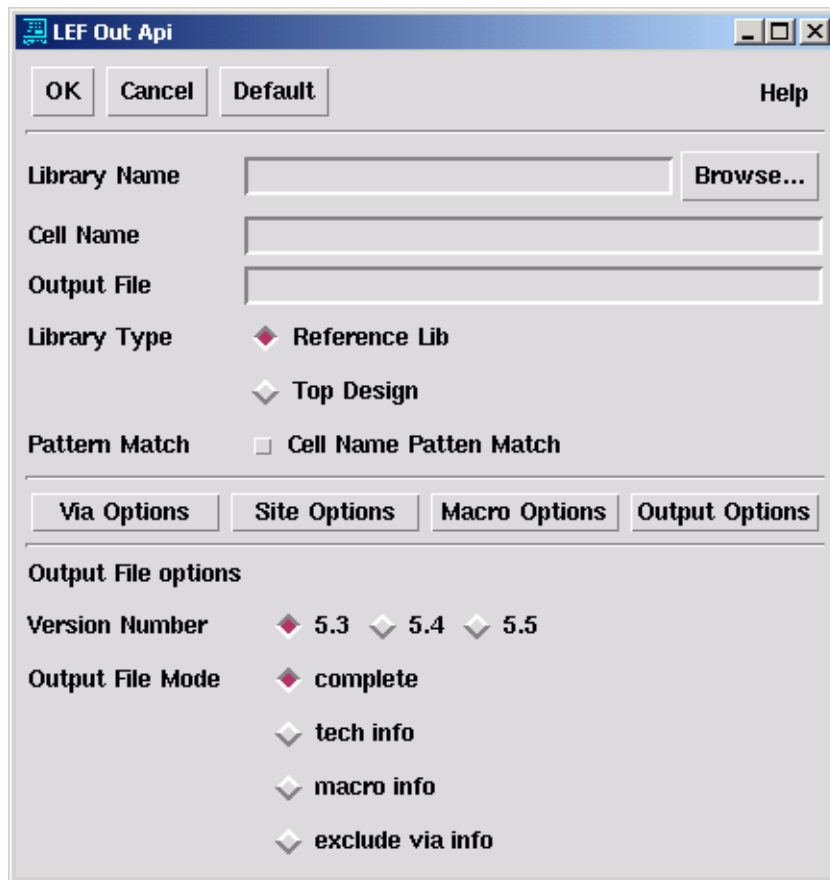
Specifies which Milkyway capacitance value to write to the LEF pin capacitance statement. Output Value has the following options:

- Rise Max - When selected, the rise maximal capacitance value is written.
- Rise Nom - When selected, the rise nominal capacitance value is written.
- Rise Min - When selected, the rise minimal capacitance value is written.

- Fall Max - When selected, the fall maximal capacitance value is written.
- Fall Nom - When selected, the fall nominal capacitance value is written.
- Fall Min - When selected, the rise minimal capacitance value is written.
- No Output - When selected, no capacitance value is written.

LEF Out Api Output Options Dialog Box

The menu command opens the following dialog box. Its options are described below.



Output File Options

Output File Options has the following options:

- Version Number: 5.3 - Default option, Outputs LEF files supported in the LEF 5.3 syntax.
- Version Number: 5.4 - Outputs LEF files supported in the LEF 5.4 syntax.
- Version Number: 5.5 - Outputs LEF files supported in the LEF 5.5 syntax.

Output File Mode

Output File Mode has the following options:

- complete - Default option. Outputs all sections to the LEF file.
- tech info - Outputs technology information (units, layers, vias, viarules, viarules generate, and nondefault rule) to the LEF file.
- macro info - Outputs macro related information (sites, arrays, and macros) to the LEF file.
- exclude via info - Outputs all sections except via information to the LEF file.

See Also

[auNDIApi](#)

[auNDOApi](#)

[auNLIApi](#)

auNewECOByNetCmp

Compares the current netlist for a design with an updated version of the netlist and makes the following changes to the layout:

- If the designer replaces a cell instance in the netlist with another cell (using the same instance name), Milkyway replaces the instance in the layout with the cell used in the netlist.

Note:

The replacement cell must have the same instance name, pin names, and pin-to-net connections as the cell it is replacing. Otherwise, Milkyway performs a deletion and addition.

- If the designer deletes a cell instance in the netlist, Milkyway deletes the instance in the layout, unless the *freeze silicon* option is turned on.

Note:

If the freeze silicon option is turned on, Milkyway marks deleted instances as spare cells and places them in a spare group called ecoBonus.

- If the designer adds a cell instance in the netlist, Milkyway adds the instance in the layout.

This command optionally imports a new or changed floor plan in Physical Design Exchange Format (PDEF).

Generates an ECO report, which contains detailed information about the comparison and update.

Caution!

You should make a backup of the cell using [geBackupCell](#) before you execute this command.

Tools and Menu Commands

Milkyway: ECO > By Net Compare

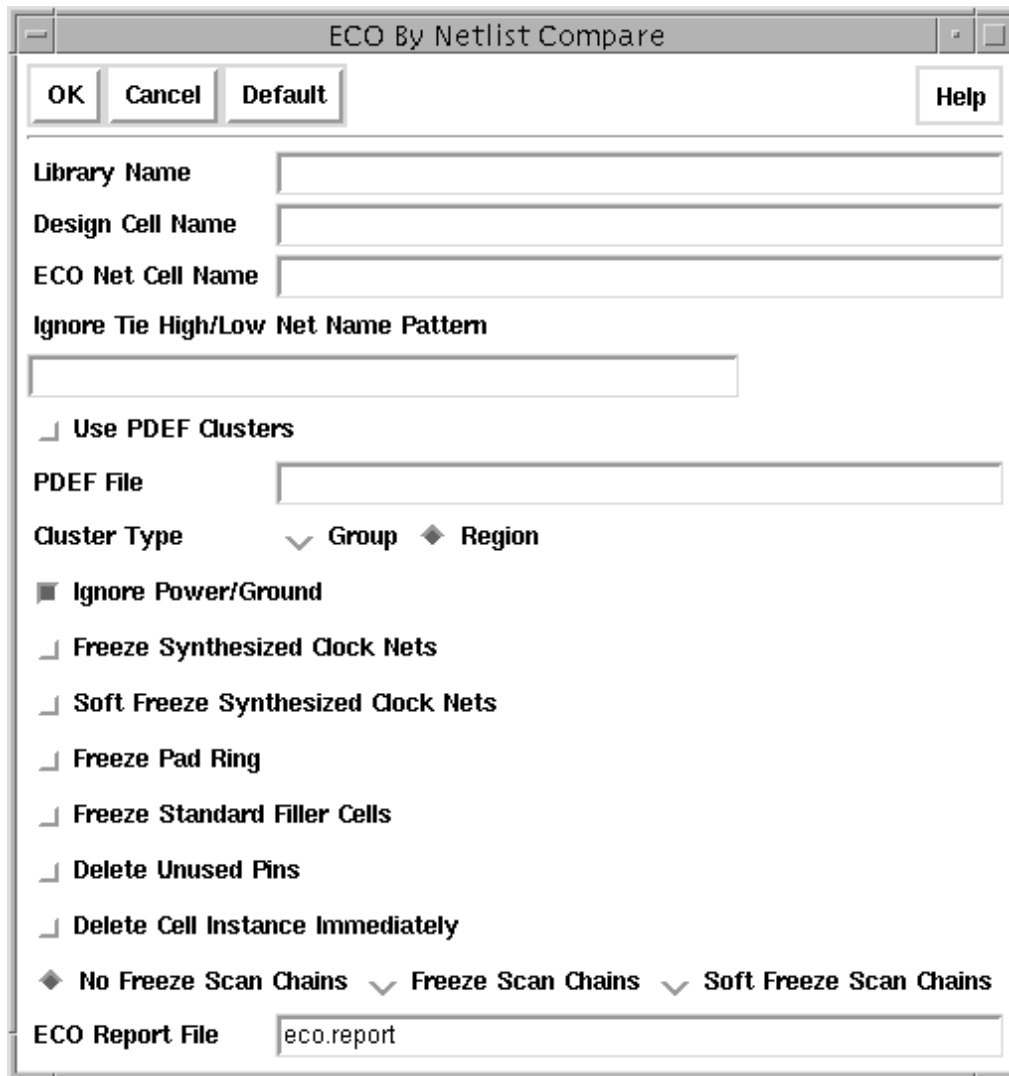
Prerequisites

Create a netlist cell for the updated netlist in the library containing the top-level cell using [auEdifIn](#).

Assume that the scan chain groups in the design cell are consistent with its netlists. If not, please re-trace the scan chains to update scan chain groups in DB.

ECO By Netlist Compare Dialog Box

The menu command opens the following dialog box. Its options are described below.



The dialog box titled "ECO By Netlist Compare" contains the following controls:

- Buttons: OK, Cancel, Default, Help
- Library Name: Text input field
- Design Cell Name: Text input field
- ECO Net Cell Name: Text input field
- Ignore Tie High/Low Net Name Pattern: Text input field
- ☐ Use PDEF Clusters
- PDEF File: Text input field
- Cluster Type: ☒ Group ☐ Region
- ☒ Ignore Power/Ground
- ☐ Freeze Synthesized Clock Nets
- ☐ Soft Freeze Synthesized Clock Nets
- ☐ Freeze Pad Ring
- ☐ Freeze Standard Filler Cells
- ☐ Delete Unused Pins
- ☐ Delete Cell Instance Immediately
- Scan Chains: ☒ No Freeze Scan Chains ☐ Freeze Scan Chains ☐ Soft Freeze Scan Chains
- ECO Report File: Text input field containing "eco.report"

Library Name

Type the name of the library containing the layout cells (and the new netlist cell) you want to update.

Design Cell Name

Type the name of the layout cell for which you want to perform ECO changes.

ECO Net Cell Name

Type the name of the netlist cell for the updated netlist.

Note:

This cell must be in the library containing the top-level cell and the old netlist.

Ignore Tie High/Low Net Name Pattern

Given the Tie high/low net name pattern(s), the engine will ignore all the matched nets during the netlist compare process. The net names must be separated by a comma.

Use PDEF Clusters

Select to use clusters defined in a PDEF file.

PDEF File

If you select *Use PDEF Clusters*, type the name of the PDEF file that defines the clusters.

Cluster Type

If you select *Use PDEF Clusters*, select the type of clusters used in the PDEF file you want to import

Option	Indicates that the clusters are ...
Group	Groups
Region	Regions

Ignore Power/Ground

Select if you want the comparison to ignore power and ground nets.

- If the netlist does not define power and ground connections, select this option.
- If you have only one power net and one ground net, Milkyway attempts to connect any new cell instances to power and ground.
- If you have multiple power and ground nets, specify port-to-net connections for any new cell instances using `aprPGConnect`.
- If the netlist defines power and ground connections, deselect this option. Milkyway connects any new cell instances to power and ground as defined.

Note:

In any case, check the ECO Report File to be sure all power and ground connections are made correctly.

Freeze Synthesized Clock Nets

Select if you want to preserve buffer cells and nets added by clock tree (or trunk) synthesis.

Soft Freeze Synthesized Clock Nets

Select if you want to preserve buffer cells and nets added by clock tree synthesis and delete or add the flip-flops.

Note:

If you turn on “Freeze Silicon” using `fphEditHierarchy` or `axgECOStatus` and you do not want any gate changes in the design during Net Compare, use the Scheme command `auECODoNotChangeGates` to set the flag. It takes a `#t` or a `#f` as an argument.

Freeze Pad Ring

Select if you want to preserve the pad ring. If this option is turned on, the existing connections to the pad cells or pad filler cells will not be modified.

Freeze Standard Filler Cells

Select if you want to preserve the standard filler cells.

Delete Unused Pins

Select if you want to delete the pins of all the ports that would be disconnected by this operation.

Delete Cell Instance Immediately

“-I” cell instances are only *marked* for deletion, but not actually deleted. They will be deleted during ECO Place. You can see that a cell is marked for deletion if the cell instance name is blank. When this option is set, “-I” cell instances will be deleted right away.

No Freeze Scan Chains

This is the default if you do not want to preserve the scan chains.

Freeze Scan Chains

Select if you want to preserve the scan chains.

Soft Freeze Scan Chains

Select if you want to preserve the scan chains except allowing the changes “+R” and “-I&+I.” For the change “-I,” deleting the beginning scan-chain cell or the ending scan-chain cell is not allowed. For the change “-I&+I,” the number of “ScanIn” or “ScanOut” ports of the “+I” cell instance in the netlist should be equal to that of the “-I” cell instance in the design cell.

ECO Report File

Type the name of the file to which you want Milkyway to write the report.

auSDFToConstraint

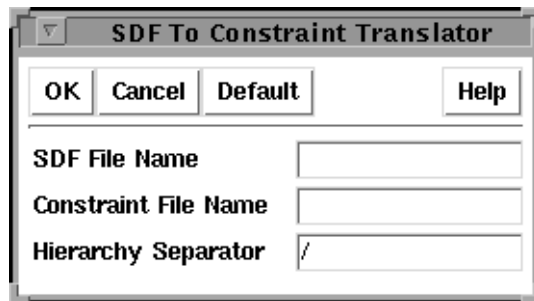
Translates a specified Standard Delay Format (SDF) file to a path constraints file. For information about path constraints files.

Tools and Menu Commands

Milkyway: Netlist In > SDF To Constraint

SDF To Constraint Translator Dialog Box

The menu command opens the following dialog box. Its options are described below.



SDF File Name

Type the name of the SDF file you want to translate.

Constraint File Name

Type the name of the path constraints file to which you want to write the translation.

Hierarchy Separator

Type the character you want to use as a separator in hierarchical names in the constraint file. This should be the same hierarchy separator you specified when you translated the netlist using [auEdifIn](#) or [auVerilogIn](#).

auSetEdifOutExternal

Specifies the location where the reference cells are written by the auEdifOut function within the output file.

Syntax

```
auSetEdifOutExternal < boolean value >
```

where the argument is as follows:

value

must be #t or #f

Description

The auSetEdifOutExternal function specifies the location where the reference cells are written by the auEdifOut function within the output file. If the boolean value is #t, the reference cell netlists are written to the “external” section. If the boolean value is #f, they are written to the “library” section (default location).

Example

The following script will generate an EDIF file with reference cell netlists written to the “external” section:

```
auSetEdifOutExternal #t
auEdifOut
setFormField "Edif Out" "Library Name" "libraryName"
setFormField "Edif Out" "Cell Name" "cellName"
setFormField "Edif Out" "Edif File Name" "dump.edif"
formOK "Edif Out"
```

auSetEmptyCellOnly

Flags and prints out only empty cells for hierarchical verilog out. The default value is 0.

Tool

Milkyway

Syntax

```
auSetEmptyCellOnly
```

auSetHierInstanceAndNetNameLong

Enables [auStreamOut](#) to output net name and cell instance name as property with a name length up to 1024 characters.

Tools and Menu Commands

Milkyway: Output > Stream Out

Syntax

```
auSetHierInstanceAndNetNameLong longName?
```

where the argument is as follows:

<i>longName?</i>	Indicates whether or not to output the long name. Valid values: #t means enable the output. #f means disable the output.
------------------	---

Examples

```
auSetHierInstanceAndNetNameLong #t
```

See Also

[auStreamOut](#)

auSetHierInstanceNameLong

Enables [auStreamOut](#) to output cell instance name as property with a name length up to 1024 characters.

Tools and Menu Commands

Milkyway: Output > Stream Out

Syntax

```
auSetHierInstanceNameLong longName?
```

where the argument is as follows:

<i>longName?</i>	Indicates whether or not to output the long instance name. Valid values: #t means enable the long instance name output. #f means the instance name will be chopped off.
------------------	--

Examples

```
auSetHierInstanceNameLong #t
```

See Also

[auStreamOut](#)

auSetHierNetNameLong

Enables [auStreamOut](#) to output net name as property with a name length up to 1024 characters.

Tools and Menu Commands

Milkyway: Output > Stream Out

Syntax

```
auSetHierNetNameLong longName?
```

where the argument is as follows:

<i>longName?</i>	Indicates whether or not to output the long net name. Valid values: #t means enable the output. #f means disable the output.
------------------	---

Examples

```
auSetHierNetNameLong #t
```

See Also

[auStreamOut](#)

auSetHierRefLibTwoLevels

Use this command to enable a two-level design/reference library hierarchy setup. Run this command before the streamOut process.

Tool

Milkyway

Syntax

```
auSetHierRefLibTwoLevels twoLevels?
```

where the argument is as follows:

twoLevels?

Valid values:

- #t means enable two-level references.
- #f means disable two-level references.

Examples

```
auSetHierRefLibTwoLevels #t
```

See Also

[auStreamOut](#)

auSetMasterCellNameLong

This switch enables [auStreamOut](#) to output cell names with name lengths up to 1024 characters.

Tools and Menu Commands

Milkyway: Output > Stream Out

Syntax

`auSetMasterCellNameLong longName?`

where the argument is as follows:

<i>longName?</i>	Indicates whether or not to output the long cell name. Valid values: #t means enable cell name up to 1024 characters. #f means cell name will be truncated to 24 characters.
------------------	---

Examples

```
auSetMasterCellNameLong #t
```

See Also

[auStreamOut](#)

auSetMultiLEFForeignName

Sets the `auStreamOut` command's "Multiple LEF Foreign Name" mode.

Syntax

`auSetMultiLEFForeignName <boolean mode>`

where the argument is as follows:

<i>mode</i>	• #t to write multiple LEF foreign names. • #f to not write multiple LEF foreign names (default).
-------------	---

Description

The `auSetMultiLEFForeignKeyName` command sets the `auStreamOut` command's "Multiple LEF Foreign Name" mode. LEF foreign names (that is, GDSII structure names) are saved in each cell during LEF input. This command controls how these foreign names are used during GDSII Stream output.

If the mode is set to `#t`, the `auStreamOut` command writes each cell that has multiple foreign cell names multiple times (once for each foreign cell name). The foreign cell information is written in place of the actual cell information.

If the mode is set to `#f`, the `auStreamOut` command will not write out multiple foreign names. It will either write out one foreign name for each cell, or it will not write out any foreign name, depending upon the "LEF Foreign Name" mode.

Example

```
auSetMultiLEFForeignKeyName #t
```

auSetPRBdry

Generates PR boundaries for cells in your library. A PR boundary is a polygon that encloses the unit tiles in a standard cell. It is on the PRboundary layer and can be larger or smaller than the cell boundary. Astro uses the PR boundary to align wire tracks and power and ground rails during cell placement.

The `auSetPRBdry` command creates the unit tile cell required for floorplanning. The unit tile cell is the smallest unit of placement. It is arranged in a repeating pattern during floorplanning to form the core area of the cell. You define wire tracks in the unit tile cells using `axgDefineWireTracks`.

After entering values and selections for other options on the dialog box, you can click *Analyze* to display suggested values for (*Bottom Boundary*) *Offset* and (*Height*) *Value* and to view a list of cells that Astro considers double, triple, or variable height.

The `auSetPRBdry` command also assigns a cell class to each standard cell. You can use cell classes to place different types of cells in particular areas of the design.

Tools and Menu Commands

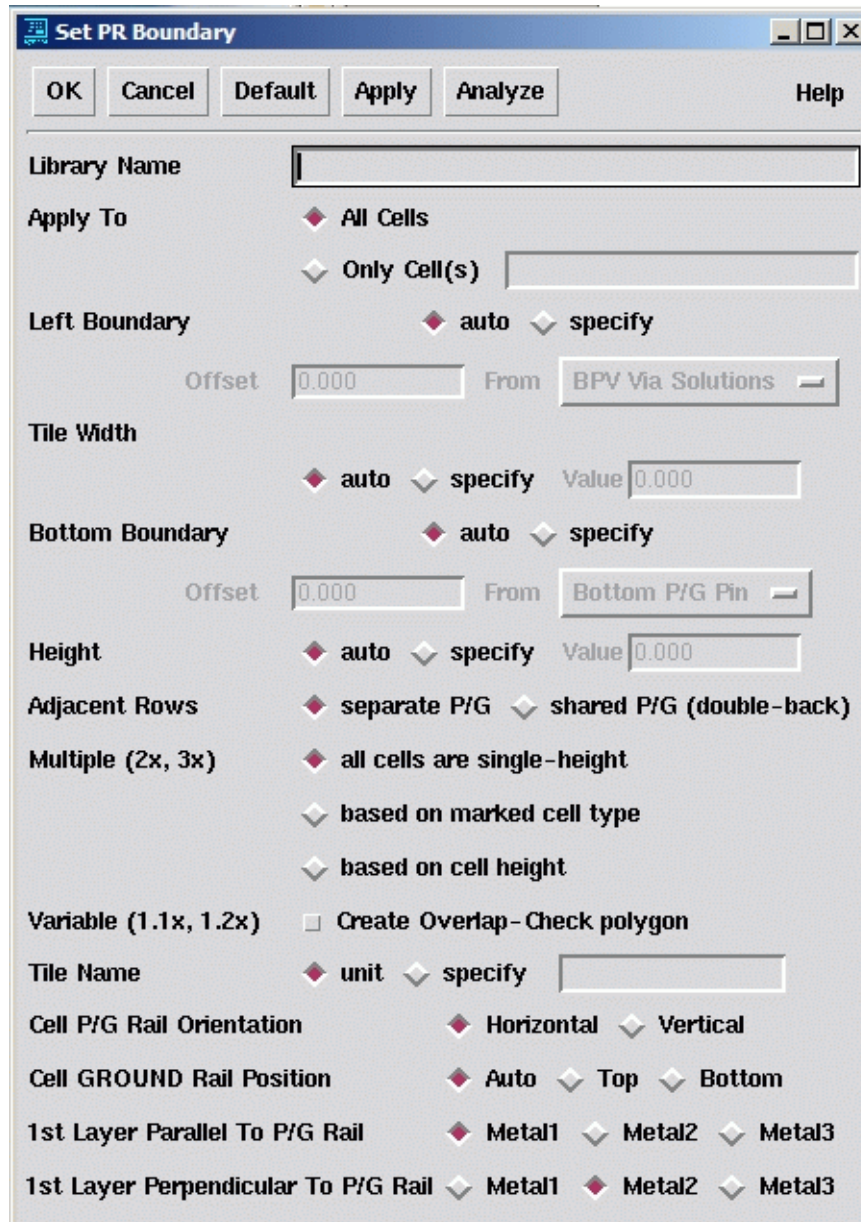
Milkyway: Cell Library > Set PR Boundary

Prerequisites

Perform pin and blockage extraction using [auExtractBlockagePinVia](#) before you generate PR boundaries.

Set PR Boundary Dialog Box

The menu command opens the following dialog box. Its options are described below.



The image shows a software dialog box titled "Set PR Boundary". It contains several sections for configuring parameters. At the top are buttons for "OK", "Cancel", "Default", "Apply", "Analyze", and "Help".

- Library Name:** A text input field.
- Apply To:** Radio buttons for "All Cells" (selected) and "Only Cell(s)" (with an associated text field).
- Left Boundary:** Radio buttons for "auto" (selected) and "specify" (with an associated text field).
- Offset:** A text field with "0.000" and a "From" dropdown menu showing "BPV Via Solutions".
- Tile Width:** Radio buttons for "auto" (selected) and "specify" (with a "Value" text field set to "0.000").
- Bottom Boundary:** Radio buttons for "auto" (selected) and "specify" (with an associated text field).
- Offset:** A text field with "0.000" and a "From" dropdown menu showing "Bottom P/G Pin".
- Height:** Radio buttons for "auto" (selected) and "specify" (with a "Value" text field set to "0.000").
- Adjacent Rows:** Radio buttons for "separate P/G" (selected) and "shared P/G (double-back)".
- Multiple (2x, 3x):** Radio buttons for "all cells are single-height" (selected), "based on marked cell type", and "based on cell height".
- Variable (1.1x, 1.2x):** A checkbox for "Create Overlap-Check polygon" which is unchecked.
- Tile Name:** Radio buttons for "unit" (selected) and "specify" (with an associated text field).
- Cell P/G Rail Orientation:** Radio buttons for "Horizontal" (selected) and "Vertical".
- Cell GROUND Rail Position:** Radio buttons for "Auto" (selected), "Top", and "Bottom".
- 1st Layer Parallel To P/G Rail:** Radio buttons for "Metal1" (selected), "Metal2", and "Metal3".
- 1st Layer Perpendicular To P/G Rail:** Radio buttons for "Metal1", "Metal2" (selected), and "Metal3".

Library Name

Type the name of the library containing the cells for which you want to create a PR boundary.

Apply To

Indicate the cells for which you want to generate PR boundaries.

Table 14-9

Option	Generates PR boundaries for ...
All Cells	All cells in the specified library.
Only Cell(s)	The cells you specify by typing the cell names. To specify multiple cells, separate the names with commas.

Left Boundary

Select how to determine the location of the left edge of the PR boundary for a cell.

Table 14-10

Option	Indicates that you want to ...
auto	Let Milkyway automatically set the location of the left edge of the PR boundary. The width of the PR boundary is a multiple of the <i>metal2</i> pitch, the left and right edges might align or extend beyond the cell boundary, and the distance from the left edge to the first <i>metal2</i> track (as defined by the via solutions generated by pin and blockage extraction) is the same for all cells in the library.
specify	Specify the location of the left edge of the PR boundary. Type the <i>Offset</i> and select the reference point <i>From</i> which you want to offset the left edge of the PR boundary.

(Left Boundary) Offset

To *specify* the *Left Boundary*, type the distance *From* the specified reference point for the left edge of the PR boundary.

Table 14-11

If you specify a ...	The left edge of the PR boundary is to the ...
Negative number	Left of the reference point.
Positive number	Right of the reference point.

(Left Boundary) From

Select the reference point to which the *(Left Boundary) Offset* applies.

Table 14-12

If you select ...	The left edge of the PR boundary is a distance equal to the specified <i>(Left Boundary) Offset</i> from ...
BPV Via Solutions	The left-most via solution generated by pin and blockage extraction.
Origin (0,0)	The origin of the cell.
Left Boundary	The left cell boundary.

Tile Width

Select from the following:

- auto – Lets Milkyway automatically set the tile width (typically the M2 pitch from the technology file).
- specify – Forces the command to generate the unit tile, using the specified width. Enter the width in the Value box.

Bottom Boundary

Select how to determine the location of the bottom edge of the PR boundary.

Table 14-13

Option	Indicates that you want to ...
auto	Let Milkyway automatically set the location of the bottom edge of the PR boundary. Select a reference point from the (<i>Bottom Boundary</i>) <i>From</i> options. Milkyway finds the cell in which the bottom edge of the cell boundary is the greatest distance from the reference point you select and sets the bottom edge of the PR boundary in that cell based on the cell row configuration you select. Then for each cell being processed, Milkyway aligns the bottom edge of the PR boundary to the cell boundary.
specify	Specify the location of the bottom edge of the PR boundary. Type the <i>Offset</i> and select the reference point <i>From</i> which you want to offset the bottom boundary.

(Bottom Boundary) Offset

To *specify* the *Bottom Boundary*, type the distance *From* the specified reference point for the bottom edge of the PR boundary.

Table 14-14

If you specify a ...	The bottom edge of the PR boundary is ...
Negative number	Below the reference point.
Positive number	Above the reference point.

Note:

To display a suggested value, select the reference point from the (Bottom Boundary) *From* options, then click Analyze.

(Bottom Boundary) From

Select the reference point to which the *(Bottom Boundary) Offset* applies.

Table 14-15

If you select ...	The bottom edge of the PR boundary is a distance equal to the specified <i>(Bottom Boundary) Offset</i> from ...
Bottom P/G Pin	The bottom power and ground rail in the cell.
Origin (0,0)	The origin of the cell.
Middle of 2 P/G Pins	A point halfway between the top and bottom power and ground rails in the cell.

Height

Select how to determine the height of the PR boundary.

Table 14-16

Option	Indicates that you want to ...
auto	Let Milkyway automatically set the height of the PR boundary. The height of the PR boundary is a multiple of the metal1 pitch and based on the height of the majority of the cells in the library. The actual height depends on the cell row configuration you select from the <i>Adjacent Rows</i> options.
specify	Specify the height of the PR boundary. Type a <i>Value</i> to indicate the height.

Value

If you want to *specify* the *Height*, type a value to indicate the height of the PR boundary.

Note:

To display a suggested value, select the reference point from the (Bottom Boundary) From options and a cell row configuration from the Adjacent Rows options, then click Analyze.

Adjacent Rows

If you selected *auto* for the *Bottom Boundary* option, select the configuration you want for adjacent cell rows.

Table 14-17

Option	Milkyway sets the height of the PR boundary based on a cell placement in which ...
separate P/G	The cells do not overlap.
shared P/G (double-back)	The cell rows are paired, with one row in each pair flipped and the cells placed so that the bottom power and ground rails of the flipped cells overlap with the top power and ground rail of the unflipped cells.

Multiple (2x, 3x)

Select the type of cells used in the design.

Table 14-18

If you select ...	Milkyway creates the PR boundary ...
all cells are single-height	With the height equal to the unit tile height for all the cells.
based on marked cell type	According to the marked cell type. Use this option if you marked cells as double or triple height using cmMarkCellType .

Table 14-18

If you select ...	Milkyway creates the PR boundary ...
based on cell height	According to the height of the cell boundary. If you marked cells as double or triple height using cmMarkCellType , the cells remain marked. If a cell is not marked double or triple height, its cell boundary is checked and Astro determines whether it is a double or triple height.

Note:

To check whether cells are double or triple height, click the Analyze button.

Variable (1.1x, 1.2x)

Select *Create Overlap - Check polygon* if a few cells are larger than other standard cells. For each cell with a cell boundary that is bigger than the PR boundary, Milkyway creates a polygon on the *overlapCheck* layer (layer number 192). Astro uses these polygons on the *overlapCheck* layer to place cells so that they do not overlap.

Tile Name

Select the unit tile for cells that will be placed in different classes of rows. If you are placing standard cells that are not double, triple, and variable height and you selected *all cells are single-height*, use the default unit tile, *unit*.

Table 14-19 Tile Name Options

Option	Description
unit	Specifies to use the default unit tile name. Milkyway creates a unit tile in the specified library called <i>unitTile</i> .

Table 14-19 Tile Name Options

Option	Description
specify	Specifies to use the name of a unit tile defined in the technology file. Milkyway creates a unit tile in the specified library called <i>name</i> Tile, where <i>name</i> is the name of the unit tile defined in the technology file. Type a tile name defined in the technology file in the <i>specify</i> entry field. See “Tile Section” in the <i>Milkyway Environment Data Preparation User Guide</i> .

Cell P/G Rail Orientation

Select the orientation, Horizontal or Vertical, in which you want the power and ground rails set.

Cell GROUND Rail Position

Select this option when you are working with multiple reference libraries with mixed ground rail positions—for example, when one library has cells with the ground rails on top and another library has the ground rails on the bottom. The option ensures that ground rail positions are consistent among all reference libraries linked in a design library.

- Auto

When this option is set to Auto (the default), the tool finds the correct rail positions for reference libraries with consistent rail positions (either bottom or top).

- Top

When you select Top for this option, the ground rails are placed on top of the cells.

- Bottom

When you select Bottom, the ground rails are placed on the bottom.

1st Layer Parallel To P/G Rail

Select the first metal layer that you want to be parallel to the power and ground rail.

1st Layer Perpendicular To P/G Rail

Select the first metal layer that you want to be perpendicular to the power and ground rail.

auStampToDB

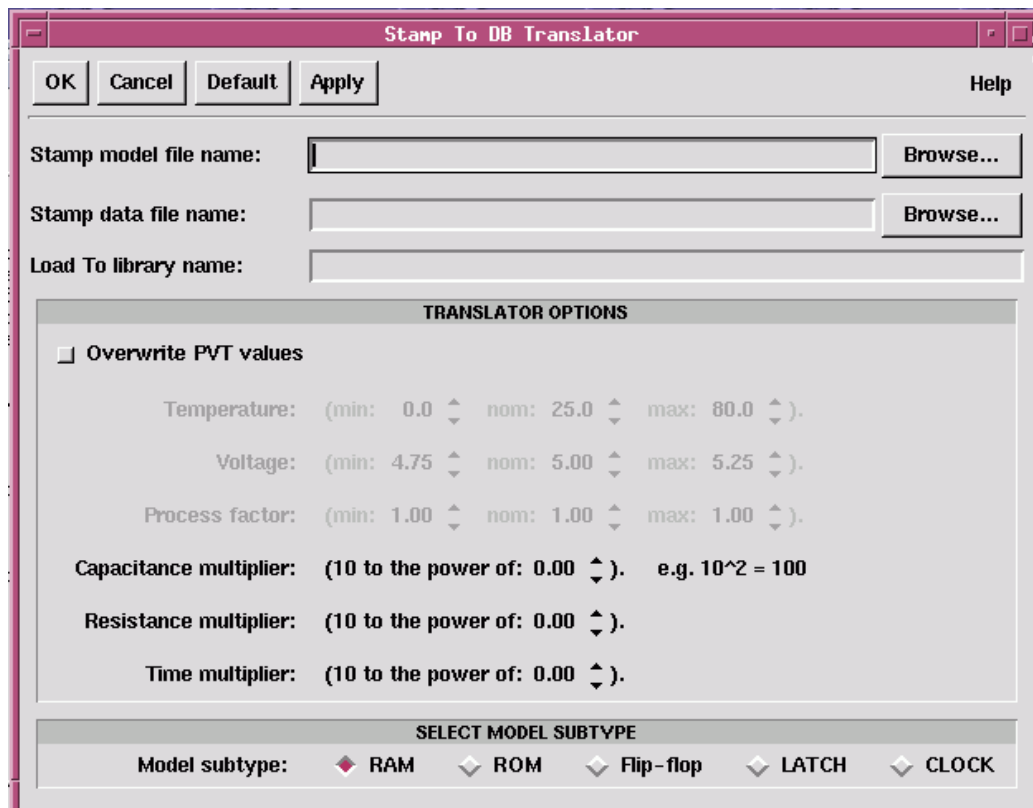
Reads a Stamp model into the Milkyway database and generates a TIM view.

Tool

Tool based on Milkyway

Dialog Box

The menu command opens the following dialog box. Its options are described below.



Stamp model file name

Specifies the name of the Stamp model file to be translated.

Stamp data file name

Specifies the name of the Stamp data file to be translated.

Load To library name

Specifies the library into which the Stamp file is to be loaded.

Overwrite PVT values

If selected, the process, voltage, and temperature (PVT) values indicated in the dialog box are used. If not selected (the default), the Stamp file's PVT values are used.

Temperature

Specifies the minimum, nominal, and maximum temperatures for the operating conditions under which to operate the chip.

Voltage

Specifies the minimum, nominal, and maximum voltages for the operating conditions under which to operate the chip.

Process factor

Specifies the minimum, nominal, and maximum process factors for the operating conditions under which to operate the chip.

Capacitance multiplier

Specifies the power of 10 by which to multiply the Stamp load values to get the actual capacitance values needed by the Milkyway application. If the value is 2, $10^2 = 100$ and all load values are multiplied by 100.

Resistance multiplier

Specifies the power of 10 by which to multiply the Stamp load values to get the actual resistance values needed by the Milkyway application. If the value is 2, $10^2 = 100$ and all load values are multiplied by 100.

Time multiplier

Specifies the power of 10 by which to multiply the Stamp load values to get the actual time values needed by the Milkyway application. If the value is 2, $10^2 = 100$ and all load values are multiplied by 100.

Model subtype

Specifies the logic model subtype of a cell. The default is RAM. See the `dbSetLModelSubType` command.

Description

The `auStampToDB` command reads a Stamp model into the Milkyway database and generates a TIM view. Milkyway calculates the minimum and maximum scaled delays by using the following:

- The k-factors specified in the Stamp data file
- The temperature, voltage, and process factor values (minimum, nominal, and maximum) specified in the Stamp To DB Translator dialog box
- The minimum and maximum scaled delays formulas:

$$D_{MinScaled} = D * (1 + (T_{min} - T_{nom}) KT) * (1 + (V_{max} - V_{nom}) KV) * (1 + (P_{min} - P_{nom}) KP)$$

$$D_{MaxScaled} = D * (1 + (T_{max} - T_{nom}) KT) * (1 + (V_{min} - V_{nom}) KV) * (1 + (P_{max} - P_{nom}) KP)$$

where the variables are described in the following table:

Variable	Description
DMinScaled	The computed minimum scaled delay for the operating condition under which to operate the chip.
DMaxScaled	The computed maximum scaled delay for the operating condition under which to operate the chip.
Tmin	The minimum temperature for the operating condition under which to operate the chip, as specified in the Stamp To DB Translator dialog box.
Tmax	The maximum temperature for the operating condition under which to operate the chip, as specified in the Stamp To DB Translator dialog box.
Tnom	The nominal temperature for the operating condition under which to operate the chip, as specified in the Stamp To DB Translator dialog box.

Variable	Description
KT	The k-factor for temperature, as specified by the k_temp value in the Stamp data file.
Vmin	The minimum voltage for the operating condition under which to operate the chip, as specified in the Stamp To DB Translator dialog box.
Vmax	The maximum voltage for the operating condition under which to operate the chip, as specified in the Stamp To DB Translator dialog box.
Vnom	The nominal voltage for the operating condition under which to operate the chip, as specified in the Stamp To DB Translator dialog box.
KV	The k-factor for voltage, as specified by the k_volt value in the Stamp data file.
Pmin	The minimum process factor for the operating condition under which to operate the chip, as specified in the Stamp To DB Translator dialog box.
Pmax	The maximum process factor for the operating condition under which to operate the chip, as specified in the Stamp To DB Translator dialog box.
Pnom	The nominal process factor for the operating condition under which to operate the chip, as specified in the Stamp To DB Translator dialog box.
KP	The k-factor for process, as specified by the k_process value in the Stamp data file.

auStampToCLF

Translates a Stamp model into Synopsys's CLF format.

After you translate the Stamp file, load the CLF information file using [auLoadCLF](#).

Milkyway determines the minimum and maximum scaled delays using the following:

- The k-factors you have specified in the Stamp data file.
- The temperature, voltage, and process factor values (minimum, nominal and maximum) that you specified on the Stamp To CLF Translator dialog box.
- Milkyway calculates the minimum and maximum scaled delays using the following formula:

$$\begin{aligned} D_{\text{MinScaled}} &= D * (1 + (T_{\text{min}} - T_{\text{nom}})KT) * \\ &\quad (1 + (V_{\text{max}} - V_{\text{nom}})KV) * (1 + (P_{\text{min}} - P_{\text{nom}})KP) \\ D_{\text{MaxScaled}} &= D * (1 + (T_{\text{max}} - T_{\text{nom}})KT) * \\ &\quad (1 + (V_{\text{min}} - V_{\text{nom}})KV) * (1 + (P_{\text{max}} - P_{\text{nom}})KP) \end{aligned}$$

where the variables are as follows:

D MinScaled

The computed minimum scaled delay for the operating conditions under which you want the chip to operate.

D MaxScaled

The computed maximum scaled delay for the operating conditions under which you want the chip to operate.

T min

The minimum temperature for the operating conditions under which you want the chip to operate (specified by the Min Temperature option on the Stamp To CLF Translator dialog box).

T max

The maximum temperature for the operating conditions under which you want the chip to operate (specified by the Max Temperature option on the Stamp To CLF Translator dialog box).

T nom

The nominal temperature for the operating conditions under which you want the chip to operate (specified by the Nom Temperature option on the Stamp To CLF Translator dialog box).

K T

The k-factor for temperature (specified by the k_temp value in the k-factor file).

V min

The minimum voltage for the operating conditions under which you want the chip to operate (specified by the Min Voltage option on the Stamp To CLF Translator dialog box).

V max

The maximum voltage for the operating conditions under which you want the chip to operate (specified by the Max Voltage option on the Stamp To CLF Translator dialog box).

V nom

The nominal voltage for the operating conditions under which you want the chip to operate (specified by the Nom Voltage option on the Stamp To CLF Translator dialog box).

K V

The k-factor for voltage (specified by the k_volt value in the K-Factor file).

P min

The minimum process factor for the operating conditions under which you want the chip to operate (specified by the Max Process Factor option on the Stamp To CLF Translator dialog box).

P max

The maximum process factor for the operating conditions under which you want the chip to operate (specified by the Max Process Factor option on the Stamp To CLF Translator dialog box).

P nom

The nominal process factor for the operating conditions under which you want the chip to operate (specified by the Nom Process Factor option on the Stamp To CLF Translator dialog box).

K P

The k-factor for process (specified by the k_process value in the k-factor file).

For example, the Synopsys tool calculates the minimum scaled delay for intrinsic_rise using the formula:

$$\text{MinScaledDelayintrinsic_rise} = \text{Delayintrinsic_rise} * \\ (1 + (\text{Tmin} - \text{Tnom})k_temp_intrinsic_rise) * \\ (1 + (\text{Vmax} - \text{Vnom})k_volt_intrinsic_rise) * \\ (1 + (\text{Pmin} - \text{Pnom})k_process_intrinsic_rise)$$

Tools and Menu Commands

Milkyway: CLF > Stamp To CLF

Prerequisites

Prepare a set of data and model STAMP files.

Stamp To CLF Translator Dialog Box

The menu command opens the following dialog box. Its options are described below.

Stamp To CLF Translator

OK Cancel Default Apply Help

INPUT AND OUTPUTS

Stamp model file name: (↓)

Stamp data file name: (↓)

Output CLF file name: (↓)

TRANSLATOR OPTIONS

☐ Overwrite PVT values

Temperature: (min: 0.0 ↓ nom: 25.0 ↓ max: 80.0 ↓).

Voltage: (min: 4.75 ↓ nom: 5.00 ↓ max: 5.25 ↓).

Process factor: (min: 1.00 ↓ nom: 1.00 ↓ max: 1.00 ↓).

Capacitance multiplier: (10 to the power of: 0.0 ↓). e.g. 10² = 100

Resistance multiplier: (10 to the power of: 0.00 ↓).

Time multiplier: (10 to the power of: 0.00 ↓).

SELECT MODEL SUBTYPE

Model subtype: ♦ RAM ▼ ROM ▼ Flip-flop ▼ LATCH ▼ CLOCK

Input and Outputs

Stamp model file name

Type the name of the Stamp model file you want to translate.

Stamp data file name

Type the name of the Stamp data file you want to translate.

Output CLF file name

Type the name you want to assign to the new CLF file.

Translator Options

The following options allow you to adjust the operating condition values under those you want the chip to operate on.

Overwrite PVT values

Click this button on when you want to use the PVT values on the dialog box . By default, this option is off so that the PVT values in the stamp file are used. When off, the input fields for specifying the PVT values will be grayed out.

Temperature: min

Type the minimum temperature for the operating conditions under which you want the chip to operate. You can also use the “up” or “down” arrow buttons to increase or decrease this value.

Nom Temperature: nom

Type the nominal temperature for the operating conditions under which you want the chip to operate. You can also use the “up” or “down” arrow buttons to increase or decrease this value.

Temperature: max

Type the maximum temperature for the operating conditions under which you want the chip to operate. You can also use the “up” or “down” arrow buttons to increase or decrease this value.

Voltage: min

Type the minimum voltage for the operating conditions under which you want the chip to operate. You can also use the “up” or “down” arrow buttons to increase or decrease the value.

Voltage: nom

Type the nominal voltage for the operating conditions under which you want the chip to operate. You can also use the “up” or “down” arrow buttons to increase or decrease the value.

Voltage: max

Type the maximum voltage for the operating conditions under which you want the chip to operate. You can also use the “up” or “down” arrow buttons to increase or decrease the value.

Process Factor: min

Type the minimum process factor for the operating conditions under which you want the chip to operate. You can also use the “up” or “down” arrow buttons to increase or decrease the value.

Process Factor: nom

Type the nominal process factor for the operating conditions under which you want the chip to operate. You can also use the “up” or “down” arrow buttons to increase or decrease the value.

Process Factor: max

Type the maximum process factor for the operating conditions under which you want the chip to operate. You can also use the “up” or “down” arrow buttons to increase or decrease the value.

Capacitance Multiplier

Type the number by which Milkyway should multiply the load values specified in the Stamp file to get the actual capacitance values needed. The values are multiplied by the power of 10 (e.g., $10^2 = 100$). You can also use the “up” or “down” arrow buttons to increase or decrease the value.

Resistance Multiplier

Type the number by which Milkyway should multiply the load values specified in the Stamp file to get the actual resistance values needed. The values are multiplied by the power of 10 (e.g., $10^2 = 100$). You can also use the “up” or “down” arrow buttons to increase or decrease the value.

Time Multiplier

Type the number by which Milkyway should multiply the load values specified in the Stamp file to get the actual time values needed. The values are multiplied by the power of 10 (e.g., $10^2 = 100$). You can also use the “up” or “down” arrow buttons to increase or decrease the value.

Model subtype

You can use this option to specify the logic model subtype of a cell within the STAMP file. Default is RAM, see the `dbSetLModelSubType` command.

auStreamIn

Translates a design from GDSII Stream format to Synopsys format, creating layout cells for all the cells in the design. For more information, see “About Layer Mapping Files” in Chapter 3 of the *Milkyway Environment Data Preparation User Guide*.

The `auStreamIn` command is order dependent. If two reference libraries contain two different cells with the same name, the cell that is read in first is used.

If the referenced cells in the top level library are present, the `auStreamIn` command automatically searches for cells in the reference libraries and binds the physical information to the top level cell in question.

Tools and Menu Commands

Milkyway: Cell Library > Stream In

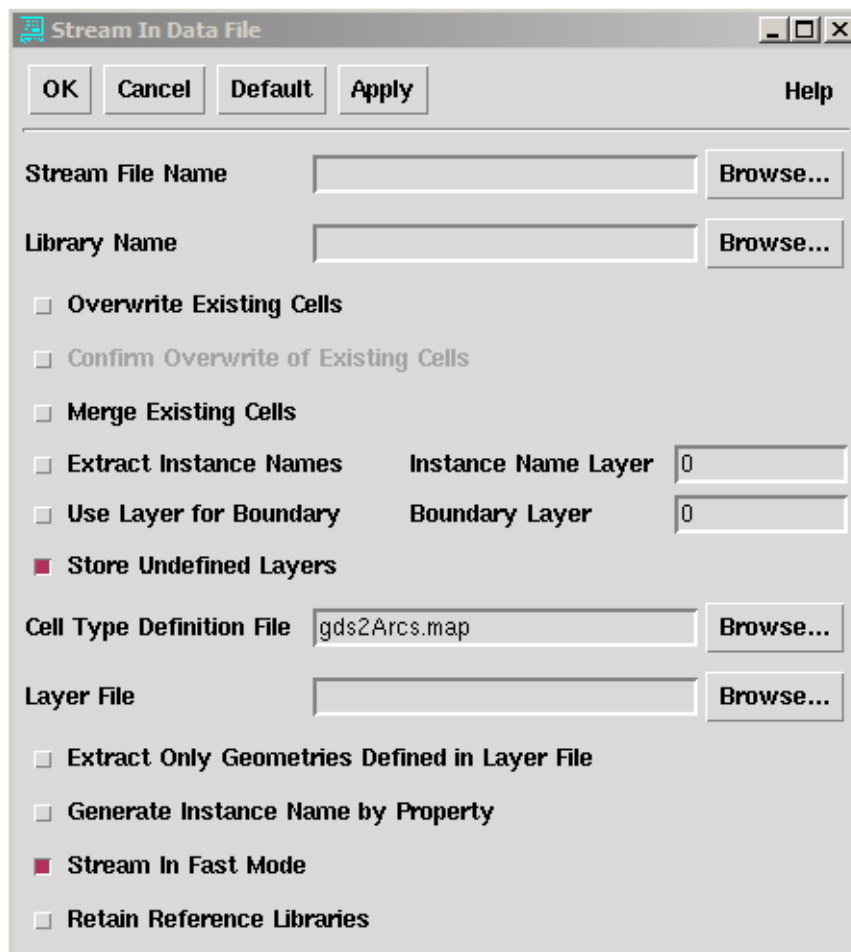
Prerequisites

Create a cell definition type file that lists cells in the layout according to type.

If you want to use a special layer mapping Scheme, you also need to create a layer file.

Stream In Data File Dialog Box

The menu command opens the following dialog box. Its options are described below.



Stream File Name

Type the name of the GDSII Stream file containing the layout definition.

Library Name

Type the name of the library in which you want to place the layout cells. Use the Browse button to help locate library files.

Browse

Select to invoke the Browse Library dialog box.

Overwrite Existing Cells

Select to overwrite any cells already in the library with the same name as cells being imported.

Confirm Overwrite of Existing Cells

Select if you want Milkyway to ask for confirmation before it overwrites a cell.

Merge Existing Cells

Select if you want to merge with existing cells.

Extract Instance Names

Select to extract the names of cell instances. If you select this option, you also need to type the *Instance Name Layer* used for the GDSII Stream instance names.

Instance Name Layer

If you selected *Extract Instance Names*, type the number of the Synopsys layer used for GDSII Stream instance names (after Milkyway maps the GDSII Stream layers to Synopsys layers, as specified by the *Layer File*).

Use Layer for Boundary

Select if you want your Synopsys tool to use the cell boundaries in the GDSII layout when placing cells in cell rows. If you select this option, you also need to type the *Boundary Layer* used for the GDSII Stream boundary.

Boundary Layer

If you selected *Use Layer for Boundary*, type the number of the Synopsys layer used for GDSII Stream boundaries (after Milkyway maps the GDSII Stream layers to Synopsys layers, as specified by the *Layer File*).

Store Undefined Layers

Select to import and store layers not defined in the technology file. Otherwise, GDSII Stream data on layers not defined in the technology file (after Milkyway maps the GDSII Stream layers to Synopsys layers, as specified by the *Layer File*) are not imported.

Cell Type Definition File

Type the name of the file you created that lists the cells by type. For information about creating a cell type definition file, see “Specifying Cell Types” on page 3-2 of the *Milkyway Data Preparation User Guide*.

Layer File

Type the name of the file containing special layer mapping instructions, if any. For information about creating a layer file, see “Specifying Layer Mapping” on page 3-4 of the *Milkyway Data Preparation User Guide*.

Extract Only Geometries Defined in Layer File

Select if want to extract only objects that are defined in the layer mapping file.

Generate Instance Name by Property

Select to generate the cell instance names according to the PROPERTY in the GDSII file. By default, this option is not selected.

Layer Mapping File Format for GDSII to Milkyway

```
MilkywayLayer[:MilkywayDataType]  
  GDSIILayer[:GDSIIDataType]
```

You can add comments to the file by preceding them with a semicolon (;). Milkyway ignores all text following a semicolon on the same line.

Where the variables are described in the following table.

Table 14-20 Variables for Mapping GDSII layers for Stream In

MilkywayLayer	Number of the Milkyway layer to which you want the GDSII Layer translated Valid values: Number of any layer defined in the technology file Note: For mapping blockage layers, use the following predefined Milkyway layer numbers: 212 for polyBlockage 216 for metal4Blockage 217 for via3Blockage 218 for metal1Blockage 219 for metal2Blockage 220 for metal3Blockage 223 for polyContBlockage 224 for via1Blockage 225 for via2Blockage
MilkywayDataType	Data type of the Milkyway data to be translated. Valid values: Any integer from 0 to 255
GDSIILayer	Number of the GDSII Stream layer to be translated Valid value: Any integer from 0 to 255
GDSIIDataType	Data type of the GDSII Stream data to be translated. Valid values: Any integer from 0 to 255

auStreamInCellBndryIgnoreText

Sets the auStreamIn command's "Cell Boundary Ignore Text" mode.

Syntax

```
auStreamInCellBndryIgnoreText <boolean mode>
```

where the argument is as follows:

mode

#t

Specifies to ignore the text bounding boxes when calculating cell boundaries.

#f

Specifies not to ignore the text bounding boxes when calculating cell boundaries (default).

Description

This mode controls whether or not each cell boundary is created by including the cell's text objects. By default, all text objects are included within the cell boundaries.

Example

```
auStreamInCellBndryIgnoreText #t
```

auStreamInDuplicateLayerAsBoundary

Forces Milkyway to copy the layer specified on the dialog box to a boundary instead of moving it. Use this command before streaming in.

Tools and Menu Commands

Milkyway: Cell Library > Stream In

Syntax

```
auStreamInDuplicateLayerAsBoundary duplicate?
```

where the argument is as follows:

<i>duplicate?</i>	Whether or not to duplicate the layer. Valid values: #t means to duplicate the layer. #f means to move the layer instead of duplicating it.
-------------------	--

Examples

```
auStreamInDuplicateLayerAsBoundary #t
```

See Also

[auStreamIn](#)

auStreamInRectilinearBoundary

This switch enables [auStreamIn](#) to input a rectilinear boundary and preserve its shape instead of converting it to a bounding box.

Tools and Menu Commands

Milkyway: Cell Library > Stream In

Syntax

```
auStreamInRectilinearBoundary rectBndry?
```

where the argument is as follows:

<i>rectBndry?</i>	Indicates whether or not to preserve the original shape of a rectilinear boundary. Valid values: • #t means enable the shape of rectilinear boundary. • #f means to convert a rectilinear boundary into a bounding box.
-------------------	--

Examples

```
auStreamInRectilinearBoundary #t
```

See Also

[auStreamIn](#)

auStreamInSetCellBoundaryDataType

This switch enables [auStreamIn](#) to set the data type of a cell boundary. Use this feature only when the “Use Layer for Boundary” option is turned on.

Tools and Menu Commands

Milkyway: Cell Library > Stream In

Syntax

```
auStreamInSetCellBoundaryDataType dataType
```

where the argument is as follows:

<i>dataType</i>	An integer number that is the data type.
-----------------	--

Examples

```
auStreamInSetCellBoundaryDataType 2
```

See Also

[auStreamIn](#)

auStreamInTextConversionFactor

This switch enables [auStreamIn](#) to control the text size.

Tools and Menu Commands

Milkyway: Cell Library > Stream In

Syntax

```
auStreamInTextConversionFactor convFactor
```

where the argument is as follows:

<i>convFactor</i>	A double-precision real number that is the magnification factor.
-------------------	--

Examples

```
auStreamInTextConversionFactor 2.0
```

See Also

[auStreamIn](#)

auStreamOut

Translates a design from Synopsys format to GDSII Stream format (version 5).

Note:

To specify options for labeling nets and pins, click Pin/Net Options on the Stream Out Data File dialog box.

If a design uses hierarchical ref. library tree structure, you must always run the `dbSetHierRefLibMode #t Scheme` command before you run the `auStreamOut` command or else the cells in reference libraries of reference libraries will not be seen from the top level.

Tools and Menu Commands

Milkyway: Output > Stream Out

Prerequisites

If you want to use a special layer mapping Scheme, create a layer file.

If you want to fill gaps or notches during the translation, generate gap/notch filling information using `geNewShortLocator`. You should not edit the cell after you generate this information.

Layer Mapping File Format for Milkyway to GDSII Stream

```
MilkywayObjType[MilkywayNetType]
  MilkywayLayer[:MilkywayDataType] GDSIILayer
    GDSIIDataType
```

Where the variables are shown in the following table.

You can add comments to the file by preceding them with a semicolon (;). Milkyway ignores all text following a semicolon on the same line.

Table 14-21 Syntax Variables for GDSII Stream Out

MilkywayObjType	Names the code for the type of object in the data to be translated Valid types Use A for all types Use T for text Use D for data
MilkywayNetType	Names the code for the net type of the object in the data to be translated Valid Net types: S for signal P for power G for ground C for clock U for Up conduction layer (upper layer in a via). Example: metal1 in an m1/m2 via D is for Down conduction layer (lower layer in a via). Example: metal2 in a nm1/m2 via X for power/ground wires or contacts with "signal" or "tie-off" routing type

Table 14-21 Syntax Variables for GDSII Stream Out (Continued)

Note:

U and D are used only for via and smashed via objects and override any other net type set for the layer when the layer is in a via.

If MilkywayNetType is omitted, Milkyway will map all Milkyway data of the specified object type to the specified GDSII layer.

If MilkywayNetType is included, Milkyway will examine every object of the specified object type to determine its net type, and maps it to the layer you specify. If an object has no net (or a net has no type), Milkyway assumes that it is a signal net type

If a layer file contains contradictory lines, the last line will overwrite any previous line.

For example, if an earlier line of a layer file specifies mapping for a particular object/net type, and a later line specifies mapping for the same object type but no net type, Milkyway will translate all data of the specified object type as specified by the later line.

MilkywayLayer	Specifies the name or number of any layer, between 0 and 225, defined in the technology file to be translated
MilkywayDataType	Data type of the Milkyway data to be translated. Valid values: Any integer from 0 to 255
GDSIILayer	Specifies the number of the GDSII Stream layer to which you want objects of MilkywayObjType on MilkywayLayer to be translated. Use any integer from 0 to 255, or use (-) to omit the Milkyway layer during translate
GDSIIDataType	Specifies the number of the GDSII Stream data type to which you want objects of MilkywayObjType (and MilkywayNetType, if given) on Milkyway Layer to be translated. Use any integer from 0 to 255

If you want Milkyway to translate any layers differently, you must specify mapping for those layers in a layer file. Each line in the layer file specifies a particular type of object (text, data, or both) on a specified Milkyway layer that you want translated to a particular GDSII Stream layer with a given data type. You can also omit a Milkyway layer during translation.

Milkyway outputs net and pin text on the same layer as the associated net or pin. For example, if Milkyway translates a pin on layer 5, the pin's text is also on layer 5.

If you specify a layer file, Milkyway maps the text layer according to the specification in the layer file. For example, if Milkyway translates a pin on layer 5 using a layer file mapping Milkyway layer 5 to GDSII Stream layer 10, the text is on layer 10 instead. Specifying a minus (-) omits transfer of all text and data on the Milkyway layer. See examples below for more detail.

Example Mapping of Milkyway Layers to GDSII Stream

```
T METAL:2 3 5      ; converts text on Milkyway layer
                   ; METAL data Type 2 to GDSII Stream
                   ; layer #3 data type 5.
                   ; Note: If you stream back into
                   ; Milkyway database without using
                   ; the layer map file. This will
                   ; switch your text on layer METAL
                   ; to layer METAL5.

TS 16 31 6         ; converts text associated with
                   ; signal nets on Milkyway layer #16
                   ; to GDSII Stream layer #31 and
                   ; data type 6

TP METAL3 16       ; converts text associated with
                   ; power on Milkyway layer METAL3
                   ; to GDSII Stream layer #16

TG 28 16           ; converts text associated with
                   ; ground on Milkyway layer #28
```

```

D METAL2 45      ; to GDSII Stream layer #16
                  ; converts data on Milkyway layer
                  ; METAL2 to GDSII Stream layer #45

DS 45:2 18 5     ; converts data associated with
                  ; signal nets on Milkyway layer #45
                  ; data Type 2 to GDSII Stream layer #18
                  ; data Type 5

A METAL6 3 4     ; converts text and data on
                  ; Milkyway Layer METAL6 to GDSII
                  ; Stream layer #3 and datatype 4

A METAL4 -       ; omits to transfer all text and
                  ; data on Milkyway layer METAL4

```

When streaming out of Milkyway, the first character of the map file is the code for the type of object to be translated. Use A for all, T for text, and D for data. The optional second character specifies the net type. For example, S for signal, P for power, and G for ground. More information on the net types can be found in [Table 14-21](#).

Objects in Milkyway That Can be Streamed Out

Only pure physical information will be streaming out from Milkyway database to the GDSII Stream. Because the GDSII Stream format contains only layout information, any connectivity information or place and route (abstract) objects such as port, net, port instance, region, and cell row cannot be streamed out.

Extension type objects such as group, property, and attached file also cannot be streamed out. [Table 14-22](#) lists Milkyway objects for Stream Out.

Table 14-22 *Stream Out Objects in Milkyway*

Milkyway Objects	GDSII Stream Objects
CellInstance	SREF
CellInstArray	AREF

Table 14-22 Stream Out Objects in Milkyway (Continued)

Milkyway Objects	GDSII Stream Objects
Contact	SREF/Boundary
ContactArray	SREF/Boundary
HorizontalWire	Path
Path	Path
Pin	Boundary
Polygon	Boundary
Rectangle	Boundary
Text	Text
VerticalWire	Path

C-API Application Considerations

The previous section covers limitation on which Milkyway objects can be streamed out to GDSII Stream. However, using the Milkyway C-API, there is no limit to what you can convert directly from a Milkyway database to a third party database. You can use C-API functions to create text from abstract objects such as port, net, and port instance.

For example, you create text objects with the origin located on a wire object region with the text name extracted by a C-API function from the net name of the wire object. The text objects can be streamed out. By this method, the net name information for the wire objects can be preserved and passed out to the stream file.

Stream Out Data File Dialog Box

The menu command opens the following dialog box. Its options are described below.

Stream Out Data File

OK Cancel Default Pin/Net Options Design Intent Help

Stream File Name Browse...

Library Name Browse...

Layer File Browse...

Cell Rename File Browse...

Child Extraction Depth

Convert ☒ All .CEL ☐ Selected Cells ☐ Specified Cell

Cell Name

☒ Convert Reference Lib Child Cells

Text Conversion Factor

Text Width

Flatten ☐ Devices ☐ Device Arrays

Device Name Prefix Notch ☐ Gap ☐ FILL

☒ Force output outdated Fill Cell

☐ Merge Additional Views

View Name

☐ Generate Instance Name As Prop

Instance Property Number

☐ Generate Geometry Property

☐ Strip Backslash from Instance/Net Names

☐ Skip Layer/DataType Mapping for GDSII Imported Cells

Stream File Name

Type the name you want to assign to the output file.

Library Name

Type the name of the library containing the layout cells. Use the Browse button to help locate library names.

Browse

Select to invoke the the Browse Library dialog box.

Layer File

Type the name of the file containing special layer mapping instructions, if any.

Cell Rename File

Specifies a mapping file used to rename a cell to a new cell name when outputting to GDSII format. Note that only master cell names can be renamed. The format of the mapping file is

```
Milkyway_cell_name GDSII_cell_name
```

An example entry for a mapping file would be as follows:

```
my_ADDER      add_1
whos_ADDER     add_2
my_NOR         nor_2
```

In this example, my_ADDER is the master cell name of an adder in the library that is renamed add_1 and written out in GDSII format. You can check the log file for the details of the renaming operation. The log file reports the following:

```
Outputting Cell add_1.CEL (Renamed from ADDFX1.CEL)
Outputting Cell add_2.CEL (Renamed from ADDHXL.CEL)
Outputting Cell nor_2.CEL (Renamed from NOR2BX1.CEL)
```

Child Extraction Depth

Type the level to which you want to convert child cells. If you are exporting the design to perform a DRC check, type a large number (20) to ensure extraction of all child cells. Milkyway exports the *.cel* version of each child cell.

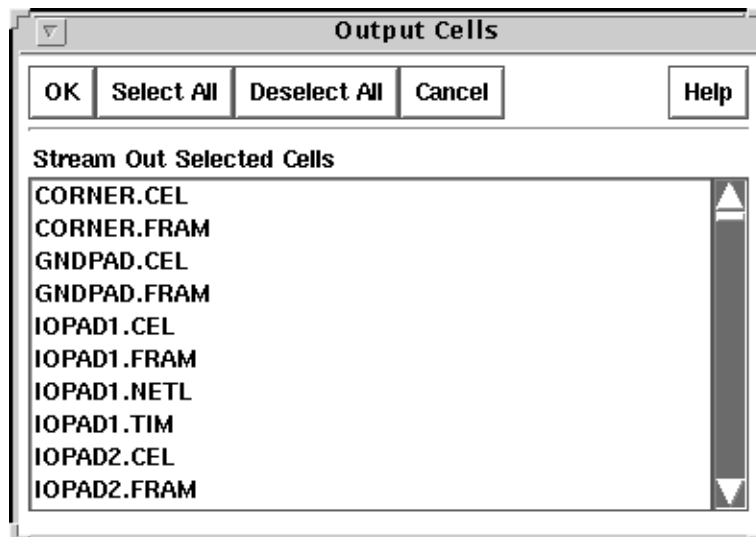
Convert

Indicate the cells you want to translate.

Option	Translates ...
All.CEL	all layout cells in the specified library.
Selected Cells	only the cells you specify. (See note below.)
Specified Cell	a cell you specify by typing its <i>Cell Name</i> .

Note:

When you click OK with the Selected Cells option selected, Milkyway displays a selection window.



- *Stream Out Selected Cells* lets you indicate the cells you want to translate by selecting them from a list of all cells in the library and clicking *OK*.
- *Selection* lets you indicate the cells you want to translate by typing the name (or pattern) of the cells and clicking *Select*, then *OK*.

You can use pattern matching to select all cells with names that match a pattern. For example, typing *TOP.** and clicking *Select* selects all cells with names that begin with *TOP*.

Cell Name

Type the name of the cell you want to translate (if you selected *Specified Cell* from the *Convert* options).

Convert Reference Lib Child Cells

Select to convert child cells in reference libraries. Milkyway converts child cells in reference libraries only if the *Child Extraction Depth* is sufficient to convert cells at that level.

Text Conversion Factor

Type the number you want to use to convert the text height to text magnification. Milkyway calculates the text magnification for text in the database as:

$$textHeight \times dbuPerUU \times txtConvFactor$$

where the variables are as follows:

- *textHeight* is the text height in Milkyway.
- *dbuPerUU* is the number of database units per user unit, as specified in your technology file.
- *txtConvFactor* is the text conversion factor you specified on the dialog box.

Text Width

Type the width you want for text. Milkyway calculates the width for text output to GDSII Stream as:

$$\text{textWidth} \times \text{dbuPerUU}$$

where the variables are as follows:

- *textWidth* is the text width you specified on the dialog box.
- *dbuPerUU* is the number of database units per user unit, as specified in your technology file.

Flatten

Select the objects you want to break down into component parts.

- If you deselect *Devices*, Milkyway translates contacts as references to the technology file.
- If you deselect *Device Arrays*, Milkyway translates contact arrays as cells with the names:

$$\text{prefixPlusContactName_xViaPitch_yViaPitch_xReps_yReps}$$

where the variables are as follows:

- *prefixPlusContactName* is the *Device Name Prefix* you specified on the dialog box plus the name assigned to the contact in the deviceTable section of your technology file. For example, if the contact name is *V1* and you specified a *Device Name Prefix* of *\$\$*, the *prefixPlusContactName* is *\$\$V1*.
- *xViaPitch* is the horizontal distance (in database units) between centers of contacts in the array.
- *yViaPitch* is the vertical distance (in database units) between centers of contacts in the array.

- *xReps* is the number of columns (contacts in the horizontal direction) in the array.
- *yReps* is the number of rows (contacts in the vertical direction) in the array.

For example, the name `$$V1_2000_3000_4_4` indicates a four-by-four array of V1 contacts, spaced with 2000 database units from center to center in the horizontal direction, and 3000 database units from center to center in the vertical direction.

Device Name Prefix

Type the prefix you want to add to the contact names. If you use the default prefix (`$$`), Milkyway translates each contact as:

`$$deviceName`

where *deviceName* is the name assigned to the contact in the deviceTable section of your technology file. For example, a contact named *via1* translates to the GDSII Stream file as `$$via1`.

Fill

Select the filling options you want to use in the translation.

Table 14-23

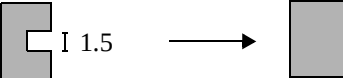
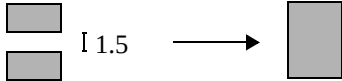
Option	Fills ...
Notch	notches that are smaller than the minimum distance required between objects on the same layer. For example, if the minimum distance required between two objects on the <i>metal3</i> layer is 1.6, selecting the <i>Notch</i> option fills the notch shown below:
	

Table 14-23

Option	Fills ...
Gap	gaps that are smaller than the minimum distance required between objects on the same layer. For example, if the minimum distance required between two objects on the <i>metal3</i> layer is 1.6, selecting <i>Gap</i> fills the gap shown below: 
FILL	notches and gaps that are stored in the .FILL cell are generated by geNewFilling. Filling patterns include both notch and gap.

Note:

Options “Notch” and “Gap” are designed for .NOTC and .GAP cells. If you select either the Notch or Gap option, you need to have generated gap/notch filling information using geNewShortLocator. Milkyway fills notches and gaps for selected cells only (not for child cells of selected cells).

Option “FILL” is designed for .FILL cells generated by geNewFilling.

Milkyway fills notches and gaps for selected cells only (not for child cells of selected cells).

View Name

When you have selected the option, Merge Additional Views, you can type the names of the additional views you want merged into the .CEL view.

Generate Instance Name As Prop

Select if you want Milkyway to attach a property specifying the instance name to each cell instance in the design.

Note:

Milkyway generates instance name properties for selected cells only (not for child cells of selected cells).

Instance Property Number

Type a number in the range of 0-127 to indicate the property number of the instance name property (if you select *Generate Instance Name As Prop*).

Generate Geometry Property

Select to attach a property to each geometry object in the design cell.

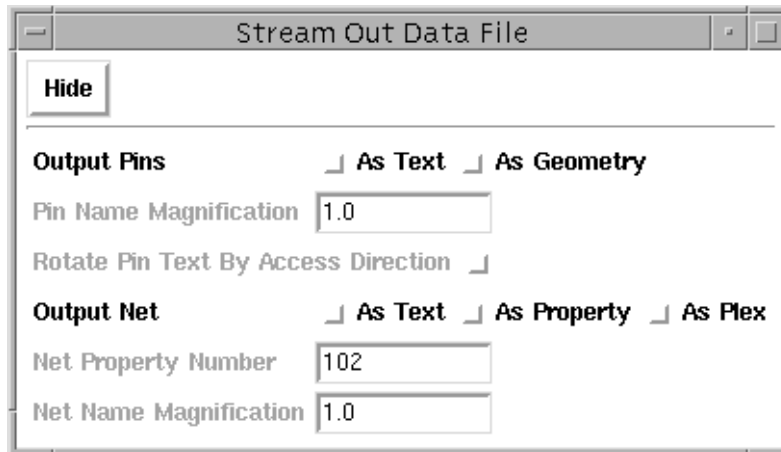
Strip Backslash from Instance/Net Names

Select to remove backslashes from hierarchical cell instance and net names which might conflict with hierarchical names.

Skip Layer/DataType Mapping for GDSII Imported Cells

Select to ignore Layer/DataType mapping for those cells created from GDSII file during *auStreamIn* process. The layer number and data type will be written exactly to GDSII file without any conversion.

Stream Out Data File Dialog Box (Pin/Net Options)



Note:

Milkyway outputs net and pin text on the same layer as the associated net or pin. For example, if Milkyway translates a pin on layer 5, the text is on layer 5. If you specify a Layer File, Milkyway maps the text layer according to the specification in the

layer file. For example, if Milkyway translates a pin on layer 5 using a layer file that maps Synopsys layer 5 to GDSII Stream layer 10, the text is on layer 10.

Output Pins

Select the options you want to use for translating pins.

Table 14-24 Options for Translating Pins

Option	Description
As Text	Translates text associated with pins. If you select this option, you must also: • Type the <i>Pin Name Magnification</i> you want for pin names. • Select <i>Rotate Pin Text By Access Directions</i> to rotate pin names for pins with top or bottom (but no left or right) access.
As Geometry	Translates geometry associated with pins.

Note:
Milkyway outputs pins for selected cells only (not for child cells of selected cells).

Pin Name Magnification

Type the magnification you want Milkyway to use for pin names (if you select *Output Pins As Text*).

Rotate Pin Text By Access Direction

Select if you want Milkyway to rotate pin names for pins with top or bottom but no left or right access (if you select *Output Pins As Text*).

Output Net

Select the option(s) you want to use for translating nets.

Table 14-25

Option	Translates nets as follows ...
As Text	generates one text string per net and places the text over an arbitrary wire associated with that net. If you select this option, then type a <i>Net Name Magnification</i> .
As Property	generates a property for each object associated with a net. If you select this option, then type a property number between 0-127 in the <i>Net Property Number</i> entry field. The property is equal to the net name, unless the name is greater than 126 characters. If a net name exceeds 126 characters, Milkyway displays a message and does not generate a property for that net. In the case of contacts, Milkyway attaches the property to the contact SREF (if not flattened) and the individual geometries (if flattened).
As Plex (automatical ly selects As Text)	assigns a plex number to each net in the cell and generates a text record with a number equal to the plex number and a value equal to the net name. Milkyway associates the plex number of each net with each object on the net. If a net name exceeds 255 characters, Milkyway flags an error and does not generate a text record. Selecting this option automatically selects <i>As Text</i> .

Note:

The most efficient method for passing net information is to select As Plex, which generates GDSII Stream files much smaller than those generated using the As Property option. If you select all three Output Net options, Milkyway generates text records with properties equal to the net names.

Milkyway outputs nets for selected cells only (not for child cells of selected cells).

Net Property Number

Type a number in the range of 0-127 to indicate the property number of the net name property (if you select *Output Net As Property*).

Net Name Magnification

Type the magnification you want Milkyway to use for net names (if you select *Output Net As Text*).

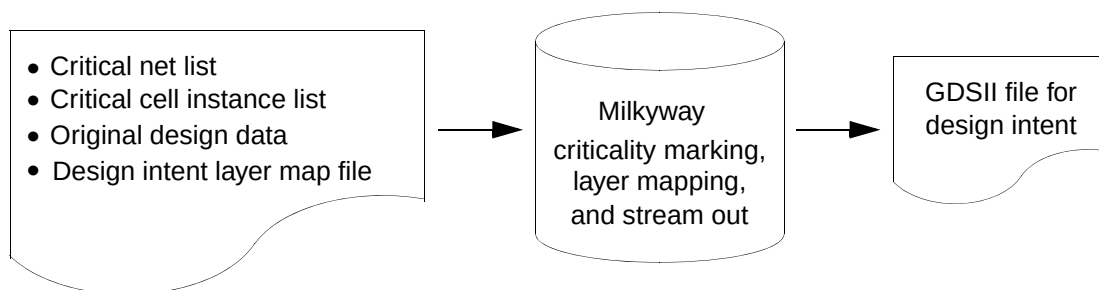
Design Intent

Design intent data are extra geometries on top of the regular GDSII design data. These geometries are used to stream out data for critical nets and cell instances in different layer data type pairs. Each extra layer data type pair is associated with one or more original design layers.

One of the main uses for design intent is to focus optical proximity correction (OPC) only on critical nets and instances in order to reduce design data processing.

Figure 14-1 shows an overview of the design intent flow.

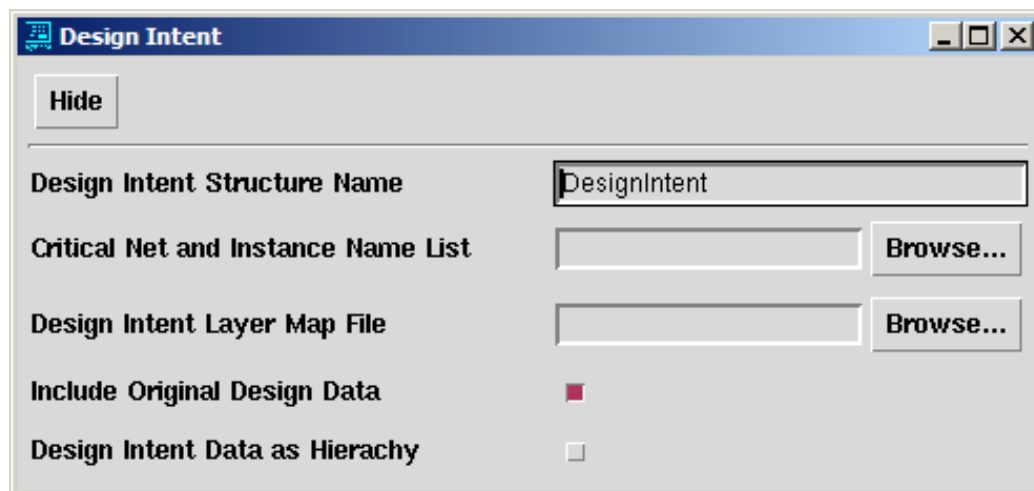
Figure 14-1 *Design Intent Flow*



Milkyway does the following in the design intent flow:

- Marks the critical entities (such as nets and cell instances) based on the input net and instance name lists.
- Calculates the design intent shapes (that is, boundaries) for critical cell instances using bounding boxes.
- Maps the original `layer:datatypes` design to the design intent `layer:datatypes` design.
- Invokes the `auStreamOut` command to stream out a GDSII file with design intent data.

Click the Design Intent button in the `auStreamOut` dialog box to open the Design Intent dialog box. Its options are described below.



Design Intent Structure Name

Specifies the structure name. Design intent data is streamed out in one level as a single structure.

Critical Net and Cell Instance Name List

Specifies the file that lists the critical net and cell instance names that are associated with design intent. (PrimeTime can provide the critical net and cell instance name lists.) The nets and instances are grouped by criticality using the keyword `DITYPE`.

[Example 14-1](#) shows the file list format.

Example 14-1 Sample Critical Net and Cell Instance List

```
; Generated by PrimeTime or other design tools
DITYPE CRITICAL_10_PERCENT
N INST1/NET12
N INST1/NET14
N INST1/NET15/Q
I INST2
DITYPE EM_SENSITIVITY
N INST1/NET12
N INST1/net16
N INST2/net14/q
I INST3
N INST4/net15/Qb
```

Design Intent Layer Map File

Maps the layer data type pairs in the original design to the design intent layer data type pairs.

Similar to the critical net and cell instance name list, the layer-mapping file is grouped by level of criticality using the keyword `DITYPE`. Because only bounding boxes are used for critical cell instances, all critical cell instances are mapped to the same layer data type pair.

Design intent layer mapping is formatted with the following statement:

```
N|I Original layer : original datatype -> DI layer : DI datatype
```

where N stands for net, I stands for cell instance, layer and data type are delimited by a colon (:), and the arrow symbol (->) specifies that the mapping is from left to right. [Table 14-26](#) shows an example of design intent layer mapping.

Table 14-26 *Design Intent Layer Mapping Example*

Layer name	Original layer number: data type	Design intent layer: data type
Metal1	31:0	31:10
Metal2	32:0	32:10
Metal3	33:0	33:10

In [Table 14-26](#), the data types are specified with identical layer numbers. As a result, the same layer is used for the original design layer and design intent layer.

Based on the information in the table, the layer mapping would be specified as follows:

```
N 31:0 -> 31:10
N 32:0 -> 32:10
N 33:0 -> 33:10
```

Note that the format of the design intent layer-mapping file is different from the format of the layer-mapping file that maps between the Milkyway database and the GDSII layer and data type.

[Example 14-2](#) shows the format of the design intent layer-mapping file.

Example 14-2 *Sample Design Intent Layer-Mapping File*

```
; Mark all nets with < 10 % timing slack.
DITYPE CRITICAL_10_PERCENT
N 31:0->31:10 ; Metal 1 critical design intent
N 32:0->32:10 ; Metal 2 critical design Intent
I 255:0->64:10 ; instance marking for critical design intent
```

```
; Mark all nets that are sensitive to signal integrity effects
DITYPE EM_SENSITIVE_NETS
N 31:0->31:15 ; Metal 1 EM design intent
N 32:0->32:15 ; Metal 2 EM design Intent
I 255:0->64:15 ; instance marking for EM design intent
```

Include Original Design Data

Specifies to include the original design when streaming out the data for critical nets and cell instances in different layer data type pairs. This option is selected by default.

Design Intent Data as Hierarchy

Specifies to use a hierarchical structure when streaming out design intent data. By default, the design intent data is streamed out in one level as a single structure. However, as the design intent data increases, the design intent data can be large if the original data is array based or if it contains hierarchical levels. In this case, using a hierarchical structure can significantly save data storage space. This option is not selected by default.

auStreamOutAllHierPinNetData

This switch enables [auStreamOut](#) to include the instance names as properties down the complete hierarchy.

Tool

Milkyway

Syntax

```
auStreamOutAllHierPinNetData #t
```

where the argument is as follows:

<code>#t</code>	Boolean, where <code>#t</code> indicates that the instance names are to be included as properties down the complete hierarchy.
-----------------	--

See Also

[auStreamOut](#)

auStreamOutArefContactArray

This switch enables [auStreamOut](#) to output each contact array based on the GDSII array reference (AREF) format. The purpose of it is to decrease the number of contact array files in the target GDSII file.

Tools and Menu Commands

Milkyway: Output > Stream Out

Syntax

```
auStreamOutArefContactArray boolean?
```

where the argument is as follows:

<i>boolean?</i>	Indicates which GDSII format will be used. Valid values: • <code>#t</code> means to use the array reference (AREF) format. • <code>#f</code> means to use the structure reference (SREF) format.
-----------------	---

Example

```
auStreamOutArefContactArray #t
```

See Also

[auStreamOut](#)

auStreamOutByLayerNumber

This switch enables [auStreamOut](#) to output geometries by layer number.

Tools and Menu Commands

Milkyway: Output > Stream Out

Syntax

```
auStreamOutByLayerNumber "layerNumber"
```

where the argument is as follows:

layerNumber A list of layer numbers.

Examples

```
auStreamOutByLayerNumber "2 4 6"
```

See Also

[auStreamOut](#)

auStreamOutCellInstanceNameAsText

This function enables the `auStreamOut` function to output a cell instance's name as a TEXT object to a specified layer and datatype.

Syntax

```
auStreamOutCellInstanceNameAsText integer1 integer2
```

Arguments

integer1

The layer that you want the TEXT object to be on

integer2

The datatype that you want the TEXT object to be on

Description

This command enables `auStreamOut` to output a cell instance's name as a TEXT object to the layer and datatype specified by the arguments: `integer1` and `integer2`.

Example

```
auStreamOutCellInstanceNameAsText 33 5
```

auStreamOutCellRenamePrefix

This switch enables [auStreamOut](#) to set a prefix character string for duplicate cells renaming.

Tools and Menu Commands

Milkyway: Output > Stream Out

Syntax

```
auStreamOutCellRenamePrefix "prefixStr"
```

where the argument is as follows:

prefixStr A string of characters.

Examples

```
auStreamOutCellRenamePrefix "_".
```

See Also

[auStreamOut](#)

auStreamOutCheckInvalidCells

Sets the `auStreamOut` command's "Check Invalid Cells" mode.

Syntax

```
auStreamOutCheckInvalidCells <boolean mode>
```

where the argument is as follows:

mode

`#t`

Specifies to quit with an error immediately if a self-referencing cell exists.

`#f`

Specifies not to check for self-referencing cells (default).

Description

This mode controls whether or not the `auStreamOut` command checks and error out immediately if a self-referencing cell (that is, an invalid cell) exists. A self-referencing cell is one that contains an instance of itself. By default, `auStreamOut` will not check for self-referencing cells.

Example

```
auStreamOutCheckInvalidCells #t
```

auStreamOutChildCellFillData

Sets the `auStreamOut` command's "Child Cell Fill Data" mode.

Syntax

```
auStreamOutChildCellFillData <boolean mode>
```

where the argument is as follows:

mode

`#t`

Specifies to write out each child cell's fill data.

`#f`

Specifies not to write out each child cell's fill data (default).

Description

This mode controls whether or not the `auStreamOut` command writes out each child cell's fill data. By default, `auStreamOut` will not write out each child cell's fill data.

Example

```
auStreamOutChildCellFillData #t
```

auStreamOutConfirmOverWriteGDSIIFile

This switch provides protection from overwriting the existing GDS file when running [auStreamOut](#).

Tools and Menu Commands

Milkyway: Output > Stream Out

Syntax

```
auStreamOutConfirmOverWriteGDSIIFile overwrite?
```

where the argument is as follows:

- | | |
|-------------------|--|
| <i>overwrite?</i> | Indicates whether or not to overwrite old GDS file.
Valid values: |
| | <ul style="list-style-type: none">• <i>#t</i> will enable the overwriting of the old GDS file.• <i>#f</i> will disable the overwriting of the old GDS file. |

Example

```
auStreamOutConfirmOverWriteGDSIIFile #t
```

See Also

[auStreamOut](#)

auStreamOutContactArrayMatrix

This switch enables [auStreamOut](#) to output contact arrays located on power and ground straps with the GDSII array reference (AREF) format.

Tools and Menu Commands

Milkyway: Output > Stream Out

Syntax

```
auStreamOutContactArrayMatrix boolean?
```

where the argument is as follows:

boolean?

Indicates which GDSII format will be used.
Valid values:

- `#t` means to use the array reference (AREF) format.
- `#f` means to use the structure reference (SREF) format.

Examples

```
auStreamOutContactArrayMatrix #t
```

See Also

[auStreamOut](#)

auStreamOutControlFileToLog

This function enables `auStreamOut` to output all reference library names defined in the control file to the Milkyway log file.

Syntax

```
auStreamOutControlFileToLog < boolean value >
```

where the argument is as follows:

value

must be `#t` or `#f`

Arguments

Boolean value

Argument valid values:

`#t`

Specifies to output the control file to log file.

`#f`

Specifies to disable the output.

Description

When the argument is set to `#t`, the command enables the `auStreamOut` command to output all reference library names defined in the control file to the Milkyway log file. Also, the control file mode must be enabled (`dbSetRefControlFileMode #t`).

Example

```
auStreamOutControlFileToLog #t
```

auStreamOutCustomViewInstance

Sets the `auStreamOut` command's "Custom View Instance" mode.

Syntax

```
auStreamOutCustomViewInstance <boolean mode>
```

where the argument is as follows:

mode

`#t`

Specifies to write out each of the child cell's specific view in addition to its CEL view.

`#f`

Specifies to write out each of the child cell's CEL view only (default).

Description

This mode controls how `auStreamOut` writes out each cell's child cells.

A cell can contain instances of a specific child cell view (usually the FRAM or CEL view, but it can be any view). If this mode is set to `#t`, that specific view is written to the GDS output file in addition to the CEL view. The parent cell references that specific view in the GDS output file. If this mode is set to `#f`, only the CEL view is written to the GDS output file.

For example, if the mode is set to `#t` and cell A contains an instance of cell B's FILL view, cell B's CEL and FILL view data are written out. In the GDS output file, cell structures "B" and "B\$FILL" are created, and cell structure "A" references structure "B\$FILL".

Example

```
auStreamOutCustomViewInstance #t
```

auStreamOutFillDataForAllCellTypes

This enables the auStreamOut function to output fill data for all cell types.

Syntax

```
auStreamOutFillDataForAllCellType < boolean value >  
value
```

must be #t or #f

Arguments

Boolean value

Argument valid values:

#t

Output fill data for all cell types

#f

Output fill data for Macro cell only

Description

When the argument is set to #t, the command enables the function auStreamOut to output fill data for all cell types. When the argument is #f, it outputs fill data for Macro cells only.

Example

```
auStreamOutFillDataForAllCellTypes #t
```

auStreamOutFillDataMapping

Sets the `auStreamOut` command's "Fill Data Mapping" mode.

Syntax

```
auStreamOutFillDataMapping <boolean mode>
```

where the argument is as follows:

mode

`#t`

Specifies to write out the fill, notch, and gap data.

`#f`

Specifies to not write out fill, notch, and gap data (default).

Description

This mode controls whether or not the `auStreamOut` command writes out fill, notch, and gap data with the user-specified layer and datatype. By default, `auStreamOut` will not write out this data.

Example

```
auStreamOutFillDataMapping #t
```

auStreamOutGDSIIVersion

This switch enables [auStreamOut](#) to set the GDSII version number instead of using the default value of five.

Tools and Menu Commands

Milkyway: Output > Stream Out

Syntax

```
auStreamOutGDSIIVersion version
```

where the argument is as follows:

<i>version</i>	An integer representing the version number.
----------------	---

Example

```
auStreamOutGDSIIVersion 4
```

See Also

[auStreamOut](#)

auStreamOutKeepDataType

This switch enables [auStreamOut](#) to output each geometry's original data type instead of using the default value of zero.

Tools and Menu Commands

Milkyway: Output > Stream Out

Syntax

```
auStreamOutKeepdataType keepDataType?
```

where the argument is as follows:

<i>keepDataType?</i>	Indicates how to output the value of the data type. Valid values: • <i>#t</i> means to output the data type with original value. • <i>#f</i> means to output the data type with the zero value.
----------------------	--

Example

```
auStreamOutKeepdataType #t
```

See Also

[auStreamOut](#)

auStreamOutNetNameBasedLayerMapping

This switch enables [auStreamOut](#) to support net name based layer mapping.

Tools and Menu Commands

Milkyway: Output > Stream Out

Syntax

```
auStreamOutNetNameBasedLayerMapping #t|#f?
```

where the argument is as follows:

<i>display?</i>	Indicates whether or not to support net name based layer mapping. Valid values: • #t means enable net name based layer mapping. • #f means disable net name based layer mapping.
-----------------	---

Example

```
auStreamOutNetNameBasedLayerMapping #t
```

See Also

[auStreamOut](#)

auStreamOutNetWithPathTypeZero

This enables the auStreamOut function to output a WIRE object with PATHTYPE 0.

Syntax

```
auStreamOutNetWithPathTypeZero < boolean value >  
value
```

Must be true (#t) or false (#f)

Arguments

Boolean value

Argument valid values:

#t

Output a WIRE object with PATHTYPE 0

#f

Disables the cast

Description

When the argument is set to #t, the function enables the command auStreamOut to output a WIRE object with PATHTYPE 0. Milkyway will output a WIRE object as “square-ended with no extension” instead of “square-ended with 1/2 width extension” (PATHTYPE 2).

If the argument is #f, this cast is disable.

Example

```
auStreamOutNetWithPathTypeZero #t
```

auStreamOutOddShapePin

This enables the auStreamOut function to output odd shaped pins as several polygons.

Syntax

```
auStreamOutOddShapePin < boolean value >
```

value

must be #t or #f

Arguments

Boolean argument valid values:

#t

Stream out the odd shape pins as several polygons

#f

Do not stream out the odd shape pins, flag a Error number and deletes the GDSII file.

Description

This switch enables the auStreamOut function to output odd shape pins as several polygons.

If this command is issued for the first time, auStreamOut reports an odd shape pin message and continues to generate the GDSII file.

Example

```
auStreamOutOddShapePin #t
```

auStreamOutOptionsToLog

This switch enables [auStreamOut](#) to output dialog box options and Scheme command switches to a Milkyway log file.

Tools and Menu Commands

Milkyway: Output > Stream Out

Syntax

```
auStreamOutOptionsToLog boolean?
```

where the argument is as follows:

<i>boolean?</i>	Indicates output options to log file or not. Valid values:
	• #t means to output the options to a log file. • #f means disable the output.

Example

```
auStreamOutOptionsToLog #t
```

See Also

[auStreamOut](#)

auStreamOutPropertyByObject

This switch enables [auStreamOut](#) to output the property information of polygons, paths, rectangles and text.

Tools and Menu Commands

Milkyway: Output > Stream Out

Syntax

```
auStreamOutPropertyByObject list
```

where the argument is as follows:

<i>list</i>	Indicates a list of objects, including polygons, paths, rectangles and text.
-------------	--

Examples

```
auStreamOutPropertyByObject '(path)
auStreamOutPropertyByObject '(polygon text path)
auStreamOutPropertyByObject ' ( )
```

Comments

Only four objects are allowed in a list. Each time *auStreamOutPropertyByObject* is used, it will cancel its previous setting and incorporate the new one. So:

```
auStreamOutPropertyByObject ' ( )
```

will cancel all settings and will not output any property information on polygons, paths, rectangles or text.

See Also

[auStreamOut](#)

auStreamOutRemoveEmptyCells

Sets the `auStreamOut` command's "Remove Empty Cells" mode.

Syntax

```
auStreamOutRemoveEmptyCells <boolean mode>
```

where the argument is as follows:

mode

`#t`

Specifies to omit the empty cells.

`#f`

Specifies to write the empty cells (default).

Description

This mode controls whether or not the `auStreamOut` command omits or writes out empty cells and references to the output file. By default, the `auStreamOut` command writes empty cells.

Example

```
auStreamOutRemoveEmptyCells #t
```

auStreamOutRenameCellName

This switch enables [auStreamOut](#) to rename a cell name by a given cell rename file. Only the master cell will be renamed.

Tools and Menu Commands

Milkyway: Output > Stream Out

Syntax

```
auStreamOutRenameCellName "renameFile"
```

where the argument is as follows:

<i>renameFile</i>	A file containing the cell names with the following format: <i>Milkyway_CellName GDSII_CellName.</i>
-------------------	---

Example

```
auStreamOutRenameCellName "rename.txt"
```

See Also

[auStreamOut](#)

auStreamOutRenameRefLibDuplicateCellFirst

This switch enables [auStreamOut](#) to rename a duplicate cell starting with the cell in the reference library instead of a random selection. This is to prevent the cell in the top design library from being renamed first.

Tools and Menu Commands

Milkyway: Output > Stream Out

Syntax

```
auStreamOutRenameRefLibDuplicateCellFirst refFirst?
```

where the argument is as follows:

<i>refFirst?</i>	Indicates whether or not to set the selection preference. Valid values: • #t means the cell is being renamed from the reference library first. • #f means a random selection.
------------------	--

auStreamOutRenameRefLibDuplicateCellFirst

Examples

```
auStreamOutRenameRefLibDuplicateCellFirst #t
```

See Also

[auStreamOut](#)

auStreamOutSelectedCells

This function allows you to specify the cells to output to a text file.

Syntax

```
auStreamOutSelectedCells <String fileName >
```

Arguments

String

argument valid value:

fileName

Name of the text file to output cell names to. String value.

Description

The `auStreamOutSelectedCells` function enables a designer to specify the cells to output to a text file.

Example

```
auStreamOutSelectedCells "SelectedCellFile"
```

auStreamOutShortCellName

Converts a cell or via name longer than 16 characters, to a unique 16 character name.

Tool

Milkyway

Syntax

```
auStreamOutShortCellName "filename" #t
```

where the arguments are as follows:

<i>filename</i>	Name of the file to which you want to output the 16 character names.
<i>#t</i>	Boolean, where <i>#t</i> indicates that the character names are to be created in the named file.

auStreamOutShowCellInfo

Enables [auStreamOut](#) to display cell version, creation date, and the last modification date on screen.

Tools and Menu Commands

Milkyway: Output > Stream Out

Syntax

```
auStreamOutShowCellInfo display?
```

where the argument is as follows:

<i>display</i>	Indicates whether or not to display cell information. Valid values:
	• #t means enable display of cell information • #f means disable the display

Example

```
auStreamOutShortCellInfo #t
```

See Also

[auStreamOut](#)

auStreamOutSkipGRouteContact

Sets the `auStreamOut` command's "Skip Global Route Contacts" mode.

Syntax

```
auStreamOutSkipGRouteContact <boolean mode>
```

where the argument is as follows:

mode

#t

Specifies to omit global route contacts.

#f

Specifies to write out global route contacts (default).

Description

This mode controls whether or not the `auStreamOut` command omits or writes out contacts created by the global router. By default, the `auStreamOut` command writes out global route contacts.

Example

```
auStreamOutSkipGRouteContact #t
```

auStreamOutSkipTheseCells

This switch enables [auStreamOut](#) to skip the output of those cells listed in a file.

Tools and Menu Commands

Milkyway: Output > Stream Out

Syntax

```
auStreamOutSkipTheseCells "cellFile"
```

where the argument is as follows:

<i>cellFile</i>	Name of the file to which you want to skip those cells. Each line should contain only one single cell name.
-----------------	--

Example

```
auStreamOutSkipTheseCells "cellList"
```

See Also

[auStreamOut](#)

auStreamOutSplitPath

This enables auStreamOut to split those paths with more than 200 pairs of XY coordinates into several path objects.

Syntax

```
auStreamOutSplitPath < boolean value >
```

value

must be #t or #f

Arguments

Boolean value

Argument valid values:

#t

Splits paths that have more than 200 pairs of XY coordinates

#f

Disables the split paths option

Description

This enables the auStreamOut command to split those paths with more than 200 pairs of XY coordinates into several path objects so that each new path object has a maximum of 200 points.

Example

```
auStreamOutSplitPath #t
```

auStreamOutUnlinkGDSIIFile

This switch enables [auStreamOut](#) to remove erroneous GDSII file. If set to `#t` (default), [auStreamOut](#) will give an error number and delete the erroneous GDSII file. If set to `#f`, [auStreamOut](#) will not delete the erroneous GDSII file and only give an error number.

Tools and Menu Commands

Milkyway: Output > Stream Out

Syntax

```
auStreamOutUnlinkGDSIIFile boolean?
```

where the argument is as follows:

<i>boolean?</i>	Indicates remove GDSII file or not. Valid values:
	• <code>#t</code> means to remove GDSII file. (default value). • <code>#f</code> means disable to remove.

Example

```
auStreamOutUnlinkGDSIIFile #f
```

See Also

[auStreamOut](#)

auStreamOutWillAbortIfCellsMissing

Flags the auStreamOut function to abort GDSII file generation if a cell is missing.

Syntax

```
auStreamOutWillAbortIfCellIsMissing < boolean value >  
value
```

must be #t or #f

Arguments

Boolean value

Argument valid values:

#t

Aborts GDSII file generation if any cell is missing

#f

Disables this option

Description

If any cell to be output is missing, the auStreamOutWillAbortIfCellsMissing function enables auStreamOut to abort GDSII file generation.

Example

```
auStreamOutWillAbortIfCellIsMissing #t
```

auStreamOutWithExtendedLayerMapping

This enables `auStreamOut` to duplicate object output based on the definition in the layer map file.

Syntax

```
auStreamOutWithExtendedLayerMapping <boolean value >
```

value

must be `#t` or `#f`

Arguments

Boolean value

Argument valid values:

`#t`

Enables output with extended layer mapping

`#f`

Disables extended layer mapping

Description

The `auStreamOutWithExtendedLayerMapping` function enables the `auStreamOut` command to duplicate object output based on the definition in the layer map file, for example

```
D 34 24 0; Convert Synopsys layer 34 to GDSII Stream layer 24 and datatype 0
D 34 14 0; Convert Synopsys layer 34 to GDSII Stream layer 14 and datatype 0
```

This function enables `auStreamOut` to create the same data on layer 24 as well as on layer 14.

Example

```
auStreamOutWithExtendedLayerMapping #t
```

auSuffixDuplicateLongCellName

This switch enables `auStreamOut` to rename a 128-character long duplicate cell name with a suffix added at the end instead of copying it to the beginning of cell name.

Tools and Menu Commands

Milkyway: Output > Stream Out

Syntax

```
auSuffixDuplicateLongCellName suffixLongName?
```

where the argument is as follows:

- | | |
|------------------|---|
| <i>refFirst?</i> | Indicates whether or not to suffix a long cell name.
Valid values: |
| | <ul style="list-style-type: none">• <code>#t</code> means to add suffix at the end.• <code>#f</code> means to copy to the beginning. |

Examples

```
auSuffixDuplicateLongCellName #t
```

See Also

[auStreamOut](#)

auSynopToCLF

Translates a Synopsys Format Report Lib file to a CLF/port information file containing the following functions:

- *defineTimeIntrinsic* (linear delay model) or [clfCreateTable](#) and *defineTimeTLU* (table lookup model).
- *definePortCapacitance*.
- *dbSetCellPortTypes*.
- *dbSetLModelSubType*.

When translating a Synopsys timing file, Milkyway identifies any cells containing clock ports or flip-flops. If your library contains cells for which this assumption is incorrect (for example, latches), edit the `dbSetLModelSubType` functions in the output file.

After you translate the Synopsys timing file, load the CLF/port information file using [auLoadCLF](#).

Milkyway determines the minimum and maximum scaled delays using the following:

- The k-factors you specify in a k-factor file.
- The temperature, voltage, and process factor values (minimum, maximum, and nominal) you specify on the Synopsys To CLF Translator dialog box.

Note:

You can change the settings for temperature, voltage, and process factor using `aprTimingOptions`.

The Synopsys tool calculates the minimum and maximum scaled delays using the formula:

$$D_{\text{MinScaled}} = D \times (1 + (T_{\text{min}} - T_{\text{nom}}) K_T) \times (1 + (V_{\text{max}} - V_{\text{nom}}) K_V) \times (1 + (P_{\text{min}} - P_{\text{nom}}) K_P)$$

$$D_{\text{MaxScaled}} = D \times (1 + (T_{\text{max}} - T_{\text{nom}}) K_T) \times (1 + (V_{\text{min}} - V_{\text{nom}}) K_V) \times (1 + (P_{\text{max}} - P_{\text{nom}}) K_P)$$

where the variables are as follows:

$D_{\text{MinScaled}}$	The computed minimum scaled delay for the operating condition under which you want the chip to operate.
$D_{\text{MaxScaled}}$	The computed maximum scaled delay for the operating condition under which you want the chip to operate.
T_{min}	The minimum temperature for the operating condition under which you want the chip to operate (specified by the <i>Min Temperature</i> option on the Synopsys To CLF Translator dialog box).
T_{max}	The maximum temperature for the operating condition under which you want the chip to operate (specified by the <i>Max Temperature</i> option on the Synopsys To CLF Translator dialog box).
T_{nom}	The nominal temperature for the operating condition under which you want the chip to operate (specified by the <i>Nom Temperature</i> option on the Synopsys To CLF Translator dialog box).
K_T	The k-factor for temperature (specified by the <i>k_temp</i> value in the k-factor file).
V_{min}	The minimum voltage for the operating condition under which you want the chip to operate (specified by the <i>Min Voltage</i> option on the Synopsys To CLF Translator dialog box).

$V_{\max}$	The maximum voltage for the operating condition under which you want the chip to operate (specified by the <i>Max Voltage</i> option on the Synopsys To CLF Translator dialog box).
V_{nom}	The nominal voltage for the operating condition under which you want the chip to operate (specified by the <i>Nom Voltage</i> option on the Synopsys To CLF Translator dialog box).
K_V	The k-factor for voltage (specified by the <i>k_volt</i> value in the K-Factor file).
$P_{\min}$	The minimum process factor for the operating condition under which you want the chip to operate (specified by the <i>Max Process Factor</i> option on the Synopsys To CLF Translator dialog box).
$P_{\max}$	The maximum process factor for the operating condition under which you want the chip to operate (specified by the <i>Max Process Factor</i> option on the Synopsys To CLF Translator dialog box).
P_{nom}	The nominal process factor for the operating condition under which you want the chip to operate (specified by the <i>Nom Process Factor</i> option on the Synopsys To CLF Translator dialog box).
K_P	The k-factor for process (specified by the <i>k_process</i> value in the k-factor file).

For example, the Synopsys tool calculates the minimum scaled delay for `intrinsic_rise` using the formula:

$$\text{MinScaledDelay}_{\text{intrinsic_rise}} = \text{Delay}_{\text{intrinsic_rise}} \times (1 + (T_{\min} - T_{\text{nom}})k_{\text{temp_intrinsic_rise}}) \times (1 + (V_{\max} - V_{\text{nom}})k_{\text{volt_intrinsic_rise}}) \times (1 + (P_{\min} - P_{\text{nom}})k_{\text{process_intrinsic_rise}})$$

Tools and Menu Commands

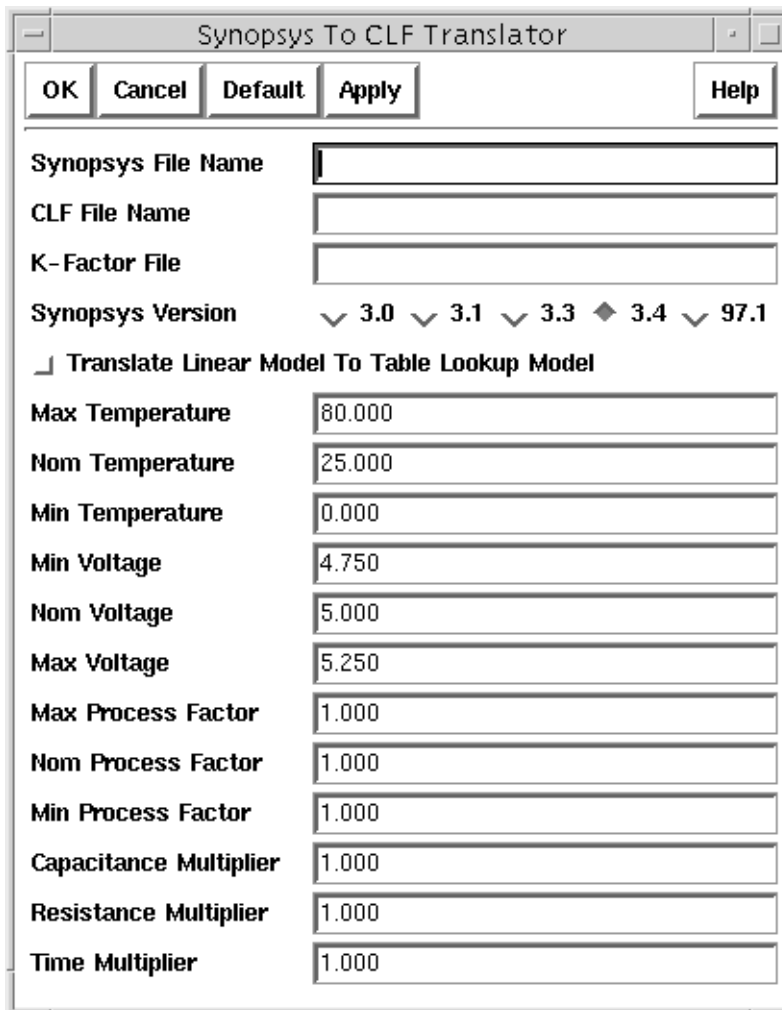
Milkyway: CLF > Synopsys To CLF

Prerequisites

Create a k-factor file.

Synopsys To CLF Translator Dialog Box

The menu command opens the following dialog box. Its options are described below.



Synopsys To CLF Translator

OK Cancel Default Apply Help

Synopsys File Name

CLF File Name

K-Factor File

Synopsys Version ▼ 3.0 ▼ 3.1 ▼ 3.3 ◆ 3.4 ▼ 97.1

☒ Translate Linear Model To Table Lookup Model

Max Temperature

Nom Temperature

Min Temperature

Min Voltage

Nom Voltage

Max Voltage

Max Process Factor

Nom Process Factor

Min Process Factor

Capacitance Multiplier

Resistance Multiplier

Time Multiplier

Synopsys File Name

Type the name of the Synopsys file you want to translate.

CLF File Name

Type the name you want to assign to the new CLF file.

K-Factor File

Type the name of the file containing the k-factors for temperature, voltage, and process, as they are defined in the Synopsys technology library.

Synopsys Version

Select the version of Synopsys that was used to create the Synopsys file.

Note:

Does not support Synopsys files generated with Synopsys version 3.0a.

Translate Linear Model To Table Lookup Model

Select to translate linear delay model information to the table lookup delay model.

Max Temperature

Type the maximum temperature for the operating condition under which you want the chip to operate.

Nom Temperature

Type the nominal temperature for the operating condition under which you want the chip to operate.

Min Temperature

Type the minimum temperature for the operating condition under which you want the chip to operate.

Min Voltage

Type the minimum voltage for the operating condition under which you want the chip to operate.

Nom Voltage

Type the nominal voltage for the operating condition under which you want the chip to operate.

Max Voltage

Type the maximum voltage for the operating condition under which you want the chip to operate.

Max Process Factor

Type the maximum process factor for the operating condition under which you want the chip to operate.

Nom Process Factor

Type the nominal process factor for the operating condition under which you want the chip to operate.

Min Process Factor

Type the minimum process factor for the operating condition under which you want the chip to operate.

Capacitance Multiplier

Type the number by which Milkyway should multiply the load values specified in the Synopsys Format Report Lib file to get the actual capacitance values needed by the Synopsys tool.

Resistance Multiplier

Type the number by which Milkyway should multiply the load values specified in the Synopsys Format Report Lib file to get the actual resistance values needed by the Synopsys tool.

Time Multiplier

Type the number by which Milkyway should multiply the load values specified in the Synopsys Format Report Lib file to get the actual time values needed by the Synopsys tool.

auVerilogIn

Note:

The auVerilogIn command has been replaced by the [auVerilogToCell](#) command. The auVerilogIn command is no longer supported.

Translates a netlist from Verilog Hardware Description Language (HDL) format to Synopsys format, creating a library (if the library does not already exist) and a netlist cell.

Tools and Menu Commands

Milkyway: Netlist In > Verilog In

Jupiter: Design Setup > Verilog In

Prerequisites

If you want to translate multiple Verilog files, create a Verilog list file, that contains the names of the files you want to translate. If the files are not in the directory where you started Milkyway, then include the full path. The format of the file is:

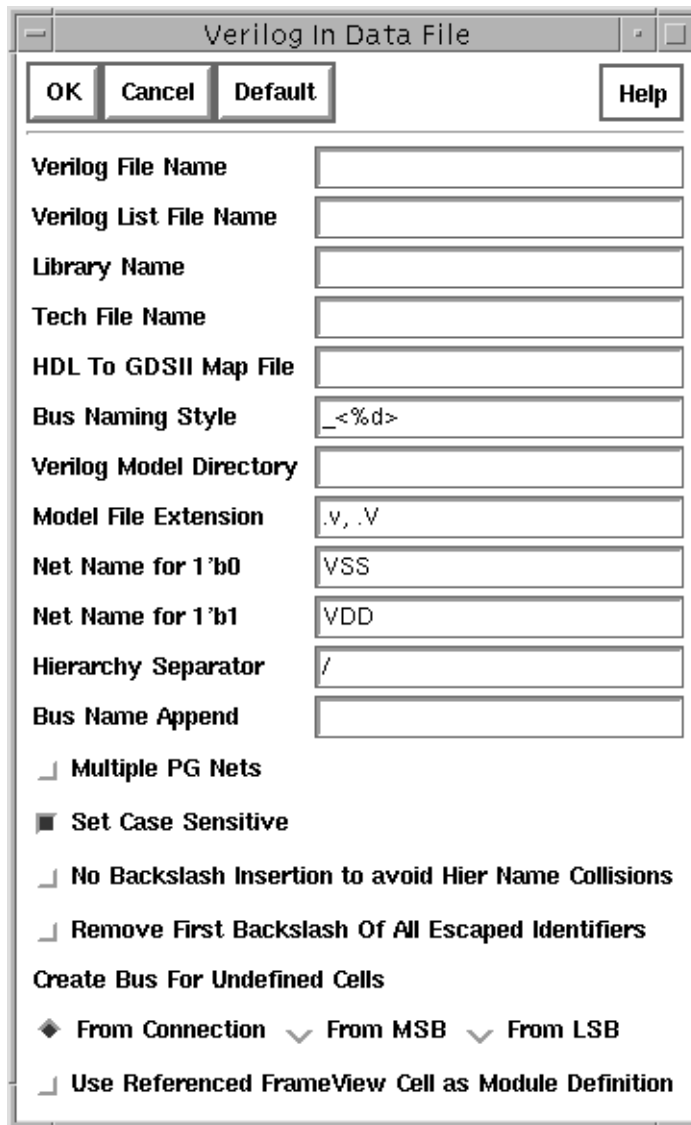
```
fileName1 fileName2 ... fileNameN
```

where *fileNames* are the names (including the paths, if necessary) of the files.

If you want to map Verilog HDL cell names to GDSII cell names, create an HDL To GDSII map file.

Verilog In Data File Dialog Box

The menu command opens the following dialog box. Its options are described below.



The image shows a dialog box titled "Verilog In Data File". It has a standard Windows-style title bar with minimize, maximize, and close buttons. Below the title bar are four buttons: "OK", "Cancel", "Default", and "Help". The main area of the dialog contains several labeled text input fields and a group of checkboxes.

Field Name	Value
Verilog File Name	
Verilog List File Name	
Library Name	
Tech File Name	
HDL To GDSII Map File	
Bus Naming Style	_<%d>
Verilog Model Directory	
Model File Extension	.v, .V
Net Name for 1'b0	VSS
Net Name for 1'b1	VDD
Hierarchy Separator	/
Bus Name Append	

Below the text fields are several checkboxes and a group of radio buttons:

- ☐ Multiple PG Nets
- ☒ Set Case Sensitive
- ☐ No Backslash Insertion to avoid Hier Name Collisions
- ☐ Remove First Backslash Of All Escaped Identifiers
- Create Bus For Undefined Cells
 - ☒ From Connection
 - ☐ From MSB
 - ☐ From LSB
- ☐ Use Referenced FrameView Cell as Module Definition

Verilog File Name

Type the name of the Verilog file you want to translate, or leave it blank when you have a Verilog list file containing file names for multiple files. If the file is not in the directory where you started Milkyway, you need to include the path.

Verilog List File Name

Type the name of the Verilog list file containing the names of multiple files to be translated, or leave blank if you are translating a single source file specified by *Verilog File Name*.

Library Name

Type the name of the library in which you want to place the netlist cell. If the library does not exist, Milkyway will create it.

Note:

If you do not type a Library Name, Milkyway creates a library named verilogHDL (if case insensitive), or VERILOGHDL (if case sensitive).

Tech File Name

Type the name of the technology file you want to load to the library. If the library already exists, Milkyway ignores this option and you can leave this field blank.

HDL To GDSII Map File

Type the name of the file you want Milkyway to use to map cell names during translation, or leave blank to use the HDL cell names.

Bus Naming Style

Type any string you want Milkyway to append to the bus name specified in the Verilog file when it translates the bus to individual ports, using *%d* to indicate the port number. The requirements are as follows:

- The string cannot contain any blanks.
- The string can contain only one *%d*.
- To specify a percent (%) in the string, use *%%*.

For example, if the Verilog file defines a bus as:

```
input [1:4] sys
```

and you specify a *Bus Naming Style* of `_<%%%d>`, Milkyway translates the bus defined by this statement as ports `sys_<%1>`, `sys_<%2>`, `sys_<%3>`, and `sys_<%4>`.

Note:

After you import the netlist, you can redefine the bus using `aprCreateBus`.

Verilog Model Directory

Type the name of the directory containing the Verilog module files, or leave blank if the modules are included in the file specified by *Verilog File Name*. Separate two or more directories with commas.

Model File Extension

Type the extensions used in the names of module files, or leave blank if the modules are included in the file specified by *Verilog File Name*. Separate two or more extensions with commas.

Net Name for 1'b0

Type the name you want to assign to the 1-bit 0 signal in the Verilog file.

Net Name for 1'b1

Type the name you want to assign to the 1-bit 1 signal in the Verilog file.

Hierarchy Separator

Type the character you want to use as a separator in hierarchical names. If this character is used in cell instance or net names, Milkyway will translate the name with a backslash (\) preceding the hierarchy separator character.

If you select the *No Backslash Insertion to Avoid Hier Name Collisions*, Milkyway will not insert a backslash before all hierarchy separators.

Bus Name Append

Type any string you want Milkyway to insert into the bus name specified in the Verilog file when it translates the bus to individual bits. The translator inserts the string before the *Bus Naming Style* string in each bus name.

For example, if you enter “Bus” in the *Bus Name Append* field and [%d] in the *Bus Naming Style*, a bus in the Verilog file called “hif_rdata[1]” is translated as “hif_rdataBus[1],” which distinguishes Verilog net and bus files for the Synopsys tool.

Caution!

If you append the bus names you will have serious back annotation problems, because the bus names will be different in the original netlist and the Synopsys tool’s data base.

Multiple PG Nets

By default, Milkyway will create:

- one power net for l'b1 and all nets that net types are “supply1”.
- one ground net for 1'b0 and all nets that net type are “supply0”.

If you turn on the Multiple PG Nets option, Milkyway will create multiple power and ground nets.

For example:

```

module top(a, b);
input a;
output [1:0] b;
supply1 vdd1, vdd2
supply0 vss1, vss2;
assign b [1] = 1'b1;
assign b [0] = 1b0;
...
endmodule
assume "Net Name for 1'0 is "VSS"
assume "Net Name for 1'1 is "VDD"

```

When this option is off (default):

- Milkyway creates power net “VDD” (1'b1, *vdd1* and *vdd2* will be replaced by “VDD”).
- Milkyway creates ground net “VSS” (1'b0, *vss1* and *vss2* will be replaced by “VSS”).

When this option is on:

- Milkyway creates power nets “VDD” (only 1'b1 will be replaced by “VDD”), “*vdd1*” and “*vdd2*”.
- Milkyway will also create three ground nets: “VSS”, “*vss1*” and “*vss2*”.

Set Case Sensitive

Deselect if you do not want the library created to be case sensitive. Libraries are case sensitive by default. If a library already exists, this option will not overwrite the existing case sensitive setting.

No Backslash Insertion to avoid Hier Name Collisions

Select if you do not want Milkyway to insert a backslash when translating cell instance, port and net names to avoid confusion with the backslash used as hierarchy separators.

If you deselect the default value, Milkyway will insert a backslash before all hierarchy separators.

Remove First Backslash Of All Escaped Identifiers

Select to remove the first backslash of all escaped identifiers.

By default, Milkyway retains the first backslash for identifiers ending with nonfunctional “*Bus Naming Style*” (this identifier is not a bus bit; for example, \a[0], while it removes the first backslash for all the others.

Caution!

If you select this option, you may get an incorrect output through SPEF-out, PDEF-out, SDF-out, DEF-out, and EDIF-out.

Create Bus For Undefined Cells

If any port of any undefined cell is a bus, Milkyway will create a bus port according to this option and how the cell is instantiated. Milkyway records how bus ports have been created in the log file for *hdl2A*.

For example:

If an undefined cell is instantiated like the following:

```
sub si (.port(net[x:y]),...);
number of bus bits = r = |x-y|+1
```

Select options for Create Bus in Log Files

Option	Defines bus origin from ...
From Connection	x > y type [r:0] port x<y type [0:r] port
From MSB	type [r:0] port
From LSB	type [0:r] port

:

Use Referenced FrameView Cell as Module Definition

auVerilogIn will not create a .NETL view when this option is turned on and a .FRAM cell exists in the referenced library.

When this option is off, auVerilogIn creates .NETL views for the cells. The bus port information is only available in .NETL view, so when no other .NETL views are found, the bus port connections cannot be generated.

auVerilogOut

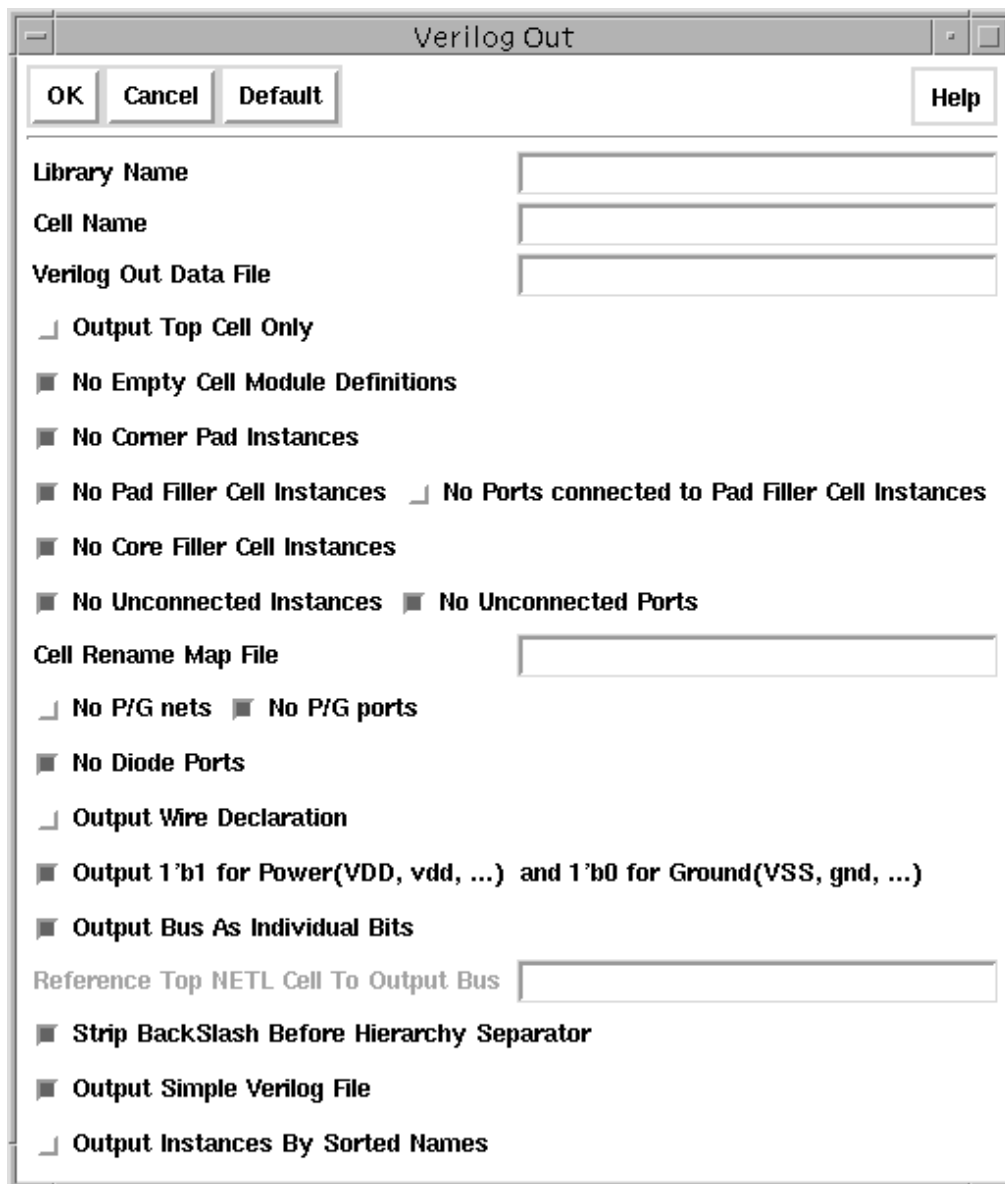
Translates connectivity information contained in a Synopsys top-level cell to a file in Verilog format.

Tools and Menu Commands

Milkyway: Output > Verilog Out

Verilog Out Dialog Box

The menu command opens the following dialog box. Its options are described below.



The image shows a dialog box titled "Verilog Out". It has a standard Windows-style title bar with minimize, maximize, and close buttons. Below the title bar are four buttons: "OK", "Cancel", "Default", and "Help". The main area of the dialog contains several input fields and checkboxes. The "Library Name" field is at the top, followed by "Cell Name" and "Verilog Out Data File". Below these are several checkboxes: "Output Top Cell Only" (unchecked), "No Empty Cell Module Definitions" (checked), "No Corner Pad Instances" (checked), "No Pad Filler Cell Instances" (checked), "No Ports connected to Pad Filler Cell Instances" (unchecked), "No Core Filler Cell Instances" (checked), "No Unconnected Instances" (checked), and "No Unconnected Ports" (checked). There is a "Cell Rename Map File" field. Below that are more checkboxes: "No P/G nets" (unchecked), "No P/G ports" (checked), "No Diode Ports" (checked), "Output Wire Declaration" (unchecked), "Output 1'b1 for Power(VDD, vdd, ...) and 1'b0 for Ground(VSS, gnd, ...)" (checked), and "Output Bus As Individual Bits" (checked). At the bottom is a "Reference Top NETL Cell To Output Bus" field, followed by "Strip BackSlash Before Hierarchy Separator" (checked), "Output Simple Verilog File" (checked), and "Output Instances By Sorted Names" (unchecked).

Verilog Out

OK Cancel Default Help

Library Name

Cell Name

Verilog Out Data File

☐ Output Top Cell Only

☒ No Empty Cell Module Definitions

☒ No Corner Pad Instances

☒ No Pad Filler Cell Instances ☐ No Ports connected to Pad Filler Cell Instances

☒ No Core Filler Cell Instances

☒ No Unconnected Instances ☒ No Unconnected Ports

Cell Rename Map File

☐ No P/G nets ☒ No P/G ports

☒ No Diode Ports

☐ Output Wire Declaration

☒ Output 1'b1 for Power(VDD, vdd, ...) and 1'b0 for Ground(VSS, gnd, ...)

☒ Output Bus As Individual Bits

Reference Top NETL Cell To Output Bus

☒ Strip BackSlash Before Hierarchy Separator

☒ Output Simple Verilog File

☐ Output Instances By Sorted Names

Library Name

Type the name of the library containing the cell for which you want to export connectivity information.

Cell Name

Type the name of the top-level cell for which you want to export connectivity information.

Verilog Out Data File

Type the name of the file to which you want to write the connectivity information.

Output Top Cell Only

Select to output top cell information only.

No Empty Cell Module Definitions

Select if you do not want Milkyway to translate empty cells.

No Corner Pad Instances

Select if you do not want Milkyway to translate corner pad cell instances.

No Pad Filler Cell Instances

Select if you do not want Milkyway to translate filler cell instances.

No Core Filler Cell Instances

Select if you do not want Milkyway to translate core filler cell instances.

No Unconnected Instances

Select if you do not want Milkyway to translate unconnected instances.

No Unconnected Ports

Select if you do not want Milkyway to translate unconnected ports. If this option is not selected, unconnected ports are generated and new nets are created to connect with the ports.

Cell Rename Map File

Type the name of the cell mapping file you want Milkyway to use to map cell names during translation, or leave blank to use GDSII cell names. If you used an HDL to GDSII map file to map cell names when you imported your netlist, use this same file to map names back to HDL.

No P/G nets

Select if you do not want Milkyway to translate power and ground nets. By default, this option is not selected, and power and ground nets (tie-up or tie-down only) are translated and output.

No P/G ports

Deselect if you want Milkyway to translate the power and ground ports. By default, this option is selected, and the power and ground ports will not be translated and output.

Output Wire Declaration

Select to add wire information to the output. By default, this option is not selected and no wire declaration is generated.

Output 1'b1 for Power (VDD, vdd, ...) and Output 1'b0 for Ground (VSS, gnd, ...)

Select to output power nets as 1'b1 and ground nets as 1'b0. For example, if you have power net VDD connected to *port1*, and ground net VSS connected to *port2*

And the Option is ...	Milkyway outputs the nets as ...
Selected	assign port1 = 1'b1 ; assign port2 = 1'b0 ;
Not selected	assign port1 = VDD ; assign port2 = VSS ;

When this option is not selected, “supply1” and “supply0” are generated for power and ground nets at the net declaration section of the verilog file. When this option is selected, as it is by default, “1'b1” and “1'b0” are created at the cell instantiation section and no net declaration is created for them.

Output Bus As Individual Bits

Select to output buses as individual bits. For example, if you have a bus with bits A[0], A[1], and A[2]:

And the Option is ...	Milkyway outputs the bus as ...
Selected	<pre>module X (A[2], A[1], A[0]); input A[2]; /*Individual Bits*/ input A[1]; /*Individual Bits*/ input A[0]; /*Individual Bits*/ endmodule</pre>
Not selected	<pre>module X (A); input [2:0] A; /*As a Bus*/ endmodule</pre>

Reference Top NETL Cell To Output Bus

Type the name of the .NETL cell which contains the bus information you want to output.

Very often, a hierarchical .NETL cell is expanded and then bound to a top cell, usually with a different name for placement and routing. Because the bus information is stored in the .NETL cells, you need to specify the original top .NETL cell name when outputting bus information.

Strip BackSlash Before Hierarchy Separator

When selected, the tool removes backslashes that appear before hierarchy separators for every identifier, including ports, nets, and instances.

Output Simple Verilog File

When selected, the tool generates a Verilog file that is smaller and easier to read. Used for backward compatibility, selecting this option allows you to generate a simple format Verilog file similar to version 2.2.

Output Instances By Sorted Names

Select this feature to control the order of instances in the Verilog Files. When selected, the feature will output filenames alphabetically. When deselected, filenames will order arbitrarily.

auVerilogToCell

Performs Verilog netlist input, flattening, and binding to a CEL view. The Verilog reader can be used to perform hierarchy preservation and is also capable of handling “dirty” netlists.

Equivalent Tcl Command

```
read_verilog
```

See the man pages for Tcl command descriptions and syntax. For more information about the man pages, enter `man help` on the command line, or enter `man command_name` for information about a specific Tcl command.

Tools

Design Planning, Astro

Verilog To Cell Dialog Box

The `auVerilogToCell` command opens the following dialog box. Its options are described below.

Verilog To Cell

OK Cancel Default Reference Library Global Net Options Help

Library Name Browse...

Verilog File Name Browse...

Output Cell Name

Optional:

Top Module Name

Top Module Name List File Browse...

Tech File Name Browse...

HDL To GDSII Map File Browse...

Ref Lib Control file Browse...

Bus Naming Style Bus Name Append

Net Name for 1'b0 Net Name for 1'b1

Hierarchy Separator

☒ Set Case Sensitive ☐ Multiple PG Nets

☐ Specified Verilog File is a Verilog List File

☐ No Backslash Insertion to avoid Hier Name Collisions

☐ Remove First Backslash Of All Escaped Identifiers

Create Bus For Undefined Cells

☒ From Connection ☐ From MSB ☐ From LSB

☐ Use Referenced FrameView Cell as Module Definition if it is consistent

☐ Precede hierarchical names with "!" ☒ Expand netlist cell with no instance

☐ Stop at FRAM view cells only ☐ Print out single pin net

☒ Initialize Hierarchy Preservation ☐ Handle Dirty Netlist

☐ Open Library and Cell When Done ☐ Verilog IEEE 2001

Library Name

Enter the name of the library in which you want to place the netlist cell. If the library does not exist, Milkyway will create it.

Verilog File Name

Enter the name of the Verilog file you want to translate, or leave it blank when you have a Verilog list file containing file names for multiple files. If the file is not in the directory where you started Milkyway, you need to include the path.

Output Cell Name

Enter the name of the hierarchical netlist cell to be created and used during auVerilogOut or auHierVerilogOut.

Top Module Name

Enter the name of the top module in the original design.

Tech File Name

Enter the name of the technology file you want to load into the library. If the library already exists, Milkyway ignores this option and you can leave this field blank.

HDL to GDSII Map File

Enter the name of the file you want Milkyway to use to map cell names during translation, or leave the field blank to use the HDL cell names.

Bus Naming Style

The string you want Milkyway to append to the bus name specified in the Verilog file when it translates the bus to individual ports, using %d to indicate the port number. The requirements are as follows:

- The string cannot contain any blanks.
- The string can contain only one %d.
- To specify a percent (%) in the string, use %%%.

For example, if the Verilog file defines a bus as:

```
input [1:4] sys
```

and you specify a Bus Naming Style of _<%%d>, Milkyway translates the bus defined by this statement as ports sys_<%1>, sys_<%2>, sys_<%3>, and sys_<%4>.

Bus Name Append

Enter any string you want Milkyway to insert into the bus name specified in the Verilog file when it translates the bus into individual bits. The translator inserts the string before the Bus Naming Style string in each bus name.

For example, if you enter Bus in the Bus Name Append field and [%d] in the Bus Naming Style, a bus in the Verilog file called hif_rdata[1] is translated as hif_rdataBus[1], which distinguishes the Verilog net and bus files for the tool.

Note:

If you append the bus names, you will have serious back annotation problems, because the bus names will be different in the original netlist and the database.

Net Name for 1'b0

Enter the name you want to assign to the 1-bit 0 signal in the Verilog file.

Net Name for 1'b1

Enter the name you want to assign to the 1-bit 1 signal in the Verilog file.

Hierarchy Separator

The character you want to use as a separator in hierarchical names. If this character is used in cell instances or net names, Milkyway will translate the name with a backslash (> preceding the hierarchy separator character.)

If you deselect the No Backslash Insertion to avoid Hierarchy Name Collisions option, Milkyway does not insert a backslash before all hierarchy separators.

Set Case Sensitive

Deselect this option if you do not want the library created to be case sensitive. Libraries are case sensitive by default. If a library already exists, this option will not overwrite the existing case sensitive setting. The default is on.

Multiple PG Nets

Milkyway creates one power net for 1'b1 and all nets with net type supply1 and one ground net for 1'b0 and all nets with net type supply0. The default is off.

For example:

```
module top(a, b);  
  input a;  
  output [1:0] b;  
  supply1 vdd1, vdd2  
  supply0 vss1, vss2;  
  assign b [1] = 1'b1;  
  assign b [0] = 1b0;  
  ...  
endmodule
```

Assume the net name for 1'0 is VSS and the net name for 1'1 is VDD.

When this option is off (the default), Milkyway creates power net VDD (1'b1, vdd1 and vdd2 is replaced by VDD) and ground net VSS (1'b0, vss1 and vss2 is replaced by VSS).

When this option is on, Milkyway creates power nets VDD (only 1'b1 is replaced by VDD), vdd1 and vdd2.

Milkyway also creates three ground nets: VSS, vss1, and vss2.

Specified Verilog File is a Verilog List File

Informs the tool that the specified Verilog File is a Verilog List File. A Verilog list file contains the names of multiple Verilog Files (one per line).

No Backslash Insertion to avoid Hier Name Collisions

Select this option if you do not want Milkyway to insert a backslash when translating cell instance, port and net names to avoid confusion with the backslash used as hierarchy separators.

If you deselect this option, Milkyway inserts a backslash before all hierarchy separators.

Remove First Backslash Of All Escaped Identifiers

Removes the first backslash of all escaped identifiers.

By default, Milkyway retains the first backslash for identifiers ending with nonfunctional Bus Naming Style (this identifier is not a bus bit; for example, `\a[0]`), while it removes the first backslash for all the others.

Note:

If you select this option, you may get an incorrect output through SPEF-out, PDEF-out, SDF-out, DEF-out, and EDIF-out.

Create Bus For Undefined Cells

If any port of any undefined cell is a bus, Milkyway creates a bus port for the undefined cell and instantiates it. It also creates a log file that contains a record of how the bus ports were created.

For example, If an undefined cell is instantiated like the following:

```
sub si (.port(net[x:y]),...);
```

the number of bus bits = $r = |x-y|+1$

Option	Defines bus origin from ...
From Connection (the default)	$x > y$ type $[r:0]$ port $x < y$ type $[0:r]$ port
From MSB	type $[r:0]$ port

Option	Defines bus origin from ...
From LSB	type [0:r] port

Precede hierarchical names with “/”

Milkyway inserts a backslash before all hierarchy names. This ensures consistency with SDF files. The default is off.

Expand netlist cell with no instance

Expands netlist cells that do not contain physical descriptions. Select this option if you have netlist instances containing only nets. If you deselect this option, and you have netlist instances with no physical description, expansion fails. The default is on.

Stop at FRAM view cells only

If this option is on, the netlist expanding process stops at .FRAM view cells only. All child cells are expanded even if the corresponding .CEL views exist in the design library. The default is off.

Initialize Hierarchy Preservation

Extracts the necessary information from the original hierarchical netlist and write it to the flattened top cell. The information is represented in the form of an object record in the Milkyway database. The default is on.

Print out single pin net

Prints out a single pin net in the graphics window. The default is off.

Handle Dirty Netlist

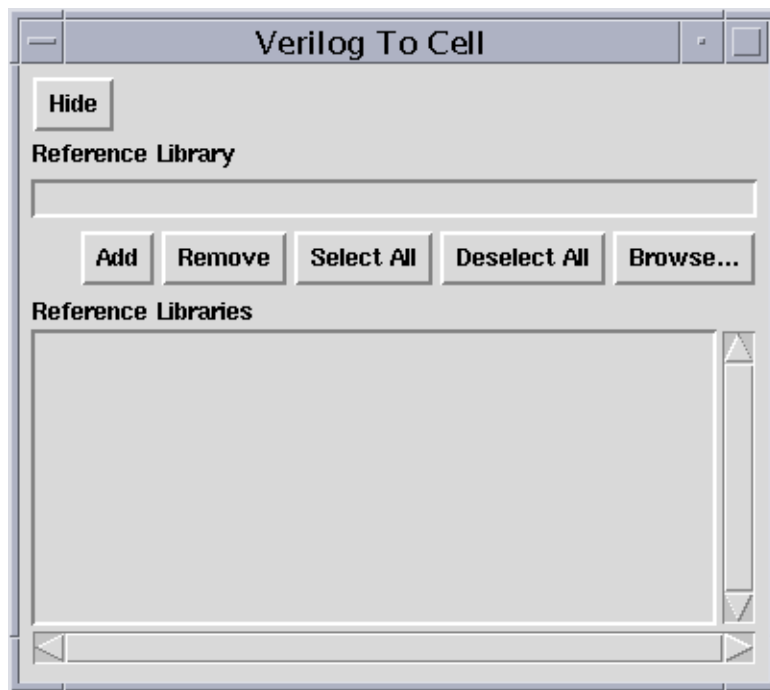
Select this option if you want the Verilog reader to handle “dirty” netlists. A dirty netlist is a netlist that contains, for example, floating pins, nets and pins (or ports) that are a mismatch

between the Verilog netlist and Milkyway libraries. When the Verilog reader reads in a dirty netlist, it checks for the following missing or incomplete information, issues warning messages, and then creates objects that allow the tool to continue. The default is off.

Missing or Incomplete Information	Action ...
Master cell is missing	Creates a new cell in the current library.
Mismatch of cell instance master with reference cell	Creates a new cell.
Missing cell in reference library	If there is no cell instance definition in the netlist or referenced cell in the reference library, Verilog input creates an empty cell. These empty cells are not tagged as leaf cells and therefore, no cell instances are created during expansion. This in turn, can cause a binding error since binding stops when there are no cell instances. In this case, the Verilog reader creates a cell instance if there is an empty cell in the NETL view under tolerant mode.
Missing port declaration	Creates a new port.
Missing port instance	Creates a new cell instance master and new cell instance.
Missing cell instance	Creates a new cell instance.

Reference Library

Opens the Verilog To Cell (Reference Library) dialog box. Its options are described below.



- Reference Library

You can select reference libraries manually by specifying each individual filename, or from a list of files (select Browse).

- Add

Select the required reference libraries from the Reference Libraries list at the top right of the panel and click the Add button.

- Remove

If you wish to remove a reference library in the Selected Reference Libraries list, simply select it and click the Remove button.

- Reference Libraries

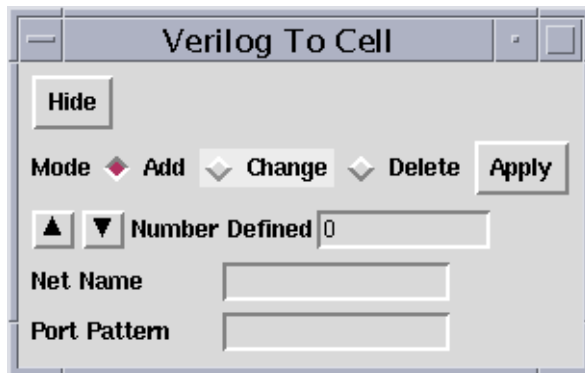
The selected libraries are added to the current list of libraries in the Selected Reference Libraries list near the bottom of the panel.

Note:

You can double click on a Library Name in the Reference Libraries list to add it.

Global Net Options

Opens the Verilog To Cell (Global Net Options) dialog box. Its options are described below.



Mode

After you enter a net name and a port pattern, select the appropriate action you want to take and click Apply.

- Add (the default)

Connects ports and nets that match the port pattern to the net specified by the net name. If the net does not exist, Milkyway creates it.

- Change

Changes previously defined connections for the net specified by the net name.

- Delete

Removes previously defined connections for the net specified by the net name.

Number Defined

Click the up or down arrows to display the nets for which you have defined connections.

Net Name

Enter the name of the top-level global net for which you want to define connections. If the net does not exist, Milkyway creates it.

Port Pattern

Enter a port pattern to match the names of the ports and nets that you want to connect to the net specified in the Net Name text field.

- If the object is a port, the tool connects it to the specified net only if it is not connected to another net.
- If the object is a net, the tool connects it to the specified net only if it is inside a cell instance and is not connected to a port.

Open Library and Cell When Done

Allows the library and cell to remain open after the cell is created in the CEL view.

- If selected, keeps both the library and cell open after cell creation.
- If not selected, keeps both the library and cell closed after cell creation (default).

Verilog IEEE 2001

If selected, Milkyway reads in the Verilog netlist using version Verilog IEEE 2001. The default version is Verilog IEEE 1995.

Limitation:

Verilog 2001 multidimensional arrays are not supported in this release. If you have a netlist that contains multidimensional arrays, `auVerilogToCell` will terminate with an error. You must invoke `auVerilogToCell` with a modified netlist that has all multidimensional arrays converted to single dimensional arrays.

auVhdlIn

Translates a netlist from VHDL format to Synopsys format, creating a library (if the library does not already exist) and a netlist cell.

Tools and Menu Commands

Milkyway: Netlist In > VHDL In

Prerequisites

If you want to translate multiple VHDL files, create a VHDL list file, which contains the names of the files you want to translate. If the files are not in the directory where you started Milkyway, include the full path. The format of the file is:

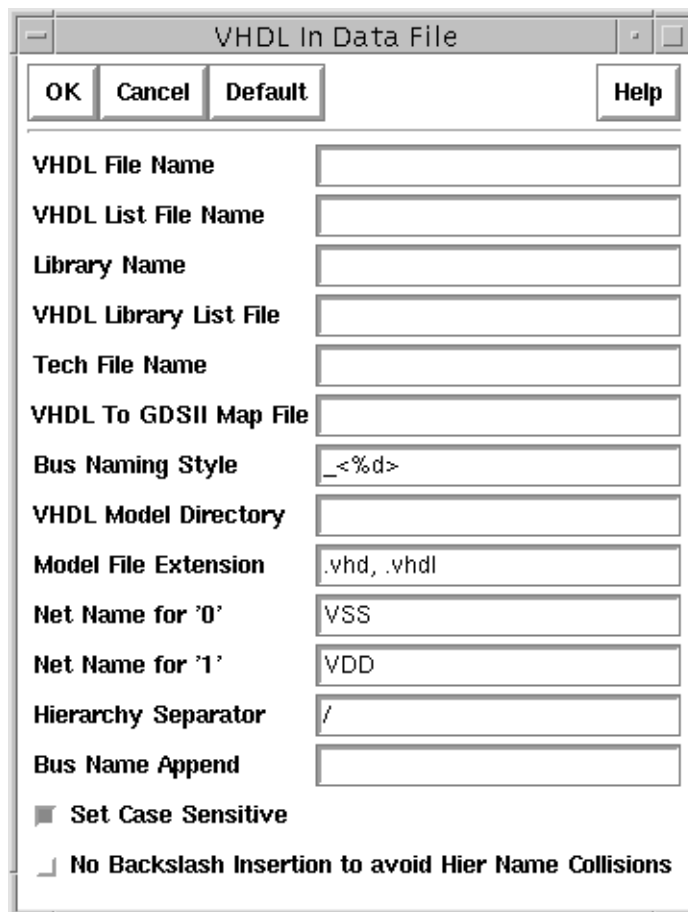
```
fileName1 fileName2 ... fileNameN
```

where *fileNames* are the names (including the paths, if necessary) of the files.

If you want to specify a Scheme for mapping VHDL cell names to GDSII cell names, create a VHDL To GDSII map file. For information about creating a VHDL To GDSII map file.

VHDL In Data File Dialog Box

The menu command opens the following dialog box. Its options are described below.



VHDL In Data File

OK Cancel Default Help

VHDL File Name

VHDL List File Name

Library Name

VHDL Library List File

Tech File Name

VHDL To GDSII Map File

Bus Naming Style _<%d>

VHDL Model Directory

Model File Extension .vhd, .vhdl

Net Name for '0' VSS

Net Name for '1' VDD

Hierarchy Separator /

Bus Name Append

☒ Set Case Sensitive

☐ No Backslash Insertion to avoid Hier Name Collisions

VHDL File Name

Type the name of the VHDL file you want to translate, or leave blank if you have a VHDL list file containing file names for multiple files. If the file is not in the directory where you started Milkyway, you need to include the path.

VHDL List File Name

Type the name of the VHDL list file containing the names of multiple files to be translated, or leave blank if you are translating a single source file specified by *VHDL File Name*.

Library Name

Type the name of the library in which you want to place the netlist cell. If the library does not exist, Milkyway creates it.

Note:

If you do not type a Library Name, Milkyway creates a library named `vhdldefault`.

VHDL Library List File

Optionally, you can type the name of the VHDL Library List File. This file contains a list of mappings from VHDL library names to file paths. VHDL library names are names in library clauses of VHDL source codes. The corresponding file paths store *vtip*-analyzed *vhdlview* files.

Three most important libraries are “work”, “ieee”, and “std”. The “ieee” and “std” libraries store *vtip*-analyzed IEEE standard packages. The “work” library stores *vhdlview* files for currently analyzed VHDL source codes.

The following is an example of VHDL Library List File content:

```
work ./vhdldefault.vtip
ieee avant-root/etc/vtiplib
reference1 ./reference1
reference2 ./reference2
```

Without an input VHDL Library List File, VHDL-In uses the following default libraries:

```
work LibraryName.vtip
ieee avant-root/etc/vhdllibs
std avant-root/etc/vhdllibs
where LibraryName is the user input Library Name
```

Tech File Name

Type the name of the technology file you want to load to the library. If the library already exists, Milkyway ignores this option and you can leave this field blank.

VHDL To GDSII Map File

Type the name of the file you want Milkyway to use to map cell names during translation, or leave blank to use VHDL cell names. For information about creating a VHDL To GDSII map file, see “Specifying Cell Name Mapping” on page 5-3 of the *Milkyway Data Preparation User Guide*.

Bus Naming Style

Type any string you want Milkyway to append to the bus name specified in the VHDL file when it translates the bus to individual ports, using *%d* to indicate the port number. The requirements are as follows:

- The string cannot contain any blanks.
- The string can contain only one *%d*.
- To specify a percent (%) in the string, use *%%*.

For example, if the VHDL file defines a bus as:

```
sys: inout bit_vector (1 to 4)
```

and you specify a *Bus Naming Style* of *_<%%>%d>*, Milkyway translates the bus defined by this statement as ports *sys_<%1>*, *sys_<%2>*, *sys_<%3>*, and *sys_<%4>*.

Note:

After you import the netlist, you can redefine the bus using *aprCreateBus*.

VHDL Model Directory

Type the name of the directory containing the VHDL module files, or leave blank if the modules are included in the file specified by *VHDL File Name*.

Model File Extension

Type the extensions used in the names of module files, or leave blank if the modules are included in the file specified by *VHDL File Name*. Separate two or more extensions with commas.

Net Name for '0'

Type the name you want to assign to the 1-bit 0 signal in the VHDL file.

Net Name for '1'

Type the name you want to assign to the 1-bit 1 signal in the VHDL file.

Hierarchy Separator

Type the character you want to use as a separator in hierarchical names. If this character is used in cell instance or net names, Milkyway translates the name with a backslash (\) preceding the hierarchy separator character.

Bus Name Append

Type any string you want Milkyway to insert into the bus name specified in the VHDL file when it translates the bus to individual ports. The translator inserts the string before the left bracket '(' in each bus name. Only [auVhdlOut](#) removes the inserted string.

For example, if you enter "Bus" in the *Bus Name Append* field, a bus in the VHDL file called "hif_rdata(1)" is translated as "hif_rdataBus(1)," which distinguishes VHDL net and bus files for the Synopsys tool.

Caution!

If you append the bus names you will have serious back annotation problems, because the bus names will be different in the original netlist and the Synopsys tool's data base.

Set Case Sensitive

Deselect if you do not want the library created to be case sensitive. Libraries are case sensitive by default. If a library already exists, this option will not overwrite the existing case sensitive setting.

No Backslash Insertion to avoid Hier Name Collisions

Select if you do not want Milkyway to insert a backslash when translating cell instance or net names to avoid confusion with the backslash used as hierarchy separators.

auVhdlOut

Translates connectivity information contained in a Synopsys top-level cell to a file in VHDL format.

Tools and Menu Commands

Milkyway: Output > VHDL Out

VHDL Out Dialog Box

The menu command opens the following dialog box. Its options are described below.

VHDL Out

OK Cancel Default Help

Library Name

Cell Name

VHDL Out Data File

☐ Output Top Cell Only

☐ No Empty Cell Module Definitions

☒ Output User-Specified Libraries

☒ Output Component Statements for Library Cells

☒ No Corner Pad Instances

☒ No Pad Filler Cell Instances

☒ No Core Filler Cell Instances

☐ No Unconnected Instances ☐ No Unconnected Ports

☐ No P/G nets ☐ No P/G ports

Cell Rename Map File

☒ Output Wire Declaration

☒ Output '1' for Power(VDD, vdd, ...) and '0' for Ground(VSS, gnd, ...)

☒ Output Bus As Individual Bits

Reference Top NETL Cell To Output Bus

☒ Strip BackSlash Before Hierarchy Separator

Library Name

Type the name of the library containing the cell for which you want to export connectivity information.

Cell Name

Type the name of the top-level cell for which you want to export connectivity information.

VHDL Out Data File

Type the name of the file to which you want to write the connectivity information.

Output Top Cell Only

Select to output top cell information only.

No Empty Cell Module Definitions

Select if you do not want Milkyway to translate empty cells.

Output User-Specified Libraries

When selected, the tool outputs user-specified libraries in the beginning of entity declaration.

User-specified libraries means the design libraries except ieee and std. If this option is selected, the library clauses and use clauses are output. Otherwise, the user-specified libraries are not output.

For example, the following is the VHDL program read into [auVhdlIn](#). Note that library lib is a user-specified library, and comp is a package containing types, components, procedures, and functions. Suppose CUB_CELL is a component of package comp.

```

library lib; -- user-specified library clause
use lib.comp.all; -- user-specified use clause
library ieee;
use ieee.std_logic_1164.all;
entity CELL_000 is
port (
A : in std_logic;
Y : out std_logic;
I : out std_logic;
O : out std_logic
);
end CELL_000;
architecture STRUCTURE of CELL_000 is
signal SIG_001 : std_logic;
signal SIG_002 : std_logic;
signal SIG_003 : std_logic;
signal SIG_004 : std_logic;
begin
INSTANCE_000_001: SUB_CELL port map (SIG_001, SIG_002);
INSTANCE_000_002: SUB_CELL port map (SIG_002, SIG_003);
end STRUCTURE;

```

Output Component Statements for Library Cells

When selected, the tool outputs components for library cells in the architecture body.

A component instantiation statement contains a label, a component name, and signal names. If the corresponding architecture of this statement contains no statements, this component is treated as a library cell. For any non-empty architectures, the information of this component should be written out. If the component statements for library cells are written out, the duplication of component definition may cause a problem for some simulators.

In the VHDL program above, the first component instantiation statement, labeled INSTANCE_000_001, shows that the signals SIG_001 and SIG_002 are connected to the ports of a SUB_CELL component. Suppose the component SUB_CELL is a library cell.

The following VHDL program is the output program when this option is selected. The component SUB_CELL in the architecture CELL_000 is the component definition for library cells. If this option is selected, the component statements for library cells are still written in the architecture body. Otherwise, the component statements are not written.

```

library ieee;
use ieee.std_logic_1164.all;
library lib;
use lib.comp.ALL;
entity SUB_CELL is
port (
A : in std_logic;
Y : out std_logic);
end SUB_CELL;
architecture SUB_CELL of SUB_CELL is
begin
end SUB_CELL;
library ieee;
use ieee.std_logic_1164.all;
library lib;
use lib.comp.ALL;
entity CELL_000 is
port (
A : in std_logic;
Y : out std_logic;
I : out std_logic;
O : out std_logic);
end CELL_000;
architecture CELL_000 of CELL_000 is
signal SIG_001 : std_logic;
signal SIG_002 : std_logic;
signal SIG_003 : std_logic;
component SUB_CEL - this is a port ( -- component A : in
std_logic; Y : out std_logic); -- definition for end
component;
-- library cell begin
INSTANCE_000_002 :SUB_CELL port map (A => SIG_002, Y =>
SIG_003);
INSTANCE_000_001 :SUB_CELL port map (A => SIG_001, Y =>
SIG_002);
end CELL_000;

```

No Corner Pad Instances

Select if you do not want Milkyway to translate corner pad cell instances.

No Pad Filler Cell Instances

Select if you do not want Milkyway to translate filler cell instances.

No Core Filler Cell Instances

Select if you do not want Milkyway to translate core filler cell instances.

No Unconnected Instances

Select if you do not want Milkyway to translate unconnected instances.

No Unconnected Ports

Select if you do not want Milkyway to translate unconnected ports. If this option is not selected, unconnected ports are generated and new nets are created to connect with the ports.

No P/G nets

Select if you do not want Milkyway to translate power and ground nets.

No P/G ports

Select if you do not want Milkyway to translate power and ground ports.

Cell Rename Map File

Type the name of the cell mapping file you want Milkyway to use to map cell names during translation, or leave blank to use GDSII cell names. If you used a VHDL to GDSII map file to map cell names when you imported your netlist, use this same file to map

names back to VHDL. For more information, see “Specifying Cell Name Mapping” on page 5-3 of the *Milkyway Data Preparation User Guide*.

Output Wire Declaration

Select to add wire information to the output.

Output ‘1’ for Power (VDD, vdd, ...) and Output ‘0’ for Ground (VSS, gnd, ...)

Select to output power nets as ‘1’ and ground nets as ‘0.’ For example, if you have power net VDD connected to *port1*, and ground net VSS connected to *port2*

And the Option is ...	Milkyway outputs the nets as ...
Selected	port1 <= ‘1’ ; port2 <= ‘0’ ;
Not selected	port1 <= VDD ; port2 <= VSS ;

When this option is selected, ‘1’ and ‘0’ are created at the cell instantiation section and no net declaration is created for them.

Output Bus As Individual Bits

Select to output buses as individual bits. For example, if you have a bus with bits A(0), A(1), and A(2):

And the Option is ...	Milkyway outputs the bus as ...
Selected	port (A_<0> : in bit; /*Individual Bits*/ A_<1> : in bit; /*Individual Bits*/ A_<2> : in bit; /*Individual Bits*/)

And the Option is ...	Milkyway outputs the bus as ...
Not selected	<pre>port (A : in bit_vector (0 to 2) /*As a Bus*/)</pre>

Reference Top NETL Cell To Output Bus

Type the name of the .NETL cell which contains the bus information you want to output.

Very often, a hierarchical .NETL cell is expanded and then bound to a top cell, usually with a different name for placement and routing. Because the bus information is stored in the .NETL cells, you need to specify the original top .NETL cell name when outputting bus information.

Hierarchical VHDL output is also supported. When generating a hierarchical output, the netlist can be a .NETL view or a .HNET view. The .HNET view can be created using the [auCreateHierNetlist](#) command, and needs to be performed prior to hier VHDL out. An example of a “flattened” VHDL output and a “hierarchical” VHDL output is shown below:

(flattened output)

```
auVhdlOut
setFormField "VHDL Out" "Library Name" "design"
setFormField "VHDL Out" "Cell Name" "top_rt.CEL"
setFormField "VHDL Out" "VHDL Out Data File" "lu.vhdl"
setFormField "VHDL Out" "Reference Top NETL Cell To Output
Bus"
"core.NETL"
formOK "VHDL Out"
```

(hier output)

```

auVhdlOut
setFormField "VHDL Out" "Library Name" "design"
setFormField "VHDL Out" "Cell Name" "hv.HNET"
setFormField "VHDL Out" "VHDL Out Data File" "luv.vhdl"
formOK "VHDL Out"
setFormField "VHDL Out" "Output '1' for Power(VDD, vdd,
...) and
'0' for Ground(VSS, gnd, ...) " "0"
setFormField "VHDL Out" "No P/G ports" "1"
setFormField "VHDL Out" "Reference Top NETL Cell To Output
Bus"
"core.NETL"
formOK "VHDL Out"

```

Strip BackSlash Before Hierarchy Separator

When selected, the tool removes backslashes that appear before hierarchy separators for every identifier, including ports, nets, and instances.

15

awe Commands for Milkyway

This chapter describes commands with the prefix *awe*. Use the *awe* commands to set Asymptotic Waveform Evaluation (AWE) information.

- [aweSetOrder](#)

For more timing commands, refer to commands with the following prefixes:

- clf (cell library format) [clf Commands for Milkyway](#)
- ata (timing) [ata Commands for Astro Family Products](#)
- tdf (top design format) [tdf Commands for Milkyway](#)

aweSetOrder

Sets the asymptotic waveform evaluation (AWE) algorithm delay calculation order.

Tool

Astro

Syntax

```
aweSetOrder order
```

where the argument is as follows:

<i>order</i>	An integer. Valid values: Normally, not greater than four.
--------------	--

Note:

The program calculates delay start from first order, then up. For small nets, it usually converges at second order. For a larger net it can go to a higher order. Control is not normally required unless a first order limit is required.

17

ax Commands for Milkyway

This chapter describes the following command with the prefix `ax`:

- `axCreateContactArray`
- `axCreateRowRecord`
- `axCreateTilePatternRecord`
- `axCreateTrackRecord`
- `axCreateWireDirRecord`
- `axSetUnitTileWidth`

axCreateContactArray

Creates (or adds) contact instance arrays in a design. Contacts are predefined in your technology file. For more information, see the “ContactCode” section in the technology file.

Syntax

```
axCreateContactArray cellId "netName" contactNumber  
transform flag Point xPitch yPitch xTimes yTimes BitMap
```

where the arguments are as follows:

cellId

Specifies the database object ID of the cell to which the information applies.

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

netName

Specifies the name of the net on which you want to add the contact array.

Valid values:

Any net in the open cell

contactNumber

Specifies the number you specified in the deviceTable section of the technology file that identifies the contact you want to create

Valid values:

Any integer from 1 to 255 used in the deviceTable section to identify a contact. For the following deviceTable section, the contact number for the contact, polymet1, is 1.

```
*deviceTable
*contact 1 polymet1
cont poly m1 1 1 1.2 2.5 2.0
defaultRowNumber 2
parasitics 0.2 0.0 0.0
*end
```

transform

Specifies whether or not to rotate all contacts in the array by 90 degrees.

Valid values:

- 0, which means the contacts are not rotated
- 1, which means the contacts are rotated 90 degrees

flag

Specifies the desired flag value.

Valid values:

Positive integers

Point

Specifies the point corresponding to the center of the contact array.

Valid syntax:

(x y)

xPitch

Specifies the center-to-center spacing of the contact cuts in the horizontal direction.

Valid values:

Number of units to define the x-dimension spacing of your array

yPitch

Specifies the center-to-center spacing of the contact cuts in the vertical direction.

Valid values:

Number of units to define the y-dimension spacing of your array

xTimes

Specifies the number of sites created in the x-direction.

yTimes

Specifies the number of sites created in the y-direction.

bitmap

If the contact array is full, there is no need to use *bitmap*; that is, *bitmap* is NULL. Otherwise, a row-wise bit representation is used to indicate that some contact in the contact array is empty when the bit is 0. An int32 can hold 32 bits. So the size of the bitmap can be determined by

```
tmp = sizeof (unit32);
```

```
mapSize = (bitMap?((xTimes*yTimes -1)/(8*tmp)+1)*tmp:0);
```

axCreateRowRecord

Creates a row record in the database.

Syntax

```
axCreateRowRecord cellId direction flip "tilePatternName"  
listLL listUR
```

where the arguments are as follows:

cellId

Specifies the database object ID of the cell to which the information applies.

Valid values:

- Cell ID of any open cell in the current library
- (`geGetEditCell`)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

direction

Specifies the preferred direction.

Valid values:

- 1, which means vertical
- 2, which means horizontal

flip

Specifies whether the tile will be flipped or not. 0 means not flipped.

tilePatternName

Specifies the name of the tile pattern in the row.

Valid values:

Any tile pattern name from the open cell whose tile pattern number you can get from the name

listLL

Specifies the coordinate that indicates the left segment [extends low] of the bounding box.

Valid syntax:

'(x y)

listUR

Specifies the coordinate that indicates the upper right of the bounding box.

Valid syntax:

'(x y)

axCreateTilePatternRecord

Creates a tile pattern record in the open cell.

Syntax

```
axCreateTilePatternRecord cellId "tileName" "patternName"  
direction flip
```

where the arguments are as follows:

cellId

Specifies the database object ID of the cell to which the information applies.

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

tileName

Specifies the tile name in the open cell.

Valid values:

If not specified (that is, == NULL), the default “unit” will be used.

patternName

Specifies the pattern name in the row.

Valid values:

Any pattern name from the open cell whose tile pattern number you can get from the name

direction

Specifies the preferred direction.

Valid values:

- 1, which means vertical
- 0, which means horizontal

flip

Specifies whether the tile will be flipped or not. 0 means not flipped.

axCreateTrackRecord

Creates a track record in the open cell.

Syntax

```
axCreateTrackRecord cellId layerNumber direction startTrack  
step xxtimes listLL listUR
```

where the arguments are as follows:

cellId

Specifies the database object ID of the cell to which the information applies.

Valid values:

- Cell ID of any open cell in the current library
- (*geGetEditCell*)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

layerNumber

Specifies the number of the cell layer to which the rectangle information applies.

Valid values:

Positive integers (0 to 256)

direction

Specifies the preferred direction.

Valid values:

- #f, which means vertical
- #t, which means horizontal

startTrack

Specifies the number of the start track.

step

Specifies the distance between each track.

xxtimes

Specifies the number of tracks created.

listLL

Specifies the coordinate that indicates the left segment [extends low] of the bounding box.

Valid syntax:

'(x y)

listUR

Specifies the coordinate that indicates the upper right of the bounding box.

Valid syntax:

'(x y)

Example

```
axCreateTrackRecord _cell 2 #t 0.32 0.64 15 '(0.0000 0.3200)
' (0.6400 9.2800)
```

axCreateWireDirRecord

Creates a wire direction record in the open cell.

Syntax

```
axCreateWireDirRecord cellId layer direction
```

where the arguments are as follows:

cellId

Specifies the database object ID of the cell to which the information applies.

Valid values:

- Cell ID of any open cell in the current library
- (*geGetEditCell*)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

layer

Specifies the number of cell layers to which the rectangle information applies.

Valid values:

Positive integers (0 to 256)

direction

Specifies the preferred direction.

Valid values:

- #f, which means vertical

- #t, which means horizontal

Example

```
axCreateWireDirRecord _cell 32 #t
```

axSetUnitTileWidth

Sets the width of a unit tile cell to be used by the [axgPlanner](#) command.

Tool

Milkyway

Syntax

```
axSetUnitTileWidth "libName" "tileName" width
```

where the arguments are as follows:

<i>libName</i>	The name of the current working library.
<i>tileName</i>	The filename of the unit tile to be used by the axgPlanner command.
<i>width</i>	The desired width of the unit tile cell you want to set in user units. Valid values: Any positive integer.

Example 1

```
axSetUnitTileWidth "lib/stdCell" "unit" 1.8
```

See Also

[axgPlanner](#) – Performs design setup on the open cell according to your specifications.

19

axg Commands for Milkyway

This chapter describes commands with the prefix *axg*. The commands are listed alphabetically. Most of these commands are used for Astro and Jupiter floorplanning.

- [axgCheckWireTrack](#)
- [axgDefineWireTracks](#)
- [axgDisplayCellWireTracks](#)
- [axgDisplayDesignWireTracks](#)
- [axgGenerateInstName](#)

axgCheckWireTrack

Displays information in the messages area and the log file about the number of pins not on wire tracks and the list of such pins. The information should look something like the following:

```
#BEGIN_CHECK_WIRETRACK
```

```
Total number of pins without on-track routeability: 1 (out of 9999)
```

```
List of pins with bad routeability
```

Cell Name	Pin Name	Layer	Direction	Track
ADD_123	Pin2	M1	Input	H&V

```
Check Wire Track successful.
```

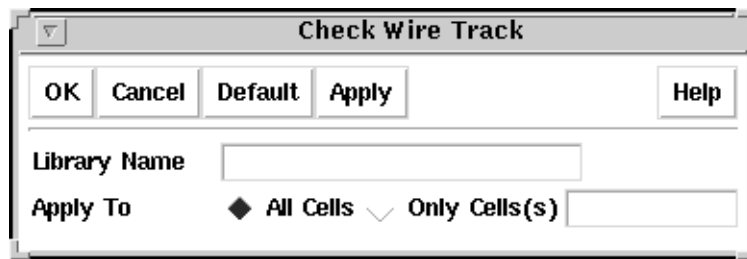
```
#END_CHECK_WIRETRACK
```

Tools and Menu Commands

Milkyway: Design Setup > Wire Tracks – Check Pin on Track

Check Wire Track Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Enter the name of the library that contains the cells to check.

Note:

The library does not have to be open.

Apply To

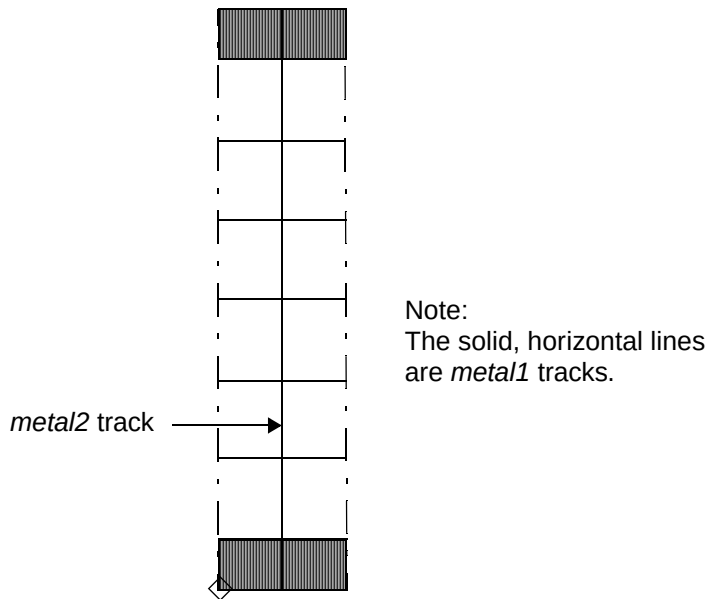
Indicate the cells to check for pins not on wire tracks.

Option	Checks ...
All Cells	all cells in all pin and blockage cells (<i>.fram</i>) in the library.
Only Cell(s)	only the cells you specify by typing the cell names. To specify multiple cell names, separate the names with commas.

axgDefineWireTracks

Creates wire tracks in the unit tile. The unit tile cell must contain wire tracks, that assists the router in placing wires for each of the routing layers.

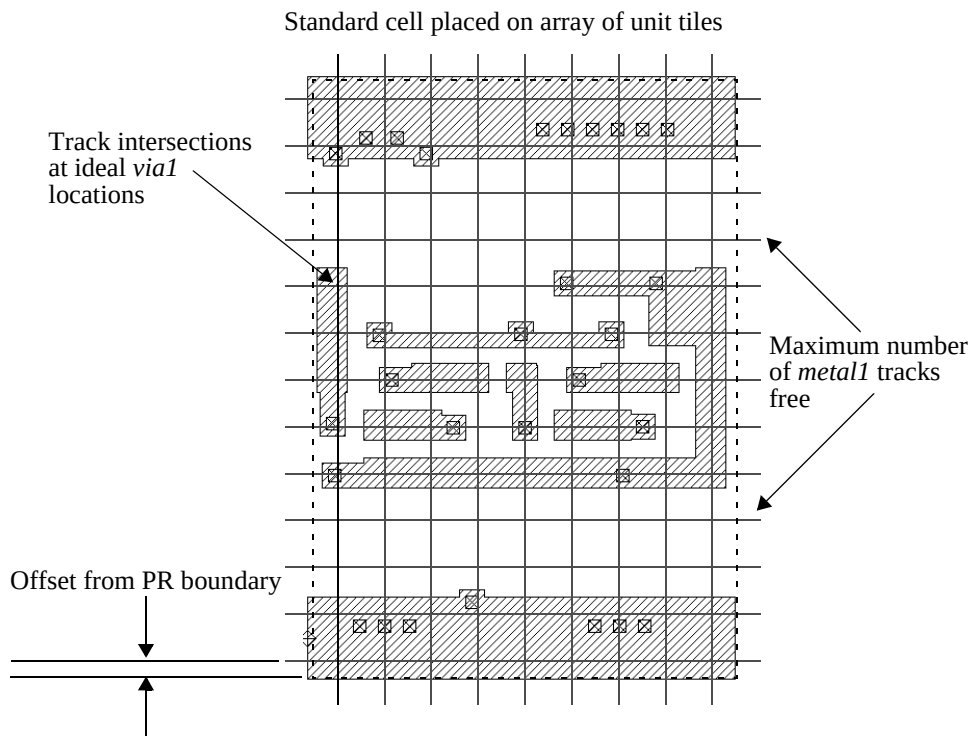
Each layer has its own set of wire tracks, that run in the preferred direction for that layer. For example, wire tracks on metal1 run along the horizontal axis. The following is an example of a unit tile with the wire tracks defined.



Use the following guidelines to ensure that the wire tracks are created to maximize routing:

- The width of the unit tile should be the *metal2* pitch.
- The distance from the bottom of the unit tile to the first wire track must be zero or one half the *metal1* pitch. Otherwise, when you use the *Double Back* option in the [axgPlanner](#) command, Astro leaves space between the cell rows, depending on the size of the space between the top track and the top of the unit tile or the bottom track and the bottom of the unit tile, and the *rowSpacing* rule in the technical file.
- If there are *metal1* routing areas inside the cell, try to maximize the number of *metal1* tracks available for routing.
- Via1 locations should intersect as many tracks as possible.

- Generally, the wire track on *metal2* should be centered vertically in the unit tile cell, although there are libraries in which the *metal2* wire track is aligned with the left boundary.



Tools and Menu Commands

Astro: Design Setup > Wire Tracks – Define Unit Tile Wire Tracks

Jupiter: Astro Floorplanning: Design Setup > Wire Tracks – Define Unit Tile Wire Tracks

Prerequisites

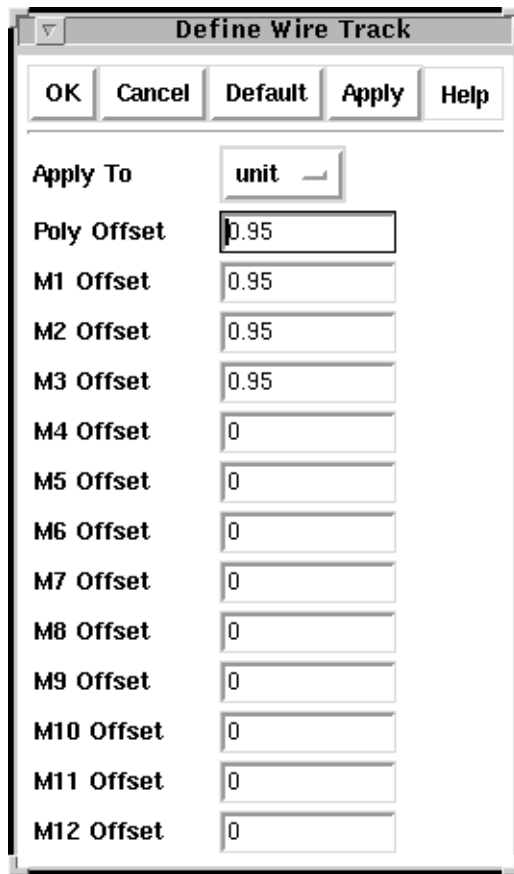
Create the unit tile, using the [auSetPRBdry](#) command and open the library that contains the unit tile.

Note:

If you select this command and the current library does not contain a unit tile cell, Astro searches the reference libraries for a unit tile cell and asks if you want to copy the unit tile cell to the current library.

Define Wire Track Dialog Box

The menu command opens the following dialog box. Its options are described below.



The image shows a dialog box titled "Define Wire Track". It has a standard Windows-style title bar with a close button. Below the title bar are five buttons: "OK", "Cancel", "Default", "Apply", and "Help". The main area of the dialog contains a list of settings. At the top, there is a label "Apply To" followed by a dropdown menu currently showing "unit". Below this are twelve rows, each with a label and a text input field. The labels are "Poly Offset", "M1 Offset", "M2 Offset", "M3 Offset", "M4 Offset", "M5 Offset", "M6 Offset", "M7 Offset", "M8 Offset", "M9 Offset", "M10 Offset", "M11 Offset", and "M12 Offset". The input fields for "Poly Offset", "M1 Offset", "M2 Offset", and "M3 Offset" all contain the value "0.95". The input fields for "M4 Offset" through "M12 Offset" all contain the value "0".

Label	Value
Apply To	unit
Poly Offset	0.95
M1 Offset	0.95
M2 Offset	0.95
M3 Offset	0.95
M4 Offset	0
M5 Offset	0
M6 Offset	0
M7 Offset	0
M8 Offset	0
M9 Offset	0
M10 Offset	0
M11 Offset	0
M12 Offset	0

Apply To

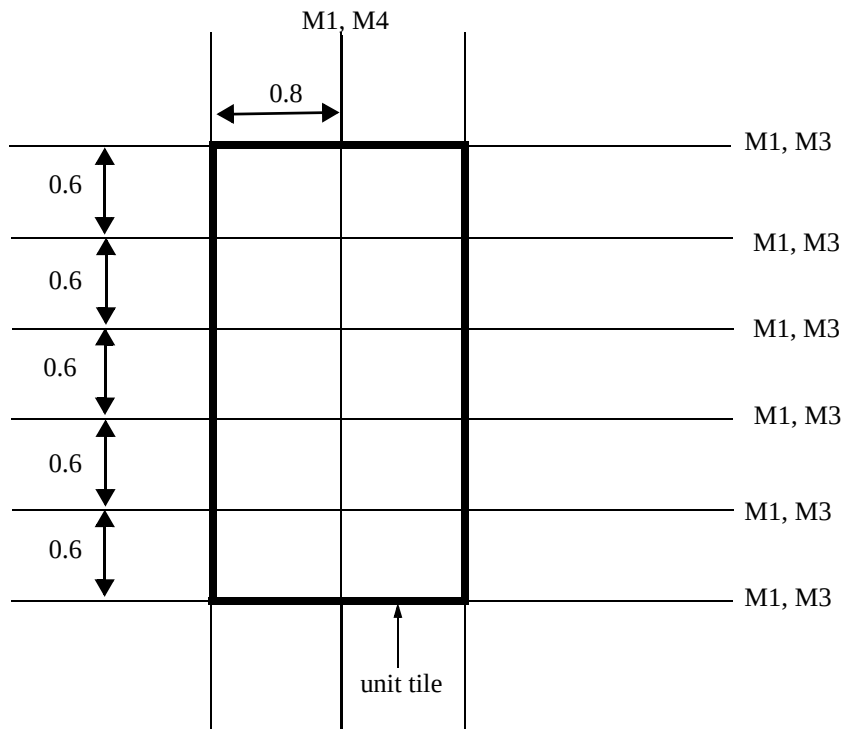
Select either unit, unit1 or specified. If you have more than one unit tile defined in the technical file, select the unit tile in which to define wire tracks.

Note:

If you are using multiple unit tiles, define the wire tracks for each of the unit tiles.

Layer Offset

Enter the distance from the lower, left corner of the unit tile cell to the first track on each layer. Wire tracks after the first track are spaced according to the layer pitch defined in the technical file. For example, if the M1 pitch is 0.6 microns and the *M2 Offset* value is 0.8 microns, the wire tracks resemble the following:



The default spacing between the first wire track and the second wire track (and all additional wire tracks) is the pitch specified for the layer in the technical file.

You can also select the preferred wire track direction after each layer offset line to either “Horizontal” or “Vertical.”

Note:

The layers listed in the technical file appear next to Offset in the dialog box.

Astro calculates a vertical track offset of zero or one-half the pitch, that generally maximizes routing. You should set a horizontal offset relative to the PR boundary. If you use the Double Back option in the [axgPlanner](#) command, the distance from the bottom of the unit tile cell to the first wire track must be zero or one half the metal1 pitch.

See Also

[axgCheckWireTrack](#)

[axgDisplayCellWireTracks](#)

[axgDisplayDesignWireTracks](#)

axgDisplayCellWireTracks

Displays wire tracks defined in the unit tile cell in a standard cell library. Use this command to check the relationship between pins and wire tracks to ensure that the pins are on or close to the wire tracks.

If you made changes to the unit tile cell and did not close the Display Cell Wire Track dialog box, you can use the *Load* button to view the changes you made.

Tools and Menu Commands

Astro: Design Setup > Wire Tracks – Display Cell Wire Tracks

Jupiter: Astro Floorplanning: Design Setup > Wire Tracks – Display Cell Wire Tracks

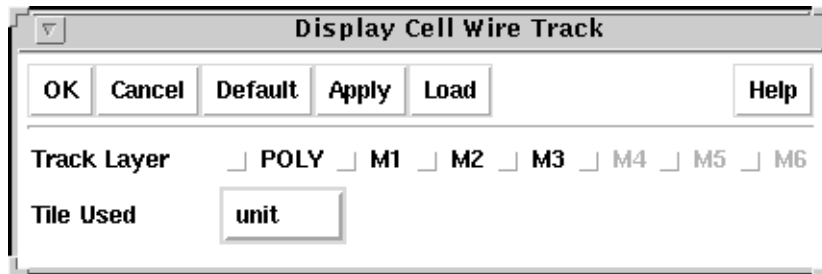
Prerequisites

Define the wire tracks in the unit tile cell, using the [“axgDefineWireTracks” on page 19-3](#) command.

Open a standard cell before you use this command.

Display Cell Wire Track Dialog Box

The menu command opens the following dialog box. Its options are described below.



Track Layer

Select the layers that contain the wire tracks you want to see.

Tile Used

Select the unit tile to use. The default is *unit*. If you have defined multiple tiles in the technical file, they appear in the *Tile Used* list.

axgDisplayDesignWireTracks

Displays wire tracks in the design. Use this command after running the design setup to view the wire tracks.

After you make the changes to the Display Design Wire Track dialog box and select the *Apply* button, use *Redraw* to view the wire tracks.

Tools and Menu Commands

Astro: Design Setup > Wire Tracks – Display Design Wire Tracks

Jupiter: Astro Floorplanning: Design Setup > Wire Tracks – Display Design Wire Tracks

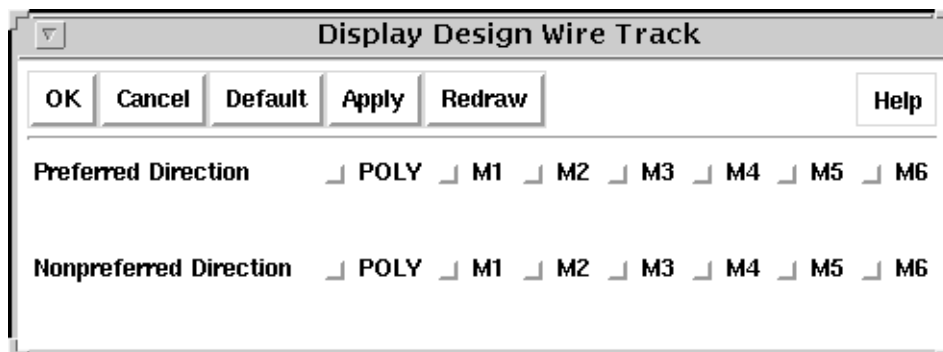
Milkyway: Wire Tracks – Display Design Wire Tracks

Prerequisites

When you change the wire tracks in a unit tile cell, using the [“axgDefineWireTracks” on page 19-3](#) command, regenerate the design setup to view the new wire tracks in the design.

Display Design Wire Track Dialog Box

The menu command opens the following dialog box. Its options are described below.



Preferred Direction

Select the layers to show the wire tracks on the layers. These wire tracks display in the direction of a layer. For example, wire tracks on metal1 display horizontally.

Nonpreferred Direction

Select the layers to show the wire tracks on the layers. These wire tracks display in the non preferred direction of a layer. For example, wire tracks on metal1 display vertically.

axgGenerateInstName

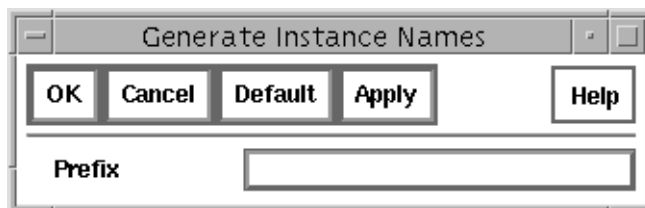
Assigns names to unnamed cell instances.

Tool

Astro

ECO Status Dialog Box

The menu command opens the following dialog box. Its options are described below.



Prefix

Enter the new cell instance name prefix. Cell instances with a “ “ name are renamed to “prefix_n,” where n is a number assigned by the tool.

21

clf Commands for Milkyway

This chapter describes commands with the prefix *clf*. The commands are listed alphabetically.

- [clfCreateTable](#)
- [clfCreateVectorTable](#)
- [clfDefineDesignCellName](#)
- [clfDump](#)
- [clfDumpAntennaProp](#)
- [clfDumpStaticNoiseMargin](#)
- [clfDumpStaticNoiseProperty](#)
- [clfLoad](#)
- [clfPurgeBooleanFunction](#)

- `clfPurgePad`

clfCreateTable

Creates a timing table to show the effects of input transition time or output capacitance on one of the following timing characteristics of an arc:

- Propagation delay when the output signal is rising
- Propagation delay when the output signal is falling
- Transition delay when the output signal is rising
- Transition delay when the output signal is falling
- Cell delay when the output signal is rising
- Cell delay when the output signal is falling

Also creates a power table to show the effects of input and output conditions on internal power, leakage power, and switching power.

Alternatively, the timing table can show the effects of transition delays at a constrained pin (for example, the data input pin) or transition delays at a related pin (for example, the clock pin) on one of the following timing characteristics of an arc:

- Delay constraint when the output signal is rising
- Delay constraint when the output signal is falling

The *defineTimeTLU* function (see [defineTimeTLU](#)) references timing tables created by *clfCreateTable* functions to characterize the arc between two ports.

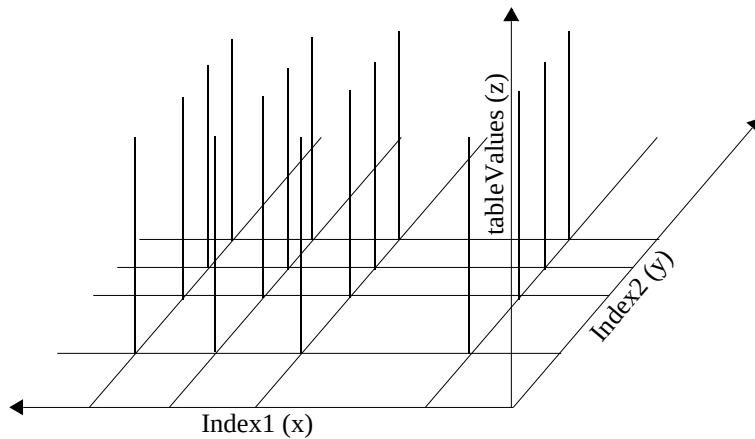
For more information about power-driven layouts, see the *Astro-Rail User Guide*.

Tool

Astro

Syntax

```
clfCreateTable "tableName" "tableType" '(["index1Type"  
(index1Values)]) ["index2Type" (index2Values)])  
'(tableValues)
```



Note:

For scalar type tables (in other words, to specify a constant delay), the syntax is as follows:

```
clfCreateTable "tableName" "tableType" '() '(tableValue)
```

where the arguments are as follows:

<i>tableName</i>	Name you want to assign to the timing table you are creating.
------------------	---

tableType

Type of timing table you are creating.

Valid values:

- *RisePropagationTable* indicates that the table defines the propagation delay of an arc when the output signal is rising.
- *FallPropagationTable* indicates that the table defines the propagation delay of an arc when the output signal is falling.
- *RiseTransitionTable* indicates that the table defines the transition delay of an arc when the output signal is rising.
- *FallTransitionTable* indicates that the table defines the transition delay of an arc when the output signal is falling.
- *RiseCellDelayTable* indicates that the table defines the cell delay of an arc when the output signal is rising.
- *FallCellDelayTable* indicates that the table defines the cell delay of an arc when the output signal is falling.
- *RiseConstraintTable* indicates that the table defines the delay constraint on an arc when the output signal is rising.
- *FallConstraintTable* indicates that the table defines the delay constraint on an arc when the output signal is falling.
- *InternalPowerTable* indicates that the table defines the power consumed when the cell inputs switch but the outputs do not.
- *SwitchingPowerTable* indicates that the table defines the power consumed by the outputs switching in response to input switching.
- *LeakagePowerTable* indicates that the table defines the power dissipated by leakage currents.

indexType

Index type you want to use in the table. To specify a one-dimensional table, specify only *index1Type*; to create a two-dimensional table, specify both *index1Type* and *index2Type*.

Valid values:

- If the *tableType* is *RisePropagationTable*, *FallPropagationTable*, *RiseTransitionTable*, *FallTransitionTable*, *RiseCellDelayTable*, *FallCellDelayTable*, *InternalPowerTable*, *LeakagePowerTable*, or *SwitchingPowerTable*, the index type(s) can be one or both of the following:
- *InputNetTransition* indicates that the related *indexValues* are input transition times to be included in the table.
- *OutputCapacitance* indicates that the related *indexValues* are output capacitances to be included in the table.
- If the *tableType* is *RiseConstraintTable* or *FallConstraintTable*, the index type(s) can be one or both of the following:
- *ConstrainedPinTransition* indicates that the related *indexValues* are transition delays at the constrained pin to be included in the table.
- *RelatedPinTransition* indicates that the related *indexValues* are transition delays at the related pin to be included in the table.

indexValues

Values to be included in the table for each index.

tableValues Delays or delay constraints (as specified by *tableType*) resulting from each *indexType* and its *indexValues* (for a one-dimensional table) or pair of *indexTypes* and their *indexValues* (for a two-dimensional table).

- If you are creating a scalar type table, this is expressed as follows:

'(tableValue)

where the list contains a single value that expresses a constant delay.

- If you are creating a one-dimensional table, this is expressed as follows:

'(indexTypeValue1TableValue
indexTypeValue2TableValue
... indexTypeValueiTableValue)

where the values in the list are the delays or delay constraints for the *indexValues* given for the specified *indexType*, respectively.

- If you are creating a two-dimensional table, this is expressed as follows:

<i>' (indexType1Value1/indexType2Value1TableValue</i>	}	first set of lines
<i>indexType1Value1/indexType2Value2TableValue</i>		
<i>:</i>		
<i>indexType1Value1/indexType2ValueiTableValue</i>	}	second set of lines
<i>indexType1Value2/indexType2Value1TableValue</i>		
<i>indexType1Value2/indexType2Value2TableValue</i>		
<i>:</i>		
<i>indexType1Value2/indexType2ValueiTableValue</i>		
<i>:</i>		
<i>:</i>		
<i>indexType1Valuei/indexType2Value1TableValue</i>	}	last set of lines
<i>indexType1Valuei/indexType2Value2TableValue</i>		
<i>:</i>		
<i>indexType1Valuei/indexType2ValueiTableValue)</i>		

where the *tableValues* in the list are the delays or delay constraints for the pairs of *indexValues* given for the *indexTypes*, in the order shown.

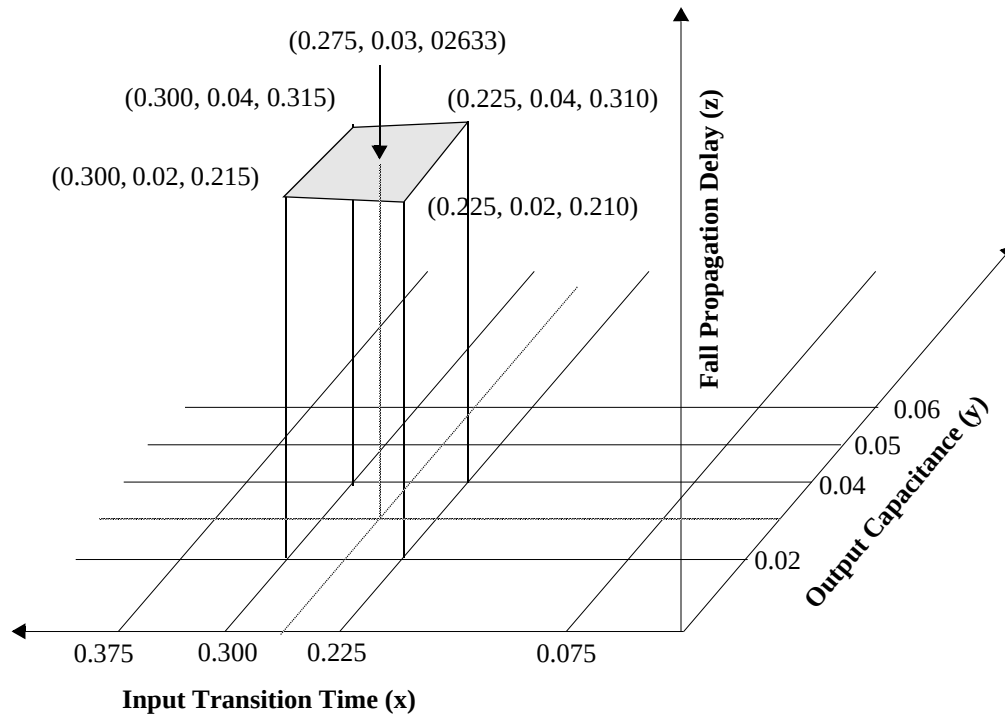
- The *tableValues* shown in the first set of lines are for the pairing of the first *indexValue* given for the first *indexType* given with each of the *indexValues* given for the second *indexType* given (in the respective order).
- The *tableValues* shown in the second set of lines are for the pairing of the second *indexValue* given for the first *indexType* given with each of the *indexValues* given for the second *indexType* given (in the respective order).
- The *tableValues* shown in the last set of lines are for the pairing of the last *indexValue* given for the first *indexType* given with each of the *indexValues* given for the second *indexType* given (in the respective order).

Example 1

```
clfCreateTable "table1" "FallPropagationTable"
'(("InputNetTransition" (0.075 0.225 0.300 0.375))
 ("OutputCapacitance" (0.02 0.04 0.05 0.06))
 '(0.200 0.300 0.350 0.400 0.210 0.310 0.360 0.410 0.215
 0.315 0.365 0.415 0.220 0.320 0.370 0.420))
```

This *clfCreateTable* function creates a two-dimensional fall propagation table *table1*.

Output Capacitance Æ Input Transition Time Ø	0.02	0.04	0.05	0.06
0.075	0.200	0.300	0.350	0.400
0.225	0.210	0.310	0.360	0.410
0.300	0.215	0.315	0.365	0.415
0.375	0.220	0.320	0.370	0.420



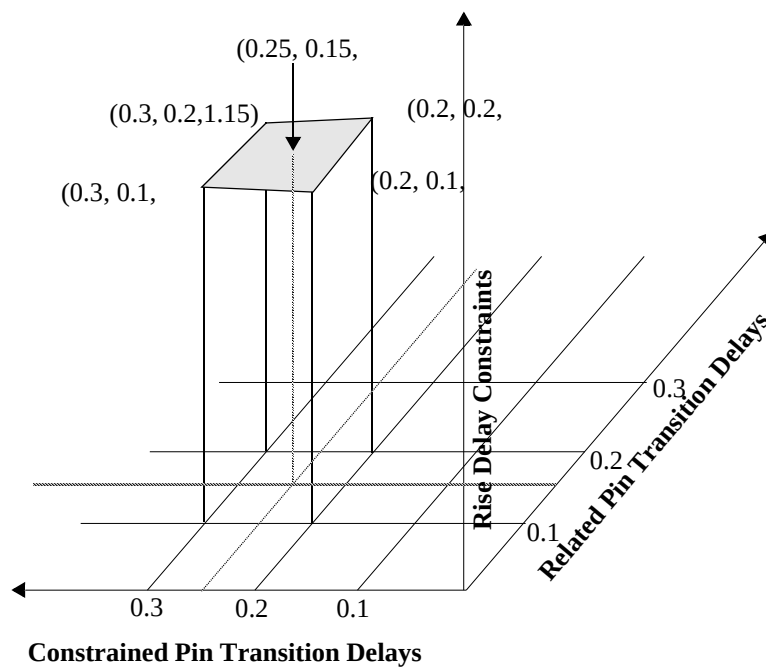
By interpolation, Astro determines the fall propagation delay for any input transition time and output capacitance based on the values in the table. For example, if the input transition time is 0.275 and the output capacitance is 0.03, the fall propagation delay is 0.2633.

Example 2

```
clfCreateTable "table2" "RiseConstraintTable"
'(("ConstrainedPinTransition" (0.1 0.2 0.3))
("RelatedPinTransition" (0.1 0.2 0.3))
'(0.05 0.75 0.95 0.25 0.95 1.21 0.55 1.15 1.50)
```

This *clfCreateTable* function creates a two-dimensional rise constraint table *table2*.

Related Pin Transition ΔE Constrained Pin Transition	0.1	0.2	0.3
0.1	0.05	0.75	0.95
0.2	0.25	0.95	1.21
0.3	0.55	1.15	1.50



By interpolation, Astro can determine the rise delay constraint for any constrained pin transition delay and related pin transition delay based on the values in the table. For example, if the constrained pin transition delay is 0.25 and the related pin transition delay is 0.15, the rise delay constraint is 0.725.

See Also

[auDefln](#) – Writes the CLF information associated with a library to a file. (Displays a dialog box.)

[auLoadCLF](#)– Loads a CLF file into a library you specify.

clfCreateVectorTable

Tabulates a standard cell's current as a function of the time, supply voltage, load capacitance, and transition time and stores the waveform data in the WAVE view.

Tool

Tool based on Milkyway

Syntax

```
clfCreateVectorTable < string tableName >  
  < string tableType >  
  < list familyOfIndices >  
  < list familyOfVectorLists >
```

tableName

Specifies the unique table name (which should be unique among all tables created by the `clfCreateVectorTable` and `clfCreateTable` commands).

tableType

Specifies the case-insensitive table type. `WaveformTable` is currently the only supported type.

familyOfIndices

Specifies the family (list) of indexes. There must be at least one and at most three indexes. Each index contains an index type and a list of knots.

One gate current waveform is created for each knot combination, which is formed by taking one knot from each index. Each index is a list with the following syntax:

```
< string indexType > < list listOfKnots >
```

The `indexType` argument specifies the case-insensitive index type. The following types are permitted:

- `SupplyVoltage`
- `InputNetTransition`
- `OutputCapacitance`
- `RisePropagationTable`
- `FallPropagationTable`
- `RiseTransitionTable`
- `FallTransitionTable`
- `RiseConstraintTable`
- `FallConstraintTable`
- `RiseCellDelayTable`
- `FallCellDelayTable`
- `SwitchingPowerTable`
- `InternalPowerTable`
- `LeakagePowerTable`
- `WaveformTable`

The `listOfKnots` argument specifies the list of knots. Each knot is a real number.

familyOfVectorLists

Specifies the family (list) of vector lists with at least 1 and at most 16,000 vector lists. However, the actual number of vector lists must match the number of waveforms specified in `familyOfIndices`.

Each vector list describes the coordinates of these waveforms and can have a varying number of vectors (which is helpful in the context of the Miller effect). However, it must have at least 1 and at most 65,536 vectors.

Each vector is also a list of coordinates (real numbers) having a dimension (number of coordinates) between 1 and 16. Vectors within the same vector list must have the same dimensions.

Description

The `clfCreateVectorTable` command tabulates a standard cell's current as a function of the time, supply voltage, load capacitance, and transition time and stores the waveform data in the WAVE view.

Note:

The `clfCreateVectorTable` command is an IC layout command and can be used only within a CLF load file (loaded using the `auLoadCLF` command). You should not call `clfCreateVectorTable` directly in the Milkyway Scheme window.

The `clfCreateVectorTable` command is similar to the `clfCreateTable` IC layout command. However, whereas `clfCreateTable` tabulates scalar quantities, `clfCreateVectorTable` tabulates vector quantities.

One gate current waveform is created for each knot combination, which is formed by taking one knot from each index. Each waveform is a piecewise linear graph with time on the horizontal axis and current on the vertical axis. Its coordinates are specified in the family of vector lists. Each waveform must have a corresponding vector list in the family of vector lists.

Example

The following example is an excerpt from a CLF load file:

```
(
  clfCreateVectorTable
  "tablexyz"
  "WaveformTable"
  '(
    ("SupplyVoltage" (2.3 2.4 2.5 2.6))
    ("InputNetTransition" (0.2 2 4))
    ("OutputCapacitance" (0.01 0.1 1))
  )
  '(
    ((0 0) (1.03 0) (2.97 11.1) (5.00 0) (7.01 0))
    ((0 0) (1.05 0) (2.97 11.2) (4.98 0) (7.02 0))
    ((0 0) (1.08 0) (3.00 11.4) (4.96 0) (7.03 0))
    .
    (36 waveforms in all)
    .
    ((0 0) (1.00 0) (3.00 11.0) (5.00 0) (7.00 0))
  )
)
```

In this example, the parts of the syntax are as follows:

```
tableName = "tablexyz"
tableType = "WaveformTable"
familyOfIndices=' ("SupplyVoltage" (2.3 2.4 2.5 2.6))
                  ("InputNetTransition" (0.2 2 4))
                  ("OutputCapacitance" (0.01 0.1 1))
                )
familyOfVectorLists ='( ((0 0) (1.03 0) (2.97 11.1) (5.00 0) (7.01 0))
                        ((0 0) (1.05 0) (2.97 11.2) (4.98 0) (7.02 0))
                        ((0 0) (1.08 0) (3.00 11.4) (4.96 0) (7.03 0))
                        .
                        (36 waveforms in all)
                        .

```

```
((0 0) (1.00 0) (3.00 11.0) (5.00 0) (7.00 0))  
)
```

Family of Indexes

In the previous example, the first index is as follows:

```
("SupplyVoltage" (2.3 2.4 2.5 2.6))
```

The first index contains the following objects:

"SupplyVoltage"

The "SupplyVoltage" object specifies the index type. It indicates that the gate current waveform is dependent on the supply voltage, V. Milkyway picks up the unit precision for voltage from the technology file.

(2.3, 2.4, 2.5, 2.6)

The (2.3, 2.4, 2.5, 2.6) object specifies the list of knots. The numbers 2.3, 2.4, 2.5, and 2.6 are the knots.

The second index is as follows:

```
("InputNetTransition" (0.2 2 4))
```

The second index contains the following:

"InputNetTransition"

The "InputNetTransition" string specifies the index type. It indicates that the gate current waveform is dependent on the transition time, $T_{\text{transition}}$. Milkyway picks up the unit precision for time from the technology file.

(0.2 2 4)

The (0.2 2 4) object specifies the list of knots. The numbers 0.2, 2, and 4 are the knots.

The last index is as follows:

```
("OutputCapacitance" (0.01 0.1 1))
```

The last index contains the following:

"OutputCapacitance"

The "OutputCapacitance" string specifies the index type. It indicates that the gate current waveform is dependent on the output capacitance, C_{output} . Milkyway picks up the unit precision for capacitance from the technology file.

(0.01 0.1 1)

The (0.01 0.1 1) object specifies the list of knots. The numbers 0.01, 0.1, and 1 are the knots.

One gate current waveform is created for each combination of knots, which is formed by taking one knot from each index, as shown (there are a total of 36 waveforms):

Waveform 1 : $V = 2.3$, $T_{\text{transition}} = 0.2$, $C_{\text{output}} = 0.01$

Waveform 2 : $V = 2.3$, $T_{\text{transition}} = 0.2$, $C_{\text{output}} = 0.1$

Waveform 3 : $V = 2.3$, $T_{\text{transition}} = 0.2$, $C_{\text{output}} = 1$

Waveform 4 : $V = 2.3$, $T_{\text{transition}} = 0.2$, $C_{\text{output}} = 0.01$

Waveform 5 : $V = 2.3$, $T_{\text{transition}} = 0.2$, $C_{\text{output}} = 0.1$

Waveform 6 : $V = 2.3$, $T_{\text{transition}} = 0.2$, $C_{\text{output}} = 1$

.
. .
.

Waveform 36 : $V = 2.6$, $T_{\text{transition}} = 4$, $C_{\text{output}} = 1$

Family of Vector Lists

In the family of vector lists, each vector list represents the vertices of one gate current waveform graphed with time on the horizontal and current on the vertical axis.

In the example earlier in this section, the first vector list describes waveform 1 with the following vertices:

O	: (0 ns, 0 nA)
A ₁	: (1.03 ns, 0 nA)
B ₁	: (2.97 ns, 11.1 nA)
C ₁	: (5.00 ns, 0 nA)
D ₁	: (7.01 ns, 0 nA)

Similarly, the second vector list describes waveform 2 with the following vertices:

O	: (0 ns, 0 nA)
A ₂	: (1.05 ns, 0 nA)
B ₂	: (2.97 ns, 11.2 nA)
C ₂	: (4.98 ns, 0 nA)

D₂ : (7.02 ns, 0 nA)

Proceeding in this manner, the last vector list describes waveform 36 with these vertices:

O : (0 ns, 0 nA)

A36 : (1.00 ns, 0 nA)

B36 : (3.00 ns, 11.0 nA)

C36 : (5.00 ns, 0 nA)

D36 : (7.00 ns, 0 nA):

The family of indexes has 36 waveforms, so the family of vector lists has 36 vector lists.

clfDefineDesignCellName

Defines the CLF design cell name.

Syntax

```
clfDefineDesignCellName <string designCellName>
```

where the argument is as follows:

<code>designCellName</code>	The design cell name.
-----------------------------	-----------------------

Description

The `clfDefineDesignCellName` command defines the CLF design cell name.

This is a CLF command and should only be called from within a CLF file (which is loaded using the `auLoadCLF` command).

Example

```
clfDefineDesignCellName "myDesign"
```

clfDump

Writes the Cell Library Format (CLF) information associated with a library to a file.

Note:

Writes only CLF functions to the file. Although you can include a `dbSetCellPortTypes` function in a CLF file, it is a database function, not a CLF function.

Syntax

```
clfDump libID fileName
```

where the arguments are as follows:

libID

ID of the library for which you want CLF information

Valid values:

ID of any library in the current directory. Either of the following can be substituted for *libID*:

- `(dbGetCurrentLibId)`
- If a cell is open in your library,
`(dbGeLibIdByCellId (geGetEditCell))`

fileName

Name of the file to which you want to write the information

Valid values:

Any string of up to 32 characters

Example

```
clfDump (dbOpenLib "demo") "demo.clf"
```

Typing this function from the input area of the Milkyway window writes the CLF information associated with the library demo to a file named demo.clf.

See Also

[auDefIn](#) writes the CLF information associated with a library to a file. (This command displays a dialog box.)

[auLoadCLF](#) loads a CLF file into a library you specify.

clfDumpAntennaProp

Dumps antenna properties of a library to a CLF file. The file can then be modified and loaded with [auLoadCLF](#).

Tool

Astro

Syntax

```
clfDumpAntennaProp libId "fileName"
```

where the arguments are as follows:

<i>libId</i>	ID of the library for which you want to dump antenna property. Valid values: ID of an open library. If the library is open, you can substitute the following for <i>libID</i> : (<i>dbGetCurrentLibId</i>)
<i>fileName</i>	Name of the file.

Example

```
clfDumpAntennaProp (dbGetCurrentLibId) "antenna.dump"
```

clfDumpStaticNoiseMargin

Dumps the noise margin values stored in a library to a file. The library has to be opened before executing this command.

Tool

Astro

Syntax

```
clfDumpStaticNoiseMargin libId "fileName"
```

where the arguments are as follows:

<i>libId</i>	ID of the library for which you want to dump noise margin values. Valid values: ID of an open library. If the library is open, you can substitute the following for <i>libID</i> : (<i>dbGetCurrentLibId</i>)
<i>fileName</i>	Name of the file.

Example

```
clfDumpStaticNoiseMargin (dbGetCurrentLibId)
"noisemargin.dump"
```

clfDumpStaticNoiseProperty

Dumps the noise margin values stored in a library currently opened to a file named "file name." The library has to be opened before executing this command.

Tool

Milkyway

Syntax

```
clfDumpStaticNoiseProperty dbGetCurrentLibId "file name"
```

where the arguments are as follows:

<i>dbGetCurrentLibId</i>	ID of the library for which you want to dump noise margin values. Valid values: ID of an open library. If the library is open, you can substitute the following for <i>libID</i> : (<i>dbGetCurrentLibId</i>)
<i>file name</i>	Name of the file.

clfLoad

Loads a CLF file into a library.

Syntax

```
clfLoad libId "worstclfname" "typicalclfname" "bestclfname"  
boolean
```

where the arguments are as follows:

<i>libId</i>	The ID of the library to which you want to load the CLF file.
<i>worstclfname</i>	Type the name of the worst case CLF file to be loaded.
<i>typicalclfname</i>	Type the name of the typical case CLF file to be loaded.
<i>bestclfname</i>	Type the name of the best case CLF file to be loaded.
<i>boolean</i>	Selection for whether to translate delay information specified using the linear delay model to the table lookup delay model when loading the CLF file. Valid values: • #t • #f Note: This option is also available on the Synopsys To CLF Translator dialog box. Whenever a CLF file is generated using auSynopToCLF, #f is entered here, and the option is selected on the Synopsys To CLF Translator dialog box.

See Also

[auLoadCLF](#) for loading a CLF file using the dialog box.

clfPurgeBooleanFunction

Purges all CLF boolean functions in the current library.

Syntax

```
clfPurgeBooleanFunction <void>
```

Description

The `clfPurgeBooleanFunction` command purges all CLF boolean functions in the current library. CLF boolean functions are defined using the `defineBooleanFunction` command.

This is a CLF command and should only be called from within a CLF file (which is loaded using the `auLoadCLF` command).

Example

```
clfPurgeBooleanFunction
```

clfPurgePad

Purges all CLF pad definitions in the current library.

Syntax

```
clfPurgePad <void>
```

Description

The `clfPurgePad` command purges all CLF pad definitions in the current library. CLF pad definitions are defined using the `definePad` command.

This is a CLF command and should only be called from within a CLF file (which is loaded using the `auLoadCLF` command).

Example

```
clfPurgePad
```


22

cm Commands for Milkyway

This chapter describes commands with the prefix *cm*. The commands are listed alphabetically.

- [cmAttachStarRCXT](#)
- [cmCheckCellDRC](#)
- [cmCheckLibrary](#)
- [cmCheckNetList](#)
- [cmCheckTechFile](#)
- [cmCopyLib](#)
- [cmCreateCapModel](#)
- [cmCreateHierPorts \(Unsupported\)](#)
- [cmCreateLib](#)

- `cmDumpHierPorts` (Unsupported)
- `cmDumpPlib`
- `cmDumpPseudoPlib`
- `cmDumpRefLibList`
- `cmDumpTech`
- `cmDumpTLUPlus`
- `cmExecSmash`
- `cmFlattenNetlist`
- `cmItfToTLUPlus`
- `cmLoadTech`
- `cmItfToTLUTest`
- `cmMarkCellType`
- `cmMarkEEQPins`
- `cmMarkLefSite`
- `cmMasterTilePattern`
- `cmPDBIn`
- `cmPurgeTech`
- `cmQueryHierPorts`
- `cmRefLib`
- `cmRenameCell`
- `cmRenameLib`

- `cmReplaceRefLib`
- `cmReplaceTech`
- `cmReplaceTLUPlus`
- `cmSetBusNameStyle`
- `cmSetMultiHeightProperty`
- `cmShowInstOrigin`
- `cmShowRefLib`
- `cmSmash`

cmAttachStarRCXT

Star-RCXT is available from within the Astro environment. Any parasitics generated by Star-RCXT can be written simply and directly into the Milkyway database. This can be done using the `astStarRCXT` command. Prior to executing Star-RCXT, the TCAD information must be added to the library.

Note:

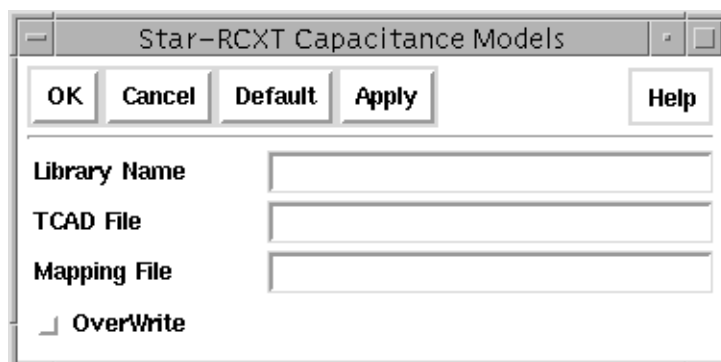
The files will be “moved” into the Milkyway database. The files that are attached will be removed. It is important to make copies of these files before attaching them into the Milkyway database.

Tools and Menu Commands

Milkyway: TechFile > StarRCXT Capacitance Model

Star-RCXT Capacitance Models Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Enter the name of the library.

Min TCAD File

The name of a minimum TCAD file (*nxtgrd*).

Nom TCAD File

The name of a nominal TCAD file (*nxtgrd*).

Max TCAD File

The name of a maximum TCAD file (*nxtgrd*).

Mapping File

The name of the mapping file.

Overwrite

Select this option to overwrite any existing data, and to remove any pre-existing *nxtgrd* TCAD files.

cmCheckCellDRC

This command is used for calling the DRC and checking the net for all cells.

Tool

Milkyway

Syntax

```
cmCheckCellDRC "libName" thinBlockageFlag
```

where the arguments are as follows:

libName

The name of the library containing the net you want to check DRC and cell consistency.

thinBlockageFlag The flag when you have thin blockage.
Valid values:
#t or #f.

cmCheckLibrary

Verifies the following for a specified library:

- That the technology file is loaded and valid.
- That no design rule check (DRC) errors exist in the .FRAM cells of the specified types.

Tools and Menu Commands

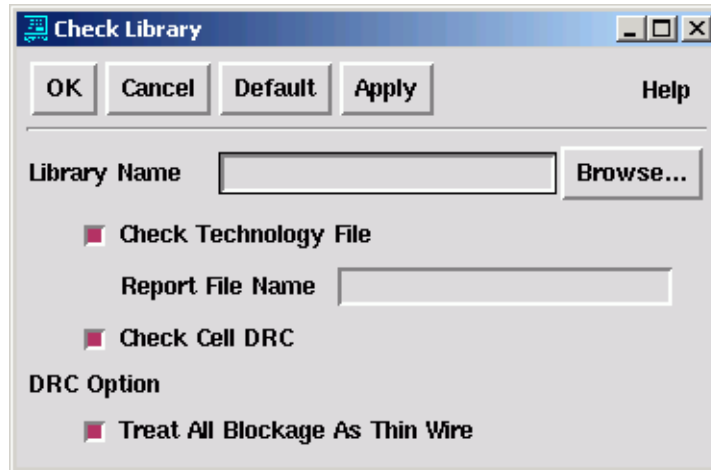
Astro: Library > Check

Milkyway: Library > Check

Jupiter: Astro Floorplanning: Library > Check

Check Library Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Type the name of the library you want to check.

Check Technology file

Select to confirm that the technology file is loaded and valid.

Report File Name

Enter a file name for the check library report.

Check Cell DRC

Select to confirm that .FRAM cell data is complete and valid.

DRC Option – Treat All Blockage As Thin Wire

Select if you want all blockages treated as thin wire instead of classifying them according to their widths.

cmCheckNetList

This command is used for checking the net SCM, pin and cell consistency.

Tool

Milkyway

Syntax

```
cmCheckNetList "libName" "cellName"
```

where the arguments are as follows:

<i>libName</i>	The name of the library containing the net you want to check SCM, pin and cell consistency.
<i>cellName</i>	The name of the cell that you want to check SCM, pin and cell consistency. Valid values: Name of a cell in the specified library.

cmCheckNetlist

Reports the following for the specified top cell:

- Net/port data consistency
- Net, cell, and port statistics
- Net and cell accessibility from I/O pads

Tools and Menu Commands

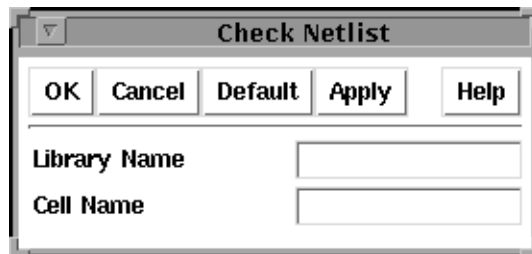
Astro: Design Setup > Netlist - Check Netlist

Jupiter: Astro Floorplanning: Design Setup > Netlist - Check Netlist

Milkyway: Netlist In > Check

Check Netlist Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Type the name of the library containing the cell that you want to check the netlist.

Cell Name

Type the name of the cell that you want to check the netlist.

cmCheckTechFile

This command is used for checking the technical file.

Tool

Milkyway

Syntax

```
cmCheckTechFile "libName"
```

where the argument is as follows:

libName

The name of the library containing the technical file that you want to check.

cmCopyLib

Copies a specified library.

Tools and Menu Commands

Astro: Library > Copy

Jupiter: Library > Copy

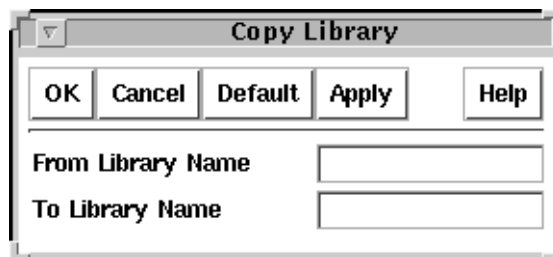
Milkyway: Library > Copy

Prerequisite

You cannot have any libraries open when executing this command.

Copy Library Dialog Box

The menu command opens the following dialog box. Its options are described below.



From Library Name

Type the name of the library you want to copy.

To Library Name

Type the name you want to assign to the copy of the library.

Report Progress

Select if you want your Synopsys tool to display messages about the progress of the copy operation in the messages area of the window.

cmCreateCapModel

Creates capacitance models (specifications) and updates the technology file with the information you provide.

Tools and Menu Commands

Milkyway: Tech File > Create Capacitance Model

Prerequisites

You must have a technology file in the design library.

Create Capacitance Model Dialog Box

The menu command opens the following dialog box. Its options are described below.

Create Capacitance Model

OK Cancel Default Apply Help

Technology Name Library Name

TCAD GRD File Name Read

LPE Mode ☐ MIN ☒ NOM ☐ MAX ☐ delete all capacitance tables

Layer Name	Thickness	Dielectric Const	Layer Name	Thickness	Dielectric Const
Air	0	1.0	Metal 3	0.7	3.9
Passivation I	0	3.9	Dielectric 2 I	0.6	3.9
Passivation II	0	3.9	Dielectric 2 II	0	3.9
Passivation III	0	3.9	Dielectric 2 III	0	3.9
Passivation IV	0	3.9	Dielectric 2 IV	0	3.9
Metal 6	0	3.9	Metal 2	0.7	3.9
Dielectric 5 I	0	3.9	Dielectric 1 I	0.6	3.9
Dielectric 5 II	0	3.9	Dielectric 1 II	0	3.9
Dielectric 5 III	0	3.9	Dielectric 1 III	0	3.9
Dielectric 5 IV	0	3.9	Dielectric 1 IV	0	3.9
Metal 5	0	3.9	Metal 1	0.7	3.9
Dielectric 4 I	0	3.9	Dielectric Poly I	0.6	3.9
Dielectric 4 II	0	3.9	Dielectric Poly II	0	3.9
Dielectric 4 III	0	3.9	Dielectric Poly III	0	3.9
Dielectric 4 IV	0	3.9	Dielectric Poly IV	0	3.9
Metal 4	0	3.9	Poly	0.3	3.9
Dielectric 3 I	0	3.9	Field Oxide	0.3	3.9
Dielectric 3 II	0	3.9			
Dielectric 3 III	0	3.9			
Dielectric 3 IV	0	3.9			

Width and Spacing Table Dimension Additional Layers

Technology Name

Type the name for the technology you are using (e.g. "tsmc025").

Library Name

Type the name for the library containing the technology file you want to update.

TCAD GRD File Name

This feature parses and reads the information stored in the TCAD .GRD file. Type the name for the TCAD .GRD file that contains information to be filled into the Create Capacitance Model dialog box.

Mapping File

Type the name of the Star-RCXT mapping file, which is to map the names of the conducting layers in the .GRD file to those in the Milkyway database. An example of the mapping file is:

```
conducting_layers  
poly cpoly  
metal1 M1
```

Note:

When no mapping file is specified, the .GRD file should use the same naming convention for conducting layers as that of the Milkyway database, such as “poly,” “metal1,” “metal2,” and so on.

LPE Mode

Select *MIN* (minimum) if you are entering minimum values in the dialog box(es).

Select *NOM* (nominal) if you are entering nominal values in the dialog box(es).

Select *MAX* (maximum) if you are entering maximum values in the dialog box(es).

The capacitance table names will reflect the selected mode.

delete all capacitance tables

Select to delete all existing or previously generated capacitance tables. By default, only the tables corresponding to the selected LPE mode will be deleted.

Note:

For the following fields, type the specifications in the appropriate technology file user units.

Layer Name - Thickness

Type the layer thickness for each specified layer. You must specify a non-zero thickness for every conducting layer in the technology file, and for at least one dielectric layer between each pair of conducting layers. The tool will ignore all layers with zero thickness.

Layer Name - Dielectric Const

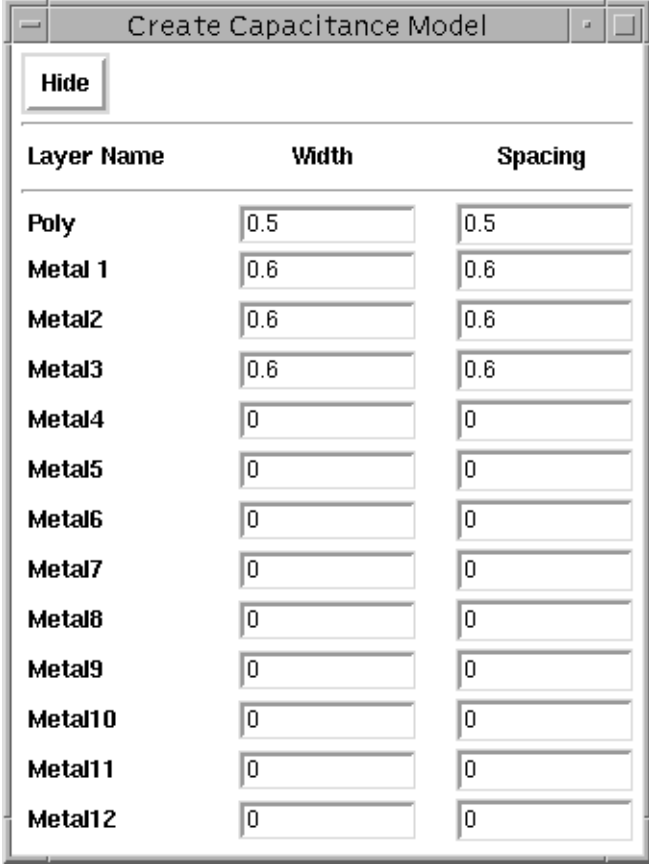
Type the dielectric constant for each specified layer. In conducting layers, the dielectric constant applies to the intra-layer dielectric, such as the dielectric that fills the space when the metal is not present.

Width and Spacing—button

Type the minimum wire width for each poly or metal layer and the minimum distance between neighboring wires for each poly or metal layer.

Width and Spacing Dialog Box

The menu command opens the following dialog box. Its options are described below.



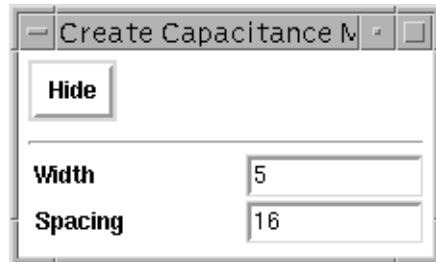
Layer Name	Width	Spacing
Poly	0.5	0.5
Metal 1	0.6	0.6
Metal2	0.6	0.6
Metal3	0.6	0.6
Metal4	0	0
Metal5	0	0
Metal6	0	0
Metal7	0	0
Metal8	0	0
Metal9	0	0
Metal10	0	0
Metal11	0	0
Metal12	0	0

Table Dimension—button

Type the width and spacing dimensions for the capacitance table.

Table Dimension Dialog Box

The menu command opens the following dialog box. Its options are described below.

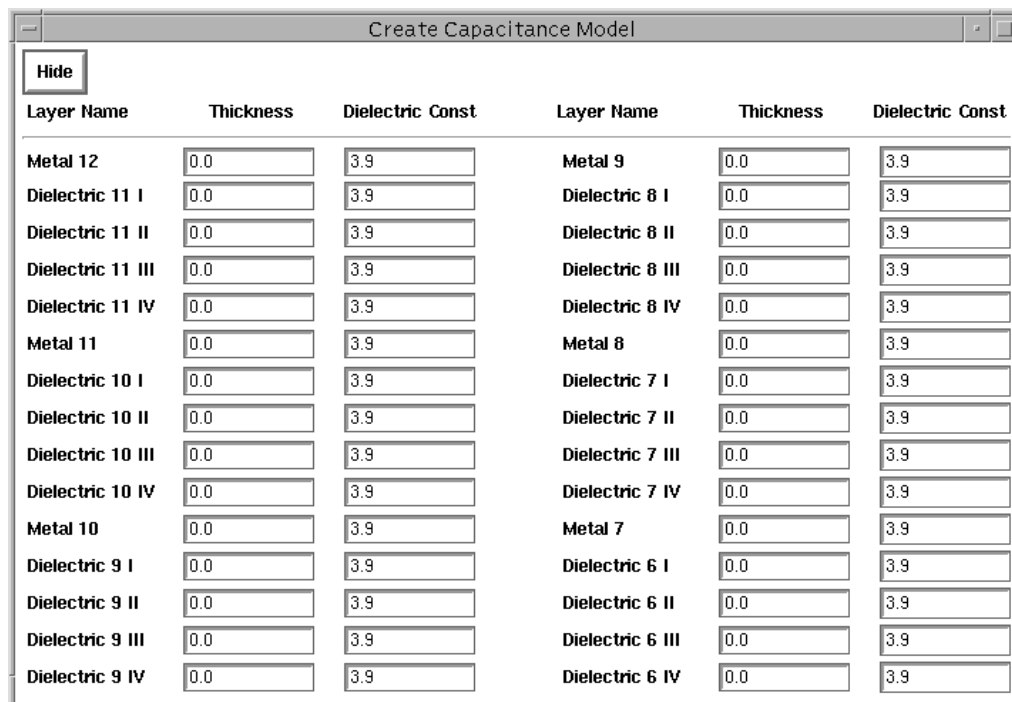


Additional Layers—button

Click to specify information for additional layers, if necessary.

Additional Layer Dialog Box

The menu command opens the following dialog box. Its options are described below.



Layer Name	Thickness	Dielectric Const	Layer Name	Thickness	Dielectric Const
Metal 12	0.0	3.9	Metal 9	0.0	3.9
Dielectric 11 I	0.0	3.9	Dielectric 8 I	0.0	3.9
Dielectric 11 II	0.0	3.9	Dielectric 8 II	0.0	3.9
Dielectric 11 III	0.0	3.9	Dielectric 8 III	0.0	3.9
Dielectric 11 IV	0.0	3.9	Dielectric 8 IV	0.0	3.9
Metal 11	0.0	3.9	Metal 8	0.0	3.9
Dielectric 10 I	0.0	3.9	Dielectric 7 I	0.0	3.9
Dielectric 10 II	0.0	3.9	Dielectric 7 II	0.0	3.9
Dielectric 10 III	0.0	3.9	Dielectric 7 III	0.0	3.9
Dielectric 10 IV	0.0	3.9	Dielectric 7 IV	0.0	3.9
Metal 10	0.0	3.9	Metal 7	0.0	3.9
Dielectric 9 I	0.0	3.9	Dielectric 6 I	0.0	3.9
Dielectric 9 II	0.0	3.9	Dielectric 6 II	0.0	3.9
Dielectric 9 III	0.0	3.9	Dielectric 6 III	0.0	3.9
Dielectric 9 IV	0.0	3.9	Dielectric 6 IV	0.0	3.9

cmCreateHierPorts (Unsupported)

This command is no longer supported. You should use the `astInitHierPreservation` command instead. For more information, see [astInitHierPreservation](#).

If you are using the `cmCreateHierPorts` command (which is still available for backward-compatibility), you begin by building the hierarchical port records and storing them into a Milkyway database (as the Flattened Cell Name's .CEL view). The information will be used for Astro's Timer to handle false path data during the timing analysis task.

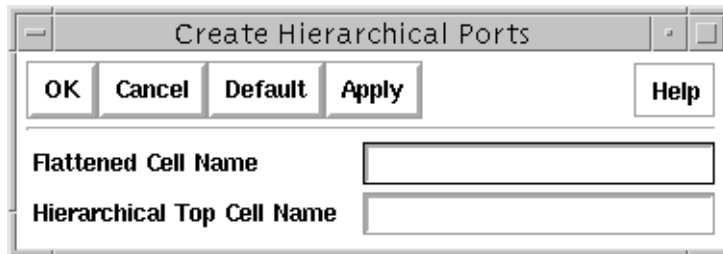
Note:

This command only need be run once as long as there is no change in the netlist (such as .NETL view data). Whenever you perform an ECO on a netlist, it is recommended that you run this command to update the hierarchical port records (previous hierarchical port records are deleted before new ones are created, so there is no danger of conflict.)

If the hierarchical port records are missing in the flattened cell, timing analysis will ignore the false path associated with that hierarchical port.

Create Hierarchical Ports Dialog Box

The menu command opens the following dialog box. Its options are described below.



Flattened Cell Name

Type the name of the cell you wish to store.

Hierarchical Top Cell Name

Type the name you want to assign to the top cell.

cmCreateLib

Creates a library catalog file—which contains the compiled technology file information—and a directory with the library name you specified to hold the cell files.

Tools and Menu Commands

Astro: Library > Create

Jupiter: Library > Create

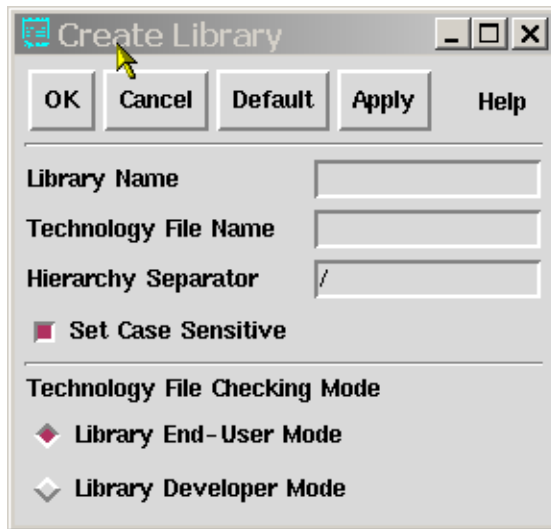
Milkyway: Library > Create

Prerequisites

You must have a technology file for the library.

Create Library Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Type the name you want for the new library. The Synopsys database imposes few restrictions on the names given to objects; however, if you plan to interface with other tools or export data to other formats, you should adhere to the following guidelines when naming your library:

- The first character must be an alphabetic character.
- The remaining characters can be any of the following:
 - Alphabetic characters.
 - Numeric characters.
 - Underscores (_).
- The length must not exceed 31 characters, except for stipple and line style pattern names, which should not exceed 15 characters.

Caution!

Failure to conform to these guidelines can result in the inability to export your data.

Note:

For detailed information about Synopsys database naming restrictions, see “Naming Guidelines” in the *Milkyway Environment Extension Language Reference Manual*.

Technology File Name

Type the name of the technology file you want to associate with the cells in the library.

Hierarchy Separator

Type the character you want to use as a separator in hierarchical names. If this character is used in cell instance or net names, Milkyway translates the name with a backslash (\) preceding the hierarchy separator character.

The following characters are allowed as hierarchy separators in the Milkyway database:

- / (forward slash)
- . (period)
- | (pipe)
- # (pound)
- @ (at)
- ^ (caret)

Set Case Sensitive

Select if you want to create a library having case sensitivity.

Technology Checking Modes

You have the following technology file checking modes to choose from:

- Library End-User Mode

This is the default level. The end-user mode is less restrictive. Use this mode if you receive technology files from a library vendor and want to create design libraries. Errors are reported with less severe labeling; therefore, they are not as useful for debugging.

- Library Developer Mode

This mode is more restrictive. It is for library developers who create technology files before passing them to the end user. This mode provides the maximum level of reported warning messages to help create clean technology files.

cmDumpHierPorts (Unsupported)

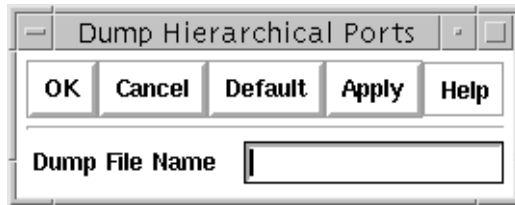
This command is no longer supported. You should use the `astDumpHierPreservation` command instead. For more information, see [astInitHierPreservation](#).

The `cmDumpHierPorts` command is still available for backward-compatibility. The `cmDumpHierPorts` command prompts a dialog box for the output file name, and then dumps all the hierarchical port records into an opened cell. You need to have an opened cell before executing this command.

The format is a sequence of reports as generated by [cmQueryHierPorts](#).

Dump Hierarchical Ports Dialog Box

The menu command opens the following dialog box. Its options are described below.



Dump File Name

Type the name of the File you wish to dump.

cmDumpPlib

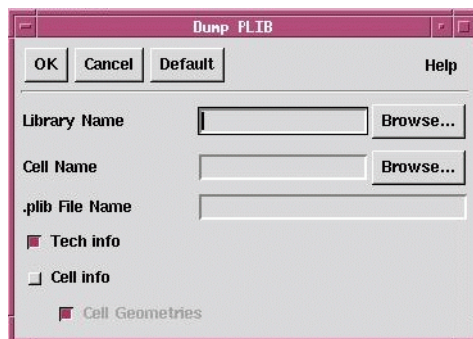
Writes a .plib file from a specified Milkyway library.

Tool and Menu Command

Milkyway: Library > Dump Plib

cmDumpPlib Dialog Box

The command opens the following dialog box. Its options are described below.



Library Name (Required)

The Milkyway library from which to generate the file.

Cell Name (Optional)

The name of the cell from the library.

If this option is not specified, all cells from the library are written out.

.plib File Name (Optional)

The name of the generated .plib file.

Tech info (Optional)

This option is selected by default. In the default mode, Milkyway writes a technology .plib file. You can also

- write a complete .plib file by selecting the Tech info option, the Cell info option, and the Cell Geometries option.
- write a .plib file with technology and cell information, but without geometry information, by selecting the Tech info option and the Cell info option and deselecting the Cell Geometries option.

Cell info

This option is not selected by default. If you select this option, Milkyway writes a .plib file with cell information. You can

- write a .plib file with cell information and geometries, but without technology information, by selecting the Cell info option and deselecting the Tech info option.
- write a .plib file with technology and cell information, but without geometry information, by selecting the Tech info option and the Cell info option and deselecting the Cell Geometries option.
- write a .plib file with cell information but without technology information and cell geometries by selecting the Cell info option and deselecting the Tech info option and the Cell Geometries option.

Cell Geometries

This option is selected by default when you select the Cell info option. In the default mode, Milkyway writes a .plib file with cell info and geometries.

Description

The `cmDumpPlib` command writes a `.plib` file from the specified Milkyway library. The `.plib` file is generated from the Milkyway library's technology file information and its FRAM view cells.

Example

```
cmDumpPlib
setFormField "Dump Plib File" "Plib File Name" "my.plib"
setFormField "Dump Plib File" "Library Name" "myLib"
setFormField "Dump Plib File" "Plib File Name" "test"
formOK "Dump Plib File"
```

cmDumpPseudoPlib

Note:

The `cmDumpPseudoPlib` command has been replaced by `cmDumpPlib`. Use the `cmDumpPlib` command to write a `.plib` file with technology and cell information, by selecting the Tech info option and the Cell info option in the Dump PLIB dialog box. For more information, see [cmDumpPlib](#).

Writes a pseudo `.plib` file from the specified Milkyway library.

Dump PseudoPLIB Dialog Box

The `cmDumpPseudoPlib` options are described below.

Option	Description
<code>.plib</code> File Name	Name of the generated pseudo <code>.plib</code> file.
Library Name	Milkyway library from which to generate the file.

Description

The `cmDumpPseudoPlib` command writes a pseudo `.plib` file from the specified Milkyway library. The pseudo `.plib` file is generated from the Milkyway library's technology file information and its FRAM view cells; however, its geometry and pin data are omitted.

Example

```
cmDumpPseudoPlib
setFormField "Dump PseudoPlib File" "PPlib File Name"
    "my.pplib"
setFormField "Dump PseudoPlib File" "Library Name" "myLib"
formOK "Dump Plib File"
```

cmDumpRefLibList

This command is used to list all the reference libraries into one file.

Tool

Milkyway

Syntax

```
cmDumpRefLibList "libName" "fileName"
```

where the arguments are as follows:

<i>libName</i>	The name of the library that information will be listed.
<i>fileName</i>	The name of the file that you want the library information written.

cmDumpTech

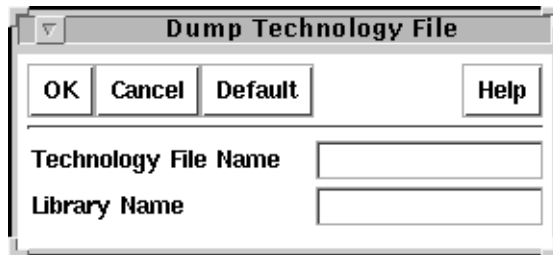
Writes the technology information associated with a library to an editable technology file.

Tools and Menu Commands

Milkyway: Tech File > Write To File

Dump Technology File Dialog Box

The menu command opens the following dialog box. Its options are described below.



Technology File Name

Type the name you want to assign to the editable technology file to be created. Generally, this name has the extension *.tf* to identify the file as an editable technology file; however, this is not mandatory.

Library Name

Type the name of the library that you want to unload the technology information.

cmDumpTLUPlus

You can view, edit, and update the TLUPlus information stored in the Milkyway database by using the `cmDumpTLUPlus` command.

Use the `cmDumpTLUPlus` command to dump the TLUPlus information to an ASCII file, which can be viewed and edited with a text editor. You can save changes made to the TLUPlus data back to the Milkyway database by using the `cmReplaceTLUPlus` command. Before updating, the `cmReplaceTLUPlus` command creates a backup TLUPlus data file named `libname.TLUPlus_Replaced`.

Note:

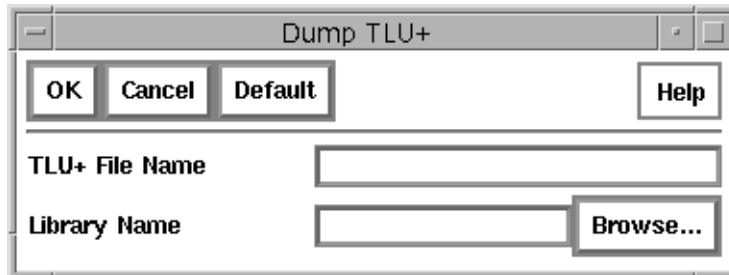
When you select the “Encrypt process data” option in the `cmIfToTLUPlus` command, the `cmDumpTLUPlus` and `cmReplaceTLUPlus` commands are disabled in order to prevent access by external tools.

Tool

Milkyway

Dump TLU+ Dialog Box

The menu command opens the following dialog box. Its options are described below.



TLU+ File Name

Type the name of the TLUPlus file you would like to dump.

Library Name

Type the name of the library from which you want to dump the TLUPlus information.

See Also

[cmReplaceTLUPlus](#)—You can view, edit, and update the TLUPlus information stored in the Milkyway database by using the `cmReplaceTLUPlus` command.

cmExecSmash

This command is used to smash (flatten) a cell.

Tool

Milkyway

Syntax

```
cmExecSmash "libName" "cellName" stopLevel \ smashStd  
smashMacro  
smashPad smashOther explodeText
```

where the arguments are as follows:

<i>libName</i>	The name of the library containing the cell whose netlist you wish to flatten. Valid values: Legal library name.
<i>cellName</i>	The name of the cell that you want the netlist flattened. Valid values: Name of a cell in the specified library.
<i>stopLevel</i>	Indicates how many levels down the hierarchy to stop.
<i>smashStd</i> <i>smashMacro</i> <i>smashPad</i> <i>smashOther</i>	These options are (#t or #f), indicating whether or not you want to smash stds, macros, pad cells or other.
<i>explodeText</i>	This will retain the text at all levels of hierarchy that are smashed.

cmFlattenNetlist

Flattens a hierarchical netlist.

Tool

Milkyway

Syntax

```
cmFlattenNetlist "libName" "cellName" "viewName"  
"expandedView"
```


where the arguments are as follows:

<i>libName</i>	The name of the library containing the cell whose netlist you wish to flatten. Valid values: Legal library name.
<i>cellName</i>	The name of the cell that you want the netlist flattened. Valid values: Name of a cell in the specified library.
<i>viewName</i>	The view name of the cell that you want the netlist flattened. Valid values: .NETL, .CEL.
<i>expandedView</i>	The view name that you want the flattened netlist to be written. Valid values: .EXP, .CEL.

cmltfToTLUPlus

Creates a library attach file for TLU capacitance and resistance models created by Star-RCXT (TLU+ models).

Tools and Menu Commands

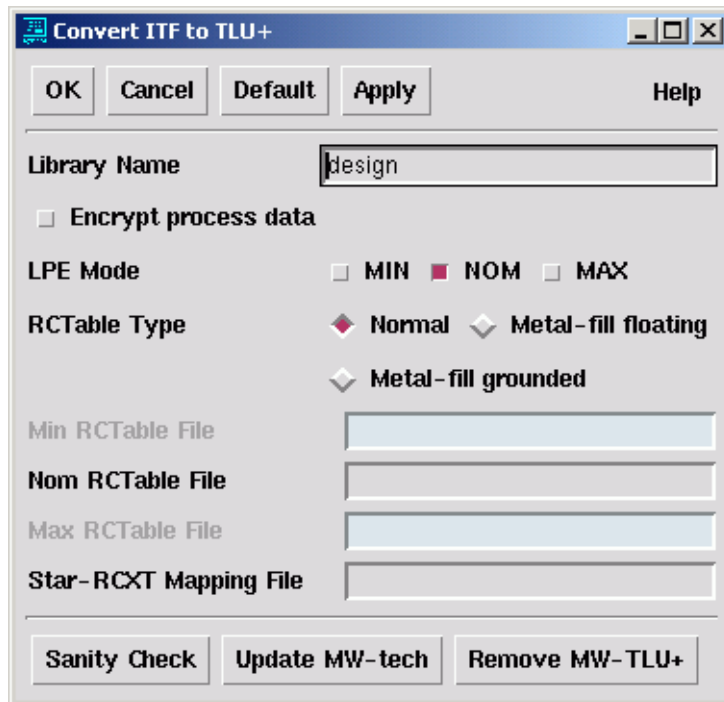
Milkyway: Tech File > ITF To TLU+

Prerequisites

You must have created one or more capacitance table files using the Star-RCXT `grdgenxo` with the `-itf2TLUPlus` option.

Convert ITF to TLU+ Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Type the name for the library for which you wish to create TLU+ capacitance models.

Encrypt process data

Select to store the ITF data in an encrypted format in a Milkyway library so it cannot be accessed and interpreted by external tools.

When this option is selected, the [cmDumpTLUPlus](#) and [cmReplaceTLUPlus](#) commands are disabled.

LPE Mode

Select any combination of MIN, NOM, or MAX capacitance models.

RCTable Type

Select one of the following RCTable types that corresponds to the RCTable File entry on the dialog box.

- Normal – The default value. Select if you want to do normal extraction or your RCTable was generated without metal-fill in the metal-fill flow.
- Metal-fill floating – Select if your RCTable was generated with floating models in the metal-fill flow. Using this option activates the floating model option in subsequent metal-fill extractions.
- Metal-fill grounded – Select if your RCTable was generated with ground models in the metal-fill flow. Using this option activates the ground model option in subsequent metal-fill extractions.

Min RCTable File

If you selected “MIN” above, enter the name of the minimum binary capacitance table file generated by Star-RCXT.

Nom RCTable File

If you selected “NOM” above, enter the name of the nominal binary capacitance table file generated by Star-RCXT.

Max RCTable File

If you selected “MAX” above, enter the name of the maximum binary capacitance table file generated by Star-RCXT.

Star-RCXT Mapping File

Type the name of the Star-RCXT mapping file.

Sanity Check

Click this button to perform a “Sanity Check” on the inputs to the `cmItfToTLUPlus` dialog box. you will get a report on the consistency checking of layer names, etch values, widths, spacing, and layer thickness by comparing the ITF and the Milkyway technology information.

Update MW-tech

Click this button when there are inconsistencies in the “Sanity Check” and you believe the values in the Milkyway technology file information should be updated.

Remove MW-TLU+

Click this button when you want to remove all TLUPlus related files attached to the Milkyway database.

cmLoadTech

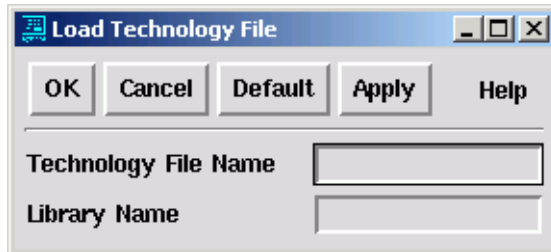
Use this command to load a technology file into a library.

Tool

Milkyway

Load Technology File Dialog Box

The menu command opens the following dialog box. Its options are described below.



Technology File Name

Type the name you want to assign to the editable technology file to be loaded. Generally, this name has the extension *.tf* to identify the file as an editable technology file; however, this is not mandatory.

Library Name

Type the name of the library that you want to load the technology information.

cmItfToTLUTest

Converts the specified ITF file to a TLUPlus file and attaches it to the specified library.

Syntax

```
cmItfToTLUTest <objectId libId> <boolean  
deleteAllCapTables> <string minTableFile> <string  
nomTableFile> <string maxTableFile> <string mapfile>
```

where the arguments are as follows:

<code>libId</code>	Object ID of the library.
<code>deleteAllCapTables</code>	This argument is ignored. Capacitance tables are always deleted.
<code>minTableFile</code>	Name of the ITF file containing the minimal capacitance table.
<code>nomTableFile</code>	Name of the ITF file containing the nominal capacitance table.
<code>maxTableFile</code>	Name of the ITF file containing the maximal capacitance table.
<code>mapfile</code>	Name of the Star-RCXT mapping file.

Description

The `cmItfToTLUTest` command converts the specified ITF file to a TLUPlus file and attaches it to the specified library. This command is the command-line equivalent to the “Convert ITF to TLU+” GUI form (`cmItfToTLUPlus`).

Example

```
cmItfToTLUTest (dbGetCurrentLibId) #t "min.itf" "nom.itf"
               "max.itf" "my.map"
```

cmMarkCellType

Lets you change the type of cell after translation to the Synopsys format.

Tools and Menu Commands

Astro: Cell > Mark Type

Jupiter: Astro Floorplanning: Cell > Mark Type

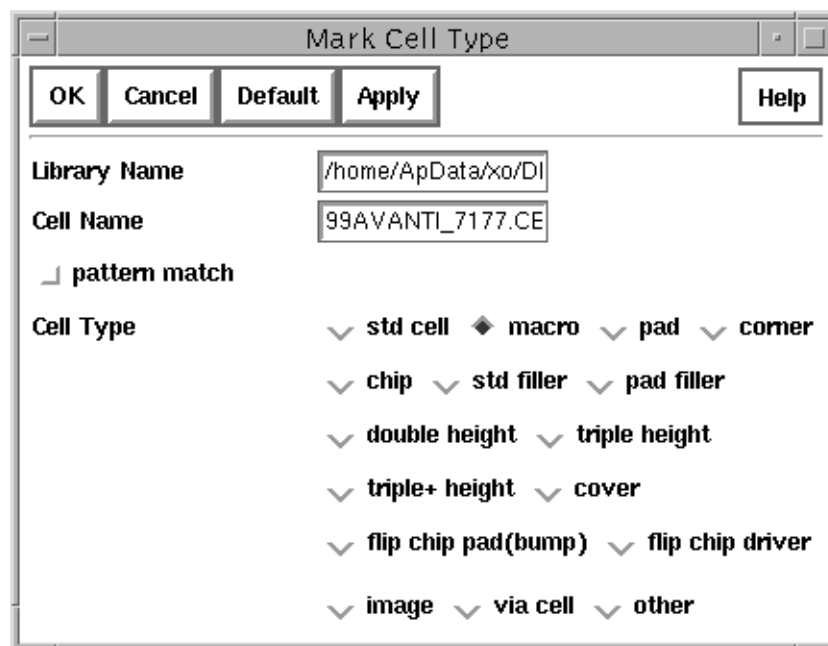
Milkyway: Cell > Mark Type

Prerequisites

Translate the library into Astro using [auStreamIn](#). The *auStreamIn* command requires a cell type definition file (*gds2Arcs.map*). The *cmMarkCellType* command lets you correct a cell types incorrectly specified in the cell type definition file.

Mark Cell Type Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Type the name of the library containing the translated cell.

Cell Name

Type the name of the cell that you want to change the type to, or select *pattern match* and type a pattern to match the names of cells that you want to change the type to.

pattern match

Select when you want to specify a pattern to match the names of cells that you want to change the type to.

Use the pattern matching option to select all nets with names that match a pattern.

Cell Type

Select the type you want to specify for the cell.

Table 22-27

Option	Specifies that the cell is ...
chip	a top-level, cell-based design.
corner	a corner pad.
cover	a cover cell.
double height	a double height cell.
flip chip driver	a flip chip driver cell.
flip chip pad (bump)	an I/O pad for flip chip designs.
image	an image cell.
macro	a block.
pad	a pad.
pad filler	a dummy pad used to control pad spacing and to complement nwell taps in pads.
std cell	a standard cell.

Table 22-27

Option	Specifies that the cell is ...
std filler	a standard filler cell with power and ground pins, wells, and nwell taps.
triple height	a triple height cell.
triple+ height	Quadruple height or more.
other	of no type and is therefore ignored by Milkyway and Astro (no pin or blockage extraction).

cmMarkEEQPins

Changes the electrical equivalency of pins for specified cells. After you execute this command, Astro considers pins on each port in the specified cell(s) connected as follows:

- If pins on adjacent metal layers (for example, metal1 and metal2) overlap, Astro considers them strongly connected to each other.
- Otherwise, Astro considers pins weakly connected to all other pins.

If you do not run *cmMarkEEQPins* on a cell, Astro considers:

- All metal pins on a port strongly connected to each other.
- All poly pins on a port weakly connected to all other pins.

Note:

Pins are the physical implementation of ports defined in the netlist.

Tools and Menu Commands

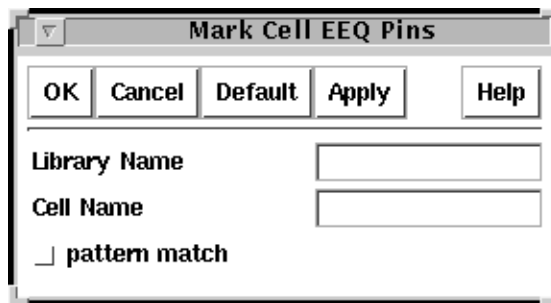
Milkyway: Cell Library > Mark EEQ Pins

Prerequisites

Create the pin and blockage cells, using the [auExtractBlockagePinVia](#) command.

Mark Cell EEQ Pins Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Type the name of the Astro library containing the cell that you want to mark pins.

Cell Name

Type the name of the cell that you want to mark pins, or select *pattern match* and type a pattern to match the names of cells that you want to mark the pins.

pattern match

Select if you want to specify a pattern to match the names of cells that you want to mark pins.

Use the pattern matching option to select all nets with names that match a pattern.

cmMarkLefSite

Lets you specify a special tile to be used by a cell master when overlapping rows are detected during placement with Astro. This command stores the defined string name in the cell master FRAM view.

Tools and Menu Commands

Milkyway: None

Prerequisites

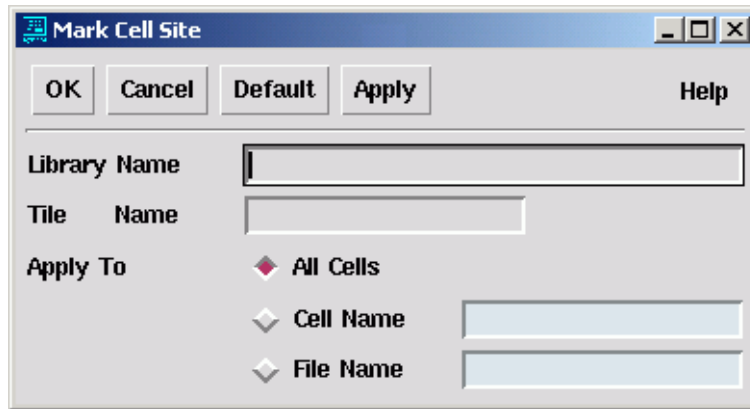
You need to define the special tile name in the Tile section of the technology file and use the `cmSetMultiHeightProperty` command before using the `cmMarkLefSite` command.

For example, to modify the technology file, enter

```
Tile "unit" {  
    width = 1.9  
    height = 22.8  
}  
Tile "Extra" {  
    width = 1.9  
    height = 45.6  
}
```

Mark Cell Site Dialog Box

The `cmMarkLefSite` command opens the following dialog box. Its options are described below.



Library Name

Type the name of the library that contains the cell you want to designate to use a special site.

Tile Name

Type the name of a special tile that is defined in the technology file for the library.

Apply In

- All Cells—All cells in the library are to use the designated site.
- Cell Name—Enter the cell name to use the designated site.
- File Name—Specify an ASCII file that contains the names of cell masters to use the designated site.

See Also

[cmSetMultiHeightProperty](#)—Adjusts the place and route boundary for multiple height cells.

cmMasterTilePattern

Allows you to view, edit, or create legal location patterns for any master in your library.

When you enter a library or master tile name, the tool displays legal location information. You may also create new legal location information to be written back into the master.

Note:

In order to avoid destroying a legal location pattern, avoid editing or modifying existing tile patterns using this command. A corrupted tile pattern will cause all instances of the given master not to be placed.

Tools and Menu Commands

Milkyway: Cell Library > View/Edit Tile Pattern

Prerequisites

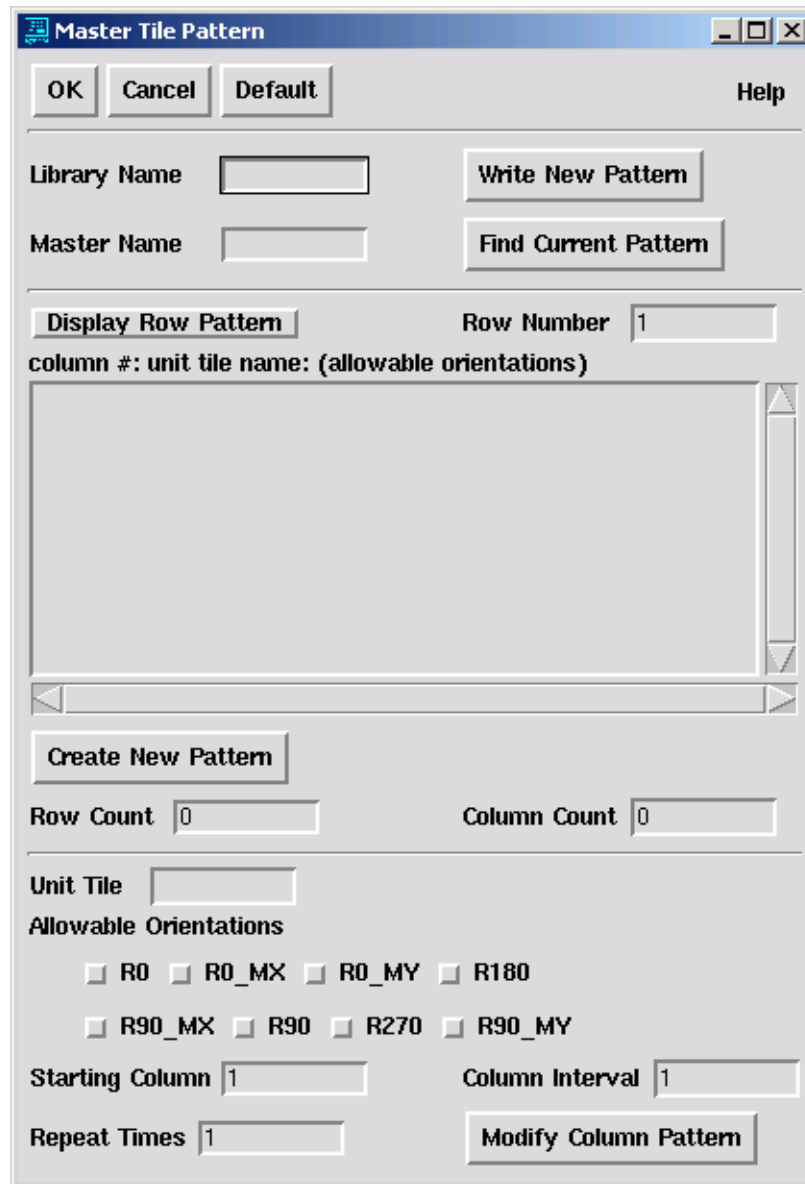
Create pin/blockage cells for layout cells, using the [auExtractBlockagePinVia](#) command.

Generate PR boundaries for cells in your library, using the [auSetPRBdry](#) command.

If your library contains multiple height cells, adjust PR boundaries, using the [cmSetMultiHeightProperty](#) command.

Master Tile Pattern Dialog Box

The menu command opens the following dialog box. Its options are described below.



The dialog box is titled "Master Tile Pattern" and features a standard Windows-style title bar with minimize, maximize, and close buttons. It contains several sections of controls:

- Buttons:** "OK", "Cancel", "Default", and "Help" are located at the top.
- Library Name:** A text input field with a "Write New Pattern" button to its right.
- Master Name:** A text input field with a "Find Current Pattern" button to its right.
- Display Row Pattern:** A button that, when clicked, displays the pattern for a specific row.
- Row Number:** A text input field with the value "1" entered.
- Pattern Display:** A large rectangular area with a vertical scrollbar on the right, intended for displaying the tile pattern. Above this area is the label "column #: unit tile name: (allowable orientations)".
- Create New Pattern:** A button located below the pattern display area.
- Row Count:** A text input field with the value "0" entered.
- Column Count:** A text input field with the value "0" entered.
- Unit Tile:** A text input field.
- Allowable Orientations:** A section containing eight checkboxes: "R0", "R0_MX", "R0_MY", "R180", "R90_MX", "R90", "R270", and "R90_MY".
- Starting Column:** A text input field with the value "1" entered.
- Column Interval:** A text input field with the value "1" entered.
- Repeat Times:** A text input field with the value "1" entered.
- Modify Column Pattern:** A button located at the bottom right.

Library Name

Type the library name containing the master you wish to edit.

Master Name

Type the name of the master you wish to edit.

Write New Pattern

After creating a new or editing an old master (as defined in the scroll window), click to write the new or altered pattern back into the master. This is the last step to be performed when writing or editing a pattern.

Find Current Pattern

Click to display in the scroll window the current legal location pattern of the master you specify.

Display Row Pattern

Click to display in the scroll window the legal location pattern of the row specified in the *Row Number* field. By default, the first row is displayed. You can choose to display any row in the case of a multiple-height master. The dimensions of the legal location pattern are displayed in the *Row Count* and *Column Count* fields.

Row Number

Type the row number of the row pattern that you want displayed
column #: unit tile name (allowable orientations)

This scroll window displays the legal location pattern of the row you chose, arranged by column number, unit tile name, and the allowable orientations of the row.

Create New Pattern

After entering the *Row Count* and *Column Count*, click to create a new set of buffers for further editing.

Row Count

If you are choosing to create a new pattern, type the number of rows you want the matrix to create. If you are working with an existing pattern, the row count of that pattern is displayed here.

Column Count

If you are choosing to create a new pattern, type the number of columns you want the matrix to create. If you are working with an existing pattern, the column count of that pattern is displayed here.

Unit Tile

Type the name of the unit tile that you want written back into the buffer.

Allowable Orientations

Choose one of eight possible orientations for the master, either *R0*, *R0_MX*, *R0_MY*, *R180*, *R90_MX*, *R90*, *R270*, or *R90_MY*.

Table 22-28

Option	Orients the master...
R0	Without changing its orientation.
R0_MX	Flipped on the y axis.
R0_MY	Flipped on the x axis.
R180	Rotated counterclockwise 180 degrees.
R90_MX	Rotated counterclockwise 90 degrees and flipped on the y axis.
R90	Rotated counterclockwise 90 degrees.
R270	Rotated counterclockwise 270 degrees.
R90_MY	Rotated counterclockwise 90 degrees and flipped on the x axis.

Note:

You can select more than one orientation option.

Starting Column

Type the number of the first column which you want to modify.

Column Interval

Type the interval number of subsequent columns following the starting column that you want your modifications to effect. For instance, if you type *1*, all subsequent columns will be effected. If you type *2*, the second (and fourth, etc.) column will be skipped and the third (and fifth, etc.) column will be modified. If you type *3*, the second and third (and fifth and sixth, etc.) columns will be skipped and the fourth (and seventh, etc.) column will be modified. Set the number of repetitions of the interval in the *Repeat Times* field.

Repeat Times

Type the number of times you want the column interval to repeat. The number of repetitions is limited by how many columns you are working with.

Modify Column Pattern

Click to perform column pattern modification once you have entered all information needed to create a new pattern.

cmPDBIn

Imports a PDB (.pdb) file into a Milkyway library, creating the technology information and cells in the library.

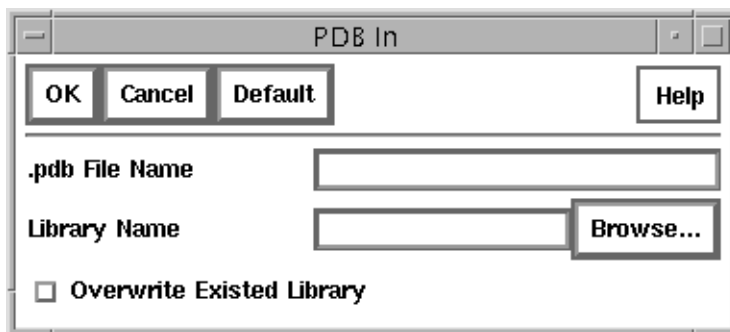
PDB (Physical Library DataBase) defines the elements of an IC process technology and associated library of cell models. It contains library information for a class of designs. Library data includes layer, via, site and macro cell definitions.

Tool

Milkyway

PDB In Dialog Box

The menu command opens the following dialog box. Its options are described below.



.pdb File Name

Type the name of the .PDB input file.

Library Name

Specifies the Milkyway library in which the library information and reference cells will be created. The string may include the path of the library, otherwise the library is resolved from the current working directory.

Poly Contact Layer

When selected, the polyCont layer is defined in the Milkyway library technology information. If not selected, the polyCont layer is not defined in the Milkyway library technology information.

See Also

cmDumpPlib – no document

cmDumpPseudoPlib – no document

cmPurgeTech

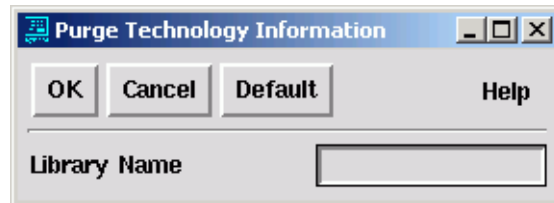
Use this command to purge the technology information of a library.

Tool

Milkyway

Purge Technology Information Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Type the name of the library that you want to purge the technology information.

cmQueryHierPorts

Prompts a dialog box for the port name, then prints out ports on the leaf cells. You need to have an cell opened before executing this command.

Following is an example of the output:

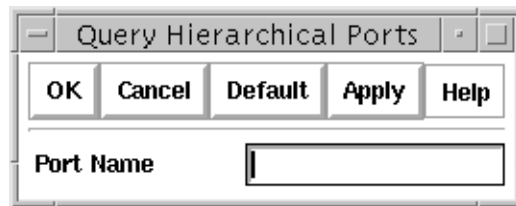
Hierarchical port A/p has leaf ports:

A/B/q
A/C/r

This means net connected to port “p” of cell instance “A” (at top level cell) connects to port “q” of cell instance “A/B” and port “r” of cell instance “A/C” in the flattened cell.

Query Hierarchical Ports Dialog Box

The menu command opens the following dialog box. Its options are described below.



Port Name

Type the name of the Hierarchical Port you wish to query.

cmRefLib

Adds a reference library to a library. If specifying a relative reference library path, the path should be relative to the main library directory, not the current working directory.

Instead of copying cells to multiple libraries, the Synopsys tool references the other libraries, this way they are referred to as “reference libraries.”

Tools and Menu Commands

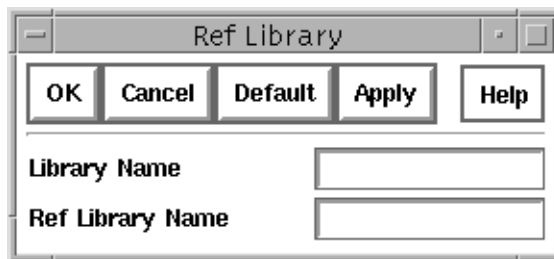
Astro: Library > Add Ref

Jupiter: Library > Add Ref

Milkyway: Library > Add Ref

Ref Library Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Name of the library that you want to add a reference library.

Ref Library Name

Name of the library you want to add as a reference library.

See Also

[cmShowRefLib](#) – Displays the reference libraries for a library.

[cmReplaceRefLib](#) – Replaces reference libraries for a library.

cmRenameCell

Renames a cell.

Tools and Menu Commands

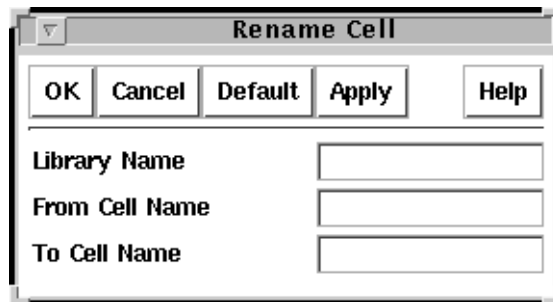
Astro: Cell > Rename

Jupiter: Cell > Rename

Milkyway: Cell > Rename

Rename Cell Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Type the name of the library containing the cell you want to rename.

From Cell Name

Type the name of the cell you want to rename.

To Cell Name

Type the name you want to assign to the cell.

cmRenameLib

Renames a library. You must close the library before renaming it.

Tools and Menu Commands

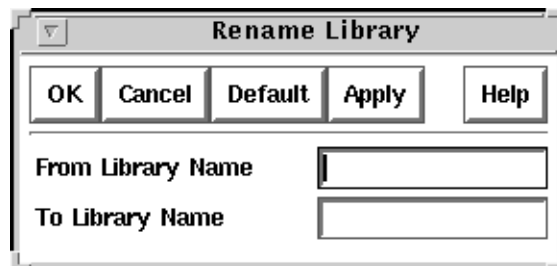
Astro: Library > Rename

Jupiter: Library > Rename

Milkyway: Library > Rename

Rename Library Dialog Box

The menu command opens the following dialog box. Its options are described below.



From Library Name

Type the name of the library you want to rename.

To Library Name

Type the name you want to assign to the library.

cmReplaceRefLib

Replaces reference libraries associated with a library.

Tools and Menu Commands

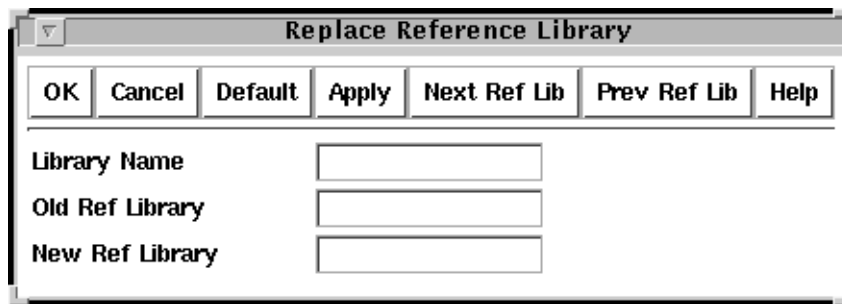
Astro: Library > Replace Ref

Jupiter: Library > Replace Ref

Milkyway: Library > Replace Ref

Replace Reference Library Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Type the name of the library that you want to change a reference library.

Old Ref Library

Type the name of the reference library you want to change, or click *Next Ref Lib* or *Prev Ref Lib* to cycle forward or backward through the reference libraries associated with the specified library.

New Ref Library

Type the name of the library you want to replace the *Old Ref Library*.

See Also

[cmRefLib](#) adds a reference library to a library.

[cmShowRefLib](#) displays the reference libraries for a library.

cmReplaceTech

Replaces the technology file associated with a library. If specifying a relative reference library path, the path should be relative to the main library directory, not the current working directory.

Caution!

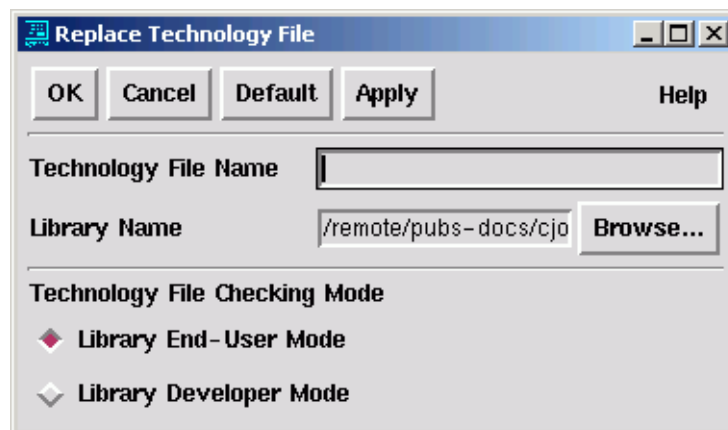
The definition of system layers in previous software versions changed in version 99.2 by running the *cmReplaceTech* command. This will enable the software and technical file to support more than six layers of metal. For more information regarding the automatic re-naming of system layers.

Tools and Menu Commands

Milkyway: Tech File > Replace

Replace Technology File Dialog Box

The menu command opens the following dialog box. Its options are described below.



Technology File Name

Type the name of the technology file you want to associate with the cells in the library.

Library Name

Type the name of the library that you want to load the technology file.

Technology File Checking Mode

Select to enable technology file checking in either Library End-User Mode or Library Developer Mode.

cmReplaceTLUPlus

You can view, edit, and update the TLUPlus information stored in the Milkyway database by using the `cmReplaceTLUPlus` command.

You can save changes made to the TLUPlus data back to the Milkyway database by using the `cmReplaceTLUPlus` command. Before updating, the `cmReplaceTLUPlus` command creates a backup TLUPlus data file named `libname.TLUPlus_Replaced`

Note:

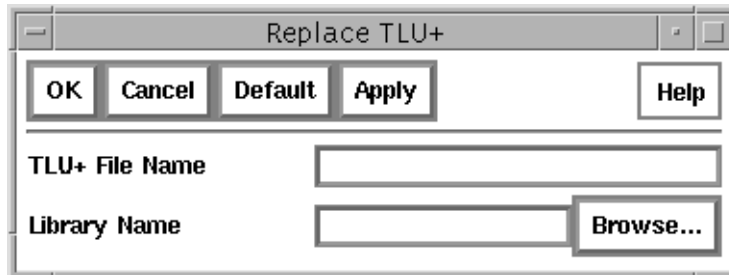
When you select the "Encrypt process data" option in the [cmIfToTLUPlus](#) command, the `cmDumpTLUPlus` and `cmReplaceTLUPlus` commands are disabled in order to prevent access by external tools.

Tool

Milkyway

Replace TLU+ Dialog Box

The menu command opens the following dialog box. Its options are described below.



TLU+ File Name

Type the name of the TLUPlus file you would like to replace.

Library Name

Type the name of the library from which you want to replace the TLUPlus information.

cmSetBusNameStyle

Sets the library bus naming style.

Tools and Menu Commands

Astro: Library > Set Bus Naming Style

Milkyway: Library > Set Bus Naming Style

Set Bus Naming Style Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Type the name of the library for which you want to set the bus name style.

Bus Naming Style

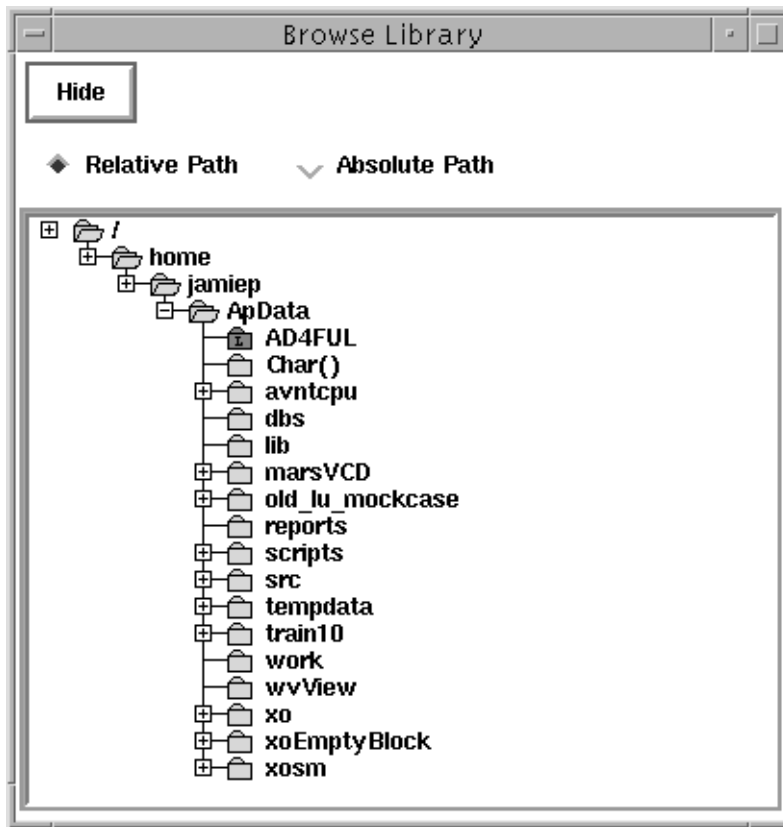
Type the Bus Name Style you want to set.

Browse

Select to invoke the Browse Library dialog box.

Browse Library Dialog Box

The menu command opens the following dialog box. Its options are described below.



Browse Library Dialog Box

Select the library to change the BusNameStyle.

cmSetMultiHeightProperty

Adjusts the PR Boundary for multiple height cells and sets the cell row orientation. Checks the power and ground height and width, suggests floor plan options, and checks the row spacing rule in the technology file. If the row spacing rule does not exist, Milkyway defines it.

Note:

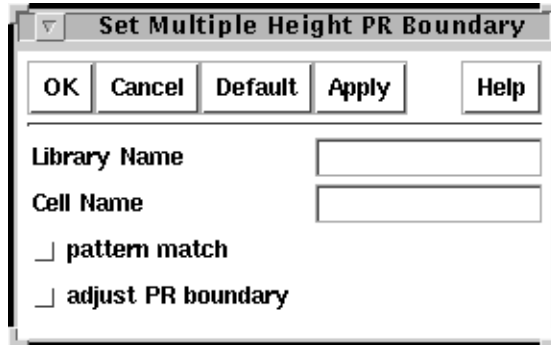
If you are using double- or triple-height cells with the standard cells in your design, you need to adjust the PR boundary for each multiple height cell by selecting the adjust PR boundary option. Astro adjusts the PR boundary and generates a tile pattern for the multiple height cells.

Tools and Menu Commands

Milkyway: Cell Library > Multi Height Cell

Set Multiple Height PR Boundary Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Type the name of the library that contains the cell that you want to adjust the PR boundary and set the cell row orientation.

Cell Name

Type the name of the cell that you want to adjust the PR boundary, or select *pattern match* and type a pattern to match the names of multiple cells that you want to adjust the PR boundary.

pattern match

Select if you want to specify a pattern to match the names of multiple cells that you want to adjust the PR boundary.

Use the pattern matching option to select all nets with names that match a pattern.

adjust PR boundary

Select to adjust the PR boundary and set the cell row orientation. Deselect to set only the cell row orientation.

cmShowInstOrigin

Generates a text file listing the source library for each cell master instantiated within the specified design cell.

Syntax

```
cmShowInstOrigin < string libName > < string cellName >  
< string fileName >
```

where the arguments are as follows:

libName

Name of the top library with the design cell

cellName

Name of the design cell

fileName

Name of the output file

Description

The `cmShowInstOrigin` function generates a text file listing the source library for each cell master instantiated within the specified design cell. The cell masters are listed in alphabetical order with their source library and its instance names.

This function only reports the cell masters instantiated at the top level (that is, instantiated directly in the design cell). It does not include cell masters instantiated at a lower level (that is, instantiated within another cell master).

Example

```
cmShowInstOrigin "mylib" "design" "report.txt"
```


The above function call generates the following information in a text file named “report.txt”:

Origin Information of cell Instance

DesignLib and TopCell:
Library:/home/mylib
TopCell:design

Cell Instance List:

cellInstMaster	cellinstance	RefLibrary

CM1		
	CM1_1	/net/u1/ref1
	CM1_2	/net/u1/ref1

CM2		
	CM2_1	/net/u2/ref2
	CM2_2	/net/u2/ref2
	CM2_3	/net/u2/ref2

#of Instance: 5		
THE END		

cmShowRefLib

Displays a list of the reference libraries for the library you specify in the messages window.

Tools and Menu Commands

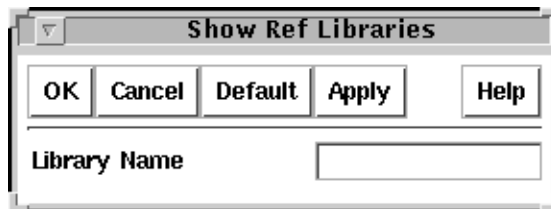
Astro: Library > Show Refs

Jupiter: Library > Show Refs

Milkyway: Library > Show Refs

Show Ref Libraries Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Type the name of the library that you want to display reference libraries.

See Also

[cmRefLib](#) adds a reference library to a library.

[cmReplaceRefLib](#) replaces reference libraries for a library.

cmSmash

Smashes a specified master cell (and the master cells for all child cells) for a specified cell type down to a specified level.

Note:

Smashing a cell that is not open changes the disk copy of the cell. Changing a cell that is open changes the copy in virtual memory. To retain the smash of an open cell, save the cell before you close it.

Tools and Menu Commands

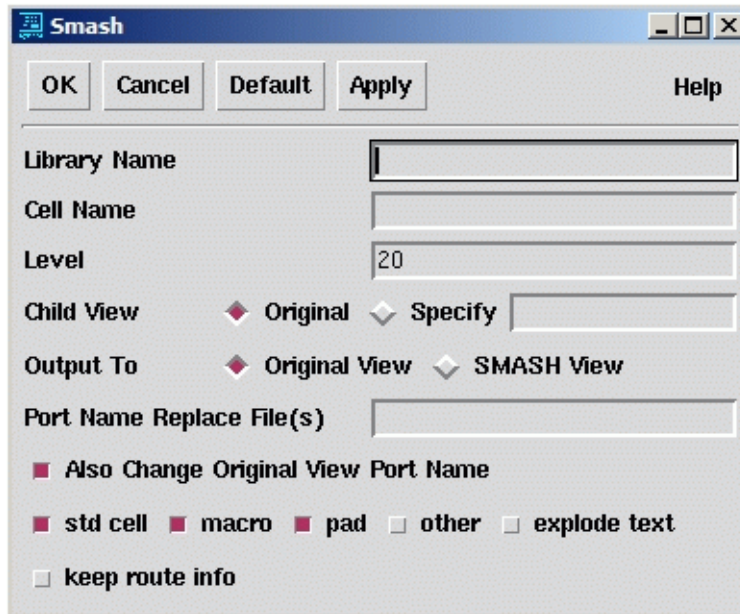
Astro: Modify > Smash Cell

Jupiter: Astro Floorplanning: Edit > Smash Cell

Milkyway: Modify > Smash Cell

Smash Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Type the name of the library containing the cell(s) you want to smash, or leave blank to smash cells in the open library.

Cell Name

Indicate the cells you want to smash.

To indicate ...

One cell.

All cells of the types you select.

Do the following:

Type the name of that cell and select its type.

Type an asterisk (*) and select from the cell types: *module cell*, *macro*, *pad*, *other*.

Level

Type the number of levels of hierarchy that you want to smash the cell(s).

Child View

This option determines which view will be used to do the smash for all child cells. By default, the option is set at the “Original View” which is the built-in view of a child cell in the hierarchy. You can specify a particular view, such as a .CON view, to a .SMASH view. If the specified view doesn’t exist for some child cell(s), the original view will be used.

Output To

Specify from the following output choices:

- Original View: will change the original view.
- SMASH View: will change and save as new smash view.

Port Name Replace File(s)

If you want to rename the port name, enter the file name that contains the rename information. The format of the rename file is `cellName oldName newName`.

Also Change Original View Port Name

Select to change the original view port name. Primarily used when you select to output to .SMASH view but you still want to keep the original view unaltered.

std cell

Select to smash all standard cells in the library when you do not specify a cell name.

macro

Select to smash all blocks in the library when you do not specify a cell name.

pad

Select to smash all pads in the library when you do not specify a cell name.

other

Select to smash any cells other than standard cells, blocks, or pads in the library when you do not specify a cell name.

explode text

Select to retain the text at all levels of hierarchy that are smashed.

keep route info

Select to keep the route information.

23

cmCmd Commands for Milkyway

This chapter describes commands with the prefix *cmCmd*. The commands are listed alphabetically.

- [cmCmdDumpMultipleECOs](#)
- [cmCmdDumpSlackGraph](#)
- [cmCmdECODump](#)
- [cmCmdECOHistory](#)
- [cmCmdExpand](#)
- [cmCmdExpandBindNet](#)
- [cmCmdLoadConstraints](#)
- [cmCmdLoadSlackGraph](#)
- [cmCmdLoadTimingRequirements](#)

cmCmdDumpMultipleECOs

Writes the details of multiple ECOs to a file. The format of this file follows the syntax of the Header Information and Detailed Information, as described in the [cmCmdECODump](#) command. Generates a new *ECOld* and deletes all the ECO records in the specified range. To view the contents of the dumped file, use the [cmCmdECODump](#) command.

Caution!

This command cannot be undone. Please make a backup of the top cell before performing this operation.

Tools and Menu Commands

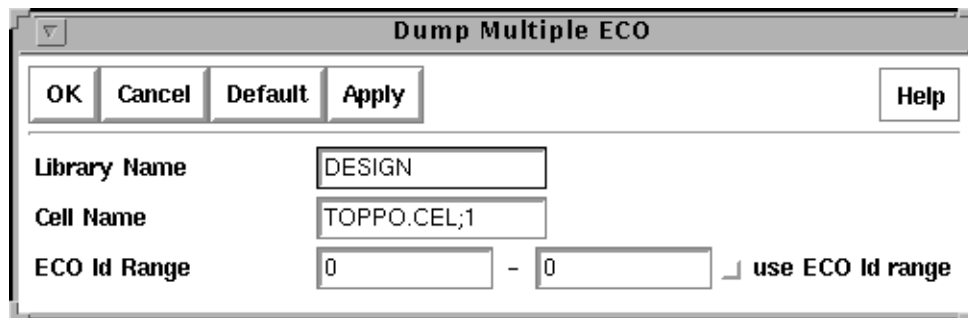
Astro: ECO > Write Multiple ECOs to file

Jupiter: ECO > Write Multiple ECOs to file

Milkyway: ECO > Write Multiple ECOs to file

Dump Multiple ECO Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Type the name of the library containing the cell for which you want to write the ECO details to a file.

Cell Name

Type the name of the cell for which you want to write the ECO details to a file.

ECO Id Range

Type the range of the ECO numbers that you want to dump to the file. Select the “use ECO Id range” button to activate the minimum and maximum entries. If the “use ECO Id range” button is not selected, then all ECO records starting from the minimum *ECOId* are dumped out.

cmCmdDumpSlackGraph

Writes timing constraints to a file in slack graph format.

Tool

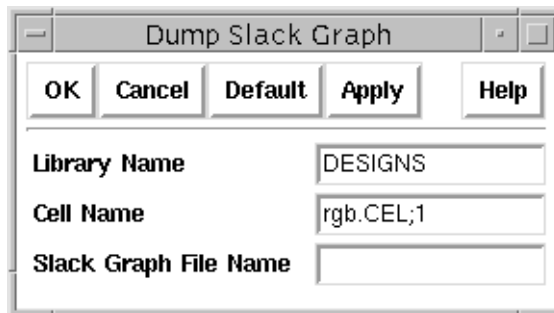
Milkyway

Prerequisites

You must first generate a slack graph in the database, using the [cmCmdLoadConstraints](#) or [cmCmdLoadSlackGraph](#) commands.

Dump Slack Graph Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Type the name of the library containing the cell for which you want to write slack graph information to a file.

Cell Name

Type the name of the cell for which you want to write slack graph information to a file.

Slack Graph File Name

Type the name of the file to which you want to write slack graph information.

cmCmdECODump

Writes the details of an ECO to a file. The format of the file follows the syntax of the Header Information and Detailed Information described here.

ECO Header Information

Each ECO begins with a header that provides the following information:

- Date and time of the change in the format

+D dayOfWeek Month Day Time Year

- Tool that made the change in the format:

+T tool

- Comments about the change in the format:

+C comment

- User responsible for the change in the format:

+A loginName

ECO Detailed Information

An ECO can contain any of the following information:

- Addition of a net in the format:

+N netName

- Deletion of a net in the format:

-N netName

- Addition of a cell instance in the format:

+I instanceName masterName

- Deletion of a cell instance in the format:

-I instanceName masterName

- Replacement of a cell instance in the format:

+R oldMasterName newMasterName instanceName

- Connection of a port in the format:

+P portName instanceName netName

- Disconnection of a port in the format:

-P portName instanceName netName

Note:

“-P” disconnects the port, it does not delete it from the database.

The hierarchical name of a cell instance consists of the names of all cell instances in the netlist hierarchy of that cell instance, top to bottom, preceded by a hierarchy delimiter. For example, a cell instance *A* contains a cell instance *B*, which contains a cell instance *C*. The hierarchical name of cell instance *C* is *A/B/C*, when using the hierarchical delimiter “/”.

Note:

All database information is written directly to the output file. Special characters, such as { } and \$, will not be escaped with a backslash or any other escape character.

Tools and Menu Commands

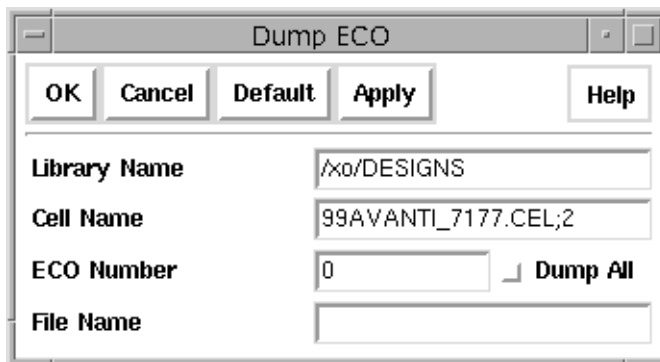
Astro: ECO > Write To File

Jup[iter: ECO > Write To File

Milkyway: ECO > Write To File

Dump ECO Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Type the name of the library containing the cell for which you want ECO details.

Cell Name

Type the name of the cell for which you want ECO details.

ECO Number

Type the number of the ECO for which you want ECO details.

Dump All

Check this button to dump all ECO records from #1 history to #end history.

File Name

Type the name of the file to which you want to write ECO details.

cmCmdECOHistory

Provides ECO history for a specified cell.

Tools and Menu Commands

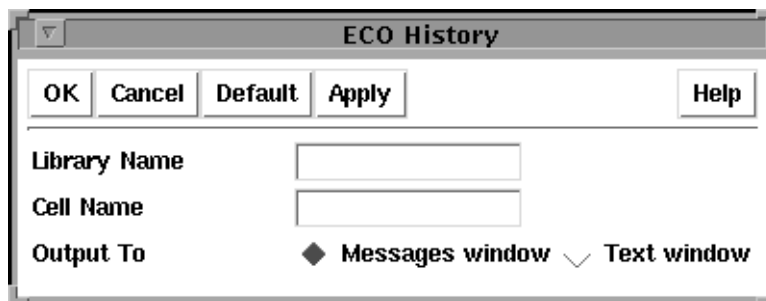
Astro: ECO > History

Jupiter: ECO > History

Milkyway: ECO > History

ECO History Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Type the name of the library containing the cell for which you want ECO history information.

Cell Name

Type the name of the cell for which you want ECO history information.

Output To

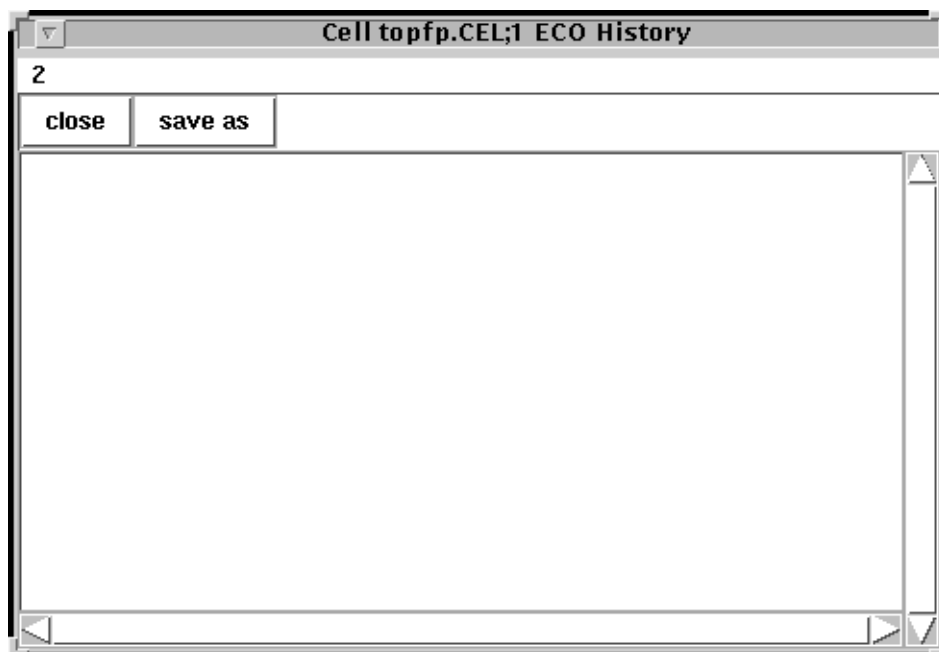
Select the place you want to display the report.

Table 23-29

Option	Description
Messages window	Displays the report in the messages area of the Synopsys window.
Text window	Displays the report in a text window, from which you can save the report to a file.

If you selected *Text window*, the ECO History Report is displayed in a text window.

ECO History Report (in a Text Window)



This report provides the following information:

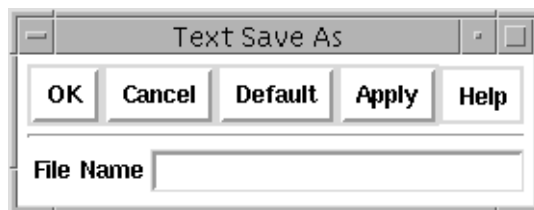
- The author of the ECO.

- The tool that executed the ECO.
- The date of the change.
- Any comments that were stored.

File menu

You can use the commands on this menu to save the text in the window to a file, load another file into the window, or close the window.

- *Save* saves the text in the window to a file with a name you assign using *Set File Name*.
- *Save As* displays the *Text Save As* dialog box.



To save the text in the window, type a *File Name* and click *OK*. The tool saves the file to the name you type.

- *Load* loads an existing file into the window.
- *Set File Name* assigns a name to the text in the window. You can then save the text to a file using *Save*.
- *Close* closes the window.

cmCmdExpand

Expands the netlist recursively until all objects in the netlist are broken down into component objects that exist in the layout, creating an expanded netlist cell in the library directory. Using the Global Net Options dialog box, you can also specify global net connections to nets and ports in the subcells.

Tools and Menu Commands

Milkyway: Netlist In > Expand

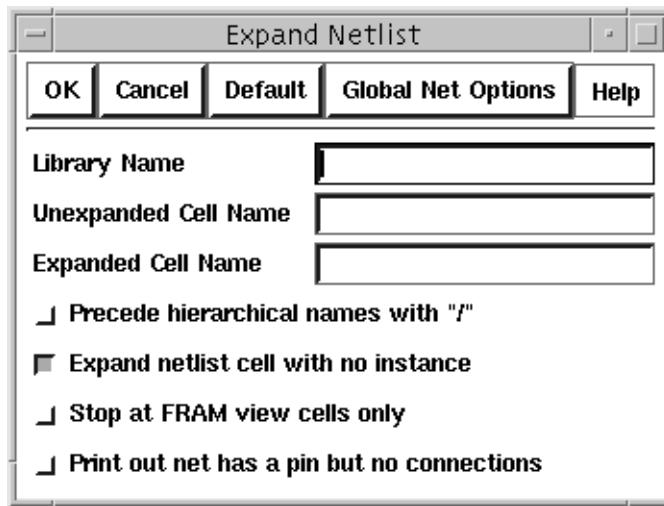
Prerequisites

You must have a netlist cell. To create the netlist cell, use [auEdifIn](#).

Also, the layout standard cells, the layout macro cells, the unexpanded cell, and any netlist cells referenced by the netlist cell you are expanding must be in the library you specify on the dialog box below (or one of its reference libraries). To make a library a reference library to another library, use [cmRefLib](#).

Expand Netlist Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Type the name of the library in which you want to place the expanded netlist cell.

Unexpanded Cell Name

Type the name of the unexpanded netlist cell created by translating the EDIF file.

Expanded Cell Name

Type the name you want to assign to the expanded netlist cell. By convention, this file has a *.exp* extension. For example, if a netlist cell is named *b5.netl*, you name the expanded netlist cell *b5.exp*.

Precede hierarchical names with "/"

Select to generate hierarchical names preceded with a slash (/) to ensure consistency with SDF files.

Expand netlist cell with no instance

Select to expand netlist cells that contains no physical descriptions. Use this option if you have netlist instances containing only nets. If this option is deselected, and you have netlist instances with no physical descriptions, expansion will fail.

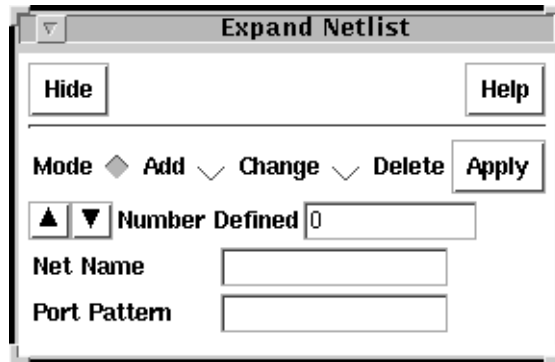
Stop at FRAM view cells only

If users turn this option on, the netlist expanding process will stop at .FRAM view cells only. All child cells will be expanded even if the corresponding .CEL views exist in the design library. the default value is off.

Print out net has a pin but no connections

Prints out a single pin net in the window.

Expand Netlist Dialog Box (Global Net Options)



Mode

After you type a *Net Name* and *Port Pattern*, select the appropriate action you want to take and click *Apply*.

Table 23-30

Option	Action
Add	Connects ports and nets that match the <i>Port Pattern</i> to the net specified by <i>Net Name</i> . If the net does not exist, Milkyway creates it.
Change	Changes previously defined connections for the net specified by <i>Net Name</i> .
Delete	Removes previously defined connections for the net specified by <i>Net Name</i> .

To display the name of a previously defined net, click the arrows.

Note:

To add to connections previously defined by this command, select Add. To overwrite connections previously defined by this command, select Change.

Apply

When you complete the dialog box, click to apply the net connection according to the information you provided in the dialog box fields and options.

Number Defined

Click the arrows to display the nets for which you have defined connections.

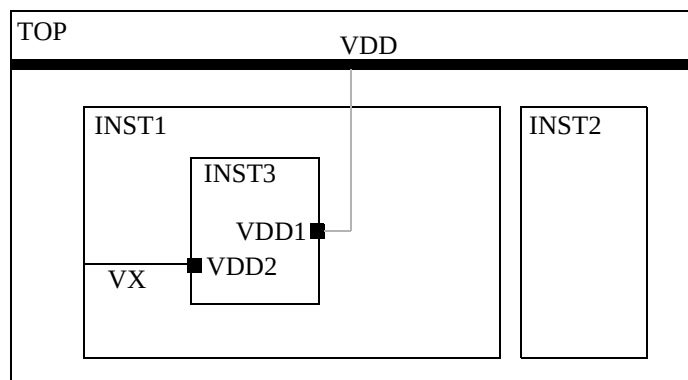
Net Name

Type the name of the top-level global net for which you want to define connections. If the net does not exist, Milkyway creates it.

Port Pattern

Type a pattern to match the names of ports and nets that you want to connect to the net specified by *Net Name*.

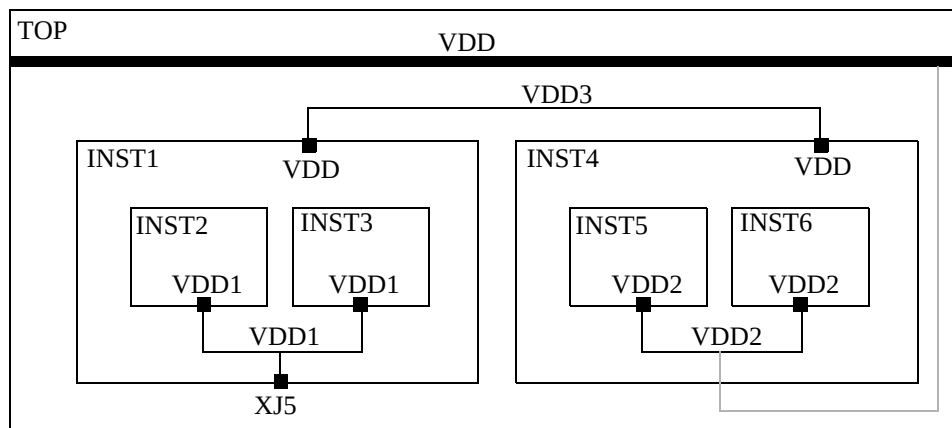
- If the object is a port, your Synopsys tool connects it to the specified net only if it is not connected to another net. For example, consider the case below.



If you specify a *Net Name* of *VDD* and a *Port Pattern* of *VDD.**:

- Your Synopsys tool connects port VDD1 to the top-level net VDD, because port VDD1 is not connected to another net.

- Your Synopsys tool does not connect port VDD2 to the top-level net VDD, because port VDD2 is already connected to net VX.
- If the object is a net, your Synopsys tool connects it to the specified net only if it is inside a cell instance and is not connected to a port. For example, consider the case below.



If you specify a *Net Name* of **VDD** and a *Port Pattern* of **VDD.***:

- Your Synopsys tool does not connect net VDD1 to the top-level net VDD, because it is connected to port XJ5.
- Your Synopsys tool connects net VDD2 to the top-level net VDD, because VDD2 is inside cell instance INST4 and is not connected to a port of INST4.
- Your Synopsys tool does not connect net VDD3 to the top-level net VDD, because it is not inside a cell instance.

cmCmdExpandBindNet

Expands the netlist recursively until all objects in the netlist are broken down into component objects that exist in the layout, creating an expanded netlist cell in the library directory. This command also binds the flattened netlist to directly produce a .CEL cell. Using the Global Net Options dialog box, you can also specify global net connections to nets and ports in the subcells.

Tool

Milkyway

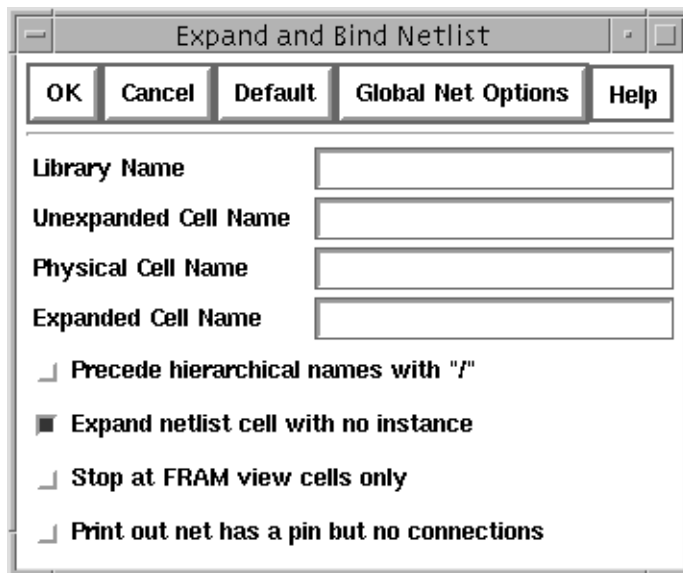
Prerequisite

You must have a netlist cell. To create the netlist cell, use [auEdifIn](#).

Also, the layout standard cells, the layout macro cells, the unexpanded cell, and any netlist cells referenced by the netlist cell you are expanding must be in the library you specify on the dialog box below (or one of its reference libraries). To make a library a reference library to another library, use [cmRefLib](#).

Expand and Bind Netlist Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Type the name of the library in which you want to place the expanded netlist cell.

Unexpanded Cell Name

Type the name of the unexpanded netlist cell created by translating the EDIF file.

Physical Cell Name

Fill in the name of .CEL cell. this space can be left blank when the cell to be processed is the cell that is being currently edited.

Expanded Cell Name

Type the name you want to assign to the expanded netlist cell. By convention, this file has a .exp extension. For example, if a netlist cell is named *b5.netl*, you name the expanded netlist cell *b5.exp*.

Precede hierarchical names with "/"

Select to generate hierarchical names preceded with a slash (/) to ensure consistency with SDF files.

Expand netlist cell with no instance

Select to expand netlist cells that contains no physical descriptions. Use this option if you have netlist instances containing only nets. If this option is deselected, and you have netlist instances with no physical descriptions, expansion will fail.

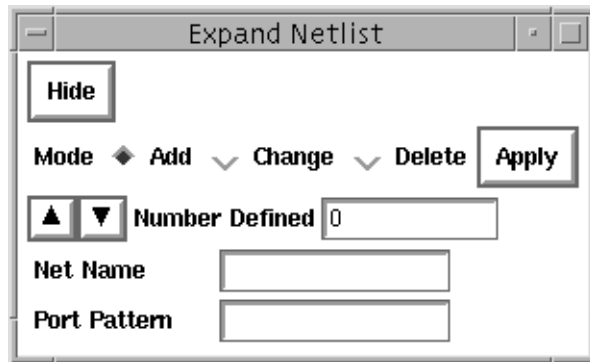
Stop at FRAM view cells only

If users turn this option on, the netlist expanding process will stop at .FRAM view cells only. All child cells will be expanded even if the corresponding .CEL views exist in the design library. the default value is off.

Print out net has a pin but no connections

Prints out a single pin net in the window.

Expand Netlist Dialog Box (Global Net Options)



Mode

After you type a *Net Name* and *Port Pattern*, select the appropriate action you want to take and click *Apply*.

Table 23-31

Option	Action
Add	Connects ports and nets that match the <i>Port Pattern</i> to the net specified by <i>Net Name</i> . If the net does not exist, Milkyway creates it.
Change	Changes previously defined connections for the net specified by <i>Net Name</i> .
Delete	Removes previously defined connections for the net specified by <i>Net Name</i> .

To display the name of a previously defined net, click the arrows.

Note:

To add to connections previously defined by this command, select Add. To overwrite connections previously defined by this command, select Change.

Apply

When you complete the dialog box, click to apply the net connection according to the information you provided in the dialog box fields and options.

Number Defined

Click the arrows to display the nets for which you have defined connections.

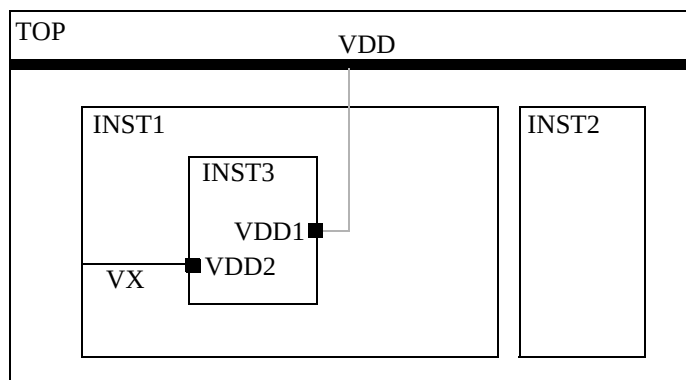
Net Name

Type the name of the top-level global net for which you want to define connections. If the net does not exist, Milkyway creates it.

Port Pattern

Type a pattern to match the names of ports and nets that you want to connect to the net specified by *Net Name*.

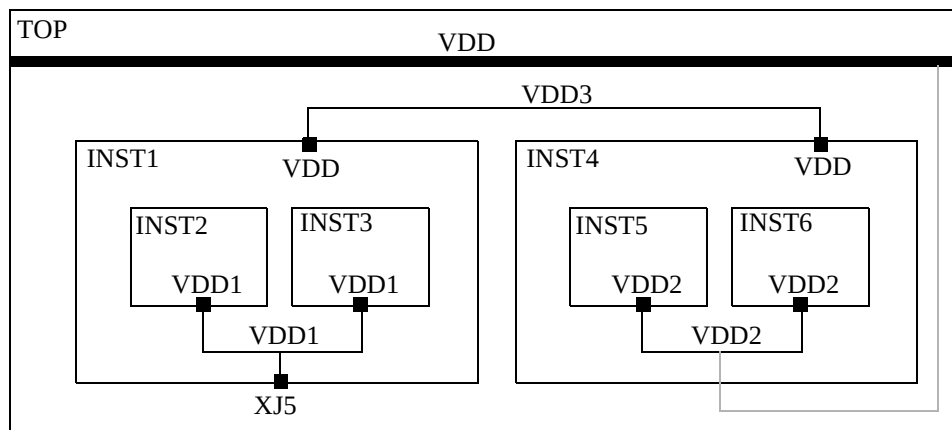
- If the object is a port, your Synopsys tool connects it to the specified net only if it is not connected to another net. For example, consider the case below.



If you specify a *Net Name* of *VDD* and a *Port Pattern* of *VDD.**:

- Your Synopsys tool connects port *VDD1* to the top-level net *VDD*, because port *VDD1* is not connected to another net.

- Your Synopsys tool does not connect port VDD2 to the top-level net VDD, because port VDD2 is already connected to net VX.
- If the object is a net, your Synopsys tool connects it to the specified net only if it is inside a cell instance and is not connected to a port. For example, consider the case below.



If you specify a *Net Name* of *VDD* and a *Port Pattern* of *VDD.**:

- Your Synopsys tool does not connect net VDD1 to the top-level net VDD, because it is connected to port XJ5.
- Your Synopsys tool connects net VDD2 to the top-level net VDD, because VDD2 is inside cell instance INST4 and is not connected to a port of INST4.
- Your Synopsys tool does not connect net VDD3 to the top-level net VDD, because it is not inside a cell instance.

cmCmdLoadConstraints

Loads boundary conditions and path constraints. The tool determines the arrival and required arrival times from the *dbSetPathConstraint* functions in the *Constraint File* you specify on the dialog box.

Note:

Loading a path constraints file generates a slack graph in the database. You can also generate a slack graph in the database using [cmCmdLoadSlackGraph](#).

Tools and Menu Commands

Astro: Timing Setup > Constraints – Load Path Constraints

Jupiter: Timing Setup > Constraints – Load Path Constraints

Milkyway: Netlist In > Load Timing Constraints

Prerequisites

You need to have a path constraints file or a Synopsys boundary file.

Load Constraint File Dialog Box

The menu command opens the following dialog box. Its options are described below.

Field	Value
Library Name	DESIGNS
Cell Name	99AVANTI_8111.CE
Constraint File	
Synopsys Boundary File	
Bus Naming Style In Design	[%d]
Bus Naming Style In Files	[%d]
Capacitance Multiplier	1.0
Resistance Multiplier	1.0
Allowable Timing Tolerance	1.0

Library Name

Type the name of the library containing the cell to which you want to load constraints.

Cell Name

Type the name of the cell to which you want to load constraints.

Constraint File

Type the name of the path constraints file you want to load.

Synopsys Boundary File

Type the name of the boundary file generated by the Synopsys Design Compiler, or leave blank if you are not using a Synopsys boundary file. You can adjust the capacitance and resistance specified in the boundary file using the *Capacitance Multiplier* and *Resistance Multiplier* options.

Capacitance Multiplier

Type the number by which the Synopsys tool should multiply the load values specified in the *Synopsys Boundary File* to get the actual capacitance values needed by the Synopsys tool.

Resistance Multiplier

Type the number by which the Synopsys tool should multiply the load values specified in the *Synopsys Boundary File* to get the actual resistance values needed by the Synopsys tool.

Allowable Timing Tolerance

Type a value indicating the maximum difference you want to allow between rise and fall delays on a path. If the difference is greater than the value you type, the software reports a warning message.

cmCmdLoadSlackGraph

Loads a slack graph to a cell. A slack graph contains timing constraints for the cell.

Tool

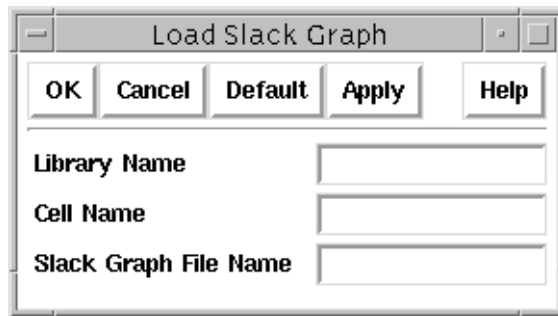
Milkyway

Prerequisites

Generate a slack graph. For information about generating a slack graph using an external timing analyzer, see the Synopsys *Timing-Driven Layout* manual.

Load Slack Graph Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Type the name of the library containing the cell to which you want to load a slack graph.

Cell Name

Type the name of the cell to which you want to load a slack graph.

Slack Graph File Name

Type the name of the file containing the slack graph you want to load.

cmCmdLoadTimingRequirements

Loads timing requirements from a Synopsys timing requirements file to the database.

Tools and Menu Commands

Astro: Timing Setup > Constraints – Load Timing Requirements

Jupiter: Timing Setup > Constraints – Load Timing Requirements

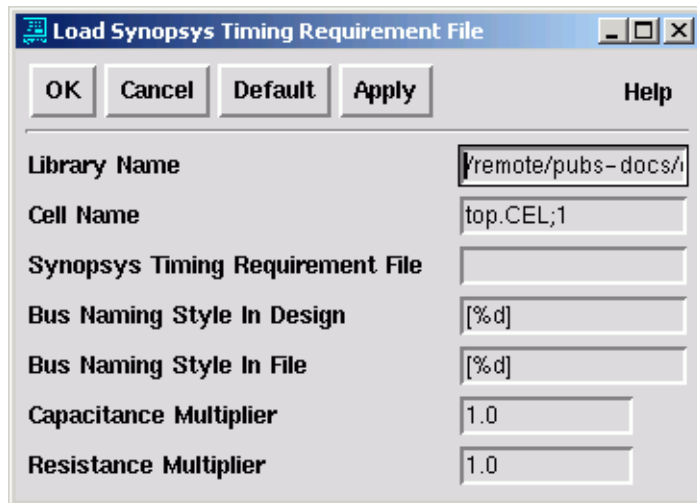
Milkyway: Netlist In > Timing Requirements

Prerequisites

You need a Synopsys timing requirements file.

Load Synopsys Timing Requirement Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Type the name of the library containing the cell for which you want to load timing requirements.

Cell Name

Type the name of the cell for which you want to load timing requirements.

Synopsys Timing Requirement File

Type the name of the file containing the timing requirements.

Bus Naming Style In Design

Enter the bus naming style used in the design.

Bus Naming Style In Files

Enter the bus naming style used in the timing requirements file.

Capacitance Multiplier

Type the number by which the Synopsys tool should multiply the load values specified in the *Synopsys Timing Requirement File* to get the actual capacitance values needed by the Synopsys tool.

Resistance Multiplier

Type the number by which the Synopsys tool should multiply the load values specified in the *Synopsys Timing Requirement File* to get the actual resistance values needed by the Synopsys tool.

25

db Commands for Milkyway

This chapter describes commands with the prefix *db*. The commands are listed alphabetically.

- [dbAbortCell](#)
- [dbAddAntennaLayerRule](#)
- [dbAddCellInstToPlanGroupByName](#)
- [dbAddCellInstToPlanGroupByNames](#)
- [dbAddCellInstToCellRegion](#)
- [dbAddFixedEdgeToScanChain](#)
- [dbAddFreeEdgeToScanChain](#)
- [dbAddInstGroupToScanChain](#)
- [dbAddIOText](#)

- dbAddLibSchemeFile
- dbAddLibTclFile
- dbAddNetSubTypeByName
- dbAddObjectToGroup
- dbAddPartialScanToScanChain
- dbAddRefLib
- dbAddViaSpot
- dbAllowToAddPGIOText
- dbArchiveCellToLib
- dbArchiveHierDesign
- dbAssignNetClassXtalkConstraints
- dbAssignNetMinMaxLayer
- dbAssignNetRouteRuleByHalfPerimeter
- dbAssignNetTimingSpacing
- dbAssignNetTopLayerProbe
- dbAssignNetVarRouteRule
- dbAssignNetXtalkClass
- dbAssignNetXtalkConstraints
- dbAssignVarRouteRule
- dbBrowser
- dbCellCheckSum

- dbCellCompare
- dbCellInstPlaceAndStatus
- dbCellReportHighFanoutNet
- dbChangeAccessMode
- dbChangeAllInstNames
- dbChangeAllNetNames
- dbChangeAllNetNamesByNetId
- dbChangeInstanceName
- dbChangeLibDbVersion19To20
- dbChangeRefLibAccessMode
- dbChangeTextLayer
- dbCheckCellData
- dbCheckCellTypeAndPortMatching
- dbCheckIdenticalCell
- dbCheckNetList
- dbCheckNetlistVsFram
- dbCheckRefLibControl
- dbClearAllNetXtalkProp
- dbClearAttachment
- dbClearCellDifferentialGroup
- dbClearCellEEQClass

- dbClearCellEQClass
- dbClearCellLEQClass
- dbClearCellFootPrintEQClass
- dbClearCellVTType
- dbClearCellXtalkConstraints
- dbClearCellXtalkLengthWeight
- dbClearLengthRouteRuleTable
- dbClearLibAntennaRules
- dbClearLibraryCellsVTType
- dbClearLRFillerRule
- dbClearMustJoinPortByCell
- dbClearNetLengthRouteRuleFromTable
- dbClearRefLibControl
- dbClearVTFiller
- dbCloseCell
- dbCloseLib
- dbCloseQueryFile
- dbCompNetHalfBBox
- dbConnect
- dbConnectPortInstByName
- dbConvertPortToDiodePort

- dbConvertRegionToMacroCell
- dbCopyCell
- dbCopyCellFromAnotherLib
- dbCopyCellToAnotherLibDisk
- dbCopyCellWithViewFromAnotherLib
- dbCopyLib
- dbCopyTextAndRectangle
- dbCreateBaseArray
- dbCreateBaseArrayRow
- dbCreateCell
- dbCreateCellBndyForChildCell
- dbCreateCellBoundary
- dbCreateCellBoundaryForCell
- dbCreateCellInst
- dbCreateCellInstArray
- dbCreateCellEEQClass
- dbCreateCellInstIgnoreECO
- dbCreateCellInstWithoutCheckStatus
- dbCreateCellLEQClass
- dbCreateCellProperty
- dbCreateCellRegion

- dbCreateCellRow
- dbCreateChildNet
- dbCreateChipPort
- dbCreateContact
- dbCreateError
- dbCreateErrorClass
- dbCreateErrorComponent
- dbCreateErrorObj
- dbCreateErrorType
- dbCreateGLink
- dbCreateGlobalNetByScanChain
- dbCreateGroup
- dbCreateHierNet
- dbCreateHorizontalWire
- dbCreateLeqPortGroup
- dbCreateLib
- dbCreateLogicView
- dbCreateModelView
- dbCreateMustJoinNets
- dbCreateNet
- dbCreateNode

- dbCreateParamSet
- dbCreateParentNet
- dbCreatePath
- dbCreatePcellDefinition
- dbCreatePin
- dbCreatePlanGroupAndStatus
- dbCreatePolygon
- dbCreatePort
- dbCreatePreRoute
- dbCreateProperty
- dbCreatePseudoPath
- dbCreateRectangle
- dbCreateRegionPin
- dbCreateRegionPort
- dbCreateScanChain
- dbCreateSGEdge
- dbCreateSGPrimaryNode
- dbCreateText
- dbCreateVerticalWire
- dbCreateWireMaster
- dbDefineAbsXtalkConstraint

- dbDefineAntennaAccumMode
- dbDefineAntennaLayerRatioScale
- dbDefineAntennaRule
- dbDefineCTSPin
- dbDefineCTSPort
- dbDefineDifferentialGroup
- dbDefineIgnorePin
- dbDefineIgnorePort
- dbDefineLengthRouteRuleTable
- dbDefineLeqPortGroup
- dbDefineSyncPin
- dbDefineSyncPort
- dbDefineVarRouteRule
- dbDefineXtalkConstraint
- dbDeleteAllVarRouteRules
- dbDeleteCell
- dbDeleteCellAttachedFile
- dbDeleteCellInstFromCellRegion
- dbDeleteCellRowDoubleBack
- dbDeleteFloatingCell
- dbDeleteInstAndStitch

- dbDeleteLibAttachedFile
- dbDeleteLeqPortGroup
- dbDeleteLogicData
- dbDeleteLogicView
- dbDeleteModelData
- dbDeleteModelView
- dbDeleteNetClass
- dbDeleteObject
- dbDeleteOneLayerRectangle
- dbDeletePcellDefinition
- dbDeletePin
- dbDeletePlacementChain
- dbDeletePolygonByLayer
- dbDeleteProperty
- dbDeletePropertyById
- dbDeletePropertyByName
- dbDeleteRectangleByLayer
- dbDeleteRefLibByName
- dbDeleteScanChain
- dbDeleteTieHiLoCell
- dbDisconnect

- dbDisconnectPortInstByName
- dbDisJoinPort
- dbDisplayCellRailProperty
- dbDisplayPlanGroup
- dbDowngradeLib
- dbDumpAllCriticalNets
- dbDumpAllNetLengthRouteRuleFromTable
- dbDumpAllNetMinMaxLayer
- dbDumpAllNetTimingSpacing
- dbDumpAllNetTopLayerProbe
- dbDumpAllVarRouteRules
- dbDumpAllVTFillers
- dbDumpCell
- dbDumpCellAttachedFile
- dbDumpCellDifferentialGroup
- dbDumpCellPlacement
- dbDumpCellVTType
- dbDumpCheckNetlistVsFram
- dbDumpChildParentInfo
- dbDumpCTSPin
- dbDumpFootPrintEQ

- dbDumpFreezeRoutingNets
- dbDumpGPortTable
- dbDumpHierRefLibControl
- dbDumpIgnorePin
- dbDumpLengthRouteRuleTable
- dbDumpLeqPortGroup
- dbDumpLeqPortGroupCandidates
- dbDumpLibAntennaRules
- dbDumpLibAttachedFile
- dbDumpLibCatalog
- dbDumpLibraryCellsVTType
- dbDumpLRFillerRule
- dbDumpNetXtalkProp
- dbDumpPlacementChains
- dbDumpPropertyTable
- dbDumpRefLibControl
- dbDumpScanChain
- dbDumpScanPortPairs
- dbDumpSingleViewPageNumber
- dbDumpSyncPin
- dbDumpTechFile

- dbDumpViewPageNumber
- dbDumpViewRecordNumber
- dbDumpVTFiller
- dbDumpXtalkConstraints
- dbEliminatePins
- dbExistsCellAttachedFile
- dbExistsLibAttachedFile
- dbExpungeAllObjByTypeId
- dbExpungeAllObjByTypeName
- dbExpungeObjTypeById
- dbExpungeObjTypeByName
- dbExtractCellPGPosition
- dbExtractPGPosition
- dbExtractPins
- dbFetchCellPropertyValueByName
- dbFetchContactCodeInfo
- dbFetchContactCodeTypes
- dbFetchContactGeom
- dbFetchContactNames
- dbFetchDesignRuleInfo
- dbFetchDesignRuleTypes

- dbFetchFringeCapInfo
- dbFetchLayerIdByName
- dbFetchLayerIdByNumber
- dbFetchLayerIdList
- dbFetchLayerInfo
- dbFetchLayerTypes
- dbFetchNonDefaultContactCodeInfo
- dbFetchObject
- dbFetchObjectField
- dbFetchPinInstByPortInst
- dbFetchProperty
- dbFetchPropertyFieldAndName
- dbFetchPropertyValueByName
- dbFetchScanTypes
- dbFetchTechInfo
- dbFetchTechTypes
- dbFileCompare
- dbFixCellData
- dbFixNetList
- dbForceRecompCellBBox
- dbForceRecursiveUpdateAllHierarchy

- `dbForceUpdateHierarchy`
- `db-foreach`
- `dbFormNetClass`
- `dbFreezeRoutingOnNets`
- `dbGenNetList`
- `dbGetLayerNumberFromName`
- `dbGetLayerNameFromNumber`
- `dbGetLibIdByCellId`
- `dbGetNetByName`
- `dbGetPinByName`
- `dbGetPortByName`
- `dbGetRegionIdByCellInstId`
- `dbGetRegionPortByName`
- `dbGetRegionPortDirection`
- `dbGetAccessMode`
- `dbGetAllInstFromFence`
- `dbGetAllInstFromGroup`
- `dbGetAllLibCellInfo`
- `dbGetCellAttachedFile`
- `dbGetCellByInstPath`
- `dbGetCellFullNameById`

- dbGetCellInstById
- dbGetCellInstByName
- dbGetCellInstRefLib
- dbGetCellNameAndViewById
- dbGetCellNameById
- dbGetCellViewVersions
- dbGetCheckLibMode
- dbGetConnectedObject
- dbGetConstraintsCellId
- dbGetCurrentLibId
- dbGetDiodePortByName
- dbGetLayerNameByMaskName
- dbGetLayerNumberByMaskName
- dbGetLibAttachedFile
- dbGetLibBusNameStyle
- dbGetLibDbVersion
- dbGetLibIdByName
- dbGetLibNameById
- dbGetLibraryPath
- dbGetLibRefCtrlFileMode
- dbGetLModelSubType

- dbGetMaxMetalRoutingLayer
- dbGetMsgLevel
- dbGetNetVarRouteRuleName
- dbGetObjectTypeName
- dbGetOneLibCellInfo
- dbGetPinNetByName
- dbGetPlanGroupByName
- dbGetRefControlFileMode
- dbGetRefLib
- dbGetTolerantMode
- dbGetUserHierDelimiter
- dbGetVarRouteRule
- dbHierCopyToExistLib
- dbHierCopyToNewLib
- dbHierDelete
- dbHierPurge
- dbHierSave
- dbHierSplit
- dbId?
- dbImportLogicData
- dbImportModelData

- dbInitShapeStruct
- dbIOPinTextToPin
- dbIsCaseSensitive
- dbIsCellOpen
- dbLastECONum
- dbLibraryCompare
- dbLibReportTiming
- dbListPcellDefinitions
- dbListUnitRange
- dbLoadPropertyDef
- dbMakeScanChainByCellInst
- dbMakeScanChainByMaster
- dbMakeWireMasterId
- dbMarkCellType
- dbMarkScanPortPairs
- dbMemProfile
- dbMemProfileReport
- dbMergeLib
- dbMoveAllCellRegions
- dbMoveAllNetMember
- dbMoveCellOrigin

- dbNetChecksum
- dbObjId?
- dbOpenCell
- dbOpenLib
- dbOpenLibrary
- dbOpenQueryFile
- dbOpenRefLibTree
- dbPadToPin
- dbPinToPad
- dbPinToPadAndSnap
- dbPrintAllNetClasses
- dbPrintApplParam
- dbPrintCellHierReport
- dbPurgeAllEmptyNet
- dbPurgeAllPlanGroup
- dbPurgeAllSingleNet
- dbPurgeCellInstByName
- dbPurgeCellInstMaster
- dbPurgeCTSPin
- dbPurgeIgnorePin
- dbPurgeNet

- dbPurgeNetByName
- dbPurgeRefLib
- dbPurgeLibStaticNoiseMargin
- dbPurgeLibStaticNoiseProperty
- dbPurgeObject
- dbPurgePin
- dbPurgePort
- dbPurgeRegionPin
- dbPurgeRegionPort
- dbPurgeSyncPin
- dbPurgeUnusedWireMasters
- dbQuery
- dbQueryCPUTime
- dbQueryMem
- dbQueryNetClassByName
- dbQueryObjectByExtendDist
- dbQueryPeakMem
- dbQueryPeriodPeakMem
- dbQueryPeriodPeakMemBegin
- dbQuitShapeStruct
- dbRebuildLib

- `dbRecoverLib`
- `dbRecursiveUpdateAllHierarchy`
- `dbRefLibChecksum`
- `dbRefreshLib`
- `dbRegisterProperty`
- `dbRemoveInstGroupFromScanChain`
- `dbRemoveObjectFromGroup`
- `dbRenameCell`
- `dbRenameInstAndNet`
- `dbRenameLib`
- `dbReOpenRefLibForWrite`
- `dbReplaceAllCellBoundary`
- `dbReplaceCellBoundary`
- `dbReplaceCellInst`
- `dbReplaceCellTextChar`
- `dbReplaceCellTextStr`
- `dbReplaceChildCell`
- `dbReplaceRegionWithInst`
- `dbReplaceRefLibByName`
- `dbReplaceScanCell`
- `dbReplaceTechFile`

- dbResetAllNetTimingSpacing
- dbResetAllNetTopLayerProbe
- dbResetCellMustJoinPin
- dbRestoreHierDesign
- dbRestorePin
- dbRollBackCell
- dbSaveCell
- dbSetArrivalTime
- dbSetBaseArraySubTile
- dbSetCellInstECOStatus
- dbSetCellInstECOStatusByName
- dbSetCellInstFixStatusByName
- dbSetCellInstOrigin
- dbSetCellInstGroupId
- dbSetCellInstPlacement
- dbSetCellInstStatus
- dbSetCellMustJoin2Pin
- dbSetCellMinMaxLayer
- dbSetCellMustJoinAllPin
- dbSetCellMustJoinPin
- dbSetCellMustJoinPinPattern

- dbSetCellPattern
- dbSetCellPortTypes
- dbSetCellPortTypeExtraView
- dbSetCellPowerValue
- dbSetCellPRBoundaryPolygon
- dbSetCellRegionBBox
- dbSetCellRegionTargetPercent
- dbSetCellRowMirror
- dbSetCellType
- dbSetCellVTType
- dbSetCheckLibMode
- dbSetCurrentRegionNumber
- dbSetEcoWeight
- dbSetErrorClassColor
- dbSetErrorClassPattern
- dbSetErrorClassStatus
- dbSetErrorObjStatus
- dbSetErrorStatus
- dbSetErrorTypeStatus
- dbSetFringeFactor
- dbSetGroupParentId

- dbSetHierRefLibMode
- dbSetIOPad
- dbSetLayerTimeInfo
- dbSetLayoutSubType
- dbSetLibBusAppendName
- dbSetLibBusNameStyle
- dbSetLibDefaultCreationAccess
- dbSetLibRefCtrlFileMode
- dbSetLibraryCellsVTType
- dbSetMacroCellOrient
- dbSetMaxMetalRoutingLayer
- dbSetMsgLevel
- dbSetLModelSubType
- dbSetMaxCapacitance
- dbSetMaxIntraCapDistRatio
- dbSetMaxTransition
- dbSetNetCriticality
- dbSetNetSubTypeByName
- dbSetNetTrace
- dbSetNetTraceById
- dbSetNetType

- dbSetNetTypeByName
- dbSetOperatingVoltage
- dbSetPathConstraint
- dbSetPinAccessDir
- dbSetPinAccessDirByName
- dbSetPinLayerNumber
- dbSetPinLocation
- dbSetPinPointArray
- dbSetPinStatusFlag1
- dbSetPlanGroupAndStatus
- dbSetPlanGroupBBox
- dbSetPlanGroupBBoxByName
- dbSetPolygonPointArray
- dbSetPortInstLocalStatus
- dbSetPortDirection
- dbSetPortInstNetId
- dbSetRefLibControl
- dbSetRefControlFileMode
- dbSetReqArrivalTime
- dbSetScanChainDefaultPriority
- dbSetSkipRefLibReadLock

- dbSetStdCell
- dbSetStdCellOrient
- dbSetTechFileName
- dbSetUserLogFile
- dbSetUseUmask
- dbSetXtalkLengthWeight
- dbSkipRouteOnNet
- dbSpecifyLRFiller
- dbSpecifyVTFiller
- dbStartCellRowDoubleBack
- dbStoreCellAttachedFile
- dbStoreLibAttachedFile
- dbSwapCellViewVersions
- dbSyncRefLib
- dbTruncateCellText
- dbTruncateText
- dbUpdateCellInstNumberOfCellRow
- dbUpdateCellProperty
- dbUpdateCellRailProperty
- dbUpdateHierarchy
- dbUpdateNetStatus

- `dbUpdatePcellDefinition`
- `dbUpdateProperty`
- `dbVerifyCellEEQClass`
- `dbVerifyCellLEQClass`
- `dbVerilogTranslateString`

dbAbortCell

Closes a cell without saving your changes. If `dbAbortCell` succeeds, the Synopsys tool returns *#t*.

Syntax

```
dbAbortCell cellId
```

where the argument is as follows:

cellId

Database object ID of any open cell in the current library that you want to close without saving

Valid values:

- Cell ID
- (`geGetEditCell`)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

See Also

[dbCloseCell](#)
[dbRollBackCell](#)
[dbSaveCell](#)

dbAddAntennaLayerRule

Defines an advanced antenna rule for the specified layer and stores it in the library.

Tool

Milkyway

Prerequisites

The antenna rule for the specified mode must first be defined with [dbDefineAntennaRule](#).

Syntax

```
dbAddAntennaLayerRule libId mode layer ratio diodeRatio
```

where the arguments are as follows:

<i>libId</i>	The Id of an opened library: (<i>dbGetCurrentLibId</i>).
<i>mode</i>	The mode in the corresponding command <i>dbDefineAntennaRule</i> . Valid values: 1 - 6.
<i>layer</i>	Name of the valid metal layer or cut layer in the library.
<i>ratio</i>	The maximum allowable ratio of the antenna area to the gate area if the antenna is not protected by any diode. Valid values: Any number.

diodeRatio

The maximum allowable ratio of the antenna area to the gate area if the antenna is not protected by any diode.

Valid values:

'(v0 v1 v2 v3): Specify the allowable ratio in the antenna area to the gate area if the antenna is protected by a diode. If the output pin protection value is (dp), the allowable ratio will be:

$((dp + v1) * v2 + v3)$, if $(dp) > (v0)$
layerMaxRatio, if $(dp) \leq (v0)$

The (dp) is specified with `defineDiodeProtection` command in the CLF file and the default value is '(0 1 0 0).

Example

```
define _libId (dbGetCurrentLibId)
dbDefineAntennaRule _libId 4 2 400 20
dbAddAntennaLayerRule _libId 4 "metal1" 400 '(0.336 -0.5 400
2400)
dbAddAntennaLayerRule _libId 4 "metal2" 400 '(0.336 -0.5 400
2400'
dbAddAntennaLayerRule _libId 4 "metal3" 400 '(0.336 -0.5 400
2400)
```

See Also

[dbClearLibAntennaRules](#) deletes all of the antenna rules stored in the library.

[dbDefineAntennaRule](#) defines an advanced antenna rule for the specified mode and stores it in the library.

[dbDumpLibAntennaRules](#) dumps all of the antenna rules stored in the library to a file.

dbAddCellInstToPlanGroupName

Adds a cell instance to the specified plan group.

dbAddCellInstToPlanGroupName

Syntax

```
dbAddCellInstToPlanGroupByName cellId pgId "ciName"
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

pgId

Database object ID of the plan group in the cell.

Valid values:

ID of any plan group in the cell

ciName

Name of the cell instance that you want to add to the plan group

dbAddCellInstToPlanGroupByNames

Adds the specified cell instances to a plan group.

Syntax

```
dbAddCellInstToPlanGroupByNames <objectId cellId>  
    <objectId pgId> <list ciNameList>
```


where the arguments are as follows:

<code>cellId</code>	Database object ID of the cell to which the information applies.
<code>pgId</code>	Database object ID of the plan group in the cell.
<code>ciNameList</code>	List of cell instance names to add to the plan group.

Description

The `dbAddCellInstToPlanGroupByNames` command adds the specified cell instances to a plan group. Invalid cell instance names (for example, names of non-existing cell instances) are skipped.

See Also

[dbAddCellInstToPlanGroupByNames](#)

Example

```
dbAddCellInstToPlanGroupByNames (geGetEditCell) #x0800  
' ("AND1" "AND2" "AND3")
```

dbAddCellInstToCellRegion

Adds a cell instance to a cell region. This is equivalent to assigning a specific cell instance to a region. The instance's visible location does not change as a result of this function.

Syntax

```
dbAddCellInstToCellRegion cellId regionId cellInstId
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the region information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

regionId

Database object ID of the region to which the cell instance information applies

Valid values:

Region ID of any region in the specified cell

cellInstId

Database object ID of the cell instance to add

Valid values:

Cell instance ID of any cell instance in the specified cell

See Also

[dbGetCellInstByName](#)

[dbDeleteCellInstFromCellRegion](#)

dbAddFixedEdgeToScanChain

Adds a set of fixed edges to a scan chain in the open cell.

Note:

If this function references a cell instance that is not explicitly placed in a scan chain group, the Synopsys tool creates a scan chain group containing only that cell instance (see [dbAddInstGroupToScanChain](#)). The tool assigns it a priority equal to the default priority (except for the primary input scan cell, which is always at the beginning of the scan chain, and the primary output scan cell, which is always at the end of the scan chain). You set the default priority using [dbSetScanChainDefaultPriority](#).

Tool

Milkyway

Prerequisites

Before adding fixed edges, create the scan chain using [dbCreateScanChain](#). Indicate whether ports on scan cells are input ports or output ports using [dbSetCellPortTypes](#).

Syntax

```
dbAddFixedEdgeToScanChain (geGetEditCell) chainName  
'(((srcInstName srcPortName)  
(destInstName destPortName))}...)
```

where the arguments are as follows:

<i>chainName</i>	Name of the scan chain to which you are adding a fixed edge. Valid values: Name of any scan chain in the open cell.
------------------	--

<i>srcInstName</i>	Name of the source cell instance in the fixed edge. Valid values: Name of a scan cell instance in the open cell for which one of the following is true: The instance is in the same scan chain group as the cell instance specified by <i>destInstName</i>. The instance is not in a user-defined scan chain group. A null string ("") indicates that the destination port is a top-level port.
<i>srcPortName</i>	Name of the output port of the specified source cell instance. Valid values: Name of any output port on the specified source cell instance.
<i>destInstName</i>	Name of the destination cell instance in the fixed edge. Valid values: Name of a scan cell instance in the open cell for which one of the following is true: The instance is in the same scan chain group as the cell instance specified by <i>srcInstName</i>. The instance is not in a user-defined scan chain group. A null string ("") indicates that the destination port is a top-level port.
<i>destPortName</i>	Name of the input port of the specified destination cell instance. Valid values: Name of any input port on the specified destination cell instance.

dbAddFreeEdgeToScanChain

Adds a set of free edges to a scan chain in the open cell.

Note:

If this function references a cell instance that is not explicitly placed in a scan chain group (see [dbAddInstGroupToScanChain](#)), the Synopsys tool creates a scan chain group containing only that cell instance. The tool assigns it a priority equal to the default priority (except for the primary input scan cell, which is always at the beginning of the scan chain, and the primary output scan cell, which is always at the end of the scan chain). You set the default priority using [dbSetScanChainDefaultPriority](#).

Tool

Milkyway

Prerequisites

Before adding free edges, create the scan chain using [dbCreateScanChain](#). Indicate whether ports on scan cells are input ports or output ports using [dbSetCellPortTypes](#).

Syntax

```
dbAddFreeEdgeToScanChain (geGetEditCell) chainName  
'({(srcInstName srcPortName)(destInstName destPortName))}...)
```

where the arguments are as follows:

<i>chainName</i>	Name of the scan chain to which you are adding a free edge. Valid values: Name of any scan chain in the specified cell.
------------------	---

<i>srcInstName</i>	Name of the source cell instance in the free edge. Valid values: Name of any scan cell instance in the open cell, or a null string ("") to indicate that the source port is a top-level port.
<i>srcPortName</i>	Name of the output port of the specified source cell instance. Valid values: Name of any output port on the specified source cell instance (or top-level cell).
<i>destInstName</i>	Name of the destination cell instance in the free edge. Valid values: Name of any scan cell instance in the specified top cell.
<i>destPortName</i>	Name of the input port of the specified destination cell instance. Valid values: Name of any input port on the specified destination cell instance.

dbAddInstGroupToScanChain

Adds a scan cell instance group to a scan chain in the open cell.

Tool

Milkyway

Prerequisites

Before adding fixed edges, create the scan chain using [dbCreateScanChain](#). Indicate whether ports on scan cells are input ports or output ports using [dbSetCellPortTypes](#).

Syntax

```
dbAddInstGroupToScanChain (geGetEditCell) priority  
chainName  
  groupName '({instName} ...)
```

where the arguments are as follows:

<i>priority</i>	Number to indicate the priority for placement of cell instances in the group. If a group has a lower priority, the Synopsys tool places cell instances in the group closer to the beginning of the scan chain. If a group has a higher priority, the Synopsys tool places cell instances in the group closer to the end of the scan chain. Valid values: Any integer from 0 to 127.
<i>chainName</i>	Name of the scan chain to which you are adding a scan cell instance group. Valid values: Name of any scan chain in the open cell.
<i>groupName</i>	Name you want to assign to the scan cell instance group. Valid values: Any string up to 31 characters.
<i>instName</i>	Name of a cell instance you want in the group. Valid values: Name of any scan cell in the open cell.

dbAddIOText

Adds I/O text.

Syntax

```
dbAddIOText cellId padType textType layerNumber textHeight
```

where the arguments are as follows:

cellId

The ID for the top-level cell.

padType

The string *fakePin* or *pad [pinName]*.

textType

Designates which data you want to appear in the text, *netName*, *portName*, or *cellInstName*.

layerNumber

The integer layer number for the text.

textHeight

The integer for text height; 10 to 30 works well.

Example

```
dbAddIOText (geGetEditCell) "pad" "netName" 5 3
```

dbAddLibSchemeFile

Registers a Scheme file that contains functions that are available to all parameterized cells used with this library.

Syntax

```
dbAddLibSchemeFile libId "supportFileName"
```


where the arguments are as follows:

libId

Database object ID of the current library from which you want to reference another library.

Valid values:

- Library ID
- (dbGetCurrentLibId)

For more information, see “Specifying Libraries” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

supportFileName

Name of the support file.

dbAddLibTclFile

Stores the specified Tcl file into the library and loads it into the current Milkyway database environment session.

Syntax

```
dbAddLibTclFile < objectId libId> < string fileName >
```

where the arguments are as follows:

<code>libId</code>	Database object ID of the library. (objectId is the type of the libID argument)
<code>fileName</code>	Name of the Tcl file.

Description

The `dbAddLibTclFile` function stores the specified Tcl file into the library and loads it into the current Milkyway session. This Tcl library support file is loaded for use with parameterized cells (Pcell). If a previous Tcl library support file exists, it is overwritten by the new file.

Example

```
dbAddLibTclFile 1 "my_tcl_file"
```

dbAddNetSubTypeByName

Adds a new net subtype to a specific net in the cell.

Syntax

```
dbAddNetSubTypeByName cellId "netName" "netType"
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- `(geGetEditCell)`

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

netName

Name of the net to which you want to add the type

Valid values:

Name of any net in the open cell

netType

Type you want to add to the net

Valid values:

"core", "pad", "coreAndPad", "CTSNet", "CTSRootNet",
"nonCTSNet"

dbAddObjectToGroup

Adds an object to a group.

Syntax

```
dbAddObjectToGroup cellId groupId objectId position
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the group information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

groupId

Database object ID of the group to which the object information applies

Valid values:

Group ID of any group in the cell

objectId

Database object ID of the object to add to the group

Valid values:

Object ID of any object within the cell

Note:

Objects do not have to be homogeneous within the group. For example, you can place a net and a cell together in one group.

position

A positive integer specifying the position of the object in the group. This is meaningful only in an ordered group.

See Also

[aprCreateGroup](#)

[dbDeleteObject](#)

[dbPurgeObject](#)

[dbRemoveObjectFromGroup](#)

dbAddPartialScanToScanChain

Adds the specified partial scan cell instances to a scan chain.

Syntax

```
dbAddPartialScanToScanChain cellId "chainName"  
'({lItem}...)
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (*geGetEditCell*)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

chainName

Name of the scan chain to which you are adding a partial scan

Valid values:

Name of any scan chain in the open cell

Item

List of the cell instance names that you want in the group

Valid values:

Names of any cell instances in the open cell

dbAddRefLib

Creates a reference library. If `dbAddRefLib` successfully links another library to the main library, the Synopsys tool displays the database object ID of the reference library.

Prerequisite

The library for which you want to create a reference library must be open in write (W) mode.

Syntax

```
dbAddRefLib libId "refLibName"
```

where the arguments are as follows:

libId

Database object ID of the current library from which you want to reference another library

Valid values:

- Library ID
- (dbGetCurrentLibId)

For more information, see “Specifying Libraries” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

refLibName

Name of the library you want the current library to reference

Valid values: Any string with 127 characters or fewer corresponding to an existing library in the current directory

See Also

[dbDeleteRefLibByName](#)

[dbReplaceRefLibByName](#)

dbAddViaSpot

Adds a via spot (a BPV contact array) for the specified pin.

Syntax

```
dbAddViaSpot<objectId cellId> <string pinName>  
    <int groupNumber> <string contactName>  
    <boolean rotate90> <string accessDir>  
    <list pointArray>
```

where the arguments are as follows:

cellId	Database object ID of the cell.
pinName	Pin for which this via spot will be added.
groupNumber	Via spot group number to which to add this via spot.
contactName	Contact code upon which the via is based.
rotate90	#t if the via is rotated 90 degrees.
accessDir	Access direction of the via. Valid strings: "L" or "l" Left access direction "R" or "r" Right access direction "U" or "u" Up access direction "D" or "d" Down access direction
pointArray	List of two points describing the via bounding box (in other words, the lower left corner and upper right corner of the bounding box).

Description

The `dbAddViaSpot` command adds a via spot (a BPV contact array) for the specified pin. The via spot is added to the specified via spot group. If the specified group does not exist, the group will be created and the via spot is added into the group. If the group is not specified, the via spot is added into the default group (the first group found in the database).

Example

```
dbAddViaSpot (geGetEditCell) "A" 1 "via1" #f "r" `((2000
2000) (2050 2050))
```

dbAllowToAddAllPGIOText

Sets the mode for the Scheme [dbAddIOText](#) function.

Syntax

```
dbAllowToAddAllPGIOText "booleanValue"
```

where the arguments are as follows:

"booleanValue"

- a Scheme boolean

valid values:

- *#t*

Specifies for `dbAddIOText` to add text to all power and ground nets, including power and ground nets with multiple pins.

- *#f*

Specifies for `dbAddIOText` to behave as before (default value).

dbAllowToAddPGIOText

Sets the mode for the Scheme [dbAddIOText](#) function.

Syntax

```
dbAllowToAddPGIOText "boolean value"
```

where the argument is as follows:

"booleanValue"

A Scheme Boolean

Valid values:

- *#t* causes `dbAddIOText` to add text to all power and ground nets, including power and ground nets with multiple pins.
- *#f* causes `dbAddIOText` to behave as before (the default).

The following table illustrates the relationship between the `dbAllowToAddPGIOText` and `dbAllowToAddAllPGIOText` functions.

dbAllowToAddPGIOText		
	#f	#t
	#f	#t
dbAllowToAddAllPGIOText	Text <i>not</i> added to power/ground nets	Text added to <i>single-pin</i> power/ground nets
	Text added to <i>all</i> power/ground nets	Text added to <i>all</i> power/ground nets

dbArchiveCellToLib

Archives a cell design to your specified archive library. All design data (including unit tiles, library cell equivalence information, and library global port tables) are copied into a single library. The archive library will not contain any hierarchical reference information.

The versions, views, and reference cells that are archived can be controlled by optional arguments. When the default settings are used, all versions, views, and reference cells of the cell design are archived.

When `dbArchiveCellToLib` is executed successfully, a *true* message is returned. When execution is unsuccessful, a *false* message is returned.

Prerequisites

The library containing the cell to be archived must be open. If the library uses reference control files, you must set the reference control file mode with the [dbSetRefControlFileMode](#) command before you open the library. Otherwise cells that are referenced by the top cell and are located in reference libraries will not be copied.

Syntax

```
dbArchiveCellToLib cellName archiveLibName  
[versionMode topCellOnly overWrite] [userDefinedProp]  
[excludeViewsProp] [includeViewsProp] [allViewCellsProp]  
[warningLevelProp] [skipDupCellsProp]
```

where the arguments are as follows:

<i>cellName</i>	Name of the source top cell. Valid values: Syntax: <i>sourceCellName</i>[:<i>sourceVersion</i>] [=<i>newCellName</i>] “,” – Separates <i>sourceCellName</i> from <i>sourceVersion</i>. “=” – Separates <i>sourceCellName</i> from <i>newCellName</i>. <i>sourceCellName</i> – Name of the source top cell to be archived. <i>sourceVersion</i> – Version of the source top cell to archive (optional). <i>newCellName</i> – New cell name to which the source top cell will be copied in the archive library (optional). • Examples: If the value is top;3, the third version of the cell called top will be archived. If the value is top=newTop, the cell called top will be archived as newTop. If the value is top;3=newTop, the third version of the cell top will be archived as newTop.
<i>archiveLibName</i>	Name of the archive library. Valid values: Any existing or nonexisting library. If the library does not exist, it will be created using the same technology information as the source library.

versionMode

Specifies which cell versions are archived (optional).

Note:

If a particular version is specified in *cellName* for the top cell, it takes effect only if *versionMode* has a 1 or 2 value.

Valid values:

0: Archives all versions (default).

1: Archives the most recent version only.

2: Archives the version used to create the particular cell instance only.

topCellOnly

Specifies whether to archive all reference cells or only the top cell (optional).

Valid values:

0: Archives the top cell and all reference cells (default).

1: Archives the top cell only.

overwrite

Specifies if existing archived cells should be overwritten (optional).

Valid values:

0: Existing cells will not be overwritten; instead new versions will be created in the archive library (default).

1: The most current version of the existing cells will be overwritten. If the cell does not exist in the archive, a new version will be created.

`userDefinedProp`

A property string that describes a user-defined property to be stored with the archived cells (optional). This property is stored with all archived cell views and versions.

Valid values:

Syntax: [*propName*=]*propValue*

"=" – Separates the property name and property value.

propName – The name of the property (optional). If not specified, the default property name will be `DB_ARCHIVE_PROP`. If the property name already exists, it will be updated if the property value is different.

propValue – The string value of the property.

Note:

`EXCLUDE_VIEWS` and `INCLUDE_VIEWS` are key property names. They cannot be used as property names in a user archive property.

`excludeViewsProp`

A property string that describes the views to be excluded from the archive library (optional). This property is not stored with the archived cells.

Valid values:

Syntax: `EXCLUDE_VIEWS=propValue`

"=" – Separates the property name and property value.

`EXCLUDE_VIEWS` – A key property name; cannot be used as a user archive property (see *userDefinedProp*).

propValue – A comma- or space-delimited list of cell views that should not be archived. All other cell views will be included in the archive.

includeViewsProp

A property string that describes the views to be included in the archive library (optional). This property is not stored with the archived cells.

Valid values:

Syntax: INCLUDE_VIEWS=*propValue*

“=” – Separates the property name and property value.

INCLUDE_VIEWS – A key property name; cannot be used as a user archive property (see above).

propValue – A comma- or space-delimited list of cell views that should be archived. All other unlisted views will be excluded from the archive.

Note:

The EXCLUDE_VIEWS and INCLUDE_VIEWS properties are mutually exclusive. They cannot be used in the same function call.

skipDupCellsProp

Can be either SKIP_DUP_CELLS=TRUE or SKIP_DUP_CELLS=FALSE. If TRUE, only the first copy of each cell is archived. Duplicated cells are not archived. By default, the value is FALSE.

See Also

[dbArchiveHierDesign](#)

dbArchiveHierDesign

Archives the specified cell design data to a specified location. To meet the requirements of portability and hierarchical referencing, `dbArchiveHierDesign` maintains the original library directory structure under the archive location.

All necessary data for the main cells referenced in the top cell are copied over to the archive location. The library references maintained in the archive libraries are updated to reflect the new locations of the reference libraries in the archive location.

If `dbArchiveHierDesign` is executed successfully, a *true* message is returned. If unsuccessful, a *false* message is returned.

Prerequisites

All libraries must be closed.

Syntax

```
dbArchiveHierDesign "libName" "cellName" "archiveDirectory"
```

where the arguments are as follows:

libName

Path (relative or absolute) and name of the library containing the cell to be archived

Valid values:

Any existing library

cellName

Cell name of the cell to be archived

archiveDirectory

Location to which the cell will be archived

Valid values:

Any existing or nonexisting directory

If the directory already exists, the contents might be overwritten.

See Also

[dbRestoreHierDesign](#)

[dbArchiveCellToLib](#)

dbAssignNetClassXtalkConstraints

You can assign crosstalk constraints to nets belonging to a crosstalk class. Each net can have more than one constraint.

Tool

Milkyway

Syntax

```
(dbAssignNetClassXtalkConstraints cellId incr '(conxId ...)
  net-class-Id
```

where the arguments are as follows:

<i>cellId</i>	ID of the cell for which you want to assign crosstalk constraints. Valid values: ID of any cell in the open library. If the cell is open, you can substitute the following for <i>cellId</i> : (<i>geGetEditCell</i>).
---------------	--

<i>incr</i>	#f: overwrite all the existing constraints #t: add to the existing constraints
-------------	---

<i>conxId</i>	ID of the crosstalk constraints. Valid values: Integer that is a valid crosstalk constraint ID.
<i>netClassId</i>	Crosstalk class ID of nets to which you want to assign the rule. Valid values: Integer that is a valid crosstalk class ID.

Examples

```
(dbAssignNetClassXtalkConstraints cellId #f '(1 3) 1)
(dbAssignNetClassXtalkConstraints cellId #f '(2 3) 2)
(dbAssignNetClassXtalkConstraints cellId #f '(1 2) 3)
```

See Also

[dbClearCellXtalkConstraints](#)
[dbDefineXtalkConstraint](#)
[dbDefineAbsXtalkConstraint](#)
[dbClearCellXtalkLengthWeight](#)
[dbSetXtalkLengthWeight](#)
[dbAssignNetXtalkClass](#)
[dbAssignNetXtalkConstraints](#)
[dbDumpXtalkConstraints](#)
[axgSetXtalkRouteOptions](#)

dbAssignNetMinMaxLayer

Sets a range of layers on which you can route specified nets in the cell.

Note:

To set the minimum to maximum layer for all nets in the design, use [dbSetCellMinMaxLayer](#).

Tool

Milkyway

Prerequisites

Open the cell containing the nets to assign to layers.

Syntax

```
dbAssignNetMinMaxLayer cellId "min_layer_name"  
"max_layer_name"  
"netNames"
```

where the arguments are as follows:

<i>cellId</i>	ID of the design cell containing the nets. If the cell is open, you can substitute the following for <i>cellId</i> : (<i>geGetEditCell</i>).
<i>min_layer_name</i>	Name of the minimum layer. To use the lowest routing layer, type "". Valid values: Routing layer.
<i>max_layer_name</i>	Name of the maximum layer. To use the highest routing layer, type "". Valid values: Routing layer.
<i>netNames</i>	Name of the nets to which you want to assign a timing-driven spacing constraint. Valid values: Name of any nets in the specified library.

Example

```
dbAssignNetMinMaxLayer cellId "M2" "M4" "AAF.*, AAD.*"
```

This example specifies net names beginning with AAF and AAD to be routed on layers M2 and M4.

dbAssignNetRouteRuleByHalfPerimeter

Assigns variable-width route rules or min/max layer constraints to nets based on the length based tables. This command is mostly used after placement and before detail routing. So, the net lengths are estimated by half perimeters of the net bounding boxes.

When *overwrite* is on, the route rules and constraints will be set to a net, whether or not there are previously assigned rules or constraints. Otherwise, the previous rules or constraints could not be overwritten.

When *pattern_match* is on, the nets are searched by pattern matching.

Tool

Milkyway

Syntax

```
dbAssignNetRouteRuleByHalfPerimeter cellId overwrite  
pattern_match '("netName" ...)
```

where the arguments are as follows:

<i>cellId</i>	ID of the design cell containing the nets. If the cell is open, you can substitute the following for <i>cellId</i> : (<i>geGetEditCell</i>).
<i>overwrite</i>	True or False.
<i>pattern_match</i>	True or False.
<i>netName</i>	A list of net names to be assigned rules or constraints.

Example

```
dbAssignNetLengthRouteRuleByTable (geGetEditCell) #f #f  
  ' ("net1" "net2")  
dbAssignNetLengthRouteRuleByTable (geGetEditCell) #f #t  
  ' ("net*")
```

dbAssignNetTimingSpacing

Specifies nets as having timing-driven spacing constraints. If crosstalk problems occur around this net or the intralayer coupling capacitance of the net is too high, Astro adds space between this net and other nets and avoids placing long, parallel wires next to long wires on this net.

Tool

Milkyway

Prerequisite

You must open the cell containing the nets to which you want to assign timing-driven spacing constraints. However, before you specify the timing-driven spacing constraints, you must first set the routing options. See `axgSetHPORouteOptions`.

Syntax

```
dbAssignNetTimingSpacing cellId ' (netName netName)
```

where the arguments are as follows:

<i>cellId</i>	ID of the design cell containing the nets. If the cell is open, you can substitute the following for <i>cellId</i> : (<i>geGetEditCell</i>).
---------------	---

netName Name of the net to which you want to assign a timing-driven spacing constraint.
Valid values: Name of any net in the specified library.

Example

```
dbAssignNetTimingSpacing geGetEditCell '("N4001" "N4652")
```

This example assigns timing-driven spacing constraints to nets N4001 and N4652.

dbAssignNetTopLayerProbe

Assign constraints to Astro's router to route only those specified nets to the top-layer metal for probing purposes. For the router to honor the constraint, `axgSetHPORouteOptions` must be used to turn on this optional feature.

Tool

Milkyway

Syntax

```
(dbAssignNetTopLayerProbe cellId type '("netnames")
```

where the arguments are as follows:

cellId ID of the cell for which you want to assign nets.
Valid values: ID of any cell in the open library. If the cell is open, you can substitute the following for *cellId*:
(*geGetEditCell*).

type “anyPort”: the net needs to reach the top layer only once; the location does not matter. This is for signal probing only.

 “allPort”: the routing from every pin has to reach the top layer prior to hitting the steiner point or another pin. This is for Post-Mask Surgery, which may want to disconnect any pin from the net.

 “outPort”: the routing from the output pin has to reach the top layer prior to hitting the steiner point or another pin. This is for Post-Mask Surgery, which may want to disconnect the driving pin of the net.

Note:

 For 2-pin nets, all three options are essentially the same. You will only notice a difference for nets with three (3) or more pins.

netnames Pattern of the name of a net to which you want to assign the constraints.

 Valid values: Name of any net in the cell.

Examples

```
dbAssignNetTopLayerProbe (geGetEditCell) "anyPort "
' ("N1 " "N2 " "N3 ")
```

This example assigns signal probing on the top layer to nets N1, N2, and N3.

See Also

axgSetHPORouteOptions
[dbDumpAllNetTopLayerProbe](#)
[dbResetAllNetTopLayerProbe](#)

dbAssignNetVarRouteRule

Assigns a set of variable-width routing rules to one or more nets.

Tool

Milkyway

Prerequisites

Define variable-width routing rules using [dbDefineVarRouteRule](#).

Syntax

```
dbAssignNetVarRouteRule cellId "ruleSetName" pattern_match  
' ({"netName"} ...)
```

where the arguments are as follows:

<i>cellId</i>	ID of the cell for which you want to assign variable-width routing rules. Valid values: ID of any cell in the open library. If the cell is open, you can substitute the following for <i>cellId</i> : <i>(geGetEditCell)</i> .
<i>ruleSetName</i>	Name of the rule set you want to assign. Valid values: Name of any set of variable-width routing rules for the cell. Rule set names are defined by the <i>ruleSetName</i> argument in dbDefineVarRouteRule .
<i>pattern_match</i>	Specify whether the net names are regular expression patterns or exact names. Valid values: #t or #f.
<i>netName</i>	Pattern of the name of a net to which you want to assign the rule. Valid values: Name of any net in the cell.

Note:

To convert a literal string to a regular expression, refer to the Scheme command `strToRegExp`.

Examples

```
dbAssignNetVarRouteRule (geGetEditCell) "VWR1" #f  
' ("n1" "n2" "n3")
```


This example assigns rule set VWR1 to nets n1, n2, and n3.

```
dbAssignNetVarRouteRule (geGetEditCell) "VWR1" #f '("a[1]"  
"a[2]" "a[3]")
```

This example assigns rule set VWR1 to nets “a[1]”, “a[2]”, and “a[3]”.

```
dbAssignNetVarRouteRule (geGetEditCell) "VWR2" #t  
'("clock.*")
```

This example assigns rule set VWR2 to nets with names matching the pattern “clock.*”.

dbAssignNetXtalkClass

Nets can be assigned up to 64 different crosstalk classes. Nets can only belong to one class. All nets also belong to class '0.'

Tool

Milkyway

Syntax

```
(dbAssignNetXtalkClass cellId classId pattern  
{ {"netName"}...})
```

where the arguments are as follows:

cellId ID of the cell for which you want to assign crosstalk classes.
Valid values: ID of any cell in the open library. If the cell is open, you can substitute the following for *cellId*: (*geGetEditCell*).

classId ID of the class you want to assign.
Valid values: 1 - 63.

pattern #f: exact net name
 #t: net name is regular expression

netName Pattern of the name of a net to which you want to assign the
 crosstalk class.
 Valid values: Name of any net in the cell.

Examples

```
(dbAssignNetXtalkClass cellId 0 #t '("."*))  
(dbAssignNetXtalkClass cellId 1 #f '("n100" "n200"))  
(dbAssignNetXtalkClass cellId 2 #t '("n3.*"))  
(dbAssignNetXtalkClass cellId 3 #f '("n40"))
```

See Also

[dbClearCellXtalkConstraints](#)
[dbDefineXtalkConstraint](#)
[dbDefineAbsXtalkConstraint](#)
[dbClearCellXtalkLengthWeight](#)
[dbSetXtalkLengthWeight](#)
[dbAssignNetXtalkConstraints](#)
[dbAllowToAddPGIOText](#)
[dbDumpXtalkConstraints](#)
[axgSetXtalkRouteOptions](#)

dbAssignNetXtalkConstraints

You can assign crosstalk constraints to nets. Each net can have more than one constraint.

Tool

Milkyway

Syntax

```
(dbAssignNetXtalkConstraints cellId incr '(conxId...)
  pattern'("netNames")
```

where the arguments are as follows:

<i>cellId</i>	ID of the cell for which you want to assign crosstalk constraints. Valid values: ID of any cell in the open library. If the cell is open, you can substitute the following for <i>cellId</i> : (<i>geGetEditCell</i>).
<i>incr</i>	#f: overwrite all the existing constraints of the net #t: add to the existing constraints of the nets
<i>conxId</i>	ID of the crosstalk constraints. Valid values: Integer that is a valid crosstalk constraint ID.
<i>pattern</i>	#f: netNames is the exact net name #t: netNames is a regular expression
<i>netNames</i>	Pattern of the name of a net to which you want to assign the rule. Valid values: Name of any net in the cell.

Examples

```
(dbAssignNetXtalkConstraints cellId #f '() #t '("."*))
(dbAssignNetXtalkConstraints cellId #f '(1) #f '("abus[1]"
  "abus[2]"))
(dbAssignNetXtalkConstraints cellId #f '(1 3) #t
  '("bbus\[.*\]"))
```

See Also

[dbClearCellXtalkConstraints](#)
[dbDefineXtalkConstraint](#)
[dbDefineAbsXtalkConstraint](#)
[dbClearCellXtalkLengthWeight](#)
[dbSetXtalkLengthWeight](#)

[dbAssignNetXtalkClass](#)
[dbAllowToAddPGIOText](#)
[dbDumpXtalkConstraints](#)
[axgSetXtalkRouteOptions](#)

dbAssignVarRouteRule

Assigns a set of variable-width routing rules to one or more nets.

Tool

Milkyway

Prerequisites

Define variable-width routing rules using [dbDefineVarRouteRule](#).

Syntax

```
dbAssignVarRouteRule cellId "ruleSetName" '({"netName"}  
...)
```

where the arguments are as follows:

<i>cellId</i>	ID of the cell for which you want to assign variable-width routing rules. Valid values: ID of any cell in the open library. If the cell is open, you can substitute the following for <i>cellId</i> : (<i>geGetEditCell</i>).
<i>ruleSetName</i>	Name of the rule set you want to assign. Valid values: Name of any set of variable-width routing rules for the cell. Rule set names are defined by the <i>ruleSetName</i> argument in dbDefineVarRouteRule .
<i>netName</i>	Pattern of the name of a net to which you want to assign the rule. Valid values: Name of any net in the cell.

Note:

To convert a literal string to a regular expression, refer to the Scheme command `strToRegExp`.

Examples

```
dbAssignVarRouteRule (geGetEditCell) "VWR1" '("n1" "n2"
"n3")
```

This example assigns rule set VWR1 to nets n1, n2, and n3.

```
dbAssignVarRouteRule (geGetEditCell) "VWR1" '("a\\[1\\]"
"a\\[2\\]" "a\\[3\\]")
```

This example assigns rule set VWR1 to match net name patterns to nets “a[1]”, “a[2]”, and “a[3]”.

dbBrowser

Invokes the database browser to search and view information in the database.

Syntax

```
dbBrowser<void>
```

Description

The `dbBrowser` command invokes the database browser to search and view information in the database.

The browser provides a graphical user interface to:

- Select and view a library and its cells.
- View the number of objects (for each object type) in the library or cell.

- View a list of all objects in the library or cell.
- Select and view the contents of an object.
- Retrieve an object by its ID number and view its contents.
- Search for an object according to user-specified parameters and view its contents.

Example

`dbBrowser`

dbCellChecksum

Calculates and generates a cell checksum report.

Syntax

```
dbCellChecksum < objectId cellId > [ string fileName ]
```

`cellId` - Database object ID of an opened cell for which to report checksum information

`fileName` - Name of the output report file (optional argument). If not specified, the report is written to the Milkyway scheme window.

Prerequisites

The library containing the specified cell must be opened prior to calling the `dbCellChecksum` function.

Description

The `dbCellChecksum` function calculates and generates a cell checksum report. The report is a text file listing checksum information about the cell's cell instances and variable route rules.

The function compiles the following information:

- Cell instance information sorted by cell type
 - Total count
 - Total length of all cell instance names
 - Total value of all cell instance names
 - Total sum of all cell instance x coordinates
 - Total sum of all cell instance y coordinates
- Cell instance information sorted by cell master
 - Total count
 - Total length of all cell instance names
 - Total value of all cell instance names
 - Total sum of all cell instance x coordinates
 - Total sum of all cell instance y coordinates
- Variable route rule information
 - Total number of rule sets
 - Total value of all rule set names
 - Total number of width rules
 - Total value of all width rule layer names
 - Total sum of all width rule widths
 - Total number of space rules
 - Total value of all space rule layer names

- Total sum of all space rule spacings
- Total number of contact rules
- Total value of all contact rule contact code names
- Total sum of all contact rule array widths
- Total sum of all contact rule array heights
- Total number of taper rules
- Total sum of all taper rule levels

For name strings, the value is calculated by assigning each alphanumeric character a numeric value and tallying up the total value.

Example

```
dbCellChecksum (geGetEditCell) "checksum.txt"
```

The above function call would generate the following in a text file named "checksum.txt":

CELL CHECK SUM REPORT
CELL NAME: top

CELL INSTANCES

SORTED BY CELL TYPE

CELLTYPE POS	COUNT TRANSFORM	NAMELEN	NAMEVAL	X-POS	Y-	
MACRO	1	10	100	1000	1000	1
STANDARD	2	20	200	2000	2000	2
OTHER	3	30	300	3000	3000	3

TOTAL	6	60	600	6000	6000	6

SORTED BY CELL INSTANCE MASTER

MASTER POS	COUNT TRANSFORM	NAMELEN	NAMEVAL	X-POS	Y-	
CM1	1	10	100	1000	1000	1
CM2	1	10	100	1000	1000	1
CM3	1	10	100	1000	1000	1
CM4	1	10	100	1000	1000	1
CM5	2	20	200	2000	2000	2

TOTAL	6	60	600	6000	6000	6


```

*****
*****
VARIABLE ROUTE RULES
*****
*****
Number of Rule Sets                : 10
Rule Set Names CheckSum           : 600
Number of Width Rules             : 20
Width - Layer Names CheckSum      : 300
Total Width                       : 800
Number of Space Rules             : 30
Space - Layer Names Value         : 600
Total Space                       : 400
Number of Contact Rules          : 20
Contact Names CheckSum           : 3000
Total Contact X                   : 60
Total Contact Y                   : 90
Number of Taper Rules            : 20
Total Taper Level                 : 50

```

dbCellCompare

Compares and reports the differences between two cells.

Syntax

```

dbCellCompare < string libName1 > < string cellName1 >
< string libName2 > < string cellName2 >
[ string outputFile ] [ int detailLevel ]

```

libName1 - Name of the library containing the first cell.

cellName1- Name of the cell to compare. The view and version may be specified using the standard database syntax (i.e. cellName.viewName;version).

libName2 - Name of the library containing the second cell.

cellName2 - Name of the cell to compare. The view and version may be specified using the standard database syntax (i.e. cellName.viewName;version).

fileName - Name of the output report file (optional). If not specified, the report is displayed to the Milkyway scheme window.

detailLevel - Indicates the amount of detail to compare (optional). The default setting compares the cells with the greatest detail.

Note that the arguments, cellName1 and cellName2, must correspond to two unique cells.

Description

The dbCellCompare function compares and reports the differences between two cells. This function compares object checksum statistics, not individual objects.

For example, to compare the cell instance positions of two cell files, it adds up all the cell instance positions of the first cell, adds up all the cell instance positions of the second cell, and compares the two totals. If the totals are equal, then the cell instance positions are equivalent. Otherwise the cell instance positions are different.

This comparison method, tallying of checksum numbers, provides a quick and efficient means of finding differences. However, it cannot provide absolute certainty that two cells are equivalent.

It also cannot provide detailed descriptions about the differences such as which net was missing a port or which cell instance was moved to another location.

The granularity of the checksum comparison is determined by the detail level you specify. The detail level may be 1 or 2 (default is 2). If you specify detail level 1, the function gathers and compares the total counts of all database objects.

Specifying detail level 2, the function gathers and compares the fields of the database objects in addition to the total counts. Also you specify detail level 2, the function provides more information as well as a more thorough comparison, at a cost of longer runtimes.

If no differences were found, then the message "No differences were found" will appear in the report. Otherwise the function outputs messages indicating the differences in the report.

Note that "report" refers to all messages generated by the function. The user can either dump the report to a file or to the Milkyway scheme window.

If a file name is specified, the report is written to the file. Otherwise the report is displayed in the Milkyway scheme window (and the Milkyway log file).

Example

```
dbCellCompare "lib1" "cell1" "lib2" "cell2" "compare.txt" 1
```

The above function call may generate the following in a text file named "compare.txt" (Note that the detail level is set to 1):

```

*****
CELL COMPARISON REPORT:
Cell 1: cell11.CEL;1
(/home/lib1)
Cell 2: cell12.CEL;1
(/home/lib2)
*****

No differences were found.

```

The report file indicates that the two cells are identical. However, if the detail level is set to 2, then the following may be generated:

```

*****
CELL COMPARISON REPORT:
Cell 1: cell11.CEL;1
(/home/lib1)
Cell 2: cell12.CEL;1
(/home/lib2)
*****

dbtPolygon (Type Id = 79)
-----
The polygon sizes on layer 17 are different.

dbtPath (Type Id = 84)
-----
The path sizes on layer 17 are different.

```

This will occur if the total number of polygon and path objects are the same in both cells, but vary in the number of objects on specific layers.

dbCellInstPlaceAndStatus

Sets the placement and status of a cell instance within the specified design cell.

Syntax

```
dbCellInstPlaceAndStatus <objectId cellId> <string  
cellInstName> <string rotationStr> <string mirrorStr>  
<string justStr> <list point> <string status>
```

cellId - Database object ID of the design cell in which to set the cell instance placement and status

cellInstName - Name of the cell instance

rotationStr - Counter-clockwise rotation in degrees

Valid Values: "0", "90", "180", "270"

mirrorStr - Direction, if any, to flip the cell instance

Valid Values: "", "X", "Y"

justStr - justification of the cell instance around the origin

Valid Values:

"ll" - Place the cell instance to the lower left of the origin

"lc" - Place the cell instance to the lower center of the origin

"lr" - Place the cell instance to the lower right of the origin

"cl" - Place the cell instance to the center left of the origin

"cc" - Centers the cell instance around the origin

"cr" - Place the cell instance to the center right of the origin

"ul" - Place the cell instance to the upper left of the origin

"uc" - Place the cell instance to the upper center of the origin

"ur" - Place the cell instance to the upper right of the origin

point - Coordinate pair representing the location of the cell instance origin

status - Placement status

Valid Values: "Unplaced", "Placed", "Fixed", "SoftFixed"

Description

The dbCellInstPlaceAndStatus function sets the placement and status of a cell instance within the specified design cell.

This function combines the functionality of dbSetCellInstPlacement and [dbSetCellInstStatus](#).

Example

```
dbCellInstPlaceAndStatus (geGetEditCell) "AND3" "90" "X"
"11" `(450 660) "Placed"
```

Note the apostrophe (') before point (450 660) above. In the Scheme language, an apostrophe precedes all lists.

dbCellReportHighFanoutNet

This command is used to output a report to a file. In that file report, each sum number of Port and PortInstance of Net in the Cell, except PowerNet and GroundNet are included. Only the summation of Port and PortInstance numbers of each Net that are larger than the value of the “threshold” will be output. Before you run this command, you must first open a library and a cell. You should also know the CellId. Use “geGetEditCell” to retrieve it.

Tool

Milkyway

Syntax

```
dbCellReportHighFanoutNet cellId threshold "filename"
```

where all three parameters are string objects and their description follows:

cellId Database object ID of the cell for which you want to report the high fanout net.
Valid values: Cell ID of any open cell in the current library (*geGetEditCell*).

threshold An integral number

filename Name of the file saved. Example: "net_remap".

Example

```
dbCellReportHighFanoutNet 3 2 "out1.txt"
```

in output file "out1.txt":

```
26 i1/DO20
2 i1/x1/o041/n206
2 i1/x1/o041/N70
14 i1/DO23
14 i1/DO24
13 i1/DO22
15 i1/DO21
3 i1/x1/o083/N57
3 i1/x1/o083/BS1_SO
```

In the first line, the number "26" is the sum number of the Port and PortInstance of the Net "i1/DO20." "i1/DO20" is a Net name.

dbChangeAccessMode

Changes the current cell access mode from read-only (R) to write (W). If `dbChangeAccessMode` succeeds, the Synopsys tool returns `#t`.

Syntax

```
dbChangeAccessMode cellId "accessMode"
```

where the arguments are as follows:

cellId

Database object ID of the cell to which information applies

Valid values:

- Cell ID of any open cell in the current library
- `(geGetEditCell)`

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

accessMode

Indicates whether you can edit the cell

Valid value:

`w`, which means that edits are allowed

dbChangeAllInstNames

Changes the cell instance names within a design cell.

Syntax

```
dbChangeAllInstNames <string libName> <string cellName>  
<string mapFileName>
```

libName - Name of library containing the cell

cellName - Name of design cell containing the cell instances whose names are to be changed

mapFileName - Name of the map file with the cell instance name remapping information

Description

The dbChangeAllInstNames function changes the cell instance names within a design cell. It utilizes the information in the map file to rename the cell instances.

The map file lists the current cell instance name and its new name. The syntax of the map file is:

```
newInstName_1 currentInstName_1  
newInstName_2 currentInstName_2  
.  
.  
.  
newInstName_N currentInstName_N
```

Example

```
dbChangeAllInstNames "mylib: "top: "map.file:
```

```
map.file:  
inst1_AND inssst1_AND  
inst2_OR inssst2_OR
```

The above function call with the indicated "map.file" will rename instance "inssst1_AND" to "inst1_AND" and "inssst2_OR" to "inst2_OR" inside of the design cell "top."

dbChangeAllNetNames

This command is used to change the net names in a Milkyway cell. The change is made using a mapping file.

The difference between this command and [dbChangeAllNetNamesByNetId](#) is how you specify the old net. This command uses a name to specify the old net; whereas, [dbChangeAllNetNamesByNetId](#) uses a net id to specify the old net.

The design must be closed before using this command, or else the operation fails and returns information showing successful completion.

Tool

Milkyway

Syntax

```
dbChangeAllNetNames "libName" "cellName" "filename"
```

where all three parameters are string objects and their description follows:

<i>libName</i>	Name of the library containing the cell with nets to be changed. Example: "designs".
----------------	---

cellName Name of the cell containing nets to be renamed.
Example: "top.CEL".

filename Name of the file with the net remapping information.

Example

Syntax of the data in *fileName*:

```
newNetName_1 oldNetName_1
newNetName_2 oldNetName_2
.
.
.
newNetName_N oldNetName_N
```

All *oldNetNames* will be changed to the *newNetNames*.

Note:

The file containing the net remapping information must be placed in the library directory for the cell view being changed. With the example above the file, mapping file is placed in the .CEL directory of the designs library to be used by the [dbChangeAllNetNames](#) command.

dbChangeAllNetNamesByNetId

This command is used to change the net names of a net id in a Milkyway cell. The change is made using a mapping file.

The difference between this command and [dbChangeAllNetNames](#) is how you specify the old net. The dbChangeAllNetNames command uses a name to specify the old net; whereas, dbChangeAllNetNamesByNetId uses a net id to specify the old net.

The net id must be specified in decimal, not hexadecimal. The design must be closed before using this command, or else the operation fails, but returns information showing successful completion.

Tool

Milkyway

Syntax

```
dbChangeAllNetNamesByNetId "libName" "cellName"
    "netIdToNewNetNamesFile"
```

where all three parameters are string objects and their description follows:

<i>libName</i>	Name of the library containing the cell with nets to be changed. Example: "designs".
<i>cellName</i>	Name of the cell containing nets to be renamed. Example: "top.CEL".
<i>netIdToNewNetNamesFile</i>	The file name containing a list. The first column is net name and the second column is net id.

Example

Syntax of the data in fileName:

```
new_n31311 5049376  
.  
.  
.
```

The *net Ids* will be changed to the *newNetNames* specified in the file. Use decimal not hexadecimal to specify the net Ids.

Note:

The file containing the net remapping information must be placed in the library directory for the cell view being changed. With the example above the file, mapping file is placed in the .CEL view directory of the designs library to be used by the [dbChangeAllNetNamesByNetId](#) command.

dbChangeInstanceName

This command is used to change the cell instance name of a given cell.

Tool

Milkyway

Prerequisites

The library for which you want to change the cell instance name must be open in write (W) mode.

Syntax

```
dbChangeInstanceName cellId "oldCellInstName"  
"newCellInstName"
```

where the arguments are as follows:

<i>cellId</i>	Database object ID of the cell for which you want to change the cell instance name. Valid values: Cell ID of any open cell in the current library (<i>geGetEditCell</i>).
<i>oldCellInstName</i>	Original name of the cell instance that you intend to change. Valid values: Name of any cell instance in the cell indicated by <i>cellId</i> .
<i>newCellInstName</i>	New cell instance name you intend to change to. Valid values: Name of any cell instance in the top-level cell. Valid values: Any string with 31 characters or less.

There are several cases where this command may fail:

- No library is opened in write mode.
- The *cellId* is invalid or is not opened in write mode.
- The *oldCellInstName* is identical to the *newCellInstName*.
- The *oldCellInstName* does not exist.
- The *newCellInstName* already exists.

dbChangeLibDbVersion19To20

Changes the db version of a library from 19 to 20. If the db version of the library is already 20, then no change will be made. If the db version of the library is neither 19 nor 20, error messages will display

and #f will be returned. A change in the db version of a library has an effect on the size of attachment objects created by the Milkyway database system. A new attachment object, created by the db system, is a larger size attachment object if the db version of the library, that the object belongs to, is 19. The new object is a smaller size attachment object if the db version of the library is 20.

Tool

Milkyway

Prerequisites

The library whose db version is to be changed must be closed.

Syntax

```
dbChangeLibDbVersion19To20 "libName"
```

where the argument is as follows:

libName

The name of the library whose db version is to be changed.

dbChangeRefLibAccessMode

Changes the access mode of a reference library.

Tool

Milkyway

Syntax

```
dbChangeRefLibAccessMode refLibId accessMode
```


where the arguments are as follows:

<code>refLibId</code>	integer; specifies the ID of the reference library to change the access mode.
<code>accessMode</code>	string; specifies the access mode to be changed. "w"/"W" stands for write, and "r"/"R" stands for read.

Description

Reference libraries are usually open for read-only. The `dbChangeRefLibAccessMode` command lets you write to a reference library by changing its access mode.

Example

```
dbChangeRefLibAccessMode 2 "w"
```

dbChangeTextLayer

Changes the layer number of a specified text object to a user specified layer number.

Tool

Milkyway

Syntax

```
dbChangeTextLayer libId "textName" layerNum
```

where the arguments are as follows:

<i>libId</i>	The current library Id. Use <code>dbGetCurrentLibId</code> can get it.
--------------	--

<i>textName</i>	The name of text object for which you want to change its layer number.
<i>layerNum</i>	The layer number for which you want the text object changed to.

dbCheckCellData

Checks data integrity of a specified cell. If there is any “ERROR” reported by this command, the DB cell may have been damaged.

Tool

Milkyway

Syntax

```
dbCheckCellData cellId
```

where the argument is as follows:

<i>cellId</i>	ID of the cell where you want to check data integrity. Valid values: ID of any cell in the open library. If the cell is open, you can substitute the following for <i>cellId</i> : (<i>geGetEditCell</i>).
---------------	---

Example

```
dbCheckCellData (geGetEditCell)
```

dbCheckCellTypeAndPortMatching

Checks the consistency of all cell’s types and ports between two views.

Tool

Milkyway

Syntax

```
dbCheckCellTypeAndPortMatching libName viewName1 viewName2  
ignorePGPort? [dumpFileName]
```

where the arguments are as follows:

<i>libName</i>	The name of a library.
<i>viewName1</i>	The name of a view.
<i>viewName2</i>	The name of a view.
<i>ignorePGPort?</i>	Checks power and ground ports or not.
<i>dumpFileName</i>	Will dump the report to a disk file.

Example

```
dbCheckCellTypeAndPortMatching "ARCHIVE" "CEL" "FRAM" #t
```

e.g. lib DESIGN has a.CEL b.CEL and a.FRAME

```
dbCheckCellTypeAndPortMatching "DESIGN" "CEL" "FRAM"
```

will check the consistency of cell type between a.CEL and a.FRAME
and report the extra ports in a.CEL/a.FRAME but not in a.FRAME/
a.CEL, then report that there is no .FRAME view of b.CEL

```
dbCheckCellTypeAndPortMatching "DESIGN" "CEL" "FRAM" #t
```

will not check power & ground port.

```
dbCheckCellTypeAndPortMatching "DESIGN" "CEL" "FRAM" #t  
"report.txt"
```

will direct the output to file report.txt not command window.

dbCheckIdenticalCell

Checks the specified library hierarchy for duplicated cell names.

Syntax

```
dbCheckIdenticalCell <string libName>
```

libName - Name of the top library where checking begins

Description

The dbCheckIdenticalCell function checks the specified library hierarchy for duplicate cell names. It compares all cell names stored in the top library and in all libraries of its library reference tree.

This function compares only the cell names cataloged in the library. It does not compare the cell view names or version numbers. It does not check if the cells exist on the disk. It also does not open any cells to verify if the cells are the same or different.

This function dumps the information to the Milkyway Database Environment scheme window. This information indicates which libraries contain the same cell names and which cell names are duplicated.

Example

```
dbCheckIdenticalCell "mylib"
```

The above example reports the following type of information

Library mylib and library replib1 share the follow cells
macro1
macro2
macro3

Library mylib and library replib2 share the follow cells
macro4
macro5
macro6

Library replib1 and library replib2 share the follow cells
None cell is shared by those two libraries

dbCheckNetList

Checks the net list of a specified cell to see whether all nets are driven properly. It will:

- report nets without any drivers.
- check multidriven nets.
- check floating primary input pins.
- check floating primary output pins.
- check floating cell instance pins.

Tool

Milkyway

Syntax

```
dbCheckNetList cellId [fileName]
```

where the arguments are as follows:

<i>cellId</i>	ID of the cell where you want to check the net list. Valid values: ID of any cell in the open library. If the cell is open, you can substitute the following for <i>cellId</i> : (<i>geGetEditCell</i>).
<i>fileName</i>	Name of the file you want to dump to. Valid values: Any valid file name. Note: This argument is optional.

Example

```
dbCheckNetList (geGetEditCell)
```

dbCheckNetlistVsFram

Checks the correspondence between netlist (*.netl*) cells and pin/blockage (*.fram*) cells in a specified library.

- For every netlist cell, it checks to make sure that there is a pin/blockage cell.
- If a pin/blockage cell exists, it checks to make sure that for every port in the netlist cell there is a corresponding port of the same name in the pin/blockage cell.

Tool

Milkyway

Syntax

```
dbCheckNetlistVsFram libName  
[netlistViewName excludeLibraryList]
```

where the arguments are as follows:

<i>libName</i>	Name of the library you want to check. Valid values: Name of any library in the current directory.
<i>netlistViewName</i>	Name of any other cell view name you want to check. By default, this command checks netlist cells with the view name “.NETL”. Valid values: Name of any cell view in the library. Note: This argument is optional.
<i>excludeLibraryList</i>	List of the name of all reference libraries you don't want to check. By default, all reference libraries of <i>libName</i> will be checked. Note: This argument is optional.

Example

This example checks netlist cells in the library *DESIGNS* and it's reference libraries, with the cell view name “.Symb”.

```
dbCheckNetlistVsFram "DESIGNS" ".Symb"
```

See Also

[dbDumpCheckNetlistVsFram](#)

dbCheckRefLibControl

Displays the comparison of the reference library setting in the control file and the prior setting in the library.

Tool

Milkyway

Syntax

```
dbCheckRefLibControl "A"
```

See Also

[dbDumpRefLibControl](#)

[dbClearRefLibControl](#)

[dbSetRefLibControl](#)

dbClearAllNetXtalkProp

Clears all net crosstalk properties in the specified cell.

Syntax

```
dbClearAllNetXtalkProp <objectId cellId>
```

where the argument is as follows:

cellId

Database object ID of the cell.

Description

The `dbClearAllNetXtalkProp` command clears all net crosstalk properties in the specified cell.

Example

```
dbClearAllNetXtalkProp (geGetEditCell)
```

dbClearAttachment

This function cleans up all attachment objects for a specified cellId.

Syntax

```
dbClearAttachment cellId
```

Arguments

cellId - The cell Id to performs attachment clean up. Integer value

Description

The the `dbClearAttachment` function cleans up all attachment objects for a specified cellId. This function forces a clean-up regardless of whether the object of an attachment is deleted. For example, if A1 is an attachment of object2, this function deletes A1 whether or not object2 is deleted.

Example

```
dbClearAttachment (geGetEditCell)
```

dbClearCellDifferentialGroup

Clears all the differential group definitions of the design.

Syntax

```
dbClearCellDifferentialGroup cellId
```

where the argument is as follows:

cellId

Cell ID to which nets of the groups belong

Example

The following clears all the differential group definitions of the current design:

```
dbClearCellDifferentialGroup (geGetEditCell)
```

dbClearCellEEQClass

Clears the electrical-equivalent class in the specified library.

Note:

Astro does not currently use the electrical-equivalent class.

Syntax

```
dbClearCellEEQClass libId
```

where *libId* is the database object ID of any library in the current directory from which you want to delete electrical-equivalent class.

See Also

[dbCreateCellEEQClass](#)

[dbClearCellEQClass](#)

[dbVerifyCellEEQClass](#)

dbClearCellEQClass

Clears both electrical-equivalent and logical-equivalent classes in the specified library.

Note:

Astro does not currently use the electrical-equivalent class.

Syntax

```
dbClearCellEQClass libId
```

where *libId* is the database object ID of any library in the current directory from which you want to delete both the electrical-equivalent and logical-equivalent classes.

See Also

[dbCreateCellEEQClass](#)

[dbCreateCellLEQClass](#)

[dbClearCellEEQClass](#)

[dbClearCellLEQClass](#)

[dbVerifyCellEEQClass](#)

[dbVerifyCellLEQClass](#)

dbClearCellLEQClass

Clears logical-equivalent class in the specified library.

Syntax

```
dbClearCellLEQClass libId
```

where *libId* is the database object ID of any library currently open from which you want to delete logical-equivalent class.

See Also

[dbCreateCellEQClass](#)

[dbClearCellEQClass](#)

[dbVerifyCellEQClass](#)

dbClearCellFootPrintEQClass

Clears any existing footprint-equivalent cell information from the design library defined by `defineFootPrintEQCell` .

Tool

Milkyway

Syntax

```
dbClearCellFootPrintEQClass libId
```

where the argument is as follows:

libId ID of the cell for which you want to remove EQ class.

dbClearCellVTType

Clears the defined voltage threshold type property of the named cell in the top cell. This command iterates every cell in the specified library; if the cell's CEL or FRAM view is instantiated in the specified top cell, the cell's voltage threshold type information will be written out.

Tool

Milkyway

Syntax

```
dbClearCellVTType topCellId cellName
```

where the arguments are as follows:

topCellId The database object ID of the top cell.

cellName The name of the cell with the property to be cleared.

Example

```
dbClearCellVTType (geGetEditCell) "AND3 "
```

See Also

[dbDumpCellVTType](#)

[dbSetCellVTType](#)

dbClearCellXtalkConstraints

To clear the crosstalk constraint definition and remove all constraints assigned to the nets.

Tool

Milkyway

Syntax

```
(dbClearCellXtalkConstraints cellId)
```

where the argument is as follows:

<i>cellId</i>	ID of the cell for which you want to clear crosstalk constraints. Valid values: ID of any cell in the open library. If the cell is open, you can substitute the following for <i>cellId</i> : (<i>geGetEditCell</i>).
---------------	---

See Also

[dbDefineXtalkConstraint](#)
[dbDefineAbsXtalkConstraint](#)
[dbClearCellXtalkLengthWeight](#)
[dbSetXtalkLengthWeight](#)
[dbAssignNetXtalkClass](#)
[dbAssignNetXtalkConstraints](#)
[dbAllowToAddPGIOText](#)
[dbDumpXtalkConstraints](#)
[axgSetXtalkRouteOptions](#)

dbClearCellXtalkLengthWeight

To clear the *LengthWeight* of all crosstalk constraint layers. Once the *LengthWeight* is cleared, all layers will have the same weight.

Tool

Milkyway

Syntax

```
(dbClearCellXtalkLengthWeight cellId)
```

where the argument is as follows:

<i>cellId</i>	ID of the cell for which you want to clear <i>LengthWeight</i> . Valid values: ID of any cell in the open library. If the cell is open, you can substitute the following for <i>cellId</i> : (<i>geGetEditCell</i>).
---------------	--

See Also

[dbClearCellXtalkConstraints](#)
[dbDefineXtalkConstraint](#)
[dbDefineAbsXtalkConstraint](#)
[dbSetXtalkLengthWeight](#)
[dbAssignNetXtalkClass](#)
[dbAssignNetXtalkConstraints](#)
[dbAllowToAddPGIOText](#)
[dbDumpXtalkConstraints](#)
[axgSetXtalkRouteOptions](#)

dbClearLengthRouteRuleTable

Removes the length based tables for variable-width routing rules and min/max layer constraints.

Tool

Milkyway

Syntax

```
dbClearLengthRouteRuleTable cellId
```

where the argument is as follows:

<i>cellId</i>	ID of an opened cell. (geGetEditCell).
---------------	---

Example

```
dbClearLengthRouteRuleTable (geGetEditCell)
```

dbClearLibAntennaRules

Deletes all of the antenna rules stored in the library.

Tool

Milkyway

Syntax

```
dbClearLibAntennaRules libId
```

where the argument is as follows:

<i>libId</i>	ID of an opened library: (dbGetCurrentLibId).
--------------	--

See Also

[dbAddAntennaLayerRule](#) defines an advanced antenna rule for the specified layer and stores it in the library.

[dbDefineAntennaRule](#) defines an advanced antenna rule for the specified mode and stores it in the library.

[dbDumpLibAntennaRules](#) dumps all of the antenna rules stored in the library to a file.

dbClearLibraryCellsVTType

Clears the defined voltage threshold type property of each top cell that is in the named cell library. This command iterates every cell in the specified library; if the cell's CEL or FRAM view is instantiated in the specified top cell, the cell's voltage threshold type property will be cleared.

Tool

Milkyway

Syntax

```
dbClearLibraryCellsVTType topCellId libraryName
```

where the arguments are as follows:

<i>topCellId</i>	The database object ID of the top cell.
<i>libraryName</i>	The name of the cell library with the properties to be cleared.

Example

```
dbClearLibraryCellsVTType (geGetEditCell) "std_lib"
```

See Also

[dbDumpLibraryCellsVTType](#)

[dbSetLibraryCellsVTType](#)

dbClearLRFillerRule

Clears the left-and-right filler rules previously added to the design database.

Tool

Milkyway

Syntax

```
dbClearLRFillerRule topCellId MasterCellList
```

where the arguments are as follows:

<i>topCellId</i>	The database object ID of the top cell.
<i>MasterCellList</i>	The names of the master cells with the properties to be cleared.

Example

```
dbClearLRFillerRule (geGetEditCell) "AN22LKT1"
```

See Also

[dbDumpLRFillerRule](#)

[dbSpecifyLRFiller](#)

dbClearMustJoinPortByCell

Clears all ports in a cell.

Syntax

```
dbClearMustJoinPortByCell "cellName"
```

where the arguments are as follows:

cellName

Name of the cell for which you want to clear the must-join port

Valid values:

Name of any cell in the library

dbClearNetLengthRouteRuleFromTable

Clears the net route rules or constraints assigned by the length based tables.

When `pattern_match` is on, the nets are searched by pattern matching.

Tool

Milkyway

Syntax

```
dbClearNetLengthRouteRuleFromTable cellId pattern_match  
' ("netName" ...)
```

where the arguments are as follows:

cellId ID of an opened cell.
(`geGetEditCell()`).

pattern_match True or False.

netName A list of net names to be cleared.

Example

```
dbClearNetLengthRouteRuleFromTable (geGetEditCell) #f  
' ("net1" "net2")  
dbClearNetLengthRouteRuleFromTable (geGetEditCell) #t  
' ("net*")
```

dbClearRefLibControl

Unlinks the reference library control file of a library. Takes the library out of the “control file” mode. It will then open the reference libraries from the setting in the library.

Tool

Milkyway

Syntax

```
dbClearRefLibControl libName
```

where the argument is as follows:

<i>libName</i>	Library name of which the reference library control file to be removed.
	Valid values: Name of any library in the current directory.

Example:

```
dbClearRefLibControl "A"
```

See Also

[dbDumpRefLibControl](#)
[dbCheckRefLibControl](#)
[dbSetRefLibControl](#)

dbClearVTFiller

Clears the filler cell list stored for the two specified voltage threshold types in the specified cell.

Tool

Milkyway

Syntax

```
dbClearVTFiller cellId vtTypeA vtTypeB
```

where the arguments are as follows

<i>cellId</i>	The database object ID of the cell.
<i>vtTypeA</i>	The first of two voltage threshold types from which the list of filler cells is to be cleared.
<i>vtTypeB</i>	The second of two voltage threshold types from which the list of filler cells is to be cleared.

Example

```
dbClearVTFiller (geGetEditCell) "vtType1" "vtType2"
```

See Also

[dbDumpAllVTFillers](#)

[dbDumpVTFiller](#)

[dbSpecifyVTFiller](#)

dbCloseCell

Closes the cell and saves your changes. If `dbCloseCell` succeeds, the Synopsys tool returns *#t*.

Syntax

```
dbCloseCell cellId
```

where the argument is as follows:

cellId

Database object ID of any open cell in the current library that you want to close and save

Valid values:

- Cell ID
- (`geGetEditCell`)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

See Also

[dbAbortCell](#)
[dbRollBackCell](#)
[dbSaveCell](#)

dbCloseLib

Closes and saves changes to a library. If `dbCloseLib` succeeds in closing and saving your library, the Synopsys tool returns *#t*.

Note:

A side effect of this command is that all cells within the library are closed.

Syntax

```
dbCloseLib [libId]
```

where *libId* is the database object ID of any library in the current directory that you want to close and save.

Note:

Omission of this argument closes the main library.

Example 1

```
dbOpenLib "train"  
db:1  
.  
.  
.  
dbCloseLib  
#t
```

The code in this example closes the train library and all open cells.

Example 2

```
define lib1 (dbOpenLib "train")  
db:1  
define lib2 (dbOpenLibrary "democ" "r")  
db:2  
.  
.  
.  
dbCloseLib lib2  
#t
```

In this example,

- The first line opens the main library and creates an alias (*lib1*) for the database object ID of the library.
- The second line opens a reference library and creates an alias (*lib2*) for the database object ID of the reference library.
- The third lines closes the reference library.

dbCloseQueryFile

Closes the current query file. If `dbCloseQueryFile` is successful, the Synopsys tool returns *#t*.

Syntax

```
dbCloseQueryFile
```

dbCompNetHalfBBox

This function returns the half perimeter of the pin associated with the given net.

Syntax

```
dbCompNetHalfBBox cellId netId
```

Arguments

cellId - The cell Id for calculating the half perimeter. Integer value

netId - The net Id for calculating the half perimeter. Integer value

Description

This returns the half perimeter of the pin associated with the given net.

Example

```
dbCompNetHalfBBox 3 327937
```

dbConnect

Connects any of the following to a net:

- Port
- Port instance
- Wire
- Contact
- Path

Use [dbConnectPortInstByName](#) to create connections of port instances to nets that are not members of the net. Generally, power and ground connections are not members of the net.

Note:

Because the netlist is changed and no ECO is recorded, be careful when you use this function.

Syntax

```
dbConnect cellId netId memberId
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the net information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

netId

Database object ID of the net to which the information applies

Valid values:

Net ID of any net in the specified cell

memberId

Database object ID of the net member to connect

Valid values:

Object ID of any object connected to the specified net

dbConnectPortInstByName

Connects a port instance of a cell instance to a net. The port instance must not currently be connected to a net.

Syntax

```
dbConnectPortInstByName cellId netId instName portInstName  
topCellName
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the port information applies.

Valid values:

Cell ID of any open cell in the current library (`geGetEditCell`)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

netId

Database object ID of the net to which the port information applies.

Valid values:

Net ID of any net in the cell

instName

Name of the cell instance to which the port information applies.

Valid values:

Name of any cell instance in the cell

portInstName

Name of the port instance to be connected to a net.

Valid values:

Name of any port in the cell

topCellName

Name of the top-level design cell.

See Also

[dbDisconnectPortInstByName](#)

dbConvertPortToDiodePort

Converts a regular port into a special diode port for `axgInsertDiode` and `axgAddDiodeForPreroute`.

Syntax

```
dbConvertPortToDiodePort libId cellName portName
```

where the arguments are as follows:

libId

ID of an opened library:

(`dbGetCurrentLibId`)

cellName

Name of the cell from which the diode port will be converted

Valid values:

Name of any cell in the library

portName

Name of the port to be converted

Valid values:

Any port in the cell specified by *cellName*

See Also

`axgAddDiodeForPreroute`

`axgInsertDiode`

dbConvertRegionToMacroCell

Copies the contents of a cell region to a macro cell. This function does not alter the existing cell region.

Syntax

```
dbConvertRegionToMacroCell "cellName" regionNumber  
"macroCellName"
```

where the arguments are as follows:

cellName

Name of the cell from which the cell region contents are to be converted

Valid values:

Name of any cell in the current library

regionNumber

Number of the cell region to convert

Valid values:

Any positive integer corresponding to a region in the cell

macroCellName

Name of the macro cell to which the cell region contents are to be copied

Valid values:

Any string of 31 characters or fewer

dbCopyCell

Creates a duplicate copy of a cell. If successful, `dbCopyCell` returns the database ID of *toCell*. If not successful, it returns a pair of parentheses (). The Scheme version of this command copies *only* the specified cell. The Scheme application program should copy any additional cell views desired.

Note:

Both *fromCell* and *toCell* are open after execution of `dbCopyCell`.

Caution!

Although you can execute `dbCopyCell` without binding the cell ID of *toCell* to a variable, it is not recommended. Most Scheme commands refer to cells by their cell ID. If the cell ID has not been bound to a variable, some commands cannot modify the cell. For example, *cellID* is one of the arguments of [dbSaveCell](#).

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

Syntax

```
dbCopyCell "fromCell" "toCell"
```

where the arguments are as follows:

fromCell

Name of the cell you want to copy

Valid values:

Name of any cell in the current library

toCell

Name of the duplicate cell you want to create

Valid values:

Any string of 31 characters or fewer

See Also

[dbCopyCellToAnotherLibDisk](#)

[dbCopyCellFromAnotherLib](#)

dbCopyCellFromAnotherLib

Copies a cell from another library to the current library. If `dbCopyCellFromAnotherLib` succeeds, the Synopsys tool returns the database object ID of the copied cell.

The copied cell, *toCell*, is open after execution of `dbCopyCellFromAnotherLib`.

Caution!

Although you can execute `dbCopyCellFromAnotherLib` without binding the cell ID of *toCell* to a variable, it is not recommended. Most Scheme commands refer to cells by their cell ID. If the cell ID has not been bound to a variable, some commands cannot modify the cell. For example, *cellID* is one of the arguments of [dbSaveCell](#).

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

Prerequisite

The library, *fromLib*, that contains the cell to be copied must be closed. A Synopsys tool supports only one library open at a time for editing.

Syntax

```
dbCopyCellFromAnotherLib "fromCell" "fromLib" "toCell"
```

where the arguments are as follows:

fromCell

Name of the cell you want to copy

Valid values:

Name of any cell in the current directory

fromLib

Name of the library containing the cell you want to copy

Valid values:

Name of any library in the current directory containing *fromCell*

toCell

Name of the cell you want to create

Valid values:

Any string of 31 characters or fewer

See Also

[dbCopyCell](#)

[dbCopyCellToAnotherLibDisk](#)

dbCopyCellToAnotherLibDisk

Copies a cell to another library from the current library. If successful, [dbCopyCell](#) returns *#t*.

Prerequisite

The library in which you want to place the copied cell needs to be closed.

Syntax

```
dbCopyCellToAnotherLibDisk "fromCell" "toLib" "toCell"
```

where the arguments are as follows:

fromCell

Name of the cell you want to copy

Valid values:

Name of any cell in the current library

toLib

Name of the library to which you want to copy *fromCell*

Valid values:

Name of any library in the current directory

toCell

Name of the cell you want to create

Note:

If a cell named *toCell* already exists in *toLib*, the Synopsys tool creates a new version of the cell. For example, the tool creates *toCell.extension;versionNumber+1*.

Valid values:

Any string of 31 characters or fewer

See Also

[dbCopyCell](#)

[dbCopyCellFromAnotherLib](#)

[dbDumpCell](#)

dbCopyCellWithViewFromAnotherLib

This function copies a cell to another one with the same view.

Syntax

```
dbCopyCellWithViewFromAnotherLib fromCellName fromViewName  
fromLibName toCellName
```

Arguments

fromCellName - The source cell's name.

fromViewName - The source cell's view name. String value

fromLibName - The source cell's library name. String value

toCellName - The destination cell's name. String value

Description

This copies a cell from the specified library to another cell with the same view in the current library.

Example

```
dbCopyCellWithViewFromAnotherLib "AND2" "FRAM"  
"referenceLib" "AND2_BK"
```

dbCopyLib

Copies a library. If `dbCopyLib` successfully creates a new library, *toLib*, the Synopsys tool returns *#t*.

Prerequisite

The library you want to copy, *fromLib*, must be closed. A Synopsys tool supports only one library open for editing.

Syntax

```
dbCopyLib "fromLib" "toLib"
```

where the arguments are as follows:

fromLib

Name of the library you want to copy

Valid values:

Name of any library in the current directory

toLib

Name you want to assign the new library

Valid values:

Any text string with 255 characters or fewer

Note:

The Synopsys tool does not open the new library, *toLib*, at the end of the copying procedure.

See Also

[dbOpenLib](#)

dbCopyTextAndRectangle

This function copies texts and rectangles of one library to another library.

Syntax

```
dbCopyTextAndRectangle fromLibName toLibName
```

Arguments

fromLibName - The source library name. String value

toLibName - The destination library name. String value

Description

This copies texts and rectangles of one library to another library.

Example

```
dbCopyTextAndRectangle "library1" "library2"
```

dbCreateBaseArray

Creates a base array object.

Syntax

```
dbCreateBaseArray cellId '({llPointArray}...) "className"  
  tileWidth tileHeight
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (*geGetEditCell*)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

llPointArray

Coordinates to indicate the size and shape of the base array

Valid syntax:

((x1 y1) (x2 y2) ...(x1 y1))

className

Name of the class to which the base array applies

Valid values:

Name of any class in the open cell.

tileWidth

Tile dimension of the width

Valid values:

Any positive integer

tileHeight

Tile dimension of the height

Valid values:
Any positive integer

dbCreateBaseArrayRow

Create a row in the base array.

Syntax

```
dbCreateBaseArrayRow cellId baseArrayId PointArray
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (*geGetEditCell*)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

baseArrayId

Database object ID of the base array in the open cell

Valid values:

ID of the base array object in the open cell

PointArray

Coordinates indicating the size and shape of the base array.

Valid syntax:

'((x1 y1) (x2 y2))

dbCreateCell

Opens an empty cell in write (W) mode. If *dbCreateCell* succeeds, the Synopsys tool returns the database object ID of the cell and opens the new cell for writing.

Caution!

Although you can execute *dbCreateCell* without binding the cell ID of toCell to a variable, it is not recommended. Most Scheme commands refer to cells by their cell ID. If the cell ID has not been bound to a variable, some commands cannot modify the cell. For example, cellID is one of the arguments of [dbSaveCell](#).

Prerequisite

A library must be open in write (W) mode.

Syntax

```
dbCreateCell "cellName" "viewName" "viewType"
```

where the arguments are as follows:

<i>cellName</i>	Name of the cell to create. Valid values: Any text string with 127 characters or less. If you include the cell extension, such as <i>.cel</i> , <i>.fram</i> , <i>.exp</i> , <i>.netl</i> , or <i>.tim</i> , the Synopsys tool ignores the argument <i>viewName</i> .
-----------------	--

<i>viewName</i>	Name of the cell view you want to create. Valid values: Any text string with 31 characters or less <i>.cel</i> <i>.fram</i> <i>.exp</i> <i>.netl</i> <i>.tim</i> If you do not specify <i>viewName</i> and do not include a cell extension, such as <i>.cel</i>, <i>.fram</i>, <i>.exp</i>, <i>.netl</i>, or <i>.tim</i>, as part of <i>cellName</i>, the Synopsys tool creates a layout cell. Besides the <i>viewName</i> that the Synopsys tool can recognize (<i>.cel</i>, <i>.fram</i>, <i>.exp</i>, <i>.netl</i>, or <i>.tim</i>), you can define your own <i>viewName</i>. However, the <i>viewName</i> must be specified when the cell is referred.
<i>viewType</i>	Type of cell view you want to create. Valid values: <i>Layout</i> <i>Netlist</i> Schematic Symbolic <i>Unknown</i>

See Also

[dbDeleteCell](#) deletes a cell.

[dbSetCellOrigin](#) sets the origin for the cell.

[dbSetCellType](#) specifies the type of a cell.

[dbSetCellMustJoinAllPin](#) instructs the Synopsys tool to externally connect together all pins on a specified port (or all ports with names matching a specified pattern).

[dbSetCellMustJoinPin](#) instructs the Synopsys tool to externally connect together specified pins on a port.

dbCreateCellBndyForChildCell

This function draws a rectilinear boundary for child cell from the top level.

Syntax

```
dbCreateCellBndyForChildCell cellId childCellName  
pointArray;
```

Arguments

cellId - The top level cell's Id. Integer value

childCellName - The child cell name to draw a rectilinear boundary on. String value

pointArray - SCM list, the coordinates of the rectilinear boundary

Description

This draws a rectilinear boundary for child cell from the top level.

Example

```
dbCreateCellBndyForChildCell (geGetEditCell) "rgb" ' ((0.0  
0.0) (0.2 0.2) (0.3 0.3) (0.4 0.4) (0.5 0.5) (0.6 0.6) (0.0  
0.0))
```

dbCreateCellBoundary

Creates a cell boundary in a cell. If a cell boundary already exists for the cell, the existing boundary will be replaced by the new one.

Syntax

```
dbCreateCellBoundary cellId pointArray
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the cell boundary information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

pointArray

Coordinates indicating the dimensions of the cell boundary

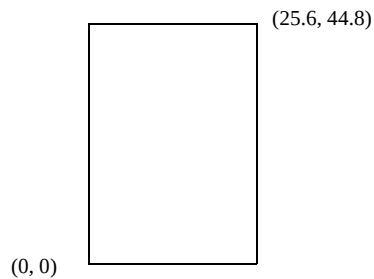
Valid syntax:

```
'((x1 y1) (x2 y2) (x3 y3) (x4 y4) (x1 y1))
```

Example

```
dbCreateCellBoundary cellId  
'((0 0) (0 44.8) (25.6 44.8) (25.6 0) (0 0))  
#t
```

This code creates the following cell boundary:



dbCreateCellBoundaryForCell

Creates a cell boundary in a cell. If a cell boundary already exists for the cell, the existing boundary will be replaced by the new one.

Syntax

```
dbCreateCellBoundary ForCell cellName pointArray
```

where the arguments are as follows:

ForCell

cellName

Name of the master cell of the soft macro where you want to create a boundary.

pointArray

Coordinates indicating the dimensions of the cell boundary

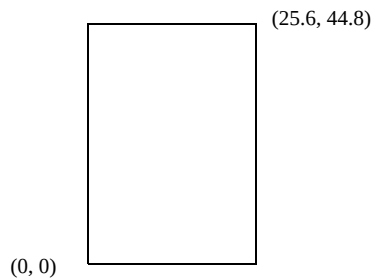
Valid syntax:

```
' ( (x1 y1) (x2 y2) (x3 y3) (x4 y4) (x1 y1) )
```

Example

```
dbCreateCellBoundary ForCell cellName  
'((0 0) (0 44.8) (25.6 44.8) (25.6 0) (0 0))  
#t
```

This code creates the following cell boundary:



dbCreateCellInst

Creates a cell instance in a cell. In the case of a Place and Route type cell, this function creates an instance marked for ignore by the ECO comparator. This allows additional instances, such as power and ground pads to be created that are ignored during ECO's.

Syntax

```
dbCreateCellInst cellId "childLibName" "childCellName"  
  "cellInstName" "rotationStr" "mirrorStr" Points  
  "topCellName"
```

where the arguments are as follows:

<i>cellId</i>	Database object ID of the cell in which you want to create the cell instance. Valid values: Cell ID of any open cell in the current library (geGetEditCell).
<i>childLibName</i>	Name of the library containing the cell from which you want to create the cell instance. Valid values: Any library in the current directory. "" specifies the current library or a reference library associated with the current library.
<i>childCellName</i>	Name of the cell from which you can get some data for creating the instance's master cell. Valid values: Name of any cell in the specified library, but cannot be the same one as the former one. If no view name is specified, the default view will be taken.
<i>cellInstName</i>	Name you want to assign to the new cell instance. Valid values: Any text string with 1024 characters or less.
<i>rotationStr</i>	Degree to which you want the cell instance rotated in reference to the master cell. Valid values: 0 90 180 270 any other values would be treated as zero "0."

<i>mirrorStr</i>	Indicates the orientation of the cell instance. Valid values: No X Y any other values would be treated as “No.”
<i>Points</i>	Reference point for the new instance. Valid syntax: (list (x y))
<i>topCellName</i>	Is always NULL, unless failure occurs when retrieving “_status.” then the property cannot be NULL. Valid Values: Name of the top cell in the open library.

dbCreateCellInstArray

Creates a cell instance array in the specified design cell.

Syntax

```
dbCreateCellInstArray <objectID cellId>
    <string childCellName> <string cellInstName>
    <string rotationStr> <string mirrorStr>
    <point origin> <float xDisplacementUU>
    <float yDisplacementUU>
    <int xTimes> <int yTimes>
```

where the arguments are as follows:

cellId

Database object ID of the design cell in which to create the cell instance array.

childCellName

Name of the child cell (the cell master) of which to instantiate in the array.

cellInstName

Name of the cell instance array.

rotationStr

Degree by which to rotate the cell instance array. Rotation string must be either "0," "90," "180," or "270." All other strings are ignored and the rotation defaults to 0.

mirrorStr

Direction by which to mirror the cell instance array. Mirror string must be either "X" (mirror horizontally) or "Y" (mirror vertically). All other strings are ignored and no mirroring occurs.

origin

Placement location of the cell instance array.

xDisplacementUU

Horizontal pitch between cell instances in the array.

yDisplacementUU

Vertical pitch between cell instances in the array.

xTimes

Number of times to repeat the cell instances horizontally (that is, the number of columns in the array).

yTimes

Number of times to repeat the cell instances vertically (that is, the number of rows in the array).

Description

The `dbCreateCellInstArray` command creates a cell instance array in the specified design cell. The cell instance array is a two-dimensional array with dimensions specified by the `xTimes` and `yTimes` arguments. The `rotationStr` and `mirrorStr` arguments describe the orientation of the entire array.

Example

```
dbCreateCellInstArray (geGetEditCell) "AND2" "instA" "90"  
"X" '(0 0) 300.0 400.0 4 2
```

dbCreateCellEEQClass

Defines electrically equivalent pairs of GDSII Stream cells. If you set a pair of cells as electrically equivalent, both cells are electrically equivalent to all other cells set electrically equivalent to either of the two.

Note:

Astro does not currently use the electrical-equivalent class.

Syntax

```
dbCreateCellEEQClass libId "cellName1" "cellName2"
```

where the arguments are as follows:

libId

Database object ID of the library in which you want to create the electrical-equivalent class

Valid values:

Library ID of any library in the current directory

cellName1

Name of the cell you want to set as electrically equivalent to another cell, *cellName2* (and to all other cells set as electrically equivalent to *cellName2*)

Valid values:

Name of any cell in the specified library

cellName2

Name of the second cell you want to set as electrically equivalent to another cell, *cellName1* (and to all other cells set as electrically equivalent to *cellName1*)

Valid values:

Name of any cell, other than *cellName1*, in the specified library

See Also

[dbClearCellEEQClass](#)

[dbClearCellEQClass](#)

[dbVerifyCellEEQClass](#)

dbCreateCellInstIgnoreECO

Creates a cell instance in a cell and sets the "Ignore ECO" flag. In the case of a Place and Route type cell, this function creates an instance marked for ignore by the ECO comparator. This allows additional instances, such as power and ground pads to be created that are ignored during ECO's.

Syntax

```
dbCreateCellInstIgnoreECO cellId "childLibName"  
"childCellName"  
"cellInstName" "rotationStr" "mirrorStr" Points  
"topCellName"
```

dbCreateCellInstIgnoreECO

where the arguments are as follows:

<i>cellId</i>	Database object ID of the cell in which you want to create the cell instance. Valid values: Cell ID of any open cell in the current library (geGetEditCell).
<i>childLibName</i>	Name of the library containing the cell from which you want to create the cell instance. Valid values: Any library in the current directory. "" specifies the current library or a reference library associated with the current library.
<i>childCellName</i>	Name of the cell from which you can get some data for creating the instance's master cell. Valid values: Name of any cell in the specified library, but cannot be the same one as the former one. If no view name is specified, the default view will be taken.
<i>cellInstName</i>	Name you want to assign to the new cell instance. Valid values: Any text string with 1024 characters or less.
<i>rotationStr</i>	Degree to which you want the cell instance rotated in reference to the master cell. Valid values: 0 90 180 270 any other values would be treated as zero "0."

<i>mirrorStr</i>	Indicates the orientation of the cell instance. Valid values: No X Y any other values would be treated as “No.”
<i>Points</i>	Reference point for the new instance. Valid syntax: (list (x y))
<i>topCellName</i>	Is always NULL, unless failure occurs when retrieving “_status.” Then the property cannot be NULL. Valid Values: Name of the top cell in the open library.

See Also

[dbCreateCellInst](#)

dbCreateCellInstWithoutCheckStatus

Creates a cell instance in a cell, but does not check the status nor the “Ignore ECO” flag. In the case of a Place and Route type cell, this function creates an instance marked for ignore by the ECO comparator. This allows additional instances, such as power and ground pads to be created that are ignored during ECO’s.

Syntax

```
dbCreateCellInstWithoutCheckStatus cellId "childLibName"
    "childCellName" "cellInstName" "rotationStr" "mirrorStr"
Points
    "topCellName"
```

where the arguments are as follows:

<i>cellId</i>	Database object ID of the cell in which you want to create the cell instance. Valid values: Cell ID of any open cell in the current library (geGetEditCell).
<i>childLibName</i>	Name of the library containing the cell from which you want to create the cell instance. Valid values: Any library in the current directory. "" specifies the current library or a reference library associated with the current library.
<i>childCellName</i>	Name of the cell from which you can get some data for creating the instance's master cell. Valid values: Name of any cell in the specified library, but cannot be the same one as the former one. If no view name is specified, the default view will be taken.
<i>cellInstName</i>	Name you want to assign to the new cell instance. Valid values: Any text string with 1024 characters or less.
<i>rotationStr</i>	Degree to which you want the cell instance rotated in reference to the master cell. Valid values: 0 90 180 270 any other values would be treated as zero "0."

<i>mirrorStr</i>	Indicates the orientation of the cell instance. Valid values: No X Y any other values would be treated as "No."
<i>Points</i>	Reference point for the new instance. Valid syntax: (list (x y))
<i>topCellName</i>	Is always NULL, unless failure occurs when retrieving "_status." Then the property cannot be NULL. Valid Values: Name of the top cell in the open library.

See Also

[dbCreateCellInst](#)

[dbCreateCellInstIgnoreECO](#)

dbCreateCellLEQClass

Defines logically equivalent pairs of GDSII Stream cells. If you set a pair of cells as logically equivalent, both cells are logically equivalent to all other cells set logically equivalent to either of the two.

Syntax

```
dbCreateCellLEQClass libId "cellName1" "cellName2"
```

where the arguments are as follows:

libId

Database object ID of the library in which you want to create a logical-equivalent class

Valid values:

Library ID of any library in the current directory

cellName1

Name of the cell you want to set as logically equivalent to another cell, *cellName2* (and to all other cells set as logically equivalent to *cellName2*)

Valid values:

Any cell in the specified library

cellName2

Name of the cell you want to set as logically equivalent to another cell, *cellName1* (and to all other cells set as logically equivalent to *cellName1*)

Valid values:

Any cell, other than *cellName1*, in the specified library

See Also

[dbClearCellLEQClass](#)

[dbClearCellEQClass](#)

[dbVerifyCellLEQClass](#)

dbCreateCellProperty

Creates a cell property.

Note:

A Synopsys tool infers the property type from the property value. List values are illegal. Only atoms may be used for the argument value.

Syntax

```
dbCreateCellProperty cellId "propName" propValue
```

where the arguments are as follows:

cellId

Database object ID of the cell to which you want to add the cell property

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

propName

Name you want to assign to the new cell property

Valid values:

Any text string of 31 characters or fewer

propValue

Value you want to assign to the new cell property

Valid value types:

- Boolean
- Float

- Integer
- String

See Also

[dbDeletePropertyByName](#)

dbCreateCellRegion

Creates a cell region with horizontal rows.

Syntax

```
dbCreateCellRegion cellId pointArray
```

where the arguments are as follows:

cellId

Database object ID of the cell to which cell you want to add the cell region

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

pointArray

Coordinates indicating the dimensions of the cell region

Valid syntax:

```
' ((x1 y1) (x2 y2))
```


See Also

[dbSetCellRegionBBox](#)
[dbSetCellRegionTargetPercent](#)
[dbAddCellInstToCellRegion](#)
[dbCreateCellRow](#)
[dbDeleteObject](#)
[dbPurgeObject](#)
[dbReplaceRegionWithInst](#)

dbCreateCellRow

Creates a cell row.

Syntax

```
dbCreateCellRow cellId regionId pointArray
```

where the arguments are as follows:

cellId

Database object ID of the cell containing the cell region to which you want to add the cell row

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

regionId

Database object ID of the cell region to which you want to add the cell row

Valid values:

Region ID of any region in the specified cell

pointArray

Coordinates indicating the dimensions of the cell row

Valid syntax:

```
' ((x1 y1) (x2 y2))
```

See Also

[dbSetCellRowMirror](#)

[dbStartCellRowDoubleBack](#)

dbCreateChildNet

Creates a child net of the specified net. Child nets are used when a net has connections in more than one region, or in a region and macro cells or pad cells.

Caution!

It is recommended that you do not use this function. It is essential that the naming conventions be consistent with those used by the Synopsys tool for proper system operation. Failure to follow the naming conventions can cause database problems.

Syntax

```
dbCreateChildNet cellId "childNetName" netId
```

where the arguments are as follows:

cellId

Database object ID of the cell to which you want to add the child net

Valid values:

- Cell ID of any open cell in the current library
- (`geGetEditCell`)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

childNetName

Name you want to assign to the new child net

Valid values:

The parent net name with the string “*nn*” when *nn* is the region number in which the net will be used.

netId

Database object ID of the existing (parent) net from which you want to create the child net

Valid values:

Net ID of any net in the master cell

See Also

[dbGetNetByName](#)

[dbPurgeNet](#)

[dbPurgeNetByName](#)

dbCreateChipPort

Creates a top-level port on a cell. The `dbCreateChipPort` function returns *nil*—a set of parentheses, `()`—when it succeeds. When `dbCreateChipPort` fails, the Synopsys tool returns an error message. For example, if a port already exists for the specified net, Astro displays the following error message:

```
Port creation failed because WARNING: Port already  
exists for the given net.  
( )
```

If you plan to do timing-driven placement and your design does not have top-level ports, you need to create them before you partition the cells.

Note:

After creating the port, set its characteristics by using [dbSetCellPortTypes](#).

Syntax

```
dbCreateChipPort libName cellName netName portName
```

where the arguments are as follows:

libName

Name of the library containing the cell to which you want to add the top-level port

Valid values:

Name of any library in the current directory

cellName

Name of the cell to which you want to add the port

Valid values:

Name of any cell in the specified library

netName

Name of the net to which you want the new port to be connected

Valid values:

Name of any net in the specified cell

portName

Name you want to assign to the new port

Valid values:

Any string of 1,023 characters or fewer

Example

```
dbCreateChipPort "demo" "topj" "VDD" "VDD"  
( )
```

This code creates a top-level port on cell topj in library demo. The port is named VDD and is connected to the VDD net.

See Also

[dbSetPortDirection](#)

dbCreateContact

Creates a contact.

Prerequisite

All contacts used in your design must be defined in the deviceTable section of the technology file. For more information, see the *Milkyway Environment Data Preparation User Guide*.

Syntax

```
dbCreateContact cellId contactNumber point
```

where the arguments are as follows:

cellId

Database object ID of the cell for which you want to create a contact

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

contactNumber

Number you specified in the deviceTable section of the technology file that identifies the contact you want to create.

Valid values:

Any integer from 1 to 255 used in the deviceTable section to identify a contact. For the following deviceTable section, the contact number for the contact, polymet1, is 1.

```
*deviceTable
```



```
*contact 1 polymet1
cont poly m1 1 1 1.2 2.5 2.0
defaultRowNumber 2
parasitics 0.2 0.0 0.0
*end
```

point

Location of the new contact

Valid syntax:

'(x y)

dbCreateError

Signals a layout versus schematic (LVS) or design rule checking (DRC) error for a specified class of errors. If successful, `dbCreateError` returns the database object ID of your error.

Caution!

Synopsys LVS and DRC do not check for user-defined errors.

Prerequisite

You must first define a class of errors, using [dbCreateErrorClass](#).

Syntax

```
dbCreateError cellId errClassId "message"
```

where the arguments are as follows:

cellId

Database object ID of the cell containing a specified class of errors for which you want to signal an error

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

errClassId

Database object ID of the class of errors for which you want to signal an error

Valid values:

Error class ID of any class of errors in the specified cell

message

Message describing the signaled error

Valid values:

Any string of 200 characters or fewer

Note:

When executing `dbCreateError`, assign the result to a variable.

See Also

[dbSetErrorStatus](#)

[dbCreateErrorClass](#)

dbCreateErrorClass

Creates a class of LVS or DRC errors. If successful, `dbCreateErrorClass` returns the database object ID of your class of errors.

Caution!

Synopsys LVS and DRC do not check for user-defined errors.

Syntax

```
dbCreateErrorClass cellId errLevel "className" "message"
```

where the arguments are as follows:

cellId

Database object ID of the cell for which you want to create a class of errors

Valid values:

- Cell ID of any open cell in the current library
- (`geGetEditCell`)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

errLevel

Number you want to assign to the severity of the error

Valid values:

Positive integer from 0 to 4

className

Name you want to assign to the new class of errors

Valid values:

Any string of 31 characters or fewer

message

Message describing the class of error

Valid values:

Any string of 200 characters or fewer

Note:

When executing `dbCreateErrorClass`, assign the result to a variable.

See Also

[dbSetErrorClassPattern](#)

[dbSetErrorClassStatus](#)

[dbCreateError](#)

dbCreateErrorComponent

Creates an error component object and attaches it to an error object. An error component consists of a set of one or more rectangles that graphically highlight the location of an error. The rectangles are supplied as an even-numbered list of coordinate pairs supplied to the function as the *pointArray* parameter. Each pair of coordinates represents opposing corners of a rectangle.

Syntax

```
dbCreateErrorComponent cellId errObjId nodeNum "netName"  
pointList
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (*geGetEditCell*)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

errObjId

Database object ID of the error object where the error component information applies. The errorObject can be created using the [dbCreateErrorObj](#) command.

Valid values:

ID of any error object in the open cell

nodeNum

Represents the integer value displayed by GUI commands such as *geAllError*

netName

Name of the net to which the error component relates. The value is not currently used.

Valid values:

Name of any net in the open cell to which the error component relates

pointList

Represents the array of points describing one or more rectangles

Valid values:

Must be an even number of points, each pair representing opposite corners of a rectangle

dbCreateErrorObj

Creates an error object. An error object represents one error. The error object is associated with a single error type and might have numerous error components. The `dbCreateErrorObj` command returns the database object ID of the new error object.

Syntax

```
dbCreateErrorObj cellId errTypeId "Info"
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (*geGetEditCell*)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

errTypeId

Database object ID of the error type where the error object information applies

Valid values:

ID of any error type in the open cell. The error type can be created using the [dbCreateErrorType](#) command.

Info

The information related to the error

Valid values:

String of characters

dbCreateErrorType

Creates an error type. An error type represents a particular kind of error and might have several related error objects. The `dbCreateErrorType` command returns the database object ID of the new error type.

Syntax

```
dbCreateErrorType cellId errorLevel "typeName" "info"
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (`geGetEditCell`)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

errorLevel

The severity of the error type. The value is not used now.

typeName

Name of the error type. The length is limited to 15 characters.

Valid values:

Name of the creating error type

Info

The information related to the error

Valid values:

String of characters

dbCreateGLink

Creates and connects a global link to a net.

Syntax

```
dbCreateGLink<objectId cellId> <objectId netId>  
    <int layerNumber> <int firstX> <int firstY>  
    <int secondX> <secondY>
```

where the arguments are as follows:

<code>cellId</code>	Database object ID of the cell.
<code>netId</code>	Database object ID of the net to which to connect the global link.
<code>layerNumber</code>	Layer number to which to add the global link.

<code>firstX</code>	X-coordinate of the global link's first point.
<code>firstY</code>	Y-coordinate of the global link's first point.
<code>secondX</code>	X-coordinate of the global link's second point.
<code>secondY</code>	Y-coordinate of the global link's second point.

Description

The `dbCreateGLink` command creates and connects a global link to a net. The created global link object ID is returned.

Example

```
dbCreateGLink (geGetEditCell) #x400 5 5000 9000 7500 9000
```

dbCreateGlobalNetByScanChain

Creates a global net for scan control (for example, scan clock or scan enable for the cell instances) of a specified scan chain in the open cell.

Note:

This operation generates an ECO.

Tool

Milkyway

Syntax

```
dbCreateGlobalNetByScanChain (geGetEditCell) chainName  
netName  
' (srcInstName srcPortName) portName
```

where the arguments are as follows:

<i>chainName</i>	Name of the scan chain for which you are creating a global net. Valid values: Any scan chain defined for the cell.
<i>netName</i>	Name of the net you are creating. Valid values: Any string up to 31 characters.
<i>srcInstName</i>	Name of the source cell instance in the scan chain, or a null string ("") to indicate that the source port is a top-level port.
<i>srcPortName</i>	Name of the source port of the scan chain.
<i>portName</i>	Name of the port in each scan cell instance to which the net should be connected.

dbCreateGroup

Creates a group of objects. If successful, `dbCreateGroup` returns the database object ID of your group.

Syntax

```
dbCreateGroup cellId "groupName" groupNumber
```

where the arguments are as follows:

cellId

Database object ID of the cell for which you want to create a group

Valid values:

- Cell ID of any cell in the open library

- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

groupName

Name you want to assign to the new cell group

Valid values:

Any string of 31 characters or fewer

Note:

Group names are always case-sensitive, regardless of the case-sensitivity setting.

groupNumber

Number you want to assign to the new group

Valid values:

A number from 0 to 255. A Synopsys tool reserves group numbers 0 to 200 and 255 for predefined purposes.

Note:

When executing `dbCreateGroup`, assign the result to a variable.

See Also

[dbDeleteObject](#)

[dbPurgeObject](#)

[dbAddObjectToGroup](#)

[dbRemoveObjectFromGroup](#)

[dbGetAllInstFromGroup](#)

[aprCreateGroup](#)

dbCreateHierNet

Creates a hierarchical net.

Syntax

```
dbCreateHierNet cellId "netName" netId cellRegionNumber
```

where the arguments are as follows:

cellId

Database object ID of the cell to which you want to add a hierarchical net

Valid values:

- Cell ID of any cell in the open library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

netName

Name you want to assign to the new hierarchical net

Valid values:

Parent net name with the string "*nn*" appended, where *nn* is the cell region number

netId

Database object ID of the parent net

Valid values:

Net ID of any net in the cell

cellRegionNumber

Number of the region in which the net is placed

Valid values:

Number from 0 to 255

dbCreateHorizontalWire

Creates an internal horizontal wire.

Syntax

```
dbCreateHorizontalWire cellId masterWireId wireLength  
xPoint yPoint
```

where the arguments are as follows:

cellId

Database object ID of the cell to which wire information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

masterWireID

Database object ID of the master wire record

Valid values:

Wire ID of any wire in the specified cell

wireLength

Length of the wire

Valid types of values:

- A positive integer, which extends the wire to the right of the origin
- A negative integer, which extends the wire to the left of the origin

xPoint

X-coordinate of the location where you want to place the wire

Valid values:

Any integer

yPoint

Y-coordinate of the location where you want to place the wire

Valid values:

Any integer

dbCreateLeqPortGroup

Creates a logically equivalent set of ports for a cell. Only one set of logically equivalent ports may be defined at this time.

Syntax

```
dbCreateLeqPortGroup cellId `("port1" "port2" ...)
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the node information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

portlist

List of the ports to be defined as logically equivalent

See Also

[dbDeleteLeqPortGroup](#)

dbCreateLib

Creates a library in the current directory. If `dbCreateLib` successfully creates and opens a new library, the Synopsys tool displays the database object ID of the new library.

Syntax

```
dbCreateLib "libName" "techfile.tf" "hierSep"
```

where the arguments are as follows:

libName

Name of the library you want to create

Valid values:

Any text string 255 characters or less

techfile.tf

Name of an existing technology file. The technology file contains information about the cells in the library.

Note:

You can use any valid technology file to create your library. After your library is created, you can modify the technology file. For more information, see the *Milkyway Environment Data Preparation User Guide*.

Valid values:

Any text string

hierSep

The character to be used as a separator in hierarchical names. This argument is optional. If you do not enter a character, hierarchical names in the library will be separated by the default slash (/) character.

See Also

[dbAddRefLib](#)

[dbCloseLib](#)

[dbSetTechFileName](#)

dbCreateLogicView

Creates the LOGIC view in the specified library.

Syntax

```
dbCreateLogicView <string libraryName>
```

where the argument is as follows:

libraryName

Name of the library.

Description

The LOGIC view stores the logic design .db files for each design cell. The `dbCreateLogicView` command only creates and initializes the library subdirectory. It does not import logic design .db files. See also the `geCreateLogicView` command.

Example

```
dbCreateLogicView "myLibrary"
```

dbCreateModelView

Creates the LM (Logic Model) view in the specified library.

Syntax

```
dbCreateModelView <string libraryName>
```

where the argument is as follows:

libraryName

Name of the library.

Description

The LM view stores the model .db files for each library. The `dbCreateModelView` command only creates and initializes the library subdirectory. It does not import any model .db files. See also the `gePrepLibs` command.

Example

```
dbCreateModelView "myRefLibrary"
```

dbCreateMustJoinNets

Creates a net for each port on a cell instance in the specified top-level cell that has must-join pins not assigned to any net. The net name is as follows:

```
MUSTJOIN_cellInstanceName_portName
```

where *cellInstanceName* is the name of the cell instance of the port and *portName* is the name of the port.

Note:

You designate must-join pins using [dbSetCellMustJoinPin](#) or [dbSetCellMustJoinAllPin](#).

Tool

Milkyway

Syntax

```
dbCreateMustJoinNets cellId
```


where the argument is as follows:

<i>cellId</i>	The ID of the top-level cell for which you want to create nets for must-join pins. If the cell is open, you can substitute the following for <i>cellId</i> : (<i>geGetEditCell</i>)
---------------	--

Example

```
dbCreateMustJoinNets (geGetEditCell)
```

This function creates a net for each port in the open cell that has must-join pins not assigned to any net. If cell instance CI has a port B with pins B1 and B2 designated as must-join but not connected to any net, this command creates a net named MUSTJOIN_CI_B.

See Also

[dbSetCellMustJoinPin](#) instructs the tool to externally connect together specified pins on a port.

[dbSetCellMustJoinAllPin](#) instructs the Synopsys tool to externally connect together all pins on a specified port (or all ports with names matching a specified pattern).

The `dbResetCellMustJoinPin` command removes external connection specifications for pins on a port.

dbCreateNet

Creates a net in a cell. When successful, `dbCreateNet` returns the database object ID of the new net.

Syntax

```
dbCreateNet cellId "netName"
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the net information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

netName

Name of the net to create

Valid values:

Any text string of 1,023 characters or fewer

Example

```
define cell1 (dbOpenCell "control.Net1" "w")
db:2
define vddNetId (dbCreateNet cell1 "VDD")
db:110034
dbMoveAllNetMember cell1 (dbGetNetByName cell1 "Logic1")
vddNetId
dbCloseCell cell1
#t
```

In this example,

- The first line opens the cell control.Net1 and assigns the database ID to cell1.

- The second line creates a new net, VDD, and assigns the net ID to vddNetId.
- The third line merges all members from Logic1 into the new net, VDD, and deletes the empty net Logic1.
- The fourth line closes the cell control.Netl.

See Also

[dbGetNetByName](#)

[dbMoveAllNetMember](#)

dbCreateNode

Creates a graph node in a cell.

Syntax

```
dbCreateNode cellId "type" nodeNo center edgeNoUp edgeNoDown
edgeNoLeft edgeNoRight
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the node information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

type

Type of cell design

Valid values:

- *singleRegion*, which specifies a single-region design
- *multipleRegion*, which specifies a multiple-region design

Note:

This field is case-insensitive.

nodeNo

Number of the graph node

Valid values:

A positive integer

center

Location of the center of the node

Valid syntax:

' (x y)

edgeNoUp

Number of the edge above the node

Valid values:

A positive integer

edgeNoDown

Number of the edge below the node

Valid values

A positive integer

edgeNoLeft

Number of the edge to the left of the node

Valid values:

A positive integer

edgeNoRight

Number of the edge to the right of the node

Valid values:

A positive integer

dbCreateParamSet

Creates a parameter set record in a cell.

Syntax

```
dbCreateParamSet cellId "applName"
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

applName

Name of the set to create

Valid values:

Any text string 31 characters or less

dbCreateParentNet

Creates a parent net in a cell.

Caution!

It is recommended that you do not use this function.

Syntax

```
dbCreateParentNet cellId "parentNetName" netId  
cellRegionNumber
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the net information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

parentNetName

Name of the net to create

Valid values:

Any text string of 1,023 characters or fewer

netId

Database object ID of an existing net from which a parent net is created

Valid values: Net ID of any net in the specified cell

cellRegionNumber

Number of the region to which the parent net is to be added

Valid values:

A number from 0 to 255, but typically 0

dbCreatePath

Creates a path record for the cell in the database.

Syntax

```
dbCreatePath cellId "layerName" pathType width routeType  
dataType PointArray
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (*geGetEditCell*)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

layerName

Name of the layer to which the path information applies

Valid values:

Name of any layer in the specified cell

pathType

The type of the created path

Valid values:

- 0, when the path edges extend with the coordinate
- 1, when the path edge is rounded
- 2, when the path edge extends a half width beyond the coordinate

width

Desired width for the creating path

routeType

Route type of the created path

Valid values:

Only bits (1-5) are used in the routing type definition

Major type - 2 bits

- 0 axcSignalRouteType
- 16 axcPGRouteType
- 32 axcClockRouteType

- 48 axcMiscRouteType

dataType

Data type to create the path

pointArray

Coordinates to indicate the size and shape of the path

Valid syntax:

'((x1 y1) (x2 y2)....)

dbCreatePcellDefinition

Registers a parameterized cell with the library.

Syntax

```
dbCreatePcellDefinition libId "pcellName" "pcellFileName"
```

where the arguments are as follows:

libId

Database object ID of the current library from which you want to reference another library

Valid values:

- Library ID
- (dbGetCurrentLibId)

For more information, see “Specifying Libraries” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

pcellName

Name of the parameterized cell (string)

pcellFileName

Full path to the file that contains the parameterized cell functions

Note:

At this point, a copy of the parameterized cell's file name is stored with the library. If you modify the file name, the parameterized cell registered in the library will not be automatically updated.

dbCreatePin

Creates a pin record for a cell.

Syntax

```
dbCreatePin cellId netId "pinName" "layerName" pointArray
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the pin information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

netId

Database object ID of the net to which the pin information applies

Valid values:

Net ID of any net in the specified cell

pinName

Name of pin to create

Valid values:

Any text string of 31 characters or fewer

layerName

Name of the layer to which the pin information applies

Valid values:

Name of any layer defined in the cell library's technology file

pointArray

Coordinates indicating the size and shape of the pin

Valid syntax:

' ((x1 y1) (x2 y2))

The first point is the lower-left coordinate of the box, and the second point is the upper-right coordinate of the box. You can specify a polygon pin by entering a set of three or more points, where the last point is the same as the first.

dbCreatePlanGroupAndStatus

Creates a floorplan group and sets its status.

Syntax

```
dbCreatePlanGroupAndStatus <objectId cellId>  
    <string planGroupName> <int groupType>  
    <int targetPercent> <int planStatus>  
    <int rigidity> <int exclusive> <int color>  
    <list pointArray>
```

where the arguments are as follows:

cellId

Database object ID of the design in which to create the floorplan group.

planGroupName

The floorplan group name. Must be a unique floorplan group name.

groupType

The floorplan group type.

Valid values are:

- 0 soft plan group
- 1 hard plan group

targetPercent

The floorplan target utilization percentage. The value is specified in units of one hundredth of a percent (for example, 7 is 0.07%, 70 is 0.7%, 700 is 7.0%, and 7000 is 70.0%).

planStatus

The floorplan group status.

Valid values are:

- 1 cell placed

- 2 invisible
- 4 standard cell move deferred
- 8 macro cell move deferred
- 16 placement fixed

rigidity

The floorplan group rigidity. Valid values include all integers from 0 through 10. Values less than 0 default to 0. Values greater than 10 default to 10.

exclusive

The floorplan group exclusivity.

Valid values are:

- 0 not exclusive
- 1 exclusive

color

The floorplan group display color index. Valid values include all color indices defined in the library technology file.

pointArray

A list of points describing the floorplan group boundary. The list can consist of two points describing two opposite vertices of a rectangular boundary, or it can consist of more than two points describing the vertices of a rectilinear boundary.

Description

The floorplan group is created in the specified cell with the specified parameters.

Example

```
dbCreatePlanGroupAndStatus (geGetEditCell) "A" 0 7000 1 1 0
28 '((0 0) (100 100))
```

See Also

[dbSetPlanGroupAndStatus](#)

dbCreatePolygon

Creates a polygon record for a cell.

Syntax

```
dbCreatePolygon cellId "layerName" pointArray
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the polygon information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

layerName

Name of the layer to which the polygon information applies

Valid values:

Name of any layer in the specified cell

pointArray

Coordinates indicating the size and shape of the polygon

Valid syntax:

$((x_1 \ y_1) \ (x_2 \ y_2) \ \dots \ (x_1 \ y_1))$

A closed set of points

dbCreatePort

Creates a port record on a specified net.

Syntax

```
dbCreatePort cellId netId "portName"
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the port information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

netId

Database object ID of the net to which the port information applies

Valid values:

Net ID of any net in the specified cell

portName

Name of the port to create and attach to the specified net

Valid values:

Any text string of 1,023 characters or fewer

dbCreatePreRoute

Creates a preroute record in a cell.

Syntax

```
dbCreatePreRoute cellId netId startPt endPt edgeNo width  
length layerNumber offset
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the preroute information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

netId

Database object ID of the net to which the preroute information applies

Valid values:

Net ID of any net in the specified cell

startPt

Point corresponding to the beginning of the preroute segment

Valid syntax:

' (x y)

endPt

Point corresponding to the end of the preroute segment

Valid syntax:

' (x y)

edgeNo

Edge to which the preroute information applies

Valid values:

Any positive integer

width

Width of the preroute segment, in user units

Valid values:

Any positive integer

length

Length of the preroute segment between pins

Valid values:

Any positive integer specifying the distance between the preroute segment and the pin

layerNumber

Number of the cell layer where the preroute record should be placed

Valid values:

Any positive integer

offset

Distance between the preroute segment and the center of the node

Valid values:

Any positive integer

dbCreateProperty

Creates a property record associated with an object. The type of the property is inferred from the value.

Syntax

```
dbCreateProperty cellId objectId "propName" value
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library

- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

objectId

Database object ID of the object to which the property information applies

Valid values:

Object ID of any object in the specified cell

propName

Name of the property record to create and associate the object with

Valid values:

Any text string of 35 characters or fewer

value

Value assigned to the property

Valid types:

- String
- Float
- Integer
- Boolean

dbCreatePseudoPath

Creates a pseudopath record in a cell.

Syntax

`dbCreatePseudoPath cellId width "layerName" pointArray`

where the arguments are as follows:

cellId

Database object ID of the cell to which the pseudopath information applies

Valid values:

- Cell ID of any open cell in the current library
- `(geGetEditCell)`

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

width

Width of the pseudopath, in user units

layerName

Name of the layer where the pseudopath record is to be placed

pointArray

Coordinates indicating the shape and size of the pseudopath

Valid syntax:

`'((x1 y1), (x2 y2), ... , (xn yn))`

dbCreateRectangle

Creates a rectangle record in a cell.

Syntax

```
dbCreateRectangle cellId "layerName" pointArray
```

or

```
dbCreateRectangle cellId layerNumber pointArray
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the rectangle information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

layerName

Name of the layer to which the rectangle information applies

Valid values:

Name of any layer in the technology file

layerNumber

Number of the cell layer to which the rectangle information applies

Valid values:

A positive integer

pointArray

Coordinates indicating the shape and size of the rectangle

Valid syntax:

'((X1 Y1) (X2 Y2))

The first point is the lower-left coordinate of the rectangle, and the second point is the upper-right coordinate of the rectangle.

dbCreateRegionPin

Creates a region pin record in a cell. If no region port exists, a side effect of this function is to create the region port. Region pins cannot exist without an associated region port.

Syntax

```
dbCreateRegionPin cellId netId regionNetId regionId  
"regionPinName" "layerName" pointArray
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the region pin information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

netId

Database object ID of the net to which the region pin information applies

Valid values:

Net ID of any net in the specified cell

regionNetId

Database object ID of the region net to which the region pin information applies

Valid values:

Net ID of any region net in the current library

regionId

Database object ID of the region to which the region pin information applies

Valid values:

Region ID of any region in the current library

regionPinName

Name of the region pin to create

Valid values:

Any text string of 31 characters or fewer

layerName

Name of the layer to which the region pin information applies

Valid values:

Name of any layer in the current library’s technology file

pointArray

Coordinates indicating the shape and size of the region pin

Valid syntax:

`'((x1 y1), (x2 y2), ... (xn yn))`

dbCreateRegionPort

Creates a region port record on a cell.

Syntax

`dbCreateRegionPort cellId netId regionNetId regionId`

where the arguments are as follows:

cellId

Database object ID of the cell to which the region port information applies

Valid values:

- Cell ID of any open cell in the current library
- `(geGetEditCell)`

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

netId

Database object ID of the net to which the region port information applies

Valid values:

Net ID of any net in the specified cell

regionNetId

Database object ID of the region net to which the region port information applies

Valid values:

Net ID of any region net in the current library

regionId

Database object ID of the region to which the region port information applies

Valid values:

Region ID of any region in the current library

dbCreateScanChain

Creates an empty scan chain in the open cell.

Tool

Milkyway

Syntax

```
dbCreateScanChain (geGetEditCell) chainName
```

where the argument is as follows:

chainName The name you want to assign to the scan chain.

dbCreateSGEdge

Creates a slack graph edge record.

Syntax

```
dbCreateSGEdge cellId status fromObjId toObjId
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the timing information applies

Valid values:

Cell ID of any cell in the current library

status

Valid values:

0

fromObjId

Database object ID of the object from which the timing signal leaves

Valid values:

Object ID of any object in the current cell

toObjId

Database object ID of the object to which the timing signal comes

Valid values:

Object ID of any object in the current cell

dbCreateSGPrimaryNode

Creates a slack graph primary node record in a specified cell.

Syntax

```
dbCreateSGPrimaryNode cellId status riseMax riseMin fallMax  
fallMin startOrEndId
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the timing information applies

Valid values:

Cell ID of any cell in the current library

status

Whether slack graph primary node should be an arrival time node or a required arrival time node

Valid values:

- 0, which means that the node is an arrival time node
- 1, which means that the node is a required arrival time node

riseMax

Latest time the signal arrives or should arrive on a rising edge

Valid values:

- If status is set to 0, value represents the latest time the signal will arrive.
- If status is set to 1, value represents the latest time the signal should arrive.

riseMin

Earliest time the signal arrives or should arrive on a rising edge

- If status is set to 0, value represents the earliest time the signal will arrive.
- If status is set to 1, value represents the earliest time the signal should arrive.

fallMax

Latest time the signal arrives or should arrive on a falling edge

- If status is set to 0, value represents the latest time the signal will arrive.
- If status is set to 1, value represents the latest time the signal should arrive.

fallMin

Earliest time the signal arrives or should arrive on a falling edge

- If status is set to 0, value represents the earliest time the signal will arrive.
- If status is set to 1, value represents the earliest time the signal should arrive.

startOrEndId

Database object ID of the object at either the start or end of the timing path

Valid values:

Object ID of any object in the specified cell

dbCreateText

Creates a text record in a cell.

Syntax

```
dbCreateText cellId "layerName" "just" "rotation" "mirror"  
height origin "text"
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the text information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

layerName

Name of the layer to which the text information applies

Valid values:

Name of any layer in the specified cell library’s technology file

just

Placement of the text around the origin

Valid values:

ll, which places the text to the lower left of the origin

lc, which places the text to the lower center of the origin

lr, which places the text to the lower right of the origin

cl, which places the text to the center left of the origin

cc, which centers the text around the origin

cr, which places the text to the center right of the origin

ul, which places the text to the upper left of the origin

uc, which places the text to the upper center of the origin

ur, which places the text to the upper right of the origin

rotation

Degree to which the text should be rotated

Valid values:

0

90

180

270

mirror

The orientation of the text

Valid values:

No

X

Y

height

Size of the text in the y direction, in user units

origin

Coordinate to which the text should be anchored

Valid syntax:

`'(x1 y1)`

text

Text string that should appear in the cell

Valid values:

Any string

dbCreateVerticalWire

Creates an internal vertical wire.

Syntax

```
dbCreateVerticalWire cellId masterWireId wireLength xPoint  
yPoint
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the wire information applies

Valid values:

- Cell ID of any open cell in the current library
- `(geGetEditCell)`

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

masterWireId

Database object ID of the master wire record

Valid values:

Wire ID of any wire in the specified cell

wireLength

Length of the wire

Valid values:

- A positive integer, which extends the wire above the origin
- A negative integer, which extends the wire below the origin

xPoint

X-coordinate of the location where you want to place the wire

Valid values:

Any integer

yPoint

Y-coordinate of the location where you want to place the wire

Valid values:

Any integer

dbCreateWireMaster

Creates a wire master record in a specified cell.

Syntax

```
dbCreateWireMaster cellId pathType layerType wireWidth  
layerNumber
```


where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

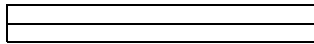
For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

pathType

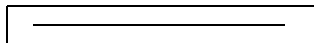
Alignment of the wire

Valid values:

- 0, which means that the path edge aligns with the coordinate



- 2, which means that the path edge extends beyond the coordinate



layerType

0

Note:

This field is currently unused.

wireWidth

Width of the wire, in database units

layerNumber

Layer number for instances of this wire master

dbDefineAbsXtalkConstraint

You can define crosstalk constraints with absolute values. For a detailed explanation of crosstalk constraints, please refer to [dbDefineXtalkConstraint](#).

The unit of absolute value is wire-length or capacitance.

For type "l", the absolute measure is the total parallel length.

For type "c", the absolute measure is the total coupling capacitance.

For type "n", the absolute measure is the total coupling capacitance scaled by the switching speed.

The measure used by [dbDefineXtalkConstraint](#) is essentially the ratio between the peak noise voltage and the supply voltage.

For type "l", the absolute measure is the (noise * total wire length of the net).

For type "c" and "n", the absolute measure is the (noise * total load capacitance of the net).

Example calculation for *absolute* measure:

peak noise voltage = 0.026 x supply voltage

Total cap on net = 0.0243743 pf

absolute measure = (0.026) (0.0243743) = 0.00063

Tool

Milkyway

Syntax

```
(dbDefineAbsXtalkConstraint cellId conxId type maxValue  
  ' ((aggressorClassId weight) ....))
```

where the arguments are as follows:

<i>cellId</i>	ID of the cell for which you want to assign variable-width routing rules. Valid values: ID of any cell in the open library. If the cell is open, you can substitute the following for <i>cellId</i> : (<i>geGetEditCell</i>).
<i>conxId</i>	ID of the crosstalk constraints. Valid values: Integers less than 256.
<i>type</i>	"l" : parallel length "c" : coupling capacitance "n" : noise
<i>maxValue</i>	The absolute crosstalk constraints value.

Example

```
(dbDefineAbsXtalkConstraint cellId 3 "l" 30.0 ' ((0 1.0))
```

See Also

[dbClearCellXtalkConstraints](#)
[dbDefineXtalkConstraint](#)
[dbClearCellXtalkLengthWeight](#)
[dbSetXtalkLengthWeight](#)
[dbAssignNetXtalkClass](#)
[dbAssignNetXtalkConstraints](#)

[dbAllowToAddPGIOText](#)
[dbDumpXtalkConstraints](#)
[axgSetXtalkRouteOptions](#)

dbDefineAntennaAccumMode

Creates an antenna accumulation mode route rule.

Syntax

```
dbDefineAntennaAccumMode <objectId libId>  
    <int accumCutToMetal> <int accumMetalToCut>
```

where the arguments are as follows:

libId

Database object ID of the library.

accumCutToMetal

Accumulation cut to metal mode.

accumMetalToCut

Accumulation metal to cut mode.

Example

```
dbDefineAntennaAccumMode (dbGetCurrentLibId) 1 2
```

dbDefineAntennaLayerRatioScale

Creates an antenna layer ratio route rule.

Syntax

```
dbDefineAntennaLayerRatioScale <objectId libId>  
    <string layerName><float layerScale>  
    <float accumScale>
```

where the arguments are as follows:

libId

Database object ID of the library.

layerName

Name of the layer.

layerScale

Layer scale factor.

accumScale

Accumulation scale factor.

Example

```
dbDefineAntennaLayerRatioScale (dbGetCurrentLibId)  
"metal1" 1000.00 2.00
```

dbDefineAntennaRule

Defines an advanced antenna rule for the specified mode and stores it in the library. *Diode mode 3* is now supported for accumulating the protections of multiple diodes on the same net. *Diode mode 2* only considers the maximum diode protection if there are multiple diodes.

Tool

Milkyway

Syntax

```
dbDefineAntennaRule libId mode diodeMode defaultMetalRatio
defaultCutRatio
```

where the arguments are as follows:

<i>libId</i>	The Id of an opened library: (<i>dbGetCurrentLibId</i>).
<i>mode</i>	Defines the way antenna areas are computed. Valid values: - Uses polygon area, ignoring all lower-layer segments. - Uses polygon area, including all lower-layer segments to the input pins. - Uses polygon area, including all lower-layer segments. - Uses side-wall area, ignoring all lower-layer segments. - Uses side-wall area, including all lower-layer segments to the input pins. - Uses side-wall area, including all lower-layer segments. Only one rule can be defined for every mode. If two commands contain the same mode number, the second command will overwrite the first one.
<i>diodeMode</i>	Defines the protection capability of the diode. By default, all output pins are considered as a diode. Valid values: 0—output pin cannot protect antenna. 1—output pin can provide unlimited protection. 2—output pin protection is limited and is defined by dbAddAntennaLayerRule and dbDefineIgnorePin (in the CLF file).
<i>defaultMetalRatio</i>	Maximum allowable ratio for metal area to gate size if the metal layer is not defined with dbAddAntennaLayerRule. If this value is 0, the ratio will be ignored. Valid value: any non-negative number.

defaultCutRatio Maximum allowable ratio for cut area to gate size if the cut layer is not defined with [dbAddAntennaLayerRule](#). If this value is 0, the ratio will be ignored.
Valid value: any non-negative number.

Example

```
define _libId (dbGetCurrentLibId)
dbDefineAntennaRule _libId 1 2 1000 0
```

See Also

[dbAddAntennaLayerRule](#) defines an advanced antenna rule for the specified layer and stores it in the library.

[dbClearLibAntennaRules](#) deletes all of the antenna rules stored in the library.

[dbDumpLibAntennaRules](#) dumps all of the antenna rules stored in the library to a file.

dbDefineCTSPin

This timing command identifies an input port on a cell instance as a CTS pin and specifies the level number at which the ports of the cell instance should be connected during clock tree synthesis.

Tool

Milkyway

Syntax

Note:

A construction enclosed by braces ({ }) and followed by three horizontal dots (...) can be repeated.

```
dbDefineCTSPin cellId "cellInstName" '({("portName"
ctsLevel
Levelno)}) ...)
```

where the arguments are as follows:

<i>cellId</i>	ID of the top-level cell containing the cell instance for which you are defining a CTS pin. Valid values: ID of any top-level cell in the current directory.
<i>cellInstName</i>	Name of the cell instance for which you are defining a CTS pin. Valid values: Name of any cell instance in the top-level cell.
<i>portName</i>	Name of the input port you are defining as a CTS point.
<i>ctsLevel</i>	Keyword for specifying the level property of the CTS pin.
<i>Levelno</i>	Integer for specifying the level number in the clock tree root at which the port instance should be connected. Valid value: Any level number between one and the maximum level number of the clock tree.

Example

```
dbDefineCTSPin (geGetEditCell) "INST1" '("P1" "ctsLevel"
3))
```

This example instructs Astro that *P1* of cell instance *INST1* must be connected at level 3 while synchronizing *P1* with all clock input ports.

See Also

[dbDumpCTSPin](#) writes a files containing information about input ports to be set as CTS pins during clock tree synthesis.

[dbPurgeCTSPin](#) changes input ports previously identified as CTS pins to non-CTS ports.

dbDefineCTSPort

This timing command identifies a port on a master cell instance as a CTS port and specifies the level number at which ports of the master cell should be connected during clock tree synthesis.

Because this command is defined on a master cell, it can be loaded through *clfload*.

Tool

Milkyway

Syntax

Note:

A construction enclosed by braces ({ }) and followed by three horizontal dots (...) can be repeated.

```
dbDefineCTSPort "masterCellName" '({("portName" ctsLevel  
Levelno)} ...)
```

where the arguments are as follows:

<i>masterCellName</i>	Name of the master cell instance for which you are defining a CTS port. Valid values: Name of any master cell instance in the top-level cell.
<i>portName</i>	Name of the input port you are defining as a CTS point.

<i>ctsLevel</i>	Keyword for specifying the level property of the CTS port.
<i>Levelno</i>	Integer specifying the level number in the clock tree root at which the cell instances of a macro cell should be connected.

Example

```
dbDefineCTSPort "INST1" ' ("P1" "ctsLevel" 3))
```

This example instructs Astro that the port named *P1* of the cell *INST1* should be connected at level 3.

dbDefineDifferentialGroup

Defines differential groups. Each group can have two or more nets.

Syntax

```
dbDefineDifferentialGroup cellId groupName netNameList
```

where the arguments are as follows:

cellId

cell Id to which nets of the groups belong

groupName

Name of the parameterized cell (string)

netNameList

List of the net names you want to form the group

Example

The following defines a group “g1” with nets “A” and “B” as its members:

```
dbDefineDifferentialGroup (geGetEditCell) "g1" ' ("A" "B")
```

dbDefineIgnorePin

This timing command sets an input port on a cell instance as a ignore pin.

This is useful when you want to instruct Astro to ignore specific ports connected to the clock network during clock tree synthesis and skew analysis operations.

Tool

Milkyway

Syntax

Note:

A construction enclosed by braces ({ }) and followed by three horizontal dots (...) can be repeated.

```
dbDefineIgnorePin cellId "cellInstName" '({("portName"}
...)
```

where the arguments are as follows:

<i>cellId</i>	ID of the top-level cell containing the cell instance for which you are defining as an ignore pin. Valid values: ID of any top-level cell in the current directory.
<i>cellInstName</i>	Name of the cell instance for which you are defining an ignore pin. Valid values: Name of any cell instance in the top-level cell.
<i>portName</i>	Name of the input port you are defining as an ignore pin.

Example

```
dbDefineIgnorePin (geGetEditCell) "INST1" '("P1" "P2")
```

This example instructs Astro to ignore the ports P1 and P2 of INST1 during CTS and skew analysis operations.

See Also

[Chapter 25, “dbDumpIgnorePin](#) writes a file containing information about input ports to be set as ignore pins during clock tree synthesis.

[dbPurgeIgnorePin](#) changes input ports previously identified as ignore pins to default ports.

dbDefineIgnorePort

This timing command sets a port on a master cell instance as a ignore port. You would use this command when you want to ignore specific ports of a master cell connected to the clock network during clock tree synthesis and skew analysis operations.

Tool

Milkyway

Syntax

Note:

A construction enclosed by braces ({ }) and followed by three horizontal dots (...) can be repeated.

```
dbDefineIgnorePort "masterCellName" '({("portName"))} ...)
```

where the arguments are as follows:

<i>masterCellName</i>	Name of the master cell instance for which you are defining a ignore port. Valid values: Name of any master cell instance in the top-level cell.
<i>portName</i>	Name of the input port you are defining as a ignore port.

Example

```
dbDefineIgnorePort "INST1" '("P1" "P2")
```

This example instructs Astro to ignore ports named *P1* and *P2* of the master cell named *INST1* during CTS and skew analysis operations

dbDefineLengthRouteRuleTable

Defines a length based table for variable-width routing rules or min/max layer constraints.

Tool

Milkyway

Syntax

```
dbDefineLengthRouteRuleTable cellId ruleType Tablelist
```

where the arguments are as follows:

<i>cellId</i>	ID of the top-level cell containing the cell instance for which you are defining length. Valid values: ID of any top-level cell in the current directory.
<i>ruleType</i>	"varRoute" for variable-width routing rules; "minMaxLayer" for min/max layer constraints.
<i>Tablelist</i>	See the Examples below.

Examples

```
dbDefineLengthRouteRuleTable (geGetEditCell) "varRoute"  
'((0 1000 "rule1") (1000 2000 "rule2")  
)  
dbDefineLengthRouteRuleTable (geGetEditCell) "minMaxLayer"  
'((0 1000 "METAL2" "METAL4") (1000 2000 "METAL4" "METAL5")  
)
```

dbDefineLeqPortGroup

Specifies logically equivalent port information in the database for a given library. To write logically equivalent port information for a given library, use [dbDumpLeqPortGroup](#).

Syntax

```
dbDefineLeqPortGroup "libraryName" "cellName"  
' ( { ("portName") }  
... )
```

where the arguments are as follows:

<i>libraryName</i>	Name of the library in which master cell exists for which <i>Leq Port Group</i> is to be defined. Valid values: Valid Object <i>libraryName</i> .
<i>cellName</i>	Name of the master cell for which <i>Leq Port Group</i> is to be defined. It should be the .FRAM view of the master cell.
<i>portName</i>	List of port names that are logically equivalent.

Example

To define the ports “A” and “B” of “AND2X1” cell in “access05” to be logically equivalent:

```
dbDefineLeqPortGroup "access05" "AND2X1.FRAME" ' ("A" "B")
```

dbDefineSyncPin

This timing command identifies an input port on a cell instance as a sync point and specifies minimum and maximum phase delays for this port at different phases. Phase delay is the delay you want Astro to add to the clock path delay at a sync point when synchronizing the inputs during clock tree synthesis. Astro uses this information to determine at which level it should connect the cell instance to the clock tree. Astro also uses this information to generate clock skew and synthesized net skew reports. This command should be initiated before loading the common graph. If you want to use the dbDefineSyncPin command after the common graph is loaded, you must update the common graph. It is recommended that you use ataDefineSyncPin instead.

For example, the phase delay at the clock input port on a macro is the delay between the clock input port of the macro and the clock input to the flip-flops inside the macro. In other words, it is the delay through the macro's internal clock tree.

If you are performing clock tree synthesis, you need to specify the phase delay for the input port on each macro in the clock tree. In some cases, you also need to specify the phase delay for the input port on each cell instance in the clock tree that is not a flip-flop or latch. You can specify phase delay for an input port on a master cell basis using `definePortPhaseDelay`.

Note:

The Set All Inputs As Synchronous Point option on the Multi-level Clock Tree Synthesis dialog box (see `aprMClkTreeSyn`) identifies all non-clock input ports connected to the net being synthesized

(and not previously identified as sync points by dbDefineSyncPin) as sync points with zero (0) phase delay for the selected phases.

Tool

Milkyway

Syntax

Note:

A construction enclosed by braces ({ }) and followed by three horizontal dots (...) can be repeated.

```
dbDefineSyncPin cellId "cellInstName" '({ ("portName"  
  {spec} ...}) ...)
```

where the arguments are as follows:

<i>cellId</i>	ID of the top-level cell containing the cell instance for which you are defining a sync point. Valid values: ID of any top-level cell in the current directory.
<i>cellInstName</i>	Name of the cell instance for which you are defining a sync point. Valid values: Name of any cell instance in the top-level cell.
<i>portName</i>	Name of the input port you are defining as a sync point.

spec

Specification of phase delays for the specified input port, in the following format:

"phase" phaseDelays

where

phase indicates the unate state and reference edge for the phase delays that follow and can be any of the following:

nonInvertRise indicates the following:

The unate state at the input port on the internal flip-flop is the same as the unate state at the input port on the cell instance (in other words, the signal is not inverted).

The input port should be synchronized with other input ports when the clock signal is rising.

nonInvertFall indicates the following:

The unate state at the input port on the internal flip-flop is the same as the unate state at the input port on the cell instance (in other words, the signal is not inverted).

The input port should be synchronized with other input ports when the clock signal is falling.

invertRise indicates the following:

The unate state at the input port on the internal flip-flop is the inverse of the unate state at the input port on the cell instance (in other words, the signal is inverted).

The input port should be synchronized with other input ports when the clock signal is rising.

invertFall indicates the following:

The unate state at the input port on the internal flip-flop is the inverse of the unate state at the input port on the cell instance (in other words, the signal is inverted).

The input port should be synchronized with other input ports when the clock signal is falling.

Note:

The reference edge you specify should be consistent with the reference edge you specify for the master cell using `defineTimeIntrinsic` or `defineTimeTLU`.

phaseDelays are the minimum and maximum phase delays that you want added to the clock path delay at the input port. You can express these in either of the following ways:

A single value, expressed as follows:

value

where *value* is a floating-point number that expresses both the minimum and maximum phase delays. You would use this format in either of the following cases:

The cell instance has a single internal clock input to a flip-flop.

The delay between the input to the cell instance and the clock inputs to all internal flip-flops is the same.

Two values, expressed as follows:

(min max)

where *min* and *max* are floating-point numbers that express the following:

min expresses the shortest delay between the input to the cell instance and the clock input to an internal flip-flop.

max expresses the longest delay between the input to the cell instance and the clock input to an internal flip-flop

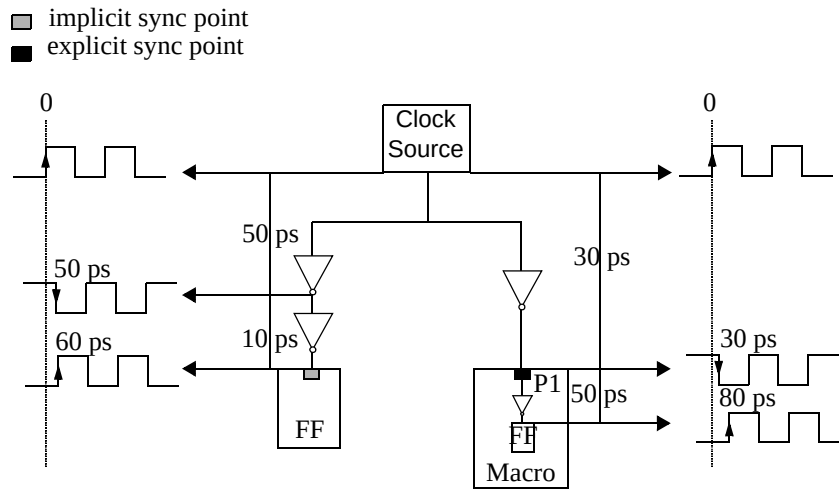
Use this format when the delays between the input to the cell instance and the clock inputs to internal flip-flops are different.

Example

```
dbDefineSyncPin (geGetEditCell) "INST1"  
'(("P1" "invertRise" 50))
```

This example instructs Astro to synchronize P1 with all clock input ports and all sync points when the clock signal is rising, adding a phase delay of 50 ps to the clock path delay at the input P1 and inverting the unate state at the input port to the cell instance. The time unit is set by the Astro technology file.

The diagram below shows the case when this pin is compared with other sync points in the nonInvertRise category of the clock skew report.



See Also

[dbDumpSyncPin](#) writes a files containing information about input ports to be synchronized during clock tree synthesis.

[dbPurgeSyncPin](#) changes input ports previously identified as sync points to nonsynchronous ports.

dbDefineSyncPort

This timing command allows you to specify minimum and maximum phase delays at different phases of an input port on a master cell of a macro. With this, the ports of cell instances of the macro are treated as sync ports by CTS and skew analysis operations.

If *dbDefineSyncPort* it is defined on a master cell, you can load this command through *clffload*.

Note:

The Set All Inputs As Synchronous Point option on the Multi-level Clock Tree Synthesis dialog box (see `aprMClkTreeSyn`) identifies all non-clock input ports connected to the net being synthesized (and not previously identified as sync points by `dbDefineSyncPin`) as sync points with zero (0) phase delay for the selected phases.

Tool

Milkyway

Syntax

Note:

A construction enclosed by braces (`{ }`) and followed by three horizontal dots (`...`) can be repeated.

```
dbDefineSyncPort "masterCellName" ' ( ( "portName"  
  {spec} ... ) } ... )
```

where the arguments are as follows:

<i>masterCellName</i>	Name of the master cell instance for which you are defining a sync point. Valid values: Name of any master cell instance in the top-level cell.
<i>portName</i>	Name of the input port you are defining as a sync point.

spec

Specification of phase delays for the specified input port, in the following format:

"phase" phaseDelays

where

phase indicates the unate state and reference edge for the phase delays that follow and can be any of the following:

nonInvertRise indicates the following:

The unate state at the input port on the internal flip-flop is the same as the unate state at the input port on the cell instance (in other words, the signal is not inverted).

The input port should be synchronized with other input ports when the clock signal is rising.

nonInvertFall indicates the following:

The unate state at the input port on the internal flip-flop is the same as the unate state at the input port on the cell instance (in other words, the signal is not inverted).

The input port should be synchronized with other input ports when the clock signal is falling.

invertRise indicates the following:

The unate state at the input port on the internal flip-flop is the inverse of the unate state at the input port on the cell instance (in other words, the signal is inverted).

The input port should be synchronized with other input ports when the clock signal is rising.

invertFall indicates the following:

The unate state at the input port on the internal flip-flop is the inverse of the unate state at the input port on the cell instance (in other words, the signal is inverted).

The input port should be synchronized with other input ports when the clock signal is falling.

Note:

The reference edge you specify should be consistent with the reference edge you specify for the master cell using `defineTimeIntrinsic` or `defineTimeTLU`.

phaseDelays are the minimum and maximum phase delays that you want added to the clock path delay at the input port. You can express these in either of the following ways:

A single value, expressed as follows:

value

where *value* is a floating-point number that expresses both the minimum and maximum phase delays. You would use this format in either of the following cases:

The cell instance has a single internal clock input to a flip-flop.

The delay between the input to the cell instance and the clock inputs to all internal flip-flops is the same.

Two values, expressed as follows:

(min max)

where *min* and *max* are floating-point numbers that express the following:

min expresses the shortest delay between the input to the cell instance and the clock input to an internal flip-flop.

max expresses the longest delay between the input to the cell instance and the clock input to an internal flip-flop.

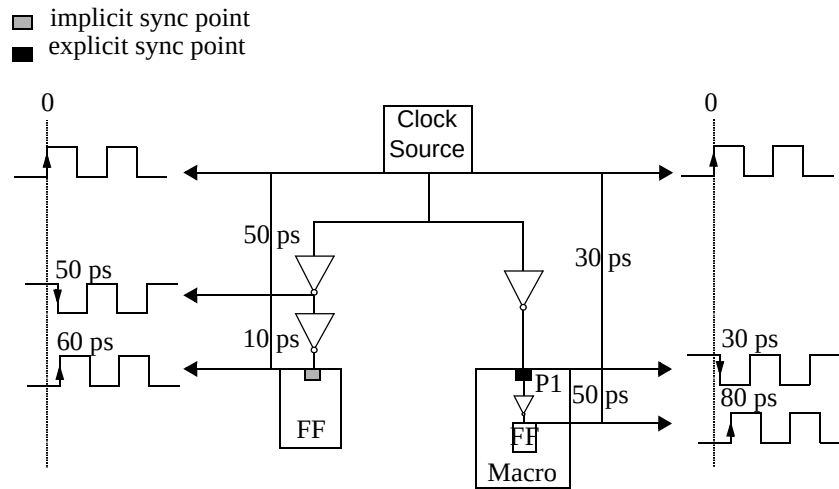
Use this format when the delays between the input to the cell instance and the clock inputs to internal flip-flops are different.

Example

```
dbDefineSyncPort "INST1" ' ("P1" "invertRise" 50))
```

This example instructs Astro to synchronize P1 with all clock input ports and all sync points when the clock signal is rising, adding a phase delay of 50 ps to the clock path delay at the input P1 and inverting the unate state at the input port to the cell instance.

The diagram below shows the case when this pin is compared with other sync points in the *nonInvertRise* category of the clock skew report.



See Also

dbDefineVarRouteRule

Specifies a set of rules for variable-width routing:

- A *width* rule defines the minimum width of objects on a layer.
- A *spacing* rule defines the minimum spacing between objects on a layer.
- A *contact* rule defines the size of array to use with a contact.

After you specify a set of rules, you can assign those rules to one or more nets using [dbAssignVarRouteRule](#).

Tool

Milkyway

Prerequisites

Before you define variable-width routing rules, you might want to delete previously set rules using [dbDeleteAllVarRouteRules](#).

Syntax

Note:

A construction enclosed by braces ({ }) and followed by three horizontal dots (...) can be repeated.

```
dbDefineVarRouteRule cellId "ruleSetName" '({(rule)} ...)
```

where the arguments are as follows:

<i>cellId</i>	ID of the cell for which you want to define variable-width routing rules. Valid values: ID of any cell in the open library. If the cell is open, you can substitute the following for <i>cellId</i> : (<i>geGetEditCell</i>).
<i>ruleSetName</i>	Name you want to assign to the set of rules. When you assign the rules to nets, you reference the set of rules using this name.

rule

Definition of rule to be included in the rule set.

Valid values:

To include a width rule, use width "*layerName*" *minWidth* where *width* is a keyword to indicate that this is a width rule, *layerName* is the name of the layer to which the rule applies, and *minWidth* is the minimum width of objects on the layer. If you omit the width rule, Astro uses the minimum width specified in the technology file.

To include a spacing rule, use spacing "*layerName*" *minSpacing* where *spacing* is a keyword to indicate that this is a spacing rule, *layerName* is the name of the layer to which the rule applies, and *minSpacing* is the minimum spacing between objects on the layer. If you omit the spacing rule, Astro uses the minimum spacing specified in the technology file.

To include a contact rule, use contact "*contactName*" [*numColumns numRows*] where *contact* is a keyword to indicate that this is a contact rule, *contactName* is the name of the contact to which the rule applies, *numColumns* is the number of columns in the array, and *numRows*.

*rule -
cont.*

To Include taper rule, use

taperLevel " " [*numTaperLevel*]

where *taperLevel* is a keyword to indicate this is a taper rule, *numTaperLevel* shows how many different steps to taper down the wire width and contact the default width.

The wire from the output pins will be wider, while the wire to the input pins will be narrower. The wire-width will change at the steiner point.

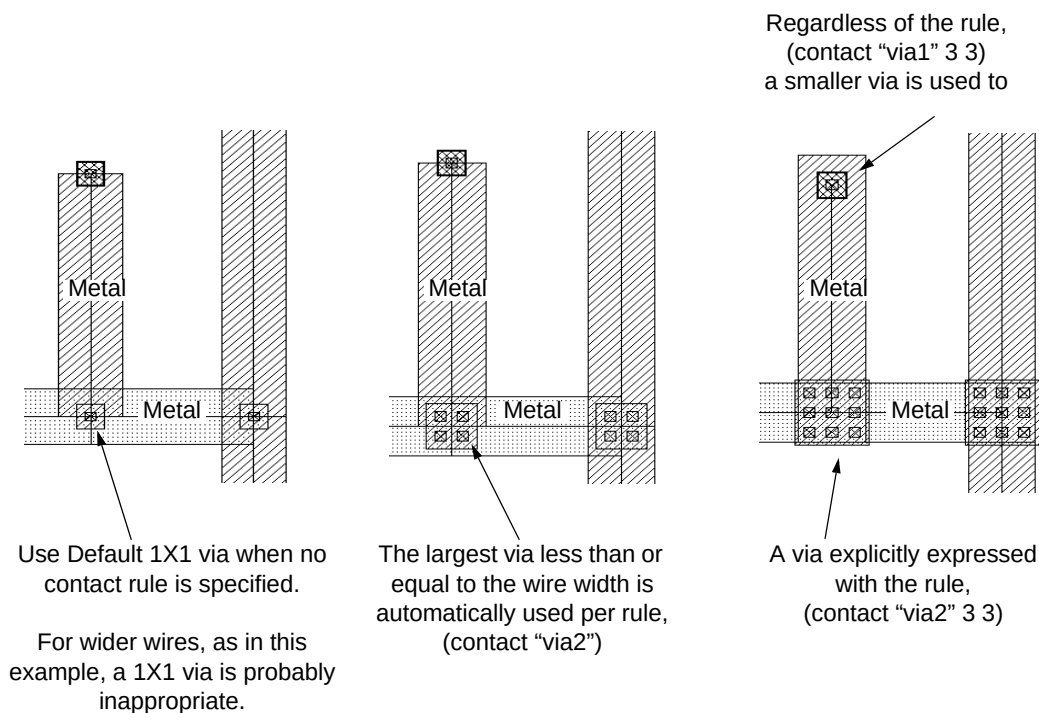
If the single row of a single column contact is specified and the contact size is bigger than the wire width, contact tapering may be applied with a bigger contact-array close to the output pin, and a smaller contact close to the input pin.

is the number of rows in the array. If you omit *numColumns* and *numRows*, Astro determines the array size based on the width rules for the layers. If you omit the contact rule, Astro uses the default via defined in the technology file.

For the *contact* rule, you have the following choices of vias:

- Default 1X1 via.
- Via automatically chosen according to the wire width.
- Via size specified in the *contact* rule.

The result of each of these three methods is shown below.



Examples

```
dbDefineVarRouteRule (geGetEditCell) "VWR1" '((width "m1"
6.0)
(width "m2" 7.0) (width "m3" 10.0) (spacing "m1" 6.5)
(spacing "m2" 6.0) (contact "v12" 2 1) (contact "v23"))
```

This example defines rule set VWR1, which includes width, spacing, and contact rules for variable-width routing on layers m1, m2, and m3. Note that contact v23 has no specification given. Astro creates a contact array equal to or smaller than 7.0 by 10.0 microns to connect m2 and m3 wires.

- To taper wire width

```
(dbDefineVarRouteRule cellId "rule3") ' ((width "M3" 4.0)
  (width "M2" 4.0) (contact "V12") (contact "V23")
  (taperLevel " " 4))
```

- To taper contact with default-width wire.

```
(dbDefineVarRouteRule cellId "rule3") ' ((contact "V12" 2
1)
  (contact "V23" 2 1) (taperLevel " " 4))
```

See Also

Once variable routing rules are defined with *dbDefineVarRouteRule*, nets that are to be routed with the rules are declared with [dbAssignVarRouteRule](#).

dbDefineXtalkConstraint

Can define the crosstalk constraints that can be honored by the router.

In general, the crosstalk is introduced when there are high coupling capacitance between different nets.

The total crosstalk of a net (the 'victim') is the sum of all the crosstalk introduced by all its neighboring nets (the 'aggressor').

The crosstalk constraints will be the maximum value allowed for the victim net.

The crosstalk can be modeled by five different types and the router will handle each of them differently.

Tool

Milkyway

Syntax

```
dbDefineXtalkConstraint cellId conxId type value threshold  
classList
```

where the arguments are as follows:

<i>cellId</i>	ID of the cell for which you want to assign variable-width routing rules. Valid values: ID of any cell in the open library. If the cell is open, you can substitute the following for <i>cellId</i> : (<i>geGetEditCell</i>).
<i>conxId</i>	ID of the crosstalk constraints. Valid values: Integers less than 256.
<i>type</i>	The type of crosstalk constraints. Valid values: "c": coupling capacitance. "n": coupling capacitance * (speed factor). "tn": type "n" add consideration of timing window, for rise and fall. "tn-r": type "n" add consideration of timing window, for rise only. "tn-f": type "n" add consideration of timing window, for fall only.

<i>value/threshold</i>	The crosstalk constraints value and the threshold for filtering out 'safe' nets that won't have crosstalk problems. The constraints is specified as a ratio to a certain net property based on the constraint type. The constraints will be considered met if: $(\text{net property}) < \text{threshold or total crosstalk} < (\text{net property}) * \text{maxRatio}$ The (net property) are computed by the router depending on the type.
<i>classList</i>	The crosstalk class ID and its weight of the aggressors that need to be considered. The default class list is (0 1.0).

Examples

```
(dbDefineXtalkConstraint cellId 2 "tn" 0.40 20.0 '((0 1.0))
```

See Also

[dbClearCellXtalkConstraints](#)
[dbAssignNetXtalkClass](#)
[dbAssignNetXtalkConstraints](#)
[dbDefineXtalkConstraint](#)
[dbDumpXtalkConstraints](#)
[axgSetXtalkRouteOptions](#)

dbDeleteAllVarRouteRules

Deletes all variable-width routing rules in the specified cell.

Tool

Milkyway

Syntax

```
dbDeleteAllVarRouteRules cellId
```


where the argument is:

<i>cellId</i>	ID of the cell for which you want to delete variable-width routing rules. Valid values: ID of any cell in the open library. If the cell is open, you can substitute the following for <i>cellId</i> : (<i>geGetEditCell</i>).
---------------	--

Example

```
dbDeleteAllVarRouteRules (geGetEditCell)
```

This example deletes all variable-width routing rules in the open cell.

dbDeleteCell

Deletes a cell. If `dbDeleteCell` succeeds, the Synopsys tool returns *#t*.

Prerequisite

The cell you want to delete must be closed. In addition, if you want to delete a cell from a reference library, the reference library must be open in write (W) mode.

Syntax

```
dbDeleteCell "cellName"
```

where the argument is as follows:

cellName

Name of the cell you want to delete

Valid values:

Name of any cell in the current library

dbDeleteCellAttachedFile

Deletes an attached file from a cell. If `dbDeleteCellAttachedFile` succeeds, the Synopsys tool returns `#t`.

Prerequisite

The cell from which you want to delete an attached file must be open in write (W) mode.

Syntax

```
dbDeleteCellAttachedFile cellId "pseudoName"
```

where the arguments are as follows:

cellId

Database object ID of the cell from which you want to delete an attached file

Valid values:

- Cell ID of any open cell in the current library
- `(geGetEditCell)`

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

pseudoName

User name of attached file you want to delete

Valid values:

Name of any attached file in the specified library

dbDeleteCellInstFromCellRegion

Deletes a cell instance's association with a cell region. The cell instance does not change its location. Only the relationship indicating that this cell instance is to be placed inside the cell region is removed.

Syntax

```
dbDeleteCellInstFromCellRegion cellId regionId cellInstId
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

regionId

Database object ID of the region to which the cell instance information applies

Valid values:

Region ID of any region in the specified cell

cellInstId

Database object ID of the cell instance to delete

Valid values:

Cell instance ID of any cell instance in the specified region

dbDeleteCellRowDoubleBack

Resets cell row placement orientation.

Note:

Use prior to executing [dbStartCellRowDoubleBack](#).

Syntax

```
dbDeleteCellRowDoubleBack cellId regionNum
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the cell row information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

regionNum

Number of the cell region in which the cell row information applies

Valid values:

Positive integers that identify regions inside the specified cell

dbDeleteFloatingCell

Deletes unconnected standard cell instances.

Syntax

```
dbDeleteFloatingCell cellId cellMasterName powerPinName  
groundPinName
```

Arguments

cellId - The ID of the cell to delete unconnected cell instances.
Integer value

cellMasterName - The name of the cell master of the cell instances
deleted. String value

powerPinName - The name of power pin. String value

groundPinName - The name of ground pin. String value

Example

```
dbDeleteFloatingCell (geGetEditCell) "AOI22X2" "VDD" "VSS"
```

dbDeleteInstAndStitch

Deletes all cell instances in the specified file.

dbDeleteInstAndStitch handles only one-input and one-
output, then stitches all its fanouts to the input net.

Syntax

`dbDeleteInstAndStitch cellId fileName`

where the arguments are as follows:

cellId

Database object ID of the cell from which you want to delete all cell instances in the specified file

Valid values:

- Cell ID of any open cell in the current library
- `(geGetEditCell)`

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

fileName

Name of the file from which you want to delete all cell instances

Valid values:

Any string of as many as 32 characters

dbDeleteLibAttachedFile

Unlinks an attached file from a library. If

`dbDeleteLibAttachedFile` successfully unlinks the attached file from the current library, the tool returns *#t*.

Caution!

In addition to unlinking the file from your library,

`dbDeleteLibAttachedFile` deletes the attached file from the current directory.

Prerequisite

The library from which you want to delete an attached file must be open in write (W) mode.

Syntax

```
dbDeleteLibAttachedFile libId "pseudoName"
```

where the arguments are as follows:

libId

Database object ID of the current library from which you want to delete an attached file

Valid values:

- Library ID
- (dbGetCurrentLibId)

For more information, see “Specifying Libraries” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

pseudoName

User name of attached file you want to delete

Valid values:

Name of any file attached to the current library

See Also

[dbStoreLibAttachedFile](#)

dbDeleteLeqPortGroup

Deletes the current logical-equivalent port information.

Syntax

```
dbDeleteLeqPortGroup cellId
```

where the argument is as follows:

cellId

Database object ID of the cell to which the cell row information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

See Also

[dbDeleteLeqPortGroup](#)

dbDeleteLogicData

Deletes the logic .db file associated with the specified design cell.

Syntax

```
dbDeleteLogicData <string libraryName> <string cellName>  
                [string dbFileName]
```

where the arguments are as follows:

libraryName

Name of the library containing the logic data to delete.

cellName

Name of design cell to which the logic data is associated.

dbFileName

Name of the logic .db file (optional).

Description

The `dbDeleteLogicData` function deletes the logic .db file associated with the specified design cell. If the logic .db file name is also specified, then it is deleted only if it matches the associated logic .db file name.

Example

```
dbDeleteLogicData "myLib" "myDesign" "myDesign.db"
```

dbDeleteLogicView

Deletes all logic .db files from the specified library.

Syntax

```
dbDeleteLogicView <string libraryName>
```

where the argument is as follows:

libraryName

Name of the library containing the logic data to delete.

Description

All logic .db files within the LOGIC view subdirectory are removed. However, the LOGIC subdirectory is not removed.

Example

```
dbDeleteLogicView "myLib"
```

dbDeleteModelData

Deletes the model .db file associated with the specified library's operating condition.

Syntax

```
dbDeleteModelData <string libraryName> <int opCond>  
                [string dbFileName]
```

where the arguments are as follows:

libraryName

Name of the library containing the model data to delete.

cellName

Operating condition to which the model data is associated.

Valid values are:

- 0 minimum
- 1 maximum
- 2 typical
- 3 other

dbFileName

Name of the model .db file (optional).

Description

The `dbDeleteModelData` function deletes the model .db file associated with the specified library's operating condition. If multiple .db files are associated with the operating condition, then only the first one is deleted.

If the model .db file name is specified, then it is deleted only if it matches the one of the associated model .db file names.

Example

```
dbDeleteModelData "myLib" 0 "myLib_min.db"
```

dbDeleteModelView

Deletes all model .db files from the specified library.

Syntax

```
dbDeleteModelView <string libraryName>
```

where the argument is as follows:

libraryName

Name of the library containing the model data to delete.

Description

The `dbDeleteModelView` function deletes all model .db files from the specified library. All model .db files within the LM view subdirectory are removed. The LM subdirectory is not removed.

Example

```
dbDeleteModelView "myLib"
```

dbDeleteNetClass

Deletes a net class from the cell. A net class can consist of nested net classes or individual nets which you specify using [dbFormNetClass](#).

Tool

Milkyway

Syntax

```
dbDeleteNetClass cellId "className"
```

where the arguments are as follows:

<i>cellId</i>	ID of the cell from which you are deleting a net class. Valid values: ID of any cell in the open library. If the cell is open, you can substitute the following for <i>cellId</i> : (geGetEditCell).
<i>className</i>	Name of the net class which you are deleting. To delete all net classes, type "*" for <i>className</i> .

Example

```
dbDeletNetClass (geGetEditCell) "A"
```

This function deletes net class A from the cell.

See Also

[dbFormNetClass](#) creates a net class with individual nets or other net classes you specify as members.

[dbQueryNetClassByName](#) displays the names of the nets in a specified net class.

[dbPrintAllNetClasses](#) displays the names of all net classes in a cell.

dbDeleteObject

Deletes an object from a cell.

Syntax

```
dbDeleteObject cellId objectId
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

objectId

Database object ID of the object to delete

Valid values:

Object ID of any object in the specified cell, such as the following:

- Cell instance ID
- Net ID
- Pin ID
- Port ID
- Property ID

- Region pin ID
- Region port ID

This is only a partial list.

dbDeleteOneLayerRectangle

Deletes all rectangles on a specified layer.

Syntax

```
dbDeleteOneLayerRectangle "libName" layerNumber
```

where the arguments are as follows:

libName

Name of the library to which the layer information applies

Valid values:

Name of any library in the current directory

layerNumber

Number of the layer from which all rectangles are to be deleted

Valid values:

Any positive integer corresponding to a layer in the specified library

dbDeletePcellDefinition

Removes a parameterized cell from the library.

Syntax

```
dbDeletePcellDefinition libId "pcellName"
```

where the arguments are as follows:

libId

Database object ID of the current library from which you want to reference another library

Valid values:

- Library ID
- (dbGetCurrentLibId)

For more information, see “Specifying Libraries” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

pcellName

Name of the parameterized cell (string).

dbDeletePin

Deletes a pin record in a cell. Serves the same function as [dbPurgeObject](#).

Syntax

```
dbDeletePin cellId pinId
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the pin information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

pinId

Database object ID of the pin to delete

Valid values:

Pin ID of any pin in the specified cell

dbDeletePlacementChain

Deletes the specified placement chain group.

Syntax

```
dbDeletePlacementChain<objectId cellId> <string chainName>
```

where the arguments are as follows:

<code>cellId</code>	Database object ID of the cell.
<code>chainName</code>	Name of the placement chain.

Description

The `dbDeletePlacementChain` function deletes the specified placement chain group.

Example

```
dbDeletePlacementChain (geGetEditCell) "chain1"
```

dbDeletePolygonByLayer

Deletes any polygon record whose layer is the same as specified.

Tool

Milkyway

Syntax

```
dbDeletePolygonByLayer cellId layer
```

where the arguments are as follows:

<i>cellId</i>	ID of the cell for which you want to delete a polygon by layer. Valid values: ID of any cell in the open library. If the cell is open, you can substitute the following for <i>cellId</i> : (<i>geGetEditCell</i>).
<i>layer</i>	A layer name or layer number is what has been defined in the technology file of the library. Entering either a name or a number will be accepted.

Example 1

```
dbDeletePolygonByLayer (geGetEditCell) "M2"
```

This example deletes all polygon objects on layer *name* “M2” in the current design cell.

Example 2

```
dbDeletePolygonByLayer (geGetEditCell) "2"
```

This example deletes all polygon objects on layer *number* “2” in the current design cell.

dbDeleteProperty

See [dbDeletePropertyById](#).

dbDeletePropertyById

Deletes a specified property record.

Syntax

```
dbDeletePropertyById cellId propertyId
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

propertyId

Database object ID of the property to delete

Valid values:

Object ID of any property in the specified cell

dbDeletePropertyByName

Deletes a specified property record from a cell.

Syntax

```
dbDeletePropertyByName cellId objectId "propName"
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

objectId

Database object ID of the object to which the information applies

Valid values:

Object ID of any object in the specified cell, such as the following:

- Cell instance ID
- Net ID
- Pin ID
- Port ID
- Region pin ID

- Region port ID

This is only a partial list.

propName

Name of the property record to delete

Valid values:

Name of any property in the specified cell

dbDeleteRectangleByLayer

Deletes any rectangle record whose layer is the same as specified.

Tool

Milkyway

Syntax

```
dbDeleteRectangleByLayer cellId layer
```

where the arguments are as follows:

<i>cellId</i>	ID of the cell for which you want to delete a rectangle by layer. Valid values: ID of any cell in the open library. If the cell is open, you can substitute the following for <i>cellId</i> : (<i>geGetEditCell</i>).
<i>layer</i>	A layer name or layer number is what has been defined in the technology file of the library. Entering either a name or a number will be accepted.

Example 1

```
dbDeleteRectangleByLayer (geGetEditCell) "M2 "
```

This example deletes all rectangle objects on layer *name* "M2" in the current design cell.

Example 2

```
dbDeleteRectangleByLayer (geGetEditCell) "2"
```

This example deletes all rectangle objects on layer *number* "2" in the current design cell.

dbDeleteRefLibByName

Deletes a reference library from a library. If `dbDeleteRefLibByName` successfully unlinks the reference library from the current library, the Synopsys tool returns *#t*.

Note:

The `dbDeleteRefLibByName` function does not delete the reference library from your directory. However, any cell within the main library that references a cell in the deleted library will be undefined.

Prerequisite

The library from which you want to delete a reference library must be open in write (W) mode.

Syntax

```
dbDeleteRefLibByName libId "refLibName"
```

where the arguments are as follows:

libId

Database object ID of the current library from which you want to unlink a reference library

Valid values:

- Library ID
- (dbGetCurrentLibId)

For more information, see “Specifying Libraries” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

refLibName

Name of the library you want to unlink from the current library

Valid values:

Name of any reference library in the specified library

See Also

[dbReplaceRefLibByName](#)

dbDeleteScanChain

Deletes a scan chain in the open cell.

Tool

Milkyway

Syntax

```
dbDeleteScanChain (geGetEditCell) chainName
```

where the argument is as follows:

chainName The name of the scan chain you want to delete.

dbDeleteTieHiLoCell

Deletes tie-hi/lo cell instances.

Syntax

```
dbDeleteTieHiLoCell cellId cellMasterName pgNetName purgeCI
```

Arguments

cellId - The ID of the cell to delete tie-hi/lo cell instances. Integer value

cellMasterName - The name of the cell master of the tie-hi/lo cell instances deleted. String value

pgNetName - The name of pgNet. String value

purgeCI - Purge tie-hi/lo cell instances or not. Boolean value

Example

```
dbDeleteTieHiLoCell (geGetEditCell) "AOI22X2" "VDD" #t
```

dbDisconnect

Disconnects an object from a net. Use [dbDisconnectPortInstByName](#) to erase connections of port instances to nets that are not members of the net. Generally, power and ground connections are not members of the net.

Note:

The netlist is changed, and no engineering change order (ECO) is recorded. Be careful when using this function.

Syntax

```
dbDisconnect cellId netId memberId
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the net information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

netId

Database object ID of the net to which the information applies

Valid values:

Net ID of any net in the specified cell

memberId

Database object ID of the net member to disconnect

Valid values:

Object ID of any object connected to the specified net

dbDisconnectPortInstByName

Disconnects power or signal connections from port instances and associated pin instances. This function can be used to disconnect power and ground connections that are not members of the net.

Important:

This function changes the netlist. No ECO is recorded.

Syntax

```
dbDisconnectPortInstByName cellId "netName" "cellInstName"  
"portInstName" "topCellName"
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the port instance information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

netName

Name of the net to which the port instance information applies

Valid values:

Name of any net in the specified cell

cellInstName

Name of the cell instance to which the port instance information applies

Valid values:

Name of any cell instance in the specified cell

portInstName

Name of the port instance to which the information applies

Valid values:

Name of any port instance associated with the specified cell instance

topCellName

Name of the top-level design cell

dbDisJoinPort

Disjoins two ports.

Syntax

```
dbDisJoinPort cellName port1Name port2Name
```

Arguments

where the arguments are as follows:

<i>cellName</i>	Name of the cell to disjoin two ports. String value
-----------------	---

<code>port1Name</code>	Name of the first port being disjointed. String value
<code>port2Name</code>	Name of the other port being disjointed. String value

Example

```
dbDisJoinPort "top" "Z" "A"
```

dbDisplayCellRailProperty

Displays the y-coordinates (in database units) of the edges of the power and ground rails in a standard cell from the bottom up. The standard cell should be opened in FRAM view.

Syntax

```
dbDisplayCellRailProperty cellId
```

where the argument is as follows:

cellId

Database object ID of the cell to which the y-coordinate applies

Valid values:

- Cell ID of any open cell in the current library
- `(geGetEditCell)`

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

See Also

[dbUpdateCellRailProperty](#)

dbDisplayPlanGroup

Controls the display of floorplan groups. To see the result, redraw the window using `geRedraw`.

Tool

Milkyway

Syntax

```
dbDisplayPlanGroup cellId display?
```

where the arguments are as follows:

<i>cellId</i>	ID of the cell for which you want to control the display of floorplan groups. Valid values: ID of any cell in the open library. If the cell is open, you can substitute the following for <i>cellId</i> : <code>(geGetEditCell)</code> .
<i>display?</i>	Whether or not to display floorplan groups. Valid values: <code>#t</code> means to display floorplan groups. <code>#f</code> means to suppress the display of floorplan groups.

Example

```
dbDisplayPlanGroup (geGetEditCell) #f
```

This function suppresses the display of floorplan groups in the open cell.

dbDowngradeLib

Makes a library in the current Synopsys tool release format revert back to the previous release format. If `dbDowngradeLib` successfully makes your library revert, the Synopsys tool displays *#t*.

Prerequisite

All libraries must be closed.

Syntax

```
dbDowngradeLib "libName"
```

where *libName* is the name of the library you want to revert back to the previous Synopsys tool release format. Valid values are the names of any library in the current directory.

For more information, see “Working With Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

dbDumpAllCriticalNets

Writes critical nets to a file. Critical nets are nets with a criticality from one (less critical) to seven (most critical). You set net criticality using [dbSetNetCriticality](#). After you write the critical nets to a file, you can edit the file and reload the edited net criticalities using the `load` command.

Tool

Milkyway

Syntax

```
dbDumpAllCriticalNets cellId "fileName"
```

where the arguments are as follows:

<i>cellId</i>	ID of the cell for which you want to write critical nets to a file. Valid values: ID of any cell in the open library. If the cell is open, you can substitute the following for <i>cellId</i> : (geGetEditCell).
<i>fileName</i>	Name of the file to which you want to write the critical nets.

See Also

[dbSetNetCriticality](#) lets you set a net criticality, which determines the probability that the router will detour a net.

dbDumpAllNetLengthRouteRuleFromTable

Dumps the net route rules or constraints assigned by the length based tables.

Tool

Milkyway

Syntax

```
dbDumpAllNetLengthRouteRuleTable cellId "fileName"
```

where the arguments are as follows:

<i>cellId</i>	ID of an opened cell. (geGetEditCell).
<i>fileName</i>	Name of the file for dumped rules and constraints.

Example

```
dbDumpAllNetLengthRouteRuleFromTable (geGetEditCell)  
"RouteRule.dump"
```

dbDumpAllNetMinMaxLayer

Dumps all cell layer constraints for the file “*filename*”, that have been previously set by [dbSetCellMinMaxLayer](#), or overwritten by [dbAssignNetMinMaxLayer](#).

Tool

Milkyway

Prerequisites

Constraints set by [dbSetCellMinMaxLayer](#) or overwritten by [dbAssignNetMinMaxLayer](#).

Syntax

```
dbDumpAllNetMinMaxLayer cellId "fileName"
```

where the arguments are as follows:

<i>cellId</i>	ID of the design cell for which you want to dump cell layer constraints. If the cell is open, you can substitute the following for <i>cellId</i> : (geGetEditCell).
<i>fileName</i>	Name of the file to which you want to dump the cell layer constraints.

dbDumpAllNetTimingSpacing

Writes the names of nets with timing spacing rules to a file.

Tool

Milkyway

Prerequisites

Open the cell containing the nets.

Syntax

```
dbDumpAllNetTimingSpacing cellId "fileName"
```

where the arguments are as follows:

<i>cellId</i>	ID of the design cell for which you want to write the names of nets to the file. If the cell is open, you can substitute the following for <i>cellId</i> : (geGetEditCell).
---------------	---

<i>fileName</i>	Name of the file to which you want to write the net names.
-----------------	--

Example

```
dbDumpAllNetTimingSpacing (geGetEditCell) "dd"
```

This example writes the names of nets containing timing-driven spacing rules in the open cell to file *dd*.

dbDumpAllNetTopLayerProbe

Writes all of the top layer probe constraints of the cell to a file.

Tool

Milkyway

Prerequisites

Open the cell containing the nets.

Syntax

```
dbDumpAllNetTopLayerProbe cellId "fileName"
```

where the arguments are as follows:

<i>cellId</i>	ID of the design cell for which you want to write the top layer probe constraints to the file. If the cell is open, you can substitute the following for <i>cellId</i> : (geGetEditCell).
---------------	---

<i>fileName</i>	Name of the file to which you want to write all of the top layer probe constraints of the open cell.
-----------------	--

Example

```
dbDumpAllNetTopLayerProbe (geGetEditCell) "probe.list"
```

This example writes all of the top layer probe constraints of the open cell to file *probe.list*.

See Also

[axgSetHPORouteOptions](#)

[dbAssignNetTopLayerProbe](#)

[dbResetAllNetTopLayerProbe](#)

dbDumpAllVarRouteRules

Writes variable-width routing rules to a file.

Note:

After you write this information to a file, you can delete the information from the cell (see [dbDeleteAllVarRouteRules](#)), edit the output file, and reload the edited information using `listFeature`.

Tool

Milkyway

Syntax

```
dbDumpAllVarRouteRules cellId "fileName"
```

where the arguments are as follows:

<i>cellId</i>	ID of the cell for which you want to write variable-width routing rules to a file. Valid values: ID of any cell in the open library. If the cell is open, you can substitute the following for <i>cellId</i> : (geGetEditCell).
<i>fileName</i>	Name of the file to which you want to write the rules.

Example

```
dbDumpAllVarRouteRules (geGetEditCell) "vwr"
```

This example writes variable-width routing rules in the open cell to file *vwr*.

dbDumpAllVTFillers

Writes all filler cells and their voltage threshold type filling information in the specified top cell to an output file.

The output file includes a section that lists the voltage threshold rules that are defined in the open design view. The voltage threshold rules from each reference library will continue to be listed.

Tool

Milkyway

Syntax

```
dbDumpAllVTFillers topCellId fileName
```

where the arguments are as follows:

topCellId The database object ID of the top cell.

fileName The name of the output file.

Example

```
dbDumpAllVTFillers (geGetEditCell) "vtFiller.dump"
```

See Also

[dbClearVTFiller](#)

[dbDumpVTFiller](#)

[dbSpecifyVTFiller](#)

dbDumpCell

Dumps the cell contents to a file.

Syntax

```
dbDumpCell cellId "fileName"
```

where the arguments are as follows:

cellId

Database object ID of the cell from which you want information

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Working With Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

fileName

Name of the file to which the cell contents is to be dumped

Valid values:

Any text string

dbDumpCellAttachedFile

This function dumps the specified attached file to the dump file.

Syntax

```
dbDumpCellAttachedFile cellId pseudoName dumpFileName
```

Arguments

cellId - ID of the cell to dump out the specified attached file. Integer value

pseudoName - Name of cell attached file to write out. String value

dumpFileName - Name of the dump file written out. String value

Example

```
dbDumpCellAttachedFile (geGetEditCell) "RGB:1_2" "outFile"
```

dbDumpCellDifferentialGroup

Dumps all the definitions of the differential groups of the design.

Syntax

```
dbDumpCellDifferentialGroup cellId fileName
```

where the arguments are as follows:

cellId

Cell ID to which nets of the groups belong

fileName

Name of the file to which you want to dump the differential groups and their nets

Example

The following dumps out all the differential groups and their nets to output.file:

```
dbDumpCellDifferentialGroup (geGetEditCell) "output.file"
```

dbDumpCellPlacement

Dumps the cell placement information to a file.

Syntax

```
dbDumpCellPlacement cellId "cellType" "fileName"
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

cellType

Type of cells for which placement information is dumped

Valid values:

- *macro*, which means that placement information for macro cells is dumped
- *standard*, which means that placement information for standard cells is dumped
- *pad*, which means that placement information for pad cells is dumped
- *cornerPad*, which that placement information for corner pad cells is dumped

fileName

Name of the file to which the cell placement information is dumped

Valid values:
Any text string

dbDumpCellVTType

Writes the voltage threshold type information of the named master cell in the top cell to an output file. This command iterates every cell in the specified library; if the cell's CEL or FRAM view is instantiated in the specified top cell, the cell's voltage threshold type information will be written out.

Tool

Milkyway

Syntax

```
dbDumpCellVTType topCellId cellMasterName fileName
```

where the arguments are as follows:

<i>topCellId</i>	The database object ID of the top cell.
<i>cellMasterName</i>	The name of the cell master.
<i>fileName</i>	The name of the output file.

Example

```
dbDumpCellVTType (geGetEditCell) "AND3" "AND3.vtType"
```

See Also

[dbClearCellVTType](#)

[dbSetCellVTType](#)

dbDumpCheckNetlistVsFram

Checks the correspondence between netlist (*.netl*) cells and pin/blockage (*.fram*) cells in a specified library, and dumps the result to a file.

- For every netlist cell, it checks to make sure that there is a pin/blockage cell.
- If a pin/blockage cell exists, it checks to make sure that for every port in the netlist cell there is a corresponding port of the same name in the pin/blockage cell.

Tool

Milkyway

Syntax

```
dbDumpCheckNetlistVsFram libName fileName  
[netlistViewName excludeLibraryList]
```

where the arguments are as follows:

<i>libName</i>	Name of the library you want to check. Valid values: Name of any library in the current directory.
<i>fileName</i>	Name of the file you want to dump to.
<i>netlistViewName</i>	Name of any other cell view name you want to check. By default, this command checks netlist cells with the view name ".NETL". Valid values: Name of any cell view in the library. Note: This argument is optional.

excludeLibraryList List of the name of all reference libraries you don't want to check. By default, all reference libraries of libName will be checked.

Note:

This argument is optional.

Example

This example checks netlist cells in the library *DESIGNS* and it's reference libraries, with the cell view name ".Symb".

```
dbDumpCheckNetlistVsFram "DESIGNS" ".Symb"
```

See Also

[dbCheckNetlistVsFram](#)

dbDumpChildParentInfo

Dumps each cell referenced in the top design and a listing of its parent cells to a text file.

Syntax

```
dbDumpChildParentInfo <string libName> <string cellName>  
<string fileName>
```

libName - Name of the design library containing the top design cell

cellName - Name of the top design cell

fileName - Name of output file where the information will be dumped

Description

The dbDumpChildParentInfo function dumps to a text file each cell referenced in the top design along with a listing of its parent cells.

Each cell is a child of the cell in which it is instantiated. The cell in which it is instantiated is called the parent cell. Each cell may be instantiated in one or more cells, and therefore may have multiple parents.

All cells in the design hierarchy (at all depths) are traversed. This means, for any particular cell, its CEL view is always opened and examined if one is found.

The output file lists the child cells in alphabetical order. Under each child cell is a list of parent cells and their libraries. The format follows:

CHILD	cellName.cellView;version (library path and name)
PARENT	cellName.cellView;version (library path and name)
PARENT	cellName.cellView;version (library path and name)
	.
	.
	.
PARENT	cellName.cellView;version (library path and name)

Example

```
dbDumpChildParentInfo "mylib" "mydesign" "childparent.txt"
```

The above function call dumps the following type of information:

```

CHILDAND.CEL;2 (/home/std_lib1)
PARENTMACRO1.CEL;3 (/home/macro_lib1)
PARENTMACRO2.CEL;1 (/home/macro_lib2)

CHILDAND.CEL;5 (/home/std_lib2)
PARENTMACRO1.CEL;2 (/home/macro_lib4)
PARENTMACRO2.CEL;2 (/home/macro_lib5)

CHILDOR.CEL;1 (/home/std_lib1)
PARENTMACRO1.CEL;3 (/home/macro_lib1)
PARENTMACRO2.CEL;3 (/home/macro_lib3)

CHILDMACRO1.CEL;2 (/home/macro_lib4)
PARENTmydesign.CEL;1 (/home/mylib)

CHILDMACRO1.CEL;3 (/home/macro_lib1)
PARENTmydesign.CEL;1 (/home/mylib)

CHILDMACRO2.CEL;1 (/home/macro_lib2)
PARENTmydesign.CEL;1 (/home/mylib)

CHILDMACRO2.CEL;2 (/home/macro_lib5)
PARENTmydesign.CEL;1 (/home/mylib)

CHILDMACRO2.CEL;3 (/home/macro_lib3)
PARENTmydesign.CEL;1 (/home/mylib)

```

dbDumpCTSPin

Writes a file containing information about CTS input ports during clock tree synthesis. You specify this information using [dbDefineCTSPin](#).

Tool

Milkyway

Syntax

```
dbDumpCTSPin cellId "fileName"
```

where the arguments are as follows:

<i>cellId</i>	ID of the top-level cell containing the cell instance for which you want information. Valid values: ID of any top-level cell in the current directory.
<i>fileName</i>	Name of the file to which you want the information written. Valid values: Any string up to 32 characters.

Example

```
dbDumpCTSPin (geGetEditCell) "CTSInfo"
```

This example writes CTS information for the open cell to the file *CTSInfo*.

See Also

[dbDefineCTSPin](#) identifies an input port on a cell instance as a CTS pin.

[dbPurgeCTSPin](#) deletes the CTS specific information defined on all ports of the cell instance.

dbDumpFootPrintEQ

Will dump all footprint EQ defined by `defineFootPrintEQCell` to a file.

Tool

Milkyway

Syntax

```
dbDumpFootPrintEQ <fileName>
```

where the argument is as follows:

<i>fileName</i>	Name of the file to which you want to dump footprint EQ information.
-----------------	--

dbDumpFreezeRoutingNets

Dumps a list of frozen nets to the file. The dumped file can be modified manually and loaded back again.

Syntax

```
dbDumpFreezeRoutingNets cellId option "dumpFileName"
```

where the arguments are as follows:

cellId

Database object ID of the cell from which you want to delete an attached file

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

option

Boolean where

- *#t* creates a dump list of frozen nets to a file.
- *#f* creates a dump list of nets that can be changed freely.

dumpFileName

Name of the file to which the cell contents is to be dumped

Valid values:

Any text string

Example 1

The following code dumps a list of frozen nets to the file freeze.list:

```
dbDumpFreezeRoutingNets (geGetEditCell) #t "freeze.list"
```

Example 2

The following code dumps a list of nets that are not frozen to the file routable.list:

```
dbDumpFreezeRoutingNets (geGetEditCell) #f "routable.list"
```

dbDumpGPortTable

Writes the library global port information to a file. To specify port information, use [dbSetCellPortTypes](#).

Syntax

```
dbDumpGPortTable "libName" "fileName"
```

where the arguments are as follows:

libName

Name of the library from which you want to write the global port information

Valid values:

Name of any library in the current directory

fileName

Name of the file to which you want to write the information

Valid values:

Any text string

dbDumpHierRefLibControl

Dumps the reference library structure of all libraries in the library hierarchy into a single control file.

Syntax

```
dbDumpHierRefLibControl <string libName> <string fileName>
```

libName - Name of the top library

fileName - Name of the dumped reference library control file

Description

The dbDumpHierRefLibControl function dumps the reference library structure of all libraries in the library hierarchy into a single control file.

Starting from the top library and going down the hierarchy, all reference library structures are added to an output control file. Because higher-level libraries can define reference structures of lower-level libraries, any redefined reference library structures in lower-level libraries are not included in the output control file.

Similar to the `dbDumpRefLibControl` function, this function takes the reference information from the internally stored reference control file if it is found.

For more information about the reference control file structure see `dbSetRefLibControl`, `dbClearRefLibControl`, `dbDumpRefLibControl`, and `dbCheckRefLibControl`.

dbDumplgnorePin

Writes a file containing information about ignore input ports during clock tree synthesis. You specify this information using [dbDefineIgnorePin](#).

Tool

Milkyway

Syntax

```
dbDumpIgnorePin cellId "fileName"
```


where the arguments are as follows:

cellId	ID of the top-level cell containing the cell instance for which you want information. Valid values: ID of any top-level cell in the current directory.
fileName	Name of the file to which you want the information written. Valid values: Any string up to 32 characters.

Example

```
dbDumpIgnorePin (geGetEditCell) "IgnoreInfo"
```

This example writes ignore information for the open cell to the file *IgnoreInfo*.

See Also

[dbDefineIgnorePin](#) identifies an input port on a cell instance as an ignore pin.

[dbPurgeIgnorePin](#) changes input ports previously identified as ignore pins to default ports.

dbDumpLengthRouteRuleTable

Dumps the length based tables for variable-width routing rules and min/max layer constraints.

Tool

Milkyway

Syntax

```
dbDumpLengthRouteRuleTable cellId "filename"
```

where the arguments are as follows:

cellId ID of an opened cell.
(geGetEditCell).

filename Name of the file for dumped tables.

Example

```
dbDumpLengthRouteRuleTable (geGetEditCell)  
"routeRuleTable.dump"
```

dbDumpLeqPortGroup

Writes the logically equivalent port information defined in the database for a given library. To specify logically equivalent port information, use [dbDefineLeqPortGroup](#).

Tool

Milkyway

Syntax

```
dbDumpLeqPortGroup "libId" "fileName"
```

where the arguments are as follows:

libId Object ID of the library from which you want to write the logically equivalent port information.

fileName Name of the file to which you want the information written.
Valid values:
Any text string up to 32 characters.

Example

```
where access05 is the library:  
setq mylib (dbopenlib "access05")  
dbDumpLeqPortGroup mylib "access05.leqPort"
```

dbDumpLeqPortGroupCandidates

Reports logic equivalent port group information defined in the database for a given library. To specify logically equivalent port information, use [dbDefineLeqPortGroup](#).

Tool

Milkyway

Syntax

```
dbDumpLeqPortGroupCandidates "libId" "fileName"
```

where the arguments are as follows:

libId Object ID of the library from which you want to write the logically equivalent port information.
Valid values:
Valid Object ID.

fileName Name of the file to which you want the information written.
Valid values:
Any text string up to 32 characters.

Example

```
where access05 is the library:  
setq mylib (dbopenlib "access05")  
dbDumpLeqPortGroupCandidates mylib "access05.leqPort"
```

dbDumpLibAntennaRules

Dumps all of the antenna rules stored in the library to a file.

Tool

Milkyway

Syntax

```
dbDumpLibAntennaRules "libName" "fileName"
```

where the arguments are as follows:

libName The name of a valid Milkyway library.

fileName The name of a specified dump file.

See Also

[dbAddAntennaLayerRule](#) defines an advanced antenna rule for the specified layer and stores it in the library.

[dbDefineAntennaRule](#) defines an advanced antenna rule for the specified mode and stores it in the library.

[dbClearLibAntennaRules](#) deletes all of the antenna rules stored in the library

dbDumpLibAttachedFile

Writes a file that is currently attached to the library to the specified file name.

Syntax

```
dbDumpLibAttachedFile <objectId libId> <string pseudoName>  
<string dumpFileName>
```

libId - Database object ID of the library which has the attached file

pseudoName - The name of the attachment (as defined when the file was originally attached)

dumpFileName - Name of the output file

Description

The dbDumpLibAttachedFile function writes a file that is currently attached to the library to the specified file name. The attached file should be previously attached to the library with the specified pseudoName. The format and content of the attached file is the responsibility of the user to define and maintain.

See Also

dbStoreLibAttachedFile
dbGetLibAttachedFile
dbDeleteLibAttachedFile

dbDumpLibCatalog

Dumps a list of all cell names, views, and versions stored in the library to a text file.

Syntax

```
dbDumpLibCatalog <string libName> <string dumpFileName>
```

libName - Name of the library

dumpFileName - Name of the output file

Description

The dbDumpLibCatalog dumps a list of all cell names, views, and versions stored in the library to a text file. If a cell is locked, the user name of the lock owner is printed next to the cell name. The list is ordered alphabetically.

The output file follows this format:

cellName.viewName;versionNumber (locked by *userName*)

.
. .
.

cellName.viewName;versionNumber (locked by *userName*)

Note: The string, (locked by *userName*) does not appear if the cell is not locked.

Example

```
dbDumpLibCatalog "myLib" "myLib.catalog"
```

The above function call generates the following file information:

```
M1.CEL;1(locked by johndoe)
M1.CEL;2
M1.NETL;1
N1.NETL;1
O1.CEL;1(locked by janedoe)
O1.NETL;1
O1.NETL;2
O1.NETL;3
O2.NETL;2
```

dbDumpLibraryCellsVTType

Writes the voltage threshold type information of each cell in the named cell library to an output file. This command iterates every cell in the specified library; if the cell's CEL or FRAM view is instantiated in the specified top cell, the cell's voltage threshold type information will be written out.

Tool

Milkyway

Syntax

```
dbDumpLibraryCellsVTType topCellId libraryName fileName
```

where the arguments are as follows:

<i>topCellId</i>	The database object ID of the top cell.
<i>libraryName</i>	The name of the library that contains the cells.

fileName

The name of the output file.

Example

```
dbDumpLibraryCellsVTType (geGetEditCell) "std_lib"  
    "std_lib.vtType"
```

See Also

[dbClearLibraryCellsVTType](#)

[dbSetLibraryCellsVTType](#)

dbDumpLRFillerRule

Writes the left-and-right filler rules that you defined with the `dbSpecifyLRFiller` command to an output file. These rules define the filler cells to be used for filling to the left and right of certain standard cells during the filler insertion process.

Tool

Milkyway

Syntax

```
dbDumpLRFillerRule topCellId fileName
```

where the arguments are as follows:

topCellId

The database object ID of the top cell.

fileName

The name of the output file.

Example

```
dbDumpLRFillerRule (geGetEditCell) "LRrulefile"
```


See Also

[dbClearLRFillerRule](#)

[dbSpecifyLRFiller](#)

dbDumpNetXtalkProp

Dumps all net crosstalk properties in the specified cell.

Syntax

```
dbDumpNetXtalkProp <objectId cellId> <string fileName>
```

where the arguments are as follows:

`cellId`

Database object ID of the cell.

`fileName`

Name of the output file.

Description

The `dbDumpNetXtalkProp` function dumps all net crosstalk properties in the specified cell. The information is written to the specified output file using the Scheme language format. The output file may be reloaded as a Scheme file.

Example

```
dbDumpNetXtalkProp (geGetEditCell) "xtalkProp.scm"
```

dbDumpPlacementChains

Dumps placement chains to a dump file.

Syntax

```
dbDumpPlacementChains cellId fileName
```

Arguments

cellId - The ID of the cell to dump out placement chains. Integer value

fileName - Name of dump output file. String value

Example

```
dbDumpPlacementChains (geGetEditCell) "outFile"
```

dbDumpPropertyTable

Dumps the run-time property definition table used for byte-swapping.

Syntax

```
dbDumpPropertyTable <void>
```

Description

The `dbDumpPropertyTable` function dumps the run-time property definition table used for byte-swapping. This is primarily a debugging and checking utility. Information is written to the application's input/output window.

Example

```
dbDumpPropertyTable
```

dbDumpRefLibControl

Outputs current hierarchical reference library structure to a file called reference library control file.

Tool

Milkyway

Syntax

```
dbDumpRefLibControl libName fileName
```

where the arguments are as follows:

<i>libName</i>	The name of the library that the hierarchical reference library structure will be dumped from.
----------------	--

<i>fileName</i>	The file name that you want to write the hierarchical reference library structure to.
-----------------	---

Example

```
dbDumpRefLibControl "A" "dump.ref"
```

This function dumps the hierarchical reference library structure of library “A” into a file named “*dump.ref*”. The format of the file would be:

```
LIBRARY A
REFERENCE B
REFERENCE C
REFERENCE D
LIBRARY B
REFERENCE E
LIBRARY C
REFERENCE F
```

where A to D are full path names of the libraries. A has reference libraries B, C, and D. B has reference library E, and C has reference Library F.

See Also

[dbSetRefLibControl](#)

[dbClearRefLibControl](#)

[dbCheckRefLibControl](#)

dbDumpScanChain

Writes scan chain information for the open cell to a file.

Tool

Milkyway

Syntax

```
dbDumpScanChain (geGetEditCell) chainName
```

where the argument is as follows:

<i>chainName</i>	The name of the scan chain you want to dump.
------------------	--

dbDumpScanPortPairs

This command allows you to dump the marked scan port pairs for a particular cell master (marked using the [dbMarkScanPortPairs](#) command).

Tool

Milkyway

Syntax

```
dbDumpScanPortPairs (geGetEditCell) cellMasterName fileName
```

where the arguments are as follows:

cellMasterName The name of the cell master for which you would like to dump the marked scan port pairs.

fileName The name of the file to which you would like to write the scan port pair information,

Example

```
dbDumpScanPortPairs (geGetEditCell) "macrocell"  
"scanportpairs.dump"
```

dbDumpSingleViewPageNumber

Dumps the database page number of the specified cell in the current opened library with the specified view name.

Syntax

```
dbDumpSingleViewPageNumber cellId "viewName"
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- `(geGetEditCell)`

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

viewName

Name of the view from which cell contents are dumped

Valid values:

Name of any view associated with the specified cell, such as the following:

- CEL
- FRAM
- NETL
- EXP

dbDumpSyncPin

Writes a file containing information about input ports to be synchronized during clock tree synthesis. You specify this information using [dbDefineSyncPin](#).

Tool

Milkyway

Syntax

```
dbDumpSyncPin cellId fileName
```

where the arguments are as follows:

<i>cellId</i>	ID of the top-level cell containing the cell instance for which you want information. Valid values: ID of any top-level cell in the current directory.
<i>fileName</i>	Name of the file to which you want the information written. Valid values: Any string up to 32 characters

Example

```
dbDumpSyncPin (geGetEditCell) "syncInfo"
```

This example writes sync point information for the open cell to the file *syncInfo*.

See Also

[dbDefineSyncPin](#) identifies an input port on a cell instance as a sync point and specifies minimum and maximum phase delays for this port at different phases.

[dbPurgeSyncPin](#) changes input ports previously identified as sync points to nonsynchronous ports.

dbDumpTechFile

Creates a technology file you can edit.

Syntax

```
dbDumpTechFile libId "outFile" dumpAllFlag
```

where the arguments are as follows:

libId

Database object ID of the library to which the technology file information applies

Valid values:

Library ID of any library in the current directory

outFile

Name of the technology file to create

Valid values:

Any text string with a file extension of *.tf*

dumpAllFlag

Whether or not all technology information is dumped

Valid values:

- *#t*, which means that all technology information—including predefined color, stipple, and so forth—is dumped
- *#f*, which means that only user-defined information is dumped

Example

```
dbDumpTechFile lib1 "train2out" #t
#t
```

This code dumps the technology information of the library *lib1* to an editable ASCII file.

See Also

[dbReplaceTechFile](#)

dbDumpViewPageNumber

Dumps the database page number of every cell in the current opened library with the specified view name.

Syntax

```
dbDumpViewPageNumber "viewName"
```

where the argument is as follows:

viewName

Name of the view from which you want the database page number of each cell. Can be the name of any view in the current library.

Valid values:

- CEL
- FRAM
- NETL
- EXP

dbDumpViewRecordNumber

Dumps the database record number of every cell in the current opened library with the specified view name.

Syntax

```
dbDumpViewRecordNumber "viewName"
```

where the argument is as follows:

viewName

Name of the view from which you want the database record of each cell. Can be the name of any view in the current library.

Valid values:

- CEL
- FRAM
- NETL
- EXP

dbDumpVTFiller

Writes the filler cell list stored for the two specified voltage threshold types in the specified cell to an output file. The output file can be reloaded.

Tool

Milkyway

Syntax

```
dbDumpVTFiller cellId vtTypeA vtTypeB fileName
```

where the arguments are as follows:

cellId The database object ID of the cell.

<i>vtTypeA</i>	The first of two voltage threshold types for which the list of filler cells is to be output.
<i>vtTypeB</i>	The second of two voltage threshold types for which the list of filler cells is to be output.
<i>fileName</i>	The name of the output file.

Example

```
dbDumpVTFiller (geGetEditCell) "vtType1" "vtType2"
"vtFiller.dump"
```

See Also

[dbClearVTFiller](#)

[dbDumpAllVTFillers](#)

[dbSpecifyVTFiller](#)

dbDumpXtalkConstraints

You can dump crosstalk constraints to a file.

Tool

Milkyway

Syntax

```
(dbDumpXtalkConstraints cellId "fileName" dumpNet)
```

where the arguments are as follows:

<i>cellId</i>	ID of the cell for which you want to dump crosstalk constraints. Valid values: ID of any cell in the open library. If the cell is open, you can substitute the following for <i>cellId</i> : (<i>geGetEditCell</i>).
<i>fileName</i>	Name of the file to which you want the information written. Valid values: Any string up to 32 characters.
<i>dumpNet</i>	#f: dump constraint definition only #t: also dump net crosstalk class and crosstalk constraints

Example

```
(dbDumpXtalkConstraints cellId "foo" #t)
```

See Also

[dbClearCellXtalkConstraints](#)
[dbDefineXtalkConstraint](#)
[dbDefineAbsXtalkConstraint](#)
[dbClearCellXtalkLengthWeight](#)
[dbSetXtalkLengthWeight](#)
[dbAssignNetXtalkClass](#)
[dbAssignNetXtalkConstraints](#)
[dbAllowToAddPGIOText](#)
[axgSetXtalkRouteOptions](#)

dbEliminatePins

Removes all the specified pins from the cell. Use this command before [auExtractBlockagePinVia](#).

Syntax

```
dbEliminatePins "libName" "cellName" "pinNameFile"
```

where the arguments are as follows:

libName

Name of the library

Valid values:

Name of any library in the current directory

cellName

Name of the cell from which the pins are to be deleted

Valid values:

Name of any cell in the current library

pinNameFile

Name of the file containing all the pin names to be deleted

Example

```
dbEliminatePins "designLib" "ram32" "pinlist"
```

dbExistsCellAttachedFile

Checks if a cell has an attached file.

Syntax

```
dbExistsCellAttachedFile cellId pseudoName
```

cellId - ID of the cell to be checked. Integer value

pseudoName - Name of the specified file to be checked. String value

Description

This function returns true (#t) if the specified file is an attached file of the cell. Otherwise it returns false (#f).

Example

```
dbExistsCellAttachedFile (geGetEditCell) "1_attached"
```

dbExistsLibAttachedFile

Checks if a library has an attached file.

Syntax

```
dbExistsLibAttachedFile libId pseudoName
```

Arguments

libId - ID of the library to be checked. Integer value

pseudoName - String; name of the specified file to be checked.
String value

Description

This function returns true if the specified file is an attached file of the library. Otherwise it returns false.

Example

```
dbExistsLibAttachedFile 1 "1_attached"
```

dbExpungeAllObjByTypeId

Expunges all objects of the specified type in the specified cell.

Syntax

```
dbExpungeAllObjById<string password> <string libName>  
    <int typeId> <string viewName> <string cellName>
```

where the arguments are as follows:

passWord	Required password to execute this command.
libName	Name of the library containing the cell.
typeId	Type ID of the objects to expunge.
viewName	Name of the cell view containing the objects to expunge.
cellName	Name of the cell containing the objects to expunge.

Description

The `dbExpungeAllObjById` function expunges all objects of the specified type in the specified cell. This is a password controlled command and should only be accessible by authorized users.

Example

```
dbExpungeAllObjById password "myLib" 82 "CEL" "myCell"
```

dbExpungeAllObjByTypeName

Expunges all objects of the specified type in the specified cell.

Syntax

```
dbExpungeAllObjByTypeName<string password> <string libName>  
    <string typeName> <string viewName>  
    <string cellName>
```

where the arguments are as follows:

<code>passWord</code>	Required password to execute this command.
<code>libName</code>	Name of the library containing the cell.
<code>typeId</code>	Type ID of the objects to expunge.
<code>viewName</code>	Name of the cell view containing the objects to expunge.
<code>cellName</code>	Name of the cell containing the objects to expunge.

Description

The `dbExpungeAllObjByTypeName` function expunges all objects of the specified type in the specified cell. This is a password controlled command and should only be accessible by authorized users.

Example

```
dbExpungeAllObjByTypeName password "myLib" "dbtText" "CEL"
"myCell"
```

dbExpungeObjTypeById

Expunges the specified object type from the specified library.

Syntax

```
dbExpungeObjTypeById<string libName><int typeId>
```

where the arguments are as follows:

`libName`

Name of the library containing the object type to expunge.

typeId

ID of the type to expunge.

Description

The `dbExpungeObjTypeById` function expunges the specified object type from the specified library. The object's database schema is removed from the library.

Example

```
dbExpungeObjTypeById "myLib" 82
```

dbExpungeObjTypeByName

Expunges the specified object type from the specified library.

Syntax

```
dbExpungeObjTypeByName<string libName><string typeName>
```

where the arguments are as follows:

libName

Name of the library containing the object type to expunge.

typeName

Name of the type to expunge.

Description

The `dbExpungeObjTypeByName` function expunges the specified object type from the specified library. The object's database schema is removed from the library.

Example

```
dbExpungeObjTypeByName "myLib" "dbtText "
```

dbExtractCellPGPosition

Extracts P/G track for cell.

Syntax

```
dbExtractCellPGPosition cellId
```

Arguments

cellId - ID of the cell to extract P/G track. Integer value

Description

This function extracts P/G track for cell and displays the result.

Example

```
dbExtractCellPGPosition (geGetEditCell)
```

dbExtractPGPosition

Extracts power and ground rails.

Tool

Milkyway

Syntax

```
dbExtractPGPosition libraryName
```

where the argument is as follows:

<i>libraryName</i>	The library that you want to extract the power and ground rails from.
--------------------	---

dbExtractPins

Deletes pins not in the specified file.

Syntax

```
dbExtractPins libname cellname pinNameFile
```

Arguments

libname - Name of the library to extract pins. String value

cellname - String; name of cell to extract pins. String value

pinNameFile - Name of the file listing all pins to be deleted. String value

Description

The dbExtractPins function deletes pins not in the specified file.

Example

```
dbExtractPins "RGB" "top" "pinsFile"
```

dbFetchCellPropertyValueByName

Returns the value of the specified cell property.

Syntax

```
dbFetchCellPropertyValueByName <objectId cellId>  
    <string propName>
```

where the arguments are as follows:

cellId

Database object ID of the cell.

propName

Name of the property.

Description

The `dbFetchCellPropertyValueByName` function returns the value of the specified cell property. The cell property is specified by its name, which was assigned during property creation. A boolean false is returned and an error message is displayed if the property does not exist.

See also the [dbCreateCellProperty](#) and [dbUpdateCellProperty](#) commands.

Example

```
dbFetchCellPropertyValueByName (geGetEditCell) "prop1"
```

dbFetchContactCodeInfo

Queries the specified library for specific contact information.

Tool

Milkyway

Syntax

```
dbFetchContactCodeInfo libId contactId techType
```

where the arguments are as follows:

<i>libId</i>	Current open library ID (<i>dbGetCurrentLibId</i>).
<i>contactId</i>	Contact code index (the defined <i>contactCodeNumber</i> within the <i>contactCode</i> section of the technology file). Valid values: Integers less than 256.
<i>techType</i>	Valid fields of the <i>contactCode</i> . Valid values: Can be obtained by entering <i>dbFetchContactCodeTypes</i> .

Example

```
dbFetchContactCodeInfo (dbGetCurrentLibId) 1  
"upperLayerEncWidth"
```

dbFetchContactCodeTypes

Returns a list of contact code object field names that exist in the currently opened library's technology file.

Syntax

```
dbFetchContactCodeTypes <void>
```

Description

The *dbFetchContactCodeTypes* function returns a list of contact code object field names that exist in the currently opened library's technology file.

The field names are used to get specific field information by name for a particular contact code object.

See also `dbFetchContactCodeInfo` or `dbFetchNonDefaultContactCodeInfo`

Example

```
dbFetchContactCodeTypes
```

The above function call may return the following list:

```
("name" "contactCodeNumber" "cutLayerNumber"
"lowerLayerNumber" "upperLayerNumber" "isDefaultContact"
"isFatContact" "cutWidth" "cutHeight" "lowerLayerEncWidth"
"lowerLayerEncHeight" "upperLayerEncWidth"
"upperLayerEncHeight" "minCutSpacing"
"maxNumRowsNonTurning" "unitMinCapacitance"
"unitNomCapacitance" "unitMaxCapacitance"
"unitMinResistance" "unitNomResistance"
"unitMaxResistance" "unitMinInductance"
"unitNomInductance" "unitMaxInductance" "temperatureCoeff"
"reserved")
```

dbFetchContactGeom

Returns the index number for a layer. With this function, you can convert a contact into real physical objects.

Syntax

```
dbFetchContactGeom cellId objectId
```

where the arguments are as follows:

cellId

Database object ID of any open cell in the current library from which you want information about contact geometry

Valid values:

- Cell ID
- (`geGetEditCell`)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

objectId

Database object ID of either the contact or contact array for which you want information about contact geometry

Valid values:

- Contact ID
- Contact array ID

Note:

Using a database object ID other than those stated above is an error and causes `dbFetchContactGeom` to return *nil* or *()*.

dbFetchContactNames

Returns a list of contact code object names that exist in the currently opened library’s technology file.

Syntax

```
dbFetchContactNames <void>
```

Description

The `dbFetchContactNames` function returns a list of contact code object names that exist in the currently opened library's technology file. These names are used to fetch by name information for a specific contact code object (see `dbFetchNonDefaultContactCodeInfo`).

Example

```
dbFetchContactNames
```

The above function call might return the following list:

```
("PCON" "VIA12" "VIA23" "VIA12FAT" "VIA23FAT")
```

dbFetchDesignRuleInfo

Queries the specified library for specific design rule information within the technology file.

Tool

Milkyway

Syntax

```
dbFetchDesignRuleInfo libId layerNum1 layerNum2  
ruletechType
```

where the arguments are as follows:

libId Current open library ID (`dbGetCurrentLibId`).

<i>layerNum1</i>	Design Rule index (the defined <i>layerNumber</i> within the <i>designRule</i> section of the technology file). Valid values: Number of a defined cut layer (must be different than <i>layer2</i>).
<i>layerNum2</i>	Design Rule index (other defined <i>layerNumber</i> within the <i>designRule</i> section of the technology file). Valid values: Number of a defined cut layer.
<i>ruletechType</i>	Valid fields of the <i>designRule</i> . Valid values: Can be obtained by entering <i>dbFetchDesignRuleTypes</i> .

Example

```
dbFetchDesignRuleInfo (dbGetCurrentLibId) 4 11 "minSpacing"
```

where *4* is the possible *layerNumber* of VIA1 and *11* is the possible *layerNumber* of the contact.

dbFetchDesignRuleTypes

Returns a list of design rule object field names that exist in the currently opened library's technology file.

Syntax

```
dbFetchDesignRuleTypes <void>
```

Description

The `dbFetchDesignRuleTypes` function returns a list of design rule object field names that exist in the currently opened library's technology file. The field names are used to find specific field information by name for a particular design rule object (see `dbFetchDesignRuleInfo`).

Example

```
dbFetchDesignRuleTypes
```

The above function call might return the following list:

```
("layerNumber1" "layerNumber2" "stackable" "minSpacing"  
"minEnclosure" "diffNetMinSpacing" "endOfLineEnclosure"  
"reserved")
```

dbFetchFringeCapInfo

Queries the specified library for specific *Fringe* capacitance information within the technology file.

Tool

Milkyway

Syntax

```
dbFetchFringeCapInfo libId layerNum1 layerNum2 techType
```

where the arguments are as follows:

<i>libId</i>	Current open library ID (<code>dbGetCurrentLibId</code>).
--------------	---

<i>layerNum1</i>	Fringe Cap index (the defined <i>layerNumber</i> within the <i>FringeCap</i> section of the technology file). Valid values: One of the two metal layers on which different metal layers cross each other.
<i>layerNum2</i>	Fringe Cap index (other defined <i>layerNumber</i> within the <i>FringeCap</i> section of the technology file). Valid values: One of the two metal layers on which different metal layers cross each other; a whole number at least one less than the number of metal layers (M1, M2, etc.).
<i>techType</i>	Valid fields of the <i>FringeCap</i>. Valid values: Can be obtained by entering <i>dbFetchFringeCapTypes</i>.

Example

```
dbFetchFringeCapInfo (dbGetCurrentLibId) 1 2 "minFringeCap"
```

where *1* is *Metal1* and *2* is *layerNumber* of *Metal2*. Only one *techType* is allowed at a time.

dbFetchLayerIdByName

Returns the index number for a layer. This function returns *nil* or *()* if the layer specified in *layerName* does not exist.

Syntax

```
dbFetchLayerIdByName libId "layerName"
```

where the arguments are as follows:

libId

Database object ID of the library currently open in write (W) mode from which you want information about a layer

Valid values:

- Library ID of the current library
- (dbGetCurrentLibId)

For more information, see “Specifying Libraries” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

layerName

Name of the layer for which you want the index number

Valid values:

Name of any layer in the current library

dbFetchLayerIdByNumber

Returns the index number for a layer. This function returns *nil* or *()* if the layer specified in *layerNumber* does not exist.

Syntax

```
dbFetchLayerIdByNumber libId layerNumber
```

where the arguments are as follows:

libId

Database object ID of the library currently open in write (W) mode from which you want information about a layer

Valid values:

- Library ID of the current library
- (dbGetCurrentLibId)

For more information, see “Specifying Libraries” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

layerNumber

Number of the layer for which you want the index number

Valid values:

Positive integer corresponding to any layer in the current library

See Also

[dbFetchLayerIdByName](#)

dbFetchLayerIdList

Returns a list of the index numbers for all the layers in a library.

Syntax

```
dbFetchLayerIdList libId
```

where the argument is as follows:

libId

Database object ID of the library currently open in write (W) mode from which you want information about a layer

Valid values:

- Library ID of the current library
- (dbGetCurrentLibId)

For more information, see “Specifying Libraries” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

dbFetchLayerInfo

Returns the technology information for a layer in the *layerTable* section of a technology file. This function returns the current value for *layerTechType*. If *layerTechType* does not exist or *libId* is a bad database object ID, dbFetchLayerInfo returns *nil* or *()*.

Syntax

```
dbFetchLayerInfo libId layerIndex "layerTechType"
```

where the arguments are as follows:

libId

Database object ID of the library currently open in write (W) mode from which you want information about a layer

Valid values:

- Library ID of the current library
- (`dbGetCurrentLibId`)

For more information, see “Specifying Libraries” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

layerIndex

Index that identifies the layer

Valid values:

Execute any of the following functions for a list of valid values.

- (`dbFetchLayerIdByName` *libId* "layerName")
- (`dbFetchLayerIdByNumber` *libId* layerNumber)

layerTechType

Layer technology type, such as display attributes or layout attributes, for which you want the current value

Valid values:

Execute `dbFetchLayerTypes` for a list of valid values.

See Also

[dbFetchLayerIdByName](#)
[dbFetchLayerIdByNumber](#)
[dbFetchLayerTypes](#)

dbFetchLayerTypes

Lists all valid layer technology types.

Syntax

`dbFetchLayerTypes`

See Also

[dbFetchLayerInfo](#)

dbFetchNonDefaultContactCodeInfo

Queries the specified contact code object and returns the specified field data.

Syntax

```
dbFetchNonDefaultContactCodeInfo <objectId libId> <string  
contactName> <string fieldName>
```

`libId` - Database object ID of the opened library containing the contact code object

`contactName` - Name of the contact code object

`fieldName` - Name of the contact code object field (a list of valid field names can be obtained using `dbFetchContactCodeTypes`)

Description

The `dbFetchNonDefaultContactCodeInfo` function queries the specified contact code object and returns the specified field data.

The contact code is specified by name. To specify the contact code by number, use `dbFetchContactCodeInfo`.

Note: Despite the function name, this function may be used to query both default and non-default contact codes.

Example

```
dbFetchNonDefaultContactCodeInfo (dbGetCurrentLibId)
"VIA12FAT" "lowerLayerNumber"
```

The above function call might return the following lower layer number:

3

dbFetchObject

Lists all the field names and the data that is contained within these fields for a given object. This function is a useful programming tool during the development and debugging of Scheme programs. The `dbFetchObject` function returns the field names and current values in the following syntax:

```
( { ( fieldName ( fieldValue ) ) } ... )
```

Syntax

```
dbFetchObject libraryOrCellId subObjectId
```

where the arguments are as follows:

libraryOrCellId

Database object ID of the cell or library for which you want field names and current values

Valid values:

- Cell ID of any cell in the current library
- (geGetEditCell)
- Library ID of the current library

- (dbGetCurrentLibId)

For more information, see “Specifying Libraries” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

subObjectId

Database object ID of the object for which you want field names and current values

Valid values:

- *nil* returns the information contained in the cell header record if *libraryOrCellId* is a cell ID, or it returns the information contained in the library header record if *libraryOrCellId* is a library ID.
- The object ID of any object within the library or cell specified in *libraryOrCellId* returns all available information for the object.

Example

This example lists all available information for a specified path.

```
dbFetchObject cellId pathId
( (layerNumber (10)) (pathWidth (5000)) ...)
```

Note:

Both *cellId* and *pathId* are defined as variables in a previous step.

See Also

->?

[dbFetchObjectField](#)

dbFetchObjectField

Retrieves the value of a specified field associated with a specified database object. This function is central to any Scheme program that accesses the Synopsys database.

Syntax

```
dbFetchObjectField fileId ObjectId fieldName
```

where the arguments are as follows:

fileId

Database object ID of the cell or library from which you want to fetch the value of a specific field.

Valid values:

File ID of any cell in the current library (*geGetEditCell*)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

ObjectId

Database object ID of the object to which the field information applies.

Valid values:

Object ID of any object within the cell

Objects do not have to be homogeneous within the group. For example, you can place a net and a cell together in one group.

fieldName

Name of the field about which information is desired.

Valid values:

Name of any field associated with the specific object.

If the first character of *fieldName* is a question mark (?), the command will display all valid field names associated with *subObjectId* or *libraryOrCellId*.

Example 1

You can select a wire in the graphics window and query the selected wire, and you will see that the wire ID is one of the printed parameters. Alternatively, from a script, *wireId* can be set in a variable from a single selected wire with the command

```
(wireId (car (geGetSelectedSet (hiGetCurrentWindow))))  
(wireMasterNumber (dbFetchObjectField (geGetEditCell)  
  wireId  
  "wireMasterIdx"))  
(wireMasterId (dbMakeWireMasterId (geGetEditCell)  
  wireMasterNumber))  
dbFetchObjectField (geGetEditCell) wireMasterId "width"
```

which returns the width of the wire. Note that this wire width must be divided by the value of “length precision” in the technology file for obtaining the actual wire width. The unit value of the resulting wire width is in units of *unitLengthName* from the technology file.

Example 2

```
dbFetchObjectField (geGetEditCell) InstanceId "bbox"  
  ((252.8 406.2) (288 451))
```

The record name “bbox” is not printed for the *InstanceId* record when queried, because it is computed at runtime and is not stored in the database. It is, nevertheless, a valid query parameter for the *InstanceId* record.

Example 3

```
define libId (dbOpenLib "DESIGN")
dbFetchObjectField libId '() "?"
    schemaMajorVersion
    schemaMinorVersion
    libFlag
    userDistUnit
    dataBaseUnitPerUserDistUnit
    dataBaseUnitPerMinGrid
    libCreator
    techFileName
    libraryName
```

Example 4

```
define cellId (geGetEditCell)
define portId (dbGetPortByName cellId "A")
dbFetchObjectField cellId portId "?"
    name
    index
    direction
    type
    numberOfPin
    netId
    propertyId
    mustJoinRootId
```

Note that “escaped” port names must be handled as an exception with Milkyway Scheme, because the character is used to denote the character immediately following it if a string is to be interpreted as a literal. So to use *dbGetPortByName* with an escaped port name such as \R[11], the command would be:

```
dbGetPortByName cellId "\\R[11]"
```

Note that when using the *geNameSelect* command, you can simply enter the escaped port name as \R[11] and the command automatically converts the name to \\R[11].

See Also

->?

[dbFetchObject](#)

dbFetchPinInstByPortInst

Returns the pin instance data associated with a port instance.

Syntax

```
dbFetchPinInstByPortInst cellId portInstId
```

where the arguments are as follows:

cellId

Database object ID of the cell containing the port instance from which you want pin instance information

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

portInstId

Database object ID of the port instance from which you want pin instance information

Valid values:

ID of any port instance in the specified cell

dbFetchProperty

Retrieves the value of a specified property.

Tool

Milkyway

Prerequisites

Open a cell.

Place in the design flow

Data preparation.

Syntax

```
dbFetchProperty cellId objId propertyName
```

where the arguments are as follows:

<i>cellId</i>	The ID of the cell which has the object.
<i>objId</i>	The ID of the object which has the property.
<i>propertyName</i>	The name of the property.

dbFetchPropertyFieldAndName

Queries the specified property object and returns its name and the specified field data in a list.

Syntax

```
dbFetchPropertyFieldAndName <objectId fileId> <objectId  
propertyName> <string fieldName>
```

fieldId - Database object ID of the opened library or cell containing the property object

propertyId - Database object ID of the property

fieldName - Name of the property object field (a list of valid field names can be obtained using dbFindObjectField)

Description

The dbFetchPropertyFieldAndName function queries the specified property and returns its name and the specified field data in a list.

This function returns a list comprised of two lists. Each list has a field name and data value. The first list contains the field name (propertyName) and its corresponding value. The second list contains the specified field name and its corresponding value.

Example

```
dbFetchPropertyFieldAndName (geGetEditCell) #x2400  
"propertyValue"
```

The above function returns a list such as,

```
(( "propertyName" "m9AntennaScale") ("propertyValue" 1.0))
```

dbFetchPropertyValueByName

Queries the specified object and returns the associated property value.

Syntax

```
dbFetchPropertyValueByName <objectId cellId> <objectId  
objId> <string propName>
```

cellId - Database object ID of the opened cell containing the object

objId - Database object ID of the object with the associated property

propName - Name of the property

Description

The dbFetchPropertyValueByName queries the specified object and returns the associated property value.

This function queries a property associated with a database object. To create and associate a property with an object, see [dbCreateProperty](#).

Example

```
dbFetchPropertyValueByName (geGetEditCell) #x800  
"MyPersonalProperty"
```

The above function call might return the following string:

```
"Created By Me"
```

dbFetchScanTypes

Lists the available scan types associated with the specified library, cell, or subobject. This function returns the available scan types in the following syntax:

```
({"scantype"} ...)
```

Syntax

```
dbFetchScanTypes libraryOrCellId subObjectIdOrNil
```

where the arguments are as follows:

libraryOrCellId

Database object ID of the cell or library for which you want the available scan types

Valid values:

- Cell ID of any cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

- Library ID of the current library
- (dbGetCurrentLibId)

For more information, see “Specifying Libraries” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

subObjectIdOrNil

Database object ID of the object from which information is desired or *nil*

Valid values:

- '()' returns the scan types associated with the library or cell.
- The object ID of any object within the specified library or cell returns the scan types associated with the object.

Example

This example lists the available scan types for a specified net.

```
dbFetchScanTypes cellId netId  
("Property" "ChildNet" "NetMember")
```

Note:

Both *cellId* and *netId* are defined as variables in a previous step.

dbFetchTechInfo

Returns the value of a technology type. This function returns the current value for *techType*. If *techType* does not exist or *libId* is a bad database object ID, *dbFetchTechInfo* returns *nil* or *()*.

Syntax

```
dbFetchTechInfo libId "techType"
```

where the arguments are as follows:

libId

Database object ID of the library currently open in write (W) mode from which you want information about the technology file

Valid values:

- Library ID of the current library
- (dbGetCurrentLibId)

For more information, see “Specifying Libraries” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

techType

Technology type for which you want the current value

Valid values: Execute [dbFetchTechTypes](#) for a list of valid values.

See Also

[dbFetchTechTypes](#)

dbFetchTechTypes

Lists all valid technology types.

Syntax

```
dbFetchTechTypes
```

See Also

[dbFetchTechInfo](#)

dbFileCompare

Compares and reports the differences between two cells or two libraries.

Syntax

```
dbFileCompare <objectId fileId1> <objectId fileId2> [string  
fileName] [int detailLevel]
```

fileId1 - Database object ID of the first opened cell or library

fileId2 - Database object ID of the second opened cell or library

fileName - Name of the output report file (optional). If not specified, the report is displayed to the Milkyway scheme window.

detailLevel - Indicates the amount of detail to compare (optional). The default setting compares the cells or libraries with the greatest detail.

Note: The arguments, fileId1 and fileId2, must correspond to two unique cells or two unique libraries.

Description

The dbFileCompare function compares and reports the differences between two cells or two libraries.

This function combines the functionality of dbCellCompare and dbLibraryCompare. It behaves the same as the dbCellCompare and dbLibraryCompare functions, except that the specified cells or libraries must be opened prior to calling this function.

See dbCellCompare or dbLibraryCompare for more information.

Example 1

```
dbFileCompare 1 2 "compare.txt" 1
```

This command generates the following in a text file named "compare.txt" (Object ID 1 and 2 are libraries),

```

*****
LIBRARY COMPARISON REPORT:
Library 1: /home/lib1
Library 2: /home/
lib2*****
****
No differences were found.

```

Example 2

```
dbFileCompare 3 4 "compare.txt" 1
```

This command generates the following in a text file named "compare.txt" (Object ID 3 and 4 are cells),

```

*****
CELL COMPARISON REPORT:
Cell 1: cell1.CEL;1
        (/home/lib1)
Cell 2: cell2.CEL;
        (/home/lib2)
*****
No differences were found.

```

dbFixCellData

Fixes corrupted net data in the specified cell's netlist.

Syntax

```
dbFixCellData <objectId cellId> <int passcode>
```

cellId - Database object ID of an opened cell that needs to be fixed

passcode - A passcode allowing access to use this function

Description

The `dbFixCellData` function fixes corrupted net data in the specified cell's netlist.

This function detaches all net objects (such as wires, paths, contacts, contact arrays, and glinks) from the nets and reattaches them according to the objects' net IDs. This fixes object linking corruptions where the nets and net objects do not point to each other correctly.

This function should only be used as an R&D debugging tool. Its access and use is controlled by a passcode. If a corruption problem occurs, please report it to Synopsys first.

See also [dbFixNetList](#).

dbFixNetList

Fixes duplicated net names in the specified cell's netlist.

Syntax

```
dbFixNetList <objectId cellId> <int passcode>
```

where the arguments are as follows:

`cellId`

database object ID of an opened cell that needs to be fixed

`passcode`

a passcode allowing access to use this function

Description

The `dbFixNetList` function fixes duplicated net names in the specified cell's netlist. It searches the netlist for duplicated net names and either merges the nets with the same name or differentiates the net names.

If a net has no input port connections (including tristate and inout ports), it is merged with the duplicated net with the same name. In all other cases, the net is unchanged, except its name is differentiated from the duplicated net name by appending a unique integer to the end of the original name. This fixes netlist corruptions resulting from non-unique net names.

This function should only be used as an R&D debugging tool. Its access and use is controlled by a passcode. If a corruption problem occurs, please report it to Synopsys first.

See Also

[dbFixCellData](#)

dbForceRecompCellBBox

Forces the system to update all instances of the database page bounding box of the specified cell and then update the cell bounding box.

Syntax

```
dbForceRecompCellBBox cellId
```


where the argument is as follows:

cellId

Database object ID of the cell for which you want to update the bounding box

Valid values:

- Cell ID of any open cell in the current library
- (`geGetEditCell`)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

dbForceRecursiveUpdateAllHierarchy

Operates within the cell hierarchy and propagates any changes to top-level cells. This command updates all cells. It is similar to [dbForceUpdateHierarchy](#), except that `dbForceUpdateHierarchy` works only on the top cell’s immediate children.

Syntax

```
dbForceRecursiveUpdateAllHierarchy cellId
```

where the argument is as follows:

cellId

Database object ID of the cell that you want to update

Valid values:

- Cell ID of any open cell in the current library

- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

Example

Cell A has an instance B, and the master cell of B has an instance C. If the master cell in C has changed, you must use `dbForceRecursiveUpdateAllHierarchy` to propagate the change to the top level for all cells.

dbForceUpdateHierarchy

Forces the system to update all cell instances of the specified cell, to make them consistent.

Syntax

```
dbForceUpdateHierarchy cellId
```

where the argument is as follows:

cellId

Database object ID of the cell

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

db-foreach

Executes a statement for each object associated with a subobject in the way specified by the scan name.

Syntax

```
db-foreach libraryOrCellId subObjectIdOrNil fieldName  
"scanName"  
loopVariable (statements)
```

where the arguments are as follows:

libraryOrCellId

Database object ID of the cell or library to which the information applies

Valid values:

- Cell ID of any cell in the current library
- Library ID of any library in the current directory

subObjectIdOrNil

Database object ID of the object from which information is desired or *nil*

Valid values:

- Object ID of any object within the specified library or cell
- *nil* (') if the scan is directly associated with the library or the cell

FieldName

Name of a valid scan. If the first character of *scanName* is '?', the command will display all valid scan names associated with *subObjectId* or *libraryOrCellId*.

scanName

Name of a valid scan. Valid values for *scanName* depend on the values of the *libraryOrCellId* and *subObjectIdOrNil* fields.

Execute [dbFetchScanTypes](#) to get a complete list.

The following values apply for a given library ID when *subObjectIdOrNil* is equal to *nil* (''):

- *CellName*
- *CellVersion*
- *RefLibrary*

The following values apply for a given cell ID when *subObjectIdOrNil* is equal to *nil* (''):

- *Trapezoid*
- *Polygon*
- *Rectangle*
- *Path*
- *Text*
- *HorizontalWire*
- *VerticalWire*
- *WireMaster*
- *Contact*

- *ContactArray*
- *PseudoPath*
- *PseudoContactArray*
- *Net*
- *NetHeader*
- *Pin*
- *Port*
- *CellInstHeader*
- *CellInstMaster*
- *CellInst*
- *PortInstMaster*
- *PortInst*
- *CellInstArray*
- *ProtectionFrame*
- *CellRegion*
- *CellRow*
- *HardFence*
- *SoftFence*
- *RegionPin*
- *RegionPort*
- *GateSite*

- *GateSiteArray*
- *WireTrack*
- *WireDir*
- *ErrorClass*
- *Error*
- *ErrorObject*
- *AttachedFile*
- *Node*
- *Edge*
- *Channel*
- *ChannelGate*
- *ChannelGatePin*
- *PreRoute*
- *InterRouteSeg*
- *PortToPort*
- *PortToPortTLU*
- *State*
- *StateTransition*
- *StateCommandClass*
- *StateCommand*
- *PointArray*

- *CellLocArray*
- *PadLocArray*
- *CellEQClass*
- *SlicingTreeNode*
- *Guide*
- *ChannelGuide*
- *Property*
- *PropertyHeader*
- *PropertyAttribute*
- *Group*
- *GroupMember*
- *PlanGroup*
- *ParameterSet*
- *DelayTable*
- *DelayValueTable*
- *UserUnformatted*

The following values are applicable to AstroGA only.

- *InternalSOGXWire*
- *InternalSOGYWire*
- *SOGWireMaster*
- *SOGContact*

- *SOGContactMaster*

The following values apply for a given cell ID when *subObjectIdOrNil* is not equal to *nil* (''):

Table 25-32 Cell ID Values

<i>subObjectId type</i>	<i>scanName</i>
Cell boundary	Valid value: <i>Property</i>
Cell EQ class	Valid value: <i>Property</i>
Cell instance	Valid values: <i>PortInst</i> <i>Property</i>
Cell instance array	Valid value: <i>Property</i>
Cell instance header	Valid value: <i>CellInst</i>
Cell instance master	Valid values: <i>CellInst</i> <i>PortInstMaster</i>
Cell name	Valid value: <i>Property</i>
Cell region	Valid values: <i>CellRow</i> <i>Property</i> <i>RegionPin</i> <i>RegionPort</i>
Cell row	Valid values: <i>CellInst</i> <i>Property</i>
Cell version	Valid value: <i>Property</i>
Channel	Valid value: <i>Property</i>

Table 25-32 Cell ID Values (Continued)

<i>subObjectId</i> type	<i>scanName</i>
Channel gate	Valid values: <i>ChannelGatePin</i> <i>Property</i>
Channel gate pin	Valid value: <i>Property</i>
Channel guide	Valid value: <i>Property</i>
Delay table	Valid value: <i>Property</i>
Edge	Valid value: <i>Property</i>
Gate site	Valid value: <i>Property</i>
Gate site array	Valid value: <i>Property</i>
Group	Valid values: <i>GroupMember</i> <i>Property</i>
Guide	Valid value: <i>Property</i>
Hard fence	Valid values: <i>CellInst</i> <i>Group</i> <i>Property</i>
Net	Valid values: <i>ChildNet</i> <i>NetMember</i> <i>Property</i>
Node	Valid value: <i>Property</i>
Parameter set	Valid value: <i>Property</i>
Path	Valid value: <i>Property</i>
Pin	Valid value: <i>Property</i>

Table 25-32 Cell ID Values (Continued)

<i>subObjectId</i> type	<i>scanName</i>
Polygon	Valid value: <i>Property</i>
Port	Valid values: <i>Pin</i> <i>Property</i>
Port-to-port delay	Valid value: <i>Property</i>
Port-to-port table	Valid value: <i>Property</i>
Preroute segment	Valid value: <i>Property</i>
Protection frame	Valid value: <i>Property</i>
Pseudopath	Valid value: <i>Property</i>
Reference library	Valid value: <i>Property</i>
Region pin	Valid value: <i>Property</i>
Region port	Valid values: <i>Property</i> <i>Region Pin</i>
Soft fence	Valid values: <i>CellInst</i> <i>Group</i> <i>Property</i>
Slicing tree node	Valid value: <i>Property</i>
Text	Valid value: <i>Property</i>
Wire track	Valid value: <i>Property</i>
Wire direction	Valid value: <i>Property</i>

loopVariable

Name of the variable to be bound to a new location and assigned a database ID for each pass through the loop. The *loopVariable* field persists only for the life of the loop.

statement

Any single statement. The statement may be a *begin* statement if multiple function calls are needed. If *statement* returns *nil*, the loop is aborted. This is equivalent to a break in C.

Important:

Avoid creating objects that would be a part of the scan during the scan. The new objects might appear in the current scan.

Example 1

```
define libName "ex4"
"ex4"
define libId (dbOpenLib libName)
db:1
(db-foreach libId '() "cellName" cellNameRec
(begin
(display (dbFetchObjectField libId cellNameRec
"cellName")))
#t)
)
`()
dbCloseLib
#t
```

In this example,

- The first line creates an alias, libName, for the library ex4.
- The second line creates an alias, libId, for the database object ID of ex4.
- The third line displays the value of the cell name field for each cell name record in the library ex4.

Example 2

```
define libId (dbOpenLib "DESIGN")
db-foreach libId '() "?" () ()
    CellName
    CellVersion
    RefLibrary
```

Example 3

```
define cellId (geGetEditCell)
define netId (dbGetNetByName cellId "VDD")
b-foreach cellId netId "?" () ()
    ChildNet
    PortInst
    Port
    NetMember
    NetWiring
```

See Also

[dbFetchScanTypes](#)

dbFormNetClass

Creates a net class in a cell with individual nets or other net classes you specify as members.

Tool

Milkyway

Syntax

```
dbFormNetClass cellId "className" ' ("member1Name"
"member2Name" ...)
```

where the arguments are as follows:

<i>cellId</i>	ID of the cell in which you want to create a net class. Valid values: ID of any cell in the open library. If the cell is open, you can substitute the following for <i>cellId</i> : (geGetEditCell).
<i>className</i>	Name of the net class you want to create.
<i>memberName</i>	Name of a net or net class you want to include in the new net class. Valid values: Any predefined net name or net class name. Pattern matching is allowed.

Example

```
dbFormNetClass (geGetEditCell) "wwInfo" ' ("N2340" "N1776")
```

This example creates a net class named *wwInfo* that includes nets *N2340* and *N1776*.

Note:

To include all nets whose names start with “N” in the example above, substitute ‘(“N.*”) for ‘(“N2340” “N1776”).

See Also

[dbDeleteNetClass](#) deletes a specified net class from the cell.

[dbQueryNetClassByName](#) displays the names of the nets in a specified net class.

[dbPrintAllNetClasses](#) displays the names of all net classes in a cell.

dbFreeCleanPages

Frees unmodified pages in the Milkyway database cache.

Tool

Milkyway

Syntax

`dbFreeCleanPages`

dbFreezeRoutingOnNets

Prevents the router from changing the specified nets. The status is stored in the database and constrains all of the following routing operations.

Syntax

`dbFreezeRoutingOnNets cellId option "net_name_pattern"`

where the arguments are as follows:

cellId

Database object ID of the cell from which you want to delete an attached file

Valid values:

- Cell ID of any open cell in the current library
- `(geGetEditCell)`

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

option

Boolean where

- *#t* freezes the routing matching the pattern specified in *net_name_pattern*
- “m” allows minor changes on the net
- *#f* allows the router to change nets matching the pattern specified in *net_name_pattern*

net_name_pattern

Name of specified nets for which routing is to be frozen or that the router is to change

Valid values:

Any text string

Example 1

The following code freezes routing on nets matching the pattern *a/.** or *b/.**:

```
dbFreezeRoutingOnNets (geGetEditCell) #t "a/.*, b/.*" 
```

Example 2

The following code allows the router to change nets matching the pattern *a/.**:

```
dbFreezeRoutingOnNets (geGetEditCell) #f "a/.*" 
```

dbGenNetList

Creates a generated (GEN) view cell containing a randomly generated netlist.

Syntax

```
dbGenNetList <string libName> <string cellName> <string  
childLibName> <int numOfNets> <int numOfCellInsts> <int  
maxInstsPerMaster> <int l2PortNet> <int l3PortNet> <int  
l4PortNet> <int l5PortNet>
```

libName - Name of the library in which to create the generated view cell

cellName - Name of the generated (GEN) view cell

childLibName - Name of the library containing the frame (FRAM) view cells from which cell instances are instantiated. Can be the same library as libName

numOfNets - Number of nets to randomly generate

numOfCellInsts - Number of cell instances to randomly instantiate

maxInstsPerMaster - Maximum number of instances to instantiate for each frame view cell (currently not used and not supported)

l2PortNet - Percentage of total nets that should connect to two ports

l3PortNet - Percentage of total nets that should connect to three ports

l4PortNet - Percentage of total nets that should connect to four ports

l5PortNet - Percentage of total nets that should connect to five ports

Note: The total percentage (sum of l#PortNet arguments) must be less than 100.

Description

The dbGenNetList function creates a generated (GEN) view cell containing a randomly generated netlist. After creating a generated view cell in the specified library, it randomly populates it with cell instances of the frame (FRAM) view cells in the child library, creates nets, and then connects the nets to the ports of the cell instances.

The number of nets with two, three, four, and five port connections are defined by the l#PortNet arguments. These arguments define the percentage of the nets that have a specific number of ports. The total percentage must be less than 100, because the remaining nets are necessary to connect the rest of the ports.

This function fails if a netlist is unobtainable with the specified parameters.

dbGetLayerNumberFromName

Returns the layer number of a specified layer.

Syntax

```
dbGetLayerNumberFromName libId "layerName"
```

where the arguments are as follows:

libId

Database object ID of the library to which layer information applies

Valid values:

Library ID of any library in the current directory

Note:

When the cell ID is used as *libId*, the correct layer number is returned.

layerName

Name of the layer of which you want the layer number

Valid values:

Name of any layer in the specified library

Example

The code in this example determines which layer number corresponds to METAL1.

```
define libId (dbOpenLib "test")
  db:1
dbGetLayerNumberFromName libId "METAL1"
9
```

See Also

[dbGetLibIdByCellId](#)

dbGetLayerNameFromNumber

Returns the layer name of a specified layer.

Syntax

`dbGetLayerNameFromNumber libId layerNumber`

where the arguments are as follows:

libId

Database object ID of the library to which the layer information applies

Valid values:

Library ID of any library in the current directory

layerNumber

Number of the layer of which you want the name

Valid values: Number of any layer in the specified library

Example

The code in this example determines the name of a given layer number.

```
define libId (dbOpenLib "test")
db:1
dbGetLayerNameFromNumber libId 9
"METAL1"
```

See Also

[dbGetLayerNumberFromName](#)

dbGetLibIdByCellId

Returns the library ID of the library that contains a specified cell.

Syntax

```
dbGetLibIdByCellId cellId
```

where the argument is as follows:

cellId

Internal ID of any open cell in the library for which you want the library ID

Valid values:

- Cell ID
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

See Also

[dbGetCurrentLibId](#)

dbGetNetByName

Returns the database object ID of a net. If the net does not exist, the function returns *nil*.

Syntax

```
dbGetNetByName cellId "netName"
```

where the arguments are as follows:

cellId

Database object ID of the cell containing the net of which you want the database object ID

Valid values:

- Cell ID of any open cell in the current library
- (`geGetEditCell`)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

netName

Name of the net of which the object ID is required

Valid values:

Local name of any net in the specified cell

Example 1

The code in this example gets the database object ID for the power net. For cells opened with [dbOpenCell](#).

```
define cellId (dbOpenCell "top" "r")
db:2
.
.
.
dbGetNetByName cellId "VDD"
db:12b002b
```

Example 2

For cells open in the layout window, this example demonstrates the substitution of *geGetEditCell* for *cellId*.

```
dbGetNetByName (geGetEditCell) "VDD"  
db:12b002b
```

Examples 1 and 2 both return the net ID of the power net VDD.

Example 3

The code in this example tests for the existence of net VDD and creates it if it does not exist.

```
(if (not (dbGetNetByName cellId "VDD"))  
(dbCreateNet cellId "VDD")  
)  
db:12b002b  
dbSetNetTypeByName cellId "VDD" "power"  
#t
```

See Also

[dbCreateNet](#)

[dbMoveAllNetMember](#)

dbGetPinByName

Returns the database object ID of a pin. If the pin does not exist, the function returns *nil*.

Syntax

```
dbGetPinByName cellId "pinName"
```

where the arguments are as follows:

cellId

Database object ID of the cell containing the pin of which you want the database object ID

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

pinName

Name of the pin of which the object ID is required

Valid values:

Name of any pin in the specified cell

dbGetPortByName

Returns the database object ID of a port. If the port does not exist, the function returns *nil*.

Syntax

```
dbGetPortByName cellId "portName"
```

where the arguments are as follows:

cellId

Database object ID of the cell containing the port of which you want the database object ID

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

pinName

Name of the port of which the object ID is required

Valid values:

Name of any port in the specified cell

dbGetRegionIdByCellInstId

Gets the region ID from a specified cell instance.

Syntax

```
dbGetRegionIdByCellInstId cellId cellInstId
```

where the arguments are as follows:

cellId

Database object ID of the cell containing the region of which you want the database object ID

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

cellInstId

Database object ID of the cell to which the region information applies

Valid values:

Cell instance ID of any cell instance in the specified cell

dbGetRegionPortByName

Gets the region port ID of a specified region port.

Syntax

```
dbGetRegionPortByName cellId "regionPortName"
```

where the arguments are as follows:

cellId

Database object ID of the cell containing the region port of which you want the database object ID

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

regionPortName

Name of the region port of which the object ID is required

Valid values:

Name of any region port in the specified cell

dbGetRegionPortDirection

Gets the direction of a specified region port.

Syntax

```
dbGetRegionPortDirection cellId regionPortId
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies.

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

regionPortId

Database object ID of the region port from which direction information is required

Valid values:

Region port ID of any region port in the specified cell

dbGetAccessMode

Returns the access mode of an opened cell or library.

Syntax

```
dbGetAccessMode <objectId fileId>
```

fileId - The database object ID of an opened cell or library

Description

The dbGetAccessMode function returns the access mode of an opened cell or library. The possible return values are null or a string representing the access mode ("r" or "w").

dbGetAllInstFromFence

Returns a list of all cell instances associated with a fence.

Syntax

```
dbGetAllInstFromFence cellId fenceId
```

where the arguments are as follows:

cellId

Database object ID of the cell containing the fence from which you want cell instance information

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

fenceId

Database object ID of the fence from which you want cell instance information

Valid values:

Object ID of any fence in the specified cell

dbGetAllInstFromGroup

Returns a list of all cell instances associated with a group.

Syntax

```
dbGetAllInstFromGroup cellId groupId
```

where the arguments are as follows:

cellId

Database object ID of the cell containing the group from which you want cell instance information

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

groupId

Database object ID of the group from which you want cell instance information

Valid values:

Object ID of any group in the specified cell

dbGetAllLibCellInfo

Displays useful cell information for all the cells in the specified library.

Syntax

```
dbGetAllLibCellInfo "library_name"
```

where *library_name* is the name of an existing library.

dbGetCellAttachedFile

Gets a cell's attached file.

Syntax

```
dbGetCellAttachedFile cellId attFileName
```

Arguments

cellId - ID of the cell for attached file. Integer value

attFileName - Name of the cell's attached file. String value

Description

The dbGetCellAttachedFile function returns a specific attached file name with the full path for a given cell.

Example

```
dbGetCellAttachedFile (geGetEditCell) "top_1"
```

If the cell's attached file is found, it returns the full path of the attached file such as the following:

```
/net/gorp/jane_doe/RGB_widget/top_1
```

If the attached file is not found, it returns false (#f).

dbGetCellByInstPath

Returns the cell ID corresponding to the specified cell instance.

Syntax

```
dbGetCellByInstPath <objectId topLibId> <objectId  
topCellId> <string instPath>
```

where the arguments are as follows:

topLibId

Database object ID of the library containing the top cell.

topCellId

Database object ID of the top cell containing the cell instance.

instPath

Path and name of the cell instance.

Description

The `dbGetCellByInstPath` function returns the cell ID corresponding to the specified cell instance. The top cell should contain the specified instance, otherwise this function returns an error.

The instance path is a list of instances, starting in the top cell and delimited by forward slashes that indicate the path to the desired cell instance. The cell ID of the last cell instance is returned.

Example

```
dbGetCellByInstPath (dbGetCurrentLibId) (geGetEditCell)  
"/instA/instB/instC/instD"
```

dbGetCellFullNameById

Gets a cell's full name from a cell identifier.

Syntax

```
dbGetCellFullNameById cellId
```

Arguments

cellId - Id of the cell for a full name. Integer value

Description

The dbGetCellFullNameById function returns the full cell name of a given cell. A full cell name is cellName.viewName;cellVersion (see the following example).

Example

```
dbGetCellFullNameById (geGetEditCell)
```

This returns the full cell name of the current cell, such as the following:

```
"top.CEL;2"
```

dbGetCellInstById

Gets a cell instance for a cell.

Syntax

```
dbGetCellInstById cellId cellInstId
```


Arguments

cellId - Id of the cell in which for a cell instance. Integer value

cellInstId - Id of the cell instance. Integer value

Description

The `dbGetCellInstById` function returns true (#t) if the cell instance exists for the cell. Otherwise it returns false (#f).

Example

```
dbGetCellInstById (geGetEditCell) 35328
```

dbGetCellInstByName

Returns the database object ID of a cell instance. If the cell instance does not exist, the function returns *nil*.

Syntax

```
dbGetCellInstByName cellId "cellInstName"
```

where the arguments are as follows:

cellId

Database object ID of the cell containing the cell instance.

Valid values:

- Cell ID of any cell in the current library
- `(geGetEditCell)`

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

cellInstName

Name of a cell instance to which information applies

Valid values:

Local name of any cell instance in the specified cell

Example

The code in this example gets the database object ID for a specified cell instance given a local instance name.

```
define cellId (geGetEditCell)
  db:2
define instanceId (dbGetCellInstByName cellId "_AAAB.I20")
  db:3001f
```

See Also

[dbConvertRegionToMacroCell](#)

dbGetCellInstRefLib

Gets the directory path of the reference library containing the specified cell instance’s master.

Syntax

```
dbGetCellInstRefLib <objectId cellId> <objectId cellInstId>
  <string option>
```

where the arguments are as follows:

cellId

Database object ID of the top cell containing the cell instance.

cellInstId

Database object ID of the cell instance.

option

Indicates the path type to return.

Valid values are:

- "name"

If this value is specified, this function returns the full path and name of the reference library.

- "path"

If this value is specified, this function returns the full path *without the name* of the reference library.

Description

The `dbGetCellInstRefLib` function returns the directory path of the reference library containing the specified cell instance's master. The specified top cell should contain the specified instance, otherwise this function returns an error.

Example

```
dbGetCellInstRefLib (geGetEditCell) #xFF00 "name"
```

dbGetCellNameAndViewById

Gets a cell's name and view.

Syntax

```
dbGetCellNameAndViewById cellId
```

Arguments

cellId - Id of the cell for cell name and view. Integer value

Description

The `dbGetCellNameAndViewById` function returns the cell name and its view of a given cell. The return is the form of `cellName.viewName` (see the following example).

Example

```
dbGetCellNameAndViewById (geGetEditCell)
```

This will return the cell name and view of the current cell, such as the following:

```
"top.CEL"
```

dbGetCellNameById

Returns a cell's name.

Syntax

```
dbGetCellNameById cellId
```

Arguments

cellId - Id of the cell for cell name. Integer value

Description

The `dbGetCellNameById` function returns the cell name of a given cell (see the following example).

Example

```
dbGetCellNameById (geGetEditCell)
```

This will return the cell name of the current cell, such as the following:

```
"top"
```

dbGetCellViewVersions

Lists all existing version numbers of the specified cell view.

Syntax

```
dbGetCellViewVersions <string libName> <string cellName>  
<string viewName>
```

libName - Name of the library (including the relative or absolute path if necessary)

cellName - Name of the cell

viewName - Name of the view (for example, CEL, FRAM)

Description

The `dbGetCellViewVersions` function lists all existing version numbers of the specified cell view. The list is displayed to the Milkyway scheme window.

Example

```
dbGetCellViewVersions "myLib" "top" "CEL"
```

The above function call displays the following list:

"top.CEL" has the following versions:

1
2
3
5
6

dbGetCheckLibMode

Returns the current check-library-mode flag.

Syntax

```
dbGetCheckLibMode
```

Description

The `dbGetCheckLibMode` function returns the current check-library-mode flag. The possible return values include the scheme `true` and `false` boolean values (`#t` and `#f`).

If the flag is `true`, libraries are checked for consistency when they are opened. If the flag is `false`, libraries are not checked. For more information, see [dbSetCheckLibMode](#).

dbGetConnectedObject

Returns a list of all connected objects. These objects include ports (I/O on this module), port instances (I/O of a cell instance) and wires.

Tool

Milkyway

Syntax

```
dbGetConnectedObject cellId objectId
```

where the arguments are as follows:

<i>cellId</i>	The ID of the current opened cell. If the cell is open, you can substitute the following for <i>cellId</i> : (geGetEditCell).
---------------	---

<i>objectId</i>	A physical object that has a net associated with it.
-----------------	--

dbGetConstraintsCellId

Returns the cell ID of a cell currently operated on by a load constraint.

Syntax

```
dbGetConstraintsCellId
```

dbGetCurrentLibId

Returns the library ID of the library currently open in write (W) mode. If no library is open, `dbGetCurrentLibId` returns *nil* or *()*.

Syntax

```
dbGetCurrentLibId
```

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

dbGetDiodePortByName

Returns the database object ID for the specified port.

Syntax

```
dbGetDiodePortByName <objectId cellId> <string portName>
```

cellId - Database object ID of the opened cell

portName - Name of the port

Description

The dbGetDiodePortByName function returns the database object ID for the specified port. If the port does not exist, then the Scheme boolean value false (#f) is returned.

dbGetLayerNameByMaskName

Returns the technology layer name for a layer mask name.

Syntax

```
dbGetLayerNameByMaskName libId maskName
```

Arguments

libId - The ID of the library. Integer value

maskName - The layer mask name. String value

Description

The dbGetLayerNameByMaskName function returns the technology layer name for a given layer mask name.

Example

```
dbGetLayerNameByMaskName 1 "metal2"
```

For mask name “metal2”, this function returns the layer name such as

```
"M2R"
```

dbGetLayerNumberByMaskName

Returns the technology layer number for a layer mask name.

Syntax

```
dbGetLayerNumberByMaskName libId maskName
```

Arguments

libId - The ID of the library. Integer value

maskName - The layer mask name for a layer. String value

Description

This returns the Technology layer number for a layer mask name.

Example

```
dbGetLayerNumberByMaskName 1 "metal2"
```

For mask name “metal2”, this function returns the layer number such as,

```
5
```

dbGetLibAttachedFile

Returns the library attached file name.

Syntax

```
dbGetLibAttachedFile libId attName
```

Arguments

libId - The ID of the library. Integer value

attName - The name of the library attached file. String value

Description

If the attached file exists, the dbGetLibAttachedFile function returns the full path of the attached file.

Otherwise, it returns false (#f).

Example

```
dbGetLibAttachedFile 1 "lib_1"
```

This function returns the full path of the attached file for library ID 1, for example,

```
/net/gorp/jane_doe/RGB/lib_1
```

If the attached file is not found, false (#f) is returned.

dbGetLibBusNameStyle

Returns the bus naming style of a library.

Syntax

```
dbGetLibBusNameStyle libName
```

Arguments

libName

The name of the library. String value

Description

This returns the bus naming style of a library if the bus naming style is saved in the library property: "Bus Naming Style".

Example

```
dbGetLibBusNameStyle "RGB"
```

This might return for example the bus naming style,

```
_<%d>
```

dbGetLibDbVersion

Returns the specified library's database version number.

Syntax

```
dbGetLibDbVersion <string library_name>
```

where the argument is as follows:

library_name

Name of the library.

Description

The `dbGetLibDbVersion` function returns the specified library's database version number.

Example

```
dbGetLibDbVersion "myLib"
```

dbGetLibIdByName

Returns the specified library's object ID.

Syntax

```
dbGetLibIdByName <string fullLibName>
```

where the argument is as follows:

`fullLibName`

Directory path and name of the library. Relative paths are resolved with respect to the current working directory. If no path is specified, the library should be located in the current working directory.

Description

The `dbGetLibIdByName` function returns the specified library's object ID. The library should be opened, otherwise an error is returned.

Example

```
dbGetLibIdByName "/home/userA/myLib"
```

dbGetLibNameById

Returns the library name of the specified library.

Syntax

```
dbGetLibNameById <objectId libId>
```

libId - Database object ID of an opened library

Description

The dbGetLibNameById function returns the library name of the specified library. The return value is a string or a boolean false (#f) if an error occurred.

Example

```
dbGetLibNameById 1
```

The above function call might return the following string:

```
"mylib"
```

dbGetLibraryPath

Returns the specified library's directory path.

Syntax

```
dbGetLibraryPath <objectId libId>
```

where the argument is as follows:

libId

Database object ID of the library.

Description

The `dbGetLibraryPath` function returns the specified library's directory path. The library should be opened, otherwise an error is returned.

Example

```
dbGetLibraryPath (dbGetCurrentLibId)
```

dbGetLibRefCtrlFileMode

Returns the current setting of the specified library's "Reference Control File Mode" flag.

Syntax

```
dbGetLibRefCtrlFileMode <string libName>
```

`libName` - Name of a library

Description

The `dbGetLibRefCtrlFileMode` function returns the current setting of the specified library's "Reference Control File Mode" flag. This flag is a boolean and determines which library reference tree is used. If the flag is `#t`, the reference control file is used. If the flag is `#f`, the internal reference structure is used.

See [dbSetLibRefCtrlFileMode](#) and [dbSetRefLibControl](#) for more information about the reference control file and mode.

Note: This flag overrides the general (non-library specific) flag accessed by the `dbSetRefControlFileMode` function and `dbGetRefControlFileMode` functions.

Example

```
dbGetLibRefCtrlFileMode "mylib"
```

The above function call might return the following boolean value:

```
#t
```

dbGetLModelSubType

Displays the logic model subtype of the specified cell.

Syntax

```
dbGetLModelSubType <objectId cellId>
```

cellId - Object database ID of an opened cell

Description

The dbGetLModelSubType function displays the logic model subtype of the specified cell. The subtype is displayed to the scheme window, but it is not returned by the function.

See dbSetLModelSubType for more information about the logic model subtype.

Example

```
dbGetLModelSubType (geGetEditCell)
```

The above function call might display the following:

The cell's lModelSubType is latch.

dbGetMaxMetalRoutingLayer

Allows you to check the maximum metal routing layer in the metal routing layers.

Note:

The default value of the maximum metal routing layer will be set during floorplanning to the highest metal in the technology file.

Use only three or four layers of metal in a five- or six-layer library.

Syntax

```
dbGetMaxMetalRoutingLayer cellId
```

where the argument is as follows:

cellId

Database object ID of cell to which net information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

Example

The following checks the maximum metal routing layer:

```
(dbSetMaxMetalRoutingLayer (geGetEditCell) 4)
(dbGetMaxMetalRoutingLayer (geGetEditCell))
-> should return 4
```

dbGetMsgLevel

Displays the current message level.

Syntax

```
dbGetMsgLevel
```

Description

The `dbGetMsgLevel` command displays the current message level. The return value is high, medium, or low.

Example

```
dbGetMsgLevel
```

If `dbSetMsgLevel` has never been issued, `dbGetMsgLevel` returns the following default message level:

```
medium
```

dbGetNetVarRouteRuleName

Fetches the via routing rule name given to the net ID.

Tool

Milkyway

Syntax

```
dbGetNetVarRouteRuleName cellId netId
```

where the arguments are as follows:

<i>cellId</i>	The ID of the cell where you want to retrieve the net.
<i>netId</i>	The ID of the net from which you want to retrieve the via routing rule name.

dbGetObjectTypeName

Returns the type name of the specified object.

Syntax

```
dbGetObjectTypeName <objectId fileId> <objectId objId>
```

fileId - Database object ID of an opened library or cell containing the object

objId - Database object ID of the object for which to obtain the type name

Description

The `dbGetObjectTypeName` function returns the type name of the specified object. The return value is an empty string if the name is not available, an empty list if an error occurs, or a string such as "CellName", "Polygon", "Rectangle", "WireMaster", and so on.

Example

```
dbGetObjectTypeName (geGetEditCell) #x500
```

The above function call might return the following string:

"Rectangle"

dbGetOneLibCellInfo

Displays useful information about the specified cell in the specified library.

Syntax

```
dbGetOneLibCellInfo "library_name" "cell_name"
```

where *library_name* is the name of an existing library and *cell_name* is the name of an existing cell in the library.

dbGetPinNetByName

Returns the net to which the specified pin is connected.

Syntax

```
dbGetPinNetByName <objectId cellId> <string pinName>
```

cellId - Database object ID of an opened cell containing the pin

pinName - Name of the pin for which to obtain the net

Description

The dbGetPinNetByName function returns the net to which the specified pin is connected. The return value is the database object ID of the net or a boolean false (#f) if an error occurs. If more than one pin exists with the same name, a warning message is displayed and the net of the first pin is returned.

Example

```
dbGetPinNetByName (geGetEditCell) "vdd"
```

The above function call might return the following database object ID:

```
1795
```

dbGetPlanGroupName

Returns the database object ID of the specified floorplan group.

Syntax

```
dbGetPlanGroupName <objectId cellId>  
<string planGroupName>
```

cellId - Database object ID of the cell containing the floorplan group

planGroupName - Name of the floorplan group

Description

The dbGetPlanGroupName function returns the database object ID of the specified floorplan group. If the floorplan group does not exist, an ID of 0 is returned.

Example

```
dbGetPlanGroupName (geGetEditCell) "myPlan"  
3408
```

dbGetRefControlFileMode

Returns the current global reference control file mode.

Syntax

```
dbGetRefControlFileMode <void>
```

Description

The `dbGetRefControlFileMode` returns the current global reference control file mode. This returns the value of the global run-time mode set by `dbSetRefControlFileMode`.

If you need the value of the library-specific mode set by `dbSetLibRefCtrlFileMode`, use `dbGetLibRefCtrlFileMode`.

Example

```
dbGetRefControlFileMode  
#f
```

See Also

[dbSetRefControlFileMode](#)
[dbSetLibRefCtrlFileMode](#)

dbGetRefLib

Returns the list of reference libraries of the specified library.

Syntax

```
dbGetRefLib <string libName>
```

`libName` - Name of the library

Description

The `dbGetRefLib` function returns the list of reference libraries of the specified library. The list contains the entire library hierarchy that is created when the library is opened. Each library is listed by their full path.

The format of the return list is as follows:

((refList) ...)

where `refList = libName ( refName ... )`

Example

```
dbGetRefLib "main"
( ("/home/main" ("/home/ref1" "/home/ref2")) ("/home/ref1"
("/home/ref3" "/home/ref4")) ("/home/ref2" ("/home/ref5"))
("/home/ref3" ("/home/ref6")))
```

dbGetTolerantMode

Returns the current “Netlist Fault Tolerant Mode” setting.

Syntax

```
dbGetTolerantMode <void>
```

Description

The `dbGetTolerantMode` command returns the current “Netlist Fault Tolerant Mode” setting. This setting is used by the netlist import and update commands (for example, `geECODumpDiff`, `auNewECOByNetCmp`, `auVerilogIn`, `auFastVerilogIn`, and `auVerilogToCell`).

If the mode setting is #t, the netlist commands create and connect all net objects and allow you to continue your design flow, although the netlist may be incomplete or erroneous. The erroneous net objects and connections are tagged, and can be fixed or deleted.

Example

```
dbGetTolerantMode
```

dbGetUserHierDelimiter

Returns the user-specified hierarchical name delimiter.

Syntax

```
dbGetUserHierDelimiter <void>
```

Description

The `dbGetUserHierDelimiter` function returns the user-specified hierarchical name delimiter (also known as the hierarchical name separator). This is set during library creation (see `dbCreateLib` and `cmCreateLib`).

Example

```
dbGetUserHierDelimiter  
"/"
```

dbGetVarRouteRule

Gets information of variable-width route rules.

Tool

Milkyway

Syntax

```
dbGetVarRouteRule cellId ruleName
```

where the arguments are as follows:

<i>cellId</i>	ID of any cell in the open library.
<i>ruleName</i>	Name of rule from which you want to get the information.

Example

```
dbGetVarRouteRule (geGetEditCell) "VWR1"  
'( (width "METAL1" 6.00) (width "METAL2" 7.00)  
  (spacing "METAL1" 6.50) (contact "VIA12" 2 1) )
```

In this example, the command fetches the information of varRouteRule 'VWR1'.

dbHierCopyToExistLib

Copies the top cell and its associated child cells into the existing library.

Syntax

```
dbHierCopyToExistLib topCellName existlibName  
["check_only"]
```

Arguments

topCellName - Top cell to copy

`existlibName` - Target libraryOptional argument. If specified, prints list, but does not perform actual operation for copy operation

`check_only` - Optional argument. If specified, prints list, but does not perform actual operation.

Description

The `dbHierCopyToExistLib` command operates similarly to the `dbHierCopyToNewLib` command except that it requires that the destination library exist.

This command does not create a new library and then copy information from the source library. Therefore before using this command, you need to make sure that both the source library and destination library share the same unit tile, technology information, library cell table information, library attached information, and reference library path.

Compared to the `dbHierCopyToNewLib` command, this command reduces the CPU time and uses less disk space. If the library remains the same, it is preferable for repeated saves to use the `dbHierCopyToExistLib` command instead of the `dbHierCopyToNewLib` command.

If a destination library does not already exist, the `dbHierCopyToExistLib` command terminates with an error message.

The largest version of any opened and unsaved cell is saved from memory into the destination library. If the cell is opened but is not the largest version, the cell is copied from disk instead of memory.

If there are any unopened cells, Milkyway copies from the disk to the destination library.

The new copied cell includes all information from the original cell, including from the CEL, PWR, and TIM views. Copying all information facilitates running-block level design concurrently using the copied library.

When you specify the `check_only` option, the command prints out the list of cells, views, and versions but does not copy information to the library.

See Also

[dbHierDelete](#)

[dbHierCopyToNewLib](#)

[dbHierSave](#)

[dbHierSplit](#)

[dbHierPurge](#)

dbHierCopyToNewLib

Copies a library and its contents to a new library.

Syntax

```
dbHierCopyToNewLib topCellName newlibName ["check_only"]
```

Arguments

`topCellName` - Top cell to copy

`newlibName` - Target library for hierarchical copy

`check_only` - Optional argument. If specified, prints list, but does not perform actual operation.

Description

The `dbHierCopyToNewLib` command creates a new library and copies information from the existing library to this new library. This information includes unit tile, technology information, library attached file information, and reference library path.

If a destination library already exists, the command terminates with an error message.

The largest version of any opened and unsaved cell is saved from memory into the destination library. If the cell is opened but is not the largest version, the cell is copied from disk instead of memory.

If the cell is unopened, the cell in the disk is copied.

The new copied cell includes all information from the original cell, including from the CEL, PWR, and TIM views. Copying all information facilitates running block-level design concurrently using the copied library.

When you specify the `check_only` option, the command prints out the list of cells, views, and versions but does not copy information to the new library.

See Also

[dbHierCopyToExistLib](#)

[dbHierDelete](#)

[dbHierSave](#)

[dbHierSplit](#)

[dbHierPurge](#)

dbHierDelete

Deletes top and child cells in all views and versions, including the cell's attached files.

Syntax

```
dbHierDelete libName topCellName ["check_only"]
```

Arguments

libName - Library of top cell

topCellName - Top cell for hierarchical deletion

check_only - Optional argument. If specified, prints list, but does not perform actual operation.

Description

The `dbHierDelete` command deletes top and child cells in all views and versions, including the cell's attached files.

The difference between `dbHierDelete` and `dbHierPurge` is that `dbHierDelete` removes all the cells and the associated attached files for a given top cell in a specified library; `dbHierPurge` retains the current version and removes all previous versions.

When you specify the `check_only` option, the command prints out the list of cells, versions, and views and does not purge any data. If you are unsure of the result of this command, run it with the `check_only` option first.

See Also

[dbHierCopyToNewLib](#)

[dbHierCopyToExistLib](#)

[dbHierSave](#)

[dbHierSplit](#)

[dbHierPurge](#)

dbHierPurge

Purges all views and versions of the top cell and instantiated child cells, except for the current version.

Syntax

```
dbHierPurge libName topCellName ["check_only"]
```

Arguments

libName - Library of top cell

topCellName - Top cell for hierarchical purge

check_only - Optional argument. If specified, prints list, but does not perform actual operation.

Description

The `dbHierPurge` command retains the current version only and removes the top cell, child cells, and attached files in all views and versions for all previous versions. When completed, it lists the names of cells, versions, and views removed from Milkyway. This command is useful for cleaning up older versions.

The difference between `dbHierPurge` and `dbHierDelete` is that `dbHierPurge` retains the current version; `dbHierDelete` removes all cells and associated attached files for a given top cell in a specified library.

When you specify the `check_only` option, the command prints out the list of cells, versions, and views and does not purge any data. If you are unsure of the result of this command, run it with the `check_only` option first.

See Also

[dbHierDelete](#)

[dbHierCopyToNewLib](#)

[dbHierCopyToExistLib](#)

[dbHierSave](#)

[dbHierSplit](#)

dbHierSave

Saves the opened and modified top cell, as well as instantiated child cells. Milkyway saves all cells as the same version. This command also saves all the cell's attached files.

Syntax

```
dbHierSave libName topCellName ["check_only"]
```

Arguments

`libName` - Library of top cell

`topCellName` - Top cell for hierarchical save

`check_only` - Optional argument. If specified, prints list, but does not perform actual operation.

Description

The `dbHierSave` command saves the opened top cell, associated child cells, and all attached files into the same version. When the command finishes copying, it prints out the list of saved cell names.

This command does not perform a close-cell operation. Therefore any open cells remain open after the hierarchical save.

When you specify the `check_only` option, the command prints out the list of cells to be saved but does not save the cells.

The `dbHierSave` and `dbHierCopyToNewLib` commands require an open library and top cell for modification.

See Also

[dbHierCopyToNewLib](#)
[dbHierCopyToExistLib](#)
[dbHierDelete](#)
[dbHierSplit](#)
[dbHierPurge](#)

dbHierSplit

Splits the top cell into different libraries. Splitting the library hierarchically facilitates concurrent design.

Syntax

```
dbHierSplit libName topCellName ["check_only"]
```

Arguments

`libName` - Library of top cell

`topCellName` - Top cell for hierarchical split

`check_only` - Optional argument. If specified, prints list, but does not perform actual operation.

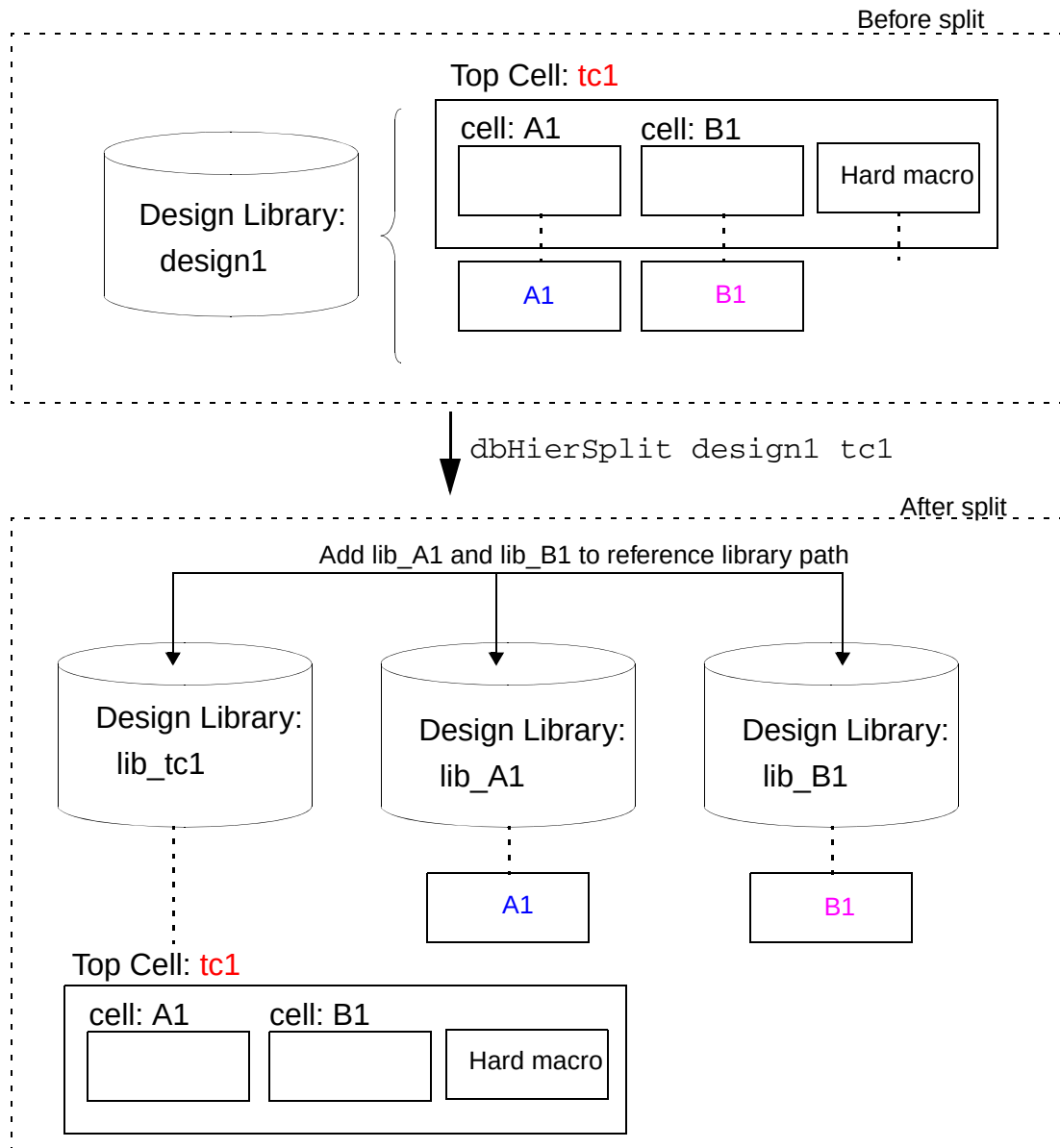
Description

The `dbHierSplit` command creates new libraries in the working directory. Each new library created has a single cell.

All new libraries inherit the library information from the original library, including the technology file and reference library path.

When you specify the `check_only` option, the command prints out the list of libraries and cells to be created (or overwritten) but does not perform the split. If you are unsure of the result of this command, run it with the `check_only` option first.

The following figure shows the top cell before and after `dbHierSplit` is run:



After the hierarchical split, the new libraries lib_tc1, lib_A1, and lib_B1 are created in the working directory. If any destination library exists already, the dbHierSplit command terminates with an error

message. Each new library except the top library contains a single cell. For example, the design library lib_A1 contains the cell A1. The design library lib_B1 contains the cell B1.

The top cell library, lib_tc1, adds lib_A1 and lib_B1 to its highest-priority reference library path. Therefore opening the top cell in lib_tc1 allows you to access the information in cell A1 from lib_A1 and in cell B1 from lib_B1.

All new libraries inherit the library information (unit tile, technology information, library attached information, reference library path, and so on) from the original library.

The new cells (A1 and B1) include all information from the original library, design1. This includes information in the CEL, PWR, and TIM views. Therefore after splitting, you can perform block-level design concurrently on the resultant libraries.

If the top cell contains cells other than a soft macro, such as a hard macro or standard cell that are located in the design library, these macro or standard cells are copied to the destination top cell library to maintain the original top cell structure. No other cells from the reference library are copied to the top cell.

If a destination library already exists, the command terminates with an error message.

See Also

[dbHierCopyToNewLib](#)
[dbHierCopyToExistLib](#)
[dbHierDelete](#)
[dbHierSave](#)
[dbHierPurge](#)

dbId?

Identifies whether or not a variable is a database object ID.

Syntax

```
dbId? variable
```

where *variable* can be any variable you need to identify as being a database object ID or not.

See Also

[->type](#)

dbImportLogicData

Imports a logic .db file into the library.

Syntax

```
dbImportLogicData <string libraryName> <string dbName>  
                  <string cellName> [boolean otherType]
```

where the arguments are as follows:

libraryName

Name of the library into which to import.

dbName

Name of the logic .db file to import.

cellName

Name of the design cell to which the logic .db file is attached.

otherType

Indicates if the logic .db file should be stored as an “other” logic file or the “design” logic file.

Description

The `dbImportLogicData` command imports a logic .db file into the library. The logic .db file is attached to the specified design cell as either an “other” logic file or the “design” logic file.

If the `otherType` argument is `#t`, the logic .db file is stored as an “other” logic file. These “other” logic files are stored in the library but are not used. There can be multiple “other” logic files per design cell in the library, but none are used until they are set as the “design” logic file.

If the `otherType` argument is `#f`, the logic .db file is stored as the “design” file. There can be only one “design” logic file per design cell at any moment. Once a new “design” logic file is set, the previous “design” logic file becomes an “other” logic file.

See also the `geCreateLogicView` command.

Example

```
dbImportLogicData "myLib" "design.db" "design" #f
```

dbImportModelData

Imports model .db files into the library.

Syntax

```
dbImportModelData <string libraryName> <string dbNames>  
                  <int fileType>
```

where the arguments are as follows:

libraryName

Name of the library into which to import.

dbNames

Names of the model .db files to import. If multiple names are specified, the string should be a space delimited list of .db files.

fileType

Indicates the operating condition to which the model .db files are stored.

Valid values:

- 0 minimum operating condition
- 1 maximum operating condition
- 2 typical operating condition
- 3 other operating condition

Description

The `dbImportModelData` command imports model .db files into the library. The files are stored as model .db files used for either the minimum, maximum, typical, or other operating condition.

There can only be one .db file list for each of the minimum, maximum, and typical operating conditions. When setting the minimum, maximum, or typical operating condition .db files, the previous files are moved to an other operating condition, and the new files are set to the specified operating condition.

However, there can be multiple other operating conditions, each having a different list of .db files. Therefore, it is not necessary to move .db files when setting an other operating condition.

See also the `gePrepLibs` command.

Example

```
dbImportModelData "refLib" "refLib1.db refLib2.db  
refLib3.db" 0
```

dbInitShapeStruct

Initializes the in-memory shape structure for the specified cell.

Syntax

```
dbInitShapeStruct <objectId cellId>
```

`cellId` - Database object ID of an opened cell

Description

The `dbInitShapeStruct` function initializes the in-memory shape structure for the specified cell. The shape structure is a quad-tree representation of the cell that allows for efficient region-based queries. The `dbQuitShapeStruct` function removes and frees the shape structure.

The shape structure requires significant additional memory overhead and is not automatically synchronized with changes in the database. Therefore, the database should not be modified while the shape structure is in use (after `dbInitShapeStruct` and before `dbQuitShapeStruct` is called). The correct usage model is

```

;; modify the database
dbInitShapeStruct cellId
dbQueryObjectByExtendDist cellId obj1 spacing
dbQueryObjectByExtendDist cellId obj2 spacing2
dbQueryObjectByExtendDist cellId obj3 spacing3
...
;; do NOT modify the database
....
dbQuitShapeStruct cellId
;; modify the database

```

Example

```
dbInitShapeStruct (geGetEditCell)
```

dbIOPinTextToPin

Creates a scheme file that will convert the specified cell's IO pin text into pin objects.

Syntax

```
dbIOPinTextToPin <objectId cellId> <int textLayer>
int pinLayer><string fileName>
```

cellId - Database object ID of the cell

textLayer - Layer number containing the pin text

pinLayer - Layer number on which to create the pins

fileName - Name of the scheme file to create

Description

The `dbIOPinTextToPin` function creates a scheme file that will convert the specified cell's IO pin text into pin objects.

For each text object found on the specified text layer, the scheme file will create a pin with the same name as the text string, at the location of the text string, and on the specified pin layer.

The pin will have the minimal dimensions allowed on the specified pin layer. The pin will be connected to the net with the same name. If the net does not exist, then the net is created as well.

Example

```
dbIOPinTextToPin (geGetEditCell) 51 31 "textToPin.scm"
```

dbIsCaseSensitive

Returns the current setting of the specified library or cell's "Case Sensitive" flag.

Syntax

```
dbIsCaseSensitive <objectId fileId>
```

fileId - Database object ID of the library or cell, or 0

Description

The `dbIsCaseSensitive` function returns the current setting of the specified library or cell's "Case Sensitive" flag. The return value is a boolean value (#t or #f).

If the argument, `fileId`, is 0, the setting of the currently opened main library is returned. The following two command calls are equivalent:

```
dbIsCaseSensitive (dbGetCurrentLibId)
dbIsCaseSensitive 0
```


Example

```
dbIsCaseSensitive (dbGetCurrentLibId)
```

The above function call may return the following boolean value:

```
#t
```

dbIsCellOpen

Checks to see if the specified cell is open.

Syntax

```
dbIsCellOpen <objectId libId> <string cellName>
```

where the arguments are as follows:

libId

Database object ID of the top library.

cellName

Name of the cell.

Description

The `dbIsCellOpen` function checks to see if the specified cell is open. If it is open, the database cell ID is returned. If the cell is not open, then 0 is returned. The cell must exist in either the top library or one of the libraries in the reference tree.

Example

```
dbIsCellOpen (dbGetCurrentLibId) "design"
```

dbLastECONum

Retrieves the last ECO number of a specified cell.

Tool

Milkyway

Syntax

```
dbLastECONum cellId
```

where the argument is as follows:

<i>cellId</i>	The ID of the cell where you want to retrieve the last ECO number.
---------------	--

dbLibraryCompare

Compares and reports the differences between two libraries.

Syntax

```
dbLibraryCompare <string libName1> <string libName2>  
[string outputFile] [int detailLevel]
```

libName1 - Name of the library to compare.

libName2 - Name of the library to compare.

fileName - Name of the output report file (optional). If not specified, the report is displayed to the Milkyway scheme window.

detailLevel - Indicates the amount of detail to compare (optional). The default setting compares the library's with the greatest detail.

Note: The arguments, `libName1` and `libName2`, must correspond to two unique libraries.

Description

The `dbLibraryCompare` function compares and reports the differences between two libraries.

This function compares object checksum statistics, not individual objects. For example, to compare contact code cut widths of two library files, it adds up all the contact code cut widths of the first cell, then adds up all the contact code cut widths of the second library, and finally compares the two totals. If the totals are equal, the contact code cut widths are equivalent. Otherwise, they are different.

This comparison method, tallying of checksum numbers, provides a quick and efficient means of finding differences. However, it cannot provide absolute certainty that two libraries are equivalent. It also cannot provide detailed descriptions about the differences such as which contact code is different or which layer name changed.

The granularity of the checksum comparison is determined by the detail level you specify. The detail level can be 1 or 2 (default is 2). If you specify a detail level of 1, the function gathers and compares the total counts of all database objects.

Specifying a detail level of 2, the function gathers and compares the fields of the database objects in addition to the total counts. Also if you specify detail level 2, the function provides more information as well as a more thorough comparison, at a cost of longer runtimes.

If no differences are found, the message "No differences were found" appears in the report. Otherwise the function outputs messages indicating the differences in the report. Note that "report" refers to all messages generated by the function. The user can either dump the report to a file or to the Milkyway scheme window.

If a file name is specified, the report is written to the file. Otherwise the report is displayed in the Milkyway scheme window (and the Milkyway log file).

Example

```
dbLibraryCompare "lib1" "lib2" "compare.txt" 1
```

The above function call might generate the following in a text file named "compare.txt" (Note that the detail level is set to 1):

```
*****
LIBRARY COMPARISON REPORT:
  Library 1: /home/lib1
  Library 2: /home/lib2
*****
No differences were found.
```

dbLibReportTiming

Used for reporting the library cell timing information.

Tool

Milkyway

Syntax

```
dbLibReportTiming libName fileName
```

where the arguments are as follows:

<i>libName</i>	The name of a valid Milkyway library.
<i>fileName</i>	The name of the output file to which you want the information written. Valid values: Any string up to 32 characters.

Example

```
dbLibReportTiming "cb18" "cb18.tim"
```

dbListPcellDefinitions

Displays a list of parameter cell defined in the specified library.

Syntax

```
dbListPcellDefinitions [objectId libId]
```

libId - Database object ID of the library (optional)

Description

The `dbListPcellDefinitions` function displays a list of parameter cells defined in the specified library. The list includes the parameter cells' name, type, and description string.

If the argument, `libId`, is not specified, then the information is retrieved from the currently opened main library.

Example

```
dbListPcellDefinitions (dbGetCurrentLibId)
```

dbListUnitRange

Displays unit names, parasitic and physical attributes, and the ranges of their values associated with the main library.

Syntax

```
dbListUnitRange "library_name"
```

where *library_name* is the name of the main library.

dbLoadPropertyDef

Loads the specified struct property definition file.

Syntax

```
dbLoadPropertyDef <string propDefFile>
```

where the argument is as follows:

propDefFile

Name of the property definition file.

Description

The `dbLoadPropertyDef` command loads the specified struct property definition file. This is provided as a transitional tool for those who have defined properties in the `./etc/misc/dbProperty.def` file and have not transitioned to the new `dbRegisterProperty` command to define struct properties.

See also the `dbRegisterProperty` and `dbDumpPropertyTable` commands.

Example

```
dbLoadPropertyDef "/installation/etc/misc/dbProperty.def"
```

dbMakeScanChainByCellInst

Creates a scan chain containing specified cell instances in the open cell.

Tool

Milkyway

Prerequisites

Indicate whether ports on scan cells are input ports or output ports using [dbSetCellPortTypes](#).

Syntax

```
dbMakeScanChainByCellInst (geGetEditCell) chainName  
'(srcInstName srcPortName)' (destInstName destPortName)  
'({instName} ...)
```

where the arguments are as follows:

<i>chainName</i>	Name of the scan chain you are creating. Valid values: Any string up to 31 characters.
<i>srcInstName</i>	Name of the source cell instance in the scan chain. Valid values: Name of any scan cell instance in the open cell, or a null string ("") to indicate that the source port is a top-level port.
<i>srcPortName</i>	Name of the source port of the scan chain. Valid values: Name of any top-level port or any output port on the specified source cell instance.

<i>destInstName</i>	Name of the destination cell instance in the scan chain. Valid values: Name of any scan cell instance in the open cell.
<i>destPortName</i>	Name of the destination port of the scan chain. Valid values: Name of any input port on the specified destination cell instance.
<i>instName</i>	Name of a cell instance you want included in the scan chain. Valid values: Name of any scan cell instance in the open cell.

dbMakeScanChainByMaster

Creates a scan chain containing all cell instances, except those specifically excluded, of a specified master cell.

Note:

`dbMakeScanChainByMaster` will accept “two scan out” ports and will issue a warning message when it encounters a “two scan out” port situation.

Tool

Milkyway

Prerequisites

Indicate whether ports on scan cells are input ports or output ports using [dbSetCellPortTypes](#).

Syntax

```
dbMakeScanChainByMaster (geGetEditCell) chainName  
  ' (srcInstName srcPortName) ' (destInstName destPortName)  
  ' ({masterName} ...) ' ({instName} ...)
```

where the arguments are as follows:

<i>chainName</i>	Name of the scan chain you are creating. Valid values: Any string up to 31 characters.
<i>srcInstName</i>	Name of the source cell instance in the scan chain. Valid values: Name of any scan cell instance in the open cell, or a null string ("") to indicate that the source port is a top-level port.
<i>srcPortName</i>	Name of the source port of the scan chain. Valid values: Name of any top-level port or any output port on the specified source cell instance.
<i>destInstName</i>	Name of the destination cell instance in the scan chain. Valid values: Name of any scan cell instance in the open cell.
<i>destPortName</i>	Name of the destination port of the scan chain. Valid values: Name of any input port on the specified destination cell instance.
<i>masterName</i>	Name of a master cell for which you want to add cell instances to the scan chain. Valid values: Name of any master scan cell with instances in the open cell.

instName

Name of an instance of a specified master cell that you want to exclude from the scan chain.

Valid values:

Name of any instance of any of the specified master cells.

dbMakeWireMasterId

Composes a real object ID from a cell ID and a wire's internal wire master ID. The database uses *internWireMasterId* when storing wires to save space in the database. When fetching a wire's master ID, the real object ID of the master must be composed with this function.

Syntax

```
dbMakeWireMasterId cellId layer internWireMasterId
```

where the arguments are as follows:

cellId

ID of the cell from which the wire is selected

Valid values:

- Cell ID of any open cell in the current library. If the cell is open, you can substitute the following for *cellId*
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

layer

Number of the layer for which you want to fetch the wire master ID. If you do not specify a layer, this command returns the wire master ID in the first 256 created wire masters, regardless of the layer.

Valid values:

Routing layer

internWireMasterId

The wire's internal wire master ID

Example)

```
define wireMasterId (dbMakeWireMasterId (geGetEditCell)
(dbFetchObjectField (geGetEditCell)
(car (geGetSelectedSet 1))
"internWireMasterId")
)
db:18b0002
```

In this example, you compose a wire master ID (assuming a wire is selected) and assign the ID to the `wireMasterId` variable.

To get information about the wire master, use [dbFetchObject](#).

```
dbFetchObject (geGetEditCell) wireMasterId
(("filler" 0) ("layerNumber" 5) ("wireWidth" 200)
("status" 0) ("pathType" 0) ("layerType" 0)
("recordType" "WireMaster"))
```

The `dbFetchObject` function lists all the information contained in the wire master record.

dbMarkCellType

Sets the layout cell subtype for the specified cell.

Syntax

```
dbMarkCellType libName cellName layoutSubType
```

where the arguments are as follows:

libName

Name of the library that contains the cell

Valid values:

Any existing library name and path

cellName

Name of the cell for which you want to set a layout cell subtype

Valid values:

Any existing cell name in the library

layoutSubType

Layout cell subtype

Valid values:

A string in the set {"padCell", "cornerCell", "moduleCell", "standardCell", "macroCell", "imageCell", "coverCell", "otherCell"}

Example

```
dbMarkCellType "libName" "cellName" "macroCell"  
dbMarkCellType  
"/home/user/libname" "cellName" "imageCell"
```

dbMarkScanPortPairs

This command allows you to mark pairs of scan ports for all instances of a particular cell master. This allows scan trace the ability to trace through cell instances in the scan chain with multiple pairs of the scan ports defined, e.g., macros.

Tool

Milkyway

Syntax

```
dbMarkScanPortPairs (geGetEditCell) cellMasterName  
  `(("Scan In" "Scan Out")...) append?
```

where the arguments are as follows:

<i>cellMasterName</i>	The name of the cell master for which you would like to dump the marked scan port pairs.
<i>ScanIn</i>	The name of the scan in port in the pair.
<i>ScanOut</i>	The name of the scan out port in the pair.
<i>append?</i>	Whether or not the information should be appended to information specified by previous calls of the dbMarkScanPortPairs command. Valid values: #t means that Astro should append any existing information with the new information. #f means that Astro should replace any existing information with the new information.

Example

```
dbMarkScanPortPairs (geGetEditCell) "macrocell" `(("si1"
"so1")
("si2" "so2")) #f
```

dbMemProfile

Starts and ends the memory profiling utility.

Syntax

```
dbMemProfile <boolean tf>
```

where the argument is as follows:

tf

#t to start memory profiling.

#f to end memory profiling.

Description

The `dbMemProfile` command starts and ends the memory profiling utility. Start and end the memory profiling utility for the commands to be profiled, then call the `dbMemProfileReport` command to report their peak memory usage.

Example

```
dbMemProfile #t
...
dbMemProfile #f
dbMemProfileReport "peak_mem.rpt"
```

dbMemProfileReport

Reports the peak memory usage.

Syntax

```
dbMemProfileReport <string fileName>
```

where the argument is as follows:

fileName

File to which the report is written.

Description

The `dbMemProfileReport` command reports the peak memory usage. Use the `dbMemProfile` command to start and end the memory profiling utility for the commands to be profiled, then call the `dbMemProfileReport` command to report their peak memory usage.

Example

```
dbMemProfile #t
...
dbMemProfile #f
dbMemProfileReport "peak_mem.rpt"
```

dbMergeLib

Copies all cell versions from one source library to one destination library.

dbMergeLib gives you three ways to “merge” two libraries:

1. Merge by appending new cells to the destination library

2. Merge by appending new cells and overwriting existing cells in the destination library
3. Merge by prompting you to choose whether to overwrite or skip existing cells in the destination library.

Note:

The term “library” refers to the top library only and only cells stored in the top library are affected by this command. It does not include the reference libraries or their cells.

The source library and the destination library cannot be the same. They must be two physically distinct libraries on the file system. If the paths to these libraries are different, but contain symbolic links to the same library, *dbMergeLib* will print an error message and fail. If the libraries are physically distinct but contain the exact same data, *dbMergeLib* will continue successfully.

If the source library does not exist, *dbMergeLib* will print an error message and fail. If the destination library does not exist, it will be created and the source library will be directly copied to the destination library.

The source library will not be modified by *dbMergeLib*.

All transactions will be described in the tool log file. The description will include the following:

- Source library name
- Destination library name
- Cell name
- An indication of whether the cell was appended, overwritten, or skipped (not modified)

Append Mode: If you choose to append the cells, the cells that already exist in the destination library will not be modified in any way. Only cells that exist in the source library and don't exist in the destination library will be copied from the source to the destination library.

Overwrite Mode: If you choose to overwrite the cells in the destination library, cells that exist in both the source and the destination library will be deleted from the destination library. All cells from the source library will be copied to the destination library.

Prompt Mode: If you choose to be prompted before copying, a dialog box will appear and ask whether the cell should be copied or skipped.

If you choose to copy, the existing cell in the destination library will be deleted, and the cell from the source library will be copied to the destination library.

If you choose to skip, the existing cell in the destination library will not be modified or deleted.

The dialog box will only appear for cells that exist in both libraries. All other cells will be automatically copied to the destination library without any prompting.

Tool

Milkyway

Syntax

```
dbMergeLib "sourceLibrary" "destinationLibrary" "mergeMode"
```

where the arguments are as follows:

<i>sourceLibrary</i>	The source library. Valid values: Any library name that exists in the current working directory, any relative path and library name that exists relative to the current working directory, or any absolute path and library name that exists.
<i>destinationLibrary</i>	The destination library. Valid values: Any library name, any relative path and library name relative to the current working directory, or any absolute path and library name.
<i>mergeMode</i>	The method used to merge the two libraries. Valid values: “Append” “A” - append mode. “Overwrite” “O” - overwrite mode. “Prompt” “P” - prompt mode. Return values: #t - TRUE on success. #f - FALSE on failure.

dbMoveAllCellRegions

Moves all cell regions of a cell to a location relative to their original locations.

Syntax

```
dbMoveAllCellRegions <objectId cellId> <list point>
```

where the arguments are as follows:

cellId - Database object ID of an opened cell

point - Coordinate pair representing a relative location to which to move each cell region relative to its original location

Description

The `dbMoveAllCellRegions` function moves all cell regions of a cell to a location relative to their original locations. All of the physical objects within the region are moved as well, including cell rows, instances, pins, paths, wires, contacts, and contact arrays.

Example

```
dbMoveAllCellRegions (geGetEditCell) `(100.50 -200.20)
```

The above function call moves all cell regions to the right by 100.50 and to the bottom by 200.20.

dbMoveAllNetMember

Moves all members from one net to another. Region ports and region pins are not modified.

Note:

Because this function ignores the parent-child net relationship, do not use this function after you generate region pins for multiple-region designs in Astro.

Syntax

```
dbMoveAllNetMember cellId fromNetId toNetId
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- `(geGetEditCell)`

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

fromNetId

Database object ID of the net from which the information is to be copied

Valid values:

- Net ID of any net in the specified cell
- `(dbGetNetByName cellId "netName")`

For more information, see [dbGetNetByName](#).

toNetId

Database object ID of the net to which the information is to be copied

Valid values:

- Net ID of any net in the specified cell
- `(dbGetNetByName cellId "netName")`

For more information, see [dbGetNetByName](#).

Example 1

The code in this example merges all member of net Logic0 into net VSS.

```
define cell1 (dbOpenCell "control.Net1" "w")
db:2
dbMoveAllNetMember cell1 (dbGetNetByName cell1 "Logic0")
(dbGetNetByName cell1 "VSS")

dbCloseCell cell1
#t
```

In this example,

- The first line opens the cell control.Net1 and assigns the object ID to cell1.
- The second lines merges all members of net Logic0 into net VSS.
- The third line saves the changes you made and closes cell1.

Example 2

The code in this example merges tiehi/tielo nets into powerNetName and groundNetName.

```

define libraryName "designs"
"designs"
define expandedCellName "top.Net1"
"top.Net1"
define powerNetName "VDD"
"VDD"
define groundNetName "VSS"
"VSS"
dbOpenLib libraryName
db:1
define cell1 (dbOpenCell expandedCellName "w")
db:2
dbMoveAllNetMember cell1 (dbGetNetByName cell1 "Logic1")
(dbGetNetByName cell1 powerNetName)
dbMoveAllNetMember cell1 (dbGetNetByName cell1 "Logic0")
(dbGetNetByName cell1 groundNetName)
dbCloseCell cell1
#t
dbCloseLib
#t

```

In this example,

- The first four lines create global variables.
- The fifth line opens the library called designs.
- The sixth line opens the expanded cell top.Net1.
- The seventh line merges the tiehi nets from net Logic1 to powerNetName.
- The eighth line merges the tielo nets from net Logic0 to groundNetName.
- The ninth line saves the changes you made and closes cell1.
- The tenth line saves and closes the library designs.

Note:

If the nets VDD and VSS did not exist prior to this example, you could delete lines three and four and substitute the following lines for lines seven and eight:

```
dbMoveAllNetMember cell11 (dbGetNetByName cell11 "Logic1")
(dbCreateNet cell11 powerNetName)
dbMoveAllNetMember cell11 (dbGetNetByName cell11 "Logic0")
(dbCreateNet cell11 groundNetName)
```

See Also

[dbCreateNet](#)

dbMoveCellOrigin

Changes the origin of a cell to a point you specify by coordinates. If successful, `dbMoveCellOrigin` returns *#t*.

Syntax

```
dbMoveCellOrigin cellId origin
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the origin information applies

Valid values:

- Cell ID of any open cell in the current library
- (`geGetEditCell`)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

origin

Origin of the cell

Valid syntax:

' (x y)

dbNetChecksum

Calculates and generates a net checksum report.

Syntax

```
dbNetChecksum < objectId cellId > [ string fileName ]
```

where the arguments are as follows:

cellId - Database object ID of an opened cell for which to report checksum information

fileName - Name of the output report file (optional argument). If not specified, the report is written to the Milkyway scheme window.

Prerequisites

The library containing the specified cell must be opened prior to calling the dbNetChecksum function.

Description

The `dbNetChecksum` function calculates and generates a net checksum report file. The report is a text file listing checksum information about the cell's netlist.

For string data, the value is calculated by assigning each alphanumeric character a numeric value and tallying up the total value.

The function compiles the following information:

Net information sorted by net type

- Total count
- Total length of all net names
- Total value of all net names

Net component information sorted by object type

- Total count
- Totals of all net component information

Port information sorted by port type

Total count
Total count of all port pins
Total length of all port names
Total value of all port names

Path information sorted by layer

Total count
Total width of paths
Total height of paths

Path information sorted by route type

Total count
Total width of paths
Total height of paths

Horizontal wire information sorted by layer

Total count

Total width of horizontal wires
 Total height of horizontal wires
 Horizontal wire information sorted by route type
 Total count
 Total width of horizontal wires
 Total height of horizontal wires
 Vertical wire information sorted by layer
 Total count
 Total width of vertical wires
 Total height of vertical wires
 Vertical wire information sorted by route type
 Total count
 Total width of vertical wires
 Total height of vertical wires
 GLink information sorted by layer
 Total count
 Total width of glinks
 Total height of glinks
 GLink information sorted by route type
 Total count
 Total width of glinks
 Total height of glinks
 Rectangle information sorted by layer
 Total count
 Total width of rectangles
 Total height of rectangles
 Rectangle information sorted by route type
 Total count
 Total width of rectangles
 Total height of rectangles
 Polygon information sorted by layer
 Total count
 Total width of polygons

- Total height of polygons
- Contact information sorted by contact code index
 - Total count
 - Total width of contact
 - Total height of contact
- Contact information sorted by route type
 - Total count
 - Total width of contacts
 - Total height of contacts
- Contact array information sorted by contact code index
 - Total count
 - Total width of contact arrays
 - Total height of contact arrays
- Contact array information sorted by route type
 - Total count
 - Total width of contact arrays
 - Total height of contact arrays

Example

```
dbNetChecksum (geGetEditCell) "checksum.txt"
```

The above function call generates as an example the following in a text file named “checksum.txt”:

NET CHECK SUM REPORT

CELL NAME: top

NETS

***** -----

SORTED BY NET TYPE

NET TYPE	COUNT	NAME LENGTH	NAME VALUE
POWER	2	6	78
GROUND	1	3	57
OTHER	77325	4881073	128564732
TOTAL	77328	4881082	128564867

NET COMPONENTS

SORTED BY COMPONENT TYPE

COMPONENT TYPE	COUNT
PORTS	884
PORT INSTANCES	219679
PATHS	334
HORIZONTAL WIRES	1548
VERTICAL WIRES	862
CONTACTS	8
CONTACT ARRAYS	21403
OTHER	1566
TOTAL	246284

PORTS

SORTED BY PORT TYPE

PORT TYPE	COUNT	PIN COUNT	NAME LENGTH	NAME VALUE
NORMAL	881	881	15515	163636
POWER	2	421	6	78

GROUND	1	264	3	57

TOTAL	884	1566	15524	163771

PORT INSTANCES

***** -----

----- SORTED BY PORT TYPE

PORT TYPE	COUNT	PIN COUNT	NAME LENGTH	NAME VALUE

NORMAL	219679	220293	376860	3420270

TOTAL	219679	220293	376860	3420270

PATHS

----- SORTED BY LAYER

LAYER	COUNT	SUM-X	SUM-Y

10	19	4457875	97675
12	10	43525	1836825
14	27	4482825	157500
16	21	58750	37600
18	257	93956925	10129875

TOTAL	334	102999900	12259475

----- SORTED BY ROUTE TYPE

ROUTE TYPE	COUNT	SUM-X	SUM-Y

P/G PIN CONNECT	334	102999900	12259475

TOTAL	334	102999900	12259475

HORIZONTAL WIRES

SORTED BY LAYER			
LAYER	COUNT	SUM-X	SUM-Y
10	827	674247500	1935600
12	21	209750	48350
14	508	74713925	1632100
16	2	201275	18950
18	190	10354125	922700
TOTAL	1548	759726575	4557700

SORTED BY ROUTE TYPE			
ROUTE TYPE	COUNT	SUM-X	SUM-Y
P/G STRIPE	85	110609225	633000
P/G PIN CONNECT	656	30475450	2325100
P/G FOLLOW PIN	807	618641900	1599600
TOTAL	1548	759726575	4557700

 VERTICAL WIRES

SORTED BY LAYER			
LAYER	COUNT	SUM-X	SUM-Y
10	58	184400	566825
12	502	1330300	150614375
14	122	356050	265650
16	180	632650	5544275
TOTAL	862	2503400	156991125

SORTED BY ROUTE TYPE			
ROUTE TYPE	COUNT	SUM-X	SUM-Y
P/G STRIPE	337	774300	146481875

P/G PIN CONNECT	525	1729100	10509250
TOTAL	862	2503400	156991125

GLINKS

SORTED BY LAYER			
LAYER	COUNT	SUM-X	SUM-Y
TOTAL	0	0	0

SORTED BY ROUTE TYPE			
ROUTE TYPE	COUNT	SUM-X	SUM-Y
TOTAL	0	0	0

RECTANGLES

SORTED BY LAYER			
LAYER	COUNT	SUM-X	SUM-Y
TOTAL	0	0	0

SORTED BY ROUTE TYPE			
ROUTE TYPE	COUNT	SUM-X	SUM-Y
TOTAL	0	0	0

POLYGONS

SORTED BY LAYER

LAYER	COUNT	SUM-X	SUM-Y
TOTAL	0	0	0

CONTACTS

SORTED BY CONTACT CODE INDEX

INDEX	COUNT	SUM-X	SUM-Y
3	2	500	500
5	2	1000	1000
6	2	900	900
7	2	900	900
TOTAL	8	3300	3300

SORTED BY ROUTE TYPE

ROUTE TYPE	COUNT	SUM-X	SUM-Y
P/G PIN CONNECT	8	3300	3300
TOTAL	8	3300	3300

CONTACT ARRAYS

SORTED BY CONTACT CODE INDEX

INDEX	COUNT	SUM-X	SUM-Y
1	10685	9766100	19237150
2	343	2116250	2210100
3	435	2577100	1369900
4	319	2726400	1456800

5	4782	37102200	33880600
6	3109	10622600	16976800
7	641	2312450	3721350
8	1089	6472200	4322100

TOTAL	21403	73695300	83174800

SORTED BY ROUTE TYPE			

ROUTE TYPE	COUNT	SUM-X	SUM-Y

SIGNAL USER	113	511200	203400
P/G STRIPE	4584	13361600	36908950
P/G PIN CONNECT	3916	21718250	21930050
P/G FOLLOW PIN	12790	38104250	24132400

TOTAL	21403	73695300	83174800

dbObjId?

Checks if a ID is a valid object ID.

Syntax

```
dbObjId? fileId, objId
```

Arguments

fileId - The ID of a library or cell in which to check a valid object ID.

objId - The ID to be checked if it's a valid object ID. Integer value

Description

The dbObjId? command returns true if there is an object whose ID is objId in the specified library or cell. Otherwise, it returns false.

Example

```
dbObjId? (dbGetCurrentLibId) 513
```

dbOpenCell

Opens the specified cell in read (R) or write (W) mode. If `dbOpenCell` succeeds, the Synopsys tool returns the database object ID of the cell and opens the cell in the specified mode.

Prerequisite

The library containing the cell you want to open must be open.

Syntax

```
dbOpenCell "cellName" "mode"
```

where the arguments are as follows:

cellName

Name of the cell to open

Valid values:

Name of any cell in the current library

mode

Indicates whether you can edit the cell

Valid values:

- *r*, which opens the cell for reading only
- *w*, which opens the cell for read and write capabilities

See Also

[dbAbortCell](#)
[dbChangeAccessMode](#)
[dbCloseCell](#)
[dbRenameCell](#)

dbOpenLib

Opens a library for editing. If `dbOpenLib` successfully opens the library, the Synopsys tool returns the database object ID of the library.

Note:

Do not reopen an already open library by using `dbOpenLib`. If you do, you will need to close the library twice.

Prerequisite

No libraries can be open for editing. A Synopsys tool supports only one library open for editing.

Syntax

```
dbOpenLib "libName"
```

where *libName* is the name of the library you want to open.

See Also

[dbCloseLib](#)

dbOpenLibrary

Opens a library in read (R) or write (W) mode. If `dbOpenLibrary` successfully opens the library, the Synopsys tool will display the database object ID of the library.

Note:

When you are opening a library in write mode, it is better to use [dbOpenLib](#). Unlike `dbOpenLib`, `dbOpenLibrary` does not specify the database object ID of the library globally. When other commands, such as [dbOpenCell](#), do not request a library ID, the following error message might appear:

```
"ERROR: No library opened."
```

For more information, see “Specifying Libraries” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

Syntax

```
dbOpenLibrary "libName" "mode"
```

where the arguments are as follows:

libName

Name of the library you want to open

Valid values:

Name of any library in the current directory

mode

Indicates whether a library can be edited

Valid values:

- *r* opens the library specified by *libName* for read-only.

- *w* opens the library specified by *libName* for read and write capabilities.

See Also

[dbCloseLib](#)

[dbOpenLib](#)

dbOpenQueryFile

Opens an output file for the output from [dbQuery](#).

Syntax

```
dbOpenQueryFile " fileName "
```

where the arguments are as follows:

fileName

Name of the file to open

Valid values:

Name of any query file in the current directory

dbOpenRefLibTree

Opens all reference libraries of a design library.

Syntax

```
dbOpenRefLibTree libId
```

Arguments

libId - The ID of the design library. Integer value

Description

The reference libraries of a design are organized as a hierarchical tree based on their referencing relations. The `dbOpenRefLibTree` command opens all reference libraries for a given design library.

Example

```
dbOpenRefLibTree (dbGetCurrentLibId)
```

dbPadToPin

Converts pad cell instances to pins in a design cell.

Syntax

```
dbPadToPin cellId
```

Arguments

cellId - The ID of the design cell. Integer value

Description

The `dbPadToPin` command converts pad cell instances, each of which has only one rectangle pin, to pins of the cell. The cell boundary of the cell is also reset as result of the conversion process.

Example

```
dbPadToPin (geGetEditCell)
```

dbPinToPad

Converts pins of a design cell to pad cells.

Syntax

```
dbPinToPad cellId
```

Arguments

cellId - The ID of the design cell. Integer value

Description

The `dbPinToPad` command converts pins of the design cell to pad cells and pad cell instances of the given design. The cell boundary is also reset as a result of the conversion process.

Example

```
dbPinToPad (geGetEditCell)
```

dbPinToPadAndSnap

Converts pins of a design cell to pad cells and locates new pad cell instances as specified by coordinate and distance values.

Syntax

```
dbPinToPadAndSnap cellId xOffset xStep yOffset yStep
```

Arguments

cellId - The ID of the design cell. Integer value

xOffset - The x-coordinate offset to locate new pad cell instances. Double precision value

xStep - The distance to locate new pad cell instances in x-coordinate. Double precision value

yOffset: - The y-coordinate offset to locate new pad cell instances.
Double precision value

yStep - The distance to locate new pad cell instances in y-coordinate. Double precision value

Description

The `dbPinToPadAndSnap` command converts pins of a design cell to pad cells and locates new pad cell instances as specified by coordinate and distance values. The cell boundary is also reset as a result of the conversion process.

Example

```
dbPinToPadAndSnap (geGetEditCell)12.3 2.2 3.4 5.0
```

dbPrintAllNetClasses

Displays all net class information in the message area.

Tool

Milkyway

Syntax

```
dbPrintAllNetClasses cellId
```

where the argument is as follows:

cellId

The ID of the cell containing the net classes you for which you want to display information.

Valid values:

ID of any cell in the open library. If the cell is open, you can substitute the following for *cellId*: `(geGetEditCell)`

Example

```
dbPrintAllNetClasses (geGetEditCell)
```

This example displays information for all net classes in the cell.

See Also

[dbFormNetClass](#) creates a net class in the cell.

[dbDeleteNetClass](#) deletes a net class from the cell.

[dbQueryNetClassByName](#) displays the name of the nets in a specified net class.

dbPrintApplParam

The command prints out all members of a property for a given cell.

Syntax

```
dbPrintApplParam cellId propertyName
```

Arguments

cellId - The cell ID of the given cell. Integer value

propertyName - The name of the property group. String value

Description

If there exists a property named propertyName in a given cell, this command reports all members of this property. Otherwise, an invalid property name message is displayed.

In the contents of this command, a property means a set of application parameters. Each application parameter is a member of the property.

For example, assume there is a property named myPlacement that has the members myUnitTile , myTILE1, myTILE2, and so on. If a cell has property myPlacement with members such as those above, running the command:

```
dbPrintApplParam cellId "myPlacement"
```

prints out myUnitTile, myTILE1, myTILE2 and so on with their respective values:

```
myUnitTile 5.000000  
myTILE1 3.000000  
myTILE2 8.000000
```

Example

```
dbPrintApplParam (geGetEditCell) "paramName"
```

dbPrintCellHierReport

Prints out the cell hierarchy of a given cell to a text file.

Syntax

```
dbPrintCellHierReport libId, cellName, viewName, fileName
```

Arguments

libId - The library ID of the cell. Integer value

cellName - The name of the cell. String value

viewName - The view name of the cell. String value

fileName - The text file name for print out. String value

Description

This command prints the cell hierarchy to a text file that is specified in the command argument fileName. The library of the cell needs to be opened for the function to operate.

Example

```
dbPrintCellHierReport (dbGetCurrentLibId) "top" "NETL"  
"top.netl.hier.txt"
```

The following is an example output file.

```
==== Cell Hierarchy Dump Report ====  
Date : Thu Jul 24 11:07:21 2003  
Working Directory : /desktop/u805-02/sort_2  
  
Format : <instance name> (<master name>)  
  
top  
  
....buf0 (buffd1)  
....buf1 (buffd1)  
....buf2 (buffd1)  
  
....(module) buffd1.FRAME  
  
.....buf0 (buffd1)  
  
.....(module) buffd1.NETL  
  
....sub1 (sub)  
  
....(module) sub.NETL  
  
.....buf1 (buffd1)  
.....buf2 (buffd1)  
.....buf3 (buffd1)  
  
.....(module) buffd1.FRAME  
  
.....buf0 (buffd1)  
  
.....(module) buffd1.NETL
```

dbPurgeAllEmptyNet

Purges all empty nets in the specified cell.

Syntax

```
dbPurgeAllEmptyNet cellId
```

where the argument is as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

dbPurgeAllPlanGroup

Purges all plan groups in the open cell.

Syntax

```
dbPurgeAllPlanGroup cellId
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

Licence Required

Milkyway

dbPurgeAllSingleNet

Purges all single-port (or port-instance) nets in the specified cell.

Syntax

```
dbPurgeAllSingleNet cellId
```

where the argument is as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

dbPurgeCellInstByName

Purges a cell instance record from the database.

Syntax

```
dbPurgeCellInstByName cellId "cellInstName"
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

cellInstName

Name of the cell instance record to purge

Valid values:

Name of any cell instance in the specified cell

dbPurgeCellInstMaster

Purges a cell instance master record from the database. A side effect of this function is to purge all cell instances of this master.

Syntax

```
dbPurgeCellInstMaster cellId "cellInstMasterName"
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

cellInstMasterName

Name of the cell instance master record to purge

Valid values:

Name of any cell instance in the specified cell

dbPurgeCTSPin

This timing command deletes the CTS specific information on the specified port. You identify input ports as CTS pins using [dbDefineCTSPin](#).

Tool

Milkyway

Syntax

```
dbPurgeCTSPin cellId "cellInstName"
```

where the arguments are as follows:

<i>cellId</i>	ID of the top-level cell containing the cell instance for which you are purging CTS pin information. Valid values: ID of any top-level cell in the current directory.
<i>cellInstName</i>	Name of the cell instance for which you are purging CTS pin information. Valid values: A string that is the name of any cell instance in the top-level cell, or "*" to indicate all cell instances in the top-level cell.

Example

```
dbPurgeCTSPin (geGetEditCell) "INST1"
```

This example instructs Astro to delete CTS specific information defined for the ports of the cell instance INST1.

See Also

[dbDefineCTSPin](#) identifies an input port on a cell instance as a CTS pin and specifies the CTS level and number.

[dbDumpCTSPin](#) writes a file containing information about input ports set as CTS pins during clock tree synthesis.

dbPurgeIgnorePin

This timing command deletes the ignore port property on the ports of the specified cell instance. You identify input ports as ignore pins using [dbDefineIgnorePin](#).

Tool

Milkyway

Syntax

```
dbPurgeIgnorePin cellId "cellInstName"
```

where the arguments are as follows:

<i>cellId</i>	ID of the top-level cell containing the cell instance for which you are purging ignore port information. Valid values: ID of any top-level cell in the current directory.
<i>cellInstName</i>	Name of the cell instance for which you are purging ignore port information. Valid values: A string that is the name of any cell instance in the top-level cell, or "*" to indicate all cell instances in the top-level cell.

Example

```
dbPurgeIgnorePin (geGetEditCell) "INST1"
```

This example instructs Astro to delete the ignore port property on all ports of the cells instanced INST1.

See Also

[dbDefineIgnorePin](#) identifies an input port on a cell instance as an ignore pin.

[dbDumpCTSPin](#) writes a file containing information about input ports set as ignore pins during clock tree synthesis.

dbPurgeNet

Purges a specified net record from the database.

Syntax

```
dbPurgeNet cellId netId
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

netId

Database object ID of the net to purge

Valid values:

Net ID of any net in the specified cell

dbPurgeNetByName

Purges a specified net record from the database.

Syntax

```
dbPurgeNetByName cellId "netName"
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

netName

Name of the net to purge

Valid values:

Name of any net in the specified cell

dbPurgeRefLib

Purges all reference libraries of a given library.

Syntax

```
dbPurgeRefLib libId
```

Arguments

libId - The ID of the library to purge reference libraries. Integer value

Description

This command purges all reference libraries of a library for a given Id. The library needs to be opened for the function to operate.

Example

```
dbPurgeRefLib (dbGetCurrentLibId)
```

dbPurgeLibStaticNoiseMargin

Deletes the noise margin values stored in a specified library.

Tool

Milkyway

Syntax

```
dbPurgeLibStaticNoiseMargin "Library Name"
```

where the argument is as follows:

<i>Library Name</i>	Name of the library for which you are purging noise margin values.
---------------------	--

Example

```
dbPurgeLibStaticNoiseMargin "LIB1"
```

This example deletes the noise margin values from the library LIB1.

dbPurgeLibStaticNoiseProperty

Deletes the noise margin values stored in the library with the name "library name."

Tool

Milkyway

Syntax

```
dbPurgeLibStaticNoiseProperty "library name"
```

where the argument is as follows:

<i>library name</i>	Name of the library for which you are purging noise margin values.
---------------------	--

dbPurgeObject

Purges an object from the database. Serves the same function as [dbDeleteObject](#).

Syntax

```
dbPurgeObject cellId objectId
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

Cell ID of any cell in the current library

objectId

Database object ID of the object to be deleted

Valid values:

Object ID of any object in the specified cell, such as the following:

- *netId*
- *portId*
- *portInstId*

dbPurgePin

Purges a pin record from the database.

Syntax

```
dbPurgePin cellId pinId
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

pinId

Database object ID of the pin to purge

Valid values:

Pin ID of any pin in the specified cell

dbPurgePort

Purges a port record from the database.

Syntax

```
dbPurgePort cellId portId topCellName
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

portId

Database object ID of the port to purge

Valid values:

Port ID of any port in the specified cell

topCellName

Database object ID of the top-level design cell

Valid values:

ID of the top cell

dbPurgeRegionPin

Purges a region pin record from the database.

Syntax

```
dbPurgeRegionPin cellId regionPinId
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

regionPinId

Database object ID of the region pin to purge

Valid values:

Region pin ID of any port in the specified cell

dbPurgeRegionPort

Purges a region port record from the database.

Syntax

```
dbPurgeRegionPort cellId regionPortId
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

regionPortId

Database object ID of the region port to purge

Valid values:

Region port ID of any port in the specified cell

dbPurgeSyncPin

This timing command changes input ports previously identified as sync points to nonsynchronous ports. You identify input ports as sync points using [dbDefineSyncPin](#).

Syntax

```
dbPurgeSyncPin cellId "cellInstName"
```

where the arguments are as follows:

<i>cellId</i>	ID of the top-level cell containing the cell instance for which you are purging sync point information. Valid values: ID of any top-level cell in the current directory.
<i>cellInstName</i>	Name of the cell instance for which you are purging sync point information. Valid values: A string that is the name of any cell instance in the top-level cell, or "*" to indicate all cell instances in the top-level cell.

Example

```
dbPurgeSyncPin (geGetEditCell) "INST1"
```

This example changes input ports for cell instance INST1 to nonsynchronous.

See Also

[dbDefineSyncPin](#) identifies an input port on a cell instance as a sync point and specifies minimum and maximum phase delays for this port at different phases.

[dbDumpSyncPin](#) writes a files containing information about input ports to be synchronized during clock tree synthesis.

dbPurgeUnusedWireMasters

Deletes all unused wire masters of a given cell.

Syntax

```
dbPurgeUnusedWireMasters cellId
```

Arguments

libId - The ID of the cell to delete unused wire masters. Integer value

Description

This command deletes all unused wire masters of a given cell. The cell needs to be opened for the function to operate.

Example

```
dbPurgeUnusedWireMasters (geGetEditCell)
```

dbQuery

Prints the contents of the record.

Syntax

```
dbQuery libraryOrCellId "recordName"
```

where the arguments are as follows:

libraryOrCellId

Database object ID of the cell or library to which the information applies

Valid values:

- Cell ID of any cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

- Library ID of the library currently open in write (W) mode
- (`dbGetCurrentLibId`)

For more information, see “Specifying Libraries” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

recordName

Name of the record to print

Valid values:

Name of any record in the specified library or cell

dbQueryCPUTime

Returns the total CPU time used by the tool.

Syntax

```
dbQueryCPUTime <void>
```

Description

The `dbQueryCPUTime` command returns the total CPU time used by the tool. The total includes all CPU time used while executing instructions in the user space of the main process and its child processes and that used by the system on behalf of the main process and its child processes.

Example

```
dbQueryCPUTime
```

dbQueryMem

Reports the memory usage.

Syntax

```
dbQueryMem outputFileName
```

Arguments

outputFileName - The file name for outputting memory usage. String value

Description

This command prints out the memory usage to a text file.

Example

```
dbQueryMem "dump_mem.txt"
```

The following is an example output:

```
Memory used = 100.29M
```

dbQueryNetClassByName

Displays the names of the nets in a specified net class in the message area. Net classes can have other net classes nested as members.

Tool

Milkyway

Syntax

```
dbQueryNetClassByName cellId "className"
```

where the arguments are as follows:

<i>cellId</i>	ID of the cell containing the net class you want to query. Valid values: ID of any cell in the open library. If the cell is open, you can substitute the following for <i>cellId</i> : (geGetEditCell).
<i>className</i>	Name of the net class you want to query.

Example

```
dbQueryNetClassByName (geGetEditCell) "A"
```

This example displays the names of the members of net class A, including nested net classes and individual nets.

See Also

[dbFormNetClass](#) creates a net class in the cell.

[dbDeleteNetClass](#) deletes a net class from the cell.

[dbPrintAllNetClasses](#) displays the names of all net classes in a cell.

dbQueryObjectByExtendDist

Returns an object ID list of objects that exist within the specified query area.

Syntax

```
dbQueryObjectByExtendDist <objectId cellId> <objectId  
objId>  
<double extensionDist>
```


cellId - Database object ID of an opened cell with an initialized shape structure

objId - Database object ID of the object whose bounding box determines the query area

extensionDist - Extension distance by which to enlarge the query area (i.e. bounding box area of the object) in all four directions

Description

The `dbQueryObjectByExtendDist` function returns an object ID list of objects that exist within the specified query area. All objects completely or partially enclosed by the query area are returned.

The query area is defined as the bounding box of the specified object enlarged by the specified extension distance. The query area is enlarged uniformly on all four sides by the specified amount (that is the total width and height are each enlarged by two times (2X) the extension distance).

The specified cell must have an initialized shape structure. Changes made to the database after initialization are not reflected in the query. See `dbInitShapeStructure` and `dbQuitShapeStructure` for additional information.

Example

```
dbQueryObjectByExtendDist (geGetEditCell) #x00FF 200.00
```

If the bounding box of the object specified by `#x00FF` has a lower-left coordinate of (0, 0) and an upper-right coordinate of (1000, 1000), the query area has a lower-left coordinate of (-200, -200) and an upper-right coordinate of (1200, 1200). The above function call returns a list of all objects enclosed by this query area, such as

(43456 34574 33451 43542 32435 41325 42132)

dbQueryPeakMem

Returns the amount of memory (in megabytes) allocated to the tool.

Syntax

```
dbQueryPeakMem <void>
```

Description

The `dbQueryPeakMem` command returns the amount of memory (in megabytes) that is allocated to the tool.

Example

```
dbQueryPeakMem
```

dbQueryPeriodPeakMem

Returns the peak memory used, in megabytes, during the query period.

Syntax

```
dbQueryPeriodPeakMem <void>
```

Description

The `dbQueryPeriodPeakMem` command returns the peak memory used, in megabytes, during the query period. The query period includes the time between the last call to the `dbQueryPeriodPeakMemBegin` command and the call to this command.

This command may be called multiple times to report the peak memory used since the last call to the `dbQueryPeriodPeakMemBegin` command.

Example

```
dbQueryPeriodPeakMemBegin  
dbQueryPeriodPeakMem
```

dbQueryPeriodPeakMemBegin

Starts the peak memory query period.

Syntax

```
dbQueryPeriodPeakMemBegin <void>
```

Description

The `dbQueryPeriodPeakMemBegin` command starts the peak memory query period. Call the `dbQueryPeriodPeakMem` command to return the peak memory used, in megabytes, since the last call to the `dbQueryPeriodPeakMemBegin` command.

Example

```
dbQueryPeriodPeakMemBegin  
dbQueryPeriodPeakMem
```

dbQuitShapeStruct

Removes and frees the in-memory shape structure for the specified cell.

Syntax

```
dbQuitShapeStruct <objectId cellId>
```

cellId - Database object ID of an opened cell

Description

The `dbQuitShapeStruct` function removes and frees the in-memory shape structure for the specified cell. The shape structure is a quad-tree representation of the cell that allows for efficient region-based queries. The `dbInitShapeStruct` function initializes the shape structure.

The shape structure requires significant additional memory overhead and is not automatically synchronized with changes in the database. Therefore, the database should not be modified while the shape structure is in use (after `dbInitShapeStruct` and before `dbQuitShapeStruct` is called). The correct usage model is

```
;; modify the database
dbInitShapeStruct cellId
dbQueryObjectByExtendDist cellId obj1 spacing
dbQueryObjectByExtendDist cellId obj2 spacing2
dbQueryObjectByExtendDist cellId obj3 spacing3
...
;; do NOT modify the database
....
dbQuitShapeStruct cellId
;; modify the database
```

Example

```
dbQuitShapeStruct (geGetEditCell)
```

dbRebuildLib

Rebuilds a library (.lib file) by scanning all the cells in the library directory. This command does not create a library property or attached files

Syntax

```
dbRebuildLib "libName"
```

where the argument is as follows:

libName

Name of the library

dbRecoverLib

Opens a corrupted library and returns its database object ID.

Syntax

```
dbRecoverLib <string libName> <string techName>
```

libName - Name of the library

techName - Name of the technology file (argument is not used)

Description

The `dbRecoverLib` function opens a corrupted library and returns its database object ID. If the specified library is corrupt and cannot be opened by the usual means (that is using `dbOpenLib` or `geOpenLib`), this function opens the library anyway. If the library is not corrupt, this function fails.

Example

```
dbRecoverLib "myCorruptedLib" "tech.tf"
```

The above function call returns 1 if “myCorruptedLib” library is corrupt.

dbRecursiveUpdateAllHierarchy

Operates in the cell hierarchy and propagates any changes to top-level cells. It updates only the cells that have time stamps later than the top cell. It is similar to [dbUpdateHierarchy](#), except that `dbUpdateHierarchy` works only on the top cell's immediate children.

Syntax

```
dbRecursiveUpdateAllHierarchy cellId
```

where the arguments are as follows:

cellId

Database object ID of the cell you want to update

Valid values:

- Cell ID of any open cell in the current library
- `(geGetEditCell)`

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

Example

Cell A has an instance B, and the master cell of B has an instance C. If the master cell in C has changed, you must run `dbRecursiveUpdateAllHierarchy` to propagate the change to the top level for cells that have a time stamp later than the top cell.

dbRefLibChecksum

Calculates and generates a library checksum report.

Syntax

```
dbRefLibChecksum <string libName> <string cellName>  
[string fileName]
```

libName - Name of the library for which to report checksum information.

cellName - Name of the top design cell or "*".

fileName - Name of the output report file (optional argument). If not specified, the report is written to the Milkyway scheme window.

Description

The `dbRefLibChecksum` function calculates and generates a library checksum report file. The report is a text file listing checksum information about the library's technology, cell catalog, and the cell masters referenced by the specified top design cell.

If the top design cell is the wildcard ("*") character, the cell masters of all cells within the library are included in the checksum.

For string data, the value is calculated by assigning each alphanumeric character a numeric value and tallying up the total value.

The function compiles the following information:

- Total length of all library object names
- Technology unit name and precision information
- Technology slot rule information
 - Total number of slot rules

- Total of threshold width and length values
- Total of minimum slot width and length values
- Total of maximum slot width and length values
- Total of minimum side space and clearance values
- Total of maximum side space and clearance values
- Total of minimum end space and clearance values
- Total of maximum end space and clearance values
- Total of maximum metal density values
- Technology stipple information
 - Total number of stipples
 - Total of stipple width and height values
- Technology tile information
 - Total number of tiles
 - Total of tile width and height values
- Technology design rule information
 - Total number of design rules
 - Total of minimum spacing values
 - Total of minimum enclosure values
 - Total of different-net minimum spacing values
 - Total of end-of-line enclosure values
 - Total number of stackable design rules
- Technology layer information
 - Total number of layers
 - Number of master slice layers
 - Number of overlap layers
 - Number of cut layers
 - Number of routing layers
 - Total of preferred wire direction values
 - Total of layer width values
 - Total of layer spacing values
- Technology contact code information
 - Total number of contact codes
 - Number of default contact codes

- Total of resistance values
- Total of geometry width and height values
- Site information
 - Number of gate sites
 - Number of core and pad sites
 - Number of sites with symmetry about the x-axis, about the y-axis, and when rotated 90 degrees

(Site information is obtained from the library's site objects, unless the top design cell is specified and contains its own set of site objects.)

- Cell master information of the specified top design cell
 - Number of cell masters
 - Number of standard cells
 - Number of macro cells
 - Number of corner and pad cells
 - Number of cover cells
 - Total of cell width values
 - Total of cell height values
 - Number of port masters
 - Number of pin masters
 - Number of diode ports
 - Number of input pins
 - Number of output pins
 - Number of inout pins
 - Number of tristate pins
 - Number of power/ground pins
 - Number of obstruction layers
 - Number of geometries (physical objects)
 - Total of geometry width values
 - Total of geometry height values
- Variable route rule information
 - Total number of rule sets

- Total value of all rule set names
- Total number of width rules
- Total value of all width rule layer names
- Total sum of all width rule widths
- Total number of space rules
- Total value of all space rule layer names
- Total sum of all space rule spacings
- Total number of contact rules
- Total value of all contact rule contact code names
- Total sum of all contact rule array widths
- Total sum of all contact rule array heights
- Total number of taper rules
- Total sum of all taper rule levels

Example

```
dbRefLibChecksum "myLib" "*" "checksum.txt"
```

The above function call generates the following in a text file named "checksum.txt":

Reference Library CheckSum

All Object Names CheckSum : 42816

Technology Section

Dielectric : 3.45e-05
Time Unit Name : ns
Time Unit Precision : 1000
Length Unit Name : micron
Length Unit Precision : 100
Grid Resolution : 1
Resistance Unit Name : kohm
Resistance Unit Precision : 1000000
Capacitance Unit Name : pf
Capacitance Unit Precision : 100000000
Inductance Unit Name : nh
Inductance Unit Precision : 100

Slot Rule Section

Total Number of Slot Rule : 0
Total widthThreshold : 0.0000
Total lengthThreshold : 0.0000
Total minSlotWidth : 0.0000
Total maxSlotWidth : 0.0000
Total minSlotLength : 0.0000
Total maxSlotLength : 0.0000
Total minSideSpace : 0.0000
Total maxSideSpace : 0.0000
Total minSideClearance : 0.0000
Total maxSideClearance : 0.0000
Total minEndSpace : 0.0000
Total maxEndSpace : 0.0000
Total minEndClearance : 0.0000
Total maxEndClearance : 0.0000
Total maxMetalDensity : 0

Stipple Section

Total Number of Stipple : 1
Total Stipple Width : 1.0000
Total Stipple Height : 1.0000

Tile Section

dbRefLibCheckSum

Total Number of Tile : 1
Total Tile Width : 0.5600
Total Tile Height : 5.0400

Design Rule Section

Total Number of Design Rule : 11
Total minSpacing : 1.1600
Total minEnclosure : 0.0000
Total diffNetMinSpacing : 0.0000
Total endOfLineEnclosure : 0.0000
Total Number of Stackable : 3

Layer Section

Total Number of Layers : 17
Number of MasterSlice Layers : 1
Number of Overlap Layers : 0
Number of Cut Layers : 8
Number of Routing Layers : 8
Horizontal/Vertical Direction : 4 / 4
Total Layer Width : 476.0000
Total Layer Spacing : 472.0000

Via Section

Total Number of Vias : 38
Number of Default Vias : 7
Total Resistance : 0.2343
Total Via Geometry Width : 12102.0000
Total Via Geometry Height : 9126.0000

Site Section

Number of Gate Sites : 0
Core/Pad : 0/0
X/Y/R90 : 0/0/0

Cell Section

Number of Cell Masters : 2
Number of Standard Cells : 0
Number of Macro Cells : 0
Number of Corner/Pad Cells : 0
Number of Cover Cells : 0
Total Cell Width : 0.0000
Total Cell Height : 0.0000
Number of Port Masters : 0

```

Number of Pin Masters      : 0
Number of Diode Ports     : 0
Number of Input Pins      : 0
Number of Output Pins     : 0
Number of InOut Pins      : 0
Number of Tristate pins   : 0
Power/Ground Pins        : 0 / 0
Number of Obs Layers      : 0
Number of Geometry        : 0
Total Geometry Width      : 0.0000
Total Geometry Height     : 0.0000

```

Variable Route Rule Section

```

-----
Number of Rule Sets       : 0
Rule Set Names CheckSum   : 0
Number of Width Rules     : 0
Width - Layer Names CheckSum : 0
Total Width               : 0
Number of Space Rules     : 0
Space - Layer Names Value : 0
Total Space               : 0
Number of Contact Rules   : 0
Contact Names CheckSum    : 0
Total Contact X           : 0
Total Contact Y           : 0
Number of Taper Rules     : 0
Total Taper Level         : 0

```

```

*****
      End of CheckSum Report
*****

```

dbRefreshLib

Restores the library on disk and keeps the library ID unchanged.

Syntax

```
dbRefreshLib libId
```

Arguments

libId: integer; the ID of the library to refresh.

Description

This command restores the version of a library on disk discarding any changes at run time. It keeps the library ID unchanged.

Example

```
dbRefreshLib 1
```

dbRegisterProperty

Registers a user-defined struct property in the run-time property definition table.

Syntax

```
dbRegisterProperty <string propertyName>  
                  <objectId ownerId> <string definition>
```

where the arguments are as follows:

propertyName

Name of the property.

ownerTypeId

Object type ID of the owning object. Valid values include integers between 1 and 1,029. Use 0 if the owner is unknown.

definition

A comma-delimited list defining the size of the struct property's fields. The elements of the list can be:

- An integer, describing the byte size of a built-in type.

For example:

1 --> 1-byte character, boolean or 8-bit field

2 --> 2-byte short integer 16-bit field

3 --> 3-byte 24-bit field
4 --> 4-byte float or integer
8 --> 8-byte double or long integer

- An integer followed by a pair of square brackets enclosing another integer, describing the byte size of an array. The first integer describes the array's element size. The number enclosed by the brackets describes the number of elements in an array.

For example:

2[4] --> an array of 4 2-byte short integers

4[8] --> an array of 8 4-byte integers

- An integer followed by a pair of square brackets enclosing the integer 1, describing the byte size of a variable sized array. Variable sized arrays can only appear at the end of the struct definition.

Description

The `dbRegisterProperty` command registers a user-defined struct property in the run-time property definition table. This information is used to perform byte swapping on Linux and other platforms which have different byte orders than that of the Sun and HP platforms.

Known limitations include:

- Unions are not supported.
- Array of structs are not supported. They need to be fully expanded and entered field by field.

- An 8-byte double type field in a structure needs to be aligned on an 8-byte boundary. If necessary, padding should be inserted explicitly. Otherwise, the property will not be cross-platform readable. Unlike SUN and HP, 32-bit Linux aligns 8-byte double on a 4-byte boundary by default.
- A multi-dimensional array needs to be converted to a one-dimensional format. For example, 4[12][3] --> 4[36].
- Only “byte fields” with sizes that are multiples of bytes are supported. Other bit fields with sizes less than 1 byte are not swapped.

Example

```
dbRegisterProperty "myPropStuct" "132" "1[32768], 4,
4[1024]"
```

dbRemoveInstGroupFromScanChain

Removes an instance group from a scan chain in the open cell.

Syntax

```
dbRemoveInstGroupFromScanChain cellId "chainName"
"instGroupName"
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell).

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

chainName

Name of the scan chain from which you want to remove the instance group

Valid values:

Name of any scan chain that contains an instance group in the open cell

instGroupName

Name of the instance group that you want to remove from the scan chain

Valid values:

Any valid name of an instance group in the open cell

dbRemoveObjectFromGroup

Removes an object from a group.

Syntax

```
dbRemoveObjectFromGroup cellId groupId objectId
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the group information applies

Valid values:

- Cell ID of any open cell in the current library

- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

groupId

Database object ID of the group from which the object is to be removed

Valid values:

Group ID of any group in the current cell

objectId

Database object ID of the object to be removed

Valid values:

Object ID of any member of the specified group, such as the following:

- *netId*
- *pinId*
- *portId*
- *portInstId*

dbRenameCell

Renames a cell. If `dbRenameCell` succeeds, the Synopsys tool returns *#t*.

Note:

This function does not create a copy of the specified cell. After execution of `dbRenameCell`, the original cell no longer exists.

Prerequisite

The cell you want to reload must be closed.

Syntax

```
dbRenameCell "oldCellName" "newCellName"
```

where the arguments are as follows:

oldCellName

Name of the existing cell you want to rename

Valid values:

Name of any cell in the current library

newCellName

Name you want to assign to the cell

Valid values:

Any text string of 31 characters or fewer

See Also

[dbCopyCell](#)

[dbCopyCellToAnotherLibDisk](#)

dbRenameInstAndNet

Renames all the cell instances and nets to meaningless numbers. This encrypts the names of all nets & cell instances to hide the design information.

Tool

Milkyway

Syntax

```
dbRenameInstAndNet cellId
```

where the argument is as follows:

<i>cellId</i>	ID of the cell you want to rename. Valid values: ID of any cell in the open library. If the cell is open, you can substitute the following for <i>cellId</i> : (<i>geGetEditCell</i>).
---------------	--

Example

```
dbRenameInstAndNet (geGetEditCell)
```

dbRenameLib

Renames a library. If `dbRenameLib` succeeds, the Synopsys tool returns `#t`.

After `dbRenameLib` renames your library, only the renamed library exists. If your renamed library is a reference library, the cells in the main library referencing cells in this library will be undefined. Replace the reference library of the old name with the renamed library, using `dbReplaceRefLibByName`.

Prerequisite

The library you want to rename must be closed.

Syntax

```
dbRenameLib "oldLibName" "newLibName"
```

where the arguments are as follows:

oldLibName

Name of the existing library to be renamed

Valid values:

Name of any library in the current directory

newLibName

Name of the new library to which data from the existing library moves

Valid values:

Any text string with fewer than 255 characters

Note:

The renamed library will not be open after you execute `dbRenameLib`.

See Also

[dbCreateLib](#)

[dbReplaceRefLibByName](#)

dbReOpenRefLibForWrite

Changes the access mode of a reference library from read-only to write.

Syntax

```
dbReOpenRefLibForWrite libId
```

Arguments

libId: integer; the ID of the main library of which the access mode of the reference library will be changed.

Description

Reference libraries usually are open for read-only. This command enables writing to a reference library by reopening it with the access mode set to write.

Example

```
dbReOpenRefLibForWrite (dbGetCurrentLibId)
#t
```

dbReplaceAllCellBoundary

Changes the boundary of all cells in a library to the size of a rectangle of a given layer.

Syntax

```
dbReplaceAllCellBoundary libId layerNumber
```

Arguments

libId: integer; the ID of the library to change boundary of cells.

layerNumber: integer; the layer number of the layer containing the rectangle used to resize the boundary of the cell.

Description

This command resets the boundary size of all cells in a library to the size of a rectangle in a given layer.

Example

```
dbReplaceAllCellBoundary 1 3
```

dbReplaceCellBoundary

Changes the boundary of a cell to the size of a rectangle of a given layer.

Syntax

```
dbReplaceCellBoundary cellId layerNumber
```

Arguments

cellId: integer; the ID of the cell to change boundary size.

layerNumber: integer; the layer number of the layer containing the rectangle used to resize the boundary of the cell.

Description

This command resets the boundary size of a cell to the size of a rectangle in a given layer.

Example

```
dbReplaceCellBoundary (geGetEditCell) 3
```

dbReplaceCellInst

Replaces the cell instance in a cell. Connections to the instance will be maintained if the port names of the new cell master match the port names of the existing cell master. Ports in the old master that have

no equivalent port in the new master are disconnected. Ports in the new master that have no equivalent in the old master are left unconnected.

Syntax

```
dbReplaceCellInst cellId instId "newCellName" "newViewName"  
"topCellName"
```

where the arguments are as follows:

cellId

Database object ID of the master cell to which the cell instance information applies

Valid values:

ID of any cell in the open library

If the cell is open, you can substitute the following for *cellId*: `(getCellEditCell)`. For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

instId

Database object ID of the cell instance to which the information applies

Valid values:

Cell instance ID of any cell instance in the current library

newCellName

Name of the master cell to replace the current master cell

Valid values:

Name of any master cell in the current library

newViewName

View name of the master cell to replace the current master cell

Valid values:

Name of any master view in the current library

topCellName

Name of the top cell in the open cell

Example

```
dbReplaceCellInst (geGetEditCell) 25619 "INVX3" ".FRAM"  
"top"
```

dbReplaceCellTextChar

Replaces a set of cell text characters in the open cell.

Syntax

```
dbReplaceCellTextChar cellId "oldCharSet" "newChar"
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

oldCharSet

Set of characters to change

newChar

Character to which all characters in the old character set will be changed

Note:

If *newChar* includes more than one character, only the first character in the string will be used for the replacement.

dbReplaceCellTextStr

Replaces a cell text string in the open cell.

Syntax

```
dbReplaceCellTextStr cellId "oldText" "newText"
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell).

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

oldText

The text objects to change

newText

New value to give to the text objects being changed

dbReplaceChildCell

Replaces all cell instances of an old child cell with a new child cell.

Syntax

```
dbReplaceChildCell cellId "oldCellName" "oldViewName"  
"newCellName" "newViewName" "topCellName"
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

oldCellName

Name of the old child cell in the open cell that you want to replace with a new child cell name

oldViewName

View name of the old child cell in the open cell that you want to replace with a new child cell view name

newCellName

Name of the new child cell that you want to replace for the old one

newViewName

View name of the new child cell that you want to replace for the old one

topCellName

Name of the top cell in the open cell

dbReplaceRegionWithInst

Replaces a cell region with a cell instance.

Syntax

```
dbReplaceRegionWithInst "cellName" regionNumber  
"macroCellName" "cellInstName"
```

where the arguments are as follows:

cellName

Name of the cell to which the cell instance information applies

Valid values:

Name of any cell in the current library

regionNumber

Number of the region that is to be replaced

Valid values:
Any positive integer

macroCellName

Name of the macro cell that is to be the cell instance's child cell

Valid values:
Name of any macro cell in the current library

cellInstName

Name of the cell instance that replaces the cell region

Valid values:
Name of any cell instance in the current cell

dbReplaceRefLibByName

Replaces an existing reference library in a library. If `dbReplaceLibByName` succeeds, the Synopsys tool returns *#t*.

Prerequisite

The library with which you want to replace a reference library must be open in write (W) mode.

Syntax

```
dbReplaceRefLibByName libId "oldRefLibName" "newRefLibName"
```

where the arguments are as follows:

libId

Database object ID of the current library with which you want to replace a reference library

Valid values:

- Library ID
- (dbGetCurrentLibId)

For more information, see “Specifying Libraries” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

oldRefLibName

Name of the existing reference library from which data moves

Valid values:

Name of any reference library attached to the current library

newRefLibName

Name of the new reference library to which data from the existing reference library moves

Valid values:

Any text string with fewer than 127 characters

Note:

This command does not open the new reference library.

See Also

[dbAddRefLib](#)

[dbDeleteRefLibByName](#)

dbReplaceScanCell

Replaces cell instances in the open cell with instances of a different master. Use this function to replace cell instances in a scan chain with instances of a different master.

Note:

Generates an ECO.

Tool

Milkyway

Syntax

```
dbReplaceScanCell (geGetEditCell) oldMasterName  
newMasterName
```

where the arguments are as follows:

oldMasterName Name of the master cell for which you are replacing instances.
Valid values:
Name of any master cell for which instances exist in a scan chain.

newMasterName Name of the master cell for which you want to use instances to replace the instances of *oldMasterName*.
Valid values:
Name of any master cell in the library.

dbReplaceTechFile

Replaces the existing technology file for a library. If `dbReplaceTechFile` succeeds, the Synopsys tool returns *#t*.

Prerequisite

The library for which you want to replace a technology file must be open in write (W) mode.

Syntax

```
dbReplaceTechFile libId "techfile.tfo"
```

where the arguments are as follows:

libId

Database object ID of the current library for which you want to replace the technology file

Valid values:

- Library ID
- (dbGetCurrentLibId)

For more information, see “Specifying Libraries” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

techfile.tfo

Name of the technology file that replaces the existing technology file of the current library. This is a binary compiled format of the technology file.

Valid values:

Name of any technology file in the current directory

See Also

[dbSetTechFileName](#)

dbResetAllNetTimingSpacing

Removes timing-driven spacing rules from all nets in the open cell.

Tool

Milkyway

Prerequisites

Open the cell containing the nets.

Syntax

```
dbResetAllNetTimingSpacing cellId
```

where the argument is as follows:

<i>cellId</i>	The design cell for which you want to specify external pin connections. If the cell is open, you can substitute the following for <i>cellId</i> : (<i>geGetEditCell</i>).
---------------	---

Example

```
dbResetAllNetTimingSpacing cellId
```

This example removes the timing-driven spacing rules from all nets in the current open cell.

dbResetAllNetTopLayerProbe

Removes all of the top-layer probe constraints from all nets in the open cell.

Tool

Milkyway

Prerequisites

Open the cell containing the nets.

Syntax

```
dbResetAllNetTopLayerProbe cellId
```

where the argument is as follows:

<i>cellId</i>	The the design cell that you want to remove the top layer probe constraints of the nets from. If the cell is open, you can substitute the following for <i>cellId</i> : (<i>geGetEditCell</i>)
---------------	--

Example

```
dbResetAllNetTopLayerProbe (geGetEditCell)
```

This example removes all of the top layer probe constraints from all nets in the current open cell.

See Also

[dbAssignNetTopLayerProbe](#)
[dbDumpAllNetTopLayerProbe](#)
[axgSetHPORouteOptions](#)

dbResetCellMustJoinPin

Removes external connection specifications for pins on a port.

Note:

This command applies to Astro and AstroGA.

Syntax

```
dbResetCellMustJoinPin cellId portname
```

where the arguments are as follows:

cellId

ID of the pin or blockage (FRAM) cell for which you want to remove specifications for external pin connections. If the cell is open, you can substitute the following for *cellId*:

`(geGetEditCell)`

portName

Name of the port for which you want to remove specifications for external pin connections

Example

```
dbResetCellMustJoinPin (geGetEditCell) "A"
```

This function removes the specifications for external pin connections on port A in the open cell.

See Also

[dbSetCellMustJoinPin](#)

[dbSetCellMustJoinAllPin](#)

[dbCreateMustJoinNets](#)

dbRestoreHierDesign

Restores the specified archived cell to the current directory. The cell design must have been archived with the `dbArchiveHierDesign` function.

To meet the requirements of portability and hierarchical referencing, `dbRestoreHierDesign` restores the original library directory structure to the current directory. All necessary data for the main and reference libraries are copied over to the current directory. All cells referenced in the top cell are copied over to the current directory. The library references maintained in the restored libraries are updated to reflect the new locations of the reference libraries in the current directory.

If `dbRestoreHierDesign` is executed successfully, a *true* message is returned. If unsuccessful, a *false* message is returned.

Prerequisite

All libraries must be closed.

Syntax

```
dbRestoreHierDesign "archiveLibName" "cellName"
```

where the arguments are as follows:

archiveLibName

Path (relative or absolute) and name of the archived library containing the cell to be restored

Valid values:

Any library that was created using `dbArchiveHierDesign`

cellName

Name of the text objects to change

See Also

[dbArchiveHierDesign](#)

[dbArchiveCellToLib](#)

dbRestorePin

Restores pins. For top-cell pins, the bits 0-3, of access-direction is not used. However, this is required for pins in .FRAM view of the child cell. The bits 4-7 need to be set properly.

Syntax

```
dbRestorePin cellId netName pinName status_flag  
access_direction  
status_flag_1 EEQ_class laynum point point
```

where the arguments are as follows:

<i>cellId</i>	Database object ID of the cell to which the information applies. If the cell is open, you can substitute the following for <i>cellId</i> : (<i>geGetEditCell</i>). Valid values: Any library that was created using dbArchiveHierDesign.
<i>netName</i>	Names of any net in the open cell.
<i>pinName</i>	Names of the pin you want the tool to restore.
<i>status_flag</i>	Status for placement. Valid values: 0 - unplaced, 1 - placed, 2 - fixed

<i>access_direction</i>	Pin's direction for route. bit0 - right access direction, bit1 - left access direction, bit2 - up access direction, bit3 - down access direction, bit4 - input port, bit5 - output port, bit6 - inout port, bit7 - tristate
<i>status_flag_1</i>	Status flag used by Jupiter for soft-macro pins.
<i>EEQ_class</i>	EEQ class number.
<i>laynum</i>	Number of the layer on which you want the pin. Valid values: Number of any layer defined in the technology file.
<i>point point</i>	Points of the rectangle of pins.

Examples

```
dbRestorePin _cell "red_out[0]" "red_out[0]" 0 40 0 0 2 '((51.250
0.000)
```

```
(51.850 0.600))
```

status_flag:

0 - unplaced,

1 - placed,

2 - fixed

access_direction:

bit0 - right access direction,
bit1 - left access direction,
bit2 - up access direction,
bit3 - down access direction,
bit4 - input port,
bit5 - output port,
bit6 - inout port,
bit7 - tristate.

status_flag_1: Used by Jupiter for soft-macro pins.

EEQclass: EEQ class number

dbRollBackCell

Replaces the current cell with the last version of the cell saved to the disk. This discards any changes made since you last saved the cell. Unlike [dbAbortCell](#), [dbRollBackCell](#) opens the last version of the cell that was saved to the disk.

If `dbRollBackCell` succeeds, the Synopsys tool returns `#t` and opens the last version of the cell that was saved to disk.

Note:

The `dbRollBackCell` function does not replace the current cell in the layout window. Choose Views > Redraw to update the layout window.

Syntax

```
dbRollBackCell cellId
```

where the argument is as follows:

cellId

Database object ID of the cell you want to roll back

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

See Also

[dbAbortCell](#)

[dbCloseCell](#)

[dbSaveCell](#)

[dbRollBackCell](#)

dbSaveCell

Saves the cell in the current window to disk. If `dbSaveCell` succeeds, the Synopsys tool returns *#t*.

Syntax

```
dbSaveCell cellId
```


where the argument is as follows:

cellId

ID of the cell to save

Valid values:

- Cell ID of any open cell in the current library
- `(geGetEditCell)`

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

See Also

[dbAbortCell](#)

[dbCloseCell](#)

[dbRollBackCell](#)

dbSetArrivalTime

This timing function has been replaced by `tdfSetInputDelay`.

dbSetBaseArraySubTile

Sets the the subTile flag of a base array subTile.

Syntax

```
dbSetBaseArraySubTile cellId baseArrayId subTile
```

Arguments

`cellId`: integer; the ID of the cell to change `subTileFlag` of the base array `subTile`.

`baseArrayId`: integer; the ID of the base array `subTile` to change `subTileFlag`.

Description

This command resets the `subTileFlag` of a base array `subTile` for a specified base array ID.

Example

```
dbSetBaseArraySubTile (geGetEditCell) 20 1
```

dbSetCellInstECOStatus

Sets the ECO status bits in the cell instance *statusFlag* field.

Syntax

```
dbSetCellInstECOStatus cellId cellInstId "ECOStatus"
```

where the arguments are as follows:

cellId

Database object ID of the master cell to which the cell instance information applies

Valid values:

- Cell ID of any open cell in the current library
- `(geGetEditCell)`

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

cellInstId

Database object ID of the cell instance to which the ECO information applies

Valid values:

Instance ID of any cell instance in the current cell

ECOStatus

ECO status of the cell instance

Valid values:

- *ECOReset*, which resets all the ECO status bits.
- *ECODelete*, which resets the *dbbPRCellInstECODelete* bit. This indicates that the ECO placer should delete the cell instance during ECO placement.
- *ECOAdd*, which sets the *dbbPRCellInstECOAdd* bit. This indicates that the cell was newly added by an ECO and that the ECO placer must place it. After the ECO placer places the cell, this bit is reset.
- *ECOREplace*, which sets the *dbbPRCellInstECOREplace* bit. This indicates that the cell was replaced by an ECO.
- *ECOIgnore*, which sets the *dbbPRCellInstECOIgnore* bit. This bit is automatically set by [dbCreateCellLEQClass](#), `geAddCell` (see Physical Implementation Online Help), and TDF *insertPad* to indicate that this cell was interactively added by the user and should not be deleted, even if it does not appear in the ECO netlist.

dbSetCellInstECOStatusByName

Sets the ECO status bits in the cell instance *statusFlag* field.

Syntax

```
dbSetCellInstECOStatus cellId cellInstName "ECOStatus"
```

where the arguments are as follows:

cellId

Database object ID of the master cell to which the cell instance information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

cellInstName

Name of the cell instance to which the ECO information applies

Valid values:

Name of any cell instance in the current directory

ECOStatus

ECO status of the cell instance

Valid values:

- *ECOReset*, which resets all the ECO status bits.

- *ECODElete*, which resets the *dbbPRCellInstECODElete* bit. This indicates that the ECO placer should delete the cell instance during ECO placement.
- *ECOAdd*, which sets the *dbbPRCellInstECOAdd* bit. This indicates that the cell was newly added by an ECO and that the ECO placer must place it. After the ECO placer places the cell, this bit is reset.
- *ECOREplace*, which sets the *dbbPRCellInstECOREplace* bit. This indicates that the cell was replaced by an ECO.
- *ECOIgnore*, which sets the *dbbPRCellInstECOIgnore* bit. This bit is automatically set by [dbCreateCellEQClass](#), `geAddCell` (see Physical Implementation Online Help), and TDF *insertPad* to indicate that this cell was interactively added by the user and should not be deleted, even if it does not appear in the ECO netlist.

dbSetCellInstFixStatusByName

Specifies the placement status of a cell instance.

Syntax

```
dbSetCellInstFixStatusByName cellId cellInstName
"placementStatus"
```

where the arguments are as follows:

cellId

Database object ID of the master cell to which the cell instance information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

cellInstName

Name of the cell instance to which the placement information applies

Valid values:

Instance name of any cell instance in the current directory, with the exception of I/O pad instances

ECOStatus

Placement status of the cell instance

Valid values:

- Unplaced
- Placed
- Fixed
- RigidFixed

For more information, see `aprCmdFixCell` in Physical Implementation Online Help.

dbSetCellInstOrigin

Establishes the origin of a cell instance.

Syntax

```
dbSetCellInstOrigin cellId "cellInstName" origin
```

where the arguments are as follows:

cellId

Database object ID of the master cell to which the cell instance information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

cellInstName

Name of the cell instance to which the origin information applies

Valid values:

Name of any cell instance in the current library

origin

Origin of the cell instance

Valid values:

(list (x y))

dbSetCellInstGroupId

Sets the cell instance group for the specified floor plan group.

Syntax

```
dbSetCellInstGroupId <objectId cellId> <objectId groupId>  
<objectId parentId>
```

cellId - Database object ID of a cell containing the groups

groupId - Database object ID of the floor plan group

parentId - Database object ID of the cell instance group

Description

The `dbSetCellInstGroupId` function sets the cell instance group for the specified floor plan group. This function does not check if the specified group IDs are valid group objects.

Example

```
dbSetCellInstGroupId (geGetEditCell) #x1111A #x2222B
```

dbSetCellInstPlacement

Places a cell instance in the open cell.

Syntax

```
dbSetCellInstPlacement cellId "cellInstName" rotationStr  
mirrorStr "justStr" Point
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library

- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

cellInstName

Name of the cell instance in the open cell to which the placement changes apply

rotationStr

Counterclockwise rotation

Valid values:

Degrees (0, 90, 180, or 270)

mirrorStr

Direction, if any, in which to flip the cell instance

Valid values:

no, X, or Y

justStr

Placement of the cell instance around the origin

Valid values:

- ll, which places the cell instance to the lower left of the origin
- lc, which places the cell instance to the lower center of the origin
- lr, which places the cell instance to the lower right of the origin
- cl, which places the cell instance to the center left of the origin
- cc, which centers the cell instance on the origin
- cr, which places the cell instance to the center right of the origin

- ul, which places the cell instance to the upper left of the origin
- uc, which places the cell instance to the upper center of the origin
- ur, which places the cell instance to the upper right of the origin

Point

The x-y coordinate pair representing the location of the cell instance origin

Valid syntax:

'(x y)

dbSetCellInstStatus

Specifies the status of a cell instance.

Syntax

```
dbSetCellInstStatus cellId cellInstId "status"
```

where the arguments are as follows:

cellId

Database object ID of the master cell to which the cell instance information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

cellInstId

Database object ID of the cell instance to which the status information applies

Valid values:

ID of any instance in the current cell

status

Placement status of the cell instance

Valid values:

- Unplaced
- Placed
- Fixed
- RigidFixed

dbSetCellMustJoin2Pin

Instructs the Synopsys tool to externally connect together specified pins on a port.

Syntax

```
dbSetCellMustJoin2Pin cellId "portName" "pinName1"  
"pinName2"
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies.

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

portName

Name of the port for which you want to specify external pin connections

pinName1

Name of the first pin you want the application to connect externally

pinName2

Name of the second pin you want the application to connect externally

dbSetCellMinMaxLayer

Sets a range of layers on which nets in the cell can be routed.

Note:

Sets the default minimum to maximum layers for all the nets in the cell. You can overwrite this default using [dbAssignNetMinMaxLayer](#).

Tool

Milkyway

Prerequisites

Open the cell containing the nets to which you want to assign to layers.

Syntax

```
dbSetCellMinMaxLayer cellId "min_layer_name"  
"max_layer_name"
```

where the arguments are as follows:

<i>cellId</i>	ID of the design cell for which you want to assign nets to particular layers. If the cell is open, you can substitute the following for <i>cellId</i> : (geGetEditCell).
<i>min_layer_name</i>	Name of the minimum layer. To use the lowest routing layer, type "".
<i>e</i>	Valid values: Routing layers.
<i>max_layer_name</i>	Name of the maximum layer. To use the highest routing layer, type "".
<i>e</i>	Valid values: Routing layers.

Note:

To unassign the nets to the routing layers, set both *min_layer_name* and *max_layer_name* to "" (null).

Example

```
dbSetCellMinMaxLayer cellId "M1" "M3"
```

This example assigns nets in the current open cell to layers M1 through M3.

dbSetCellMustJoinAllPin

Instructs the Synopsys tool to externally connect together all pins on a specified port (or all ports with names matching a specified pattern).

Note:

Astro support must-join pins for all cells.

Tool

Milkyway

Prerequisites

Create the pin/blockage cell using `auExtractBlockagePinVia`, then open the pin/blockage cell (*.fram*) using `geOpenCell`.

Syntax

```
dbSetCellMustJoinAllPin cellId portname
```

where the arguments are as follows:

<i>cellId</i>	ID of the pin/blockage (<i>.fram</i>) cell for which you want to specify external pin connections. If the cell is open, you can substitute the following for <i>cellId</i> : <code>(geGetEditCell)</code> .
---------------	---

portName

Name of the port for which you want to specify external pin connections, or a pattern to match the names of multiple ports. For information about pattern matching, see “Specifying Objects by Pattern” on page 9 of the Place and Route Getting Started User Guide.

Example

```
dbSetCellMustJoinAllPin (geGetEditCell) "A"
```

This function marks all pins on port A in the open cell as pins to be connected externally by the router.

See Also

[dbSetCellMustJoinPin](#) instructs the tool to externally connect together specified pins on a port.

[dbCreateMustJoinNets](#) creates a net for each port in the specified cell that has must-join pins not assigned to any net.

[dbResetCellMustJoinPin](#) - removes external connection specifications for pins on a port.

dbSetCellMustJoinPin

Instructs the Synopsys tool to externally connect together specified pins on a port.

Note:

Astro support must-join pins for all cells.

Tool

Milkyway

Prerequisites

Create the pin/blockage cell using `auExtractBlockagePinVia`, then open the pin or blockage cell (*.fram*) using `geOpenCell`.

Note:

The `auExtractBlockagePinVia` command assigns unique names to pins on the same port, as follows: The first pin inherits the name of the port, and Milkyway assigns the other pin names by adding a blank and a number to the port name. For example, if port A has three pins, the pin names are A, A 1, and A 2.

Syntax

```
dbSetCellMustJoinPin cellId portname ' (pinName1 pinName2
... pinNameN)
```

where the arguments are as follows:

<i>cellId</i>	ID of the pin/blockage (<i>.fram</i>) cell for which you want to specify external pin connections. If the cell is open, you can substitute the following for <i>cellId</i> : (<code>geGetEditCell</code>).
<i>portName</i>	Name of the port for which you want to specify external pin connections.
<i>pinNames</i>	Names of the pins you want the tool to connect externally.

Example

```
dbSetCellMustJoinPin (geGetEditCell) "A" ' ("A" "A 1" "A 2"
"A 4")
```

This function marks pins A, A 1, A 2, and A 4 on port A in the open cell as pins to be connected externally by the router.

See Also

[dbSetCellMustJoinAllPin](#) instructs the tool to externally connect together all pins on a specified port (or all ports with names matching a specified pattern).

[dbCreateMustJoinNets](#) creates a net for each port in the specified cell that has must-join pins not assigned to any net.

The `dbResetCellMustJoinPin` command removes external connection specifications for pins on a port.

dbSetCellMustJoinPinPattern

Establishes the external connection specification for all pins for a port uniquely identified by the given pattern.

Syntax

```
dbSetCellMustJoinPinPattern cellId "portNamePattern"
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies.

Valid values:

- Cell ID of any open cell in the current library
- `(geGetEditCell)`

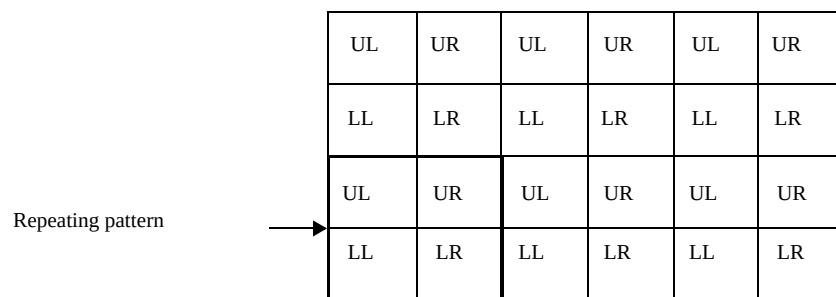
For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

portNamePattern

Specifies the pattern that uniquely identifies a port in the cell

dbSetCellPattern

Defines the legal orientations of the cell when placed on each of the tiles in the repeating pattern of a tile array. The tiles are referred to by their positions as “LL” (lower left), “LR” (lower right), “UL” (upper left), and “UR” (upper right).



This function returns *#t* when successful.

Syntax

```
dbSetCellPattern cellId patternNumber1 patternNumber2  
                patternNumber3 patternNumber4
```

where the arguments are as follows:

cellId

Database object ID that must be a cell ID

Valid values:

- Cell ID of any cell in the open library

- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

patternNumber1

Legal orientation for the lower-left tile

Valid values:

Any combination of the following:

- If bit0 is set, AstroGA places the cell with no transformation.
- If bit1 is set, AstroGA places the cell flipped on the y-axis.
- If bit2 is set, AstroGA places the cell flipped on the x-axis.
- If bit3 is set, AstroGA places the cell rotated 180 degrees.
- If bit4 is set, AstroGA places the cell flipped on the x-axis and rotated 90 degrees.
- If bit5 is set, AstroGA places the cell rotated 90 degrees.
- If bit6 is set, AstroGA places the cell rotated 270 degrees.
- If bit7 is set, AstroGA places the cell flipped on the y-axis and rotated 90 degrees.

patternNumber2

Legal orientation for the lower-right tile

Valid values:

Same as for *patternNumber1*

patternNumber3

Legal orientation for the upper-left tile

Valid values:

Same as for *patternNumber1*

patternNumber4

Legal orientation for the upper-right tile

Valid values:

Same as for *patternNumber1*

dbSetCellPortTypes

Specifies the following information for ports in a cell:

- Whether a port is input, output, inout, or tristate
- Whether a port is connected to a power, ground, or clock net
- Whether a port has secondary power and ground pins (pins not on the power and ground rail)
- Whether any unconnected ports in a cell should tie to power or ground
- Whether a port asynchronous with respect to a clock is active high or active low
- Whether a port is the input or output to a clocked cell
- Whether a port is the address input of a memory cell (RAM or ROM)
- Whether a port is the enable port for the scan clock
- Whether a port is a scan input, scan output, or scan clock input for a scannable cell
- Whether a port is a control line for a tristate cell

- Whether a port is for connecting to a flip-chip bump

Some of this information is stored in the database in the FRAM cell, and some is stored as a library attached file. This function does *not* store any information in the CEL cell. Use [dbSetPortDirection](#) instead.

For more information, see the section on specifying port information for advanced operations in the *Milkyway Environment Data Preparation User Guide*.

Syntax

```
dbSetCellPortTypes "libName" "cellName" '({("portName"
{"type"} ...} ...)) append?
```

where the arguments are as follows:

libName

Name of the library containing the cell to which port information applies

Valid values:

Name of any library in the current directory

cellName

Name of the cell to which port information applies

Valid values:

Name of any cell in the specified library

portName

Name of the port to which information applies

Valid values:

Name of any port in the specified cell

type

Type that applies to the specified port

Valid values:

For any port, you can specify one or more of the following:

- `Input` means that the port is an input port.
- `Output` means that the port is an output port.
- `Inout` means that the port is an inout port.
- `Tristate` means that the port is a tristate port.
- `Power` means that the port should be connected to a power net.
- `Ground` means that the port should be connected to a ground net.
- `specialPG` means that the port has secondary power and ground pins (pins not on the power and ground rail). When this port type is specified, secondary ground and power pins are checked for shorts (helps to eliminate design rule violations on vertical power and ground preroutes).
- `Clock` means that the port should be connected to a clock net.
- `Tieup` means that if the port is not connected, it should be tied to power.
- `Tiedown` means that if the port is not connected, it should be tied to ground.
- `Async_rising` means that the port is asynchronous with respect to a clock and is active high.

- `Async_falling` means that the port is asynchronous with respect to a clock and is active low.
- `DataIn` means that the port is the input to a clocked cell.
- `DataOut` means that the port is the output from a clocked cell.
- `AddressIn` means that the port is the address input of a memory cell (RAM or ROM).
- `Enable` means that the port is the enable port for the scan clock.
- `FlipChip` means the port is for connecting to a bump.
- `ScanIn` means that the port is a scan input for a scannable cell.
- `ScanOut` means that the port is a scan output for a scannable cell.
- `ScanClock` means that the port is a scan clock input for a scannable cell.
- `TristateDisable` means that the port is a control line for a tristate cell.

Note:

If you change the tie-up or tie-down information after floorplanning, you must update connections in the cells, using the `aprPGConnect` command.

append?

Whether or not the information should be appended to information specified by previous `dbSetCellPortTypes` functions

Valid values:

- #t means that Astro should append any existing information with the new information.
- #f means that Astro should replace any existing information with the new information.

Note:

To write current port information to a file, use the [dbDumpGPortTable](#) command.

Example

```
dbSetCellPortTypes "demo" "AND4" '(
("VDD" "Inout" "Power")
("C" "Input" "Tieup")
("A" "Input" "Tieup")
("B" "Input" "Tieup")
("D" "Input" "Tieup")
("X" "Output")
("GND" "Inout" "Ground")
) #f
```

This example provides port information for ports VDD, C, A, B, D, X, and GND in cell AND4, which is in the library called demo. The #f indicates that the information should replace any existing port information for the cell AND4.

See Also

[dbSetCellPortTypeExtraView](#)

dbSetCellPortTypeExtraView

Specifies the extra view name, with the exception of CEL view, FRAM view, or TIM view, to be used in the [dbSetCellPortTypes](#) command.

Syntax

```
dbSetCellPortTypeExtraView "viewName"...
```

where the arguments are as follows:

"viewName"

Name of the view that you want to use in the [dbSetCellPortTypes](#) command

Valid values:

"SMASH" or another view name

The maximum number of extra view names is two.

Example

```
dbSetCellPortTypeExtraView "SMASH" "BARR"
```

See Also

[dbSetCellPortTypes](#)

dbSetCellPowerValue

Sets the power value of a specified cell.

Prerequisite

The power units must be specified in the unit section of your technology file. For example,

```
*unit  
power mw 1000  
*end
```

For more information, see the power section in the Technology File appendix in the *Milkyway Environment Data Preparation User Guide*.

Syntax

```
dbSetCellPowerValue cellId powerValue
```

where the arguments are as follows:

cellId

ID of the cell to which the power value applies

Valid values:

ID of any open cell in the current library

powerValue

Amount of power the cell consumes

Valid types:

- Floating point
- Integer

dbSetCellPRBoundaryPolygon

Sets the PR boundary for the specified cell.

Syntax

```
dbSetCellPRBoundaryPolygon <objectId cellId>  
<list pointArray>
```

where the arguments are:

cellId - Database object ID of the cell

pointArray - List of points that define the boundary (maximum of 20 points)

Description

The `dbSetCellPRBoundaryPolygon` function sets the PR boundary for the specified cell. A PR boundary is a polygon that encloses the unit tiles in a standard cell. Astro uses the PR boundary to align wire tracks, power rails, and ground rails during cell placement. A PR boundary is on the PRBoundary layer and can be larger or smaller than the cell boundary.

The specified point list can contain up to 20 points that define the boundary points in sequential order. If you specify more than 20 points, only the first 20 points are used in the boundary. If the last point does not match the first point, the function automatically completes the polygon by appending the first point to the end of the list. This appended point is not counted against the 20 point maximum.

Example

```
dbSetCellPRBoundaryPolygon (geGetEditCell) '((0 0) (1000 0)
(1000 500) (1500 500) (1500 1000) (0 1000))
```

dbSetCellRegionBBox

Specifies a bounding box for a cell region.

Syntax

```
dbSetCellRegionBBox cellId regionId pointArray
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the cell region information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

regionId

Database object ID of the region to which the information applies

Valid values:

Region ID of any region in the specified cell

pointArray

Coordinates of the bounding box specified in the following format:

'((x1 y1) (x2 y2))

The first point is the lower-left corner, and the second point is the upper-right corner.

Example

```
define_cell (geGetEditCell)
define_cellinst (dbGetCellInstByName_cell "block1/inst2")
define_cellRegion
(dbGetRegionIdByCellInstId_cell_cellinst)
dbSetCellRegionBBox_cell_region '((100 100) (200 200))
```

dbSetCellRegionTargetPercent

Specifies the target percentage of cells for a region.

Syntax

```
dbSetCellRegionTargetPercent cellId regionId targetPercent
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the cell region information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

regionId

Database object ID of the region to which the information applies

Valid values:

Region ID of any region in the specified cell

targetPercent

Percentage of cells to be placed in this region

Valid values:

Any positive integer less than 15,000

(Note that the region percentage is scaled by 100.)

dbSetCellRowMirror

Specifies the placement orientation of a cell row.

Syntax

```
dbSetCellRowMirror cellId cellRowId orient
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the cell row information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

cellRowId

Database object ID of the cell row to which the information applies

Valid values:

Cell row ID of any cell row in the specified cell

orient

Whether or not the cell row should be mirrored

Valid values:

- #t mirrors the cell row.
- #f leaves the cell row unmodified.

dbSetCellType

Specifies the type of a cell.

Syntax

```
dbSetCellType cellId "cellType"
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

cellType

Type that applies to the cell

Valid values:

- maskLayout
- netList

- schematic
- symbolic
- pcbLayout
- behavior
- logicModel
- graphic
- document
- stranger
- unknown

dbSetCellVTType

Marks a given cell master as a certain voltage threshold type by attaching a unique ID string to it. The ID string is used in conjunction with what is specified with the `dbSpecifyVTFiller` command, to identify the voltage threshold type of cell instances during mixed-voltage threshold filler cell insertion. (You can use the `dbSetLibraryCellsVTType` command to mark all the cells in a given library.)

Tool

Milkyway

Syntax

```
dbSetCellVTType topCellId cellMasterName vtTypeId
```


where the arguments are as follows:

<i>topCellId</i>	The database object ID of the top cell.
<i>cellMasterName</i>	The name of the cell master.
<i>vtTypeId</i>	The string identifier for the voltage threshold type. Use the format vtType#, with # being an integer unique to the type.

Example

```
dbSetCellVTType (geGetEditCell) "cellA" "vtType1"
```

See Also

[dbClearCellVTType](#)

[dbDumpCellVTType](#)

[dbSetLibraryCellsVTType](#)

[dbSpecifyVTFiller](#)

dbSetCheckLibMode

Sets the “Check Library Mode” flag.

Syntax

```
dbSetCheckLibMode <boolean mode>
```

where the argument is:

mode - Either true or false (#t or #f)

Description

The `dbSetCheckLibMode` function sets the “Check Library Mode” flag. If the flag is true, libraries are checked for consistency when they are opened. If the flag is false, then libraries are not checked.

By default, the flag is true. When a library and its reference libraries are initially opened, Milkyway checks for consistency between the main library’s and its reference libraries’ technology and settings. If discrepancies are found, warning messages are displayed in the scheme window. However, Milkyway continues to open the library and its reference libraries.

For each reference library, the following values should be consistent with the main library:

- case sensitivity
- time unit and precision
- length unit and precision
- resistance unit and precision
- capacitance unit and precision
- inductance unit and precision
- grid resolution
- layers
- contact codes

See also `dbGetCheckLibMode`.

Example

```
dbSetCheckLibMode #t
```

dbSetCurrentRegionNumber

Sets the current region number for the specified cell.

Syntax

```
dbSetCurrentRegionNumber <objectId cellId> <int regNum>
```

where the arguments are:

cellId - Database object ID of an opened cell

regNum - Current region number

Description

The `dbSetCurrentRegionNumber` function sets the current region number for the specified cell. If set, the current region number is used by the `dbSetCellInstPlacement` and `dbCellInstPlaceAndStatus` functions to add instances into the corresponding region.

The current region number is only valid for the specified cell and persistent for the current Milkyway session. It is unset whenever the `dbSetCellInstPlacement`, `dbCellInstPlaceAndStatus`, and `dbSetCellInstFixStatusByName` functions are called for a different cell.

Example

```
dbSetCurrentRegionNumber (geGetEditCell) 5
```

dbSetEcoWeight

Resets all ECO weight in the design. The higher the ECO weight, the higher the priority during overlap removal, and therefore, the less displacement performed by the ECO placer on a cell instance.

After timing fine-tuning and adding new cell instances, ECO-Place is usually run to removal cell overlaps. It is advisable to set ECO weight to those timing - important cell instances to minimize location disturbance during overlap removal.

Tool

Milkyway

Syntax

```
dbSetEcoWeight (geGetEditCell) "cellInstNamePattern"  
ecoWeight
```

where the arguments are as follows:

<i>cellInstNamePattern</i>	Name of the full cell instance or a name pattern containing a wild card or '?' for matching.
<i>ecoWeight</i>	Integer: 1, 2, 3, ... where: 1: clears 2 or higher: sets

Example

```
dbSetECOWeight (geGetEditCell) ".*" 2
```

dbSetErrorClassColor

Specifies the color of an error class.

Syntax

```
dbSetErrorClassColor cellId errClassId color
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the error information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

errClassId

Database object ID of the error class to which information applies

Valid values:

Error class ID of any error class in the specified cell

color

Color that applies to the error

Valid values:

An integer from 0 to 255 that indexes the color map

dbSetErrorClassPattern

Specifies the pattern of an error class.

Syntax

```
dbSetErrorClassPattern cellId errClassId pattern
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the error information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

errClassId

Any database object ID that is also an error class ID

Valid values:

Error class ID of any error class in the specified cell

pattern

Pattern that applies to the error

Valid values:

An integer from 0 to 255 that indexes the fill pattern map

dbSetErrorClassStatus

Specifies the status of an error class.

Syntax

```
dbSetErrorClassStatus cellId errClassId status
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the error information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

errClassId

Database object ID of the error class to which the information applies

Valid values:

Error class ID of any error class in the specified cell

status

Level that applies to the error

Valid values:

- 0 gives an informational error.
- 1 gives a warning.
- 2 gives an error.
- 3 gives a severe error.

dbSetErrorObjStatus

Sets the value of the status field of an error object. The error status is not currently used by Synopsys tools.

Syntax

```
dbSetErrorObjStatus cellId ErrorObjId status
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

errorObjId

Database identifier of the error object being updated

Valid values:

ID of any error type in the open cell

status

Value to assign to the status field of the error type

dbSetErrorStatus

Specifies the status of an error.

Syntax

```
dbSetErrorStatus cellId errId status
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the error information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

errId

Database object ID of the error to which the information applies

Valid values:

Error class ID of any error class in the specified cell

status

Level that applies to the error

Valid values:

- 0, which gives an informational error
- 1, which gives a warning

- 2, which gives an error
- 3, which gives a severe error

dbSetErrorTypeStatus

Sets the value of the status field of an error type. The error status is not currently used by Synopsys tools.

Syntax

```
dbSetErrorTypeStatus cellId ErrorTypeId status
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

errorTypeId

Database identifier of the error type being updated

Valid values:

ID of any error type in the open cell

status

Value to assign to the status field of the error type

dbSetFringeFactor

Specifies the fringe factor between two objects.

Syntax

```
dbSetFringeFactor layerNo1 layerNo2 minFringeFactor  
maxFringeFactor
```

where the arguments are as follows:

layerNo1

Number of the layer in which the first object is located

Valid values:

Any layer in the current library

layerNo2

Number of the layer in which second object is located

Valid values:

Any layer in the current library except the layer specified in

layerNo1

minFringeFactor

Minimum value for the fringe factor

Valid values:

A positive or negative integer

maxFringeFactor

Maximum value for the fringe factor

Valid values:

A positive or negative integer

dbSetGroupParentId

Sets the parent object for the specified group.

Syntax

```
dbSetGroupParentId <objectId cellId>  
<objectId groupId> <objectId parentGroupId>
```

where the arguments are as follows:

cellId - Database object ID of the cell containing the specified group and parent.

groupId - Database object ID of the group.

parentGroupId - Database object ID of the parent object.

Description

The `dbSetGroupParentId` function sets the parent object for the specified group. A group's parent can be the region, floorplan group, or hard/soft fence to which the group belongs.

Example

```
dbSetGroupParentId (geGetEditCell) #x4000 #x5000
```

dbSetHierRefLibMode

Sets the child cell search order to either the hierarchical reference library mode or the nonhierarchical reference library mode.

Syntax

```
dbSetHierRefLibMode flag
```

where the argument is as follows:

flag

Is Boolean

Valid values:

- *#t*

Sets the library cell search order to the hierarchical mode. The current library is the library that contains the parent cell with the cell instance (child cell reference). Cell searching starts in the current library, then traverses through the current library's reference libraries. This is the default setting.

- *#f*

Sets the library search order to the nonhierarchical mode. Cell searching always starts in the top (main) library, then traverses through the top library's reference libraries. The access depth is limited to two levels only.

Example

The following code sets the library search order in hierarchical mode:

```
dbSetHierRefLibMode #t  
#t
```

dbSetIOPad

Sets the cell flag of all cell instances of the cell (of the specified cell ID) to the I/O pad cell. However, if the cell flag has been set to standard cell, this command will not change it to the I/O pad cell.

Syntax

```
dbSetIOPad cellId
```

where *cellId* indicates the internal ID of any open cell in the current library to which you want to set the cell flag.

See Also

[dbSetStdCell](#)

dbSetLayerTimeInfo

Specifies the following timing information for a layer:

- Resistance
- Capacitance
- Inductance

Syntax

```
dbSetLayerTimeInfo layerNumber minRes maxRes minCap maxCap  
minInd maxInd
```

where the arguments are as follows:

layerNumber

Number identifying the layer to be modified

Valid values:

A positive integer that corresponds to an existing layer

minRes

Value that specifies the minimum resistance allowable for the layer specified by *layerNumber*

maxRes

Value that specifies the maximum resistance allowable for the layer specified by *layerNumber*

minCap

Value that specifies the minimum capacitance allowable for the layer specified by *layerNumber*

maxCap

Value that specifies the maximum capacitance allowable for the layer specified by *layerNumber*

minInd

Value that specifies the minimum inductance allowable for the layer specified by *layerNumber*

maxInd

Value that specifies the maximum inductance allowable for the layer specified by *layerNumber*

dbSetLayoutSubType

Specifies the layout subtype of a cell.

Syntax

```
dbSetLayoutSubType cellId "layoutSubType"
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (getCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

layoutSubType

Layout subtype that applies to the specified cell

Valid values:

- ioPad
- padFillerCell
- cornerPad
- standardCell
- standardCell2
- stdFillerCell
- macro
- chip
- image
- cover
- unknown

dbSetLibBusAppendName

Enables you to set the attribute, *Bus Append Name* of your library, which causes all bits of buses to be followed by the specified string.

Tool

Milkyway

Syntax

```
dbSetLibBusAppendName "libName" "appendName"
```

where the arguments are as follows:

<i>libName</i>	Name of the library for which you want to modify the bus append name. Valid values: Name of any library in the current directory.
<i>appendName</i>	Any string you want Milkyway to insert into the bus name. Valid values: _bus, _array, BUS

Example

This example sets the append name of *_bus* to the library named *DESIGNS*.

```
dbSetLibBusAppendName "DESIGNS" "_bus"
```

dbSetLibBusNameStyle

Enables you to set the *Bus Naming Style* property of your library.

Tool

Milkyway

Syntax

```
dbSetLibBusNameStyle "libName" "busStyle"
```

where the arguments are as follows:

<i>libName</i>	Name of the library for which you want to modify the bus naming style. Valid values: Name of any library in the current directory.
<i>busStyle</i>	Name of the bus style you wish to set. Type any string you want Milkyway to append to the bus name using <i>%d</i> to indicate the port number. The requirements are as follows: The string can contain only one <i>%d</i>. To specify a percent (%) in the string, use <i>%%</i>. For example, if your Verilog file defines a bus as <pre>input [1:4] test_bus</pre> and you specify a <i>Bus Naming Style</i> of <i>_<i><%%<i>%d</i>></i></i>, Milkyway translates the bus defined by this statement as ports <i>test_bus_<%1></i>, <i>test_bus_<%2></i>, <i>test_bus_<%3></i>, and <i>test_bus_<%4></i>. Valid values: [<i>%d</i>], <i>_<i><%d</i>></i>, <i>_<i>%d</i>_</i>

Example

This example sets the bus naming style of [*%d*] to all bits of buses in the library named "test_bus".

```
dbSetLibBusNameStyle "test_bus" "[ %d] "  
#t
```

dbSetLibDefaultCreationAccess

Enables you to set an access value to a newly created library.

Tool

Milkyway

Syntax

```
dbSetLibDefaultCreationAccess accessValue
```

where the argument is as follows:

<i>accessValue</i>	The default access value (Scheme number). Valid Values: 0 or 511 will set the default access mode to "0777". Any other value will set the default access mode to "0755". Return Values: Returns #t if the default access mode is set to "0777". Returns #f if the default access mode is set to "0755".
--------------------	---

Note:

The default access mode only applies to libraries created in the current tool's environment. However, a similar effect can be achieved by setting the environmental variable "MK_LIB_DEFAULT_CREATION_ACCESS" with the corresponding "accessMode". By using the global environment variable instead, the default access mode applies to all tools.

Example

This example sets the access mode of the current library to "0777".

```
dbSetLibDefaultCreationAccess 511
dbSetLibDefaultCreationAccess #o777
dbSetLibDefaultCreationAccess #d511
dbSetLibDefaultCreationAccess 0
```

dbSetLibRefCtrlFileMode

Sets the library reference mode to the “Reference Control File Mode” or the “Internal Reference Mode.” If #t, all library references are determined by the reference control file. If #f, all library references are determined by the internal references.

Tool

Milkyway

Syntax

```
dbSetLibRefCtrlFileMode libName #t/#f
```

Note:

There are currently two functions to set the reference control file mode:

The old function, [dbSetRefControlFileMode](#), stores the mode in a global runtime variable, and therefore sets the mode for the current session only.

The new function, `dbSetLibRefCtrlFileMode`, stores the mode in the library and therefore sets the mode for the specified library indefinitely for all sessions.

The `dbSetLibRefCtrlFileMode` function has a higher precedence than the old function. Once you use the new function to set the

mode in the library, the old function will no longer have any effect on that particular library.

Certain processes (such as `auNDIApi` and `auNDOApi`) run in a separate session, and therefore you must use the new function, `dbSetLibRefCtrlFileMode`, in order to set the correct mode.

See Also

[dbCheckRefLibControl](#)
[dbClearRefLibControl](#)
[dbDumpRefLibControl](#)
[dbSetRefLibControl](#)

dbSetLibraryCellsVTType

Marks all cell masters in a given library as a certain voltage threshold type by attaching a unique ID string to it. The ID string is used in conjunction with what is specified with the `dbSpecifyVTFiller` command to identify the voltage threshold type of cell instances during mixed-voltage threshold filler cell insertion. (You can use the `dbSetCellVTType` command to mark a given cell master.)

Tool

Milkyway

Syntax

```
dbSetLibraryCellsVTType topCellId LibraryName vtType#
```

where the arguments are as follows:

topCellId The database object ID of the top cell.

<i>LibraryName</i>	The name of the library that contains the cells.
<i>vtType#</i>	The string identifier for the voltage threshold type. Use the format vtType#, with # being an integer unique to the type.

Example

```
dbSetLibraryCellsVTType (geGetEditCell) "LibraryA"
    "vtType1"
```

See Also

[dbClearLibraryCellsVTType](#)

[dbDumpLibraryCellsVTType](#)

[dbSetCellVTType](#)

[dbSpecifyVTFiller](#)

dbSetMacroCellOrient

Enables you to control the placement orientations of the macro cells. Takes effect as a result of the [geMove](#) command or the macro placement command.

Note:

The same is true for [dbSetStdCellOrient](#).

This command applies to the Astro tool.

Syntax

```
dbSetMacroCellOrient libName cellName orientation
```

where the arguments are as follows:

libName

Name of the library containing the cell to which the orientation specification applies.

Valid values:

Name of any library in the current directory.

cellName

Name of the cell to which the orientation specification applies

Valid values:

- Name of any cell in the specified library.
- Pattern matching to select all cells with names that match a pattern. For example, typing TOP.* specifies all cells with names that begin with TOP. For information about pattern matching, see *Physical Implementation Getting Started*.

orientation

One of {Any Reflect Norelect} or a combination of {R0 R0_MX R0_MY R180 R90 R90_MX R90_MY R270}

Example:

```
"R0 R0_MY R180 R90_MX"
```

These valid values are case-insensitive and are separated by blanks.

Example

```
dbSetMacroCellOrient "GOLDROAD" "RAM101" "Reflect"  
  (same as "R0_MX R180")  
dbSetMacroCellOrient "GOLDROAD" "ROM102" "R0 R0_MY R180"
```

The first example instructs Astro to place macro instances of RAM101 with either R0_MX or R180 (reflected) orientation. The second example further allows R0 as a legal orientation for any instance of master ROM102.

dbSetMaxMetalRoutingLayer

Allows you to control the routing layer usage by setting the maximum metal routing layer.

The default value of the maximum metal routing layer is set during floorplanning to the highest metal in the technology file. You can check this value by using the [dbGetMaxMetalRoutingLayer](#) command.

Note:

Use only three or four layers of metal in a five- or six-layer library.

Syntax

```
dbSetMaxMetalRoutingLayer cellId metal_num
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the net information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

metal_num

Number of layers of metal in a metal routing layer

Example

```
(dbSetMaxMetalRoutingLayer (geGetEditCell) 4)
```

This command will route up to metal4.

dbSetMsgLevel

Sets the message level, which controls the number of messages displayed.

Syntax

```
dbSetMsgLevel [string level]
```

where the argument is as follows:

Level

"high", "medium" or "low" (short forms) "h", "m" or "l"

Description

The `dbSetMsgLevel` command sets the message level, which determines how many messages are displayed. There are three levels: high, medium, and low. The higher the level, the greater number of messages displayed. The default level is medium.

Example

To see all user messages, set the message level to high by typing the following on the command line:

```
dbSetMsgLevel "high"
```

or

```
dbSetMsgLevel "h"
```

dbSetLModelSubType

Specifies the logic model subtype of a cell, including any latch, flip-flop, RAM, ROM, or clock cell in the library.

Prerequisite

Because `dbSetLModelSubType` functions set the types on the timing cells, you need to be sure to create these timing cells before you load the `dbSetLModelSubType` functions. You create timing cells by loading the `defineTimeIntrinsic` and `definePortCapacitance` functions.

Note:

You can include `dbSetLModelSubType` functions in a CLF file, along with the other functions that are used to define the timing library for slack graph generation (`defineTimeIntrinsic`, `definePortCapacitance`, and `dbSetCellPortTypes`).

However, you need to place the `defineTimeIntrinsic` and `definePortCapacitance` functions before the `dbSetLModelSubType` functions in the file so that the timing cells exist when the `dbSetLModelSubType` functions attempt to set the types.

Syntax

```
dbSetLModelSubType cellId "LModelSubType"
```

where the arguments are as follows:

cellId

ID of the timing cell for which you are specifying a type

Valid values:

ID of any timing cell in the current directory (If you do not know the cell ID, see the section on specifying cell IDs in the *Static Timing Analyzer User Guide*.)

LModelSubType

Type of the cell

Valid values:

Any of the following:

- latch
- flipflop
- ram
- rom
- clock (clock buffer definition for clock tree synthesis)

Example

```
setq cellId (dbOpenCell "latch1.tim" "w")
dbSetLModelSubType cellId "latch"
dbCloseCell cellId
```

These statements do the following:

- The first line opens the cell called latch1.tim and creates the `cellId` variable for its cell ID.
- The second line specifies that the cell is a latch.
- The third line closes the cell.

See Also

[defineTimeIntrinsic](#)
[definePortCapacitance](#)
[dbSetCellPortTypes](#)

“Matrices for Timing Functions” in the appendix of the *Static Timing Analyzer User Guide*, which contains an annotated list of timing functions.

dbSetMaxCapacitance

Sets the maximum capacitance for a port. Clock tree synthesis will try to meet this constraint.

Tool

Milkyway

Syntax

```
dbSetMaxCapacitance cellId portName maxCap
```

where the arguments are as follows:

<i>cellId</i>	ID of the cell for which you want to set pin location constraints. If the cell is open, you can substitute the following for <i>cellId</i> : (geGetEditCell).
<i>portName</i>	Name of the port for which you want to set the maximum transition delay.
<i>maxCap</i>	The maximum total net loading for the net that connects to the port. Valid values: Any real number.

Example

This example sets the maximum capacitance for PHI2

```
dbSetMaxCapacitance (geGetEditCell) "PHI2" 0.08
```

dbSetMaxIntraCapDistRatio

Sets the boundary to which the LPE program searches for coupling wire capacitance. The default boundary is three (3), which means LPE searches three times the minimum spacing distance.

Tool

Milkyway

Syntax

```
dbSetMaxIntraCapDistRatio ratio layerNumber
```

where the arguments are as follows:

<i>ratio</i>	Multiple of the minimum spacing distance within which you want the LPE program to search for coupling capacitance. Valid values: Any integer.
<i>layerNumber</i>	Number of the layer on which you want coupling capacitance checked. Valid values: Any layer defined in the technology file.

Example

This example tells the LPE program to search wires on layer 3 for coupling capacitance within four times the minimum spacing distance.

```
dbSetMaxIntraCapDistRatio 4 3
```

dbSetMaxTransition

Sets the maximum transition delay for ports. View the transition delay violations with the `apVerifyDelay` command after detail routing. Clock tree synthesis will also try to meet the transition delay constraint set with this command.

Tool

Milkyway

Syntax

```
dbSetMaxTransition cellId "portName" max_transition
```

where the arguments are as follows:

<i>cellId</i>	ID of the cell for which you want to set transition delay constraints. If the cell is open, you can substitute the following for <i>cellId</i> : <code>(geGetEditCell)</code> .
<i>portName</i>	Name of the port for which you want to set the maximum transition delay.
<i>max_transition</i>	The maximum transition delay you want to apply to a port. Valid values: Any real number.

Example

This example sets the maximum transition delay for port Y to 0.05.

```
dbSetMaxTransition (geGetEditCell) "Y" 0.05
```

The Design Constraints Verifier window produced by the `aprVerifyDelay` shows the constraints as follows:

```
*** Transition Delay Violation ***
Driving Pin Name <From Pin> Rise Fall (Constraint)
U182/Y (NAND4X1) <A> 0.1771 0.1909 (0.0500)
U182/Y (NAND4X1) <B> 0.1624 0.1794 (0.0500)
U182/Y (NAND4X1) <C> 0.1500 0.1711 (0.0500)
U182/Y (NAND4X1) <D> 0.1422 0.1655 (0.0500)
U188/Y (NAND4X1) <A> 0.1551 0.1670 (0.0500)
U188/Y (NAND4X1) <B> 0.1380 0.1526 (0.0500)
U188/Y (NAND4X1) <C> 0.1291 0.1473 (0.0500)
U188/Y (NAND4X1) <D> 0.1202 0.1405 (0.0500)
```

dbSetNetCriticality

Lets you set a net criticality, which determines whether the router will use the most direct route for a net.

Tool

Milkyway

Syntax

```
dbSetNetCriticality cellId netName value
```


where the arguments are as follows:

<i>cellId</i>	ID of the cell for which you want to set the maximum capacitance. Valid values: ID of any cell in the open library. If the cell is open, you can substitute the following for <i>cellId</i> : (geGetEditCell).
<i>netName</i>	Name of the net for which you want to set the maximum capacitance. Valid values: Name of any net in the specified cell.
<i>value</i>	Number from 0 to 7 to indicate the criticality. 0 indicates that the net is not critical. This is the default for all nets. 7 indicates that the net is the most critical. In this case, the router attempts to route the net with the shortest wire length possible.

See Also

[dbDumpAllCriticalNets](#) writes critical nets to a file. You can then edit the file and reload the edited net criticalities using the `load` command.

dbSetNetSubTypeByName

Sets a subtype for the net specified by its name.

Syntax

```
dbSetNetSubTypeByName cellId "netName" "netType"
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

netName

Names of any net in the open cell

netType

Net subtype

Valid values:

- core
- pad
- coreAndPad
- CTSNet
- CTSRootNet
- nonCTSNet

dbSetNetTrace

Sets and unsets net tracing for the specified net.

Syntax

```
dbSetNetTrace <objectId cellId> <string netName>  
<boolean onOff> [boolean pattern]
```

where the arguments are as follows:

cellId - Database object ID of the cell

netName - Name or pattern name of the net.

onOff - #t to set net tracing, #f to unset net tracing.

pattern - #t if the specified net name is a pattern name, #f if the specified net name is the actual name.

Description

The `dbSetNetTrace` function sets and unsets net tracing for the specified net.

If you use a pattern name, all nets that match the pattern name are set or unset. Regular expression pattern matching is used by default, unless you select UNIX style pattern matching with the `setPatternAsWildcard` function.

See also `geSetNetTrace` and `dbSetNetTraceById`.

Example

```
dbSetNetTrace (geGetEditCell) "VDD" $t #f
```

dbSetNetTraceById

Sets and unsets net tracing for the specified nets.

Syntax

```
dbSetNetTraceById <objectId cellId> <boolean onOff>  
<list netIdList>
```

where the arguments are as follows:

cellId - Database object ID of the cell

onOff - #t to set net tracing, #f to unset net tracing.

netIdList - Database object ID list of the nets

Description

The `dbSetNetTraceById` function sets and unsets net tracing for the specified nets. Each net listed by its database object ID is set or unset accordingly.

See also `geSetNetTrace` and `dbSetNetTrace`.

Example

```
dbSetNetTraceById (geGetEditCell) #t '(4000 5000)
```

dbSetNetType

Specifies the type of a net.

Syntax

```
dbSetNetType cellId netId "netType"
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the net information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

netId

Database object ID of the net to which the information applies

Valid values:

Net ID of any net in the specified cell

netType

Type that applies to the specified net

Valid values:

- power

The net is a power net

- ground

The net is a ground net

- clock

The net is a clock net

- signal

The net is a signal net

dbSetNetTypeByName

Specifies the type of a net by name.

Syntax

```
dbSetNetTypeByName cellId "netName" "netType"
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the net information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

netName

Name of the net to which the information applies

Valid values:

Name of any port in the specified cell

netType

Type that applies to the specified net

Valid values:

- power

The net is a power net

- ground

The net is a ground net

- clock

The net is a clock net

- signal

The net is a signal net

dbSetOperatingVoltage

The command has been replaced by `astSetVoltageGroup`.

dbSetPathConstraint

Sets the minimum and maximum rise and fall delays for a path. This function sets values to specify that the signal propagating through a defined path should arrive

- No sooner than
 - *riseMin* for a rising signal at the endpoint of the path
 - *fallMin* for a falling signal at the endpoint of the path
- No later than
 - *riseMax* for a rising signal at the endpoint of the path
 - *fallMax* for a falling signal at the endpoint of the path

Syntax

```
dbSetPathConstraint riseMin riseMax fallMin fallMax  
"startPoint" "endPoint" ['({interimPoint} ...)]
```

where the arguments are as follows:

riseMin

Minimum delay of the transition of the input to the rise of the output

Valid values:

Any floating-point number less than *riseMax*

riseMax

Maximum delay of the transition of the input to the rise of the output

Valid values:

Any floating-point number greater than *riseMin*

fallMin

Minimum delay of the transition of the input to the fall of the output

Valid values:

Any floating-point number less than *fallMax*

fallMax

Maximum delay of the transition of the input to the fall of the output

Valid values:

Any floating-point number greater than *fallMin*

startPoint

Name of the first pin along the path

Valid values:

If the pin is a top-level pin, specify the name of the pin. If the pin is on a cell instance, specify the name of the pin in the following format:

pinName/instanceName

where *pinName* is the name of the pin and *instanceName* is the name of the cell instance.

endPoint

Name of the last pin along the path

Valid values:

Same as *startPoint*

interimPoint

Name of an interim pin along the path

Valid values:

Same as *startPoint*

When specifying multiple pins, specify them in proper order and separate the names with spaces.

Note:

The square brackets ([]) indicate that this argument is optional. If you are setting a constraint between two pins with no interim pins, you do not need to specify this argument.

Example 1

```
dbSetPathConstraint 2.0 3.0 2.1 3.1 "P1/I255" "P20" ' ("P1  
I133" "P2/I115")
```

This function sets a constraint for a path from pin P1 on cell instance I255 to pin P1 on cell instance I133 to pin P2 on cell instance I115 to the top-level pin P20. During timing-driven placement, Astro attempts to place cells such that a signal propagating through the path arrives

- Rising no sooner than 2.0 ns and no later than 3.0 ns
- Falling no sooner than 2.1 ns and no later than 3.1 ns

Example 2

```
dbSetPathConstraint 2.0 3.2 1.9 3.1 "P4/I135" "P12"
```

This function sets a constraint for a direct path from pin P4 on cell instance I135 to the top-level pin P12. During timing-driven placement, Astro attempts to place cells such that a signal propagating through the path arrives

- Rising no sooner than 2.0 ns and no later than 3.2 ns
- Falling no sooner than 1.9 ns and no later than 3.1 ns

See Also

“Matrices for Timing Functions” in the appendix of the *Static Timing Analyzer User Guide* contains an annotated list of timing functions.

dbSetPinAccessDir

Specifies the access direction of a pin.

Syntax

```
dbSetPinAccessDir cellId pinId "accessDir"
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the pin information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

pinId

Database object ID of the pin to which the information applies

Valid values:

Pin ID of any pin in the specified cell

accessDir

Direction you want Astro to use when routing the pin

Valid values:

- L, which means that the pin is accessible only from the left
- R, which means that the pin is accessible only from the right
- U, which means that the pin is accessible only from above
- D, which means that the pin is accessible only from below
- LR, which means that the pin is accessible from the left and right

- LU, which means that the pin is accessible from the left and above
- LD, which means that the pin is accessible from the left and below
- RU, which means that the pin is accessible from the right and above
- RD, which means that the pin is accessible from the right and below
- UD, which means that the pin is accessible from above and below
- LRUD, which means that the pin is accessible from all directions
- LRU, which means that the pin is not accessible from below
- LRD, which means that the pin is not accessible from above
- LUD, which means that the pin is not accessible from the right
- RUD, which means that the pin is not accessible from the left

dbSetPinAccessDirByName

Specifies the access direction of a pin.

Syntax

```
dbSetPinAccessDirByName cellId "pinName" "accessDir"
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the pin information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

pinName

Name of the pin to which the information applies

Valid values:

Name of any pin in the specified cell

accessDir

Direction you want Astro to use when routing the pin

Valid values:

- L, which means that the pin is accessible only from the left
- R, which means that the pin is accessible only from the right
- U, which means that the pin is accessible only from above
- D, which means that the pin is accessible only from below
- LR, which means that the pin is accessible from the left and right

- LU, which means that the pin is accessible from the left and above
- LD, which means that the pin is accessible from the left and below
- RU, which means that the pin is accessible from the right and above
- RD, which means that the pin is accessible from the right and below
- UD, which means that the pin is accessible from above and below
- LRUD, which means that the pin is accessible from all directions
- LRU, which means that the pin is not accessible from below
- LRD, which means that the pin is not accessible from above
- LUD, which means that the pin is not accessible from the right
- RUD, which means that the pin is not accessible from the left

dbSetPinLayerNumber

Specifies the layer number of a pin and creates a pin.

Syntax

```
dbSetPinLayerNumber cellId pinId layerNumber
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the pin information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

pinId

Database object ID of the pin to which the information applies

Valid values:

Pin ID of any pin in the specified cell

layerNumber

Number of the cell layer on which the pin is to be placed

Valid values:

Any layer defined in the cell library’s technology file

dbSetPinLocation

Sets pin location constraints. Generally, you use this function for rectilinear designs. For other designs, you can use the TDF command pin.

Tool

Milkyway

Syntax

```
dbSetPinLocation cellId "pinName" "layer" pointArray
```

where the arguments are as follows:

<i>cellId</i>	ID of the cell for which you want to set pin location constraints. If the cell is open, you can substitute the following for <i>cellId</i> : (geGetEditCell).
<i>pinName</i>	Name of the pin to be constrained. Valid values: Name of any pin in the design.
<i>layer</i>	Name of the layer on which you want the pin. Valid values: Name of any layer defined in the technology file.
<i>pointArray</i>	Coordinates to indicate the location and size of the pin in the format: '((LLX LLY) (URX URY)) where <i>LLX</i> and <i>LLY</i> are the coordinates of the lower left corner of the pin, and <i>URX</i> and <i>URY</i> are the coordinates of the upper right corner of the pin.

Example

```
dbSetPinLocation (geGetEditCell) "p1" "m1" '((0 10) (10 20))
```

This example instructs the Synopsys tool to place pin p1 on layer m1 with its lower left corner at point (0, 10) and upper right corner at point (10, 20).

dbSetPinPointArray

Sets the point array of the specified pin.

Syntax

```
dbSetPinPointArray <objectId cellId> <objectId pinId>  
<list pointArray>
```

where the arguments are as follows:

cellId - Database object ID of the cell

pinId - Database object ID of the pin

pointArray - List of points

Description

The `dbSetPinPointArray` function sets the point array of the specified pin. The point array describes the physical geometry of the pin.

The number of points in the point array must match the number of points that the pin is allowed to have.

For rectangular pins, this is usually two points. The first point represents the lower left corner and the second point represents the upper right corner of the pin.

For polygonal pins, the point array describes each point of the pin geometry. The number of allowable points can vary.

Example

```
dbSetPinPointArray (geGetEditCell) #x3500 `((0 0) (800 800))
```

dbSetPinStatusFlag1

Specifies the follow pin information for the pin of a cell.

Syntax

```
dbSetPinStatusFlag1 cellId pinName flag
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the pin information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

pinName

The name of a pin in the cell

flag

The desired flag value

Valid values:

- "pinFollowInRow"
- "pinFollowInCol"

- "pinNoFollow"

dbSetPlanGroupAndStatus

Sets an existing floorplan group and its status.

Syntax

```
dbSetPlanGroupAndStatus <objectId cellId>  
    <string planGroupName> <int groupType>  
    <int targetPercent> <int planStatus>  
    <int rigidity> <int exclusive> <int color>  
    <list pointArray>
```

where the arguments are as follows:

cellId

Database object ID of the design in which the floorplan group exists.

planGroupName

The name of the floorplan group that you wish to set.

groupType

The floorplan group type.

Valid values are:

- 0 soft plan group
- 1 hard plan group

targetPercent

The floorplan target utilization percentage. The value is specified in units of one hundredth of a percent (for example, 7 is 0.07%, 70 is 0.7%, 700 is 7.0%, and 7000 is 70.0%).

planStatus

The floorplan group status.

Valid values are:

- 1 cell placed
- 2 invisible
- 4 standard cell move deferred
- 8 macro cell move deferred
- 16 placement fixed

rigidity

The floorplan group rigidity. Valid values include all integers from 0 through 10. Values less than 0 default to 0. Values greater than 10 default to 10.

exclusive

The floorplan group exclusivity.

Valid values are:

- 0 not exclusive
- 1 exclusive

color

The floorplan group display color index. Valid values include all color indices defined in the library technology file.

pointArray

A list of points describing the floorplan group boundary. The list can consist of two points describing two opposite vertices of a rectangular boundary, or it can consist of more than two points describing the vertices of a rectilinear boundary.

Description

The floorplan group is created in the specified cell with the specified parameters.

Example

```
dbSetPlanGroupAndStatus (geGetEditCell) "A" 0 7000 1 1 0 28  
'((0 0) (100 100))
```

See Also

[dbCreatePlanGroupAndStatus](#)

dbSetPlanGroupBBox

Sets the bounding box for the specified floor plan group.

Syntax

```
dbSetPlanGroupBBox <objectId cellId> <objectId planGroupId>  
<list pointArray>
```

where the arguments are as follows:

cellId - Database object ID of any opened cell.

planGroupId - Database object ID of the floor plan group.

pointArray - List of two points that define the bounding box

Description

The `dbSetPlanGroupBBox` function sets the bounding box for the specified floor plan group. The specified points can be either the lower-left and upper-right points or the upper-left and lower-right points.

Example

```
dbSetPlanGroupBBox (geGetEditCell) #x4ABCD  
                    `((100 100) (200 200))
```

dbSetPlanGroupBBoxByName

Sets the bounding box for the specified floor plan group.

Syntax

```
dbSetPlanGroupBBoxByName <objectId cellId>  
<string planGroupName><list pointArray>
```

where the arguments are as follows:

`cellId` - Database object ID of any opened cell.

`planGroupName` - Name of the floor plan group.

`pointArray` - List of two points that define the bounding box

Description

The `dbSetPlanGroupBBoxByName` function sets the bounding box for the specified floor plan group. The specified points can be either the lower-left and upper-right points or the upper-left and lower-right points.

Example

```
dbSetPlanGroupBBoxByName (geGetEditCell) "group_one"  
                          `((100 100) (200 200))
```

dbSetPolygonPointArray

Changes the point array in the polygon record. The length of the new point array must match the old length.

Syntax

```
dbSetPolygonPointArray cellId polygonId pointArray
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the polygon information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

polygonId

Database object ID of the polygon to which the point array information applies

Valid values:

Polygon ID of any polygon in the specified cell

pointArray

Coordinates of the polygon

Valid values:

$((x_1 \ y_1), (x_2 \ y_2), \dots, (x_n \ y_n))$

dbSetPortInstLocalStatus

Sets the local status for the specified port instance.

Syntax

```
dbSetPortInstLocalStatus <objectId cellId> <objectId  
portInstId>  
<string portLocalStatus>
```

where the arguments are as follows:

cellId - Database object ID of any opened cell.

portInstId - Database object ID of the port instance

portLocalStatus - Status of the port instance

Description

The `dbSetPortInstLocalStatus` function sets the local status for the specified port instance. The only valid status strings are "autoTie," "tieHigh," and "notChild."

Example

```
dbSetPortInstLocalStatus (geGetEditCell) #x12345 "tieHigh"
```

dbSetPortDirection

Specifies the direction of a port.

Syntax

```
dbSetPortDirection cellId "portName" "portDir"
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the port information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

portName

Name of the port to which the direction specification applies

Valid values:

Any port in the specified cell

portDir

Direction that applies to the port

Valid values:

- input
- output
- inputoutput
- tristateoutput
- tristateinputoutput
- none

dbSetPortInstNetId

Specifies the net ID of a port instance without making the port instance a member of the net. Use this function when connecting power and ground nets to the port instance.

Syntax

```
dbSetPortInstNetId cellId portInstId netId
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the port instance information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

portInstId

Database object ID of the port instance to which the net information applies

Valid values:

Port instance ID of any port instance in the specified cell

netId

Database object ID of the net

Valid values:

Net ID of any net connected to the specified port instance

dbSetRefLibControl

Links the reference library control file to a library. The hierarchical structure of control file will replace the reference libraries settings stored in the library. It will also show the difference between the original reference setting and loaded reference setting.

Tool

Milkyway

Syntax

```
dbSetRefLibControl "libName" "fileName"
```

where the arguments are as follows:

<i>libName</i>	Library name of which the hierarchical reference Library structure to be loaded in.
<i>fileName</i>	Name of the file name in which you want to load the hierarchical reference library structure.

Example

```
dbSetRefLibControl "A" "control.ref"
```

Suppose the "control.ref" has content as follows:

```
LIBRARY A
```

```
REFERENCE B
```

```
REFERENCE C
```

```
REFERENCE D
```

```
LIBRARY B
```

```
REFERENCE G
```

```
LIBRARY C
```

```
REFERENCE F
```

where the original setting is

```
LIBRARY A
```

```
REFERENCE B
```

```
REFERENCE C
```

```
REFERENCE D
```

```
LIBRARY B
```

```
REFERENCE E
```

```
LIBRARY C
```

```
REFERENCE F
```

then the reference library "G" will replace the library "E".

See Also

[dbDumpRefLibControl](#)

[dbClearRefLibControl](#)

[dbCheckRefLibControl](#)

[dbSetRefControlFileMode](#)

dbSetRefControlFileMode

Enables the [dbCheckRefLibControl](#), [dbClearRefLibControl](#), [dbDumpRefLibControl](#), and [dbSetRefLibControl](#) functions for control file operation.

Tool

Milkyway

Syntax

```
dbSetRefControlFileMode [#t/#f]
```

See Also

[dbCheckRefLibControl](#)
[dbClearRefLibControl](#)
[dbDumpRefLibControl](#)
[dbSetRefLibControl](#)

dbSetReqArrivalTime

This timing function has been replaced by `tdfSetOutputDelay`.

dbSetScanChainDefaultPriority

Sets the default priority for scan chain groups. If a scan chain edge constraint references a cell instance that is not explicitly placed in a scan chain group, the Synopsys tool creates a scan chain group containing only that cell instance. The tool assigns it a priority equal to the default priority (except for the primary input scan cell, which is always at the beginning of the scan chain, and the primary output scan cell, which is always at the end of the scan chain).

Syntax

```
dbSetScanChainDefaultPriority cellId priority "chainName"
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library.
- `(geGetEditCell)`

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

priority

Default priority for cell instances that you place in an edge constraint but do not assign to a scan chain group

Valid values:

Any integer from 0 to 160 (0 is the lowest priority and 160 is the highest priority)

chainName

Name of the scan chain for which you are setting a default priority

Valid values:

Name of any scan chain in the open cell

dbSetSkipRefLibReadLock

Bypasses the lock on a reference library when you are certain the reference library is read-only. No one else can modify the library.

Syntax

```
dbSetSetSkipRefLibReadLock
```

where the Boolean value is `#t` or `#f`:

- `#t` disables the lock on the reference library.

- #f turns the lock back on.

dbSetStdCell

Sets the cell flag of all cell instances of the cell (of the specified cell ID) to standard cell.

Syntax

```
dbSetStdCell cellId
```

where *cellId* indicates the internal ID of any open cell in the current library to which you want to set the cell flag.

See Also

[dbSetIOPad](#)

dbSetStdCellOrient

Sets the allowable placement orientations of a standard cell master. This command takes effect during execution of the [geMove](#) command or during the standard cell placement process.

Note:

This command also applies to AstroGA.

Syntax

```
dbSetStdCellOrient libName cellName orientation
```

where the arguments are as follows:

libName

Name of the library containing the cell to which the orientation specification applies

Valid values:

Name of any library in the current directory

cellName

Name of the cell to which the orientation specification applies

Valid values:

- Name of any cell in the specified library
- Pattern matching to select all cells with names that match a pattern

For example, typing *TOP.** specifies all cells with names that begin with TOP. For information about pattern matching, see the section on specifying objects by pattern in *Physical Implementation Getting Started*.

orientation

Valid values:

- One of {Any Reflect Norelect}
- Or a combination of {R0 R0_MX R0_MY R180 R90 R90_MX R90_MY R270}

For example, "R0 R0_MY R180 R90_MX"

These valid values are case-insensitive and are separated by blanks.

Examples

```
dbSetStdCellOrient "GOLDROAD" "DFFRP" "Reflect"  
  (same as "R0_MX R180")  
dbSetStdCellOrient "GOLDROAD" "INV1" "R0 R0_MY R180"  
dbSetStdCellOrient "GOLDROAD" "NAND3" "R0 R180"
```

The third example instructs Astro to allow only R0 and R180 orientations during the placement process or during execution of the `geMove` command.

See Also

[geMove](#)

dbSetTechFileName

Attaches a technology file to a library. If `dbSetTechFileName` succeeds, the Synopsys tool returns *#t*.

Prerequisite

The library to which you want to attach a technology file must be open in write (W) mode.

Syntax

```
dbSetTechFileName "techFileName"
```

where *techfileName* is the name of the technology file you want to attach to the current library.

See Also

[dbReplaceTechFile](#)

dbSetUserLogFile

Starts or stops logging to a specified file. This command does not affect the normal logging of the application. It only enables or disables logging to a specified file.

Syntax

```
dbSetUserLogFile <string fileName>
```

where the argument is as follows:

fileName

Name of the file to record the log information. This file is opened for overwriting.

Description

The `dbSetUserLogFile` command starts or stops logging to a specified log file (depending on whether a *fileName* argument is specified). This command does not affect the normal logging of the application.

To begin logging, enter the following command with a specified *fileName* argument in the command window:

```
dbSetUserLogFile "my_log_file"
```

To stop logging to the specified log file, enter the following command without the *fileName* argument in the command window:

```
dbSetUserLogFile
```

Note that only one log file is supported at a time for one logging session. If you want to log to another log file, you must first end logging on the current log file. Then enter the `dbSetUserLogFile` command again, specifying a different log file name.

dbSetUseUmask

Use *dbSetUseUmask* in conjunction with [dbSetLibDefaultCreationAccess](#) to cause the tool to create libraries and cells based on your umask setting.

Tool

Milkyway

Syntax

```
dbSetUseUmask mode
```

where the mode is as follows:

<i>mode</i>	1 use the umask
	0 do not use the umask

Example

```
dbSetUseUmask 1
dbSetLibDefaultCreationAccess 0
```

In this example, the library and cells created will have the permissions set according to your umask setting.

See Also

[dbSetLibDefaultCreationAccess](#) – Enables you to set an access value to a newly created library.

dbSetXtalkLengthWeight

For crosstalk constraints of the 'parallel length' type, see [dbDefineXtalkConstraint](#). You can assign different weights to different layers and also assign the maximum distance between two wires when the cross between them has to be considered. If this is not specified, the default value for the weight is 1.0 and the default value of the maximum distance is the pitch of the layer.

Tool

Milkyway

Syntax

```
(dbSetXtalkLengthWeight cellId "layerName" maxDist weight)
```

where the arguments are as follows:

<i>cellId</i>	ID of the cell for which you want to assign weight to. Valid values: ID of any cell in the open library. If the cell is open, you can substitute the following for <i>cellId</i> : (<i>geGetEditCell</i>).
<i>layerName</i>	Name of the layer for which you want to determine weight.
<i>maxDist</i>	The max span between two wires when the crosstalk effect needs to be considered.
<i>weight</i>	The weight of the specified layer.

Examples

```
(dbSetXtalkLengthWeight cellId "meta11" 2.0 0.9)  
(dbSetXtalkLengthWeight cellId "meta12" 3.0 1.1)
```

See Also

[dbClearCellXtalkConstraints](#)
[dbDefineXtalkConstraint](#)
[dbDefineAbsXtalkConstraint](#)
[dbClearCellXtalkLengthWeight](#)
[dbAssignNetXtalkClass](#)
[dbAssignNetXtalkConstraints](#)
[dbAllowToAddPGIOTextdbDumpXtalkConstraints](#)
[axgSetXtalkRouteOptions](#)

dbSkipRouteOnNet

Specifies the net that should not be routed by the router.

Tool

Milkyway

Syntax

```
dbSkipRouteOnNet dbId string
```

where the arguments are as follows:

<i>dbId</i>	ID of the cell. Valid values: ID of any cell in the open library. If the cell is open, you can substitute the following for <i>cellId</i> : (<i>geGetEditCell</i>).
<i>string</i>	The name of the net not to be routed.

Examples

```
dbSkipRouteOnNet (geGetEditCell) "net 4"
```

dbSleep

Suspends the current Milkyway session for the specified number of microseconds.

Syntax

```
dbSleep <int microseconds>
```

where the argument is as follows:

microSeconds - Number of microseconds to suspend the current session.

Description

The `dbSleep` function suspends the current Milkyway session for the specified number of microseconds. The specified value must be a positive, non-zero integer. When Milkyway is suspended, it will not respond to user input.

Example

```
dbSleep 10000000
```

The above function call will suspend Milkyway for 10 seconds.

dbSpecifyLRFiller

Defines the filler cells to be used for filling to the left and right of the specified standard cells. You can write the left-and-right filler rules that you define to an output file with the `dbDimpLRFillerRule` command.

Tool

Milkyway

Syntax

```
dbSpecifyLRFiller topCellId fillerL fillerR cellMasterNames
```

where the arguments are as follows:

<i>topCellId</i>	The database object ID of the top cell.
<i>fillerL</i>	The filler cell to be placed to the left of the named standard cell.
<i>fillerR</i>	The filler cell to be placed to the right of the named standard cell.
<i>cellMasterNames</i>	The names of the standard cell masters (the names must be separated by commas).

Example

```
dbSpecifyLRFiller (geGetEditCell) "FILL1" "FILLH1" "BUFDX1"
```

This syntax specifies that filler cell FILL1 is to be inserted to the left of standard cell BUFDX1 and filler cell FILLH1 is to be inserted to the right of standard cell BUFDX1 during the filler cell insertion process.

See Also

[dbClearLRFillerRule](#)

[dbDumpLRFillerRule](#)

dbSpecifyVTFiller

Defines special filler cells to be used for filling between the cells of certain voltage threshold types.

This command creates the voltage threshold rule table information, which maps the filler cells to various combinations of different voltage threshold cells. You apply the information in the table to your design by using the `axgAddFillerCell` or `axgAddFillerCellByArea` command.

The voltage threshold rule table information allows filler cells to honor one or more voltage threshold rules. Support for multiple rules, in the form of a two-dimensional table, is useful when your design requires that filler cells be inserted between multiple voltage threshold types.

Astro supports an empty string as the default type for *vtType1* or *vtType2*. This concept allows a filler cell to be placed to the left or right of a voltage threshold cell for the following conditions:

- End of a row
- Beginning of a row
- Space between a nonstandard cell (a macro cell) and a standard cell (a voltage threshold cell)
- Space between a placement blockage and a standard cell (a voltage threshold cell)

Tool

Milkyway

Syntax

```
dbSpecifyVTFiller cellId vtTypeA vtTypeB fillerCellNames
```

where the arguments are as follows:

<i>cellId</i>	The database object ID of the cell.
<i>vtTypeA</i>	The first of two voltage threshold types to be defined.
<i>vtTypeB</i>	The second of two voltage threshold types to be defined.
<i>fillerCellNames</i>	The names of the filler cells to be used for filling (the names must be separated by commas).

Example 1

```
dbSpecifyVTFiller (geGetEditCell) "vtType1" "vtType2"  
                    "fillerA, fillerB, fillerC"
```

Example 2

The following syntax defines an empty string as the default type for *vtType2*.

```
dbSpecifyVTFiller (geGetEditCell) "vtType1" "  
                    "fillerA, fillerB, fillerC"
```

See Also

[axgAddFillerCell](#)

[axgAddFillerCellByArea](#)

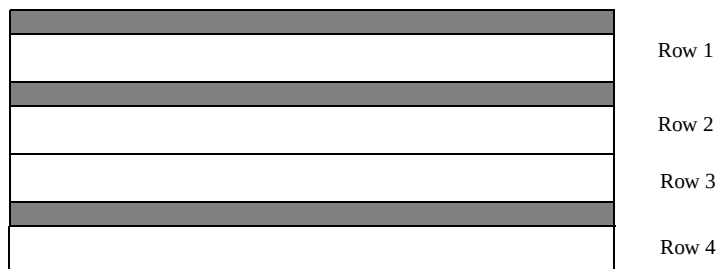
[dbClearVTFiller](#)

[dbDumpVTFiller](#)

[dbDumpAllVTFillers](#)

dbStartCellRowDoubleBack

Specifies the placement orientation of the cell row. When doubleback is set, Astro abuts the cell rows. Rows 2 and 3 in the figure are abutted.



Prerequisites

Before executing `dbStartCellRowDoubleBack`, use the [dbDeleteCellRowDoubleBack](#) to reset cell row placement.

Syntax

```
dbStartCellRowDoubleBack cellId regionNum startCellRowNo
```

where the arguments are as follows:

cellId

Database object ID of the cell in which the cell row information applies

Valid values:

- Cell ID of any open cell in the current library
- (`geGetEditCell`)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

regionNum

Number of the cell region in which the cell row information applies

Valid values:

Positive integers that identify regions inside the specified cell

startCellRowNo

Number of the cell row where cell abutment is to begin

Valid values:

Positive integers that identify cell rows inside the specified region

dbStoreCellAttachedFile

Attaches a file to a cell. If `dbStoreCellAttachedFile` succeeds, the Synopsys tool returns *#t*.

Syntax

```
dbStoreCellAttachedFile cellId "attachedFileName"  
"pseudoName" "comment"
```

where the arguments are as follows:

cellId

Database object ID of the cell to which you want to attach a file

Valid values:

- Cell ID of any open cell in the current library
- `(geGetEditCell)`

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

attachedFileName

Name of the file to be attached to a cell

Valid values:

Name of any file in the current library

pseudoName

User name you want to associate with the attached file

Valid values: Any text string of 16 characters or fewer

comment

Text that is to be associated with the attached file

Valid values:

Any text string of 32 characters or fewer

dbStoreLibAttachedFile

Attaches a file to a specified library. If `dbStoreAttachedFile` succeeds, the Synopsys tool returns `#t`.

Prerequisite

The library to which you want to attach a file must be open in write (W) mode.

Syntax

```
dbStoreLibAttachedFile libId "attachedFileName"  
"pseudoName" "comment"
```

where the arguments are as follows:

libId

Database object ID of the current library to which the file attaches

Valid values:

- Library ID
- (dbGetCurrentLibId)

For more information, see “Specifying Libraries” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

attachedFileName

Name of the actual cell to be attached to a library

Valid values:

Name of any cell in the current directory

pseudoName

Name that is to be associated with the actual cell

Valid values:

Any text string of 16 characters or fewer

comment

Text that is to be associated with the attached file

Valid values:

Any text string of 32 characters or fewer

dbSwapCellViewVersions

Swaps the specified versions of the a cell view.

Syntax

```
dbSwapCellViewVersions <string libraryName> <string  
cellName>  
<string viewName> <int versionA> <int versionB>
```

where the arguments are as follows:

libraryName - Name of the library containing the cell view

cellName - Name of the cell

viewName - Name of the view

versionA - One of the version numbers to swap

versionB- One of the version numbers to swap

Description

The `dbSwapCellViewVersions` function swaps the specified versions of a cell view. The two specified versions must be different numbers. When completed, versionA will be versionB and versionB will be versionA of the cell view. The attached files corresponding to these versions are also swapped.

When parent cells are opened (in write mode), its cell instance masters are automatically updated to reflect changes in the latest child cell views. The latest version is the highest existing version. Therefore, if a child cell view's highest version was swapped with a lower version, the parent cell uses the cell view version that was the previously lower version.

Example

```
dbSwapCellViewVersions "mylib" "top" "CEL" "3" "2"
```

dbSyncRefLib

Updates all reference libraries of the current opened library.

Syntax

```
dbSyncRefLib
```

Arguments

No argument is required.

Description

The `dbSyncRefLib` command saves all reference library changes. If there are changes, at run time, it reopens the reference libraries for synchronization.

Note that there is no need to enter any argument for the command. The command itself finds the current opened library to synchronize the reference libraries.

Example

```
dbSyncRefLib
```

dbTruncateCellText

Removes all characters appearing after the first occurrence of the character in *startChar* from all text objects appearing in a cell. Truncation includes the character in *startChar*.

Tool

Milkyway

Syntax

```
dbTruncateCellText cellId "startChar"
```

where the arguments are as follows:

<i>cellId</i>	Database object ID of the cell to change.
<i>startChar</i>	Characters after which all characters are removed. If <i>startChar</i> is more than one char, then only the first character in the string will be used as a start char to truncate in the text.

Example

```
dbTruncateCellText (geGetEditCell) "T"
```

See Also

[dbTruncateText](#)

dbTruncateText

Removes all characters appearing after the first occurrence of the character in *startChar* from all text objects appearing in all cells of a library. Truncation includes the character in *startChar*.

Tool

Milkyway

Syntax

```
dbTruncateText libId "startChar"
```


where the arguments are as follows:

<i>libId</i>	Database object ID of the library.
<i>startChar</i>	If <i>startChar</i> is more than one char, then only the first character in the string will be used as a <i>start char</i> to truncate in the text.

Example

```
dbTruncateText 1 "T"
```

See Also

[dbTruncateCellText](#)

dbUpdateCellInstNumberOfCellRow

Updates the number of cell instances of the “cell row” object for the specified cell.

Syntax

```
dbUpdateCellInstNumberOfCellRow cellId
```

Arguments

cellId - ID of the cell to update the cell row object. Integer value

Description

Any deletion or addition of cell instances in a cell causes inconsistency to the number of cell instances of rows in the “cell row” object. The command updates the “cell row” object with the number of cell instances in the cell.

Example

```
dbUpdateCellInstNumberOfCellRow (geGetEditCell)
```

dbUpdateCellProperty

Updates the value of the specified property of the cell. The standard cell should be opened in FRAM view.

Syntax

```
dbUpdateCellProperty cellId "propName" value
```

where the arguments are as follows:

cellId

Database object ID of the cell to which the information applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

propName

Name of the cell property to be updated in the specified cell

Valid values:

Name of any property

value

Value to which you want Astro to update the cell property

Valid data types:

- string
- float
- integer
- boolean

Note:

If *propName* exists, the type of the value must match the type of the property. If the property does not exist, the value can be of any type, and the property with the specified value will be created. In the case of a string property, the length of the new string must match the length of the old string.

dbUpdateCellRailProperty

Updates the y-coordinate of an edge of a power or ground rail in a standard cell.

Syntax

```
dbUpdateCellRailProperty cellId railSpec side yCoord
```

where the arguments are as follows:

cellId

Database object ID of the standard cell to which the y-coordinate applies

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

railSpec

Specification of the rail to which the y-coordinate applies

Valid values:

- A string that is the name of the pin (rail)
- An integer that is the number of the rail from the bottom up, starting with 1

side

Side of the rail to which the y-coordinate applies

Valid values:

- “low”

Indicates the bottom side of the rail

- “high”

Indicates the top side of the rail

yCoord

Y-coordinate of the rail edge in database units

See Also

[dbDisplayCellRailProperty](#)

dbUpdateHierarchy

Compares the time stamps of the parent cell and cell instances and updates the cell instances if necessary.

Syntax

`dbUpdateHierarchy cellId`

where the arguments are as follows:

cellId

Database object ID of the parent cell

Valid values:

- Cell ID of any open cell in the current library
- (`geGetEditCell`)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

dbUpdateNetStatus

Updates the status of all nets. Execute this function to correct the status of the net single-port connection.

Syntax

`dbUpdateNetStatus cellId`

where the arguments are as follows:

cellId

Database object ID of the cell to which the net information applies

Valid values:

- Cell ID of any open cell in the current library

- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

dbUpdatePcellDefinition

Updates the pcell with the specified definition.

Syntax

```
dbUpdatePcellDefinition libId pcell_name fileName
```

Arguments

libId - ID of the library to update pcell. Integer value

pcell_name - Name to the pcell to be updated. String value

fileName - Name of the definition used to update the pcell. String value

Description

The `dbUpdatePcellDefinition` command updates the definition of the pcell called “pcell_name” with the specified object called “fileName”.

Example

```
dbUpdatePcellDefinition 1 "pmos" "pObj_name"
```

dbUpdateProperty

Updates the value of the specified property of the specified object in the cell.

Syntax

```
dbUpdateProperty cellId objectId "propName" value
```

where the arguments are as follows:

cellId

Database object ID of the cell in which the information is to be updated

Valid values:

- Cell ID of any open cell in the current library
- (geGetEditCell)

For more information, see “Specifying Cells” in Chapter 4 of the *Milkyway Environment Extension Language Reference Manual*.

objectId

Database object ID of the object to be updated

Valid values:

- *cellInstId*
- *netId*
- *pinId*
- *portId*
- *portInstId*

- *regionPinId*
- *regionPortId*

Note:

This is a partial list of possible values.

propName

Name of the object's property to which the information applies

value

Value to which you want Astro to update the object property

Valid types:

- string
- float
- integer
- boolean

Note:

If the property exists, the type of the value must match the type of the property. If the property does not exist, the value can be of any type, and the property with the specified value will be created. In the case of a string property, the length of the new string must be the same as the length of the old property.

dbVerifyCellEEQClass

Verifies the consistency of the content in the electrical-equivalent class of the library.

Syntax

```
dbVerifyCellEEQClass libId
```

where the argument is as follows:

libId

Database object ID that must be a library ID

dbVerifyCellLEQClass

Verifies the consistency of the content in the logical-equivalent class of the library.

Syntax

```
dbVerifyCellLEQClass libId
```

where the argument is as follows:

libId

Database object ID that must be a library ID

dbVerilogTranslateString

Outputs the string stored in the Milkyway database using a verilog identifier.

Tool

Milkyway

Syntax

```
dbVerilogTranslateString "libName" "string"
```

where the arguments are as follows:

<i>libName</i>	Name of the library for which you want to query. <i>Valid values:</i> Name of any library in the current directory.
<i>string</i>	The identifier you want to query.

Note:

Backslash should be escaped by a second backslash, i.e. "\\" stands for one backslash.

Examples

```
scalar without special characters: "net1"  
dbVerilogTranslateString "DESIGNS" "net1"
```

```
scalar with special characters: "\net\@!"  
dbVerilogTranslateString "DESIGNS" "\\net\\@!"
```

```
single bus bit: "net [0]"  
dbVerilogTranslateString "DESIGNS" "net [0]"
```

```
multiple bits bus: 3 bits bus "net" (from bit 2 to bit 4)  
dbVerilogTranslateString "DESIGNS" "net [2:4]"
```

27

define Commands for Milkyway

This chapter describes commands with the prefix *define*. The commands are listed alphabetically.

- [defineAntennaArea](#)
- [defineBindKey](#)
- [defineBooleanFunction](#)
- [defineCellDontTouch](#)
- [defineCellDontUse](#)
- [defineCellFanoutLoad](#)
- [defineCellMaxCapacitance](#)
- [defineCellMaxFanout](#)
- [defineCellMaxTransition](#)

- `defineCellMinCapacitance`
- `defineCellMinFanout`
- `defineCellMinTransition`
- `defineCondMinClkPeriod`
- `defineCondMinClkPulseWidth`
- `defineCondTimeIntrinsic`
- `defineCondTimeTLU`
- `defineDiodeProtection`
- `defineExtAntennaArea`
- `defineFlipFlopFunction`
- `defineFootPrintEQCell`
- `defineGateSize`
- `defineHierAntennaProp`
- `defineInternalPower`
- `defineLatchFunction`
- `defineLeakagePower`
- `defineMinClkPeriod`
- `defineMinClkPulseWidth`
- `definePad`
- `definePortCapacitance`
- `definePortFeedthru`

- `definePortPhaseDelay`
- `definePortTieHigh`
- `definePortTieLow`
- `defineStaticNoiseMargin`
- `defineStaticNoiseWidth`
- `defineSwitchingPower`
- `defineTimeIntrinsic`
- `defineTimeTLU`
- `defineTriStateFunction`

defineAntennaArea

Defines the area for any pins in the library for computing antenna rules. By default, the antenna checker uses the pin area in the .FRAM view to compute antenna ratio.

If this value is specified, the router uses the value defined by this command instead of the area computed from .FRAM view.

This command can only be in a CLF file and loaded with [auLoadCLF](#).

Tool

Milkyway

Syntax

```
defineAntennaArea "cellName" "portName"  
  ' ("layerName" isSideWall area) ....  
)
```

where the arguments are as follows:

<i>cellname</i>	The name of the cell.
<i>portName</i>	Name of the port.
<i>layerName</i>	Name of the layer in the port.
<i>isSideWall</i>	Indicates whether the area is a side-wall or not. Valid values: #t : area is sideWall. #f : area is a polygon area.

Example

```
(defineAntennaArea "and3x2" "A" '(  
  ("metal1" #t 3.6)  
  ("via1" #t 1.6)  
  ("metal2" #t 2.7)
```

defineBindKey

Lets you reassign commands to a specific key or combination of keys.

Tool

Milkyway

Syntax

```
defineBindKey "key" "command"
```

where the arguments are as follows:

<i>key</i>	The key or combination of keys that you want to assign to a command. Valid values: • Any single key that represents a printing character. Any combination of a single key that represents a printing character with the following keys: • Meta • Control • Any single function key from F1 to F10 or any combination of this type of key with the Meta key. The exceptions are: • Any keys or key combinations used by the window manager • Control-C • Control-Z • Numbers without Meta or Control. • Indicate the non-printing keys as follows: • <meta> for the Meta key • <ctrl> for the Control key • <F#> for any function key, where # is the number of the key
<i>command</i>	Name of the Scheme command or function that you want to assign to the key.

See Also

[listBindKey](#)

defineBooleanFunction

Defines the output pin function of a combinational cell.

Tool

Milkyway

Syntax

```
defineBooleanFunction "cell_name" "output_pin_name"  
    "input_function"
```

where the arguments are as follows:

<i>cellName</i>	The name of the cell whose output pin function you wish to define in boolean values.
<i>output_pin_name</i>	The name of the output pin or inout pin.
<i>input_function</i>	The boolean value of an output pin or inout pin in terms of input pins or inout pins.

Example

```
defineBooleanFunction "aoi4_1" "y" "(!((a b)+(c d)))"
```

defineCellDontTouch

This command has been replaced by astSetDontTouch.

defineCellDontUse

This command has been replaced by astSetDontUse.

defineCellFanoutLoad

Specifies the fanout load value of a cell port.

Tool

Milkyway

Syntax

```
defineCellFanoutLoad "cell_name" "pin_name" float_value
```

where the arguments are as follows:

cellName The name of the cell whose fanout value you wish to define.

pin_name The name of the input pin, output pin, or inout pin of the cell.

float_value Any legal floating-point value.

Example

```
defineCellFanoutLoad "buf1x" "z" 1.0
```

defineCellMaxCapacitance

Specifies the maximal capacitance constraint in cell-pin based.

Tool

Milkyway

Syntax

```
defineCellMaxCapacitance "cell_name" "pin_name" float_value
```

where the arguments are as follows:

<i>cellName</i>	The name of the cell whose maximal capacitance constraint you wish to define for it's cell-pin.
<i>pin_name</i>	The name of the cells input, output, or inout pin.
<i>float_value</i>	Any legal floating-point value.

Example

```
defineCellMaxCapacitance "buf1x" "z" 0.3
```

defineCellMaxFanout

Specifies the maximal fanout constraint in cell-pin based.

Tool

Milkyway

Syntax

```
defineCellMaxFanout "cell_name" "pin_name" float_value
```

where the arguments are as follows:

<i>cell_name</i>	The name of the cell whose maximal fanout constraint you wish to define for it's cell-pin.
<i>pin_name</i>	The name of the cells input, output, or inout pin.
<i>float_value</i>	Any legal floating-point value.

Example

```
defineCellMaxFanout "buf1x" "z" 5.2
```

defineCellMaxFanout

defineCellMaxTransition

Specifies the maximal transition constraint in cell-pin based.

Tool

Milkyway

Syntax

```
defineCellMaxTransition "cell_name" "pin_name" float_value
```

where the arguments are as follows:

<i>cell_name</i>	The name of the cell whose maximal transition constraint you wish to define for it's cell-pin.
<i>pin_name</i>	The name of the cells input, output, or inout pin.
<i>float_value</i>	Any legal floating-point value.

Example

```
defineCellMaxTransition "buf1x" "z" 1.0
```

defineCellMinCapacitance

Specifies the minimal capacitance constraint in cell-pin based.

Tool

Milkyway

Syntax

```
defineCellMinCapacitance "cell_name" "pin_name" float_value
```

where the arguments are as follows:

<i>cell_name</i>	The name of the cell whose minimal capacitance constraint you wish to define for it's cell-pin.
<i>pin_name</i>	The name of the cells input, output, or inout pin.
<i>float_value</i>	Any legal floating-point value.

Example

```
defineCellMinCapacitance "buf1x" "z" 0.0
```

defineCellMinFanout

Specifies the minimal fanout constraint in cell-pin based.

Tool

Milkyway

Syntax

```
defineCellMinFanout "cell_name" "pin_name" float_value
```

where the arguments are as follows:

<i>cell_name</i>	The name of the cell whose minimal fanout constraint you wish to define for it's cell-pin.
<i>pin_name</i>	The name of the cells input, output, or inout pin.
<i>float_value</i>	Any legal floating-point value.

defineCellMinFanout

Example

```
defineCellMinFanout "buf1x" "z" 0.0
```

defineCellMinTransition

Specifies the minimal transition constraint in cell-pin based.

Tool

Milkyway

Syntax

```
defineCellMinTransition "cell_name" "pin_name" float_value
```

where the arguments are as follows:

<i>cell_name</i>	The name of the cell whose minimal transition constraint you wish to define for its cell-pin.
<i>pin_name</i>	The name of the cells input, output, or inout pin.
<i>float_value</i>	Any legal floating-point value.

Example

```
defineCellMinTransition "buf1x" "z" 0.0
```

defineCondMinClkPeriod

Used in a CLF file or on the Astro command line, this command specifies the minimum allowable time for one complete cycle of clock pins under specified conditions.

Note:

Conditional delay allows you to specify different delays of the output pin of a cell with respect to the various states of the input pins.

Tool

Milkyway

Syntax

```
defineCondMinClkPeriod "cellName" "pinName" "enaCond"  
  '(minPeriod nomPeriod maxPeriod)
```

where the arguments are as follows:

<i>cellName</i>	Name of the cell that you are setting the minimum clock period. Valid values: Name of any cell in the library.
<i>pinName</i>	Name of a clock pin instance that you want set the minimum period. Valid values: Name of any instance in the library or use pattern matching (*) to set the period of instances that match the pattern you enter. Use the pattern matching option to select all nets with names that match a pattern..
<i>enaCond</i>	For the timing types PATH, CLOCK_RISING and CLOCK_FALLING: It specifies the logic condition TRUE, when the delay values are applicable. For other timing types, this argument is empty.
<i>minPeriod</i>	Minimum elapsed time between repetitions of the clock (1/frequency). Valid values: Any floating-point number.

<i>nomPeriod</i>	Nominal elapsed time between repetitions of the clock (1/frequency). Valid values: Any floating-point number.
<i>maxPeriod</i>	Maximum elapsed time between repetitions of the clock (1/frequency). Valid values: Any floating-point number.

Note:

The min/max is scaled by the following k_factors:

k_temp_min_period

k_volt_min_period

k_process_min_period

Example

This example specifies the minimum allowable time for one complete cycle of the clock pin “clk” when the condition “CD” is true.

```
defineCondMinClkPeriod "FF2" "clk" "CD" '(0.69 0.69 0.69)
```

See Also

[tdfSetClkDef](#) sets source characteristics for a clock.

defineCondMinClkPulseWidth

Used in a CLF file or on the Astro command line, this command specifies the minimum clock pulse width under specified conditions. The minimum clock pulse width is the minimum allowable time for either the positive or negative phase of the clock pin.

Note:

Conditional delay allows you to specify different delays of the output pin of a cell with respect to the various states of the input pins.

Should supersede [defineMinClkPulseWidth](#), which does not consider k_factor effects or enable conditions.

Tool

Milkyway

Syntax

```
defineCondMinClkPulseWidth "cellName" "pinName"  
{ "High" | "Low" }  
"enaCond" ' (min nom max)
```

where the arguments are as follows:

<i>cellName</i>	Name of the cell that port-to-port timing is specified. Valid values: Name of any cell in the library.
<i>pinName</i>	Name of a pin instance that you want set a minimum pulse. Valid values: Name of any instance in the library or use pattern matching (*) to set the power to instances that match the pattern you enter.
<i>High/Low</i>	Phase of the clock pin that you are setting the minimum allowable time. Valid values: HIGH indicates the positive phase of the clock pin, or the clock pulse width when the clock is equal to one (1). Low indicates the negative phase of the clock pin, or the clock pulse width when the clock is equal to zero (0).

<i>enaCond</i>	For the timing types PATH, CLOCK_RISING and CLOCK_FALLING: It specifies the logic condition TRUE, when the delay values are applicable. For other timing types, this argument is empty.
<i>min</i>	Minimum elapsed time between repetitions of the clock (1/frequency). Valid values: Any floating-point number.
<i>nom</i>	Nominal elapsed time between repetitions of the clock (1/frequency). Valid values: Any floating-point number.
<i>max</i>	Maximum elapsed time between repetitions of the clock (1/frequency). Valid values: Any floating-point number.

Note:

The scaling factors for min/max are derived from the following:

k_temp_min_pulse_width_high

k_volt_min_pulse_width_high

k_process_pulse_width_high

Example

This example specifies the minimum allowable time for the negative (low) phase of the clock pin “CP” when the condition “CD” is true.

```
defineCondMinClkPulseWidth "FF1" "CP" "LOW" "CD"
' (0.52 0.52 0.52)
```

defineCondTimeIntrinsic

Used in a CLF file or on the Astro command line, this command specifies the parameters associated with the arc between a specified source port and a specified destination port on a master cell, including the following:

- Unateness
- Timing type
- Intrinsic delay or timing rules (depending on the timing type)
- Slope factor for adjusting the intrinsic delay for the arc due to transition delay on the source port. This delay adjustment models the changing delay characteristics of a circuit when the driving signal has a faster or slower rise/fall time.
- Output resistance

You need to specify these parameters for every arc included in the path constraints.

Note:

Conditional delay allows you to specify different delays of the output pin of a cell with respect to the various states of the input pins.

To calculate the slope factor, you need to understand the model Astro uses to calculate the actual intrinsic delay between a source port and a destination port. The following terms and example explain the model:

- Transition delay is the time it takes for a signal to change state due to loading at an output port. The formula is as follows:

`output resistance 5 load`

Note:

When specifying output resistance, you should already have the threshold voltage factored in.

- Connect delay is the time it takes for a state transition to propagate through an interconnect. There is one connect delay for each output-input pair.
- Intrinsic delay is the internal delay between the source port and destination port when the output loading is zero (0).
- Actual intrinsic delay is the intrinsic delay adjusted for changing delay characteristics. The formula is as follows:

Intrinsic delay	+	Slope delay
(a constant)		(calculated by Synopsys tool)

- Slope factor is the number you provide for each operating condition (riseMin, riseMax, fallMin, fallMax) so that Astro can calculate the slope delays for each. The formula is as follows:

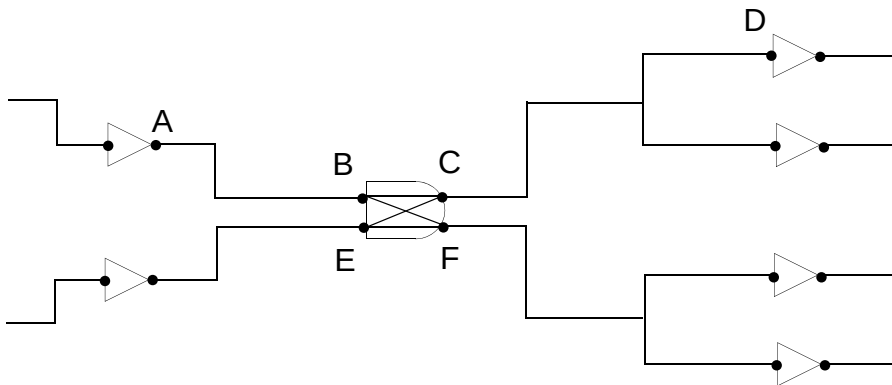
$\text{Slope delay} = \text{Slope factor} \times \text{Transition delay}$

Note:

When specifying the slope factor, you should already have the difference between transition delay and real slew time factored in.

- Transition delay is 0 (or 100)-to-threshold switching time.
- Real slew time is normally defined as 10-to-90% switching time.

The figure below provides an example.



Astro uses the following to determine the delay from B to D:

- Transition delay at port C due to down-stream loading
- Connect delay from port C to port D
- Intrinsic delay from port B to port C
- Slope delay on arc B-C due to the transition delay at port A

Note:

You can extract certain timing characteristics for a design using the Post Route > Create Timing Model command. For information, see `aprCreateTimingModel`.

Tool

Milkyway

Syntax

```
defineCondTimeIntrinsic "cellName" "sourcePort" "destPort"  
    "unate" "timingType" "enaCond" "startCond" "endCond"  
    delayOrRuleSet slopeSet resistSet
```

where the arguments differ slightly depending on the timingType:

<i>cellName</i>	Name of the cell that internal delay between ports is specified. Valid values: Name of any cell in the library.
<i>sourcePort</i>	Name of the source port. Valid values: Any input port in the cell specified by cellName.
<i>destPort</i>	For intrinsic delay types: Name of the output port. For timing rule types: Name of the clock port. Valid values: Any output/clock port in the cell specified by cellName.

unate

For intrinsic delay types: Specification of the type of output signal generated by a rising edge input signal.

For timing rule types: Not used.

Valid values:

Any of the following:

- **noninvert** indicates that a rising edge at the source generates a rising edge at the destination.
- **invert** indicates that a rising edge at the source generates a falling edge at the destination.
- **nonunate** indicates that a rising edge at the source generates an unknown edge at the destination. Because the polarity can no longer be ascertained, Astro takes the maximum of the falling/rising edge parameters, which reduces the accuracy of the timing estimate.

Note that for timing rule types, you need to specify one of the valid values, even though it is ignored.

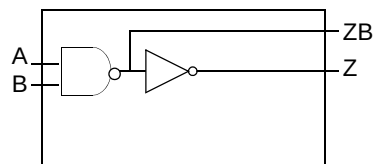
timingType

Timing type of the arc.

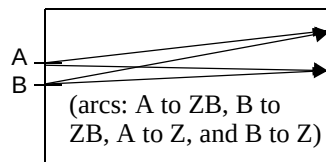
Valid values:

Any type in either of the following categories:

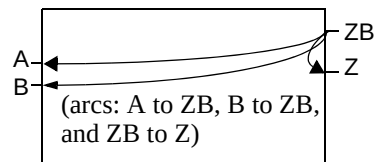
- Intrinsic delay types:
- Path (standard combinatorial delay) indicates the arc type shown below.



Correct

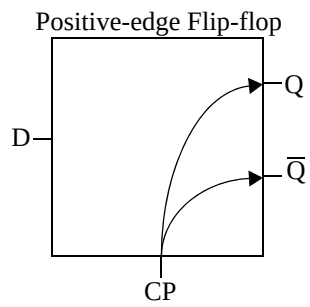


Incorrect

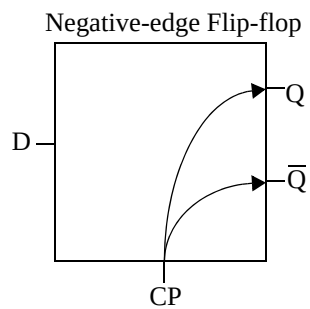


defineCondTimeIntrinsic

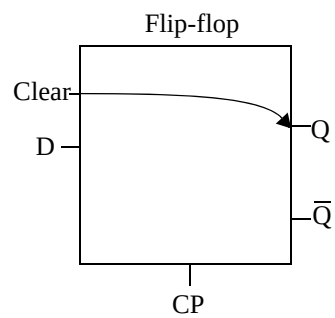
- Clock_rising indicates the arc type shown below.



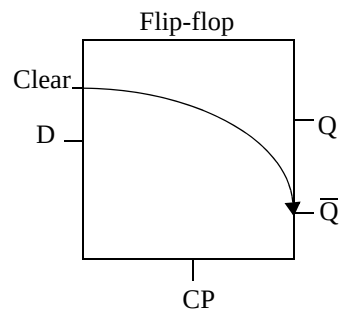
- Clock_falling indicates the arc type shown below.



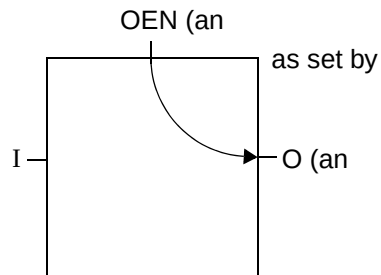
- Clear indicates the arc type shown below.



- Preset indicates the arc type shown below.

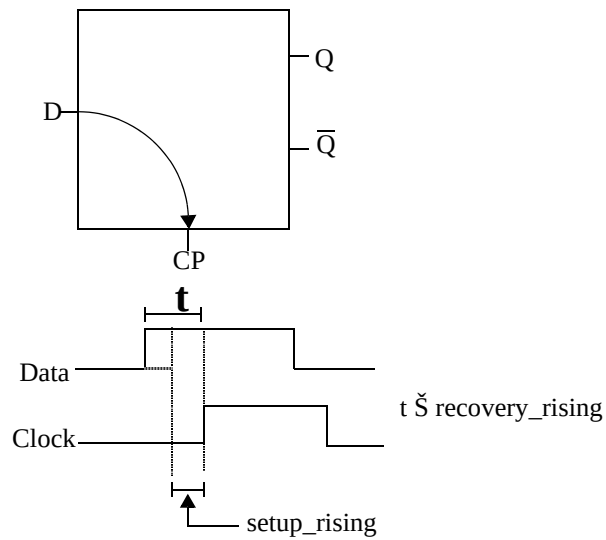


- Disable indicates the arc type shown below.



- Timing rule types:
- Setup_rising indicates the arc type shown below.

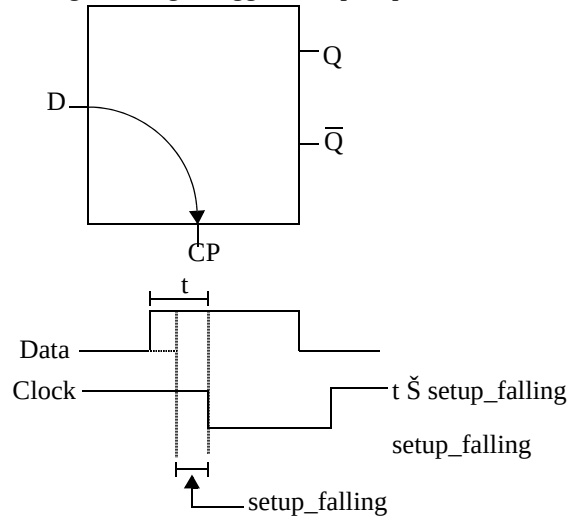
Positive-edge Triggered Flip-flop



defineCondTimeIntrinsic

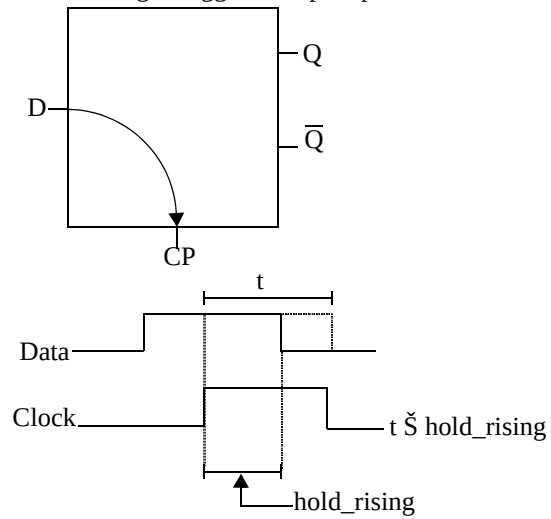
- Setup_falling indicates the arc type shown below.

Negative-edge Triggered Flip-flop

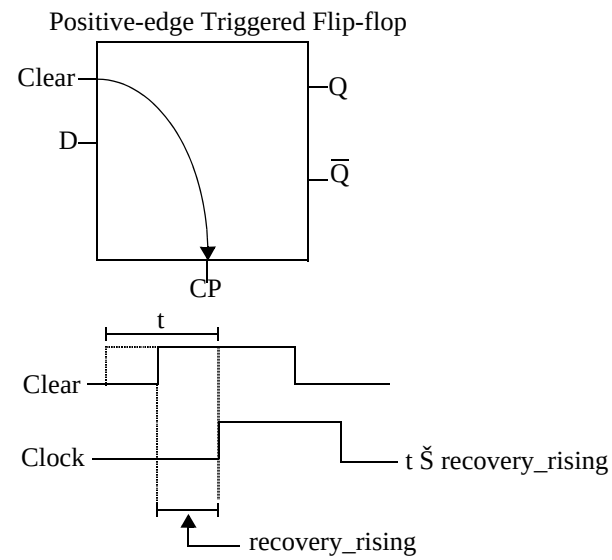


- Hold_rising indicates the arc type shown below.

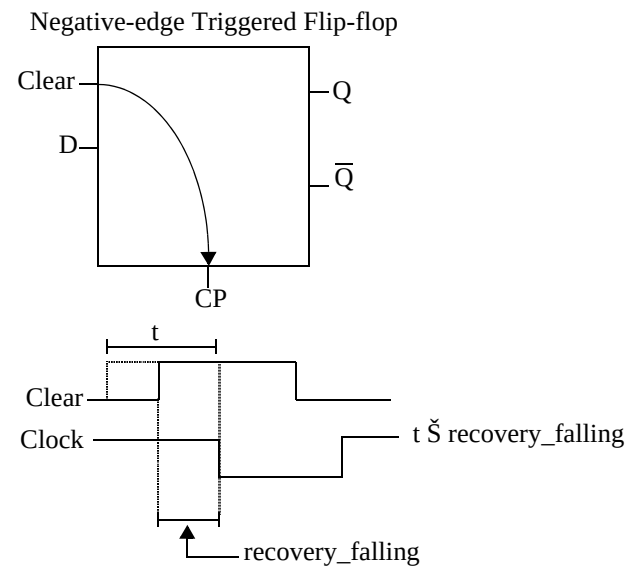
Positive-edge Triggered Flip-flop



- Recovery_rising indicates the arc type shown below.



- Recovery_falling indicates the arc type shown below.



enaCond Specifies the logic condition TRUE, when the delay values are applicable, for timing types PATH, CLOCK_RISING and CLOCK_FALLING. For other timing types, including SETUP, HOLD, and RECOVERY, this argument is empty.
Valid values:
An empty or non-empty string. Non-empty strings are Boolean equations with port names defined in the cell.

Note:

If you specify *enaCond*, leave *startCond* and *endCond* empty. If either *startCond* or *endCond* is specified, leave *enaCond* empty.

startCond Specifies the logic condition TRUE, when the delay values are applicable, for timing rule types (SETUP_RISING, SETUP_FALLING, HOLD_RISING, HOLD_FALLING, RECOVERY_RISING, RECOVERY_FALLING, REMOVAL_RISING and REMOVAL_FALLING). For other timing types, including SAMPLE and DELAY arc types, this argument is empty.
Valid values:
An empty or non-empty string. Non-empty strings are Boolean equations with port names defined in the cell.

endCond Specifies the logic condition TRUE, when the delay values are applicable, for timing rule types (SETUP_RISING, SETUP_FALLING, HOLD_RISING, HOLD_FALLING, RECOVERY_RISING, RECOVERY_FALLING, REMOVAL_RISING and REMOVAL_FALLING). For other timing types, including SAMPLE and DELAY arc types, this argument is empty.
Valid values: An empty or non-empty string. Non-empty strings are Boolean equations with port names defined in the cell.

Note:

An empty argument does not mean the argument is omitted. You must specify an empty string; "".

delayOr
RuleSet

A value or set of values that expresses the following:

- For intrinsic delay types: The minimum delay between the specified sourcePort and the specified destPort (in this case, output port) when the output signal is rising.
For timing rule types: The minimum timing rule between the specified sourcePort and the specified destPort (in this case, clock port) when the input signal is rising.
- For intrinsic delay types: The minimum delay between the specified sourcePort and the specified destPort (in this case, output port) when the output signal is falling.
For timing rule types: The minimum timing rule between the specified sourcePort and the specified destPort (in this case, clock port) when the input signal is falling.
- For intrinsic delay types: The nominal delay between the specified sourcePort and the specified destPort (in this case, output port) when the output signal is rising.
For timing rule types: The nominal timing rule between the specified sourcePort and the specified destPort (in this case, clock port) when the input signal is rising.
- For intrinsic delay types: The nominal delay between the specified sourcePort and the specified destPort (in this case, output port) when the output signal is falling.
For timing rule types: The nominal timing rule between the specified sourcePort and the specified destPort (in this case, clock port) when the input signal is falling.

defineCondTimeIntrinsic

- For intrinsic delay types: The maximum delay between the specified sourcePort and the specified destPort (in this case, output port) when the output signal is rising.
For timing rule types: The maximum timing rule between the specified sourcePort and the specified destPort (in this case, clock port) when the input signal is rising.
- For intrinsic delay types: The maximum delay between the specified sourcePort and the specified destPort (in this case, output port) when the output signal is falling.
For timing rule types: The maximum timing rule between the specified sourcePort and the specified destPort (in this case, clock port) when the input signal is falling.

Valid values:

Astro needs six values, which you can express in any of the following ways:

- A single value, expressed as follows:

value

where:

value is a non-negative, floating-point number that expresses the minimum, nominal, and maximum delay/timing rule when the destination (output) signal is rising and the minimum, nominal, and maximum delay/timing rule when the destination (output) signal is falling.

- A list of two values, expressed as follows:

'(riseTime fallTime)

where:

- riseTime is a non-negative, floating-point number that expresses the minimum, nominal, and maximum delay/timing rule when the destination (output) signal is rising.

- fallTime is a non-negative, floating-point number that expresses the minimum, nominal, and maximum delay/timing rule when the destination (output) signal is falling.

- A list of four values, expressed as follows:

```
'(minRiseTime minFallTime  
nomMaxRiseTime nomMaxFallTime)
```

where:

- minRiseTime is a non-negative, floating-point number that expresses the minimum delay/timing rule when the destination (output) signal is rising.
- minFallTime is a non-negative, floating-point number that expresses the minimum delay/timing rule when the destination (output) signal is falling.
- nomMaxRiseTime is a non-negative, floating-point number that expresses the nominal and maximum delay/timing when the destination (output) signal is rising.
- nomMaxFallTime is a non-negative, floating-point number that expresses the nominal and maximum delay/timing rule when the destination (output) signal is falling.

- A list of six values, expressed as follows:

```
'(minRiseTime minFallTime nomRiseTime  
nomFallTime maxRiseTime maxFallTime)
```

where:

- minRiseTime is a non-negative, floating-point number that expresses the minimum delay/timing rule when the destination (output) signal is rising.

- minFallTime is a non-negative, floating-point number that expresses the minimum delay/timing rule when the destination (output) signal is falling.
- nomRiseTime is a non-negative, floating-point number that expresses the nominal delay/timing when the destination (output) signal is rising.
- nomFallTime is a non-negative, floating-point number that expresses the nominal delay/timing rule when the destination (output) signal is falling.
- maxRiseTime is a non-negative, floating-point number that expresses the maximum delay/timing when the destination (output) signal is rising.
- maxFallTime is a non-negative, floating-point number that expresses the maximum delay/timing rule when the destination (output) signal is falling.

slopeSet

For intrinsic delay types: A value or set of values that express the following:

- Slope factor used to adjust the minimum delay between the specified sourcePort and the specified destPort when the destination (output) signal is rising.
- Slope factor used to adjust the minimum delay between the specified sourcePort and the specified destPort when the destination (output) signal is falling.
- Slope factor used to adjust the nominal delay between the specified sourcePort and the specified destPort when the destination (output) signal is rising.
- Slope factor used to adjust the nominal delay between the specified sourcePort and the specified destPort when the destination (output) signal is falling.
- Slope factor used to adjust the maximum delay between the specified sourcePort and the specified destPort when the destination (output) signal is rising.
- Slope factor used to adjust the maximum delay between the specified sourcePort and the specified destPort when the destination (output) signal is falling.

Note:

When specifying slope factor, you should already have the difference between transition delay and real slew time factored in.

For timing rule types: Not used

Valid values:

Astro needs six values, which you can express in any of the following ways:

defineCondTimeIntrinsic

- A single value, expressed as follows:

value

where: value is a non-negative, floating-point number that expresses the slope factor used to adjust the minimum, nominal, and maximum delay when the destination (output) signal is rising and the minimum, nominal, and maximum delay when the destination (output) signal is falling.

- A list of two values, expressed as follows:

'(riseTime fallTime)

where:

- riseTime is a non-negative, floating-point number that expresses the slope factor used to adjust the minimum, nominal, and maximum delay when the destination (output) signal is rising.
- fallTime is a non-negative, floating-point number that expresses slope factor used to adjust the minimum, nominal, and maximum delay when the destination (output) signal is falling.

- A list of four values, expressed as follows:

'(minRiseTime minFallTime nomMaxRiseTime
nomMaxFallTime)

where:

- minRiseTime is a non-negative, floating-point number that expresses the slope factor used to adjust the minimum delay when the destination (output) signal is rising.
- minFallTime is a non-negative, floating-point number that expresses the slope factor used to adjust the minimum delay when the destination (output) signal is falling.

- nomMaxRiseTime is a non-negative, floating-point number that expresses the slope factor used to adjust the nominal and maximum delay when the destination (output) signal is rising.
- nomMaxFallTime is a non-negative, floating-point number that expresses the slope factor used to adjust the nominal and maximum delay when the destination (output) signal is falling.
- A list of six values, expressed as follows:
'(minRiseTime minFallTime nomRiseTime
nomFallTime maxRiseTime maxFallTime)
where:
 - minRiseTime is a non-negative, floating-point number that expresses the slope factor used to adjust the minimum delay when the destination (output) signal is rising.
 - minFallTime is a non-negative, floating-point number that expresses the slope factor used to adjust the minimum delay when the destination (output) signal is falling.
 - nomRiseTime is a non-negative, floating-point number that expresses the slope factor used to adjust the nominal delay when the destination (output) signal is rising.
 - nomFallTime is a non-negative, floating-point number that expresses the slope factor used to adjust the nominal delay when the destination (output) signal is falling.
 - maxRiseTime is a non-negative, floating-point number that expresses the slope factor used to adjust the maximum delay when the destination (output) signal is rising.
 - maxFallTime is a non-negative, floating-point number that expresses the slope factor used to adjust the maximum delay when the destination (output) signal is falling.

Note:

For timing rule types, you need to specify one of the valid values, even though it is ignored.

resistSet

For intrinsic delay types: A value or set of values that express the following:

- Output resistance (after factoring in threshold voltage) associated with the specified sourcePort and the specified destPort when the destination (output) signal is at its fastest speed and the transition is a rise.
- Output resistance (after factoring in threshold voltage) associated with the specified sourcePort and the specified destPort when the destination (output) signal is at its fastest speed and the transition is a fall.
- Output resistance (after factoring in threshold voltage) associated with the specified sourcePort and the specified destPort when the destination (output) signal is at its nominal speed and the transition is a rise.
- Output resistance (after factoring in threshold voltage) associated with the specified sourcePort and the specified destPort when the destination (output) signal is at its nominal speed and the transition is a fall.
- Output resistance (after factoring in threshold voltage) associated with the specified sourcePort and the specified destPort when the destination (output) signal is at its slowest speed and the transition is a rise.

- Output resistance (after factoring in threshold voltage) associated with the specified sourcePort and the specified destPort when the destination (output) signal is at its slowest speed and the transition is a fall.

For timing rule types: Not used.

Valid values: Astro needs four values, which you can express in any of the following ways:

- A single value, expressed as follows:

value

where: value is a non-negative, floating-point number that expresses the resistance when the destination(output) signal is at it slowest, nominal, or fastest speed and the transition is a rise or a fall.

- A list of two values, expressed as follows:

'(riseResistance fallResistance)

where:

- riseResistance is a non-negative, floating-point number that expresses the resistance when the destination (output) signal is at it slowest, nominal, or fastest speed and the transition is a rise.
- fallResistance is a non-negative, floating-point number that expresses the resistance when the destination (output) signal is at it slowest, nominal, or fastest speed and the transition is a fall.

- A list of four values, expressed as follows:

```
'(minRiseResistance minFallResistance
nomMaxRiseResistance
nomMaxFallResistance)
```

where:

- minRiseResistance is a non-negative, floating-point number that expresses the resistance when the destination (output) signal is at it fastest speed and the transition is a rise.
- minFallResistance is a non-negative, floating-point number that expresses the resistance when the destination (output) signal is at it fastest speed and the transition is a fall.
- nomMaxRiseResistance is a non-negative, floating-point number that expresses the resistance when the destination (output) signal is at its nominal or slowest speed and the transition is a rise.
- nomMaxFallResistance is a non-negative, floating-point number that expresses the resistance when the destination(output) signal is at its nominal or slowest speed and the transition is a fall.

- A list of six values, expressed as follows:

```
'(minRiseResistance minFallResistance
nomRiseResistance nomFallResistance)
maxRiseResistance maxFallResistance)
```

where:

- minRiseResistance is a non-negative, floating-point number that expresses the resistance when the destination (output) signal is at it fastest speed and the transition is a rise.

- minFallResistance is a non-negative, floating-point number that expresses the resistance when the destination (output) signal is at its fastest speed and the transition is a fall.
- nomRiseResistance is a non-negative, floating-point number that expresses the resistance when the destination (output) signal is at its nominal speed and the transition is a rise.
- nomFallResistance is a non-negative, floating-point number that expresses the resistance when the destination (output) signal is at its nominal speed and the transition is a fall.
- maxRiseResistance is a non-negative, floating-point number that expresses the resistance when the destination (output) signal is at its slowest speed and the transition is a rise.
- maxFallResistance is a non-negative, floating-point number that expresses the resistance when the destination (output) signal is at its slowest speed and the transition is a fall.

Note:

For timing rule types, you need to specify one of the valid values, even though it is ignored.

Example (for timing rule arc type)

```
defineCondTimeIntrinsic "FF1" "D" "CP" "" "" "Q+D"
'("t1" "t2" "t3" "t4" "t5" "t6") `()
```

This above conditional timing arc is used when the boolean equation (Q+D) is TRUE.

See Also

[defineCondTimeTLU](#) specifies the timing characteristics of an arc.

defineCondTimeTLU

This timing command specifies the timing characteristics of an arc under specified conditions.

Note:

Conditional delay allows you to specify different delays of the output pin of a cell with respect to the various states of the input pins.

Tool

Milkyway

Prerequisite

Before you can specify the timing characteristics of an arc, you must create timing tables using [clfCreateTable](#).

Syntax

```
defineCondTimeTLU "cellName" "sourcePort" "destPort"  
"unate" "timingType" "enaCond" "tartCond" "endCond"  
propOrCellOrConstrTable tranTable
```

where the arguments are as follows:

<i>cellName</i>	Name of the cell that port-to-port timing is specified. Valid values: Name of any cell in the library.
<i>sourcePort</i>	Name of the start port. Valid values: Any port in the cell specified by cellName.

destPort

For intrinsic delay types: Name of the output port.

For timing rule types: Name of the clock port.

Valid values:

Any output/clock port in the cell specified by cellName.

unate

For intrinsic delay types: Specification of the type of destination (output) signal generated by a rising edge source signal.

For timing rule types: Not used.

Valid values: Any of the following:

- noninvert indicates that a rising edge source generates a rising edge destination.
- invert indicates that a rising source input generates a falling edge destination.
- nonunate indicates that a rising edge source generates an unknown edge destination. Because the polarity can no longer be ascertained, Astro takes the maximum of the falling/rising edge parameters, which reduces the accuracy of the timing estimate.

Note:

For timing rule types, you need to specify one of the valid values, even though it is ignored.

timingType

Timing type of the arc.

Valid values:

Any type in either of the following categories:

- Intrinsic delay types:
- Path
- Clock_rising
- Clock_falling
- Clear
- Preset
- Timing rule types:
- Setup_rising
- Setup_falling
- Hold_rising
- Hold_falling
- Recovery_rising
- Recovery_falling

Note:

For descriptions of these timing types, see the `timingType` argument for [defineTimeIntrinsic](#).

enaCond

Specifies the logic condition TRUE, when the delay values are applicable, for timing types PATH, CLOCK_RISING and CLOCK_FALLING. For other timing types, this argument is empty.

Valid values:

An empty or non-empty string. Non-empty strings are Boolean equations with port names defined in the cell.

Note:

If you specify `enaCond`, leave `startCond` and `endCond` empty. If either `startCond` or `endCond` is specified, leave `enaCond` empty.

<i>startCond</i>	Specifies the logic condition TRUE, when the delay values are applicable, for timing rule types (SETUP_RISING, SETUP_FALLING, HOLD_RISING, HOLD_FALLING, RECOVERY_RISING, RECOVERY_FALLING, REMOVAL_RISING and REMOVAL_FALLING). For other timing types, this argument is empty. Valid values: An empty or non-empty string. Non-empty strings are Boolean equations with port names defined in the cell.
<i>endCond</i>	Specifies the logic condition TRUE, when the delay values are applicable, for timing rule types (SETUP_RISING, SETUP_FALLING, HOLD_RISING, HOLD_FALLING, RECOVERY_RISING, RECOVERY_FALLING, REMOVAL_RISING and REMOVAL_FALLING). For other timing types, this argument is empty. Valid values: An empty or non-empty string. Non-empty strings are Boolean equations with port names defined in the cell.

Note:

An empty argument does not mean the argument is omitted. You must specify an empty string; "".

<i>propOrCellOrConstrTable</i>	Names of timing tables that express propagation or cell delays (for intrinsic delay types) or delay constraints (for timing rule types) for the arc, in the following format: For intrinsic delay types: <pre>'(minRisePOrCTable minFallPOrCTable [nomRisePOrCTable nomFallPOrCTable] maxRisePOrCTable maxFallPOrCTable)</pre> where: minRisePOrCTable is the name of the timing table that expresses the minimum propagation or cell delays for the arc when the signal is rising.
--------------------------------	--

Note:

This table must have a tableType of RisePropagationTable or RiseCellDelayTable (see [clfCreateTable](#)).

defineCondTimeTLU

- minFallPOrCTable is the name of the timing table that expresses the minimum propagation or cell delays for the arc when the signal is falling.

Note:

This table must have a tableType of FallPropagationTable or FallCellDelayTable (see [clfCreateTable](#)).

- nomRisePOrCTable is the name of the timing table that expresses the nominal propagation or cell delays for the arc when the signal is rising. If you omit this argument, Astro uses the value given for maxRisePOrCTable.

Note:

This table must have a tableType of RisePropagationTable or RiseCellDelayTable (see [clfCreateTable](#)).

- nomFallPropTable is the name of the timing table that expresses the nominal propagation or cell delays for the arc when the signal is falling. If you omit this argument, Astro uses the value given for maxFallPOrCTable.

Note:

This table must have a tableType of FallPropagationTable or FallCellDelayTable (see [clfCreateTable](#)).

- maxRisePOrCTable is the name of the timing table that expresses the maximum propagation or cell delays for the arc when the signal is rising.

Note:

This table must have a tableType of RisePropagationTable or RiseCellDelayTable (see [clfCreateTable](#)).

- maxFallPOrCTable is the name of the timing table that expresses the maximum propagation or cell delays for the arc when the signal is falling.

Note:

This table must have a tableType of FallPropagationTable or FallCellDelayTable (see [clfCreateTable](#)).

- For timing rule types:
'(minRiseConstrTable minFallConstrTable
[nomRiseConstrTable nomFallConstrTable]
maxRiseConstrTable maxFallConstrTable)
where:
- minRiseConstrTable is the name of the timing table that expresses the minimum delay constraints for the arc when the signal is rising.

Note:

This table must have a tableType of RiseConstraintTable (see [clfCreateTable](#)).

- minFallConstrTable is the name of the timing table that expresses the minimum delay constraints for the arc when the signal is falling.

Note:

This table must have a tableType of FallConstraintTable (see [clfCreateTable](#)).

- nomRiseConstrTable is the name of the timing table that expresses the nominal delay constraints for the arc when the signal is rising. If you omit this argument, Astro uses the value given for maxRiseConstrTable.

Note:

This table must have a tableType of RiseConstraintTable (see [clfCreateTable](#)).

- nomFallConstrTable is the name of the timing table that expresses the nominal delay constraints for the arc when the signal is falling. If you omit this argument, Astro uses the value given for maxFallConstrTable.

Note:

This table must have a tableType of FallConstraintTable (see [clfCreateTable](#)).

- maxRiseConstrTable is the name of the timing table that expresses the maximum delay constraints for the arc when the signal is rising.

defineCondTimeTLU

Note:

This table must have a `tableType` of `RiseConstraintTable` (see [clfCreateTable](#)).

- `maxFallConstrTable` is the name of the timing table that expresses the maximum delay constraints for the arc when the signal is falling.

Note:

This table must have a `tableType` of `FallConstraintTable` (see [clfCreateTable](#)).

tranTable

Names of the timing tables that express transition delays (for intrinsic delay types) for the arc, in the following format:

- For intrinsic delay types:
'(minRiseTranTable minFallTranTable
[nomRiseTranTable nomFallTranTable]
maxRiseTranTable maxFallTranTable)
where:
 - `minRiseTranTable` is the name of the timing table that expresses the minimum transition delays for the arc when the signal is rising.

Note:

This table must have a `tableType` of `RiseTransitionTable` (see [clfCreateTable](#)).

- `minFallTranTable` is the name of the timing table that expresses the minimum transition delays for the arc when the signal is falling.

Note:

This table must have a `tableType` of `FallTransitionTable` (see [clfCreateTable](#)).

- `nomRiseTranTable` is the name of the timing table that expresses the nominal transition delays for the arc when the signal is rising. If you omit this argument, Astro uses the value given for `maxRiseTranTable`.

Note:

This table must have a tableType of RiseTransitionTable (see [clfCreateTable](#)).

- nomFallTranTable is the name of the timing table that expresses the nominal transition delays for the arc when the signal is falling. If you omit this argument, Astro uses the value given for maxFallTranTable.

Note:

This table must have a tableType of FallTransitionTable (see [clfCreateTable](#)).

- maxRiseTranTable is the name of the timing table that expresses the maximum transition delays for the arc when the signal is rising.

Note:

This table must have a tableType of RiseTransitionTable (see [clfCreateTable](#)).

- maxFallTranTable is the name of the timing table that expresses the maximum transition delays for the arc when the signal is falling.

Note:

This table must have a tableType of FallTransitionTable (see [clfCreateTable](#)).

- For timing rule types:
'()

Example 1

```
defineCondTimeTLU "cell11" "p1" "p2" "invert" "Path" "!p" "" ""  
' ("t1" "t2" "t3" "t4") ' ("t17" "t18" "t19" "t20")
```

This defineCondTimeTLU function indicates the following timing arc is used when the Boolean equation “p!” is TRUE:

- A rising edge on port p1 generates a falling edge on port p2.

defineCondTimeTLU

- The timing type of the arc is Path.
- Timing tables t1, t2, t3, and t4 express the propagation delays between port p1 and port p2 in cell1.
- Timing tables t17, t18, t19, and t20 express the transition delays between port p1 and port p2 in cell1.

See Also

[defineCondTimeIntrinsic](#) specifies the parameters associated with the arc between a specified source port and a specified destination port on a master cell.

defineDiodeProtection

Defines the diode protection value in the CLF file.

Tool

Milkyway

Syntax

```
defineDiodeProtection cellName portName
    '(protection_values...)
```

where the arguments are as follows:

<i>cellName</i>	Name of the cell for which capacitance of a port is specified. Valid values: Name of any cell in the library.
<i>portName</i>	Name of the port for which capacitance is specified. Valid values: Any port in the cell specified by <i>cellName</i> .

protection_value The protection values used in computing the allowable ratio when antenna is protected by a diode
s The first value of the list is for “metal1”, the second value is for “metal2”, and so on. If the value of higher layers is the same as the value of lower layers, they can be omitted.

Example

```
defineDiodeProtection "and3x1" "Y" '(0.25)  
defineDiodeProtection "adn2x4" "Y" '(1.05)
```

See Also

[dbAddAntennaLayerRule](#) defines an advanced antenna rule for the specified layer and stores it in the library.

[dbClearLibAntennaRules](#) deletes all of the antenna rules stored in the library.

[dbDumpLibAntennaRules](#) dumps all of the antenna rules stored in the library to a file.

[dbDefineAntennaRule](#) defines an advanced antenna rule for the specified mode and stores it in the library.

defineExtAntennaArea

Defines the external antenna area for any pins in the design (for example routing at next level of hierarchy) for computing antenna rules. By default, the antenna checker assumes the external antenna area for internal pins to compute antenna ratios.

If this value is specified, the router uses the value defined by this command.

This command can only be used in a TDF file and loaded with axgLoadTDF .

Tool

Milkyway

Syntax

```
defineExtAntennaArea portName ' ("layerName" isSideWall  
area)  
    ....  
)
```

where the arguments are as follows:

<i>portName</i>	Name of the port.
<i>layerName</i>	Name of the layer in the port.
<i>isSideWall</i>	Indicates whether the area is a sideWall or not. Valid values: • #t : area is sideWall. • #f : area is a polygon area.

Example

```
(defineExtAntennaArea "and3x2" "A" '(  
  ("metal1" #t 3.6)  
  ("via1" #t 1.6)  
  ("metal2" #t 2.7)
```

defineFlipFlopFunction

Defines the function of a flip-flop cell.

Tool

Milkyway

Syntax

```
defineFlipFlopFunction "cell_name" "pseudo_q" "pseudo_qn"  
"clock" "next_state" "reset" "set" "var_when_both1"  
"var_when_both2" "clocked_on_also"
```

where the arguments are as follows:

<i>cell_name</i>	The name of the flip-flop cell whose functionality you wish to define.
<i>pseudo_q</i>	The state of the non-inverting output of the flip-flop or latch.
<i>pseudo_qn</i>	The state of the inverting output of the flip-flop or latch.
<i>clock</i>	The state of the clock input of the flip-flop.
<i>next_state</i>	The logic equation written in terms of inout pins or <i>pseudo_q</i> , it specifies the value of <i>pseudo_q</i> at the next active transition of the clock.
<i>reset</i>	The active value of the clear input of the flip-flop or latch.
<i>set</i>	The active value of the preset input of the flip-flop or latch.
<i>var_when_both1</i>	The value that <i>pseudo_q</i> has when reset and set are both active at the same time.
<i>var_when_both2</i>	The value that <i>pseudo_qn</i> has when reset and set are both active at the same time.
<i>clocked_on_also</i>	Specifies the active edge of the clock signal.

Example

```
defineLatchFunction "dff1x" "IQ" "IQN" "CK" "D" "CLR" "PRE"
```

defineFlipFlopFunction

```
"X"  
  "X" "CA"
```

is converted from synopsys .lib format

```
ff (IQ,IQN) {  
  clocked_on   : CK ;  
  next_state   : D ;  
  clear        : CLR ;  
  preset       : PRE ;  
  clear_preset_var1 : X ;  
  clear_preset_var2 : X ;  
  clocked_on_also : CA;  
}
```

defineFootPrintEQCell

Specifies that two cells, cellA and cellB, have identical footprints and are logically equivalent, or they belong to the same voltage group, have identical logic functions, and identical port names. Can specify multiple cells in the same class by linking together through multiple calls. Footprint EQ is best used for cells with have the same boundary and same blockage such that they can be swapped without any routing impact.

The above command is to be used for the design library and not a reference library.

Tool

Milkyway

Syntax

```
defineFootPrintEQCell "cellA" "cellB"
```

where “cellA” and “cellB” are two standard cells in the reference library.

Example

```
defineFootPrintEQCell "nd2" "nd2a"  
defineFootPrintEQCell "nd2a" "nd2b"
```

would specify “nd2,” “nd2a,” “nd2b” all within the same footprint EQ.

See Also

astPowerRecovery

defineGateSize

Defines gate length and width in the CLF file. This command must be included in the CLF file if you want to check charge-collecting antenna.

Tool

Milkyway

Syntax

```
defineGateSize cellName portName gate_size
```

where the arguments are as follows:

<i>cellName</i>	Name of the cell that capacitance of a port is specified. Valid values: Name of any cell in the library.
-----------------	--

portName Name of the port that capacitance is specified.
Valid values:
Any port in the cell specified by *cellName*.

gate_size The poly gate area (W x L), in user units.
Valid values:
Any real floating-point number.

Prerequisites For Antenna Checking

1. Create the CLF file containing the `defineGateSize` function, or add the gate size definitions to an existing CLF file.
2. Load the CLF file using [auLoadCLF](#).
3. Turn ON antenna checking and specify the ratio in Astro. To do this, enter:

```
axSetIntParam "droute" "doAntennaConx" 3  
axSetIntParam "droute" "maxAntennaRatio" 35
```

where 35 is the ratio of gate to metal area.

Example

This is an example of the `defineGateSize` format in the CLF file.

```
defineGateSize "AND3B" "A" 5.30  
defineGateSize "AND3B" "B" 5.30  
defineGateSize "AND3B" "C" 5.30  
defineGateSize "AND4" "A" 5.15  
defineGateSize "AND4" "B" 5.15
```

defineHierAntennaProp

Defines the hierarchical antenna property in the CLF file. This command is not intended for user to use directly. Should only be generated by [auDumpCLF](#) in the CLF file so they can be loaded to a different library. The values should only be generated by `axComputeHierAntennaProp` or by Hercules and no documentation or explanation is provided for any of the arguments.

Tool

Milkyway

Syntax

```
defineHierAntennaProp cellName portName `(list of values)
```

where the arguments are as follows:

<i>cellName</i>	Name of the cell for which the antenna property of a port is specified. Valid values: Name of any cell in the library
<i>portName</i>	Name of the port that antenna property is specified. Valid values: Any port in the cell specified by <i>cellName</i> .
<i>`(list of values)</i>	The list of valid values. Please use the values as they are generated by “auDumpCLF” on page 14-34 . None of the values should be modified.

See Also

[auDumpCLF](#)

[auLoadCLF](#)

[axComputeHierAntennaProp](#)

defineInternalPower

This CLF command defines the power consumption of the cell when the input(s) switch but outputs do not. In this case, although the logic state(s) of the output(s) do not change, those of internal nodes may change and dissipate energy. Internal power could be substantial in complex gates.

Use this command to create power views (cellName.PWR) of the cells in a library just as you define timing characteristics by timing view (cellName.TIM) and connectivity relationships by frame view (cellName.FRAME).

Tool

Milkyway

Syntax

```
defineInternalPower "cellName" "inputCond" "outputCond"  
"modelType" "modelArg"
```

where the arguments are as follows:

<i>cellName</i>	Name of the cell or cell master that you are specifying power requirements. Valid values: Name of any cell in the library.
-----------------	--

inputCond Signal values or transitions of the inputs separated by the character “*” to indicated a Boolean AND operation in the following format:

signalName signalCond * signalName signalCond

where the arguments are as follows:

signalName – Name of the input that you are defining a condition.

signalCond – one of the following six conditions:

- “0” – signal stays at 0
- “1” – signal stays at 1
- “-” – doesn’t matter, signal can be either 0 or 1
- “r” – signal has a rising transition
- “f” – signal has a falling transition
- “t” – signal toggles

If the signalCond is not specified, it is treated as having “universal” condition and the model information applies regardless of the signal condition.

outputCond Signal values or transitions of the outputs separated by the character “*” to indicated a Boolean AND operation in the following format:

signalName signalCond * signalName signalCond

where the arguments are as follows:

signalName – Name of the output that you are defining a condition.

signalCond – one of the following six conditions:

- “0” – signal stays at 0
- “1” – signal stays at 1
- “-” – doesn’t matter, signal can be either 0 or 1
- “r” – signal has a rising transition
- “f” – signal has a falling transition
- “t” – signal toggles

If the signalCond is not specified, it is treated as having “universal” condition and the model information applies regardless of the signal condition.

Note:

If all input and output signals are of the same condition, you can omit the signalName. For example, if all input signals stay at “1” in a cell of three inputs “a,” “b,” and “c,” instead of typing “a1*b1*c1”, just type “1.”

Note:

You must define at least one input at least one input signal experiencing a transition but not the output signals.

modelType Specifies the type of power model defined.
Valid values:

Constant – Power model is a constant value. In this case, the field modelArg of the command has to be a floating-point number. Constant type is allowed in definition of all power components.

Linear – Power model is a constant multiplier. In this case, the field modelArg of the command has to be a floating-point number. Linear type is used when the contribution of short-circuit power is unknown and is approximated by multiplying the power for charging/ discharging the loading capacitance by a factor. Therefore, its only allowed in [defineSwitchingPower](#).

TLU – Power model is a look-up table. In this case, the field modelArg of the command is the name of the look-up table that must be defined prior to the appearance of current command. The type of the referred look-up table has to be the same as the type of the command. If a table is referred to in the definition of a defineInternalPower command, the type of the table has to be “InternalPowerTable.”

modelArg Specifies the corresponding argument of the selected model type. See details in the explanation of model type.

Example 1

```
(clfCreateTable "AOI33_1X:i2:ipp140" "InternalPowerTable"
'(("InputNetTransition" (130.000000 250.000000)))
' (0.035000 0.034000 )
)
defineInternalPower "AOI33_1X" "i2f*i10*i31*i40*i50*i60"
"y1"
"TLU" "AOI33_1X:i2:ipp140"
```

This example defines the internal power consumption for cell AOI33_1X. The input conditions are as follows:

Signal Input Name	Signal Input Condition
i2	Falls
i1	Stays at 0
i3	Stays at 1
i4	Stays at 0
i5	Stays at 0
i6	Stays at 0

The input signal “y” stays at 1, The model type is TLU, and the model argument is the name of the look-up table defined with [clfCreateTable](#) prior to using the `defineInternalPower` command.

Example 2

```
(clfCreateTable "AOI211_1X:int:a" "InternalPowerTable"
'(("InputNetTransition" (0.200 0.500 1.000 1.500)))
'(0.249 0.267 0.320 0.392)
)
defineInternalPower "AOI211_1X" "ar" "-" "TLU"
"AOI211_1X:in:a"
```

This example defines the internal power consumption for cell AOI211_1X. The input conditions specify that input “a” is rising and the output conditions are labeled as insignificant (the signal conditions do not matter.) The model type is TLU and the model argument is the name of the InternalPowerTable defined with [clfCreateTable](#) prior to using the `defineInternalPower` command.

See Also

[defineLeakagePower](#) defines the power dissipated by leakage currents through reverse-biased P-N junctions and subthreshold conduction channels.

[defineSwitchingPower](#) defines the power dissipated by the cell when the output(s) of the cell switches in response to its input(s) switching.

[clfCreateTable](#) creates a power table to show the effects of input and output conditions on internal power, leakage power, and switching power.

defineLatchFunction

Defines the function of a latch cell.

Tool

Milkyway

Syntax

```
defineLatchFunction "cell_name" "pseudo_q" "pseudo_qn"  
"enable"  
"data_in" "reset" "set" "var_when_both1" "var_when_both2"  
"enable_also"
```

where the arguments are as follows:

<i>cell_name</i>	The name of the latch cell whose functionality you wish to define.
<i>pseudo_q</i>	The state of the non-inverting output of the latch.
<i>pseudo_qn</i>	The state of the inverting output of the latch.

<i>enable</i>	The state of the enable latch of the tri-state buffer.
<i>data_in</i>	The state of the data input of the latch.
<i>reset</i>	The active value of the clear input of the latch.
<i>set</i>	The active value of the preset input of the latch.
<i>var_when_both1</i>	The value that pseudo_q has when reset and set are both active at the same time.
<i>var_when_both2</i>	The value that pseudo_qn has when reset and set are both active at the same time.
<i>enable_also</i>	Specifies the active edge of the enable signal.

Example

```
defineLatchFunction "lat1x" "IQ" "IQN" "EN" "D" "CLR" "PRE"
"X"
"X" "EA"
```

is converted from synopsys .lib format

```
latch (IQ,IQN) {
enable   : EN ;
data_in  : D ;
clear    : CLR ;
preset   : PRE ;
clear_preset_var1 : X ;
clear_preset_var2 : X ;
enable_also : EA;
}
```

defineLeakagePower

This CLF command defines the power dissipated by leakage currents through reverse-biased P-N junctions and subthreshold conduction channels. Although the majority of power consumption

originates from changes in signal values, a cell still dissipates certain amount of energy constantly even if all signal values stay unchanged. That energy is defined as leakage power.

Use this command to create power views (cellName.PWR) of the cells in a library just as you define timing characteristics by timing view (cellName.TIM) and connectivity relationships by frame view (cellName.FRAME).

Tool

Milkyway

Syntax

```
defineLeakagePower "cellName" "inputCond" "outputCond"  
"modelType" "modelArg"
```

where the arguments are as follows:

<i>cellName</i>	Name of the cell or cell master that you are specifying power requirements. Valid values: Name of any cell in the library.
<i>inputCond</i>	Signal values or transitions of the inputs separated by the character "*" to indicated a Boolean AND operation in the following format: signalName signalCond * signalName signalCond

where the arguments are as follows:

signalName – Name of the input that you are defining a condition.

signalCond – one of the following six conditions:

- “0” – signal stays at 0
- “1” – signal stays at 1
- “-” – doesn’t matter, signal can be either 0 or 1
- “r” – signal has a rising transition
- “f” – signal has a falling transition
- “t” – signal toggles

If the signalCond is not specified, it is treated as having “universal” condition and the model information applies regardless of the signal condition.

outputCond Signal values or transitions of the outputs separated by the character “*” to indicated a Boolean AND operation in the following format:

signalName signalCond * signalName signalCond

where the arguments are as follows:

signalName – Name of the output that you are defining a condition.

signalCond – one of the following six conditions:

- “0” – signal stays at 0
- “1” – signal stays at 1
- “-” – doesn’t matter, signal can be either 0 or 1
- “r” – signal has a rising transition
- “f” – signal has a falling transition
- “t” – signal toggles

If the signalCond is not specified, it is treated as having “universal” condition and the model information applies regardless of the signal condition.

Note:

If all input and output signals are of the same condition, you can omit the signalName. For example, if all input signals stay at “1” in a cell of 3 inputs “a,” “b,” and “c,” instead of typing “a1*b1*c1”, just type “1.”

Note:

No transition condition can be specified for any input or output signal.
The power model definition in violation of these rules is discarded.

<i>modelType</i>	Specifies the type of power model defined. Valid values: Constant – Power model is a constant value. In this case, the field <i>modelArg</i> of the command has to be a floating-point number. Constant type is allowed in definition of all power components. Linear – Power model is a constant multiplier. In this case, the field <i>modelArg</i> of the command has to be a floating-point number. Linear type is used when the contribution of short-circuit power is unknown and is approximated by multiplying the power for charging/ discharging the loading capacitance by a factor. Therefore, its only allowed in defineSwitchingPower. TLU – Power model is a look-up table. In this case, the field <i>modelArg</i> of the command is the name of the look-up table that has to be defined prior to the appearance of current command. The type of the referred look-up table has to be the same as the type of the command. If a table is referred to in the definition of a <i>defineLeakagePower</i> command, the type of the table has to be "LeakagePowerTable."
<i>modelArg</i>	Specifies the corresponding argument of the selected model type. See details in the explanation of model type.

Example

The following example defines the leakage power for cell AOI33_1X. The input and output signal conditions do not matter. The model type is Constant, so the value is a floating-point number.

```
defineLeakagePower "AOI33_1X" "-" "-" "Constant"  
47470.504000
```


See Also

[defineInternalPower](#) defines the power consumption of the cell when the input(s) switch but outputs do not.

[defineSwitchingPower](#) defines the power dissipated by the cell when the output(s) of the cell switches in response to its input(s) switching.

[clfCreateTable](#) creates a power table to show the effects of input and output conditions on internal power, leakage power, and switching power.

defineMinClkPeriod

Used in a CLF file or on the Astro command line, this command specifies the minimum allowable time for one complete cycle of clock pins.

Tool

Milkyway

Syntax

```
defineMinClkPeriod "cellName" "pinName" '(minPeriod  
nomPeriod  
maxPeriod)
```

where the arguments are as follows:

<i>cellName</i>	Name of the cell that port-to-port timing is specified. Valid values: Name of any cell in the library.
-----------------	--

<i>pinName</i>	Name of a pin instance that you want set the clock period for. Valid values: Name of any instance in the library or use pattern matching (*) to set the power to instances that match the pattern you enter.
<i>minPeriod</i>	Minimum elapsed time between repetitions of the clock (1/frequency). Valid values: Any floating-point number.
<i>nomPeriod</i>	Nominal elapsed time between repetitions of the clock. (1/frequency). Valid values: Any floating-point number.
<i>maxPeriod</i>	Maximum elapsed time between repetitions of the clock (1/frequency). Valid values: Any floating-point number.

Example

This example specifies the minimum allowable time for one complete cycle of the clock pin “clk.”

```
defineMinClkPeriod "FF2" "clk" '(0.69 0.69 0.69)
```

See Also

[tdfSetClkDef](#) sets source characteristics for a clock.

defineMinClkPulseWidth

Used in a CLF file or on the Astro command line, this command specifies the minimum clock pulse width. The minimum clock pulse width is the minimum allowable time for either the positive or negative phase of the clock pin.

Tool

Milkyway

Syntax

```
defineCondMinClkPulseWidth "cellName" "pinName"  
{ "High" | "Low" }  
' (min nom max)
```

where the arguments are as follows:

<i>cellName</i>	Name of the cell that port-to-port timing is specified. Valid values: Name of any cell in the library.
<i>pinName</i>	Name of a pin instance that you want set a minimum pulse with for. Valid values: Name of any instance in the library or use pattern matching (*) to set the power to instances that match the pattern you enter.
<i>High/Low</i>	Phase of the clock pin that you are setting the minimum allowable time. Valid values: HIGH indicates the positive phase of the clock pin. Low indicates the negative phase of the clock pin.
<i>min</i>	Minimum elapsed time between repetitions of the clock (1/frequency). Valid values: Any floating-point number.
<i>nom</i>	Nominal elapsed time between repetitions of the clock (1/frequency). Valid values: Any floating-point number.

defineMinClkPulseWidth

max Maximum elapsed time between repetitions of the clock (1/frequency).
Valid values:
Any floating-point number.

Example

This example specifies the minimum allowable time for the negative (low) phase of the clock pin “CP.”

```
defineCondMinClkPulseWidth "FF1" "CP" "LOW" '(0.52 0.52  
0.52)
```

definePad

Defines the orientation of a pad cell in a CLF file. You can also use the `definePad` command to rotate corner pads.

Tool

Milkyway

Syntax

```
definePad padCellName padRotation
```

where the arguments are as follows:

<i>PadCellName</i>	Name of the pad cell to which the <i>padRotation</i> value applies. Valid values: Name of a cell in the library defined as a <code>gdsPadCell</code> in a cell type definition file or <code>"*"</code> to represent all I/O cells in the library.
--------------------	--

padRotation Rotation value.
Valid values:
0, 90, 180, 270

Example

To rotate pad cells,

```
definePad "pad1" "0"  
definePad "pad2" "90"  
definePad "pad3" "180"  
definePad "pad4" "270"
```

To rotate corner pad cells,

```
definePad "cpad1" "0"  
definePad "cpad2" "90"  
definePad "cpad3" "180"  
definePad "pcad4" "270"
```

definePortCapacitance

This timing command defines the actual capacitance of a port. You need to specify capacitance for both input ports and output ports.

Note:

You can extract this and other timing characteristics for a design using the Post Route > Create Timing Model command. For information, see `aprCreateTimingModel`.

Tool

Milkyway

Syntax

```
definePortCapacitance "cellName" "portName" "condition"  
value
```

where the arguments are as follows:

<i>cellName</i>	Name of the cell that capacitance of a port is specified. Valid values: Name of any cell in the library.
<i>portName</i>	Name of the port that capacitance is specified. Valid values: Any port in the cell specified by <i>cellName</i> .
<i>condition</i>	The operating condition under which the specification applies. Valid values: Any of the following: • rise min means that the specification applies when the signal is coming in or going out at its fastest speed and the transition is a rise. • rise nom means that the specification applies when the signal is coming in or going out at its nominal speed and the transition is a rise. • rise max means that the specification applies when the signal is coming in or going out at its slowest speed and the transition is a rise. • fall min means that the specification applies when the signal is coming in or going out at its fastest speed and the transition is a fall. • fall nom means that the specification applies when the signal is coming in or going out at its nominal speed and the transition is a fall. • fall max means that the specification applies when the signal is coming in or going out at its slowest speed and the transition is a fall.

Note:

For most technologies, rise, nom, and fall port capacitances are the same. The six operating conditions are provided for those libraries that may require different rise, nom, and fall specifications.

value Capacitance of the specified port under the specified operating condition.
Valid values:
Any non-negative, floating-point number.

Example

```
definePortCapacitance "c1" "p1" "rise min" .017
definePortCapacitance "c1" "p1" "rise nom" .017
definePortCapacitance "c1" "p1" "rise max" .017
definePortCapacitance "c1" "p1" "fall min" .02
definePortCapacitance "c1" "p1" "fall nom" .02
definePortCapacitance "c1" "p1" "fall max" .02
```

This definePortCapacitance function indicates that the capacitance of input port p1 on cell c1 is:

- .017 when the signal is coming in at its fastest speed and the transition is a rise.
- .017 when the signal is coming in at its nominal speed and the transition is a rise.
- .017 when the signal is coming in at its slowest speed and the transition is a rise.
- .02 when the signal is coming in at its fastest speed and the transition is a fall.
- .02 when the signal is coming in at its nominal speed and the transition is a fall.
- .02 when the signal is coming in at its slowest speed and the transition is a fall.

definePortFeedthru

Marks a port as a feedthru port in a CLF file so the router can have multiple connections on the port, even if all the other ports only allow a single connection. This command can only be put in a CLF file.

Tool

Milkyway

Syntax

```
definePortFeedthru cellName portName isFeedThru
```

where the arguments are as follows:

<i>cellName</i>	Name of the cell. Valid values: Name of any cell in the library.
<i>portName</i>	Name of the port. Valid values: Any port in the cell specified by <i>cellName</i> .
<i>isFeedThru</i>	Whether the port can be used as a feedthru port or not. Valid values: #t: If the port is to be used as a feedthru port. #f: If the port is not to be used as a feedthru port.

Example

```
definePortFeedthru "AND4" "A" #t
```

definePortPhaseDelay

This timing command specifies the phase delay for the clock input port on a cell. Phase delay is the delay between the clock input port of a cell and the clock input to the flip-flops inside the cell. In other words, it is the delay through the internal clock tree of the cell. Synopsys tools use this information during clock tree synthesis to determine at which level it should connect cells to the clock tree.

Note:

To specify phase delay for the clock input port on a cell instance, use [dbDefineSyncPin](#). This function lets you specify minimum and maximum phase delays at different phases.

In the future, `definePortPhaseDelay` will be similar to [dbDefineSyncPin](#), except master based.

You can extract this and other timing characteristics for a design using the Timing Analysis > Create Timing Model command. For information, see `aprCreateTimingModel`.

Tool

Milkyway

Syntax

```
definePortPhaseDelay "cellName" "portName" delay
```

where the arguments are as follows:

<i>cellName</i>	Name of the cell that you are specifying phase delay for a port. Valid values: Name of any cell in the library.
<i>portName</i>	Name of the port that you are specifying phase delay. Valid values: Name of any port on cell <i>cellName</i> .
<i>delay</i>	Phase delay of the port. Valid values: Any non-negative, floating-point number.

Example

```
definePortPhaseDelay "block1" "p2" 1.5  
definePortPhaseDelay "block2" "p5" 2.5
```

These `definePortPhaseDelay` functions indicate the following:

- Phase delay for port p2 on cell block1 is 1.5 ns.
- Phase delay for port p5 on cell block2 is 2.5 ns.

definePortTieHigh

`definePortTieHigh` is used to declare a port to always be tied to the power supply voltage as a property that can be used in later tools, such as crosstalk analysis.

Tool

Milkyway

Syntax

```
definePortTieHigh cellName portName define
```

where the arguments are as follows:

<i>cellName</i>	Name of the cell the tie-high port is to be define in. Valid values: Name of any cell in the library.
<i>portName</i>	Name of the port the tie-high port is to be define in. Valid values: Any port in the cell specified by <i>cellName</i> .
<i>define</i>	Use 1 if you want to define a tie-high port and 0 when you want to undefine a tie-high port.

definePortTieLow

definePortTieLow is used to declare a port to always be tied to the ground as a property that can be used in later tools, such as crosstalk analysis.

Tool

Milkyway

Syntax

```
definePortTieLow cellName portName define
```

where the arguments are as follows:

<i>cellName</i>	Name of the cell the tie-low port is to be define in. Valid values: Name of any cell in the library.
-----------------	--

<i>portName</i>	Name of the port the tie-low port is to be define in. Valid values: Any port in the cell specified by cellName.
<i>define</i>	Use 1 if you want to define a tie-low port and 0 when you want to undefine a tie-low port.

defineStaticNoiseMargin

Specifies the static noise margins of a pin named “pin name” and of a cell of named “cell name.” The nominal and maximum values are optional. When only one pair of values are specified, all three corners (min, nom and max) will have the same values.

Tool

Milkyway

Syntax

```
defineStaticNoiseMargin "cell name" "pin name"
'((riseMin fallMin) (riseNom fallNom) (riseMax fallMax))
```

where the arguments are as follows:

<i>cell name</i>	Name of the cell you are specifying the static noise margins. Valid values: Name of any cell in the library.
<i>pin name</i>	Name of the pin you are specifying the static noise margins. Valid values: Any pin in the cell specified by cell name.

defineStaticNoiseWidth

Specifies the static noise width threshold of a pin named "pin name" and of a cell of named "cell name." The nominal and maximum values are optional. When only one pair of values are specified, all three corners (min, nom and max) will have the same values.

The noise margin/width values are stored in the .FRAM view and is used during Astro-Xtalk noise analysis.

Tool

Milkyway

Syntax

```
defineStaticNoiseWidth "cellName" "pinName"  
'((riseMin fallMin) (riseNom fallNom) (riseMax fallMax))
```

where the arguments are as follows:

<i>cellName</i>	Name of the cell you are specifying the static noise margins. Valid values: Name of any cell in the library.
<i>pinName</i>	Name of the pin you are specifying the static noise margins. Valid values: Any pin in the cell specified by cell name.

defineSwitchingPower

This CLF command defines the power dissipated by the cell when the output(s) of the cell switches in response to its input(s) switching. This value includes the energy used to change the logic state(s) of the output(s), as well as the short-circuit power caused by nonzero transition times of input signals.

Use this command to create power views (cellName.PWR) of the cells in a library just as you define timing characteristics by timing view (cellName.TIM) and connectivity relationships by frame view (cellName.FRAME).

Tool

Milkyway

Syntax

```
defineSwitchingPower "cellName" "inputCond" "outputCond"  
    "modelType" "modelArg"
```

where the arguments are as follows:

<i>cellName</i>	Name of the cell or cell master that you are specifying power requirements. Valid values: Name of any cell in the library.
-----------------	--

inputCond Signal values or transitions of the inputs separated by the character “*” to indicated a Boolean AND operation in the following format:

signalName signalCond * signalName signalCond

where the arguments are as follows:

signalName – Name of the input that you are defining a condition.

signalCond – one of the following six conditions:

- “0” – signal stays at 0
- “1” – signal stays at 1
- “-” – doesn’t matter, signal can be either 0 or 1
- “r” – signal has a rising transition
- “f” – signal has a falling transition
- “t” – signal toggles

If the signalCond is not specified, it is treated as having “universal” condition and the model information applies regardless of the signal condition.

outputCond Signal values or transitions of the outputs separated by the character “*” to indicated a Boolean AND operation in the following format:

signalName signalCond * signalName signalCond

where the arguments are as follows:

signalName – Name of the output that you are defining a condition.

signalCond – one of the following six conditions:

- “0” – signal stays at 0
- “1” – signal stays at 1
- “-” – doesn’t matter, signal can be either 0 or 1
- “r” – signal has a rising transition
- “f” – signal has a falling transition
- “t” – signal toggles

If the signalCond is not specified, it is treated as having “universal” condition and the model information applies regardless of the signal condition.

Note:

If all input and output signals are of the same condition, you can omit the signalName. For example, if all input signals stay at “1” in a cell of three inputs “a,” “b,” and “c,” instead of typing “a1*b1*c1”, just type “1.”

Note:

You must define at least one input and one output signal that has transitions, for example, rising, falling, or toggling.

<i>modelType</i>	Specifies the type of power model defined. Valid values: Constant – Power model is a constant value. In this case, the field modelArg of the command has to be a floating-point number. Constant type is allowed in definition of all power components. Linear – Power model is a constant multiplier. In this case, the field modelArg of the command has to be a floating-point number. Linear type is used when the contribution of short-circuit power is unknown and is approximated by multiplying the power for charging/ discharging the loading capacitance by a factor. Therefore, it is only allowed in defineSwitchingPower. TLU – Power model is a look-up table. In this case, the field modelArg of the command is the name of the look-up table that has to be defined prior to the appearance of current command. The type of the referred look-up table has to be the same as the type of the command. If a table is referred in the definition of a defineSwitchingPower command, the type of the table must be “SwitchingPowerTable.”
<i>modelArg</i>	Specifies the corresponding argument of the selected model type. See details in the explanation of model type.

Example

```
(clfCreateTable "AOI33_1X:i1:y:swp0f"  
"SwitchingPowerTable"  
'(("InputNetTransition" (130.000000 250.000000))  
("OutputCapacitance" (20.000 40.000)))  
'(0.178000 0.177000 0.217000 0.219000)  
)  
defineSwitchingPower "AOI33_1X" "i1r*i21*i31*i40*i50*i60"  
"yf"  
"TLU" "AOI33_1X:i1:y:swp0f"
```

This example defines the switching power consumption for cell AOI33_1X. The input conditions are as follows:

Signal Input Name	Signal Input Condition
i1	Falls
i2	Stays at 1
i3	Stays at 1
i4	Stays at 0
i5	Stays at 0
i6	Stays at 0

The input signal “y” is falling. The model type is TLU, and the model argument is the name of the look-up table defined with [clfCreateTable](#) prior to using the [defineSwitchingPower](#) command.

Example

```
(clfCreateTable "AOI211_1X:sw:aqr" "SwitchingPowerTable"  
'(("InputNetTransition" (0.200 0.500 1.000 1.500))  
("OutputCapacitance" (0.080 0.160 0.240 0.400)))  
'(3.779 5.559 7.709 11.449 3.857 5.797 7.947 11.757 4.32 6.20  
8.05 12.03 4.852 6.732 8.582 12.342)  
defineSwitchingPower "AOI211_1X" "at" "qr" "TLU"  
"AOI211_1X:sw:aqr"
```

This example defines the switching power consumption for cell AOI211_1X. The input signal “a” toggles and the output signal “q” rises. The model type is TLU and the model argument is the name of the look-up table defined with [clfCreateTable](#) prior to using the [defineSwitchingPower](#) command.

See Also

[defineInternalPower](#) defines the power consumption of the cell when the input(s) switch but outputs do not.

[defineLeakagePower](#) defines the power dissipated by leakage currents through reverse-biased P-N junctions and subthreshold conduction channels.

[clfCreateTable](#) creates a power table to show the effects of input and output conditions on internal power, leakage power, and switching power.

defineTimeIntrinsic

This timing command specifies the parameters associated with the arc between a specified source port and a specified destination port on a master cell, including the following:

- Unateness
- Timing type
- Intrinsic delay or timing rules (depending on the timing type)

- Slope factor for adjusting the intrinsic delay for the arc due to transition delay on the source port. This delay adjustment models the changing delay characteristics of a circuit when the driving signal has a faster or slower rise/fall time.
- Output resistance

You need to specify these parameters for every arc included in the path constraints.

To calculate the slope factor, you need to understand the model Astro uses to calculate the actual intrinsic delay between a source port and a destination port. The following terms and example explain the model:

- Transition delay is the time it takes for a signal to change state due to loading at an output port. The formula is as follows:

```
output resistance 5 load
```

Note:

When specifying output resistance, you should already have the threshold voltage factored in.

- Connect delay is the time it takes for a state transition to propagate through an interconnect. There is one connect delay for each output-input pair.
- Intrinsic delay is the internal delay between the source port and destination port when the output loading is zero (0).
- Actual intrinsic delay is the intrinsic delay adjusted for changing delay characteristics. The formula is as follows:

Intrinsic delay	+	Slope delay
(a constant)		(calculated by Synopsys tool)

defineTimeIntrinsic

- Slope factor is the number you provide for each operating condition (riseMin, riseMax, fallMin, fallMax) so that Astro can calculate the slope delays for each. The formula is as follows:

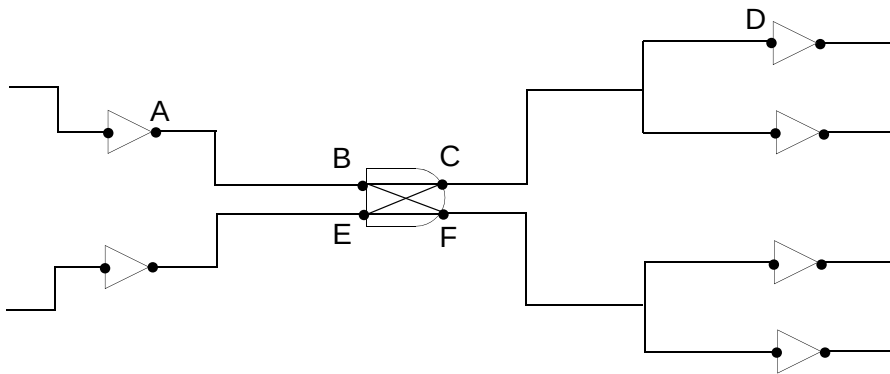
$$\text{Slope delay} = \text{Slope factor} \times \text{Transition delay}$$

Note:

When specifying the slope factor, you should already have the difference between transition delay and real slew time factored in.

- Transition delay is 0 (or 100)-to-threshold switching time.
- Real slew time is normally defined as 10-to-90% switching time.

The figure below provides an example.



Astro uses the following to determine the delay from B to D:

- Transition delay at port C due to down-stream loading
- Connect delay from port C to port D
- Intrinsic delay from port B to port C
- Slope delay on arc B-C due to the transition delay at port A

Note:

You can extract certain timing characteristics for a design using the Post Route > Create Timing Model command. For information, see `apCreateTimingModel`.

Tool

Milkyway

Syntax

```
defineTimeIntrinsic "cellName" "sourcePort" "destPort"  
"unate"  
"timingType" delayOrRuleSet slopeSet resistSet
```

where the arguments differ slightly depending on the `timingType`.

<i>cellName</i>	Name of the cell that internal delay between ports is specified. Valid values: Name of any cell in the library.
<i>sourcePort</i>	Name of the source port. Valid values: Any input port in the cell specified by <i>cellName</i> .
<i>destPort</i>	For intrinsic delay types: Name of the output port. For timing rule types: Name of the clock port. Valid values: Any output/clock port in the cell specified by <i>cellName</i> .

unate

For intrinsic delay types: Specification of the type of output signal generated by a rising edge input signal.

For timing rule types: Not used.

Valid values:

Any of the following:

- *noninvert* indicates that a rising edge at the source generates a rising edge at the destination.
- *invert* indicates that a rising edge at the source generates a falling edge at the destination.
- *nonunate* indicates that a rising edge at the source generates an unknown edge at the destination. Because the polarity can no longer be ascertained, Astro takes the maximum of the falling/rising edge parameters, which reduces the accuracy of the timing estimate.

Note:

For timing rule types, you need to specify one of the valid values, even though it is ignored.

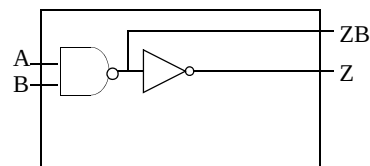
timingType

Timing type of the arc.

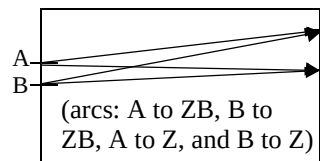
Valid values:

Any type in either of the following categories:

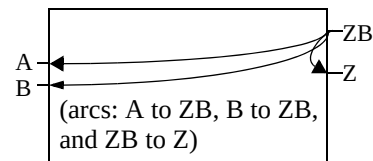
- Intrinsic delay types:
- Path (standard combinatorial delay) indicates the arc type shown below.



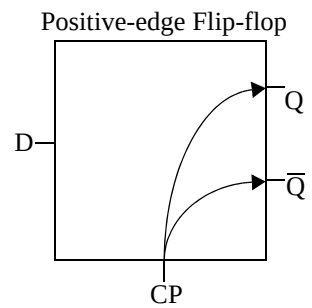
Correct



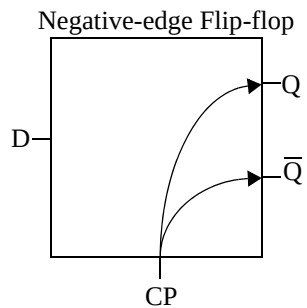
Incorrect



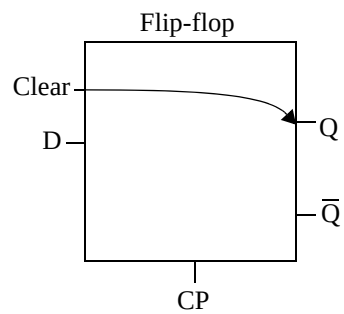
- Clock_rising indicates the arc type shown below.



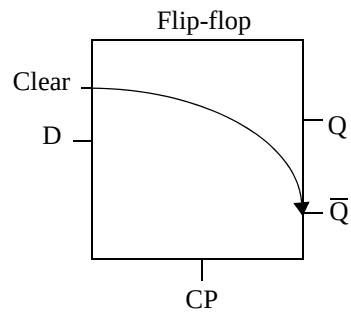
- Clock_falling indicates the arc type shown below.



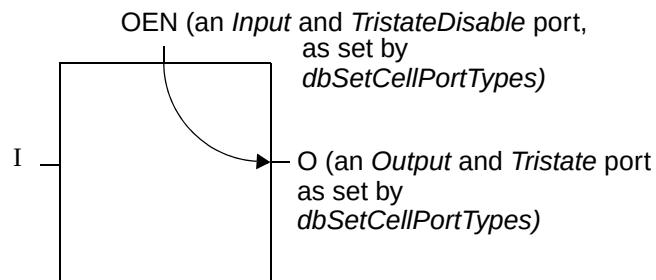
- Clear indicates the arc type shown below.



- Preset indicates the arc type shown below.

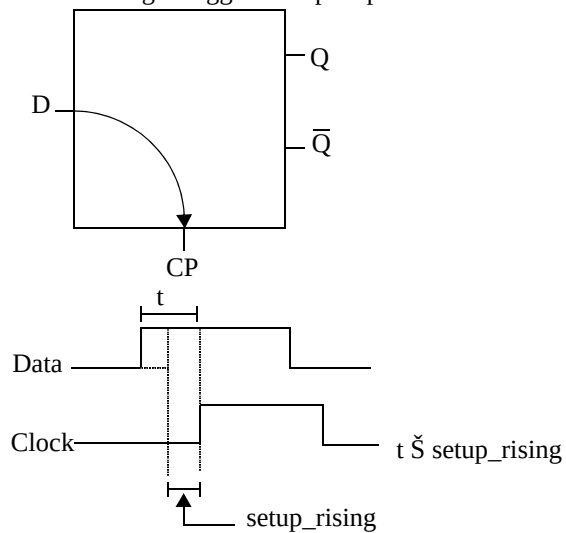


- Disable indicates the arc type shown below.



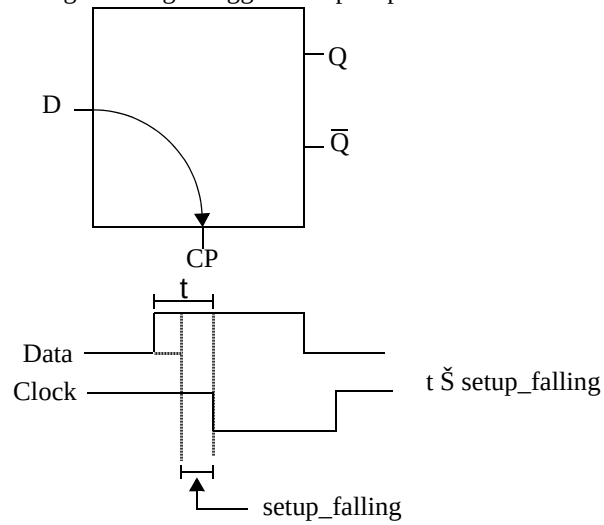
- Timing rule types:
- Setup_rising indicates the arc type shown below.

Positive-edge Triggered Flip-flop



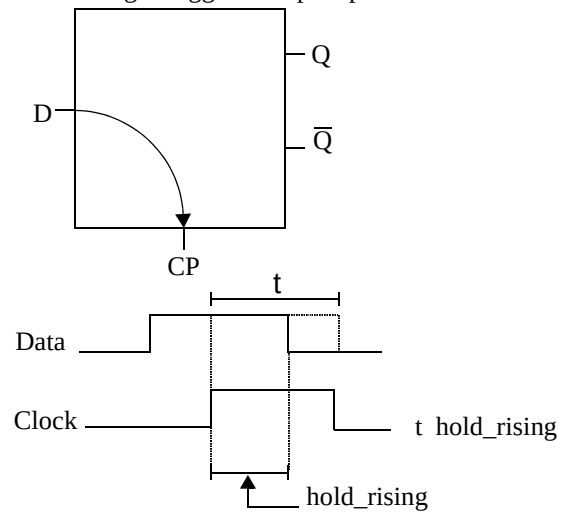
- Setup_falling indicates the arc type shown below.

Negative-edge Triggered Flip-flop



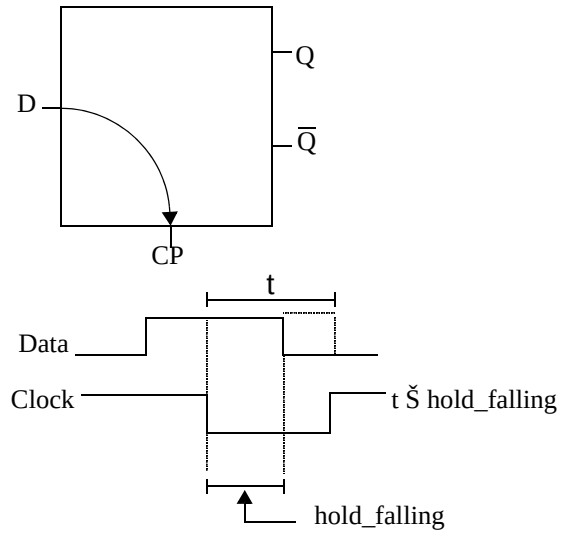
- Hold_rising indicates the arc type shown below.

Positive-edge Triggered Flip-flop



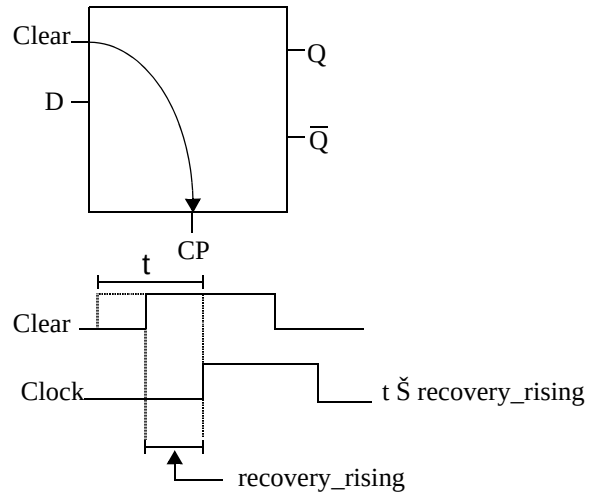
- Hold_falling indicates the arc type shown below.

Negative-edge Triggered Flip-flop



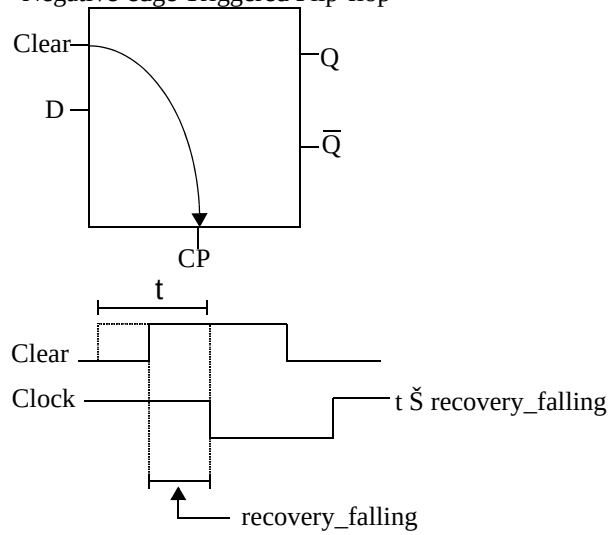
- Recovery_rising indicates the arc type shown below.

Positive-edge Triggered Flip-flop



- Recovery_falling indicates the arc type shown below.

Negative-edge Triggered Flip-flop



delayOrRule
Set

A value or set of values that expresses the following:

- For intrinsic delay types: The minimum delay between the specified sourcePort and the specified destPort (in this case, output port) when the output signal is rising. For timing rule types: The minimum timing rule between the specified sourcePort and the specified destPort (in this case, clock port) when the input signal is rising.
- For intrinsic delay types: The minimum delay between the specified sourcePort and the specified destPort (in this case, output port) when the output signal is falling. For timing rule types: The minimum timing rule between the specified sourcePort and the specified destPort (in this case, clock port) when the input signal is falling.
- For intrinsic delay types: The nominal delay between the specified sourcePort and the specified destPort (in this case, output port) when the output signal is rising. For timing rule types: The nominal timing rule between the specified sourcePort and the specified destPort (in this case, clock port) when the input signal is rising.
- For intrinsic delay types: The nominal delay between the specified sourcePort and the specified destPort (in this case, output port) when the output signal is falling. For timing rule types: The nominal timing rule between the specified sourcePort and the specified destPort (in this case, clock port) when the input signal is falling.
- For intrinsic delay types: The maximum delay between the specified sourcePort and the specified destPort (in this case, output port) when the output signal is rising. For timing rule types: The maximum timing rule between the specified sourcePort and the specified destPort (in this case, clock port) when the input signal is rising.

- For intrinsic delay types: The maximum delay between the specified sourcePort and the specified destPort (in this case, output port) when the output signal is falling
- For timing rule types: The maximum timing rule between the specified sourcePort and the specified destPort (in this case, clock port) when the input signal is falling.

Valid values: Astro needs six values, which you can express in any of the following ways:

- A single value, expressed as follows:

value

where: value is a non-negative, floating-point number that expresses the minimum, nominal, and maximum delay/timing rule when the destination (output) signal is rising and the minimum, nominal, and maximum delay/timing rule when the destination (output) signal is falling.

- A list of two values, expressed as follows:

'(riseTime fallTime)

where:

- riseTime is a non-negative, floating-point number that expresses the minimum, nominal, and maximum delay/timing rule when the destination (output) signal is rising.
- fallTime is a non-negative, floating-point number that expresses the minimum, nominal, and maximum delay/timing rule when the destination (output) signal is falling.
- A list of four values, expressed as follows:

'(minRiseTime minFallTime
nomMaxRiseTime nomMaxFallTime)

where:

- minRiseTime is a non-negative, floating-point number that expresses the minimum delay/timing rule when the destination (output) signal is rising.
- minFallTime is a non-negative, floating-point number that expresses the minimum delay/timing rule when the destination (output) signal is falling.
- nomMaxRiseTime is a non-negative, floating-point number that expresses the nominal and maximum delay/timing when the destination (output) signal is rising.
- nomMaxFallTime is a non-negative, floating-point number that expresses the nominal and maximum delay/timing rule when the destination (output) signal is falling.
- A list of six values, expressed as follows:

'(minRiseTime minFallTime nomRiseTime
nomFallTime maxRiseTime maxFallTime)

where:

- minRiseTime is a non-negative, floating-point number that expresses the minimum delay/timing rule when the destination (output) signal is rising.
- minFallTime is a non-negative, floating-point number that expresses the minimum delay/timing rule when the destination (output) signal is falling.
- nomRiseTime is a non-negative, floating-point number that expresses the nominal delay/timing when the destination (output) signal is rising.
- nomFallTime is a non-negative, floating-point number that expresses the nominal delay/timing rule when the destination (output) signal is falling.
- maxRiseTime is a non-negative, floating-point number that expresses the maximum delay/timing when the destination (output) signal is rising.
- maxFallTime is a non-negative, floating-point number that expresses the maximum delay/timing rule when the destination (output) signal is falling.

slopeSet

For intrinsic delay types: A value or set of values that express the following:

- Slope factor used to adjust the minimum delay between the specified sourcePort and the specified destPort when the destination (output) signal is rising.
- Slope factor used to adjust the minimum delay between the specified sourcePort and the specified destPort when the destination (output) signal is falling.
- Slope factor used to adjust the nominal delay between the specified sourcePort and the specified destPort when the destination (output) signal is rising.
- Slope factor used to adjust the nominal delay between the specified sourcePort and the specified destPort when the destination (output) signal is falling.
- Slope factor used to adjust the maximum delay between the specified sourcePort and the specified destPort when the destination (output) signal is rising.
- Slope factor used to adjust the maximum delay between the specified sourcePort and the specified destPort when the destination (output) signal is falling.

Note:

When specifying slope factor, you should already have the difference between transition delay and real slew time factored in.

defineTimeIntrinsic

For timing rule types: Not used

Valid values:

Astro needs six values, which you can express in any of the following ways:

- A single value, expressed as follows:

value

where: value is a non-negative, floating-point number that expresses the slope factor used to adjust the minimum, nominal, and maximum delay when the destination (output) signal is rising and the minimum, nominal, and maximum delay when the destination (output) signal is falling.

- A list of two values, expressed as follows:

'(riseTime fallTime)

where:

- riseTime is a non-negative, floating-point number that expresses the slope factor used to adjust the minimum, nominal, and maximum delay when the destination (output) signal is rising.
- fallTime is a non-negative, floating-point number that expresses slope factor used to adjust the minimum, nominal, and maximum delay when the destination (output) signal is falling.

- A list of four values, expressed as follows:

```
'(minRiseTime minFallTime nomMaxRiseTime
nomMaxFallTime)
```

where:

- minRiseTime is a non-negative, floating-point number that expresses the slope factor used to adjust the minimum delay when the destination (output) signal is rising.
- minFallTime is a non-negative, floating-point number that expresses the slope factor used to adjust the minimum delay when the destination (output) signal is falling.
- nomMaxRiseTime is a non-negative, floating-point number that expresses the slope factor used to adjust the nominal and maximum delay when the destination (output) signal is rising.
- nomMaxFallTime is a non-negative, floating-point number that expresses the slope factor used to adjust the nominal and maximum delay when the destination (output) signal is falling. A list of six values, expressed as follows:

```
'(minRiseTime minFallTime nomRiseTime
nomFallTime maxRiseTime maxFallTime)
```

where:

- minRiseTime is a non-negative, floating-point number that expresses the slope factor used to adjust the minimum delay when the destination (output) signal is rising.
- minFallTime is a non-negative, floating-point number that expresses the slope factor used to adjust the minimum delay when the destination (output) signal is falling.

- nomRiseTime is a non-negative, floating-point number that expresses the slope factor used to adjust the nominal delay when the destination (output) signal is rising.
- nomFallTime is a non-negative, floating-point number that expresses the slope factor used to adjust the nominal delay when the destination (output) signal is falling.
- maxRiseTime is a non-negative, floating-point number that expresses the slope factor used to adjust the maximum delay when the destination (output) signal is rising.
- maxFallTime is a non-negative, floating-point number that expresses the slope factor used to adjust the maximum delay when the destination (output) signal is falling.

Note:

For timing rule types, you need to specify one of the valid values, even though it is ignored.

resistSet

For intrinsic delay types: A value or set of values that express the following:

- Output resistance (after factoring in threshold voltage) associated with the specified sourcePort and the specified destPort when the destination (output) signal is at its fastest speed and the transition is a rise.
- Output resistance (after factoring in threshold voltage) associated with the specified sourcePort and the specified destPort when the destination (output) signal is at its fastest speed and the transition is a fall.
- Output resistance (after factoring in threshold voltage) associated with the specified sourcePort and the specified destPort when the destination (output) signal is at its nominal speed and the transition is a rise.
- Output resistance (after factoring in threshold voltage) associated with the specified sourcePort and the specified destPort when the destination (output) signal is at its nominal speed and the transition is a fall.
- Output resistance (after factoring in threshold voltage) associated with the specified sourcePort and the specified destPort when the destination (output) signal is at its slowest speed and the transition is a rise.
- Output resistance (after factoring in threshold voltage) associated with the specified sourcePort and the specified destPort when the destination (output) signal is at its slowest speed and the transition is a fall.

For timing rule types: Not used

Valid values: Astro needs four values, which you can express in any of the following ways:

defineTimeIntrinsic

- A single value, expressed as follows:
value
where: value is a non-negative, floating-point number that expresses the resistance when the destination(output) signal is at it slowest, nominal, or fastest speed and the transition is a rise or a fall.
- A list of two values, expressed as follows:
'(riseResistance fallResistance)
where:
 - riseResistance is a non-negative, floating-point number that expresses the resistance when the destination (output) signal is at it slowest, nominal, or fastest speed and the transition is a rise.
 - fallResistance is a non-negative, floating-point number that expresses the resistance when the destination (output) signal is at it slowest, nominal, or fastest speed and the transition is a fall.
- A list of four values, expressed as follows:
'(minRiseResistance minFallResistance
omMaxRiseResistance
nomMaxFallResistance)
where:
 - minRiseResistance is a non-negative, floating-point number that expresses the resistance when the destination (output) signal is at it fastest speed and the transition is a rise.
 - minFallResistance is a non-negative, floating-point number that expresses the resistance when the destination (output) signal is at it fastest speed and the transition is a fall.

- nomMaxRiseResistance is a non-negative, floating-point number that expresses the resistance when the destination (output) signal is at its nominal or slowest speed and the transition is a rise.
- nomMaxFallResistance is a non-negative, floating-point number that expresses the resistance when the destination (output) signal is at its nominal or slowest speed and the transition is a fall.
- A list of six values, expressed as follows:
'(minRiseResistance minFallResistance
nomRiseResistance nomFallResistance)
maxRiseResistance maxFallResistance)
where:
 - minRiseResistance is a non-negative, floating-point number that expresses the resistance when the destination (output) signal is at its fastest speed and the transition is a rise.
 - minFallResistance is a non-negative, floating-point number that expresses the resistance when the destination (output) signal is at its fastest speed and the transition is a fall.
 - nomRiseResistance is a non-negative, floating-point number that expresses the resistance when the destination (output) signal is at its nominal speed and the transition is a rise.
 - nomFallResistance is a non-negative, floating-point number that expresses the resistance when the destination (output) signal is at its nominal speed and the transition is a fall.
 - maxRiseResistance is a non-negative, floating-point number that expresses the resistance when the destination (output) signal is at its slowest speed and the transition is a rise.
 - maxFallResistance is a non-negative, floating-point number that expresses the resistance when the destination (output) signal is at its slowest speed and the transition is a fall.

Note:

For timing rule types, you need to specify one of the valid values, even though it is ignored.

Example (for intrinsic delay arc type)

```
defineTimeIntrinsic "cell1" "p1" "p2" "invert" "Path"  
'(4 6 5 7) .02 '(.25 .33)
```

This defineTimeIntrinsic function indicates the following:

- A rising edge input of a signal traveling between port p1 and port p2 in cell1 generates a falling edge output.
- The timing type of the arc is Path.
- The delay between the fall of the input to p1 and the rise of the output to p2 is between four (4) and five (5) ns.
- The delay between the rise of the input to p1 and the fall of the output to p2 is between six (6) and seven (7) ns.
- A slope factor of 0.02 is used to adjust the minimum and maximum delay when the output signal is rising and the minimum, nominal, and maximum delay when the output signal is falling (slope delay time = 0.02 5 transition delay time).
- The resistance associated with ports p1 and p2 when the p2 signal is a rising is 0.25.
- The resistance associated with ports p1 and p2 when the p2 signal is a falling is 0.33.

Example (for timing rule arc type)

```
defineTimeIntrinsic "FF1" "D" "CP" "nonunate" "Setup_rising"  
'(11 22) 0 0
```

This defineTimeIntrinsic function indicates the following:

- The timing type of the arc is Setup_rising.
- The timing rule between the rise of the input to D and the rise of the clock to CP is 11 ns.
- The timing rule between the fall of the input to D and the rise of the clock to CP is 22 ns.

defineTimeTLU

This timing command specifies the timing characteristics of an arc.

Tool

Milkyway

Prerequisite

Before you can specify the timing characteristics of an arc, you must create timing tables using [clfCreateTable](#).

Syntax

```
defineTimeTLU "cellName" "sourcePort" "destPort" "unate"  
              "timingType" "propOrCellOrConstrTable" "tranTable"
```

where the arguments are as follows:

<i>cellName</i>	Name of the cell that port-to-port timing is specified. Valid values: Name of any cell in the library.
-----------------	--

<i>sourcePort</i>	Name of the start port. Valid values: Any port in the cell specified by cellName.
<i>destPort</i>	For intrinsic delay types: Name of the output port. For timing rule types: Name of the clock port. Valid values: Any output/clock port in the cell specified by cellName.
<i>unate</i>	For intrinsic delay types: Specification of the type of destination (output) signal generated by a rising edge source signal. For timing rule types: Not used Valid values: Any of the following: • noninvert indicates that a rising edge source generates a rising edge destination. • invert indicates that a rising source input generates a falling edge destination. • nonunate indicates that a rising edge source generates an unknown edge destination. Because the polarity can no longer be ascertained, Astro takes the maximum of the falling/rising edge parameters, which reduces the accuracy of the timing estimate.

Note:

For timing rule types, you need to specify one of the valid values, even though it is ignored.

<i>timingType</i>	Timing type of the arc. Valid values: Any type in either of the following categories: Intrinsic delay types: • Path • Clock_rising • Clock_falling
-------------------	--

- Clear
- Preset
- Timing rule types:
- Setup_rising
- Setup_falling
- Hold_rising
- Hold_falling
- Recovery_rising
- Recovery_falling

Note:

For descriptions of these timing types, see the `timingType` argument for [defineTimeIntrinsic](#).

*propOrCellOr
ConstrTable*

Names of timing tables that express propagation or cell delays (for intrinsic delay types) or delay constraints (for timing rule types) for the arc, in the following format:

- For intrinsic delay types:

```
'(minRisePOrCTable minFallPOrCTable
[nomRisePOrCTable nomFallPOrCTable]
maxRisePOrCTable maxFallPOrCTable)
```

where:

- `minRisePOrCTable` is the name of the timing table that expresses the minimum propagation or cell delays for the arc when the signal is rising.

Note:

This table must have a `tableType` of `RisePropagationTable` or `RiseCellDelayTable` (see [clfCreateTable](#)).

- `minFallPOrCTable` is the name of the timing table that expresses the minimum propagation or cell delays for the arc when the signal is falling.

Note:

This table must have a `tableType` of `FallPropagationTable` or `FallCellDelayTable` (see [clfCreateTable](#)).

defineTimeTLU

- `nomRisePOrCTable` is the name of the timing table that expresses the nominal propagation or cell delays for the arc when the signal is rising. If you omit this argument, Astro uses the value given for `maxRisePOrCTable`.

Note:

This table must have a `tableType` of `RisePropagationTable` or `RiseCellDelayTable` (see [clfCreateTable](#)).

- `nomFallPropTable` is the name of the timing table that expresses the nominal propagation or cell delays for the arc when the signal is falling. If you omit this argument, Astro uses the value given for `maxFallPOrCTable`.

Note:

This table must have a `tableType` of `FallPropagationTable` or `FallCellDelayTable` (see [clfCreateTable](#)).

- `maxRisePOrCTable` is the name of the timing table that expresses the maximum propagation or cell delays for the arc when the signal is rising.

Note:

This table must have a `tableType` of `RisePropagationTable` or `RiseCellDelayTable` (see [clfCreateTable](#)).

- `maxFallPOrCTable` is the name of the timing table that expresses the maximum propagation or cell delays for the arc when the signal is falling.

Note:

This table must have a `tableType` of `FallPropagationTable` or `FallCellDelayTable` (see [clfCreateTable](#)).

- For timing rule types:
`'(minRiseConstrTable minFallConstrTable`
`[nomRiseConstrTable nomFallConstrTable]`
`maxRiseConstrTable maxFallConstrTable)`
 where:
 - `minRiseConstrTable` is the name of the timing table that expresses the minimum delay constraints for the arc when the signal is rising.

Note:

This table must have a tableType of RiseConstraintTable (see [clfCreateTable](#)).

- minFallConstrTable is the name of the timing table that expresses the minimum delay constraints for the arc when the signal is falling.

Note:

This table must have a tableType of FallConstraintTable (see [clfCreateTable](#)).

- nomRiseConstrTable is the name of the timing table that expresses the nominal delay constraints for the arc when the signal is rising. If you omit this argument, Astro uses the value given for maxRiseConstrTable.

Note:

This table must have a tableType of RiseConstraintTable (see [clfCreateTable](#)).

- nomFallConstrTable is the name of the timing table that expresses the nominal delay constraints for the arc when the signal is falling. If you omit this argument, Astro uses the value given for maxFallConstrTable.

Note:

This table must have a tableType of FallConstraintTable (see [clfCreateTable](#)).

- maxRiseConstrTable is the name of the timing table that expresses the maximum delay constraints for the arc when the signal is rising.

Note:

This table must have a tableType of RiseConstraintTable (see [clfCreateTable](#)).

- maxFallConstrTable is the name of the timing table that expresses the maximum delay constraints for the arc when the signal is falling.

Note:

This table must have a tableType of FallConstraintTable (see [clfCreateTable](#)).

tranTable

Names of the timing tables that express transition delays (for intrinsic delay types) for the arc, in the following format:

- For intrinsic delay types:

```
'(minRiseTranTable minFallTranTable  
[nomRiseTranTable nomFallTranTable]  
maxRiseTranTable maxFallTranTable)
```

where:

- minRiseTranTable is the name of the timing table that expresses the minimum transition delays for the arc when the signal is rising.

Note:

This table must have a tableType of RiseTransitionTable (see [clfCreateTable](#)).

- minFallTranTable is the name of the timing table that expresses the minimum transition delays for the arc when the signal is falling.

Note:

This table must have a tableType of FallTransitionTable (see [clfCreateTable](#)).

- nomRiseTranTable is the name of the timing table that expresses the nominal transition delays for the arc when the signal is rising. If you omit this argument, Astro uses the value given for maxRiseTranTable.

Note:

This table must have a tableType of RiseTransitionTable (see [clfCreateTable](#)).

- nomFallTranTable is the name of the timing table that expresses the nominal transition delays for the arc when the signal is falling. If you omit this argument, Astro uses the value given for maxFallTranTable.

Note:

This table must have a tableType of FallTransitionTable (see [clfCreateTable](#)).

- maxRiseTranTable is the name of the timing table that expresses the maximum transition delays for the arc when the signal is rising.

Note:

This table must have a tableType of RiseTransitionTable (see [clfCreateTable](#)).

- maxFallTranTable is the name of the timing table that expresses the maximum transition delays for the arc when the signal is falling.

Note:

This table must have a tableType of FallTransitionTable (see [clfCreateTable](#)).

- For timing rule types:
'()

Example 1

```
defineTimeTLU "cell1" "p1" "p2" "invert" "Path"
' ("t1" "t2" "t3" "t4") ' ("t17" "t18" "t19" "t20")
```

This defineTimeTLU function indicates the following:

- A rising edge on port p1 generates a falling edge on port p2.
- The timing type of the arc is Path.
- Timing tables t1, t2, t3, and t4 express the propagation delays between port p1 and port p2 in cell1.
- Timing tables t17, t18, t19, and t20 express the transition delays between port p1 and port p2 in cell1.

Example 2

```
defineTimeTLU "cellA" "pA" "pB" "invert" "Setup_rising"
' ("t31" "t32" "t33" "t34") '()
```

defineTimeTLU

This defineTimeTLU function indicates the following:

- The timing type of the arc is Setup_rising.
- Timing tables t31, t32, t33, and t34 express the delay constraints between port pA and port pB in cellA.

defineTriStateFunction

Defines the tri-state function of a cell. The output is tri-stated if the enable_function is true.

Tool

Milkyway

Syntax

```
defineTriStateFunction "cell_name" "output_pin_name"  
"enable"  
"input_function"
```

where the arguments are as follows:

<i>cellName</i>	The name of the cell whose tri-state function you wish to define.
<i>output_pin_name</i>	The name of the output pin or inout pin of the cell.
<i>enable</i>	The state of the enable input of latch of the tri-state buffer. The boolean value of an output pin or inout pin in terms of input pins or inout pins.
<i>input_function</i>	The boolean value of an output pin or inout pin in terms of input pins or inout pins.

Example

```
defineTriStateFunction "tribuf1x" "z" "!en" "din"
```


28

eip Commands for Milkyway

This chapter describes commands with the prefix *eip*. The commands are listed alphabetically. These commands are used for edit-in-place operations.

- [eipPop](#)
- [eipPush](#)
- [eipQuit](#)

eipPop

Quits current edit-in-place operations and switches the editing environment to the parent cell at one level higher.

Tools and Menu Commands

Astro: Cell > Edit-In-Place – Up One Level!

Jupiter: Cell > Edit-In-Place – Up One Level!

Milkyway: Cell > Edit-In-Place – Up One Level!

See Also

[eipPush](#) – For editing any child cell in the context of the current design cell (Edit-In-Place).

[eipQuit](#) – Quits all edit-in-place operations and switches the environment back to the top design cells.

eipPush

Edits any child cell in the context of the current design cell (the edit-in-place operation).

To use this command, first select one and *only one* cell instance. Once the command is invoked, the editing environment is switched to the selected child cell with the child cell placed as in the top design cell. All the routing of the top design cell is still visible, but only the objects in the current cell can be edited. If the child cell is in a read-only library, the editing mode is switched to read-only automatically.

By default, this command switches to the view of the selected cell instance. You can switch to a different view by using the `geWindowOption` command to set the *Display Child Viewname* box.

For example, if you enter

```
geWindowOption  
setFormField "Window Options" "Display Child Viewname" "CEL"  
formCancel "Window Options"
```

the command opens the .CEL view for editing instead of the original .FRAM view.

You can apply this command recursively to any child cell at lower levels of the hierarchy.

Tools and Menu Commands

Astro: Cell > Edit-In-Place – Edit Selected !

Jupiter: Cell > Edit-In-Place – Edit Selected !

Milkyway: Cell > Edit-In-Place – Edit Selected !

See Also

[eipPop](#) – Quits current edit-in-place operations and switches the editing environment to the parent cell at one level higher.

[eipQuit](#) – Quits all edit-in-place operations and switches the environment back to the top design cells.

[geWindowOption](#) – Allows you to change the window options for the current window (and the default options for windows you subsequently open during the current Synopsys session).

eipQuit

Quits all edit-in-place operations and switches the environment back to the top design cells.

Tools and Menu Commands

Astro: Cell > Edit-In-Place – Quit EIP!

Jupiter: Cell > Edit-In-Place – Quit EIP!

Milkyway: Cell > Edit-In-Place – Quit EIP!

See Also

[eipPop](#) – Quits current edit-in-place operations and switches the editing environment to the parent cell at one level higher.

[eipPush](#) – For editing any child cell in the context of the current design cell (edit-in-place operation).

30

ek Commands for Milkyway

This chapter describes commands with the prefix *ek*. The commands are listed alphabetically. These commands are used for extraction.

- [ekGetPostRouteCapMultiplier](#)
- [ekGetPostRouteResMultiplier](#)
- [ekSetPostRouteCapMultiplier](#)
- [ekSetPostRouteResMultiplier](#)

ekGetPostRouteCapMultiplier

Prints out the current values of the
`ekSetPostRouteCapMultiplier` command.

Tool

Milkyway

Syntax

`ekGetPostRouteCapMultiplier`

See Also

[ekSetPostRouteCapMultiplier](#)

ekGetPostRouteResMultiplier

Prints out the current values of the
`ekSetPostRouteResMultiplier` command.

Tool

Milkyway

Syntax

`ekGetPostRouteResMultiplier`

See Also

[ekSetPostRouteResMultiplier](#)

ekSetPostRouteCapMultiplier

Scales the minimum and maximum parasitic capacitance that is extracted during layout parasitic extraction, or read from a PARA view, for a routed cell.

To enable this scaling, you must select the minimum and maximum operating condition options, using the Parasitics page of the AstroTime Timing Setup dialog box (`atTimingSetup`).

You can print out the current values of this command with the `ekGetPostRouteCapMultiplier` command.

Tool

Milkyway

Syntax

```
ekSetPostRouteCapMultiplier minCapMult MaxCapMult
```

where the arguments are as follows:

<i>minCapMult</i>	Specifies the minimum capacitance multiplier number.
<i>MaxCapMult</i>	Specifies the maximum capacitance multiplier number.

Set the multipliers to 0 to delete these cell properties.

Example

```
ekSetPostRouteCapMultiplier 0.90 1.10
```

See Also

[ekGetPostRouteCapMultiplier](#)

ekSetPostRouteResMultiplier

Scales the minimum and maximum parasitic resistance that is extracted during layout parasitic extraction, or read from a PARA view, for a routed cell.

To enable this scaling, you must select the minimum and maximum operating condition options, using the Parasitics page of the AstroTime Timing Setup dialog box (`atTimingSetup`).

You can print out the current values of this command with the `ekGetPostRouteResMultiplier` command.

Tool

Milkyway

Syntax

```
ekSetPostRouteResMultiplier minResMult MaxResMult
```

where the arguments are as follows:

<i>minResMult</i>	Specifies the minimum resistance multiplier number.
-------------------	---

<i>MaxResMult</i>	Specifies the maximum resistance multiplier number.
-------------------	---

Set the multipliers to 0 to delete these cell properties.

Example

```
ekSetPostRouteResMultiplier 0.95 1.05
```


See Also

[ekGetPostRouteResMultiplier](#)

[ekSetPostRouteCapMultiplier](#)

31

ek Commands (No license)

This chapter describes commands with the prefix *ek*. The commands are listed alphabetically. These commands are used for extraction.

- [ekGetCoupleCapThreshold](#)
- [ekSetCoupleCapThreshold](#)
- [ekSetLPEMode](#)
- [ekSetLPEOperatingConds](#)
- [ekSetParasiticSource](#)

ekGetCoupleCapThreshold

Displays the current threshold setting of the design cell. The absolute value is displayed in the Synopsys database (.db) format.

Tool

Astro

Syntax

```
ekGetCoupleCapThreshold
```

ekSetCoupleCapThreshold

Sets the percentage and absolute thresholds for the coupling capacitance threshold. The command has two arguments. The first argument is a percentage threshold. The second argument is an absolute threshold. The unit is femtofarad (fF).

Tool

Astro

Syntax

```
ekSetCoupleCapThreshold perThreshold absThreshold
```

Example

```
ekSetCoupleCapThreshold 0.02 5
```

Sets the percentage threshold to 2 percent and the absolute threshold to 5 fF.

ekSetLPEMode

Sets the extraction mode for runtime layout parasitic extraction (LPE).

Tool

Astro

Syntax

```
ekSetLPEMode  
"Virtual_RC" | "Real_R_Virtual_C" | "Real_RC" | "Auto"
```

The extraction mode translates into routing and coupling modes based on the following table, where

VR = virtual routing

RR = real routing

CDC = congestion-map-based coupling

RC = real coupling

Table 31-1

Type of net	Virtual_RC	Real_R_Virtual_C	Real_RC
Unrouted net	VR, CDC	VR, CDC	VR, CDC
Global routed net	VR, CDC	RR, CDC	RR, CDC
Detail routed net	VR, CDC	RR, CDC	RR, RC

In “Auto” mode, the tool uses “Real_RC” if most nets are routed and “Virtual_RC” otherwise.

ekSetLPEOperatingConds

Sets the operating conditions for runtime layout parasitic extraction.

Tool

Astro

Syntax

```
ekSetLPEOperatingConditions ({ "Min" | "Nom" | "Max" } ...)
```

Examples

```
ekSetLPEOperatingConditions "Nom"  
ekSetLPEOperatingConditions "Min" "Max"
```

ekSetParasiticSource

Sets the source of parasitic information.

Tool

Astro

Syntax

```
ekSetParasiticSource "LPE" | "DB" | "DB_then_LPE"
```

Table 31-2

Option	Source of parasitic information
LPE	Runtime extraction
DB	Parasitic view
DB_then_LPE	Parasitic view if existing, otherwise runtime extraction

33

fc Commands for Milkyway

This chapter describes the following command with the prefix *fc*. This command is used for flip-chip operations.

- [fcPad](#)

fcPad

Specifies the bump cell location by absolute coordinates in the TDF file.

Tool

Milkyway

Syntax

```
fcPad "instName" xoffset yoffset "pattern"
```

where the arguments are as follows:

<i>instName</i>	The bump cell instance name.
<i>xoffset</i>	Specify the x coordinate.
<i>yoffset</i>	Specify the y coordinate.
<i>pattern</i>	Specify the bump cell orientation.

36

ge Commands for Milkyway

This chapter describes commands with the prefix *ge*. The commands are listed alphabetically.

For lower level commands with the prefix *ge*, see “Scheme Graphic Editor Extensions” in the online Help.

- [geAddBlockage](#)
- [geAddBoundary](#)
- [geAddCell](#)
- [geAddContact](#)
- [geAddContactArray](#)
- [geAddPath](#)
- [geAddPolygon](#)

- `geAddRectangle`
- `geAddRectilinearBoundary`
- `geAddRectilinearBoundaryForChildCell`
- `geAddRectPin`
- `geAddRoutingBlockage`
- `geAddSMPin`
- `geAddText`
- `geAddWire`
- `geAllError`
- `geBackupCell`
- `geCheckMinGrid`
- `geCheckPinsOnUserGrid`
- `geCloseLib`
- `geCloseWindow`
- `geConfirmCloseLib`
- `geConvertInstPinToPin`
- `geConvertWireToPin`
- `geCopy`
- `geCopyCell`
- `geCreateCell`
- `geCreateConnView`

- `geCreateLogicView`
- `geCreateModelView`
- `geCurrentError`
- `geCurrentErrorType`
- `geDefineBindKey`
- `geDelete`
- `geDeleteCell`
- `geDeleteRuler`
- `geDeselectAll`
- `geDeselectAllObject`
- `geDeselectObject`
- `geErrorBrowser`
- `geFirstError`
- `geFirstErrorType`
- `geFit`
- `geFitBorder`
- `geFitSelSet`
- `geGetEditCell`
- `geGetEnvValue`
- `geGetInputData`
- `geGetSelectedSet`

- `geGoToError`
- `geGridOption`
- `geImportPDB`
- `geImposeCell`
- `geIsLayerSelectable`
- `geIsLayerVisible`
- `geIsObjectSelectable`
- `geIsObjectVisible`
- `geLastView`
- `geLayerPanel`
- `geLineDeselect`
- `geLineSelect`
- `geListCoordinatesOfAllErrorType`
- `geListErrorSummary`
- `geListHierarchy`
- `geLoadDetailRouteErrCell`
- `geLoadErrorCell`
- `geMerge`
- `geMergeFatBlockage`
- `geModify`
- `geMove`

- `geMoveCell`
- `geMoveGroup`
- `geMovePoint`
- `geMoveRegion`
- `geMoveSMPin`
- `geNameSelect`
- `geNewMakeMacro`
- `geNewShortLocator`
- `geNextError`
- `geNextErrorType`
- `geOpenCell`
- `geOpenLib`
- `geOpenLocator`
- `geOutputBBox`
- `geOutputLine`
- `geOutputPoint`
- `gePan`
- `gePan2`
- `gePointDeselect`
- `gePointSelect`
- `gePrepLibs`

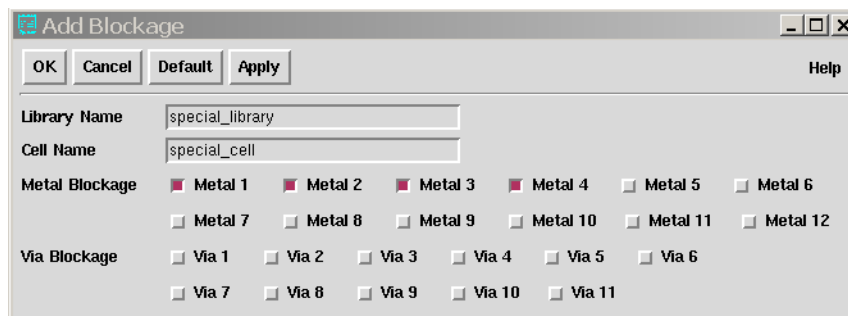
- `gePreviousError`
- `gePurgeCell`
- `geQueryError`
- `geQueryGroup`
- `geQueryGroupName`
- `geQueryNet`
- `geQueryNetConstraint`
- `geQueryNetName`
- `geQueryObject`
- `geRedo`
- `geRedraw`
- `geReShape`
- `geRollbackCell`
- `geRuler`
- `geSaveAllOpenCells`
- `geSaveAs`
- `geSelectAll`
- `geSelectInstancesOnNet`
- `geSelectObject`
- `geSetCellInstColor`
- `geSetConfirmPinDeletionFlag`

- `geSetCustomRouteTypesVis`
- `geSetErrorType`
- `geSetEnvValue`
- `geSetLayerSelectable`
- `geSetLayerVisible`
- `geSetObjectSelectable`
- `geSetObjectVisible`
- `geSetNetTrace`
- `geSetNetTypesVis`
- `geSetViewCell`
- `geShowCellList`
- `geSmashCellInst`
- `geSplit`
- `geStretch`
- `geSummary`
- `geSuppressHltRedraw`
- `geSwapCells`
- `geTransform`
- `geTurnErrorOff`
- `getXWindowId`
- `geUndo`

- `geUnpackCellInstArray`
- `geViewDown`
- `geViewLeft`
- `geViewRight`
- `geViewUp`
- `geWindowDeselect`
- `geWindowOption`
- `geWindowSelect`
- `geWindowStretch`
- `geZoomIn`
- `geZoomInBy2`
- `geZoomOutBy2`
- `geZoomOutBy4`
- `geZoomOutBy8`
- `setInstViewLevel`
- `addRelativePoint`
- `layoutWindowGrabFocus`

geAddBlockage

Adds cell blockages to the specified cell or cells. The `geAddBlockage` command opens the Add Blockage dialog box, as shown. Its options are described below.



Syntax

(See [Example](#))

Library Name

Name of the library specified in the "Library Name" field.

Cell Name

The name of the cell or wildcard (*) in which to create cell blockages. The wildcard generates blockages in all macro cells in the library, and the function skips all other cell types.

Metal Blockage

The metal blockage layers on which to create a blockage. Select appropriate check boxes: Metal 1 through Metal12.

Via Blockage

The via blockage layers on which to create a blockage. Select appropriate check boxes: Via 1 through Via11.

Description

The `geAddBlockage` function adds cell blockages to the specified cell or cells. For each specified metal and via blockage layer, this function creates a polygon on the corresponding blockage layer. These polygons have the same dimensions as the cell boundary. In the following example, 1 enables blockage and 0 disables blockage creation.

Example

```
geAddBlockage
setFormField "Add Blockage" "Library Name" "design"
setFormField "Add Blockage" "Cell Name" "*"
setFormField "Add Blockage" "Metal1" "1"
setFormField "Add Blockage" "Metal2" "1"
setFormField "Add Blockage" "Metal3" "1"
setFormField "Add Blockage" "Metal4" "1"
setFormField "Add Blockage" "Metal5" "1"
setFormField "Add Blockage" "Via1" "1"
setFormField "Add Blockage" "Via2" "1"
setFormField "Add Blockage" "Via3" "1"
setFormField "Add Blockage" "Via4" "1"
setFormField "Add Blockage" "Via5" "1"
formOK "Add Blockage"
```

geAddBoundary

Adds a rectangular cell boundary you specify by entering diagonal corners.

Tools and Menu Commands

Astro: Create > Boundary

Jupiter: Astro Floorplanning: Create > Boundary

Milkyway: Create > Boundary

geAddCell

Adds an instance of a master cell at a point you specify.

Tools and Menu Commands

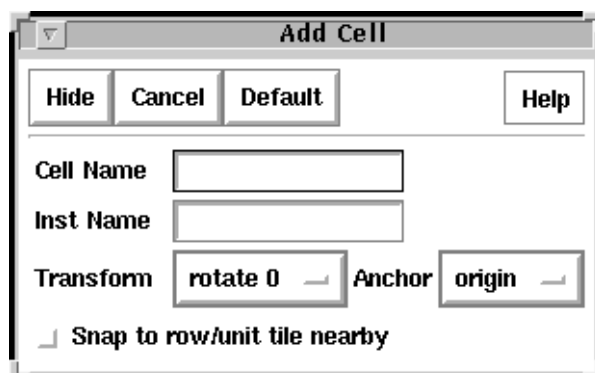
Astro: Create > Cell

Jupiter: Astro Floorplanning: Create > Cell

Milkyway: Create > Cell

Add Cell Dialog Box

The menu command opens the following dialog box. Its options are described below.



Cell Name

Type the name of the master cell that you want to use. The cell master must be in the current library or in a reference library.

Inst Name

Type the name that you want to assign to the instance of the master cell.

Transform

Select the orientation you want for placement of the cell.

Option	Adds the cell ...
rotate 0	Without changing its orientation.
rotate 90	Rotated counterclockwise 90 degrees.
rotate 180	Rotated counterclockwise 180 degrees.
rotate 270	Rotated counterclockwise 270 degrees.
mirror X	Flipped on the y axis (in the X direction).
mirror Y	Flipped on the x axis (in the Y direction).
rotate 90 mirror X	Rotated 90 degrees and flipped on the y axis (in the X direction).
rotate 90 mirror Y	Rotated 90 degrees and flipped on the x axis (in the Y direction).

Anchor

Select the point in the cell that you want placed at the point you specify.

Option	Places the following point in the cell at the point you specify:
origin	Origin.
left-bottom	Bottom left corner.
center-bottom	Center of the bottom edge.
right-bottom	Bottom right corner.
left-center	Center of the left edge.

Option	Places the following point in the cell at the point you specify:
center	Center.
right-center	Center of the right edge.
left-top	Top left corner.
center-top	Center of top edge.
right-top	Top right corner.

Snap to row/unit tile nearby

Snaps the newly created cell instance to the nearest row or unit tile.

geAddContact

Adds a contact at a point you specify by clicking in the design window.

Tools and Menu Commands

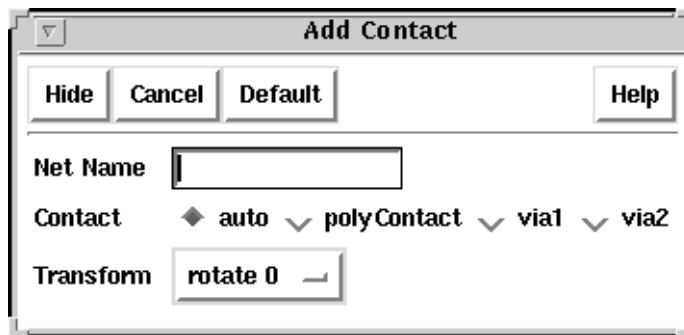
Astro: Create > Contact

Jupiter: Astro Floorplanning: Create > Contact

Milkyway: Create > Contact

Add Contact Dialog Box

The menu command opens the following dialog box. Its options are described below.



Net Name

Type the name of the net to which you want the contact connected.

Contact

Select the type of contact you want to add.

Option	Adds a contact that is ...
auto	Automatically selected from the contacts defined in the technology file.
all others	Of a specific type. The dialog box displays an option for each contact defined the technology file.

Transform

Select the orientation you want for placement of the contact.

Option	Adds the contact ...
rotate 0	Without changing its orientation.
rotate 90	Rotated counterclockwise 90 degrees.
rotate 180	Rotated counterclockwise 180 degrees.
rotate 270	Rotated counterclockwise 270 degrees.

Option	Adds the contact ...
mirror X	Flipped on the y axis (in the X direction).
mirror Y	Flipped on the x axis (in the Y direction).
rotate 90 mirror X	Rotated 90 degrees and flipped on the y axis (in the X direction).
rotate 90 mirror Y	Rotated 90 degrees and flipped on the x axis (in the Y direction).

geAddContactArray

Adds a contact array at a point you specify by clicking in the design window.

Tools and Menu Commands

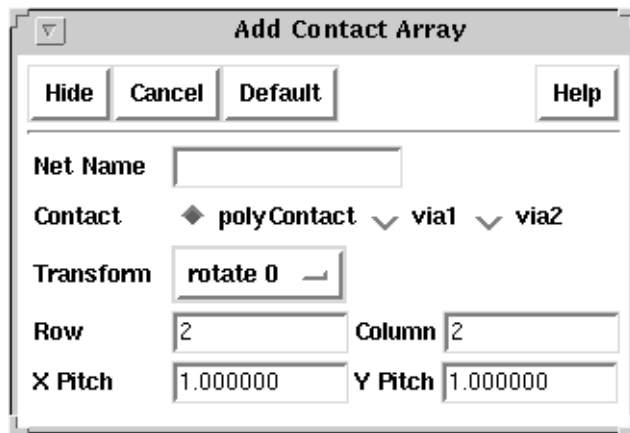
Astro: Create > Contact – Array

Jupiter: Astro Floorplanning: Create > Contact – Array

Milkyway: Create > Contact – Array

Add Contact Array Dialog Box

The menu command opens the following dialog box. Its options are described below.



Net Name

Type the name of the net to which you want the contact array connected.

Contact

Select the type of contacts you want to add from those on the dialog box. The dialog box displays an option for each contact defined in the technology file.

Transform

Select the orientation you want for placement of the added contacts.

Option	Adds the contacts ...
rotate 0	Without changing their orientation.
rotate 90	Rotated counterclockwise 90 degrees.
rotate 180	Rotated counterclockwise 180 degrees.
rotate 270	Rotated counterclockwise 270 degrees.
mirror X	Flipped on the y axis (in the X direction).

Option	Adds the contacts ...
mirror Y	Flipped on the x axis (in the Y direction).
rotate 90 mirror X	Rotated 90 degrees and flipped on the y axis (in the X direction).
rotate 90 mirror Y	Rotated 90 degrees and flipped on the x axis (in the Y direction).

Row

Type the number of horizontal rows you want in the contact array.

Column

Type the number of vertical columns you want in the contact array.

X/YPitch

Type the distance you want between the centers of the contacts in a row (X Pitch), or a column (Y Pitch).

geAddPath

Adds a path you specify by entering points in the design window. You can only add a path on one layer at a time.

Tools and Menu Commands

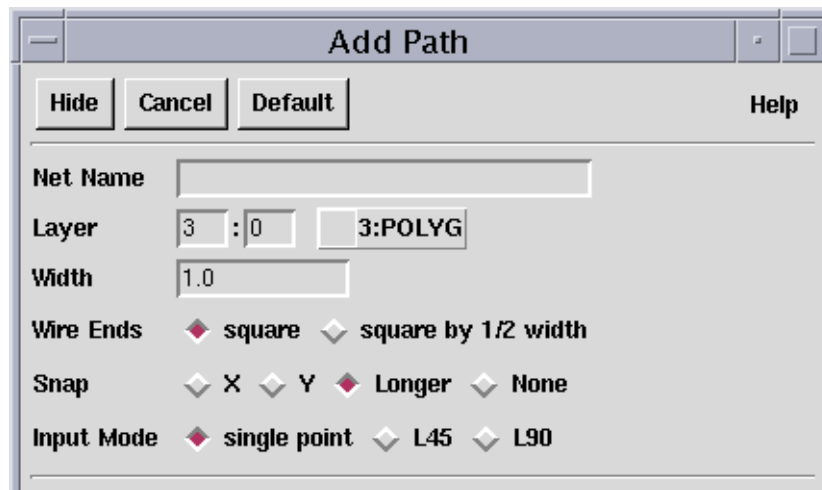
Astro: Create > Path

Jupiter: Astro Floorplanning: Create > Path

Milkyway: Create > Path

Add Path Dialog Box

The menu command opens the following dialog box. Its options are described below.



Net Name

Type the name of the net to which you want the path connected.

Layer

Type the name or number of the layer on which you want the path, or use the Edit Layer Panel to select a layer from the available layers. To display the Edit Layer Panel, use [geLayerPanel](#).

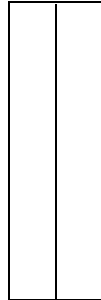
Width

Type the width you want for the path.

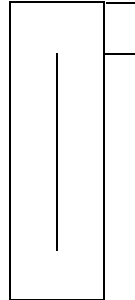
Wire Ends

Select the shape you want, either *square* or *square by 1/2 width*, for the end of the wires.

Wire Ends: square
looks like this:



Wire Ends: square by 1/
2 width looks like this:



Distance to end is 1/2
of the wire width

Input Mode

Select the shape you want for drawing the path.

Option	Draws the portion of the path between two specified points ...
single point	With only one segment of any angle (does not have to be horizontal or vertical).
L45	With a 45-degree segment, then a horizontal or vertical segment.
L90	With a horizontal segment and a vertical segment, using the specified <i>Snap Rule</i> .

Snap

Option	Draws the segments ...
X	First in the horizontal direction, then in the vertical direction.
Y	First in the vertical direction, then in the horizontal direction.

Option	Draws the segments ...
longer	First in the direction (horizontal or vertical) that is of a greater distance (between the two points). For example, if the distance between the first point and the second point is 5 in the horizontal direction and 7 in the vertical direction, your Synopsys tool draws the first segment in the vertical direction and the second segment in the horizontal direction.
None	Without any snap applied.

Select the rule you want to use when the *Snap Shape* is *L90*.

See Also

[geAddWire](#) adds a wire you define by entering points.

geAddPolygon

Adds a polygon you specify by entering points in the design window.

Tools and Menu Commands

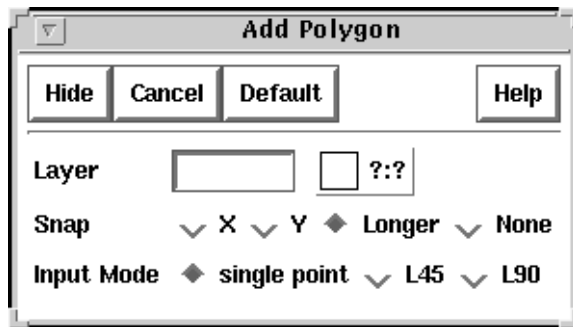
Astro: Create > Polygon

Jupiter: Astro Floorplanning: Create > Polygon

Milkyway: Create > Polygon

Add Polygon Dialog Box

The menu command opens the following dialog box. Its options are described below.



Layer

Type the name or number of the layer on which you want the polygon, or use the Edit Layer Panel to select a layer from the available layers. To display the Edit Layer Panel, use [geLayerPanel](#).

Snap

Option	Draws the segments ...
X	First in the horizontal direction, then in the vertical direction.
Y	First in the vertical direction, then in the horizontal direction.
Longer	First in the direction (horizontal or vertical) that is of a greater distance (between the two points). For example, if the distance between the first point and the second point is 5 in the horizontal direction and 7 in the vertical direction, your Synopsys tool draws the first segment in the vertical direction and the second segment in the horizontal direction.
None	Without any snap applied.

Select the rule you want to use when the *Snap Shape* is *L90*.

Input Mode

Select the shape you want for the polygon.

Option	Draws the portion of the polygon between two specified points ...
single point	With only one segment of any angle (does not have to be horizontal or vertical).
L45	With a 45-degree segment, then a horizontal or vertical segment.
L90	With a horizontal segment and a vertical segment, using the specified <i>Snap Rule</i> .

geAddRectangle

Adds a rectangle you specify by entering diagonal corners.

Tools and Menu Commands

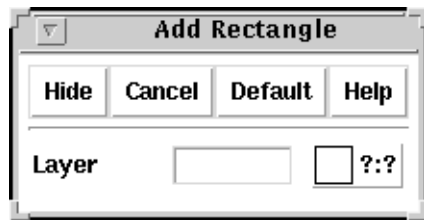
Astro: Create > Rectangle

Jupiter: Astro Floorplanning: Create > Rectangle

Milkyway: Create > Rectangle

Add Rectangle Dialog Box

The menu command opens the following dialog box. Its options are described below.



Layer

Type the name or number of the layer on which you want the rectangle, or use the Edit Layer Panel to select a layer from the available layers. To display the Edit Layer Panel, use [geLayerPanel](#).

geAddRectilinearBoundary

Adds a rectilinear cell boundary that you specify by entering points within the cell. If the cell boundary already exists, it will be overwritten.

Tools and Menu Commands

Astro: Create > Rectilinear Boundary

Jupiter: Astro Floorplanning: Create > Rectilinear Boundary

Milkyway: Create > Rectilinear Boundary

geAddRectilinearBoundaryForChildCell

Adds a rectilinear boundary to the child cell you specify.

Tools and Menu Commands

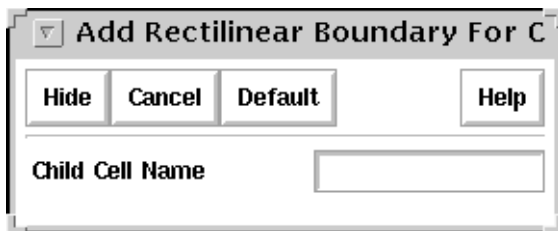
Astro: Create > Rectilinear Bndy For Child Cell

Jupiter: Astro Floorplanning: Create > Rectilinear Bndy For Child Cell

Milkyway: Create > Rectilinear Bndy For Child Cell

Add Rectilinear Boundary for Child Cell Dialog Box

The menu command opens the following dialog box. Its options are described below.



Child Cell Name

Type the name of the child cell you want to add the rectilinear boundary to.

geAddRectPin

Adds a pin at a point you specify.

Tools and Menu Commands

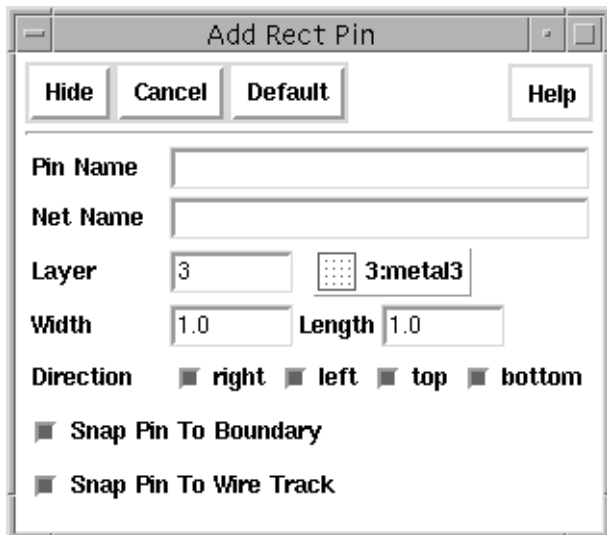
Astro: Create > Pin

Jupiter: Astro Floorplanning: Create > Pin

Milkyway: Create > Pin

Add Rect Pin Dialog Box

The menu command opens the following dialog box. Its options are described below.



Pin Name

Type the name you want to assign to the pin.

Note:

Pin names are limited to 32 characters.

Net Name

Type the name of the net to which you want the pin connected.

Layer

Type the name or number of the layer on which you want the pin, or use the Edit Layer Panel to select a layer from the available layers. To display the Edit Layer Panel, use [geLayerPanel](#).

Width

Type the horizontal dimension you want for the pin.

Length

Type the vertical dimension you want for the pin.

Direction

Select the direction(s) from which you want to permit the router to access the pin.

Option	Lets the router access the pin from ...
right	The right side.
left	The left side.
top	The top.
bottom	The bottom.

Snap Pin To Boundary

Choose this option if you want the pin to be snapped to the nearest cell boundary.

Snap Pin To Wire Track

Choose this option if you want the pin to be snapped to the nearest wire track. If there are no wire tracks in the design, this constraint is ignored.

geAddRoutingBlockage

This command is used to generate geometry on the corresponding metal blockage layer (or route guide layer). It prevents the router from routing through areas with metal blockages (or route guides) in a cell. You may also use it to prevent problems with Xtalk.

Tools and Menu Commands

Astro: Cell > Add Routing Blockage

Jupiter: Cell > Add Routing Blockage

Milkyway: Cell > Add Routing Blockage

Add Routing Blockage Dialog Box

The menu command opens the following dialog box. Its options are described below.

Add Routing Blockage

Library Name

Input Cell Name

Create in FRAM View ☒ Metal Blockage ☐ Route Guide

Route Guide Type ☒ Signal ☐ Automatic Preroutes

Metal Blockage Generation Information

	Source Layer	Oversize Value
Poly		0.000
Metal1		0.000
Metal2		0.000
Metal3		0.000
Metal4		0.000
Metal5		0.000
Metal6		0.000
Metal7		0.000
Metal8		0.000
Metal9		0.000
Metal10		0.000
Metal11		0.000
Metal12		0.000

☐ Trim by Pins ☒ Treat All Blockage as Thin Wire

☐ Delete Existing Blockage

Library Name

Type the name of the library containing the layout cells which you want to add routing blockages.

Input Cell Name

Indicate the layout cell(s) where you want to create routing blockages on its .FRAM view. To indicate one cell, type the name of that cell. To indicate all cells, leave the field empty.

Create In FRAM View

Generate routing blockage on either the “Metal Blockage” layer or the “Route Guide” layer. By default, the routing blockage will be generated on the “Metal Blockage” layers.

Route Guide Type

Selects to generate the routing blockage on the “Route Guide” layer. The type of route guide will be for “Signal” nets (by default) or for “Automatic Preroutes.”

Metal Blockage Generation Information

For any metal layer (Poly, Metal1 ~ Metal 12), you can specify a layer as the source shape and location of the corresponding routing blockage. *geAddRoutingBlockage* will generate the blockage according to the geometry on the specified layers and modify them based on the following options:

- *Oversize Value* - If you want to oversize the source geometry by some value, type in the value in the corresponding metal layer.
- *Trim by Pins* - If you want to cut the source geometry around the pins, select this option.
- *Treat All Blockages as Thin Wire* - By default, the spacing reserved around pins will treat all blockage as thin wires when this button is on. If you want to treat all blockage as fat wires, please turn off this option to reserve a larger space around pins according to the fat table defined in the technology file.

- Delete Existing Blockage - By default, `geAddRoutingBlockage` will add the routing blockage geometry to the specified layer(s) without deleting the existing routing blockage. If you do not want the existing metal blockage (or route guide) geometry, select this button on to delete them.

Note:

`geAddRoutingBlockage` was designed for adding “fake” blockages to .FRAM views of soft-macros. It is not intended for real blockages. .FRAM views should have all the real blockages.

By default, both the power and ground router and the signal router will honor the “Route Guide,” but will ignore “fake” blockages on “metal-blockage layers.” The purpose of a “fake” blockage is to reserve extra spacing around signal routing inside macros to avoid potential crosstalk problems. Because of the extra constraint, the signal router has an option to honor these constraints with (`axSetIntParam “droute” “readMetalBlockageLayer” 1`).

Normally, std-cells are “hard” from GDS-in. If the “source layer” exists in the original GDS, you can simply map the layer to either the `polyBlockage` or `metal1Blockage` layer during GDS-in without using `geAddRoutingBlockage`.

geAddSMPin

Please see `fphAddSMPin` for more information.

geAddText

Adds text at a point you specify.

Tools and Menu Commands

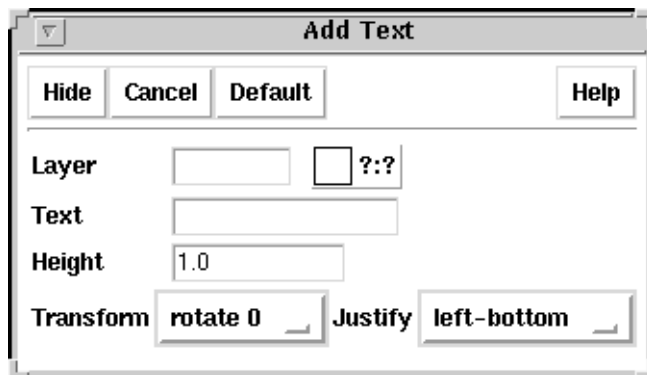
Astro: Create > Text

Jupiter: Astro Floorplanning: Create > Text

Milkyway: Create > Text

Add Text Dialog Box

The menu command opens the following dialog box. Its options are described below.



Layer

Type the name or number of the layer on which you want the text, or use the Edit Layer Panel to select a layer from the available layers. To display the Edit Layer Panel, use [geLayerPanel](#).

Text

Type the text that you want to add.

Height

Type the height you want for the text.

Transform

Select the orientation you want for placement of the text.

Option	Places the text ...
rotate 0	Without changing its orientation.
rotate 90	Rotated counterclockwise 90 degrees.
rotate 180	Rotated counterclockwise 180 degrees.
rotate 270	Rotated counterclockwise 270 degrees.
mirror X	Flipped on the y axis (in the X direction).
mirror Y	Flipped on the x axis (in the Y direction).

Option	Places the text ...
rotate 90 mirror X	Rotated 90 degrees and flipped on the y axis (in the X direction).
rotate 90 mirror Y	Rotated 90 degrees and flipped on the x axis (in the Y direction).

Justify

Select the point in the text line that you want placed at the point you specify.

Option	Places the following point in the text line at the point you specify:
left-bottom	Bottom left corner.
center-bottom	Center of the bottom edge.
right-bottom	Bottom right corner.
left-center	Center of the left edge.
center	Center.
right-center	Center of the right edge.
left-top	Top left corner.
center-top	Center of top edge.
right-top	Top right corner.

geAddWire

Adds a wire you define by entering points.

When you click *Commit*, your Synopsys tool applies the specified layer, width, and wire ends (*Layer 1*, *Width 1*, and *Wire Ends 1* on the dialog box) to the entire portion of the wire (or the portion drawn since the last time you clicked *Commit*), and you can continue entering points. To add a via and change to the alternate layer, width, and wire ends (*Layer 2*, *Width 2*, and *Wire Ends 2* on the dialog box), click *Via*. To terminate the wire, click with the right mouse button.

Note:

While drawing the wire, you can delete previously specified segments and vias using the Delete key.

Tools and Menu Commands

Astro: Create > Wire

Jupiter: Astro Floorplanning: Create > Wire

Milkyway: Create > Wire

Add Wire Dialog Box

The menu command opens the following dialog box. Its options are described below.

Net Name

Type the name of the net to which you want the wire connected.

Layer 1

Type the name or number of the layer on which you want the current portion of the wire, or use the Edit Layer Panel to select a layer from the available layers. To display the Edit Layer Panel, click on the buttons to the right of the Layer 1 fields (shown as 1:metal1 above).

Width 1

Type the width you want for the current portion of the wire.

Wire Ends 1

Select the type of wire ends you want for the current portion of the wire.

Option	Finishes the ends of the wire segments
square	At the specified points.
square by 1/2 width	Half the wire width beyond the specified points.

Layer 2

Type the name or number of the layer on which you want the portion of the wire to be drawn following placement of the next via, or use the Edit Layer Panel to select a layer from the available layers. To display the Edit Layer Panel, click on the buttons to the right of the Layer 2 fields (shown as 2:metal2 above).

Width 2

Type the width you want for the portion of the wire to be drawn following placement of the next via.

Wire Ends 2

Select the type of wire ends you want for the portion of the wire to be drawn following placement of the next via.

Option	Finishes the ends of the wire segments
square	At the specified points.
square by 1/2 width	Half the wire width beyond the specified points.

Snap

Select the snap rule you want to use when the *Input Mode* is L90.

Option	Draws the segments
X	First in the horizontal direction, then in the vertical direction.
Y	First in the vertical direction, then in the horizontal direction.
Longer	First in the direction (horizontal or vertical) that is of a greater distance (between the two points). For example, if the distance between the first point and the second point is 5 in the horizontal direction and 7 in the vertical direction, your Synopsys tool draws the first segment in the vertical direction and the second segment in the horizontal direction.
None	Without any snap applied.

Select the rule you want to use when the *Snap Shape* is L90.

Input Mode

Select the shape you want for drawing the wire.

Option	Draws the portion of the wire between two specified points
L90	With a horizontal segment and a vertical segment, using the specified <i>Snap Rule</i> .
single point	With only one segment of any angle (does not have to be horizontal or vertical).

Mark as

Select the route type of the new wire segment.

Option	Mark as
Signal	Depends on the "and as" setting
Clock	Depends on "and as" setting

Option	Mark as
P/G	Depends on the "and as" setting
Bus	Bus route type
Shield	Shield (fix) route type

and as

These options work with the Mark as options and are listed below.

Option	and as
User	User Entered route type
Detailed	Signal Route route type
Ring	P/G Ring or "CLK Ring"
Strap	Clk Strap or P/G Strap route type
Tie-off	Signal Route route type
Std-cell pin conn	P/G Standard Cell Pin Connection route type
Macro/ IO pin conn	P/G Macro or IO Pin Connectivity route type

Skip wire shorter than half-width

Elect to skip wires that are shorter than half the specified wire width.

Automatic pushing

This feature is no longer supported.

See Also

[geAddPath](#) adds a path you specify by entering points.

geAllError

Displays all errors of all types in the loaded error cell. The error cell contains the results of running DRC or LVS.

Tools and Menu Commands

Astro: Verify > Set Error Type – Show Errors of All Types

Prerequisites

Open the cell and load the error cell. If you used the default error cell name when running DRC or LVS, the Synopsys tool automatically loads the error cell. To manually load an error cell, use [geLoadErrorCell](#).

geBackupCell

Saves the current cell to disk, then increments the version number, creating a new version of the cell in virtual memory.

Tools and Menu Commands

Astro: Cell > Backup

Jupiter: Cell > Backup

Milkyway: Cell > Backup

geCheckMinGrid

Checks to be sure that the edges of any object of the specified type in the specified cell are placed on the grid.

Note:

The grid is defined in the technology file.

Tools and Menu Commands

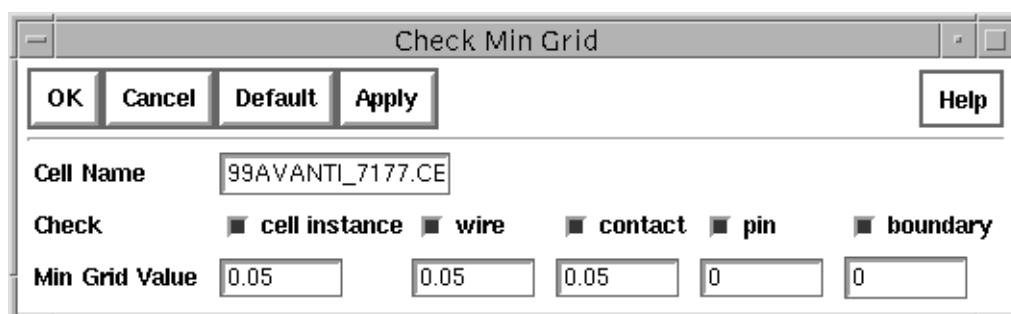
Astro: Verify > Check Min Grid

Prerequisites

Open the library containing the cell you want to check; you do not need to open the cell. To check cell instances, perform placement on the cell. To check paths and contacts, perform routing on the cell.

Check Min Grid Dialog Box

The menu command opens the following dialog box. Its options are described below.



Cell Name

Type the name of the cell you want to check.

Check

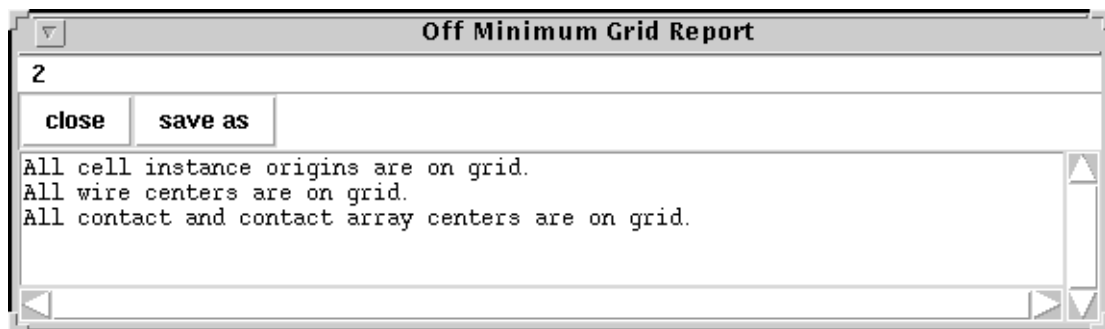
Select the type of object for which you want the tool to check.

Option	Checks to be sure ...
cell instance	the left edge of each cell instance is on the grid.
wire	the left edge of each wire is on the grid.
contact	the left edge of each contact is on the grid.
pin	all edges of each pin are on the grid.
boundary	all edges of each boundary are on the grid.

Min Grid Value

Type the minimum grid value for each object type you want to check. The default is 0.05.

Off Minimum Grid Report

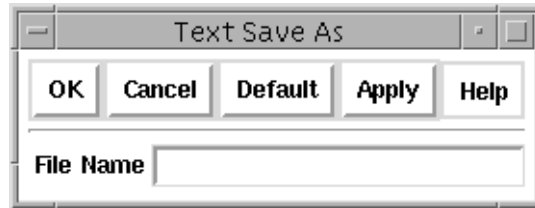


File menu

You can use the commands on this menu to save the text in the window to a file, load another file into the window, or close the window.

- **Save** saves the text in the window to a file with a name you assign using *Set File Name*.

- *Save As* displays the Text Save As dialog box.



To save the text in the window, type a *File Name* and click *OK*. The tool saves the file to the name you type.

- *Load* loads an existing file into the window.
- *Set File Name* assigns a name to the text in the window. You can then save the text to a file using *Save*.
- *Close* closes the window.

geCheckPinsOnUserGrid

Checks whether or not the top level pins and pin instances of certain cell instances are located on user defined grids. The default value of the pitch of the user defined grid for each layer is picked from the technical file.

Note:

Depending on the number of metal layers in the design, the required number of metal layers in the dialog box will be active.

Error cells will be generated for each kind of errors, such as an error cell for top level pins, an error cell for soft macro pins etc. The types of errors are classified by the types of pins. The report will provide the name of the error file.

Tool

Milkyway

Check Pins Are On User Grid Dialog Box

The menu command opens the following dialog box. Its options are described below.

Check Pins Are On User Grid

OK Cancel Default Apply Help

Check The Pins Of ☒ Soft Macros

☐ Hard Macros

☐ Standard Cells

☐ Pad Cells

☐ Top Level

Layer	Start Value	Min Grid Value
M1	1.550	1.900
M2	0.250	1.900
M3	1.550	1.900
M4	0.0	0.0
M5	0.0	0.0
M6	0.0	0.0
M7	0.0	0.0
M8	0.0	0.0
M9	0.0	0.0
M10	0.0	0.0
M11	0.0	0.0
M12	0.0	0.0

☐ Dump Report To File

Check The Pins Of

- Soft Macros - Select this option to check whether or not the pins of soft macros are on user defined grids.

- Hard Macros - Select this option to check whether or not the pins of hard macros are on user defined grids.
- Standard Cells - Select this option to check whether or not the pins of standard cells are on user defined grids.
- Pad Cells - Select this option to check whether or not the pins of pad cells are on user defined grids.
- Top Level - Select this option to check whether or not the top level pins are on user defined grids.

Layer

- Start Value - This is the starting coordinate of the user defined track. When the preferred direction of the track is horizontal, this is the Y-coordinate. Otherwise, it is the X-coordinate. The unit is in microns.
- Min Grid Value - This is the pitch of the user defined grid. The unit is in microns.

Dump Report To File

Choose this option when you want the report to be dumped to a text file. The default is to dump the report to the message window. Enter the name of the report file as well.

geCloseLib

Closes the current library. If any cells changed since they were opened, your Synopsys tool displays the Save Cells window to let you select the cells you want to save.

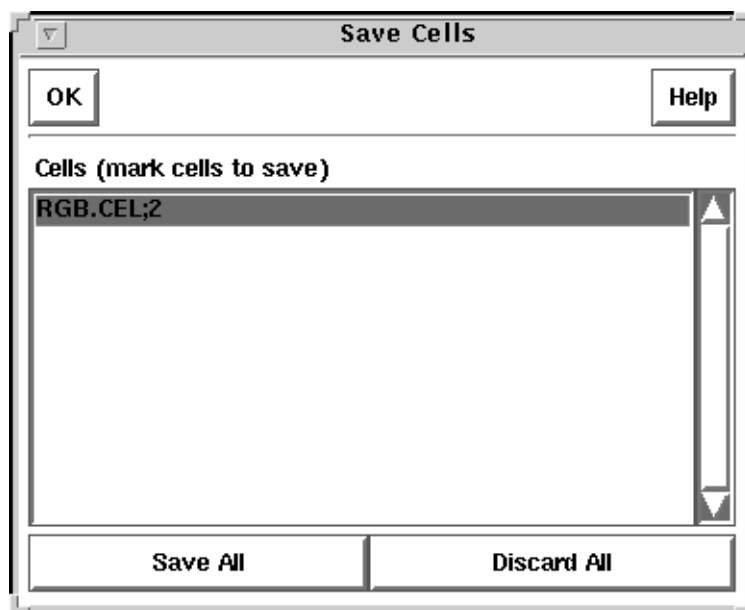
Tools and Menu Commands

Astro: Library > Close

Jupiter: Library > Close

Milkyway: Library > Close

Save Cells Window



Cells (Click to save)

Lists cells that changed. Select the cell(s) you want to save before closing the library.

geCloseWindow

Closes the current window and performs any other actions you specify. These can include any of the following:

- You can select to close all related windows. A related window is any window containing a cell that meets the following criteria:
 - The cell is open with write permission in the current window.
 - The cell is open with write permission in the related window.

To find out if two windows are related, place the cursor in one of the windows and see if it is displayed in the other window. If the cursor is displayed in both windows, the windows are related.

- If a cell open with write permission in the current window will be open in another window after the command executes, you can select one of the following actions:
 - Save the cell.
 - Defer the choice of saving or discarding the cell.

Either of these actions keeps the cell in virtual memory.

- If a cell open with write permission in the current window will not be open in another window after the command executes, you can select one of the following actions:
 - Close the cell. This action saves the cell and removes it from virtual memory.
 - Discard the cell. This action removes the cell from virtual memory.

- Defer the choice of saving or discarding the cell. This action keeps the cell in virtual memory, and you can select to save or discard it when you exit your Synopsys tool (or reopen the cell).

The dialog box lists the names of cells for which you need to select an action. First indicate whether you want to *Close all related windows*. Then click each cell name until the action you want for that cell appears to the right of its name (in parentheses), or use the *Save/Close All*, *Discard All*, or *Defer All* button to set the action the same for all cells. Click *OK* to perform the actions.

Tools and Menu Commands

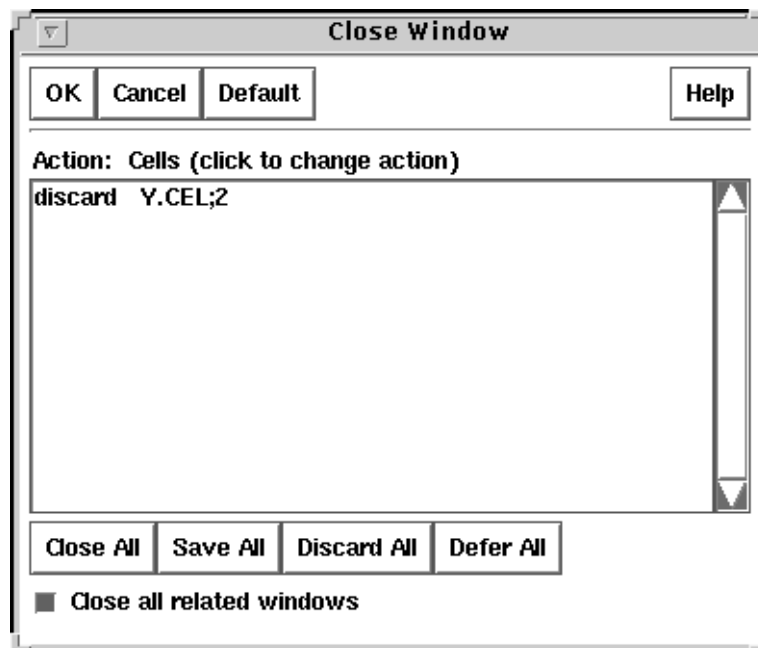
Astro: Cell > Close

Jupiter: Cell > Close

Milkyway: Cell > Close

Close Window Dialog Box

The menu command opens the following dialog box. Its options are described below.



Action:Cells (click to change action)

Lists the names of all cells for which you need to select an action. Click each name to select the action you want to perform for that cell, or click the *Close All*, *Save All*, *Discard All*, or *Defer All* button to set the action the same for all cells.

- *save* saves the cell and keeps it in virtual memory.
- *close* saves the cell and removes it from virtual memory.
- *discard* discards the cell and removes it from virtual memory.
- *defer* defers the choice of saving or discarding the cell and keeps it in virtual memory.

Close All

Select to set the action to *close* for all cells.

Save All

Select to set the action to *save* for all cells.

Discard All

Select to set the action to *discard* for all cells (except those cells that will be open in another window after the command executes).

Defer All

Select to set the action to *defer* for all cells.

Close all related windows

Select to close all related windows.

geConfirmCloseLib

Closes an open design library. A dialog box will appear asking for your confirmation.

The functionality is the same as [geCloseLib](#), except for your confirmation. *geconfirmCloseLib* is added in the menu so you don't invoke *gecloselib* accidentally.

Tools and Menu Commands

Astro: Library > Close

Jupiter: Library > Close

Milkyway: Library > Close

Syntax

```
geConfirmCloseLib
```

geConvertInstPinToPin

Converts cell instance pin(s) into top level pin(s) for specified net name(s) in the selected object set (cells).

Note:

If the specified net name does not exist in the top cell, nothing will be converted. That is, no additional port (net) will be created in the top cell and the net list will not be changed.

If geConvertInstPinToPin is used to convert power and ground pin instance(s) into the top level, then you must invoke the aprPGConnect command before performing geConvertInstPinToPin.

After invoking geConvertInstPinToPin for a cell, the corresponding .FRAM view of the cell must be updated by the [geNewMakeMacro](#) command for consistency.

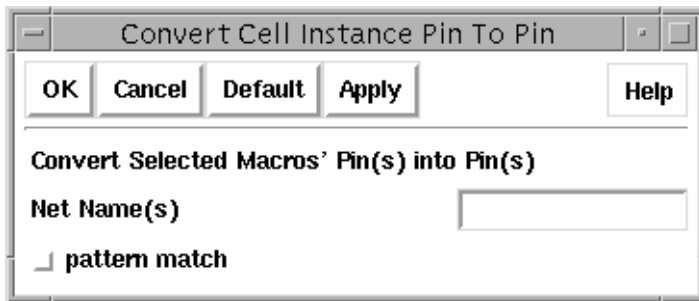
Tools and Menu Commands

Astro: Modify > Convert Instance Pin to Pin

Milkyway: Modify > Convert Instance Pin to Pin

Convert Pin to Pin Dialog Box

The menu command opens the following dialog box. Its options are described below.



Convert Selected Macros' Pin(s) into Pin(s)

For those instance cells in the selected object set, *geConvertInstPinToPin* will convert the instance pin(s) for the nets specified in the field of "Net Name(s)" into the top level pin(s).

Net Name(s)

Type the name of the net(s) where you want to convert *instance pins* to *top level pins*, or select "pattern match" and type a pattern to match the names of the multiple nets.

geConvertWireToPin

Converts wire(s) to pin(s) for either all nets, or, specified net(s) in the selected object set.

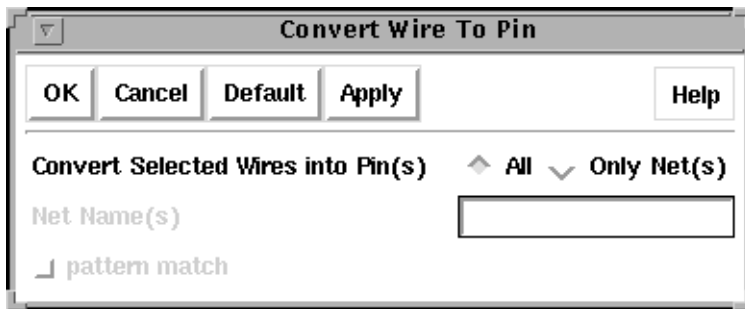
Tools and Menu Commands

Astro: Modify > Convert Wire to Pin

Milkyway: Modify > Convert Wire to Pin

Convert Wire to Pin Dialog Box

The menu command opens the following dialog box. Its options are described below.



All

Select to convert wires to pins for all nets in the selected object set. This is the default. When selected, the *Net Name(s)* and *pattern match* fields are inoperable.

Only Net(s)

Select to convert wires to pins for specific nets in the selected object set. When selected, the *Net Name(s)* and *pattern match* fields become active. Type the name of the net(s) for which you want to convert wires to pins, or select *pattern match* and type a pattern to match the names of multiple nets.

geCopy

Copies and places specified object(s) at a specified location.

Tools and Menu Commands

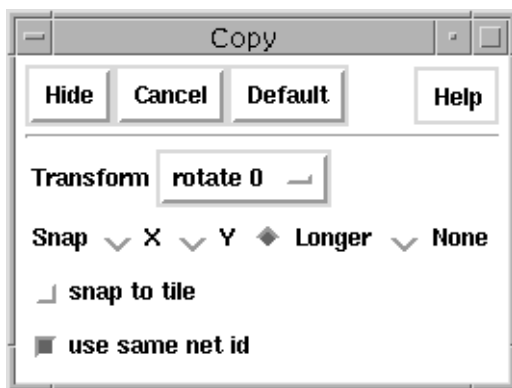
Astro: Modify > Copy

Jupiter: Astro Floorplanning: Edit > Copy

Milkyway: Modify > Copy

Copy Dialog Box

The menu command opens the following dialog box. Its options are described below.



Transform

Select the orientation you want for placement of the copied object(s).

Table 36-9

Option	Places the object(s) ...
rotate 0	Without changing their orientation.
rotate 90	Rotated counterclockwise 90 degrees.
rotate 180	Rotated counterclockwise 180 degrees.
rotate 270	Rotated counterclockwise 270 degrees.
mirror X	Flipped on the y axis (in the X direction).
mirror Y	Flipped on the x axis (in the Y direction).

Table 36-9

Option	Places the object(s) ...
rotate 90 mirror X	Rotated 90 degrees and flipped on the y axis (in the X direction).
rotate 90 mirror Y	Rotated 90 degrees and flipped on the x axis (in the Y direction).

Direction

Select the direction to move the copy of the object for placement.

Table 36-10

Option	Lets you move the object ...
X	Horizontally from the original object.
Y	Vertically from the original object.
Longer	In the direction you move the cursor the farthest. For example, if you move the cursor farther horizontally than vertically, the object moves in the horizontal direction.
None	In any direction.

snap to tile

Select to snap objects to a unit tile.

use same net id

Select if you want the copy to be on the same net as the original object.

geCopyCell

Serves two purposes. Copies a specified cell to a new cell, and places the copy within the current library. It also copies a specified cell from one library to another. To do the latter, open the target library and specify the source library within the menu.

Tools and Menu Commands

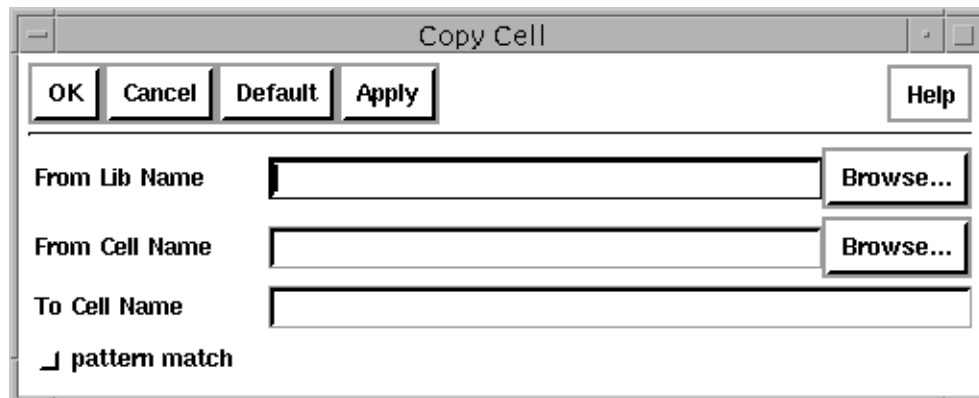
Astro: Cell > Copy

Jupiter: Cell > Copy

Milkyway: Cell > Copy

Copy Cell Dialog Box

The menu command opens the following dialog box. Its options are described below.



From Lib Name

Type the name of the library containing the cell you want to copy, or select *Browse* to see the list of library names to select from.

From Cell Name

Type the name of the cell you want to copy, or select *Browse* to see the list of cell names available in the *From Lib Name* library to select from.

To Cell Name

Type the name you want to assign to the copy of the cell.

pattern match

Select if you want to specify a pattern to match the names of multiple cells.

geCreateCell

Creates an empty cell.

Tools and Menu Commands

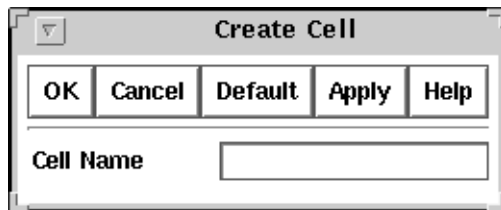
Astro: Cell > Create

Jupiter: Astro Floorplanning: Cell > Create

Milkyway: Cell > Create

Create Cell Dialog Box

The menu command opens the following dialog box. Its options are described below.



Cell Name

Type the name you want to assign the new cell.

geCreateConnView

Generates the connectivity views for hard macro cells to be used in *P/G Rail Analysis*, when *Consider Cell P/G Pin* is selected for the *Macro P/G Modeling*. It also generates the connectivity views for I/O pad cells to be used in *P/G Rail Analysis* taking into account the effects of their internal power and ground structures on the voltage drops.

Note:

At the moment, .CON views are to be used exclusively in Astro-Rail.

Tools and Menu Commands

Astro: Data Prep > Cell Library > Create Connectivity View

Create Connectivity View Dialog Box

The menu command opens the following dialog box. Its options are described below.

Create Connectivity View

OK

Cancel

Default

Apply

Help

Library Name

Cell Name

Hierarchical Extraction

On/Off

Unknown Cell Type

As Macro

Ignore

Delete SMASH View After CONN View Generation

Power/Ground Port Only

Hierarchical Power/Ground Net Information

Net Name	Net Type	Port Pattern(s)
	power	ground pattern match
	power	ground pattern match
	power	ground pattern match
	power	ground pattern match

Information From File

Extract Pin by Text

Check Hierarchy

Identify Pin By Pin Text

Extract Connectivity

through

polyCont

via1

via2

via3

via4

via5

via6

via7

via8

via9

via10

via11

text fall through

double fall through

Poly Text		Metal1 Text	
Metal2 Text		Metal3 Text	
Metal4 Text		Metal5 Text	
Metal6 Text		Metal7 Text	
Metal8 Text		Metal9 Text	
Metal10 Text		Metal11 Text	
Metal12 Text		Fall Through Text	

Library Name

Type the name of the Library where you wish to generate the views.

Cell Name

Type the name of the cell for which you wish to generate views.

Hierarchical Extraction

When the input cell is a hierarchical cell, turn the option “Hierarchical Extraction” on so that this hierarchical flow does not need the command “cmSmash” to run before running this command. While this option is on, you must provide the top level power and ground net naming information in the “Hierarchical Power/Ground Net Information” table below. When this option is off and when the input cell is hierarchical, the program will look at the .SMASH view of the given cell and use the flattened approach to create a .CON view.

Unknown Cell Type

Any flattened cell with an unknown cell type, will be skipped by this command. That is, no .CON view will be generated for unknown flattened cells. As to a hierarchical cell, this option is to encourage you to mark each child cell with power and ground objects of the correct cell type and to mark those child cells without power and ground objects of “unknown” cell types (with the command “cmMarkCelltype”). In this way, the program will not go through each geometry in an unknown cell or do checks to see whether or not the geometry is a power and ground object. Therefore, the performance can be speed up. In another words, from the performance aspect, it is important to mark the cell type of each child cell carefully.

An extreme example is, when the whole hierarchical child cells are of an unknown cell type, but all of them are with power and ground objects, you can select the option “As Macro” to treat all unknown cells as macro cells.

Note:

This is not a good discipline for data preparation, but is provided as a convenience.

Delete SMASH View After CONN View Generation

This option deletes the .SMASH view of the input cell after the resulting .CON view is created.

Power/Ground Port Only

To improve performance, select this option to extract only the power and ground pin and via geometries in the .CON view. In a hierarchical cell, this option is always selected on.

Hierarchical Power/Ground Net Information

Currently, this command can extract four different power and ground nets for the input cell. However, you must type in the net name, and select the net type for each power and ground net to be extracted. When the power and ground naming is different between the given library and the reference libraries, you need to enter all possible port name (text) patterns into the “Port Pattern(s)” field and/or turn on the pattern match. This information is necessary for a hierarchical cell but is optional for a flattened cell. When you enter the information for a flattened cell, it will speed up the performance.

- Net Name - Please enter the power and ground net name that you want to name the top power and ground net under the “Net Name” column.
- Net Type - Select the correct net type (power or ground) for each net.
- Port Pattern(s) - The “Port Pattern(s)” column is used when the naming of the power and ground ports is different from the top net name; or when the naming in the reference libraries is

inconsistent from the design library. All port names that are under the top net name, should be entered into this field. Therefore, the port name and the net names will be consistent in the .CON view.

Extract Pin by Text

Used for identifying pins by pin text for connectivity extraction.

Check Hierarchy

Before or after running the geCreateConnView command, when you want to know the hierarchical cell relationship and the current.CON view generation status for all whole child cells, press this button and the result will be located in the log file.

For some hierarchical gdsIn data, when no text exists in a child cell and a pin needs to be identified by text, the program will generate an empty .CON view for the child cell.

When several power and ground geometries exists in this kind of cell, the cell is called a PG_BRIDGE_CELL. That is, a power and ground geometry in a PG_BRIDGE_CELL will be identified through the geometry touching the power and ground geometry in adjacent cells.

Currently, for performance consideration, only check the first level geometry of a PG_BRIDGE_CELL to see whether or not there is a power and ground bridge connected to the adjacent cells. When you want all level geometries touching, check with the adjacent cells of some hierarchical PG_BRIDGE_CELL. You may need to provide a .SMASH view for such a non-text hierarchical PG_BRIDGE_CELL. In the log file, the status report will show “(may need a .SMASH view)” for a non-text hierarchical gdsIn cell.

Identify Pin By Pin Text

For *gdsIn* data, select this option to identify pins by text(s), text fall-through, or double fall-through in the same manner as the [auExtractBlockagePinVia](#) command. Default is set on with *Extract Connectivity* and *Extract Via*.

Extract Connectivity

Select if you want Milkyway to trace the connectivity of a specified multiple-layer pin and extract the entire pin. If the layer numbers of texts for pins are identical to the corresponding metal layer numbers, you can leave the fields for text layer numbers blank.

through

The options are as follows:

- polyCont
- via1
- via2
- via3
- via4
- via5
- via6
- via7
- via8
- via9
- via10
- via11

The connectivity can be extracted through any specified via(s). The default is to extract connectivity through via1 to via11 (metal1 - metal12) when “Extract Connectivity” is selected.

text fall through

Select and specify the *Fall Through Text* layer if you want Milkyway to look at the layer(s) beneath the origin of each text label to identify the geometry annotated by the text. If more than one geometry exists beneath the origin of the text label, Milkyway selects the geometry to associate with the label according to the following priorities, from highest to lowest:

- Fall Through Text
- Metal12
- Metal11
- Metal10
- Metal9
- Metal8
- Metal7
- Metal6
- Metal5
- Metal4
- Metal3
- Metal2
- Metal1
- Poly Text

For example, if one of the geometries is *metal3*, Milkyway selects this geometry. If not, Milkyway looks for *metal2*, then *metal1*, then poly.

When the text layers match the layers for the geometries they identify, deselect *text fall through*. You do not need to specify the text layers associated with the interconnection metals. When text layers are not specified (and *text fall through* is not selected), Milkyway assumes they match the geometry layers.

double fall through

Select and specify the *Fall Through Text* layer if you want Milkyway to look at the layer(s) beneath the origin of each text label on a *metal2* geometry to identify a geometry (in addition to the *metal2* geometry) annotated by the text. If more than one geometry (except the *metal2* geometry) exists beneath the origin of the text label, Milkyway selects the second geometry to associate with the label according to the following priorities, from highest to lowest:

- Poly Text
- Metal1
- Metal2
- Metal3
- Metal4
- Metal5
- Metal6
- Metal7
- Metal8

- Metal9
- Metal10
- Metal11
- Metal12
- Fall Through Text

For example, if one of the geometries is *metal1*, Milkyway selects this geometry. If not, Milkyway looks for poly.

Poly Text

Type the name or number of the layer used for the text identifying poly geometries (or leave blank if the text layer matches the geometry layer).

Metal# Text

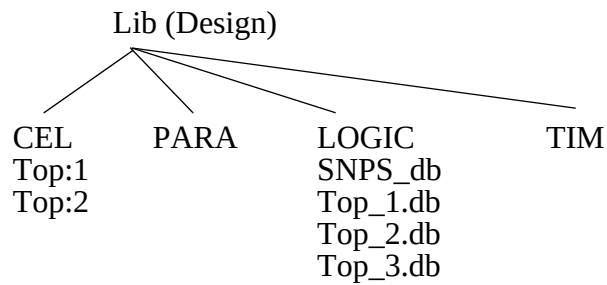
Type the name or number of the layer used for the text identifying metal# geometries (or leave blank if the text layer matches the geometry layer).

Fall Through Text

Type the name or number of the layer used for text if you select *text fall through* or *double fall through*.

geCreateLogicView

Use the geCreateLogicView command to create a .LOGIC view for the design library. The .LOGIC views might have one .db for a .CEL view cell and multiple versions of the .db. However, each version can only have one .db. You can put a .db into the .LOGIC view by specifying parameters in the Create Logic View dialog box.



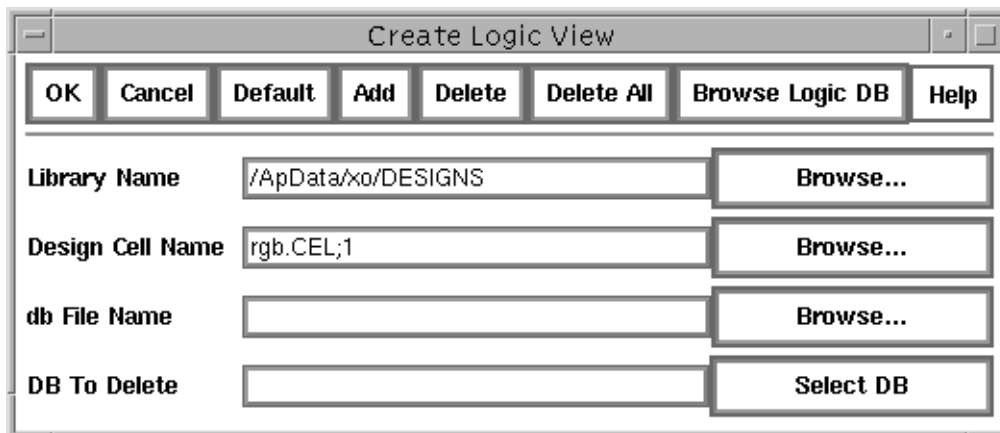
Tools and Menu Commands

Astro

Milkyway

Create Logic View Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Enter the name of the library you want to open. Enter “design” to open the Synopsys “design” library.

This command does not actually open libraries or cells. This is the library that contains the design cell. After completion, the library is closed.

Design Cell Name

Type the name of the cell you want to open.

This is the design cell to which the .db file is attached. After completion, this cell is closed if it was not already opened.

db File Name

Type the name of the database that you want to add to the design cell. Click Add.

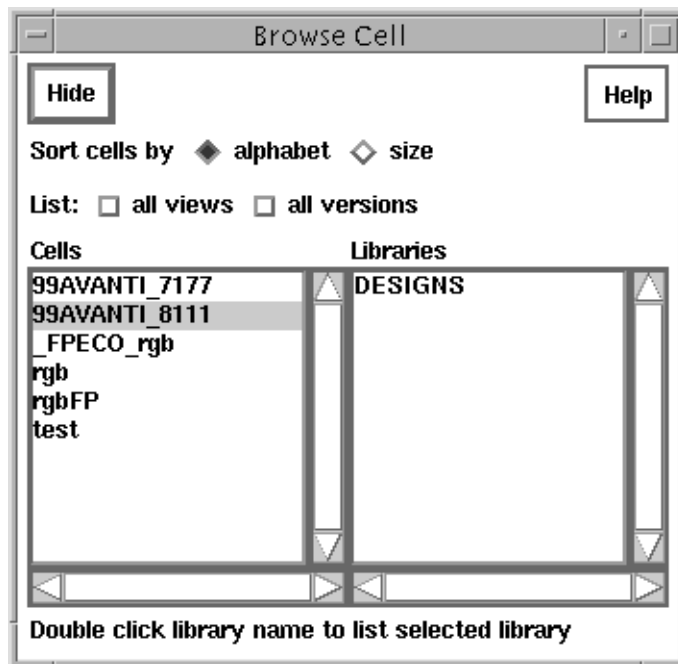
DB To Delete

Type the name of the database you want to delete. Click Delete.

Browse

Select to invoke the Browse dialog boxes.

For example, when you click Browse next to the Design Cell Name box, the Browse Cell dialog box appears, as shown below.



geCreateModelView

This command has been replaced by the [gePrepLibs](#) command.

geCurrentError

Displays the current error in the loaded error cell. The error cell contains the results of running DRC or LVS.

Tools and Menu Commands

Astro: Verify > Set Error Type – Show Errors of Current Type

Prerequisites

Open the cell and load the error cell. If you used the default error cell name when running DRC or LVS, the tool automatically loads the error cell. To manually load an error cell, use the [geLoadErrorCell](#) command.

geCurrentErrorType

Displays all errors of the current type in the loaded error cell. The error cell contains the results of running DRC or LVS.

Tools and Menu Commands

Astro: Verify > Set Error Type – Show Errors of Current Type

Prerequisites

Open the cell open and load the error cell. If you used the default error cell name when running DRC or LVS, the tool automatically loads the error cell. To manually load an error cell, use [geLoadErrorCell](#).

geDefineBindKey

See [defineBindKey](#).

geDelete

Deletes the currently selected object(s).

Tools and Menu Commands

Astro: Modify > Delet

Jupiter: Edit > Delete

Milkyway: Modify > Delet

geDeleteCell

Deletes the specified cell(s) in the open library.

Tools and Menu Commands

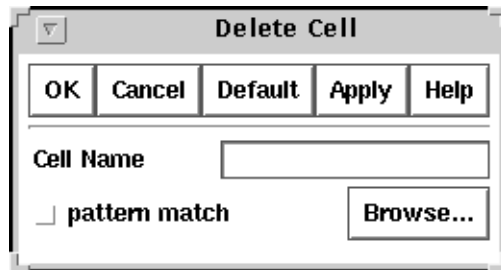
Astro: Cell > Delete

Jupiter: Cell > Delete

Milkyway: Cell > Delete

Delete Cell Dialog Box

The menu command opens the following dialog box. Its options are described below.



Cell Name

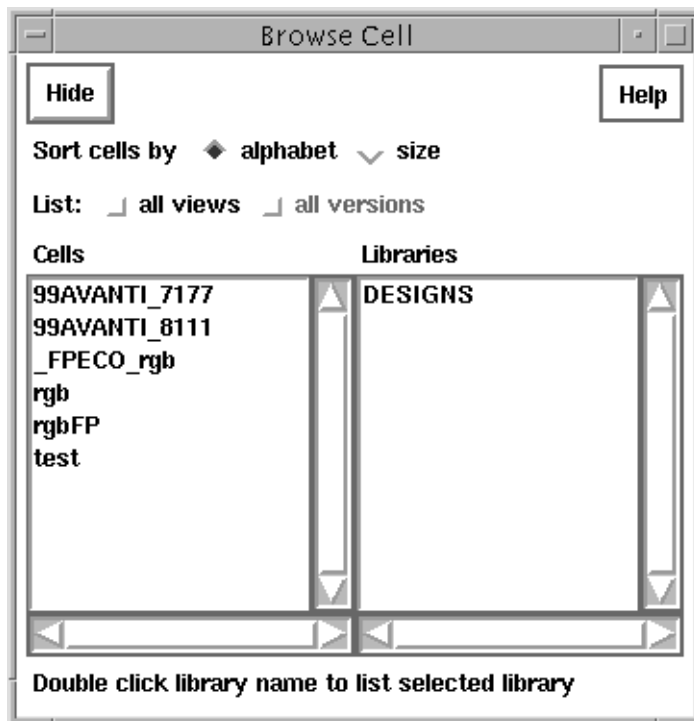
Type the name of the cell you want to delete, or select *pattern match* and type a pattern to match the names of multiple cells.

pattern match

Select if you want to specify a pattern to match the names of multiple cells to delete.

Browse

Click to display the Browse Cell window from which you can choose from any cell in the libraries. You can also sort the cells by selecting either *alphabet* or *size*. Your choice will allow you to either select sorted cells alphabetically, or by file size.



geDeleteRuler

Deletes a ruler display you specify by clicking in the graphics window.

Note:

You create a ruler display, using the [geRuler](#) command.

Tools and Menu Commands

Astro: Query > Ruler – Remove Ruler

Jupiter: Query > Ruler – Remove

Milkyway: Query > Ruler – Remove Ruler

See Also

[geRuler](#) – Provides a ruler for measuring distances in your design.

geDeselectAll

Deselects all selected objects in the current window.

Tools and Menu Commands

Astro: Select > Deselect – Deselect All

Jupiter: Select > Deselect – All

Milkyway: Select > Deselect – Deselect All

geDeselectAllObject

Deselects all previously selected objects.

Syntax

```
geDeSelectAllObject windowNumber
```

where the arguments are as follows:

windowNumber

Number of the layout window from which you want to delete selected objects

Valid values:

Any positive integer that corresponds to an open window

See Also

[geDeselectObject](#)
[geSelectObject](#)

geDeselectObject

Deselects one of the previously selected objects.

Syntax

```
geDeselectObject windowNumber objectId
```

where the arguments are as follows:

windowNumber

Number of the window to which the object information applies

Valid values:

Any positive integer corresponding to an open window

objectId

Database ID of any object you want to subtract from the selected set of windows

Valid values:

Object ID of any object in an open window

See Also

[geDeselectAllObject](#)
[geSelectObject](#)

geErrorBrowser

This command replaces [geLoadErrorCell](#).

Used for design verification. The browser displays all errors stored in an error cell; in the graphic sorted tree mode, offers dynamic interaction between error objects and physical layouts, and helps to analyze and resolve design errors more efficiently.

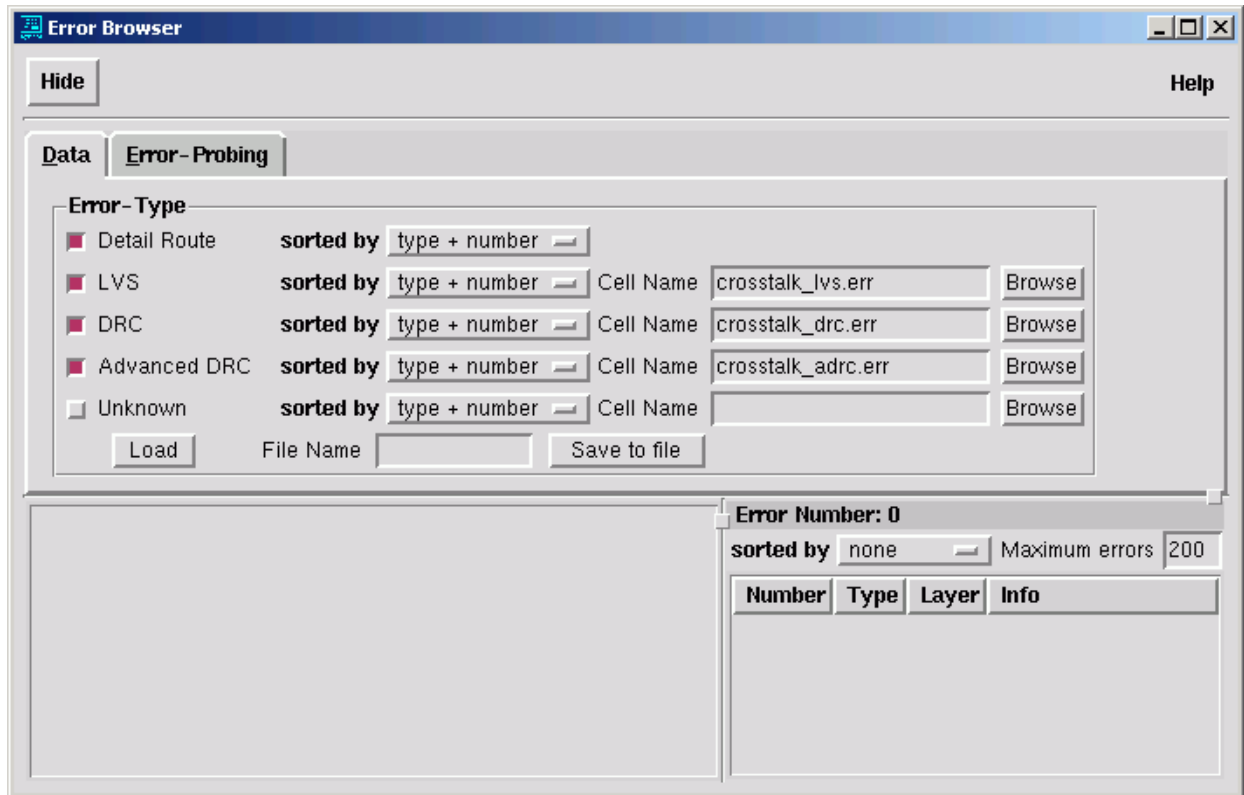
This is a static dialog box, meaning it can be nested with other dialog boxes. The error tree is automatically reloaded when the new design rule checking is finished.

Tools and Menu Commands

Astro: Verify > Error Browser

Error Browser Dialog Box

The menu command opens the following dialog box. Its options are described below.



Data Panel

Set the following options:

Error-Type

Choose the type of errors to be displayed in the error tree pane (lower-left). Select from the following:

- Detail Route – Displays the errors produced by the axgRouterVerify command.
- LVS – Displays the errors produced by the geNewLVS command.
- DRC – Displays the errors produced by the geNewDRC command.

- Advanced DRC – Displays the errors produced by the geAdvDRC command.
- Unknown – Displays the errors produced by other tools.

sorted by

In the list box, select from the following options:

- type + number – Sorts the error by type first. When the type is the same, then it sorts by the error number.
- layer + number – Sorts the error by layer first. When the layer is the same, then it sorts by the error number.
- type + name – Sorts the error by type first. When the type is the same, then it sorts by the error name.
- layer + name – Sorts the error by layer first. When the layer is the same, then it sorts by the error name.

Cell Name

For any of the error types, enter the name of the error cell for which you want information displayed.

Browse – Click to list and select your error cells.

File Name

Enter the file name in which to save the error results.

Save to file

Click to save the error results to the named file.

Load

Click to load the error tree in the pane located at the lower left. This displays the errors stored in the named error cells in a tree-style graphic,

Error Object Pane

The number of errors are reported at the top of the pane (next to Error Number). Set the following options:

sorted by

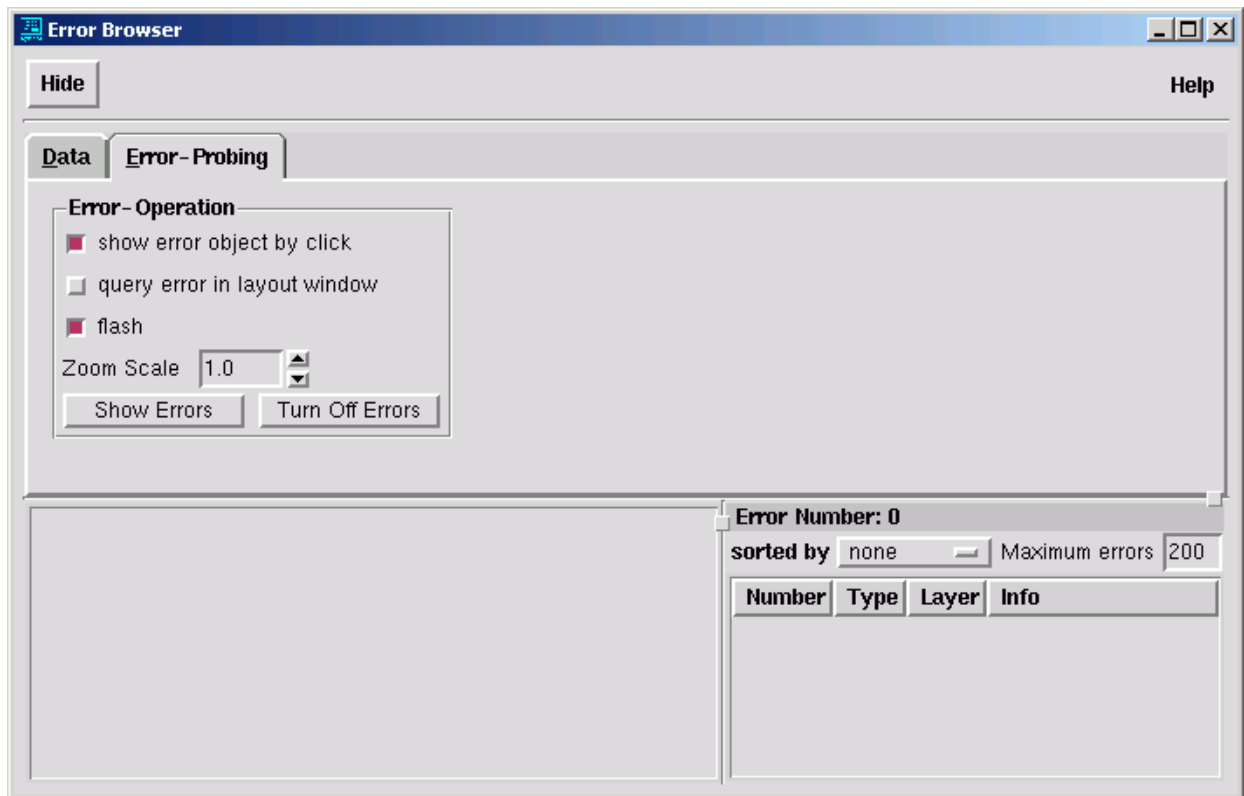
In the list box, select a sorting method for the error objects (none, type, layer, info). The default is none, which means the errors are listed in numerical order.

Maximum errors

Enter the maximum number of error objects to be listed.

Error-Probing Panel

Clicking the Error-Probing tab opens the following page.



Set the following options:

Error-Operation

Select from the following options:

- show error object by click – Selecting any error object from the list in the error object pane, highlights that error in the layout window.
- query error in layout window – Selecting any highlighted error object in the graphics window queries this error object and prints information in the message area of the Astro window.
- flash – Flashes the highlighted errors in the graphics window.
- Zoom Scale – Enter a number for zooming the layout window.
- Show Errors – Click to enable the highlighting of errors in the graphics window.
- Turn Off Errors – Click to turn off the highlighting of errors in the graphics window.

geFirstError

Displays the first error of the current type.

Tools and Menu Commands

Astro: Verify > Show Errors – Show First Error

Prerequisites

Open the cell open and load the error cell. If you used the default error cell name when running DRC or LVS, the Synopsys tool automatically loads the error cell. To manually load an error cell, use [geLoadErrorCell](#).

Set the error types you want to display using [geSetErrorType](#), and set an error type current using [geFirstErrorType](#) and [geNextErrorType](#).

geFirstErrorType

Makes the first error type current.

Tools and Menu Commands

Astro: Verify > Set Error Type – Set First Error Type

Prerequisites

Open the cell open and load the error cell. If you used the default error cell name when running DRC or LVS, the Synopsys tool automatically loads the error cell. To manually load an error cell, use [geLoadErrorCell](#).

Set the error types you want to display using [geSetErrorType](#).

geFit

Fits the entire design in the graphics window. Use this command after panning or zooming. You can also use the icons on the left side of the design window to fit the design in the graphics window.

Tools and Menu Commands

Astro: Views > Fit

Jupiter: Views > Fit

Milkyway: Views > Fit

geFitBorder

Fits the entire design plus a border equaling 10 percent of the design size in the graphics window. Use this command after panning or zooming. You can also use the icons on the left side of the design window to fit the design in the graphics window.

Tools and Menu Commands

Astro: Views > Fit+

Jupiter: Views > Fit+

Milkyway: Views > Fit+

geFitSelSet

Fits the selected as well as the highlighted objects in the graphics window.

Tools and Menu Commands

Astro: Select > Fit Selected !

Jupiter: Views > Fit Selected ! or Select > Fit Selected !

Milkyway: Select > Fit Selected !

geGetEditCell

Returns the cell ID of a cell open in the current layout window.

Prerequisite

The cell for which you want the cell ID must be opened using the `geOpenCell` function. For more information about `geOpenCell`, see Physical Implementation Online Help.

Syntax

```
geGetEditCell
```

Note:

The `dbOpenCell` function (see [dbOpenCell](#)) does not open cells in the layout window. Therefore `geGetEditCell` does not recognize those cells.

If you did not use the `geOpenCell` function to open your cells, `geGetEditCell` returns a set of parentheses `()`.

See Also

[geSelectObject](#)
[“Specifying Cells” topic](#)

geGetEnvValue

Returns the current setting of an environment variable for a layout window.

Syntax

```
geGetEnvValue windowNumber "envName"
```

where the arguments are as follows:

windowNumber

Number of the layout window whose environment variables you want information about

Valid values:

- Any positive integer corresponding to an open window
- (*hiGetCurrentWindow*)

envName

Name of the environment variable for which you want the current setting

Valid values:

- `FilterSize`
- `ViewList`
- `ViewLevels`
- `YMajorGridSpacing`
- `XMajorGridSpacing`
- `YGridSpacing`
- `XGridSpacing`
- `YSnapGrid`
- `XSnapGrid`

- MaxDragObjects
- SelectionAperture
- SnapMode
- UseLocalName
- DisplayMajorGrid
- DisplayGrid
- SnapToGrid
- ShowPathOutline
- ShowPathCenter

geGetInputData

Get input data from the mouse click. The input data can be a point, a bounding box, or a line.

Tool

Milkyway

Syntax

```
geGetInputData "shape" wait_for_single_input
```

where the arguments are as follows:

<i>shape</i>	The shape of the input data from the mouse click. Valid values: • for point: “Point” or “point”. • for line: “Line” or “line”. • for bounding box: “BBox” or “bbox”.
<i>wait_for_single_input</i>	<i>#t</i> or <i>#f</i> <i>#t</i> : The tool will wait for you to input a point, line or bounding box. <i>#f</i> : The tool will accept multiple inputs.

geGetSelectedSet

Returns the object IDs for selected objects.

Prerequisites

The cell containing the objects you want to select must be open in a layout window. In other words, the cell must be opened using the `geOpenCell` function. For more information, see `geOpenCell` in Physical Implementation Online Help. Also, you must select objects in the layout window.

Syntax

`geGetSelectedSet` *windowNumber*

where the arguments are as follows:

windowNumber

Number of the window that contains the selected objects for which you want the ID information

Valid values:

- Any positive integer corresponding to an open window
- (*hiGetCurrentWindow*)

Example

```
geGetSelectedSet 1  
(db:1270011 db:1270013 db:2d1007a db:3f5000b)
```

geGoToError

Displays an error of the current type, which you specify by error number.

Tools and Menu Commands

Astro: Verify > Show Errors – Go To Error

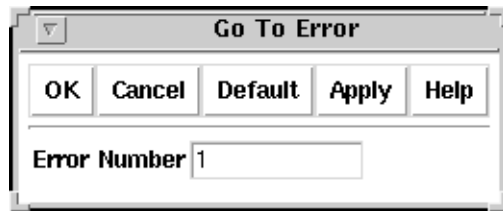
Prerequisites

Open the cell and load the error cell. After running DRC or LVS, the Synopsys tool automatically loads the error cell. To reload an error cell later, use [geLoadErrorCell](#).

Set the error types you want to display using [geSetErrorType](#), and set an error type current using [geFirstErrorType](#) and [geNextErrorType](#).

Go To Error Dialog Box

The menu command opens the following dialog box. Its options are described below.



Error Number

Type the number of the error you want to display.

geGridOption

Sets the grid options for the current editing session.

Tools and Menu Commands

Astro: Options > Grid

Jupiter: Options > Grid

Milkyway: Options > Grid

Grid Options Dialog Box

The menu command opens the following dialog box. Its options are described below.

Grid Option

OK Cancel Default Apply Help

☒ **Display Grid** Filter pixel

Display Grid Offset	X	0	Y	0
Level 0 Step	X	0.001	Y	0.001
Level 1 Step	X	0.01	Y	0.01
Level 2 Step	X	0.1	Y	0.1
Level 3 Step	X	1	Y	1
Level 4 Step	X	10	Y	10
Level 5 Step	X	100	Y	100
Level 6 Step	X	1000	Y	1000
Level 7 Step	X	10000	Y	10000

☒ **Snap To Grid**

Snap Grid Offset	X	0.250	Y	0
Snap Grid Step	X	1.900	Y	0.05
From Wire Track	X	3		3:metal3
From Wire Track	Y	3		3:metal3

Display grid

Select to display the grid.

Filter pixel

Type the threshold distance to control when the grid display will switch to the next highest level. When grid points are closer together then the *Filter* amount, the grid display changes to the next level up, so that you can always have visible grid points.

Display Grid Offset X/Y

Type the horizontal (X) and the vertical (Y) distance (in user units) by which you want to offset the grid from origin.

Level 0-7 Step X/Y

Type the horizontal (X) and the vertical (Y) distance (in user units) you want between points in the grid. You can specify up to 8 grid levels with increasing scale, or distance between grid points.

Snap To Grid

Select to have your cursor snap to grid points.

Snap Grid Offset X/Y

Type the horizontal (X) and the vertical (Y) distance (in user units) by which you want to offset the cursor snap-to points from the grid points.

Snap Grid Step X/Y

Type the horizontal (X) and the vertical (Y) distance (in user units) you want between cursor snap-to points.

From Wire Track

Click to have the layout tool automatically calculate the Snap Grid Offset from the wire tracks.

geImportPDB

This command has been replaced by the gePrepLibs command.

geImposeCell

Overlays the gap and notch cells onto your design cell so that you can view in the graphics window the gap and notch filling information which is generated for a specified cell by geNewFilling.

Note:

This functions of this command also can be exercised using the Scheme command [geSetViewCell](#).

Tool

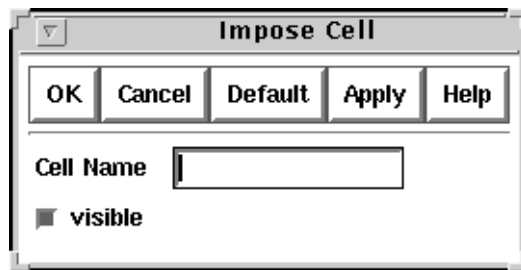
Milkyway

Prerequisites

Generate gap and notch filling information for filling gaps and notches that are smaller than the minimum distance required between objects on the same layer, using geNewFilling.

Impose Cell Dialog Box

The menu command opens the following dialog box. Its options are described below.



Cell Name

Type the name of the cell for which you want gap and notch information displayed.

visible

Select if you want to view gap and notch information in the graphics window for the cell you have specified.

geIsLayerSelectable

Specifies whether or not a layer can be selected. This function returns the selection option for a layer when *layerNameOrNumber* is specified; otherwise it returns a list of the selection options for all layers.

Syntax

```
geIsLayerSelectable windowNumber [layerNameOrNumber]
```

where the arguments are as follows:

windowNumber

Number of the layout window containing the layer for which you want information

Valid values:

Any positive integer corresponding to an open window

Note:

If *windowNumber* is 0, `geIsLayerSelectable` returns the default settings that will be applied to new windows.

layerNameOrNumber

Layer for which you want to set selection options

geIsLayerVisible

Specifies whether or not a layer is visible. This function returns the visibility option for a layer when *layerNameOrNumber* is specified; otherwise it returns a list of the visibility options for all layers.

Syntax

```
geIsLayerVisible windowNumber [layerNameOrNumber]
```

where the arguments are as follows:

windowNumber

Number of the layout window containing the layer for which you want information

Valid values:

Any positive integer corresponding to an open window

Note:

If *windowNumber* is 0, `geIsLayerVisible` returns the default settings that will be applied to new windows.

layerNameOrNumber

Layer for which you want to set visibility options

geIsObjectSelectable

Specifies whether or not an object can be selected. This function returns the selection option for an object when *objectType* is specified; otherwise it returns a list of the selection options for all objects.

Syntax

```
geIsObjectSelectable windowNumber ["objectType"]
```

where the arguments are as follows:

windowNumber

Number of the layout window containing the objects for which you want information

Valid values:

Any positive integer corresponding to an open window

Note:

If *windowNumber* is 0, `geIsObjectSelectable` returns the default settings that will be applied to new windows.

objectType

Type of object for which you want to set selection options

geIsObjectVisible

Specifies whether or not an object is visible. This function allows greater control of the visibility of objects than the `geWindowOption` command. For example, `geIsObjectVisible` gives you separate control of rectangles and polygons, whereas the Window Options dialog box combines the two types as polygons.

The `geIsObjectVisible` function returns the visibility option for an object when *objectType* is specified; otherwise it returns a list of the visibility options for all objects.

Syntax

```
geIsObjectVisible windowNumber ["objectType"]
```

where the arguments are as follows:

windowNumber

Number of the layout window containing the objects for which you want information

Valid values:

Any positive integer corresponding to an open window

Note:

If *windowNumber* is 0, `geIsObjectVisible` returns the default settings that will be applied to new windows.

objectType

Type of object for which you want to set visibility options

geLastView

Returns the graphics window to its state before you executed the last viewing command. You can also use the navigation buttons on the left side of the design window to repeat the last view of the design.

Tools and Menu Commands

Astro: Views > Previous View

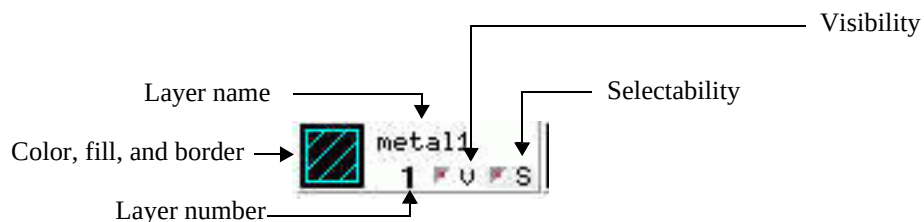
Jupiter: Views > Previous View

geLayerPanel

Allows you to view or change layer display attributes, add and delete layers, and restrict the ability to edit an object on a particular layer. You can apply these layer changes, save them (excluding editability) to the technology file, or write them to a file to retrieve them later. For each layer, the technology file specifies the following display attributes:

- Color
- Fill
- Border
- Visibility
- Selectability

The Layer Panel dialog box contains a box that shows the display attributes and editability for each layer.



By changing the attributes on the button and applying the changes, you can change the display of layers in the graphics window.

To change the visibility or selectability of a specified layer, select or deselect the V or the S in that layer's box as needed.

- To change the color, fill, or border of a specific layer, select the layer you wish to change, then select from the lower portion of the layer panel the color, fill, or border to which you want the layer to be changed.

Tools and Menu Commands

Astro: Options > Layer

Jupiter: Options > Layer

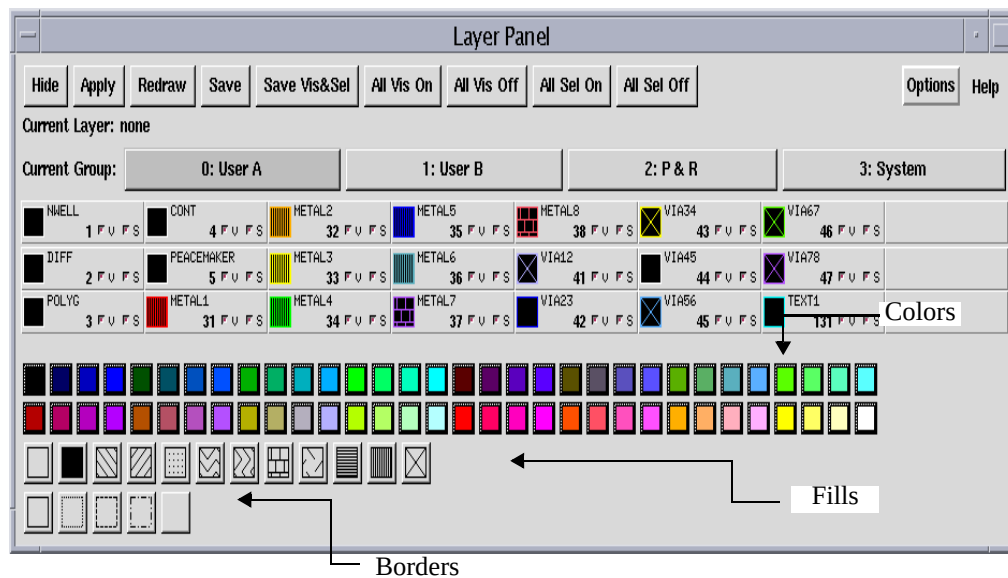
Milkyway: Options > Layer

Prerequisites

Open the library in which you want to view or manipulate layers.

Layer Panel Dialog Box

The menu command opens the following dialog box. Its options are described below.



Hide

Click this button to hide the Layer Panel dialog box.

Apply

Click this button to apply any changes you have made to your layer specifications.

Redraw

Click this button to refresh the graphics in your design after you have made changes.

Save

Click this button to save all changes you have made to your layer specifications.

Save Vis & Sel

Click this button to save your changes in visibility and selectability only.

All Vis On

Click this button to make all layers visible.

An object is visible only if both of the following are true:

- The layer of the object is visible.
- The type of the object is visible.

All Vis Off

Click this button to make all layers invisible.

All Sel On

Click this button to make all layers selectable.

An object is selectable only if both of the following are true:

- The layer of the object is visible and selectable.
- The type of the object is visible and selectable.

All Sel Off

Click this button to make all layers unselectable.

Options > order layer by

Select the order in which you want to display layers.

Option	Arranges layers from ...
column	Top to bottom
row	Left to right

Options > move layer to Group

Select if you want to move a selected layer to another group. You must first select which layer you want to move, then select which panel to which you want the layer moved, either *0: User A*, *1: User B*, *2: P&R*, or *3: System*.

Current Group

Select the group whose layer panel you wish to be displayed, either *0: User A*, *1: User B*, *2: P&R*, or *3: System*. Groups are normally determined by where the current user fits into the design process.

geLineDeselect

Lets you deselect objects by drawing a line through them.

Tools and Menu Commands

Astro: Select > Deselect – Deselect By Line

Jupiter: Select > Deselect – By Line

Milkyway: Select > Deselect – Deselect By Line

geLineSelect

Lets you select objects by drawing a line through them.

Tools and Menu Commands

Astro: Select > Select – Select By Line

Jupiter: Select > Select – By Line

Milkyway: Select > Select – Select By Line

geListCoordinatesOfAllErrorType

When you use this command, you should first open the top cell and then load the error cell. Just like the Scheme command [geListErrorSummary](#). The result of this Scheme command is similar to below:

```
DRC Error Summary
Error type: Cont Spacing Num: 3
(425300 453650) (425500 453900)
(725200 412100) (725650 412350)
(660600 495700) (660750 495950)
```

It prints all the coordinates of each error point of each type. Use this command to also print the LVS error summary.

Tool

Milkyway

Prerequisites

Open the cell containing the errors before you load the error cell.

See Also

[geListErrorSummary](#) displays a summary of the violations.

geListErrorSummary

Displays a summary of errors in the error cell.

Tools and Menu Commands

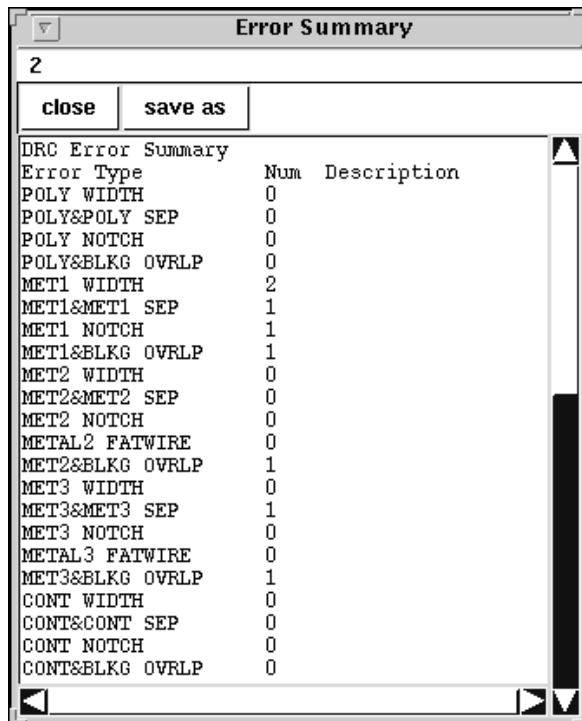
Astro: Verify > Init Errors – List Error Summary

Prerequisites

Load the error cell. If you used the default error cell name when running DRC or LVS, the Synopsys tool automatically loads the error cell. To manually load an error cell, use [geLoadErrorCell](#).

Error Summary Forms

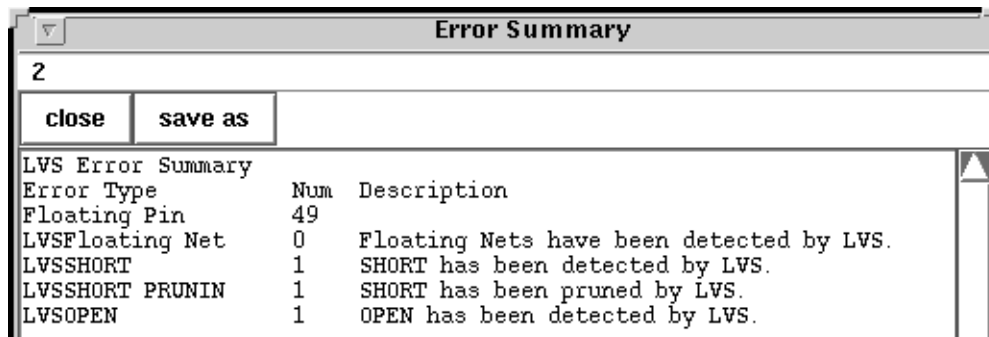
(DRC)



The screenshot shows a window titled "Error Summary" with a tab labeled "2". It contains a table with the following data:

Error Type	Num	Description
POLY WIDTH	0	
POLY&POLY SEP	0	
POLY NOTCH	0	
POLY&BLKG OVRLP	0	
MET1 WIDTH	2	
MET1&MET1 SEP	1	
MET1 NOTCH	1	
MET1&BLKG OVRLP	1	
MET2 WIDTH	0	
MET2&MET2 SEP	0	
MET2 NOTCH	0	
METAL2 FATWIRE	0	
MET2&BLKG OVRLP	1	
MET3 WIDTH	0	
MET3&MET3 SEP	1	
MET3 NOTCH	0	
METAL3 FATWIRE	0	
MET3&BLKG OVRLP	1	
CONT WIDTH	0	
CONT&CONT SEP	0	
CONT NOTCH	0	
CONT&BLKG OVRLP	0	

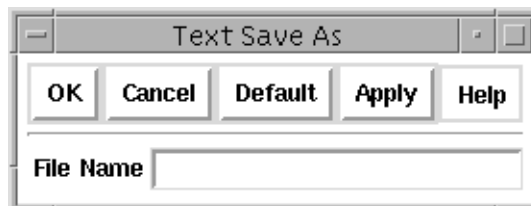
(LVS)



File menu

You can use the commands on this menu to save the text in the window to a file, load another file into the window, or close the window.

- *Save* saves the text in the window to a file with a name you assign using *Set File Name*.
- *Save As* displays the *Text Save As* dialog box.



To save the text in the window, type a *File Name* and click *OK*. The Synopsys tool saves the file to the name you type.

- *Load* loads an existing file into the window.
- *Set File Name* assigns a name to the text in the window. You can then save the text to a file using *Save*.
- *Close* closes the window.

geListHierarchy

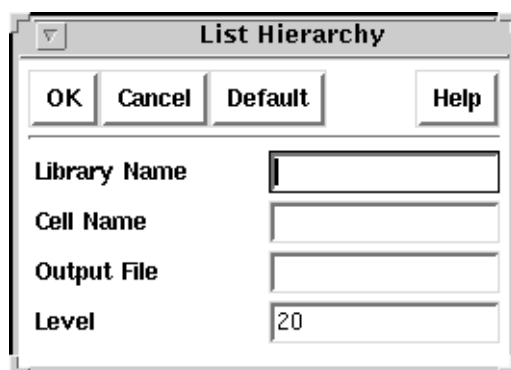
Writes the hierarchy of a specified cell to a file.

Tool

Milkyway

List Hierarchy Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Type the name of the library containing the cell for which you want hierarchy information.

Cell Name

Type the name of the cell for which you want to write out the hierarchy information.

Output File

Type the name of the output file to which you want to save the information.

Level

Type the number of levels of hierarchy you want written out for the cell.

geLoadDetailRouteErrCell

This command is obsolete. It is replaced by the [geErrorBrowser](#) command.

Loads a routing error cell so that you can examine the errors found.

Tool and Menu Command

Astro: Route Utility > Router Verify – Load DRC Error Cell

Prerequisites

Open the cell containing the errors before you load the error cell.

See Also

[geListErrorSummary](#) displays a summary of the violations.

geLoadErrorCell

This command is obsolete. It is replaced by the [geErrorBrowser](#) command.

Loads a DRC or LVS error cell so that you can examine the errors found.

Note:

If you used the default error cell name when running DRC or LVS, the Synopsys tool automatically loads the error cell.

Tools and Menu Commands

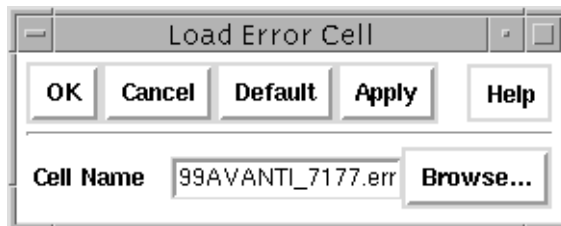
Astro: Verify > Error Viewer – Load Error Cell

Prerequisites

Open the cell containing the errors before you load the error cell.

Load Error Cell Dialog Box

The menu command opens the following dialog box. Its options are described below.



Cell Name

Enter the name of the error cell.

Browse – Click to list and select your error cells.

geMerge

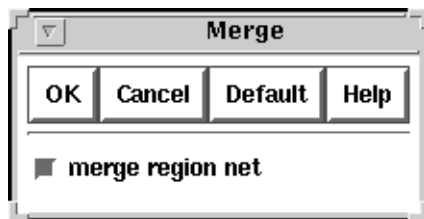
Merges two or more paths. Select the paths before invoking the command.

Tool

Milkyway

Merge Dialog Box

The menu command opens the following dialog box. Its options are described below.



merge region net

Select to merge the region nets of the selected paths.

geMergeFatBlockage

Modifies metal geometry in the *.FRAM* view of the input cell(s) so that connected consecutive sliced rectangles will be merged along the blockage boundary to avoid a metal blockage merge at the routing stage.

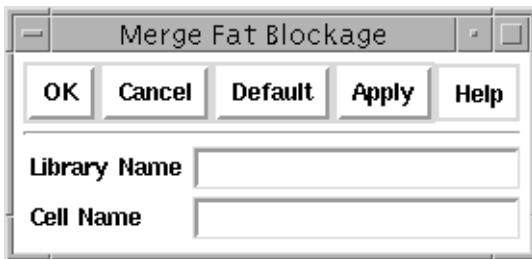
Should be run after [auExtractBlockagePinVia](#) or [geNewMakeMacro](#), if a design has defined a “fat threshold.”

Tool

Milkyway

Merge Fat Blockage Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Type the name of the library containing the layout cells where you want the metal blockage to be merged.

Cell Name

Indicate the layout cell(s) where you want the metal blockage to be merged. To indicate one cell, type the name of that cell. To indicate all cells, type an asterisk(*) or leave the field empty.

geModify

Allows you to display or change any of the following types of Astro objects: wire, cell instance, text, path, contact, contact array, rectangle, and polygon.

All object-specific modifications are available through either mouse or keyboard input. You can invoke `geModify`, use the mouse to select an object, and make your editing changes in the object dialog box. Alternatively, you can select multiple objects of the same type

with a selection command (such as `gePointSelect`), invoke `geModify`, and make keyboard-based modifications in the object dialog box.

Note:

If you select a single object, all the fields in the dialog box can be modified. If you select multiple objects of the same type, only the common fields in the dialog box can be modified.

Although you can also change the instance name, the change is treated as an engineering change order (ECO) and has an impact on back-annotation. For example, changing the instance name from `A/B/C/U1` to `A_B_C_U1` implies that you want to move an instance from a lower level to the top. This change would cause a mismatch in back-annotation, so it is recommended that you not change instance names without good reason.

Tools and Menu Commands

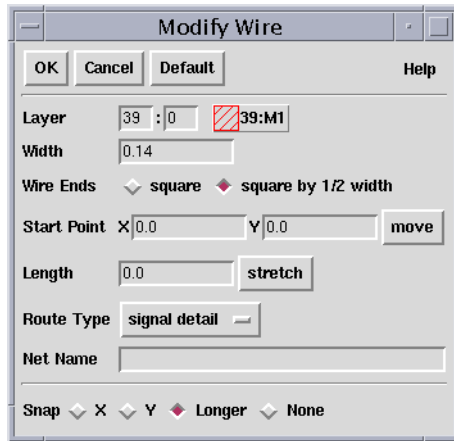
Astro: Modify > Modify

Jupiter: Astro Floorplanning: Modify > Modify

Milkyway: Modify > Modify

Modify Wire Dialog Box

Select a wire in the design. Next, enter `geModify` or choose Modify > Modify to open the Modify Wire dialog box. Its options are described below.



Layer

Type the name or number of the layer on which you want the wire, or use the Edit Layer panel to select a layer from the available layers. To display the Edit Layer panel, use [geLayerPanel](#).

Width

Type the width you want for the wire.

Wire Ends

Select the type of wire ends you want for the current portion of the wire.

Option	Result finishes the ends of the wire segments
square	at the specified points.
square by 1/2 width	at half the wire width beyond the specified points.

Start Point

Fill in the X and Y fields to specify the wire's location. Alternatively, you can specify the wire's location by using the mouse.

To use the mouse to determine the wire's location,

1. Select a Snap option to control the mouse movement.
2. Click the move button to move the wire. The reference point is the startpoint. The X and Y values change as you move the mouse.
3. Click OK to confirm the change.

Length

Fill in the Length field to specify the wire's length. Alternatively, you can specify the wire's length by using the mouse.

To use the mouse to determine the wire's length,

1. Select a Snap option to control the mouse movement.
2. Click the stretch button. The stretch point is the endpoint of the wire. The length value changes as you move the mouse.
3. Click OK to confirm the change.

Route Type

Select a route type for the object being modified. The list includes

signal user

signal global

signal detail (default)

PG ring

PG stripe

PG pin connect

PG follow pin

clock ring

clock stripe

clock zero skew

bus

fixed shielding

reroutable shielding

fill track

Net Name

The net to which the wire connects.

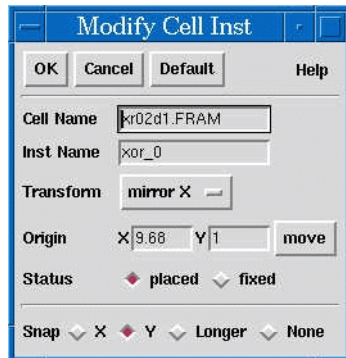
Snap

Controls the mouse movement, as follows:

Option	Description
X	The wire can be moved in the X direction. Start Point X changes as you move the mouse.
Y	The wire can be moved in the Y direction. Start Point Y changes as you move the mouse.
Longer	The wire can be moved in both the X and Y directions. Either Start Point X or Start Point Y can be changed.
None	The wire can be moved in any direction. Both Start Point X and Start Point Y can be changed.

Modify Cell Inst Dialog Box

Select a cell instance in the design. Next, enter `geModify` or choose `Modify > Modify` to open the Modify Cell Inst dialog box. Its options are described below.



Caution!

Changing a cell instance changes the netlist.

Cell Name

Type the name of a master cell in the library. (By default, the dialog box displays the current cell master of the selected cell instance.)

Inst Name

This field displays the selected instance name.

Transform

Select the orientation you want for placement of the cell instance.

Option	Orientation of cell placement
rotate 0	No change to orientation
rotate 90	Rotated counterclockwise 90 degrees
rotate 180	Rotated counterclockwise 180 degrees
rotate 270	Rotated counterclockwise 270 degrees
mirror X	Flipped on the y axis (in the X direction)
mirror Y	Flipped on the x axis (in the Y direction)

Option	Orientation of cell placement
rotate 90 mirror X	Rotated 90 degrees and flipped on the y axis (in the X direction)
rotate 90 mirror Y	Rotated 90 degrees and flipped on the x axis (in the Y direction)

Origin

The X, Y value of the bottom-left corner.

Status

The status options are

- placed

Allows movement of the cell during automatic placement and routing.

- fixed

Does *not* allow movement of the cell during any automatic operation.

Snap

Controls the mouse movement, as follows:

Option	Description
X	The wire can be moved in the X direction. Start Point X changes as you move the mouse.
Y	The wire can be moved in the Y direction. Start Point Y changes as you move the mouse.
Longer	The wire can be moved in both the X and Y directions. Either Start Point X or Start Point Y can be changed.
None	The wire can be moved in any direction. Both Start Point X and Start Point Y can be changed.

Modify Text Dialog Box

Select the text you want to modify. Next, enter `geModify` or choose `Modify > Modify` to open the Modify Text dialog box. Its options are described below.



Layer

Type the name or number of the layer on which you want the text, or use the Edit Layer panel to select a layer from the available layers. To display the Edit Layer panel, use [geLayerPanel](#).

Text

Specify the text you want to modify.

Height

Type the height you want for the text.

Transform

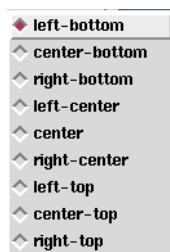
Select the orientation you want for placement of the text.

Option	Orientation of text placement
rotate 0	No changes to orientation
rotate 90	Rotated counterclockwise 90 degrees

Option	Orientation of text placement
rotate 180	Rotated counterclockwise 180 degrees
rotate 270	Rotated counterclockwise 270 degrees
mirror X	Flipped on the y axis (in the X direction)
mirror Y	Flipped on the x axis (in the Y direction)
rotate 90 mirror X	Rotated 90 degrees and flipped on the y axis (in the X direction)
rotate 90 mirror Y	Rotated 90 degrees and flipped on the x axis (in the Y direction)

Justify

Justifies the text according to the following options:



Placement

The X, Y placement specifies the left-bottom corner point of the text.

Snap

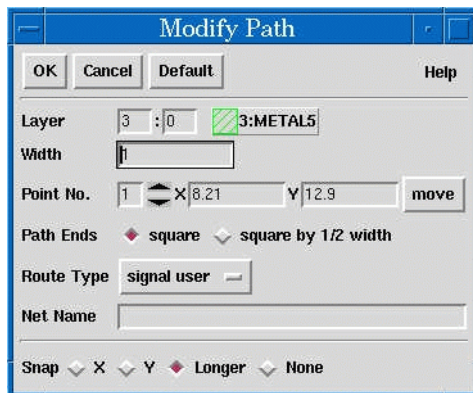
Controls the mouse movement, as follows:

Option	Description
X	The wire can be moved in the X direction. Start Point X changes as you move the mouse.

Option	Description
Y	The wire can be moved in the Y direction. Start Point Y changes as you move the mouse.
Longer	The wire can be moved in both the X and Y directions. Either Start Point X or Start Point Y can be changed.
None	The wire can be moved in any direction. Both Start Point X and Start Point Y can be changed.

Modify Path Dialog Box

Select a path in the design. Next, enter `geModify` or choose `Modify > Modify` to open the Modify Path dialog box. Its options are described below.



Layer

Type the name or number of the layer for the path, or use the Edit Layer panel to select a layer from the available layers. To display the Edit Layer panel, use [geLayerPanel](#).

Width

Type the width you want for the path.

Point No.

Allows you to change the Point No. (The corresponding X and Y value also changes).

- X

Specifies the X value of any point.

- Y

Specifies the Y value of any point.

Path Ends

Select the type of path ends you want for the current portion of the path.

Option	Result finishes the ends of the path segments
square	at the specified points.
square by 1/2 width	at half the wire width beyond the specified points.

Route Type

Select a route type for the object being modified. The list includes

signal user

signal global

signal detail (default)

PG ring

PG stripe

PG pin connect

PG follow pin

clock ring

clock stripe

clock zero skew

bus

fixed shielding

reroutable shielding

fill track

Net Name

The net to which the path connects.

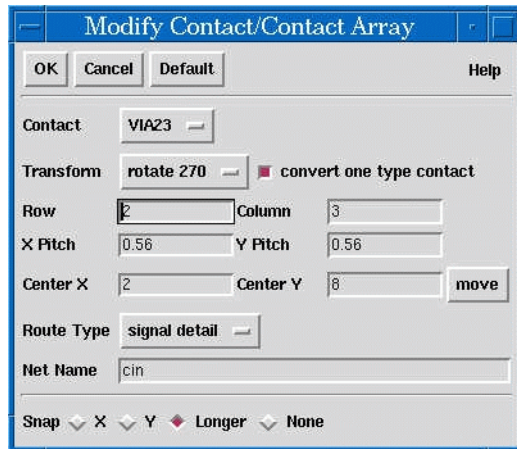
Snap

Controls the mouse movement, as follows:

Option	Description
X	The wire can be moved in the X direction. Start Point X changes as you move the mouse.
Y	The wire can be moved in the Y direction. Start Point Y changes as you move the mouse.
Longer	The wire can be moved in both the X and Y directions. Either Start Point X or Start Point Y can be changed.
None	The wire can be moved in any direction. Both Start Point X and Start Point Y can be changed.

Modify Contact/Contact Array Dialog Box

Select a contact or contact array in the design. Next, enter geModify or choose Modify > Modify to open the Modify Contact/Contact Array dialog box. Its options are described below.



Note:

When a *contact* is selected and the row or column value is more than 1, the contact is changed to a contact array after you click the OK button. When a *contact array* has row and column values that are both equal to 1, the contact array is changed to a contact after you click OK.

Contact

Select the type of contact you want to change.

Transform

Select the orientation you want for placement of the contact.

Option	Orientation of contact placement
rotate 0	No changes to orientation
rotate 90	Rotated counterclockwise 90 degrees
rotate 180	Rotated counterclockwise 180 degrees
rotate 270	Rotated counterclockwise 270 degrees
mirror X	Flipped on the y axis (in the X direction)

Option	Orientation of contact placement
mirror Y	Flipped on the x axis (in the Y direction)
rotate 90 mirror X	Rotated 90 degrees and flipped on the y axis (in the X direction)
rotate 90 mirror Y	Rotated 90 degrees and flipped on the x axis (in the Y direction)

convert one type contact

When selected, changes all contacts of the same type. This feature is used only for a contact.

Row

Type the number of horizontal rows you want in the contact array.

Column

Type the number of vertical columns you want in the contact array.

X Pitch

Type the distance you want between the centers of the contacts in a row.

Y Pitch

Type the distance you want between the centers of the contacts in a column.

Center X

The X coordinate of the contact.

Center Y

The Y coordinate of the contact.

Route Type

Select a route type for the object being modified. The list includes

signal user

signal global

signal detail (default)

PG ring

PG stripe

PG pin connect

PG follow pin

clock ring

clock stripe

clock zero skew

bus

fixed shielding

reroutable shielding

fill track

Net Name

The net to which the contact connects.

Snap

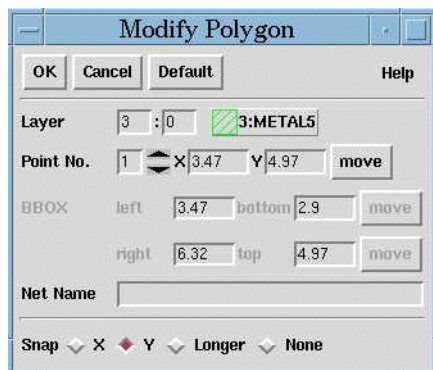
Controls the mouse movement, as follows:

Option	Description
X	The wire can be moved in the X direction. Start Point X changes as you move the mouse.

Option	Description
Y	The wire can be moved in the Y direction. Start Point Y changes as you move the mouse.
Longer	The wire can be moved in both the X and Y directions. Either Start Point X or Start Point Y can be changed.
None	The wire can be moved in any direction. Both Start Point X and Start Point Y can be changed.

Modify Polygon Dialog Box

Select a polygon in the design. Next, enter `geModify` or choose `Modify > Modify` to open the Modify Polygon dialog box. Its options are described below.



Note:

The Point No. and BBOX fields are disabled when you modify multiple polygons because the fields are specific to single polygons. However, you can change all the polygons to the same layer data type or connect them to the same net.

Layer

Type the name or number of the layer on which you want the polygon, or use the Edit Layer panel to select a layer from the available layers. To display the Edit Layer panel, use [geLayerPanel](#).

Point No.

Allows you to modify any point in the polygon.

- X

The X coordinate of the point.

- Y

The Y coordinate of the point.

BBOX

This option is disabled for polygons. The bounding box is automatically calculated.

Net Name

The net to which the polygon connects.

Snap

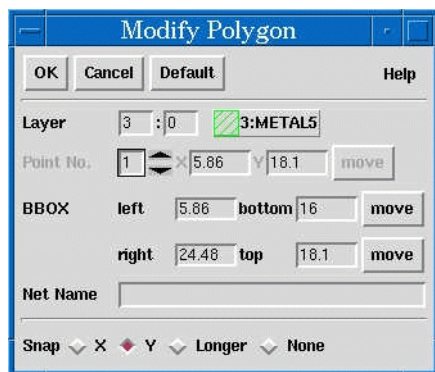
Controls the mouse movement, as follows:

Option	Description
X	The wire can be moved in the X direction. Start Point X changes as you move the mouse.

Option	Description
Y	The wire can be moved in the Y direction. Start Point Y changes as you move the mouse.
Longer	The wire can be moved in both the X and Y directions. Either Start Point X or Start Point Y can be changed.
None	The wire can be moved in any direction. Both Start Point X and Start Point Y can be changed.

Modify Polygon Dialog Box (Modify Rectangle)

Select a rectangle in the design. Next, enter `geModify` or choose `Modify > Modify` to open the Modify Polygon dialog box, which allows you to modify rectangles. Its options are described below.



Layer

Type the name or number of the layer on which you want the rectangle, or use the Edit Layer panel to select a layer from the available layers. To display the Edit Layer panel, use [geLayerPanel](#).

Point No.

This option is disabled for rectangles. Any point change will convert the rectangle to a polygon.

BBOX

Use to scale the size of the rectangle.

Net Name

The net to which the rectangle connects.

Snap

Controls the mouse movement, as follows:

Option	Description
X	The wire can be moved in the X direction. Start Point X changes as you move the mouse.
Y	The wire can be moved in the Y direction. Start Point Y changes as you move the mouse.
Longer	The wire can be moved in both the X and Y directions. Either Start Point X or Start Point Y can be changed.
None	The wire can be moved in any direction. Both Start Point X and Start Point Y can be changed.

geMove

Moves a selected object. Click on the object in the design window to select it, click again to set a reference point, then click on the place you want the object to be moved. Click the right mouse button to end the move function.

Tools and Menu Commands

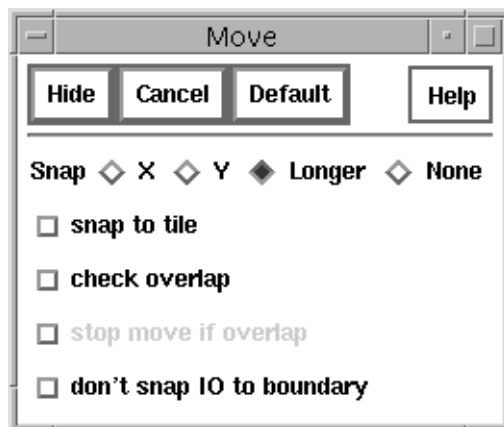
Astro: Modify > Move

Jupiter: Edit > Move

Milkyway: Modify > Move

Move Dialog Box

The menu command opens the following dialog box. Its options are described below.



Direction

Table 36-11

Option	Lets you move the object ...
X	Horizontally.
Y	Vertically.
Longer	In the direction in which you move the cursor the farthest. For example, if you move the cursor farther horizontally than vertically, the object moves in the horizontal direction.
None	In any direction.

Select the direction in which you want to move the object.

snap to tile

Select to snap the object to a unit tile.

check overlap

Select to view a list of cells that the moved object overlaps.

stop move if overlap

Select to not move the object if it overlaps other cells.

“don’t snap IO to boundary”

When this option is selected on, you can move the I/O pads to any location. When this option is not selected (off), you won’t be able to move the I/O pads outside the cell boundary or inside core. The default is off.

geMoveCell

Lets you move a standard cell to a specified point. Also lets you specify the orientation and the status of the cell.

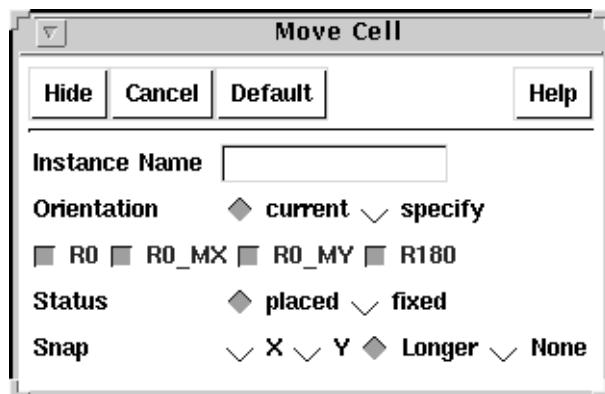
Tools and Menu Commands

Astro: Modify > Move Cell By Name

Milkyway: Modify > Move Cell By Name

Move Cell By Name Dialog Box

The menu command opens the following dialog box. Its options are described below.



Instance Name

Type the instance name of the cell to move. The instance will be highlighted in the design. Click on the cell instance, move the cursor to the location where you want to cell moved, then click again. Click the right mouse button to end the operation.

Orientation

Select *current* to keep the current orientation of the cell instance. Select *specify* to change the orientation of the cell instance and select the orientation options.

Table 36-12

Option	Moves the cell instance ...
R0	Without changing its orientation.
R0_MX	Flipped on the y axis.
R0_MY	Flipped on the x axis.
R180	Rotated counterclockwise 180 degrees.

Note:

You can select more than one orientation option. Astro snaps the cell to the closest row and determines the appropriate orientation for the specified row. If the row accepts more than one orientation, Astro uses the leftmost option.

Status

Select the status of the cell placement.

Table 36-13

Option	Marks the cell instance ...
placed	Placed. This allows movement of the cell during automatic placement and routing.
fixed	Fixed. This does not allow movement of the cell during any automatic operation.

Direction

Table 36-14

Option	Lets you move the cell instance ...
X	Horizontally.
Y	Vertically.
straight	In any direction.
longer	In the direction in which you move the cursor the farthest. For example, if you move the cursor farther horizontally than vertically, the object moves in the horizontal direction.

Select the direction in which you want to move the object.

geMoveGroup

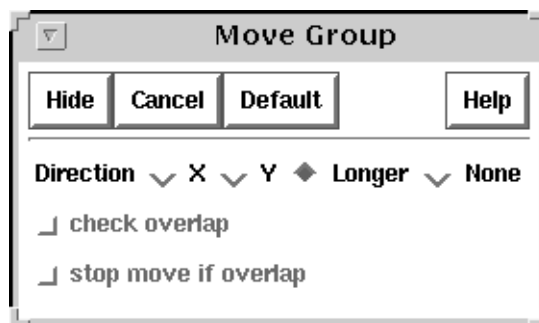
Lets you move a specified floorplan group.

Tools and Menu Commands

Jupiter: Groups > Move Group

Move Group Dialog Box

The menu command opens the following dialog box. Its options are described below.



Direction

Table 36-15

Option	Lets you move the object...
X	Horizontally.
Y	Vertically.
longer	In the direction in which you move the cursor the farthest. For example, if you move the cursor farther horizontally than vertically, the object moves in the horizontal direction.

Table 36-15

Option	Lets you move the object...
None	In any direction.

Select the direction in which you want to move the object.

check overlap

Select to view a list of cells that the moved group overlaps.

stop move if overlap

Select to not move the group if it will overlap cells.

geMovePoint

Lets you move a specified point in an object.

Note:

This command prompts you to select the object for which you want to move a point. If you have objects selected when you issue the command, you are restricted to those selected objects.

Tools and Menu Commands

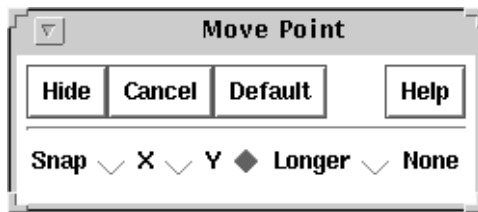
Astro: Modify > Move Point

Jupiter: Astro Floorplanning: Edit > Move Point

Milkyway: Modify > Move Point

Move Point Dialog Box

The menu command opens the following dialog box. Its options are described below.



Direction

Select the direction in which you want to move the point.

Table 36-16

Option	Lets you move the point ...
X	Horizontally.
Y	Vertically .
straight	In any direction.
longer	In the direction in which you move the cursor the farthest. For example, if you move the cursor farther horizontally than vertically, the point moves in the horizontal direction.

geMoveRegion

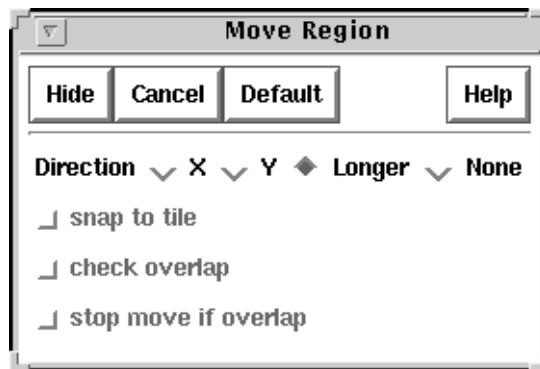
Lets you move a specified region.

Tool

Milkyway

Move Region Dialog Box

The menu command opens the following dialog box. Its options are described below.



Direction

Table 36-17

Option	Lets you move the region ...
X	Horizontally.
Y	Vertically.
Longer	In the direction in which you move the cursor the farthest. For example, if you move the cursor farther horizontally than vertically, the object moves in the horizontal direction.
None	In any direction.

Select the direction in which you want to move the region.

snap to tile

Select to snap the region to a unit tile.

check overlap

Select to view a list of cells that the moved region overlaps.

stop move if overlap

Select to not move the region if it will overlap cells.

geMoveSMPin

Allows you to move a soft macro pin or a group of soft macro pins.
For more information see fphMoveSMPin.

Note:

The `fphMoveSMPin` command is aliased to this command.

Tools and Menu Commands

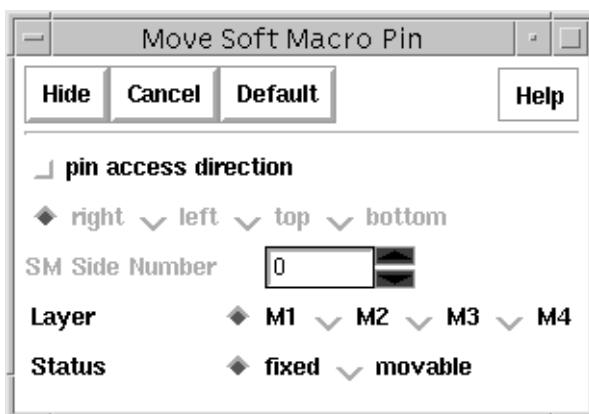
Milkyway (no menu command)

Astro (no menu command)

Jupiter: Astro Floorplanning: Soft Macro Pin > Pin Editing – Move

Move Soft Macro Pin Dialog Box

The menu command opens the following dialog box. Its options are described below.



geNameSelect

Lets you select or deselect an object or members of an object by specifying its name and type.

Tools and Menu Commands

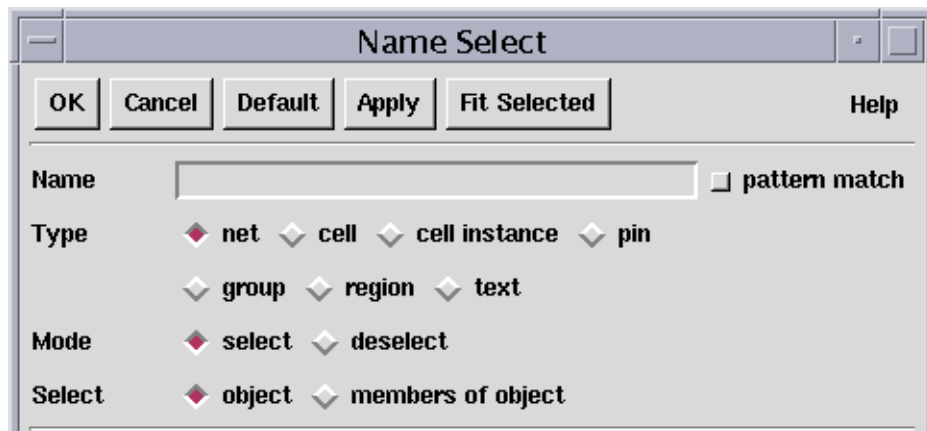
Astro: Select > (De)select By Name

Jupiter: Select > (De)select By Name

Milkyway: Select > (De)select By Name

Name Select Dialog Box

The menu command opens the following dialog box. Its options are described below.



Name

Type the name of the object you want to select (or deselect), or select *pattern match* and type a pattern to match the names of multiple objects.

- If you select a *Type* of *net*, *cell instance*, *group*, *region* or *pin*, you need to type the name (or pattern to match the names) of a net(s), cell instance(s), group(s), region(s) or pin(s) in the open cell.
- If you select a *Type* of *cell*, you need to type the name (or pattern to match the names) of a master cell(s) including view name (.CEL, .FRAM) used in the open cell.

pattern match

Select if you want to specify a pattern to match the names of multiple objects.

Type

Table 36-18

Option	Selects (or deselects) ...
net	a net with the specified name.
cell	a master cell with the specified name.
cell instance	a cell instance with the specified name.
pin	a pin with the specified name.
group	a group with the specified name.
region	a region with the specified name.

Select the type of object you want to select (or deselect).

Mode

Select whether you want to *select* or *deselect* the specified object(s).

Select

Select whether you want to select (or deselect) the object or the individual members of the object.

- *object* selects the object only.
- *members of object* selects all objects in a set. If you selected a *Type* that is a set of objects (*net*, *cell*, *region* or *group*) and want to select (or deselect) the objects in the set, you need to select this option. If you want to perform editing functions, such as moving or deleting the objects, you need to select this option.

Table 36-19

If you select a <i>Type</i> of ...	Selecting <i>members of object</i> selects ...
net	all wires/paths and contacts on the net.
cell	all instances of the master cell.
group	all objects in the group.
region	all cell instances in the region.

Note:

If an object is already selected, you cannot select its members. You must deselect the selected object first. This applies to nets, cells, groups and regions.

Fit Selected

Selecting this button will fit the screen onto the bounding box of all currently selected objects.

geNewMakeMacro

Creates pin or blockage (.fram) cells for a macro cell (block) created using a Synopsys tool, and allows for flexible control of blocking or creating feed-throughs in a macro cell. This is necessary if you want to place the cell in a Synopsys design.

Tools and Menu Commands

Astro: Cell > Make Macro Abstract

Jupiter: Astro Floorplanning: Cell > Make Macro Abstract

Milkyway: Cell > Make Macro Abstract

Make Macro Dialog Box

The menu command opens the following dialog box (by default, the Extract Blockage options are displayed). Its options are described below.

Make Macro

OK Cancel Default Apply Help

Library Name /remote/pubs-docs/cjohnson/astro/training_lab/AST1_2003.06/lab3/design_1

Cell Name PostPlaceOpt1.CEL;1

☐ Execute Hierarchical Smart Smash by Hercules

Extract Blockage Extract Pin by Text Extract Via

Extract Blockage ☐ Preserve All Metal Blockage

☐ Treat All Blockage as Thin Wire

☐ Open Soft Macro Push-down Shapes

Routing Blockage Output Layer

◇ Metal Blockage ◇ Route Guide ◇ zero min-spacing Route Guide

Treat Blockage				Threshold		Block Core	FeedThru
				X	Y	With Margin	Layer
Poly	☐ As Thin	◇ block all	◇ merge	0.000	0.000	-1	
Metal 1	☐ As Thin	◇ block all	◇ merge	0.000	0.000	-1	
Metal 2	☐ As Thin	◇ block all	◇ merge	0.000	0.000	-1	
Metal 3	☐ As Thin	◇ block all	◇ merge	0.000	0.000	-1	
Metal 4	☐ As Thin	◇ block all	◇ merge	0.000	0.000	-1	
Metal 5	☐ As Thin	◇ block all	◇ merge	0.000	0.000	-1	
Metal 6	☐ As Thin	◇ block all	◇ merge	0.000	0.000	-1	
Metal 7	☐ As Thin	◇ block all	◇ merge	0.000	0.000	-1	
Metal 8	☐ As Thin	◇ block all	◇ merge	0.000	0.000	-1	
Metal 9	☐ As Thin	◇ block all	◇ merge	0.000	0.000	-1	
Metal 10	☐ As Thin	◇ block all	◇ merge	0.000	0.000	-1	
Metal 11	☐ As Thin	◇ block all	◇ merge	0.000	0.000	-1	
Metal 12	☐ As Thin	◇ block all	◇ merge	0.000	0.000	-1	

Margin -1: block none, 0: auto compute

Library Name

Displays the name of the open library containing the cell or type the name of the library containing the layout cells for which you want to create blockages.

Cell Name

Displays the open top-level cell or type the name of the cell for which you want to create blockage cells.

Execute Hierarchical Smart Smash by Hercules

Use this option instead of the `cmSmash` command to improve the runtime and memory required to finish a large macro such as a RAM or ROM. When you enable this option, Astro generates a SMASH view containing much less geometry in comparison with the SMASH view generated by the `cmSmash` command.

You must have a Hercules license to perform this function, and the design data must be in the current working directory for Hercules.

Extract Blockage

Click to display the extraction blockage options, as described in [Extract Blockage Options](#).

Extract Pin by Text

Click to display the extract pin by text options, as described in [Extract Pin By Text Options](#).

Extract Via

Click to display the extract via options, as described in [Extract Via Options](#).

Extract Blockage Options

Select the appropriate extract blockage options and click Apply to extract the specified contact(s). By default, no contacts are extracted.

Preserve All Metal Blockage

By default, this option is off. When selected, all metal blockages are retained at their original shapes (such as not cutting around pins or trimming them for any reason).

Treat All Blockage as Thin Wire

Enable to have the router treat all blockages as thin wires in the macro or pad cells. This is typically used for designs of 130 nm and above.

This option is off by default. You can also use the As Thin option to specify thin wire individually for each layer.

Open Soft Macro Push-down Shapes

Leaves the push-down wire shapes open for the wires generated by the `fphCutPreRoute` command. The generated FRAM view will work together with shapes that remain at the top-level. The “Soft Macro Push-down Shapes” must be pushed by the `fphCutPGPreRoute` command.

Routing Blockage Output Layer

Metal Blockage – This option controls the output layer of routing blockages generated by this command.

Route Guide – The default selection is Route Guide for backward compatibility with ApolloII. However, Astro users should set the Routing Blockage Output Layer to Metal Blockage, which is more

convenient than the route guide layer from a viewing and editing perspective. The Astro router honors both Route Guide and Metal Blockage as routing blockages.

zero min-spacing Route Guide – Select to allow a Route Guide with a minimum spacing of zero.

Treat Blockage

Select from the following:

- As Thin – Specifies thin wire for individual layers
- block all – Creates a blockage on the entire macro area for the specified layer. Select this option when you want the FRAM view of a macro to contain very few areas for over-the-cell routing.
- merge – Merges the nearby geometries on the specified layer, according to the X and Y threshold values you specify.

Threshold

Specifies the threshold value to be used to control the X- and Y- directions for merging the nearby geometries on the specified layer. When the X and Y values are both zero, the blockage geometry is kept in its original shape. If a large value is specified, the nearby geometries will be merged into a larger blockage.

You can enable the “precise trimming-wires-by-pins” function by selecting “merge” (the default is “block all”) and making sure that both the X and Y thresholds are set to zero. These settings turn on the BPV_PRECISE_TRIM_WIRES_BY_PINS parameter (the default is off). Use this feature when you want the FRAM view of a macro that is placed and routed to expose the detailed geometries contained in its CEL view.

Block Core With Margin

If you want a core blockage on the corresponding routing layer, enter a positive value in this field when the Preserve All Metal Blockage option is off to automatically add a core blockage. The margin value is used to control the amount of open areas between the core blockage and the cell boundary.

A margin value of -1 means no blockage; a value of 0 means automatic computation of the margin; and a positive value means blocking cores with the specified margin. The default value is zero so the tool blocks cores with automatically computed margins of two times the pitch value on the corresponding mask.

FeedThru Layer

Enter the layer name to define (reserve) some areas in which the `geNewMakeMacro` command retains the specified area as it was in the original CEL view. This selection prevents the `geNewMakeMacro` command from covering the specified area with a blockage, and prevents merging from occurring within the specified area on the corresponding routing layer. For example, layer 190 (route guide layer) could be a feed-through layer for Metal 3.

For other layers (Metal 1 through Metal 12) the usage is identical. By default, the “block all” option is enabled for all mask layers.

Extract Pin By Text Options

In the Make Macro dialog box, click the Extract Pin by Text button to change to the extract pin options, as shown in the following dialog box and described below.

Make Macro

OK Cancel Default Apply Help

Library Name /remote/pubs-docs/cjohnson/astro/training_lab/AST1_2003.06/lab3/design_1

Cell Name PostPlaceOpt1.CEL;1

☐ Execute Hierarchical Smart Smash by Hercules

Extract Blockage Extract Pin by Text Extract Via

☒ Identify Macro Pin By Pin Text

☐ Extract Connectivity

through ☐ polyCont ☒ via1 ☒ via2 ☒ via3 ☒ via4 ☒ via5

☒ via6 ☒ via7 ☒ via8 ☒ via9 ☒ via10 ☒ via11

☒ expand small pin on blockage for contact connection

☐ text fall through ☐ double fall through

Poly Text		Metal1 Text	
Metal2 Text		Metal3 Text	
Metal4 Text		Metal5 Text	
Metal6 Text		Metal7 Text	
Metal8 Text		Metal9 Text	
Metal10 Text		Metal11 Text	
Metal12 Text		Fall Through Text	

Pin must-connect area layer number Auto-compute pin must-connect area

Identify Macro Pin By Pin Text

For gdsIn data, select this option to identify pins by text, text fall-through, or double fall-through in the same manner as [auExtractBlockagePinVia](#).

Extract Connectivity

Select if you want Milkyway to trace the connectivity of a specified multiple-layer pin and extract the entire pin.

through

Select from the following vias:

- polyCont
- via1
- via2
- via3
- via4
- via5
- via6
- via7
- via8
- via9
- via10
- via11

The connectivity can be extracted through any specified vias. The default is to extract connectivity through via1 to via11 (Metal 1 through Metal 12) when the Extract Connectivity option is selected.

expand small pin on blockage for contact connection

Select if you have small metal1 pins that are insufficient to the minimal width inside the metal1 blockage.

- If you do not select this option, the router cannot access the small metal1 pins because the pin and its surrounding open area is too small.
- If you select this option, Milkyway cuts an extra large space through the metal1 blockage around the small metal1 pins to allow the router access the small metal1 pins. In this case, the router will route in metal2 over the metal1 blockage and drop vias in the sufficient metal1 open space to access the small metal1 pins.

text fall through

Select and specify the “text fall through” layer if you want Milkyway to look at the metal layer(s) beneath the origin of each text label to identify the geometry annotated by the text. If more than one geometry exists beneath the origin of the text label, Milkyway selects one geometry to associate with the label according to the priority: from the highest metal layer to the lowest metal layer.

For example, if one of the geometries is on the metal1 layer, Milkyway selects this geometry. If not, Milkyway looks for a geometry on the poly layer.

double fall through

Select and specify the “double fall through” layer if more than one geometry exists beneath the origin of a text label in two different masks and you want the two geometries being named the same as the text label. Milkyway selects the first and second geometry to associate with the text label according to the priority: from the highest metal layer to the lowest metal layer.

For example, if two geometries are located beneath the origin of some text, one is in metal2, the other is in metal1; if the “double fall through” mode is on, both geometries will be named as the same as the text; if only the “text fall through” mode is on, only the metal2 geometry will be identified as a pin of the text.

Note:

Currently, for a design with more than three metal layers, the “text fall through” or “double fall through” mode to identify a pin geometry is seldom used. The “text fall through” and “double fall through” are retained for less than three mask designs.

In general, there are two styles to match between the pin text and the pin geometry. First, the pin text and its pin geometry are on the same layer. For example, all metal1 pin texts are located on the metal1 layer, same as metal1 geometry.

Second, there is a pin text layer corresponding to each metal layer. For example, all metal1 pin texts are located on layer MET1_TEXT, and all metal1 geometry on the metal1 layer.

For the first style, when the text and its identified geometry are on the same layer you don’t need to specify the text layer associated with the corresponding metal text field. For the second style, you need to specify the name or number of the text layer used for identifying its metal geometries in the corresponding metal text field. When the text layers are not specified (and “text fall through” mode is not selected), Milkyway assumes the pin text layers match the geometry layers.

Poly Text

Enter the name or number of the layer used for the text identifying poly geometries (or leave blank if the text layer matches the geometry layer).

Metal# Text

Enter the name or number of the layer used for the text identifying Metal# geometries (or leave blank if the text layer matches the geometry layer).

Fall Through Text

Enter the name or number of the layer used for text if you select “text fall through” or “double fall through.”

Pin Connection Options

Click “Pin must-connect area layer number” to display the additional options, as described below.

Make Macro

OK Cancel Default Apply Help

Library Name /remote/pubs-docs/cjohnson/astro/training_lab/AST1_2003.06/lab3/design_1

Cell Name PostPlaceOpt1.CEL;1

☐ Execute Hierarchical Smart Smash by Hercules

Extract Blockage Extract Pin by Text Extract Via

☐ Identify Macro Pin By Pin Text

☐ Extract Connectivity

through ☐ polyCont ☒ via1 ☒ via2 ☒ via3 ☒ via4 ☒ via5

☒ via6 ☒ via7 ☒ via8 ☒ via9 ☒ via10 ☒ via11

☐ expand small pin on blockage for contact connection

☐ text fall through ☐ double fall through

Poly Text		Metal1 Text	
Metal2 Text		Metal3 Text	
Metal4 Text		Metal5 Text	
Metal6 Text		Metal7 Text	
Metal8 Text		Metal9 Text	
Metal10 Text		Metal11 Text	
Metal12 Text		Fall Through Text	

Pin must-connect area layer number Auto-compute pin must-connect area

Pin Must-Connect Area Layer Number

Poly Pin		M1 Pin		M2 Pin		M3 Pin	
M4 Pin		M5 Pin		M6 Pin		M7 Pin	
M8 Pin		M9 Pin		M10 Pin		M11 Pin	
M12 Pin							

Pin Must-Connect Area Layer Number

Select this option when you want the router to connect a pin

through a specified area. Enter the corresponding layer number with metal pins, for example poly, metal1, ..., metal12. The geNewMakeMacro command reads in the specified areas from the entered layer number to generate a proper routing guide to have the router connect the pin through the specified area. (See also [auExtractBlockagePinVia](#)).

Note:

The layer number entered in a field must be defined in the technology file of the library.

Auto-compute pin must-connect area

As an alternate to drawing a polygon to designate the via solution, click the “Auto-compute pin must-connect area” button to display the Pin Must-Connect Area Width Threshold options.

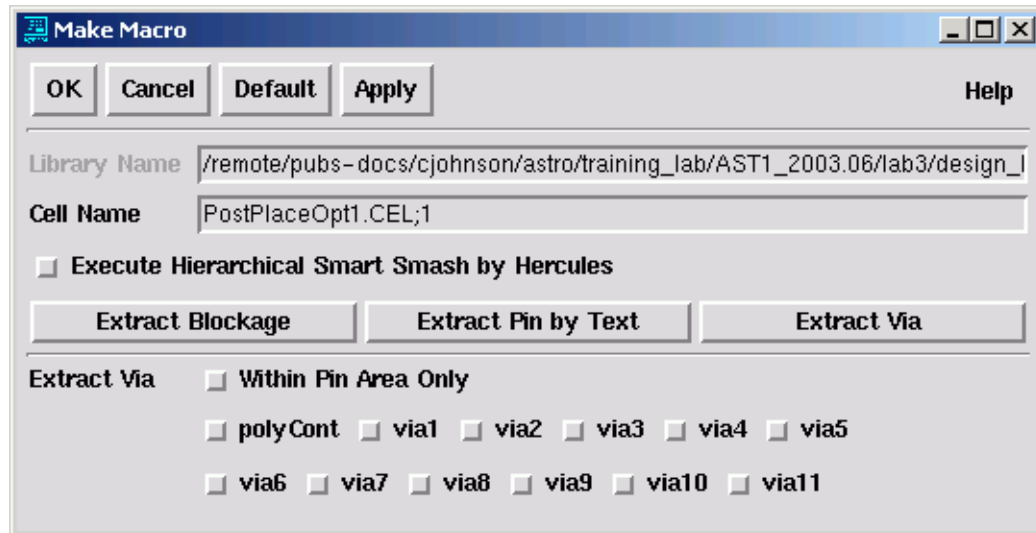
The screenshot shows a dialog box titled "Auto-compute pin must-connect area". At the top, there are two buttons: "Metal12 Text" and "Fall Through Text". Below these are two more buttons: "Pin must-connect area layer number" and "Auto-compute pin must-connect area". The main section is titled "Pin Must-Connect Area Width Threshold" and contains a grid of input fields for various layers. The layers and their corresponding threshold values are:

Layer	Threshold
Poly	0.000
M1	0.000
M2	0.000
M3	0.000
M4	0.000
M5	0.000
M6	0.000
M7	0.000
M8	0.000
M9	0.000
M10	0.000
M11	0.000
M12	0.000

These options let you specify a threshold value for automatically computing an area in an irregular pin that is large enough for the router to connect and excludes small areas where you do not want a pin connected. For example, the following figure shows an irregular pin. For details, see [auExtractBlockagePinVia](#).

Extract Via Options

In the Make Macro dialog box, click Extract Via to display the Extract Via options, as described below.



Extract Via

Within Pin Area Only – extracts vias from within the pin area only.

Select from the following vias:

- polyCont
- via1
- via2
- via3
- via4
- via5
- via6
- via7
- via8
- via9

- via10
- via11

geNewShortLocator

Checks for shorts between specified nets or between a specified net and the null net. (Any object that is on an interconnect layer but not connected to a net is considered to be on the null net.)

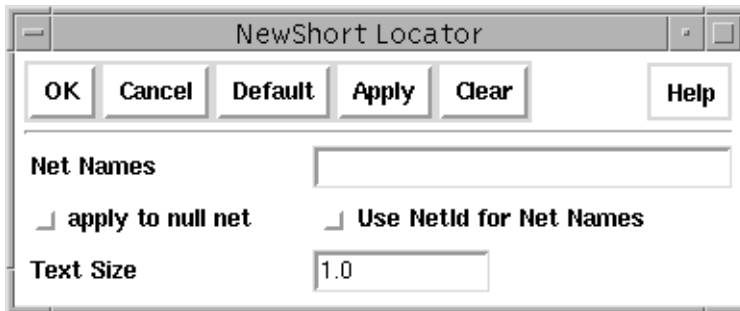
After you specify the nets for which you want to locate shorts and click *OK*, the Synopsys tool displays the approximate location of each short in the messages window and flashes a marker in the graphics window to identify each short. The marker includes the names of the nets causing the short and an arrow pointing toward the approximate location of the short.

Tools and Menu Commands

Astro: Verify > Open/Short – Locate Short

NewShort Locator Dialog Box

The menu command opens the following dialog box. Its options are described below.



Net Names

Type the name(s) of the net(s) you want to check for shorts. To indicate multiple nets, separate the names with commas. The Synopsys tool reports shorts for each pair of nets. If you select *apply to null net*, the Synopsys tool also reports shorts between each specified net and the null net.

Note:

If you do not select *apply to null net*, you need to type at least two net names.

apply to null net

Select if you want to check for shorts between the nets you specify and the null net.

Note:

Any object that is on an interconnect layer but not connected to a net is considered to be on the null net.

Use NetId for Net Names

Select if you would like to use the *NetId* as the input for *Net Names* (see above).

Text Size

Type the size you want for displaying the marker associated with a short. The marker includes the names of the nets causing the short and an arrow pointing toward the approximate location of the short.

geNextError

Displays the next error of the current type.

Tools and Menu Commands

Astro: Verify > Show Errors – Next Error Type

Prerequisites

Open the cell open and load the error cell. After running DRC or LVS, the Synopsys tool automatically loads the error cell. To reload an error cell later, use [geLoadErrorCell](#).

Set the error types you want to display using [geSetErrorType](#), and set an error type current using [geFirstErrorType](#) and [geNextErrorType](#).

geNextErrorType

Makes the next error type current.

Tools and Menu Commands

Astro: Verify > Set Error Type – Next Error Type

Prerequisites

Open the cell open and load the error cell. If you used the default error cell name when running DRC or LVS, the Synopsys tool automatically loads the error cell. To manually load an error cell, use [geLoadErrorCell](#).

Set the error types you want to display using [geSetErrorType](#), and set the first error type current using [geFirstErrorType](#).

geOpenCell

Opens a specified cell. You can see all the cells in the libraries when you click the *Browse* button.

Tools and Menu Commands

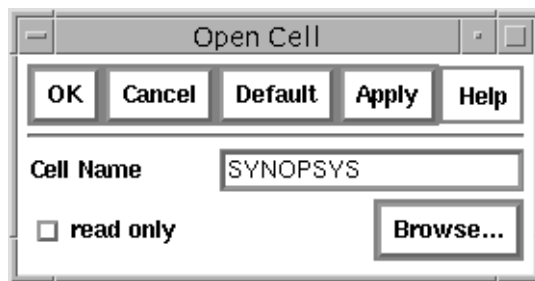
Astro: Cell > Open

Jupiter: Cell > Open

Milkyway: Cell > Open

Open Cell Dialog Box

The menu command opens the following dialog box. Its options are described below.



Cell Name

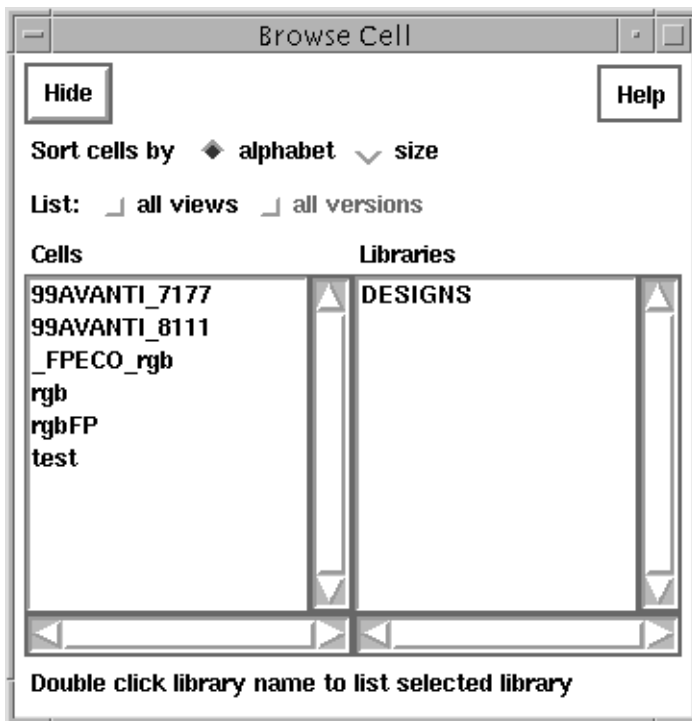
Type the name of the cell you want to open.

read only

Select if you want to prevent any changes to the cell while you have it opened.

Browse

Click to display the Browse Cell window from which you can choose from any cell in the libraries. You can also sort the cells by selecting either *alphabet* or *size*. Your choice will allow you to either select sorted cells alphabetically, or by file size.



geOpenLib

Opens a specified library.

Tools and Menu Commands

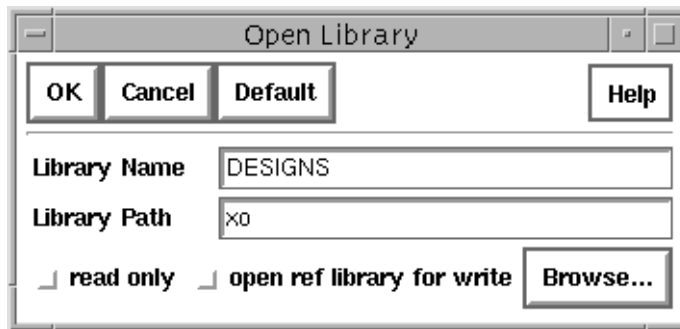
Astro: Library > Open

Jupiter: Library > Open

Milkyway: Library > Open

Open Library Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Type the name of the library you want to open. Type *sample* to open the Synopsys sample library.

Library Path

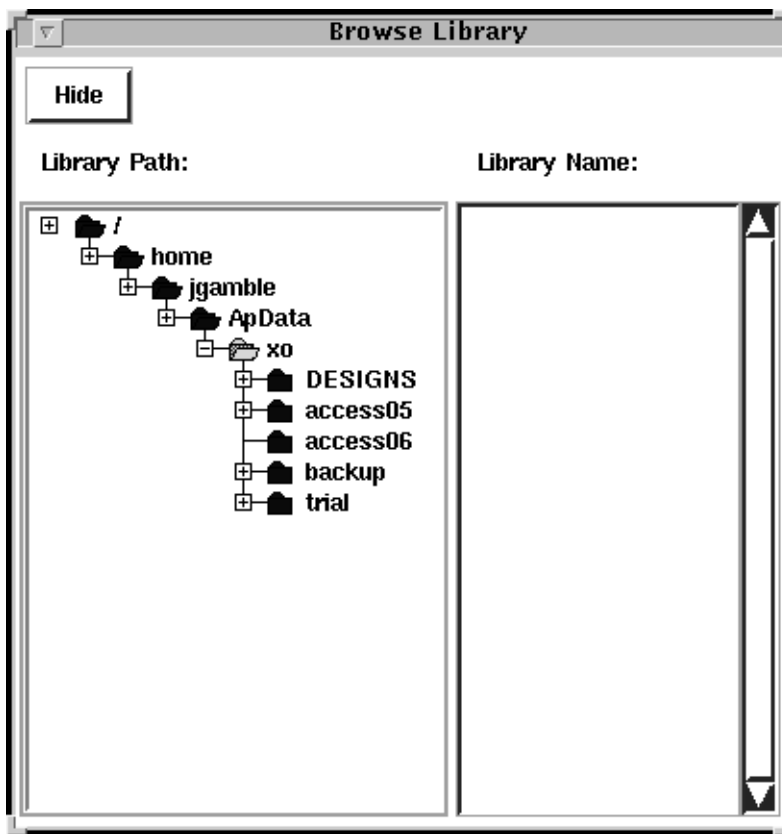
If you invoked your Synopsys tool from the directory containing your libraries, you can leave this blank. Otherwise, you need to type the full path (or the path from the directory from which you invoked your Synopsys tool) to the directory containing your library directories. To specify the path, you can use any UNIX notation (for example, “..” and “~”).

Browse

Select to invoke the Browse Library dialog box.

Browse Library Dialog Box

The menu command opens the following dialog box. Its options are described below.



Browse Library Dialog Box

Select the library containing the cells for your design by clicking on the appropriate icon in the directory hierarchy.

geOpenLocator

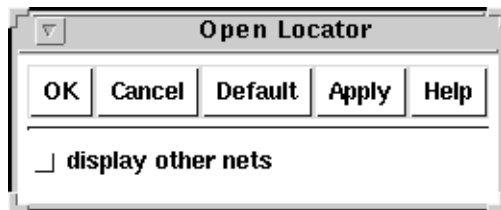
Manipulates the display to facilitate locating opens in a design.

Tools and Menu Commands

Astro: Verify > Open/Short – Locate Open

Open Locator Dialog Box

The menu command opens the following dialog box. Its options are described below.



display other nets

Select to display all nets in the design. Deselect to display only the net that contains the open.

geOutputBBox

Outputs the coordinates of the bounding box from the mouse click.

Syntax

`geOutputBBox`

geOutputLine

Outputs the coordinates of the line from the mouse click.

Syntax

`geOutputLine`

geOutputPoint

Outputs the coordinates of the point from the mouse click.

Syntax

```
geOutputPoint
```

gePan

Pans the design in the graphics window so that a specified point in the design is in the center of the graphics window. You can also use the navigation buttons on the left side of the graphics window to pan around the design.

Tools and Menu Commands

Astro: Views > Pan

Jupiter: Views > Pan

Milkyway: Views > Pan

gePan2

Pans the design in the graphics window by an amount and in a direction you specify by entering two points. For example, if you enter the second point 300 microns to the right of the first point, your Synopsys tool pans the design 300 microns to the right.

You can also use the navigation buttons on the left side of the graphics window to pan around the design.

Tools and Menu Commands

Astro: Views > Pan – 2-point

Jupiter: Views > Pan – 2-point

Milkyway: Views > Pan – 2-point

gePointDeselect

Lets you deselect objects by clicking on them.

Tools and Menu Commands

Astro: Select > Deselect – By Point

Jupiter: Select > Deselect – By Point

Milkyway: Select > Deselect – By Point

gePointSelect

Allows you to select objects by clicking on them.

Tools and Menu Commands

Astro: Select > Select – By Point

Jupiter: Select > Select – By Point

Milkyway: Select > Select – By Point

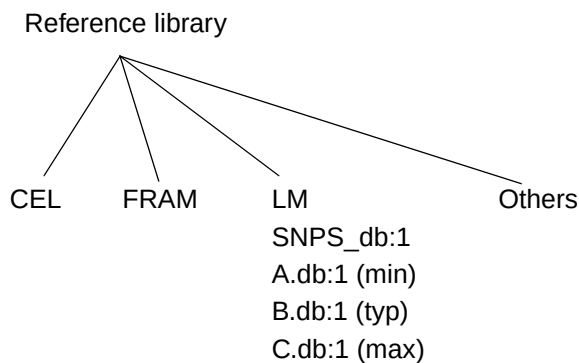
gePrepLibs

Creates the logic model (LM) view in the Milkyway database. You can invoke the `gePrepLibs` dialog box either by typing `gePrepLibs` on the command line or by completing the following steps from the Data Prep menu in either Astro or Milkyway:

1. Choose Tools > DataPrep
2. Choose Cell Library > Library Preparation
3. Click the Prepare Logical Library button
4. Click the LIB/DB button

The LM view stores the Synopsys library .db information. Tools based on Milkyway use the LM view to access the library's eight main objects: library, cell, pin, timing, internal power, leakage power, electromigration, and test cell. The `gePrepLibs` command creates the LM view and does the following:

- Attaches all logical, timing, and power libraries to the LM view.
- Registers the minimum, typical, and maximum logical libraries in the LM view, as shown:



The LM view replaces the TIM and PWR views. However, the LM view has full backward compatibility with both the TIM and PWR views. When you create an LM view within the Milkyway reference library, you can store the .db files and read them directly into the Astro timer.

The LM view is created as part of the library preparation process, which must be completed prior to reading in any designs. For more information about library preparation, see the *Astro User Guide*.

Note:

The `gePrepLibs` command replaces the `geImportPDB` and `geCreateModelView` commands.

Tools and Menu Commands

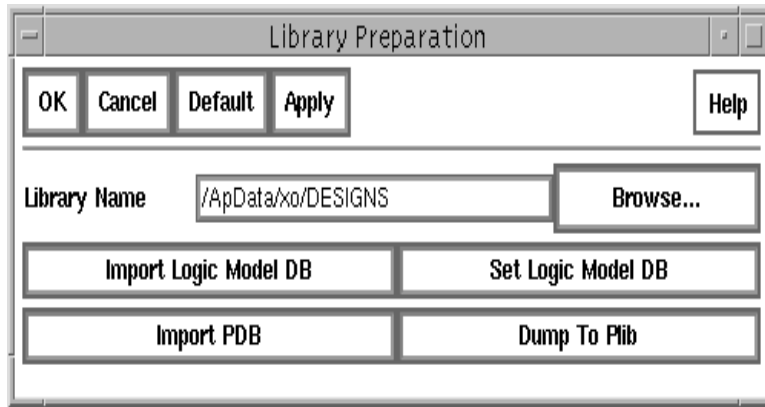
Astro: Cell Library > Library Preparation

Jupiter: Astro Floorplanning: Cell Library > Library Preparation

Milkyway: Cell Library > Library Preparation

Library Preparation Dialog Box

The Cell Library > Library Preparation menu command opens the following dialog box. Its options are described below.



Library Name

Type the name of the library you want to prepare.

Import Logic Model DB

Clicking this button expands the window. From the Import Logic Model DB area, you can

- Click the From .lib button to import the LM view from the .lib file to the named Milkyway reference library. Clicking this button further expands the window, allowing you to do the following:

Enter the filenames of one or more .lib files for the maximum, minimum, and typical operating conditions, in the Max DB Lib File, Min DB Lib File, and Typical DB Lib File boxes, respectively. You can also enter the filenames of one or more .lib files, for any other operating condition, in the Other DB Lib File box.

Select Suppress Library Compiler Warnings, to prevent Library Compiler warnings from appearing in the input/message area of the application window.

Select Set Port Direction, to update current port directions and bus information according to the information in the .lib files.

Select Overwrite, to overwrite the existing .db file that physically resides in the library directory. The new files are copied into the library directory and overwrite any existing files (that is, files with the same name) in the directory. This means that any tool or application that refers to this file also uses the new file. When Overwrite is deselected (the default), the new files are compared against the existing files in the library directory. When the file name already exists, the new file is copied to a new name (that is, the original file name is appended with a numeric suffix). When the file name doesn't exist, the new file is simple copied, without any name change.

Select Append, to add the new files to the current list of .db files in the .lib file. When Append is deselected (the default), the new files overwrite the current list of .db files in the .lib file.

Important:

“From .lib” calls Library Compiler. You must have an installation of Library Compiler in your path for this option to work.

- Click Select DB, to import the LM view from the .db file to the named Milkyway reference library. Clicking this button expands the window, allowing you to do the following:

Enter the filenames of one or more .db files, for the maximum, minimum, and typical operating conditions, in the Max DB File, Min DB File, and Typical DB File boxes, respectively. You can also enter the filenames of one or more .db files, for any other operating condition, in the Other DB File box.

Select Set Port Direction, to update current port directions and bus information according to the information in the .lib files.

Select Overwrite, to overwrite the existing .db file that physically resides in the library directory. The new files are copied into the library directory and overwrite any existing files (that is, files with the same name) in the directory. This means that any tool or application that refers to this file also uses the new file. When Overwrite is deselected (the default), the new files are compared against the existing files in the library directory. When the file name already exists, the new file is copied to a new name (that is, the original file name is appended with a numeric suffix). When the file name doesn't exist, the new file is simple copied, without any name change.

Select Append, to add the new files to the current list of .db files in the .lib file. When Append is deselected (the default), the new files overwrite the current list of .db files in the .lib file.

- Click Delete DB, to delete .db files that were imported to the Milkyway reference library. Clicking this button expands the window, allowing you to do one of the following:

Enter the names of the set of .db files for an operating condition you want to delete, in the DB To Delete box. You must list all the .db files for an operating condition and you must list them in the original input order. You can click Select DB to view and select a set of .db files to delete.

Select Delete All Logic Model DB, to delete all .db files.

Note:

If you use both the DB To Delete and the Delete All options, the latter has the higher precedence and all the .db files will be deleted.

Import PDB

Clicking this button expands the window. From the Import Physical DB area, you can

- Click From FRAM View, to do the following: translate the technology information and FRAM view cells in the Milkyway reference library to .plib format, compile the .plib format into .pdb format, and import the .pdb file to the named library. Clicking this button expands the window, allowing you to do one of the following:

Select Create Macro-Only PDB (the default), to import only the macro information (without importing the physical technology information). This is the default because the technology information is generally set during data preparation.

Select Suppress Library Compiler Warnings, to prevent Library Compiler warnings from appearing in the input/message area of the application window.

Select Overwrite, to overwrite the existing .pdb file that physically resides in the library directory. The new files are copied into the library directory and overwrite any existing files (that is, files with the same name) in the directory. This means that any tool or application that refers to this file also uses the new file. When Overwrite is deselected (the default), the new files are compared against the existing files in the library directory. When the file name already exists, the new file is copied to a new name (that is, the original file name is appended with a numeric suffix). When the file name doesn't exist, the new file is simple copied, without any name change.

- Click From Plib, to compile the .plib format into .pdb format and then import the .pdb file to the named Milkyway reference library. Clicking this button expands the window, allowing you to do the following:

Enter the names of one or more .plib files to be compiled, in the Plib File Name box.

Select Create Macro-Only PDB (the default), to import only the macro information (without importing the physical technology information). This is the default because the technology information is generally set during data preparation.

Select Suppress Library Compiler Warnings, to prevent Library Compiler warnings from appearing in the input/message window.

Select Overwrite, to overwrite the existing .pdb file that physically resides in the library directory. The new files are copied into the library directory and overwrite any existing files (that is, files with the same name) in the directory. This means that any tool or application that refers to this file also uses the new file. When Overwrite is deselected (the default), the new files are compared against the existing files in the library directory. When the file name already exists, the new file is copied to a new name (that is, the original file name is appended with a numeric suffix). When the file name doesn't exist, the new file is simple copied, without any name change.

- Click Select PDB, to specify the name of one or more .pdb files to be imported to the named Milkyway reference library. Clicking this button expands the window. You can enter the filename in the PDB File Name box.

Select Overwrite, to overwrite the existing .pdb file that physically resides in the library directory. The new files are copied into the library directory and overwrite any existing files (that is, files with the same name) in the directory. This means that any tool or application that refers to this file also uses the new file. When Overwrite is deselected (the default), the new files are compared against the existing files in the library directory. When the file name already exists, the new file is

copied to a new name (that is, the original file name is appended with a numeric suffix). When the file name doesn't exist, the new file is simple copied, without any name change.

- Click Delete PDB, to specify the name of a .pdb file to be deleted from the Milkyway reference library. Clicking this button expands the window. You can enter the filename in the PDB To Delete box. You can also click Select DB to view and select a list of .db files to delete.

Dump To Plib

Clicking this button expands the window. From the Dump Plib from FRAM view area, enter the name of the .plib file to be dumped. During this process, the information from the Milkyway FRAM view is translated to .plib format and written to the named .plib file.

The Dump Plib from FRAM view area has three options:

- Tech info (Optional)

This option is selected by default. When Tech info is selected, Milkyway writes a technology .plib file. You can also

- write a complete .plib file by selecting the Tech info option, the Cell info option, and the Cell Geometries option.
- write a .plib file with technology and cell information, but without geometry information, by selecting the Tech info option and the Cell info option and deselecting the Cell Geometries option.

- Cell info

This option is not selected by default. If you select this option, Milkyway writes a .plib file with cell information. You can

- write a .plib file with cell information and geometries, but without technology information, by selecting the Cell info option and deselecting the Tech info option.
- write a .plib file with technology and cell information, but without geometry information, by selecting the Tech info option and the Cell info option and deselecting the Cell Geometries option.
- write a .plib file with cell information but without technology information or cell geometries by selecting the Cell info option and deselecting the Tech info option and the Cell Geometries option.
- Cell Geometries

This option is selected by default when you select the Cell info option. When Cell Geometries is selected, Milkyway writes a .plib file with cell info and geometries.

Set Logic Model DB

Clicking this button expands the window. From the Set Logic Model DB area, you can do the following:

- For a reference library that already has LM view information, you can view and delete the current sets of .db files and you can reassign them to other operating conditions.

Select Is The Above Library A Reference Library to indicate that the specified library is a reference library.

Enter the filenames of one or more .db files, for the maximum, minimum, and typical operating conditions, in the Max DB Name, Min DB Name, and Typical DB Name boxes, respectively. You must list all the .db files for an operating condition and you must list them in the original input order. You can click Select DB to view and select a set of .db files.

- For a design library that references a library with LM view information, you can assign sets of .db files to be used by a specific design cell. Usually a design cell uses the .db files assigned to a reference library during library preparation. These are the default files. However, you can use the options in the Set Logic Model DB area to override these .db files and to assign different .db files, to be used by a specific design cell.

Deselect Is The Above Library A Reference Library to indicate that the specified library is a design library.

Enter the name of the design cell for which to assign the default sets of .db files, in the Design Cell Name box.

Enter the name of the reference library from which the set of .db files are assigned from. You can specify different sets of default .db files for each reference library by setting the fields appropriately and clicking Apply for each reference library.

Enter the filenames of one or more .db files, for the maximum, minimum, and typical operating conditions from the specified reference library, in the Max DB Name, Min DB Name, and Typical DB Name boxes, respectively. You must list all the .db files for an operating condition and you must list them in the original input order. You can click Select DB to view and select a set of .db files.

OK or Apply

When you click OK or Apply, Astro attaches each set of .db or .lib files to the corresponding operating condition (the .lib files are converted to .db files first) and replaces the current set of .db files for that operating condition. The current set of files are moved to the “other” operating condition. You can have multiple sets of other operating conditions.

gePreviousError

Displays the previous error of the current type.

Tools and Menu Commands

Astro: Verify > Show Errors – Previous Error

Prerequisites

Open the cell and load the error cell. After running DRC or LVS, the Synopsys tool automatically loads the error cell. To reload an error cell later, use [geLoadErrorCell](#).

Set the error types you want to display using [geSetErrorType](#), and set an error type current using [geFirstErrorType](#) and [geNextErrorType](#).

gePurgeCell

Deletes old versions of the specified cell(s).

Tools and Menu Commands

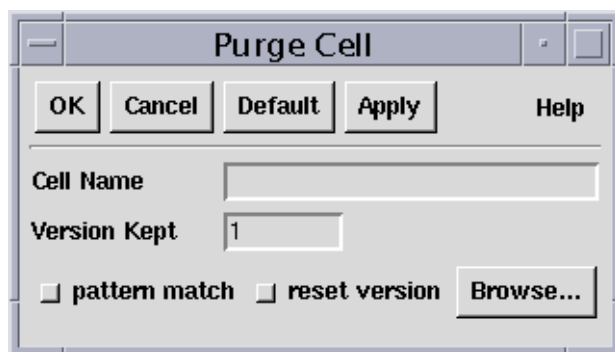
Astro: Cell > Purge

Jupiter: Cell > Purge

Milkyway: Cell > Purge

Purge Cell Dialog Box

The menu command opens the following dialog box. Its options are described below.



Cell Name

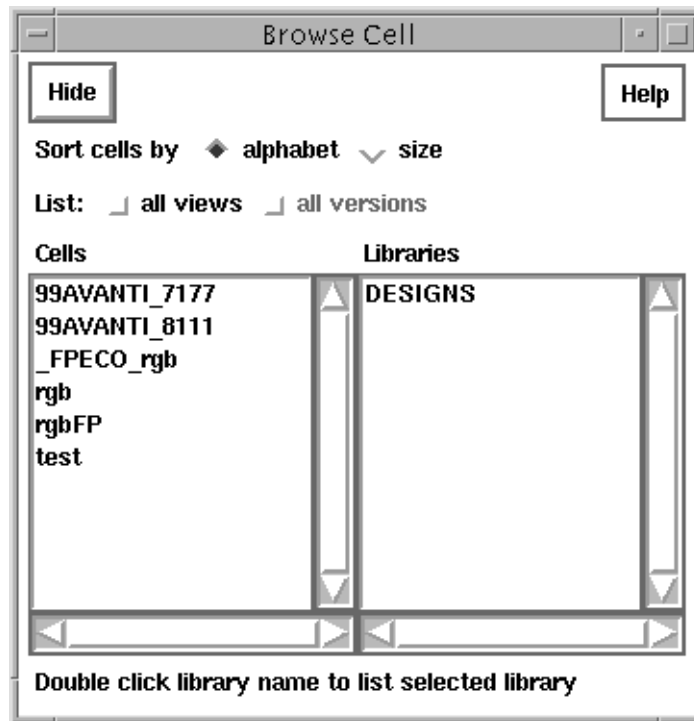
Type the name of the cell you want to purge, or select *pattern match* and type a pattern to match the names of multiple cells.

of versions kept

Type the number of versions of the cell you want to keep.

Browse

Click to display the Browse Cell window from which you can choose from any cell in the libraries. You can also sort the cells by selecting either *alphabet* or *size*. Your choice will allow you to either select sorted cells alphabetically, or by file size.



pattern match

Select if you want to specify a pattern to match the names of multiple cells to purge.

geQueryError

Displays information about an error object when you click on the error in the graphics window. An error cell must first be loaded and displayed.

Tools and Menu Commands

Astro: Verify > Show Errors - Query Error !

geQueryGroup

Displays information about groups.

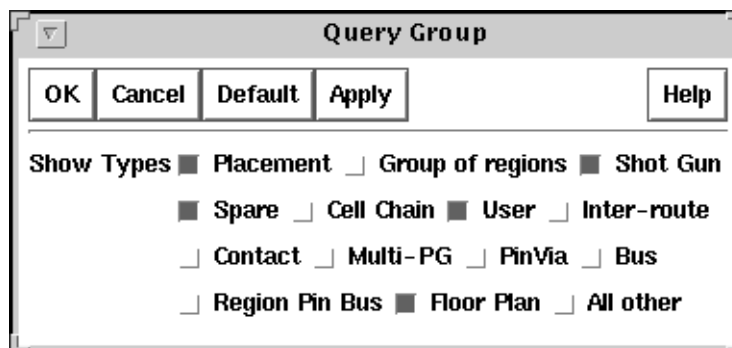
Tools and Menu Commands

Milkyway (no menu command)

Jupiter: Query > Groups

Query Group Dialog Box

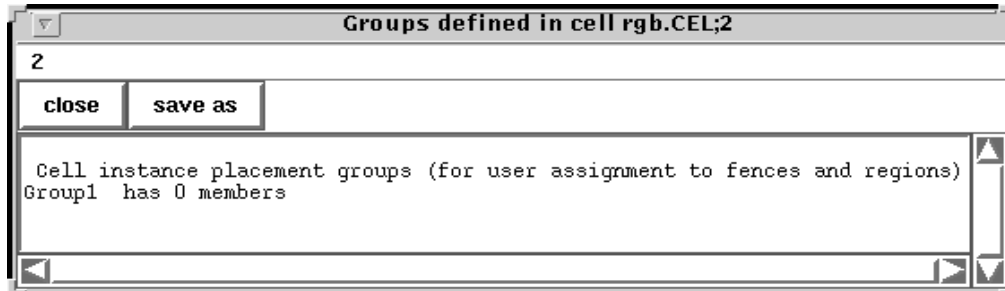
The menu command opens the following dialog box. Its options are described below.



Show Types

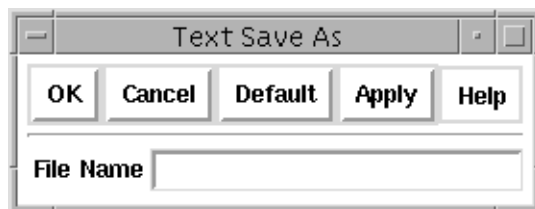
Select the types of groups for which you want to display information.

Groups Report



You can use the commands on this menu to save the text in the window to a file, load another file into the window, or close the window.

- *Save As* displays the *Text Save As* dialog box.



To save the text in the window, type a *File Name* and click *OK*. The tool saves the file to the name you type.

- *Close* closes the window.

geQueryGroupName

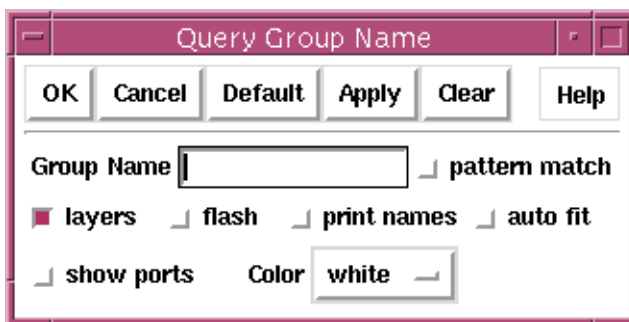
Highlights the nets in the given groups by the given group name.

Tools and Menu Commands

Astro: Query > Net – Query Group By Name

Query Group Name Dialog Box

The menu command opens the following dialog box. Its options are described below.



Group Name

Type the names of groups to highlight, or select *pattern match* and type a pattern to match the names of groups to highlight.

Pattern match

Select if you want to specify a pattern to match the names of the multiple groups you want to highlight.

Use pattern matching to select all groups with names that match a pattern. For more information about pattern matching see *Specifying Objects by Pattern*.

Layers

Select to draw the nets of the group(s) with the display attributes of the layers of its wires. In this case, the color you select is overlaid on the layer.

flash

Select if you want the nets in the group(s) to flash (blink).

Print names

Select if you want to display the net names and pins in the message area.

auto fit

Automatically change the window size to include all the group(s).

Show ports

Select if you want to highlight the ports.

color

Select one of 15 color choices you want for the net display.

geQueryNet

Highlights objects on a net you specify by clicking in the graphics window, and displays the net name in the messages area.

To remove the highlighting, click *Clear*.

Tools and Menu Commands

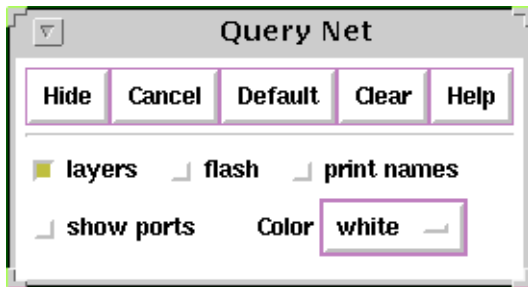
Astro: Query > Net – Query Net By Point

Jupiter: Query > Net – Query Net By Point

Milkyway: Query > Net – Query Net By Point

Query Net Dialog Box

The menu command opens the following dialog box. Its options are described below.



layers

Select to draw the net with the display attributes of the layers of its wires. In this case, the *Color* you select is overlaid on the layer.

flash

Select if you want the net to flash (blink).

print names

Select if you want to display the net name and pin in the messages area.

show ports

Select if you want to display the port name in the messages area.

Color

Select one of 15 colors you want for the net display.

geQueryNetConstraint

Highlights objects on a net for query by a specified name.

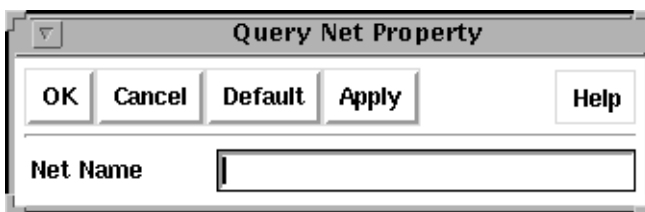
Tools and Menu Commands

Astro: Route > Constraint – Query Net Constraints or

Query > Net – Query Net Constraints

Query Net Constraint Dialog Box

The menu command opens the following dialog box. Its options are described below.



Net Name

Enter the name of the net that you want to highlight and query, or click on the net in the open cell that you wish to highlight and query.

geQueryNetName

Highlights objects on a net you specify by name.

Tools and Menu Commands

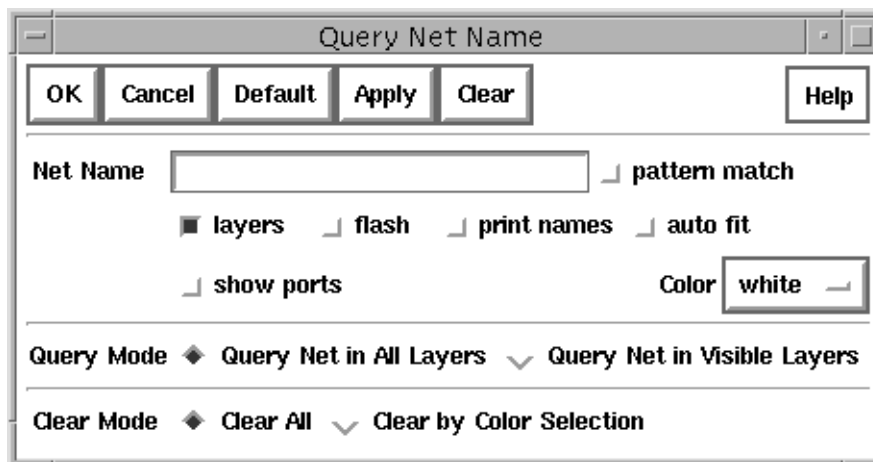
Astro: Query > Net – Query Net By Name

Jupiter: Query > Net – Query Net By Name

Milkyway: Query > Net – Query Net By Name

Query Net Name Dialog Box

The menu command opens the following dialog box. Its options are described below.



This table describes the buttons located at the top of the dialog box.

Button	Function
OK	Executes the command and closes the dialog box.
Cancel	Closes the dialog box without executing the command.
Default	Sets default values on all dialog box options.
Apply	Executes the command but does not close the dialog box.
Clear	Removes the highlighting of all nets.
Help	Initializes the online Help program for this command.

Net Name

Type the name of the net you want to highlight, or select *pattern match* and type a pattern to match the names of multiple nets.

pattern match

Select if you want to specify a pattern to match the names of multiple nets to highlight.

layers

Select to draw the net with the display attributes of the layers of its wires. In this case, the *Color* you select is overlaid on the layer.

flash

Select if you want the net to flash (blink).

print names

Select if you want to display the net name and pin in the messages area.

auto fit

Select to auto fit highlighted nets the in display.

show ports

Select to show ports of specified net name.

Color

Select one of 15 color choices you want for the net display.

Query Mode

The Query Mode options are as follows:

- Query Net in All Layers - You can see or clear the highlighting of all the queried net wires in all layers.
- Query Net in Visible Layers - You can see or clear the highlighting of all the queried net wires in only the visible layers. If you change the visibility of layers, you need to click Save or Save Vis&Sel button in the Layer Panel dialog box (see [geLayerPanel](#)) before you can query the visible layers.

Clear Mode

The Clear Mode options are as follows:

- Clear All - Clear highlighted nets.
- Clear by Color Selection - Clear highlighted nets by specified color.

geQueryObject

Displays information about an object you select by clicking on it in the graphics window. It will also show the net status indicating whether or not a net is a dummy net (through copy/push down).

Tools and Menu Commands

Astro: Query > Object – Query Object

Jupiter: Query > Object – Query Object

Milkyway: Query > Object – Query Object

geRedo

Re-does an action after you use *Modify > Undo* to undo the action.

Tools and Menu Commands

Astro: Modify > Redo

Jupiter: Astro Floorplanning: Edit > Redo

Milkyway: Modify > Redo

geRedraw

Refreshes the current window after an operation in order for you to see the results.

Tools and Menu Commands

Astro: Views > Redraw

Jupiter: Views > Redraw

Milkyway: Views > Redraw

geReShape

Redefines a portion of a path, rectangle, or polygon you select by clicking in the graphics window. You specify the change by entering points. To indicate that you are finished entering points, click the right mouse button. To select the correct interpretation of the redefined portion, click *Toggle* until the intended change is displayed.

Tools and Menu Commands

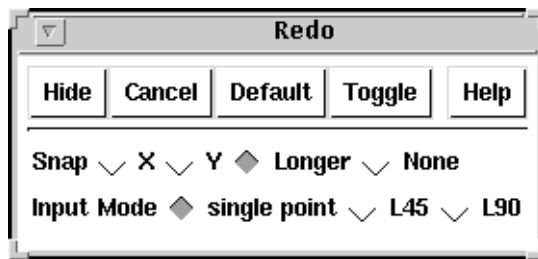
Astro: Modify > Redo

Jupiter: Astro Floorplanning: Edit > Redo

Milkyway: Modify > Redo

Redo Dialog Box

The menu command opens the following dialog box. Its options are described below.



Snap Shape

Select the shape you want for redrawing the object.

Table 36-20

Option	Redraws the object ...
edge	First in the direction of the previous segment, then in the alternate direction. For the first segment, this option uses <i>L90</i> .
L90	With a horizontal segment and a vertical segment, using the specified <i>Snap Rule</i> .
L45	With a 45-degree segment, then a horizontal or vertical segment.
manhattan	With only one segment in the direction (horizontal or vertical) that is of a greater distance (between the two points).
straight	With only one segment of any angle (does not have to be horizontal or vertical).

Snap Rule

Table 36-21

Option	Draws the object ...
X	First in the horizontal direction, then in the vertical direction.
Y	First in the vertical direction, then in the horizontal direction.

Table 36-21

Option	Draws the object ...
longer	First in the direction (horizontal or vertical) that is of a greater distance (between the two points). For example, if the distance between the first point and the second point is 5 in the horizontal direction and 7 in the vertical direction, the your Synopsys tool draws the first segment in the vertical direction and the second segment in the horizontal direction.

Select the rule you want to use when the *Snap Shape* is *L90*.

geRollbackCell

Replaces the current cell with the last version of the cell saved to the disk. This discards any changes made since you last saved the cell.

Tools and Menu Commands

Astro: Cell > Refresh

Jupiter: Cell > Refresh

Milkyway: Cell > Refresh

geRuler

Provides a ruler for measuring distances in your design. By clicking in the graphics window, indicate the two points between which you want to measure the distance.

To remove the ruler display:

- If you created the ruler display with the *keep* option deselected, redraw the screen using [geRedraw](#).
- If you created the ruler display with the *keep* option selected, do one of the following:
 - To remove all rulers, click *Clear*.
 - To remove individual rulers, use [geDeleteRuler](#).

Tools and Menu Commands

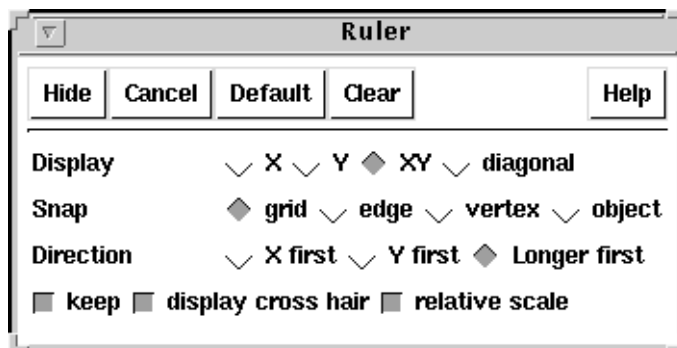
Astro: Query > Ruler – Add Ruler

Jupiter: Query > Ruler

Milkyway: Query > Ruler – Add Ruler

Ruler Dialog Box

The menu command opens the following dialog box. Its options are described below.



Display

Select the direction in which you want to display the ruler (and measure the distance).

Table 36-22

Option	Measures ...
X	A horizontal distance.
Y	A vertical distance.
XY	Both a horizontal and a vertical distance.
diagonal	A distance other than a horizontal or vertical distance.

Snap

Table 36-23

Option	Snaps the ruler to ...
grid	The grid, which is defined in the technology file.
edge	The edge of the nearest visible object.
vertex	The vertex of the nearest visible object.

Table 36-23

Option	Snaps the ruler to ...
object	The center of the nearest visible object if that object is a wire, path, or contact.

Select the point to which you want the ruler to snap.

Note:

You can control the visibility of an object using [geWindowOption](#).

Direction

Select the direction you want for the first segment of the ruler when *Display* is set to *XY*.

Table 36-24

Option	Draws ...
X first	The horizontal segment of the ruler first.
Y first	The vertical segment of the ruler first.
longer first	The longer segment of the ruler first. For example, if the distance between the first point and the second point is 5 in the horizontal direction and 7 in the vertical direction, your Synopsys tool draws the first segment in the vertical direction and the second segment in the horizontal direction.

keep

Select if you want the ruler display to remain until you close the cell, or delete the display by clicking *Clear* or using [geDeleteRuler](#). Ruler displays are not saved with a cell. If you deselect *keep*, you can remove the ruler display using [geRedraw](#).

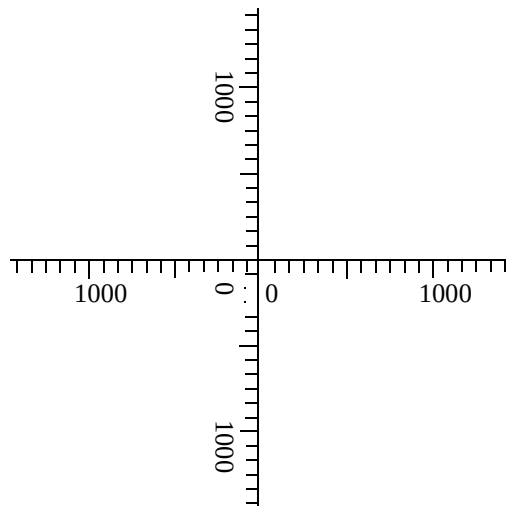
display cross hair

Select if you want to display a crosshair cursor when the cursor is in the graphics window. The cursor is labeled as a ruler.

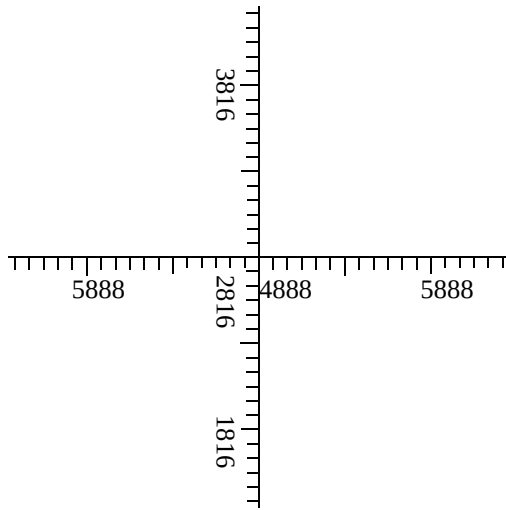
relative scale

Select if you want the ruler labeled using a scale relative to zero.

- If you select *relative scale*, the start point of the ruler (and center of the crosshair cursor) is always labeled zero (0), with the other marks on the ruler labeled relative to zero.



- If you deselect *relative scale*, each mark on the ruler is labeled according to the coordinate beneath it.



See Also

[geDeleteRuler](#) -- deletes a ruler display.

geSaveAllOpenCells

Saves all open and modified cells in the current working library. A dialog box appears for your confirmation. All cells remain open after saving.

Tools and Menu Commands

Astro: Cell > Save All Open Cells

Jupiter: Cell > Save All Open Cells

Milkyway: Cell > Save All Open Cells

Syntax

`geSaveAllOpenCells`

geSaveAs

Saves the cell in the current window with a name you specify. This leaves the version of the cell on the disk unchanged and creates a new cell with the new name.

Tools and Menu Commands

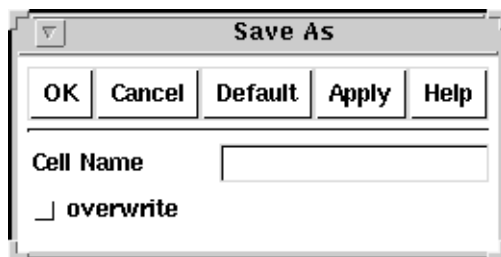
Astro: Cell > Save As

Jupiter: Cell > Save As

Milkyway: Cell > Save As

Save As Dialog Box

The menu command opens the following dialog box. Its options are described below.



Cell Name

Type the name you want to assign to the new cell.

overwrite

Select to overwrite any existing cell with the same name.

geSelectAll

Highlights and selects all objects in the current window.

Tools and Menu Commands

Astro: Select > Select – Select All

Jupiter: Select > Select – Select All

Milkyway: Select > Select – Select All

geSelectInstancesOnNet

Lets you select all instances on a given net.

Tools and Menu Commands

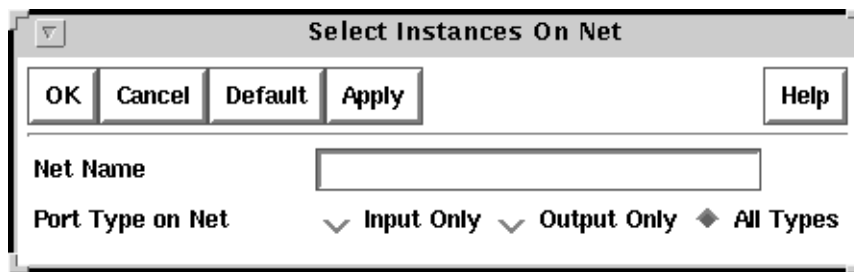
Astro: Select > Select Insts On Net

Jupiter: Select > Select Insts On Net

Milkyway: Select > Select Insts On Net

Select Instances On Net Dialog Box

The menu command opens the following dialog box. Its options are described below.



Net Name

Type the name of the net on which you want to select all instances.

Port Type on Net

Select the type of port on the net you want to select.

Table 36-25

Option	Selects ...
Input Only	Instances that have an input port on the net you select. This option does not include I/O ports.
Output Only	Instances that have an output port on the net you select. This option does not include I/O ports.
All Types	Instances that have a port of any type on the net you select. This option includes I/O ports.

geSelectObject

Selects objects from an open window.

Syntax

```
geSelectObject windowNumber objectId
```

where the arguments are as follows:

windowNumber

Number of the window from which you select objects

Valid values:

Any positive integer corresponding to an open window

objectId

Database ID of the object you want to select

Valid values:

ID of any object in the selected window

Example

```
geSelectObject 1  
(dbGetCellInstByName (geGetEditCell) "inst1")  
#t
```

In this example, Astro selects the instance inst1.

geSetCellInstColor

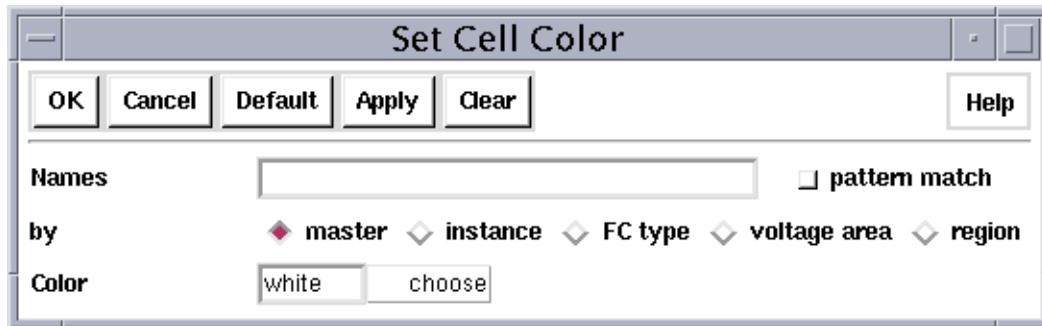
Draws specified cell instances in specified colors.

Tool

Milkyway

Set Cell Color Dialog Box

The menu command opens the following dialog box. Its options are described below.



Names

Type the names or name patterns of the cell instances that you want to set color to. Names or name patterns are separated by a comma ‘,’.

pattern match

Select this option to locate net names that match the pattern typed in the *Names* field.

Use the pattern matching option to select all nets with names that match a pattern.

by

Select the type of cells you want to color that belong to your choice (master, instance, FC type, voltage area, or region).

For example, when you select “voltage area,” the specified color is applied to all the cell instances that are assigned to the voltage area you created (see `aprCreateVoltageArea`).

Color

Select the color you want for the cell instance display.

geSetConfirmPinDeletionFlag

This function will set the “Confirm Pin Deletion” flag to a specified value. If you set it to *true*, you will be prompted for confirmation before the pin is deleted using the Scheme function [geDelete](#). If you set it to *false*, you will not be prompted for confirmation, and the pin will be deleted. The flag is initially set to *false*.

Tool

Milkyway

Syntax

```
geSetConfirmPinDeletionFlag booleanFlag
```

where the argument is as follows:

booleanFlag A Scheme boolean (either #t or #f).
g

geSetCustomRouteTypesVis

Allows you to query and change the visibility status of all custom route types. If no arguments are specified, `geSetCustomRouteTypesVis` prints out the visibility status of all custom route types.

Syntax

```
geSetCustomRouteTypesVis '(routetype1 routetype2...) #t/#f
```

where the arguments are as follows:

<i>routeType1, routeType2, ...</i>	Specifies the custom route type number.
	The following custom route type numbers are valid: {10 12 24 26 28 30 38 40 42 44 46}
<i>#t</i>	Enables the visibility of the specified custom route types.
<i>#f</i>	Disables the visibility of the specified custom route types.

geSetErrorType

Sets the types of errors you want to examine.

Tools and Menu Commands

Astro: Verify > Set Error Type – Set Types to View

Prerequisites

Open the cell open and load the error cell. If you used the default error cell name when running DRC or LVS, the Synopsys tool automatically loads the error cell. To manually load an error cell, use [geLoadErrorCell](#).

Set Error Type Dialog Box (DRC)

The menu command opens the following dialog box. Its options are described below.

Set Error Type

OK Cancel Default All On All Off Help

Visible Type ☐ VIA3&VIA4 OVRLP ☐ VIA3&VIA5 OVRLP ☐ VIA3

☐ VIA4&VIA5 OVRLP ☐ VIA4 ☐ VIA5

☐ MAX STACK ☐ POLY WIDTH ☐ POLY&POLY SEP

☐ POLY NOTCH ☐ POLY&BLKG OVRLP ☐ MET1 WIDTH

☐ MET1&MET1 SEP ☐ MET1 NOTCH ☐ METAL1 FATWIRE

☐ MET1&BLKG OVRLP ☐ MET2 WIDTH ☐ MET2&MET2 SEP

☐ MET2 NOTCH ☐ METAL2 FATWIRE ☐ MET2&BLKG OVRLP

☐ MET3 WIDTH ☐ MET3&MET3 SEP ☐ MET3 NOTCH

☐ METAL3 FATWIRE ☐ MET3&BLKG OVRLP ☐ MET4 WIDTH

☐ MET4&MET4 SEP ☐ MET5 NOTCH ☐ METAL4 FATWIRE

☐ MET4&BLKG OVRLP ☐ MET5 WIDTH ☐ MET5&MET5 SEP

☐ MET5 NOTCH ☐ MET5&BLKG OVRLP ☐ MET6 WIDTH

☐ MET5&MET5 SEP ☐ MET6 NOTCH ☐ MET6&BLKG OVRLP

☐ CONT WIDTH ☐ CONT&CONT SEP ☐ CONT NOTCH

☐ CONT&BLKG OVRLP ☐ VIA1 WIDTH ☐ VIA1&VIA1 SEP

☐ VIA1 NOTCH ☐ VIA1&BLKG OVRLP ☐ VIA2 WIDTH

☐ VIA2&VIA2 SEP ☐ VIA2 NOTCH ☐ VIA2&BLKG OVRLP

☐ VIA3 WIDTH ☐ VIA3&VIA3 SEP ☐ VIA3 NOTCH

☐ VIA3&BLKG OVRLP ☐ VIA4 WIDTH ☐ VIA3&VIA4 SEP

☐ VIA4 NOTCH ☐ VIA4&BLKG OVRLP ☐ VIA5 WIDTH

☐ VIA3&VIA5 SEP ☐ VIA5 NOTCH ☐ VIA5&BLKG OVRLP

☐ CONT&VIA1 SEP ☐ CONT&VIA1 OVRLP ☐ CONT&VIA2 OVRLP

☐ CONT&VIA3 OVRLP ☐ CONT&VIA4 OVRLP ☐ CONT&VIA5 OVRLP

☐ CONT ☐ VIA1&VIA2 OVRLP ☐ VIA1&VIA3 OVRLP

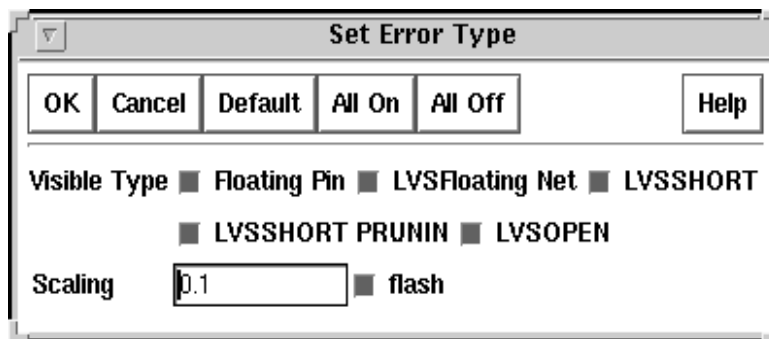
☐ VIA1&VIA4 OVRLP ☐ VIA1&VIA5 OVRLP ☐ VIA1

☐ VIA2&VIA3 OVRLP ☐ VIA2&VIA4 OVRLP ☐ VIA2&VIA5 OVRLP

☐ VIA2

Scaling ☐ flash

(LVS)



Visible Type

Select the type(s) of errors you want to examine.

Scaling

Type the scaling factor you want to use for viewing errors individually. The smaller the scaling factor, the closer you zoom into the error. The larger the scaling factor, the more area you see around the error.

flash

Select if you want the errors to flash (blink).

geSetEnvValue

Sets the following environment variables for a layout window.

Syntax

```
geSetEnvValue windowNumber "envName" value
```

where the arguments are as follows:

windowNumber

Number of the layout window for which you want information about the environment variables

Valid values:

Any positive integer corresponding to an open window

envName

Name of the environment variable for which you want the current setting

Valid values: See the following sections.

value

Depends on the value of *envName*. See the following sections.

If `geSetEnvValue` is successful, it returns *#t*. If `geSetEnvValue` fails, it returns *nil* or *()*.

Note:

The `InfixMode` and `CursorLocation` variables are unsupported.

Environment Variables That Are Unique to Each Window

The environment variables in the following list are unique for each window. For example, the same cell can be viewed in two windows at different depths.

- `FilterSize` controls whether or not all objects are visible. Any object smaller than `FilterSize` is not drawn. `FilterSize` is a percent of the screen size.
- `ViewLevels` indicates the number of levels of hierarchy displayed in the window during drawing.

Environment Variables That Are Unique to Each Cell

The environment variables in the following list are unique for each cell. For example, two windows containing the same cell share the value for these environment variables.

- `ViewList` controls which cell master is displayed when viewing levels are deeper than 0.
- `YMajorGridSpacing`, `XMajorGridSpacing`, `YGridSpacing`, `XGridSpacing`, `set grid options`.

Note:

When a library is opened, this value is set to the minimum grid specified in the technology file for the library.

- `YSnapGrid` and `XSnapGrid` specify the grid to which the cursor snaps.
- `UseLocalName` controls whether commands use the hierarchical or flat name for nets and cell instances.

Valid values:

- `#t` requires you to type the flat name for cell instances or nets in all cell instance or net name fields.
- `#f` requires you to type the hierarchical name for cell instances or nets in all cell instance or net name fields.
- `DisplayMajorGrid` and `DisplayGrid` are boolean values that specify the display options for grids.

Valid values:

- `#t` displays the grid or major grid.
- `#f` blocks display of the grid or major grid.

Environment Variables That Are Global and Apply to All Windows

The environment variables in the following table are global environment variables and apply to all windows. For example, if `ShowPathOutlines` is turned off, all windows will block display of path outlines.

- `MaxDragObjects` specifies the maximum number of objects that are real-time displayed during certain data modification commands. A value of 0 implies no limit.
- `SelectionAperture`, which is a percent of the screen size, controls the maximum distance to consider when you select objects.
- `SnapAperture` controls the maximum distance a coordinate can be moved. This environment variable is currently unused.

- `SnapMode` is an environment variable used by data entry commands.
- `SnapToGrid` is a boolean value that controls whether the grid is visible.

Valid values:

- `#t` blocks display of the grid.
- `#f` displays of the grid or major grid.

Note:

Some commands display a grid regardless of the value of `SnapToGrid`.

- `ShowPathOutline` and are boolean values that control the display of paths.

Valid values for `ShowPathOutline`:

- `#t` displays the outline of a path.
- `#f` blocks display of the outline of a path.

Valid values for `ShowPathCenter`:

- `#t` displays the center of a path.
- `#f` blocks display of the center of a path.

geSetLayerSelectable

Specifies whether or not a layer can be selected.

Syntax

```
geSetLayerSelectable windowNumber layerNameOrNumber select?
```

where the arguments are as follows:

windowNumber

Number of the layout window containing the layer for which you want information

Valid values:

Any positive integer corresponding to an open window

Note:

If *windowNumber* is 0, `geIsLayerSelectable` returns the default settings that will be applied to new windows.

layerNameOrNumber

Layer for which you want to set selection options

Valid values:

select?

Whether or not you can select the layer

Valid values:

- *#t* means you can select the layer.
- *#f* means you cannot select the layer.

If `geSetLayerSelectable` is successful, it returns *#t*. If `geSetLayerSelectable` fails, it returns *nil* or *()*.

geSetLayerVisible

Specifies whether or not a layer is visible.

Syntax

```
geSetLayerVisible windowNumber layerNameOrNumber visible?
```

where the arguments are as follows:

windowNumber

Number of the layout window containing the layer for which you want information

Valid values:

Any positive integer corresponding to an open window

Note:

If *windowNumber* is 0, `geIsLayerVisible` returns the default settings that will be applied to new windows.

layerNameOrNumber

Layer for which you want to set visibility options

Valid values:

select?

Whether or not the layer is visible

Valid values:

- *#t* means the layer is visible.
- *#f* means the layer is not visible.

If `geSetLayerSelectable` is successful, it returns *#t*. If `geSetLayerSelectable` fails, it returns *nil* or *()*.

geSetObjectSelectable

Specifies whether or not an object can be selected.

Note that an object can be selected only if both of the following are true:

- The type of the object is selectable.
- The layer of the object is selectable.

Syntax

```
geSetObjectSelectable windowNumber "ObjectType" select?
```

where the arguments are as follows:

windowNumber

Number of the layout window containing the objects for which you want information

Valid values:

Any positive integer corresponding to an open window

Note:

If *windowNumber* is 0, `geIsObjectSelectable` returns the default settings that will be applied to new windows.

objectType

Type of object for which you want to set selection options

Valid values:

select?

Whether or not you can select the object type

Valid values:

- *#t* means you can select the object type.
- *#f* means you cannot select the object type.

If `geSetObjectSelectable` is successful, it returns *#t*. If `geSetObjectSelectable` fails, it returns *nil* or *()*.

geSetObjectVisible

Specifies whether or not an object is visible.

Note that an object is visible only if both of the following are true:

- The type of the object is visible.
- The layer of the object is visible.

Syntax

```
geSetObjectVisible windowNumber "objectType" visible?
```

where the arguments are as follows:

windowNumber

Number of the layout window containing the objects for which you want information

Valid values:

Any positive integer corresponding to an open window

Note:

If *windowNumber* is 0, `geIsObjectVisible` returns the default settings that will be applied to new windows.

objectType

Type of object for which you want to set visibility options

Valid values:

select?

Whether or not the object type is visible.

Valid values:

- *#t* means the object type is visible.
- *#f* means the object type is not visible.

If `geSetObjectVisible` is successful, it returns *#t*. If `geSetObjectVisible` fails, it returns *nil* or *()*.

geSetNetTrace

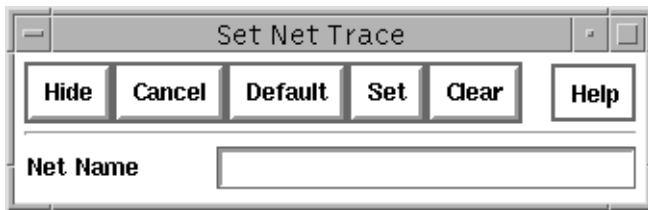
This command is written to conveniently set/unset net tracing with a GUI. It is normally used after setting the un-traced nets to “dim” with the `geTraceMode` command.

Tool

Milkyway

Set Net Trace Dialog Box

The menu command opens the following dialog box. Its options are described below.



Net Name

Shows the selected net name when the mouse clicks on a net on the view port.

Set

Select to set the net tracing.

Clear

Resets the net tracing.

Nets not traced

- visible - When this option is selected on, the nets that are not traced are visible.
- selectable - When this option is selected on, the nets that are not traced are selectable.
- dimmed - When this option is selected on, the nets that are not traced are dimmed.

Apply

Click to apply the selection of the three check buttons.

Clear All Net Trace

Click to clear all traced nets.

geSetNetTypesVis

Allows you to query and change the visibility status of paths, wires, contacts, and contact arrays by net type. If no arguments are specified, `geSetNetTypesVis` prints out the current visibility status.

Syntax

```
geSetNetTypesVis '(netTypeName1 netTypeName2...) #t/#f
```

where the arguments are as follows:

netTypeName1,
netTypeName2, ...

Specifies the net type.

The following net types are valid:

- Unknown
- Power
- Ground
- Signal
- Clock
- Split clock
- Split clock & clock

#t

Enables the visibility of the specified net types.

#f

Disables the visibility of the specified net types.

geSetViewCell

Overlays the gap and notch cells onto your design cell so that you can view in the graphics window the gap and notch filling information which is generated for a specified cell by geNewFILLING.

Note:

This functions of this command also can be exercised using the dialog box command [geImposeCell](#).

Tool

Milkyway

Prerequisites

Generate gap and notch filling information for filling gaps and notches that are smaller than the minimum distance required between objects on the same layer, using geNewFILLING.

Syntax

```
geSetViewCell "cellName" visible
```

where the arguments are as follows:

<i>cellName</i>	The name of the cell whose gap or notch information you wish to view. Valid values: Any gap or notch cell containing gap or notch information that you want to make visible.
-----------------	--

visible Whether or not you would like gap and notch information to be visible in the graphics window.

Valid boolean values:

- *#t* tells the tool to make gap and notch information visible.
- *#f* tells the tool to not make gap and notch information visible.

geShowCellList

Displays a cell browser window which stays visible while you use other commands. You can display the cells of any library in the directory.

Tools and Menu Commands

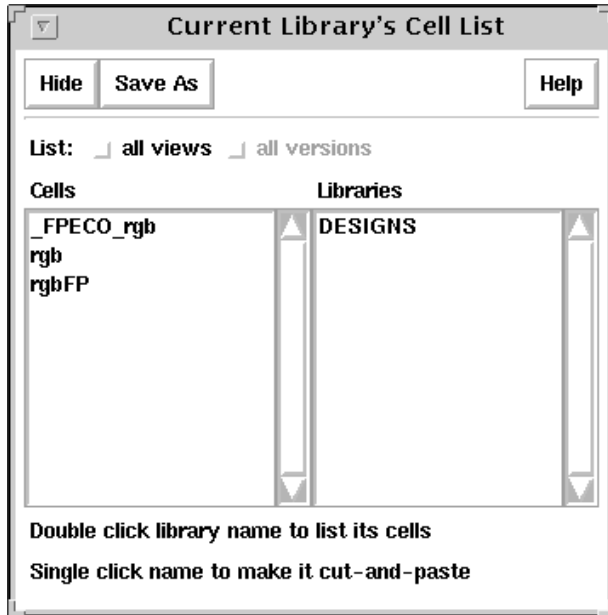
Astro: Library > Show Cells

Jupiter: Library > Show Cells

Milkyway: Library > Show Cells

To close the dialog box, click the Hide button or enter *geCloseCellList*.

Current Library's Cell List



List: all views

Select to list all the cell views in the current library. By default, the browser displays the *.CEL* views of the cells in the design library. If you select *all views*, the browser displays all views, such as *.NETL*, *.CEL*, *.Exp*, *GAP*, *.err*, and *.NOTC* cell views.

List: all versions

Select to list all the versions of the cells in the current library.

geSmashCellInst

Smashes the currently selected cell instance(s) to a specified level of hierarchy.

Tools and Menu Commands

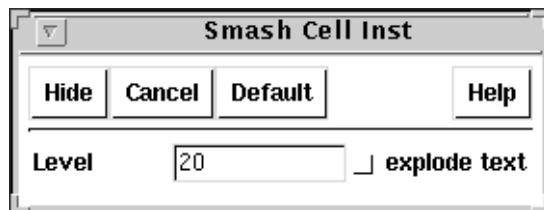
Astro: Modify > Smash – Smash Cell Instance

Jupiter: Astro Floorplanning: Edit > Smash Cell – Instance

Milkyway: Modify > Smash – Smash Cell Instance

Smash Cell Inst Dialog Box

The menu command opens the following dialog box. Its options are described below.



Level

Type the number of levels of hierarchy to which you want to smash the selected cell instance.

explode text

Select to retain the text at all levels of hierarchy that are smashed.

See Also

[cmSmash](#) smashes a specified master cell down to a specified level.

geSplit

Splits path segments, polygons, and rectangles intersected by a line you specify by entering two points.

Tools and Menu Commands

Astro: Modify > Split

Jupiter: Edit > Split

Milkyway: Modify > Split

geStretch

Lets you stretch an object. The `geStretch` command allows the continuous stretch of other objects after stretching the cell boundary.

Note:

This command prompts you to select the object you want to stretch. If you have objects selected when you issue the command, you are restricted to those selected objects.

Caution!

This command is not used to stretch the core object. To stretch the core object, use the `fphStretchCore` command.

Tools and Menu Commands

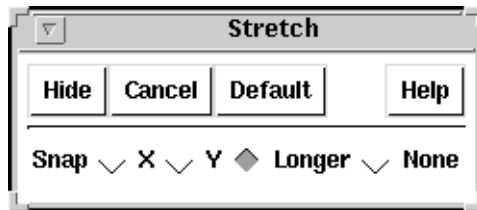
Astro: Modify > Stretch

Jupiter: Astro Floorplanning: Edit > Stretch

Milkyway: Modify > Stretch

Stretch Dialog Box

The menu command opens the following dialog box. Its options are described below.



Snap

Table 36-26

Option	Lets you stretch the object ...
X	Horizontally.
Y	Vertically.
Longer	In the direction in which you move the cursor the farthest. For example, if you move the cursor farther horizontally than vertically, the object stretches in the horizontal direction.

Select the direction in which you want to stretch the object.

geSummary

Reports a summary for selected objects. The summary report contains information such as the following: the number of rectangles, polygons, paths, wires, and texts on the specified layer; the total number of cell instances, cell instance arrays, contact, contact arrays, and pins in the selected object's set; and the fit bounding box of the set.

Syntax

`geSummary widgetId layerNumber`

where the arguments are as follows:

<i>widgetId</i>	Specifies the ID of the layout window.
<i>layerNumber</i>	Specifies the number of the layer associated with the selected object for which you want the summary report.
	Valid layer numbers: 0 - 255 (-1 specifies all layers)

geSuppressHltRedraw

When an object is highlighted, the cell window is redrawn. Use this command to suppress this redrawing. You can reduce time when a list of objects is highlighted by ID, then the cell window is redrawn only once. For example:

```
hilite obj1
redraw
hilite obj2
redraw
....
```

This command highlights all objects then redraws them. For example,

```
hilite obj1
hilite obj2
...
redraw
```

Tool

Milkyway

Syntax

`geSuppressHltRedraw #t/#f`

where the arguments are as follows:

`#t` to suppress the redraw.

`#f` to cancel suppress redraw.

geSwapCells

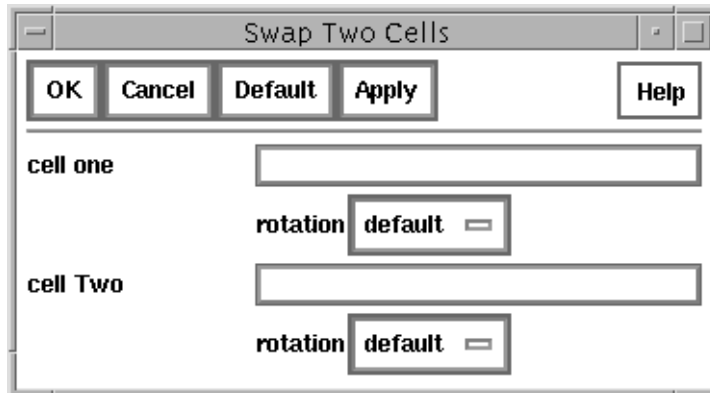
Allows you to swap the locations of two selected cells.

Tool

Milkyway

Swap Two Cells Dialog Box

The menu command opens the following dialog box. Its options are described below.



cell one

Type the name of the first selected cell to swap with the second.

cell two

Type the name of the second selected cell to swap with the first.

geTransform

Lets you move and rotate/flip a specified object.

Tools and Menu Commands

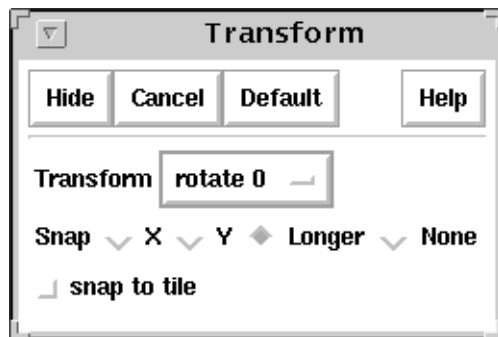
Astro: Modify > Transform

Jupiter: Modify > Transform

Milkyway: Modify > Transform

Transform Dialog Box

The menu command opens the following dialog box. Its options are described below.



Transform

Select the orientation you want for placement of the object.

Table 36-27

Option	Adds the object ...
rotate 0	Without changing its orientation.
rotate 90	Rotated counterclockwise 90 degrees.
rotate 180	Rotated counterclockwise 180 degrees.
rotate 270	Rotated counterclockwise 270 degrees.
mirror X	Flipped on the y axis (in the X direction).
mirror Y	Flipped on the x axis (in the Y direction).
rotate 90 mirror X	Rotated 90 degrees and flipped on the y axis (in the X direction).
rotate 90 mirror Y	Rotated 90 degrees and flipped on the x axis (in the Y direction).

Snap

Select the direction in which you want to move the object.

Table 36-28

Option	Lets you move the object ...
X	Horizontally.
Y	Vertically.
Longer	In the direction in which you move the cursor the farthest. For example, if you move the cursor farther horizontally than vertically, the object moves in the horizontal direction.
None	Without any snap applied.

snap to tile

Select to snap the object to a unit tile.

geTurnErrorOff

Removes highlighting from the errors displayed in a DRC or LVS error cell.

Tools and Menu Commands

Astro: Verify > Init Errors – Turn Off Errors

getXWindowId

Retrieves the X window ID of the given layout window.

Syntax

`getXWindowId widgetId`

where the argument is as follows:

<i>widgetId</i>	Specifies the ID of the layout window.
-----------------	--

geUndo

Undoes the operation performed by the last command executed, if that command is one of the following:

- [geMove](#)
- [geCopy](#)
- [geDelete](#)
- [aprAssignNet](#)
- [geModify](#)
- [geMovePoint](#)
- [geRedo](#)
- [ioReplacePads](#)
- [geSplit](#)
- [geStretch](#)
- [geWindowStretch](#)
- [geTransform](#)
- [geUndo](#)

Tools and Menu Commands

Astro: Modify > Undo

Jupiter: Modify > Undo

Milkyway: Modify > Undo

geUnpackCellInstArray

This command is used to unpack selected cell instance array(s) into cell instances.

Tools and Menu Commands

Astro: Modify > Unpack Cell Inst Array

Milkyway: Modify > Unpack Cell Inst Array

geViewDown

Moves the display in the graphics window down by half.

Note:

On Sun workstations, the default bind key for this function is the down arrow.

Tools and Menu Commands

Astro: Views > Down

Jupiter: Views > Down

Milkyway: Views > Down

geViewLeft

Moves the display in the graphics window left by half.

Note:

On Sun workstations, the default bind key for this function is the left arrow.

Tools and Menu Commands

Astro: Views > Left

Jupiter: Views > Left

Milkyway: Views > Left

geViewRight

Moves the display in the graphics window right by half.

Note:

On Sun workstations, the default bind key for this function is the right arrow.

Tools and Menu Commands

Astro: Views > Right

Jupiter: Views > Right

Milkyway: Views > Right

geViewUp

Moves the display in the graphics window up by half.

Note:

On Sun workstations, the default bind key for this function is the up arrow.

Tools and Menu Commands

Astro: Views > Up

Jupiter: Views > Up

Milkyway: Views > Up

geWindowDeselect

Lets you deselect objects by enclosing them with a rectangle.

Tools and Menu Commands

Astro: Select > Deselect – Deselect By Window

Jupiter: Select > Deselect – By Window

Milkyway: Select > Deselect – Deselect By Window

geWindowOption

Allows you to change the window options for the current window (and the default options for windows you subsequently open during the current Synopsys session). After changing the options, redraw the window by clicking Redraw in the Window Options dialog box.

When setting the visibility and selectability of objects, you can use the buttons at the top of the dialog box as follows:

- To make all object types visible, click All Vis On.
- To make all object types invisible, click All Vis Off.
- To make all object types selectable, click All Sel On.
- To make all object types unselectable, click All Sel Off.
- To filter out wires and contacts of any route types for viewing or selection, click Route Type.

Tool and Menu Command

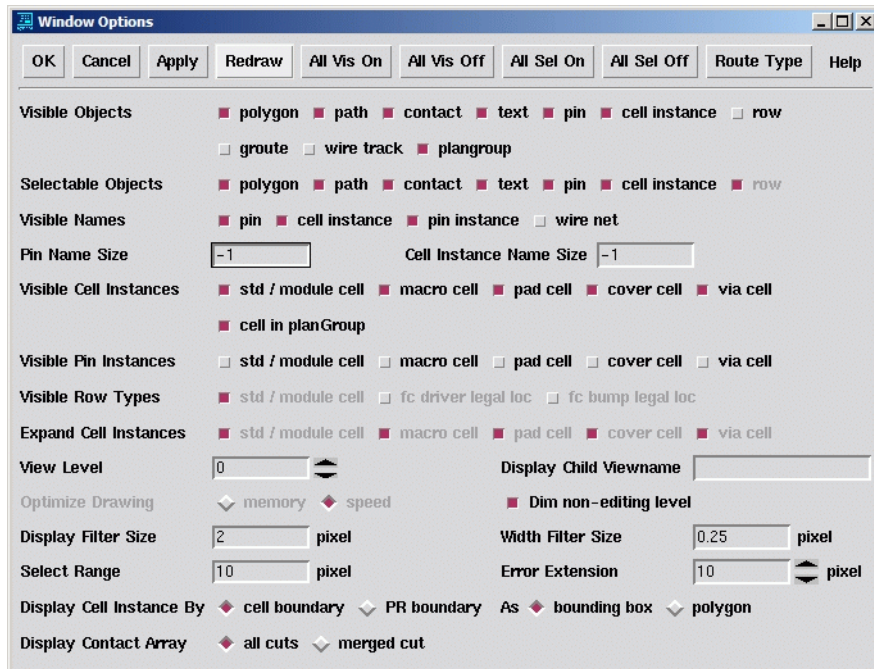
Astro: Options > Window

Jupiter: Options > Window

Milkyway: Options > Window

Window Options Dialog Box

The command opens the following dialog box. Its options are described below.



Visible Objects

Select the types of objects you want to display in the window.

Selectable Objects

Select the types of objects you want to be able to select.

Visible Names

Select the types of names you want to display in the window.

Options include

- pin

Turns the pin name on and off. This option is on by default.

- cell instance

Turns the cell instance name on and off. This option is on by default.

- pin instance

Turns the pin instance name on and off. This option is on by default.

- wire net

Turns the wire net names display on and off. The net name text stays within the width of the wire and within the viewable area. This option is off by default.

Pin Name Size

Controls the pin name font size.

Cell Instance Name Size

Scales the cell instance name. Enter a value to customize the text size. The smallest size you can specify is 0.001 if the length precision is 1000. (The length precision is defined in the technology file.) The name size is set to -1 by default.

Visible Cell Instances

If you chose to make cell instance names visible, select the types of cell instances you want to display in the window.

Visible Pin Instances

If you chose to make pin instance names visible, select the types of pin instances you want to display in the window.

View Level

The number of hierarchy levels you want to display. The default is 0, which displays only the top level.

Optimize Drawing

Optimizes the window redrawing for memory or for speed.

Display Child Viewname

The extension of the child cell names you want to display. For a design in which you bound the EDIF netlist and the GDSII Stream layout, the default is the pin/blockage cell (.fram). For a design in which you imported the GDSII Stream data but did not perform the binding process, the default is the layout cell (.cel).

Display Filter Size

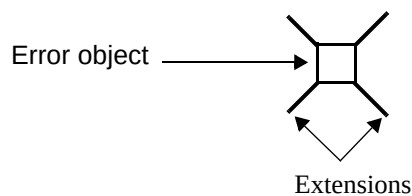
The minimum screen pixel size of an object that you want the tool to display. For example, if the filter size is 2, the Synopsys tool displays only objects that are equal to or greater than 2 pixels on your display monitor.

Select Range

The distance around a point you specify within which you want all objects to be selected. The distance from the point is measured in pixels.

Error Extension

The number of pixels by which you want to extend error objects so that they can be more easily visible.



Display Cell Instance By

Select the display style of the cell instance.

Option	Description
cell boundary	Displays the cell instance with the boundary as stored in the layout cell (.cel).
PR boundary	Displays the cell instance with the place and route boundary.

As

Select the shape you want to use to display cell instances.

Option	Description
bounding box	Displays cell instances with rectangles outlining the boundaries. If you select this option, the tool displays rectilinear cell instances as rectangles.
polygon	Displays cell instances with polygons outlining the boundaries. If you select this option, the tool displays rectilinear cell instances with their actual shapes.

Display Contact Array

Displays contact arrays for all of the cuts of the contact array or for only a single merged cut.

geWindowSelect

Allows you to select objects by enclosing them with a rectangle.

Tools and Menu Commands

Astro: Select > Select – Select By Window

Jupiter: Select > Select – By Window

Milkyway: Select > Select – Select By Window

geWindowStretch

Stretches or moves paths, polygons, and rectangles in a rectangular area. After you specify diagonal corners of the area by entering points, you pick a reference point and a destination point to indicate the direction and extent of the stretch or move.

If an object is partially inside the area, the Synopsys tool stretches the object in the direction of the destination point. If an object is entirely inside the area, the Synopsys tool moves the object in the direction of the destination point.

Tools and Menu Commands

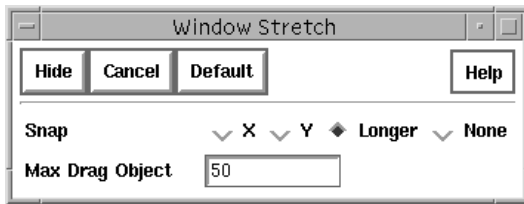
Astro: Modify > Window Stretch

Jupiter: Astro Floorplanning: Edit > Stretch – By Window

Milkyway: Modify > Window Stretch

Window Stretch Dialog Box

The menu command opens the following dialog box. Its options are described below.



Snap

Select the direction in which you want to stretch the objects in the specified area.

Table 36-29

Option	Lets you stretch the objects ...
X	Horizontally from the original object.
Y	Vertically from the original object.
Longer	In the direction in which you move the cursor the farthest. For example, if you move the cursor farther horizontally than vertically, the object moves in the horizontal direction.
None	Without any snap applied.

Max Drag Object

Type the maximum number of objects you want your Synopsys tool to drag. If the specified area contains more than the maximum number of objects, your Synopsys tool chooses objects partially inside the area before those entirely inside the area.

geZoomIn

Zooms into an area in the graphics window that you specify by entering diagonal corners of a rectangle.

Tools and Menu Commands

Astro: Views > Zoom In

Jupiter: Views > Zoom In

Milkyway: Views > Zoom In

geZoomInBy2

Zooms into the graphics window by a factor of 2.

Tools and Menu Commands

Astro: Views > Zoom In X2

Jupiter: Views > Zoom In X2

Milkyway: Views > Zoom In X2

geZoomOutBy2

Zooms out in the graphics window by a factor of two (2).

Tools and Menu Commands

Astro: Views > Zoom Out X2

Jupiter: Views > Zoom Out X2

Milkyway: Views > Zoom Out X2

geZoomOutBy4

Zooms out in the graphics window by a factor of four (4).

Tools and Menu Commands

Astro: Views > Zoom Out X4

Jupiter: Views > Zoom Out X4

Milkyway: Views > Zoom Out X4

geZoomOutBy8

Zooms out in the graphics window by a factor of eight (8).

Tools and Menu Commands

Astro: Views > Zoom Out X8

Jupiter: Views > Zoom Out X8

Milkyway: Views > Zoom Out X8

setInstViewLevel

Allows you to query and set the view level and view name of any cell instance in the open design.

Syntax

```
setInstViewLevel cellInstanceName viewName viewLevel
```

where the arguments are as follows:

<i>cellInstanceName</i>	Specifies the name of the cell instance for which you want to set the view level and view name.
-------------------------	---

<i>viewName</i>	Specifies the name of the view you want to set.
-----------------	---

Valid view names:

- CEL
- FRAM

Use " " to specify the default view name.

<i>viewLevel</i>	Specifies the view level you want to set.
------------------	---

Valid view levels:

0 - 256

Usually a design has 5 to 6 levels.

addRelativePoint

Emulates a mouse click when an editing command has a specified starting point.

Syntax

```
addRelativePoint widgetId '(x y)
```

where the arguments are as follows:

<i>widgetId</i>	Specifies the ID of the widget.
-----------------	---------------------------------

<i>(x y)</i>	Specifies the distance on the x- and y- axis, respectively.
--------------	---

layoutWindowGrabFocus

Allows you to query and configure whether to grab the focus from a window launched by key binding back to the layout window. If no arguments are specified, `layoutWindowGrabFocus` returns the current configuration.

Syntax

```
layoutWindowGrabFocus #t/#f
```

where the arguments are as follows:

<code>#t</code>	Grabs the focus from a window launched by key binding back to the layout window.
<code>#f</code>	Does not grab the focus from a window.

37

hdl Commands for Milkyway

This chapter describes commands with the prefix *hdl*. The commands are listed alphabetically.

- [hdlGetBusDirectionAsSameWay](#)
- [hdlGetNoPGPortForStdCellsOnly](#)
- [hdlGetNoPortNetInconsistent](#)
- [hdlSetNotCloseCell](#)
- [hdlGetOutputLowerCase](#)
- [hdlSetBusDirectionAsSameWay](#)
- [hdlSetNoPortNetInconsistent](#)
- [hdlSetOutputLowerCase](#)

hdlGetBusDirectionAsSameWay

This command is used for retrieving the current flag setting. The default is 0.

Tool

Milkyway

Syntax

```
hdlGetBusDirectionAsSameWay
```

hdlGetNoPGPortForStdCellsOnly

Retrieves the “No PG Port For Standard Cells Only” Verilog flag.

Syntax

```
hdlGetNoPGPortForStdCellsOnly
```

No argument is required.

Description

The hdlGetNoPGPortForStdCells function returns the “No PG Port For Standard Cells Only” Verilog flag.

The auVerilogOut and auHierVerilogOut functions use this flag to determine if power and ground ports of standard cells should be written to the Verilog file.

If the value is 1, standard cell power and ground ports are not written. If the value is 0, they are written. The flag’s default value is 0.

See Also

`hdlSetNoPGPortForStdCellsOnly`

Example

```
hdlGetNoPGPortForStdCells
```

hdlGetNoPortNetInconsistent

This command is used for retrieving the current setting of the flag set by [hdlSetNoPortNetInconsistent](#). The default is 0.

Tool

Milkyway

Syntax

```
hdlGetNoPortNetInconsistent
```

hdlSetNotCloseCell

Sets the “Not Close Cell” Verilog flag.

Syntax

```
hdlSetNotCloseCell <int value>
```

value - 1 (cells not closed) or 0 (cells closed)

The default is 0.

Description

The `hdlSetNotCloseCell` function sets the “Not Close Cell” Verilog flag. The `auVerilogOut` and `auHierVerilogOut` functions use this flag to determine if cells should be closed if they are opened when generating the Verilog file. If the value is 1, cells are not closed. If the value is 0, cells are closed.

The flag's default value is 0. Leaving the cells opened can speed up the function and decrease run-time.

See Also

[hdlSetNotCloseCell](#)

Example

```
hdlSetNotCloseCell 1
```

hdlGetOutputLowerCase

This command is used in retrieving the value of the [hdlSetOutputLowerCase](#) flag setting. The default value is 0.

Tool

Milkyway

Syntax

```
hdlGetOutputLowerCase
```

hdlSetBusDirectionAsSameWay

This command is used for creating a hierarchical verilog only after Saturn or Scan optimization. You cannot use a normal hierarchy.

The default value is 0. *Verilog out* will output individual bits of information if some of the *bus bits* direction have changed.

When the value is set to 1, *verilog out* will output *bus* as a whole, even if some *sub bits* I/O directions change because of optimization.

Tool

Milkyway

Syntax

```
hdlSetBusDirectionAsSameWay value
```

hdlSetNoPortNetInconsistent

When the value is set to 1, *verilog out* will force the I/O Port name to be the same as it's net name. This avoids an assigned statement in the verilog.

This is particularly useful if Clock Tree Synthesis added a buffer on a clock net going to an output port.

Tool

Milkyway

Syntax

```
hdlSetNoPortNetInconsistent value
```

hdlSetOutputLowerCase

This command is used to force the output of the data stream in lower case characters in the commands [auHierVerilogOut](#) and [auVerilogOut](#).

Tool

Milkyway

Syntax

```
hdlSetOutputLowerCase 1/0
```

where the arguments are as follows:

<i>1</i>	True. To force the lowercase stream.
<i>0</i>	False. The default value. Will not force the lowercase stream.

40

hi Commands for Milkyway

This chapter describes commands with the prefix *hi*. The commands are listed alphabetically. Commands prefixed with *hi* are obsolete and are replaced by commands prefixed with *ge*.

- [hiGetCurrentWindow](#)
- [hiQuit](#)

hiGetCurrentWindow

This Scheme command will return the ID of the current active widget.

Tool

Milkyway

Syntax

`getEditWindow`

hiQuit

Exits the Synopsys tool. When exiting, if any cells changed since they were opened, the Synopsys tool displays the *Save Cells* window to let you select the cells you want to save.

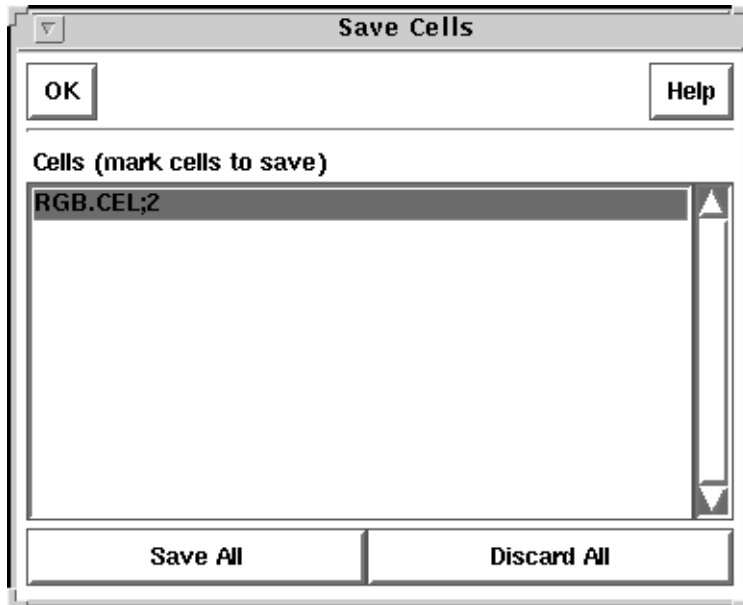
Tools and Menu Commands

Astro: Tools > Quit

Jupiter: Tools > Quit

Milkyway: Tools > Quit

Save Cells Window



Select cells to save before exit

Lists cells that changed. Select the cell(s) you want to save before exiting.

Selection

Use to select the cell(s) you want to save by typing the name of the cell and clicking *Select*. You can also use pattern matching to select all cells with names that match a pattern. For example, typing *TOP.** selects all cells with names that begin with *TOP*.

Use the pattern matching option to select all nets with names that match a pattern.

41

hmi Commands for Milkyway

This chapter describes commands with the prefix *hmi*. The commands are listed alphabetically.

The *hmi* prefix indicates that the command deals with the Synopsys Hercules Verification and Explorer commands. For more information on these commands see the Synopsys *Hercules Manual*.

- [hmiA2DRC](#)
- [hmiA2LVS](#)
- [hmiADRC](#)
- [hmiADRCAddFile](#)
- [hmiADRCDelFile](#)
- [hmiExplorerDRC](#)
- [hmiExplorerLVS](#)

- [hmiExplorerOptions](#)
- [hmiHercules](#)
- [hmiHierAntenna](#)

hmiA2DRC

Converts the design rules stored in a Milkyway database into commands in a Hercules runset file, and invokes Hercules to perform a verification.

Tools and Menu Commands

Astro: Verify > Hercules – Auto DRC

Milkyway: Hercules > Verification – Auto DRC

A2DRC Dialog Box

The menu command opens the following dialog box. Its options are described below.

A2DRC

OK Cancel Default Help

Library Name

Cell Name

Library Path

Output Library Path

Group Directory

Include User Runset File

☒ Use Predefined Mask Layers Only

☐ Use Hierarchical Reference Library

☐ Disable Technology File Checks

☐ Intra-Blockage Checking ☐ Treat All Blockages as Thin Wires

☐ Runset Only ☐ Graphics Output ☐ Temporary Output

☐ Fill Notch/Gap Only ☐ Fill Notch/Gap then Checks

☐ Show Message in Apollo

Database Mode DRC_BLACK_BOX DRC_FULL DRC_AUTO

Black Box File

Black Box Cell(s)

Library Name

Type the name of the library on which you want to perform DRC.
This field has no default and must be filled out.

Cell Name

Type the name of the top cell on which you want to perform DRC.
This field has no default and must be filled out.

Library Path

Type the path of the library. This defines the *layout_path* in the DRC/runset file. The default is the current library.

Output Library Path

Type the path of the output library. This defines the *output_layout_path* in the DRC/runset file.

Group Directory

Type the group directory. This defines *group_dir* in the DRC/runset file.

Include User Runset File

Type the optional user runset file name. This file will be included in the DRC/runset file after the ASSIGN section.

Use Predefined Mask Layers Only

When selected, the tool uses only predefined mask layers.

Use Hierarchical Reference Library

When selected, the equivalent flag for Astro's [dbSetRefControlFileMode](#) in Hercules (HIER_REF_LIB_MODE) will be set TRUE.

Disable Technology File Checks

When selected, only the design rules in the included User Runset file will be verified, not the design rules in the technology file.

Intra-Blockage Checking

When selected, the tool checks external spacing between blockages. This applies to DRC_BLACK_BOX mode only.

Treat All Blockage as Thin Wire

When selected, all blockages are treated as thin wire blockages.

Runset Only

When selected, the tool creates a runset only. Hercules is not invoked.

Graphics Output

When selected, the tool writes a graphics section in the DRC/ runset file.

Temporary Output

When selected, the tool writes output to TEMPORARY layers.

Fill Notch/Gap Only

When selected, the runset will only generate notch/gap fill data.

Fill Notch/Gap then Checks

When selected, the runset will generate notch/gap fill data, and then run DRC checks on the resultant modified database.

Show Message in Astro

When selected, the output message will show in Astro instead of the standard output.

Database Mode

Select one of the database mode options:

- DRC_BLACK_BOX
- DRC_FULL
- DRC_AUTO

Black Box File

Type the name of a file which contains a list of black box cell names. This defines the BLACK_BOX_FILE option in the HEADER{} section of the DRC runset.

Black Box Cell(s)

Type a list of black box cell names separated by spaces. This defines *black_box_cells* in the DRC/runset file.

hmiA2LVS

Creates a runset file named “cellname.ev,” and invokes Hercules to perform a verification. The results are stored in the LVS directory. Also creates the equivalence file and the schematic netlist file based on the Logical Design Data stored in the database. The schematic netlist file is written in Hercules format. Only top level compression is performed.

Tools and Menu Commands

Astro: Verify > Hercules – Auto LVS

Milkyway: Hercules > Verification – Auto LVS

A2LVS Dialog Box

The menu command opens the following dialog box. Its options are described below.

AZLVS

OK Cancel Default Help

Required:

Library Name

Layout Cell Name

Optional:

Schematic Netlist ◆ Default ▼ Milkyway ▼ Verilog ▼ Hercules

Milkyway Netlist View

Verilog Schematic Netlist

Hercules Schematic Netlist

Schematic Cell Name

Library Path

Output Path

Group Directory

Include User Runset File

☐ Use Hierarchical Reference Library

☐ Runset Only ☐ Graphics Output

☐ Show Message in Apollo

Library Name

Type the name of the library on which you want to perform LVS. This field has no default and must be filled out.

Layout Cell Name

Type the name of the top layout cell on which you want to perform LVS. This field has no default and must be filled out.

Schematic Netlist

In LVS, the schematic can be in one of the following formats:

Option	Creates...
Default	A schematic netlist on the tool's default cell.
Milkyway Netlist View	A schematic netlist view on the cell in Milkyway format when selected.
Verilog Schematic Netlist	A schematic netlist on the cell in Verilog format when selected.
Hercules Schematic Netlist	A schematic netlist on the cell in Hercules format when selected.
Schematic Cell Name	Type in the name of the schematic top block. this is not the same as the Layout top block.

Library Path

Type the library path. This defines the *layout_path* in LVS/runset file. The default is the current directory.

Output Path

Type the path of the output library. This defines the *output_layout_path* in the LVS/runset file.

Group Directory

Type the group directory. This defines *group_dir* in the LVS/runset file.

Include User Runset File

Type the optional user runset file. This file will be included in the LVS/runset file after the ASSIGN section.

Use Hierarchical Reference Library

When selected, the equivalent flag for Astro's `dbSetRefControlFileMode` in Hercules (HIER_REF_LIB_MODE) will be set TRUE.

Runset Only

When selected, the tool creates a runset only. Hercules is not invoked.

Graphics Output

When selected, the tool writes a graphics section in the DRC/ runset file.

Show Message in Astro

When selected, the output message will show in Astro instead of the standard output.

hmiADRC

This is the Advanced DRC option. It uses the attached file in the opened library to perform a design rule check. You should have a Hercules DRC license to run this command. Before executing this command, you need to first open a library. After completing this command, use `geLoadErrorCell` to load the error cell that is produced by hmiADRC. It is always the same as the selected Cell Name. The other operations are the same as the general DRC flow.

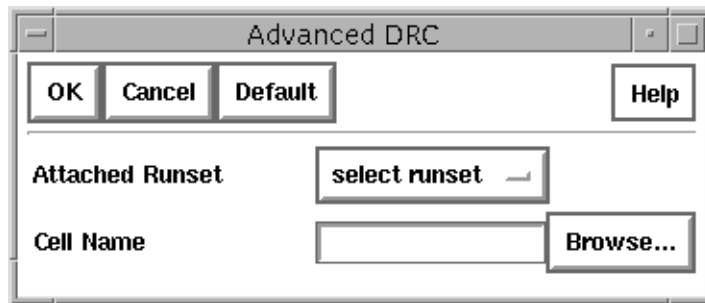
Tools

Milkyway

Astro

Advanced DRC Dialog Box

The menu command opens the following dialog box. Its options are described below.



Attached Runset

Choose the file you want to use for DRC checking.

Cell Name

Type the name of the top block on which you want to perform your Advanced DRC check. This field has no default and must have an input.

hmiADRCAddFile

This is the command to attach the runset rule file of an Advanced DRC.

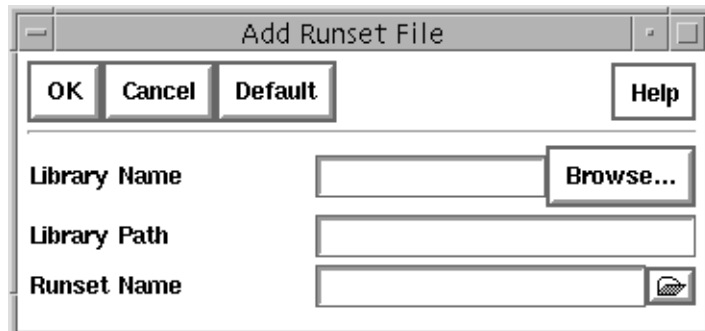
Tools

Milkyway

Astro

Add Runset File Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Type the name of the library on which you want to attach the Advanced DRC rule file. This field is required.

Library Path

Type the path name of the library.

Runset Name

Type the filename of the Advanced DRC rule file (and path, when not in the current directory). The filename should have the extension “ev”.

hmiADRCDeIFile

This is the command to delete the attached rule file of an Advanced DRC. Before executing this command, you must first close the library.

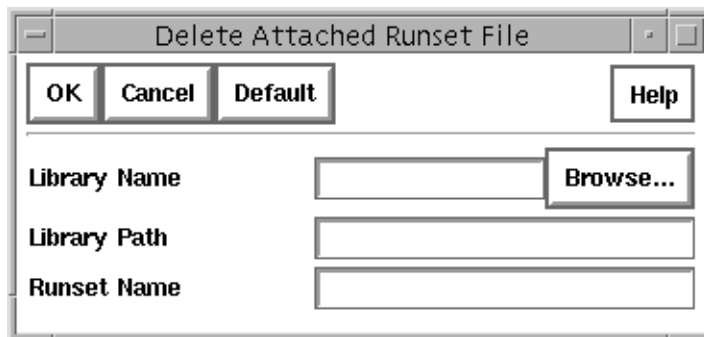
Tools

Milkyway

Astro

Delete Attached Runset File Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Type the name of the library on which you want to delete the attached Advanced DRC rule file. This field is required.

Library Path

Type the path name of the library.

Runset Name

Type the filename of the attached Advanced DRC rule file. This field is required.

hmiExplorerDRC

Invokes the Explorer executable, *hxdrc*. This command is provided as an interface between the Synopsys place and route tools and the Synopsys Explorer verification tool.

Tools and Menu Commands

Astro: Verify > Explorer – DRC

Milkyway: Hercules > Explorer – DRC

hmiExplorerLVS

Invokes the Explorer executable, *hxlvs*. This command is provided as an interface between the Synopsys place and route tools and the Synopsys Explorer verification tool.

Tools and Menu Commands

Astro: Verify > Explorer – LVS

Milkyway: Hercules > Explorer – LVS

hmiExplorerOptions

Allows the addition of space-separated user command-line options prior to invoking Hercules Explorer. For information and descriptions of all Explorer command-line options, refer to *hxdrc-usage* or *hxlvs-usage* in the Hercules documentation.

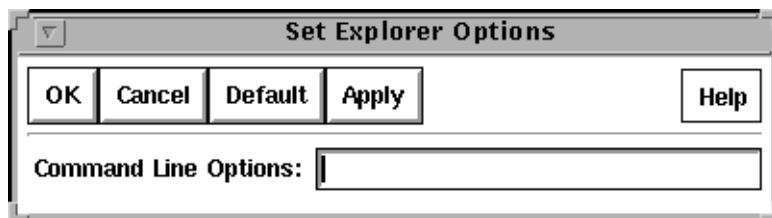
Tools and Menu Commands

Astro: Verify > Explorer – Options

Milkyway (no menu command)

Set Explorer Options Dialog Box

The menu command opens the following dialog box. Its options are described below.



Command Line Options

Type the Hercules Explorer command-line option(s), along with applicable arguments, all separated by single spaces. Options must begin with a hyphen "-".

hmiHercules

Invokes Hercules on a user-specified runset, input layout library, and top cell name. Other runtime options can also be specified. For more information on running Hercules, refer to *hercules-usage* within the Hercules documentation.

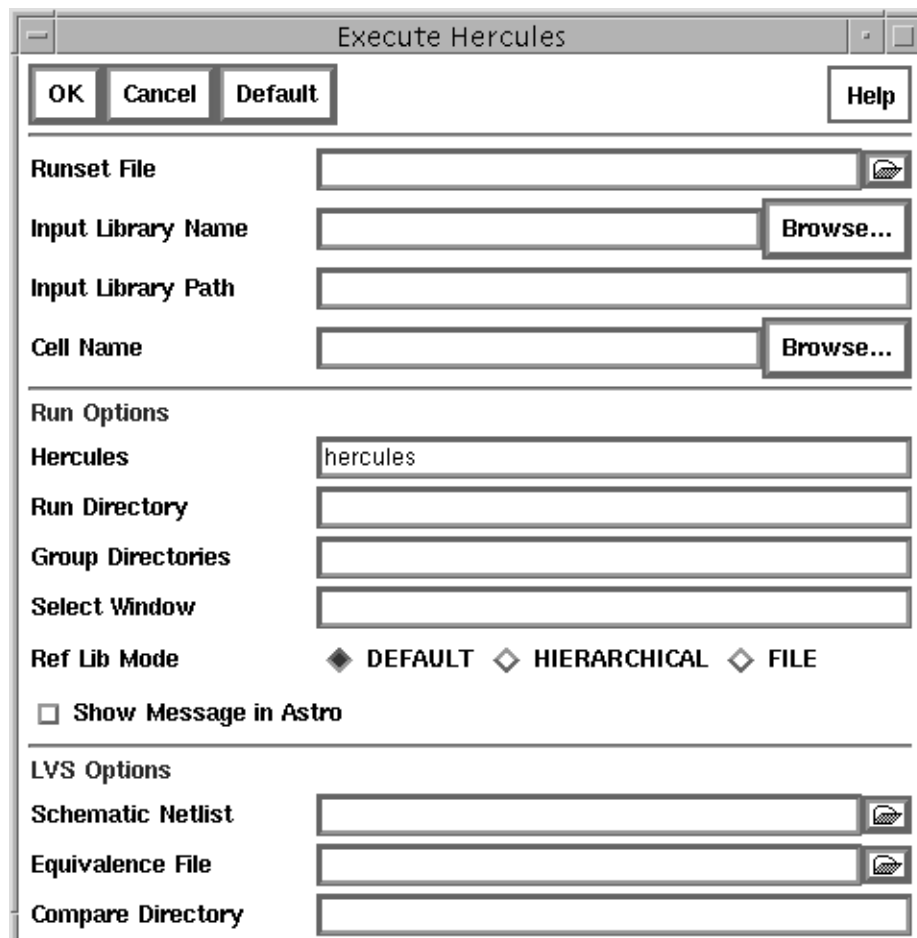
Tools and Menu Commands

Astro: Verify > Hercules – User Runset

Milkyway: Hercules > User Runset

Execute Hercules Dialog Box

The menu command opens the following dialog box. Its options are described below.



The dialog box is titled "Execute Hercules". It features a standard Windows-style title bar with minimize, maximize, and close buttons. Below the title bar is a row of buttons: "OK", "Cancel", "Default", and "Help". The main area is divided into several sections. The first section contains four text input fields: "Runset File", "Input Library Name", "Input Library Path", and "Cell Name". The "Runset File" field has a file selection icon on its right. The "Input Library Name" and "Cell Name" fields have "Browse..." buttons to their right. The second section, titled "Run Options", contains a text input field for "Hercules" (pre-filled with "hercules"), and three more empty text input fields for "Run Directory", "Group Directories", and "Select Window". Below these is a "Ref Lib Mode" section with three radio button options: "DEFAULT" (selected), "HIERARCHICAL", and "FILE". There is also a checkbox labeled "Show Message in Astro". The third section, titled "LVS Options", contains three text input fields: "Schematic Netlist", "Equivalence File", and "Compare Directory". The "Schematic Netlist" and "Equivalence File" fields have file selection icons on their right.

Runset File

Required. Type the Hercules runset filename (and path, if not in the current directory).

Input Library Name

Type the name of the library on which you want to perform your customized Hercules run. This field over-rides the library name specified within the runset file.

Input Library Path

Type the path name of the library on which you want to perform your customized Hercules run. This field over-rides the library name specified within the runset file.

Cell Name

Type the name of the top block on which you want to perform your customized Hercules run. This field over-rides the top block name specified within the runset file.

Hercules

Required. Type the name of the Hercules executable. If no path is specified, the user's PATH environment variable will determine where the binary is found.

Run Directory

Type the name of the Hercules run directory. All Hercules output will be stored in a directory hierarchy starting with the run directory as the top level directory. This field over-rides the corresponding entry in the runset.

Group Directories

Type the name(s) of group directories. See Hercules documentation for more information. This field over-rides the corresponding entry in the runset.

Select Window

Type the coordinates of layout window(s) as arguments to the Hercules `SELECT_WINDOW` runset option. The coordinates specify area(s) of the design on which to run verification. The coordinates must be specified as one or more comma-separated tuples of four comma-separated numbers within square brackets: `[x1, y1, x2, y2], [...], ...`

Ref Lib Mode

Has three options: `DEFAULT`, `HIERARCHICAL`, and `FILE`.

If `DEFAULT` is selected, no option is inserted into the setup file. The default reference structure is used.

If `HIERARCHICAL` is selected, Milkyway forces Hercules to use the library's (internal) reference structure (created by `cmRefLib` and `cmReplaceLib`).

If `FILE` is selected, Milkyway forces Hercules to use the library's library's reference control file (set by `dbSetRefLibControl`).

Show Message in Astro

When selected, the outputted message will show in Astro instead of the standard out.

Schematic Netlist

Type the name of the schematic netlist for use by Hercules LVS Compare. This field over-rides the corresponding entry in the runset.

Equivalence File

Type the name of the equivalence file for use by Hercules LVS Compare. This field over-rides the corresponding entry in the runset.

Compare Directory

Type the name of the compare directory. This field over-rides the corresponding entry in the runset.

hmiHierAntenna

Invokes Hercules on an auto-generated runset with an input layout library, and a top cell name. You can use this command to input information required to extract the hierarchical antenna properties of a hard macro. This information, along with information obtained from the design's technology file, can be used to create and execute a Hercules runset that performs the actual data extraction.

When the dialog box has been completed and the "OK" button has been clicked, you will see the comment "<Hercules has started>" in the Astro command window. The interface will use the information you've provided, along with information obtained directly from the design's technology file, to create a *Hercules* runset that will extract the hierarchical antenna properties from your design. It will then call *Hercules* and execute that runset.

This process will be transparent and you will only see a succession of dots being generated in the Astro command window indicating *Hercules* is running. When *Hercules* has finished, you will see the comment "<Hercules is done>" in the command window, and the hierarchical antenna properties extracted by *Hercules* will have been added to the .FRAM view of your database. Depending on the size and complexity of the macro this interface is being applied to, *Hercules* execution can range from a few minutes to a few hours.

The data reported by this interface includes information on the cumulative size of all gates connected to the net being reported. This area is determined by the logic AND all poly layers with all diffusion layers. Similarly, MOS source and drain regions are defined as *diffusions* that are not covered by a gate but that are touching that gate.

This interface does not explicitly check for shorts, but because a short merges two (or more) different nets, you might see unreasonable values for the area of a given net, simplifying the Astro and Hercules *Hierarchical Antenna Property* generation.

Tools

Milkyway

Astro

Hierarchical Antenna Report Dialog Box

The menu command opens the following dialog box. Its options are described below.

Hierarchical Antenna Report

OK Cancel Default Help

Library Name Browse...

Cell Name Browse...

Library Path

Pins for ☒ Top Cell Only ☐ All Cells

☐ Treat MOS source/drain as protection diodes

☐ Report Protection Diodes

Data Layers

Diffusion	
Poly	
Contact	

Text Layers

Poly	
Metal 1	
Metal 2	
Metal 3	
Metal 4	
Metal 5	
Metal 6	
Metal 7	
Metal 8	
Metal 9	
Metal 10	
Metal 11	
Metal 12	

Library Name

Type the name of the library containing the hard macro where you want to extract the hierarchical antenna properties. This is a required field.

Cell Name

Type the name of the top block (hard macro) where you want to extract the hierarchical antenna properties. This is a must input field.

Note:

If you want to use the “Browse...” button, you must first open the target library with [geOpenLib](#) so you can browse the target cell name by GUI.

Library Path

Type the path name of the library where you want to extract the hierarchical antenna properties from, if it's not in the current working directory. If this is not specified, the path defaults to the current working directory.

Pins for

There are two option modes for Pins:

- *Top Cell Only*: This only applies to top cell pins. Extracts hierarchical antenna properties only for nets connecting to pins in the top cell of the macro. This is the default.
- *All Cells*: This applies to child cell pins that are referenced by a top cell and to top cell pins. Extracts hierarchical antenna properties for nets that connect to pins in the top cell of the macro as well as pins in the child cells contained within the macro.

Treat MOS source/drain as protection diodes

When selected, Hercules treats MOS source/drain as protection diodes and considers any MOS source and drain regions connected to a net to be a part of the diode protection area available to that net. When this option is not selected, these source and drain regions are ignored when protection diodes are being considered. The default is TRUE and treats MOS source/drain regions as protection diodes.

Note:

If this option is selected, the previous option must also be selected in order for this information to be reported to Astro.

Report Protection Diodes

When selected, Hercules reports the location of generated protection diodes. The default is TRUE and includes information on existing diodes in the data given to Astro.

Hercules includes information on the total area of any diodes already present on a given net in the data it provides to Astro. When this option is not selected, Hercules does not include information on existing diodes in this data.

Note:

This interface considers any instance where a diffusion layer is connected through a contact layer and the appropriate metal and via layers to the net in question to be a protection diode. As a result, if your design contains other devices with this structure, the diode area reported to Astro will include the area of these other devices.

Data Layers

Diffusion

Input the GDSII layer number for all diffusion layers in the design that *hmiHierAntenna* can't automatically get from Astro technology file. The valid input is number 0-255 separated by ",". An example of a valid input is "1,2,3". All layers specified in this field will be OR'd together by Hercules for processing. This is a must input field.

Poly

If the poly layer doesn't exist in Astro technology file, please input the layer numbers of all poly layers in the design. Valid entries are any number in the range 0-255, and multiple layers will be OR'd together by Hercules for processing. This is a required field only if the interface is unable to identify the poly layer from the technology file.

Contact

If the contact layer doesn't exist in Astro technology file, please input the layer number(s) on the contact layers (connecting poly to metal-1) in the design. Valid entries are any number in the range 0-255, and multiple layers will be OR'd together by Hercules for processing. This is a required field only if the interface is unable to identify the contact layer from the technology file.

Text Layers

With the exception of the predefined layer in Astro technology file, if you have an additional text layer, such as "Metal 1 layer", please enter it in the appropriate field. Multiple layer numbers are allowed.

42

hnet Commands for Milkyway

This chapter describes commands with the prefix *hnet*. The commands are listed alphabetically.

- [hnetChangeInstName](#)
- [hnetCreate](#)
- [hnetCreateHierNetlist](#)
- [hnetCreateHierVlogOut](#)
- [hnetDetachPortInstFromPGNet](#)

hnetChangeInstName

Scans any instance with the prefix “ecoInst_”, and find its driving instance or driving port, and then “pushes down” the “ecoInst_” into its driving hierarchy where possible. Should be used when Astro does not preserve the module boundary.

Tool

Astro

Syntax

```
hnetChangeInstName "libName" "cellName"
```

where the arguments are as follows:

<i>libName</i>	Name of the library for which you want to modify the bus naming style. Valid values: Name of any library in the current directory.
<i>cellName</i>	Name of the cell for which capacitance of a port is specified. Valid values: Name of any cell in the library.

If using scan chain re-ordering and/or Saturn, also run the following commands before doing *hnetCreate*.

Example

For Saturn:

```
hnetChangeInstName "test_bus" "test_cell"  
#t
```

For scan chain optimization use *scoRenameChainNet*. This fixes any same-name scan problems causing hierarchical verilog out to rename the nets for scanning.

hnetCreate

This command has been replaced by [auCreateHierVlogOut](#).

hnetCreateHierNetlist

Regenerates a hierarchical netlist for a flat layout cell after netlist changes in place and route, based on a reference hierarchical netlist. These netlist changes can result from clock tree synthesis (CTS), scan chain optimization (reordering), Solar and Saturn optimization (cell sizing, buffer or repeater insertion, logic optimization), or any other netlist changes.

To reflect the changes back to the original hierarchical netlist, the .NETL view cells corresponding to the original hierarchical netlist need to be used. The cell view name for the top module of the .NETL view cell is required for command input.

HNET is the view name for the newly created hierarchical netlist cells. After the HNET view cell creation, you can use the hierarchical VerilogOut command to output the hierarchical netlist. This output contains the changes resulting from place and route activities.

If your netlist is hierarchical, expand it with [cmCmdExpand](#). Subsequent place and route steps, such as clock tree synthesis and scan chain optimization, change the expanded hierarchical netlist,

making it difficult for the logic designer to work with. The `hnetCreateHierNetlist` command generates a new hierarchical netlist reflecting the changes made to the design during place and route.

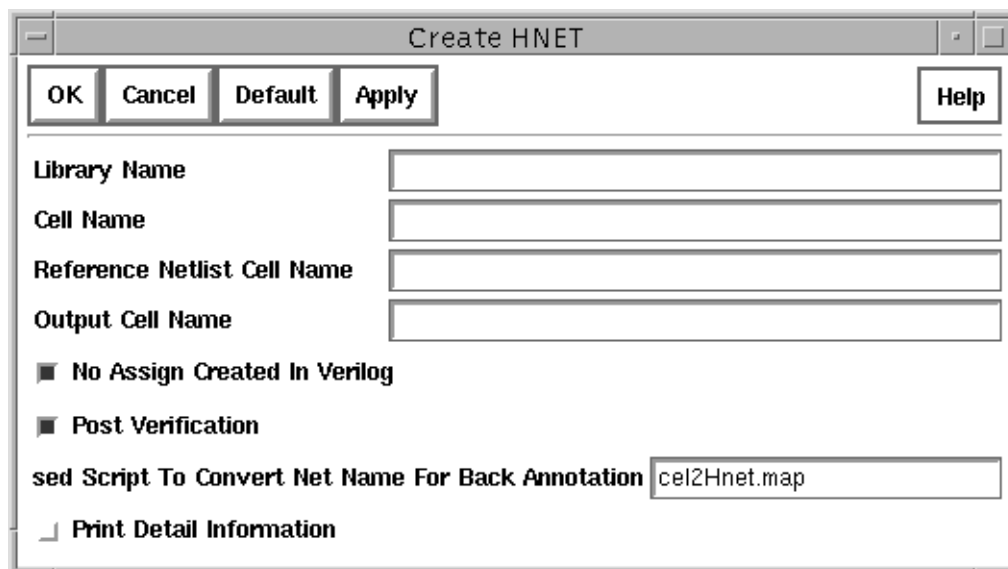
The newly generated hierarchical netlist is created by the [auHierVerilogOut](#) command.

Tools and Menu Commands

Milkyway: Output > Create HNET

Create HNET Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

The name of the library containing the cell for which you want to build a hierarchical netlist, and the reference cell.

Cell Name

The name of the top-level expanded cell for which a new netlist hierarchy is being created. By default, this is a .CEL view cell. For example, top.CEL.

Reference Netlist Cell

The name of the top-level, unexpanded netlist cell from which the place and route cell was generated.

Output Cell Name

The name of the hierarchical netlist cell to be created and used during VerilogOut or hierarchical VerilogOut.

No Assign Created In Verilog

When selected, the tool will remove assign statements (some port names will be modified to remove these assign statements).

After layout, some connections may become changed. To retain original port names, usually a new net is created which is connected with the ports. An assignment statement is generated for this connection whenever you dump the cell through commands [auVerilogOut](#) or [auHierVerilogOut](#).

Post Verification

When selected, the tool performs a quick check of the cell created by this command. If found, the tool will report the following five types of problems:

Problem Type	Description
Directions of Ports are Changed	After layout, the original port direction cannot drive all port instances it connected. Therefore, the direction has been changed.
Extra Ports are Found	Some extra ports may be created when using this command. This is acceptable as long as they do not cause net loop or break the connection.

Problem Type	Description
Net Loops	Net loops may be formed from the hierarchical ports and nets in the netlist. Net loops may cause problem n front-end tools.
Mismatch Between Flattened Cell and Layout Cell	Corresponding connections in both cells cannot be found. This is a serious problem and can result in shorts or opens. Please call Synopsys Technical Support for help.
Net Mapping Between Flatten Cell and Layout Cell	Can result in net name changes. Corresponding nets can be determined using the reported mapping messages. Names can be replaced in back-annotation files.

sedScript To Convert Net Name For Back Annotation

Because some nets may be renamed after running this command, you need to use the 'sed' function to replace nets for your timing information file with the new ones (LPE, DSPF, SPEF, or SPICE).

Whenever you enter a file name in this field, the tool creates a file containing the required sed script. You can use this script to run the following sed function:

```
sed -f script <oldTimeFile> newTimeFile
```

After you perform this function, all nets in oldTimeFile are renamed, as needed.

Print Detail Information

When selected, the tool shows a detailed listing of the build process.

hnetCreateHierVlogOut

Regenerates a hierarchical netlist for a flat layout cell after netlist changes in place and route, based on a reference hierarchical netlist. These netlist changes can result from clock tree synthesis (CTS), scan chain optimization (reordering), Solar and Saturn optimization (cell sizing, buffer or repeater insertion, logic optimization), or any other netlist changes.

To reflect the changes back to the original hierarchical netlist, the .NETL view cells corresponding to the original hierarchical netlist need to be used. The cell view name for the top module of the .NETL view cell is required for command input.

HNET is the view name for the newly created hierarchical netlist cells. After the HNET view cell creation, you can use the hierarchical VerilogOut command to output the hierarchical netlist. This output contains the changes resulting from place and route activities.

If your netlist is hierarchical, expand it with [cmCmdExpand](#). Subsequent place and route steps, such as clock tree synthesis and scan chain optimization, change the expanded hierarchical netlist, making it difficult for the logic designer to work with. The hnetCreateHierVlogOut command generates a new hierarchical netlist reflecting the changes made to the design during place and route.

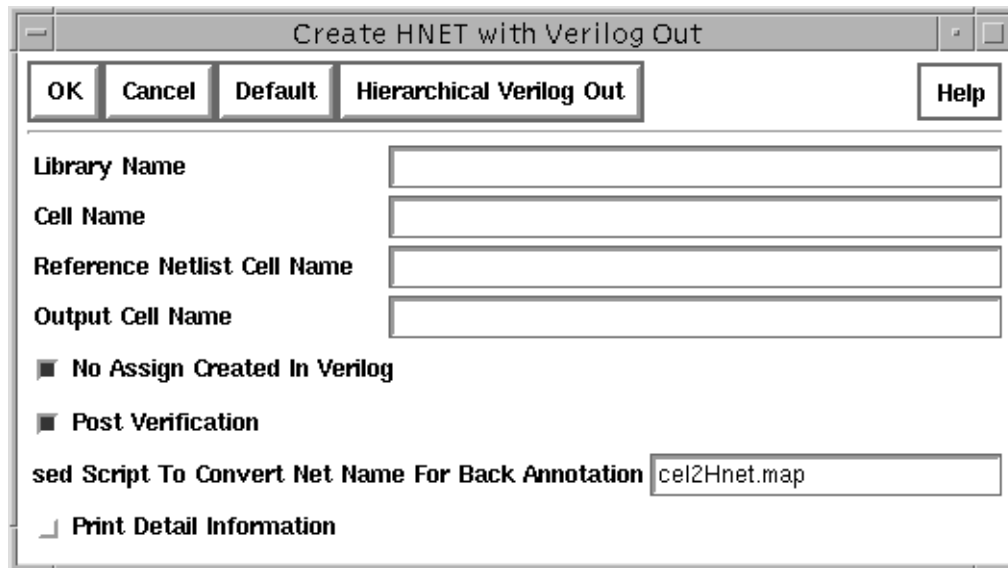
The newly generated hierarchical netlist is created automatically.

Tool

Milkyway

Create HNET with Verilog Out Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

The name of the library containing the cell for which you want to build a hierarchical netlist, and the reference cell.

Cell Name

The name of the top-level expanded cell for which a new netlist hierarchy is being created. By default, this is a .CEL view cell. For example, top.CEL.

Reference Netlist Cell Name

The name of the top-level, unexpanded netlist cell from which the place and route cell was generated.

Output Cell Name

The name of the hierarchical netlist cell to be created and used during VerilogOut or hierarchical VerilogOut.

No Assign Created In Verilog

When selected, the tool will remove assign statements (some port names will be modified to remove these assign statements).

After layout, some connections may become changed. To retain original port names, usually a new net is created which is connected with the ports. An assignment statement is generated for this connection whenever you dump the cell through commands [auVerilogOut](#) or [auHierVerilogOut](#).

Post Verification

When selected, the tool performs a quick check of the cell created by this command. If found, the tool will report the following five types of problems:

Problem Type	Description
Directions of Ports are Changed	After layout, the original port direction cannot drive all port instances it connected. Therefore, the direction has been changed.
Extra Ports are Found	Some extra ports may be created when using this command. This is acceptable as long as they do not cause net loop or break the connection.
Net Loops	Net loops may be formed from the hierarchical ports and nets in the netlist. Net loops may cause problem in front-end tools.
Mismatch Between Flattened Cell and Layout Cell	Corresponding connections in both cells cannot be found. This is a serious problem and can result in shorts or opens. Please call Synopsys Technical Support for help.

Problem Type	Description
Net Mapping Between Flatten Cell and Layout Cell	Can result in net name changes. Corresponding nets can be determined using the reported mapping messages. Names can be replaced in back-annotation files.

Whenever you enter a file name in this field, the tool creates a file containing the required sed script. You can use this script to run the following sed function:

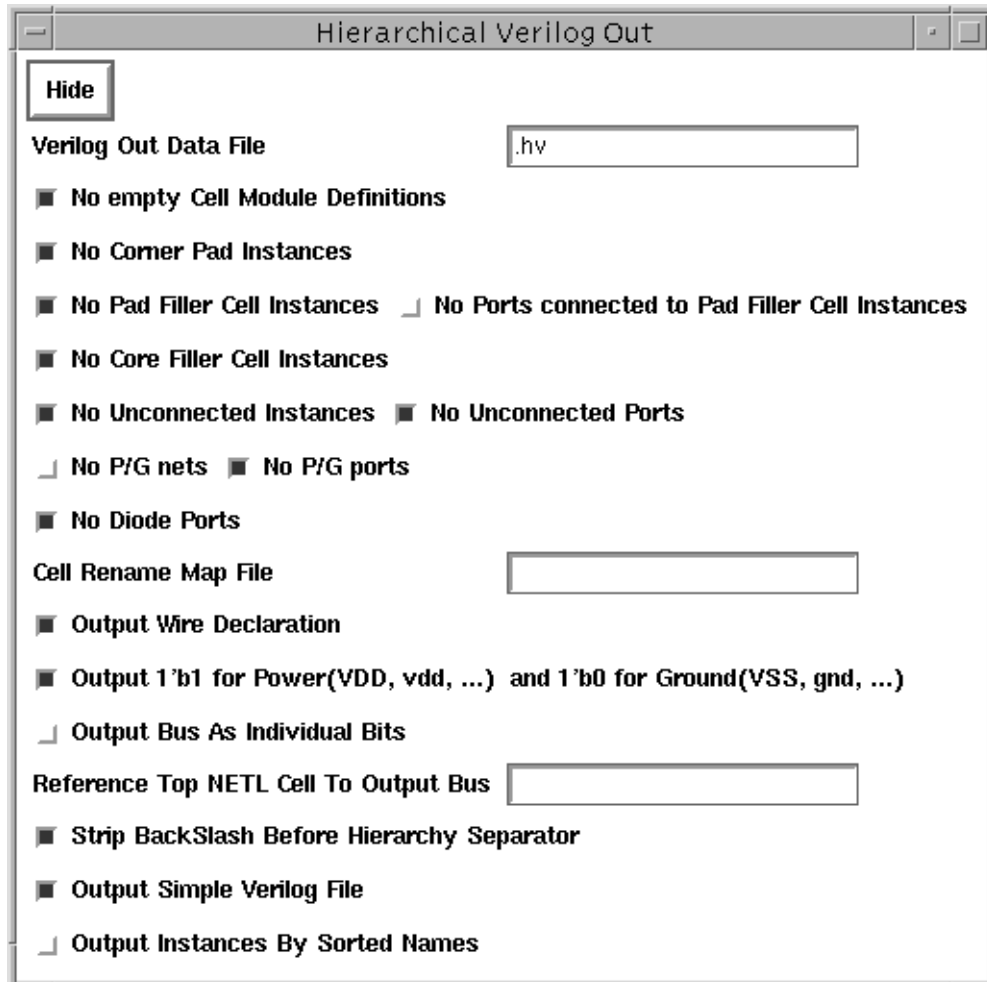
```
sed -f script <oldTimeFile> newTimeFile
```

After you perform this function, all nets in oldTimeFile are renamed, as needed.

Print Detail Information

When selected, the tool shows a detailed listing of the build process.

Hierarchical Verilog Out Dialog Box



Hierarchical Verilog Out

Hide

Verilog Out Data File

☒ No empty Cell Module Definitions

☒ No Corner Pad Instances

☒ No Pad Filler Cell Instances ☐ No Ports connected to Pad Filler Cell Instances

☒ No Core Filler Cell Instances

☒ No Unconnected Instances ☒ No Unconnected Ports

☐ No P/G nets ☒ No P/G ports

☒ No Diode Ports

Cell Rename Map File

☒ Output Wire Declaration

☒ Output 1'b1 for Power(VDD, vdd, ...) and 1'b0 for Ground(VSS, gnd, ...)

☐ Output Bus As Individual Bits

Reference Top NETL Cell To Output Bus

☒ Strip BackSlash Before Hierarchy Separator

☒ Output Simple Verilog File

☐ Output Instances By Sorted Names

Verilog Out Data File

Type the name of the file to which you want to write the verilog information.

No empty Cell Module Definitions

Select to prevent Milkyway from translating empty cells.

No Corner Pad Instances

Select when you do not want Milkyway to translate corner pad cell instances.

No Pad Filler Cell Instances

Select when you do not want Milkyway to translate pad filler cell instances.

No Ports connected to Pad Filler Cell Instances

Select when you do not want Milkyway to translate the ports that are connected to the pad filler cell instances.

No Core Filler Cell Instances

Select when you do not want Milkyway to translate core filler cell instances.

No Unconnected Instances

Select when you do not want Milkyway to translate unconnected instances.

No Unconnected Ports

Select when you do not want Milkyway to translate unconnected ports. If this option is not selected, unconnected ports will be generated and new nets will be created to connect the ports. The naming style for these nets is AVANT_UNCONNECTED_%d.

No P/G nets

Select if you do not want Milkyway to translate power and ground nets.

No P/G ports

Select if you do not want Milkyway to translate power and ground ports.

No Diode Ports

When selected, Milkyway diode ports are not written to the LEF file. If not selected, Milkyway diode ports are written.

Cell Rename Map File

Type the name of the cell mapping file you want Milkyway to use to map cell names during translation, or leave blank to use GDSII cell names. If you used an EDIF to GDSII map file to map cell names when you imported your netlist, use this same file to map names back to EDIF.

Output Wire Declaration

Select to add wire information to the output. By default, this option is not selected and no wire declaration is generated.

Output 1'b1 for Power(VDD, vdd, ...) and 1'b0 for Ground(VSS, gnd, ...)

Select to output power nets as 1'b1 and ground nets as 1'b0. For example, if you have power net VDD connected to port1, and ground net VSS connected to port2

And the Option is ...	Milkyway outputs the nets as ...
Selected	assign port1 = 1'b1 ; assign port2 = 1'b0 ;
Not selected	assign port1 = VDD ; assign port2 = VSS ;

When this option is not selected, "supply1" and "supply0" are generated for power and ground nets at the net declaration section of the verilog file. When this option is selected, as it is by default, "1'b1" and "1'b0" are created at the cell instantiation section and no net declaration is created for them.

Output Bus As Individual Bits

Select to output buses as individual bits. For example, if you have a bus with bits A[0], A[1], and A[2]:

And the Option is ...	Milkyway outputs the bus as ...
Selected	<pre>module X (A[2], A[1], A[0]); input A[2]; /*Individual Bits*/ input A[1]; /*Individual Bits*/ input A[0]; /*Individual Bits*/ endmodule</pre>
Not selected	<pre>module X (A); input [2:0] A; /*As a Bus*/ endmodule</pre>

When this options is not selected, all bits of a bus are combined into a bus port (or net). In such case, Milkyway ignores No Unconnected Ports and No P/G Nets, so that unconnected ports and tie-off nets are generated.

Reference Top NETL Cell TO Output Bus

Type the name of the .NETL cell which contains the bus information you want to output.

Very often, a hierarchical .NETL cell is expanded and then bound to a top cell, usually with a different name for placement and routing. Because the bus information is stored in the .NETL cells, you need to specify the original top .NETL cell name when outputting bus information.

Strip BackSlash Before Hierarchy Separator

When selected, the tool removes backslashes that appear before hierarchy separators for every identifier, including ports, nets, and instances.

Output Simple Verilog File

When selected, the tool generates a Verilog file that is smaller and easier to read. Used for backward compatibility, selecting this option allows you to generate a simple format Verilog file similar to version 2.2.

Output Instances By Sorted Names

Select this feature to control the order of instances in the Verilog Files. When selected, the feature will output filenames alphabetically. When deselected, filenames will order arbitrarily.

hnetDetachPortInstFromPGNet

Detaches port instances from the power and ground nets.

Tool

Milkyway

Syntax

```
hnetDetachPortInstFromPGNet libName cellName
```

where the arguments are as follows:

<i>libName</i>	Name of the port instance you want to detach from the power and ground net. Valid values: Name of any library in the current directory.
<i>cellName</i>	Name of the cell for which capacitance of a port is specified. Valid values: Name of any cell in the library.

43

io Commands for Milkyway

This chapter describes commands with the prefix *io*. The commands are listed alphabetically.

- [ioDumpIOLocation](#)
- [ioDumpPinLocation](#)
- [ioDumpTdf](#)
- [ioDumpTdfForChildCell](#)
- [ioLoadTdf](#)
- [ioReplacePads](#)

ioDumpIOLocation

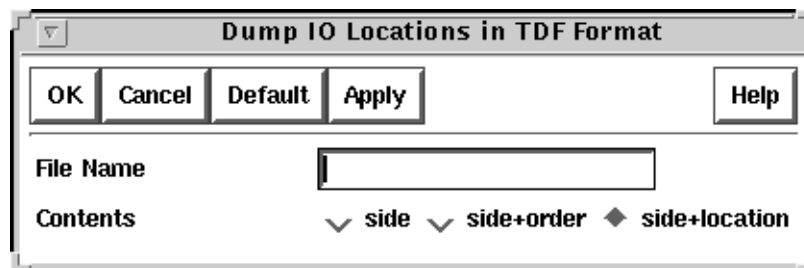
Writes the I/O pin information to a TDF file. Use this command to save changes made to the I/O pins since you last loaded a TDF file.

Tools and Menu Commands

Astro: Design Setup > TDF - Dump IO Pins

Dump IO Locations in TDF Format Dialog Box

The menu command opens the following dialog box. Its options are described below.



File Name

Type the name of the file to create.

Contents

Select the *Contents* option.

Option	Saves information about the ...
side	side on which each pin is placed. The sides are left, right, top, and bottom.
side+order	side on which each pin is placed and lists the pins the order relative to other pins on the same side.

Option	Saves information about the ...
side+location	side on which each pin is placed and the X,Y location of each pin.

ioDumpPinLocation

Writes the boundary I/O and cPin information to a TDF file. Use this command to save changes made to pins since you last loaded a TDF file.

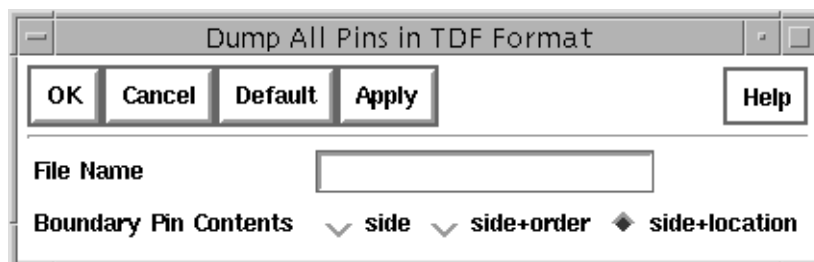
Tools and Menu Commands

Astro: Design Setup > TDF - Dump All Pins

Jupiter: Timing Setup > TDF - Dump All Pins

Dump All Pins in TDF Format Dialog Box

The menu command opens the following dialog box. Its options are described below.



File Name

Type the name of the file to create.

Boundary Pin Contents

Select the Boundary Pin *Contents* options.

Option	Saves information about the ...
side	Side on which each boundary I/O is placed. The sides are left, right, top, and bottom.
side+order	Side on which each boundary I/O is placed and lists the pins the order relative to other pins on the same side.
side+location	Side on which each boundary I/O is placed and the X,Y location of each pin.

ioDumpTdf

Used in conjunction with ataWriteTC, writes the TDF information associated with a specified cell to a file.

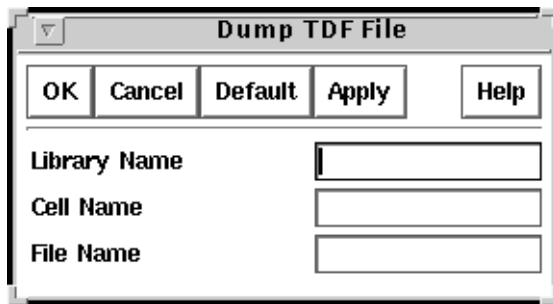
Tools and Menu Commands

Astro: Timing > TDF - Dump TDF

Jupiter: Timing Setup> TDF - Dump TDF

Dump TDF File Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Type the name of the library containing the cell for which you want TDF information.

Cell Name

Type the name of the cell for which you want TDF information.

File Name

Type the name of the file to which you want to write TDF information.

See Also

[tdfDump](#) writes the TDF information associated with a cell to a file. (This function does not display a dialog box.)

[tdfDumpIOToTDF](#) writes I/O constraints of a specified type to a file in TDF format.

ioDumpTdfforChildCell

Dumps the TDF (top design format) file for the child cell from the top level.

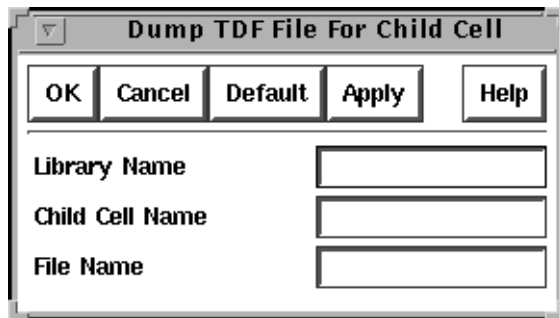
Tools and Menu Commands

Astro: Timing > TDF - Dump TDF for Child Cell

Jupiter: Timing Setup > TDF - Dump TDF for Child Cell

Dump TDF File For Child Cell Dialog Box

The menu command opens the following dialog box. Its options are described below.



Library Name

Type the name of the library containing the cell for which you want to dump TDF information.

Child Cell Name

Type the name of the child cell.

File Name

Type the name of the file to which you want to dump TDF information.

ioLoadTdf

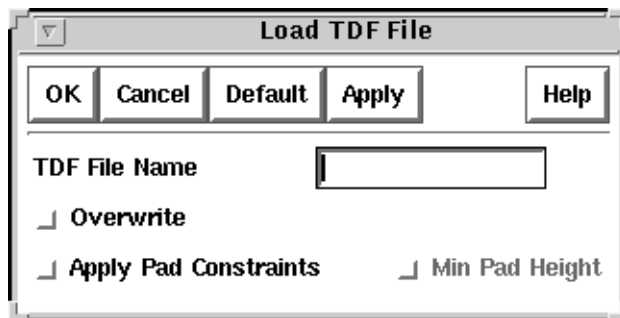
Loads a TDF file.

Tools and Menu Commands

Jupiter: Timing Setup > Load TDF

Load TDF File Dialog Box

The menu command opens the following dialog box. Its options are described below.



TDF File Name

Type the name of the TDF file you want to load.

Overwrite

Select to replace the TDF information currently loaded in the cell with the information in the specified file. By default, this option is deselected, which updates the currently loaded information with the information in the specified file as follows:

- Adds directives in the new file to old directives.
- Replaces old directives that are contradicted by directives in the new file.

Note:

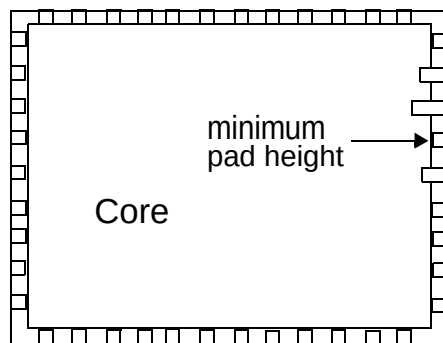
You cannot overwrite or update `markNet` or `insertPad` functions. After nets or pads are identified or created, you cannot change or remove them using TDF directives. To remove a pad, select and delete it using [geDelete](#).

Apply Pad Constraints

Select to apply all *pad* and *insertPad* functions after loading the new TDF file.

Min Pad Height

Select to have Astro set the core area based on the minimum pad height.



ioReplacePads

Adjusts the placement of unconstrained pins/pads. Optionally, this command also adjusts the size of the core area and the size of the cell boundary.

Tools and Menu Commands

Jupiter: Basic Floorplan > Pads - (Re-)place Pads

Replace Pads Dialog Box

The menu command opens the following dialog box. Its options are described below.

Replace Pads

OK Cancel Default Apply

IO Style ▾ Net Weighted Optimization ▾ Flip Chip IO ▾ Center pack ▾ Distribute in core ◆ Distribute ▾ Distribute and abut

Core To Left 50.0 Core To Right 50.0

Core To Bottom 50.0 Core To Top 50.0

Adjust Core ◆ Do Not Change ▾ By Macros/Regions ▾ Maintain Offset From Pads

☐ Min Pad Height

IO Style

Select the style you want to use for distributing the pads.

Option	Distributes pads as follows:
Net Weighted Optimization	Optimize I/O pads based on the net length and net weight.
Flip Chip IO	Optimize the Flip Chip driving cell's location based on the corresponding bump.
Center pack	Pulls pads to the center of each side.
Distribute in core	Distributes pads evenly on each side within the area defined by the core.
Distribute	Distributes pads evenly on each side without regard to the core.
Distribute and abut	Distributes pads evenly on each side without regard to the core, abutting left/right and top/bottom pads.
Old style	Meets constraints with no biasing.

Core To Left

Type the distance you want between the core and the left side of the boundary. The tool uses this value to adjust the cell boundary.

Core To Right

Type the distance you want between the core and the right side of the boundary. The tool uses this value to adjust the cell boundary.

Core To Bottom

Type the distance you want between the core and the bottom of the boundary. The tool uses this value to adjust the cell boundary.

Core To Top

Type the distance you want between the core and the top of the boundary. The tool uses this value to adjust the cell boundary.

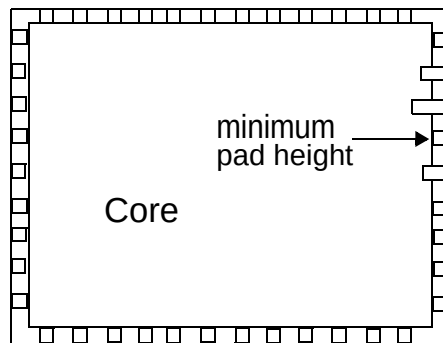
Adjust Core

Select the method you want to use for determining the core area.

Option	Description
Do Not Change	Uses the existing core area.
By Macros/ Regions	Adjusts the core area to the minimum size required to enclose the macros and cell regions.
Maintain Offset From Pads	Adjusts the core area while maintaining offsets from the pads.

Min Pad Height

Select to have Astro set the core area based on the minimum pad height.



49

po Commands for Milkyway

This chapter describes commands with the prefix *po* for the Milkyway tool. The commands are listed alphabetically.

- [poAndPolygons](#)
- [poConnViewPreparation](#)
- [poCreateConnView](#)
- [poCreateConnViewByHerc](#)
- [poDumpCurrSource](#)
- [poFillPolygonsToRectangles](#)
- [poGenCurrSource](#)
- [poNotPolygons](#)
- [poOrPolygons](#)

- `poOversizePolygon`
- `poPathToPolygon`
- `poPolygonArea`
- `poPolygonToRectangles`
- `poPurgeCurrSource`
- `poXorPolygons`

poAndPolygons

This command performs Boolean operations on polygons. Specifically it returns a list of polygons that cover the areas common to two polygons. Note that in Scheme, lists must be quoted or built by using the list procedure. For example,

```
'((0 0) (0 1) (1 1) (1 0) (0 0))
```

```
(list '(0 0) (list 0 y) (list x y) (list x 0) '(0 0))
```

Syntax

```
poAndPolygons <poly1> <poly2>
```

poConnViewPreparation

Like the [poCreateConnViewByHerc](#) command, automatically invokes the Hercules connectivity engine to generate connectivity (CONN) views for nonstandard cells, either flat or hierarchical, but with less memory usage.

The command is designed to replace the `poCreateConnViewByHerc` command. When creating CONN views for hard macros, `poConnViewPreparation` provides the “skip cell” feature that reduces data size and improves performance of the CONN view generation process, without losing the accuracy on IR drop analysis. Creating a white box macro model by integrating current sources is also supported.

When you are creating a full-chip CONN view or an IP block CONN view for rail analysis, it is sometimes better not to generate CONN views for standard cells to remove the fingers. After the fingers are

removed, the current sources located at the fingers are snapped to the center of standard cell power and ground pins. Skipping bit cells for memories saves memory usage in the downstream processing steps.

When a cell is skipped during CONN view creation but not during current source file generation, running `poConnViewPreparation` removes the cell content from the CONN view, but the current sources at the diffusion contacts in the cell are snapped to the center of cell instance boundaries that interact with top-level power and ground metals.

Because the tool cannot distinguish the cell types from GDSII data, the Skip Cell feature is available only to standard cells. For other cell types, current sources are snapped to the cell boundaries only when the center points of the power and ground pin rectangles are inside the cell boundaries.

Note:

You should close the library and cell of interest before running this command.

The number of cells printed in each Hercules command is limited to 300 to prevent the Hercules parser from breaking.

Tool and Menu Command

Astro-Rail: Design Setup > Prepare CONN View

Syntax

`poConnViewPreparation`

Connectivity View Preparation Dialog Box

The command opens the following dialog box. Its options are described below.

CONN View Preparation

OK Cancel Default Apply Help

Library Name Browse...

Cell Name Browse...

Hierarchical Power/Ground Net Information

Net Name	Net Type	Port Pattern(s)	
	☒ power ☐ ground		☐ pattern match
	☒ power ☐ ground		☐ pattern match
	☐ power ☒ ground		☐ pattern match
	☐ power ☒ ground		☐ pattern match

Information From File

Extract Pin by Text Generate CSF

☒ Identify Pin By Pin Text

☒ Extract Connectivity

through ☐ polyCont ☒ via1 ☒ via2 ☒ via3 ☒ via4 ☒ via5

☒ via6 ☒ via7 ☒ via8 ☒ via9 ☒ via10 ☒ via11

☐ text fall through ☐ double fall through

Poly Text		Metal1 Text	
Metal2 Text		Metal3 Text	
Metal4 Text		Metal5 Text	
Metal6 Text		Metal7 Text	
Metal8 Text		Metal9 Text	
Metal10 Text		Metal11 Text	
Metal12 Text		Fall Through Text	

☐ Skip cells Browse...

Note:

You should close the library and cell of interest before running this command.

Library Name

Enter the name of the library or click Browse to select a name from the list that appears.

Cell Name

Enter the single cell name for which you want to generate the CONN view, or click Browse to select a name from the list that appears.

Hierarchical Power/Ground Net Information

Specify the required information for the power and ground nets to be extracted.

- Net Name – Enter the names of the top-level power and ground nets to be extracted.
- Net Type – Select the net type (power or ground) for each net.
- Port Pattern(s) – Enter all possible port names that are under the net you specify in the Net Name text field. This ensures that the port name and the net name can be consistent in the CONN view. Specify the port patterns when
 - The naming of the power and ground ports is different from that in the top-level net
 - The naming in the reference libraries is inconsistent with that in the design library

Note:

The port pattern information is necessary for a hierarchical cell but is optional for a flat cell. If you enter the information for a flat cell, performance will be faster.

Information From File

Specify the name of the file that contains additional information on the power and ground nets in the ASCII format. This option is useful when you have more than four power and ground nets or port patterns to be extracted. The format of the file is

```
netName, netType [, portPattern, patternMatch]
```

where `portPattern` and `patternMatch` are optional.

For example, to extract all pins under the VDD power net when \$vdd\$ is used to identify the VDD power pin without pattern matching, specify the information on the VDD power net in the file as follows:

```
VDD, power, VDD*, patternMatch
```

To comment a line, place a semicolon (;) or a pound sign (#) at the beginning of the line.

Extract Pin By Text

Click to specify information for extracting pins by text.

- Identify Pin By Pin Text

For the CEL view with gdsin or streamin, select to identify pins by text, text fall-through, or double fall-through in the same manner as with the `auExtractBlockagePinVia` command. The default is on.

- Extract Connectivity

Select if you want the tool to trace the connectivity of a specified multiple-layer pin and extract the entire pin. If the layer numbers of texts for pins are identical to the corresponding metal layer numbers, you can leave the following text layer number fields blank. Otherwise you need to specify the appropriate text layer numbers in the corresponding fields, as defined in the technology file. The default is on.

Specify the vias for extracting the connectivity through

- polyCont
- via1
- via2
- via3
- via4
- via5
- via6
- via7
- via8
- via9
- via10
- via11

The connectivity can be extracted through any specified vias. The default is to extract connectivity through via1 to via11 (metal1 to metal12) when the Extract Connectivity option is selected.

- text fall through

Select and specify the Fall Through Text layer if you want Astro-Rail to look at the layers beneath the origin of each text label for identifying the geometry annotated by the text. If more than one geometry exists beneath the origin of the text label, Astro-Rail selects the geometry to associate with the label according to the following priorities, from highest to lowest:

- Fall Through Text
- Metal12
- Metal11
- Metal10
- Metal9
- Metal8
- Metal7
- Metal6
- Metal5
- Metal4
- Metal3
- Metal2
- Metal1
- Poly Text

For example, if one of the geometries is metal3, Astro-Rail selects this geometry; otherwise Astro-Rail looks for metal2, then metal1, and then poly.

When the text layers match the layers for the geometries they identify, deselect the “text fall through” option. You do not need to specify the text layers associated with the interconnection

metals. In other words, when text layers are not specified (that is, the “text fall through” option is not selected), Astro-Rail assumes they match the geometry layers.

- double fall through

Select and specify the Fall Through Text layer if you want Astro-Rail to look at the layers beneath the origin of each text label on a metal2 geometry to identify a geometry (in addition to the metal2 geometry) annotated by the text. If more than one geometry (except the metal2 geometry) exists beneath the origin of the text label, Astro-Rail selects the second geometry to associate with the label based on the following priorities, from highest to lowest:

- Poly Text
- Metal1
- Metal2
- Metal3
- Metal4
- Metal5
- Metal6
- Metal7
- Metal8
- Metal9
- Metal10
- Metal11
- Metal12
- Fall Through Text

For example, if one of the geometries is metal1, Astro-Rail will select this geometry; otherwise Astro-Rail looks for poly.

You can do the following with each text label:

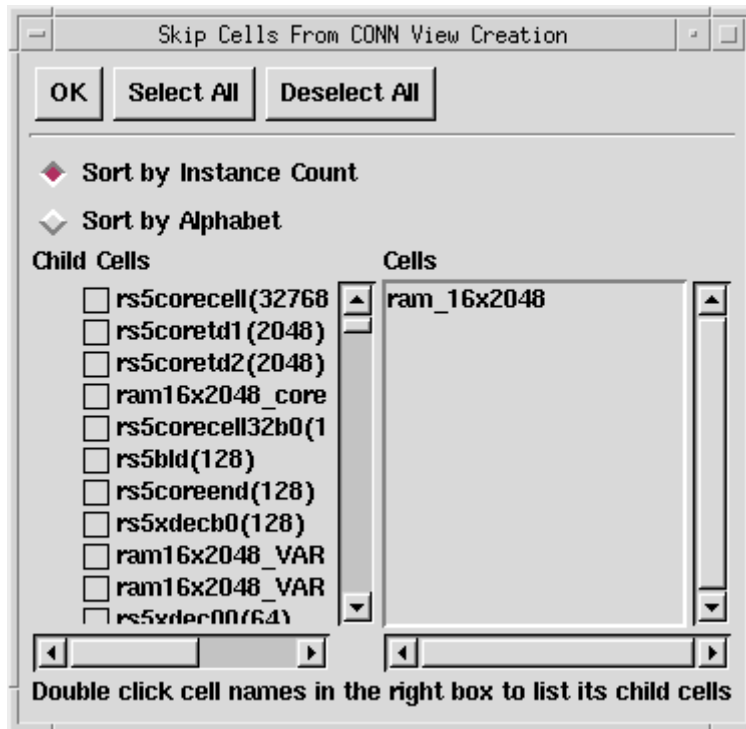
Poly Text – Enter the name or number of the layer used for the text identifying poly geometries (or leave blank if the text layer matches the geometry layer).

Metal# Text – Enter the name or number of the layer used for the text identifying metal# geometries (or leave blank if the text layer matches the geometry layer).

Fall Through Text – Enter the name or number of the layer used for text if you select “text fall through” or “double fall through.”

- **Skip cells** – Select and click the Browse button to specify the cells to be excluded from the CONN view generation.

The Skip Cells From CONN View Creation dialog box opens.



- Sort by Instance Count – Lists the child cells in the order of cell instance count (from high to low).
- Sort by Alphabet – Lists the child cells in the order of cell name.
- In the Child Cells box, select any child cells to be excluded.
- In the Cells box, the name of the cell is shown. This cell name is the same as the input cell name.

Click OK to save the chosen skip cells and close the dialog box.

Click Select All to select all the child cells as skip cells.

Click Deselect All to deselect all the selected cells.

Generate CSF

Click to specify options for generating current source files during the CONN view generation. When clicked, the available options are displayed in the lower section of the Connectivity View Preparation dialog box.

Figure 49-1 CONN View Preparation Dialog Box—Generate CSF

CONN View Preparation

OK Cancel Default Apply Help

Library Name: Browse...

Cell Name: Browse...

Hierarchical Power/Ground Net Information

Net Name	Net Type	Port Pattern(s)	
	power	ground	☐ pattern match
	power	ground	☐ pattern match
	power	ground	☐ pattern match
	power	ground	☐ pattern match

Information From File:

Extract Pin by Text Generate CSF

☐ **Generate Current Source File**

- ☒ Generate Current Source at Via1
- ☒ Generate Current Source at Diffusion Contact

Select Diffusion Mask Names

P Diffusion: N Diffusion:

☐ Exclude Well-tap Diffusion Contacts

P Well: N Well:

Tap Net Names:

☐ Skip cells:

☐ Attach CSF to CONN View

- **Generate Current Source File** – Enables the current source file generation process. Note that the generated current source file is net-based, meaning each power and ground net has a corresponding current source file.

- Generate Current Source At Via1 – Generates current sources at the locations of metal1-metal2 intersections. When selected, the current source file name is in the form *net_name-via1.csf*.
- Generate Current Source At Diffusion Contact – Generates current sources at the locations of diffusion contact cuts. When selected, the current source file name is in the form *net_name-contact.csf*.
- Select Diffusion Mask Names – Specifies the diffusion mask names for locating MOS transistor terminals. Select this option when you want to generate the current source file at the locations of diffusion contact cuts.

Select the mask names from the menus for p-type and n-type diffusions. The tool searches for keywords of "pdiff," "ndiff," and other user-defined mask names in the Milkyway technology file and lists them in the menu.

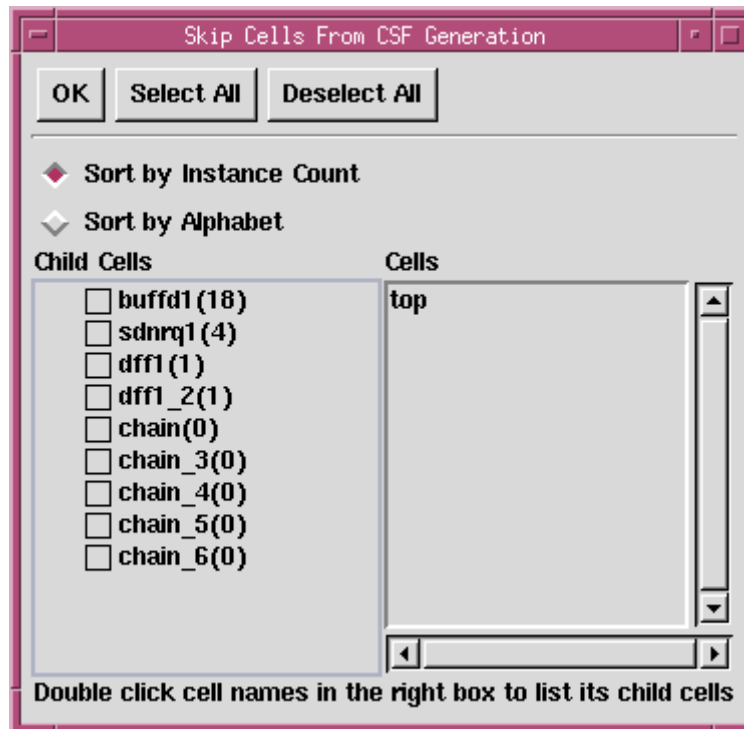
- Exclude Well-tap Diffusion Contacts – Excludes the well-tap diffusion contacts in order to generate more accurate current source locations.

Select the mask name from the menus. The tool searches for keywords of "pwell," "nwell," and other user-defined mask names in the Milkyway technology file and lists them in the menu. The tool can work with P-well only, N-well only, or both well layers.

- Tap Net Names – Enter the tap net names to generate the current source files with locations on the non-transistor-terminal diffusion contacts separately. This option is useful when the design being studied uses a different power or ground net (other than the main power or ground net) to connect to the non-transistor-terminal diffusion contact cuts.

- Skip cells – Select and click the Browse button to specify the cell to be exclude from the current source file generation.

The Skip Cells From CSF Generation dialog box opens.



- Sort by Instance Count – Lists the child cells in the order of cell instance count.
- Sort by Alphabet – Lists the child cells in the order of cell name.
- In the Child Cells box, select any child cells to be excluded.
- In the Cells box, the name of the cell is shown. This cell name is the same as the input cell name.

Click OK to save the chosen skip cells and close the dialog box.

Click Select All to select all the child cells as skip cells.

Click Deselect All to deselect all the selected cells.

- Attach CSF to CONN View – Attaches the generated current source files to the CONN view.

See Also

[auExtractBlockagePinVia](#) in the Physical Implementation Online Help

[poCreateConnViewByHerc](#)

poCreateConnView

Generates connectivity views for nonstandard cells, such as hard macro cells, I/O pad cells, power cells, and cover cells. Astro-Rail uses these views in rail analysis by taking into account the effects of their internal power and ground network and power consumption on voltage drop and electromigration.

When the `poCreateConnView` command is used to create the CONN view for a hierarchical cell, the cell's text needs to be designed hierarchically from the bottom level to the top level. Astro-Rail uses the text to identify a pin when the text is located inside some pin geometry. The bottom-level pins are used for connectivity extraction among the same-level or upper-level cell geometries.

If text is missing at the bottom level of a pin or is not designed hierarchically, `poCreateConnView` will not be able to find the corresponding bottom-level geometry of the pin. If this is the case, use the `poCreateConnViewByHerc` command (see [page 49-27](#)) instead.

Prerequisite

If you want to create a CONN view for a hierarchical cell, you need to first create a SMASH view, using the `cmSmash` command (see [“cmSmash” on page 22-65](#)). Otherwise, you must provide the hierarchical information of the nets in the Create CONN View dialog box.

Tool and Menu Command

Astro: Power > Data Preparation – Create Connectivity View

Astro-Rail: Design Setup > Create Connectivity View

Create Connectivity View Dialog Box

The command opens the following dialog box. Its options are described below.

Create Connectivity View

OK

Cancel

Default

Apply

Help

Library Name

Browse...

Cell Name

Browse...

Hierarchical Extraction

☐

On/Off

Unknown Cell Type

☐

As Macro

☒

Ignore

☐ Delete SMASH View After CONN View Generation

☒ Power/Ground Port Only

Hierarchical Power/Ground Net Information

Net Name	Net Type	Port Pattern(s)
	☒ power ☐ ground	☐ pattern match
	☒ power ☐ ground	☐ pattern match
	☐ power ☒ ground	☐ pattern match
	☐ power ☒ ground	☐ pattern match

Information From File

Extract Pin by Text

Check Hierarchy

☒ Identify Pin By Pin Text

☒ Extract Connectivity

through

☐ polyCont

☒ via1

☒ via2

☒ via3

☒ via4

☒ via5

☒ via6

☒ via7

☒ via8

☒ via9

☒ via10

☒ via11

☐ text fall through

☐ double fall through

Poly Text		Metal1 Text	
Metal2 Text		Metal3 Text	
Metal4 Text		Metal5 Text	
Metal6 Text		Metal7 Text	
Metal8 Text		Metal9 Text	
Metal10 Text		Metal11 Text	
Metal12 Text		Fall Through Text	

Library Name

Enter the name of the library or click Browse to select a name from the list that appears.

Cell Name

Enter the name of the cell for which you want to generate the CONN view, or click Browse to select a name from the list that appears.

Hierarchical Extraction

When this option is on, you must also provide top-level power and ground net naming information in the Hierarchical Power/Ground Net Information table later in the dialog box.

When this option is deselected and the input cell is hierarchical, Astro-Rail checks the SMASH view of the given cell and employs the flattened approach to create the CONN view.

Unknown Cell Type

Select to skip any flattened cell with an unknown cell type; in other words, no CONN view will be generated for unknown flattened cells. If your input cell is hierarchical, use this option to mark each child cell with power and ground objects with the correct cell type, and to mark those child cells without power and ground objects of “unknown” cell types by using the `cmMarkCellType` command (see [“cmMarkCellType” on page 22-36](#)). From a performance aspect, it is important to carefully mark the cell type of each child cell. If you do this, Astro-Rail does not need to go through each geometry in an unknown cell or check to see whether the geometry is a power and ground object. Consequently performance can be improved.

If all of the hierarchical child cells are of the unknown cell type but all have power and ground objects, you can select the As Macro option to treat all unknown cells as macro cells.

The Ignore option is selected by default. Selecting this option is not a good discipline for data preparation, but the option is provided as a convenience.

Delete SMASH View After CONN View Generation

Select to delete the SMASH view of the input cell after the resulting CONN view is created. The default is off.

Power/Ground Port Only

To improve performance, select this option to extract only the power and ground pin and via geometries in the CONN view. In a hierarchical cell, this option is always on.

Hierarchical Power/Ground Net Information

Specify the name and net type for each power and ground net to be extracted.

- Net Name – Enter the names of the top-level power and ground nets to be extracted.
- Net Type – Select the correct net type (power or ground) for each net.
- Port Pattern(s) – Enter all possible port names that are under the net you specify in the Net Name text field. This ensures that the port name and the net name can be consistent in the CONN view. Specify the port patterns when
 - The naming of the power and ground ports is different from that in the top-level net
 - The naming in the reference libraries is inconsistent with that in the design library

Note:

The port pattern information is necessary for a hierarchical cell but is optional for a flat cell. If you enter the information for a flat cell, performance will be faster.

Information From File

Specify the name of the file that contains additional information on the power and ground nets in the ASCII format. This option is useful when you have more than four power and ground nets or port patterns to extract. The format of the file is

```
netName, netType [, portPattern, patternMatch]
```

where `portPattern` and `patternMatch` are optional.

For example, to extract all pins under the VDD power net when \$vdd\$ is used to identify the VDD power pin without pattern matching, specify the information on the VDD power net in the file as follows:

```
VDD, power, VDD*, patternMatch
```

To comment a line, place a semicolon (;) or a pound sign (#) at the beginning of the line.

Extract Pin by Text

Click to view additional pin extraction options.

Check Hierarchy

Select to write out the hierarchical cell relationship and the current CONN view generation status for all child cells to the log file before or after running the `poCreateConnView` command.

For some hierarchical gdsIn data, an empty CONN view is generated for a child cell when no text exists in the child cell and a pin needs to be identified by text.

When several power and ground geometries exist in such a cell, the cell is called power-and-ground bridge cell, or bridge cell. That is, the power and ground geometries in the bridge cell are identified by connecting to the power and ground geometries of adjacent cells.

For better performance, it is recommended that you check only the first level of geometry of the bridge cell to see whether the bridge cell connects to the adjacent cells. If you want all levels of geometries of the bridge cell to be connected, check the adjacent cells of this hierarchical bridge cell instead. You might need to provide a SMASH view for such a nontext hierarchical bridge cell. The status of the nontext hierarchical gdsIn cell is shown by the phrase “(may need a SMASH view)” in the log file.

Identify Pin By Pin Text

For the CEL view with gdsin or steamin, select this option to identify pins by text, text fall-through, or double fall-through in the same manner as with the `auExtractBlockagePinVia` command (see [“auExtractBlockagePinVia” on page 14-57](#)). The default is on.

Extract Connectivity

Select if you want Astro-Rail to trace the connectivity of a specified multiple-layer pin and extract the entire pin. If the layer numbers of texts for pins are identical to the corresponding metal layer numbers, you might leave the following text layer number fields blank. Otherwise you need to specify the appropriate text layer numbers in the corresponding fields, as defined in the technology file.

Specify the vias for extracting the connectivity through

- polyCont
- via1

- via2
- via3
- via4
- via5
- via6
- via7
- via8
- via9
- via10
- via11

The connectivity can be extracted through any specified via or vias. The default is to extract connectivity through via1 to via11 (metal1 to metal12) when the Extract Connectivity option is selected.

text fall through

Select and specify the Fall Through Text layer if you want Astro-Rail to look at the layers beneath the origin of each text label to identify the geometry annotated by the text. If more than one geometry exists beneath the origin of the text label, Astro-Rail selects the geometry to associate with the label according to the following priorities, from highest to lowest:

- Fall Through Text
- Metal12
- Metal11

- Metal10
- Metal9
- Metal8
- Metal7
- Metal6
- Metal5
- Metal4
- Metal3
- Metal2
- Metal1
- Poly Text

For example, if one of the geometries is metal3, Astro-Rail selects this geometry. If not, Astro-Rail looks for metal2, then metal1, and then poly.

If the text layers match the layers for the geometries they identify, deselect the “text fall through” option. You do not need to specify the text layers associated with the interconnection metals. In other words, when text layers are not specified (that is, the “text fall through” option is not selected), Astro-Rail assumes they match the geometry layers.

double fall through

Select and specify the Fall Through Text layer if you want Astro-Rail to look at the layers beneath the origin of each text label on a metal2 geometry to identify a geometry (in addition to the metal2 geometry) annotated by the text. If more than one

geometry (except the metal2 geometry) exists beneath the origin of the text label, Astro-Rail selects the second geometry to associate with the label according to the following priorities, from highest to lowest:

- Poly Text
- Metal1
- Metal2
- Metal3
- Metal4
- Metal5
- Metal6
- Metal7
- Metal8
- Metal9
- Metal10
- Metal11
- Metal12
- Fall Through Text

For example, if one of the geometries is metal1, Astro-Rail selects this geometry. If not, Astro-Rail looks for poly.

You can do the following with each text label:

- Poly Text – Enter the name or number of the layer used for the text identifying poly geometries (or leave blank if the text layer matches the geometry layer).

- Metal# Text – Enter the name or number of the layer used for the text identifying metal# geometries (or leave blank if the text layer matches the geometry layer).
- Fall Through Text – Enter the name or number of the layer used for text if you select “text fall through” or “double fall through.”

See Also

[auExtractBlockagePinVia](#)

[cmMarkCellType](#)

[cmSmash](#)

[poConnViewPreparation](#)

[poCreateConnViewByHerc](#)

poCreateConnViewByHerc

Automatically invokes the Hercules connectivity engine to generate connectivity (CONN) views for nonstandard cells to be used in the rail analysis.

The `poCreateConnViewByHerc` command does not require the design in question to have text at all levels of the hierarchy. Astro-Rail uses the values specified in the Create CONN View By Hercules dialog box and those in the Milkyway library's technology file to create a Hercules runset file, called `runset.ev`, in the working directory. The Hercules connectivity engine then uses this file and creates a SMASH view in the database. Because the Hercules

connectivity engine takes the user-specified net names and propagates the net names throughout the design, you do not need to have the text at each hierarchical level in the design.

After the SMASH view is created, the command uses this data to create a CONN view for the cell in question in the same library.

If the memory usage of the process is too high, use the `poConnViewPreparation` command instead (see [page 49-3](#)).

Astro-Rail updates the `runset.ev` file each time you rerun the command.

Note:

Because the Hercules engine runs on the CEL view of the cell, not GDSII data, make sure the cell in question exists in the Milkyway database before running this command.

When running `poCreateConnViewByHerc` to create the `runset` file, the tool merges all the metal layers and data types. To avoid this, set `"nomask=1"` in the technology file.

Using `poCreateConnViewByHerc` to Generate CONN Views for Hard Macros

The `poCreateConnViewByHerc` command offers a more straightforward approach to generate CONN views for hierarchical hard macros. Astro-Rail does not run the Hercules engine inside the tool, and there is no Hercules software coded inside Astro-Rail. When you click Apply or OK in the Create CONN View By Hercules dialog box, an independent job starts in the same UNIX directory where Astro-Rail is installed. Therefore you need to be aware of the following:

- Before you start Astro-Rail, make sure Hercules is installed in the path where Astro-Rail is installed.
- The Star-RCXT package of executable files distributed by Synopsys no longer contains the Hercules executable files. When you use Star-RCXT, you must download the appropriate files for Hercules and save them to the path where Astro-Rail is installed.
- The license package that comes with the basic Hercules or Star-RCXT tool allows the `poCreateConnViewByHerc` runset to be executed. Either one is acceptable.
- You will not see either of the licenses checked out by Astro-Rail with any license-querying function inside the tool. The licenses have to exist in your system before Astro-Rail is invoked.

Tool and Menu Command

Astro-Rail: Design Setup > Create CONN View By Hercules

Create CONN View By Hercules Dialog Box

The command opens the following dialog box. Its options are described below.

Create CONN View By Hercules

OK Cancel Default Apply Help

Library Name Browse...

Cell Name Browse...

Hierarchical Power/Ground Net Information

Net Name	Net Type	Port Pattern(s)	
	☒ power ☒ ground		☐ pattern match
	☒ power ☒ ground		☐ pattern match
	☒ power ☒ ground		☐ pattern match
	☒ power ☒ ground		☐ pattern match

Information From File

Extract Pin by Text

☒ Identify Pin By Pin Text

☒ Extract Connectivity

through ☐ polyCont ☒ via1 ☒ via2 ☒ via3 ☒ via4 ☒ via5

☒ via6 ☒ via7 ☒ via8 ☒ via9 ☒ via10 ☒ via11

☐ text fall through ☐ double fall through

Poly Text		Metal1 Text	
Metal2 Text		Metal3 Text	
Metal4 Text		Metal5 Text	
Metal6 Text		Metal7 Text	
Metal8 Text		Metal9 Text	
Metal10 Text		Metal11 Text	
Metal12 Text		Fall Through Text	

Note:

You should close the library and cell of interest before running this command.

Library Name

Enter the name of the library or click Browse to select a name from the list that appears.

Cell Name

Enter the single cell name for which you want to generate the CONN view, or click Browse to select a name from the list that appears.

Hierarchical Power/Ground Net Information

Specify the required information for the power and ground nets to be extracted.

- Net Name – Enter the names of the top-level power and ground nets to be extracted.
- Net Type – Select the net type (power or ground) for each net.
- Port Pattern(s) – Enter all possible port names that are under the net you specify in the Net Name text field. This ensures that the port name and the net name can be consistent in the CONN view. Specify the port patterns when
 - The naming of the power and ground ports is different from that in the top-level net
 - The naming in the reference libraries is inconsistent with that in the design library

Note:

The port pattern information is necessary for a hierarchical cell but is optional for a flat cell. If you enter the information for a flat cell, performance will be faster.

Information From File

Specify the name of the file that contains additional information on the power and ground nets in the ASCII format. This option is useful when you have more than four power and ground nets or port patterns to be extracted. The format of the file is

```
netName, netType [, portPattern, patternMatch]
```

where `portPattern` and `patternMatch` are optional.

For example, to extract all pins under the VDD power net when the pattern `$vdd$` is used to identify the VDD power pin without pattern matching, specify the information on the VDD power net in the file as follows:

```
VDD, power, VDD*, patternMatch
```

To comment a line, place a semicolon (;) or a pound sign (#) at the beginning of the line.

Extract Pin by Text

Select to view additional pin extraction options.

Identify Pin By Pin Text

For the CEL view with `gdsin` or `streamin`, select to identify pins by text, text fall-through, or double fall-through in the same manner as with the `auExtractBlockagePinVia` command. The default is on.

Extract Connectivity

Select if you want Astro-Rail to trace the connectivity of a specified multiple-layer pin and extract the entire pin. If the layer numbers of texts for pins are identical to the corresponding metal layer numbers, you can leave the following text layer number fields blank. Otherwise you need to specify the appropriate text layer numbers in the corresponding fields, as defined in the technology file. The default is on.

Specify the vias for extracting the connectivity through

- polyCont
- via1
- via2
- via3
- via4
- via5
- via6
- via7
- via8
- via9
- via10
- via11

The connectivity can be extracted through any specified vias. The default is to extract connectivity through via1 to via11 (metal1 to metal12) when the Extract Connectivity option is selected.

text fall through

Select and specify the Fall Through Text layer if you want Astro-Rail to look at the layers beneath the origin of each text label for identifying the geometry annotated by the text. If more than one geometry exists beneath the origin of the text label, Astro-Rail selects the geometry to associate with the label according to the following priorities, from highest to lowest:

- Fall Through Text
- Metal12
- Metal11
- Metal10
- Metal9
- Metal8
- Metal7
- Metal6
- Metal5
- Metal4
- Metal3
- Metal2
- Metal1
- Poly Text

For example, if one of the geometries is metal3, Astro-Rail selects this geometry; otherwise Astro-Rail looks for metal2, then metal1, and then poly.

When the text layers match the layers for the geometries they identify, deselect the “text fall through” option. You do not need to specify the text layers associated with the interconnection metals. In other words, when text layers are not specified (that is, the “text fall through” option is not selected), Astro-Rail assumes they match the geometry layers.

double fall through

Select and specify the Fall Through Text layer if you want Astro-Rail to look at the layers beneath the origin of each text label on a metal2 geometry to identify a geometry (in addition to the metal2 geometry) annotated by the text. If more than one geometry (except the metal2 geometry) exists beneath the origin of the text label, Astro-Rail selects the second geometry to associate with the label based on the following priorities, from highest to lowest:

- Poly Text
- Metal1
- Metal2
- Metal3
- Metal4
- Metal5
- Metal6
- Metal7
- Metal8
- Metal9
- Metal10

- Metal11
- Metal12
- Fall Through Text

For example, if one of the geometries is metal1, Astro-Rail will select this geometry; otherwise Astro-Rail looks for poly.

You can do the following with each text label:

- Poly Text – Enter the name or number of the layer used for the text identifying poly geometries (or leave blank if the text layer matches the geometry layer).
- Metal# Text – Enter the name or number of the layer used for the text identifying metal# geometries (or leave blank if the text layer matches the geometry layer).
- Fall Through Text – Enter the name or number of the layer used for text if you select “text fall through” or “double fall through.”

See Also

[auExtractBlockagePinVia](#) in the [Physical Implementation Online Help](#)

[poConnViewPreparation](#)

poDumpCurrSource

Writes the current source data of a hard macro cell from a CONN view attached file to a text file.

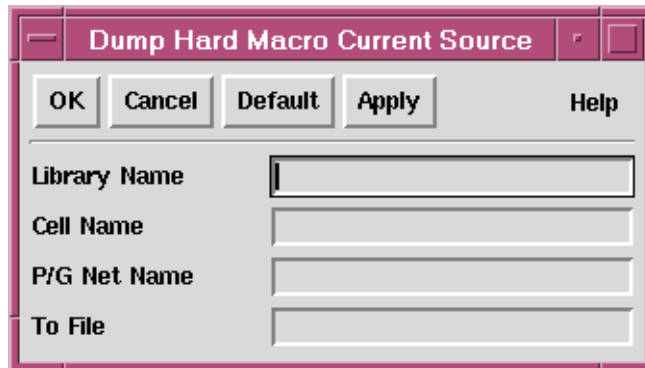
Tool and Menu Command

Astro: Power > Report – Dump Macro Current Source

Astro-Rail: Report > Dump Macro Current Source

Dump Hard Macro Current Source Dialog Box

The command opens the following dialog box. Its options are described below.



Library Name

Enter the name of the library.

Cell Name

Enter the name of the hard macro cell for which you want to dump the current source data.

P/G Net Name

Enter the name of the power and ground net.

To File

Enter the name of the current source file to be created.

See Also

[“poGenCurrSource” on page 49-39](#)

[poTxGenCurrSource](#)

poFillPolygonsToRectangles

Converts polygons to rectangles in the input cell's FILL view and then stores the modified cell in the FILL view of the specified output cell. The input cell is not modified unless the output cell name is the same as the input cell name.

Tool

Astro

Milkyway

Equivalent Tcl Command

```
convert_fill_polygons
```

Syntax

```
poFillPolygonsToRectangles <libraryName> <cellName>  
<newCellName>
```

where the arguments are as follows:

libraryName

Specifies the name of the Milkyway library containing the design cell to be modified.

cellName

Specifies the name of the design cell to be modified.

newCellName

Specifies the name of the output cell that contains the converted rectangles.

poGenCurrSource

Generates current sources of macro cells for all power and ground nets and saves them in the files attached to the CONN view.

Running this command is considered to be performing power analysis on a CONN view cell.

Tool and Menu Command

Astro: Power > Analysis – Generate Macro Current Source

Astro-Rail: Power Analysis > Generate Macro Current Source

Generate Hard Macro Current Source Dialog Box

The command opens the following dialog box. Its options are described below.

Generate Hard Macro Current Source

OK Cancel Default Apply Help

Library Name

Cell Name

◆ Output Current Source

Intersection Type

☒ polyCont ☐ Metal 1 - Metal 2

☐ Metal 2 - Metal 3 ☐ Metal 3 - Metal 4

☐ Metal 4 - Metal 5 ☐ Metal 5 - Metal 6

☐ Metal 6 - Metal 7 ☐ Metal 7 - Metal 8

☐ Metal 8 - Metal 9 ☐ Metal 9 - Metal 10

☐ Metal 10 - Metal 11 ☐ Metal 11 - Metal 12

✓ Input Current Source

Net Name

File Name

Library Name

Enter the name of the library.

Cell Name

Enter the name of the macro cell for which you want to generate current sources.

Output Current Source

Select to generate the current sources of the hard macro cells for all power and ground nets.

- Intersection Type – Generate current sources for the GDSII hard macros for any user-specified adjacent metal layers. The default intersection is between Metal 1 and Metal 2.

Input Current Source

Select to load and attach a current source file to the CONN view.

- Net Name – Enter the name of the net whose current sources you want to read in.
- File Name – Enter the name of the current source file to be loaded.

Note:

For macros that are placed and routed, use the `poGenWhiteBox` command instead.

See Also

[poGenWhiteBox](#)

[poPurgeCurrSource](#)

[poTxGenCurrSource](#)

poNotPolygons

This command performs Boolean operations on polygons. Specifically it returns a list of polygons that cover the areas covered by a first polygon, but not covered by a second polygon.

Syntax

```
poNotPolygons <poly1> <poly2>
```

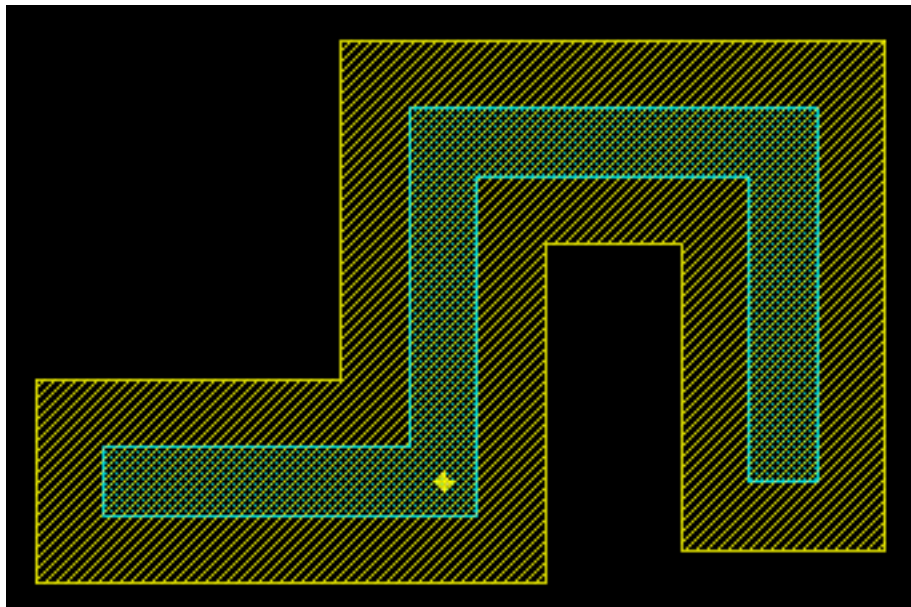
poOrPolygons

Documentation not available.

poOversizePolygon

This command performs returns a list of polygons with edges that have been pushed outwards (away from the area covered by a polygon) by a specified distance.

The size can be negative to undersize the polygon, in which case the edges are pushed inwards. The result of an oversize operation is shown below:



Syntax

```
poOversizePolygons <poly> <size>
```

poOversizeUndersizePolygon

This command performs an oversize operation immediately followed by an undersize operation of the given distance. The result of this operation fills any gaps less than twice the size units. Note that this procedure returns a single polygon, in contrast to that returned by poOversizePolygon.

Syntax

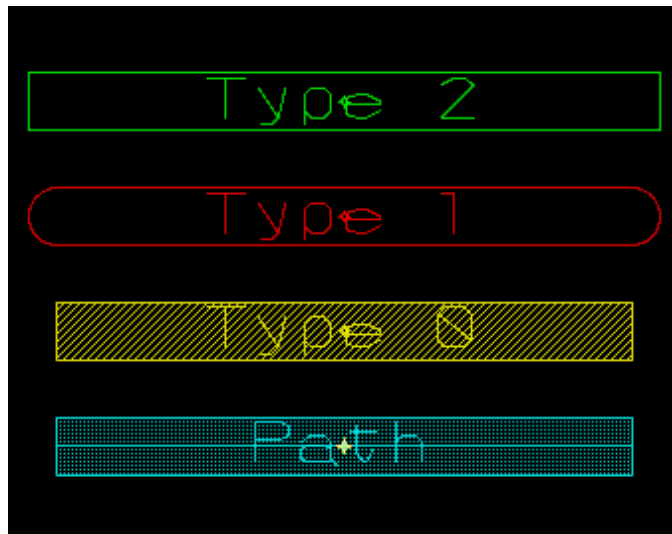
```
poOversizeUndersizePolygons <poly1> <size>
```

poPathToPolygon

This command converts a path (list of points and width) to a polygon. The type indicates how the ends should be finished. The possible values are as follows:

- 0: The ends are square and not extended
- 1: The ends are rounded (circular arcs are approximated using a 32-side polygon)

- 2: The ends are square and extended by half the path width as shown in the following diagram:



Syntax

```
poOversizeUndersizePolygons <poly1> <width> <type>
```

poPolygonArea

This command returns the area covered by a polygon.

Syntax

```
poPolygonArea <poly>
```

poPolygonToRectangles

This command converts a polygon into a list of rectangles. The result is that the rectangles are mutually exclusive. A 5-point polygon, which forms a rectangle. For example a list of points as follows:

((0 0) (0 1) (1 1) (1 0) (0 0))

Syntax

poXorPolygons <poly>

poPurgeCurrSource

Deletes the current sources of macro cells for a power or ground net.

Prerequisite

Because Astro-Rail saves the current sources in the files that are attached to the CONN view, make sure the CONN view is open before you run this command.

Tools

Astro

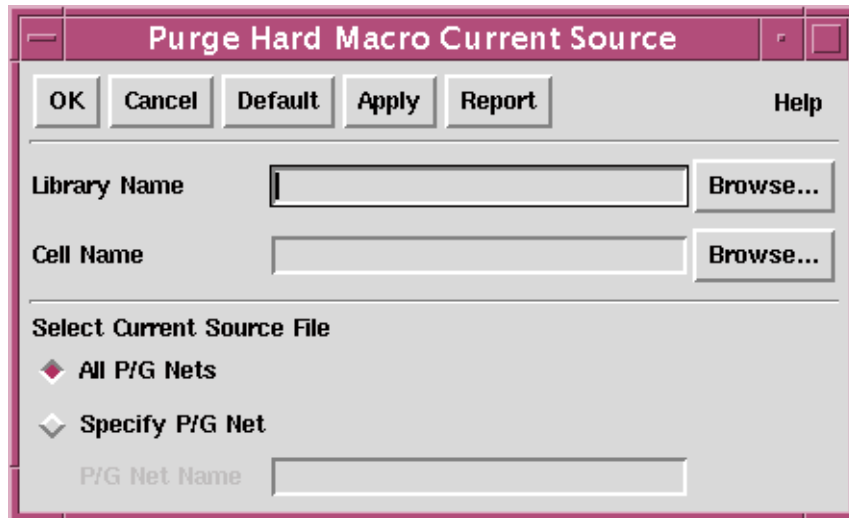
Astro-Rail

Syntax

poPurgeCurrSource

Purge Hard Macro Current Source Dialog Box

The command opens the following dialog box. Its options are described below.



Library Name

Enter the name of the design library or click Browse to select a name from the list that appears.

Cell Name

Enter the name of the cell whose current sources are to be deleted, or click Browse to select a name from the list that appears.

Select Current Source File

Specify the net whose current sources are to be deleted.

- All P/G Nets – Delete the current sources of all the power and ground nets in a cell.
- Specify P/G Net – Delete the current sources of a specific power or ground net in a cell. Enter the name of the power or ground net in the P/G Net Name text field.

poXorPolygons

Performs Boolean operations on polygons. Specifically it returns a list of polygons that cover the areas covered by either polygon, but not both.

Syntax

```
poXorPolygons <poly1> <poly2>
```


51

pvr Commands for Milkyway

This chapter describes commands with the prefix *pvr*. The commands are listed alphabetically.

- [pvrDSPFn](#)
- [pvrSPEFn](#)

pvrDSPFIn

pvrDSPFIn reads the RC information of nets from the given DSPF file, stores it into the Milkyway database, and then generates the parasitic and netlist views of the cell.

Tools and Menu Commands

Astro: Timing > Parasitic Input - DSPF In

Prerequisites

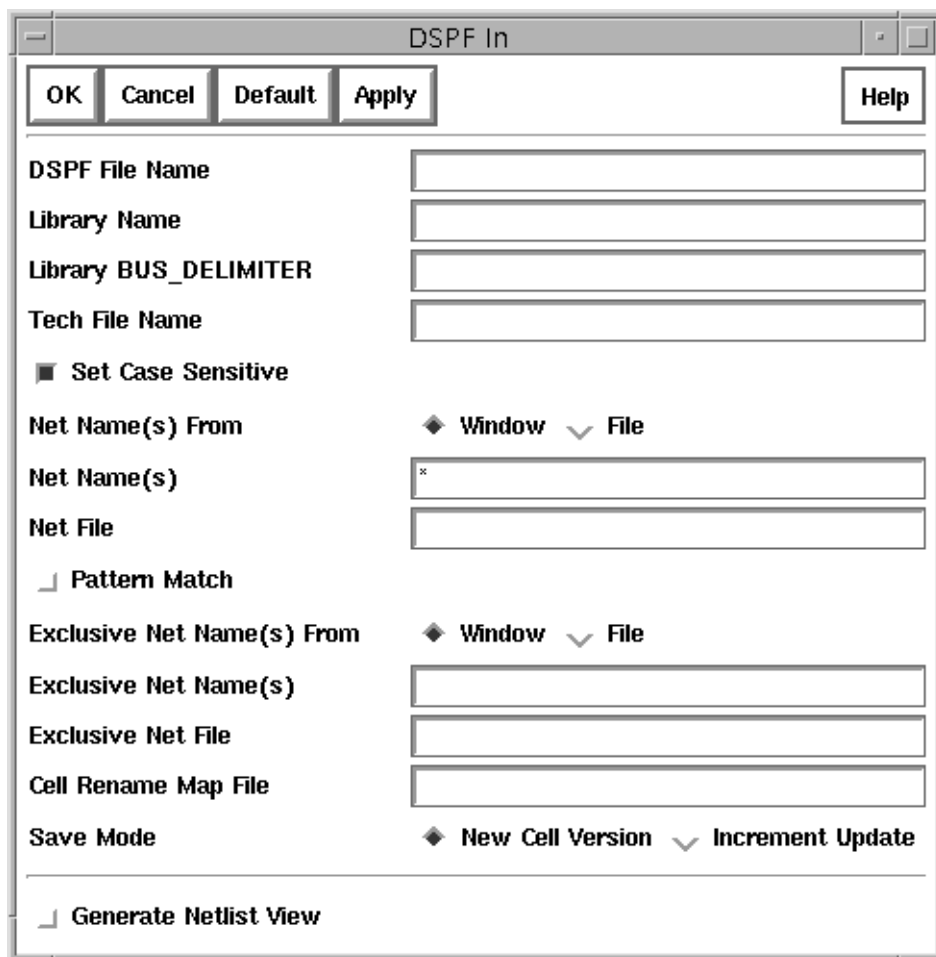
Prepare the DSPF file that you want to read.

Prepare a library name when the library you need is not open.

Prepare a technology file name when the library you need does not exist.

DSPF In Dialog Box

The menu command opens the following dialog box. Its options are described below.



DSPF In

OK Cancel Default Apply Help

DSPF File Name

Library Name

Library BUS_DELIMITER

Tech File Name

☒ Set Case Sensitive

Net Name(s) From ◆ Window ▾ File

Net Name(s)

Net File

☐ Pattern Match

Exclusive Net Name(s) From ◆ Window ▾ File

Exclusive Net Name(s)

Exclusive Net File

Cell Rename Map File

Save Mode ◆ New Cell Version ▾ Increment Update

☐ Generate Netlist View

DSPF File Name

Type the name of the DSPF file.

Library Name

Type the library name when the library you need is not open. If this library is not the same as the current opened library, open the user typed library. If this library does not exist, create a new library.

Library BUS_DELIMITER

Specify the library bus delimiter. If it is not specified, use the library's bus name style. Use the default "[%d]" if the library has no bus name style.

Tech File Name

When the library you need does not exist, you should also enter the technology file name.

Set Case Sensitive

When creating a new library, set whether the library is case sensitive or not. Its default value is "1."

Net Name(s) From

Determine the method net names are specified by selecting either *Window* or *File*. *Window* is the default option, and is specified through the Net Name(s) field. Multiple net names can be entered with a comma in-between, or select *Pattern Match* and type a pattern to match the names. When *File* is selected, the net names are determined in the Net File field.

Net Name(s)

Type the name of the net that will read the RC information from the DSPF file. The default net name is "*", representing all net names.

Net File

Specify the name of the file containing the net name that will read the RC information from the DSPF file.

Pattern Match

Select to read DSPF information for all nets matching the pattern specified in the Net Name field. For example, typing "N*" in the Net Name field and selecting *Pattern Match* will read the DSPF

information for all net names beginning with the letter “N.” Use the pattern matching option to select all nets with names that match a pattern.

If you deselect *Pattern Match*, you can specify multiple nets as follows:

- Type multiple net names separated with commas.
- Type an asterisk (*) to specify all nets.

Exclusive Net Name(s) From

Select *Window* to choose the *Exclusive Net Name(s)* extraction input from a window list, or select *File* to choose the input information from a file, with the *Exclusive Net Name(s)* information in ASCII (standard text) format.

Exclusive Net Name(s)

If an *Exclusive Net Name* is chosen by selecting *Window*, you should input the net name(s) that you do not want RC information read from in the *Exclusive Net Name* entry. The default net name is "". When typing multiple names, separate the names with a comma.

Exclusive Net File

If an *Exclusive Net Name* is determined by selecting *File*, you should input a file name in the *Exclusive Net File* entry. The net names should also be separated by a comma in the *Exclusive Net File*.

Cell Rename Map File

If you have a *Cell Rename Map File*, type the name in the *Cell Rename Map File* box. In the *Cell Rename Map File*, each line should only have two names separated by one blank space. The

first name should be the cell name in the SPEF file, and the second name should be the map name in Milkyway. Any file listed that cannot meet these requirements will cause an error.

Save Mode

Select a .PARA view generation mode, it can either be *New Cell Version* or *Increment Update*. Choose either *New Cell Version* to generate a new cell version and keep the old one, or *Increment Update* to update information about pointed nets based on the old parasitic view.

Generate Netlist View

Select whether or not to generate netlist view. If it is set “on,” it will create a new netlist view.

See Also

astDSPFOut reports interconnect capacitance and resistance for all nets in *Detailed Standard Parasitic Format* (DSPF).

astSPEFOut reports interconnect capacitance and resistance for all nets in *Standard Parasitic Exchange Format* (SPEF).

[pvrSPEFIn](#) reads the RC information of nets from the SPEF file, and generates the parasitic view of cell.

pvrSPEFIn

The *pvrSPEFIn* command reads the RC information of nets from the given SPEF file, stores it into the Milkyway database, and generates the parasitic and netlist views of the cell.

Tools and Menu Commands

Astro: Timing > Parasitic Input - SPEF In

Prerequisites

Prepare the SPEF file that you want to read. Prepare a library name if the library you need is not open. Prepare a technology file name if the library you need does not exist.

SPEF In Dialog Box

The menu command opens the following dialog box. Its options are described below.

SPEF In

OK Cancel Default Apply Help

SPEF File Name

Library Name

Library BUS_DELIMITER

Tech File Name

☒ Set Case Sensitive

Net Name(s) From ☒ Window ☐ File

Net Name(s)

Net File

☐ Pattern Match

Exclusive Net Name(s) From ☒ Window ☐ File

Exclusive Net Name(s)

Exclusive Net File

Cell Rename Map File

DB Content Mode ☒ Basic ☐ MinMax ☐ Geometry

DB Create Mode ☒ New Cell Version ☐ Incremental Cell Version

☐ Generate Netlist View

SPEF File Name

Type the name of the SPEF file.

Library Name

Type the library name if the library you need is not open. If this library is not the same as the current opened library, open the user typed library. If this library does not exist, create a new library.

Library BUS_DELIMITER

Specify the library bus delimiter. If it is not specified, use the library's bus name style. Use the default "[%d]" if the library has no bus name style.

Tech File Name

If the library you need does not exist, you should also type the technology file name.

Set Case Sensitive

When you create a new library, set whether the library is case sensitive or not. Its default value is "1".

Net Name(s) From

Determine the method net names are specified by selecting either *Window* or *File*. *Window* is the default option, and is specified through the Net Name(s) field. When *File* is selected, the net names are determined in the Net File field.

Net Name(s)

If the net name method is determined by selecting *Window*, type the net name(s) where you want the RC information read. The default net name is "*", representing all net names. When typing multiple net names, separate them with a comma.

Net File

If the net name method is determined by selecting *File*, specify the file name containing the net names where the RC information will be read. The net names in the net file should also be separated with a comma.

Pattern Match

Select to read SPEF information for all nets matching the pattern specified in the Net Name field. For example, typing "N*" in the Net Name field and selecting *Pattern Match* will read the SPEF information for all net names beginning with the letter "N." Use the pattern matching option to select all nets with names that match a pattern.

If you deselect *Pattern Match*, specify multiple nets as follows:

- Type multiple net names separated with commas.
- Type an asterisk (*) to specify all nets.

Exclusive Net Name(s) From

Select Window to choose the *Exclusive Net Name(s)* extraction input from a window list, or select *File* to choose the input information from a file, with the *Exclusive Net Name(s)* information in ASCII (standard text) format. All the information for exclusive nets will be skipped in the later parse.

Exclusive Net Name(s)

If an *Exclusive Net Name* is chosen by selecting *Window*, you should input the net name(s) that you do not want RC information read from in the *Exclusive Net Name* entry. The default net name is "". When typing multiple names, separate the names with a comma.

Exclusive Net File

If an *Exclusive Net Name* is determined by selecting *File*, you should input a file name in the *Exclusive Net File* entry. The net names should also be separated by a comma in the *Exclusive Net File*.

Cell Rename Map File

If you have a *Cell Rename Map File*, type the name in the *Cell Rename Map File* box. In the *Cell Rename Map File*, each line should only have two names separated by one blank space. The first name should be the cell name in the SPEF file, and the second name should be the map name in Milkyway. Any file listed that cannot meet these requirements will cause an error.

DB Content Mode

The three selections in the *DB Content Mode* are *Basic*, *MinMax*, or *Geometry*. The *MinMax* mode is basically the same as the *Basic* mode except that the parasitic value is tripled in *MinMax* mode. When the “Geometry” option is set on, the layer and geometry information of the node and resistor are stored in the database. The geometry and layer information exists in the SPEF file as a comment. For example,

```
// $11x=663.720 $11y=385.200 $urx=664.080 $ury=386.260  
$1v1=2
```

This option only has effect on parasitic views, not on netlist views.

DB Create Mode

Select a .PARA view generation mode, it can either be *New Cell Version* or *Incremental Cell Version*. Choose either *New Cell Version* to generate a new cell version and keep the old one, or *Incremental Cell Version* to update information about pointed nets based on the old parasitic view.

Generate Netlist View

Select whether or not to generate a netlist view. When the option is set “on,” it will create a new netlist view.

See Also

[pvrDSPFIn](#) reads the RC information of nets from a *Detailed Standard Parasitic Format* (DSPF) file, and generates a parasitic view of the cell.

astDSPFOut reports interconnect capacitance and resistance for all nets in Detailed Standard Parasitic format (DSPF).

astSPEFOut reports interconnect capacitance and resistance for all nets in Standard Parasitic Exchange format (SPEF).

54

tdf Commands for Milkyway

This chapter describes commands with the prefix *tdf*. The commands are listed alphabetically.

- [tdfDump](#)
- [tdfDumpIOToTDF](#)
- [tdfDumpPinToTDF](#)
- [tdfDumpPowerInfo](#)
- [tdfPurgeClks](#)
- [tdfPurgeClockPhaseDelays](#)
- [tdfPurgeClockWaveforms](#)
- [tdfPurgeDrivingCell](#)
- [tdfPurgeInputDelay](#)

- `tdfPurgeIOSpaceConstr`
- `tdfPurgeNetCapAndTransTimes`
- `tdfPurgeNetCapTransAndDelayTimes`
- `tdfPurgeNetConstr`
- `tdfPurgeNetTransitionDelays`
- `tdfPurgeNoTouchCellInst`
- `tdfPurgeNoTouchNet`
- `tdfPurgeOutputDelay`
- `tdfPurgePadConstr`
- `tdfPurgeTimeRequirements`
- `tdfResetPath`
- `tdfSetCaseAnalysis`
- `tdfSetClkDef`
- `tdfSetClockPhaseDelay`
- `tdfSetClockPortTransition`
- `tdfSetClockWaveform`
- `tdfSetDisableTiming`
- `tdfSetDrivingCell`
- `tdfSetFalsePath`
- `tdfSetGeneratedClk`
- `tdfSetInputDelay`

- tdfSetInputPortTransition
- tdfSetLoadCap
- tdfSetMaxDelayPath
- tdfSetMinDelayPath
- tdfSetMinIOSpacing
- tdfSetMultiCyclePath
- tdfSetNetCapAndTransTime
- tdfSetNetCapTransAndDelayTime
- tdfSetNetTransitionDelay
- tdfSetNoTouchCellInst
- tdfSetNoTouchNet
- tdfSetOutputDelay
- tdfSetPowerSupply
- tdfSetRestoreTiming
- tdfSetSrcCapacitance
- tdfSetSrcCapacitanceList
- tdfSetSrcResistance
- tdfSetSrcResistanceList
- tdfSetTimeBorrowLimit

tdfDump

Writes the TDF information associated with the cell in the current window to a file.

Tool

Milkyway

Syntax

```
tdfDump (geGetEditCell) fileName
```

where the argument is as follows:

<i>fileName</i>	The name you want to give to the TDF file.
-----------------	--

Example

```
tdfDump (geGetEditCell) "fp.tdf"
```

Typing this function from the input area of the Synopsys window writes the TDF information associated with the cell in the current window to a file named *fp.tdf*.

See Also

[ioDumpTdf](#) writes the TDF information associated with a specified cell to a file. (Displays a dialog box.)

[tdfDumpIOToTDF](#) writes I/O constraints of a specified type to a file in TDF format.

tdfDumpIOToTDF

Writes I/O constraints for a cell to a file in TDF format.

Tool

Milkyway

Syntax

```
tdfDumpIOToTDF cellId constraintType tdfFileName
```

where the arguments are as follows

cellId ID of the cell for which you want to write I/O constraints.
Valid values:
ID of any cell in the open library. If the cell is open, you can substitute the following for *cellId*:
(geGetEditCell).

constraintType Type of constraints you want to include in the file.
Valid values:
Any of the following strings:

- *side* writes constraints that specify the side (or corner) of the chip to place an I/O.
- *order* writes constraints that specify the order of an I/O relative to other IOs on the same side of the chip.
- *location* writes constraints that specify the I/O offset from the bottom or left edge of the boundary.

tdfFileName Name of the file to which you want to write I/O constraints.
Valid values: Any string up to 32 characters.

:

Example

```
tdfDumpIOToTDF (geGetEditCell) "order" "ioCnstr.tdf"
```

Typing this function from the input area of the Synopsys window writes the I/O order constraints associated with the cell in the current window to a file named *ioCnstr.tdf*.

See Also

[ioDumpTdf](#) writes all TDF information associated with a specified cell to a file. (Displays a dialog box.)

[tdfDump](#) writes all TDF information associated with a cell to a file. (Does not display a dialog box.)

tdfDumpPinToTDF

Writes boundary I/O and cPin constraints for a cell to a file in TDF format.

Tool

Milkyway

Syntax

```
tdfDumpPinToTDF cellId constraintType tdfFileName
```

where the arguments are as follows

<i>cellId</i>	ID of the cell for which you want to write pin constraints. Valid values: ID of any cell in the open library. If the cell is open, you can substitute the following for <i>cellId</i> : (geGetEditCell).
---------------	--

<i>constraintType</i>	Type of constraints for boundary I/O that you want to include in the file. Valid values: Any of the following strings: • <i>side</i> writes constraints that specify the side (or corner) of the chip to place an I/O. • <i>order</i> writes constraints that specify the order of an I/O relative to other I/Os on the same side of the chip. • <i>location</i> writes constraints that specify the I/O offset from the bottom or left edge of the boundary.
<i>tdfFileName</i>	Name of the file to which you want to write pin constraints. Valid values: Any string up to 32 characters.

:

Example

```
tdfDumpPinToTDF (geGetEditCell) "order" "ioCnstr.tdf"
```

Typing this function from the input area of the Synopsys window writes the pin constraints associated with the cell in the current window to a file named *ioCnstr.tdf*.

See Also

[ioDumpPinLocation](#) writes all pin information associated with a cell to a file.

[ioDumpTdf](#) writes all TDF information associated with a specified cell to a file. (Displays a dialog box.)

[tdfDump](#) writes all TDF information associated with a cell to a file. (This function does not display a dialog box.)

[tdfDumpIOTdTDF](#) writes only boundary I/O constraints associated with a cell to a file.

tdfDumpPowerInfo

Writes the power supply information of a given cell to a file.

Tool

Astro

Syntax

```
tdfDumpPowerInfo cellId "fileName"
```

where the arguments are as follows:

<i>cellId</i>	ID of the cell for which you want to write power supply information.
<i>fileName</i>	Name of the file to which you want to write power supply information.

See Also

poDumpPowerInfo.

tdfPurgeClks

Instructs Astro to delete all clock source characteristics set previously by [tdfSetClkDef](#) functions.

Tool

Milkyway

Syntax

`tdfPurgeClks`

tdfPurgeClockPhaseDelays

Instructs Astro to delete all clock phase delays set previously by [tdfSetClockPhaseDelay](#).

Tool

Milkyway

Syntax

`tdfPurgeClockPhaseDelays`

tdfPurgeClockWaveforms

Instructs Astro to delete all clock waveforms set previously by [tdfSetClockWaveform](#).

Tool

Milkyway

Syntax

`tdfPurgeClockWaveforms`

tdfPurgeDrivingCell

Instructs Astro to delete all driving cell information set previously by [tdfSetDrivingCell](#) functions.

Tool

Milkyway

Syntax

```
tdfPurgeDrivingCell "portName"
```

where the argument is as follows:

<i>portName</i>	Name of the top-level input port for which you are defining driving cells.
-----------------	--

tdfPurgeInputDelay

Instructs Astro to delete all input delays set previously by [tdfSetInputDelay](#).

Tool

Milkyway

Syntax

```
tdfPurgeInputDelay "portName"
```

where the argument is as follows:

<i>portName</i>	Name of the top-level input port for which you are defining driving cells.
-----------------	--

tdfPurgeIOSpaceConstr

Deletes all *tdf* constraints related to I/O pad spacing. See also [tdfSetMinIOSpacing](#).

Note:

Please be aware that a tdf Scheme command must be in a tdf file.

Tool

Milkyway

Syntax

```
tdfPurgeIOSpaceConstr
```

See Also

[tdfSetMinIOSpacing](#)

tdfPurgeNetCapAndTransTimes

Removes the default capacitance and transition time settings.

Tool

Milkyway

Syntax

```
tdfPurgeNetCapAndTransTimes
```

tdfPurgeNetCapTransAndDelayTimes

Removes the default capacitance and transition time settings.

Tool

Milkyway

Syntax

```
tdfPurgeNetCapTransAndDelayTimes
```

tdfPurgeNetConstr

The *tdfPurgeNetConstr* function instructs the Synopsys tool to delete all net location constraints created previously by *net* functions.

Tool

Milkyway

Syntax

```
tdfPurgeNetConstr
```

tdfPurgeNetTransitionDelays

Instructs Astro to delete all net transition delays set previously by [tdfSetNetTransitionDelay](#).

Tool

Milkyway

Syntax

```
tdfPurgeNetTransitionDelays
```

tdfPurgeNoTouchCellInst

Purges all “No Touch” constraints previously applied to cell instances.

Tool

Milkyway

Syntax

```
tdfPurgeNoTouchCellInst
```

See Also

Use [tdfPurgeNoTouchNet](#) to remove all “No Touch” constraints previously applied to nets.

Use [tdfSetNoTouchNet](#) to apply the “No Touch” constraint to specified cell instances and nets.

tdfPurgeNoTouchNet

Purges the “No Touch” constraint applied to all nets.

Tool

Milkyway

Syntax

```
tdfPurgeNoTouchNet
```

See Also

Use [tdfPurgeNoTouchCellInst](#) to remove all “No Touch” constraints previously applied to cell instances.

Use [tdfSetNoTouchNet](#) to apply the “No Touch” constraint to specified cell instances and nets.

tdfPurgeOutputDelay

Instructs Astro to delete all output delays set previously by [tdfSetOutputDelay](#).

Tool

Milkyway

Syntax

```
tdfPurgeOutputDelay "portName"
```


where the argument is as follows:

<i>portName</i>	Name of the top-level input port for which you are defining driving cells.
-----------------	--

tdfPurgePadConstr

The *tdfPurgePadConstr* function instructs the Synopsys tool to delete all pin or pad location constraints created previously by *pin* or *pad* functions.

Tool

Milkyway

Syntax

```
tdfPurgePadConstr
```

tdfPurgeTimeRequirements

Instructs Astro to delete all timing requirements set previously by any of the following functions:

- [tdfSetFalsePath](#)
- [tdfSetMultiCyclePath](#)
- [tdfSetMaxDelayPath](#)
- [tdfSetMinDelayPath](#)
- [tdfSetTimeBorrowLimit](#)

- [tdfSetDisableTiming](#)
- [tdfSetRestoreTiming](#)
- [tdfResetPath](#)

Tool

Milkyway

Syntax

`tdfPurgeTimeRequirements`

tdfResetPath

Resets timing arcs for specified paths to single-cycle timing, removing delay constraints set by [tdfSetMaxDelayPath](#), [tdfSetMinDelayPath](#), [tdfSetFalsePath](#), and [tdfSetMultiCyclePath](#).

Tool

Milkyway

Syntax

```
tdfResetPath "arcType" "refEdge" '(srcPoints)' (destPoints)
[ '(thruPoints) ]
```

where the arguments are as follows

<i>arcType</i>	Type of timing arcs you are resetting. Valid values:
	• <i>setup</i> indicates setup arcs. • <i>hold</i> indicates hold arcs. • <i>setuphold</i> indicates both setup and hold arcs.

<i>refEdge</i>	Edge of the timing arcs you are resetting. Valid values: • <i>rise</i> indicates rising edges. • <i>fall</i> indicates falling edges. • <i>risefall</i> indicates both rising and falling edges.
<i>srcPoints</i>	Source points of the paths for which you are resetting timing arcs, or "*" to reset timing arcs for all paths ending at the points specified by <i>destPoints</i>. Specify source points in the following format: <i>"srcPt1" "srcPt2"... "srcPtn"</i> where a <i>srcPt</i> can be any of the following: • Name of a top-level input port. • Name of a clock pin on a cell instance. • Name of a clock to indicate all top-level input ports and clock pins related to the clock. • Name of a cell instance to indicate all clock pins on the cell instance.
<i>destPoints</i>	Destination points of the paths for which you are resetting timing arcs, or "*" to reset timing arcs for all paths starting at the points specified by <i>srcPoints</i>. Specify destination points in the following format: <i>"destPt1" "destPt2"... "destPtn"</i> where a <i>destPt</i> can be any of the following: • Name of a top-level output port. • Name of a data input pin on a cell instance. • Name of a clock to indicate all top-level output ports and data input pins related to the clock. • Name of a cell instance to indicate all data input pins on the cell instance.

thruPoints Thru points of the paths for which you are resetting timing arcs, or "*" to reset timing arcs for all paths starting at the points specified by *srcPoints*.

Specify thru points in the following format:

```
"thruGroup1" "thruGroup2"... "thruGroupn"
```

where a *thruGroup* consists of a set of candidate thru points in the following format:

```
thruPt1 thruPt2 ... "thruPtn"
```

where a *thruPt* can be any of the following:

- Name of a top-level output port.
- Name of a data input pin on a cell instance.
- Name of a clock to indicate all top-level output ports and data input pins related to the clock.
- Name of a cell instance to indicate all data input pins on the cell instance.
- Name of a net.

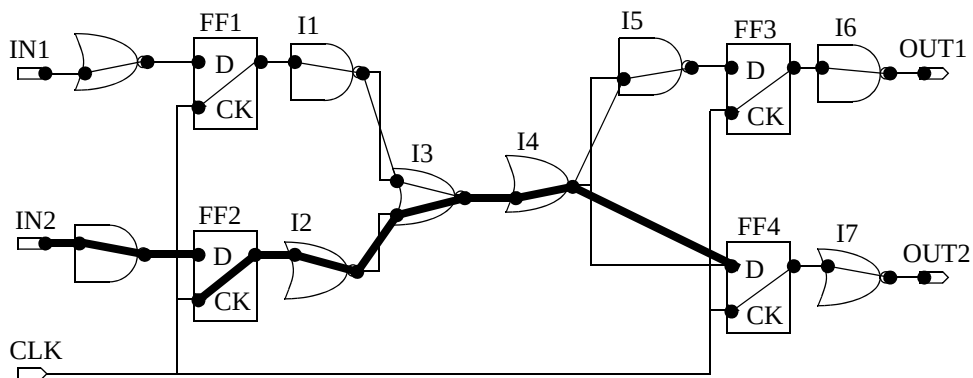
notice that at least one *thruPt* will be picked within each *thruGroup*.

:

Example 1

```
tdfResetPath "setup" "rise" ' ("IN2" "FF2/CK") ' ("FF2/D" "FF4/D")
```

Resets setup-rising timing arcs for the paths shown in bold.



tdfSetCaseAnalysis

Under Construction.

Tool

Milkyway

Syntax

`tdfSetCaseAnalysis`

tdfSetClkDef

Sets source characteristics for a clock, including the following:

- The earliest and latest the clock will rise at the source point relative to time zero (0).
- The earliest and latest the clock will fall at the source point relative to time zero (0).

- The maximum delta between the delay from the source point to any destination point.
- The elapsed time between repetitions of the clock.

You can purge clock definitions using the [tdfPurgeClks](#) function.

Note:

Generally, netlist includes port directions. If not, you need to specify the direction of the source point of the clock using `dbSetPortDirection` command.

Tool

Milkyway

Syntax

```
tdfSetClkDef "sourcePoint" riseMin riseMax fallMin fallMax
            skew period
```

where the arguments are as follows:

sourcePoint The source point of the clock.
Valid values:

- Any top-level input port.
- Any cell instance that is not a flip-flop and output port on that cell instance, in the following format:
cellInstName/outputPortName
where *cellInstName* is the name of the cell instance and *outputPortName* is the name of the output port.
- Any cell instance that is a flip-flop and clock input port on that cell instance, in the following format:
flipFlopInstName/clockInputPortName
where *flipFlopInstName* is the name of the flip-flop and *clockInputPortName* is the name of the clock input port.

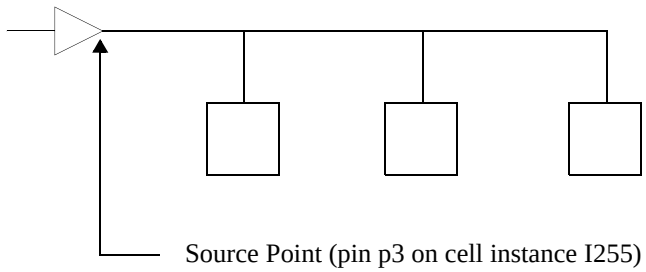
<i>riseMin</i>	Minimum delay between the time zero (0) reference point of the chip and the rise of the clock at the source. Valid values: Any floating-point number less than <i>riseMax</i> .
<i>riseMax</i>	Maximum delay between the time zero (0) reference point of the chip and the rise of the clock at the source. Valid values: Any floating-point number greater than <i>riseMin</i> .
<i>fallMin</i>	Minimum delay between the time zero (0) reference point of the chip and the fall of the clock at the source. Valid values: Any floating-point number less than <i>fallMax</i> .
<i>fallMax</i>	Maximum delay between the time zero (0) reference point of the chip and the fall of the clock at the source. Valid values: Any floating-point number greater than <i>fallMin</i> .
<i>skew</i>	Maximum delta between the delay for any two clocked elements. Valid values: Any floating-point number.
<i>period</i>	Elapsed time between repetitions of the clock (1/frequency). Valid values: Any floating-point number.

Example

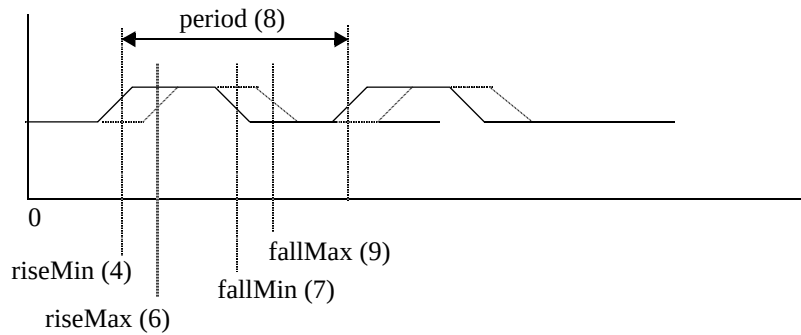
```
tdfSetClkDef "I255/p3" 4 6 7 9 .5 8
```

Using these source characteristics, Astro performs timing-driven layout based on the following:

- The clock will rise at pin *p3* on cell instance *I255* between 4 and 6 ns from time zero (0).
- The clock will fall at pin *p3* on cell instance *I255* between 7 and 9 ns from time zero (0).
- The delta between the delay from the source point to any destination point is less than or equal to 0.5 ns.
- The elapsed time between repetitions of the clock is 8 ns.



Timing Waveform at Source Point



tdfSetClockPhaseDelay

Defines clock delay characteristics, including source delay from the source point to clock port. It also defines network delay from clock port to register clock pin and skew for the clock.

Tool

Milkyway

Syntax

```
tdfSetClockPhaseDelay "objList" sourceDelaySet
networkDelaySet
deviationSet
```


where the arguments are as follows::

<i>objList</i>	Name of the <i>ports</i> , <i>portInsts</i> . Valid values: String.
<i>sourceDelaySet</i>	A value or set of values that expresses the delay at clock from the clock source point to clock port, including the following: • The minimum delay when the source signal is rising. • The minimum delay when the source signal is falling. • The maximum delay when the source signal is rising. • The maximum delay when the source signal is falling. Valid values: Astro needs four values, which you can express in any of the following ways: • A single value, expressed as follows: value where value is a floating-point number that expresses the minimum and maximum delays when the signal is rising or falling. • A list of two values, expressed as follows: '(riseTime fallTime) where • <i>riseTime</i> is a floating-point number that expresses the minimum and maximum delays when the signal is rising. • <i>fallTime</i> is a floating-point number that expresses the minimum and maximum delays when the signal is falling. • A list of four values, expressed as follows: '(minRiseTime minFallTime maxRiseTime maxFallTime)

sourceDelaySet
- *cont.*

where:

- *minRiseTime* is a floating-point number (less than *maxRiseTime*) that expresses the minimum delay when the signal is rising.
- *minFallTime* is a floating-point number (less than *maxFallTime*) that expresses the minimum delay when the signal is falling.
- *maxRiseTime* is a floating-point number (greater than *minRiseTime*) that expresses the maximum delay when the signal is rising.
- *maxFallTime* is a floating-point number (greater than *minFallTime*) that expresses the maximum delay when the signal is falling.

networkDelaySet

A value or set of values that expresses the delay at clock from the clock port to the register clock pin, including the following:

- The minimum delay when the source signal is rising.
- The minimum delay when the source signal is falling.
- The maximum delay when the source signal is rising.
- The maximum delay when the source signal is falling.

Valid values: Astro needs four values, which you can express in any of the following ways:

- A single value, expressed as follows:

value

where *value* is a floating-point number that expresses the minimum and maximum delays when the signal is rising or falling.

- A list of two values, expressed as follows:

'(riseTime fallTime)

where

- *riseTime* is a floating-point number that expresses the minimum and maximum delays when the signal is rising.
- *fallTime* is a floating-point number that expresses the minimum and maximum delays when the signal is falling.

- A list of four values, expressed as follows:

*'(minRiseTime minFallTime
maxRiseTime maxFallTime)*

tdfSetClockPhaseDelay

networkDelaySet
- *cont.*

where

- *minRiseTime* is a floating-point number (less than *maxRiseTime*) that expresses the minimum delay when the signal is rising.
- *minFallTime* is a floating-point number (less than *maxFallTime*) that expresses the minimum delay when the signal is falling.
- *maxRiseTime* is a floating-point number (greater than *minRiseTime*) that expresses the maximum delay when the signal is rising.
- *maxFallTime* is a floating-point number (greater than *minFallTime*) that expresses the maximum delay when the signal is falling.

deviationSet

A value or set of values that expresses the skew at the clock, including the following:

- Positive uncertainty when signal is rising.
- Positive uncertainty when signal is falling.
- Negative uncertainty when signal is rising.
- Negative uncertainty when signal is falling.

Valid values: Astro needs four values, which you can express in any of the following ways:

- A single value, expressed as follows:

value

where value is a floating-point number that expresses deviation (uncertainty).

- A list of two values, expressed as follows:

'(riseDeviation fallDeviation)

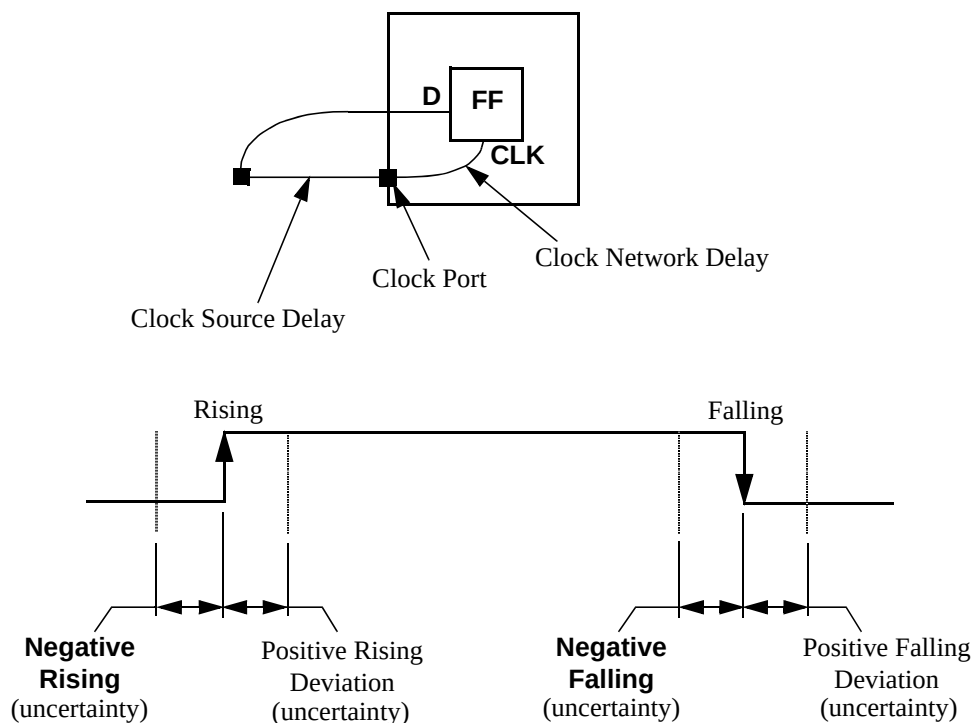
where

- *riseTime* is a floating-point number that expresses the negative deviation and positive deviation when the signal is rising.
- *fallTime* is a floating-point number that expresses the negative deviation and positive deviation when the signal is falling.
- A list of four values, expressed as follows:
'(posRiseDeviation posFallDeviation
negRiseDeviation negFallDeviation)

deviationSet -
cont.

where

- *posRiseDeviation* is a floating-point number that expresses the positive uncertainty when the signal is rising.
- *posFallDeviation* is a floating-point number that expresses the positive uncertainty when the signal is falling.
- *negRiseDeviation* is a floating-point number that expresses the negative uncertainty when the signal is rising.
- *negFallDeviation* is a floating-point number that expresses the negative uncertainty when the signal is falling.



Examples

Example 1

tdfSetClockPhaseDelay

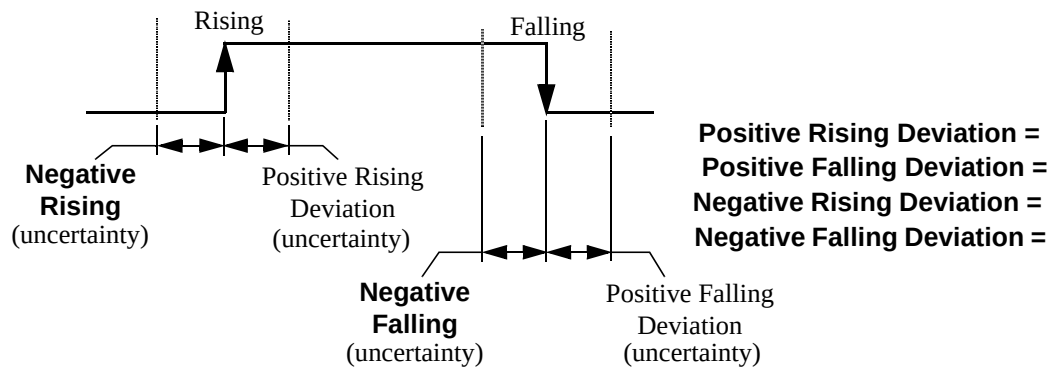
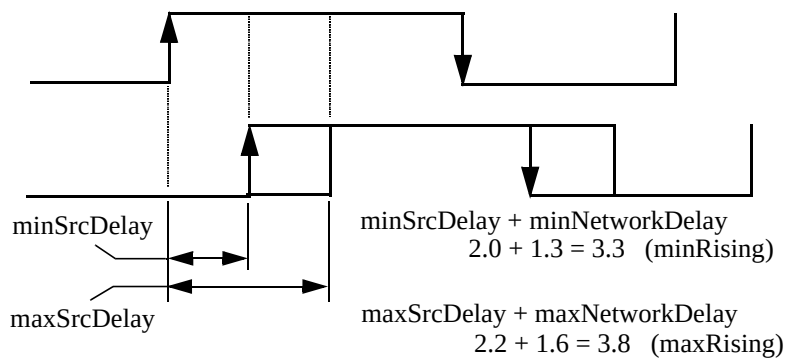
```
tdfSetClockPhaseDelay "clk" 2.1 1.5 0.05
```

Example 2

```
tdfSetClockPhaseDelay "clk" '(2.05 2.05 2.15 2.15) 1.5 0
```

Example 3

```
tdfSetClockPhaseDelay "clk"
'(2.0 2.1 2.2 2.3)
'(1.3 1.4 1.6 1.7)
'(0.01 0.02 0.03 0.04)
```



tdfSetClockPortTransition

Sets the transition time at clock ports and clock port instances. This transition time propagates along the clock network.

Tool

Milkyway

Syntax

```
tdfSetClockPortTransition "objList" transitionDelaySet
```

where the arguments are as follows:

<i>objList</i>	Name for the <i>ports</i> and <i>portInsts</i> . Valid Values: String
<i>transitionDelaySet</i>	A scalar or four-value set, expressed as follows: '(minRiseTime minFallTime maxRiseTime maxFallTime) where: • <i>minRiseTime</i> is a floating-point number (less than <i>maxRiseTime</i>) that expresses the minimum transition time when the signal is rising. • <i>minFallTime</i> is a floating-point number (less than <i>maxFallTime</i>) that expresses the minimum transition time when the signal is falling. • <i>maxRiseTime</i> is a floating-point number (greater than <i>minRiseTime</i>) that expresses the maximum transition time when the signal is rising. • <i>maxFallTime</i> is a floating-point number (greater than <i>minFallTime</i>) that expresses the maximum transition time when the signal is falling.

Example 1

```
tdfSetClockPortTransition "clk" 0.25
```

Example 2

```
tdfSetClockPortTransition '("clk" "top_clk") '(0.1 0.2  
0.4 0.4)
```

tdfSetClockWaveform

Defines the ideal clock source waveform which includes rise and fall clock edges at the source point relative to time zero (0), and sets the period of the clock.

Tool

Milkyway

Syntax

```
tdfSetClockWaveform "waveFormName" period rise fall  
"objList"
```

where the arguments are as follows:

<i>waveFormName</i>	Name for the waveform. Valid values: String.
<i>period</i>	Elapsed time between repetitions of the clock (1/ frequency). Valid values: A real number.
<i>rise</i>	Rise of the clock edge relative to time zero. Valid values: A real number.
<i>fall</i>	Fall of the clock edge relative to time zero. Valid values: A real number.

objList Name for the ports and portInsts
Valid values:
String.

Example

```
tdfSetClkWaveform "systemClock" 10 0 5 "clk"
```

tdfSetDisableTiming

Disables specified timing arcs, this way removing path constraints for all paths that cross the specified arcs.

Note:

To restore the timing arcs, use [tdfSetRestoreTiming](#).

Tool

Milkyway

Syntax

```
tdfSetDisableTiming '(srcOrDestPoints)' '(destPoints)
```

where the arguments are as follows:

srcOrDestPoints Source or destination points of the paths for which you are disabling timing arcs.

Specify points in the following format:

"pt1" "pt2" ... "ptn"

where a *pt* can be any of the following:

- Name of a cell instance to disable all timing arcs inside the cell instance. In this case, the *destPoints* argument is ignored.

Name of a pin on a cell instance to disable timing arcs as follows:

- If you specify null (""), this disables timing arcs beginning or ending at the pin.
- If you specify pins for *destPoints*, this disables timing arcs between the pin specified by the *pt* and the pins specified for *destPoints*.

destPoints Destination points of the timing arcs you are disabling, or null (""), to indicate that arcs you are disabling are those ending at the pins specified by *srcPoints*.

Specify destination points in the following format:

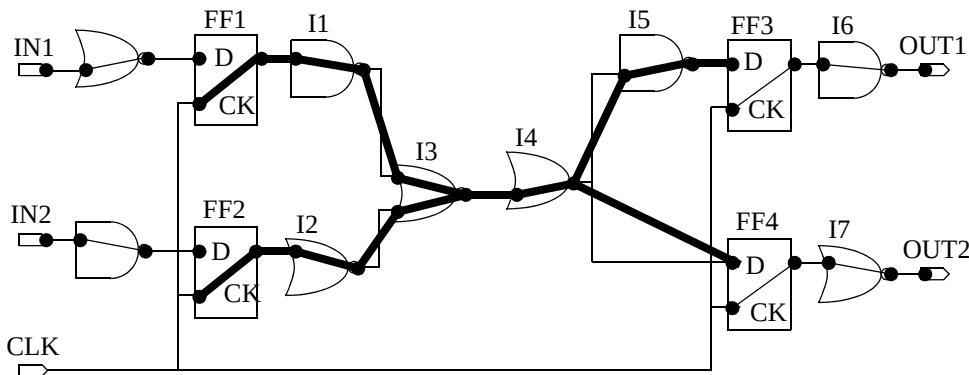
"destPt1" "destPt2" ... "destPtn"

where a *destPt* is the name of a pin.

Example 1

```
tdfSetDisableTiming ' ("I4") ' ("" )
```

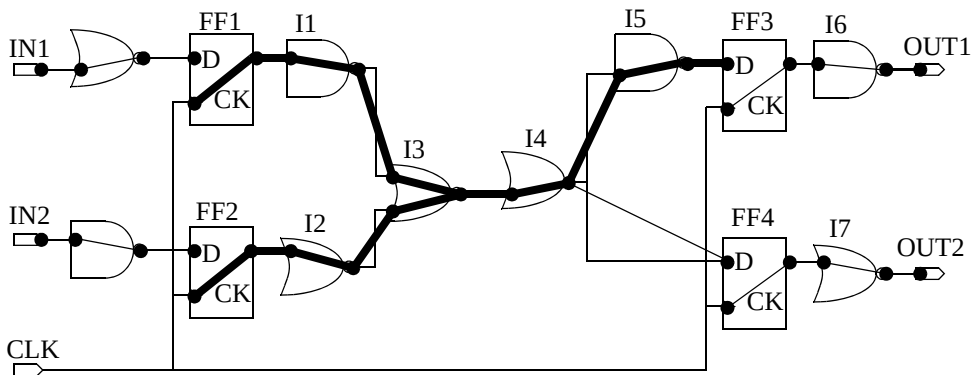
Disables the timing arc inside cell instance I4, this way removing path constraints for the paths shown in bold.



Example 2

```
tdfSetDisableTiming '("FF3/D") '("")
```

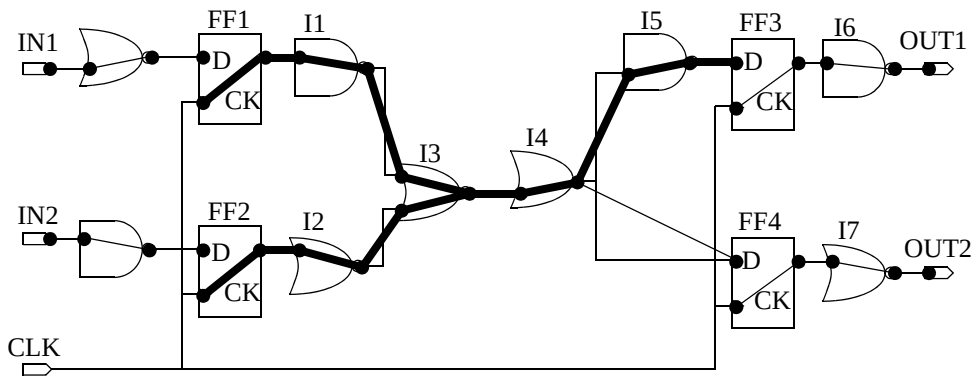
Disables the timing arc beginning or ending at FF3/D, this way removing path constraints for the paths shown in bold.



Example 3

```
tdfSetDisableTiming '("I5/OUT") '("FF3/D")
```

Disables the timing arcs between I5/OUT and FF3/D, this way removing path constraints for the paths shown in bold.



tdfSetDrivingCell

Provides information about the cells driving a top-level input port.

Note:

If the library does not contain timing information about the driving cells, you need to specify the capacitance and resistance of each through each input port using the following:

- [tdfSetSrcCapacitance](#) indicates the capacitance of the device driving the chip or block through a specified input port under a specified operating condition.
- [tdfSetSrcResistance](#) indicates the resistance of the device driving the chip or block through a specified input port under a specified operating condition.

You can purge driving cell information using the [tdfPurgeDrivingCell](#) function.

Tool

Milkyway

Syntax

```
tdfSetDrivingCell portName riseCell fallCell numDriveCells
```

where the arguments are as follows:

<i>portName</i>	Name of the top-level input port for which you are defining driving cells.
<i>riseCell</i>	Name of the driving cell with a rising transition for the port.
<i>fallCell</i>	Name of the driving cell with a falling transition for the port.
<i>numDriveCells</i>	Number of driving cells for the port.

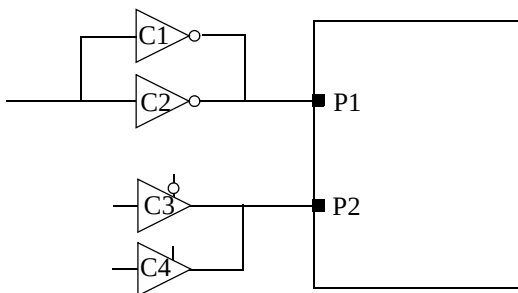
Note:

The “riseCell” and “fallCell” functions can follow the same formats of “libName/CellName/port” or “CellName/port”, where “libName” is a Milkyway library and in Milkyway format. Synopsys formatted library names are currently not accepted.

Example

```
tdfSetDrivingCell P1 C1 C2 2  
tdfSetDrivingCell P2 C3 C4 2
```

Defines the driving cells for ports P1 and P2, shown in the following diagram.



tdfSetFalsePath

Excludes specified timing paths from timing analysis.

Tool

Milkyway

Syntax

```
tdfSetFalsePath "arcType" "refEdge" '(srcPoints)
               '(destPoints)
               ['(thruPoints)]
```

("["]" denotes that this is an optional argument)

where the arguments are as follows:

<i>arcType</i>	Type of timing arcs you are excluding. Valid values: • <i>setup</i> indicates setup arcs. • <i>hold</i> indicates hold arcs. • <i>setuphold</i> indicates both setup and hold arcs.
<i>refEdge</i>	Edge of the timing arcs you are excluding. Valid values: • <i>rise</i> indicates rising edges. • <i>fall</i> indicates falling edges. • <i>risefall</i> indicates both rising and falling edges.

srcPoints

Source points of the paths for which you are excluding timing arcs, or "*" to exclude timing arcs for all paths ending at the points specified by *destPoints*.

Specify source points in the following format:

"srcPt1" "srcPt2" ... "srcPtn"

where a *srcPt* can be any of the following:

- Name of a top-level input port.
- Name of a clock pin on a cell instance.
- Name of a clock to indicate all top-level input ports and clock pins related to the clock.
- Name of a cell instance to indicate all clock pins on the cell instance.

destPoints

Destination points of the paths for which you are excluding timing arcs, or "*" to exclude timing arcs for all paths starting at the points specified by *srcPoints*.

Specify destination points in the following format:

"destPt1" "destPt2" ... "destPtn"

where a *destPt* can be any of the following:

- Name of a top-level output port.
- Name of a data input pin on a cell instance.
- Name of a clock to indicate all top-level output ports and data input pins related to the clock.
- Name of a cell instance to indicate all data input pins on the cell instance.

thruPoints

Thru points of the paths for which you are excluding timing arcs, or "*" to exclude timing arcs for all paths starting at the points specified by *srcPoints*.

Specify thru points in the following format:

"thruGroup1" "thruGroup2" ... "thruGroupn"

where a *thruGroup* consists of a set of candidate thru points in the following format:

thruPt1 thruPt2 ... thruPtn"

where a *thruPt* can be any of the following:

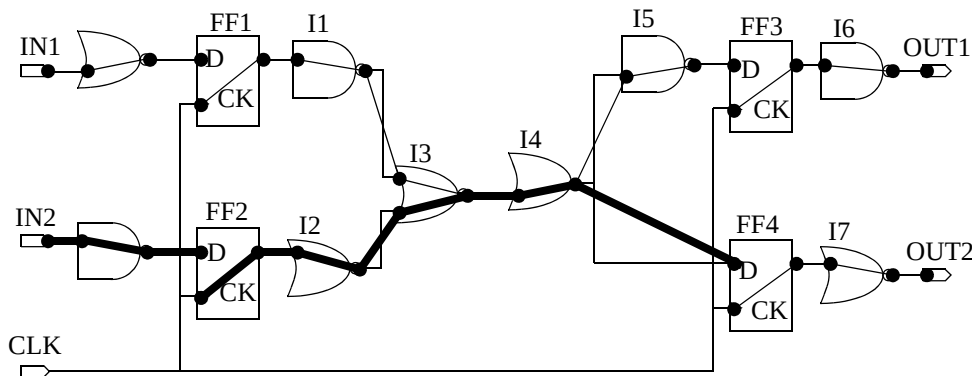
- Name of a port instance.
- Name of a net.

Notice that at least one *thruPt* will be picked within each *thruGroup*.

Example 1

```
tdfSetFalsePath "setup" "rise" ' ("IN2" "FF2/CK")  
' ("FF2/D" "FF4/D")
```

Excludes setup-rising timing arcs for the paths shown in bold.



Example 2

```
tdfSetFalsePath "setuphold" "risefall" ' ("A") ' ("B") ' ("T1" "T2")
```

Specifies that all paths that start from "A", go through T1 and T2 and end at "B", are false paths.

Example 3

```
tdfSetFalsePath "setuphold" "risefall" '("A") '("B")  
'("T1 T2" "T3 T4")
```

Specifies that all paths that start from “A”, go through (T1 or T2) and (T3 or T4) and end at “B”, are false paths.

tdfSetGeneratedClk

Sets the generated clock.

Tool

Milkyway

Syntax

```
tdfSetGeneratedClk genClkName srcClkName objList  
mult_factor  
duty invert edgeList shiftList
```

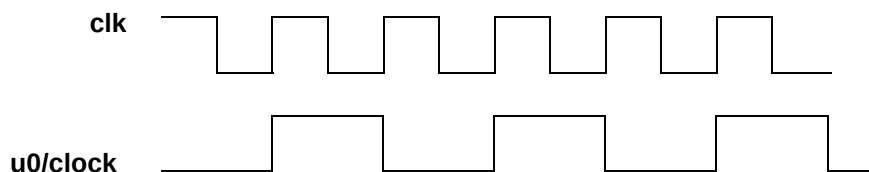
where the arguments are as follows:

<i>genClkName</i>	The generated clock name.
<i>srcClkName</i>	The source clock name.
<i>objList</i>	ports, portInsts
<i>mult_factor</i>	The multiplication factor by which the frequency is multiplied by. This should be a power of 2 (e.g 0.25 , 4) Values greater than two (2) will produce a higher frequency and a lower period. Values less than two will produce a lower frequency and a longer period.
<i>duty</i>	The duty cycle for the high pulse width.
<i>invert</i>	“yes”/“no” is/is not the inversion of the waveform.
<i>edgeList</i>	A list of three (3) or more numbers. If you omit this argument, specify '(0).
<i>shiftList</i>	A list of three (3) or more numbers. The number of the shift_list should be equal to edge_list, if you omit this argument, specify '(0).

Example 1

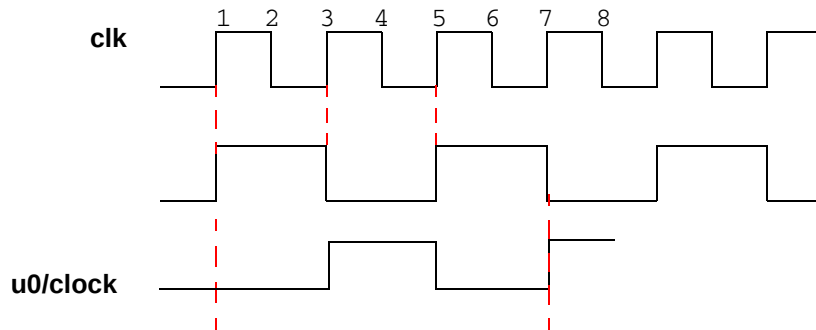
```
tdfSetGeneratedClk "u0/clock" "clk" "u0/Q" 0.5 0.5 "no"  
'(0) '(0)
```

This example specifies that “u0/clock” is derived from “clk” and the period is twice that of “clk”.



Example 2

```
tdfSetGeneratedClk "u0/clock" "clk" "u0/Q" 0 0 "no" `(1
3 5)
`(2 2 2)
```



tdfSetInputDelay

Sets the input delay at the start of a path referenced to a specified clock edge.

Note:

If you specify source capacitance and resistance using [tdfSetSrcCapacitance](#) and [tdfSetSrcResistance](#), the total input delay is the input delay you set using `tdfSetInputDelay` plus the delay caused by the source resistance and capacitance.

Tool

Milkyway

Syntax

```
tdfSetInputDelay "sourcePort" delaySet "refClk" "refEdge"  
"cellType"
```

where the arguments are as follows:

<i>sourcePort</i>	Name of the source port for the path. Valid values: Name of any top-level port in the top cell.
-------------------	--

delaySet

A value or set of values that expresses the input delay at *sourcePort* referenced to the specified clock edge, including the following:

- The minimum delay when the signal is rising.
- The maximum delay when the signal is rising.
- The minimum delay when the signal is falling.
- The maximum delay when the signal is falling.

Valid values: Astro needs four values, which you can express in any of the following ways:

- A single value, expressed as follows:

value

where *value* is a floating-point number that expresses the minimum and maximum delays when the signal is rising or falling.

- A list of two values, expressed as follows:

'(riseTime fallTime)

where

- *riseTime* is a floating-point number that expresses the minimum and maximum delays when the signal is rising.
- *fallTime* is a floating-point number that expresses the minimum and maximum delays when the signal is falling.
- A list of four values, expressed as follows:

*'(minRiseTime minFallTime
maxRiseTime maxFallTime)*

where

- *minRiseTime* is a floating-point number (less than *maxRiseTime*) that expresses the minimum delay when the signal is rising.

- *minFallTime* is a floating-point number (less than *maxFallTime*) that expresses the minimum delay when the signal is falling.
- *maxRiseTime* is a floating-point number (greater than *minRiseTime*) that expresses the maximum delay when the signal is rising.
- *maxFallTime* is a floating-point number (greater than *minFallTime*) that expresses the maximum delay when the signal is falling.

refClk Name of the clock on the external flip-flop or latch to be used for the reference.
Valid values: Name of any clock defined by a [tdfSetClkDef](#) function. (The name of the clock is the source point specified in the definition.)

refEdge Reference edge of the specified clock.
Valid values:

- *rise* indicates the reference edge is the rising edge.
- *fall* indicates the reference edge is the falling edge.

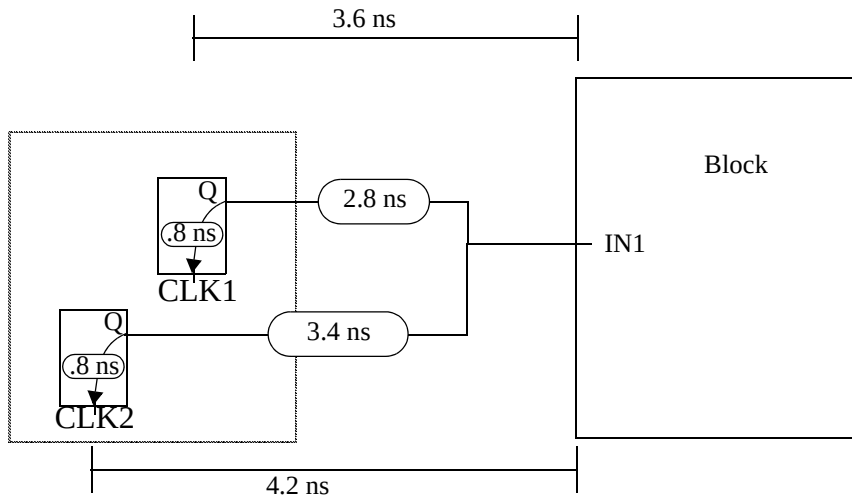
cellType Type of the external cell to which the source port is connected.
Valid values: Any of the following:

- *latch* indicates the cell is a latch.
- *flip-flop* indicates the cell is a flip-flop.

Example

```
tdfSetInputDelay "IN1" 3.6 "C1" "rise" "latch"
tdfSetInputDelay "IN1" 4.2 "C2" "rise" "latch"
```

Indicate input delays for IN1, as shown in the following figure.



tdfSetInputPortTransition

Sets the transition time at input ports and port instances.

Tool

Milkyway

Syntax

```
tdfSetInputPortTransition "objList" transitionDelaySet
```


where the arguments are as follows:

<i>objList</i>	Name for the <i>ports</i> and <i>portInsts</i> . Valid Values: String.
<i>transitionDelaySet</i>	A scalar or four-value set, expressed as follows: '(minRiseTime minFallTime maxRiseTime maxFallTime) where: • <i>minRiseTime</i> is a floating-point number (less than <i>maxRiseTime</i>) that expresses the minimum transition time when the signal is rising. • <i>minFallTime</i> is a floating-point number (less than <i>maxFallTime</i>) that expresses the minimum transition time when the signal is falling. • <i>maxRiseTime</i> is a floating-point number (greater than <i>minRiseTime</i>) that expresses the maximum transition time when the signal is rising. • <i>maxFallTime</i> is a floating-point number (greater than <i>minFallTime</i>) that expresses the maximum transition time when the signal is falling.

Example 1

```
tdfSetInputPortTransition "In" 0.25
```

Example 2

```
tdfSetInputPortTransition '("in1" "in2") '(0.1 0.2 0.4  
0.4)
```

tdfSetInputPortTransition

tdfSetLoadCap

Indicates the capacitance of the device driven by the chip or block through a specified output port.

Astro can calculate the load on a net that has no external connections. However, primary outputs such as output pads are connected to external loads that increase the net delay. The *tdfSetLoadCap* function specifies this external load capacitance.

Note:

If your EDIF netlist does not define ports, you must first create a port on an existing net using the `dbCreateChipPort` command. For syntax information, see the command reference for Astro.

Tool

Milkyway

Syntax

```
tdfSetLoadCap "portName" loadCapacitance
```

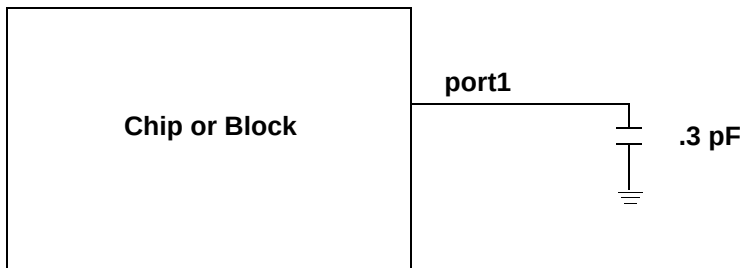
where the arguments are as follows:

<i>portName</i>	Name of the output port. Valid values: Name of any output port in the chip or block.
<i>loadCapacitance</i>	Capacitance of the device driven by the chip or block through the specified port. Valid values: Any non-negative, floating-point number.

Example

```
tdfSetLoadCap "port1" .3
```

This directive tells Astro that the device driven by the chip or block through port *port1* has a capacitance of 0.3 pF.



Astro uses this information to evaluate path delay during timing-driven placement.

tdfSetMaxDelayPath

Sets the maximum delay for specified paths.

Tool

Milkyway

Syntax

```
tdfSetMaxDelayPath riseDelay fallDelay ['(srcPoints)  
['(destPoints) ['(thruPoints)]
```

("["]" denotes that this is an optional argument)

where the arguments are as follows:

riseDelay Maximum delay when the signal is rising.

<i>fallDelay</i>	Maximum delay when the signal is falling.
<i>srcPoints</i>	Source points of the paths for which you are setting maximum delay, or "*" to set maximum delay for all paths ending at the points specified by <i>destPoints</i>. Specify source points in the following format: <i>"srcPt1" "srcPt2" ... "srcPtn"</i> where a <i>srcPt</i> can be any of the following: • Name of a top-level input port. • Name of a clock pin on a cell instance. • Name of a clock to indicate all top-level input ports and clock pins related to the clock. • Name of a cell instance to indicate all clock pins on the cell instance.
<i>destPoints</i>	Destination points of the paths for which you are setting maximum delay, or "*" to set maximum delay for all paths starting at the points specified by <i>srcPoints</i>. Specify destination points in the following format: <i>"destPt1" "destPt2" ... "destPtn"</i> where a <i>destPt</i> can be any of the following: • Name of a top-level output port. • Name of a data input pin on a cell instance. • Name of a clock to indicate all top-level output ports and data input pins related to the clock. • Name of a cell instance to indicate all data input pins on the cell instance.

thruPoints

Thru points of the paths for which you are setting maximum delay, or "*" to set maximum delay for all paths starting at the points specified by *srcPoints*.

Specify thru points in the following format:

"thruGroup1" "thruGroup2" ... "thruGroupn"

where a *thruGroup* consists of a set of candidate thru points in the following format:

thruPt1 thruPt2 ... thruPtn"

where a *thruPt* can be any of the following:

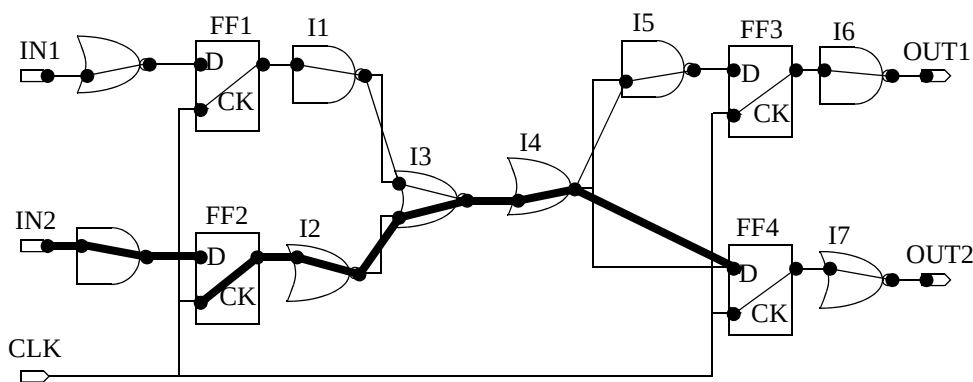
- Name of a port instance.
- Name of a net.

notice that at least one *thruPt* will be picked within each *thruGroup*.

Example 1

```
tdfSetMaxDelayPath 3.2 3.7 ' ("IN2" "FF2/CK") ' ("FF2/D" "FF4/D")
```

Sets maximum delay to 3.2 ns for the rising edge and 3.7 ns for the falling edge of the paths shown in bold.



tdfSetMinDelayPath

Sets the minimum delay for paths.

Tool

Milkyway

Syntax

```
tdfSetMinDelayPath riseDelay fallDelay ' (srcPoints)  
' (destPoints) [' (thruPoints)]
```

("["]" denotes that this is an optional argument)

where the arguments are as follows:

<i>riseDelay</i>	Minimum delay when the signal is rising.
<i>fallDelay</i>	Minimum delay when the signal is falling.
<i>srcPoints</i>	Source points of the paths for which you are setting minimum delay, or "*" to set minimum delay for all paths ending at the points specified by <i>destPoints</i>. Specify source points in the following format: "srcPt1" "srcPt2" ... "srcPtn" where a <i>srcPt</i> can be any of the following: • Name of a top-level input port. • Name of a clock pin on a cell instance. • Name of a clock to indicate all top-level input ports and clock pins related to the clock. • Name of a cell instance to indicate all clock pins on the cell instance.

destPoints

Destination points of the paths for which you are setting minimum delay, or "*" to set minimum delay for all paths starting at the points specified by *srcPoints*.

Specify destination points in the following format:

"*destPt1*" "*destPt2*" ... "*destPtn*"

where a *destPt* can be any of the following:

- Name of a top-level output port.
- Name of a data input pin on a cell instance.
- Name of a clock to indicate all top-level output ports and data input pins related to the clock.
- Name of a cell instance to indicate all data input pins on the cell instance.

thruPoints

Thru points of the paths for which you are setting minimum delay, or "*" to set minimum delay for all paths starting at the points specified by *srcPoints*.

Specify thru points in the following format:

"*thruGroup1*" "*thruGroup2*" ... "*thruGroupn*"

where a *thruGroup* consists of a set of candidate thru points in the following format:

thruPt1 *thruPt2* ... *thruPtn*

where a *thruPt* can be any of the following:

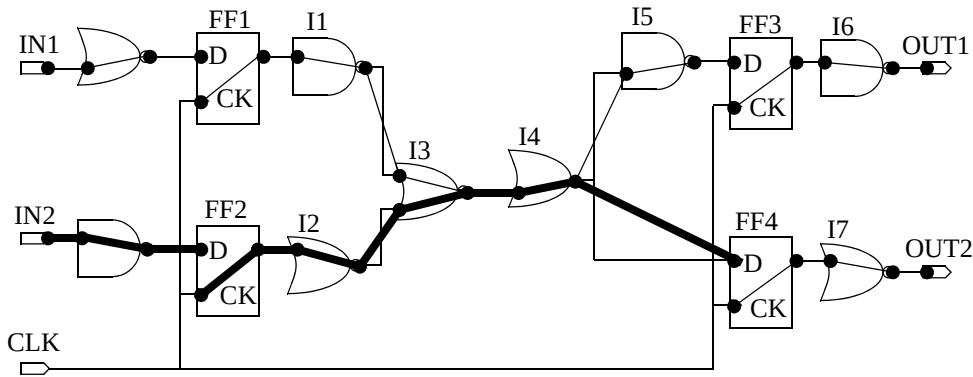
- Name of a port instance.
- Name of a net.

Notice that at least one *thruPt* will be picked within each *thruGroup*.

Example 1

```
tdfSetMinDelayPath 3.0 3.5 ' ("IN2" "FF2/CK") ' ("FF2/D" "FF4/D")
```

Sets minimum delay to 3.0 ns for the rising edge and 3.5 ns for the falling edge of the paths shown in bold.



tdfSetMinIOSpacing

Sets the minimum spacing of an I/O pad.

Note:

Please be aware that a tdf Scheme command must be in a tdf file.

Tool

Milkyway

Syntax

```
tdfSetMinIOSpacing padName leftSpace rightSpace leftAbut
rightAbut
```

where the arguments are as follows:

<i>padName</i>	Name of the I/O pad to which you want to apply the minimum spacing constraint. Valid values: Name of any I/O pad in the netlist.
<i>leftSpace</i>	The minimum spacing to the left.
<i>rightSpace</i>	The minimum spacing to the right.

<i>leftAbut</i>	0 to abut left. 1 not to abut left.
<i>rightAbut</i>	0 to abut right. 1 not to abut right.

See Also

[tdfPurgeIOSpaceConstr](#)

tdfSetMultiCyclePath

Specifies the number of cycles of timing arcs for specified paths. If a design contains timing paths that are longer than a single cycle, use this function to specify those paths.

Tool

Milkyway

Syntax

```
tdfSetMultiCyclePath numCycles "arcType" "refEdge"
"refClock"
'(srcPoints)' (destPoints) ['(thruPoints)']
```

("[" denotes that this is an optional argument)

where the arguments are as follows:

<i>numCycles</i>	Number of cycles for the timing arcs. Valid values: Any integer.
------------------	---

<i>arcType</i>	Type of timing arcs for which you are setting the number of cycles. Valid values: • <i>setup</i> indicates setup arcs. • <i>hold</i> indicates hold arcs. • <i>setuphold</i> indicates both setup and hold arcs.
<i>refEdge</i>	Edge of the timing arcs for which you are setting the number of cycles. Valid values: • <i>rise</i> indicates rising edges at the path endpoint. • <i>fall</i> indicates falling edges at the path endpoint. • <i>risefall</i> indicates both rising and falling edges at the path endpoint.
<i>refClock</i>	Indicates reference clock information in the multi-cycle path. Valid values: • <i>start</i> • if “setup” is specified, the reference launching edge will be moved backward (numCycles-1) clock cycles at start clock. • if “hold” is specified, the reference launching edge will be moved forward (numCycles) clock cycles at start clock. • <i>end</i> • if “setup” is specified, the reference receiving edge will be moved forward (numCycles-1) clock cycles at start clock. • if “hold” is specified, the reference receiving edge will be moved backward (numCycles) clock cycles at start clock.

srcPoints

Source points of the paths for which you are setting the number of cycles, or "*" to set the number of cycles for all paths ending at the points specified by *destPoints*.

Specify source points in the following format:

"srcPt1" "srcPt2" ... "srcPtn"

where a *srcPt* can be any of the following:

- Name of a top-level input port.
- Name of a clock pin on a cell instance.
- Name of a clock to indicate all top-level input ports and clock pins related to the clock.
- Name of a cell instance to indicate all clock pins on the cell instance.

destPoints

Destination points of the paths for which you are setting number of cycles, or "*" to set number of cycles for all paths starting at the points specified by *srcPoints*.

Specify destination points in the following format:

"destPt1" "destPt2" ... "destPtn"

where a *destPt* can be any of the following:

- Name of a top-level output port.
- Name of a data input pin on a cell instance.
- Name of a clock to indicate all top-level output ports and data input pins related to the clock.
- Name of a cell instance to indicate all data input pins on the cell instance.

thruPoints Thru points of the paths for which you are excluding timing arcs, or "*" to exclude timing arcs for all paths starting at the points specified by *srcPoints*.

Specify thru points in the following format:

"thruGroup1" "thruGroup2" ... "thruGroupn"

where a *thruGroup* consists of a set of candidate thru points in the following format:

thruPt1 thruPt2 ... thruPtn"

where a *thruPt* can be any of the following:

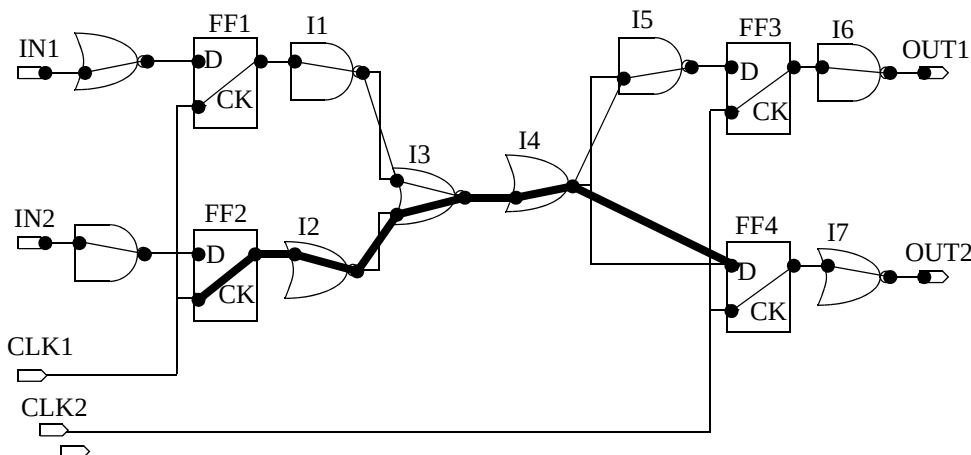
- Name of a port instance.
- Name of a net.

notice that at least one *thruPt* will be picked within each *thruGroup*.

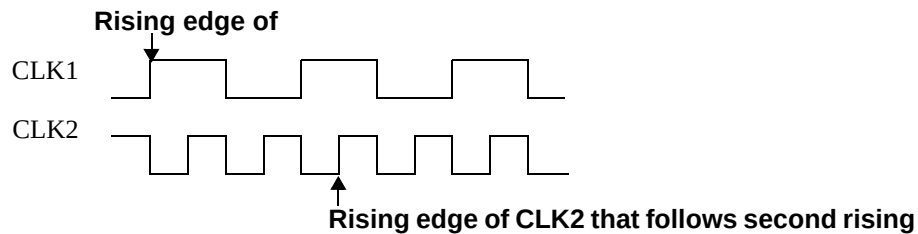
Example 1

```
tdfSetMultiCyclePath 2 "setup" "risefall" "start" ' ("FF2/  
CK")  
' ("FF4/D")
```

Sets the number of cycles to 2 in reference to the rising edge of CLK1 for the path shown in bold.



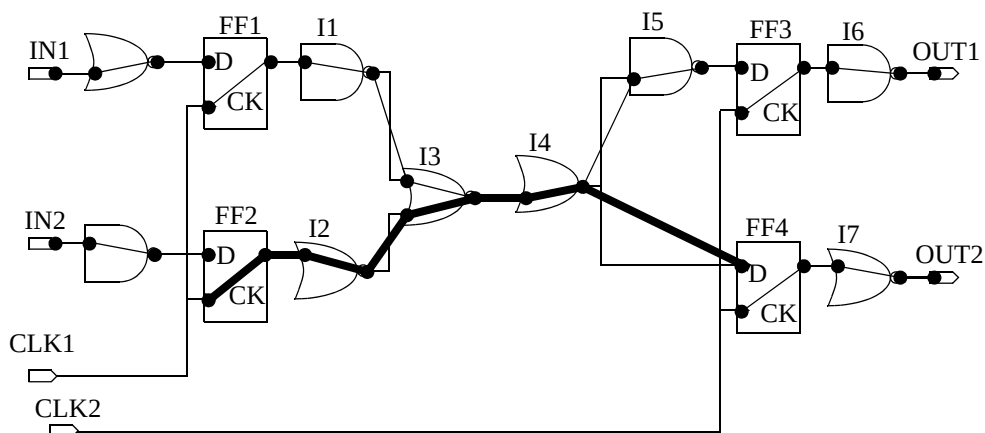
In this case, the maximum delay allowed for the path is the delay between the rising edge of CLK1 and the rising edge of CLK2 that follows the second rising edge of CLK1.



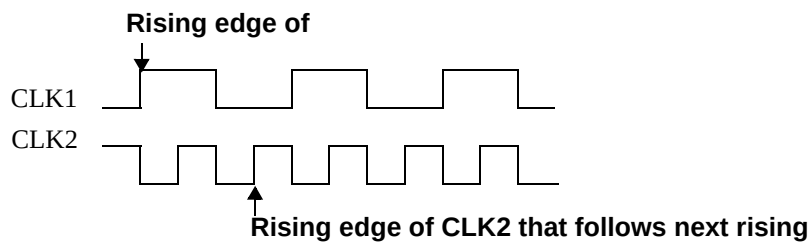
Example 2

```
tdfSetMultiCyclePath 2 "setup" "risefall" "end" ' ("FF2/CK")
' ("FF4/D")
```

Sets the number of cycles to 2 in reference to the rising edge of CLK2 for the path shown in bold.

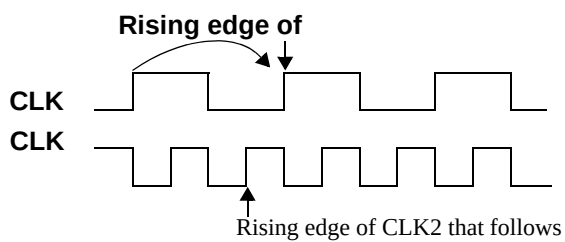


In this case, the maximum delay allowed for the path is the delay between the rising edge of CLK1 and the rising edge of CLK2 that follows the next rising edge of CLK2.



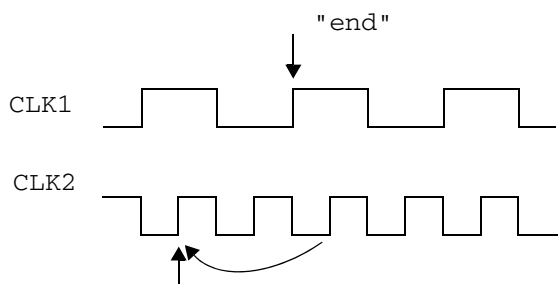
Example 3

```
tdfSetMultiCyclePath 1 "hold" "risefall" "start" ' ("FF2/CK")
' ("FF4/D")
```



Example 4

```
tdfSetMultiCyclePath 1 "hold" "risefall" "start" ' ("FF2/CK")
' ("FF4/D")
```



tdfSetNetCapAndTransTime

Sets the default capacitance and transition time for nets. The timing analyzer will use the default capacitance value to compute the driving cell delay and use the default transition time to transitive fanouts. This is mainly used to estimate the average cap and transition time for high fanout net before fanout optimization is done.

Tool

Milkyway

Syntax

```
tdfSetNetCapAndTransTime objList "capacitanceSet"  
                          "transitionTimeSet"
```

where the arguments are as follows:

<i>objList</i>	Name of the ports, port instances, nets. Valid values: String.
----------------	--

capacitanceSet

- A scalar or four-value set, expressed as follows:
'(minRiseCapacitance minFallCapacitance
maxRiseCapacitance maxFallCapacitance)
where:
 - *minRiseCapacitance* is a floating-point number (less than *maxRiseCapacitance*) that expresses the minimum capacitance when the signal is rising.
 - *minFallCapacitance* is a floating-point number (less than *maxFallCapacitance*) that expresses the minimum capacitance when the signal is falling.
 - *maxRiseCapacitance* is a floating-point number (greater than *minRiseCapacitance*) that expresses the maximum capacitance when the signal is rising.
 - *maxFallCapacitance* is a floating-point number (greater than *minFallCapacitance*) that expresses the maximum capacitance when the signal is falling.

transitionTimeSet

- A scalar or four-value set, expressed as follows:
'(minRiseTime minFallTime
maxRiseTime maxFallTime)
where:
 - *minRiseTime* is a floating-point number (less than *maxRiseTime*) that expresses the minimum transition time when the signal is rising.
 - *minFallTime* is a floating-point number (less than *maxFallTime*) that expresses the minimum transition time when the signal is falling.
 - *maxRiseTime* is a floating-point number (greater than *minRiseTime*) that expresses the maximum transition time when the signal is rising.
 - *maxFallTime* is a floating-point number (greater than *minFallTime*) that expresses the maximum transition time when the signal is falling.

tdfSetNetCapTransAndDelayTime

Sets the default *total capacitance*, *transition time*, and *net delay* for the nets. The timing analyzer will use the default capacitance value to compute the driving cell delay, and use the default transition time and delay to transitive fanouts.

This is mainly used to estimate the average cap, transition time, and delay for high fanout nets before fanout optimization is finished.

Tool

Milkyway

Syntax

```
tdfSetNetCapTransAndDelayTime "objList" "capacitanceSet"  
    "transitionTimeSet" "delaySet"
```

where the arguments are as follows:

<i>objList</i>	Name of the ports, port instances, nets. Valid values: <i>String</i> .
----------------	--

capacitanceSet

- A scalar or four-value set, expressed as follows:
'(minRiseCapacitance minFallCapacitance
maxRiseCapacitance maxFallCapacitance)
where:
 - *minRiseCapacitance* is a floating-point number (less than *maxRiseCapacitance*) that expresses the minimum capacitance when the signal is rising.
 - *minFallCapacitance* is a floating-point number (less than *maxFallCapacitance*) that expresses the minimum capacitance when the signal is falling.
 - *maxRiseCapacitance* is a floating-point number (greater than *minRiseCapacitance*) that expresses the maximum capacitance when the signal is rising.
 - *maxFallCapacitance* is a floating-point number (greater than *minFallCapacitance*) that expresses the maximum capacitance when the signal is falling.

transitionTimeSet

- A scalar or four-value set, expressed as follows:
'(minRiseTime minFallTime
maxRiseTime maxFallTime)
where:
 - *minRiseTime* is a floating-point number (less than *maxRiseTime*) that expresses the minimum transition time when the signal is rising.
 - *minFallTime* is a floating-point number (less than *maxFallTime*) that expresses the minimum transition time when the signal is falling.
 - *maxRiseTime* is a floating-point number (greater than *minRiseTime*) that expresses the maximum transition time when the signal is rising.
 - *maxFallTime* is a floating-point number (greater than *minFallTime*) that expresses the maximum transition time when the signal is falling.

delaySet

- A scalar or four-value set, expressed as follows:
'(minRiseDelay minFallDelay
maxRiseDelay maxFallDelay)
where:
 - *minRiseDelay* is a floating-point number (less than *maxRiseDelay*) that expresses the minimum delay when the signal is rising.
 - *minFallDelay* is a floating-point number (less than *maxFallDelay*) that expresses the minimum delay when the signal is falling.
 - *maxRiseDelay* is a floating-point number (greater than *minRiseDelay*) that expresses the maximum delay when the signal is rising.
 - *maxFallDelay* is a floating-point number (greater than *minFallDelay*) that expresses the maximum delay when the signal is falling.

tdfSetNetTransitionDelay

Sets the transition time for nets. The timing analyzer will use the default transition time to transitive fanouts. This function is mainly used to estimate the transition time for high fanout net before fanout optimization is finished.

Tool

Milkyway

Syntax

```
tdfSetNetTransitionDelay "objList" "transitionDelaySet"
```

where the arguments are as follows:

<i>objList</i>	Name of the ports, port instances, nets. Valid values: String.
<i>transitionDelaySet</i>	• A scalar or four-value set, expressed as follows: '(minRiseTime minFallTime maxRiseTime maxFallTime) where: • <i>minRiseTime</i> is a floating-point number (less than <i>maxRiseTime</i>) that expresses the minimum transition time when the signal is rising. • <i>minFallTime</i> is a floating-point number (less than <i>maxFallTime</i>) that expresses the minimum transition time when the signal is falling. • <i>maxRiseTime</i> is a floating-point number (greater than <i>minRiseTime</i>) that expresses the maximum transition time when the signal is rising. • <i>maxFallTime</i> is a floating-point number (greater than <i>minFallTime</i>) that expresses the maximum transition time when the signal is falling.

Example 1

```
tdfSetNetTransitionDelay "clk" 2.5
```

Example 2

```
tdfSetNetTransitionDelay '("cellA/z" "top_clk")  
'(0.1 0.2 0.4 0.4)
```

tdfSetNoTouchCellInst

Sets the specified cell instances as “No Touch,” so that the instances can not be changed during the design layout process.

Tool

Milkyway

Syntax

```
tdfSetNoTouchCellInstance ' ("cellName1" "cellName2"... )
```

where the argument is as follows:

<i>cellName</i>	Name of the cell instance to which you want to apply a “No Touch” constraint. Valid values: Name of any cell instance in the netlist.
-----------------	--

See Also

Use [tdfPurgeNoTouchNet](#) to remove all “No Touch” constraints previously applied to nets.

Use [tdfPurgeNoTouchCellInst](#) to remove all “No Touch” constraints previously applied to cell instances.

Use [tdfSetNoTouchNet](#) to apply the “No Touch” constraint to specified nets.

tdfSetNoTouchNet

Sets the specified nets as “No Touch,” so that the nets can not be changed during the design layout process.

Tool

Milkyway

Syntax

```
tdfSetNoTouchNet ' ("netName1" "netName2" . . . )
```

where the argument is as follows:

<i>netName</i>	Name of the net to which you want to apply a “No Touch” constraint. Valid values: Name of any net in the netlist.
----------------	--

See Also

Use [tdfSetNoTouchNet](#) to set specified cell instances as “No Touch.”

Use [tdfPurgeNoTouchNet](#) to remove all “No Touch” constraints previously applied to nets.

Use [tdfPurgeNoTouchCellInst](#) to remove all “No Touch” constraints previously applied to cell instances.

tdfSetOutputDelay

Sets the output delay at the destination of a path referenced to a specified clock edge. Output delay is the delay between the destination port for the path and the input to the external flip-flop or latch, plus the setup time.

Tool

Milkyway

Syntax

```
tdfSetOutputDelay "destPoint" delaySet "refClk" "refEdge"  
"cellType"
```

where the argument is as follows:

<i>destPoint</i>	Name of the destination port for the path. Valid values: Name of any top-level port in the top cell.
------------------	---

delaySet

A value or set of values that expresses the delay at *destPoint* referenced to the specified clock edge, including the following:

- The minimum delay when the signal is rising.
- The maximum delay when the signal is rising.
- The minimum delay when the signal is falling.
- The maximum delay when the signal is falling.

Valid values: Astro needs four values, which you can express in any of the following ways:

- A single value, expressed as follows:

value

where *value* is a floating-point number that expresses the minimum and maximum delays when the signal is rising or falling.

- A list of two values, expressed as follows:

'(riseTime fallTime)

where

- *riseTime* is a floating-point number that expresses the minimum and maximum delays when the signal is rising.
- *fallTime* is a floating-point number that expresses the minimum and maximum delays when the signal is falling.

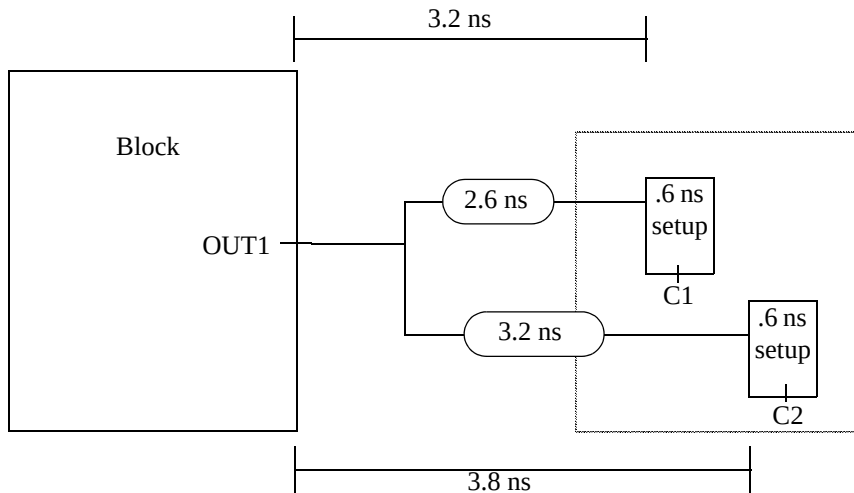
- A list of four values, expressed as follows:
`'(minRiseTime minFallTime
maxRiseTime maxFallTime)`
where
- *minRiseTime* is a floating-point number (less than *maxRiseTime*) that expresses the minimum delay when the signal is rising.
- *minFallTime* is a floating-point number (less than *maxFallTime*) that expresses the minimum delay when the signal is falling.
- *maxRiseTime* is a floating-point number (greater than *minRiseTime*) that expresses the maximum delay when the signal is rising.
- *maxFallTime* is a floating-point number (greater than *minFallTime*) that expresses the maximum delay when the signal is falling.

<i>refClk</i>	Name of the clock on the external flip-flop or latch to be used for the reference. Valid values: Name of any clock defined by a tdfSetClkDef function. (The name of the clock is the source point specified in the definition.)
<i>refEdge</i>	Reference edge of the specified clock. Valid values: • <i>rise</i> indicates the reference edge is the rising edge. • <i>fall</i> indicates the reference edge is the falling edge.
<i>cellType</i>	Type of the external cell to which the destination port is connected. Valid values: Any of the following: • <i>latch</i> indicates the cell is a latch. • <i>flip-flop</i> indicates the cell is a flip-flop.

Example

```
tdfSetOutputDelay "OUT1" 3.2 "C1" "rise" "flipflop"
tdfSetOutputDelay "OUT1" 3.8 "C2" "rise" "flipflop"
```

These functions indicate output delays for OUT1, as shown in the following figure.



tdfSetPowerSupply

Defines the power supply voltages. Mars-Rail allows definition of multiple supply voltages using multiple *tdfSetPowerSupply* commands.

Tool

Milkyway

Syntax

```
tdfSetPowerSupply (netName) (voltage)
```

where the argument is as follows:

<i>netName</i>	Name of the net to which you want to apply a “No Touch” constraint. Valid values: Name of any net in the netlist.
----------------	--

tdfSetRestoreTiming

Restores timing arcs for specified paths previously disabled using [tdfSetDisableTiming](#).

Tool

Milkyway

Syntax

```
tdfSetRestoreTiming ' (srcOrDestPoints) ' (destPoints)
```

where the arguments are as follows:

srcOrDestPoints Source or destination points of the paths for which you are restoring timing arcs.

Specify points in the following format:

"pt1" "pt2" ... "ptn"

where a *pt* can be any of the following:

- Name of a cell instance to restore all timing arcs inside the cell instance. In this case, the *destPoints* argument is ignored.
- Name of a pin on a cell instance to restore timing arcs as follows:
- Specifying null ("") for *destPoints* restores timing arcs beginning or ending at the pin.
- Specifying pins for *destPoints* restores timing arcs between the pin specified by the *pt* and the pins specified for *destPoints*.

destPoints Destination points of the timing arcs you are restoring, or null ("") to indicate that arcs you are restoring are those ending at the pins specified by *srcPoints*.

Specify destination points in the following format:

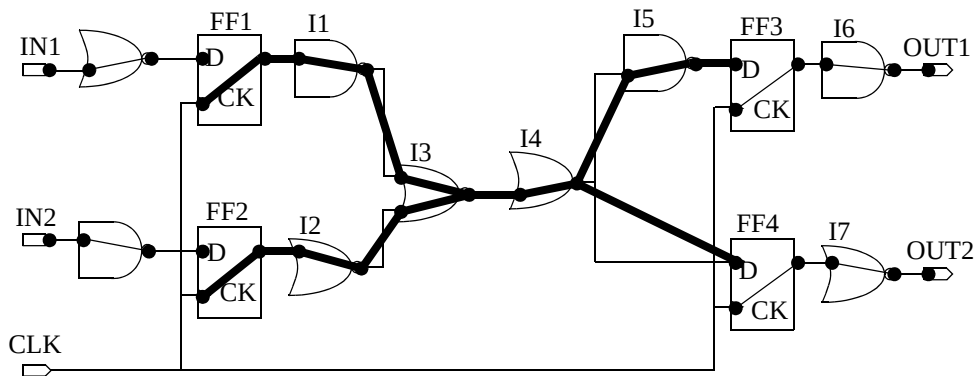
"destPt1" "destPt2" ... "destPtn"

where a *destPt* is the name of a pin.

Example 1

```
tdfSetRestoreTiming '("I4") '("")
```

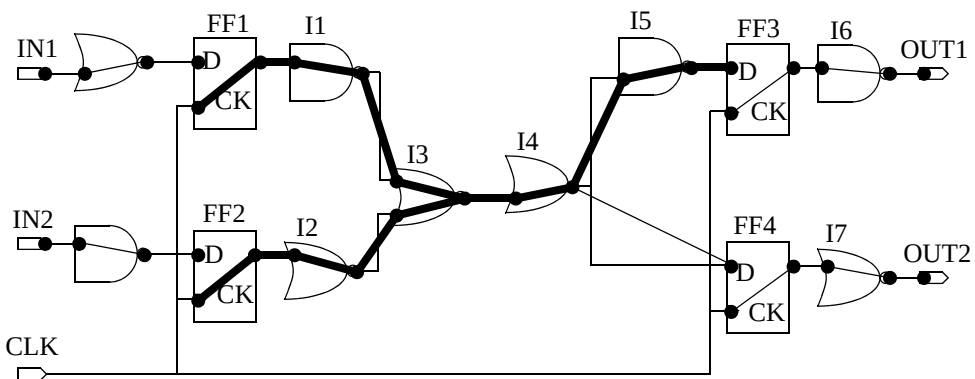
Restores the timing arc inside cell instance I4, this way restoring path constraints for the paths shown in bold.



Example 2

```
tdfSetRestoreTiming '("FF3/D") '("")
```

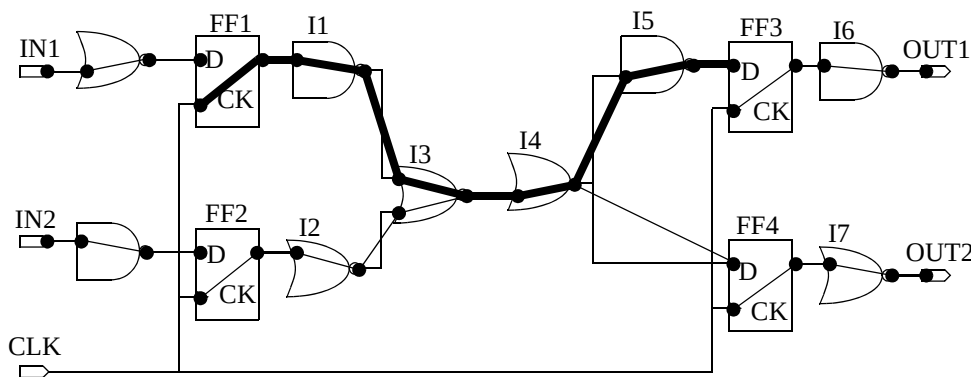
Restores the timing arc beginning or ending at FF3/D, this way restoring path constraints for the paths shown in bold.



Example 3

```
tdfSetRestoreTiming '("FF1/CK") '("FF3/D")
```

Restores the timing arcs between FF1/CK and FF3/D, this way restoring path constraints for the paths shown in bold.



tdfSetSrcCapacitance

Indicates the capacitance of the device driving the chip or block through a specified input port under a specified operating condition. You can only set the capacitance for one operating condition at a time with this command. Use [tdfSetSrcCapacitanceList](#) to set the capacitance for all operating conditions.

Astro calculates the slope (slew) of signals that originate within the chip or block. However, primary inputs such as pads are driven by external devices. Because the timing characteristics for these devices are not in the cell library, Astro cannot know the strength of the driver or the capacitance of the wiring that connects the driver to the primary input. Therefore, you must specify these parameters:

- [tdfSetSrcResistance](#) specifies the strength of the driver, or use [tdfSetSrcResistanceList](#), which allows you to specify the strength of the driver for all operating conditions.

- `tdfSetSrcCapacitance` specifies the capacitance of the wiring that connects the driver to the primary input or use [tdfSetSrcCapacitanceList](#), which allows you to specify the capacitance of the wiring that connects the driver to the primary input for all operating conditions.

Note:

If your EDIF netlist does not define ports, you must first create a port on an existing net using [dbCreateGlobalNetByScanChain](#).

Tool

Milkyway

Syntax

```
tdfSetSrcCapacitance "portName" "condition" value
```

where the arguments are as follows:

<i>portName</i>	Name of the input port being specified. Valid values: Name of any port in the chip or block.
-----------------	---

condition

The operating condition under which the specification applies.

Valid values:

- *rise min* means that the specification applies when the signal is coming in at its fastest speed and the transition is a rise.
- *rise nom* means that the specification applies when the signal is coming in at its nominal speed and the transition is a rise.
- *rise max* means that the specification applies when the signal is coming in at its slowest speed and the transition is a rise.
- *fall min* means that the specification applies when the signal is coming in at its fastest speed and the transition is a fall.
- *fall nom* means that the specification applies when the signal is coming in at its nominal speed and the transition is a fall.
- *fall max* means that the specification applies when the signal is coming in at its slowest speed and the transition is a fall.

value

The capacitance of the device driving the chip or block through the specified input port under the specified operating condition.

Valid values: Any non-negative, floating-point number.

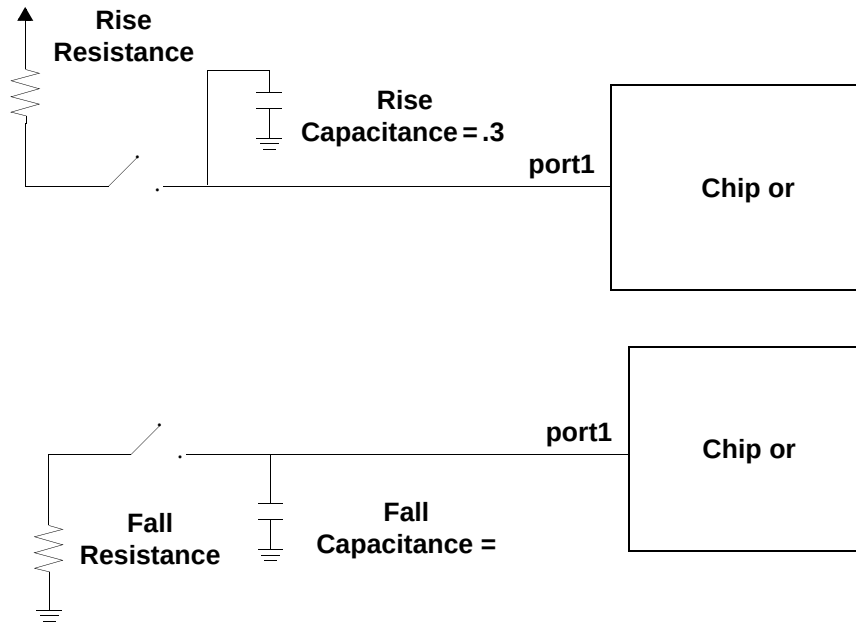
Example

```
tdfSetSrcCapacitanceList "port1" .3  
tdfSetSrcCapacitance "port1" "fall max" .3
```

Using these directives, Astro evaluates incoming signals to the chip or block through *port1* as follows:

- When a signal is coming in through *port1* at its slowest speed and the transition is a rise (*rise max*), the capacitance is 0.3 pF.
- When a signal is coming in through *port1* at its slowest speed and the transition is a fall (*fall max*), the capacitance is 0.3 pF.

tdfSetSrcCapacitance



tdfSetSrcCapacitanceList

Indicates the capacitance of the device driving the chip or block through a specified input port for all operating conditions.

Astro calculates the slope (slew) of signals that originate within the chip or block. However, primary inputs such as pads are driven by external devices. Because the timing characteristics for these devices are not in the cell library, Astro cannot know the strength of the driver or the capacitance of the wiring that connects the driver to the primary input. Therefore, you must specify these parameters:

- [tdfSetSrcResistance](#) specifies the strength of the driver or use [“tdfSetSrcResistanceList” on page 54-85](#), which allows you to specify the strength of the driver for all operating conditions.

- [tdfSetSrcCapacitance](#) specifies the capacitance of the wiring that connects the driver to the primary input or use *tdfSetSrcCapacitanceList*, which allows you to specify the capacitance of the wiring that connects the driver to the primary input for all operating conditions.

Note:

If your EDIF netlist does not define ports, you must first create a port on an existing net using [dbCreateGlobalNetByScanChain](#).

Tool

Milkyway

Syntax

```
tdfSetSrcCapacitance "portName" '(values)
```

where the arguments are as follows:

<i>portName</i>	Name of the input port being specified. Valid values: Name of any port in the chip or block.
<i>value</i>	The capacitance of the device driving the chip or block through the specified input port under all operating conditions. Specify one (1), two (2), three (3), four(4), or six (6) values to apply to one or more of the six conditions listed below. Valid values: Any non-negative, floating-point number.

The operating conditions under which the specified values apply are:

Condition...	The specification applies when the signal is coming in at its...
rise min	Fastest speed and the transition is a rise.
rise nom	Nominal speed and the transition is a rise.
rise max	Slowest speed and the transition is a rise.

tdfSetSrcCapacitanceList

Condition...	The specification applies when the signal is coming in at its...
fall min	Fastest speed and the transition is a fall.
fall nom	Nominal speed and the transition is a fall.
fall max	Slowest speed and the transition is a fall.

The values apply in the following way to the operating conditions:

```
'(A) applies to all 6 conditions
'(A B) rise min, rise nom, and rise max = A
fall min, fall nom, and fall max = B
'(A B C)rise min and fall min = A
rise nom and fall nom = B
rise max and fall max = C
'(A B C D)rise min = A
fall min = B
rise nom and rise max = C
fall nom and fall max = D
'(A B C D E F)rise min = A
fall min = B
rise nom = C
fall nom = D
rise max = E
fall max = F
```

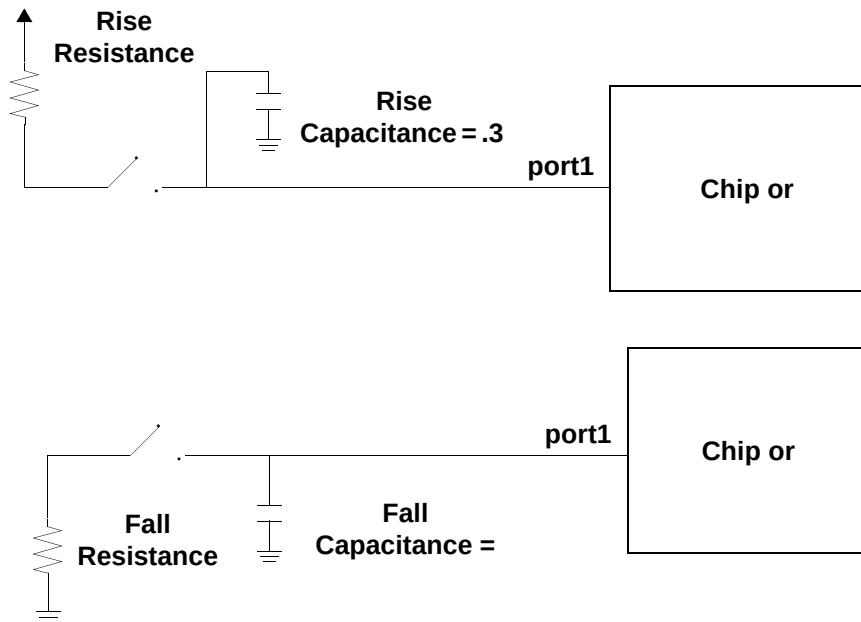
Example 1

In the first example, the capacitance of the device driving the chip or block through *port1* is set to 0.3 for all 6 operating conditions.

```
tdfSetSrcCapacitanceList "port1" .3
```

Using these directives, Astro evaluates incoming signals to the chip or block through *port1* as follows:

- When a signal is coming in through *port1* at its slowest speed and the transition is a rise (*rise max*), the capacitance is 0.3 pF.
- When a signal is coming in through *port1* at its slowest speed and the transition is a fall (*fall max*), the capacitance is 0.3 pF.



Examples

In this example, the capacitance of the device driving the chip or block through port1 is set to 0.3 for *rise min*, *rise nom*, and *rise max*. The capacitance for the same port is 0.4 for *fall min*, *fall nom*, and *fall max*.

```
tdfSetSrcCapacitanceList "port1" '(.3, .4)
```

In this example, the capacitance of the device driving the chip or block through port1 is set to 0.3 for *rise min* and *fall min*; it is set to 0.4 for *rise nom* and *fall nom*; and set to 0.5 for *rise max* and *fall max*.

```
tdfSetSrcCapacitanceList "port1" '(.3, .4, .5)
```

In this example, the capacitance of the device driving the chip or block through port1 is set to 0.3 for *rise min*, 0.4 for *fall min*, 0.5 for *rise nom* and *rise max*, and 0.6 for *fall nom* and *fall max*.

tdfSetSrcCapacitanceList

```
tdfSetSrcCapacitanceList "port1" '(.3,.4,.5,.6)
```

In this example, the capacitance of the device driving the chip or block through port1 is set to 0.3 for *rise min*, 0.4 for *fall min*, 0.5 for *rise nom*, 0.6 for *fall nom*, 0.7 for *rise max* and 0.8 for *fall max*.

```
tdfSetSrcCapacitanceList "port1" '(.3,.4,.5,.6, .7, .8)
```

tdfSetSrcResistance

Indicates the resistance of the device driving the chip or block through a specified input port under a specified operating condition.

Astro calculates the slope (slew) of signals that originate within the chip or block. However, primary inputs such as pads are driven by external devices. Because the timing characteristics for these devices are not in the cell library, Astro cannot know the strength of the driver or the capacitance of the wiring that connects the driver to the primary input. Therefore, you must specify these parameters:

- *tdfSetSrcResistance* specifies the strength of the driver, or use [tdfSetSrcResistanceList](#), which allows you to specify the strength of the driver for all operating conditions.
- [tdfSetSrcCapacitance](#) specifies the capacitance of the wiring that connects the driver to the primary input or use [tdfSetSrcCapacitanceList](#), which allows you to specify the capacitance of the wiring that connects the driver to the primary input for all operating conditions.

Note:

If your EDIF netlist does not define ports, you must first create a port on an existing net using [dbCreateGlobalNetByScanChain](#).

Tool

Milkyway

Syntax

```
tdfSetSrcResistance "portName" "condition" value
```

where the arguments are as follows:

<i>portName</i>	Name of the input port being specified. Valid values: Name of any port in the chip or block.
<i>condition</i>	The operating condition under which the specification applies. Valid values: • <i>rise min</i> means that the specification applies when the signal is coming in at its fastest speed and the transition is a rise. • <i>rise nom</i> means that the specification applies when the signal is coming in at its nominal speed and the transition is a rise. • <i>rise max</i> means that the specification applies when the signal is coming in at its slowest speed and the transition is a rise. • <i>fall min</i> means that the specification applies when the signal is coming in at its fastest speed and the transition is a fall. • <i>fall nom</i> means that the specification applies when the signal is coming in at its nominal speed and the transition is a fall. • <i>fall max</i> means that the specification applies when the signal is coming in at its slowest speed and the transition is a fall.

value

The resistance of the device driving the chip or block through the specified input port under the specified operating condition.

Valid values: Any non-negative, floating-point number.

Note:

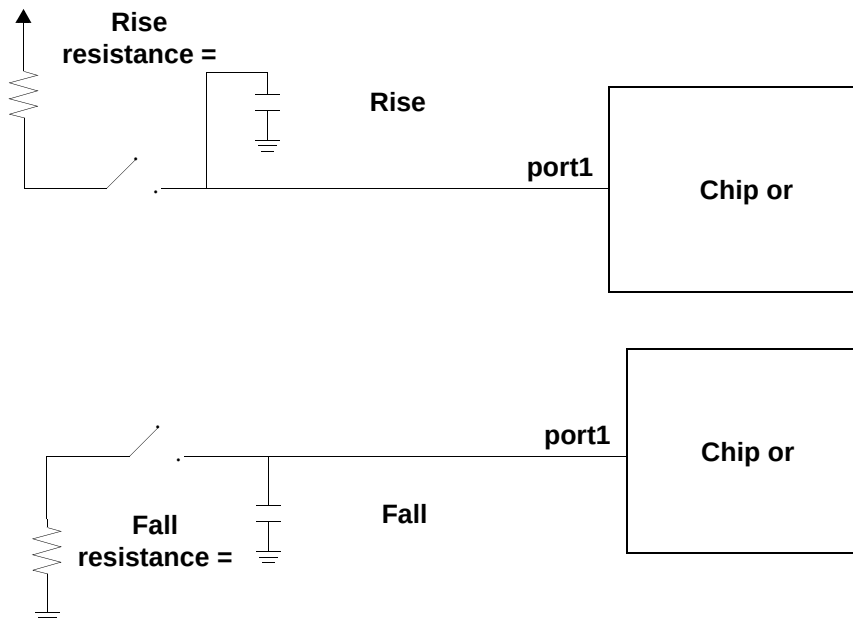
The source resistance value you provide should already have threshold voltage factored in.

Example

```
tdfSetSrcResistance "port1" "rise max" .4
tdfSetSrcResistance "port1" "fall max" .3
```

Using these directives, Astro evaluates incoming signals to the chip or block through *port1* as follows:

- When a signal is coming in through *port1* at its slowest speed and the transition is a rise (*rise max*), the resistance is 0.4 kohm.
- When a signal is coming in through *port1* at its slowest speed and the transition is a fall (*fall max*), the resistance is 0.3 kohm.



tdfSetSrcResistanceList

Indicates the resistance of the device driving the chip or block through a specified input port for all operating conditions.

Astro calculates the slope (slew) of signals that originate within the chip or block. However, primary inputs such as pads are driven by external devices. Because the timing characteristics for these devices are not in the cell library, Astro cannot know the strength of the driver or the capacitance of the wiring that connects the driver to the primary input. Therefore, you must specify these parameters:

- [tdfSetSrcResistance](#) specifies the strength of the driver, or use *tdfSetSrcResistanceList*, which allows you to specify the strength of the driver for all operating conditions.
- [tdfSetSrcCapacitance](#)) specifies the capacitance of the wiring that connects the driver to the primary input or use [tdfSetSrcCapacitanceList](#), which allows you to specify the capacitance of the wiring that connects the driver to the primary input for all operating conditions.

Note:

If your EDIF netlist does not define ports, you must first create a port on an existing net using [dbCreateGlobalNetByScanChain](#).

Tool

Milkyway

Syntax

```
tdfSetSrcResistanceList "portName" '(value)
```

where the arguments are as follows:

<i>portName</i>	Name of the input port being specified. Valid values: Name of any port in the chip or block
<i>value</i>	The resistance of the device driving the chip or block through the specified input port under all operating conditions. Specify one (1), two (2), three (3), four (4), or six (6) values to apply to the six conditions as listed below. Valid values: Any non-negative, floating-point number. Note: The source resistance value you provide should already have threshold voltage factored in.

The operating conditions under which the specified values apply are:

Condition...	The specification applies when the signal is coming in at its...
rise min	Fastest speed and the transition is a rise.
rise nom	Nominal speed and the transition is a rise.
rise max	Slowest speed and the transition is a rise.
fall min	Fastest speed and the transition is a fall.
fall nom	Nominal speed and the transition is a fall.
fall max	Slowest speed and the transition is a fall.

The values apply in the following way to the operating conditions:

```
'(A) applies to all 6 conditions
'(A B) rise min, rise nom, and rise max = A
fall min, fall nom, and fall max = B
'(A B C)rise min and fall min = A
rise nom and fall nom = B
rise max and fall max = C
```



```

'(A B C D)rise min = A
fall min = B
rise nom and rise max = C
fall nom and fall max = D
'(A B C D E F)rise min = A
fall min = B
rise nom = C
fall nom = D
rise max = E
fall max = F

```

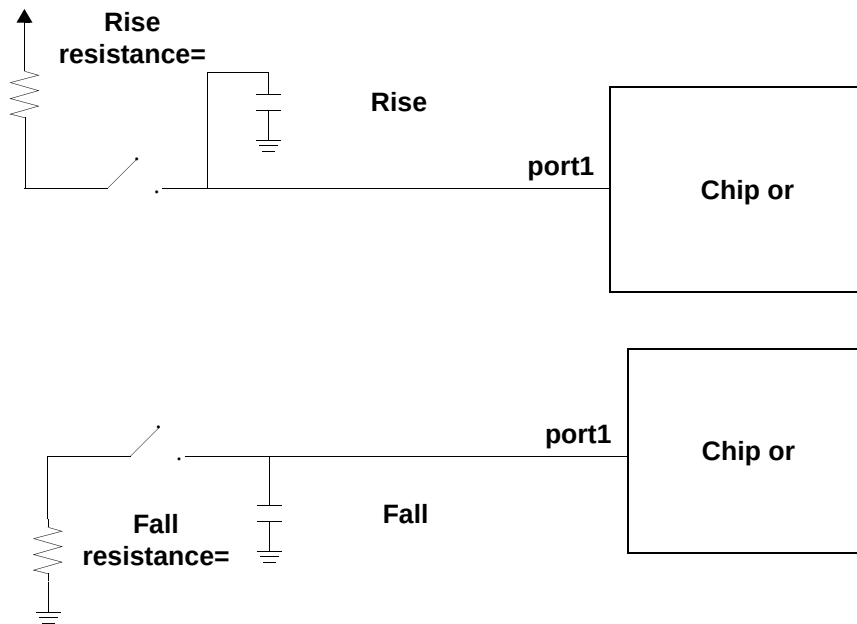
Example

In the first example, the resistance of the device driving the chip or block through *port1* is set to 0.4 for *rise min*, *rise nom*, and *rise max* and to 0.3 for *fall min*, *fall nom*, and *fall max*.

```
tdfSetSrcResistanceList "port1" '(.4, .3)
```

Using these directives, Astro evaluates incoming signals to the chip or block through *port1* as follows:

- When a signal is coming in through *port1* at its slowest speed and the transition is a rise (*rise max*), the resistance is 0.4 kohm.
- When a signal is coming in through *port1* at its slowest speed and the transition is a fall (*fall max*), the resistance is 0.3 kohm.



tdfSetTimeBorrowLimit

Sets the maximum amount of borrowed time allowed on specified latches.

Tool

Milkyway

Syntax

```
tdfSetTimeBorrowLimit maxTime '(objects)
```

where the arguments are as follows:

<i>maxTime</i>	Maximum amount of borrowed time allowed on specified latches. Valid values: Any floating-point number less than the pulse width of the reference clock minus the setup time.
----------------	---

objects

Objects related to latches for which you are setting the maximum amount of borrowed time allowed.

Specify objects in the following format:

"obj1" "obj2" ... "objn"

where an *obj* can be any of the following:

- Name of a clock. In this case, the *maxTime* applies to any latch in the transitive fanout of a source of the clock.
- Name of an enable pin of a latch.
- Name of a cell instance (latch).

Example

```
tdfSetTimeBorrowLimit 2.8 ' ("LATCH3" "C1")
```

This function limits the amount of borrowed time allowed on LATCH3 and any latch in the transitive fanout of a source of clock C1 to 2.8 ns.

26

tg Commands for Astro Family Products and Milkyway

This chapter describes commands with the prefix *tg*. Use the *tg* commands for timing operations. The commands are listed alphabetically.

- [tgCheckTimingConstraint](#)

For more timing commands, refer to commands with the following prefixes:

- *clf* (cell library format)
- *ta* (timing)
- *tdf* (top design format)

tgCheckTimingConstraint

Checks timing data and constraints. Initializes the timing structure and constraints graph.

Tools and Menu Commands

Astro: Timing > Check Timing Data

Jupiter: Timing > Check Timing Data

Astro Floorplanning: Timing > Check Timing Data

Syntax

```
tgCheckTimingConstraint CellId
```

where the argument is as follows:

<i>cellId</i>	Database object ID of any open cell in the current library that applies to this command. The argument <i>cellId</i> can be either of the following: • Cell ID • (geGetEditCell) For information, see “Specifying Cells” on page 6 of the Place and Route Getting Started User Guide.
---------------	--

57

tr Commands for Milkyway

This chapter describes commands with the prefix *tr*, listed alphabetically.

Commands with the *tr* prefix are mainly routing-related commands and are used with Astro, Astro Interactive Ultra, or Columbia. Most of the commands in this group correspond to some actions on the routing Control Panel, which you access by entering [trPanel](#) on the command line in the message/input area or by choosing Auto Ultra > Control Panel.

General routing commands with the *tr* prefix require an Astro license, and interactive routing commands require an Astro Interactive Ultra license.

The commands are listed below.

- [trCancel](#)

Note:

Unless otherwise noted, all command examples are shown as you would enter them in the message/input area, which can differ from Tcl or Scheme syntax. If you want to add commands to startup files or use them in user-defined programs, you must use exact Tcl or Scheme syntax (depending on your language setting on startup).

For more information on Scheme commands and equivalent Tcl commands, see the *Milkyway Environment Extension Language Reference Manual*. Synopsys does not distribute general Tcl documentation, but a useful Tcl/tk online reference manual is available at <http://www.tcltk.com/TclTkMan/TclTkManPages.html>.

trCancel

Cancels or disables the routing Control Panel.

Invoke this command by clicking the Cancel button on the routing Control Panel (see [trPanel](#)).

59

tu Commands for Milkyway

This chapter describes the commands with the prefix *tu*. The prefix *tu* stands for TCL user interface. Use the *tu* commands for viewing and managing Synopsys online help as well as GUI menus and icons. The commands are listed alphabetically.

- [tuHelp](#)
- [tuHelpCancel](#)

Note:

Unless otherwise noted, all command examples are shown as you would enter them from the command line which can differ from Scheme syntax. If you want to add commands to startup files, or use them in user-defined programs, you must use exact Astro Scheme syntax. Please see the *Milkyway Environment Extension Language Reference Manual* for more information.

tuHelp

Brings up the Help default page. The Help system uses a standard browser such as Firefox or Mozilla. Previously, the `tuHelp` command opened the documentation browser.

The `tuHelp` command is the same as the `help` command. To find out how to use the standard browser Help, click the Help QuickStart link at the bottom of each HTML Help page.

Tools and Menu Commands

Astro: Help > Astro

Jupiter: Help > Jupiter

Milkyway: Help > Milkyway

tuHelpCancel

Closes the online Help documentation browser by specifying its browser ID. The first documentation browser window opened is given the browser ID of zero (0). Each successive browser you open is numbered incrementally.

Tools and Menu Commands

Milkyway: Documentation Browser: File > Close Browser

Syntax

```
tuHelpCancel [<id>]
```

where the argument is as follows:

<id> ID

Example

The following example closes the second on-line browser.

```
tuHelpCancel 1
```

See Also

[tuHelp](#)

58

wv Commands for Milkyway

This chapter describes commands with the prefix `wv` which stands for waveform viewer. You can use `wv` commands to control viewing and data within Milkyway's waveform window. The commands are listed alphabetically.

Note:

Unless otherwise noted, all command examples are shown as you would enter them from the command line which can differ from Scheme syntax. If you want to add commands to startup files, or use them in user-defined programs, you must use exact Scheme syntax.

- [`wvCloseSource`](#)
- [`wvCloseView`](#)
- [`wvDeleteGraph`](#)
- [`wvDeleteView`](#)

- `wvDeleteWave`
- `wvNewGraph`
- `wvNewView`
- `wvOpenSource`
- `wvOpenView`
- `wvPlotWave`
- `wvSelectWave`
- `wvSetGraphCurrent`
- `wvSetSourceCurrent`
- `wvSetView`
- `wvSetViewCurrent`
- `wvViewClear`
- `wvViewDebug`

wwCloseSource

This commands closes the data source and withdraws all displayed waveforms of the data source.

Tool

Milkyway

Syntax

```
wwCloseSource "sourceName"
```

wwCloseView

Closes the view of *viewName*. If the view is the current view, select the next one as the current view. The graphs and their waveforms are retained. *wwOpenView* (page 601), with *viewName*, bring back the original graphs and waveforms.

Tool

Milkyway

Syntax

```
wwCloseView "viewName"
```

wwDeleteGraph

Deletes the graph of *graphName* inside the view of *viewName*. If the graph is the current graph, select the next graph as the current graph inside the view of *viewName*.

Tool

Milkyway

Syntax

```
wvDeleteGraph "viewName" "graphName"
```

wvDeleteView

Deletes the view of *viewName*. If it is the current view, select the next one as the current view. The graphs and their waveforms are deleted. *wvOpenView* (page 601), with *viewName*, creates a brand new view.

Tool

Milkyway

Syntax

```
wvDeleteView "viewName"
```

wvDeleteWave

Deletes the waveform of the signal with *SignalId* of the data source of *sourceName*, on the graph of *graphName* inside the view of *viewName*.

Tool

Milkyway

Syntax

```
wwDeleteWave "viewName" "graphName" "sourceName" "SignalId"
```

wwNewGraph

Opens a new graph window inside the view of *viewName* and sets the graph to the current graph inside the view of *viewName*.

Tool

Milkyway

Syntax

```
wwNewGraph "viewName" "graphName"
```

wwNewView

Opens a new view window with *viewName* and sets it to the current view. If a view of the same *viewName* is closed by `wwCloseView` (page 598), then `wwOpenView` (page 601) brings back all graphs and their waveforms.

Tool

Milkyway

Syntax

```
wwNewView "viewName"
```

See Also

[wwCloseView](#)

wwOpenSource

Opens the data source of *sourceName* and makes it the current source. Currently only the “SPICE” format is supported. The “SPICE” format is TR0/AC0/DC0 output file from Star-HSPICE.

Note:

sourceName must be unique among all sources.

Tool

Milkyway

Syntax

```
wwOpenSource "Format" "sourceName"
```

wwOpenView

Brings back all graphs and their waveforms into view. *wwOpenView* with the same *viewName* creates a brand new view.

Tool

Milkyway

Syntax

```
wwOpenView "viewName"
```

wwPlotWave

Plots the signal with *SignalId* of the data source of *sourceName* on the graph of *graphName* inside the view of *viewName*.

Tool

Milkyway

Syntax

```
wwPlotWave "graphName" "sourceName" "SignalId"
```

wwSelectWave

Sets the selection status of the waveform of the signal with *SignalId* of the data source of *sourceName* on the graph of *graphName* inside the view of *viewName*.

Tool

Milkyway

Syntax

```
wwSelectWave "viewName" "graphName" "sourceName" "Signalid"  
"selectStatus"
```

selectStatus is a string, the valid *selectStatus* are:

"select", "unselect" and "toggle"

wwSetGraphCurrent

Sets the graph of *graphName* as the current graph inside the view of *viewName*.

Tool

Milkyway

Syntax

```
wwSetGraphCurrent "viewName" "graphName"
```

wwSetSourceCurrent

Sets the data source as the current source

Tool

Milkyway

Syntax

```
wwSetSourceCurrent "sourceName"
```

wwSetView

Sets the view of *viewName*

Tool

Milkyway

Syntax

```
wwSetView [-w <#>] [-p <#>] [[x1 y1 x2 y2]]
```

where the arguments are as follows:

-w <#> Window Id

-p <#> Panel ID

[x1 y1 x2 y2] Bounding Box : x1 y1 x2 y2

wwSetViewCurrent

Sets the view of *viewName* as the current view.

Tool

Milkyway

Syntax

```
wwSetViewCurrent "viewName"
```

wwViewClear

Deletes all the waveform view data.

Tool

Milkyway

Syntax

wwViewClear

wwViewDebug

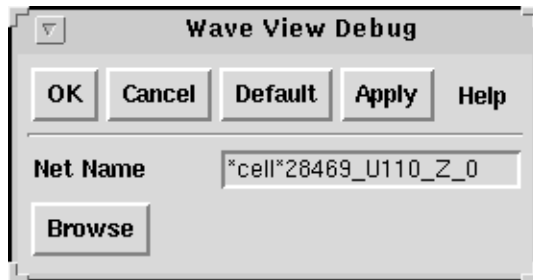
Displays waveform generated from *AWE Analysis* and *HSPICE Simulation*. To run *wwViewDebug*, the waveform view data must be prepared first.

Tool

Milkyway

Wave View Debug Dialog Box

The menu command opens the following dialog box. Its options are described below.

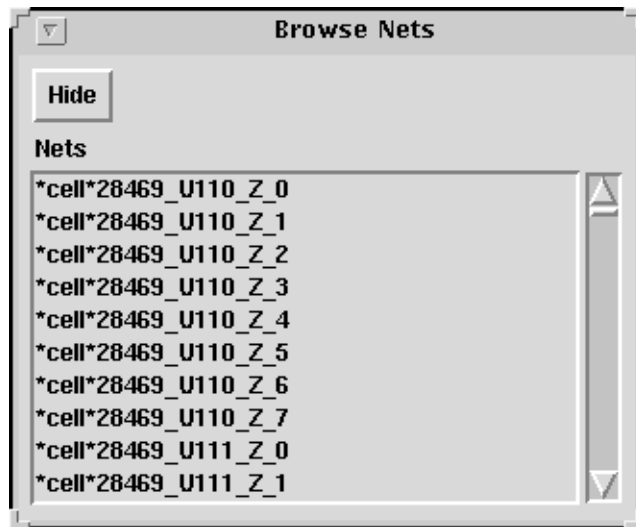


Net Name

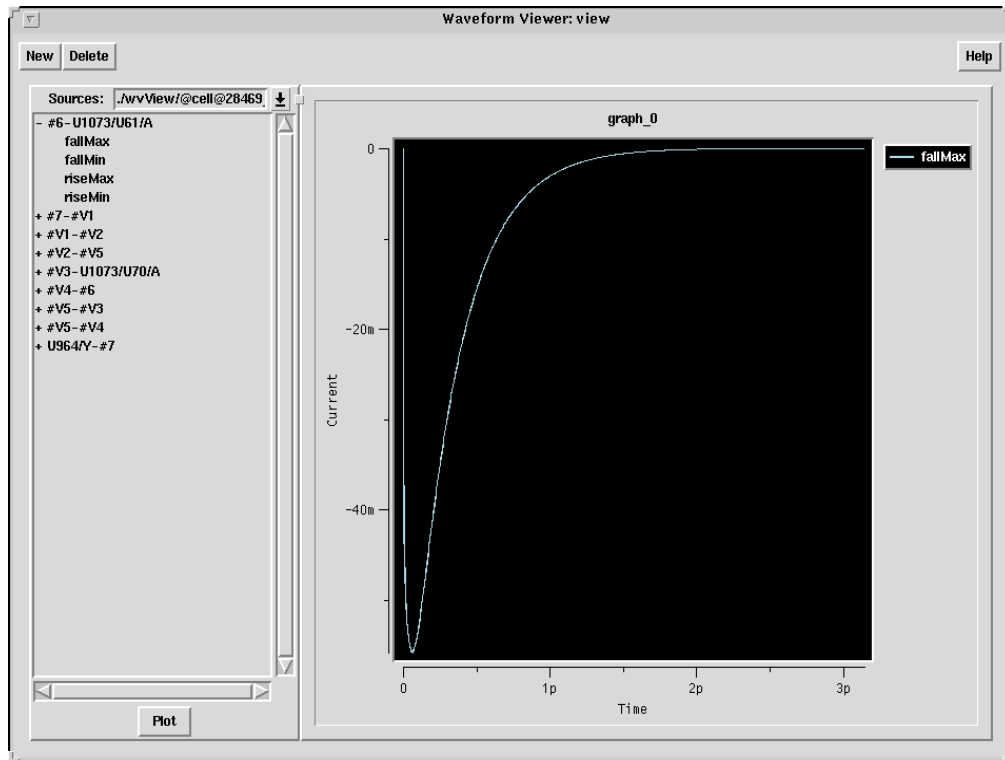
Type the name of the net where you want to run waveform view debug. You can only type in one net name.

Browse

Click to display the *Browse Nets* window from which you can choose from any net.



Waveform View Debug Display in a Graph Window



The graph window displays either voltage waveform or current waveform depending on how the waveform view data is prepared before running *wvViewDebug*. In the example shown above (prepared in *signalEMAnalysis*), it shows the current waveform on each resistor in the net.

See Also

poSignalEMAnalysis

64

Commands With No Prefix (No License)

This chapter describes commands with no categorized prefix. The commands are listed alphabetically. They do not require a license.

For lower level commands with no prefix, see the *Milkyway Environment Extension Language Reference Manual*.

addUserMenu

Allows you to define and customize menus.

The customized menu has a blue foreground to differentiate from the system menu. Both “entry_name” and “command_name” must be in the string list.

Tool

Astro

Syntax

```
addUserMenu menu_name (list menu_item ...)
menu_item: (cons "menu_entry" "command_name") # regular
entry "menu_label" # a label "" # separator
```

Example

```
addUserMenu "UserMenu" (list "Open" (cons " Library"
"geOpenLib")
(cons " Cell" "geOpenCell") "" "Option" (cons " Window"
"geWindowOption")(cons " Layer" "geLayerPanel") "" "Help"
(cons "
geOpenLib" "help 'geOpenLib") (cons " geOpenCell" "help
'geOpenCell"))
```

advSearch

Provides advanced search capabilities for Help.

Tools

Astro

Jupiter

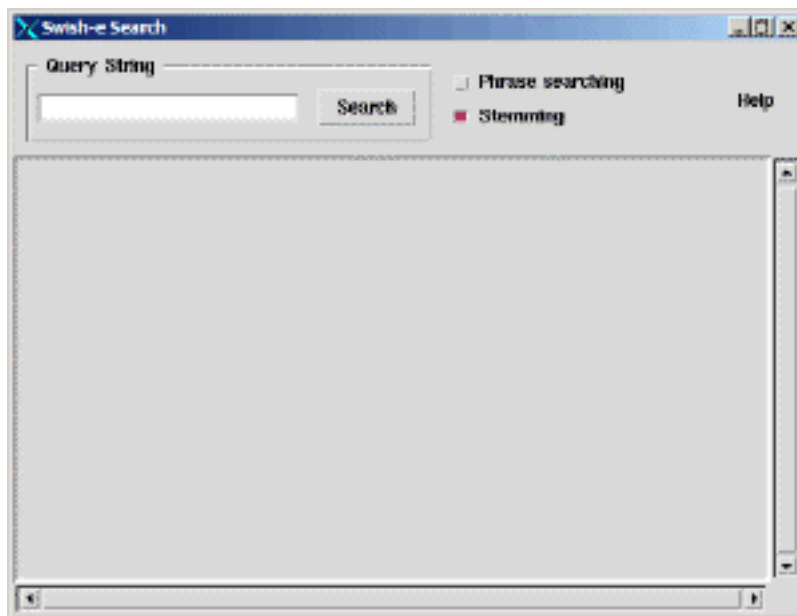
Milkyway

Syntax

`advSearch`

Description

The `advSearch` command provides fast and flexible search capabilities. Fast search is provided by the Swish-e search engine; for details, including licensing, see the Swish-e Web site at <http://www.swish-e.org>. Flexibility is provided by stemming and phrase searching.



The `advSearch` command opens a Tcl dialog box. When you type a query in the query string box and click the Search button, a list of links matching your query is returned.

With advanced search, you can search on exact phrases. Alternatively, you can choose stemming to query words having the same root word. For example, a query for "floorplan" can return results for "floorplanning," "floorplanner," and others. Advanced search also allows you to use Boolean operators to specify a search order. These topics and more are covered in more detail below.

Phrase Searching

A phrase is an exact sequence of characters. To search for a phrase, select the “Phrase searching” checkbox. For example,

```
Commands for importing LEF
```

finds this phrase in the Help system. The phrase search is not case-sensitive. The phrase

```
commands for importing LEF
```

returns the same results.

If you type

```
geOpenLib and dbOpenCell
```

the search engine tries to find the exact phrase “geOpenLib and dbOpenCell.”

Stemming

Stemming is the conversion of a word to its root form (stem). Key terms of a query are represented by stems rather than by the original words. To search by stemming, select the Stemming checkbox.

With stemming enabled, a search for the string

```
running
```

returns *run*, *runs*, *running*, and so on.

If you use a wildcard search with stemming enabled, you might get unexpected results. Consider a query on the string

```
running*
```

The `advSearch` command's search is based on the root word *run*. That is, the query is actually specifying

```
run*
```

which returns *run* and *runs*, as well as *runabout*, *runaround*, *runway*, and so on.

Boolean Operators

You can use Boolean operators to refine your search. Without Boolean operators, Swish-e assumes you are ANDing words together. Use of Boolean operators is not case-sensitive. The phrase

```
geOpenLib and dbOpenCell
```

returns pages where both words occur somewhere on the page. The phrase

```
geOpenLib or dbOpenCell
```

returns results where at least one of the words occurs on the page.

The not operator inverts the results of a search. For example,

```
not dbOpenCell
```

returns every file that does not have the string `dbOpenCell`.

For additional search control, you can use parentheses to group searches. For example,

```
not (geOpenLib and dbOpenCell)
```

See "[Search Order](#)" below for more information.

To search for the words *and*, *or*, or *not*, place these words in double quotation marks. Remember to protect the quotes from being evaluated by the shell using single quotes or the escape character (backslash).

```
geOpenLib \"and\" dbOpenCell
```

The above query finds all three words, but in any order. Or consider the following query:

```
(geOpenLib and dbOpenCell) and not dbCloseCell
```

This query finds first the files that contain both the word `geOpenLib` and the word `dbOpenCell`. Then the search finds files that do not contain the word `dbCloseCell`.

Wildcard Searches

The wildcard asterisk (*) can be used only at the end of a word. Otherwise the wildcard is considered a normal character. For example,

```
dbHier*
```

returns all files whose name includes `dbHier*`, such as `dbHierSave`, `dbHierDelete`, and `dbHierPurge`.

Search Order

The search process proceeds from left to right. For example,

```
geOpenLib not dbOpenCell and dbCloseCell
```


first finds documents that have `geOpenLib`. Then it finds all documents that do not have `dbOpenCell`. Those two results are ANDed, resulting in a list of documents that include `geOpenLib` but not `dbOpenCell`. Finally, that list is ANDed with the list of documents that contain `dbCloseCell`.

It can be helpful to combine parentheses with Boolean operators to force the order of evaluation. For example,

```
geOpenLib not (dbOpenCell and dbCloseCell)
```

retrieves files with `geOpenLib` that do not contain both words `dbOpenCell` and `dbCloseCell`.

cnvGetBuffering

Returns the current canvas buffering state.

Syntax

```
cnvGetBuffering <void>
```

Arguments: none

Description

The `cnvGetBuffering` command returns the current canvas buffering state. It returns `#t` if buffering is enabled and `#f` if buffering is disabled.

Example

```
cnvGetBuffering
```

cnvSetBuffering

Sets the canvas buffering state.

Syntax

```
cnvSetBuffering <boolean boolValue>
```

where the argument is as follows:

boolValue

#t to turn buffering on

#f to turn buffering off (default)

Description

The `cnvSetBuffering` command sets the canvas buffering state. Buffering is off by default.

Example

```
cnvSetBuffering #t
```

exit

Exits the application.

Syntax

```
exit
```

Description

The `exit` function exits the application. If any modified or unsaved cells are opened, the application prompts you to save or discard them before exiting.

Example

```
exit
```

FeatureCheckedOut

Verifies whether a license key has been checked out. This function returns #t, if the license key has been checked out, or #f, if it has not been checked out.

Tool

Milkyway

Syntax

```
FeatureCheckedOut? keyName
```

where the argument is as follows:

<i>keyName</i>	The name of the license key.
----------------	------------------------------

Example

In the following example, you would be verifying whether the license key “MDataPrep” has been checked out.

```
FeatureCheckedOut? "MDataPrep"  
#t
```

functions

Lists commands and functions in the messages area of the Synopsys tool window.

Caution!

Some functions included in the list are not documented. Support of undocumented functions is subject to change without notice.

Tool

Milkyway

Syntax

```
functions "pattern"
```

where the argument is as follows:

pattern

Pattern to match the name of the commands and functions you want to list.

Valid values:

Any pattern. All commands and functions with the pattern appearing anywhere in its name is returned.

Example

```
functions "geSet"
```

This example lists all commands with the string geSet in the messages window of the Synopsys tool window, as shown below:

```
geSetLayerSelectable  
geSetErrorType  
geSetObjectSelectable  
geSetLayerVisible  
geSetObjectVisible  
geSetViewCell
```

getFeature

Gets licenses for Synopsys feature sets.

MILKYWAY

The Milkyway package includes the following licenses for using Milkyway features:

- Milkyway provides the basic capabilities of the tool, including data preparation and basic cell editing.
- MDynLink license, which provides the capability to link object modules to a Synopsys tool.
- MDataPrep provides the data conversion features in the Milkyway tool.

AP

The Astro package includes the following licenses for using Astro features:

- All the licenses of the MILKYWAY set described above.
- Astro provides floorplanning, placement, routing, and LVS and DRC checking in Astro.
- APCS provides clock tree synthesis capabilities.
- APTIME provides timing operations capabilities.

This license can include the following optional sublicenses:

- APDP provides data path layout capabilities.
- APHPO provides high-performance routing capabilities.
- APGA4AP provides GA capabilities within Astro.
- APpwr provides power analysis capabilities within Astro.

APGA

The AstroGA package includes the following licenses for using AstroGA features:

- All the licenses of the MILKYWAY set described above.
- AstroGA provides floorplanning, placement, routing, and LVS and DRC checking in Astro.
- APGACS provides clock tree synthesis capabilities.
- APGATime provides timing operations capabilities.

This license can include the following optional sublicenses:

- APGADP provides data path layout capabilities.
- APGAHPO provides high-performance routing capabilities.
- APGApwr provides power analysis cap within Astro.

SolarII

The SolarII package includes the following licenses for using SolarII features:

- APSolar provides critical path optimization, high fanout net synthesis, and net optimization.
- APSolarII provides design optimizations including global restructuring and boolean transformations.

SolarGAII

The SolarII package includes the following licenses for using SolarII features:

- APGASolar provides critical path optimization, high fanout net synthesis, and net optimization.
- APGASolarII provides design optimizations including global restructuring and boolean transformations.

PPAP

The Jupiter package includes the following licenses for using Jupiter features:

- All the licenses of the MILKYWAY set described above.
- Jupiter provides physical floorplanning in Jupiter.
- FPDP provides the ability to handle soft macro bus pins.
- FPABP provides the ability to explore different block placement results.
- FPTime provides the ability to perform area estimates based on timing-driven placement.
- FPTA provides timing analysis capabilities.

The Jupiter license can also include the following optional sublicense:

- PPPRT provides physical retiming capabilities.

AP-D

The Very Deep Submicron (VDSM) package includes licenses for using all standard Astro and Jupiter features for your VDSM designs.

Note:

To release a license, use the [releaseFeature](#) command.

Tool

Milkyway

Syntax

```
getFeature "license" wait?
```

where the arguments are as follows:

- | | |
|----------------|--|
| <i>license</i> | Indicates the license (or sublicense) you want to use.
Valid values: <ul style="list-style-type: none">• For Astro: Milkyway, Astro, APCS, APTIME, APDP, APHPO, APGA4AP, APpwr• For AstroGA: Milkyway, AstroGA, APGACS, APGATIME, APGADP, APGAHPO, APGApwr• For Milkyway: Milkyway, MDynLink, MDataPrep• For Jupiter: Milkyway, JUAPlan, FPDP, FPABP, FPTIME, FPTA, PPPRT• For SolarII: APSolar, APSolarII• For SolarGAI: APGASolar, APGASolarII |
| <i>wait?</i> | Indicates whether you want to wait for the feature set if it is not currently available.
Valid values: <ul style="list-style-type: none">• <i>#t</i> indicates that you want to wait (generally used in scripts to indicate that you want to suspend execution of the script until a license becomes available).• <i>#f</i> indicates that you do not want to wait. |

See Also

[releaseFeature](#) – Releases licenses for Synopsys tool feature sets.

getFunctionLicensing

Displays a list of commands and functions, the tool(s) it is supported in, and license options that are required.

Tool

Milkyway

Syntax

```
getFunctionLicensing [pattern action]
```

where the arguments are as follows:

pattern A part or all of a command or Scheme function name.

action function|command

Example 1

```
getFunctionLicensing
```

Displays all commands and functions, the tool it is supported in, and the license options that are required.

Example 2 (with part of a pattern)

```
getFunctionLicensing "Timi" "command"
```

Displays:

```
tgTimingConstrGenerator (Milkyway) (no option needed)
axTransformTimingToCrit (Astro) (no option needed)
cmCmdLoadTimingRequirements (Milkyway) (no option needed)
dbAssignNetTimingSpacing (Milkyway) (no option needed)
tgCheckTimingConstraint (Milkyway) (no option needed)
tgCheckTimingData (Milkyway) (no option needed)
aprCreateTimingModel (Astro Jupiter) (JUAPlan)
```

.....(partial listing)....

Example 3

```
getFunctionLicensing "ntTimingVerify" "command"
```

Displays:

```
ntTimingVerify (Astro AstroGA AstroCH SolarII SolarIIGA  
Jupiter) (APTime APGATime FPTime FPTA FPPRT APSolar  
APGASolar APSolarII APGASolarII JUAPlan JUATime) #t
```

which means that ntTimingVerify is supported by the following tools:

Astro AstroGA AstroCH SolarII SolarIIGA Jupiter and requires at least one of the following licenses:

APTime APGATime FPTime FPTA FPPRT APSolar APGASolar
APSolarII APGASolarII JUAPlan JUATime

See Also

[FeatureCheckedOut](#)

[getFeature](#)

[listFeature](#)

[releaseFeature](#)

listFeature

Lists all keys checked out by the active tool.

Tool

Milkyway

Syntax

`listFeature`

releaseFeature

Releases licenses for a Synopsys tool. Releasing a license for a Synopsys tool also releases all associated sublicenses.

Synopsys licenses include the following:

MILKYWAY

The Milkyway package includes the following licenses for using Milkyway features:

- Milkyway provides the basic capabilities of the tool, including data preparation and basic cell editing.
- MDynLink license, which provides the capability to link object modules to a Synopsys tool.
- MDataPrep provides the data conversion features in the Milkyway tool.

AP

The Astro package includes the following licenses for using Astro features:

- All the licenses of the MILKYWAY set described above.

- Astro provides floorplanning, placement, routing, and LVS and DRC checking in Astro.
- APCS provides clock tree synthesis capabilities.
- APTIME provides timing operations capabilities.

This license can include the following optional sublicenses:

- APDP provides data path layout capabilities.
- APHPO provides high-performance routing capabilities.
- APGA4AP provides GA capabilities within Astro.
- APpwr provides power analysis capabilities within Astro.

APGA

The AstroGA package includes the following licenses for using AstroGA features:

- All the licenses of the MILKYWAY set described above.
- AstroGA provides floorplanning, placement, routing, and LVS and DRC checking in Astro.
- APGACS provides clock tree synthesis capabilities.
- APGATIME provides timing operations capabilities.

This license can include the following optional sublicenses:

- APGADP provides data path layout capabilities.
- APGAHPO provides high-performance routing capabilities.
- APGApwr provides power analysis cap within Astro.

SolarII

The SolarII package includes the following licenses for using SolarII features:

- APSolar provides critical path optimization, high fanout net synthesis, and net optimization.
- APSolarII provides design optimizations including global restructuring and boolean transformations

SolarGAII

The SolarII package includes the following licenses for using SolarII features:

- APGASolar provides critical path optimization, high fanout net synthesis, and net optimization.
- APGASolarII provides design optimizations including global restructuring and boolean transformations

PPAP

The Jupiter package includes the following licenses for using Jupiter features:

- All the licenses of the MILKYWAY set described above.
- Jupiter provides physical floorplanning in Jupiter.
- FPDP provides the ability to handle soft macro bus pins.
- FPABP provides the ability to explore different block placement results.
- FPTIME provides the ability to perform area estimates based on timing-driven placement.

- FPTA provides timing analysis capabilities.

The Jupiter license can also include the following optional sublicense:

- PPPRT provides physical retiming capabilities

AP-D

The Very Deep Submicron (VDSM) package includes licenses for using all standard Astro and Jupiter features for your VDSM designs.

Note:

To get a license, use the `getFeature` command.

Tool

Milkyway

Syntax

```
releaseFeature "license"
```

where the arguments are as follows:

<i>license</i>	Indicates the license (or sublicense) you want to use. Valid values:
	• For Astro: Milkyway, Astro, APCS, APTIME, APDP, APHPO, APGA4AP, APpwr • For AstroGA: Milkyway, AstroGA, APGACS, APGATIME, APGADP, APGAHPO, APGApwr • For Milkyway: Milkyway, MDynLink, MDataPrep • For Jupiter: Milkyway, JUAPlan, FPDP, FPABP, FPTIME, FPTA, PPPRT • For SolarII: APSolar, APSolarII • For SolarGAI: APGASolar, APGASolarII

See Also

[getFeature](#) gets licenses for your Synopsys tool feature sets.

removeUserMenu

Allows you to remove menus.

Note:

If no argument is given for removeUserMenu, all the users menus are removed.

Tool

Milkyway

Syntax

```
removeUserMenu (menu_name)
```

Example

```
removeUserMenu "UserMenu" #Remove "UserMenu"  
#Remove all user defined menu
```

setPostCallBack

Assigns a Scheme command to a Tcl form button that is evaluated after the button is activated and the button action is performed.

Syntax

```
setPostCallBack <string buttonName> <string schemeCmd>
```

buttonName

The actual name of the button

schemeCmd

The Scheme command

Description

The setPostCallBack function assigns a Scheme command to a Tcl form button that is evaluated after the button is activated and the button action is performed. The specified command overrides the previous command, if any, that was assigned. This function is similar to the setPreCallBack function.

The actual button name can be found in the application's command history window after the button is activated. For example:

Form Button	Actual Button Name
Browse...	browse...
Next Ref Lib	nextLib
All Vis On	allVisOn

The standard form buttons do not have names that appear in the command history window. Their corresponding names are:

Standard Form Button	Actual Button Name
OK	ok
Apply	apply
Cancel	cancel
Default	default
Help	help

Within the specified Scheme command, the current Tcl form name can be accessed through the Scheme variable, FORM_NAME.

Example

```
setPostCallBack "ok" "(display \"OK button clicked\\n\") "  
setPostCallBack "allSelOff" "(display FORM_NAME) "
```

setPreCallBack

Assigns a Scheme command to a Tcl form button that is evaluated after the button is activated, but before the button action is performed.

Syntax

```
setPreCallBack <string buttonName> <string schemeCmd>
```

buttonName

The actual name of the button

schemeCmd

The Scheme command

Description

The setPreCallBack function assigns a Scheme command to a Tcl form button that is evaluated after the button is activated but before the button action is performed. The specified command overrides the previous command, if any, that was assigned. This function is similar to the setPostCallBack function

The actual button name can be found in the application's command history window after the button is activated. For example:

Form Button	Actual Button Name
Browse...	browse...
Next Ref Lib	nextLib
All Vis On	allVisOn

The standard form buttons do not have names that appear in the command history window. Their corresponding names are:

Standard Form Button	Actual Button Name
OK	ok
Apply	apply
Cancel	cancel
Default	default
Help	help

Within the specified Scheme command, the current Tcl form name can be accessed through the Scheme variable, FORM_NAME.

Example

```
setPreCallback "ok" "(display \"OK button clicked\\n\\n\")"
setPreCallback "allSelOff" "(display FORM_NAME)"
```

vars

Lists all Scheme variables that contain the specified search string in their names.

Syntax

```
vars <string searchStr>
```

searchStr

The search string

Description

The vars function lists all Scheme variables that contain the specified search string in their names. The search string is case-sensitive. In the displayed list, procedures are followed by the “<procedure>” string and variables are followed by their corresponding value.

Example

```
vars "Open"
```

This above function call might return something like this:

```
geBrowseOpenCell  <procedure>  
myOpenLibName    \"userLibrary\"  
dbOpenLibrary    <procedure>  
dbOpenLib        <procedure>
```

varValue

Returns the value of the specified Scheme variable.

Syntax

```
varValue <string varName>
```

searchStr

Name of the scheme variable

Description

The varValue function returns the value of the specified Scheme variable. If the variable is undefined, the function return false (#f).

Example

```
varValue "geOpenCell"  
varValue "myOpenLibName"  
varValue "myOpenLibName2"
```

This above function calls might return respectively something like this:

```
#<primitive-procedure geOpenCell>  
"userLibrary"  
#f
```

63

Commands With No Prefix for Milkyway

This chapter describes commands with no categorized prefix. The commands are listed alphabetically.

For lower level commands with no prefix, see the *Milkyway Environment Extension Language Reference Manual*.

- `->?`
- `->type`
- `augLibCompiler`
- `batchMode`
- `bell`
- `boolNotPolygons`
- `clearAll`

- `closeImageCell`
- `CONSTANT`
- `cPin`
- `cPinF`
- `createImageCell`
- `createRowRecord`
- `createTilePatternRecord`
- `createTrackRecord`
- `createWireDirRecord`
- `cycle`
- `debug`
- `demOpen`
- `dvSnapWindow`
- `FileExists?`
- `FileIsExecutable?`
- `FileIsReadable?`
- `FileIsWritable?`
- `flipChipMatrix`
- `flipChipPad`
- `formDefault`
- `fpGetFPOption`

- fpSetFPOption
- genUniformTileInLib
- getCaseSensitive
- getDisplayTrackingMsg
- getEditWindow
- getFormField
- getFormFieldValue
- go
- hvSetDebugNetMapping
- hiliteNet
- IsApplicationIn64BitMode
- lib2lclf
- listBindKey
- loadFromPath
- loadUserForm
- loadUserFormFromPath
- make-db
- net
- netCapacitance
- netDelay
- netLength

- `netTransition`
- `notDoneYet`
- `obsoleteCmd`
- `pad`
- `pin`
- `Plot Commands`
- `plotAddBoxText`
- `plotAddPath`
- `plotAddPolygon`
- `plotAddRectangle`
- `plotAddScreenText`
- `plotAddSegment`
- `plotAddText`
- `plotClear`
- `plotFlush`
- `plotSelectWindow`
- `plotSetDrawAttr`
- `plotSetScale`
- `plotWindow`
- `printSystemInfo`
- `repeatTile`

- `replay`
- `rPin`
- `scmxConcatFunction`
- `scmxLoad`
- `scmxPreprocess`
- `setCaseSensitive`
- `setConfigurationPrinting`
- `setDisplayTrackingMsg`
- `setFieldDefault`
- `setIconificationPrinting`
- `ssClearAllSSViewports`
- `ssSetViewportMode`
- `strToRegExp`
- `system`
- `systemExec`
- `tcl`
- `tclLoad`
- `undefineBindKey`
- `userFormCancel`
- `userGetTclVar`

->?

Lists the valid field names for a cell or library. This function is used for program development.

Syntax

->? *id*

where *id* is the database object ID of any cell or library in the current directory from which you want a list of valid field names.

Example

```
define lib1 (dbOpenLib "ex4")
db:1
.
.
.
->? lib1
"attachedFile, catalog, libHeader, object, eqClass,
property, cellRecordNumber"
```

In this example, the function ->? lists the fields of the library *lib1*.

Note:

The [dbFetchObject](#) function is more versatile than ->?. Unlike ->?, `dbFetchObject` is not limited to cells and libraries; it lists the field names as well as the values for any database object.

See Also

[dbFetchObject](#)

->type

Identifies whether a database ID is a library ID or not.

Syntax

`->type objectId`

where *objectId* is a database object ID of any object in the current directory from which you want ID information.

If *objectId* is a library ID, `->type` returns "*DB library Id*". If *objectId* is not a library ID, `->type` returns the empty list, `()`.

Example

```
define lib1 (dbOpenLib "design")
db:1
define cell1 (dbOpenCell "top" "w")
db:2
.
.
.
->type lib1
"DB library Id"
->type cell1
()
```

In this example, `->type` confirms that the *lib1* variable is a library ID and that *cell1* is not.

augLibCompiler

Compiles a reference library description (in .lib format) to Synopsys' CLF format, and translates the .lib file into three discrete files with the following postfixes:

- .logic
- .power
- .time

For example, if you enter an input file name of `cb35os141d_typ.lib` in the *Synopsys LIB filename* field, and an output file name of `cb35.clf` in the *Output CLF filename* field, three CLF are generated after the compilation and are named:

- `cb35.clf.logic`
- `cb35.clf.power`
- `cb35.clf.time`

Milkyway determines the minimum and maximum scaled delays using the following:

- The k-factors you specify in a `.lib` file.
- The temperature, voltage, and process factor values (minimum, maximum, and nominal) you specify on the Synopsys To CLF Translator dialog box.

The Synopsys tool calculates the minimum and maximum scaled delays using the formula

$$D_{MinScaled} = D \times (1 + (T_{min} - T_{nom}) K_T) \times (1 + (V_{max} - V_{nom}) K_V) \times (1 + (P_{min} - P_{nom}) K_P)$$
$$D_{MaxScaled} = D \times (1 + (T_{max} - T_{nom}) K_T) \times (1 + (V_{min} - V_{nom}) K_V) \times (1 + (P_{max} - P_{nom}) K_P)$$

where the variables are as follows:

$D_{MinScaled}$	The computed minimum scaled delay for the operating condition under which you want the chip to operate.
$D_{MaxScaled}$	The computed maximum scaled delay for the operating condition under which you want the chip to operate.

T_{min}	The minimum temperature for the operating condition under which you want the chip to operate (specified by the Min Temperature option on the Synopsys To CLF Translator dialog box).
T_{max}	The maximum temperature for the operating condition under which you want the chip to operate (specified by the Max Temperature option on the Synopsys To CLF Translator dialog box).
T_{nom}	The nominal temperature for the operating condition under which you want the chip to operate (specified by the Nom Temperature option on the Synopsys To CLF Translator dialog box).
K_T	The k-factor for temperature (specified by the k_temp value in the k-factor file).
V_{min}	The minimum voltage for the operating condition under which you want the chip to operate (specified by the Min Voltage option on the Synopsys To CLF Translator dialog box).
V_{max}	The maximum voltage for the operating condition under which you want the chip to operate (specified by the Max Voltage option on the Synopsys To CLF Translator dialog box).
V_{nom}	The nominal voltage for the operating condition under which you want the chip to operate (specified by the Nom Voltage option on the Synopsys To CLF Translator dialog box).
K_V	The k-factor for voltage (specified by the k_volt value in the K-Factor file).
P_{min}	The minimum process factor for the operating condition under which you want the chip to operate (specified by the Max Process Factor option on the Synopsys To CLF Translator dialog box).
P_{max}	The maximum process factor for the operating condition under which you want the chip to operate (specified by the Max Process Factor option on the Synopsys To CLF Translator dialog box).

P_{nom}	The nominal process factor for the operating condition under which you want the chip to operate (specified by the Nom Process Factor option on the Synopsys To CLF Translator dialog box).
K_P	The k-factor for process (specified by the k_process value in the k-factor file).

For example, the tool calculates the minimum scaled delay for `intrinsic_rise` using the formula:

$$\text{MinScaledDelay}_{\text{intrinsic_rise}} = \text{Delay}_{\text{intrinsic_rise}} \times (1 + (T_{\text{min}} - T_{\text{nom}})k_{\text{temp_intrinsic_rise}}) \times (1 + (V_{\text{max}} - V_{\text{nom}})k_{\text{volt_intrinsic_rise}}) \times (1 + (P_{\text{min}} - P_{\text{nom}})k_{\text{process_intrinsic_rise}})$$

Tools and Menu Commands

Milkyway: CLF > .lib To CLF

Prerequisites

After you translate the Synopsys .lib file, load the CLF/port information file using [auLoadCLF](#). In the generated CLF file, you will see a new CLF command called "defineLibAttributes." This information is used by the timer and [auLoadCLF](#). When loading the CLF, it checks this information and perform the unit conversion automatically.

Synopsys LIB To CLF Compiler Dialog Box

The menu command opens the following dialog box. Its options are described below.

Synopsys LIB To CLF Compiler

OK Cancel Default Help

INPUT AND OUTPUTS

Synopsys LIB filename: (↓)

Output CLF filename: (↓)

Cell Rename Map filename: (↓)

COMPILER OPTIONS

Temperature: (min: ↑ ↓ nom: ↑ ↓ max: ↑ ↓).

Voltage: (min: ↑ ↓ nom: ↑ ↓ max: ↑ ↓).

Process factor: (min: ↑ ↓ nom: ↑ ↓ max: ↑ ↓).

Synopsys LIB filename

Type the name of the Synopsys reference library you want to compile.

Output CLF filename

Type the name you want to assign to the new CLF file.

Note:

This command does not support Synopsys files generated with Synopsys version 3.0a.

Cell Rename Map filename:

When you want to change a cell name automatically with the “HDL To GDSII Map” file used in [auVerilogIn](#), type the file name that you want Milkyway to use to map cell names during the translation, or leave it blank to use the HDL cell names.

Compiler Options

The following options allow you to adjust the operating condition values under which you want the chip to operate.

Temperature: min

Type the minimum temperature for the operating condition under which you want the chip to operate. You can also use the “up” or “down” arrow buttons to increase or decrease the value.

Nom Temperature: nom

Type the nominal temperature for the operating condition under which you want the chip to operate. You can also use the “up” or “down” arrow buttons to increase or decrease the value.

Temperature: max

Type the maximum temperature for the operating condition under which you want the chip to operate. You can also use the “up” or “down” arrow buttons to increase or decrease the value.

Voltage: min

Type the minimum voltage for the operating condition under which you want the chip to operate. You can also use the “up” or “down” arrow buttons to increase or decrease the value.

Voltage: nom

Type the nominal voltage for the operating condition under which you want the chip to operate. You can also use the “up” or “down” arrow buttons to increase or decrease the value.

Voltage: max

Type the maximum voltage for the operating condition under which you want the chip to operate. You can also use the “up” or “down” arrow buttons to increase or decrease the value.

Process Factor: min

Type the minimum process factor for the operating condition under which you want the chip to operate. You can also use the “up” or “down” arrow buttons to increase or decrease the value.

Process Factor: nom

Type the nominal process factor for the operating condition under which you want the chip to operate. You can also use the “up” or “down” arrow buttons to increase or decrease the value.

Process Factor: max

Type the maximum process factor for the operating condition under which you want the chip to operate. You can also use the “up” or “down” arrow buttons to increase or decrease the value.

batchMode

Sets batch mode for the Synopsys tool. When running in batch mode, the software does not display dialog boxes and displays windows as icons.

Tool

Milkyway

Syntax

`batchMode on?`

where the argument is as follows:

on?

Indicates whether you want to run in batch mode
Valid values:

- *#t* indicates that you want to run in batch mode.
- *#f* indicates that you do not want to run in batch mode.

bell

Rings the system bell.

Syntax

```
bell [int numberOfRings]
```

numberOfRings

Indicates the number of times to ring the bell (optional).

Description

The `bell` function rings the system bell. By setting the `numberOfRings` parameter, you can specify the number of times to ring the bell. The default is one ring.

Example

```
bell 10
```

boolNotPolygons

Generates polygons that exist in the first specified layer but not in the second specified layer.

Syntax

```
boolNotPolygons <string cellName> <string layer1Name>  
                <double overSize1> <string layer2Name>  
                <double overSize2> <string outLayerName>
```

cellName

Name of a cell containing the layers to compare.

layer1Name

Name of the first input layer.

overSize1

Amount to extend (oversize) each side of the polygons that exist on the first layer when comparing layers. If the value is negative, overSize1 is the amount to reduce (undersize) each side of the polygons.

layer2Name

Name of the second input layer.

overSize2

Amount to extend (oversize) each side of the polygons that exist on the second layer when comparing layers. If the value is negative, overSize2 is the amount to reduce (undersize) each side of the polygons.

outLayerName

Name of the layer on which to generate the polygons. This can be the same layer as one of the input layers.

Description

The boolNotPolygons function generates polygons that exist in the first specified layer but not in the second specified layer. The polygons on the two input layers are compared, and then the polygons are generated as rectangles on the specified output layer.

For the purposes of comparison, the polygons on the input layers can be oversized or undersized by a specified amount. However the polygons are not really modified in the cell.

An oversize value of 0 (zero) compares the polygon as is. A positive oversize value extends all sides of the polygon. A negative oversize value reduces all sides of the polygon. A polygon disappears, if you reduced it by a value greater than half its narrowest width.

If the cell is not already opened, the generated polygons are saved to the cell automatically. If the cell is already opened, the cell remains open and you can manually save the cell. In order to see the generated polygons, you might need to redraw the cell GUI window.

Example

```
boolNotPolygons "design" "metal1" 0 "metal1Pin" 0 "metal1Pin"
```

The above function call compares the polygons on the “metal1” layer against the “metal1Pin” layer. Polygons that exist on the “metal1” layer, but not in the “metal1Pin” layer, are created on the “metal1Pin” layer.

clearAll

Unhighlights all highlighted objects in the specified widget window.

Syntax

```
clearAll <int widgetId>
```

`widgetId`

Widget ID of the window

Description

The `clearAll` function unhighlights all highlighted objects in the specified widget window.

See also `hilite` and `hiliteNet`.

Example

```
clearAll 10
```

closeImageCell

Divides the image cell into global routing cells, which are used in congestion analysis during placement. Also displays the following information about the image cell:

- Global routing cell dimensions and placement.
- Number of cell rows.
- Number of tile patterns.
- Number of wire tracks.
- Wire direction for each layer.

Tool

Milkyway

Prerequisites

Create the image cell using `cluBreakDownHierarchy`.

Define tile patterns in the image cell using [createTilePatternRecord](#).

Define cell rows used for each tile pattern in the image cell using [createRowRecord](#).

Define the wire tracks in the image cell using [createTrackRecord](#).

Set the preferred direction for each routing layer in the image cell using [createWireDirRecord](#).

Syntax

```
closeImageCell
```

CONSTANT

Returns the specified database constant value.

Syntax

```
CONSTANT <string name>
```

name

Name of the constant.

Description

The CONSTANT function returns the specified database constant value. If the specified constant name is an empty string (""), a list of all available constants and their values are displayed, and the function returns true. If the specified constant name does not exist, the function returns false.

Example

```
CONSTANT "dbcCellLEQClass"
```

The above function call returns:

cPin

Creates a TDF pin constraint with the specified width and depth.

Syntax

```
cPin <string pinName> <string layer> <double width>  
<double depth>
```

```
cPin <string pinName> <int layer> <double width> <double  
depth>
```

pinName

Name of the pin

layer

Number or name of the pin layer

width

Width of the pin

depth

Depth of the pin

Description

The cPin function creates a TDF pin constraint with the specified width and depth. This is a TDF scheme function and must be used within a TDF file.

Example

```
cPin "VDD 1" 5 10 10
```

cPinF

Creates a TDF pin constraint with the specified bounding box.

Syntax

```
cPinF <string pinName> <string layer> <list pointLL>  
<list pointUR>  
cPinF <string pinName> <int layer> <list pointLL> <list  
pointUR>
```

pinName

Name of the pin

layer

Number or name of the pin layer

pointLL

Lower-left point of the bounding box

pointUR

Upper-right point of the bounding box

Description

The cPinF function creates a TDF pin constraint with the specified bounding box. This is a TDF scheme function and must be used within a TDF file.

Example

```
cPinF "VDD 1" 5 '(100 100) '(110 110)
```

createImageCell

Creates an image cell, which defines the base array where Astro can place cells and route nets in a gate-array design.

Tool

Milkyway

Syntax

```
createImageCell libName imageCellName direction
```

where the arguments are as follows:

<i>libName</i>	Name of the library in which you want to create the image cell. Valid values: Name of any library in the current directory. You must have write permission to the library.
<i>imageCellName</i>	Name of the image cell.
<i>direction</i>	Direction of the image cell. Valid values: • X indicates the image cell contains a row structure and, therefore, cell rows are placed horizontally. • Y indicates the image cell contains a column structure and, therefore, cell rows are placed vertically.

Example

```
createImageCell "toplib" "ImageCell" "X"
```

This example creates an image cell called *ImageCell* in the library *toplib*. The row direction of the image cell is horizontal.

See Also

[createTilePatternRecord](#) defines tile patterns in the image cell.

[createRowRecord](#) defines cell rows used for each tile pattern in the image cell.

[createTrackRecord](#) defines the wire tracks in the image cell.

[createWireDirRecord](#) sets the preferred direction for each routing layer in the image cell.

[closeImageCell](#) verifies the image cells and ends the image cell creation.

createRowRecord

Defines cell rows in the tile array of the image cell.

Tool

Milkyway

Prerequisites

Define the tile pattern using [createTilePatternRecord](#).

Syntax

```
createRowRecord "patternName" width height '(lowerLeft)
channelSpace nRows
```

where the arguments are as follows:

<i>patternName</i>	Name of the tile pattern. Valid values: Tile pattern defined using createTilePatternRecord .
--------------------	--

<i>width</i>	Width of the cell row. If you chose column, or Y, orientation in <code>cluBreakDownHierarchy</code> , then width is defined in the Y direction.
<i>height</i>	Height of the cell row.
<i>lowerLeft</i>	Coordinate of the lower left corner of the bottom cell row if the image cell direction is X or the leftmost cell row if the image cell direction is Y.
<i>channelSpace</i>	Amount of channel space. Valid values: User unit.
<i>nRows</i>	Number of cell rows. Valid values: The number of rows must be able to fit in the cell placement array area.

Example

```
createRowRecord "NP" 1344 67.2 '(892.8 876) 0 20
```

This example creates 20 cell rows with a width of 1344, and height of 67.2, based on the tile pattern *NP*. There is no channel spacing between the cell rows.

createTilePatternRecord

Defines a tile pattern in an image cell. The tile pattern defines the tiles and their orientation used in the base array.

Tool

Milkyway

Prerequisites

Create the image cell using `cluBreakDownHierarchy`.

Syntax

```
createTilePatternRecord "patternName" (list (list "Tile"  
  ' ("Orientation")) (list "Tile" ' ("Orientation")))
```

Note:

You can specify multiple tiles by repeating the following syntax as many times as necessary:

```
(list "Tile" ' ("Orientation"))
```

where the arguments are as follows:

<i>Tile</i>	Name of the tile.
	Valid values:
	Tile defined in the <code>tileTable</code> section of the technology file.

Orientation Allowable orientation of a cell placed on the tile
Valid values:

- R0 indicates that a cell can be placed without transformation ($\emptyset$).
- R90 indicates that a cell can be placed rotated 90 degrees ($\perp$).
- R180 indicates that a cell can be placed rotated 180 degrees ($\downarrow$).
- R270 indicates that a cell can be placed rotated 270 degrees ($\dashv$).
- R0_MX indicates that a cell can be placed mirrored on the X axis ($\neg$).
- R90_MX indicates that a cell can be placed rotated 90 degrees and mirrored on the X axis ($\perp$).
- R270_MX indicates that a cell can be placed rotated 270 degrees and mirrored on the X axis ($\dashv$).
- R0_MY indicates that a cell can be placed mirrored on the Y axis (E).

patternName Name of the tile pattern.

Example

```
createTilePatternRecord "NP" (list (list "NPTRAN" '("R0"))
  (list "NPTRAN" '("R0_MY")))
```

This example creates a tile pattern called *NP*, which consists of the tile *NPTRAN* on which cells are placed without transformation and the same tile (*NPTRAN*) on which cells are placed mirrored on the Y axis.

createTrackRecord

Defines the routing tracks for each layer in the image cell. Add this command to the file defining the image cell.

Tool

Milkyway

Prerequisites

Create the image cell using `cluBreakDownHierarchy`.

Syntax

```
createTrackRecord layerNumber "dir" start step nTracks  
lowerLeft  
    upperRight
```

where the arguments are as follows:

<i>layerNumber</i>	Number of the layer on which to define the wire tracks. Valid values: Routing layer.
<i>dir</i>	Direction of the wire tracks. Valid values: X for horizontal or Y for vertical.
<i>start</i>	Lower x coordinate if the direction is X or lower y coordinate if the direction is Y.
<i>step</i>	Spacing between wire tracks.
<i>nTracks</i>	Number of wire tracks.
<i>lowerLeft</i>	Lower left coordinate of the area to place the wire tracks.
<i>upperRight</i>	Upper right coordinate of the area to place the wire tracks.

Example

```
createTrackRecord 5 "Y" 855 5.6 234 '(834 855) '(2290 2165)
```

This example creates 234 wire tracks on layer 5 in the vertical direction in the area enclosing the coordinates 834, 855 to 2290, 2165.

createWireDirRecord

The *createWireDirRecord* command sets the preferred direction for each routing layer in the image cell.

Tool

Milkyway

Prerequisites

Create the image cell using *cluBreakDownHierarchy*.

Syntax

```
createWireDirRecord layerNumber "dir"
```

where the arguments are as follows:

<i>layerNumber</i>	Number of the layer on which to define the wire tracks. Valid values: Routing layer.
<i>dir</i>	Direction of the wire tracks. Valid values: X for horizontal or Y for vertical.

cycle

Iterates the objects located under the last mouse click point in the cell GUI window.

Syntax

```
cycle
```

Description

The cycle function iterates the objects located under the last mouse click point in the cell GUI window. This function only works if the current mouse command supports this feature (for example, geQueryObject, geSelectPoint, and geQueryNet). This function is identical to using the “F1” key to iterate objects.

Example

```
cycle
```

debug

Starts the Scheme Debugger.

Syntax

```
debug [string fileName]
```

fileName

Scheme source file name (optional)

Description

The debug function starts the Scheme Debugger. The Scheme Debugger is a dynamic extension of the Synopsys scheme language. It provides a debugging environment and a graphic interface in which you can debug scheme programs at the source level.

You can use it to interactively replay a command file in a controlled manner, with the capability of experimenting with different command options along the way.

The specified scheme source file is displayed in the source window and set as the default debugging program in the debug session until explicitly changed by the "load" or "run" commands later in the debugging process.

You can interact with the debugger in the command window. All the debug information is displayed in the command window. However the output from the program itself, if any, is displayed to its normal location. This is usually the main application's scheme command window from where the debugger was invoked.

The User Interface

The debugger has its own graphical user interface (GUI), which consists of the following components:

Source Window

The source window displays the current source code and shows the break points and execution stop point of the program, which are automatically updated as debugging proceeds. You can use the mouse in the source window for various selections:

- To select a line to set and clear breakpoints
- To select a variable name or an expression to print

Command Window

The command window allows you to interact with the debugger. You have full control of the debugger when typing commands in the command window. The debugger displays its response in the command window and updates the source window when needed.

Command Buttons

A row of command buttons are located between the source and command window. These buttons provide short cuts to some of the commonly used debugging commands.

Menu Bar

Two menus at the top of the interface provide access to other debugger functions.

Status Bars

There are two status bars. One is at the top of the source window, which shows the current filename. Another is at the bottom of the command window, which displays information about the current mouse position.

When the debugger is running, the main application window is still active and all the scheme commands are still accessible from there. The debugger starts a new scheme debugging environment without turning off the original scheme environment.

It is important to understand that these two environments share the same internal scheme evaluator and external data on your hard drives. This is a convenient feature that can greatly enhance and complement your debugging process if used properly.

Setting Breakpoints

You can set breakpoints using the "break" command or the "break" button. Breakpoints are set in the currently displayed source file, and marked in yellow. If a program is composed in more than one source files, you can use the "file" command to display a file and then set/clear breakpoints without changing the default main program.

You can also clear breakpoints by clicking the "clear" and "clearall" command/button. The "clear" command/button clears a single breakpoint in the currently displayed file.

The "clearall" command/button clears all the breakpoints in all the files previously used. It is recommended that you perform a "clearall" before starting to debug a new program.

Debugging a Program

Use the "run" command to start a program in the debugger. You can provide a filename as its argument, in which case, the program is executed and also set as the default main program.

The execution stops if an error occurs or a breakpoint is encountered. When a program stops for any reason, the stop position is marked in the source window in orange. If there is an error, an error message is displayed in the command window.

If the program is stopped at a breakpoint, you can continue the execution in three ways, "step", "next" or "cont", depending on how you want the program to be stopped again.

- The "step" command/button stops the program at the next scheme expression.
- The "next" command/button stops the program at the next line of code.
- The "cont" command/button stops the program at next breakpoint, if any.

At any stop, you can examine the status of the program, set/clear breakpoints, check the values of variables (using the "print" command), or even modify variable values (using the "assign" command) to change the execution direction of the program.

Tip: If you are running an scheme command file with GUI forms (dialog boxes), you have the option to manually override the form settings before stepping through the "formOK" or "formApply" command.

Quitting the Debugger

The "exit" and "quit" commands terminate the debug process, close the debug windows, and return you to the top scheme command window.

You can also exit the debugger from the "Program > Exit" menu.

Note:

Quitting the debugger does not automatically close any libraries and cells that were opened in the debug session.

Debugger Commands

Debugger commands fall into one of the following four functionality groups:

- File loading commands
- Execution commands
- Breakpoint commands
- Other commands

File Loading Commands

file <string filename>

This loads the file specified by "filename" into source window, and lets you examine and set breakpoints. If the file has previously been accessed and has set breakpoints, the old breakpoints are marked. Note that if the file has been modified, the old breakpoints might be set on the wrong lines.

GUI equivalence:

Button: None

Menu: Click on "Program > File". This opens a file selection window, which allows you to type in a file. The "Set as main program" button should not be checked.

load [string filename]

The "load" command is similar to the "file" command, except that the debugger re-sets the default execution program to "filename". Also the "filename" argument is optional, if the argument is missing, the current default program is loaded into the source window.

GUI equivalence:

Button: None

Menu: Click on "Program > File". This opens a file selection window, which allows you to type in a file. The "Set as main program" button should be checked.

Execution Commands

run [string filename]

The "run" command begins execution of the program.

If the "filename" argument is given, the debugger loads the file into the source window, sets it as the default program, and then execute the program. If no argument is given, the debugger loads and executes the current default program (previously set by the "run filename", "load filename", or "debug filename" command).

GUI equivalence:

Button: The [run] button executes the current default program.

Menu: None

step [int n]

The "step" command executes the next "n" scheme expressions starting from the current expression.

Any breakpoints set in the next "n" expressions, is ignored. If the argument "n" is missing, the default number is 1. If the ending expression in the evaluation path is defined in a different file, that file is loaded into the source window first.

GUI equivalence:

Button: The [step] button is equivalent to "step 1".

Menu: None

next

The "next" command executes all the expressions in the current source line, and stops at the next line in the same file. However, in the event that one of the expressions leads to the execution of commands defined in another file, that file is loaded into the source window, and the execution stops at the appropriate line within it.

GUI equivalence:

Button: The [next] button is equivalent to "next".

Menu: None

cont

The "cont" command continues execution from the current stop point until the next breakpoint.

GUI equivalence:

Button: The [cont] button is equivalent to "cont".

Menu: None

Breakpoints Commands

break [string filename] <int line>

The “break” command sets a breakpoint at <line> in the file specified by <filename>. If there is only one argument, it is assumed to be <line>, and the breakpoint is set in the currently displayed file.

GUI equivalence:

Button: Select the line in source window with the left button, then click on the [break] button.

Menu: Select a line in source window with the left button, then select “Breakpoint > Break”.

`clear [string filename] <int line>`

The “clear” command clears the breakpoint at <line> in the file specified by <filename>. If there is only one argument, it is assumed to be <line>, and the breakpoint at <line> in the currently displayed file is cleared.

GUI equivalence:

Button: Select a line in source window with the left button, then click on the [clear] button.

Menu: Select a line in source window with the left button, then select “Breakpoint > Clear”.

`clearall`

The “clearall” command clears all breakpoints in all files previously accessed.

GUI equivalence:

Button: None

Menu: Click on “Breakpoint > clearall”.

`status`

The “status” command prints the file name of the main program, running status, and breakpoints set in all files.

GUI equivalence:

Button: The [status] button is equivalent to “status”.

Menu: Click on “Breakpoint > status”.

Other Commands

`print <string exp>`

The “print” command executes the scheme expression `<exp>` and prints out the result in the command window. It can be used to examine the values of variables at a stop or to run a general scheme expression.

GUI equivalence:

Button: Select a variable or expression in the source window (by either holding the left mouse button and dragging, or double-clicking the left mouse button over the variable) and click the [print] button.

Menu: None.

`assign <string var> <string exp>`

The “assign” command assigns the value of scheme `<exp>` to variable `<var>`. If `<var>` is an existing variable in the current environment, its value is updated, otherwise a new global variable is created.

GUI equivalence: None.

`search <string searchString>`

The “search” command does a forward-search of the text `<searchString>` in the currently displayed file, where `<searchString>` can be a regular expression. If found, the text `<searchString>` is marked in the source file .

Note that the search normally starts from the beginning of the file, unless the last search command searched and found the same <searchString>. In this case, the current search starts right after its last location.

GUI equivalence:

Button: None.

Menu: Click on “Program > Search in File”. This opens the Debugger Search window, from where you can type in a string, select "Down" for forward-search, and then click the OK button to search the string.

`bsearch <string searchString>`

The “bsearch” command is the same as the “search” command, except that it does a backward search of the <searchString>.

GUI equivalence:

Button: None.

Menu: Click on “Program > Search in File”. This opens the Debugger Search window, from where you can type in a string, select "Up" for backward-search, and then click the OK button to search the string.

`cd <string dir>`

The “cd” command changes the current working directory to <dir>.

GUI equivalence: None.

`pwd`

The “pwd” command prints the current working directory.

GUI equivalence: None.

quit

The “quit” command exits the debug session. This closes the debug window, and returns focus to the top scheme command window. Note that this does not automatically close any libraries and cells opened in the debug session.

GUI equivalence:

Button: None.

Menu: Click on “Program > exit”.

exit

The “exit” command is the same as "quit".

Example

debug

demOpen

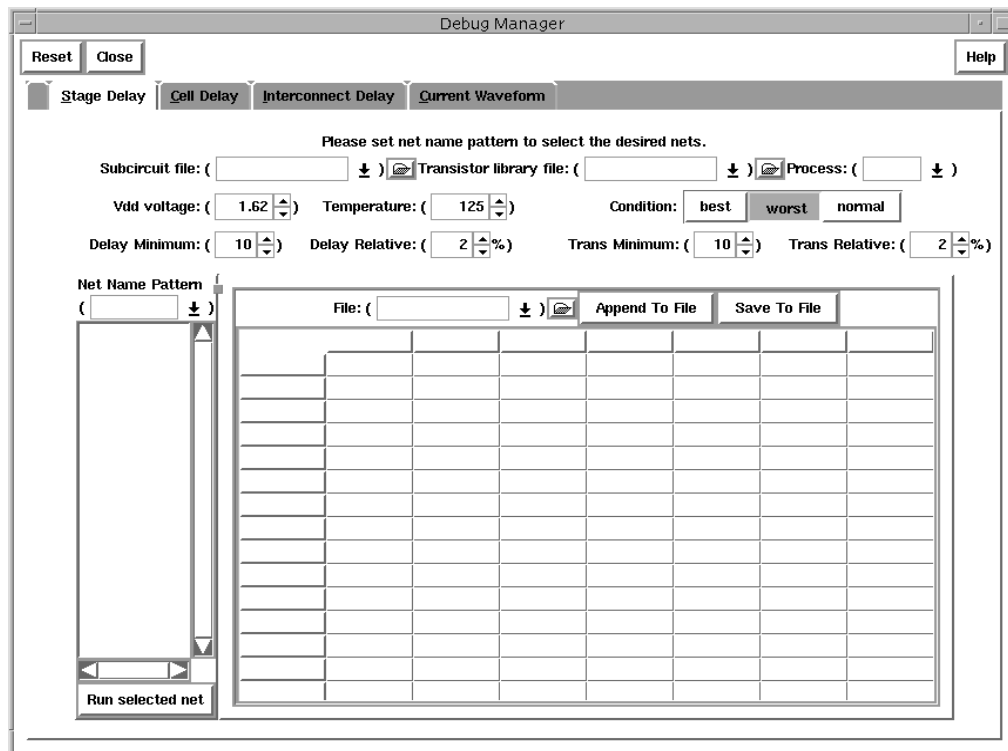
Much of demOpen is discussed with greater detail in the Astro User Guide.

Tools and Menu Commands

Astro: Verify > Debugger - Manager

Stage Delay Dialog Box

The menu command opens the following dialog box. Its options are described below.



Subcircuit file:

A subcircuit file is a text file. It contains subcircuits corresponding to the standard cells of the standard cell library. As an example, let there be two standard cells named INV and AND. The subcircuit file looks something like this:

```
.SUBCKT INV A Z
...
...
...
.ENDS
.SUBCKT AND A B Z
...
...
...
.ENDS
```

Transistor library file:

Type the name of the transistor library file. Suppose that the transistor library file is named transLib.sp. You should type transLib.sp in the box. Suppose that the transistor library file is named foo1.sp. Type foo1.sp in the box.

Process:

There is, not one, but three sets of MOSFET models. The three sets correspond to the three corner conditions:

- worst-case
- best-case
- typical-case

The vendor of the standard cell library types special symbols to distinguish between the three sets of MOSFET models. The special symbols are known as process tags.

Vdd voltage:

Type the value of VDD. Suppose that you have picked the condition to be the worst case. Type the worst-case VDD---1.62 in the example above---in the box Similarly for the best-case VDD and the typical-case VDD.

The unit of VDD is V.

Temperature:

Type the temperature. Suppose that you have picked the condition to be the worst case. Type the worst-case temperature--125 in the example above---in the box. Similarly for the best-case VDD and the typical-case VDD.

The unit of the temperature is degrees Celsius.

Condition: - best, worst, normal

Cell delay and cell transition time tables are characterized for three corner conditions:

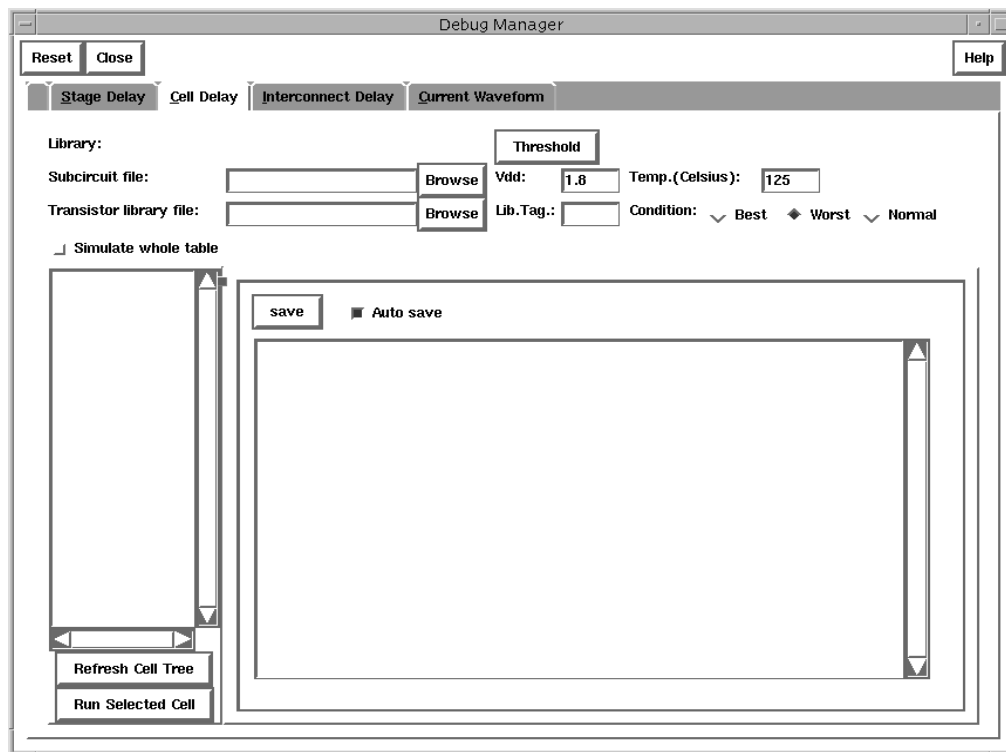
- Best-case
- Typical -case
- Worst-case

You should pick one of these corner conditions.

The dialog box has three radio buttons corresponding to the three corner conditions You should pick one of the radio buttons.

Cell Delay Dialog Box

The menu command opens the following dialog box. Its options are described below.



Subcircuit file:

A subcircuit file is a text file. It contains subcircuits corresponding to the standard cells of the standard cell library. As an example, let there be two standard cells named INV and AND. The subcircuit file looks something like this:

```
.SUBCKT INV A Z
...
...
...
.ENDS
.SUBCKT AND A B Z
...
...
...
.ENDS
```

VDD:

The cell delay and the cell transition time tables are provided by the vendor of the standard cell library. The vendor also provides a data book as part of the documentation. The data book quotes three values of VDD, for example,

1.62, 1.80, 1.98

Interpret the three values as follows:

- worst-case VDD = $\min \{1.62, 1.80, 1.98\} = 1.62$
- best-case VDD = $\max \{1.62, 1.80, 1.98\} = 1.98$
- typical-case VDD = 1.80

The dialog box has a box where you can type the value of VDD. Suppose you have picked the condition to be the worst case. Type the worst-case VDD---1.62 in the example above---in the box. Similarly for the best-case VDD and the typical-case VDD.

The unit of VDD is V.

temp.(celsius):

The cell delay and the cell transition time tables are provided by the vendor of the standard cell library. The vendor also provides a data book as part of the documentation. The data book quotes three values of temperature, for example,,

{-40, 25, 125}

You should interpret the three values as follows:

- worst-case temperature = $\max \{-40, 25, 125\} = 125$
- best-case temperature = $\min \{-40, 25, 125\} = -40$
- typical-case temperature = 25

The GUI dialog box has a box where you can type the temperature. Suppose that you have picked the condition to be the worst case. Type the worst-case temperature---125 in the example above---in the box. Similarly for the worst-case temperature and the typical-case temperature.

The unit of the temperature is degrees Celsius.

Transistor library file: Lub.Tag.:

Standard cells are made up of MOSFET circuits. Therefore, the vendor of the standard cell library must type the MOSFET models in one or more files.

Suppose that the vendor has typed the MOSFET models in a single file. Let the file be named transLib.sp. The file transLib.sp is known as the transistor library file.

Suppose that the vendor has typed the MOSFET models in multiple files. It is required that the multiple files be arranged in a hierarchy. The word “hierarchy” suggests that one of the files in the hierarchy of files is the “top file.” You must ascertain, perhaps from the vendor, which file in the hierarchy is the top file. Suppose that the top file is named foo1.sp. The file foo1.sp is the transistor library file.

Condition: Best, Worst, Normal

Cell delay and cell transition time tables are characterized for three corner conditions:

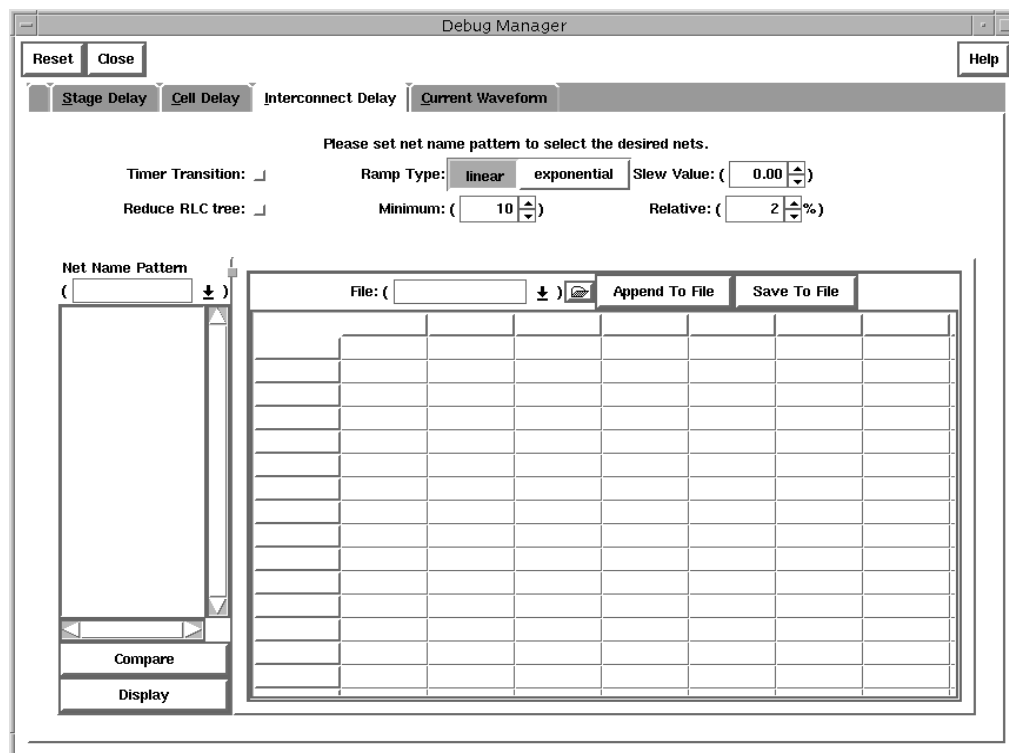
- Best-case
- Typical -case
- Worst-case

Pick one of these corner conditions.

The GUI dialog box has three radio buttons corresponding to the three corner conditions Pick one of the radio buttons.

Interconnect Delay Dialog Box

The menu command opens the following dialog box. Its options are described below.



Timer Transition:

Select to get a real transition by running the timer. Otherwise, the value shown in Option Slew Value is used as the input transition.

Ramp Type:

Select linear when you want to specify the input waveform as a linear ramp, or select exponential when you want to specify the input waveform as an exponential.

Slew Value:

Set the input transition in nano-seconds.

Reduce RLC tree:

Select when you want to use reduced RLC trees to calculate the interconnect delay.

Minimum:

Set the minimum delay in femto-seconds to be considered in error checking.

Relative:

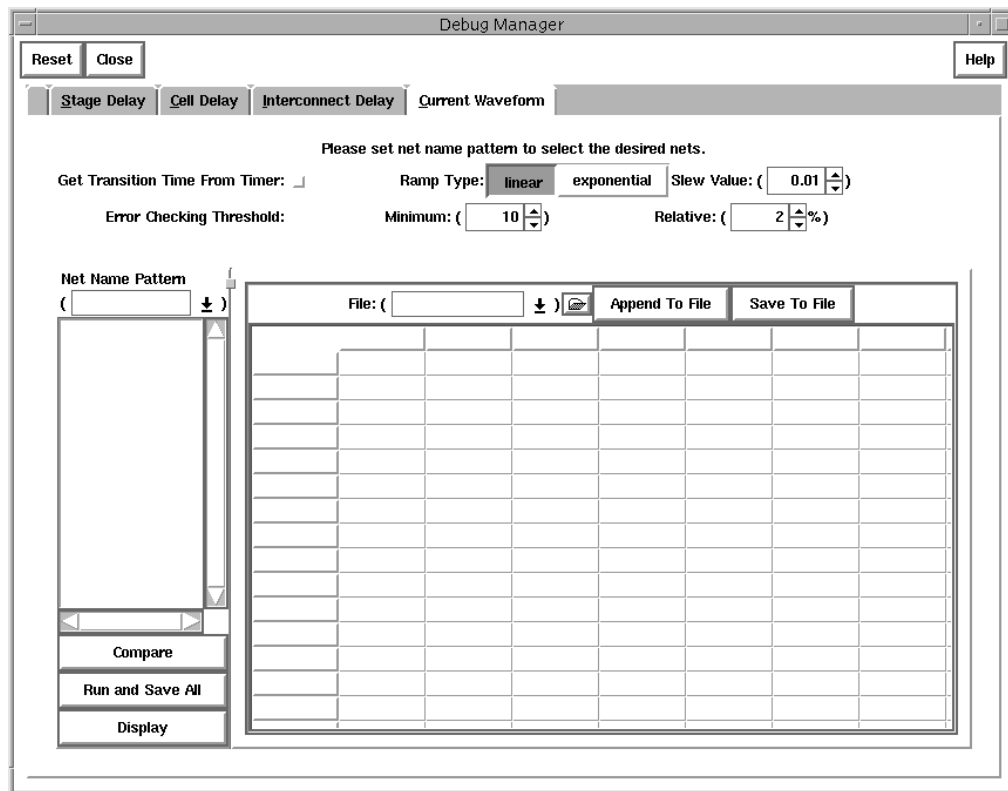
Sets the relative percentages in error checking. This option is considered together with the option Minimum in error checking. When interconnect delays are displayed, delays that are greater than the minimum delay value and when the percentage errors are greater than the relative percentages, is highlighted in red.

Net Name Pattern

Type the file name of the net for which you want to calculate the interconnect delay, or type a pattern to match the names of multiple nets.

Current Waveform Dialog Box

The menu command opens the following dialog box. Its options are described below.



Timer Transition:

Select to get a real transition by running the timer. Otherwise, the value shown in Option Slew Value is used as the input transition.

Ramp Type:

Select linear when you want to specify the input waveform as a linear ramp, or select exponential when you want to specify the input waveform as an exponential.

Slew Value:

Set the input transition in nano-seconds.

Minimum:

Set the minimum current in milli-ampere to be considered in error checking.

Relative:

Sets the relative percentages in error checking. This option is considered together with the option Minimum in error checking. When the current on the resistance edge of the interconnect RC network is displayed, the current that is greater than the minimum ampere value and when the percentage errors are greater than the relative percentages, is highlighted in red.

Net Name Pattern

Type the file name of the net for which you want to calculate the interconnect delay, or type a pattern to match the names of multiple nets.

Compare

Run the selected net and display the results in a table.

Run And Save All

Run all the nets in the design in a batch mode, the results are stored into a file named "SignalEM.RUN". This command requires a lot of disk space, and is not recommended for use.

Display

When the corner of a resistance edge is selected, for example, riseMax, this shows the wave that is calculated by AWE and Hspice into a graph.

dvSnapWindow

Takes a “snapshot” of the Astro graphics window.

Note:

The window ID and filename must be specified.

Tool

Milkyway

Syntax

`dvSnapWindow`

FileExists?

Checks to see if a file exists.

Syntax

`FileExists? <string fileName>`

`fileName`

Name of the file.

Description

The `FileExists?` function checks to see if a file exists. It returns true (#t) if the file exists, otherwise it returns an error message. You can specify the full path if needed.

Example

```
FileExists? "myFile"  
FileExists? "/home/user/myFile"
```

FileIsExecutable?

Checks to see if a file is executable.

Syntax

```
FileIsExecutable? <string fileName>
```

fileName

Name of the file.

Description

The `FileIsExecutable?` function checks to see if a file is executable. It returns true (#t) if the file is executable, otherwise it returns an error message. You can specify the full path if needed.

Example

```
FileIsExecutable? "myFile"  
FileIsExecutable? "/home/user/myFile"
```

FileIsReadable?

Checks to see if a file is readable.

Syntax

```
FileIsReadable? <string fileName>
```

fileName

Name of the file.

Description

The `FileIsReadable?` function checks to see if a file is readable. It returns true (#t) if the file is readable, otherwise it returns an error message. You can specify the full path if needed.

Example

```
FileIsReadable? "myFile"  
FileIsReadable? "/home/user/myFile"
```

FileIsWritable?

Checks to see if you can write to the file.

Syntax

```
FileIsWritable? <string fileName>
```

fileName

Name of the file.

Description

The `FileIsWritable?` function checks to see if you can write to the file. It returns true (`#t`) if the file is writable, otherwise it returns an error message. You can specify the full path if needed.

Example

```
FileIsWritable? "myFile"  
FileIsWritable? "/home/user/myFile"
```

flipChipMatrix

Defines the flip chip matrix within the TDF file.

Tool

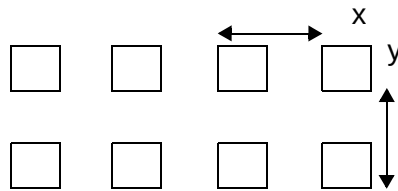
Milkyway

Syntax

```
flipChipMatrix "matrixName" xpitch ypitch
```

where the arguments are as follows:

<i>matrixName</i>	The name of the flip chip matrix. This name is also used in the flipChipPad command.
<i>xpitch/ypitch</i>	Where x and y pitch are the horizontal and vertical distances between the slots in the matrix.



See Also

[flipChipPad](#)

flipChipPad

Define the flip chip pads within the TDF file.

Tool

Milkyway

Syntax

```
flipChipPad "matrixName" "instName" rowIndex rowWeight  
yoffset  
colIndex colWeight xoffset orientation
```

where the arguments are as follows:

<i>matrixName</i>	The name of the flip chip matrix.
-------------------	-----------------------------------

<i>instName</i>	Name of a flip chip pad.
<i>rowIndex</i>	A specific row in which to place the pad. <i>rowIndex</i> and <i>rowWeight</i> are exclusive, use only one of them and enter "0" for the other.
<i>rowWeight</i>	Controls the order of cells in a row.
<i>yoffset</i>	Offsets in the Y direction from the calculated slot of the matrix. Offsets are measured from the lower left corner of matrix slots, not from the matrix origin.
<i>colIndex</i>	A specific column in which to place the pad. <i>colIndex</i> and <i>colWeight</i> are exclusive, use only one of them and enter "0" for the other.
<i>colWeight</i>	Controls the order of cells in a column.
<i>xoffset</i>	Offsets in the X direction from the calculated slot of the matrix. Offsets are measured from the lower left corner of matrix slots, not from the matrix origin.
<i>orientation</i>	Choose one of eight possible orientations for the pad, either R0, R0_MX, R0_MY, R180, R90_MX, R90, R270, or R90_MY.

[Table 63-4](#) describes the eight possible orientations for the flip chip pad.

Table 63-3 *Orientations for Flip Chip Pads*

Option	Orients the pad...
R0	Without changing its orientation.
R0_MX	Flipped on the y axis.
R0_MY	Flipped on the x axis.
R180	Rotated counterclockwise 180 degrees.
R90_MX	Rotated counterclockwise 90 degrees and flipped on the y axis.

Table 63-3 Orientations for Flip Chip Pads

Option	Orients the pad...
R90	Rotated counterclockwise 90 degrees.
R270	Rotated counterclockwise 270 degrees.
R90_MY	Rotated counterclockwise 90 degrees and flipped on the x axis.

See Also

[flipChipMatrix](#)

formDefault

Set all the fields in the dialog box back to thier default values. This command is the equivalent to clicking the "Default" button on the dialog box. The default values can be changed with the Scheme commands [setFieldDefault](#) or [setToggleFieldDefault](#).

Tool

Milkyway

Syntax

```
formDefault "formName"
```

where the argument is as follows:

formName The name of the dialog box.

Examples

```
geOpenLib  
setFieldDefault "Open Library" "read only" "1"  
setFieldDefault "Open Library" "Library Name" "stdlib"
```

Open Library

OK Cancel Default Help

Library Name: stdlib

Library Path:

☒ read only Browse...

```
setFormField "Open Library" "Library Name" "design"  
setFormField "Open Library" "read only" "0"
```

Open Library

OK Cancel Default Help

Library Name: design

Library Path:

☐ read only Browse...

```
formDefault "Open Library"
```

Open Library

OK Cancel Default Help

Library Name:

Library Path:

☐ read only Browse...

fpGetFPOption

Gets the current value of a floorplanning property.

To set the default options for place and route operations, use the [fpSetFPOption](#) command.

Syntax

```
fpGetFPOption windowNumber "optionName"
```

where the arguments are as follows:

windowNumber

Number of the window in which the cell is located

Valid values:

Any integer corresponding to an open window

optionName

Name of the property for which you want the current value

Valid values:

- Drag Flylines
- Snap To Row
- Max Pins/Net
- Row Alignment
- Use Y First
- Balance Slice

- Maintain Topography
- Delete Filler
- Route Mode
- Compact Channel
- Optimize After
- Reset Origin
- No M2 Strap overlap
- Default Core Power Net
- Default Core Ground Net
- Default Core Clock Nets
- Freeze Footprint
- Freeze Silicon

Note:

If you want the current values for all of the floorplanning options, specify “ALL” as *optionName*.

Example

The following example lists the current values for all floorplanning options:

```
fpGetFPOption 1 "ALL"
(("Default Core Clock Nets" ()))
("Default Core Ground Net" "VSS")
("Default Core Power Net" "VDD")
("Freeze Footprint" #f) ("Freeze Silicon" #f)
("No M2 Strap overlap" #f) ("Optimize After" #t)
("Compact Channel" #t) ("Delete Filler" "Routing")
("Route Mode" "3")
("Reset Origin" #t)
("Maintain Topology" #t) ("Balance Slice" #t)
("Use Y First" #f)
("Max Pins/Net" 0) ("Row Alignment" "Ground")
("Snap To Row" #f) ("Drag Flylines" #f))
```

See Also

[fpSetFPOption](#)

fpSetFPOption

Sets the default place and route options by using the Scheme language.

Note:

The `fpSetFPOption` function does not preset any of the floorplanning values; it sets the place and route parameters for the current Synopsys session.

To display the values for the current option settings, use the [fpGetFPOption](#) command.

Syntax

```
fpSetFPOption windowNumber "optionName" value
```

where the arguments are as follows:

windowNumber

Number of the window in which the object is located

Valid values:

Any integer corresponding to an open window

Note:

Zero (0) corresponds to a new window.

optionName

The options available depend on what was selected for *windowNumber*. See the following tables.

value

Value depends on the value of *optionName*. See the following tables.

The following combinations apply for all *windowNumber* values.

Table 63-4 *Combinations That Apply to All windowNumber Values*

optionName	value
Drag Flylines	Indicates whether flylines are visible. Valid values: • #t displays the flylines. • #f hides the flylines.
Snap to Row	Indicates whether the Synopsys tool automatically snaps to and adjusts rows when the cell is moved. Valid values: • #t automatically snaps to and adjusts the row when the cell is moved. • #f does not automatically snap to and adjust the row when the cell is moved.

Table 63-4 *Combinations That Apply to All windowNumber Values*

optionName	value
Max Pins/Net	Maximum number of pins you want the displayed nets to have when Drag Flylines is turned on. If a net has more pins than the value specified, the net is not displayed.
Row Alignment	Sets the default alignment options for floorplanning. Valid values: • LowerLeft aligns the bottom of each cell with the bottom of the cell row. • UpperLeft aligns the top of each cell with the top of the cell row. • Origin aligns the origin of each cell with the bottom of the cell row. • Center aligns the center of each cell with the center of the cell row. • Power aligns the top of the power pin in each cell with the top of the cell row. • Ground aligns the bottom of the ground pin in each cell with the bottom of the cell row. • CenterPG aligns the point halfway between the bottom of the power pin and the top of the ground pin in each cell with the center of the cell row.
Use Y First	Indicates whether the Synopsys tool slices vertically or horizontally first. Valid values: • #t means that the Synopsys tool slices vertically first. • #f means that the Synopsys tool slices horizontally first. This is the inverse of “Use X for first cut” in the Place and Route Options dialog box.

Table 63-4 *Combinations That Apply to All windowNumber Values*

optionName	value
Balance Slice	Indicates whether the Synopsys tool balances the slicing structure. Valid values: • #t balances the slicing structure. • #f does not balance the slicing structure.
Maintain Topology	Indicates whether the Synopsys tool maintains the current slicing structure. Valid values: • #t maintains the current slicing structure. • #f does not maintain the current slicing structure.
Delete Filler	Indicates whether the Synopsys tool deletes the filler cells prior to placement or routing. Valid values: • Placement deletes filler cells prior to placement. • Routing deletes filler cells prior to routing.
Route Mode	Valid values: • 2 indicates two-layer metal routing. • 3 indicates three-layer metal routing.
Compact Channel	Indicates whether the Synopsys tool compacts the channels. Valid values: • #t means that the channel is compacted. • #f means that the channel is not compacted.
Optimize After	Indicates whether the Synopsys tool should perform postroute optimization (which reduces wire length and eliminates unnecessary vias). Valid values: • #t means that the Synopsys tool optimizes the cell after detail routing. • #f means that the Synopsys tool does not optimize the cell after detail routing.

Table 63-4 *Combinations That Apply to All windowNumber Values*

optionName	value
Reset Origin	Indicates whether the Synopsys tool resets the origin to the lower-left corner (0,0) after detail routing of a single-region design or interregion detail routing of a multiple-region design. Valid values: • <i>#t</i> resets the origin after routing. • <i>#f</i> does not reset the origin after routing.
No M2 Strap overlap	Indicates whether the Synopsys tool can place cells beneath metal2 straps. Valid values: • <i>#t</i> means that the Synopsys tool refrains from placing any cells beneath metal2 straps. • <i>#f</i> means that metal2 straps and cells can overlap.
Freeze Footprint	Indicates whether the Synopsys tool can modify pin and pad locations of the top-level cell. Valid values: • <i>#t</i> means that the Synopsys tool attempts to maintain pin and pad locations of the top-level cell. • <i>#f</i> means that pin and pad locations are able to change.
Freeze Silicon	Indicates whether the Synopsys tool can modify pin and pad locations of the top-level cell and placement of standard and macro cells. Valid values: • <i>#t</i> means that the Synopsys tool attempts to maintain pin and pad locations of the top-level cell and placement of standard and macro cells. • <i>#f</i> means that cell placement is able to change.

The following combinations only apply for *windowNumber* !=0.

Table 63-5 *Combinations That Apply Only to windowNumber !=0*

optionName	value
Default Core Power Net	Name of the power net for which you want to generate straps during automatic floorplanning operations Valid syntax: netName "
Default Core Ground Net	Name of the ground net for which you want to generate straps during automatic floorplanning operations Valid syntax: Valid syntax: netName "
Default Core Clock Nets	Name of the clock nets for which you want to generate straps during automatic floorplanning operations Valid syntax: "} ...) netName , netName ()

Note:

When *windowNumber* is set to 0, only the values of Drag Flylines and MaxPins/Net are always copied to the new window. If an existing place and route cell is open, the values for that cell become the default for any new cell.

Example

```
fpSetFPOption 0 "Route Mode" "2"  
#t  
fpSetFPOption 0 "Delete Filler" "Placement"  
#t  
fpSetFPOption 0 "Optimize After" #t  
#t  
fpSetFPOption 0 "Reset Origin" #t  
#t  
fpSetFPOption 0 "Row Alignment" "CenterPG"  
#t
```

See Also

[fpGetFPOption](#)

genUniformTileInLib

Creates a file containing the tiles and tile patterns for each cell. Use this file to check that the tile orientations are correct. If the orientation is incorrect, edit this file and use `listFeature` to read the file in Milkyway.

Tool

Milkyway

Syntax

```
genUniformTileInLib "libName" tileOrientList  
  "direction" "fileName"
```

where the arguments are as follows:

<i>libName</i>	Name of the library containing the cells. Valid values: Name of any library in the current directory.
----------------	---

tileOrientList Orientation of the tile defined as follows:
 (list (defineTile "tileName" '("orient"))
 where the arguments are as follows:

- tileName indicates the tile name defined in the tileTable section of the technology file.
- orient indicates the allowable orientation of a cell placed on the tile.

Valid values:

- R0 indicates that a cell can be placed without transformation (F).
- R90 indicates that a cell can be placed rotated 90 degrees (ㄥ).
- R180 indicates that a cell can be placed rotated 180 degrees (ㄣ).
- R270 indicates that a cell can be placed rotated 270 degrees (ㄏ).
- R0_MX indicates that a cell can be placed mirrored on the X axis (ㄑ).
- R90_MX indicates that a cell can be placed rotated 90 degrees and mirrored on the X axis (ㄥ).
- R270_MX indicates that a cell can be placed rotated 270 degrees and mirrored on the X axis (ㄏ).
- R0_MY indicates that a cell can be placed mirrored on the Y axis (ㄵ).

direction Direction of the tile.
 Valid values:

- X indicates the tile is horizontal.
- Y indicates the tile is vertical.

fileName Name of the output file.

Example

```
genUniformTileInLib "design1" (list (defineTile "tileA"
  '("R90")) "Y" "tiles1.out"
```

This example creates a file called tiles1.out based on the cells in the library design1. This file shows the defineTilePattern for the tile tileA.

getCaseSensitive

Returns the case-sensitivity mode of your Synopsys session.

- `#t` indicates that the Synopsys tool is running in case-sensitive mode.
- `#f` indicates that the Synopsys tool is running in case-insensitive mode.

To change the mode, use [setCaseSensitive](#).

Tool

Milkyway

Syntax

```
getCaseSensitive
```

See Also

[setCaseSensitive](#) changes the case-sensitivity mode of your Synopsys session.

getDisplayTrackingMsg

Shows you the netlist change tracking message display mode. It returns either #t (displays all netlist change tracking messages on the command window) or #f (displays all netlist change tracking messages on the log file).

Tool

Milkyway

Syntax

```
getDisplayTrackingMsg
```

getEditWindow

Returns the ID of the current active widget.

Tool

Milkyway

Syntax

```
getEditWindow
```

getFormField

Retrieves specified content of a dialog box. This content is usually set using the command setFormField.

Note:

This function works only when the specified dialog box is open.

Tool

Milkyway

Syntax

```
getFormField form_name field_name
```

where the arguments are as follows:

form_name Name of the dialog box that you want to retrieve content.

form_field Name of the field that you want to retrieve content.

Example

In the following example, the library and cell have been opened and you want to retrieve dialog box content for the geAddWire function.

```
geAddWire  
getFormField "Add Wire" "Input Mode"
```

The return for this example should be either “single point” or “L90” depending on the “Add Wire” default value.

getFormFieldValue

Returns the current dialog box field value. This content is usually set using the command setFormField.

Note:

This function works only when the specified dialog box is open.

Tool

Milkyway

Syntax

```
getFormFieldValue form_name field_name
```

where the arguments are as follows:

form_name Name of the dialog box that you want to retrieve content.

field_name Name of the field that you want to retrieve content.

Example

```
geOpenLib  
getFormField "Add Wire" "Input Mode"
```

go

Sets the center of the cell display to be at the coordinates (x, y) as specified. There must be an opened cell for this command to work.

Tool

Milkyway

Syntax

```
go "x-coordinate" "y-coordinate"
```

hiliteNet

Highlights a net. This command is useful for cross-probing through the interprocess communication (IPC) protocol.

Syntax

```
hiliteNet "netName"
```

where the arguments are as follows:

netName

Name of any net in the current window that you want to highlight

hvSetDebugNetMapping

This function sets whether or not net mapping information is needed when running hvPostVerification.

Tool

Milkyway

Syntax

```
hvSetDebugNetMapping Boolean
```

where the argument is as follows:

- | | |
|----------------|--|
| <i>Boolean</i> | <ul style="list-style-type: none">• #t turn ON the net mapping.• #f turn OFF the net mapping. |
|----------------|--|

IsApplicationIn64BitMode

Checks if the current application is running in 64-bit mode.

Syntax

```
IsApplicationIn64BitMode
```

Description

The IsApplicationIn64BitMode checks if the current application is running in 64-bit mode. If it is, true is returned; otherwise false is returned.

Example

```
IsApplicationIn64BitMode
```

lib2lclf

Converts a Synopsys .LIB file into logical CLF format. If you plan to perform design optimization with Saturn, use this command to prepare your logic library. After preparing the logic library with this command, load the CLF file with [auLoadCLF](#).

Tool

Milkyway

Syntax

```
lib2lclf "libName" "clFFileName"
```

where the arguments are as follows:

<i>libName</i>	Name of the Synopsys library (.LIB format file) you want to convert to CLF format. Type the name of the library with the .lib filename extension.
<i>clFfileName</i>	The name you want assigned to the generated CLF file created by this function. Type the name of the CLF file with the .clf filename extension.

listBindKey

Displays the functions of keys on the keyboard.

Tools and Menu Commands

Astro: Query > Bind Keys

Milkyway: Options > Show Bind Keys!

See Also

[defineBindKey](#)

loadFromPath

Loads a Scheme file located in any of the directories listed in the “SCHEME_LOAD_PATH” environment variable.

Syntax

```
loadFromPath <string fileName>
```

fileName

Name of a Scheme file.

Description

The `loadFromPath` function loads a Scheme file located in any of the directories listed in the “`SCHEME_LOAD_PATH`” environment variable.

The specified file name can also include the absolute or relative path that are resolved relative to any of the directories listed in the “`SCHEME_LOAD_PATH`” environment variable. If the specified file is located in multiple directories, the file found in the first directory is used.

Example

```
loadFromPath "replay.cmd"
loadFromPath "scripts/replay.cmd"
loadFromPath "/home/user/scripts/replay.cmd"
```

loadUserForm

Loads a user-defined TCL file.

Tool

Milkyway

Syntax

```
loadUserForm tclFile
```

where the argument is as follows:

<code>tclFile</code>	Where the <code>tclFile</code> is a user provided tcl/tk file that contains user defined GUI forms.
----------------------	---

See Also

[loadUserFormFromPath](#)
[userFormCancel](#).

loadUserFormFromPath

Loads a user-defined Tcl file from the specified path.

Tool

Milkyway

Syntax

```
loadUserFormFromPath <tclFile>
```

where the argument is as follows:

<code>tclFile</code>	Where the <code>tclFile</code> is a user provided Tcl/tk file that contains user defined GUI forms. The path is defined in environment variable "MWX-TCL-LIBRARY."
----------------------	--

See Also

[loadUserForm](#)
[userFormCancel](#)

make-db

Converts an integer into a database object ID.

Syntax

```
make-db <int dbId>
```

dbId

Any positive integer

Description

The make-db function converts an integer into a database object ID. The corresponding database object ID is returned.

Example

```
make-db 5
```

net

Creates a TDF net constraint with the specified weights.

Syntax

```
net <string netName> <int horWeight> <int verWeight>
```

netName

Name of a net

horWeight

Horizontal weight

verWeight

Vertical weight

Description

The net function creates a TDF net constraint with the specified weights. This is a TDF scheme function and must be used within a TDF file.

The weight arguments must be greater or equal to 0. A value of 1 indicates no weight. All negative values default to a value of 1.

Example

```
net "VDD" 5 5
```

netCapacitance

Sets the minimum and maximum capacitance for a net, thus setting timing constraints for the net. During placement, Astro attempts to place cells connected to nets requiring faster signal speeds closer together.

Note:

Alternatively, you can use the `netDelay` function to set net timing constraints.

Tool

Milkyway

Syntax

```
netCapacitance "netName" capMin capMax
```

where the arguments are as follows:

<i>netName</i>	Name of the net to be constrained. Valid values: Name of any net in the design, including any net created using a <code>markNet</code> function.
<i>capMin</i>	Minimum capacitance for the net. Valid values: Any non-negative, floating-point number less than <code>capMax</code> .

capMax Maximum capacitance for the net.
Valid values:
Any non-negative, floating-point number greater than capMin.

Example

```
netCapacitance "LNCLK2" 300 800  
netCapacitance "CTL1" 500 1000
```

Using these net timing constraints, Astro attempts to place the design so that

- The capacitance of LNCLK2 is between 300 and 800 pF.
- The capacitance of CTL1 is between 500 and 1000 pF.

netDelay

Sets the minimum and maximum rise and fall delays for the net. During placement, Astro attempts to place cells connected to nets requiring faster signal speeds closer together.

Note:

Alternatively, you can use the netCapacitance function to set timing constraints.

Tool

Milkyway

Syntax

```
netDelay "netName" riseMin riseMax fallMin fallMax
```

where the arguments are as follows:

<i>netName</i>	Name of the net to be constrained. Valid values: Name of any net in the design, including any net created using a markNet function.
<i>riseMin</i>	Minimum rise delay on the net. Valid values: Any non-negative, floating-point number less than riseMax.
<i>riseMax</i>	Maximum rise delay on the net. Valid values: Any non-negative, floating-point number greater than riseMin.
<i>fallMin</i>	Minimum fall delay on the net. Valid values: Any non-negative, floating-point number less than fallMax.
<i>fallMax</i>	Maximum fall delay on the net. Valid values: Any non-negative, floating-point number greater than fallMin.

Example

```
netDelay "LNCLK2" .5 3 0 5
netDelay "CTL1" 1 4 2 8
```

Using these net timing constraints, Astro attempts to place the design so that

- The rise delay on net LNCLK2 is between .5 and 3 ns.
- The fall delay on net LNCLK2 is between 0 and 5 ns.
- The rise delay on net CTL1 is between 1 and 4 ns.
- The fall delay on net CTL1 is between 2 and 8 ns.

netLength

Type the length of the net during placement optimization.

Note:

This command must reside inside the TDF file. Otherwise, it fails.

Tool

Milkyway

Syntax

```
netLength "netName" minLength maxLength
```

where the arguments are as follows:

netName Name of the net to be constrained.
Valid values:
Name of any net in the design, including any net created using
a markNet function.

minLength Minimum length of the net.

maxLength Maximum length of the net.

netTransition

Sets the minimum and maximum rise and fall transition delays for the net. During placement, Astro attempts to place cells connected to nets requiring faster signal speeds closer together.

Tool

Milkyway

Syntax

```
netTransition "netName" riseMin riseMax fallMin fallMax
```

where the arguments are as follows:

<i>netName</i>	Name of the net to be constrained. Valid values: Name of any net in the design, including any net created using a markNet function.
<i>riseMin</i>	Minimum rise transition delay on the net. Valid values: Any non-negative, floating-point number less than riseMax.
<i>riseMax</i>	Maximum rise transition delay on the net. Valid values: Any non-negative, floating-point number greater than riseMin.
<i>fallMin</i>	Minimum fall transition delay on the net. Valid values: Any non-negative, floating-point number less than fallMax.
<i>fallMax</i>	Maximum fall transition delay on the net. Valid values: Any non-negative, floating-point number greater than fallMin.

Example

```
netTransition "LNCLK2" .5 3 0 5  
netTransition "CTL1" 1 4 2 8
```

Using these transition delay constraints, Astro attempts to place the design so that:

- The rise transition delay on net LNCLK2 is between 0.5 and 3 ns.
- The fall transition delay on net LNCLK2 is between 0 and 5 ns.
- The rise transition delay on net CTL1 is between 1 and 4 ns.

- The fall transition delay on net CTL1 is between 2 and 8 ns.

notDoneYet

Does nothing and returns TRUE.

Syntax

`notDoneYet`

Description

The `notDoneYet` function performs no computation and returns TRUE (#t).

Example

`notDoneYet`

obsoleteCmd

Displays an obsolete command message.

Syntax

`obsoleteCmd <string cmdName>`

`cmdName`

Name of the command

Description

The `obsoleteCmd` function displays an obsolete command message. The message has the format:

"Obsolete Command : `cmdName`"

Example

```
obsoleteCmd "myFunc"
```

This function call displays:

```
Obsolete Command : myFunc
```

pad

Creates a TDF pad constraint with the specified values.

Syntax

```
pad <string padName> <string padSide> [<int order>  
[float offset]]
```

padName

Name of the pad.

padSide

Side constraint of the pad (that is "none", "right", "left", "top", or "bottom").

order

Positive integer indicating the order constraint. A value less than or equal to zero indicates no order constraint.

offset

Distance from the left edge to the center of the pad for top and bottom pads, or the distance from the bottom edge to the center of the pad for left and right pads.

Description

The pad function creates a TDF pad constraint with the specified values. This is a TDF scheme function and must be used within a TDF file.

Example

```
pad "pad1" "right" 1 100.500
```

pin

Creates a TDF pin constraint with the specified values.

Syntax

```
pin <string pinName> <string layerName> <int pinWidth>  
<int pinDepth> <string pinSide> [<int order> [flat offset]]  
pin <string pinName> <int layerNum> <int pinWidth>  
<int pinDepth> <string pinSide> [<int order> [flat offset]]
```

pinName

Name of the pin.

layerName

Name of the pin layer.

layerNum

Number of the pin layer.

pinWidth

Width of the pin.

pinDepth

Depth of the pin.

`pinSide`

Side constraint of the pin (that is “none”, “right”, “left”, “top”, or “bottom”).

`order`

Positive integer indicating the order constraint. A value less than or equal to zero indicates no order constraint.

`offset`

Distance from the left edge to the left side of the pin for top and bottom pin, or the distance from the bottom edge to the bottom side of the pin for left and right pins.

Description

The `pin` function creates a TDF pin constraint with the specified values. This is a TDF scheme function and must be used within a TDF file.

This command is similar to the `rPin` command, but uses different values for the `sideNumber` attribute. Additional information on `rPin` attributes as well as a diagram can be found in the Milkyway Data Preparation User Guide

Example

```
pin "pin 1" 3 50 100 "right" 1 100.500
```

See Also

[rPin](#)

rPin

Creates a TDF rectangular pin constraint with the specified values.

Syntax

```
rPin pinName layer width height sideNumber [pinOrder]  
[pinOffset]
```

pinName

Name of the pin.

layer

Number of layer for pin.

width

Dimension of the pin along the cell boundary.

height

Dimension of the pin from the boundary to the center of the cell.

sideNumber

The side number is a positive integer that starts from 1. Given a rectangular shape, the lower left-most edge is the starting edge (side number 1). The side number of the next edge, going clockwise, is 2, and so on.

pinOrder

Order the pin placement relative to the other pins on the same side. Optional.

pinOffset

Relative offset of the pin placement from the bottom edge or the left edge of the cell boundary. Optional.

Description

The `rPin` command creates a TDF rectangular pin constraint with the specified values. This is a TDF scheme command and must be used within a TDF file.

This command is similar to the `pin` command, but uses different values for the `sideNumber` attribute. Additional information on `rPin` attributes as well as a diagram can be found in the *Milkyway Data Preparation User Guide*.

Example

```
rPin "p10" 6 6 2 2 0 1600
```

See Also

[pin](#)

Plot Commands

The following are `plot*` commands that provide text and drawing capabilities to Milkyway-based tools.

plotAddBoxText

Draws text in the given box on the graphic window.

Syntax

```
plotAddBoxText "string" '(point0_X point0_Y) '(point1_X  
point1_Y) "rotate" "justification"
```

Arguments

string

The text which displays on the graphic window

point0_X point0_Y

The box's starting point coordinate

point1_X point1_Y

The box's ending point coordinate

rotate

Not supported

justification

Not supported

Description

The plotAddboxText command draws text in the given box on the graphic window.

Example

```
plotAddBoxText "AVANTICORP" '(0 0) '(15 5) "270" "IC"
```

plotAddPath

Draws path on the graphic window.

Syntax

```
plotAddPath '((point0_X point0_Y) (point1_X point1_Y)  
(point2_X point2_Y).....)
```

Arguments

point0_X point0_Y

The point's coordinate

Description

The plotAddPath command draws path on the graphic window.

Example

```
plotAddPath '((0 0) (5 5) (0 5) (0 10) (10 10))
```

plotAddPolygon

Draws polygon on the graphic window.

Syntax

```
plotAddPolygon '((point0_X point0_Y) (point1_X  
point1_Y).....(point0_X point0_Y))
```

Arguments

point0_X point0_Y

The point's coordinate

Description

The plotAddPolygon command draws polygon on the graphic window.

Example

```
plotAddPolygon '((0 0) (5 10) (10 10) (15 0) (0 0))
```

plotAddRectangle

Draws rectangle on the graphic window.

Syntax

```
plotAddRectangle '(point0_X point0_Y) '(point1_X point1_Y)
```

Arguments

point0_X point0_Y

The rectangle's starting point coordinate

point1_X point1_Y

The rectangle's ending point coordinate

Description

The plotAddRectangle command draws rectangle on the graphic window.

Example

```
plotAddRectangle '(4 5) '(8 10)
```

plotAddScreenText

Draws text on the graphic window.

Syntax

```
plotAddScreenText "text" height 'origin point  
"justification"
```

Arguments

text

What displays on screen

height

Text's height

origin point

The point in which text is based on to display

justification

Not supported at this time

Description

The `plotAddScreenText` command draws text on the graphic window. Text does not move with view changes.

Example

```
plotAddScreenText "AVNT" 25 '(50 50) "IC"
```

plotAddSegment

Draws line on the graphic window.

Syntax

```
plotAddSegment '(point0_X point0_Y) '(point1_X point1_Y)
```

Arguments

point0_X point0_Y

The line's starting point coordinate

point1_X point1_Y

The line's ending point coordinate

Description

The plotAddSegment command draws line on the graphic window

Example

```
plotAddSegment '(0 0) '(10 5)
```

plotAddText

Draws text on the graphic window with given height.

Syntax

```
plotAddText "string" height '(point0_X point0_Y) "rotate"  
"justification"
```

Arguments

string

The text which displays on the graphic window

height

The text's height

point0_X point0_Y

The text starting point coordinate

rotate

Not supported at this time

justification

Not supported at this time

Description

The plotAddText command draws text on the graphic window with a given height on the graphic window.

Example

```
plotAddText "AVANTICORP" 1 '(0 0) "270" "IC"
```

plotClear

Clears the current graphic window.

Syntax

```
plotClear
```

Description

The plotClear command clears the current graphic window.

Example

```
plotClear
```

plotFlush

Updates the graphic window.

Syntax

```
plotFlush
```


Description

The `plotFlush` command updates the graphic window without redrawing after plotting.

Example

```
plotFlush
```

plotSelectWindow

Selects graphic window to plot.

Syntax

```
plotSelectWindow windowID scale
```

Arguments

`windowID`

the ID of the opened graphic window

`scale`

All plot commands use this scale instead of `lengthPrecision` in `techFile`

Description

The `plotSelectWindow` command selects from two or more graphic window to plot.

Example

```
plotSelectWindow 3 100
```

See Also

[plotSetScale](#)

plotSetDrawAttr

Sets plot attributes.

Syntax

```
plotSetDrawAttr "color" "fillPattern" "lineStyle" width
```

Arguments

color

The plot's color, including black, white, orange, lead, salmon, purple, aqua, drab brown, red, green, yellow, blue, magenta, and cyan

fillPattern

The fill's pattern, including blank, solid, and rectangle

lineStyle

The line's style, including none, solid, and dash

width

Not supported at this time

Description

The `plotSetDrawAttr` command sets the plot attributes, including color, fill pattern, linestyle, and line width.

Example

```
plotSetDrawAttr "white" "solid" "solid" 1
```

plotSetScale

Set the scale of the current graphic window.

Syntax

```
plotSetScale scale
```

Arguments

scale

All plot commands use this scale instead of lengthPrecision in techFile

Description

The `plotSetScale` command sets the the scale of the current graphic window.

Example

```
plotSetScale 100
```

See Also

[plotSelectWindow](#)

plotWindow

Closes the graphic window.

Syntax

```
plotWindow windowId option_close"
```

Arguments

windowID

The ID of the opened graphic window

option_close

Closes the graphic window. Other options not available.

Description

The `plotWindow` provides a commands to close or save the graphic window. Currently only the close option is supported.

Example

```
plotWindow (gePlotWindow) "close"
```

printSystemInfo

Prints system computer and software information in a brief report into the Message/Input area of Astro, Jupiter and Enterprise. This information comprises:

- Computer Information
 - Operating system name, release, version, source, and bits
 - Node name
 - Hardware model
- Software Information (sizes in bits)
 - Pointer
 - Long integer

- Integer
- Short integer
- Double prec.
- Floating

Tool

Milkyway

Syntax

`printSystemInfo`

Example

```
printSystemInfo
***** Computer Info *****
O.S. Name = SunOS
Node Name = av20026
O.S. Release = 5.7
O.S. Version = Generic
Hardware & Model = sun4u
O.S. Source = SUN
O.S. Bits = 64
Num of CPUs = 1
Memory Page Size = 8K bytes
Num of Memory Pages = 32768
Physical Memory = 256.00M bytes

***** Software Info *****
Pointer          Size = 32 bits
Long Integer     Size = 32 bits
Integer          Size = 32 bits
Short Integer    Size = 16 bits
Double Prec.     Size = 64 bits
Floating         Size = 32 bits
```

repeatTile

Returns a string containing the tile names and repetition number.

Syntax

```
repeatTile <int rTimes> <list tileList>
```

rTimes

Number of repetitions.

tileList

List of tile names.

Description

The `repeatTile` function returns a string containing the tile names and repetition number.

Example

```
repeatTile 5 `("unit" "core" "core2")
```

The above function call returns:

```
"( unit core core2  5 )"
```

replay

Replays a command file, which the Synopsys tool reads, executing the commands in the file. For more information about command files, see "Creating and Executing Command Files."

Tool

Milkyway

Syntax

```
replay fileName
```

where the argument is as follows:

fileName The name of the command file.

scmxConcatFunction

Concatenates a scheme function onto another scheme function.

Syntax

```
scmxConcatFunction <string mainFunc> <string addOnFunc>  
<string args>
```

mainFunc

 Name of the main function.

addOnFunc

 Name of the add-on function.

args

 Names of the function's formal arguments (parameters).

Description

The `scmxConcatFunction` concatenates a scheme function on to another scheme function. The main function is redefined to call the original main function first and then the add-on function.

Each function is called with its specified formal arguments. The redefined main function returns the add-on function's return value. The add-on function is not modified in any way.

Example

```
scmxConcatFunction "myFunc1" "myFunc2" "cellId netName"
```

The above function call redefines the myFunc1 function as:

```
(define _avnt1 myFunc1)
(define _avnt2 myFunc2)
(define (myFunc1cellId netName)
  (begin
    (_avnt1 cellId netName)
    (_avnt2 cellId netName)
  )
)
```

scmxLoad

Loads the specified Scheme file.

Syntax

```
scmxLoad <string fileName>
```

fileName

Name of the Scheme file.

Description

The scmxLoad function loads the specified Scheme file. If the file is successfully loaded, true (#t) is returned, otherwise false (#f) is returned.

The specified Scheme file string can also include the absolute or relative path, which is resolved in relation to the current working directory.

Example

```
scmxLoad "replay.cmd"  
scmxLoad "scripts/replay.cmd"  
scmxLoad "/home/user/scripts/replay.cmd"
```

scmxPreprocess

Preprocesses the specified scheme command by adding necessary parenthesis and formatting for regular spacing.

Syntax

```
scmxPreprocess <string schemeCmd>
```

schemeCmd

The Scheme command (limited to 256 characters)

Description

The `scmxPreprocess` function preprocesses the specified Scheme command by adding necessary parenthesis and formatting for regular spacing. The resulting Scheme command string is returned.

Example

```
scmxPreprocess "display \"help\" "
```

Returns:

```
"(display \"help\" )"
```

setCaseSensitive

Changes the case-sensitivity mode of your Synopsys session. You cannot open a case-insensitive library while running your Synopsys tool in case-sensitive mode, and vice versa. Therefore, you cannot change the case-sensitivity mode of a session after you open a library.

To find out the case-sensitivity mode of your Synopsys session, use [getCaseSensitive](#).

Note:

For more information about case sensitivity, see “Using Case-Sensitive Mode.”

Tool

Milkyway

Syntax

```
setCaseSensitive sensitive?
```

where the argument is as follows:

- | | |
|-------------------|---|
| <i>sensitive?</i> | Indicates whether or not you want the session to be case-sensitive. |
| • #t | changes the case-sensitivity mode to case-sensitive. |
| • #f | changes the case-sensitivity mode to case-insensitive. |

See Also

[getCaseSensitive](#) returns the case-sensitivity mode of your Synopsys session.

setConfigurationPrinting

Provides a switch to turn OFF the printing of window reconfiguration messages.

Tool

Milkyway

Syntax

```
setConfigurationPrinting Boolean
```

where the argument is as follows:

Boolean

- #t turn ON the printing of window reconfiguration messages.
- #f turn OFF the printing of window reconfiguration messages.

Example

```
setConfigurationPrinting #f
```

setDisplayTrackingMsg

This Scheme command allows you to control whether or not the netlist change tracking message is displayed in the command window. The default mode is ON, that is all the netlist change tracking messages are display on the command window. If you use the command “setDisplayTrackingMsg #f,” all the netlist change tracking messages are put into the log file only.

Tool

Milkyway

Syntax

```
setDisplayTrackingMsg #t( or #f)
```

setFieldDefault

To change the default value for a type-in field or boolean option, use the `setFieldDefault` command. In most forms, `setFieldDefault` also changes the current value of the field to the default value. If the value of the field has been changed, either click the “Default” button on the dialog box or use the Scheme command [formDefault](#) to change the fields on the dialog box back to the default values.

Tool

Milkyway

Syntax

```
setFieldDefault formName fieldName value
```

where the arguments are as follows:

<i>formName</i>	Name of the dialog box. Valid values: A dialog box with type-in fields or boolean options.
<i>fieldName</i>	Name of the type-in field or boolean option that you want to change the default setting. Valid values: Name of any type-in field or boolean option on the dialog box.

value

Value you want to set as the default.

Valid values:

A number or string for a type-in field, and one of the following for a boolean option:

- #t (or 1) turns the option ON.
- #f (or 0) turns the option OFF.

Example 1

```
setFieldDefault "geOpenCell" "Cell Name" "topj"
```

Sets the default Cell Name on the Open Library dialog box to topj.

Example 2

```
setFieldDefault "geOpenCell" "read only" #t
```

Sets the default to on for the read only option on the Open Library dialog box.

setIconificationPrinting

Provides a switch to turn OFF the printing of window iconifying messages.

Tool

Milkyway

Syntax

```
setIconificationPrinting Boolean
```

where the argument is as follows:

Boolean

- #t turn ON the printing of window iconifying messages.
- #f turn OFF the printing of window iconifying messages.

Example

```
setIconificationPrinting #f
```

ssClearAllSSViewports

Clears all viewports previously defined with the `setViewport` or `updateViewport` commands in the current layout window. This command is called automatically by the `updateViewport` command.

Tool

Milkyway

Syntax

```
ssClearAllSSViewPorts levels
```

where the argument is as follows:

levels

The number of level of physical hierarchy that is traversed when viewports are cleared.

ssSetViewportMode

Enables Astro Interactive Ultra to prepare and display shape structure objects for viewports defined with the `setViewport` or `updateViewport` commands.

Tool

Astro Interactive Ultra

Syntax

```
ssSetViewportMode 1 | 0
```

where a 0 argument disables the viewport mode and 1 enables the viewport mode.

strToRegExp

Converts a literal string to a regular expression, as required for [dbAssignVarRouteRule](#).

Tool

Milkyway

Syntax

```
strToRegExp inp
```

where the argument is as follows:

inp A literal string.

Example

The following example is a clip from a log file:

```
;; geOpenCell
;; setFormField "Open Cell" "Cell Name" "ADDFX1"
;; formOK "Open Cell"
Open Cell ADDFX1.CEL;1 OK
;; dbAssignVarRouteRule (getEditCell) (strToRegExp
"name[20]")
'()
Can't find varRouteRule [name\[20\]] in cell
()
```

Note that `strToRegExp` converts the string “name[20]” to “name\[20\]”. Because the “[]” brackets are special characters in regular expressions, the “\” characters were added before and after the brackets for escape purposes.

See Also

[dbAssignVarRouteRule](#) assigns a set of variable-width routing rules to one or more nets.

system

Passes the specified command string to the shell as input.

Syntax

```
system <string commandStr>
```

`commandStr`

A shell command string

Description

The system function passes the specified command string to the shell as input.

Unlike the systemExec function, this function waits for the shell to complete and returns the command's return code. In effect, the command is executed as if the command string was typed at the terminal.

Example

```
system "ls -l"
```

systemExec

Passes the specified command string to the shell as input.

Syntax

```
systemExec <string commandStr>
```

commandStr

A shell command string (less than 1024 characters)

Description

The systemExec function passes the specified command string to the shell as input. As soon as this function passes the command string, 0 is returned.

Unlike the system function, this function does not wait for the shell to complete. The command is executed as if you typed the command string at the terminal with the ampersand (&) designating a background command.

Example

```
systemExec "ls -l"
```

tcl

Passes the specified command string to the Tcl interpreter as input.

Syntax

```
tcl <string commandStr>
```

commandStr

A Tcl command string (less than 2048 characters)

Description

The tcl function passes the specified command string to the Tcl interpreter as input. This function displays the Tcl return value and returns true (#t).

The command string can reference Tcl variables and commands defined by a preceding tcl or tclLoad function call.

Example

```
tcl "puts stdout \"Hello World\""
```

tclLoad

Loads the specified Tcl file into the Tcl interpreter.

Syntax

```
tclLoad <string tclFileName>
```

tclFileName

Name of a Tcl file

Description

The tclLoad function loads the specified Tcl file into the Tcl interpreter. If the file is successfully loaded, this function returns true (#t). Otherwise this function returns false (#f).

The file may reference Tcl variables and commands defined by a preceding tcl or tclLoad function call.

Example

```
tclLoad "myTclFile.tcl"
```

undefineBindKey

Unassigns commands from the specified key or combination of keys.

Syntax

```
undefineBindKey <string key>
```

key

The key or combination of keys. See defineBindKey for valid key strings.

Description

The undefineBindKey functions unassigns commands from the specified key or combination of keys. The key should be previously assigned by the defineBindKey function. To see a list of currently assigned keys, use the listBindKey function.

Example

```
undefineBindKey "ctrl-p"
```

userFormCancel

Destroys (cancels) a previously loaded user-defined Tcl dialog box.

Tool

Milkyway

Syntax

```
userFormCancel <tclFile>
```

where the argument is as follows:

<code>tclFile</code>	Where the <code>tclFile</code> is a user provided Tcl or .tk file that contains user defined GUI dialog boxes.
----------------------	--

See Also

[loadUserForm](#)

[loadUserFormFromPath](#)

userGetTclVar

Fetches a specified variable's value in a Scheme-mode that was defined in the Tcl-mode. If it fails to fetch the value, the return is `#f`.

Tool

Milkyway

Syntax

```
userGetTclVar "varName" ["arrIndex"]
```

where the arguments are as follows:

<code>varName</code>	The name of the variable defined formerly in Tcl-mode. Valid values: The name of the variable whose value you want to fetch.
<code>arrIndex</code>	The name of the index, if the specified variable is bound to an array in Tcl-mode. (optional) Valid values: The name of the index in the specified array.

Index

Symbols

-> type 6

->? 6

A

access modes

changing 79

specifying

when opening cells 440

when opening libraries 442

adding

cell boundaries 10

cell instances to cell regions 31

contact arrays 15

contacts 13

instances of a master cell 11

objects to groups 41

paths 17

pins 23, 24

placement blockage area 2, 3

polygons 20

rectangles 22

reference libraries 50

text 31

wires 33

addRelativePoint 238

antenna

prerequisites for checking 51

ApolloII layouts, translating to GDSII Stream

format 197

arcs

setting characteristics of 38, 101

arrays

contact 15

arrival times, specifying 42

attached files

deleting 232

from libraries 236

storing

in cells 617

in libraries 618

auCreateBaseArray 9

auCreateHierNetlist 11

auCreateHierVlogOut 18

auGenCellBoundary 81

auGenLibCellBoundary 82

auHECOByFile 87

auHierECO 90

auSetEdifOutExternal 160

auSetMultiLEFForeignName 165

auStreamInCellBndryIgnoreText 192

auStreamOutCellInstanceNameAsText 220

auStreamOutCheckInvalidCells 222

auStreamOutChildCellFillData 223

auStreamOutControlFileToLog 225

auStreamOutCustomViewInstance 226

auStreamOutFillDataForAllCellTypes 228

- auStreamOutFillDataMapping 229
- auStreamOutNetWithPathTypeZero 232
- auStreamOutOddShapePin 233
- auStreamOutRemoveEmptyCells 235
- auStreamOutSelectedCells 238
- auStreamOutSkipGRouteContact 240
- auStreamOutSplitPath 242
- auStreamOutWillAbortIfCellIsMissing 244
- auStreamOutWithExtendedLayerMapping 245
- axg Commands 1
- B
- base array
 - defining cell rows 22
- batch mode 13
- bBox
 - setting for cell regions 537
 - updating 334
- bell 14
- bind keys
 - displaying functions for 72
 - setting 5
- blockage areas, defining 57
- blocks
 - created using ApolloII 139
- boolNotPolygons 14
- borrowed time
 - setting maximum amount on specified latches 88
- boundaries, see cell boundaries
- browsing
 - cells 215
- C
- capacitance
 - specifying
 - device driven by the chip or block 48
 - device driving the chip or block 75, 78
 - port 67, 70
- case sensitivity
 - querying 66
 - setting 103
- cell boundaries
 - adding 10
 - generating 78
- cell groups
 - displaying information about 177
- cell groups, adding objects to 41
- cell hierarchy
 - writing to a file 104
- cell instance masters, deleting 453
- cell instances
 - adding to cell region 31
 - converting names
 - hierarchical to local 115
 - creating 139
 - deleting 233, 453
 - getting object IDs 367
 - listing those associated with a fence 362
 - listing those associated with a group 363
 - replacing 493
 - cell regions 498
 - setting
 - origin 516
 - status 520
 - specifying ECO status 512
 - updating
 - for consistency 336
 - hierarchy 626
 - using to get regionId 358
- cell instances, adding to a
 - design 11
 - scan chain 36
- cell layers, setting time information
 - capacitance 556
 - inductance 556
 - resistance 556
- cell library format file, see CLF files
- cell regions
 - adding
 - cell instances to 31
 - hierarchical nets 160
 - creating 142
 - cell rows for 143

- deleting, cell instances from 233
 - getting ID 358
 - replacing with cell instances 498
 - setting
 - bBox 537
 - target percent 539
- cell rows
 - defining in base array 22
- cell rows, placement orientation
 - deleting doubleback option 234
 - setting 540, 616
- cells
 - bBox, updating 334
 - browsing 215
 - changing
 - types after translation 36
 - changing access mode 79
 - closing 108
 - copying 116, 56, 194
 - to a library other than current 117, 119
 - creating 180, 231
 - cell boundaries 128
 - cell instances 139
 - cell properties 140
 - contact records 148
 - EEQ class 134
 - empty cell 57
 - graph nodes 167
 - LEQ class 139
 - nets 167
 - new versions 39
 - parameter set records 171
 - parent nets 172
 - pin records 176
 - preroute records 182
 - pseudo path records 185
 - SG edge records 192
 - deleting
 - all empty nets 451
 - all single port nets 452
 - attached files from 232
 - by name 231
 - old versions 174
 - specified cells 71
 - specified properties 248, 249
 - dumping
 - database page numbers 291
 - to file 265, 275, 348
 - expanded netlist cells, creating 11, 17
 - exploding, see smashing
 - flattening, see smashing
 - getting, cell ID 83
 - groups
 - displaying information about 177
 - groups, see cell groups
 - highlighting 2
 - instances, see cell instances
 - layout cells
 - creating 188, 193
 - listing cell names 215
 - moving 128
 - netlist cells, creating 46, 254, 277
 - opening 440, 156
 - pin/blockage cells, creating 57
 - placement information, dumping to files 267
 - preventing movement during automatic placement 2
 - printing record content 467
 - regenerating boundaries 78
 - regions, see cell regions
 - renaming 51, 488
 - replacing
 - current cell with last saved version 188
 - with a different master 489, 500
 - replacing current cell with last saved version 509
 - reverting to last update 27
 - rows, see cell rows
 - saving 108, 510
 - with a new name 194
 - setting

- model subtype 569
 - origin 425
- smashing
 - instances of specified masters 65
 - specified instances 216, 221
- specifying
 - access modes when opening 440
 - layout subtype 557
 - type 541
- storing attached files 617
- updating
 - hierarchy of cell instances 626
 - values of properties 624
- chains
 - scan
 - creating 191
 - deleting 247, 250, 252
 - writing to a file 290
- checking
 - for objects not placed on grid 40
 - for shorts 153
 - netlists 8
- child nets, creating 144
- clearAll 16
- clearing
 - EEQ class 96
 - LEQ class 97
- CLF
 - defining gate size in 52
- CLF commands
 - defineCondMinClkPeriod 12
 - defineCondTimeTLU 38
 - defineInternalPower 54
 - defineLeakagePower 59
 - defineSwitchingPower 76
- CLF files
 - from Synopsys Format Reference Lib files 7
 - from Synopsys Format Report Lib files 247
 - loading into libraries 105, 11, 21
- CLF functions
 - defineGateSize 51
- CLF functions, definePortPhaseDelay 71
- CLF information, writing to a file 34, 20
- clfCreateTable function 3
- clfDefineDesignCellName 19
- clfPurgeBooleanFunction 24
- clfPurgePad 25
- clock
 - defining minimum cycle 14, 63
 - minimum pulse width 64
- clock cycle
 - setting 12
- clock nets
 - deleting definitions 8
 - setting source characteristics 19
- clock waveforms, deleting 9
- closing
 - cells 108
 - graphics windows 47
 - libraries 108, 45
 - query files 110
- cmCmdDumpMultipleECOs 2
- cmDumpPlib 23
- cmDumpPseudoPlib 25
- cmDumpTLUPlus command 28, 56
- cmItfToTLUTest 35
- cmShowInstOrigin 62
- cnvGetBuffering 7
- cnvSetBuffering 8
- command files
 - replaying 98
- commands
 - assigning to keys 5
 - cmDumpTLUPlus 28, 56
 - dbClearCellFootPrintEQClass 98
 - listing 9
- conditional clock cycle 12
- conditional constraints
 - arguments
 - enaCond 26
 - endCond 26

- startCond 26
- conditional delay
 - definition 15
- conditional intrinsic delay
 - setting 17
- congestion maps, displaying 2, 3, 4
- connecting, port instances to nets 111
- connections
 - displaying for
 - entire design 8
 - specified objects 11
- CONSTANT 18
- constant power model 62
- constraints
 - conditional
 - intrinsic delay 17
 - intrinsic delay 80
 - loading path delays 23, 26
 - net capacitance 76
 - net delay 77
 - net timing 79
 - on cell locations 2
 - setting pin location 589
- constraints file
 - loading 23
- contact arrays, adding 15
- contact records, creating 148
- contacts, adding 13
- converting
 - wires to pins 51, 52
- converting, regions to macro cells 115
- copying
 - cells 116, 56, 194
 - to a library other than current 117, 119
 - libraries 10, 163
 - objects 53
- cPin 19
- cPinF 20
- creating
 - cell boundaries 128
 - cell instances 139

- cell regions 142
- cell rows 143
- child nets 144
- contact records 148
- EEQ class 134
- empty cells 231, 57
- error
 - class records 151
 - object records 158
 - records 149
- expanded netlist cells 11, 17
- graph nodes 167
- groups 158
- hierarchical nets 160
- internal X wire records 161
- internal Y wire records 197
- layout cells 188, 193
- LEQ class 139
- libraries 18, 163
- macro cells 115
- netlist cells 46, 254, 277
- nets 167
- new versions of a cell 39
- parameter set records 171
- parent nets 172
- pin records 176, 588
- pin/blockage cells 57
- polygon records 180
- port records 181
- preroute records 182
- property records 184
- pseudo path records 185
- rectangle records 187
- reference libraries 43
- region pins 188
- region ports 190
- SG edge records 192
- SG primary node records 193
- text records 195
- wire tracks 3
- criticality, setting for nets 574

cycle 27
 D
 database object IDs
 getting for
 cell instances 367
 nets 354
 pins 356
 identifying a library ID 6
 dbAddAntennaLayerRule 27
 dbAddCellInstToPlanGroupByNames 30
 dbAddLibTclFile 39
 dbAddViaSpot 45
 dbAllowToAddPGIOText 47
 dbAssignNetTopLayerProbe 60
 dbBrowser 67
 dbCellAttachedFile 364
 dbCellChecksum 68
 dbCellCompare 72
 dbCellInstPlaceAndStatus 76
 dbCellReportHighFanoutNet 77
 dbChangeAllInstNames 79
 dbCheckIdenticalCell 90
 dbClearAllNetXtalkProp 94
 dbClearAttachment 95
 dbClearCellFootPrintEQClass command 98
 dbClearCellVTType 98
 dbClearLibAntennaRules 102
 dbClearLibraryCellsVTType 103
 dbClearVTFiller 107
 dbCompNetHalfBBox 110
 dbCopyCellWithViewFromAnotherLib 120
 dbCopyTextAndRectangle 122
 dbCreateCellBndyForChildCell 127
 dbCreateCellInstArray 132
 dbCreateGLink 156
 dbCreateLogicView 165
 dbCreateModelView 165
 dbCreatePlanGroupAndStatus 177
 dbDefineAntennaAccumMode 202
 dbDefineAntennaLayerRatioScale 202
 dbDefineAntennaRule 203
 dbDefineDiodeProtection 209
 dbDeleteFloatingCell 235
 dbDeleteLogicData 238
 dbDeleteLogicView 239
 dbDeleteModelData 240
 dbDeleteModelView 241
 dbDeletePlacementChair 246
 dbDeleteRectangleByLayer 250
 dbDeleteTieHiLoCell 253
 dbDisJoinPort 256
 dbDumpAllNetTopLayerProbe 262
 dbDumpAllVTFillers 264
 dbDumpCellAttachedFile 266
 dbDumpCellVTType 269
 dbDumpChildParentInfo 271
 dbDumpHierRefLibControl 277
 dbDumpLeqPortGroup 280
 dbDumpLibAntennaRules 282
 dbDumpLibAttachedFile 283
 dbDumpLibCatalog 284
 dbDumpLibraryCellsVTType 285
 dbDumpNetXtalkProp 287
 dbDumpPlacementChains 288
 dbDumpPropertyTable 288
 dbDumpVTFiller 296
 dbDumpVTFillers 3
 dbExistsCellAttachedFile 299
 dbExistsLibAttachedFile 300
 dbExpungeAllObjByTypeId 300
 dbExpungeAllObjByTypeName 301
 dbExpungeObjTypeById 302
 dbExpungeObjTypeByName 303
 dbExtractCellPGPosition 304
 dbExtractPins 305
 dbFetchCellPropertyValueByName 305
 dbFetchContactCodeInfo 306
 dbFetchContactCodeTypes 307
 dbFetchContactNames 309
 dbFetchDesignRuleInfo 310
 dbFetchDesignRuleTypes 311
 dbFetchFringeCapInfo 312

dbFetchNonDefaultContactCodeInfo 318	dbInitShapeStruct 404
dbFetchPropertyFieldAndName 325	dbIOPinTextToPin 405
dbFetchPropertyValueByName 326	dbIsCaseSensitive 406
dbFileCompare 330	dbLibraryCompare 408
dbGenNetList 350	dbLoadPropertyDef 412
dbGetAccessMode 360	dbIsCellOpen 407
dbGetCellByInstPath 365	dbMemProfile 420
dbGetCellFullNameById 366	dbMemProfileReport 421
dbGetCellInstById 366	dbMoveAllCellRegions 424
dbGetCellInstRefLib 368	dbNetChecksum 430
dbGetCellNameAndViewById 369	dbObjId? 439
dbGetCellNameById 370	dbOpenRefLibTree 443
dbGetCellViewVersions 371	dbPadToPin 444
dbGetCheckLibMode 372	dbPinToPad 444
dbGetDiodePortByName 374	dbPinToPadAndSnap 445
dbGetLayerNameByMaskName 374	dbQueryCPUTime 468
dbGetLayerNumberByMaskName 375	dbQueryObjectByExtendDist 470
dbGetLibAttachedFile 376	dbQueryPeakMem 472
dbGetLibBusNameStyle 376	dbQueryPeriodPeakMem 472
dbGetLibDbVersion 377	dbQueryPeriodPeakMemBegin 473
dbGetLibIdByName 378	dbQuitShapeStruct 473
dbGetLibNameById 379	dbRecoverLib 475
dbGetLibraryPath 379	dbRefLibChecksum 477
dbGetLibRefCtrlFileMode 380	dbRefreshLib 483
dbGetLModelSubType 381	dbRegisterProperty 484
dbGetMsgLevel 383	dbReOpenRefLibForWrite 491
dbGetObjectTypeName 384	dbReplaceAllCellBoundary 492
dbGetPinNetByName 385	dbReplaceCellBoundary 493
dbGetPlanGroupByName 386	dbResetAllNetTopLayerProbe 503
dbGetRefControlFileMode 386	dbSetBaseArraySubTile 511
dbGetRefLib 387	dbSetCellInstGroupId 517
dbGetTolerantMode 388	dbSetCellPowerValue 535
dbGetUserHierDelimiter 389	dbSetCellPRBoundaryPolygon 536
dbHierCopyToExistLib 390	dbSetCellVTType 542
dbHierCopyToNewLib 392	dbSetCheckLibMode 543
dbHierDelete 394	dbSetCurrentRegionNumber 544
dbHierPurge 395	dbSetGroupParentId 554
dbHierSave 396	dbSetLibDefaultCreationAccess 561
dbHierSplit 397	dbSetMsgLevel 568
dbImportLogicData 401	dbSetNetTrace 577
dbImportModelData 402	dbSetNetTraceById 578

- dbSetPathConstraint function 581
- dbSetPinPointArray 591
- dbSetPlanGroupAndStatus 593
- dbSetPlanGroupBBox 595
- dbSetPlanGroupBBoxByName 596
- dbSetPortInstLocalStatus 598
- dbSetUserLogFile 608
- dbSleep 612
- dbSwapCellViewVersions 620
- dbSyncRefLib 621
- dbUpdateCellInstNumberOfCellRow 623
- dbUpdatePcellDefinition 628
- debug 28
- defineBooleanFunction 7
- defineCellDontTouch 7
- defineCellDontUse 7
- defineCellFanoutLoad 8
- defineCellMaxCapacitance 8
- defineCellMaxFanout 9
- defineCellMaxTransition 10
- defineCellMinCapacitance 10
- defineCellMinFanout 11
- defineCellMinTransition 12
- defineCondMinClkPeriod 12
- defineCondMinClkPulseWidth 14
- defineCondTimeIntrinsic function 17
- defineCondTimeTLU function 38
- defineFlipFlopFunction 48
- defineGateSize 51
- defineLatchFunction 58
- defineOUTER_MOST_WELL 76
- definePortCapacitance function 67, 70
- definePortPhaseDelay function 71
- defineTimeIntrinsic function 80
- defineTimeTLU function 101
- defineTriStateFunction 108
- defining
 - wire tracks 3
- defining power use
 - in the CLF file 54, 59, 76
- delay

- conditional 15
 - setting phase delay 71
- deleting
 - attached files 232
 - from libraries 236
 - blockage geometries in layout cells 74, 102, 304
 - cell instance masters 453
 - cell instances 233, 453
 - cells 71
 - cells, by name 231
 - clock definitions 8
 - clock waveforms 9
 - driving cell definitions 10
 - input delays 10
 - net transition delays 13
 - nets 457, 458
 - objects 243, 460, 70
 - from groups 487
 - old versions of a cell 174
 - output delays 14
 - phase delays 9
 - pin records 245, 462
 - port records 463
 - properties 248, 249
 - rectangle records, from layers 244
 - reference libraries 251
 - region pins 464
 - region ports 465
 - ruler display 73
 - scan chains 247, 250, 252
 - timing requirements 15
- deselecting
 - all selected objects 74
 - objects by
 - clicking 163
 - drawing a line 100
 - enclosing with a rectangle 228
 - name 136
- deselecting, objects 2, 74, 75
- disconnecting, nets 111, 254

- from port instances 255
- displaying
 - bind key functions 72
 - cells
 - congestion maps 2, 3, 4
 - errors
 - all 39
 - all errors of the current type 70
 - current error 69
 - error of the current type by number 87
 - first error of the current type 80
 - next error of the current type 155
 - previous error of the current type 174
 - summary of errors 101
 - floorplan groups 258
 - flylines for
 - a specified object 11
 - an object edge 4
 - the entire design 8
 - flylines for a specified pin 13
 - information about groups of cells 177
 - net connections for
 - all nets 8
 - an object edge 4
 - net information about a specified
 - net 180
 - object 11
 - object edge 4
 - object information 185
 - ruler 188
 - wire tracks in
 - design 10
 - unit tile cell 9
- DRC
 - displaying
 - all errors 39
 - all errors of the current type 70
 - current error 69
 - first error of the current type 80
 - next error of the current type 155
 - previous error of the current type 174
 - specified error of the current type 87
 - summary of errors 101
 - loading error cells 101, 105
 - making the first error type current 81
 - making the next error type current 155
 - removing highlighting from errors 224
 - setting the error types to see 200
- driving cells
 - deleting definitions 10
- dumping scan chains 2
- E
- ECO status, setting for cell instances 512
- ECOs
 - comparing two netlists and updating layout 153
 - displaying history information 8
 - writing details to a file 5
- edges in a scan chain, adding 33, 34
- EDIF netlists
 - translating from ApolloII to 52
 - translating to ApolloII format 46
- eding 126
- editing commands, undoing 225
- EEQ class
 - clearing 96
 - creating 134
 - verifying consistency 630
- electrical equivalency, changing in pins 39
- enaCond argument 26
- endCond argument 26
- engineering change orders, see ECOs
- error cells, loading 105
- errors
 - creating
 - error class records 151
 - error object records 158
 - records 149
 - displaying
 - all 39
 - all errors of the current type 70
 - current error 69

- first error of the current type 80
 - next error of the current type 155
 - previous error of the current type 174
 - specified error of the current type 87
 - summary of 101
- loading error cells 105
- making the first error type current 81
- making the next error type current 155
- removing highlighting 224
- setting
 - color of error class records 546
 - pattern of error class records 547
 - status of error class records 548
 - status of error records 551
- setting the types you want to see 200
- etCellViewVersions 371
- exit 8
- exiting a Synopsys application 2
- expanding netlists 11, 17
- exploding, see smashing
- extracting
 - pin geometries 74, 102, 304
 - pins and blockage areas 57
- F
- fences, listing associated cell instances 362
- fields
 - listing valid names and current values 319
 - listing valid names for cell or library IDs 6
 - netId, setting for port instances 600
 - retrieving current values 321
- files
 - constraints
 - loading 23
- filling gaps and notches 91, 214
- fitting designs in graphics windows 81, 82
- flattening, see smashing
- flip chip
 - defining the pads 52
- flylines
 - displaying for a specified pin 13
- flylines, displaying for
 - a specified object 11
 - an object edge 4
 - the entire design 8
- fringe factors, setting 553, 58
- functions, listing 9
- G
- gaps, filling 91, 214
- gates
 - defining gate size 51
- GDSII Stream layouts
 - translating ApolloII to 197
 - translating to ApolloII format 187
- geAddBlockage 9
- geAddRectilinearBoundary 23
- geAddRectilinearBoundaryForChildCell 23
- geConvertWireToPin 52
- generating
 - PR boundaries 166
- geNewMakeMacro 139
- geQueryNetConstraint 181
- geSetCustomRouteTypesVis 199
- geSetNetTypesVis 213
- geShowCellList 215
- geSummary 219
- getting
 - cell IDs 83
 - floor planning options 56
 - layer
 - names 352
 - numbers 351
 - object IDs 86
 - for region ports 359
 - for regions 358
 - path constraints 373
- getting licenses 10
- geWindowDeselect 228
- geWindowOption 229
- geXWindowId 224
- global nets
 - creating for scan chains 157
 - specifying connections to subcells 11, 17

- graph nodes, creating 167
- graphics windows
 - closing 47
 - fitting designs in 81, 82
 - moving
 - the display down 226
 - the display left 227
 - the display right 227
 - the display up 228
 - panning 162
 - redrawing 186
 - returning to previous state 95
 - zooming in 236
 - zooming out 236, 237
- grid
 - checking for objects not placed on 40
 - setting options 88
- groups
 - creating 158
 - listing associated cell instances 363
 - moving 131
- groups, floorplanning
 - displaying 258
- GUI components 29
- guidelines for naming libraries, cells, and objects 19
- H
 - hdlSetNotCloseCell 3
 - HECODiff 88
- hierarchial nets, creating 160
- hierarchy
 - writing out to a file 104
- highlighting
 - cells 2
 - nets 180, 182
 - see also displaying
- hiliteNet 70
- hmiExplorerOptions 14
- hmiHercules 15
- I
 - image cell
 - creating 21
 - defining cell rows 22
 - defining routing tracks 25
 - defining tile pattern 23
 - verifying 2, 21, 17
 - input conditions
 - in defining power use 60
 - input delays, deleting 10
 - internal power
 - defining 54
 - internal X wire records, creating 161
 - internal Y wire records, creating 197
 - intrinsic delay
 - setting 80
 - IO constraints, writing to a file 5, 6
 - ioDumpTdffforChildCell 5
 - K
 - keepout regions, see blockage areas
 - key commands
 - resetting bind keys 5
 - keys
 - displaying functions for 72
 - L
 - labels, see text
 - layer panel 96
 - layers
 - deleting, rectangle records 244
 - displaying 96
 - getting
 - names 352
 - numbers 351
 - layout cells
 - creating 188, 193
 - layouts
 - translating
 - ApolloII to GDSII Stream format 197
 - GDSII Stream to ApolloII format 187
 - updating per netlist-to-netlist comparison 153
 - layoutWindowGrabFocus 239
 - leakage power

- defining 59
- LEQ class
 - clearing 97
 - creating 139
 - verifying consistency 631
- libraries
 - attached files, storing 618
 - closing 108, 45
 - copying 10, 163
 - creating 18, 163
 - deleting, attached files 236
 - global port information, dumping to files 276
 - loading CLF files 105, 11, 21
 - logically equivalent port information, dumping to database 291
 - opening 442, 158
 - printing record content 467
 - reference
 - adding 50
 - creating 43
 - deleting 251
 - listing 63
 - replacing 53, 499
 - renaming 490
 - replacing technology files 55
 - setting, techfileName 607
 - specifying access modes when opening 442
 - verifying consistency
 - of EEQ class 630
 - of LEQ class 631
- library
 - renaming 52
- Library preparation for LM view 165
- licenses
 - definitions of 10
 - getting 10
 - releasing 17
- linear power model 62
- listing
 - commands and functions 9
 - reference libraries 63
- LM view
 - library preparation 165
- loadFromPath 72
- loading
 - CLF files 105, 11, 21
 - error cells 105
 - slack graphs 25
 - TDF files 6
 - technology files 55
- loading a constraints file 23
- locating
 - opens 160
 - shorts 153
- log files 608
- LVS
 - displaying
 - all errors 39
 - all errors of the current type 70
 - current error 69
 - first error of the current type 80
 - next error of the current type 155
 - previous error of the current type 174
 - specified error of the current type 87
 - summary of errors 101
 - loading error cells 101, 105
 - making the first error type current 81
 - making the next error type current 155
 - removing highlighting from errors 224
 - setting the error types to see 200
- M
- macro cells, creating 115
- macros
 - created using ApolloII 139
- make-db 74
- manual
 - routing 33
- maximum delay
 - setting 49
- measuring distances 188
- merging

- objects 106
- merging paths 106
- minimum clock cycle
 - defining 63
 - defining conditionally 14
- minimum delay
 - setting 52
- models
 - power models 62
- modifying
 - objects 106
- modifying objects
 - moving 126
 - moving cells 128
- moving
 - groups 131
 - objects 133, 135, 234
 - points in objects 132
- moving cells 128
- moving objects 126
- moving, net members 425
- multiple ECOs
 - writing to a file 2
- N
- names
 - guidelines/restrictions 19
- naming guidelines/restrictions 19
- net 75
 - query constraints 178, 181
- net connections, displaying
 - for a specified object 11
 - for entire design 8
- net constraints, setting
 - capacitance 76
 - delay 77
 - source characteristics for clock 19
 - transition delay 79
- net information, displaying
 - for a specified object 11
- net transition delays, deleting 13
- netCapacitance function 76

- netDelay function 77
- netlist cells, creating 46, 254, 277
- netlists
 - checking 8
 - comparing for layout update 153
 - expanding 11, 17
 - translating
 - ApolloII to EDIF format 52
 - ApolloII to hierarchical Verilog format 97
 - ApolloII to Verilog format 261, 283
 - EDIF to ApolloII format 46
 - Verilog to ApolloII format 254, 277
- nets
 - assigning
 - variable-width rules to 61, 66
 - child, see child nets
 - connecting, port instances to 111
 - creating 167
 - creating for scan chains 157
 - deleting 457, 458
 - all empty nets 451
 - all single port nets 452
 - disconnecting
 - from members 111, 254
 - from port instances 255
 - displaying
 - information about 180
 - getting object IDs 354
 - global nets, specifying connections to sub-cells 11, 17
 - hierarchical, see hierarchical nets
 - highlighting 180, 182
 - moving all members 425
 - parent, see parent nets
 - setting
 - criticality 574
 - switch rate 578
 - type 578, 580
 - writing criticality to a file 259
- netTransition function 79

- notches, filling 91, 214
- notDoneYet 81
- O
- object edges, displaying net information about 4
- object IDs, getting 86
 - for region ports 359
 - for regions 358
- objects
 - adding to groups 41
 - copying 53
 - deleting 243, 70
 - from groups 487
 - deselecting 2, 74, 75
 - all selected 74
 - by clicking 163
 - by drawing a line 100
 - by name 136
 - deselecting by
 - enclosing with a rectangle 228
 - displaying
 - information about 185
 - information about edges 4
 - net information about 11
 - measuring 188
 - merging 106
 - moving 126, 133, 135, 234
 - name guidelines/restrictions 19
 - redefining 186
 - selecting 196
 - selecting by
 - clicking 163, 195
 - drawing a line 100
 - enclosing with a rectangle 233
 - name 136
 - setting, fringe factors 553, 58
 - splitting 217
 - stretching 218, 234
 - updating values of properties 629
- obsoleteCmd 81
- opening
 - cells 156
 - in read access mode 440
 - in write access mode 440
 - libraries 158
 - in read access mode 442
 - in write access mode 442
- opens, locating 160
- output conditions
 - for defining power use 61
- output delays, deleting 14
- P
- pad 82
- pad constraints
 - writing to a file 5, 6
- pads
 - adjusting placement of 8
 - writing constraints to a file 5, 6
- panning inside graphics windows 162
- panning up 228
- parameter set records, creating 171
- parent nets, creating 172
- path constraints
 - removing 32
- path constraints files
 - creating by translating SDF files 159
 - loading 23, 26
- path constraints, getting 373
- path constraints, setting 581
- paths
 - adding 17
 - constraints
 - creating by translating SDF files 159
 - merging 106
 - specifying arrival times 42
 - specifying required arrival times 61, 63, 67, 68, 69, 72
 - splitting 217
- PDEF
 - writing a design to a file 6
- period
 - defining conditional minimum clock 14

- defining minimum clock 63
- phase delay
 - setting 71
- phase delays, deleting 9
- Physical Design Exchange Format, see PDEF 5
- pin 83, 85
- pin connections
 - showing 13
- pin constraints
 - writing to a file 5, 6
- pin records
 - creating 176, 588
 - deleting 245, 462
 - setting, access direction 584, 586
- pin/blockage cells, creating 57
- pins
 - adding 23, 24
 - adjusting placement of 8
 - cancelling external connection specifications 504
 - changing the electrical equivalency of 39
 - converting to wires 51, 52
 - defining 57
 - display number off wire tracks 2
 - setting location constraints 589
 - specifying external connection of logically equivalent pins 4, 524, 525, 545
 - writing constraints to a file 5, 6
- pins, getting object IDs 356
- placement
 - congestion map 2, 3, 4
 - defining a blockage area 2, 3
- plotAddBoxText 86, 95
- plotAddPath 87
- plotAddPolygon 88
- plotAddRectangle 89
- plotAddScreenText 89
- plotAddSegment 90
- plotAddText 91
- plotClear 92
- plotFlush 92
- plotSelectWindow 93
- plotSetDrawAttr 94
- plotSetScale 95
- point arrays, setting for polygon records 597
- points in objects, moving 132
- polygon records 180
 - creating 180
 - setting point array 597
- polygons, adding 20
- port instances
 - connecting to nets 111
 - setting, netId 600
- port records
 - creating 181
 - deleting 463
 - setting, direction 599
- ports
 - defining capacitance 67, 70
- power models
 - types 62
- power use
 - defining in the CLF 54, 59, 76
- PR boundaries, generating 166
- preroute records, creating 182
- printing
 - record content 467
- property records, creating 184
- protection frames, see blockage areas
- pseudo path records, creating 185
- pulse width
 - setting the minimum 64
- purging
 - clock source characteristics 8, 10
 - clock waveforms 9
 - input delays 10, 12, 13
 - output delays 14
 - phase delays 9
 - timing requirements 15
- Q
- query
 - net constraints 178, 181

- query files
 - closing 110
 - opening, output files 443
- querying
 - groups of cells 177
- quitting a Synopsys application 2
- R
- rectangle records
 - creating 187
 - deleting, from layers 244
- rectangles, adding 22
- redefining portions of objects 186
- redo 185
- redrawing graphics windows 186
- reference libraries
 - adding 50
 - listing for a library 63
 - replacing 53
- region pins
 - creating 188
 - deleting 464
- region ports
 - creating 190
 - deleting 465
 - getting
 - direction 360
 - ID 359
- regular expressions
 - converting 109
- releasing licenses 17
- removing, see deleting
- renaming cells 51
- renaming libraries 52
- repeating an action 185
- repeatTile 98
- replaying command files 98
- required arrival times, specifying 61, 63, 67, 68, 69, 72
- resistance
 - setting for a device driving the chip or block 82, 85
- reverting
 - cells to last update 27
 - to last saved version of a cell 509
- reverting to last
 - saved version of a cell 188
 - view of a cell 95
- routing
 - deleting
 - variable-width rules 230
 - manual 33
 - with variable-width rules 61, 66, 224
 - writing variable-width rules to a file 263
- rPin 99
- ruler
 - displaying 188
 - removing 73
- rules, variable width
 - assigning 61, 66
 - deleting 230
 - specifying 224
 - writing to a file 263
- S
- saving
 - cells 108, 510
 - cells with new name 194
- scan chain
 - writing to a file 2
- scan chains
 - adding
 - cell instance groups 36
 - fixed edges 33
 - free edges 34
 - creating
 - containing instances of specified masters 414
 - containing specified instances 413
 - empty scan chain 191
 - global nets for 157
 - deleting 247, 250, 252
 - replacing instances with a different master 489, 500

- writing to a file 290
- scmxConcatFunction 100
- scmxLoad 102
- scmxPreprocess 102
- SDF files
 - translating to path constraints files 159
- selecting
 - objects by
 - clicking 163, 195
 - drawing a line 100
 - enclosing with a rectangle 233
 - name 136
- selecting, objects 196
- setFieldDefault 106
- setInstViewLevel 237
- setPostCallBack 21
- setPreCallBack 23
- setting
 - cell origin 425
 - fringe factors 553
 - grid options 88
 - path constraints 581
 - source characteristics for a clock 19
 - window options 229
- SG edge records, creating 192
- SG primary node records, creating 193
- shorts, locating 153
- single-cycle timing
 - resetting timing arcs to 16
- slack graph
 - loading to a cell 25
 - writing to a file 3
- smashing
 - instances of specified master cells 65
 - specified cell instances 216, 221
- specifying
 - arrival times 42
 - net capacitance 76
 - port capacitance 67, 70
 - required arrival times 61, 63, 67, 68, 69, 72
- splitting

- objects 217
- startCond argument 26
- Stream, see GDSII Stream layouts
- stretching
 - objects 218, 234
- strToRegExp 109
- switch rate, setting for nets 578
- switching power
 - defining 76
- Synopsys
 - Reference Lib file, compiling to CLF file 7
 - Report Lib file, translating to CLF file 247
- system 110
- systemExec 111
- T
- tcl 112
- tclLoad 112
- TDF files
 - creating 2, 3, 4
 - loading 6
- tdfPurgeClockPhaseDelays 9
- tdfPurgeClockWaveforms 9
- tdfPurgeNetTransitionDelays 13
- tdfPurgeTimeRequirements function 15
- tdfResetPath function 16
- tdfSetClockPortTransition 29
- tdfSetDisableTiming function 32
- tdfSetFalsePath function 37
- tdfSetInputPortTransition 46
- tdfSetMaxDelayPath function 49
- tdfSetMinDelayPath function 52
- tdfSetMultiCyclePath function 55
- tdfSetRestoreTiming function 73
- tdfSetTimeBorrowLimit function 88
- technology files
 - dumping ASCII format to the outFile 293
 - replacing 501
 - replacing (loading) 55
 - writing compiled information to editable
 - format 27
- text records, creating 195

- text, adding 31
- tile patterns
 - checking 64
 - defining 23
- timing arcs
 - disabling 32
 - excluding from timing analysis 37
 - resetting to single-cycle timing 16
 - restoring 73
 - specifying number of cycles 55
- timing constraints
 - writing to a file 3
- timing functions
 - defineCondTimeIntrinsic 17
 - defineCondTimeTLU 38
 - defineTimeIntrinsic 80
 - defineTimeTLU 101
 - setting net capacitance 76
 - setting net delay 77
 - setting phase delay 71
 - setting transition delays 79
- timing requirements, deleting 15
- timing, path constraints
 - getting 373
 - setting 581
- TLU power model 62
- top design format files, see TDF files
- tracks
 - defining in image cell 25
- translating
 - ApolloII layouts to GDSII Stream format 197
 - ApolloII netlists to EDIF format 52
 - ApolloII netlists to hierarchical Verilog format 97
 - ApolloII netlists to Verilog format 261, 283
 - EDIF netlists to ApolloII format 46
 - GDSII Stream layouts to ApolloII format 187
 - Verilog netlists to ApolloII format 254
 - VHDL netlists to ApolloII format 277
- U
 - undefineBindKey 113
 - undoing editing commands 225
 - unit tile cell
 - creating wire tracks in 3
 - displaying wire tracks 9
 - unit tiles
 - defining wire tracks in 3
- V
 - variable width routing
 - assigning rules 61, 66
 - deleting rules 230
 - specifying rules 224
 - writing rules to a file 263
 - vars 24
 - varValue 25
 - Verilog netlists, translating
 - from ApolloII format 97, 261, 283
 - to ApolloII format 254, 277
 - versions, creating in cells 39
 - vias
 - defining areas for 57
 - viewing
 - cells 2
- W
 - window options, setting 229
 - wire tracks
 - creating 3
 - defining 3
 - display number of pins not on 2
 - displaying in
 - design 10
 - unit tile cell 9
 - wires
 - adding 33
 - converting to pins 51, 52
 - splitting 217
 - writing out PDEF information 6
- Z
 - zooming in 236
 - zooming out 236, 237